

Kubernetes and Cloud Native Associate

Sean Bigelow

- KCNA
 - Kubernetes and Cloud Native Associate
- Lodestar Ledgers: KCNA
 - Copyright
 - For Those About to Navigate
 - How to Use This Guide
 - Quick Reference: Domain Weights
- KCNA Certification Study Guide
 - A Lodestar Ledgers Publication
 - Detailed Table of Contents
 - Front Matter
 - Chapter 1: Taking Departure
 - Chapter 2: Cargo in Standard Crates
 - Chapter 3: The Ship's Company
 - Chapter 4: Records of Intent
 - Chapter 5: The Smallest Vessel
 - Chapter 6: Fleets, Not Vessels
 - Chapter 7: Assigning the Berth
 - Chapter 8: Standing the Watch
 - Chapter 9: Every Pod Has an Address
 - Chapter 10: Traffic from Beyond the Cluster
 - Chapter 11: Below the Waterline
 - Chapter 12: Locks, Keys, and Watchstanders
 - Chapter 13: When the Cluster Won't Answer
 - Chapter 14: Packing for the Voyage
 - Chapter 15: The Chart Is the Truth
 - Chapter 16: Your Application, Their Cluster
 - Chapter 17: The Fleet and Its Charts
 - Chapter 18: Reading the Instruments
 - Chapter 19: Bearings Before Landfall
 - Chapter 20: Full Mock Exam
 - Back Matter
- Chapter 1: Taking Departure
 - *"Ninety minutes, four domains, and a curriculum that moved"*
 - Attention Budget
 - 📍 Soundings

- .1 What the KCNA Is, and Who It's For
- .1 Ninety Minutes: The Exam as Published
- .11 The Curriculum That Moved Under Everyone's Feet
- 🕒 Taking Your Bearings: The Exam and the Blueprint
- .1 The Phrase We Haven't Defined Yet
- .1 How This Book Is Built
- .1 Three Ways to Read This Book
- 🕒 Taking Your Bearings: Using the Instruments
- Chapter Summary
- The Voyage Ahead
- Chapter 2: Cargo in Standard Crates
 - *"Why the shipping container beat the ship"*
 - Attention Budget
 - 📍 Soundings
 - Why This Chapter Matters
 - What You'll Learn
 - §1 — .1 What a Container Actually Is
 - §2 — .1 What's Inside an Image
 - §3 — .11 Registries, Tags, and Digests
 - 🕒 Taking Your Bearings #1: Containers, Images, and Identity
 - §4 — .11 The Container Runtime Interface
 - §5 — .11 The Open Container Initiative
 - 🕒 Taking Your Bearings #2: Who Runs It, and Who Standardized It
 - §6 — .11 When Kubernetes Pulls, and When It Doesn't
 - §7 — .11 Not All Isolation Is Equal: RuntimeClass
 - 🕒 Taking Your Bearings #3: Pull Behavior and Runtime Selection
 - §8 — .11 The Crate, Not the Cargo
 - Exam Alert 🚨
 - Practice Questions
 - Chapter Summary
 - The Voyage Ahead
- Chapter 3: The Ship's Company
 - *"Everyone aboard has one job, and none of them is 'be in charge'"*
 - Attention Budget
 - 📍 Soundings
 - Why This Chapter Matters
 - What You'll Learn
 - §1 — .1 How the Cluster Got the Shape It Has
 - §2 — .1 The Control Plane
 - §3 — .1 Node Components in Context
 - 🕒 Taking Your Bearings #1: The Ship's Company
 - §4 — .11 Addons, and What Else Is Optional

- §5 — .ii The Only Door In
- 🕒 Taking Your Bearings #2: Arrangement and Optionality
- §6 — .ii Controllers and the Control Loop
- 🕒 Taking Your Bearings #3: Controllers and the Loop
- §7 — .iii Nobody Is in Charge
- Exam Alert! 🚨
- Practice Questions
- Chapter Summary
- The Voyage Ahead
- Chapter 4: Records of Intent
 - *"You don't give Kubernetes orders. You file a declaration."*
 - Attention Budget
 - 🕒 Soundings
 - Why This Chapter Matters
 - What You'll Learn
 - §1 — .i You File a Declaration
 - §2 — .i The Anatomy of a Record
 - 🕒 Taking Your Bearings #1: The Declarative Frame and Object Anatomy
 - §3 — .ii Where a Name Lives
 - §4 — .ii Configuration Kept Outside the Image
 - 🕒 Taking Your Bearings #2: Scoping and Configuration Objects
 - §5 — .ii The Universal Join
 - 🕒 Taking Your Bearings #3: Labels, Selectors, and Annotations
 - §6 — .iii A Declaration, Not an Order
 - Exam Alert! 🚨
 - Practice Questions
 - Chapter Summary
 - 🚢 Safe Harbor
 - The Voyage Ahead
- Chapter 5: The Smallest Vessel
 - *"A Pod is not a container, and that distinction is worth points"*
 - Attention Budget
 - 🕒 Soundings
 - Why This Chapter Matters
 - What You'll Learn
 - .i §1 — The Pod as the Unit of Scheduling
 - .ii §2 — More Than One Container Aboard
 - .ii §3 — Everything That Must Happen First
 - 🕒 Taking Your Bearings #1: What a Pod Is
 - .i §4 — Scheduled Once, Replaced Never
 - .ii §5 — Pod Phases and Container States
 - 🕒 Taking Your Bearings #2: Lifetime, Phase, and State

- .I §6 — A Pod's Identity
- .II §7 — Three Probes, Three Jobs
- .III §8 — What a Pod Is Owed
- 🕒 Taking Your Bearings #3: Identity, Health, and What a Pod Is Owed
- ☼ §9 — The Smallest Deployable Unit
- Exam Alert! 🚨
- Practice Questions
- Chapter Summary
- The Voyage Ahead
- Chapter 6: Fleets, Not Vessels
 - *"Nobody sails one Pod"*
 - Attention Budget
 - 🕒 Soundings
 - Why This Chapter Matters
 - What You'll Learn
 - .I §1 — The Resource That Holds the Intent
 - .I §2 — A Loop You Can Watch Working
 - .II §3 — How a Controller Knows Its Own
 - 🕒 Taking Your Bearings #1: Intent, Loops, and Ownership
 - .II §4 — Changing the Fleet Under Way
 - .II §5 — Every Rollout Is a Revision
 - 🕒 Taking Your Bearings #2: Changing the Fleet Under Way
 - .II §6 — When Pods Are Not Interchangeable
 - .I §7 — One Per Node, and Work That Ends
 - .III §8 — The Control Loop, Extended
 - 🕒 Taking Your Bearings #3: The Rest of the Fleet, and Extending It
 - ☼ §9 — Nobody Sails One Pod
 - Exam Alert! 🚨
 - Practice Questions
 - Chapter Summary
 - The Voyage Ahead
 - 🚢 Safe Harbor
- Chapter 7: Assigning the Berth
 - *"Filter, score, bind — and then a coin flip"*
 - Attention Budget
 - 🕒 Soundings
 - Why This Chapter Matters
 - What You'll Learn
 - .I §1 — One Decision, Made Once
 - .II §2 — What Makes a Node Feasible
 - 🕒 Taking Your Bearings #1 — How the Decision Is Made
 - .II §3 — Asking for a Particular Berth

- .II §4 — When the Berth Refuses You
- ⌚ Taking Your Bearings #2 — The Two Directions
- .III §5 — Placing Pods Relative to Each Other
- .III §6 — Overruling the Scheduler, and Replacing It
- ⌚ Taking Your Bearings #3 — Relative Placement, and Opting Out
- ☀ §7 — Everything Is a Filter or a Score
- Exam Alert! ⚠
- Practice Questions
- Chapter Summary
- The Voyage Ahead
- Chapter 8: Standing the Watch
 - *"The commands you'll actually type, and the versions that bite"*
 - Attention Budget
 - ⌚ Soundings
 - Why This Chapter Matters
 - What You'll Learn
 - .I §1 — The Grammar of a Command
 - .II §2 — Three Gates and a Logbook
 - .II §3 — Dividing a Shared Cluster
 - ⌚ Taking Your Bearings #1 — The Path a Command Takes In
 - .II §4 — Taking a Node Out of Service
 - .I §5 — Who Owns the Control Plane
 - ⌚ Taking Your Bearings #2 — The Machine, and Whose It Is
 - .III §6 — Versions That Are Allowed to Disagree
 - .II §7 — The One Backup That Matters
 - ⌚ Taking Your Bearings #3 — The Two Things You Cannot Improvise
 - .I §8 — Rules, or Consequences
 - Exam Alert! ⚠
 - Practice Questions
 - Chapter Summary
 - ⚓ Safe Harbor
 - The Voyage Ahead
- Chapter 9: Every Pod Has an Address
 - *"Flat networks, stable names, and the abstraction that survives churn"*
 - Attention Budget
 - ⌚ Soundings
 - Why This Chapter Matters
 - What You'll Learn
 - .I §1 — Four Rules and a Plugin
 - .I §2 — The Address That Doesn't Last
 - .II §3 — Four Ways to Be Reachable
 - ⌚ Taking Your Bearings #1

- .II §4 — The List Behind the Name
- .III §5 — When You Don't Want a Single Address
- 🕒 Taking Your Bearings #2
- .II §6 — The Component That Makes It Real
- .II §7 — Names, and Where They Resolve
- 🕒 Taking Your Bearings #3
- ☼ §8 — A Query With a Name
- Exam Alert! ⚠️
- Practice Questions
- Chapter Summary
- The Voyage Ahead
- Chapter 10: Traffic from Beyond the Cluster
 - *"Frozen, superseded, and inert without a controller"*
 - Attention Budget
 - 🕒 Soundings
 - Why This Chapter Matters
 - What You'll Learn
 - .I §1 — Where LoadBalancer Runs Out
 - .II §2 — Routing by Host and Path
 - .I §3 — The Object Is Not the Implementation
 - 🕒 Taking Your Bearings #1
 - .I §4 — Frozen, Not Deprecated
 - .III §5 — Roles, Not Just Routes
 - 🕒 Taking Your Bearings #2
 - .II §6 — Allowing, Never Denying
 - .III §7 — What NetworkPolicy Cannot Do
 - 🕒 Taking Your Bearings #3
 - ☼ §8 — Nothing Happens Without a Controller
 - Exam Alert! ⚠️
 - Practice Questions
 - Chapter Summary
 - The Voyage Ahead
- Chapter 11: Below the Waterline
 - *"Storage outlives the Pod that asked for it"*
 - Attention Budget
 - 🕒 Soundings
 - Why This Chapter Matters
 - What You'll Learn
 - .I §1 — Three Lifetimes, and the Volumes That Have Them
 - .I §2 — The Claim and the Supply
 - 🕒 Taking Your Bearings #1
 - .II §3 — Provisioning on Demand

- .ii §4 — Access Modes and What Happens After
- 🕒 Taking Your Bearings #2
- .ii §5 — Who Actually Provides the Storage
- .ii §6 — Pods That Are Not Interchangeable, Revisited
- 🕒 Taking Your Bearings #3
- ☀️ §7 — Outliving the Pod That Asked
- Exam Alert! ⚠️
- Practice Questions
- Chapter Summary
- The Voyage Ahead
- Chapter 12: Locks, Keys, and Watchstanders
 - *"RBAC has no deny rule, and Secrets aren't encrypted"*
 - Attention Budget
 - 🕒 Soundings
 - Why This Chapter Matters
 - What You'll Learn
 - .i §1 — Four Layers and Four Phases
 - .i §2 — Who You Are
 - .ii §3 — What You May Do
 - 🕒 Taking Your Bearings #1
 - .ii §4 — Secrets Are Not Encrypted
 - .ii §5 — What a Pod May Do to Its Node
 - .ii §6 — Three Levels, Three Modes
 - 🕒 Taking Your Bearings #2
 - .iii §7 — Trusting What You Ship
 - .ii §8 — Rules That Watch
 - 🕒 Taking Your Bearings #3
 - ☀️ §9 — Additive, Never Deny
 - Exam Alert! ⚠️
 - Practice Questions
 - Chapter Summary
 - The Voyage Ahead
- Chapter 13: When the Cluster Won't Answer
 - *"Read the phase before you read the logs"*
 - Attention Budget
 - 🕒 Soundings
 - Why This Chapter Matters
 - What You'll Learn
 - .i §1 — Whose Problem Is This, and What to Read First
 - .ii §2 — Pods That Never Start
 - .i §3 — Looking Inside
 - 🕒 Taking Your Bearings: Pods That Never Start, and the Commands That Tell You Why

- .ii §4 — Pods That Start and Then Don't Stay
- .iii §5 — When the Node Is the Problem
- 🕒 Taking Your Bearings: Started, Then Gone — and the Node Beneath
- .iii §6 — Versions That Don't Agree
- .ii §7 — Numbers Nobody Collects by Default
- 🕒 Taking Your Bearings: Skew, Metrics, and What Isn't There
- ⚡ §8 — Read the Phase First
- Exam Alert! ⚠️
- Practice Questions
- Chapter Summary
- The Voyage Ahead
- Chapter 14: Packing for the Voyage
 - *"A chart is not a release, and templates are not the point"*
 - Attention Budget
 - 🕒 Soundings
 - Why This Chapter Matters
 - What You'll Learn
 - .i §1 — Why a Folder of YAML Stops Working
 - .i §2 — What a Chart Contains
 - .ii §3 — Chart, Release, Revision
 - .i §4 — Where Charts Come From
 - 🕒 Taking Your Bearings: The Package and the Thing You Installed
 - .ii §5 — Patching Instead of Templating
 - .ii §6 — Which One, When
 - 🕒 Taking Your Bearings: Patching, and Choosing
 - ⚡ §7 — A Package, Not a Template
 - Exam Alert! ⚠️
 - Practice Questions
 - Chapter Summary
 - ⚓ Safe Harbor
 - The Voyage Ahead
- Chapter 15: The Chart Is the Truth
 - *"GitOps is the control loop you already learned, pointed at a repository"*
 - Attention Budget
 - 🕒 Soundings
 - Why This Chapter Matters
 - What You'll Learn
 - .i §1 — Twelve Factors, and the Ones Kubernetes Already Solved
 - .ii §2 — Ways to Replace What's Running
 - 🕒 Taking Your Bearings 1: The Application, and the Shapes of a Release
 - .ii §3 — Push, or Pull
 - .ii §4 — An Agent That Watches a Repository

- 🧭 Taking Your Bearings 2: Push, Pull, and the Agent
- .ii §5 — Ordering the Sync
- .ii §6 — The Other Agent, and More Than One Cluster
- 🧭 Taking Your Bearings 3: Ordering, and the Other Agent
- ⚙️ §7 — The Control Loop, Pointed at a Repository
- ⚓ Safe Harbor
- Exam Alert! 🚨
- Practice Questions
- Chapter Summary
- The Voyage Ahead
- Chapter 16: Your Application, Their Cluster
 - *"Four questions that separate your bug from theirs"*
 - Attention Budget
 - 📍 Soundings
 - Why This Chapter Matters
 - What You'll Learn
 - .i §1 — Handed Back
 - .ii §2 — When It Never Got Started
 - .ii §3 — Getting Inside, and Adding What Isn't There
 - 🧭 Taking Your Bearings: Taking the Handoff, and Getting Inside
 - .ii §4 — Is Anything Even Selected
 - .i §5 — Bypassing the Service on Purpose
 - 🧭 Taking Your Bearings: Reachability — What Selects, What Routes, What Proves
 - .ii §6 — When Each Replica Is Its Own
 - .ii §7 — Before You Ship It
 - 🧭 Taking Your Bearings: Identity, Storage, and the Limits of Reproducing It Locally
 - ⚙️ §8 — Mine, or the Platform's
 - Exam Alert! 🚨
 - Practice Questions
 - Chapter Summary
 - The Voyage Ahead
- Chapter 17: The Fleet and Its Charts
 - *"Meshes, functions, autoscalers, and the foundation that keeps the map"*
 - Attention Budget
 - 📍 Soundings
 - Why This Chapter Matters
 - What You'll Learn
 - .i §1 — What "Cloud Native" Actually Names
 - .i §2 — Sandbox, Incubating, Graduated, and Who Decides
 - .ii §3 — Small Pieces, Replaced Whole
 - 🧭 Taking Your Bearings: The Definition, the Ladder, and Who Decides
 - .ii §4 — Every Place Kubernetes Lets You In

- 📄 §5 — A Network That Knows What It's Carrying
- 🧭 Taking Your Bearings: Extension Points, and the Layer That Knows What It's Carrying
- 📄 §6 — Code Without a Server to Put It On
- 📄 §7 — Four Things That Scale
- 📄 §8 — How the Project Actually Runs, and How You'd Join
- 🧭 Taking Your Bearings: Scaling, Serverless, and How the Project Runs
- ⚙️ §9 — One Pluggability Story
- Exam Alert! 🚨
- Practice Questions
- Chapter Summary
- ⚓ Safe Harbor
- The Voyage Ahead
- Chapter 18: Reading the Instruments
 - *"Four signals, and the question they exist to answer"*
 - Attention Budget
 - 🧭 Soundings
 - Why This Chapter Matters
 - What You'll Learn
 - 📄 §1 — What You Can Ask, and What You Already Know
 - 📄 §2 — Four Signals
 - 📄 §3 — Numbers Over Time
 - 🧭 Taking Your Bearings: What You Can Ask, and What the Numbers Report Against
 - 📄 §4 — Pulling, Not Being Pushed
 - 📄 §5 — Following One Request
 - 🧭 Taking Your Bearings: Pull, Push, and the Path One Request Took
 - 📄 §6 — Lines From Everywhere
 - 📄 §7 — Is the Service Doing What Users Expect
 - 🧭 Taking Your Bearings: Lines, Signals, and the Question They Answer
 - ⚙️ §8 — One Question, Four Instruments
 - Exam Alert! 🚨
 - Practice Questions
 - Chapter Summary
 - ⚓ Safe Harbor
 - The Voyage Ahead
- Chapter 19: Bearings Before Landfall
 - *"Everything that connects, and the traps that don't"*
 - Attention Budget
 - 🧭 Soundings
 - Why This Chapter Matters
 - What You'll Learn
 - ⚙️ §1 — Nine Threads Through Eighteen Chapters
 - 🧭 Taking Your Bearings: The Threads

- 📊 §2 — The Pairs That Cost Points
- 🕒 Taking Your Bearings: The Pairs
- 📊 §3 — Ninety Minutes
- 📊 §4 — Where the Weight Actually Is
- 📊 §5 — Using The Lodestar
- 📊 §6 — The Week Before
- Exam Alert! 🚨
- Practice Questions
- Chapter Summary
- ⚓ Safe Harbor
- The Voyage Ahead
- Chapter 20: Full Mock Exam
 - *"Ninety minutes. No notes. Find out."*
 - Instructions
 - Exam
 - Mock Exam Answers & Walkthroughs
 - Scoring Rubric
- The Lodestar — KCNA
 - 1. The exam's published shape — *lookup*
 - 2. Exam-day pacing — *memorize; there is no scratch paper*
 - 3. The four hazards where intuition is wrong — *drill*
 - 4. The version-skew numbers — *lookup, but read it twice*
 - 5. The confusion-pair discriminators — *drill*
 - 6. Governance and institutional vocabulary — *lookup; the cheapest block to refresh*
 - 7. The one sentence to carry in
- Glossary
 - A
 - B
 - C
 - D
 - E
 - F
 - G
 - H
 - I
 - J
 - K
 - L
 - M
 - N
 - O
 - P

- Q
- R
- S
- T
- V
- W
- Acronyms

KCNA

Kubernetes and Cloud Native Associate

Lodestar Ledgers: KCNA

Kubernetes and Cloud Native Associate Exam Preparation Guide

KCNA curriculum effective 2025-11-24 (4 domains: 44/28/16/12)

LODESTAR LEDGERS

Study less. Pass once.

Copyright

Lodestar Ledgers: KCNA Kubernetes and Cloud Native Associate Exam Preparation Guide

Copyright (c) 2026 Lodestar Ledgers. All rights reserved.

No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the publisher, except in the case of brief quotations embodied in critical reviews and certain other noncommercial uses permitted by copyright law.

Trademarks

Kubernetes and the Kubernetes logo are registered marks of The Linux Foundation. KCNA, KCSA, CKA, CKAD, CKS, and the CNCF logo are marks of the Cloud Native Computing Foundation. Helm, Prometheus, Argo, Flux, Envoy, Jaeger, OpenTelemetry, containerd, CRI-O, Cilium, Falco, Harbor, KEDA, Knative, Kyverno and Notary are projects of the Cloud Native Computing Foundation. All other trademarks are the property of their respective owners.

Lodestar Ledgers is not affiliated with, endorsed by, or sponsored by the Cloud Native Computing Foundation or The Linux Foundation.

Disclaimer

This study guide is designed to help candidates prepare for the KCNA examination. While every effort has been made to ensure accuracy, the author and publisher make no warranty, express or implied, regarding the completeness or accuracy of the information contained herein.

This guide contains no actual examination questions. Every practice question in these pages was written for this book.

Examination content and requirements are subject to change. Candidates should verify current exam specifications at training.linuxfoundation.org and at the CNCF curriculum repository, github.com/cncf/curriculum.

Edition Information

First Edition: 2026 Version: 1.0

ISBN: [To be assigned]

Author: Sean Bigelow

Published by Lodestar Ledgers lodestarledgers.com

For Those About to Navigate

Most people preparing for this exam do not study too little. They study too much, in the wrong direction, and arrive knowing a great many facts they cannot connect.

That is a navigation problem, not an effort problem. A navigator who knows every star in the sky and cannot take a bearing is not close to being useful — they are missing the one thing that turns knowledge into position. The KCNA is a conceptual exam. It does not ask you to type anything. It asks whether you can recognize the right idea among three that sound almost as good, which means the work is not accumulating more facts but knowing which distinctions carry weight and which do not.

So this book is organized around distinctions. Where two things are confused, it names the confusion and gives you a one-line test. Where the documentation says something surprising, it quotes it and tells you where it came from. Where this book is reasoning rather than citing, it says so — you should be able to tell the difference between what the project publishes and what an author concluded, because on exam day only one of those is reliable.

There is one more thing worth saying before you start. The curriculum moved under everyone's feet on 24 November 2025: five domains became four, Observability stopped being a domain of its own, and Cloud Native Application Delivery doubled in weight. A great deal of the study material you will find online still teaches the old shape. This book targets the current one, and Chapter 1 shows you exactly what changed and what it costs you.

Take a bearing. Then set out.

How to Use This Guide

Understanding the Navigation Markers

Throughout this guide, you'll encounter consistent markers that help you navigate the material:

Marker	Name	Meaning
📍 Soundings	Pre-chapter diagnostic -- what you already know before you read	
☆ Fixed Point	A key concept to anchor in memory -- the essential takeaway	
🕒 Taking Your Bearings	Self-assessment checkpoint to verify your understanding	
— Dead Reckoning	Facts-only explanation without metaphors or analogies	
⚠️ Navigational Hazards	Warning about common mistakes or exam traps	
⚓ Safe Harbor	Chapter complete -- summary of what you've mastered	

| ☀️ **Zenith** | A synthesis moment — where several threads resolve into one idea | | 🗄️ **Worth Securing** | A point worth fastening down before moving on | | 📌 **Snag** | A place the obvious reading is the wrong one | | 🔍 **Closer Look** | Deeper than the exam requires, included because it explains the rest | | 🧠 **Mnemonic** | A memory hook for something arbitrary | | 🚨 **Exam Alert** | The highest-priority facts in the chapter | | 🗺️ → 🌊 → 🌅 **Voyage Progress** | Chart, Passage, Dawn — how far the voyage stands |

Difficulty scale. Every numbered section carries one of four markers: 1️⃣ **Foundation** (you must have this), 2️⃣ **Standard** (ordinary exam material), 3️⃣ **Advanced** (a distinction that separates candidates), 4️⃣ **Expert** (beyond what the exam requires, included for coherence).

Reading for Maximum Retention

First pass: Read the Soundings first and answer them badly. Getting a question wrong before you read the material is what makes the material stick; the discomfort is the mechanism, not a side effect.

At checkpoints: Close the book. Answer from memory, out loud or on paper. If you can only recognize the answer when you see it, you have not learned it yet — recognition and recall are different, and the exam tests the second one.

Between sessions: Leave a gap. A day is better than an hour, and a week is better than a day, up to a point. Re-reading immediately feels productive and does almost nothing.

Before the exam: Work `the-lodestar.md`, this book's one-page reference. Chapter 19 §5 tells you exactly how to use it in the last hour, and what to leave alone.

Chapter Structure

Each chapter follows a consistent structure:

1. **Attention Budget** -- Recommended time allocation and study conditions
2. **Soundings** -- A short diagnostic so you can skip what you already know
3. **Why This Chapter Matters** -- Context and motivation
4. **Core Content** -- The material you need to know
5. **Taking Your Bearings** -- Self-assessment checkpoints
6. **Exam Alert** -- High-priority topics and common traps
7. **Practice Questions** -- With explanations of why the wrong answers are wrong
8. **Chapter Summary** -- Key concepts in table format
9. **Safe Harbor** -- Chapter completion confirmation
10. **The Voyage Ahead** -- Preview of the next chapter

Study Schedule Recommendations

Available Time	Recommended Approach
One week	Chapters 1, 19 and 20 first — the shape of the exam, the confusion pairs, and a full mock to find out where you actually stand. Then the two weakest domains the mock exposes. Do not read front to back; you will not finish.
One month	One Part per week, in order, with the checkpoints worked rather than read. Take the Chapter 20 mock at the three-week mark, not at the end, so its results still have time to change what you do.
Three months	Front to back, one chapter per sitting, checkpoints closed-book. Re-take the mock twice — once at the halfway point and once in the final week — and use the per-domain tally rather than the total.

What This Guide Covers

This guide aligns with the CNCF's published KCNA curriculum, effective 24 November 2025:

- **Kubernetes Fundamentals** (44%) — Core Concepts, Administration, Scheduling, Containerization — *Chapters 2–8*
- **Container Orchestration** (28%) — Networking, Security, Troubleshooting, Storage — *Chapters 9–13*
- **Cloud Native Application Delivery** (16%) — Application Delivery, Debugging — *Chapters 14–16*
- **Cloud Native Architecture** (12%) — Observability, Cloud Native Ecosystem and Principles, Cloud Native Community and Collaboration — *Chapters 17–18*

The per-chapter percentages this book prints are **authored judgment**, not published data. CNCF publishes weights per domain and names the competencies beneath them; it publishes no sub-weights and no topic list. Where this book allocates, it says so.

What This Guide Doesn't Replace

- **Official CNCF and Linux Foundation resources** — the published curriculum, the candidate handbook, and the Kubernetes documentation itself. This book cites them; it does not supersede them.

- **Hands-on experience** — the KCNA has no terminal, so you can pass it without a cluster. You will understand it far better with one, and everything after it requires one.
- **Timed practice** — Chapter 20 is one full-length mock. One is enough to calibrate; it is not enough to practice pacing. Sit it under real conditions.
- **Your judgment** -- If something here conflicts with official guidance, defer to it

Quick Reference: Domain Weights

Kubernetes Fundamentals		44%	Ch 2-8
Container Orchestration		28%	Ch 9-13
Cloud Native App Delivery		16%	Ch 14-16
Cloud Native Architecture		12%	Ch 17-18

Nearly half the exam is Kubernetes Fundamentals, so Part II is where the hours go — but the two smallest domains are where candidates most often lose points they could have kept, because they are the easiest to skim and the hardest to bluff. Weight your study by the percentages; weight your *review* by where your mock results are weakest.

Your voyage begins on the next page.

KCNA Certification Study Guide

A Lodestar Ledgers Publication

Detailed Table of Contents

Front Matter

- About This Book
 - How to Read This Guide
-

Chapter 1: Taking Departure

"Ninety minutes, four domains, and a curriculum that moved"

Read chapter ->

- .i What the KCNA Is, and Who It's For
 - .i Ninety Minutes: The Exam as Published
 - .ii The Curriculum That Moved Under Everyone's Feet
 - .i The Phrase We Haven't Defined Yet
 - .i How This Book Is Built
 - .ii Three Ways to Read This Book
-

Chapter 2: Cargo in Standard Crates

"Why the shipping container beat the ship"

Read chapter ->

- \$1 — .i What a Container Actually Is
- \$2 — .i What's Inside an Image
- \$3 — .ii Registries, Tags, and Digests
- \$4 — .ii The Container Runtime Interface
- \$5 — .ii The Open Container Initiative
- \$6 — .iii When Kubernetes Pulls, and When It Doesn't

- §7 — .|| Not All Isolation Is Equal: RuntimeClass
 - §8 — .|| The Crate, Not the Cargo
-

Chapter 3: The Ship's Company

"Everyone aboard has one job, and none of them is 'be in charge'"

Read chapter ->

- §1 — .|| How the Cluster Got the Shape It Has
 - §2 — .|| The Control Plane
 - §3 — .|| Node Components in Context
 - §4 — .|| Addons, and What Else Is Optional
 - §5 — .|| The Only Door In
 - §6 — .|| Controllers and the Control Loop
 - §7 — .|| Nobody Is in Charge
-

Chapter 4: Records of Intent

"You don't give Kubernetes orders. You file a declaration."

Read chapter ->

- §1 — .|| You File a Declaration
 - §2 — .|| The Anatomy of a Record
 - §3 — .|| Where a Name Lives
 - §4 — .|| Configuration Kept Outside the Image
 - §5 — .|| The Universal Join
 - §6 — .|| A Declaration, Not an Order
-

Chapter 5: The Smallest Vessel

"A Pod is not a container, and that distinction is worth points"

Read chapter ->

- .|| §1 — The Pod as the Unit of Scheduling
- .|| §2 — More Than One Container Aboard
- .|| §3 — Everything That Must Happen First
- .|| §4 — Scheduled Once, Replaced Never

- .ll §5 — Pod Phases and Container States
 - .l §6 — A Pod's Identity
 - .ll §7 — Three Probes, Three Jobs
 - .ll §8 — What a Pod Is Owed
 - ☼ §9 — The Smallest Deployable Unit
-

Chapter 6: Fleets, Not Vessels

"Nobody sails one Pod"

Read chapter ->

- .l §1 — The Resource That Holds the Intent
 - .l §2 — A Loop You Can Watch Working
 - .ll §3 — How a Controller Knows Its Own
 - .ll §4 — Changing the Fleet Under Way
 - .ll §5 — Every Rollout Is a Revision
 - .ll §6 — When Pods Are Not Interchangeable
 - .l §7 — One Per Node, and Work That Ends
 - .ll §8 — The Control Loop, Extended
 - ☼ §9 — Nobody Sails One Pod
-

Chapter 7: Assigning the Berth

"Filter, score, bind — and then a coin flip"

Read chapter ->

- .l §1 — One Decision, Made Once
 - .ll §2 — What Makes a Node Feasible
 - .ll §3 — Asking for a Particular Berth
 - .ll §4 — When the Berth Refuses You
 - .ll §5 — Placing Pods Relative to Each Other
 - .ll §6 — Overruling the Scheduler, and Replacing It
 - ☼ §7 — Everything Is a Filter or a Score
-

Chapter 8: Standing the Watch

"The commands you'll actually type, and the versions that bite"

Read chapter ->

- .l §1 — The Grammar of a Command
 - .ll §2 — Three Gates and a Logbook
 - .ll §3 — Dividing a Shared Cluster
 - .ll §4 — Taking a Node Out of Service
 - .l §5 — Who Owns the Control Plane
 - .lll §6 — Versions That Are Allowed to Disagree
 - .ll §7 — The One Backup That Matters
 - .l §8 — Rules, or Consequences
-

Chapter 9: Every Pod Has an Address

"Flat networks, stable names, and the abstraction that survives churn"

Read chapter ->

- .l §1 — Four Rules and a Plugin
 - .l §2 — The Address That Doesn't Last
 - .ll §3 — Four Ways to Be Reachable
 - .ll §4 — The List Behind the Name
 - .lll §5 — When You Don't Want a Single Address
 - .ll §6 — The Component That Makes It Real
 - .ll §7 — Names, and Where They Resolve
 - ☼ §8 — A Query With a Name
-

Chapter 10: Traffic from Beyond the Cluster

"Frozen, superseded, and inert without a controller"

Read chapter ->

- .l §1 — Where LoadBalancer Runs Out
 - .ll §2 — Routing by Host and Path
 - .l §3 — The Object Is Not the Implementation
 - .l §4 — Frozen, Not Deprecated
 - .lll §5 — Roles, Not Just Routes
 - .ll §6 — Allowing, Never Denying
 - .lll §7 — What NetworkPolicy Cannot Do
 - ☼ §8 — Nothing Happens Without a Controller
-

Chapter 11: Below the Waterline

"Storage outlives the Pod that asked for it"

Read chapter ->

- .I §1 — Three Lifetimes, and the Volumes That Have Them
 - .I §2 — The Claim and the Supply
 - .II §3 — Provisioning on Demand
 - .II §4 — Access Modes and What Happens After
 - .II §5 — Who Actually Provides the Storage
 - .II §6 — Pods That Are Not Interchangeable, Revisited
 - ☼ §7 — Outliving the Pod That Asked
-

Chapter 12: Locks, Keys, and Watchstanders

"RBAC has no deny rule, and Secrets aren't encrypted"

Read chapter ->

- .I §1 — Four Layers and Four Phases
 - .I §2 — Who You Are
 - .II §3 — What You May Do
 - .II §4 — Secrets Are Not Encrypted
 - .II §5 — What a Pod May Do to Its Node
 - .II §6 — Three Levels, Three Modes
 - .III §7 — Trusting What You Ship
 - .II §8 — Rules That Watch
 - ☼ §9 — Additive, Never Deny
-

Chapter 13: When the Cluster Won't Answer

"Read the phase before you read the logs"

Read chapter ->

- .I §1 — Whose Problem Is This, and What to Read First
- .II §2 — Pods That Never Start
- .I §3 — Looking Inside
- .II §4 — Pods That Start and Then Don't Stay
- .III §5 — When the Node Is the Problem

- .ll §6 — Versions That Don't Agree
 - .ll §7 — Numbers Nobody Collects by Default
 - ☼ §8 — Read the Phase First
-

Chapter 14: Packing for the Voyage

"A chart is not a release, and templates are not the point"

Read chapter ->

- .ll §1 — Why a Folder of YAML Stops Working
 - .ll §2 — What a Chart Contains
 - .ll §3 — Chart, Release, Revision
 - .ll §4 — Where Charts Come From
 - .ll §5 — Patching Instead of Templating
 - .ll §6 — Which One, When
 - ☼ §7 — A Package, Not a Template
-

Chapter 15: The Chart Is the Truth

"GitOps is the control loop you already learned, pointed at a repository"

Read chapter ->

- .ll §1 — Twelve Factors, and the Ones Kubernetes Already Solved
 - .ll §2 — Ways to Replace What's Running
 - .ll §3 — Push, or Pull
 - .ll §4 — An Agent That Watches a Repository
 - .ll §5 — Ordering the Sync
 - .ll §6 — The Other Agent, and More Than One Cluster
 - ☼ §7 — The Control Loop, Pointed at a Repository
-

Chapter 16: Your Application, Their Cluster

"Four questions that separate your bug from theirs"

Read chapter ->

- .ll §1 — Handed Back
- .ll §2 — When It Never Got Started

- 📄 §3 — Getting Inside, and Adding What Isn't There
 - 📄 §4 — Is Anything Even Selected
 - 📄 §5 — Bypassing the Service on Purpose
 - 📄 §6 — When Each Replica Is Its Own
 - 📄 §7 — Before You Ship It
 - ☀️ §8 — Mine, or the Platform's
-

Chapter 17: The Fleet and Its Charts

"Meshes, functions, autoscalers, and the foundation that keeps the map"

Read chapter ->

- 📄 §1 — What "Cloud Native" Actually Names
 - 📄 §2 — Sandbox, Incubating, Graduated, and Who Decides
 - 📄 §3 — Small Pieces, Replaced Whole
 - 📄 §4 — Every Place Kubernetes Lets You In
 - 📄 §5 — A Network That Knows What It's Carrying
 - 📄 §6 — Code Without a Server to Put It On
 - 📄 §7 — Four Things That Scale
 - 📄 §8 — How the Project Actually Runs, and How You'd Join
 - ☀️ §9 — One Pluggability Story
-

Chapter 18: Reading the Instruments

"Four signals, and the question they exist to answer"

Read chapter ->

- 📄 §1 — What You Can Ask, and What You Already Know
 - 📄 §2 — Four Signals
 - 📄 §3 — Numbers Over Time
 - 📄 §4 — Pulling, Not Being Pushed
 - 📄 §5 — Following One Request
 - 📄 §6 — Lines From Everywhere
 - 📄 §7 — Is the Service Doing What Users Expect
 - ☀️ §8 — One Question, Four Instruments
-

Chapter 19: Bearings Before Landfall

"Everything that connects, and the traps that don't"

Read chapter ->

- ☼ §1 — Nine Threads Through Eighteen Chapters
 - 📊 §2 — The Pairs That Cost Points
 - ⌚ §3 — Ninety Minutes
 - 📊 §4 — Where the Weight Actually Is
 - ⌚ §5 — Using The Lodestar
 - 📊 §6 — The Week Before
-

Chapter 20: Full Mock Exam

"Ninety minutes. No notes. Find out."

Read chapter ->

- Instructions
 - Exam
 - Mock Exam Answers & Walkthroughs
 - Scoring Rubric
-

Back Matter

- Glossary
- The Lodestar (single-page reference)

Chapter 1: Taking Departure

"Ninety minutes, four domains, and a curriculum that moved"

Chapter Type: Orientation | **Domain Weight:** — | **Complexity:** Mixed | **Novelty:** Moderate | **Prerequisites:** None

Attention Budget

Total time: ~45 minutes | **Recommended:** Single session

Section	Time	Attention Cost	Best Time to Study
📍 Soundings	7 min	Low	Anytime
What the KCNA Is, and Who It's For	4 min	Low	Anytime
Ninety Minutes: The Exam as Published	5 min	Low	Anytime
The Curriculum That Moved Under Everyone's Feet	10 min	Medium	When alert
🧭 Taking Your Bearings: The Exam and the Blueprint	4 min	Medium	After brief pause
The Phrase We Haven't Defined Yet	2 min	Low	Anytime
How This Book Is Built	5 min	Low	Anytime
Three Ways to Read This Book	3 min	Low	Anytime
🧭 Taking Your Bearings: Using the Instruments	5 min	Low	Anytime

Attention Cost Key:

- **Low:** Concrete, familiar concepts — study anytime
- **Medium:** New concepts requiring focus — study when alert
- **High:** Abstract or complex material — study at peak attention

If you only have 15 minutes: read "The Curriculum That Moved Under Everyone's Feet" and take the first Taking Your Bearings checkpoint. That section is the one piece of this chapter that changes what you do next.

"Take your departure while the landmarks are still in sight. The open sea is no place to discover your chart is wrong." — Lodestar Ledgers

🔊 Soundings

Before reading this chapter, try these five questions. They test what you're arriving with, not what this chapter teaches. Every one is answerable from general IT experience, and no score here is a bad score. Your result is a reading instruction, not a verdict.

1. A container and a virtual machine both isolate an application from its neighbors. What does a container share with its host that a virtual machine does not?
2. Kubernetes is often described as "the thing that runs your containers." Is that accurate? If not, what does Kubernetes actually do, and what runs the containers?
3. Who controls the Kubernetes project — a single vendor, a commercial company that licenses it, or an open-source foundation?
4. When someone says an application is "cloud native," what do most people take that to mean?
5. Have you written Terraform, Ansible playbooks, CloudFormation templates, or anything similar: a file that describes what infrastructure *should* look like, rather than a script that performs steps to build it?

Answers + reading strategy

1. The host's **operating system** — that is the documentation's word [source: k8s-docs-overview-2026-08-23]. Practitioners sharpen it: what is actually shared is the host's **kernel**, and this book uses that sharper register where the precision earns its keep. Hold both — the first is the wording to recognize on an answer sheet. *[cross-bearing: see Ch 2 §1 — what a container actually is, and why both registers are correct]*
2. Not accurate. Kubernetes is an **orchestrator** — it decides what should run where. A separate **container runtime** on each machine does the work of actually starting the containers [source: k8s-docs-cluster-architecture-2026-08-23]. *[cross-bearing: see Ch 2 §4 — the Container Runtime Interface]*
3. An open-source foundation. Kubernetes is hosted by the **Cloud Native Computing Foundation**, which is part of the nonprofit **Linux Foundation** [source: cncf-who-we-are-2026-08-23]. *[cross-bearing: see Ch 17 §2 — CNCF governance and the project lifecycle]*
4. Most people take it to mean "runs in a public cloud." That is the near-universal assumption, and it is not what the term means. We'll come back to it in §4 below, and *[cross-bearing: see Ch 17 §1 — the CNCF definition of cloud native]*.
5. No right answer here; this one is calibration only, and it isn't scored. If you've written any of those, you already hold a distinction that Chapter 4 will name for you, and you'll move through it fast. If you haven't, Chapter 4 is worth reading slowly. *[cross-bearing: see Ch 4 §1 — declarative versus imperative]*

Scoring: question 5 isn't scored. Of the four scored questions —

3–4 right: you arrive with the platform priors this book assumes. Read Chapters 2 and 3 at normal pace. The calibration in §6 will point you toward the chapters where your real gaps are. They are probably not where you expect.

1–2 right: this is the expected starting point for this credential. Read at normal pace throughout. The book is built for you.

0 right: read carefully, and give Chapters 2 and 3 a session of their own. Nothing here is beyond you. The vocabulary is new, that's all, and this book introduces every term before it uses it.

.. What the KCNA Is, and Who It's For

Here is a question worth carrying for the next few pages: **why does so much of the KCNA study material online describe a different exam than the one you are going to sit?**

Not a slightly different exam. A structurally different one: a different number of domains, different weights, and one whole domain that no longer exists. The material isn't fraudulent and its authors aren't careless. Something moved underneath them, and much of it hasn't caught up. Section 3 has the details. By the end of this chapter you'll be able to spot a stale guide yourself, in about fifteen seconds of skimming.

First, the credential itself.

The **Kubernetes and Cloud Native Associate**, the KCNA, is the usual entry point to the cloud native certification family. The Linux Foundation, which publishes the exam, describes it as demonstrating "a user's foundational knowledge and skills in Kubernetes and the wider cloud native ecosystem" [source: lf-kcna-exam-page-2026-08-23]. It is a CNCF credential, and CNCF is part of the nonprofit Linux Foundation [source: cncf-who-we-are-2026-08-23]. Experience level: beginner. Prerequisites: none [source: lf-kcna-exam-page-2026-08-23].

Take "no prerequisites" literally. No required course. No logged hours. No prior certification, no attestation of experience, no manager's signature. If you can pay the fee and stay awake for ninety minutes, you can sit it. That's unusual in this industry, and it fits what the credential is for: it gives people a defensible way to say *I understand how this world is put together* before they've had a job that let them prove it.

It is a **multiple-choice, online, proctored exam** [source: lf-kcna-exam-page-2026-08-23]. That matters more than it sounds, and it's the first place people misjudge this credential.

The hands-on Kubernetes certifications, the ones that drop you into a live terminal with a broken cluster and a running clock, measure whether you can *do* the thing [source: lf-cloud-native-certification-catalog-2026-08-23]. The KCNA measures whether you can *discriminate*. Given several plausible-sounding statements about how the scheduler decides where a Pod goes, can you identify the one that's true?

Given a symptom, can you name the layer where the problem lives? That is a genuinely different skill, and it is not a lesser one. Practitioners who can execute a command from muscle memory but can't say why it works are everywhere, and they're the ones who stall at three in the morning when the situation stops matching the runbook.

📖 **Worth Securing:** A conceptual exam deserves a conceptual study method. The instinct to "just get hands on a cluster and practice" is a good instinct for CKA and a partially misdirected one here. Hands-on time helps, because it makes abstractions concrete, but you will not pass this exam by drilling `kubect1` commands, and you can fail it while typing them fluently. Study for discrimination: for every concept, ask "what is the thing this is most often confused with, and what's the difference?"

Let me be direct about what this book is not. It is **not a kubect1 drill book**. There are commands in it, because you cannot understand a Service without seeing how one is described, but the commands are here to illuminate concepts, not to build reflexes. When you want the reflexes, and if you stay in this field you will, the Certified Kubernetes Administrator is the next voyage: a genuinely hands-on exam taken in a live terminal [source: [lf-cloud-native-certification-catalog-2026-08-23](#)]. Lodestar Ledgers publishes a guide for that one too.

The wider certification family — the other associate-level exams, the specialist tracks, how they relate to one another — belongs in Chapter 17, alongside the ecosystem material it's part of. [*cross-bearing: see Ch 17 — the cloud native certification landscape*]

.. **Ninety Minutes: The Exam as Published**

Everything in this section comes from the Linux Foundation's own exam page. Where the page is silent, I'll say so. That silence turns out to be one of the more useful facts in the chapter.

Dead Reckoning: The KCNA is an online, proctored, multiple-choice exam. Duration is 90 minutes. There are no prerequisites. Registration includes a 12-month eligibility window in which to schedule and sit the exam, two exam attempts, and an exam preparation handbook. The certification is valid for two years. Pricing at the time this book's sources were captured (2026-08-23): \$250 for the exam alone; \$299 for the exam bundled with the LFS250 course; \$495 for the exam bundled with the THRIVE-ONE annual subscription [source: [lf-kcna-exam-page-2026-08-23](#)].

Two of those facts deserve a moment.

Two attempts are included. Not a consolation prize. A structural feature. The second attempt is part of what you bought. This is not permission to sit the first one unprepared, but it does mean the catastrophe you may be quietly bracing for isn't on the menu: failing once costs you time and morale, not money. Adjust your anxiety accordingly.

Two years of validity. Cloud native tooling moves fast enough that a five-year credential would be a fiction. Two years is honest. Plan for it.

🔔 **Closer Look:** Prices and bundles are the most volatile facts in this section and the least load-bearing for your studying. The figures above are as of 2026-08-23. Check the exam page before you buy; if the number has moved, nothing else in this book changes.

Now a disclosure most study guides skip.

The two numbers everyone quotes are both published — just not where you would look for them. The passing score is in the Linux Foundation's candidate FAQ for its multiple-choice exams, which states that a score of 75% or above must be earned to pass [source: lf-mc-exam-faq-2026-08-23]. The question count is in the same handbook, on the "Multiple Choice Exams: Important Instructions" page, which states that the exam "consists of 60* multiple-choice questions" [source: provenance-kcna-60-questions-2026-08-31]. Both are published for multiple-choice exams **as a class**, and KCNA is placed in that class by its own exam page. Neither figure is on the exam page most candidates read, which states the format, the duration, the eligibility terms, the price, and the domain weights and stops there [source: lf-kcna-exam-page-2026-08-23]. What it does not do is repeat what the handbook already says.

You will nonetheless see "60 questions, 75% to pass" repeated across study sites, videos, and forum threads, stated flatly, as one fact. It is two facts with one pedigree and a shared problem: the certifying body publishes both, in a handbook, for a class of exams — and almost nobody who repeats them has read that handbook or cites it. What travels on repetition is not the numbers. It is the *claim to know where they come from*.

⚠️ Navigational Hazards

The hazard here is not the exam. It's the habit of treating a widely-repeated number as a published one.

A study plan built on "60 questions in 90 minutes" yields a pacing rule of 90 seconds per question, and that rule feels precise and authoritative. If the real count is different, the rule is wrong, and you find that out at minute forty of a ninety-minute exam, with a proctor watching and no way to reset.

The correction costs nothing: pace by *proportion of elapsed time*, not by question number. "A quarter of the way through the questions when a quarter of my time is gone" survives any question count. "Question 15 at the 22-minute mark" does not.

This book flags where each figure comes from every time either appears, including in Chapter 20, whose full-length mock is sized to the published count. That mock is a calibrated practice exam built at the documented size, and it is framed as such. It is not a claim about what your particular exam will contain. [cross-bearing: see Chapter 20 — the full mock exam, and how it is sized] Exam-day pacing gets its own treatment once you have content to pace through. [cross-bearing: see Ch 19 §3 — pacing and time discipline]

There's a broader habit here, and it's worth naming: **know which of your facts are published and which are inherited**. That distinction outlasts this exam. It's the same instinct that stops you quoting a Stack Overflow answer as documentation.

..1 The Curriculum That Moved Under Everyone's Feet

Here's the answer to the question I opened with.

The KCNA exam blueprint was restructured, effective no earlier than **November 24, 2025** [source: lf-kcna-program-changes-2026-08-23]. Five domains became four. Weights shifted substantially. And the domain that disappeared didn't shrink. It was absorbed.

These are the four domains and weights you are being examined against:

☆ Fixed Point

The current KCNA blueprint is four domains: Kubernetes Fundamentals 44%, Container Orchestration 28%, Cloud Native Application Delivery 16%, Cloud Native Architecture 12% [source: lf-kcna-exam-page-2026-08-23].

If you memorize one thing from this chapter, memorize those four numbers. They are the index to everything else. Every study decision you make from here should be checkable against them.

Their named competencies:

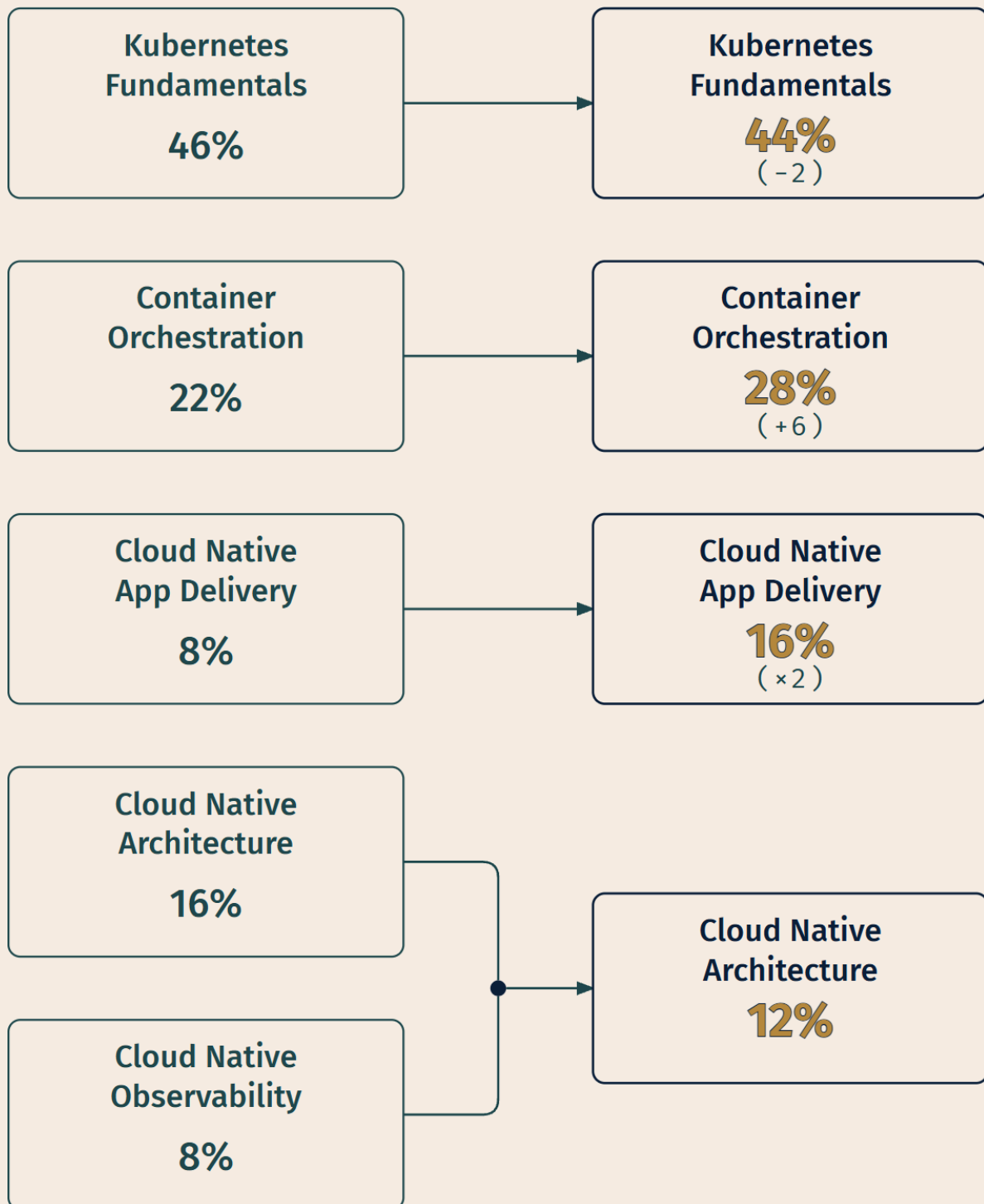
Domain	Weight	Competencies
Kubernetes Fundamentals	44%	Kubernetes Core Concepts; Administration; Scheduling; Containerization
Container Orchestration	28%	Networking; Security; Troubleshooting; Storage
Cloud Native Application Delivery	16%	Application Delivery; Debugging
Cloud Native Architecture	12%	Observability; Cloud Native Ecosystem and Principles; Cloud Native Community and Collaboration

[source: lf-kcna-exam-page-2026-08-23] [source: cncf-kcna-curriculum-pdf-2026-08-23]

And here is what they replaced. The retired blueprint had five domains: Kubernetes Fundamentals 46%, Container Orchestration 22%, Cloud Native Architecture 16%, **Cloud Native Observability 8%**, and Cloud Native Application Delivery 8% [source: lf-kcna-program-changes-2026-08-23].

RETIRED • 5 DOMAINS

CURRENT • 4 DOMAINS



Observability folds into Architecture as a competency—it's no longer a standalone domain.

The 2025 KCNA blueprint restructure. Two movements matter most: Application Delivery doubled, and Observability stopped being a domain of its own.

Three things changed in ways that should change your behavior.

Cloud Native Observability is gone as a standalone domain. Observability did not stop being tested. It was folded in as a competency under Cloud Native Architecture, alongside Ecosystem and Principles and Community and Collaboration [source: lf-kcna-program-changes-2026-08-23]. Prometheus, OpenTelemetry, the difference between metrics and traces: all still fair game. But the 8% once reserved for observability alone now shares a 12% domain with two other competencies. If you were planning to spend a fifth of your study time on Prometheus, that plan came off the old chart. *[cross-bearing: see Ch 18 §1 — observability under the current blueprint]*

Cloud Native Application Delivery doubled, from 8% to 16% [source: lf-kcna-program-changes-2026-08-23]. That is the largest proportional change among the domains that survived the restructure, and material built for the old blueprint will under-serve it by roughly half — a topic budgeted for 8% of an exam doesn't retroactively grow when the weight moves. GitOps, Helm, deployment strategies, debugging deployed applications: this is now one exam question in six. *[cross-bearing: see Ch 14–16 — the Application Delivery domain in full]*

Container Orchestration rose six points, from 22% to 28% [source: lf-kcna-program-changes-2026-08-23]. Networking, security, troubleshooting, and storage together now account for more than a quarter of the exam.

Kubernetes Fundamentals eased slightly, 46% to 44% [source: lf-kcna-program-changes-2026-08-23], a rounding-level change that shouldn't move anything in your plan. It remains, by a wide margin, the largest domain.

📌 **Snag:** "Cloud Native Architecture" appears in both blueprints at different weights, 16% before and 12% now, but it isn't the same domain in a smaller hat. It *gained* observability while *losing* weight, so the material it does cover is compressed harder than the number alone suggests. Don't reason about it by comparing 16 to 12.

Now the practical use of all this.

You are going to encounter other study material. Videos, blog posts, practice question sets, competing books, a colleague's notes from when they passed. Some of it is excellent. Here is how to check whether it describes your exam, in about fifteen seconds:

Count the domains. If the material is organized around **five** domains, or carries a standalone chapter or section titled "Cloud Native Observability" presented as a domain in its own right, it was built for the retired blueprint. That doesn't make its facts wrong; a Pod worked the same way in 2024. But its *weighting* is wrong, which makes its emphasis wrong, which makes its practice sets wrong, which makes the picture it hands you of where the exam's mass sits wrong. Specifically, it will under-serve Application Delivery by roughly half.

Use it for facts if you like. Don't use it to allocate your time.

🧠 **Mnemonic:** The weights descend: 44 · 28 · 16 · 12. Four numbers, each smaller than the last, no ties to confuse. If you want a hook, **44 and 28 together make 72%**. Kubernetes itself and its orchestration mechanics are nearly three-quarters of the exam. The remaining 28% is everything *around* Kubernetes: how you deliver to it, and the ecosystem it lives in.

One disclosure, made here where it's checkable: **the CNCF publishes weights at the domain level only.** There is no published per-competency or per-topic weighting [source: lf-kcna-exam-page-2026-08-23] [source: cncf-kcna-curriculum-pdf-2026-08-23]. Where this book assigns emphasis *within* a domain, meaning how many chapters a competency gets and how much depth each receives, that is authored judgment, derived from concept count and prerequisite load rather than from a published number. Section 5 says more about how that judgment maps to chapters.

🕒 Taking Your Bearings: The Exam and the Blueprint

Three questions. They test discrimination, which is the skill the whole exam is built on, so treat this checkpoint as a preview of the format as well as a check on the section.

1. Under the current KCNA blueprint, where does observability live?

A) It is its own domain, Cloud Native Observability, weighted 8% B) It is a competency under Cloud Native Architecture C) It is a competency under Container Orchestration, alongside Troubleshooting D) It was removed from the exam entirely in the 2025 restructure

2. Which of the following does the Linux Foundation's KCNA exam page state?

A) The number of questions on the exam B) The score you need to earn to pass C) The weight of each of the four domains D) The number of study hours recommended before sitting

3. You have limited study time and want to allocate it well. Which statement best describes how to use the domain weights?

A) Spend time in proportion to the number of chapters each domain has in this book, since chapter count reflects difficulty B) Spend the most time on Kubernetes Fundamentals (44%), then adjust for which competencies you personally find hardest C) Split time evenly across the four domains, since every domain contains questions D) Prioritize Cloud Native Application Delivery, since it doubled in the 2025 restructure

Answers with Explanations

1 — B. Observability is a competency under **Cloud Native Architecture** in the current four-domain blueprint [source: lf-kcna-program-changes-2026-08-23].

- **A is wrong**, and it's the trap that matters. Cloud Native Observability was a standalone 8% domain, in the blueprint retired in the 2025 restructure. If you picked A, you may be studying from the retired blueprint, or thinking in its terms. That's the whole point of §3.
- **C is wrong**. Container Orchestration's four competencies are Networking, Security, Troubleshooting, and Storage. Troubleshooting and observability are related in practice, but the blueprint catalogs them separately.
- **D is wrong**. Nothing was removed. It was reorganized. Observability content is still examinable; it simply shares a smaller domain now.

2 — C. The **four domain weights** (44 / 28 / 16 / 12) are stated on the exam page, and they're the most useful published fact you have [source: lf-kcna-exam-page-2026-08-23].

- **A is wrong**, though less wrong than it looks. The question count is not on the exam page — but it is published, on the candidate handbook's "Multiple Choice Exams: Important Instructions" page, for multiple-choice exams as a class [source: provenance-kcna-60-questions-2026-08-31]. The trap is the page you would think to check, not the fact.
- **B is wrong**, for a subtler reason than A. The 75% passing score is published by the Linux Foundation — in its candidate FAQ for multiple-choice exams, not on the exam page the question asks about [source: lf-mc-exam-faq-2026-08-23]. Keep the two straight by *where they live*: both are official, and both are in the candidate handbook rather than on the exam page the question asks about.
- **D is wrong**. No study-hour recommendation appears anywhere on the page. Certifying bodies rarely prescribe preparation time, for the obvious reason that it depends entirely on what the candidate walks in carrying.

Believing the inherited numbers isn't dangerous; *building a strategy on them* is. Answer every question, pace by elapsed time rather than by question number, and neither figure is one you need.

3 — B. Weight-proportional first, personal-weakness-adjusted second. Kubernetes Fundamentals is 44% of the exam, nearly half, and no allocation that under-serves it can be right.

- **A is wrong**. Chapter count tracks *how much explaining a topic needs*, not *how many questions it generates*. Those correlate loosely at best. This book's chapter allocation deviates from the published domain weights by at most 2.8 percentage points, deliberately, and §5 says so out loud. Where the two disagree, the exam weights win. They're the published number.
- **C is wrong**. An even split sounds fair and is simply inaccurate. It gives Cloud Native Architecture (12%) more than twice the attention its share of the exam warrants, and starves Kubernetes Fundamentals (44%).

- **D is wrong**, and it's the trap §3 sets by accident. A domain that doubled from a small base is still small. Growth rate is not exam share: Application Delivery moved from 8% to 16%, which makes it the fastest-growing domain and still the second-smallest one. Sixteen does not outrank forty-four.

Checkpoint: You've Now Mastered

✓ The four current domains and their weights — 44 / 28 / 16 / 12 ✓ What the 2025 restructure changed, and which topic moved most ✓ How to tell, in seconds, whether a study resource targets your exam ✓ The difference between an exam fact that's published and one that's inherited

That last one is a habit rather than a fact, and it's the most portable thing in this chapter.

.. The Phrase We Haven't Defined Yet

You've read "cloud native" perhaps a dozen times by now. It's in the credential's name. It's in two of the four domain names. And I haven't defined it.

That's deliberate, and I'd rather tell you than have you wonder.

"Cloud native" is doing real technical work in those names. It is not decoration and it is not a synonym for "modern." It names a specific set of characteristics that a system either has or doesn't. And critically, **it does not mean "runs in a public cloud."** The CNCF's own definition covers workloads deployed across public, private, and hybrid environments [source: cncf-cloud-native-definition-2026-08-23]. That's a common assumption on arrival, it's the one the Soundings tested, and it will actively obstruct you if you carry it into Chapter 17. A rack of hardware in your own building can be thoroughly cloud native, and renting cloud instances does not by itself make a system cloud native.

The CNCF maintains a published definition with named characteristics — the Cloud Native Definition, currently at version 1.1 [source: cncf-cloud-native-definition-2026-08-23]. You will get it in full in Chapter 17: the actual document, unabridged, each characteristic examined. [*cross-bearing: see Ch 17 §1 — the CNCF cloud native definition and its characteristics*]

Why wait four hundred pages for a definition I could give you in a paragraph? Because those are two different experiences of the same sentence. Read on page ten, the definition is vocabulary: a string of adjectives you'd nod at and forget by Tuesday. Read in Chapter 17, after you have met containers, orchestration, declarative APIs, services, and delivery pipelines in person, every clause lands as a description of something you now recognize. The definition stops being a thing to memorize and becomes a summary of what you already know.

So Chapter 1 leaves you with the question rather than the answer. Hold it open. It's the only thing this book asks you to carry that far.

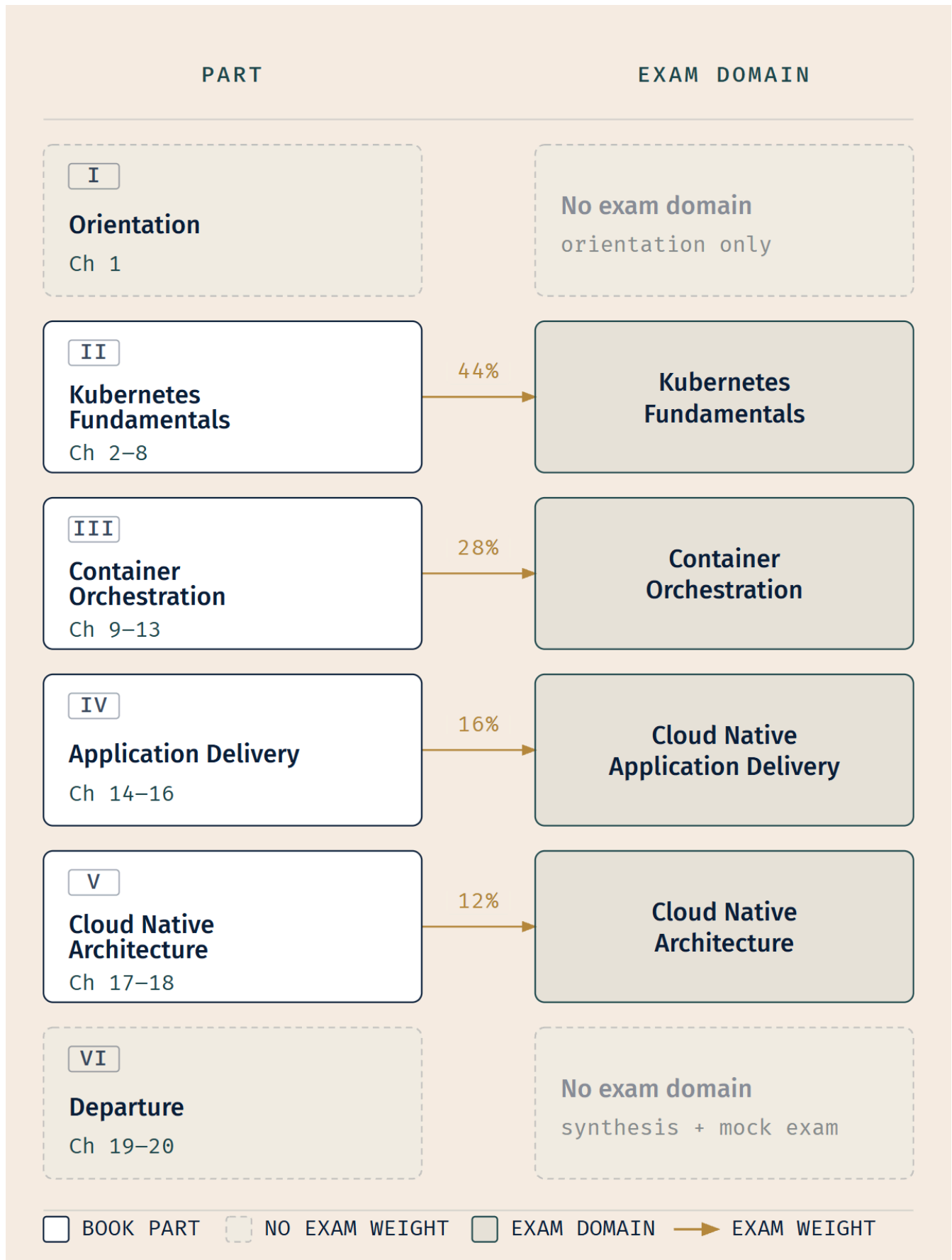
Extended Analogy: You have been handed a chart whose legend is printed on the last page.

That sounds like an error, and for most charts it would be. But this one is drawn for someone who will be sailing the water it depicts. The symbols on it, the marks for depth and hazard and anchorage, mean very little to a reader who has never seen the seabed they describe. Shown the legend first, you'd memorize that a certain mark means "rocky bottom, poor holding," and you'd have learned a fact.

Sail the water first. Anchor once on rocky bottom and spend a night listening to the cable grind. Then turn to the legend, and the mark doesn't teach you anything. It *names* something your hands already know. That's what this book is trading four hundred pages for.

How This Book Is Built

The instrument panel. Six Parts, twenty chapters.



Where you are in the book is where you are in the blueprint. Parts II through V correspond one-to-one with the four domains.

Parts II through V map one-to-one onto the four exam domains. That isn't a coincidence of organization; it's the point. "I've finished Part III" and "I've covered the Container Orchestration domain" are the same statement, so your progress through the book reads as progress through the blueprint with no translation needed.

The disclosure I promised in §3, placed where you can check it against the map above: the per-chapter emphasis in this book, meaning how many chapters a domain gets and how much depth each competency receives, is authored judgment. The CNCF publishes weights at the domain level only [source: [lf-kcna-exam-page-2026-08-23](#)]. Across the seventeen content chapters, the largest divergence between a Part's share of chapters and its domain's share of the exam is **2.8 percentage points** — Part II holds seven of seventeen chapters, or 41.2%, against Kubernetes Fundamentals' 44%. The other three Parts land within 1.6 points of their domains. Where chapter allocation and exam weight diverge, it's because some concepts take more pages to explain than their exam share would suggest, and some take fewer. The exam weights are the published fact. The chapter distribution is my best reading of what it takes to get you there. Where the two disagree, trust the weights.

The markers

You'll meet these throughout. Each one signals a specific kind of content so you can decide, at a glance, whether to slow down or move on.

Marker	Name	What it signals
Ⓢ	Soundings	Pre-chapter self-test; calibrates your reading pace
🧭	Taking Your Bearings	Post-reading checkpoint; verifies comprehension
☆	Fixed Point	Must-memorize. If you retain one thing, this
⚠	Navigational Hazards	A common mistake or exam trap, treated at length
—	Dead Reckoning	Facts only. No metaphor, no framing
⚓	Safe Harbor	Chapter complete
☀	Zenith	The moment separate concepts connect
🗺 → 🌊 → 🌅	Voyage Progress	Chart → Passage → Dawn: how far the voyage stands

Four smaller glyphs appear inline, mid-prose, as short asides:

Glyph	Name	What it signals
🔒	Worth Securing	A practitioner's tip worth anchoring
📄	Snag	A specific, common slip — briefer and narrower than ⚠️
🔍	Closer Look	Deeper than the exam requires; optional
🧠	Mnemonic	A memory hook

Two sidebar types run longer: **Logbook Entry** (a story from practice) and **Extended Analogy** (a metaphor developed at length). Both are opt-in depth; you can skip either without losing the thread.

And throughout, you'll see italic bracketed pointers like *[cross-bearing: see Ch 6 §6 — StatefulSets and stable identity]*. These are cross-bearings: forward and back references to where a concept gets its full treatment. Follow them or don't; they're there so you're never stuck wondering "did the book cover this?"

Difficulty is marked on section headings: 🌱 Foundation, 🌿 Standard, 🌳 Advanced, 🌲 Expert.

Two mechanisms that look like padding and aren't

The Soundings at the start of each chapter. These are not quizzes. Nothing is scored, recorded, or held against you. They exist for two reasons. Pre-testing improves subsequent learning even when you get the answers wrong, because attempting a question primes you to notice its answer when you meet it. And a score gives you something more useful than a grade: a reading instruction. High score, skim. Low score, slow down. Skipping them to save six minutes costs you more than six minutes.

Later chapters will test earlier chapters' material. Chapter 13's checkpoint will ask you something from Chapter 8. This is going to feel, the first few times, like the book forgot it already covered that. It didn't. Retrieving a fact after you've had time to partly forget it strengthens the memory far more than rereading it would. That's a well-established effect and the single highest-leverage thing this book does structurally. When you hit one of those questions and think *we did this already*, that thought is the mechanism working. Answer it anyway.

The book-level artifacts

Three things ship alongside the chapters:

- **the-`lodestar`.md** — a single page holding the exam-critical facts, distinctions, and traps, distilled from the whole book. It's the last thing to read before the exam. Chapter 19 walks you through using it. *[cross-bearing: see Ch 19 §5 — using The Lodestar]*
- **The glossary** — every term this book introduces, defined, with the chapter that introduces it.
- **The mock exam** — Chapter 20. A full-length, timed practice exam with worked answers. *[cross-bearing: see Ch 20 — full mock exam]*

Three Ways to Read This Book

You now have a Soundings score. Use it.

If you have no Kubernetes exposure at all: read linearly, start to finish, no skipping. Give Chapter 2 and Chapter 3 a study session each rather than folding them into a longer sitting. They carry the conceptual load everything else rests on, and rushing them is a false economy that shows up six chapters later as a vague sense that nothing is quite landing. Take every Soundings. Yours are the scores that will move most, and watching them move is worth something.

If you come from operations and are new to Kubernetes specifically: read linearly, but check your Chapter 2 Soundings score first. Scored well on the container questions? Chapter 2's container-fundamentals material is skimmable, and you should skim it. Chapter 3 onward is where your new material starts. Your likely blind spots are the parts of the ecosystem that aren't infrastructure: Part IV's delivery tooling, and Chapter 17's ecosystem and community material.

If you're a developer who has deployed to a cluster someone else runs: read this paragraph twice. Chapters 2, 4, 5, and 6 will feel familiar. They are not as familiar as they feel. You have working knowledge of the objects you touch daily, which is not the same as the complete model the exam tests, and confident partial knowledge is harder to correct than no knowledge at all. Your reliable gaps are Chapter 8, Chapter 12, and Chapter 17. Of those, Chapter 17's community and collaboration material is, in my experience, what technically strong candidates skip most often. It looks like soft content next to the technical chapters. It is examinable content in a 12% domain, it is easy to learn, and skipping it is the most avoidable way to lose points on this exam. *[cross-bearing: see Ch 8, Ch 12, and Ch 17 — the three chapters this reader most often needs]*

📖 **Snag:** Your Soundings score is a reading strategy, not a verdict. A 1/8 on Chapter 9's Soundings means "read Chapter 9 carefully" and nothing else. It is not a prediction about the exam, and it is certainly not information about you.

On study time, honestly rather than promotionally: this is a beginner-level, ninety-minute, conceptual exam. For someone with general IT literacy and no Kubernetes exposure, a few weeks of consistent evening study is a realistic frame. For someone already working around clusters, considerably less. Anyone quoting you a precise number ("Pass in 14 days!") is guessing, however confidently, because the number depends entirely on what you walk in carrying. What I can tell you is that the material is finite and well-bounded. This is not a credential you grind. It's one you understand.

Logbook Entry: Picture a candidate who sits the KCNA this year having studied hard from a course recorded in 2024.

They'd done everything right by their own lights: worked through the whole course, drilled the practice sets, built a mental map of the exam. That map had five domains on it. It had a standalone observability domain worth 8%, and they had studied Prometheus with the thoroughness an 8% standalone domain deserves. They knew the exporters. They knew the scrape model. They could talk about PromQL at a whiteboard.

They passed. A conceptual exam is forgiving of a misallocated study plan when the underlying facts are sound. But they described the first ten minutes as recalibration, question by question, as it dawned on them that the exam's mass was not where they'd been told it was. Application Delivery kept coming up. They had given it the attention appropriate to 8% of an exam, and it is 16%.

Ten minutes of a ninety-minute exam spent recalibrating is over a tenth of your time, and it goes at exactly the moment your composure matters most. That is what §3 of this chapter is for.

🕒 Taking Your Bearings: Using the Instruments

Three questions. Two are about method rather than content, because the habits they describe determine whether the next nineteen chapters work. The third checks the one claim this chapter asks you to carry forward.

1. You take the Soundings at the start of Chapter 9 and score 1 out of 8. What should you do?

A) Reread Chapter 1, since a low score means you've missed something foundational B) Skip Chapter 9's Soundings answers and come back to them after reading the chapter C) Read Chapter 9 carefully and slowly, giving it more time than a chapter you scored well on D) Read Chapter 9 at normal pace — a Soundings score reflects only what you knew beforehand, so it says nothing about how to read the chapter

2. Chapter 13's Taking Your Bearings checkpoint includes a question about material from Chapter 8. Why?

A) The book is repetitive; the editors didn't catch the duplication B) Retrieving material after partly forgetting it strengthens memory more than rereading does C) Chapter 13 requires Chapter 8 as an immediate prerequisite, so it's checking you're ready D) It fills out the question count to meet a target

3. Your company runs Kubernetes on hardware in its own building, with no public-cloud footprint anywhere in the stack. Can that platform be described as cloud native?

A) No — "cloud native" requires running in a public cloud B) No, unless it consumes at least some managed cloud services C) Yes — "cloud native" describes how a system is built and operated, not where it runs D) Only if the same workloads also run with a second provider

Answers with Explanations

1 — C. A Soundings score is a pacing instruction for the chapter *ahead*. Low score means slow down, take it in a dedicated session, don't skim.

- **A is wrong**, and it's the trap. A low Soundings score is not a signal that you failed to absorb something earlier. The questions test *priors you brought with you*, not material the book has taught. Chapter 9's Soundings measures what you knew about Chapter 9's subject before opening it. Rereading Chapter 1 would tell you nothing.
- **B is wrong**. The answers are part of the instrument. They're where the reading-strategy rubric lives, and several of them carry cross-bearings pointing at where the material is covered. Reading them costs a minute and shapes how you read the chapter.
- **D is wrong**, and its first half is right, which is what makes it tempting. The score genuinely says nothing about your ability. But it says a great deal about the gap between what you brought and what the chapter assumes — and that gap is exactly a pacing instruction. Discarding the score because it isn't a verdict throws away the only thing it was ever for.

2 — B. Retrieval practice. Recalling something after a delay, once you've partly forgotten it, builds a substantially more durable memory than rereading the same passage would. Spacing those retrievals across chapters is what makes the material stick past exam day.

- **A is wrong**, and this is the reflex worth inoculating against right now. When a later checkpoint asks about earlier material, it will feel like repetition. It isn't. It's the one structural feature of this book most likely to be skipped by readers trying to be efficient, and skipping it costs the most.
- **C is wrong**. Retrieval questions are scheduled by *spacing interval*, not by prerequisite relationship. Chapter 13 does not necessarily depend on Chapter 8; the question appears there because enough time has passed for the retrieval to be worth something.
- **D is wrong**. Question counts in this book are budgeted per chapter, but no question exists solely to fill a slot. Retrieval questions are chosen for what they reinforce.

3 — C. "Cloud native" describes characteristics of how a system is built and operated. The CNCF's definition explicitly spans public, private, and hybrid environments [source: [cncf-cloud-native-definition-2026-08-23](#)], so owning the hardware disqualifies nothing.

- **A is wrong**, and it's a common assumption on arrival — the one the Soundings tested and \$4 exists to dislodge. The phrase contains the word "cloud," which does most of the misleading on its own.
- **B is wrong**. This is the halfway version of the same error: conceding that location isn't the test, then smuggling it back in as "well, *some* cloud services, surely." Managed services are one way to build such a system. They are not in the definition.

- **D is wrong.** Multi-cloud is a deployment choice an organization makes for its own reasons. It appears nowhere in the definition, and a single-site platform is not disqualified by it.

The positive half of this — what the characteristics actually are — is Chapter 17's, and it's worth the wait. *[cross-bearing: see Ch 17 §1 — the CNCF cloud native definition and its characteristics]*

Checkpoint: You've Now Mastered

✓ What a Soundings score means and what to do with it ✓ Why later chapters test earlier material — and why to answer those questions rather than skip them ✓ Which reading path fits your starting point ✓ That "cloud native" is a statement about how a system is built, not about where it runs

Chapter Summary

Concept	Remember This
What the KCNA is	Beginner-level, no prerequisites, multiple-choice, online and proctored — a test of discrimination, not of typing speed
Published exam facts	90 minutes · no prerequisites · 12-month eligibility window · 2 attempts included · valid 2 years · \$250 exam-only as of 2026-08-23
Not on the exam page	The question count (60 — published in the candidate handbook's Important Instructions for MC exams) and the passing score (75% — published in the Linux Foundation's candidate FAQ). Both official; neither on the exam page
The four domains	44 / 28 / 16 / 12 — Kubernetes Fundamentals · Container Orchestration · Cloud Native Application Delivery · Cloud Native Architecture
The 2025 restructure	Effective no earlier than 2025-11-24. Five domains → four. Observability folded into Architecture. App Delivery doubled (8% → 16%). Orchestration +6 points
Spotting stale material	Five domains, or a standalone "Cloud Native Observability" section, means the retired blueprint. Facts may be fine; weighting isn't
Weight disclosure	CNCF publishes domain-level weights only. Per-chapter emphasis in this book is authored judgment, diverging from the published weights by at most 2.8 points
"Cloud native"	Deliberately undefined until Chapter 17. It does not mean "runs in a public cloud"
How the book maps	Parts II–V ↔ the four domains, one-to-one. Progress through the book = progress through the blueprint
Soundings	A pacing instrument, not a quiz. Low score = read slowly. Never a verdict
Retrieval questions	Later chapters test earlier material on purpose. When it feels repetitive, that's the mechanism working

[source: lf-kcna-exam-page-2026-08-23] [source: lf-kcna-program-changes-2026-08-23] [source: cncf-cloud-native-definition-2026-08-23]

🚢 Safe Harbor

Chapter 1 complete. You know what you're sitting, when the blueprint moved, and how to tell whether any given resource knows that. You've got a reading path calibrated to your own starting point, and a question about "cloud native" that you're carrying, unanswered, for the next four hundred pages.

That's the whole job of an orientation chapter, and it's done.

📊 Chart → 🌀 Passage → 🌅 Dawn

The Voyage Ahead

Chapter 2 ends where its title begins: with a shipping container. An actual one: corrugated steel, standardized corner fittings, the sort that stacks by the thousand on a container ship.

That is not decoration, and it is not a metaphor chosen for the scenery. The intermodal shipping container changed global trade not by being a better box, but by being a box that cranes, truck beds, railcars, and ships' holds could all agree to handle identically. The contents stopped mattering to the infrastructure. That one decoupling, between what's inside and what moves it, is precisely what a software container does, and it's why Chapter 2 starts there rather than with a definition.

The definition comes first; the crate is the payoff. By the time it arrives, you'll already see why it had to be that way.

44% of your exam begins on the next page.

"A chart is only as good as the survey behind it. Check the date in the corner before you trust the depths."

Chapter 2: Cargo in Standard Crates

"Why the shipping container beat the ship"

Exam domain: Kubernetes Fundamentals (44% of the exam) — competency: Containerization [source: cncf-kcna-certification-page-2026-08-23] [source: cncf-kcna-curriculum-pdf-2026-08-23] | **Estimated share of the exam: ~9% (authored allocation — CNCF publishes domain weights, not competency weights** [source: cncf-kcna-curriculum-pdf-2026-08-23]; **see front matter)** | **Complexity: Mixed** | **Novelty: Moderate**

Attention Budget

Total time: ~85 minutes | **Recommended: single session if you're fresh; otherwise split at Taking Your Bearings #2**

Section	Time	Attention Cost	Best Time to Study
§1 What a Container Actually Is	8 min	Low	Anytime
§2 What's Inside an Image	12 min	Low	Anytime
§3 Registries, Tags, and Digests	14 min	Medium	When alert
🕒 Taking Your Bearings #1	6 min	Medium	After a short break
§4 The Container Runtime Interface	12 min	Medium	When alert
§5 The Open Container Initiative	10 min	Medium	When alert
🕒 Taking Your Bearings #2	6 min	Medium	After a short break
§6 When Kubernetes Pulls, and When It Doesn't	9 min	High	Peak attention
§7 Not All Isolation Is Equal: RuntimeClass	6 min	Medium	When alert
🕒 Taking Your Bearings #3	6 min	Medium	After a short break
§8 The Crate, Not the Cargo	4 min	Low	Anytime

Attention Cost Key:

- **Low:** Concrete, familiar concepts. Study anytime.
- **Medium:** New vocabulary requiring focus. Study when alert.
- **High:** Conditional rules that must be recalled precisely. Study at peak attention.

If you only have 15 minutes: read §3 and §4, then take Taking Your Bearings #1 and #2. Those two sections carry two of the chapter's three Fixed Points, and in the author's judgment they are the material a reader is least likely to reconstruct on their own.

"The crate outlives the ship. Standardize the crate." — Lodestar Ledgers

🔗 Soundings

Before reading this chapter, try these eight questions. Your score decides how to approach the material. No shame in any score, just different reading strategies.

1. An application is running in a container, and one line of a configuration file inside it needs to change. Is editing that file in the running container the intended way to make the change persist, or is the intended workflow something else entirely?
2. You pull `myapp:v2` today. A teammate pulls the same reference next week. Which statement is true?
A) The bytes are identical, because the reference string is identical B) The bytes may differ C) The bytes are identical, because registries reject re-pushing an existing reference D) The bytes are identical only if both pulls hit the same registry replica
3. You write an image reference as just `nginx`, with no hostname in front of it. Where does that image get fetched from?
4. You build two images from the same base and push both to the same registry. Is that shared base stored once, or twice?
5. Kubernetes runs containers. Which best describes what that requires on the machines running them?
A) Docker specifically must be installed B) Any of several different runtimes will do C) Nothing — Kubernetes starts containers itself D) A runtime is required, but only on control-plane machines
6. Who publishes the specification that defines the container image format — the layout of the artifact you push to a registry and pull onto a machine?
A) The Docker company B) The Kubernetes project C) An industry standards body D) Each registry vendor, independently
7. A workload must run untrusted third-party code with a stronger boundary than an ordinary container provides. What are your options inside a single cluster?
A) None — you need a separate cluster, because isolation strength is fixed B) You can select a stronger isolation model for just that workload C) You can raise the isolation level for the whole cluster, but not per workload D) You can restrict the image it runs, which is the isolation mechanism

8. A machine has already downloaded a particular image. The next time it needs to start a container from that image, does it download it again?

Click for answers + reading strategy

1. **The intended workflow is something else.** The change belongs in a new image, not in a live container. §2 covers why this is the rule rather than a preference.
2. **B.** *[cross-bearing: see Ch 2 §3 — tags, digests, and what an image reference actually resolves to]*
3. **From a default registry that Kubernetes assumes when you don't name one.** §3 names it and shows the other two defaults hiding in the same short string.
4. **Once.** §2 has the structure.
5. **B.** §4 explains what makes them interchangeable.
6. **C.** §5 names who does.
7. **B.** §7 covers the mechanism and, more importantly, why anyone would want it.
8. **It depends, and what it depends on will surprise you.** §6 has the four-case rule, and it hinges on how you wrote the reference, not on what you configured.

If you got 6+ right: skim. Focus on the ☆ Fixed Points and △ Navigational Hazards callouts, and take all three Taking Your Bearings checkpoints. Do not skip §5 (OCI) or §7 (RuntimeClass) regardless of your score. Those two sections score well on intuition and still repay a careful read.

If you got 3–5 right: read at normal pace. This chapter is calibrated for you.

If you got 0–2 right: read carefully, in its own session. This is the chapter every later chapter rests on, and there's no shortcut through it. Nothing here is difficult; there's simply a lot of new vocabulary, and every term is defined before it's used.

Why This Chapter Matters

Here is a sentence that should stop you: **Kubernetes cannot run a container.** Not one, not ever, on any cluster you will ever touch. Every container on every node is started by software that is not Kubernetes, reached across a boundary that Kubernetes only defines. That is not a limitation someone forgot to fix. It is the design, it is deliberate, and by the time you finish §4 you will be able to draw it.

Chapter 1 told you that an orchestrator is not a runtime *[cross-bearing: see Ch 1 §Soundings A2 — orchestrator versus runtime]*. That was a one-line correction. This chapter turns it into a mechanism.

What you are building here is discrimination: the ability to tell near-identical things apart. That is the competence the whole exam measures, and it is the competence this chapter installs at the deepest level in the stack. A practitioner who has this chapter can be handed any unfamiliar container tool, a runtime they've never heard of, a new build system, somebody's registry product, and place it on a map in about ten seconds. Does this thing *produce* images, *distribute* them, or *run* them, and which specification is it conforming to? That placement instinct separates someone who has memorized the words "containerd, CRI-O, runC" from someone who understands why those three names are not a list of alternatives.

The stakes here are structural, and worth stating plainly once. This is the first content chapter because containerization is the deepest root in the book's dependency graph, not because it's an easy warm-up. A reader who leaves this chapter treating "container" as an undefined primitive will find that Chapter 3's runtime boundary, Chapter 13's failed-image-pull diagnosis, and Chapter 17's entire argument about how Kubernetes is extended each land slightly wrong. Not catastrophically. Just slightly, three times, in ways that are hard to trace back here. Better to spend the time now.

What You'll Learn

By the end of this chapter, you'll be able to:

- **Distinguish** a container from a virtual machine by what each one shares with the host, and explain why that single difference produces every other difference people cite.
- **Explain** what a container image contains, what it deliberately does not contain, and why the correct response to a needed change is a new image rather than an edited container.
- **Describe** how an image is assembled from stacked filesystem layers, and why two images built from a common base share that base rather than duplicating it.
- **Predict** which bytes a given image reference resolves to, including when a tag will silently resolve to something different than it did last week, and when a digest guarantees that it cannot.
- **Trace** the path from a Pod's image field to a running process, naming each component in the chain and stating which specification governs each hop.
- **Choose** an `imagePullPolicy` deliberately, and state what Kubernetes will do by default when you don't.
- **Recognize** the interface-and-implementations pattern that CRI exemplifies, and name the other places Kubernetes applies it.

You'll also be able to look at an unfamiliar container tool and say what layer it operates at, which is how practitioners actually navigate this ecosystem.

§1 — .1 What a Container Actually Is

If you remember nothing else: a container is repeatable because everything it needs travels with it.

Start with what a container buys you, because the definition follows from the benefit. Each container that you run is repeatable; the standardization that comes from having dependencies included means that you get the same behavior wherever you run it [source: k8s-docs-containers-2026-08-23]. That's the whole proposition. Containers decouple applications from the underlying host infrastructure, which is what makes deployment easier across different cloud or operating-system environments [source: k8s-docs-containers-2026-08-23].

Notice what that claim is *not*. It is not "containers are fast." It is not "containers are small." Speed and size are consequences. The proposition is *sameness*: the thing that ran on your laptop is the thing that runs on the node, because the dependencies are not assumptions about the host. They're contents.

Now the comparison everyone reaches for, done at the level that actually explains it.

Virtualization lets you run multiple virtual machines on a single physical server's CPU, isolating applications between VMs so that one application's information cannot be freely accessed by another. Each VM is a full machine running all the components, including its own operating system, on top of virtualized hardware [source: k8s-docs-overview-2026-08-23]. Read that last clause again: *its own operating system*. Ten VMs on a host means ten operating systems booted, patched, and consuming memory before a single line of your application runs.

Containers are similar to VMs, but they have relaxed isolation properties in order to share the operating system among the applications, and *therefore* containers are considered lightweight. Similar to a VM, a container has its own filesystem, share of CPU, memory, process space, and more; because containers are decoupled from the underlying infrastructure, they are portable across clouds and OS distributions [source: k8s-docs-overview-2026-08-23].

Two registers are in use here, and both are correct. The Kubernetes documentation and the CNCF glossary say a container **shares the operating system** [source: cncf-glossary-container-2026-08-24]. That is, in the author's judgment, the phrasing an exam item is likeliest to echo, and the one to recognize on an answer sheet. Practitioners and the container-runtime documentation usually sharpen it: a container **shares the host's kernel**, meaning the part of the operating system that talks to hardware, schedules processes, and enforces boundaries. Everything above the kernel that the application needs, its libraries, its files, its view of the process table, comes from the container itself. That is why a container has its own filesystem and its own process space while still running on somebody else's kernel. [source: docker-docs-what-is-a-container-2026-08-24]

The sharpening is not a stylistic preference; the mechanism only parses one way. Kubernetes describes a sandboxed alternative runtime as one that supplies "a user-space kernel (such as gVisor)" [source: k8s-docs-runtime-class-2026-08-23]. That phrase is only meaningful if the ordinary, non-sandboxed case is the *host's* kernel. Hold both registers: **operating system** as the published wording, **kernel** as the mechanism underneath it.

Now the single most useful move you can make with this material: stop memorizing the comparison table and derive it instead. Every difference between a container and a VM that anybody cites comes out

of that one architectural choice.

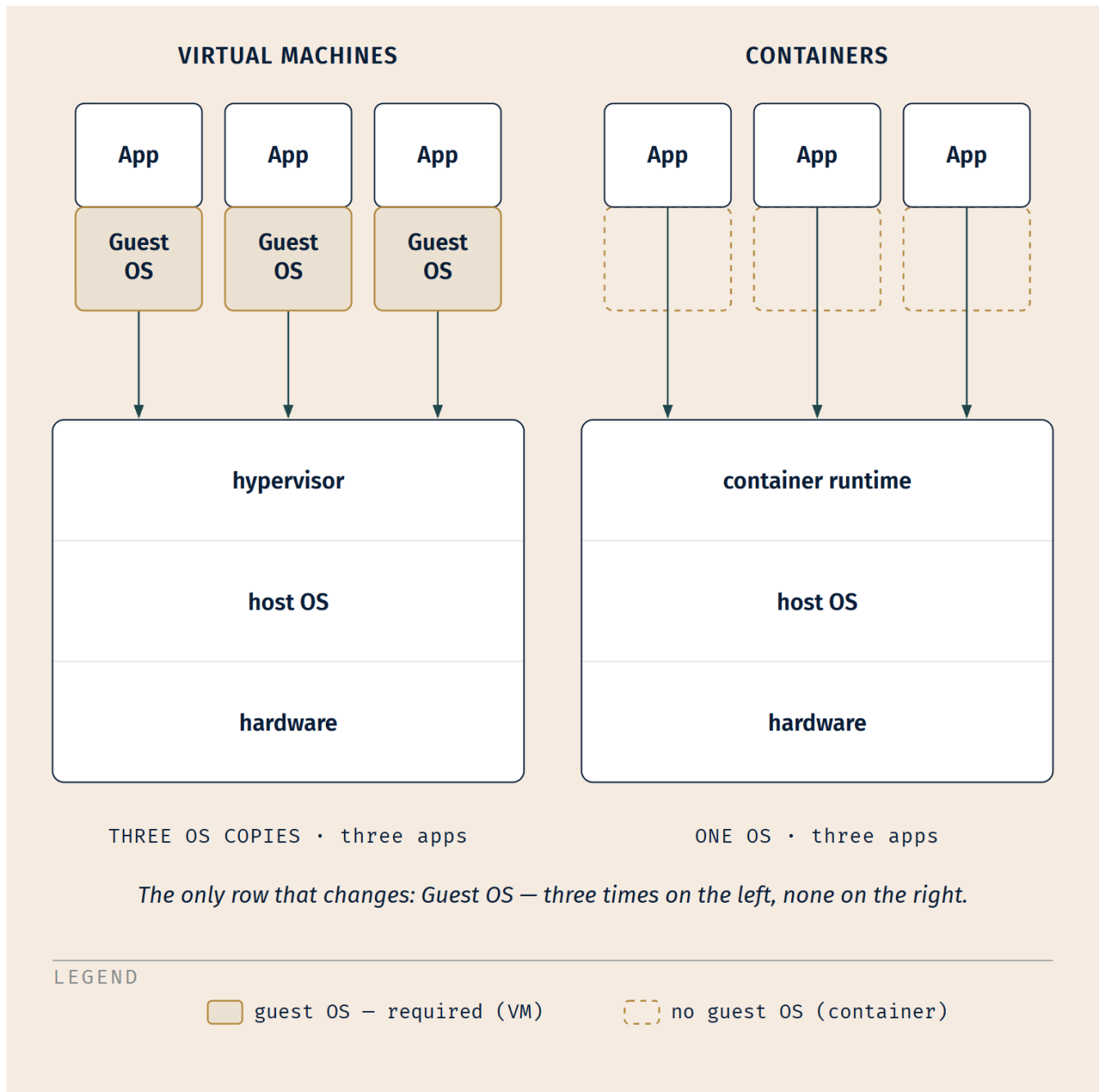


Figure 2-1. The duplication on the left is the entire story. Notice what to look for: the guest-OS row repeats once per workload on the VM side and does not appear at all on the container side. Start-up time, image size, and how many workloads fit on a host are all downstream of that one row.

Derive it yourself. Booting a guest operating system takes time, so VMs start slowly and containers start quickly. A guest operating system occupies disk and memory, so VM images are large and containers are small. Three guest kernels on one host means three times the baseline resource cost, so container density is higher. You do not need three facts. You need one fact and the willingness to follow it.

🔒 **Worth Securing:** "Relaxed isolation" is a *tradeoff*, not a deficiency. The docs use exactly that word, *relaxed* [source: k8s-docs-overview-2026-08-23], and being honest about it is what makes the rest of the picture coherent. You gave up a boundary in exchange for weight, and because it was a trade, it can be renegotiated per workload when a particular workload deserves the boundary back. [cross-bearing: see Ch 2 §7 — when relaxed isolation is not enough]

One forward plant, then we move on. Containers in a Pod are co-located and co-scheduled to run on the same node [source: k8s-docs-containers-2026-08-23]. You do not need to know what a Pod is yet. It is Chapter 5's whole subject, and it comes with a shared network namespace [source: k8s-docs-network-model-2026-08-23] and a lifecycle of its own [cross-bearing: see Ch 5 §1 — the Pod as the unit of scheduling]. For now, register only this: containers are not the unit Kubernetes schedules. Something wraps them.

§2 — .1 What's Inside an Image

A container is a running thing. An image is the package it runs from, and the contents repay precision, because the exam tests the contents and, more usefully, because the negative space in that list explains §1 backwards.

A container image is a ready-to-run software package containing everything needed to run an application: the code and any runtime it requires, application **and** system libraries, and default values for any essential settings [source: k8s-docs-containers-2026-08-23]. Said another way, a container image represents binary data that encapsulates an application and all its software dependencies; images are executable software bundles that can run standalone and that make very well-defined assumptions about their runtime environment. You typically create a container image of your application and push it to a registry before referring to it in a Pod [source: k8s-docs-images-2026-08-23].

That phrase, *very well-defined assumptions about their runtime environment*, is doing quiet work. An image is not self-sufficient. It is *explicit* about what it expects from underneath. And the largest thing it expects from underneath is the one thing it does not carry.

An image contains no kernel. That is not a separate fact to memorize, and it is deliberately not presented here as a quotation, because no authority states it as a negative. It is §1 read in reverse. If containers get the kernel by sharing the host's [source: k8s-docs-overview-2026-08-23], the image cannot be shipping one. A shipped kernel would have to be booted, and booting it is precisely what a virtual machine does and a container does not. Every time you see a question implying that a container image bundles an operating system kernel, you are looking at the same misconception wearing a different hat.

System libraries, though, *are* included [source: k8s-docs-containers-2026-08-23], which is where people get tangled, because system libraries feel like part of the OS. They belong to the operating system's userspace rather than to its kernel, and userspace is exactly what the image supplies, which follows from the same sharing model. Kernel below, everything above it in the crate.

Immutability, and why it's a rule rather than a preference

Containers are intended to be stateless and immutable: you should not change the code of a container that is already running. If you have a containerized application and want to make changes, the correct process is to build a new image that includes the change, then recreate the container to start from the updated image [source: k8s-docs-containers-2026-08-23].

Read the shape of that instruction. It does not say "editing a running container is discouraged." It says the *correct process* is different in kind: build, then recreate. Two steps, neither of which is "edit."

This is the habit that readers with virtual-machine or bare-metal backgrounds find hardest to break, and the reason deserves naming, because the reason is honorable. If you have spent years administering long-lived servers, the skill you built was *repair*: log in, diagnose, fix in place, verify. That skill is genuinely valuable and it is genuinely the wrong instinct here. In a container world, the fix goes in the image and the container gets replaced. Repair-in-place produces a running thing that no image can reproduce, which forfeits the one property you adopted containers to get: sameness.

The payoff is concrete. Image immutability is exactly what makes quick and efficient rollbacks possible [source: k8s-docs-overview-2026-08-23]. You can go back to the previous version because the previous version still exists, unchanged, as a distinct artifact. Edit containers in place and there is nothing to go back to.

Layers

An image is not a single opaque blob. The OCI Image Format encompasses the image manifest, **filesystem layer serialization**, and image configuration needed to launch applications on target platforms [source: oci-overview-2026-08-23]. In other words, the contents arrive as a set of filesystem layers, described by a manifest that names them, plus a configuration that says how to start the thing. (The image *manifest* here is an OCI artifact — a different thing from the Kubernetes manifests, the YAML files, you'll meet in Chapter 4 [*cross-bearing: see Ch 4 §2 — the anatomy of a record*].)

Three consequences follow from that structure, and all three matter downstream.

The layers stack, in order. The manifest lists them as an ordered set: a base at the bottom, and each subsequent layer applied on top of what came before. The image you finally run is the result of that stack resolved into a single filesystem view.

A shared base is a shared layer. If two images are built starting from the same base, both of their manifests name that same base layer. The layer has one identity, so it is one object: stored once and transferred once, rather than duplicated per image.

Position in the stack determines rebuild cost. Because each layer is applied on top of the ones beneath it, changing a lower layer invalidates everything above it. This is why the conventional advice is to put the parts that change least at the bottom.

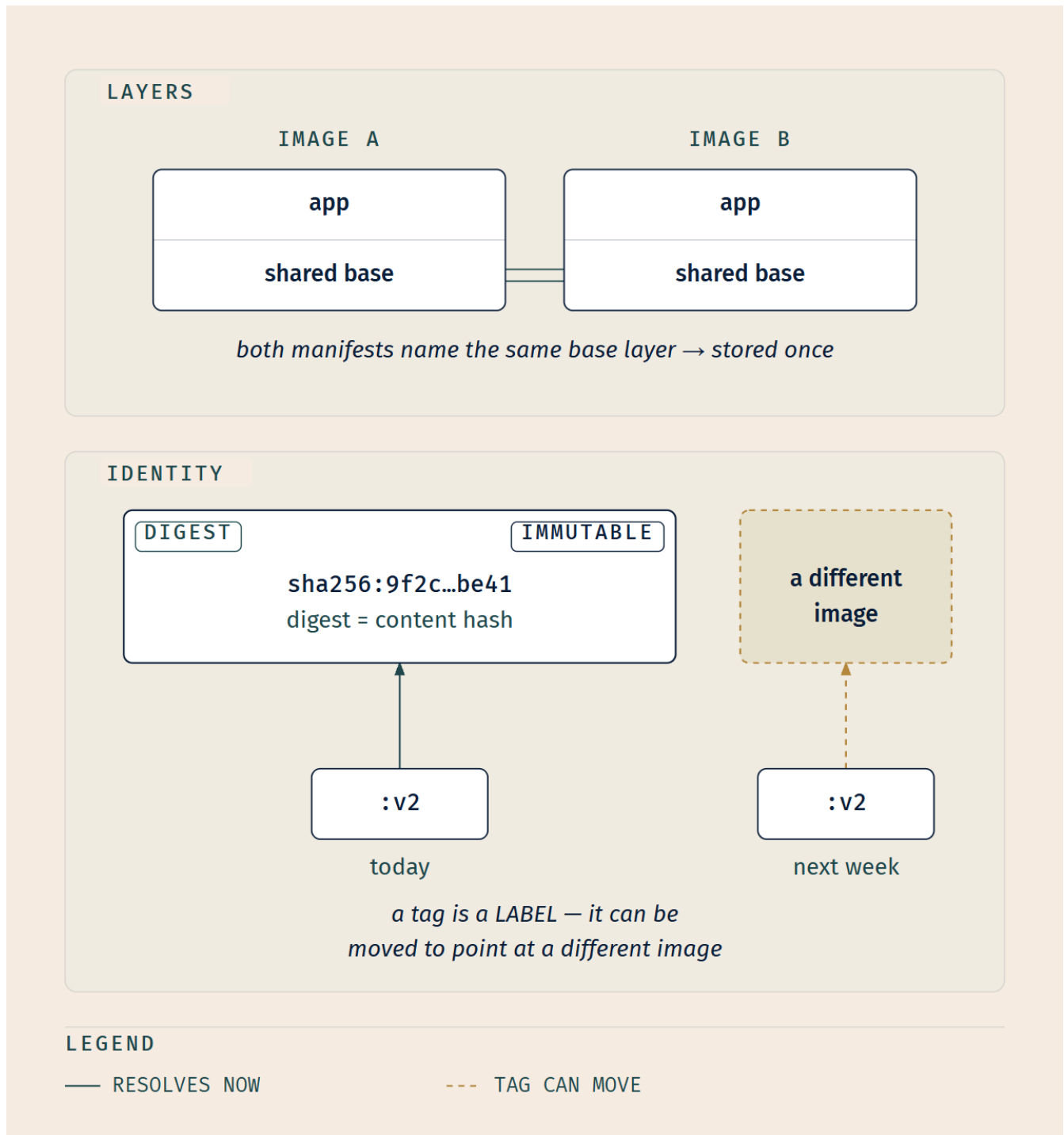


Figure 2-2. Two ideas in one frame, because §3 depends on the right half. What to notice: the dashed arrow. The tag :v2 is attached to the image, not part of it, and an attachment can be re-attached.

Note what the layer structure buys forward: the layers described here are precisely the thing that a specification standardizes, which is why any registry can serve any image and any conformant runtime can unpack one [cross-bearing: see Ch 2 §5 — the OCI image specification].

Building images without writing build files

You do not have to hand-author a build file to produce an image, and the exam's ecosystem awareness extends to knowing the alternative exists.

A **buildpack** is software that transforms application source code into runnable artifacts by analyzing the code and determining the best way to build it. A **builder** is an OCI image containing an ordered combination of buildpacks and a build-time base image, a lifecycle binary, and a reference to a runtime base image. The base images come in a pair: the *build image* is the environment in which the buildpacks run, and the *run image* is the base for the final application image; that pairing is called the **stack**. A **platform**, either the pack CLI or a CI system, orchestrates the process by invoking the lifecycle binary together with the buildpacks and the application source to produce a runnable OCI image. The lifecycle runs in phases: **detect** (determine which buildpacks apply), **build** (compile and assemble the application), and **export** (create the final OCI image with reproducible layers). Cloud Native Buildpacks is a CNCF graduated project [source: buildpacks-concepts-2026-08-23].

🔒 **Worth Securing:** The build-image / run-image split is the idea to take from Buildpacks even if you never use them. The environment that *compiles* your application and the environment that *runs* it do not have to be the same environment, and they usually shouldn't be. Compilers, headers, and build tooling are attack surface that your production image has no use for. Buildpacks make that separation structural rather than a discipline you have to remember. The same instinct scales down to base-image choice generally: a smaller base carries fewer things that can be exploited.

🔍 **Closer Look:** The export phase produces *reproducible* layers [source: buildpacks-concepts-2026-08-23], meaning the same input reliably produces the same layer bytes rather than layers that differ run to run because of timestamps or file ordering. (That gloss on "reproducible" is the author's, not the specification's wording.) The property is deeper than the exam requires, but it's the hinge on which supply-chain verification swings: you cannot meaningfully attest to an artifact whose bytes change when nothing changed. [cross-bearing: see Ch 12 — signing, attestation, and the software supply chain]

Supply-chain security, meaning scanning, signing, and bills of materials, is a real concern that attaches to everything in this section, and it is not this chapter's. It gets a full treatment later [cross-bearing: see Ch 12 — securing the image supply chain].

§3 — Registries, Tags, and Digests

This section carries the chapter's sharpest single distinction, and it is one most practitioners carry wrong for years without consequence, right up until the afternoon it costs them.

An image reference has parts

Container images are usually given a name such as `pause`, `example/mycontainer`, or `kube-apiserver`. Images can also include a registry hostname, for example `fictional.registry.example/imagename`, and possibly a port number as well. **If you don't specify a registry hostname, Kubernetes assumes that you mean the Docker public registry.** After the image name, you can add a tag or a digest. Tags let you identify different versions of the same series of images. **If you don't specify a tag, Kubernetes assumes you mean the tag `latest`.** So `busybox` is equivalent to `docker.io/library/busybox:latest` [source: k8s-docs-images-2026-08-23].

Here are the defaults as a table, because that is what a table is for:

What you write	What it resolves to	Which default filled the gap
<code>busybox</code>	<code>docker.io/library/busybox:latest</code>	registry and tag
<code>busybox:<version></code>	<code>docker.io/library/busybox:<version></code>	registry
<code>registry.k8s.io/pause</code>	<code>registry.k8s.io/pause:latest</code>	tag
<code>registry.k8s.io/pause:3.5</code>	exactly that	none
<code>registry.k8s.io/pause@sha256:1ff6...</code>	exactly those bytes	none

The `busybox` equivalence and both `registry.k8s.io/pause` forms are the documentation's own examples [source: k8s-docs-images-2026-08-23]; `<version>` is a placeholder, not a real published tag.

📌 **Snag:** A bare name is not a bare name. `busybox` is three defaults stacked in a trench coat: a registry you didn't name, a namespace you didn't name, and a tag you didn't name. Every one of the three is a decision somebody made on your behalf, and at least two of them will matter to you eventually.

The distinction this section exists for

Tags let you identify different versions of the same series of images. Digests are a unique identifier for a specific version of an image, **a hash of the image's content**, and are immutable; **tags can be moved to point to different images** [source: k8s-docs-images-2026-08-23].

Sit with the asymmetry. A digest is *derived from* the image. Change one byte of the image and you get a different digest, necessarily, because that is what a content hash is. You cannot move a digest to point at different content, in the same way that you cannot move the number 4 to mean 5.

A tag is *attached* to the image. It is a label, and labels come off.

☆ **Fixed Point:**

A tag identifies a series and can be moved to point at a different image. A digest is a hash of the image's content and is immutable. Two pulls of `myapp:v2` a week apart are not guaranteed to be the same bytes. Two pulls of an image by digest are.

If you recite one sentence from §1 through §3 cold, recite that one.

⚠ Navigational Hazards

You should avoid using the `:latest` tag when deploying containers in production, as it is harder to track which version of the image is running and more difficult to roll back properly. Instead, specify a meaningful tag such as `v1.42.0` and/or a digest [source: k8s-docs-images-2026-08-23].

That is the documented guidance, and most candidates absorb it as a naming-hygiene rule: untidy, faintly unprofessional, not really *dangerous*. That reading is incomplete, and the incompleteness is the trap. The `:latest` problem has a second half that has nothing to do with naming. **The tag you write also silently determines when Kubernetes will go looking for a new copy of the image.** Same field, second effect, and the caution above doesn't mention it. §6 completes it. [cross-bearing: see Ch 2 §6 — *the tag determines the default pull policy*]

Logbook Entry:

The failure looks like this from the inside, and it pays to know the shape of it before you meet it.

A deployment goes out. It behaves badly in a way nobody can reproduce locally. The manifest is checked: unchanged, same commit, same image reference as the version that has been running quietly for three weeks. So the manifest is checked again, this time by two people, out loud, line by line, because the alternative explanations are all worse. It is genuinely unchanged. The reference says `:v2` and it said `:v2` before.

Someone eventually proposes the rollback, which is when the afternoon turns philosophical. Rolling back to the previous configuration produces the *same reference*, which produces the *same bytes*, which produces the same bad behavior. The rollback rolls back to exactly where you already are. There is a particular quality of silence in a room where four competent people have just realized that the thing they were treating as an identity was a label the whole time.

Nobody moved the tag maliciously. Somebody upstream rebuilt `:v2` with a dependency bump, which is a completely reasonable thing to do to a tag, because a tag identifies a series. The manifest was never the problem. The mental model was. And the tell, in hindsight, was that we spent an hour proving the manifest hadn't changed instead of one minute asking whether an unchanged manifest guarantees unchanged bytes.

Registries, briefly

A registry is where images live between being built and being run. You push there and nodes pull from there [source: k8s-docs-images-2026-08-23]. It is a distribution layer, and it speaks a standardized API for

the purpose, which is a fact §5 will hand you *[cross-bearing: see Ch 2 §5 — the distribution specification]*.

Private registries may require credentials to read images from them. Credentials can be supplied by: configuring nodes to authenticate to the private registry; a kubelet credential provider that fetches credentials dynamically; pre-pulled images; specifying `imagePullSecrets` on a Pod, which references a Secret of type `kubernetes.io/dockerconfigjson`; or vendor-specific and local extensions [source: k8s-docs-images-2026-08-23].

Five paths, named and left there deliberately. The one most teams reach for, `imagePullSecrets`, requires understanding Secrets, which you do not have yet and which arrive with their own security caveats. *[cross-bearing: see Ch 4 §4 — Secrets, and the `dockerconfigjson` type]* Registry access is also a genuine security boundary rather than a convenience feature *[cross-bearing: see Ch 12 — restricting who can pull what]*.

🕒 Taking Your Bearings #1: Containers, Images, and Identity

Four questions on §1 through §3. Answer before reading on.

1. A team needs to run a workload that requires a different operating-system kernel version than the one on the host machines. Which of the following is true?

A) Use a container and specify the required kernel in the image B) Use a container and set the kernel version in the container's configuration defaults C) A container is the wrong instrument here; the workload needs a virtual machine or a host with the required kernel D) Use a container built from a base image matching the required kernel version

2. A running container's application needs a configuration change that must survive the container being replaced. Which describes the correct process?

A) Edit the file in the container; the change is written to the image's top layer B) Build a new image containing the change, then recreate the container from it C) Edit the file and restart the container so the change is committed D) Edit the file and record it in the Pod spec so it is reapplied on restart

3. You deploy a manifest referencing `myapp:v2` in April. In May, you redeploy the identical manifest — no changes of any kind — to the same cluster. Which statement is correct?

A) The bytes are guaranteed identical, because the reference is identical B) The bytes are guaranteed identical, because tags are immutable by specification C) The bytes may differ, because a tag can be moved to point at a different image D) The bytes may differ, because the kubelet's cache expires and the image is re-pulled

4. A Pod specifies the image `redis`. Nothing else. What does that reference resolve to?

A) `redis:latest` from the cluster's configured default registry, whatever that is B)
`docker.io/library/redis:latest` C) `registry.k8s.io/redis:latest` D) It fails to resolve; a registry
 hostname is mandatory

Answers with Explanations

1 — C. A container shares the host's operating system [source: [k8s-docs-overview-2026-08-23](#)], so a kernel-version requirement cannot be satisfied by the container. A virtual machine boots its own guest operating system on virtualized hardware [source: [k8s-docs-overview-2026-08-23](#)], which is exactly the capability needed.

- **A is wrong**, and it is a very common misconception in this chapter: an image bundles application and system libraries [source: [k8s-docs-containers-2026-08-23](#)], and since the kernel comes from the host, the image is not supplying one.
- **B is wrong** because the "default values for essential settings" an image carries are application settings, not kernel selection.
- **D is wrong**, and it is the tempting one, because base images genuinely do carry a distribution's userspace: the libraries and files that make an image "look like" Debian or Alpine. What they do not carry is that distribution's kernel. Choosing a base image changes what is above the kernel, never the kernel itself.

2 — B. Containers are intended to be stateless and immutable; the correct process to make a change is to build a new image that includes it, then recreate the container from the updated image [source: [k8s-docs-containers-2026-08-23](#)].

- **A is wrong** in mechanism and in outcome. Writes in a running container do not update the image; the image is the artifact you built, and nothing about running a container edits it. What you get is a running thing no image can reproduce.
- **C is wrong**. "Committing" a running container's state is not part of the documented workflow, and it inverts the direction of the process: images produce containers, not the other way round.
- **D is wrong**. The Pod spec references an image; it is not a place to record file edits, and re-applying an edit on every restart is exactly the repair-in-place habit immutability replaces.

3 — C. Tags can be moved to point to different images [source: [k8s-docs-images-2026-08-23](#)]. An identical reference is not a guarantee of identical content; only a digest gives you that, because a digest is a hash of the image's content and is immutable [source: [k8s-docs-images-2026-08-23](#)].

- **A is wrong** for exactly that reason. This is the prior many readers arrive with.
- **B is wrong** and inverts the specification: digests are immutable; tags are movable.
- **D is wrong**, and it is worth separating carefully, because it confuses two different sections. Caching is keyed on digest, and when the kubelet re-fetches is a *pull-policy* question (§6), not an *identity* question. The reason the bytes can differ is that the tag itself may now name different content, which is true whether or not anything was cached.

4 — B. Omitting the registry hostname means Kubernetes assumes the Docker public registry, and omitting the tag means it assumes `latest`; the documentation's own worked example is `busybox` $\equiv$ `docker.io/library/busybox:latest` [source: [k8s-docs-images-2026-08-23](#)].

- **A is wrong** because the default is not a cluster-configurable "whatever that is." The assumption is specific.
- **C is wrong.** `registry.k8s.io` hosts Kubernetes project images and is used in the docs' examples, but it is not the assumed default.
- **D is wrong.** The whole point is that the hostname is optional and defaulted.

Checkpoint: You've Now Mastered ✓ Why a container is lightweight, and why every other container-versus-VM difference follows from that ✓ What an image contains, what it cannot contain, and why immutability is the process rather than a preference ✓ The tag/digest distinction, the chapter's sharpest single fact ✓ The three defaults hiding inside a bare image name

If you scored 0–2: stop here and reread §3, specifically the ☆ Fixed Point and the table of reference forms. §4 through §6 all assume you can look at an image reference and say what it resolves to. That is about eight minutes of rereading and it will save you the rest of the chapter.

Two components left in the stack, and then you'll be able to draw the whole path from a manifest to a running process.

§4 — The Container Runtime Interface

Time to pay off the opening. Kubernetes cannot run a container, and here is the machinery.

The runtime is a node component

The container runtime is a fundamental component that empowers Kubernetes to run containers effectively. It is responsible for managing the execution and lifecycle of containers within the Kubernetes environment. Kubernetes supports container runtimes such as **containerd**, **CRI-O**, and **any other implementation of the Kubernetes CRI (Container Runtime Interface)** [source: [k8s-docs-containers-2026-08-23](#)] [source: [k8s-docs-cluster-architecture-2026-08-23](#)].

Read the third item in that list carefully, because it is not padding. It is a promise about the shape of the system: the list of supported runtimes is *open*. Two are named because two are widely deployed; the qualifying condition is conformance to an interface, not membership in a list.

The runtime is a **node** component. It runs on every machine that runs workloads, alongside the kubelet and (optionally) kube-proxy [source: [k8s-docs-components-2026-08-23](#)]. A container runtime, containerd or CRI-O, must be installed on every node [source: [k8s-docs-setup-tooling-2026-08-23](#)]. Not on the control plane's behalf, not centrally: on each node, because that is where containers actually run.

The **kubelet** is the agent that runs on each node in the cluster. It makes sure that containers are running in a Pod: the kubelet takes a set of PodSpecs provided through various mechanisms and ensures that the containers described in those PodSpecs are running and healthy. Notably, the kubelet doesn't manage containers which were not created by Kubernetes [source: k8s-docs-cluster-architecture-2026-08-23].

So the kubelet is the component that *wants* containers to exist. The runtime is the component that *makes* them exist. Between them is the interface.

CRI is an interface, not an implementation

Kubernetes' extension points include, under infrastructure extensions, the "container runtime (CRI, the Container Runtime Interface, to support alternative container runtimes)" [source: k8s-docs-extending-kubernetes-2026-08-23]. That single parenthetical is the whole design in one line. CRI exists *in order to* support alternatives. It is a socket, and the socket is the product.

And beneath the CRI runtime there is one more hop. Docker donated its container runtime, **runC**, to the Open Container Initiative to serve as the cornerstone of that effort [source: oci-overview-2026-08-23], and an OCI runtime's job is defined by the runtime specification: it runs a filesystem bundle that has been unpacked on disk [source: oci-overview-2026-08-23]. That is the lowest hop, the one where a process actually starts.

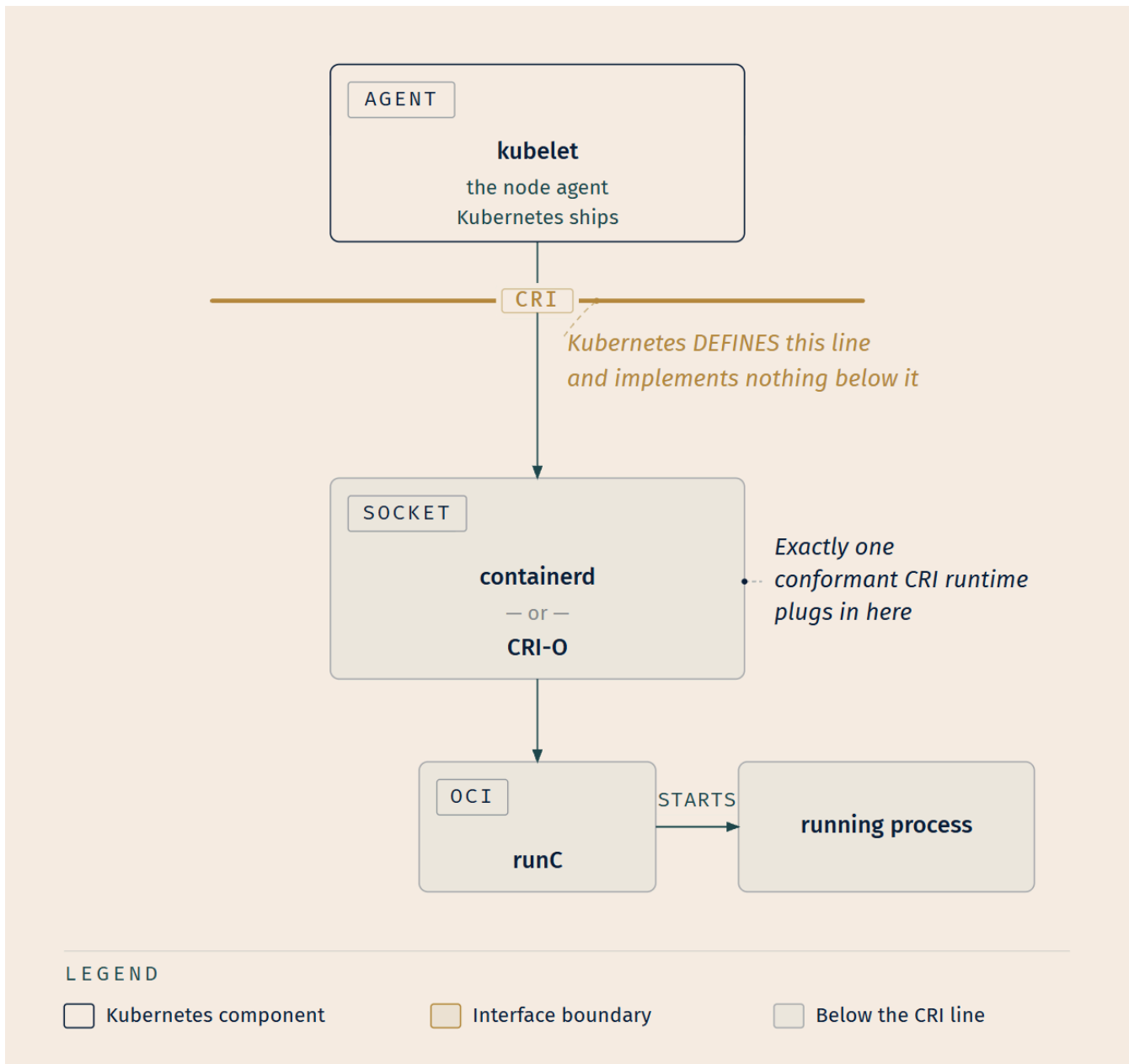


Figure 2-3. What to notice is the socket, not the boxes. containerd and CRI-O are not two parallel paths through the system; they are two things that fit the same opening. Swap one for the other and the kubelet above does not change.

☆ **Fixed Point:**

kubelet → CRI → containerd or CRI-O → runC → a running process.

Kubernetes defines the CRI. It does not implement it. Kubernetes never starts a container itself.

📌 **Snag:** "Docker" is four things wearing one name: a company, a command-line tool, a build format, and, historically, a runtime. So when someone says "Kubernetes runs Docker containers," at least three of those four meanings are doing no work at all. These are easy to confuse, and the confusion is historical rather than careless; for years the shorthand was accurate enough. The precise statement now is that Kubernetes runs containers via a CRI-conformant runtime, of which several exist [source: k8s-docs-containers-2026-08-23]. The era that produced the shorthand is Chapter 3's material [*cross-bearing: see Ch 3 §1 — how the cluster got the shape it has*].

Both of the runtimes named in the documentation, containerd and CRI-O, are CNCF **graduated** projects, meaning they are considered stable, widely adopted, and production ready [source: cncf-project-maturity-levels-2026-08-23]. That matters less as trivia than as evidence: the pluggable position in the middle of Figure 2-3 is not theoretical. It has two mature occupants.

Usually, you can allow your cluster to pick the default container runtime for a Pod. If you need more than one container runtime in your cluster, you can specify a RuntimeClass for a Pod to make sure Kubernetes runs those containers using a particular container runtime [source: k8s-docs-containers-2026-08-23]. Most of the time, then, the runtime is invisible to you, which is exactly why §7 exists.

The pattern to name now

📌 **Worth Securing:** Give this move a name in your head, because you are about to see it three more times: **Kubernetes defines an interface and lets the ecosystem implement it.** The published extension points include not just CRI for runtimes but CSI (the Container Storage Interface) for storage types and CNI (the Container Network Interface) for pod networking, alongside device plugins and API extensions [source: k8s-docs-extending-kubernetes-2026-08-23]. One design instinct behind all of them — and the four this book tracks as its canon (CRI, CNI, CSI, API extensions) are the sockets the exam cares about. You are looking at the first.

That is not a stray observation; it is the spine of the book's closing argument. [*cross-bearing: see Ch 9 §1 — CNI and pod networking*] [*cross-bearing: see Ch 11 — CSI and storage drivers*] [*cross-bearing: see Ch 6 §8 — CRDs and extending the API*] [*cross-bearing: see Ch 17 §4 — the four pluggable interfaces, collected*]

Two more pointers, then §5. The kubelet gets its full treatment as one node component among several [*cross-bearing: see Ch 3 §3 — node components in context*]. And knowing there is a layer below the Kubernetes API is what makes a particular diagnostic tool make sense: `crictl` is a node-level diagnostic listed under debugging Kubernetes nodes [source: k8s-docs-debug-overview-2026-08-23], and it is occasionally exactly what you need [*cross-bearing: see Ch 13 §5 — debugging nodes with crictl*].

§5 — .11 The Open Container Initiative

You now know that Kubernetes talks to a runtime through an interface. Underneath *that* is a second question the interface does not answer: who decided what an image looks like, how it moves across a

network, and what it means to run one? Not Kubernetes. Kubernetes is a consumer of those answers, not their author.

What the OCI is

The Open Container Initiative is an **open governance structure** for the express purpose of creating open industry standards around container formats and runtimes. It was established in **June 2015** by Docker and other leaders in the container industry [source: oci-overview-2026-08-23], and it operates under the Linux Foundation [source: oci-overview-2026-08-23].

Governance structure. Not software. It publishes documents.

☆ Fixed Point:

The OCI is a governance body that publishes three specifications: the image specification, the distribution specification, and the runtime specification. It is not a runtime, not a company, and not a product you install.

The three specifications, each explaining a section you already read

Here is the satisfying part. Each specification retroactively explains something from earlier in this chapter.

- The **Image Specification** (`image-spec`) defines the OCI Image Format, which encompasses the image manifest, filesystem layer serialization, and image configuration needed to launch applications on target platforms [source: oci-overview-2026-08-23]. That is §2's layers. The stack-with-a-shared-base is not a convention that happens to be widely followed; it is a published format.
- The **Distribution Specification** (`distribution-spec`) reached v1.0 in May 2020 and was introduced to standardize the API to distribute container images [source: oci-overview-2026-08-23]. That is §3's registries. The reason any registry can serve an image built by any tool is that the transfer protocol is specified.
- The **Runtime Specification** (`runtime-spec`) outlines how to run a "filesystem bundle" that is unpacked on disk [source: oci-overview-2026-08-23]. That is §4's bottom hop. runC's job description is a document.

A **filesystem bundle** is the artifact in the middle of that last item, and it earns a clause of definition rather than staying a term you only meet in an answer key. It is the unpacked, on-disk form of an image: the container's filesystem contents plus the configuration a runtime needs in order to start it.

And the flow that connects them, in three beats: at a high level, an OCI implementation would download an OCI Image, then unpack that image into an OCI Runtime **filesystem bundle**; at that point the bundle would be run by an OCI Runtime [source: oci-overview-2026-08-23].

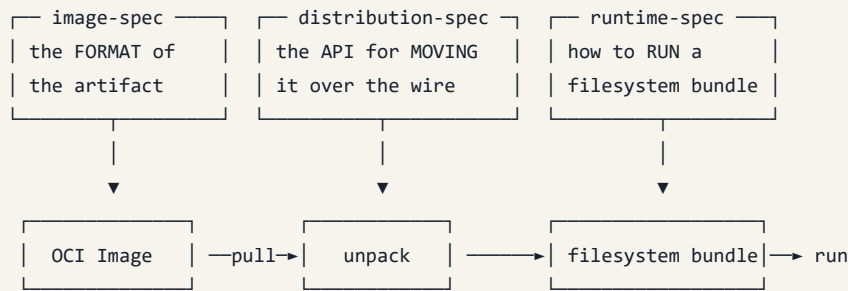


Figure 2-4. Specifications on top, artifact lifecycle underneath, one column each. What to notice: nothing in this figure is Kubernetes. Compare it against Figure 2-3. Those are two different planes, and the next callout is about exactly that.

🔑 **Mnemonic: Build it, ship it, run it** — image-spec, distribution-spec, runtime-spec. Three specs, in the order the flow uses them.

⚠️ Navigational Hazards

CRI and OCI are different layers, and conflating them is the hardest discrimination in this chapter.

CRI is how *Kubernetes* talks to a container runtime. It exists because Kubernetes wanted to support alternative runtimes [source: k8s-docs-extending-kubernetes-2026-08-23]. Its two endpoints are the kubelet and a runtime. It is a Kubernetes concern; a container system that has never heard of Kubernetes has no use for it.

OCI is how *images* are formatted and distributed, and how *bundles* are executed [source: oci-overview-2026-08-23]. Its endpoints are build tools, registries, and runtimes. It is an industry concern; it would exist, unchanged, in a world where Kubernetes was never written.

Two three-letter abbreviations. Both involve the word "runtime." Both sit underneath your Pod. They are easy to confuse, and the confusion is not carelessness, since a single component can genuinely participate in both planes at once. The discipline is to ask which *direction* you're looking. Up toward Kubernetes, that's CRI. Sideways toward the rest of the industry, that's OCI. Lay Figure 2-3 and Figure 2-4 side by side; the planes are drawn in the same grammar precisely so you can see that they are different planes.

🔍 **Closer Look:** Notice the dates. The OCI was founded in June 2015, and the distribution specification did not reach v1.0 until May 2020 [source: oci-overview-2026-08-23], arriving well after the other two. That ordering tells you something about which problems felt urgent. The industry settled *what an image is* and *what it means to run one* before it formalized *how it moves*. This is depth beyond what the exam asks. What the exam is more likely to ask — again the author's judgment — is which specification governs the registry API, and the answer is the one that arrived last.

runC closes the loop: Docker donated its container runtime, runC, to the OCI to serve as the cornerstone of this new effort [source: oci-overview-2026-08-23]. The company that popularized containers gave away the piece that runs them. That is what an open governance structure is *for*, and it is the same institutional instinct you will meet again when the book turns to how CNCF itself is governed [cross-bearing: see Ch 17 §2 — CNCF governance and project maturity].

🕒 Taking Your Bearings #2: Who Runs It, and Who Standardized It

Four questions on §4 and §5. At least one of them requires holding both sections at once.

1. Which component in a standard Kubernetes cluster actually creates and starts a container?

A) The kube-apiserver, via the control plane B) The kubelet, directly C) A CRI-conformant container runtime such as containerd or CRI-O D) The kube-scheduler, at bind time

2. What is the Open Container Initiative?

A) A container runtime maintained by the Linux Foundation B) An open governance structure that publishes specifications for container formats and runtimes C) A company formed by Docker in 2015 to commercialize container tooling D) A CNCF graduated project providing a registry implementation

3. A colleague describes a component as "the thing that defines how an unpacked filesystem bundle on disk gets executed." Which governs that, and what is the artifact called?

A) The CRI; the artifact is a container image B) The CRI; the artifact is a filesystem bundle C) The OCI runtime specification; the artifact is a filesystem bundle D) The OCI image specification; the artifact is an image manifest

4. Match the concern to the specification that standardizes it: (i) the layout of image layers and the manifest, (ii) the API a node uses to fetch an image from a registry, (iii) executing an unpacked bundle.

A) i = runtime-spec, ii = image-spec, iii = distribution-spec B) i = image-spec, ii = distribution-spec, iii = runtime-spec C) i = distribution-spec, ii = runtime-spec, iii = image-spec D) i = image-spec, ii = runtime-spec, iii = distribution-spec

Answers with Explanations

1 — **C**. The container runtime is responsible for managing the execution and lifecycle of containers, and Kubernetes supports containerd, CRI-O, and any other implementation of the CRI [source: k8s-docs-containers-2026-08-23].

- **A is wrong**. The API server exposes the Kubernetes HTTP API [source: k8s-docs-components-2026-08-23]; it is nowhere near a container process.

- **B is wrong**, and it is the most tempting distractor. The kubelet *ensures* that the containers described in PodSpecs are running and healthy [source: k8s-docs-cluster-architecture-2026-08-23]. It is the component that wants them to exist, not the one that creates them. It reaches a runtime across the CRI to do that.
- **D is wrong**. The scheduler assigns Pods to nodes [source: k8s-docs-components-2026-08-23] and then its work is finished.

2 — B. The OCI is an open governance structure for the express purpose of creating open industry standards around container formats and runtimes, established in June 2015 [source: oci-overview-2026-08-23].

- **A is wrong**, and this is the most frequently held misconception about the OCI. It publishes specifications; it does not ship a runtime. runC was *donated to* it [source: oci-overview-2026-08-23], which is a different relationship.
- **C is wrong**. Docker was a founding participant, along with other container-industry leaders, but the OCI is a governance structure rather than a company, and "commercialize" inverts its purpose.
- **D is wrong**. It is not a CNCF project and does not provide a registry; it specifies the *API* that registries implement.

3 — C. The runtime specification outlines how to run a filesystem bundle that is unpacked on disk [source: oci-overview-2026-08-23]. The term "filesystem bundle" is the specification's own.

- **A is wrong** on both counts: wrong governing body and wrong artifact, since an image is what gets *unpacked into* a bundle.
- **B is wrong** on the governing body. This is the OCI/CRI conflation the $\triangle$ callout in §5 exists for. The artifact is right, the layer is not. The CRI governs how Kubernetes talks to a runtime, not how a bundle executes.
- **D is wrong**. The image specification defines the format of the image, meaning manifest, layer serialization, configuration [source: oci-overview-2026-08-23], not its execution.

4 — B. The image specification encompasses the image manifest, filesystem layer serialization, and image configuration; the distribution specification standardizes the API to distribute container images; the runtime specification outlines how to run a filesystem bundle [source: oci-overview-2026-08-23].

- **A is wrong** on all three. It assigns the artifact's internal layout to *runtime-spec*, which governs execution, and hands the registry API to *image-spec*, which governs format.
- **C is wrong** on all three in the opposite rotation: it makes *distribution-spec* responsible for the manifest and *runtime-spec* responsible for the transfer over the wire.
- **D is wrong** on two, and it is the tempting one precisely because it gets *image-spec* right. It then swaps *moving* and *running*: the registry API is *distribution-spec* (v1.0, May 2020), and executing an unpacked bundle is *runtime-spec*. If you picked D, the fix is to run the lifecycle in order, image exists, then it moves, then it runs, and read the specification names off that sequence.

Checkpoint: You've Now Mastered ✓ The full path from a Pod's image field to a running process, and which component owns each hop ✓ That Kubernetes defines the CRI and implements nothing below it ✓ The OCI as a governance body with three specifications, and which artifact each one governs ✓ The CRI/OCI boundary, the discrimination that Chapter 17 will lean on

Two sections left, and both are about behavior rather than structure: when Kubernetes goes looking for an image, and how strong the walls around a container are.

§6 — When Kubernetes Pulls, and When It Doesn't

This is the fiddliest material in the chapter, and in the author's judgment the highest value per minute of anything in it. The rules are small, entirely conditional, and almost nobody guesses the defaults correctly.

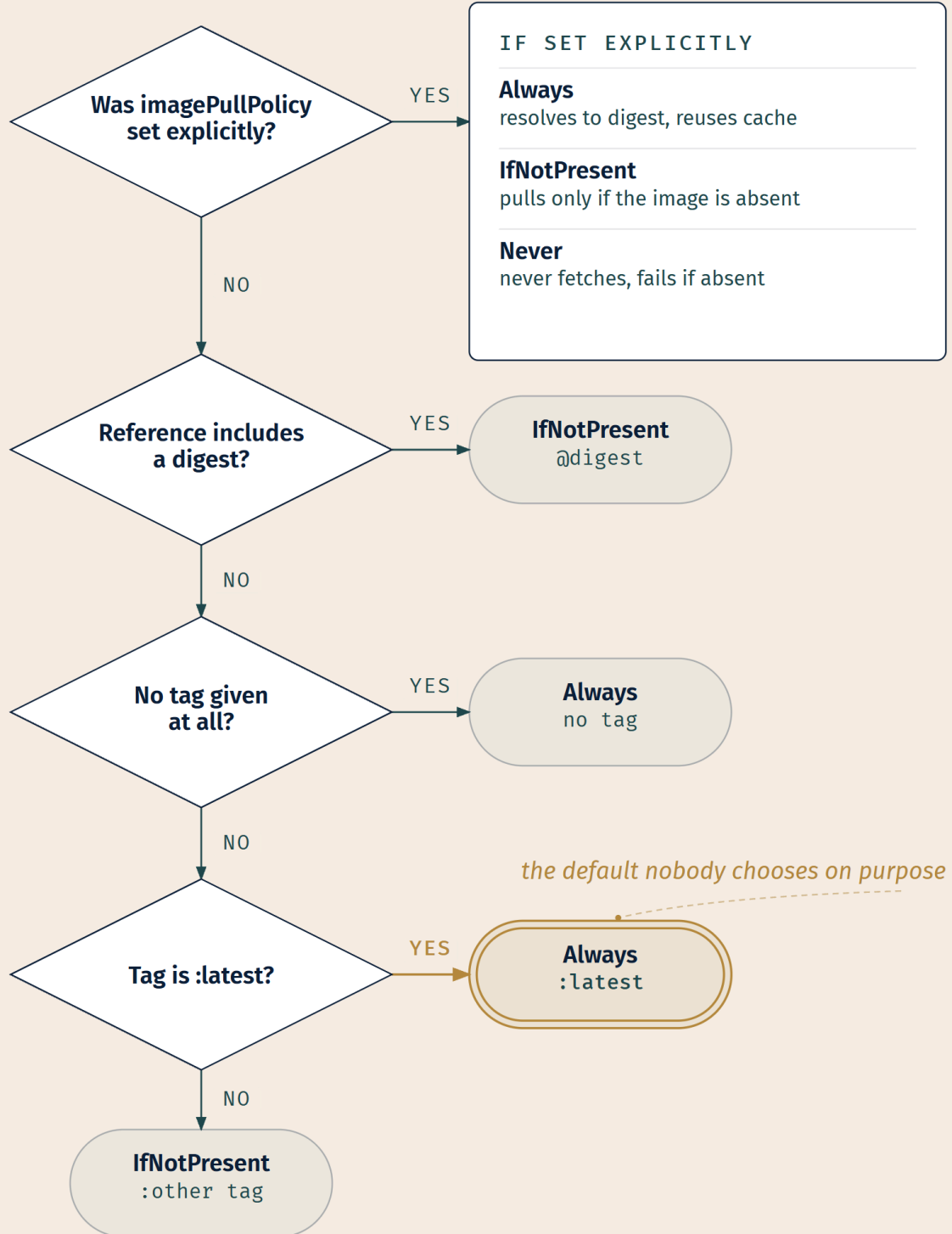
Dead Reckoning: Three pull policies. **IfNotPresent:** the image is pulled only if it is not already present locally. **Always:** every time the kubelet launches a container, it queries the container image registry to resolve the name to an image digest; if the kubelet has a container image with that exact digest cached locally, it uses its cached image, otherwise it pulls the image with the resolved digest. **Never:** the kubelet does not try fetching the image; if the image is somehow already present locally, the kubelet attempts to start the container, otherwise startup fails.

Four defaults, applied when `imagePullPolicy` is omitted. With a **digest:** `IfNotPresent`. With the tag `:latest`: `Always`. With **no tag** specified: `Always`. With a tag **other than** `:latest`: `IfNotPresent`.

Once a Pod is created, `imagePullPolicy` is not updated if the image's tag or digest changes later.

When the kubelet cannot pull an image, the container sits in `ImagePullBackOff`. That means the container could not start because Kubernetes could not pull the image, for reasons such as an invalid image name or pulling from a private registry without credentials. The **BackOff** part indicates that Kubernetes will keep trying, with an increasing back-off delay, up to a compiled-in limit of 300 seconds (5 minutes). [source: k8s-docs-images-2026-08-23]

That block is the whole exam surface for this section, stated flat. Now the two things worth saying about it.



LEGEND



Decision



Outcome



Unchosen default



Branch

Figure 2-5. What to notice, and it isn't the branches: the right-hand side of this tree is reached by *not making a decision*. The reference form you chose for identity reasons quietly chose your pull behavior too.

That is the second half of §3's `:latest` hazard, now complete [*cross-bearing: see Ch 2 §3 — the `:latest` naming caution*]. Writing `:latest` is not merely untidy. It flips the default from `IfNotPresent` to `Always` [source: k8s-docs-images-2026-08-23], which means the kubelet consults the registry on every container launch. So the tag that made you unsure which version is running is also the tag that maximizes the number of opportunities for the answer to change. Two problems, one field, and the documentation's caution names only the first.

The `Always` behavior also repays a careful reading, because its name oversells it. `Always` does not mean "always download." It means always *check*: resolve the name to a digest at the registry, and if the local cache already holds that exact digest, use the cached copy [source: k8s-docs-images-2026-08-23]. `Always` re-resolve; download only on a miss. This is a classic distractor shape, and now you know why the distractor is wrong.

📌 **Snag:** Once a Pod is created, `imagePullPolicy` is not updated if the image's tag or digest changes later [source: k8s-docs-images-2026-08-23]. The policy was resolved when the Pod was created, and moving the tag afterwards does not retroactively change how that existing Pod behaves. If you need new behavior, you need a new Pod, which is §2's immutability principle showing up in an unexpected place.

One name to bank and not chase. `ImagePullBackOff` is reported as a container in the **Waiting** state [source: k8s-docs-images-2026-08-23], and container states are Chapter 5's material [*cross-bearing: see Ch 5 §5 — Pod phases and container states*]. Diagnosing a stuck pull, meaning reading the events and checking whether the image name is right and whether it was actually pushed [source: k8s-docs-debug-pods-2026-08-23], is Chapter 13's [*cross-bearing: see Ch 13 §2 — diagnosing ImagePullBackOff*]. What Chapter 2 owes you is the name, the cause, and the retry behavior. You have all three.

§7 — 🏠 Not All Isolation Is Equal: `RuntimeClass`

Readers skip this section. It earns your attention anyway, for a reason worth stating plainly: it is the one place in the chapter where a rule you were just taught turns out to be adjustable, and that is exactly what a well-written exam item likes to probe.

§1 established that a container shares the host operating system, and the 🏠 callout there insisted that this was a *tradeoff* rather than a deficiency. Tradeoffs can be renegotiated. This is the renegotiation.

`RuntimeClass` is a feature for selecting the container runtime configuration used to run a Pod's containers [source: k8s-docs-runtime-class-2026-08-23].

Take the motivation before the mechanism, because the mechanism without the motivation is unmemorable trivia.

You can set a different RuntimeClass between different Pods to provide a balance of performance versus security. For example, if part of your workload deserves a high level of information-security assurance, you might choose to schedule those Pods so that they run in a container runtime that uses **hardware virtualization** (such as Kata Containers) or a **user-space kernel** (such as gVisor). You'd then benefit from the extra isolation of the alternative runtime, at the expense of some additional overhead. You can also use RuntimeClass to run different Pods with the same container runtime but with different settings [source: k8s-docs-runtime-class-2026-08-23].

Read what that makes possible. Two workloads, same cluster, same API, manifests shaped the same way, and different isolation floors. The workload handling untrusted user-submitted code gets hardware virtualization. The internal batch job that nobody worries about gets the default, and doesn't pay for a boundary it doesn't need. This is the answer to a question that sounds unanswerable: "containers are less isolated than VMs, so how can I run genuinely untrusted code?" You change the floor for that workload.

🔒 **Worth Securing: "Container" names an interface, not an isolation level.** That sentence is what makes this section stick. Everything you learned in §1 through §6, the image format, the pull behavior, the CRI socket, is unchanged whether the thing on the other end of the socket shares the host kernel directly, interposes a user-space kernel, or boots a lightweight virtual machine. The contract is stable; the strength of the walls is a parameter.

The mechanism, at the depth the exam reaches. Configure the CRI implementation on your nodes; each configuration has a corresponding **handler** name. Create the corresponding RuntimeClass resources (apiVersion: node.k8s.io/v1, kind: RuntimeClass, with a handler field). Once RuntimeClasses are configured for the cluster, specify a runtimeClassName in the Pod spec to use one; **if no runtimeClassName is specified, the default runtime handler is used** [source: k8s-docs-runtime-class-2026-08-23].

Two levels of indirection, which is the part to hold onto: the Pod names a RuntimeClass, and the RuntimeClass names a handler that was configured on the nodes. The Pod author does not name a runtime. They name a *class of runtime configuration* that a cluster administrator has already established, which is why this works as a self-service mechanism rather than as a way for application teams to request arbitrary runtimes.

A RuntimeClass can also carry scheduling constraints (nodeSelector, tolerations) so that Pods land on nodes which actually support the handler, and a **Pod overhead** so the scheduler accounts for the runtime's resource cost [source: k8s-docs-runtime-class-2026-08-23]. Both of those are scheduling concepts, and scheduling has its own chapter [*cross-bearing: see Ch 7 — node selection, tolerations, and accounting for overhead*]. Register that they exist; the reasoning behind them arrives later.

🔗 **Closer Look:** Kata Containers and gVisor are two genuinely different answers to the same question. Kata uses **hardware virtualization**; gVisor uses a **user-space kernel** [source: k8s-docs-runtime-class-2026-08-23]. Kata's approach is, roughly, borrowing back the isolation model §1 traded away: a real virtualization boundary underneath the workload. gVisor's is to interpose a kernel implementation that is not the host's. Different techniques, same goal, different overheads. This is depth: an exam item is far likelier to ask *why RuntimeClass exists* than to ask which sandbox uses which technique. Know the motivation cold; know this as a bonus.

The security guidance is consistent with all of it: to protect compute at runtime, use a container runtime that provides security restrictions [source: k8s-docs-cloud-native-security-2026-08-23]. RuntimeClass is the Kubernetes-shaped way to say *which* one, per workload. Sandboxed runtimes come back as one control among several in the security lifecycle [cross-bearing: see Ch 12 — runtime protection for compute].

🕒 Taking Your Bearings #3: Pull Behavior and Runtime Selection

Four questions on §6 and §7.

1. A Pod specifies image: `internal.registry.example/api-gateway` and does not set `imagePullPolicy`. What pull policy applies?

A) IfNotPresent, because a registry hostname was given explicitly B) Always, because no tag was specified
C) IfNotPresent, because that is the global default D) Always, because `imagePullPolicy` defaults to Always whenever a registry hostname is present

2. A Pod's `imagePullPolicy` is set to Always. Which statement describes what happens each time the kubelet launches the container?

A) The image is downloaded from the registry every time, unconditionally B) The kubelet queries the registry to resolve the name to a digest, and reuses its cached image if it already holds that exact digest
C) The kubelet checks whether the local image is older than the registry's copy by timestamp D) The kubelet pulls only if the tag has changed since the last launch

3. A container reports `ImagePullBackOff`. Which of the following is the correct reading of that status?

A) The image was pulled but failed its integrity check, and Kubernetes has given up B) The container could not start because the image could not be pulled, and Kubernetes will keep retrying with increasing delay up to 300 seconds
C) The image pull succeeded but the container crashed on start-up, and the kubelet is backing off before restarting it D) The node has insufficient disk space, so the pull has been deferred indefinitely

4. A platform team must run customer-submitted code on the same cluster as internal services, and the customer workloads require a stronger isolation boundary than the internal ones. Which instrument fits,

and why?

- A) A separate cluster, because isolation strength is a cluster-level property
- B) RuntimeClass, to run the customer Pods on a runtime using hardware virtualization or a user-space kernel, at the cost of extra overhead
- C) RuntimeClass, applied cluster-wide so that all workloads get the strongest available runtime
- D) A separate namespace with a restrictive NetworkPolicy, which segments the customer workloads

Answers with Explanations

1 — B. If no tag is specified, the default `imagePullPolicy` is `Always` [source: k8s-docs-images-2026-08-23]. The reference names a registry and an image but no tag, so the no-tag case applies.

- **A is wrong.** The registry hostname has no effect on pull policy; the *tag form* determines the default.
- **C is wrong.** There is no single global default. There are four conditional defaults, and `IfNotPresent` is only two of them: `digest`, and any tag other than `:latest` [source: k8s-docs-images-2026-08-23].
- **D is wrong,** and it reaches the right answer by an invented rule, which is the more dangerous kind of wrong. The policy here is `Always`, but because no tag was given, not because a hostname was. Apply this half-rule to `internal.registry.example/api-gateway:v3` and it produces `Always`, where the correct answer is `IfNotPresent`.

2 — B. With `Always`, the kubelet queries the registry to resolve the name to a digest, and if it already has an image with that exact digest cached locally, it uses the cached image; otherwise it pulls [source: k8s-docs-images-2026-08-23].

- **A is wrong,** and it is the distractor the policy's name invites. `Always` means always *check*, not always *download*.
- **C is wrong.** The resolution is by digest, not by timestamp comparison.
- **D is wrong.** The kubelet does not track tag changes between launches; it re-resolves the reference each time.

3 — B. `ImagePullBackOff` means a container could not start because Kubernetes could not pull the image, whether from an invalid image name or a private registry without credentials, and the `BackOff` indicates continued retries with increasing delay up to a compiled-in limit of 300 seconds [source: k8s-docs-images-2026-08-23].

- **A is wrong** twice: the pull did not succeed, and Kubernetes has not given up. Retrying is what `BackOff` describes.
- **C is wrong.** That describes a different failure mode entirely; here the image never arrived.
- **D is wrong.** Disk pressure is a distinct node condition and is not what this status reports.

4 — B. `RuntimeClass` exists precisely to provide a balance of performance versus security: a workload deserving high information-security assurance can be scheduled onto a runtime using hardware

virtualization (Kata Containers) or a user-space kernel (gVisor), benefiting from extra isolation at the expense of additional overhead [source: k8s-docs-runtime-class-2026-08-23].

- **A is wrong.** Isolation strength is selectable per Pod via `runtimeClassName`, which is the entire point of the feature. A separate cluster may be defensible for other reasons, but not because isolation is cluster-fixed.
- **C is wrong**, and it misses that the tradeoff runs both ways. The stronger runtime costs overhead; applying it to everything makes every workload pay for a boundary most of them don't need. Selectivity is the feature.
- **D is wrong**, and it is a real control aimed at the wrong axis. Namespaces and NetworkPolicy segment *network* reachability; the requirement here is a stronger boundary between the workload and the host it runs on. Network segmentation does nothing about that. [cross-bearing: see Ch 10 — NetworkPolicy]

Checkpoint: You've Now Mastered ✓ The three pull policies and, more importantly, the four conditional defaults ✓ Why Always does not mean "always download" ✓ ImagePullBackOff — what it reports and what BackOff implies ✓ RuntimeClass, and the fact that isolation strength is a per-workload decision

Everything in this chapter is now on the table. One short section to see why it all has the shape it has.

§8 — .11 The Crate, Not the Cargo

The subtitle comes due.

The intermodal shipping container did not win because it was a better box. It won because the industry standardized the **interface**, the dimensions and the corner fittings, rather than the contents. Once that interface was published, a crane could be designed once, a truck chassis could be designed once, a rail flatcar and a ship's hold could each be designed once, and every one of them could then handle anything. The cargo stopped mattering. Nobody at the crane has an opinion about what is inside.

☀ Zenith

You have spent this chapter looking at that same move from five angles without necessarily seeing that it was one move.

The OCI standardized the image format, so any build tool can produce an artifact any runtime can unpack. It standardized the distribution API, so any registry can serve any image. It standardized bundle execution, so any conformant runtime can run one [source: oci-overview-2026-08-23]. And Kubernetes, for its part, standardized nothing about containers at all. It published an *interface*, the CRI, and let the ecosystem supply the implementations [source: k8s-docs-extending-kubernetes-2026-08-23].

Which is why Kubernetes never needed to know what is in the crate. It doesn't know what language your application is written in, what its dependencies are, which build tool produced it, or which of two mature runtimes will start it on any given node. It knows the shape of the fitting. That is the entire reason a system this large can be this indifferent to the workloads it runs, and the reason the answer to "can Kubernetes run *my* thing?" is that if an application can run in a container, it should run great on Kubernetes [source: k8s-docs-overview-2026-08-23].

Figure 2-6 (Zenith illustration — to be commissioned). Identical crates moving between incompatible carriers, each carrier built once against a published specification, the contents never inspected and never mattering. Rendered from the Communications Officer's vantage; narrator not depicted. The synthesis must read as being *about interfaces* — a reader who sees only an attractive freight scene has received a decorative illustration. Minimal labels by design; two or three at most. Register check against `illustrator-brief.md` required before commissioning: the crate motif must work in this book's era placement rather than defaulting to a 20th-century dockside.

Extended Analogy:

Consider what the world looks like before the standard, because that is the part usually skipped. A ship arrives. In the hold are barrels, sacks, crates of a dozen different sizes, machinery in wooden frames, loose timber. Every item is handled individually, by hand, and every item is handled *differently*, because there is no general procedure for a thing whose shape you cannot predict. Unloading is slow in a way that has nothing to do with how fast anyone is working. Most of the ship's economic life is spent stationary, being emptied.

Now consider what changed. Not the ship; ships were fine. Not the cargo; the cargo was always the point. What changed is that somebody wrote down the dimensions of a box and the geometry of the fittings at its corners, and enough of the industry agreed to it that building machinery against the specification became rational. A crane that can lift one standard container can lift every standard container that will ever exist, including ones holding goods that had not been invented when the crane was built. The crane's designer never had to think about the cargo. That is not a convenience; it is the whole gain, and it is *larger* than any improvement to the crane.

Now read this chapter again in that light. An image is the crate: a published format with a manifest describing its contents, produced by any of a dozen build tools. A registry is the terminal: it moves crates according to a published API, without opening them. A CRI runtime is the crane: built once against an interface, able to handle any conformant crate, swappable for a different crane without redesigning the port. And Kubernetes is the port authority, coordinating, scheduling, deciding what goes where, and never once lifting anything itself.

The failure mode of the pre-standard world is also worth keeping. It was not that ships were slow. It was that every combination of cargo and carrier was a bespoke problem, so effort spent solving one combination taught you nothing about the next. That is exactly the failure mode that a platform without published interfaces has, and it is exactly why the next four chapters keep finding the same design decision underneath different subjects.

And that is the plant. This is the **first** of four times you will see this move. Storage does it. Networking does it. The API itself does it. When those three have all landed, the book collects them, and the collecting is meant to feel like recognition rather than a fourth list [*cross-bearing: see Ch 17 §4 – the four pluggable interfaces, collected*].

Exam Alert 🚨

High-Priority Topics

1. **Tag versus digest.** A tag identifies a series and can be moved. A digest is a hash of the image's content and is immutable [source: k8s-docs-images-2026-08-23]. Every claim anyone makes about reproducible deployment rests on this distinction.
2. **The kubelet → CRI → containerd/CRI-O → runC chain.** Kubernetes defines the interface and implements nothing below it [source: k8s-docs-containers-2026-08-23] [source: k8s-docs-extending-kubernetes-2026-08-23]. This is the most reused idea in the chapter.
3. **OCI's three specifications, and that the OCI is a governance body rather than software.** image-spec (format), distribution-spec (the registry API, v1.0 May 2020), runtime-spec (running a filesystem bundle) [source: oci-overview-2026-08-23].
4. **imagePullPolicy defaults.** IfNotPresent with a digest; Always with :latest; Always with no tag; IfNotPresent with any other tag [source: k8s-docs-images-2026-08-23]. Four cases, and an item can give you a reference and ask for the behavior.
5. **The interface-and-implementations pattern itself.** CRI is the first of four published extension points: CRI, CNI, CSI, and API extensions [source: k8s-docs-extending-kubernetes-2026-08-23]. Recognizing the *move*, not just this instance of it, is what Chapter 17 will ask of you.

Common Traps

Trap	Where this chapter defuses it
"A container image includes the OS kernel"	§2 — derived from §1's sharing model; Bearings #1 Q1
"You patch a running container"	§2 immutability — the correct process is build a new image, then recreate [source: k8s-docs-containers-2026-08-23]; Bearings #1 Q2
"OCI is a runtime"	§5 ☆ Fixed Point — it is a governance structure publishing three specifications
Conflating OCI with CRI	§5 △ Navigational Hazards. These are easy to confuse: two three-letter abbreviations, both mentioning "runtime," both below your Pod. Ask which direction you're looking
"Docker is the container runtime Kubernetes uses"	§4 ☞ Snag. Also easy to confuse, for historical reasons — "Docker" names four things, and the supported set is defined by CRI conformance [source: k8s-docs-containers-2026-08-23]
"latest is just a naming-hygiene issue"	§3 △ plus §6 — the tag also sets the default pull policy [source: k8s-docs-images-2026-08-23]
"Always re-downloads the image every launch"	§6 Dead Reckoning — it re-resolves to a digest and reuses a matching cached image [source: k8s-docs-images-2026-08-23]
"Container isolation strength is fixed"	§7 — RuntimeClass selects it per Pod [source: k8s-docs-runtime-class-2026-08-23]

Two of these traps are flagged for a specific reason. The OCI/CRI conflation and the Docker-as-runtime shorthand appear here because they are conceptually slippery, not because this book has data on how often they show up. The book does not make frequency claims it cannot support.

Practice Questions

Twenty-seven questions. Seven require combining two sections; those are marked. Explanations cover every option.

Attempt all twenty-seven before checking anything. The answer key follows the full set, so nothing here spoils itself.

1. Which single architectural difference accounts for containers starting faster than virtual machines?

A) Containers use a more efficient filesystem format B) Containers share the host operating system rather than booting their own C) Containers are always smaller than virtual machine images D) Containers skip hardware initialization by using paravirtualized drivers

2. Which of these does a container have its own of?

A) Kernel B) Hypervisor C) Filesystem D) Hardware

3. A workload is described as needing to be "portable across clouds and OS distributions." Which property of containers delivers that?

A) They are decoupled from the underlying infrastructure B) They are scheduled by Kubernetes C) They use a shared kernel D) They can be restarted automatically

4. Which is included in a container image?

A) Default values for essential settings B) The cluster's kubeconfig C) The node's kernel modules D) The Pod specification that will run it

5. An application inside a running container needs a configuration change that must survive the container being replaced. What is the correct process?

A) Edit the file in the running container and take a snapshot B) Build a new image that includes the change, then recreate the container from the updated image C) Edit the file in the running container; changes persist by default D) Mount the file from the host so the container never has to change

6. Two images are built from the same base. What follows about storage and transfer?

A) The base is duplicated once per image B) The base is a shared layer, named by both images' manifests C) The images must be merged into one before they can share anything D) Sharing only occurs if both images live in the same registry namespace

7. Cloud Native Buildpacks: what is the value proposition?

A) They compile applications faster than a hand-authored build file by parallelizing layer creation B) They transform application source code into a runnable OCI image without a hand-authored build file, by analyzing the code to determine how to build it C) They replace the container runtime with a source-level execution engine D) They guarantee that the produced image contains no known vulnerabilities

8. In the Buildpacks model, what is a **builder**?

A) A CI system that runs the lifecycle B) An OCI image containing an ordered combination of buildpacks and a build-time base image, a lifecycle binary, and a reference to a runtime base image C) The final application image produced by the export phase D) The pairing of a build image and a run image

9. [integrative: \$2 + \$5] The layer structure of an image — manifest, serialized filesystem layers, configuration — is standardized by which specification?

A) The OCI runtime specification B) The OCI image specification C) The Container Runtime Interface D) The OCI distribution specification

10. What does the reference `busybox` resolve to?

A) `busybox:latest` on whichever registry the node is configured to prefer B) `docker.io/library/busybox:latest` C) `docker.io/busybox:latest` D) `registry.k8s.io/library/busybox:latest`

11. Which statement is correct?

A) A tag is immutable; a digest can be re-pointed B) A digest is a content hash and cannot be re-pointed; a tag can be moved C) Both are immutable once pushed to a registry D) Both can be moved, but only by the registry operator

12. Why does the documentation advise against `:latest` in production?

A) It is harder to know which version is running, and harder to roll back properly B) `:latest` images are excluded from registry caching C) `:latest` always points to the most recently pushed image, so it is guaranteed current D) `:latest` is re-resolved on every request, so two nodes may run different builds of the same Deployment

13. Which is a documented way to supply credentials for a private registry?

A) Embedding credentials in the image B) Specifying `imagePullSecrets` on a Pod, referencing a Secret of type `kubernetes.io/dockerconfigjson` C) Configuring the kube-apiserver to authenticate to the registry on the nodes' behalf D) Pre-pulling the image on one node, which makes it available cluster-wide

14. [integrative: \$3 + \$5] A registry serves images to nodes over a standardized API. Which specification standardizes that API, and when did it reach v1.0?

A) `image-spec`, June 2015 B) `distribution-spec`, May 2020 C) `runtime-spec`, June 2015 D) The CRI, May 2020

15. Which component is responsible for managing the execution and lifecycle of containers on a node?

A) The kubelet B) kube-proxy C) The container runtime D) The kube-controller-manager

16. What is the CRI?

A) A container runtime maintained by the Kubernetes project B) The interface Kubernetes defines so that alternative container runtimes can be used C) An OCI specification for container images D) The protocol nodes use to fetch images from registries

17. Which two runtimes does the Kubernetes documentation name as supported, and what qualifies any other?

A) Docker and containerd; any runtime with an OCI image B) containerd and CRI-O; any other implementation of the CRI C) runC and containerd; any runtime that supports the runtime specification D) CRI-O and Kata; any runtime installed on the node

18. What was runC's origin?

A) It was written by the Kubernetes project as a reference CRI implementation B) Docker donated it to the OCI to serve as the cornerstone of the effort C) It was developed by the Linux Foundation alongside the runtime specification D) It is the CNCF's graduated low-level runtime, donated by Google

19. Which best describes the Open Container Initiative?

A) An open governance structure that publishes specifications; it ships no runtime of its own B) A CNCF working group that maintains containerd and CRI-O C) A Linux Foundation certification program for container images D) An open-source container runtime project founded in June 2015

20. [integrative: \$4 + \$5] A colleague says, "this runtime conforms to the CRI, so Kubernetes can use it, and it also handles OCI images." Is this coherent?

A) No — a component cannot participate in both the CRI and the OCI B) Yes — the CRI governs how Kubernetes reaches the runtime; the OCI governs the image format and bundle execution the runtime works with C) No — the CRI supersedes the OCI specifications for Kubernetes clusters D) Yes, but only because the runtime is a CNCF graduated project

21. A Pod specifies `image: myapp:v3.2` and omits `imagePullPolicy`. Which policy applies?

A) Always B) IfNotPresent C) Never D) It is an error to omit the policy when using a version tag

22. [integrative: \$3 + \$6] A team pins deployments with `:latest` "so we always know we're current." Which pair of consequences follows?

A) The image is easy to identify, and pulls are minimized B) Version identity is unclear and rollback is harder, and the default pull policy becomes Always C) The pull policy becomes Never, and the image must be pre-pulled D) The digest is recomputed on each pull, guaranteeing freshness

23. A Pod exists, running from `myapp:v3`. Someone moves the `:v3` tag to a new image. What happens to the existing Pod's pull policy?

A) It is recalculated from the new tag B) It is unchanged — `imagePullPolicy` is not updated when the image's tag or digest changes later C) It reverts to the cluster default D) The Pod is evicted and recreated with the new policy

24. [integrative: \$1 + \$7] Which statement best characterizes container isolation in Kubernetes?

A) Isolation is fixed by the container model; stronger isolation requires abandoning containers B) Isolation is relaxed by default to keep containers lightweight, and can be strengthened per Pod by selecting a runtime configuration via `RuntimeClass` C) Isolation is configured per node, so all Pods on a node share one isolation level permanently D) Isolation strength is determined by the image's base layer

25. A Pod omits `runtimeClassName`. What runs it?

A) The Pod is rejected until a `RuntimeClass` is specified B) The default runtime handler C) The most secure available handler, as a safe default D) The handler configured on whichever node the scheduler picked

26. [integrative: \$2 + \$6] A team fixes a bug by rebuilding their image and pushing it under the same tag, `api:v4`, then waits for the running Pods to pick it up. Nothing changes. Which pair of facts explains it?

A) The image was rebuilt but not re-tagged, so the registry rejected the push B) `imagePullPolicy` was resolved when the Pod was created and is not updated when the tag later moves; and with a tag other than `:latest` the default is `IfNotPresent`, so the kubelet never goes looking C) Kubernetes caches images by tag, so a moved tag is only detected after the cache expires D) The rebuild produced a new digest, and the running Pods are pinned to the digest they started with

27. [integrative: \$4 + \$5] Kubernetes supports containerd, CRI-O, "and any other implementation of the CRI." Which design principle does that phrasing express, and where else does the documentation apply it?

A) A compatibility guarantee — Kubernetes commits to supporting any runtime that has ever worked, and applies the same commitment to deprecated APIs B) An extension point — Kubernetes publishes an interface and lets the ecosystem supply implementations, and it does the same for pod networking and for storage C) A conformance program — runtimes are certified by the CNCF, as are storage drivers and network plugins D) A vendor-neutrality policy — Kubernetes refuses to name a default runtime, and likewise ships no default networking or storage

Practice Question Answers

1 — B. Containers have relaxed isolation properties in order to share the OS among the applications, and are therefore lightweight [source: k8s-docs-overview-2026-08-23]. A VM is a full machine running all components including its own OS on top of virtualized hardware [source: k8s-docs-overview-2026-08-23], and booting an OS is what takes the time.

- **A is wrong:** filesystem format is not the mechanism, and containers have their own filesystem in both models.
- **C** is a consequence of B, not a cause, and stating it as the cause is the error the chapter's derivation exercise is designed to prevent.
- **D is wrong:** paravirtualized drivers are a virtualization technique and have nothing to do with why containers start quickly.

2 — C. Similar to a VM, a container has its own filesystem, share of CPU, memory, process space, and more [source: k8s-docs-overview-2026-08-23].

- **A is wrong:** the kernel is the shared thing, and that sharing is the definition.
- **B is wrong:** a hypervisor belongs to the virtualization model, and no per-container hypervisor exists in the container model.
- **D is wrong:** hardware is the host's, in both models.

3 — A. Because containers are decoupled from the underlying infrastructure, they are portable across clouds and OS distributions [source: k8s-docs-overview-2026-08-23]; the standardization from having dependencies included is what produces the same behavior wherever you run it [source: k8s-docs-containers-2026-08-23].

- **B is wrong:** portability is a property of containers themselves and holds without an orchestrator.
- **C is a true statement** about containers but the wrong causal link. Kernel sharing produces lightness, not portability; in fact it is the *constraint* on portability across kernel families.
- **D describes something an orchestrator provides** [source: k8s-docs-overview-2026-08-23], not a source of portability.

4 — A. An image is a ready-to-run package containing the code and any runtime it requires, application and system libraries, and default values for any essential settings [source: k8s-docs-containers-2026-08-23].

- **B is wrong:** kubeconfig is a client credential file used to reach a cluster's API [source: k8s-docs-kubectl-overview-2026-08-23], unrelated to image contents.
- **C is wrong:** the kernel comes from the host, so kernel modules are not image contents either.
- **D is wrong** and inverts the relationship: a Pod spec *references* an image.

5 — B. Containers are intended to be stateless and immutable; if you want to make changes, the correct process is to build a new image that includes the change, then recreate the container to start from the updated image [source: k8s-docs-containers-2026-08-23].

- **A is wrong:** "edit then snapshot" produces an artifact whose provenance nobody can reproduce, forfeiting the repeatability that is the reason to use containers at all [source: k8s-docs-containers-2026-08-23].
- **C is wrong:** the documentation says explicitly you should not change the code of a running container.
- **D is a real technique** for some problems but not the answer to "the application needs a different configuration baked in," and it trades away portability by binding the container to a particular host's contents.

6 — B. An image is assembled from filesystem layers described by a manifest [source: oci-overview-2026-08-23]; two images built on the same base name the same base layer, and a layer named by both is one object rather than two.

- **A is wrong.** Both manifests reference the same base layer by identity, so the registry and the node each hold one copy, not one per image. Duplication would defeat the reason images are layered at all. This is the intuition Soundings Q4 pre-tests.
- **C is wrong:** layer sharing requires no merging; it is a property of how manifests reference layers.
- **D is wrong:** sharing is a function of layer identity, not naming conventions.

7 — B. A buildpack is software that transforms application source code into runnable artifacts by analyzing the code and determining the best way to build it; the lifecycle's export phase produces a runnable OCI image, and Cloud Native Buildpacks is a CNCF graduated project [source: buildpacks-concepts-2026-08-23].

- **A is wrong.** Build speed is not the claim the project makes, and nothing in the model promises parallel layer creation.

- **C is wrong.** Buildpacks produce an ordinary OCI image; the runtime story is entirely unchanged, which is the point of producing an OCI image.
- **D is wrong.** Vulnerability scanning is a supply-chain concern handled by other tools entirely [source: k8s-docs-cloud-native-security-2026-08-23]; no build system can guarantee a clean image.

8 — B. That is the definition verbatim in substance [source: buildpacks-concepts-2026-08-23].

- **A** describes the *platform*, which orchestrates by invoking the lifecycle [source: buildpacks-concepts-2026-08-23].
- **C** describes the export phase's output, not the builder.
- **D** describes the *stack*, the build-image/run-image pairing [source: buildpacks-concepts-2026-08-23].

9 — B. The Image Specification defines the OCI Image Format, encompassing the image manifest, filesystem layer serialization, and image configuration needed to launch applications on target platforms [source: oci-overview-2026-08-23].

- **A** governs running an unpacked filesystem bundle, not the format of the artifact you unpack.
- **C** is a Kubernetes-to-runtime interface [source: k8s-docs-extending-kubernetes-2026-08-23] and standardizes nothing about image internals. This is the OCI/CRI conflation.
- **D** standardizes the API for distributing images, not their internal layout.

10 — B. The documentation gives this exact equivalence: `busybox` is equivalent to `docker.io/library/busybox:latest` [source: k8s-docs-images-2026-08-23].

- **A is wrong:** the assumed registry is specific, not a node preference.
- **C** drops the `library` namespace present in the documented equivalence.
- **D** names a different registry, which is not the assumed default.

11 — B. Digests are a unique identifier for a specific version of an image, a hash of the image's content, and are immutable; tags can be moved to point to different images [source: k8s-docs-images-2026-08-23].

- **A** inverts it, which is the misconception this chapter's Fixed Point targets.
- **C** would make version pinning by tag safe, which is precisely what the documentation cautions against.
- **D is wrong**, and it is worth naming why: it treats mutability as a permissions question. A digest cannot be re-pointed by anyone, including the registry operator, because it is derived from the content. Change the content and you have a different digest by definition.

12 — A. You should avoid using the `:latest` tag when deploying containers in production as it is harder to track which version of the image is running and more difficult to roll back properly; specify a meaningful tag such as `v1.42.0` and/or a digest instead [source: k8s-docs-images-2026-08-23].

- **B** is invented; caching behavior is governed by pull policy, not by the tag's name.

- **C is wrong**, and it is a widely held belief. `:latest` is an ordinary tag with a conventional name. It points wherever it was last moved, which is usually but not necessarily the most recent push, and "most recently pushed" is not the same as "the version you want."
- **D is wrong**, and it conflates tag mutability with pull policy. Re-resolution is a *pull-policy* behavior, and it is Always that re-resolves [source: k8s-docs-images-2026-08-23], which `:latest` happens to default to (see Q22), but the two facts are separate, and the "guaranteeing freshness" framing gets the mechanism backwards.

13 — B. Credentials can be provided by configuring nodes to authenticate, a kubelet credential provider, pre-pulled images, specifying `imagePullSecrets` on a Pod (a Secret of type `kubernetes.io/dockerconfigjson`), or vendor-specific and local extensions [source: k8s-docs-images-2026-08-23].

- **A** is circular: you would need the credentials to pull the image containing the credentials.
- **C is wrong**, and it is the control-plane/node confusion worth catching now. Configuring nodes to authenticate is a documented path, but it is the *nodes*, because the kubelet is what pulls. The API server is not on the image-pull path at all [source: k8s-docs-components-2026-08-23].
- **D is wrong** in its second clause. Pre-pulled images are a genuine documented path [source: k8s-docs-images-2026-08-23], but pre-pulling is per-node: an image cached on one node does nothing for a Pod scheduled onto another.

14 — B. The Distribution Specification reached v1.0 in May 2020 and was introduced to OCI as an effort to standardize the API to distribute container images [source: oci-overview-2026-08-23].

- **A** names the wrong specification, and pairs it with June 2015, which is the OCI's founding date rather than the distribution specification's v1.0.
- **C** names the specification governing bundle execution.
- **D** conflates a Kubernetes interface with an OCI specification and attaches a date that belongs to the latter.

15 — C. The container runtime is responsible for managing the execution and lifecycle of containers within the Kubernetes environment [source: k8s-docs-containers-2026-08-23] [source: k8s-docs-cluster-architecture-2026-08-23].

- **A** is the closest wrong answer: the kubelet ensures that containers described in PodSpecs are running and healthy [source: k8s-docs-cluster-architecture-2026-08-23]. It drives; it does not execute.
- **B** maintains network rules on nodes to implement Services [source: k8s-docs-components-2026-08-23].
- **D** runs controller processes in the control plane [source: k8s-docs-components-2026-08-23], not on the node's container path.

16 — B. Among Kubernetes' infrastructure extension points is the container runtime, "CRI, the Container Runtime Interface, to support alternative container runtimes" [source: k8s-docs-extending-kubernetes-2026-08-23].

2026-08-23]; Kubernetes supports containerd, CRI-O, and any other implementation of the CRI [source: k8s-docs-containers-2026-08-23].

- **A is wrong** and is a consequential misreading: Kubernetes defines the interface and ships no runtime.
- **C and D** are both OCI concerns, `image-spec` and `distribution-spec` respectively [source: oci-overview-2026-08-23], which is the conflation §5's hazard callout addresses.

17 — B. Kubernetes supports container runtimes such as containerd, CRI-O, and any other implementation of the Kubernetes CRI [source: k8s-docs-containers-2026-08-23]; a container runtime, containerd or CRI-O, must be installed on every node [source: k8s-docs-setup-tooling-2026-08-23].

- **A** names Docker, which is the historical shorthand rather than the documented set, and gets the qualifying condition wrong.
- **C** names runC, which is an OCI runtime donated to the OCI [source: oci-overview-2026-08-23] rather than one of the two CRI implementations the docs name.
- **D** names Kata, which is an example of a runtime reachable *through* RuntimeClass [source: k8s-docs-runtime-class-2026-08-23], and its qualifying condition ("installed on the node") omits conformance.

18 — B. Docker donated its container runtime, runC, to the OCI to serve as the cornerstone of this new effort [source: oci-overview-2026-08-23].

- **A is wrong:** runC sits below the CRI, and Kubernetes did not write it.
- **C is wrong** on origin. The OCI operates under the Linux Foundation [source: oci-overview-2026-08-23] but did not author runC; it received it.
- **D is wrong** on both donor and framing; the CNCF's graduated container runtimes are containerd and CRI-O [source: cncf-project-maturity-levels-2026-08-23].

19 — A. The OCI is an open governance structure for the express purpose of creating open industry standards around container formats and runtimes, established in June 2015 by Docker and other leaders in the container industry, and it publishes three specifications [source: oci-overview-2026-08-23]. runC was donated to it [source: oci-overview-2026-08-23], which is a different relationship than shipping one.

- **B is wrong** on both body and projects: the OCI is not part of the CNCF, and containerd and CRI-O are CNCF graduated projects [source: cncf-project-maturity-levels-2026-08-23], not OCI deliverables.
- **C is wrong.** It publishes specifications; it does not certify images.
- **D is wrong,** and note that it carries the *correct founding date*. Getting the date right will not save you if you have the kind of body wrong, which is exactly why the Fixed Point in §5 leads with "governance body," not with "2015."

20 — B. The CRI exists to let Kubernetes support alternative runtimes [source: k8s-docs-extending-kubernetes-2026-08-23]; the OCI specifications define the image format, its distribution, and how a

filesystem bundle is executed [source: oci-overview-2026-08-23]. Different layers, and a runtime naturally sits at the intersection.

- **A is wrong** and states the conflation as a rule.
- **C is wrong:** a Kubernetes interface does not supersede industry specifications about artifact formats, since they are not competing.
- **D** reaches a correct conclusion by an irrelevant route; maturity level [source: cncf-project-maturity-levels-2026-08-23] has no effect on which specifications a component implements.

21 — B. If you omit `imagePullPolicy` and the tag is something other than `:latest`, the default is `IfNotPresent` [source: k8s-docs-images-2026-08-23].

- **A is wrong** and confuses this case with the `:latest` and no-tag cases, which *do* default to `Always` [source: k8s-docs-images-2026-08-23].
- **C is wrong.** `Never` is only ever a policy you set explicitly; it is never a default.
- **D is invented:** omitting the field is always legal, and a default applies.

22 — B. The documentation warns that `:latest` makes it harder to know which version is running and harder to roll back properly [source: k8s-docs-images-2026-08-23]; separately, the default policy when the tag is `:latest` is `Always` [source: k8s-docs-images-2026-08-23]. Two effects, one field.

- **A** inverts both halves.
- **C** describes a policy that is never a default.
- **D** garbles the mechanism: with `Always` the kubelet resolves the *name* to a digest at the registry and reuses a matching cached image [source: k8s-docs-images-2026-08-23]; it does not recompute a digest to establish freshness.

23 — B. Once a Pod is created, `imagePullPolicy` is not updated if the image's tag or digest changes later [source: k8s-docs-images-2026-08-23].

- **A is wrong:** the resolution happened at creation.
- **C is wrong** twice. Nothing reverts, and there is no single cluster default policy, only four conditional ones.
- **D is invented:** moving a tag in a registry does not trigger eviction.

24 — B. Containers have relaxed isolation properties in order to share the OS and are therefore lightweight [source: k8s-docs-overview-2026-08-23]; `RuntimeClass` lets you balance performance against security per Pod, scheduling security-sensitive workloads onto a runtime using hardware virtualization or a user-space kernel at the expense of some overhead [source: k8s-docs-runtime-class-2026-08-23].

- **A** is the belief $\$7$ exists to dislodge.
- **C** is half-right in a misleading way: handlers *are* configured on nodes, but a Pod selects among them with `runtimeClassName`, and a `RuntimeClass` can carry scheduling constraints so Pods land on supporting nodes [source: k8s-docs-runtime-class-2026-08-23].
- **D is wrong:** the base image supplies userspace contents, not isolation boundaries.

25 — B. If no `runtimeClassName` is specified, the default `RuntimeHandler` is used [source: k8s-docs-runtime-class-2026-08-23]; ordinarily you can allow your cluster to pick the default container runtime for a Pod [source: k8s-docs-containers-2026-08-23].

- **A is wrong:** specifying a `RuntimeClass` is optional, which is why most practitioners never think about runtimes.
- **C is wrong** and inverts the tradeoff. The stronger runtime costs overhead, so defaulting to it would make every workload pay for a boundary most don't need.
- **D is wrong**, and it is the plausible one, because handlers genuinely *are* configured per node. But the default is a cluster-level concept, not a per-node lottery, and a `RuntimeClass` carries `nodeSelector` and `tolerations` [source: k8s-docs-runtime-class-2026-08-23] precisely so that handler availability and node placement stay aligned.

26 — B. Two facts compose. `imagePullPolicy` is resolved at Pod creation and is not updated when the image's tag or digest changes later [source: k8s-docs-images-2026-08-23]; and with a tag other than `:latest`, the default policy is `IfNotPresent` [source: k8s-docs-images-2026-08-23], so a kubelet that already holds an image for that reference never consults the registry.

- **A is wrong:** re-pushing a tag is ordinary and permitted, which is precisely why tags are movable [source: k8s-docs-images-2026-08-23].
- **C is wrong.** Caching is keyed on digest, not on tag, and there is no tag-cache expiry mechanism. Always re-resolves the name to a digest at the registry every launch [source: k8s-docs-images-2026-08-23]; the other policies do not.
- **D is wrong**, and it is the strongest distractor because it is *nearly* right: the rebuild does produce a new digest. But the Pod is pinned to the *reference*, not to a resolved digest, which is exactly the tag-versus-digest discrimination §3 installs. Had the Pod referenced the image by digest, it would have been pinned to content, and the situation would be intentional rather than surprising.

27 — B. The published extension points include CRI "to support alternative container runtimes," CNI (the Container Network Interface) "to implement pod networking," and CSI (the Container Storage Interface) "to add new storage types" [source: k8s-docs-extending-kubernetes-2026-08-23]. One design instinct, applied in several places.

- **A is wrong.** Nothing in that phrasing is a support commitment; the qualifying condition is conformance to a published interface, not historical compatibility.
- **C is wrong**, and it is the strongest distractor. CNCF conformance and maturity programs genuinely exist, and containerd and CRI-O genuinely *are* CNCF graduated projects [source: cncf-project-maturity-levels-2026-08-23], but graduation is a project-maturity status, not a certification that makes a runtime usable by Kubernetes. What qualifies a runtime is implementing the CRI [source: k8s-docs-containers-2026-08-23].
- **D is wrong.** It over-reads pluggability into "Kubernetes ships nothing," which §7's default-handler behavior contradicts [source: k8s-docs-runtime-class-2026-08-23]: clusters do have a default runtime, they just aren't locked to it.

Chapter Summary

Concept	Remember This
Container	Repeatable because dependencies travel with it; shares the host OS, which is why it's lightweight
Container vs VM	A VM boots its own guest OS on virtualized hardware; a container does not. Every other difference follows
Container image	Code, its runtime, application and system libraries, default settings. No kernel
Immutability	Don't change a running container. Build a new image, recreate the container
Layers	An image is serialized filesystem layers plus a manifest and a configuration. They stack in order; a shared base is a shared layer; changing a lower layer invalidates what sits above it
Buildpacks	Source → runnable OCI image without hand-authored build files. detect → build → export. CNCF graduated
Image reference	<code>[registry[:port]/]name[:tag]</code>
Tag vs digest	A tag is a movable label. A digest is a content hash and is immutable
:latest	Two problems: unclear version identity <i>and</i> a default pull policy of Always
Container runtime	Per-node component that manages container execution and lifecycle. containerd, CRI-O, or any CRI implementation
kubelet	Node agent that ensures PodSpec containers are running and healthy. Drives; does not execute
CRI	The interface Kubernetes defines so alternative runtimes can be plugged in. Kubernetes implements nothing below it
runC	Donated by Docker to the OCI as the cornerstone of the effort. Runs a filesystem bundle
OCI	Governance structure, June 2015. Three specifications , not software
image-spec	The image format: manifest, layer serialization, configuration
distribution-spec	The API for distributing images. v1.0 May 2020
runtime-spec	How to run a filesystem bundle unpacked on disk
Filesystem bundle	The unpacked, on-disk form of an image: contents plus the configuration a runtime needs to start it
OCI vs CRI	OCI = format, distribution, bundle execution (industry). CRI = Kubernetes-to-runtime (Kubernetes)
imagePullPolicy	Always (re-resolve, reuse matching cache), IfNotPresent, Never

Concept	Remember This
Pull defaults	digest → IfNotPresent · :latest → Always · no tag → Always · other tag → IfNotPresent
ImagePullBackOff	Container can't start because the image couldn't be pulled. Retries with increasing delay, capped at 300s
RuntimeClasses	Selects the runtime <i>configuration</i> per Pod. Kata (hardware virtualization) or gVisor (user-space kernel) buy isolation at the cost of overhead
The pattern	Kubernetes defines interfaces and lets the ecosystem implement them. CRI is the first of four

The Voyage Ahead

You can now describe a container without hand-waving, predict what an image reference resolves to, name every component between a manifest and a running process, and say which specification governs each hop. That is the bottom of the stack, and it is genuinely the hardest part to reconstruct later if it's shaky, because everything above it is described in terms of it.

Which raises the obvious next question. §4 put a runtime on every node and a kubelet beside it, then said almost nothing about what else is on that node, or what is *above* the nodes deciding which containers should exist in the first place. There is a whole apparatus that has been quietly implied all chapter: something that holds the desired state, something that decides which node a workload lands on, something that notices when reality has drifted and corrects it. Chapter 3 opens the cluster that the runtime sits inside, names every part of it, and tells the story of how it came to have that shape.

You will also find the pattern from §4 waiting there. The kubelet you just met is one node component among several, and the reason it can talk to *any* conformant runtime turns out to be the same reason the cluster can use any conformant network plugin and any conformant storage driver. You have seen the move once. Watch for the second.

⚓ **Safe Harbor reached — Containerization complete.**

You've finished the deepest chapter in the book's dependency graph. Everything from here builds upward.

📊 Chart → ⌘ **Passage** → 🌅 Dawn

"Standardize the fitting, and you stop needing to know what's inside. That is not indifference. That is leverage."

Chapter 3: The Ship's Company

"Everyone aboard has one job, and none of them is 'be in charge'"

Domain Weight: ~6% of exam (authored estimate) | Complexity: Mixed | Novelty: Paradigm-shifting

Attention Budget

Total time: ~85 minutes | Recommended: Single session if you're fresh; otherwise split after the second checkpoint

Section	Time	Attention Cost	Best Time to Study
\$1 How the Cluster Got the Shape It Has	11 min	Low	Anytime
\$2 The Control Plane	15 min	Medium	Mid-session
\$3 Node Components in Context	10 min	Medium	Mid-session
🕒 Taking Your Bearings #1	6 min	Medium	After a brief break
\$4 Addons, and What Else Is Optional	6 min	Low	Anytime
\$5 The Only Door In	11 min	High	Peak attention
🕒 Taking Your Bearings #2	5 min	Medium	After a brief break
\$6 Controllers and the Control Loop	14 min	High	Peak attention
🕒 Taking Your Bearings #3	5 min	Medium	After a brief break
\$7 Nobody Is in Charge	7 min	Medium	Anytime after \$5 and \$6

Attention Cost Key:

- **Low:** Concrete, familiar concepts — study anytime
- **Medium:** New concepts requiring focus — study when alert
- **High:** Abstract or complex material — study at peak attention

If you only have 15 minutes: read \$6 and take Taking Your Bearings #3. Later chapters retrieve \$6 by name, more than any other section here. Nothing else in this chapter is retrieved as often.

"Kubernetes is not a mere orchestration system. In fact, it eliminates the need for orchestration." — the Kubernetes documentation [source: k8s-docs-overview-2026-08-23]

🔊 Soundings

Before reading this chapter, try these questions. Your score determines how to approach the content. No shame in any score; the score points you at a reading strategy, nothing more. Seven of these test what you already know coming in; one is a deliberate look back at Chapter 2.

1. You have three copies of an application running and you want five. How would you make that happen?
2. It's 3 a.m. A machine holding some of your application's copies fails. Who notices, and what do they do about it?
3. Does every machine in a Kubernetes cluster run the same software?
4. On a worker machine, which component is responsible for making sure the containers described for that machine are actually running? *[retrieval: ch2]*
5. A command-line tool, an internal scheduler, and fifty machine-level agents all need to read and write the same configuration data. How many of them should be allowed to talk to the datastore directly?
6. Does Kubernetes build your container images?
7. Was Kubernetes written from scratch, or does it descend from an earlier internal system at some company?
8. "Orchestration" — is that a loose compliment people pay to container platforms, or is it a technical term with a specific meaning?

Click for answers + reading strategy

1. Most people with scripting experience answer "start two more." §6 is written for that answer.
2. Common answers: a human, woken by a pager. Also common: "something automatic." §6 tells you which one Kubernetes chose and why.
3. No. Kubernetes clusters have two kinds of machine running two different sets of software. §2 and §3 name them.
4. The kubelet. Chapter 2 taught the chain from the kubelet down through the CRI to the runtime. If that came back easily, §3 will be a review. *[retrieval: ch2]*
5. One. Most people with API-design exposure answer this correctly on instinct; §5 is about the consequences, not the rule.
6. No. Images are built elsewhere and pulled. §1 has the rest of that list.

7. It descends. Kubernetes drew on Google's internal container-orchestration experience: the Borg system, and its research successor Omega [source: k8s-history-ten-years-2026-08-23].
8. It's a technical term with a specific meaning, and the meaning is narrower than the way the industry uses the word. §1 plants it; §7 settles it.

If you got 6+ right: Skim, with two exceptions. Read §6 at full pace regardless of your score, because six later chapters retrieve it by name. And read §4 rather than skimming it; the optionality material scores well on intuition and still catches people on exam day.

If you got 3–5 right: Read at normal pace. This chapter is calibrated for you.

If you got 0–2 right: Read carefully, in its own session, and don't move on to Chapter 4 the same day. Chapter 1 told you that Chapters 2 and 3 carry the conceptual load everything else rests on; this is the second of the two. Nothing here is difficult. There are eight component names and one idea, and the idea is worth more than the names.

Why This Chapter Matters

There is no component in Kubernetes whose job is to be in charge.

That should bother you. If you have ever built or operated a distributed system, the obvious way to make fifty machines agree on something is to put one of them in charge: a coordinator that holds the plan, hands out assignments, and tracks completion. Every workflow engine, every job scheduler, every deployment tool you have used works roughly that way. Kubernetes has no such component. You are about to meet eight components, you will go looking for the one that runs things, and you will not find it. Hold that question. It resolves in §6 and §7, and how it resolves is the most useful thing this chapter has to give you.

What you're building here isn't component recall. It's layer attribution. A practitioner who owns this chapter can be handed a symptom (Pods not starting, a Service with no endpoints, a node gone quiet) and reason about *which loop stopped closing*, instead of guessing which service to restart. Chapter 1 established that this exam measures discrimination rather than memorization. Here the discrimination is architectural: knowing not just what each piece is called but what it can and cannot do, and what has to be true for it to act at all. The component census is the vocabulary. The control loop is the grammar. The exam tests both. Practitioners get paid for the second.

The stakes are structural rather than dramatic. This chapter covers roughly six points of exam surface on this book's own judgment. CNCF and the Linux Foundation publish weights per domain, with competencies named but not individually weighted [source: cncf-kcna-curriculum-pdf-2026-08-23] [source: lf-kcna-exam-page-2026-08-23], so treat that number as an estimate the author is accountable for, not as a figure from the curriculum. What makes the chapter large isn't its exam weight. It's that six later chapters retrieve the control loop by name, and one of them, Chapter 15, is built entirely around

you recognizing it somewhere unfamiliar. A reader who finishes here able to recite eight component names but unable to sketch a reconciliation loop from memory has the smaller half of what's on offer.

What You'll Learn

By the end of this chapter, you'll be able to:

- **Explain** why the container deployment era produced a system shaped like Kubernetes, and state three things Kubernetes deliberately does not do.
- **Name** every control-plane and node component, state its one job, and mark which two are optional and under what circumstances.
- **Trace** what happens between a submitted request and a running container, naming which component acts at each step, and which component never does.
- **Draw** a control loop from memory: desired state, current state, and the action that closes the gap, repeating forever.
- **Distinguish** orchestration in the technical sense (first do A, then B, then C) from a set of independent control processes, and explain why Kubernetes claims to eliminate the need for the first.

You'll also stop looking for the component that's in charge, which is the single most useful habit this chapter can leave you with.

§1 — .1 How the Cluster Got the Shape It Has

Systems have shapes, and the shapes have reasons. Kubernetes looks the way it looks because of a specific sequence of problems the industry solved one at a time, each solution creating the conditions for the next.

[cross-bearing: see Ch 2 §4 — the kubelet, CRI, and the container runtime chain]

The three deployment eras

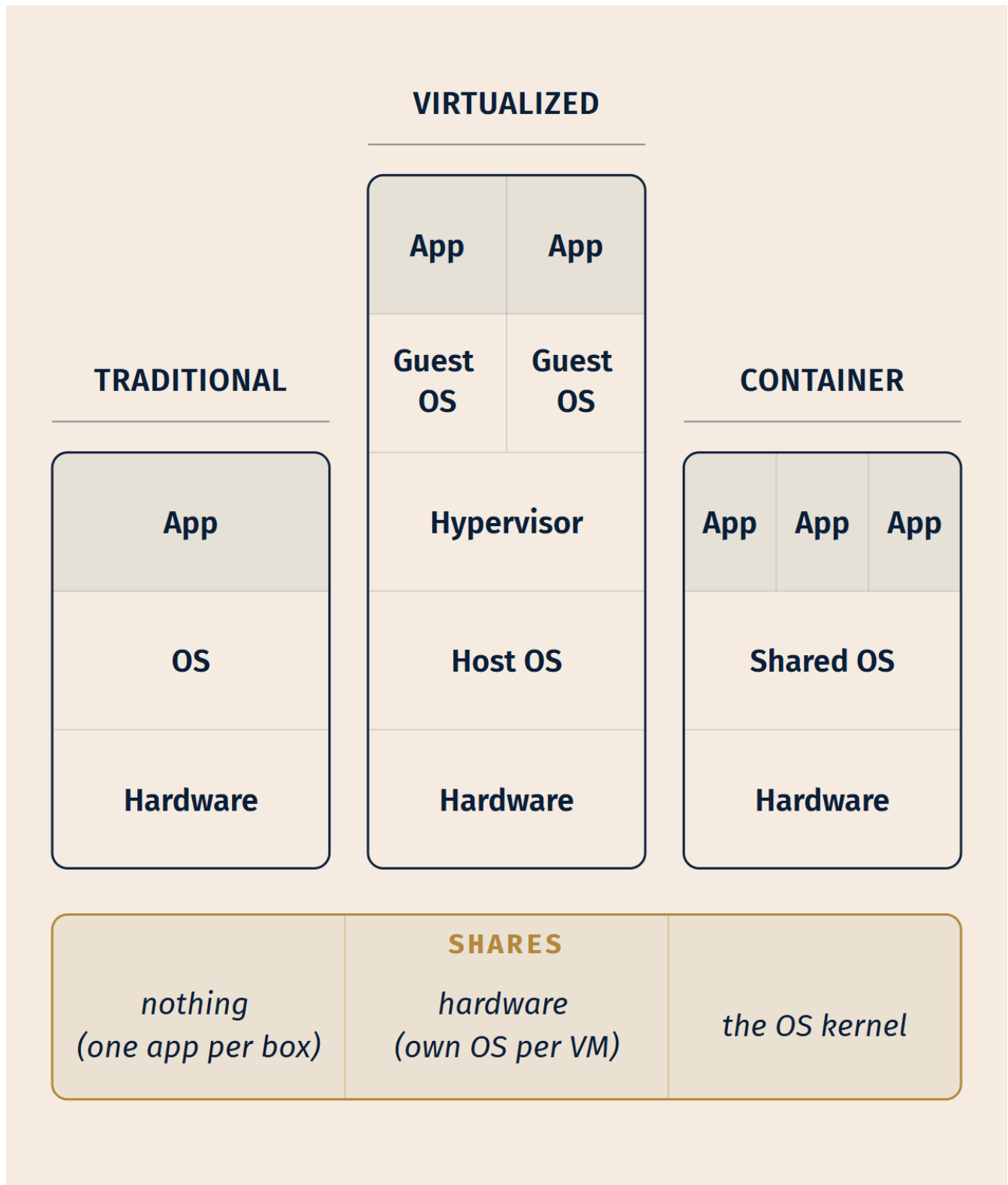
Traditional deployment era. Early on, organizations ran applications on physical servers. There was no way to define resource boundaries for applications on a physical server, and this caused resource-allocation issues. If multiple applications run on one physical server, one application can take up most of the resources, and the others underperform. The available solution was to run each application on a different physical server, which did not scale, because resources sat underutilized and maintaining many physical servers was expensive [source: k8s-docs-overview-2026-08-23].

Virtualized deployment era. Virtualization allowed multiple Virtual Machines to run on a single physical server's CPU. Applications became isolated between VMs, which provides a level of security because the

information of one application cannot be freely accessed by another. Utilization improved, hardware costs dropped, and scalability improved because an application could be added or updated easily. With virtualization you can present a set of physical resources as a cluster of disposable virtual machines. But each VM is a full machine running all the components, including its own operating system, on top of virtualized hardware [source: k8s-docs-overview-2026-08-23].

Container deployment era. Containers are similar to VMs, but they have relaxed isolation properties that let them share the operating system among the applications. That's why containers are considered lightweight. Like a VM, a container has its own filesystem, share of CPU, memory, process space, and more. Because they are decoupled from the underlying infrastructure, they are portable across clouds and OS distributions [source: k8s-docs-overview-2026-08-23].

That last sentence is the documentation's, and it says *operating system*. The sharper statement, that what a container shares with the host is specifically the **kernel**, is this book's, not the documentation's, and you'll see it written that way where the precision earns its keep. Treat it as our gloss on an accurate but coarse sentence, not as a quotation.



What changes across the eras is not the application. It's what the application shares with everything else on the machine. Read the bottom row.

This chapter isn't re-teaching the container/VM architecture; Chapter 2 did that in detail. What matters here is the *consequence*. Once your unit of deployment is small, fast, and portable, you stop having a handful of servers with one application each and start having hundreds of interchangeable processes scattered across a pool of machines. That is a fundamentally different management problem, and it's the problem Kubernetes was built for.

[cross-bearing: see Ch 2 §1 — the architectural container-versus-VM contrast]

What Kubernetes provides

The published capability list reads like marketing until you read it as a list of container-era problems, each with a name attached [source: k8s-docs-overview-2026-08-23]:

Capability	The problem it solves
Service discovery and load balancing	Containers move and get new IPs; clients need a stable name. Kubernetes can expose a container using a DNS name or its own IP address, and can load balance and distribute traffic when it's high.
Storage orchestration	Containers are ephemeral; some data isn't. Kubernetes lets you automatically mount a storage system of your choice — local, public cloud, and more.
Automated rollouts and rollbacks	Updating hundreds of copies by hand is unmanageable. You describe the desired state and Kubernetes changes actual state to desired state at a controlled rate.
Automatic bin packing	Placing containers on machines by hand wastes capacity. You tell Kubernetes how much CPU and memory each container needs, and it fits containers onto nodes to make best use of resources.
Self-healing	Things fail constantly at scale. Kubernetes restarts containers that fail, replaces containers, kills containers that don't respond to your health check, and doesn't advertise them to clients until they are ready to serve.
Secret and configuration management	Baking credentials into images is both a security problem and a rebuild problem. You can deploy and update secrets and application configuration without rebuilding images and without exposing secrets in your stack configuration.
Batch execution	Not every workload is a service. Kubernetes can manage batch and CI workloads, replacing containers that fail if desired.
Horizontal scaling	Demand changes. Scale up and down with a command, a UI, or automatically based on CPU usage.
IPv4/IPv6 dual-stack	Allocation of IPv4 and IPv6 addresses to Pods and Services.
Designed for extensibility	Add features to your cluster without changing upstream source code.

Read that right-hand column again. Notice how many entries describe something that has to *keep being true* rather than something that has to *happen once*. Self-healing isn't an action; it's a standing condition. Hold that thought. It's the shape of §6.

What Kubernetes is not

This list is more useful than the last one, and it is easier to get wrong than its length suggests. Kubernetes is not a traditional, all-inclusive PaaS. Because it operates at the container level rather than the hardware level, it provides some features common to PaaS offerings (deployment, scaling, load balancing) and lets users integrate their own logging, monitoring, and alerting. But Kubernetes is not monolithic, and those default solutions are optional and pluggable. It provides the building blocks for building developer platforms while preserving user choice [source: k8s-docs-overview-2026-08-23].

Specifically, Kubernetes [source: k8s-docs-overview-2026-08-23]:

- **Does not limit the types of applications supported.** It aims to support an extremely diverse variety of workloads — stateless, stateful, data-processing. If an application can run in a container, it should run great on Kubernetes.
- **Does not deploy source code and does not build your application.** CI/CD workflows are determined by organizational culture and preferences as well as technical requirements.
- **Does not provide application-level services** — no middleware (message buses), no data-processing frameworks (Spark), no databases (MySQL), no caches, no cluster storage systems (Ceph) as built-in services. Such things can run *on* Kubernetes, or be accessed by applications running on it.
- **Does not dictate logging, monitoring, or alerting solutions.** It provides some integrations as proof of concept, and mechanisms to collect and export metrics.
- **Does not provide nor mandate a configuration language or system.** It provides a declarative API that arbitrary declarative specifications may target.
- **Does not provide nor adopt any comprehensive machine configuration, maintenance, management, or self-healing systems.**

And one more, which we are going to leave sitting here unresolved:

📌 **Snag:** The documentation also says Kubernetes "is not a mere orchestration system" and that "it eliminates the need for orchestration" [source: k8s-docs-overview-2026-08-23]. The industry uses "orchestration" loosely, as a compliment, and this book used it that way in Chapter 1. The word has a precise technical meaning, and Kubernetes explicitly rejects it. We'll settle this in §7.

[cross-bearing: see Ch 3 §7 — the orchestration disclaimer, resolved]

That Snag deserves a moment of honesty, because it's the book correcting its own earlier wording rather than correcting you. Chapter 1 told you that "Kubernetes is an orchestrator — it decides what should run where." At orientation altitude, that's the right answer to the question being asked (orchestrator versus runtime), and it's how practically everyone in the industry talks. But it's the loose sense of the word. The precise sense is the one on the exam, and sharpening it is §7's job.

Where it came from

On June 6th, 2014, the first commit of Kubernetes was pushed to GitHub: 250 files and 47,501 lines of Go, bash, and markdown [source: k8s-history-ten-years-2026-08-23]. The project did not appear from

nothing. Kubernetes drew on Google's internal container-orchestration experience with the Borg system and its research successor Omega, and was written in Go [source: k8s-history-ten-years-2026-08-23]. It arrived into a container moment that Docker had opened the year before [source: k8s-history-ten-years-2026-08-23]. A popular, portable unit of deployment created the demand for something to manage those units at scale.

Kubernetes was announced publicly at DockerCon in June 2014, reached v1.0 in July 2015, and was donated by Google to the newly formed Cloud Native Computing Foundation as its first project [source: k8s-history-ten-years-2026-08-23]. CNCF is part of the nonprofit Linux Foundation [source: cncf-who-we-are-2026-08-23]. One more beat of that history earns its keep here, because it explains a shorthand you will still meet: "Kubernetes runs Docker containers." It was once literally true — early Kubernetes drove Docker Engine directly through a built-in shim, and that shim, dockershim, has since been removed from Kubernetes. The images never noticed: an image built with Docker is a standard OCI image, and the runtimes underneath run it natively [source: k8s-blog-dockershim-faq-2026-08-24].

The name comes from the Greek word for helmsman or pilot; "K8s" is the numeronym, with eight letters between the K and the s [source: k8s-history-ten-years-2026-08-23]. The brand you're reading did not pick the maritime register to be cute about it. The subject arrived that way.

🔗 **Closer Look:** Borg and Omega were two different projects, not one renamed. Borg was the production system; Omega is described as its research successor [source: k8s-history-ten-years-2026-08-23]. Kubernetes inherited from both. This is deeper than the exam requires.

[cross-bearing: see Ch 17 §1 — the CNCF, its governance, and the cloud native definition]

§2 — .1 The Control Plane

Start with the shape of the thing. A ship's company sorts itself before it works: some stations keep the reckoning, others keep the deck.

A Kubernetes cluster consists of a control plane plus a set of worker machines, called nodes, that run containerized applications. Every cluster needs at least one worker node in order to run Pods. The worker nodes host the Pods that are the components of the application workload. The control plane manages the worker nodes and the Pods in the cluster. In production environments, the control plane usually runs across multiple computers and a cluster usually runs multiple nodes, providing fault tolerance and high availability [source: k8s-docs-cluster-architecture-2026-08-23].

So: two kinds of machine, two sets of software. That answers Soundings question 3, and it's the first thing to fix in your head, because every component name you're about to learn hangs off it.

CONTROL PLANE

kube-apiserver

etcd

kube-scheduler

kube-controller-manager

cloud-controller-manager

NODE – REPEATS PER WORKER

kubelet

kube-proxy

Container Runtime

containerd

LEGEND



ALWAYS PRESENT



OPTIONAL

The whole census on one page. Two regions, eight components, two of them dashed. The dashes are worth as many exam points as the names.

Now the five components of the control plane. The control plane's components make global decisions about the cluster — scheduling, for example — as well as detecting and responding to cluster events, such as starting up a new Pod when a controller's declared replica count is unsatisfied [source: k8s-docs-cluster-architecture-2026-08-23]. A Pod, for now, is the unit Kubernetes schedules and runs — Chapter 5 is its whole subject.

kube-apiserver

The API server is the component of the control plane that exposes the Kubernetes API. **The API server is the front end for the Kubernetes control plane.** kube-apiserver is designed to scale horizontally: it scales by deploying more instances. You can run several instances and balance traffic between them [source: k8s-docs-cluster-architecture-2026-08-23]. In the components summary the docs describe it plainly as the core component server that exposes the Kubernetes HTTP API [source: k8s-docs-components-2026-08-23].

That "front end" phrase is doing enormous work, and §5 is devoted to unpacking it. For now: everything comes in through here.

[cross-bearing: see Ch 3 §5 — what "front end" actually implies about the cluster's shape]

etcd

Consistent and highly-available key value store used as Kubernetes' backing store for all cluster data. If your Kubernetes cluster uses etcd as its backing store, make sure you have a back up plan for the data [source: k8s-docs-cluster-architecture-2026-08-23].

That's the whole official description, and it's terser than it looks. Two words deserve unpacking, and what follows is this book's explanation of them rather than more documentation. "All cluster data" means all of it: every object you created, every object the system created, every piece of state the cluster knows about. And "consistent" is the property the rest of the architecture leans on. In plain terms, it's what lets independent components reading the same shared state work from the same picture of the world rather than from private, drifting copies. Take that as the gloss it is; the formal guarantees are etcd's own subject, not this chapter's.

🔒 **Worth Securing:** etcd is where the cluster lives. It is the backing store for *all* cluster data [source: k8s-docs-cluster-architecture-2026-08-23], which means the other components can be killed, restarted, or replaced and rebuild their working picture from what etcd holds. etcd itself holds the thing that cannot be rebuilt from anywhere else. Losing it is not something you recover from by restarting a process. It's something you recover from by having taken a backup, which is why the documentation's one piece of unsolicited advice about etcd is "make sure you have a back up plan."

[cross-bearing: see Ch 8 — etcd backup and restore in practice] [cross-bearing: see Ch 12 — Secrets live in etcd, which is why encryption at rest is a separate decision you have to make]

kube-scheduler

Control plane component that watches for newly created Pods with no assigned node, and selects a node for them to run on. Factors taken into account for scheduling decisions include individual and collective resource requirements, hardware/software/policy constraints, affinity and anti-affinity specifications, data locality, inter-workload interference, and deadlines [source: k8s-docs-cluster-architecture-2026-08-23].

One component boundary needs stating precisely right now, because it heads off a mistake that costs points: **the scheduler selects a node and records that choice. It does not start anything.** The scheduler notifies the API server about its decision, in a process called binding [source: k8s-docs-kube-scheduler-2026-08-23]. The container that eventually runs is started by the kubelet on the chosen node. The scheduler makes a decision; something else acts on it. That split will look familiar by the end of §6.

[cross-bearing: see Ch 7 — how the scheduler actually chooses, in detail]

kube-controller-manager

Control plane component that runs controller processes. Logically, each controller is a separate process, but to reduce complexity, they are all compiled into a single binary and run in a single process. There are many different types of controllers — for example: the Node controller (noticing and responding when nodes go down); the Job controller (watching for Job objects that represent one-off tasks, then creating Pods to run those tasks to completion); the EndpointSlice controller (populating EndpointSlice objects to provide a link between Services and Pods); and the ServiceAccount controller (creating default ServiceAccounts for new namespaces) [source: k8s-docs-cluster-architecture-2026-08-23].

⚠️ **Navigational Hazards:** Read that second sentence twice. *Logically*, each controller is a separate process. That's the conceptual model. *Actually*, they are all compiled into a single binary and run in a single process. This is a single sentence in the documentation and it reads like a trivial implementation note, which is exactly why it gets tested. Candidates who skimmed it will tell you confidently that there's one process per controller, or one container per controller, or one Pod per controller. All three are wrong for the same reason: there is no per-controller unit of any kind. One binary. One process. Many logical controllers inside it.

cloud-controller-manager

A Kubernetes control plane component that embeds cloud-specific control logic. The cloud controller manager lets you link your cluster into your cloud provider's API, and separates out the components that interact with that cloud platform from components that only interact with your cluster. It runs only controllers that are specific to your cloud provider. **If you are running Kubernetes on your own premises, or in a learning environment inside your own PC, the cluster does not have a cloud controller manager** [source: k8s-docs-cluster-architecture-2026-08-23].

That last sentence is the one to hold onto. This component is genuinely optional. Not "optional but everyone runs it" — *absent* from an entire class of real clusters, including the one on your laptop. §4 comes back to it.

🔗 **Mnemonic:** Don't reach for an acronym here; the structure *is* the memory hook. Five components, and they sort themselves. **Three named kube-** (apiserver, scheduler, controller-manager) are the core Kubernetes control-plane processes. **One named cloud-** (cloud-controller-manager) is the optional bridge to somebody else's infrastructure. **One with no prefix at all**, etcd, because it isn't Kubernetes software; it's a general-purpose datastore that Kubernetes uses. If you can reconstruct that sorting rule, you can reconstruct the list.

§3 — Node Components in Context

[cross-bearing: see Ch 2 §4 — the kubelet, CRI, and container runtime chain, which this section places in its architectural context]

Node components run on every node, maintaining running Pods and providing the Kubernetes runtime environment [source: k8s-docs-components-2026-08-23]. Three of them. Keep that framing sentence: the node's collective job is to *maintain* running Pods, not to start them once and stand down.

kubelet

An agent that runs on each node in the cluster. It makes sure that containers are running in a Pod. The kubelet takes a set of PodSpecs that are provided through various mechanisms and ensures that the containers described in those PodSpecs are running and healthy. **The kubelet doesn't manage containers which were not created by Kubernetes** [source: k8s-docs-cluster-architecture-2026-08-23].

A PodSpec, for now, is simply the description of what containers should run. Chapter 4 gives you spec in general; Chapter 5 gives the PodSpec its proper treatment.

[cross-bearing: see Ch 5 — Pods, PodSpecs, and what "running and healthy" means precisely]

🔒 **Worth Securing:** That last clause has practical teeth. If you SSH to a node and start a container by hand with the container runtime directly, the kubelet does not manage it [source: k8s-docs-cluster-architecture-2026-08-23]. It won't restart it, won't maintain it, won't clean it up. Whatever the cluster knows about that node, the hand-started container isn't part of it. This cuts both ways: it's why hand-started containers on a node don't turn up through `kubectl`, and it's why node-level debugging needs a node-level tool rather than the Kubernetes API.

[cross-bearing: see Ch 13 §5 — `crictl`, and why a node-level tool exists below the Kubernetes API]

kube-proxy (optional)

kube-proxy is a network proxy that runs on each node in your cluster, implementing part of the Kubernetes Service concept. kube-proxy maintains network rules on nodes. These network rules allow network communication to your Pods from network sessions inside or outside of your cluster. **If you use a network plugin that implements packet forwarding for Services by itself, and providing equivalent behavior to kube-proxy, then you do not need to run kube-proxy on the nodes in your cluster** [source: k8s-docs-cluster-architecture-2026-08-23].

Note the precise hedge: it implements *part* of the Service concept. Service is a Kubernetes object with its own chapter; kube-proxy is the node-level machinery that makes some of it work. Note the optionality condition too, because "optional" here has a specific trigger, something else doing the same job, not a general "you can skip it if you like."

[cross-bearing: see Ch 9 — Services, and how kube-proxy implements them]

Container runtime

A fundamental component that empowers Kubernetes to run containers effectively. It is responsible for managing the execution and lifecycle of containers within the Kubernetes environment. Kubernetes supports container runtimes such as containerd, CRI-O, and any other implementation of the Kubernetes CRI (Container Runtime Interface) [source: k8s-docs-cluster-architecture-2026-08-23].

You already own this one. Chapter 2 walked the chain: the kubelet speaks CRI, the CRI implementation (containerd, CRI-O) manages container lifecycle, and below that sits the low-level runtime that actually creates the process. What §3 adds is *position*. The container runtime is the one node component that is not Kubernetes software at all. It's a separate project, swappable, and not authored by the Kubernetes project: Kubernetes defines the interface and accepts any implementation of it [source: k8s-docs-cluster-architecture-2026-08-23] — the pattern Chapter 2 told you to name: *Kubernetes defines an interface and lets the ecosystem implement it* [cross-bearing: see Ch 2 §4 — the Container Runtime Interface]. etcd reflects an adjacent instinct, worth keeping distinct: no interface is defined there. Where a good general-purpose component already exists, Kubernetes simply uses it rather than building its own.

That's the census. Eight components, and here it is in one place.

☆ Fixed Point

Control plane: kube-apiserver, etcd, kube-scheduler, kube-controller-manager, and cloud-controller-manager (*optional — absent on premises and on your laptop*).

Node (on every node): kubelet, kube-proxy (*optional — unnecessary if a network plugin provides equivalent packet forwarding*), and the container runtime.

Eight names. Two of them optional, for two different reasons. One of them — the container runtime — is not Kubernetes software. [source: k8s-docs-components-2026-08-23] [source: k8s-docs-cluster-architecture-2026-08-23]

Dead Reckoning: kube-apiserver runs on the control plane and exposes the Kubernetes HTTP API. etcd runs on the control plane and stores all cluster data. kube-scheduler runs on the control plane and assigns unscheduled Pods to nodes. kube-controller-manager runs on the control plane and runs controller processes in a single binary. cloud-controller-manager runs on the control plane when a cloud provider is involved and integrates with that provider's API. kubelet runs on every node and ensures the containers described for that node are running. kube-proxy runs on every node unless something else does its job, and maintains node network rules for Services. The container runtime runs on every node and executes containers. [source: k8s-docs-components-2026-08-23] [source: k8s-docs-cluster-architecture-2026-08-23]

🕒 Taking Your Bearings #1: The Ship's Company

Five questions. One of them reaches back to Chapter 2.

Q1. .. Which of the following components runs on *every node* rather than on the control plane?

A) kube-proxy B) kube-scheduler C) etcd D) cloud-controller-manager

Q2. .. "Watches for newly created Pods with no assigned node, and selects a node for them to run on." Which component?

A) kube-controller-manager B) kubelet C) kube-apiserver D) kube-scheduler

Q3. .. How does kube-controller-manager run its controllers?

A) All controllers compiled into a single binary, running in a single process B) One operating-system process per controller, supervised by a parent process C) One container per controller, all within a single Pod D) One Pod per controller, all in the kube-system namespace

Q4. .. What does etcd store?

A) Only the objects you explicitly created B) All cluster data C) Only Pod definitions and node registrations D) Cluster data plus cached container image layers

Q5. ... [retrieval: ch2] A Pod has been assigned to a node and its containers need to start. Which node component causes a container to actually start, and through what interface does it do so?

A) kube-proxy, via the iptables rules it maintains on the node B) kube-scheduler, by connecting directly to that node's container runtime C) The container runtime, by polling the API server for its own assignments D) The kubelet, via the Container Runtime Interface (CRI)

Answers with Explanations:

Q1 — A. kube-proxy is a node component; it runs on each node in the cluster, maintaining network rules [source: k8s-docs-cluster-architecture-2026-08-23].

- **B is wrong** — kube-scheduler is a control plane component. It's a common slip because scheduling *results* in something happening on a node, but the scheduler itself lives on the control plane.
- **C is wrong** — etcd is a control plane component and the backing store for all cluster data.
- **D is wrong** — cloud-controller-manager is a control plane component, and an optional one at that.

Q2 — D. That is the kube-scheduler's published description, near-verbatim [source: k8s-docs-cluster-architecture-2026-08-23].

- **A is wrong** — the controller-manager runs controllers. Several of them create Pods, but none of them chooses which node a Pod lands on.
- **B is wrong** — the kubelet acts *after* a node has been chosen, ensuring the containers described for its own node are running. It doesn't select nodes.
- **C is wrong** — the API server serves the API. It stores and serves the fact that a Pod has no node yet; it doesn't decide the node.

Q3 — A. Logically each controller is a separate process, but to reduce complexity they are all compiled into a single binary and run in a single process [source: k8s-docs-cluster-architecture-2026-08-23].

- **B is wrong** — and it's wrong in the most tempting way, because the documentation's own sentence begins "Logically, each controller is a separate process." That word *logically* is the whole question.
- **C is wrong** for the same reason B is: there is no per-controller unit of any kind, container or otherwise. The unit is the controller-manager itself, and every controller lives inside it.
- **D is wrong** for that same reason. Whatever the controller-manager happens to be packaged as in your cluster, there is *one* of it, not one per controller.

Q4 — B. etcd is the backing store for all cluster data [source: k8s-docs-cluster-architecture-2026-08-23].

- **A is wrong** — the cluster creates plenty of objects you didn't (default ServiceAccounts, EndpointSlices, Leases), and those are stored there too.
- **C is wrong** — a subset answer. Believing etcd holds only *some* of the state is the specific misconception that leads people astray later, when Chapter 12 asks why Secret encryption at rest is a separate configuration decision.

- **D is wrong** — image layers live in a registry and, once pulled, are cached on node disk by the kubelet and runtime [source: k8s-docs-images-2026-08-23]. etcd stores API objects, not binary payloads.

Q5 — D. [retrieval: ch2] The kubelet ensures the containers described in its PodSpecs are running [source: k8s-docs-cluster-architecture-2026-08-23], and it reaches the runtime through the Container Runtime Interface, the extension point Kubernetes defines for alternative container runtimes [source: k8s-docs-extending-kubernetes-2026-08-23].

- **A is wrong** — kube-proxy handles network rules for Services. It has nothing to do with starting containers.
- **B is wrong** — the scheduler selects a node and records the decision. It never contacts a node's runtime. This is the highest-value wrong answer on the page, because the belief behind it ("the scheduler places the Pod on the node, so it must talk to the node") is genuinely widespread.
- **C is wrong** — the container runtime doesn't poll the API server. It sits below the kubelet and is driven by it through CRI. The runtime doesn't know Kubernetes exists.

If you scored 5/5: The census is yours. Move straight on; §4 is short.

If you scored 3–4: Solid. Review the ones you missed, particularly if Q3 or Q5 was among them, and continue.

If you scored 0–2: Go back to §2 and §3 and rebuild the census from the ☆ Fixed Point, then re-read the Dead Reckoning block. Ten minutes. The rest of the chapter assumes you can place any component on the right side of the control-plane/node line without thinking about it.

Checkpoint: You've Now Mastered ✓ The control-plane/node split, and what each side is for ✓ All eight components and the one job each holds ✓ Which two are optional, and the fact that the reasons differ ✓ That the scheduler decides and the kubelet acts — a split you'll see again

📖 **Chapter 3 · Voyage Progress:** census complete → now the arrangement → then the loop

§4 — Addons, and What Else Is Optional

This is the shortest section in the chapter and it exists for one sentence at the end of it.

Addons

Addons extend the functionality of Kubernetes. The published examples are: **DNS**, for cluster-wide DNS resolution; **Web UI (Dashboard)**, for cluster management via a web interface; **Container Resource Monitoring**, for collecting and storing container metrics; and **Cluster-level Logging**, for saving container logs to a central log store [source: k8s-docs-components-2026-08-23] [source: k8s-docs-cluster-addons-2026-08-24].

Note the word *extend*. Addons are not components in the sense that the eight names in §3 are components. They run *on* the cluster rather than constituting it. A cluster with none of them installed is still a cluster.

Three kinds of optional

You have now met eight components and four addons. **Two of those twelve carry the word "optional" in the documentation itself:** kube-proxy and cloud-controller-manager [source: k8s-docs-components-2026-08-23]. They carry it for two genuinely different reasons. The addons are a third kind of optional, but that framing is this book's rather than the documentation's: the docs say addons *extend* the cluster, and never apply the word "optional" to them.

What	Why it's optional	Whose word
kube-proxy	Because something else can do its job. If your network plugin implements equivalent packet forwarding for Services, kube-proxy is redundant [source: k8s-docs-cluster-architecture-2026-08-23].	The documentation's
cloud-controller-manager	Because there may be no cloud to talk to. On premises or on your laptop, the cluster simply does not have one [source: k8s-docs-cluster-architecture-2026-08-23].	The documentation's
Addons	Because they extend rather than constitute [source: k8s-docs-components-2026-08-23]. The cluster is a cluster without them.	This book's framing

⚠ **Navigational Hazards:** Two very common wrong beliefs live here, and both are cheap to get right once you've noticed the word *optional*.

"kube-proxy runs on every node, always." The documentation says node components run on every node, then marks kube-proxy optional in the same breath. Both are true: *when it runs, it runs on every node*, but it may not run at all, if a network plugin does the same work.

"Every cluster has a cloud-controller-manager." No. Every cluster on a cloud provider that integrates one does. On premises, or on a laptop, that component is simply absent.

The shared lesson is duller and more valuable than either fact: **read the word "optional" when the documentation prints it.** It's printed rarely and it's printed deliberately, on exactly two of the eight components.

Cluster DNS, and the pattern to name

Cluster DNS is the honest illustration. It is, formally, an addon: a cluster extension, not a control-plane or node component. It is also described as a built-in Kubernetes service, launched automatically by the addon manager [source: k8s-docs-dns-cluster-addon-2026-08-24]. That is why, in the author's experience

of real clusters, you will almost always find it present. Services are addressed by name, and Pods expect to be able to resolve those names.

That second half is an observation, not a documented claim. Hold the two apart: *launched automatically where the addon manager runs* is sourced; *practically universal* is the author's report.

Which brings us to the sentence this section exists for.

🔒 **Worth Securing — the absent-component pattern.** An object can exist while nothing at all happens, if the component that would act on it is absent. The object is real; it's stored; `kubectl get` will show it to you. But an object is a *description*, and descriptions don't do anything by themselves. Something has to be watching for it and willing to act. Remember the phrase: **an object without its component does nothing**. You're going to meet it four more times in this book, and each time it will look like a completely different problem until you recognize it.

[cross-bearing: see Ch 9 — CoreDNS as the cluster DNS addon, and the Service DNS records it serves]

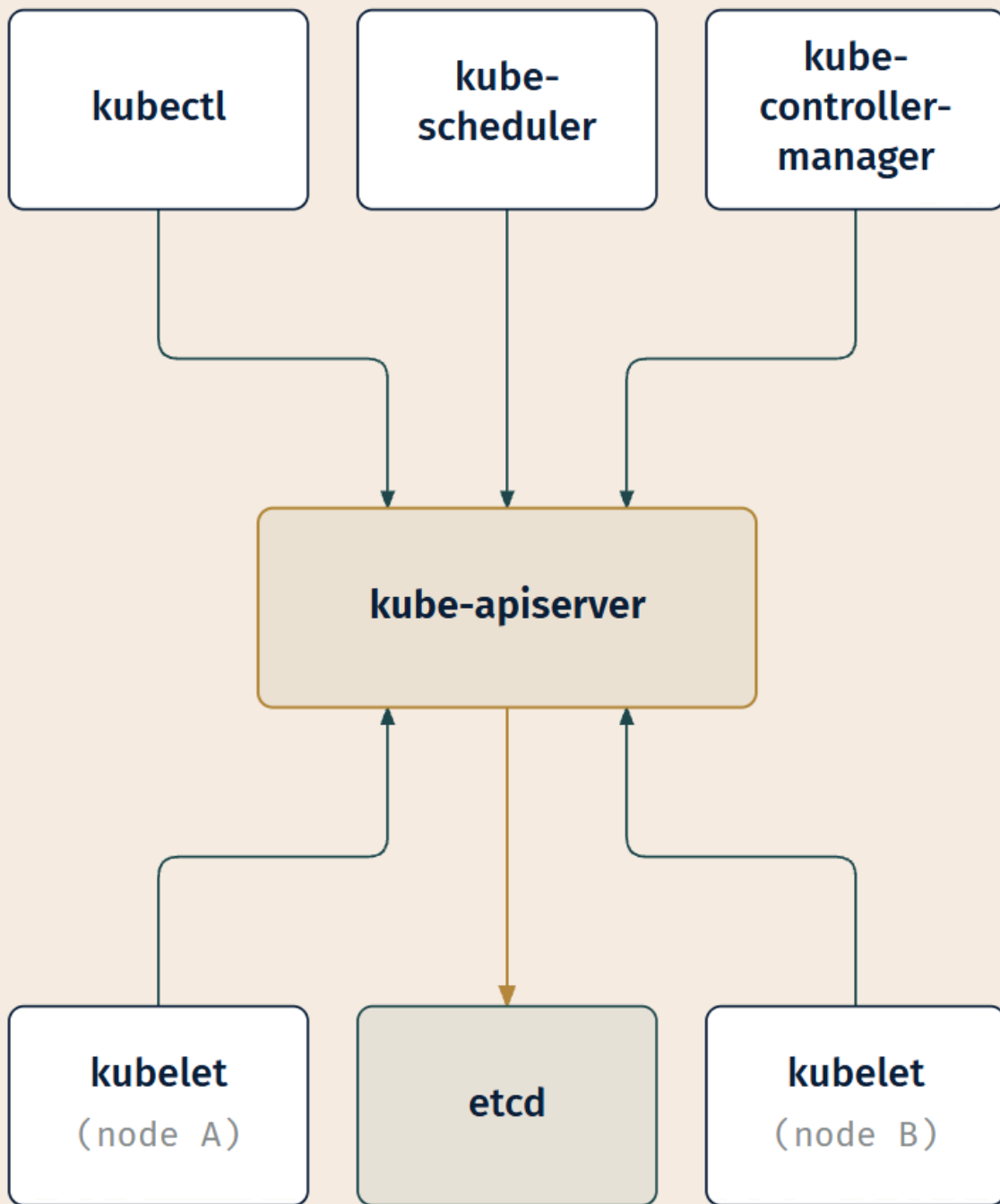
[cross-bearing: see Ch 10 — an Ingress with no Ingress controller does nothing at all: the same pattern, first recurrence] [cross-bearing: see Ch 13 — `kubectl top` with no metrics-server installed] [cross-bearing: see Ch 17 — VPA is an addon and is not shipped by default]

§5 — .1 The Only Door In

You have the census. Now the arrangement, because eight components in a list is a vocabulary, and eight components with a shape is an architecture.

Go back to one phrase from §2: **the API server is the front end for the Kubernetes control plane** [source: k8s-docs-cluster-architecture-2026-08-23]. Add the other fact you already have: etcd is the backing store for all cluster data [source: k8s-docs-cluster-architecture-2026-08-23]. Add a third, from the API-communication documentation: Kubernetes has a hub-and-spoke API pattern. All API usage from nodes and the Pods they run terminates at the API server, and none of the other control-plane components are designed to expose remote services [source: k8s-docs-control-plane-node-communication-2026-08-24]. And add one more, from the controllers documentation: a controller "might carry the action out itself; more commonly, in Kubernetes, a controller will send messages to the API server that have useful side effects" [source: k8s-docs-controllers-2026-08-23].

Put those together and a shape appears. Not a chain of command. A chart table: one surface everyone works from, and no orders issued across it.

**LEGEND** FOCAL HUB DATA STORE CLIENT

The state path. Every arrow terminates at the API server, and the API server is the one component with an edge to etcd. Now look for the arrow that isn't drawn: there is none between any two of the surrounding components.

What follows from a hub

Consider what has to be true for that picture to work.

Every actor in the cluster interacts with the same front end: the CLI you type into, the scheduler making placement decisions, the controller-manager running its many controllers, every kubelet on every node. None of them keeps its own copy of the truth. None of them reaches around the front end to instruct another component. They all read from and write to one place.

The datastore behind that front end deserves one precise sentence, because the precision matters in Chapter 12. The API server is the component that reads and writes etcd. Access to etcd is equivalent to root permission in the cluster, so ideally only the API server should have access to it [source: k8s-docs-etcd-access-control-2026-08-24]. Read that as what it is: a strong recommendation with a security reason behind it, not a law of physics. Anything else you give etcd access to, you have given root.

One qualification, and it matters later. The hub describes how *state* moves. Everything that reads or writes the cluster's state does so through the API server. It is not a claim that no connection ever runs outward. The API server does open connections to kubelets: that is how fetching Pod logs, attaching to a running container, and port-forwarding work [source: k8s-docs-control-plane-node-communication-2026-08-24]. Those paths carry a session, not an instruction. Nothing traveling on them tells a kubelet what ought to be running.

[cross-bearing: see Ch 13 — the debugging commands that ride those outbound paths]

Now consider what the hub means for two components that appear to cooperate. When the scheduler assigns a Pod to a node and the kubelet on that node starts the containers, it *looks* like a handoff. It looks like the scheduler told the kubelet to do something. It didn't. The scheduler recorded a decision. The kubelet, independently, was watching for exactly that kind of record, saw it, and acted. Neither one sent the other anything.

 **Worth Securing:** The coordination mechanism in Kubernetes is **watching, not telling**. Components don't send each other instructions. They observe shared state and act on what they see. That single sentence explains most of what looks mysterious about Kubernetes' behavior, and it's the sentence Chapter 15 will retrieve when a controller called Argo CD watches a Git repository that sits outside the cluster [source: argocd-overview-2026-08-23]: the same architecture, with a different thing in the hub position.

[cross-bearing: see Ch 15 — the same shape, with a Git repository where etcd sits here]

The submission story

Concretely, at component altitude. You submit a request describing something that should exist. (Chapter 4 covers what that description looks like and how you write it; the mechanics matter and they're that chapter's to teach.)

1. The request arrives at the API server. It is validated and persisted. Nothing is running yet. Nothing has been "launched."
2. From that moment, the description is simply part of the cluster's state, visible to anything watching.
3. The scheduler, which watches for Pods with no assigned node, notices one and selects a node. It records that selection back through the API server [source: k8s-docs-kube-scheduler-2026-08-23].
4. The kubelet on that node, which watches for Pods assigned to it, notices and starts the containers through the container runtime.
5. Status flows back the same way, through the API server, into etcd, where anything watching can see it.

Read the verbs. *Notices. Selects. Records. Notices.* At no point does one component instruct another. Each one independently observes a state it cares about, does its own job, and writes the result somewhere everyone can see it.

🔗 **Closer Look:** Doesn't a single front end become a bottleneck? The documented answer is that kube-apiserver is designed to scale horizontally: you run several instances and balance traffic between them [source: k8s-docs-cluster-architecture-2026-08-23]. A hub with N interchangeable instances behind a load balancer is architecturally still one door, and operationally is not one machine. This is deeper than the exam requires.

[cross-bearing: see Ch 8 — the authentication, authorization, and admission gates a request actually passes through on its way in] [cross-bearing: see Ch 4 — how you write the description that gets submitted]

🕒 Taking Your Bearings #2: Arrangement and Optionality

Four questions.

Q1. .. Under what condition is kube-proxy unnecessary on your nodes?

A) When the cluster has only a single node and no cross-node traffic B) Never — kube-proxy is a required node component in every cluster C) When your network plugin implements packet forwarding for Services itself D) When you have not created any Services in the cluster yet

Q2. .. You have three clusters: one on a managed cloud provider, one in your company's own data center, and one running on your laptop for learning. In which of them does a cloud-controller-manager

run?

A) All three, since every cluster needs one to manage its infrastructure B) The managed cloud cluster only C) The managed cloud cluster and the data center cluster D) All three, but on the laptop and in the data center it runs with no provider configured

Q3. Cluster DNS is described in the documentation as an addon rather than a component. What follows from that classification?

A) It cannot be used in production clusters B) It runs on the control plane rather than on the nodes C) It is required on every cluster and is simply categorized separately D) It extends the cluster's functionality rather than constituting the cluster

Q4. When kube-scheduler assigns a Pod to a node and the kubelet on that node then starts the Pod's containers, what has passed between the scheduler and the kubelet?

A) A direct gRPC call from the scheduler to the kubelet on the chosen node B) An instruction the scheduler wrote straight into etcd, which the kubelet then read C) Nothing directly — the scheduler recorded its decision through the API server, and the kubelet independently observed it D) A message routed by kube-proxy from the control plane to the chosen node

Answers with Explanations:

Q1 — C. If you use a network plugin that implements packet forwarding for Services by itself, providing equivalent behavior to kube-proxy, you do not need to run kube-proxy [source: k8s-docs-cluster-architecture-2026-08-23].

- **A is wrong** — cluster size has nothing to do with it. A single-node cluster still needs Service networking to work.
- **B is wrong** — the documentation labels kube-proxy optional explicitly. This is trap #1 in this chapter's list, and it catches people precisely because "runs on each node" and "optional" sit two sentences apart.
- **D is wrong** — and instructively so. The optionality has nothing to do with your usage patterns. It's about whether *something else already does the job*.

Q2 — B. If you are running Kubernetes on your own premises, or in a learning environment inside your own PC, the cluster does not have a cloud controller manager [source: k8s-docs-cluster-architecture-2026-08-23].

- **A is wrong** — this is the misconception itself. The component exists to link a cluster to a cloud provider's API; with no cloud provider, there is nothing to link.
- **C is wrong** — a data center cluster running on your own premises is exactly the case the documentation names as *not* having one.

- **D is wrong**, and it's the most interesting wrong answer here. The component isn't present-but-idle; it is absent. An idle-but-present component would still turn up in a component listing on that cluster. This one doesn't.

Q3 — D. Addons extend the functionality of Kubernetes [source: k8s-docs-components-2026-08-23] [source: k8s-docs-cluster-addons-2026-08-24].

- **A is wrong** — addons are entirely production-appropriate. Cluster DNS in particular is launched automatically by the addon manager [source: k8s-docs-dns-cluster-addon-2026-08-24].
- **B is wrong** — the addon/component distinction is about role, not placement. Addons run as ordinary workloads on the cluster.
- **C is wrong** — and this is the useful confusion to clear up. Something can be *near-universal in practice* and still *not part of the cluster's definition*. The gap between those two is exactly where the absent-component pattern lives.

Q4 — C. The scheduler notifies the API server of its decision [source: k8s-docs-kube-scheduler-2026-08-23]; the kubelet watches the API server and acts on what it sees. Coordination in Kubernetes is watching, not telling.

- **A is wrong** — this is the most natural assumption in the whole chapter and it's the one to unlearn. It's how you'd build it. It's not how it's built: none of the other control-plane components are designed to expose remote services [source: k8s-docs-control-plane-node-communication-2026-08-24]. Note the precise claim — not that no connection ever exists between the control plane and a node, but that no component *directs* another.
- **B is wrong** — the API server is the component that reads and writes etcd, and everything else goes through it [source: k8s-docs-etcd-access-control-2026-08-24].
- **D is wrong** — kube-proxy maintains network rules for Services. It is not a message bus for control-plane traffic.

Design note: Q4 is the integrative item, and the one that most rewards having actually read §5 rather than skimmed it. If you got it right by reasoning rather than recall, you have the chapter's second-most-important idea.

If you scored 4/4: You have the arrangement. §6 is the payoff.

If you scored 3: Solid. If Q4 was the one you missed, that's the one to go back for.

If you scored 0–2: Re-read §5, particularly "The submission story," and look again at the figure, specifically at the arrow that isn't drawn. Then re-read the  Navigational Hazards in §4. Both traps in this checkpoint turn on the word "optional," and both are cheap points on exam day.

Checkpoint: You've Now Mastered ✓ What "optional" means for each of the three optional things — and whose word it is in each case ✓ The addon/component distinction, and the absent-component pattern ✓ The hub arrangement: state moves through the API server, and the API server is what reaches etcd ✓ That apparent cooperation between components is independent observation of shared state

≡ **Chapter 3 • Voyage Progress:** census complete → arrangement complete → now the loop

§6 — .11 Controllers and the Control Loop

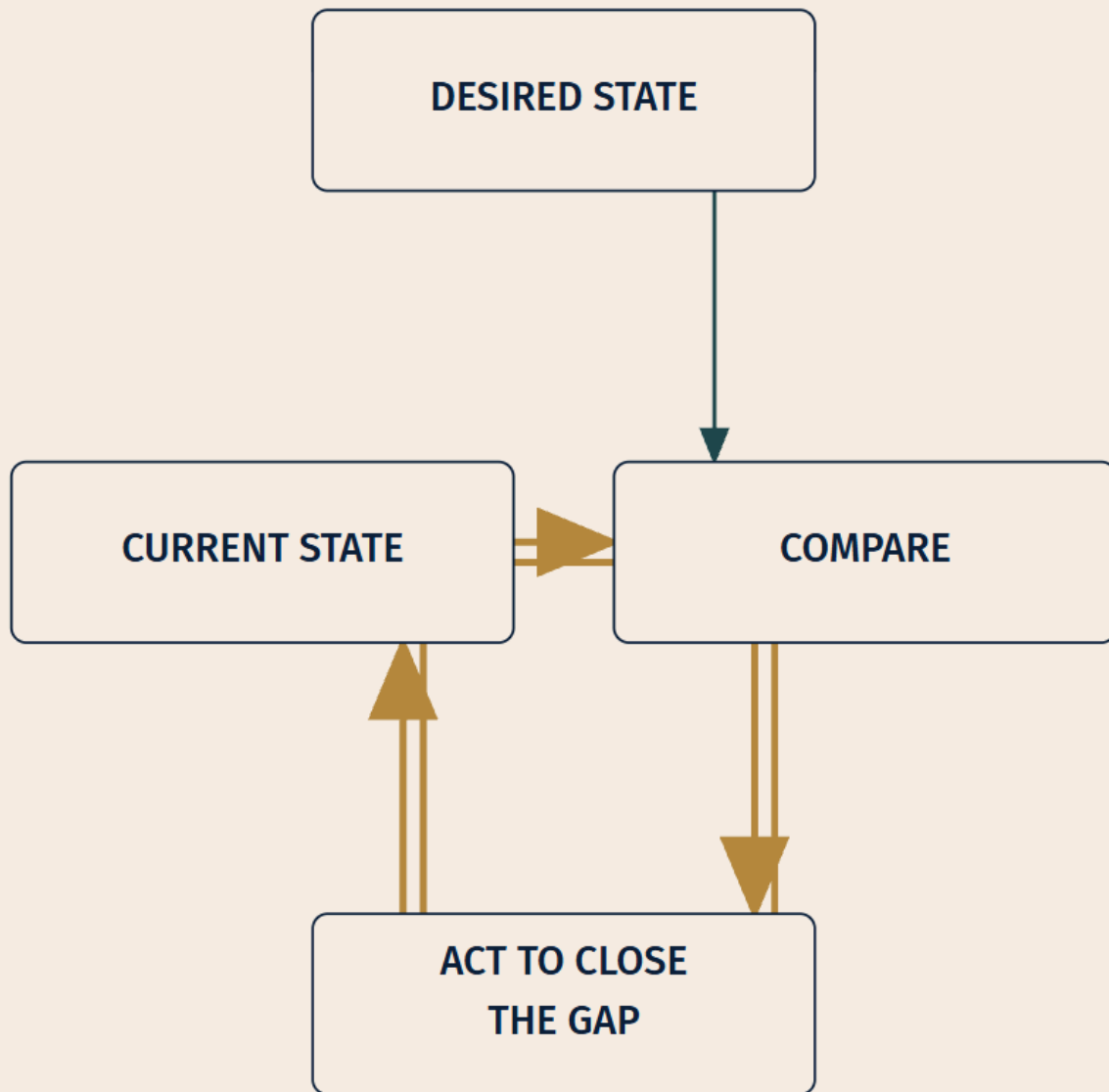
This is the section the rest of the book leans on. If you read one section of this chapter at full attention, read this one.

Start where the documentation starts

In robotics and automation, a control loop is a non-terminating loop that regulates the state of a system. Here is one example: a thermostat in a room. When you set the temperature, that's telling the thermostat about your **desired state**. The actual room temperature is the **current state**. The thermostat acts to bring the current state closer to the desired state, by turning equipment on or off [source: k8s-docs-controllers-2026-08-23].

Sit with how ordinary that is. A thermostat doesn't execute a heating plan. It doesn't calculate that reaching 20°C will require 14 minutes of furnace time and then run the furnace for 14 minutes. It compares two numbers and acts on the difference, then does it again, and again, forever. It never finishes. If someone opens a window, the thermostat doesn't need to be told; the gap widens and it acts. If someone lights a fire, same story in the other direction. Nobody wrote a rule about windows or fires.

In Kubernetes, controllers are control loops that watch the state of your cluster, then make or request changes where needed. Each controller tries to move the current cluster state closer to the desired state [source: k8s-docs-controllers-2026-08-23].



no start. no end. no exit condition.

LEGEND

== CLOSED LOOP

— FIXED INPUT

Notice there is no entry arrow and no terminus. A loop drawn with a beginning teaches the wrong thing: this one was already running before your request arrived and will still be running after it's satisfied.

The controller pattern, precisely

A controller tracks at least one Kubernetes resource type. Those objects carry a field that represents the desired state, and the controller for that resource is responsible for making the current state come closer to it. The controller might carry the action out itself; more commonly, in Kubernetes, a controller will send messages to the API server that have useful side effects [source: k8s-docs-controllers-2026-08-23].

Read that last clause carefully, because it is the distinction most people get wrong.

Control via API server. The Job controller is the documentation's own example of a built-in controller. A Job is a Kubernetes resource that runs a Pod, or perhaps several Pods, to carry out a task and then stop. When the Job controller sees a new task it makes sure that, somewhere in your cluster, the kubelets on a set of Nodes are running the right number of Pods to get the work done. **The Job controller does not run any Pods or containers itself. Instead, the Job controller tells the API server to create or remove Pods. Other components in the control plane act on the new information** — there are new Pods to schedule and run — and eventually the work is done [source: k8s-docs-controllers-2026-08-23].

(The Job *resource*, how you write one and when to reach for it, is Chapter 6's material. We're using it here only because it's the documentation's own chosen example of a controller, and swapping in a different one would cost precision for no gain.)

🔖 **Snag:** "The controller does the work" is the intuitive reading and it's wrong. A controller almost never touches a container. It writes something down. Then a different component, one that has never heard of this controller, notices what was written and acts. If you take one habit from this section, take this one: when something happens in Kubernetes, ask *which component actually performed the action*, and expect the answer to be different from *which controller wanted it*.

Direct control. The less common shape. Some controllers need to make changes to things outside your cluster. If you use a control loop to make sure there are enough Nodes in your cluster, that controller needs something outside the current cluster to set up new Nodes. Controllers that interact with external state find their desired state from the API server, then communicate directly with an external system to bring the current state closer in line [source: k8s-docs-controllers-2026-08-23].

Note what's the same and what's different. The loop is identical: desired state read from the API server, current state observed, act to close the gap. Only the *direction of the action* changes: inward through the API server in the common case, outward to some external system in the uncommon one.

Controllers also update the objects that configure them. Once the work is done for a Job, the Job controller updates that Job object to mark it Finished [source: k8s-docs-controllers-2026-08-23]. The loop reports on itself through the same shared state it reads from.

☆ Fixed Point

A control loop is: a desired state, a current state, and an action that closes the gap between them — repeating, without terminating.

A Kubernetes controller is a control loop that watches cluster state and acts to move current state closer to desired state. It does this continuously, not once. It usually acts by asking the API server to change something, not by doing the thing itself. [source: k8s-docs-controllers-2026-08-23] When later chapters say **reconciliation**, this closing-the-gap work is exactly what the word names.

Extended Analogy:

A ship's company is not a workflow. There is no master schedule pinned in the wardroom listing every action the crew will take between departure and arrival, in order, with dependencies. Such a document would be worthless within the hour, because the sea does not consult it.

What exists instead is standing orders. The helmsman's standing order is a heading: compare the compass to the ordered course, and correct. Not once, but continuously, every few seconds, for the whole watch. The lookout's standing order is a horizon: observe, and report anything on it. The engineer's standing order is a pressure range: watch the gauge, act when it drifts. Each rating holds one comparison and one response, and each of them performs it forever, without waiting to be told and without coordinating with the others.

The vessel arrives on course not because someone executed a plan but because a few dozen small corrections were made continuously by people who were each watching one thing. No one aboard is running the voyage. The voyage is what all of that watching adds up to.

The reason this analogy earns its place here rather than in the prose: what a control loop replaces is *the plan*, and that's easier to feel in a setting where you can picture the plan being useless.

Desired versus current state

Now the claim that unsettles people, and the most quietly radical idea in the chapter.

Kubernetes takes a cloud-native view of systems, and is able to handle constant change. Your cluster could be changing at any point as work happens and control loops automatically fix failures. This means that, potentially, **your cluster never reaches a stable state. As long as the controllers for your cluster are running and able to make useful changes, it doesn't matter if the overall state is stable or not** [source: k8s-docs-controllers-2026-08-23].

That is not a caveat. That is a design position, and it is unusual enough to deserve a moment.

Most systems you've operated treat "converged and quiet" as the healthy state and "constantly changing" as an alarm. Kubernetes inverts it. Constant change is expected: machines fail, load shifts, images get updated, someone deletes something they shouldn't have. A cluster that is never quite finished reconciling isn't malfunctioning; it's a vessel underway, where the small correction is the normal

condition and perfect stillness would be the thing worth investigating. The health question isn't *"has it settled?"* It's *"are the loops running, and can they still make useful changes?"*

Now go back to Soundings question 1: three copies, you want five. The script answer works exactly once, in exactly the conditions you wrote it for. The loop answer handles the same request, plus the machine that dies at 3 a.m., plus the one that dies while you're recovering from the first one, plus the copy someone deletes by hand next Tuesday, without anybody writing a rule for any of those cases. You didn't handle those cases. You stated a desired state and left something watching.

This is also where \$1's capability list cashes out. Self-healing was listed as something Kubernetes *provides* [source: k8s-docs-overview-2026-08-23]: restarting containers that fail, replacing containers, killing containers that fail a health check. Read it now with \$6's vocabulary and it stops being a feature and becomes a description of loops running. A gap opens between what you asked for and what exists, and something closes it. Nobody triggered anything.

[cross-bearing: see Ch 4 — the field that holds desired state, and its status counterpart] [cross-bearing: see Ch 6 — ReplicaSet, a control loop you can watch working in real time] [cross-bearing: see Ch 15 — the same loop, with a Git repository holding desired state]

🕒 Taking Your Bearings #3: Controllers and the Loop

Four questions. These are the ones that matter most.

Q1. ... A component continuously compares a recorded target replica count against the number of Pods actually running, and creates or deletes Pods when they differ. Is this a control loop, and what are its two states?

A) Yes — desired state is the recorded target count; current state is the number actually running B) Yes — desired state is the number actually running; current state is the recorded target count C) No — it's a scheduled task that runs at intervals, and it has no states D) No — control loops observe state but never create or delete objects

Q2. ... The Job controller receives a new Job. What does it actually do?

A) Starts the containers for the Job directly on the nodes it selects B) Tells the API server to create Pods; other components act on that information C) Chooses which node each of the Job's Pods will run on, then starts them D) Connects to each node's kubelet and instructs it to run the Job's Pods

Q3. ... Two controllers: Controller A reconciles the number of running replicas against a declared count. Controller B ensures enough Nodes exist in the cluster, provisioning new machines when needed. Which uses control via the API server, and which uses direct control?

A) Both use control via the API server B) A uses direct control; B uses control via the API server C) A uses control via the API server; B uses direct control D) Both use direct control

Q4. A cluster's state is changing continuously and never settles into a steady configuration. Is this a malfunction?

A) No — as long as the controllers are running and able to make useful changes, overall stability doesn't matter B) Yes — a healthy cluster converges on a stable state and then stays there C) Yes — continuous change means some controller is stuck repeating its reconciliation D) No, but only in clusters that have some form of autoscaling enabled

Answers with Explanations:

Q1 — A. Controllers are control loops that watch cluster state and make or request changes, each trying to move current state closer to desired state [source: k8s-docs-controllers-2026-08-23].

- **B is wrong** — and it's the reversal to catch. Desired state is what you asked for; current state is what's true. Getting them backwards inverts the entire model.
- **C is wrong** — a scheduled task runs at intervals and completes. A control loop is non-terminating and driven by the gap between two states, not by a clock.
- **D is wrong** — creating and deleting objects is exactly how most Kubernetes controllers close the gap.

Q2 — B. The Job controller does not run any Pods or containers itself; it tells the API server to create or remove Pods, and other components in the control plane act on the new information [source: k8s-docs-controllers-2026-08-23].

- **A is wrong** — controllers don't touch containers. That's the kubelet's job, and only for its own node.
- **C is wrong** — node selection is kube-scheduler's job, and it happens after the Pods exist as objects. The Job controller doesn't choose placement.
- **D is wrong** — nothing on the control plane instructs a kubelet. The kubelet watches the API server and acts on what it finds there.

Q3 — C. Controller A works entirely inside the cluster: it reads desired state from the API server and asks the API server to create or remove Pods, which is control via the API server. Controller B needs something outside the cluster to set up new Nodes, so it finds desired state from the API server and then communicates directly with an external system, which is direct control [source: k8s-docs-controllers-2026-08-23].

- **A is wrong** — no amount of API-server messaging can conjure a machine that doesn't exist. B has to reach outside.
- **B is wrong** — the pairing is backwards. Replica reconciliation is the textbook in-cluster case.
- **D is wrong** — direct control is explicitly the less common shape; treating it as the default inverts the documentation's framing.

Q4 — A. Your cluster could be changing at any point as work happens and control loops automatically fix failures; potentially, your cluster never reaches a stable state, and as long as the controllers are running

and able to make useful changes, it doesn't matter if the overall state is stable [source: k8s-docs-controllers-2026-08-23].

- **B is wrong** — this is the intuition most operators bring from other systems, and it's the one Kubernetes explicitly discards.
- **C is wrong** — "stuck in a loop" imports a pathology that doesn't apply. The loop is *supposed* to run forever; running forever isn't the symptom.
- **D is wrong** — the documentation's claim isn't conditional on any feature. It's a statement about the system's design position generally.

If you scored 4/4: You have the idea the rest of the book is built on. §7 is the payoff, and it's short.

If you scored 3: Review the one you missed and continue.

If you scored 0–2: Re-read §6 before continuing, and pay particular attention to the ☆ Fixed Point and the Job-controller paragraph. Chapter 4 opens on the mechanics of desired state and assumes you have the concept.

Checkpoint: You've Now Mastered ✓ The control loop: desired state, current state, action, repeating ✓ What a controller actually does — and what it delegates ✓ Control via API server versus direct control, and which is common ✓ Why a cluster that never settles isn't a broken cluster

📈 **Chapter 3 • Voyage Progress:** census → arrangement → **loop complete**

§7 — 📊 Nobody Is in Charge

☀ Zenith

Put §5 and §6 side by side.

From §5: all state moves through one hub, and components coordinate by watching that state rather than by instructing each other. None of the other control-plane components are even designed to expose remote services [source: k8s-docs-control-plane-node-communication-2026-08-24]. From §6: many independent loops each watch that state and act on their own account, continuously, without terminating.

Now the question this chapter opened with. Where is the component that's in charge?

There isn't one, and now you can see *why* there isn't one, which is more useful than the fact. There is no component whose job is to hold the plan, because there is no plan in that sense. There's a description of what should be true, and a set of independent processes each responsible for one gap between description and reality. Nobody is executing steps, because nobody wrote steps. What looked like a missing piece of the architecture is the architecture.

The disclaimer, paid off

Back to §1's Snag, in the documentation's own words:

Kubernetes is not a mere orchestration system. In fact, it eliminates the need for orchestration. **The technical definition of orchestration is execution of a defined workflow: first do A, then B, then C.** In contrast, Kubernetes comprises a set of independent, composable control processes that continuously drive the current state towards the provided desired state. It shouldn't matter how you get from A to C. Centralized control is also not required. This results in a system that is easier to use and more powerful, robust, resilient, and extensible. [source: k8s-docs-overview-2026-08-23]

☆ Fixed Point

Orchestration, technically, is the execution of a defined workflow: first do A, then B, then C.

Kubernetes disclaims it. Kubernetes comprises a set of independent, composable control processes that continuously drive current state toward desired state — and it shouldn't matter how you get from A to C. Centralized control is not required. [source: k8s-docs-overview-2026-08-23]

Everyone in the industry, including this book in Chapter 1, calls Kubernetes an orchestrator. In the loose sense, meaning a thing that manages containers across machines, that's a perfectly good description, and it was the right altitude for an orientation chapter. In the precise sense the documentation is using, it's the one thing Kubernetes says it is not. The exam tests the precise sense. Now you have both, and you know which is which.

[cross-bearing: see Ch 1 @ Soundings A2 — where this book first called Kubernetes an orchestrator, in the industry's loose sense]

What this buys

A distinction you can't cash out is just trivia, so: why is this worth building a system around?

A system with no coordinator has no component whose failure leaves the others without instructions.

Not because everything keeps working — some things very much stop working — but because none of them were *taking* instructions in the first place. A kubelet's standing job is not an order it received; it is a comparison it holds, against the PodSpecs it already has [source: k8s-docs-cluster-architecture-2026-08-23]. There is no instruction queue to drain and no coordinator to wait on, because the work was never expressed that way.

A system that describes outcomes rather than sequences absorbs changes to the outcome without anyone rewriting the sequence.

Change the described state and every loop that cares about it recomputes its own gap. Nobody edits a workflow, because nobody wrote one. It shouldn't matter how you get from A to C, so when C changes, nothing about the path needs redesigning.

⚠ **One precision, because the heading overstates.** "Nobody is in charge" is a good chapter title and a bad thesis, and you should leave with the narrower version. The control plane *does* make global

decisions about the cluster [source: k8s-docs-cluster-architecture-2026-08-23]. The API server is a hub that everything depends on. etcd is a single store whose loss loses the cluster. There are absolutely critical components here, and Chapters 8, 12, and 13 all depend on you knowing which.

The accurate claim is narrower and better: **there is no component that executes a workflow, and no component that tells another component what to do.** The hub holds state and serves it. It does not direct. That's a claim about *coordination*, not about *availability*, and the difference between those two is worth more to you on exam day than the slogan.

What you're carrying forward

You now own a shape. Desired state, current state, an action closing the gap, repeating without end. You will meet it again in Chapter 6, where a ReplicaSet lets you watch it work in real time. In Chapter 15, where the desired state lives in a Git repository outside the cluster and a controller inside the cluster reaches out for it, which will look like a completely new technology until you recognize it. And in Chapter 17, where it turns out to be one of the things "cloud native" means.

That's why §6 was worth more than the eight names, and why the eight names were worth learning anyway: you can't reason about which loop stopped closing if you can't name the pieces.

⚓ **Safe Harbor reached — the architecture is behind you.**

[cross-bearing: see Ch 6 — controllers you configure yourself] [cross-bearing: see Ch 15 — the loop, pointed somewhere unexpected] [cross-bearing: see Ch 17 — the same pattern, named as a principle]

Exam Alert! 🚨

High-Priority Topics:

1. **The census, with optionality marked.** Eight components across two planes. kube-proxy and cloud-controller-manager are the two the documentation marks optional, and for different reasons. Pure recall, cheap to get right once you've noticed it.
2. **The control loop.** Desired state, current state, an action that closes the gap, never terminating. The highest-value idea in this chapter by a wide margin.
3. **kube-controller-manager: many logical controllers, one binary, one process.** One sentence in the documentation, and the whole distinction rides on it.
4. **Kubernetes is not a mere orchestration system.** The technical definition of orchestration is execution of a defined workflow: first A, then B, then C. Kubernetes disclaims it in favor of independent, composable control processes.
5. **What Kubernetes is not.** Not a traditional all-inclusive PaaS. Does not build your source. Does not ship middleware, databases, or caches. Does not mandate logging or configuration solutions.

Common Traps:

- **"kube-proxy is required on every node."** — Optional when a network plugin provides equivalent packet forwarding for Services [source: k8s-docs-cluster-architecture-2026-08-23].
 - **"Every cluster has a cloud-controller-manager."** — Absent on premises and in a learning environment on your own PC [source: k8s-docs-cluster-architecture-2026-08-23].
 - **"The controller-manager runs one process per controller."** — Logically separate, but compiled into a single binary and run in a single process [source: k8s-docs-cluster-architecture-2026-08-23].
 - **"Kubernetes is an orchestrator that runs A then B then C."** — That is the technical definition of orchestration, which Kubernetes explicitly disclaims [source: k8s-docs-overview-2026-08-23].
 - **"Kubernetes is a PaaS."** — Not a traditional, all-inclusive PaaS; the PaaS-like features it does offer are optional and pluggable [source: k8s-docs-overview-2026-08-23].
 - **"The scheduler places the Pod on the node."** — The scheduler *selects* a node and notifies the API server of that decision [source: k8s-docs-kube-scheduler-2026-08-23]. The kubelet on that node starts the containers. (How the scheduler chooses is Chapter 7's.)
 - **"The controller does the work."** — A controller usually asks the API server to change something. A different component performs the action.
-

Practice Questions

Q1. .. In the traditional deployment era, what was the only practical way to give an application resource isolation on a physical server?

A) Configure per-application resource quotas in the operating system B) Run each application under a separate OS user account with per-user limits C) Run each application on a different physical server D) Run each application inside its own hypervisor partition

Q2. .. [retrieval: ch2] Containers have relaxed isolation properties compared with virtual machines. Architecturally, what is it that containers share with each other that separate VMs do not?

A) The operating system B) The physical hardware underneath the machine C) Their filesystem and process space D) The network interface on the host

Q3. .. Which of the following does Kubernetes explicitly *not* do?

A) Restart containers that fail their health check B) Automatically mount a storage system of your choice C) Expose a container using a DNS name or its own IP address D) Deploy your source code and build your application

Q4. .. Kubernetes drew on which internal Google system, and what language is it written in?

A) MapReduce, a data-processing framework; written in C++ B) Borg, and its research successor Omega; written in Go C) Spanner, a globally distributed database; written in Java D) Omega alone, with no production predecessor; written in Rust

Q5. . Kubernetes was donated by Google to which foundation, and what does the name mean?

A) The Apache Software Foundation, as an incubating project; the name is an acronym B) The Cloud Native Computing Foundation, as one of several launch projects; the name is Latin for shepherd C) The Cloud Native Computing Foundation, as its first project; the name is Greek for helmsman or pilot D) The Open Container Initiative, alongside runC; the name is Greek for cluster

Q6. . The documentation says that in production environments the control plane usually runs across multiple computers. What does that buy?

A) Higher throughput, because each control-plane machine serves a different namespace B) Fault tolerance and high availability C) Reduced etcd storage, because cluster data is sharded across the machines D) A higher ceiling on the number of worker nodes a single cluster can hold

Q7. . Which component is described as the front end for the Kubernetes control plane?

A) kube-apiserver B) kube-controller-manager C) etcd D) kubelet

Q8. . What is etcd's role in a Kubernetes cluster?

A) It runs the containers that make up the control-plane components B) It caches container images so that Pods start faster on each node C) It routes API traffic across multiple kube-apiserver instances D) It is a consistent, highly-available key value store holding all cluster data

Q9. . kube-apiserver is described as designed to scale horizontally. What does that mean in practice?

A) It automatically grows its memory allocation as request load increases B) It spawns one worker process for each connected client C) You deploy more instances of it and balance traffic between them D) It shards cluster data across multiple independent etcd clusters

Q10. . Which of these is *not* listed among the factors kube-scheduler takes into account?

A) How recently each candidate node was rebooted B) Individual and collective resource requirements C) Affinity and anti-affinity specifications D) Data locality

Q11. . The kubelet is given a set of PodSpecs. What is its responsibility?

A) To choose which node each PodSpec should run on B) To store the PodSpecs durably so that they survive a restart C) To ensure the containers described in those PodSpecs are running and healthy D) To forward the PodSpecs to kube-proxy for network configuration

Q12. . You SSH to a worker node and start a container by hand using the container runtime directly. What does the kubelet do about it?

A) It restarts it if it exits, since it is running on a managed node B) Nothing — the kubelet doesn't manage containers that were not created by Kubernetes C) It reports it to the API server as an unmanaged Pod on that node D) It terminates it immediately as an unauthorized workload on a managed node

Q13. Which node component is *not* Kubernetes software — that is, a separate project that Kubernetes drives through a defined interface?

A) kubelet B) kube-proxy C) All three node components are Kubernetes project components D) The container runtime

Q14. A cluster has an object created and stored, but the component that would normally act on that kind of object is not installed. What happens?

A) The API server rejects the object at creation time as unserviceable B) kube-controller-manager automatically installs the missing component C) Nothing happens — the object exists as a description, but nothing acts on it D) The kubelet on each node takes over the missing component's responsibilities

Q15. A kubelet needs to know which Pods it is responsible for. Where does it get that, and what else does it talk to in order to get it?

A) From kube-scheduler, which pushes each assignment directly to the node it selected B) From the API server, which it watches — not from the scheduler or another kubelet C) From etcd, which it reads directly, bypassing the API server for lower latency D) From kube-proxy on the same node, which relays control-plane traffic inward

Q16. In Kubernetes, when a controller "wants" a Pod to exist, what does it typically do?

A) Sends messages to the API server that have useful side effects B) Creates the container directly through the node's container runtime C) Writes the Pod definition into etcd itself, bypassing the API server D) Instructs the target node's kubelet over a dedicated control channel

Q17. What are the two states a control loop compares?

A) Requested state and allocated state B) Declared state and validated state C) Desired state and current state D) Committed state and applied state

Q18. §1 lists self-healing among the things Kubernetes provides: restarting containers that fail, replacing containers, killing containers that don't respond to a health check. In §6's terms, what makes self-healing a standing condition rather than an action?

A) It is scheduled to run at a fixed interval, re-checking every container each time B) It is a loop comparing the state you asked for against the state that exists, acting whenever the two differ C) It runs once when a Pod is created and then hands responsibility to the kubelet D) It is triggered by an operator or an external monitoring system when a failure is detected

Q19. A controller needs to provision a new virtual machine from a cloud provider because the cluster needs more Nodes. Which control shape is this, and where does it get its desired state?

A) Direct control; it finds desired state from the API server, then talks to the external system B) Control via the API server; it finds desired state in the cloud provider's own API C) Direct control; it finds desired

state in a configuration file on the control plane D) Control via the API server; it reads desired state directly out of etcd

Q20. ... A system is described as "executing a defined workflow: first do A, then B, then C." In the documentation's terms, what is that system doing?

A) Reconciliation B) Declarative configuration C) Automatic bin packing D) Orchestration

Q21. ... The documentation says Kubernetes eliminates the need for orchestration. What does it say Kubernetes comprises instead?

A) A centralized coordinator that computes an optimal execution order for the cluster B) A workflow engine with pluggable, user-defined step definitions C) A message bus over which the components exchange instructions with each other D) A set of independent, composable control processes driving current state toward desired state

Q22. ... The Node controller notices when nodes go down and responds. Which component is it running inside, and what does "responds" mean in control-loop terms?

A) It runs inside kube-controller-manager; it compares expected node state against observed, and asks the API server to act B) It runs inside the kubelet on each node, and it restarts that node's containers when they stop C) It runs inside kube-scheduler, and it moves the failed node's Pods to other nodes itself D) It runs as its own separate control-plane component and instructs the affected node's kubelet directly

Q23. ... kube-proxy and cloud-controller-manager are both marked optional in the documentation. Is it for the same reason?

A) No — one because another component may already do its job; the other because there may be nothing for it to integrate with B) Yes — both are optional because a cluster below a certain size does not need either of them C) Yes — both are optional because an addon can be installed to replace either of them D) No — one is optional only in single-node clusters; the other only in clusters that define no Services

Q24. ... [retrieval: ch2] You need to change one line of configuration baked into a running container's filesystem. What is the correct process?

A) Edit the file inside the running container and restart its process B) Build a new image that includes the change, then recreate the container from it C) Mount a writable layer over the container and patch the file in place D) Ask the kubelet to hot-patch the container's filesystem on the node

Q25. ... [retrieval: ch2] What does a container image contain?

A) The application binary only — libraries come from the node at run time B) A full guest operating system, including its own kernel, like a VM disk image C) The application's source code, which the node

compiles when the container starts D) The code, its runtime, application and system libraries, and default settings

Answers with Explanations:

Q1 — C. There was no way to define resource boundaries for applications on a physical server; the solution was to run each application on a different physical server, which didn't scale and was expensive [source: k8s-docs-overview-2026-08-23].

- **A is wrong** — the documented problem statement is precisely that no such boundary mechanism existed.
- **B is wrong**, and instructively so. Separate user accounts were a real historical practice, and they do constrain *some* things, but not the ones that mattered. One application could still consume most of the machine's CPU or memory and starve the rest, which is exactly the failure the documentation describes.
- **D is wrong** — hypervisors are the *next* era. That's the whole point of the progression.

Q2 — A. [retrieval: ch2] Containers have relaxed isolation properties that let them share the operating system among the applications, which is why they are considered lightweight; a VM, by contrast, is a full machine running all the components, including its own operating system, on top of virtualized hardware [source: k8s-docs-overview-2026-08-23]. Ch 2 §1 holds both registers: "operating system" is the published wording and the one to recognize on an answer sheet; "kernel" is the mechanism underneath it.

- **B is wrong** — VMs share the physical hardware too. That's what virtualization *is*, and it's the previous era's change, not this one.
- **C is wrong** — and it's the reversal worth catching. A container has *its own* filesystem, and its own share of CPU, memory, and process space [source: k8s-docs-overview-2026-08-23]. Those are among the things that stay separate.
- **D is wrong** — networking is configurable in both models and isn't the architectural distinction the era transition turns on.

Q3 — D. Kubernetes does not deploy source code and does not build your application; CI/CD workflows are determined by organizational culture, preferences, and technical requirements [source: k8s-docs-overview-2026-08-23].

- **A, B, C are wrong** — all three appear in the published capability list as things Kubernetes does: self-healing, storage orchestration, and service discovery respectively [source: k8s-docs-overview-2026-08-23].

Q4 — B. Kubernetes drew on Google's internal container-orchestration experience, Borg and its research successor Omega, and was written in Go [source: k8s-history-ten-years-2026-08-23].

- **A is wrong** — MapReduce is a data-processing framework, an unrelated lineage, and the language is wrong.

- **C is wrong** — Spanner is a distributed database, and the language is wrong.
- **D is wrong** — Omega alone is incomplete. Borg came first and was the production system; Omega is described as its research successor. The language is also wrong.

Q5 — C. Kubernetes was donated by Google to the newly formed Cloud Native Computing Foundation as its first project, and the name comes from the Greek word for helmsman or pilot [source: k8s-history-ten-years-2026-08-23].

- **A is wrong** on both halves; the CNCF is part of the nonprofit Linux Foundation [source: cncf-who-we-are-2026-08-23], not Apache, and the name is not an acronym.
- **B is wrong** on the second half, which is the discriminating half. "First project" is right; the etymology is Greek, not Latin, and it is helmsman, not shepherd.
- **D is wrong** — the OCI is a separate Linux Foundation body governing container format and runtime specifications [source: oci-overview-2026-08-23], and it is where Docker donated runC, not where Google donated Kubernetes.

Q6 — B. In production environments the control plane usually runs across multiple computers and a cluster usually runs multiple nodes, providing fault tolerance and high availability [source: k8s-docs-cluster-architecture-2026-08-23].

- **A is wrong** — nothing partitions control-plane duties by namespace. kube-apiserver instances are interchangeable [source: k8s-docs-cluster-architecture-2026-08-23].
- **C is wrong** — running the control plane across several machines is not a data-sharding scheme, and the documentation describes no such sharding.
- **D is wrong** — a plausible-sounding benefit that the documentation does not claim here. The stated purpose is availability, not capacity.

Q7 — A. The API server is the front end for the Kubernetes control plane [source: k8s-docs-cluster-architecture-2026-08-23].

- **B is wrong** — the controller-manager runs controllers; it's a client of the front end, not the front end.
- **C is wrong** — etcd is the backing store *behind* the front end, not the front end itself. This is the most tempting wrong answer if you think of "front end" as "where the data is."
- **D is wrong** — the kubelet is a node agent.

Q8 — D. Verbatim role from the documentation: a consistent and highly-available key value store used as Kubernetes' backing store for all cluster data [source: k8s-docs-cluster-architecture-2026-08-23].

- **A is wrong** — etcd is a datastore, not a runtime.
- **B is wrong** — image caching happens on nodes, managed by the kubelet and runtime [source: k8s-docs-images-2026-08-23].
- **C is wrong** — that describes a load balancer sitting in front of multiple kube-apiserver instances, which is a real thing but not etcd.

Q9 — C. kube-apiserver is designed to scale horizontally: it scales by deploying more instances, and you can run several and balance traffic between them [source: k8s-docs-cluster-architecture-2026-08-23].

- **A is wrong** — that's vertical scaling, and it isn't what the documentation describes.
- **B is wrong** — a process-per-client model is not what "scales horizontally" means here; the unit of scaling is the whole instance.
- **D is wrong** — data sharding across etcd clusters is not part of the described design.

Q10 — A. The listed factors are resource requirements, hardware/software/policy constraints, affinity and anti-affinity specifications, data locality, inter-workload interference, and deadlines [source: k8s-docs-cluster-architecture-2026-08-23]. Time-since-reboot is plausible as a stability heuristic, and some systems do weigh something like it, but it is not among the factors Kubernetes lists.

- **B, C, D are wrong as answers** because all three *are* listed factors.

Q11 — C. The kubelet takes a set of PodSpecs provided through various mechanisms and ensures that the containers described in those PodSpecs are running and healthy [source: k8s-docs-cluster-architecture-2026-08-23].

- **A is wrong** — node selection belongs to kube-scheduler.
- **B is wrong** — durable storage is etcd's job, reached through the API server.
- **D is wrong** — kube-proxy maintains network rules; it isn't downstream of the kubelet's PodSpec handling.

Q12 — B. The kubelet doesn't manage containers which were not created by Kubernetes [source: k8s-docs-cluster-architecture-2026-08-23].

- **A is wrong** — "running on a managed node" isn't the criterion; *created by Kubernetes* is.
- **C is wrong** — the kubelet's stated responsibility is the containers described in its PodSpecs. A container it didn't create isn't one of those, so there is nothing for it to maintain or account for.
- **D is wrong** — the kubelet doesn't police the node. It doesn't manage what isn't its business, which is different from acting against it.

Q13 — D. Kubernetes supports container runtimes such as containerd, CRI-O, and any other implementation of the Kubernetes CRI [source: k8s-docs-cluster-architecture-2026-08-23]. These are separate projects driven through the Container Runtime Interface [source: k8s-docs-extending-kubernetes-2026-08-23].

- **A and B are wrong** — kubelet and kube-proxy are Kubernetes project components.
- **C is wrong** — the whole design point of CRI is that the runtime is pluggable and external.

Q14 — C. The object exists as a stored description; something has to be watching for it and willing to act, and if that component is absent, nothing acts. This is the absent-component pattern from §4.

- **A is wrong** — the API server validates the object's structure, not whether a consumer for it exists.

- **B is wrong** — nothing in Kubernetes self-installs missing components in response to an object appearing.
- **D is wrong** — the kubelet has exactly one job and doesn't inherit others.

Q15 — B. All API usage from nodes and the Pods they run terminates at the API server, and none of the other control-plane components are designed to expose remote services [source: k8s-docs-control-plane-node-communication-2026-08-24]. The kubelet watches the API server and acts on what it finds.

- **A is wrong** — the scheduler notifies the API server of its decision [source: k8s-docs-kube-scheduler-2026-08-23]. It does not push anything to a node.
- **C is wrong** — the API server is the component that reads and writes etcd; access to etcd is root-equivalent and is meant to be confined to it [source: k8s-docs-etcd-access-control-2026-08-24].
- **D is wrong** — kube-proxy maintains network rules for Services. It is not a control-plane relay.

Q16 — A. A controller might carry the action out itself, but more commonly a controller sends messages to the API server that have useful side effects [source: k8s-docs-controllers-2026-08-23].

- **B is wrong** — the Job controller explicitly does not run any Pods or containers itself [source: k8s-docs-controllers-2026-08-23].
- **C is wrong** — controllers go through the API server, not around it.
- **D is wrong** — there is no such dedicated channel; the kubelet watches the API server.

Q17 — C. Desired state and current state: the thermostat's set temperature and the room's actual temperature, generalized [source: k8s-docs-controllers-2026-08-23].

- **A, B, D are wrong** — plausible-sounding pairs that appear nowhere in the documentation. If any of them felt right, that's worth noticing: real terminology and invented terminology read identically until you've anchored the real pair.

Q18 — B. Controllers are control loops that watch cluster state and act to move current state closer to desired state, continuously [source: k8s-docs-controllers-2026-08-23]. Self-healing — restarting failed containers, replacing containers, killing ones that fail a health check [source: k8s-docs-overview-2026-08-23] — is that comparison running, not a procedure someone invoked.

- **A is wrong** — a clock-driven sweep is a scheduled task. A control loop is driven by the gap between two states, not by an interval.
- **C is wrong** — "runs once and hands off" is exactly the model §6 replaces. Nothing about the loop terminates when a Pod comes up.
- **D is wrong** — no external trigger is involved. The whole point is that nobody has to notice and initiate.

Q19 — A. Controllers that interact with external state find their desired state from the API server, then communicate directly with an external system to bring the current state closer in line, and provisioning Nodes is the documentation's own example [source: k8s-docs-controllers-2026-08-23].

- **B is wrong** — desired state always comes from the API server, even for direct-control controllers. That's what makes them Kubernetes controllers.
- **C is wrong** — no local config file holds desired state; that would break the whole model.
- **D is wrong** on both halves: this is direct control, and components reach etcd through the API server rather than around it.

Q20 — D. The technical definition of orchestration is execution of a defined workflow: first do A, then B, then C [source: k8s-docs-overview-2026-08-23].

- **A is wrong** — reconciliation is the opposite shape: continuous comparison, no defined sequence.
- **B is wrong** — declarative configuration describes an end state without a path.
- **C is wrong** — bin packing is a placement optimization, not a workflow model.

Q21 — D. Kubernetes comprises a set of independent, composable control processes that continuously drive the current state towards the provided desired state; it shouldn't matter how you get from A to C, and centralized control is not required [source: k8s-docs-overview-2026-08-23].

- **A is wrong** — and it names the exact thing the documentation disclaims: centralized control is explicitly not required.
- **B is wrong** — a workflow engine with pluggable steps is still a workflow engine. Making the steps configurable doesn't stop the system from executing a defined sequence.
- **C is wrong**, and it is the mental model §5 spent a whole section dismantling. Components do not exchange instructions; they observe shared state. None of the other control-plane components are even designed to expose remote services [source: k8s-docs-control-plane-node-communication-2026-08-24].

Q22 — A. The Node controller is one of the controllers run by kube-controller-manager — noticing and responding when nodes go down — and every controller in that binary runs in the same single process [source: k8s-docs-cluster-architecture-2026-08-23]. "Responds" means what it means for every controller: compare, then ask the API server to change something [source: k8s-docs-controllers-2026-08-23].

- **B is wrong** — the kubelet manages containers on its own node. It is not where the Node controller lives, and a node that has gone down is not running a kubelet you can rely on.
- **C is wrong** — kube-scheduler assigns unscheduled Pods to nodes; it doesn't monitor node health, and it doesn't move Pods itself.
- **D is wrong** — the Node controller is not a separate component, and nothing on the control plane instructs a kubelet directly.

Q23 — A. kube-proxy is unnecessary when a network plugin implements equivalent packet forwarding for Services; cloud-controller-manager is absent when you run on your own premises or on your own PC, because there is no cloud provider to link to [source: k8s-docs-cluster-architecture-2026-08-23].

- **B is wrong** — cluster size is irrelevant to both.
- **C is wrong** — addons don't substitute for either.
- **D is wrong** — both halves invent conditions the documentation doesn't state.

Q24 — B. [retrieval: ch2] Containers are intended to be stateless and immutable; you should not change the code of a container that is already running. The correct process is to build a new image that includes the change, then recreate the container from the updated image [source: k8s-docs-containers-2026-08-23].

- **A is wrong** — editing in place is exactly the practice immutability rules out, and the change vanishes on the next restart.
- **C is wrong**, but not because the mechanism doesn't exist. A container's writable layer is real. It's wrong because anything written there belongs to *that container instance* and is gone the moment the container is replaced, which is a thing Kubernetes does constantly. That's the practical teeth of immutability: the change lives in the image, or it doesn't survive.
- **D is wrong** — there is no such kubelet capability. The kubelet starts and stops containers through the CRI [source: k8s-docs-extending-kubernetes-2026-08-23]; it doesn't reach inside their filesystems. Note the connection to this chapter, though: the control loop's response to a changed desired state is the same instinct — replace, don't mutate.

Q25 — D. [retrieval: ch2] A container image is a ready-to-run software package containing everything needed to run an application: the code and any runtime it requires, application and system libraries, and default values for any essential settings [source: k8s-docs-containers-2026-08-23].

- **A is wrong** — the standardization from having dependencies included is precisely what makes a container repeatable across environments [source: k8s-docs-containers-2026-08-23]. If libraries came from the node, that guarantee would be gone.
 - **B is wrong** — and it's the VM confusion again. A full machine with its own operating system is the virtualized era's unit, not the container era's [source: k8s-docs-overview-2026-08-23].
 - **C is wrong** — an image is ready to run. Nothing is compiled at container start.
-

Chapter Summary

Concept	Remember This
Three deployment eras	Traditional (one app per server, nothing shared) → virtualized (own OS per VM, hardware shared) → container (operating system shared, lightweight, portable)
What Kubernetes is not	Not an all-inclusive PaaS. Doesn't build your source. No built-in databases, middleware, or caches. Doesn't mandate logging or config solutions. Not a mere orchestration system.
Origin	Descended from Google's Borg (and its research successor Omega). Written in Go. First commit 2014-06-06. v1.0 July 2015. CNCF's first donated project. Name = Greek for helmsman.
Cluster shape	A control plane plus a set of worker nodes. At least one worker node is needed to run Pods. In production the control plane runs across multiple computers, for fault tolerance and high availability.
kube-apiserver	Front end for the control plane. Exposes the Kubernetes HTTP API. Scales horizontally by adding instances.
etcd	Consistent, highly-available key value store. Backing store for <i>all</i> cluster data. Access to it is root-equivalent. Have a backup plan.
kube-scheduler	Watches for Pods with no assigned node and selects one, then notifies the API server (binding). Selects and records — does not start anything.
kube-controller-manager	Runs controller processes. Logically separate, but one binary, one process .
cloud-controller-manager	Optional. Links the cluster to a cloud provider's API. Absent on premises and on your laptop.
kubelet	On every node. Ensures the containers described in its PodSpecs are running and healthy. Doesn't manage containers it didn't create.
kube-proxy	Optional. On every node when present. Maintains node network rules implementing part of the Service concept. Unnecessary if a network plugin does the same job.
Container runtime	On every node. Runs the containers. Not Kubernetes software — containerd, CRI-O, or any CRI implementation.
Addons	Extend the cluster rather than constituting it. DNS, Dashboard, resource monitoring, cluster-level logging.
Absent-component pattern	An object without its component does nothing. The description exists; nothing acts on it.
The hub	State moves through the API server, and the API server is what reads and writes etcd. Coordination is watching, not telling. (Outbound API-server→kubelet paths exist for logs, attach, and port-forward; they carry sessions, not instructions.)

Concept	Remember This
☆ Control loop	Desired state, current state, an action that closes the gap — repeating, non-terminating.
Controller behavior	Usually asks the API server to change something; other components act on that. Direct control (reaching outside the cluster) is the less common shape.
Never stable	A cluster may never reach a stable state, and that's fine — what matters is that the controllers are running and able to make useful changes.
☆ Orchestration	Technically: execution of a defined workflow, A then B then C. Kubernetes disclaims it in favor of independent, composable control processes. Centralized control is not required.

The Voyage Ahead

You've met the crew and you know how they coordinate. What you don't yet have is the thing they're all watching.

Every component in this chapter reads from and writes to the same shared state, and that state is made of *objects*, the descriptions you submit and the system maintains. §6 kept saying "the field that says what should be true" and cross-bearing forward, because that field has a name, and a counterpart that reports what actually is true, and a set of conventions for how you write both. Chapter 4 gives them to you.

That's when the control loop stops being an architecture diagram and becomes something you operate. You'll write a description, submit it, and watch a loop you now understand go to work on it. The reason that will feel unremarkable rather than magical is that you did the work in this chapter first.

Now that you look at a Kubernetes cluster and see loops rather than services, the rest of the book gets considerably easier.

"The vessel arrives on course not because someone executed a plan, but because a few dozen small corrections were made continuously by people who were each watching one thing."

Chapter 4: Records of Intent

"You don't give Kubernetes orders. You file a declaration."

Domain: Kubernetes Fundamentals — Kubernetes Core Concepts | **Estimated chapter weight:** ~6%
Complexity: Mixed | **Novelty:** Paradigm-shifting | **Prerequisites:** Chapter 3

The published domain weight is Kubernetes Fundamentals at 44% of the exam [source: [cncf-kcna-certification-page-2026-08-23](#)]. CNCF does not publish weights for individual competencies inside a domain [source: [cncf-kcna-curriculum-pdf-2026-08-23](#)]; the ~6% above is this book's estimate, and the front matter explains how every such estimate was derived.

Attention Budget

Total time: ~115 minutes | **Recommended:** split across two sessions (\$1–\$3, then \$4–\$6)

Section	Time	Attention Cost	Best Time to Study
\$1 You File a Declaration	12 min	Low	Anytime
\$2 The Anatomy of a Record	20 min	Medium	When alert
🕒 Taking Your Bearings #1	8 min	Medium	After a brief pause
\$3 Where a Name Lives	15 min	Medium	When alert
\$4 Configuration Kept Outside the Image	25 min	High	Peak attention
🕒 Taking Your Bearings #2	7 min	Medium	After a brief pause
\$5 The Universal Join	20 min	Medium	When alert
🕒 Taking Your Bearings #3	7 min	Medium	After a brief pause
\$6 A Declaration, Not an Order	6 min	Low	Anytime
Practice Questions	28 min	High	Peak attention

Attention Cost Key:

- **Low:** concrete, familiar concepts. Study anytime.
- **Medium:** new concepts requiring focus. Study when alert.
- **High:** abstract or dense material, or a misconception that needs dismantling. Study at peak attention.

If you only have 15 minutes: read §2 and take Bearings #1. The two field names in that section are the vocabulary every remaining chapter of this book assumes you have.

"It shouldn't matter how you get from A to C." — Kubernetes documentation, Overview [source: k8s-docs-overview-2026-08-23]

🎧 Soundings

Eight questions before you begin. Your score picks the reading strategy and nothing else; no result here is a bad one. Nothing below requires material from this chapter. It is all either general IT knowledge or something Chapters 2 and 3 already gave you.

1. Some tools ask you to describe what your infrastructure *should* look like; others ask you to write a script that performs the steps to build it. What is that distinction usually called, and what does the first approach buy you that the second doesn't?
2. Many configuration file formats put a version or schema identifier at the very top of the file. What problem is that field solving?
3. A control loop has two states in it, and a third thing that closes the gap between them. Name all three.
4. You submit a description of something you want to exist in a Kubernetes cluster. Which component receives that submission, and what actually stores it?
5. The same container image runs in development and in production, and needs a different database address in each. Where does the difference come from?
6. Is base64 encoding a form of encryption? If not, what is it for?
7. In a system you already know well — SQL, cloud resource tags, a ticketing system, a mail client — how do you ask for "all the things that have this attribute"?
8. Two teams share one system, and both want to call something database. What would you expect a well-designed multi-tenant system to offer them?

Answers + reading strategy

1. **Declarative versus imperative.** Declarative buys you idempotency (applying the same description twice is harmless), the ability to state an outcome without knowing the current state, and a description that outlives the session that produced it.

2. **Schema evolution.** The version field lets the consumer know which set of rules to parse the document under, so old documents keep working when the format changes.
3. **Desired state, current state, and the controller** (or loop, or reconciler) that observes the difference and acts to close it.
4. **The kube-apiserver receives it; etcd stores it.** Every write to cluster state goes through the API server, and etcd is the backing store.
5. **From outside the image:** an environment variable, or a file placed into the container when it launches. If the difference lived *inside* the image you'd need two images, which defeats the point of having one.
6. **No.** Base64 is a transport encoding: a way to represent arbitrary binary data using a restricted set of text characters. It is trivially reversible by anyone, requires no key, and provides no confidentiality whatsoever. Hold onto that answer; §4 will want it.
7. **A query with a predicate over an attribute:** `WHERE environment = 'production'`, a tag filter, a saved search. The general shape is that things carry attributes, and you name a subset by describing the attributes rather than listing the members.
8. **A scope for names:** a namespace, a tenant, a project, a schema. Something that makes "database" mean different things in different contexts without either team having to rename anything.

If you got 6+ right: skim this chapter. Focus on the ☆ Fixed Points and the ⚠ Navigational Hazards callouts, and take all three Taking Your Bearings checkpoints at full effort. You have the priors; this chapter is giving them Kubernetes-specific names and one genuinely counterintuitive detail in §4.

If you got 3–5 right: read at normal pace. The material is well within reach and this chapter is calibrated for you.

If you got 0–2 right: read carefully, and take the sections in order: §2 depends on §1, and §5 depends on §3. **If questions 3 and 4 were among your misses, re-read Chapter 3 §5–§6 before you start §2.** Those two are the load-bearing prerequisites for this entire chapter, and §2 in particular assumes them.

Why This Chapter Matters

Here is something unusual about the chapter you are starting: **nothing in it is a verb.**

You have spent three chapters learning about a system that runs software. You will spend this one writing files that describe what *should be true*, then handing them to something that never receives an instruction. You will not, anywhere in the pages ahead, tell Kubernetes to do anything. You will state what you want to exist. Something else does the doing, and you never address it directly. If you arrived from imperative operations tooling, from a world of scripts and runbooks and commands that execute

and finish, this is genuinely strange for about a chapter. Better to name the strangeness than smooth it over, so we name it now, live with it through §2 to §5, and resolve it in §6.

Chapter 3 gave you a system to look at. Chapter 4 gives you something to write, and that is the moment the control loop stops being a diagram on a page. It is also, in practical terms, the moment you become able to read Kubernetes configuration you have never seen before. Hand someone who has finished this chapter a manifest for a resource type they have never heard of, one that did not exist when this book was printed, and they can still parse it, because the four top-level fields are the same four fields every single time. That transferability is the honest reason this material matters more than its estimated six points suggest.

The stakes here are structural, and they only need saying once. Every chapter after this one reads *through* this one. Chapter 5's Pod is an object with a spec. Chapter 6's controllers select their workloads by label. Chapter 9's Service selects its backends by label. Chapter 12's role-based access control is, underneath the terminology, a namespaced-versus-cluster-scoped problem wearing a costume. A reader who leaves this chapter able to recite the four required fields but unable to explain what *status* is *for* will re-learn this chapter four more times, in worse conditions, under exam pressure. So take §2 slowly. It is the cheapest hour in the book.

Dead Reckoning: A Kubernetes object is a persistent record stored by the cluster that describes something you want to exist. You write it as a file, usually YAML, with four top-level fields, and submit it with `kubectl apply -f`. The cluster stores it and then works continuously to make reality match it. This chapter covers the shape of that record, where its name lives, how you group records into sets, and two specific record types that hold configuration.

What You'll Learn

By the end of this chapter, you'll be able to:

- **Read** any Kubernetes manifest you have never seen before, name its four required fields, and say what each one is doing.
- **Distinguish** `spec` from `status` (who writes each, which one you set, which one the system maintains) and connect both to the control loop you already know.
- **Name** the three techniques `kubectl` offers for managing an object, and say why an object should be managed with only one of them.
- **Predict** whether a given resource is namespaced or cluster-scoped, and explain why that answer determines who can be given permission over it.
- **Select** a set of objects by label using both equality-based and set-based syntax, and explain why an annotation cannot be selected on.
- **Choose** between a ConfigMap and a Secret for a piece of configuration, and state precisely what protection the Secret does and does not add.

You'll also stop reading YAML as an opaque wall of configuration and start reading it as four fields, one of which is the interesting one.

§1 — You File a Declaration

Start where the documentation starts.

Kubernetes objects are **persistent entities** in the Kubernetes system, and the cluster uses them to represent its own state. Specifically, they describe three things: what containerized applications are running and on which nodes; the resources available to those applications; and the policies around how those applications behave, meaning restart policies, upgrades, and fault-tolerance [source: k8s-docs-objects-2026-08-23].

Then the phrase this chapter is named for:

A Kubernetes object is a **"record of intent"** — once you create the object, the Kubernetes system will constantly work to ensure that the object exists [source: k8s-docs-objects-2026-08-23].

Read that second clause again, because the whole chapter follows from it. *Constantly*. Not once. Creating an object is not a request that gets serviced and then completed. It is a statement that gets **maintained**. When you create an object, you are effectively telling the Kubernetes system what you want your cluster's workload to look like, and that is your cluster's desired state [source: k8s-docs-objects-2026-08-23].

The mechanics, at the coarsest altitude: to work with Kubernetes objects, whether to create, modify, or delete them, you use the Kubernetes API. When you use the `kubectl` command-line interface, the CLI makes the necessary API calls for you [source: k8s-docs-objects-2026-08-23]. That is the tool you will use for the rest of this book, and notice where it sits: not beside the cluster, but in front of the one door Chapter 3 showed you [*cross-bearing: see Ch 3 §5 — the API server as the only way in*]. Your file does not reach the cluster. It reaches the API server, which reaches the cluster.

Which brings us to the distinction Chapter 1 promised this section would name [*cross-bearing: see Ch 1 @ Soundings A5 — the distinction this section names*].

An **imperative** interface is one where you instruct the server what to do. A **declarative** interface is one where you declare the desired state of your resource, and a controller keeps the current state of objects in sync with your declared desired state [source: k8s-docs-custom-resources-2026-08-23]. Kubernetes' object model is the second kind. The documentation is blunt about how far this goes:

Kubernetes is not a mere orchestration system. In fact, it eliminates the need for orchestration. The technical definition of orchestration is execution of a defined workflow: first do A, then B, then C. In contrast, Kubernetes comprises a set of independent, composable control processes that continuously drive the current state towards the provided desired state. It shouldn't matter how you get from A to C [source: k8s-docs-overview-2026-08-23].

Three ways to manage an object

That distinction is not just philosophical. It shows up as three concrete techniques, and the documentation names all three [source: k8s-docs-object-management-2026-08-24]:

Management technique	Operates on	Recommended environment	Supported writers
Imperative commands	Live objects	Development projects	1+
Imperative object configuration	Individual files	Production projects	1
Declarative object configuration	Directories of files	Production projects	1+

Imperative commands. A user operates directly on live objects in a cluster, providing operations to `kubectl` as arguments or flags. `kubectl create deployment nginx --image nginx` runs an instance of the `nginx` container by creating a `Deployment` object [source: k8s-docs-object-management-2026-08-24]. Notice that the documentation is not sniffy about this. It calls imperative commands *the recommended way to get started or to run a one-off task in a cluster*, and it names the cost precisely: because the technique operates directly on live objects, **it provides no history of previous configurations** [source: k8s-docs-object-management-2026-08-24].

Imperative object configuration. The command specifies the operation (`create`, `replace`, and so on), optional flags, and at least one file name. The file must contain a full definition of the object in `YAML` or `JSON` [source: k8s-docs-object-management-2026-08-24]. You said what to do *and* handed over the document.

Declarative object configuration. The user operates on configuration files stored locally, but **does not define the operations to be taken on them**. `Create`, `update`, and `delete` operations are automatically detected per-object by `kubectl` [source: k8s-docs-object-management-2026-08-24]. This is the technique that gives you `kubectl apply -f configs/` over a whole directory, and it is the one the rest of this chapter assumes.

And one warning, stated by the documentation in its own alarmed voice:

A Kubernetes object should be managed using only one technique. Mixing and matching techniques for the same object results in undefined behavior [source: k8s-docs-object-management-2026-08-24].

The verb you will use to submit a record is `kubectl apply`, which applies a configuration change to a resource from a file or standard input [source: k8s-docs-kubectl-overview-2026-08-23]. That sentence is the entire treatment `apply` gets in this chapter. You cannot teach a record of intent without showing how a record gets filed, so `apply` appears here; the full command surface, its verbs, its flags, how it authenticates, belongs to Chapter 8 [cross-bearing: see *Ch 8 — kubectl, in full*].

🔍 **Closer Look:** The three techniques differ in one thing that matters more than syntax: **where the record of what you wanted lives**. With an imperative command it lives only in the cluster, which is why the documentation says the technique gives you no history. With object configuration it lives in a file, which can go into source control, be reviewed before it is pushed, and serve as a template for the next object [source: k8s-docs-object-management-2026-08-24]. The trade-off is stated just as plainly in the other direction: object configuration requires a basic understanding of the object schema and the additional step of writing a YAML file. Nothing here is free, and the documentation does not pretend otherwise.

One precision note before we go on, because the chapter subtitle is a slogan and slogans overclaim. Kubernetes accepts plenty of imperative instruction. `kubectl delete` deletes. `kubectl scale` updates the size of a workload. `kubectl exec` executes a command inside a running container and has no declarative reading at all [source: k8s-docs-kubectl-overview-2026-08-23]. The accurate claim is narrower and more interesting than the slogan: **the objects are declarations, and the imperative commands work by changing declarations**. We will make that precise in §6. For now, hold the narrow version.

Extended Analogy: Think about the papers a vessel files before departure.

A declaration lodged with the harbormaster is not an instruction to anyone. It does not tell the pilot to steer, the loading crew to load, or the watch officer to stand a watch. It is a statement of fact and intent: this is what is aboard, this is how deep she sits, this is where we are bound, these are the hazards we carry. It gets signed once and then it sits there.

But everything that happens afterward is checked against it. The pilot consults it before boarding. The loading officer consults it while loading, and again when the manifest is queried three days later. The port authority consults it during an inspection nobody scheduled. If what's in the hold stops matching what's on the paper, the paper does not change. The hold does. And critically, none of those people received an order from you. They read a record, compared it to what they observed, and acted on the difference.

That is the entire relationship between you and a Kubernetes cluster. You file the papers. Independent parties consult them, indefinitely, and correct reality toward them. The record outranks the moment it was written, and it outlasts the session that produced it.

You now know what an object *is*. §2 tells you what is inside one.

§2 — .1 The Anatomy of a Record

Chapter 3 owed you something. This is where it comes due.

That chapter taught the control loop in plain language, *those objects carry a field that represents the desired state*, and deliberately withheld the field's name, closing with a promise that Chapter 4 would

supply it [*cross-bearing: see Ch 3 §6 — the field that holds desired state, and its status counterpart*]. Here it is. But first, the shape of the document it lives in.

The four fields

When you create an object in Kubernetes, you must provide the object spec that describes its desired state, plus some basic information about the object, such as a name. Most often you provide that information to `kubectl` in a file known as a **manifest**, and by convention, manifests are YAML [source: k8s-docs-objects-2026-08-23].

In the manifest file for the object you want to create, you set values for four fields [source: k8s-docs-objects-2026-08-23]:

- **apiVersion:** which version of the Kubernetes API you're using to create this object
- **kind:** what kind of object you want to create
- **metadata:** data that helps uniquely identify the object, including a name string, a UID, and an optional namespace
- **spec:** what state you desire for the object

Then you apply it: `kubectl apply -f <manifest>` [source: k8s-docs-objects-2026-08-23].

That is the complete structural vocabulary. Not a starting subset that gets extended for advanced resources. The complete thing. A Pod manifest has those four fields — a Pod, for now, being the unit Kubernetes schedules and runs; it is Chapter 5's whole subject. A NetworkPolicy manifest has those four fields. A custom resource for a database operator that some vendor shipped last week has those four fields. This is why the transferability claim from *Why This Chapter Matters* is not marketing: the structure does not vary by resource type, and it does not expire. [*cross-bearing: see Ch 6 — custom resources and operators*]

Dead Reckoning: Four top-level fields to know — required on almost every object; the documentation's own hedge is "*for objects that have a spec*" [source: k8s-docs-objects-2026-08-23].

apiVersion: which version of the Kubernetes API you're using to create this object. Selects the schema. **kind:** a string naming the resource type. Selects what is being created. **metadata:** an object holding identity. **name** (you supply), **uid** (the system supplies), and an optional namespace. Selects which specific instance. **spec:** an object holding the desired state. Its internal shape is defined by **kind**.

Submit with `kubectl apply -f <file>`. Nothing else is structurally required — though a few configuration-holding types (ConfigMap, Secret) carry their payload in **data** instead of a **spec** [source: k8s-api-ref-secret-v1-2026-08-24]. §4 cashes that asterisk.

🧠 **Mnemonic:** The four fields answer four questions, always in the same order. **Which API? Which kind of thing? Which one, specifically? What should it look like?** If you can hold those four questions, you can read any manifest, because reading a manifest is just answering them in order.

The two halves of identity

`metadata` carries two different kinds of identity, and the difference is worth ten seconds.

A **name** is a client-provided string that refers to an object in a resource URL. Only one object of a given kind can have a given name at a time, but if you delete the object, you can make a new one with the same name [source: k8s-docs-names-and-uids-2026-08-24]. Names are reusable. That is the point of them.

A **UID** is a system-generated string, and it is not reusable at all. Every object created over the whole lifetime of a Kubernetes cluster has a distinct UID, and the documentation states its purpose exactly: it is **intended to distinguish between historical occurrences of similar entities** [source: k8s-docs-names-and-uids-2026-08-24]. That phrase is doing quiet work. It means the cluster can tell the difference between the `web-1` you deleted this morning and the `web-1` you created this afternoon, even though the name is identical. Hold onto it; it comes back on the last page of this chapter.

`metadata` holds more than name and UID. Two of its other residents, **labels** and **annotations**, are load-bearing enough to get their own section, so they are named here and deferred [*cross-bearing: see Ch 4 §5 — labels, selectors, and annotations*]. Everything else that can appear in `metadata` sits above associate tier.

Here is what one object looks like, laid out. Note the boundary running through the middle of it, because that boundary is the second half of this section.

YOU AUTHOR THIS

<code>apiVersion</code> ...	which API version
<code>kind</code> ...	what kind of object
<code>metadata:</code>	which one, specifically
<code>name</code> ...	
<code>uid</code> ...	
<code>namespace</code> ...	(optional)
<code>spec:</code>	what it should look like
...	(more fields)

AUTHORSHIP BOUNDARY

THE SYSTEM AUTHORS THIS

<code>status:</code>	what is actually true
...	(more fields)

Figure: the anatomy of a Kubernetes object. The line through the middle is not decoration; it is the distinction the rest of this section is about. Four fields above it are yours. The field below it is not.

The two fields that matter most

Almost every Kubernetes object includes two nested object fields that govern the object's configuration: the object **spec** and the object **status**. They are not symmetrical, and the asymmetry is the point.

For objects that have a spec, you have to set this when you create the object, providing a description of the characteristics you want the resource to have: its desired state. The status describes the current state of the object, **supplied and updated by the Kubernetes system and its components**. The Kubernetes control plane continually and actively manages every object's actual state to match the desired state you supplied [source: k8s-docs-objects-2026-08-23].

A ship's log carries two kinds of entry, and they are never made by the same hand: what the master intends, and what the watch actually observed. Both live in the same book. Neither one is allowed to be written in the other's column.

The documentation's own worked example is the cleanest illustration available, and you can already read it. A Deployment is an object that can represent an application running on your cluster. When you create the Deployment, you might set its spec to specify that you want three replicas of the application running. The Kubernetes system reads the spec and starts three instances, updating the status to match. If one of those instances fails (*a status change*), the system responds to the difference between spec and status by making a correction: starting a replacement instance [source: k8s-docs-objects-2026-08-23].

That is a control loop, described in field names. Chapter 3 told you a controller compares desired state against current state and acts on the difference [source: k8s-docs-controllers-2026-08-23]. Now the two states have names. Desired state is `spec`; you wrote it. Current state is `status`; the system wrote it. The difference between them is the thing every controller in the cluster exists to close.

(Deployment is Chapter 6's resource and the instances it manages are Chapter 5's. Take the example for its shape — a number in a spec, a reality in a status, a correction — and meet the resource properly later. [cross-bearing: see Ch 6 — Deployments and ReplicaSets])

☆ **Fixed Point:** `spec` is what you want. `status` is what is. You write `spec`. The system writes `status`. Every controller in the cluster exists to close the distance between them.

That is the most reused sentence in this book. Chapter 5 reads it against a Pod's phase, Chapter 6 reads it against a replica count, Chapter 13 reads it as the first thing you check when something is wrong. Learn it in that exact shape.

📌 **Snag:** `status` is not something you write. The documentation's word on authorship is plain: `status` is supplied and updated by the system and its components [source: [k8s-docs-objects-2026-08-23](#)] — so anything you type into a `status` block is not what the object will report. It is a report, not a request. Practitioners arriving from imperative tooling try this at least once, usually at two in the morning, while trying to make a stuck object look healthy.

What actually happens when you apply

Put the pieces together. You have a file. You have a verb. You have a component that receives it, a store that keeps it, and a loop that reads it. Here is the round trip.

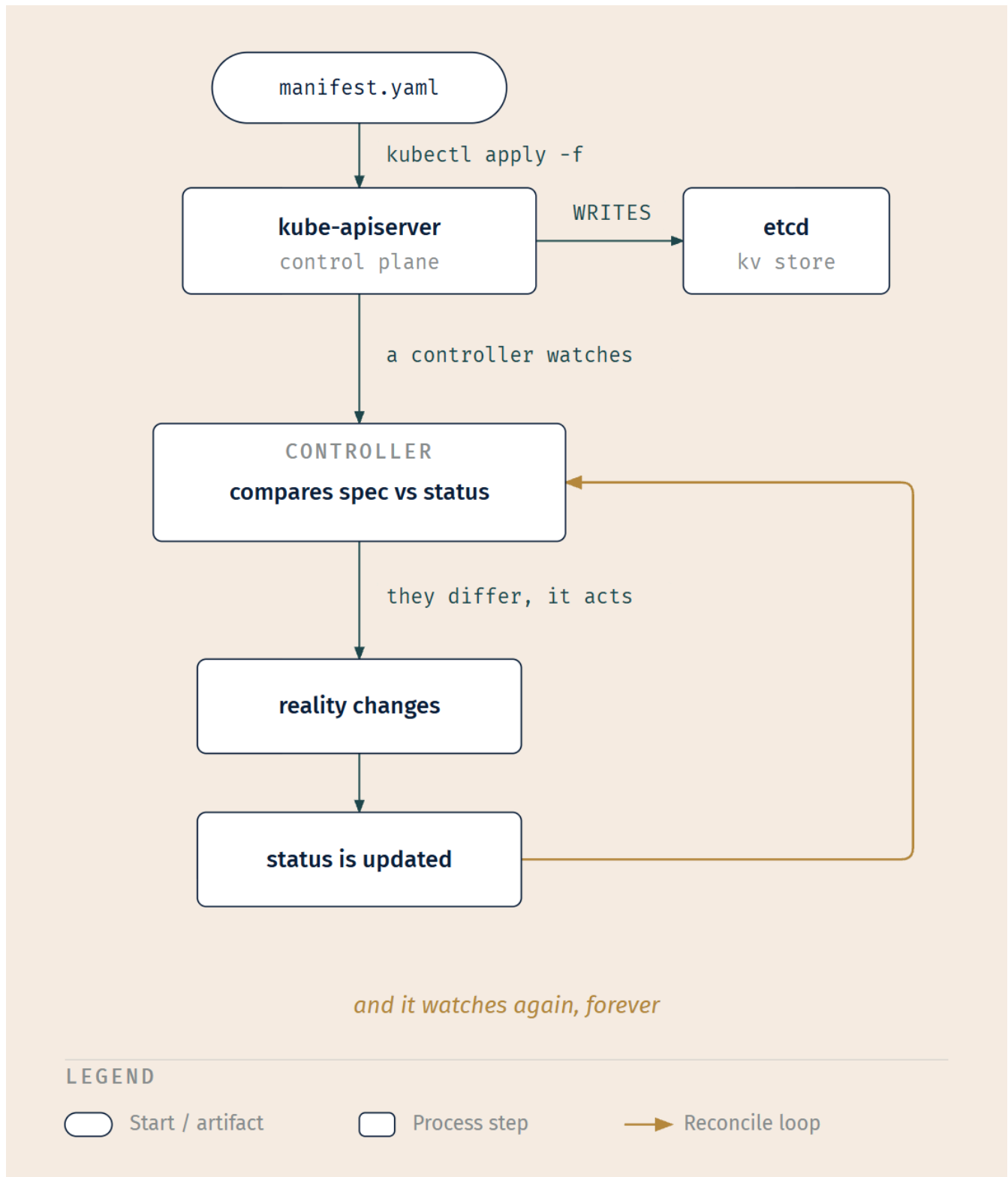


Figure: one declaration, in sequence, over time. The loop at the bottom never terminates. That is not a diagram convention; it is the actual behavior. Compare this to Chapter 3's request-path figure, which showed which components talk to the API server; this one shows what happens to a single object after they do.

Nothing in that path is a command. You submitted a description; the API server stored it; a controller noticed it; the controller acted; the result was reported back into the same object you wrote. The object is the medium and the message both.

🔦 **Closer Look:** Notice where the controller gets its information. It does not receive a message from you, and it does not receive one from `kubectl`. It watches the API server. This is why the same declaration can be applied by a person at a terminal, a CI pipeline, or a tool that reconciles from a Git repository, and the cluster behaves identically in all three cases. The cluster cannot tell the difference and does not care. That indifference is a design choice with consequences we will pick up much later [*cross-bearing: see Ch 15 — the same declaration, kept in a repository*].

One more command earns a mention here, because it converts a memorization problem into a lookup one: `kubectl explain` gets documentation for resources [source: [k8s-docs-kubectl-overview-2026-08-23](#)]. The four top-level fields you now know are the map. `kubectl explain` is how you read the territory inside spec for any resource type you have not memorized, which at associate tier is most of them, and that is fine.

🧭 Taking Your Bearings #1: The Declarative Frame and Object Anatomy

Five questions. The last one is the hardest on purpose.

1. . You are handed a manifest for a resource type this book never covers — `kind: CronTab`, from some vendor's operator. Name the four top-level fields you expect to find, and say what each one is doing.
2. . Of the two nested fields that govern an object's configuration, which do you write, and which does the system write?

A) You write `spec`; the system writes `status` B) You write `status`; the system writes `spec` C) You write both; the system reads both D) The system writes both; you read both
3. . You apply a manifest. The object appears. You read it back and `status` does not match `spec`. Is this a failure?

A) Yes — a mismatch means the apply was rejected B) Yes — `status` is the authoritative record, so a mismatch means your manifest was invalid C) Not necessarily — a gap between `spec` and `status` is the normal condition while the system reconciles D) Not necessarily — but only for objects that have no controller watching them
4. . What does `apiVersion` select, and why does it live on each object rather than once per file?
5. . **[retrieval: ch3]** Chapter 3 said a controller compares desired state against current state and acts on the difference. Name both fields those states live in, and name the component that stores the object containing them.

Answers with Explanations

1. `apiVersion` (which version of the API this object belongs to; selects the schema), `kind` (what kind of object, here `CronTab`), `metadata` (identity: a name, a UID, an optional namespace), and `spec` (the desired state, whose internal shape is defined by `kind`) [source: `k8s-docs-objects-2026-08-23`]. You will not know what belongs inside `spec` for an unfamiliar resource; that is what `kubectl explain` is for. You will always know the four fields wrapping it. That is the whole claim of this section.

2. Answer: A.

- **B is wrong** and it is the diagnostic miss: it means the direction of authorship hasn't landed yet. Re-read the Fixed Point.
- **C is wrong** because `status` is not yours to write — it is supplied and updated by the system [source: `k8s-docs-objects-2026-08-23`]; what you typed is not what the object will report.
- **D is wrong** because `spec` is precisely the field you must set when you create the object [source: `k8s-docs-objects-2026-08-23`].

3. Answer: C.

- **A is wrong** because rejection happens at submission. A rejected apply produces an error and no object. An object that exists was accepted.
- **B is wrong** and inverts the authority relationship. `spec` is authoritative for intent; `status` merely reports observation.
- **D is wrong**: the presence of a controller is what *closes* the gap, not what causes it. Objects with no controller watching them are the ones where a gap would persist.
- **C is right** because the control plane "continually and actively manages every object's actual state to match the desired state you supplied" — the continuous gap-closing Chapter 3 named **reconciliation** [source: `k8s-docs-objects-2026-08-23`]. Continual management implies there are moments when the states differ. That is not breakage; that is the system working. Hold onto this one. Chapter 13 depends on your not having the opposite instinct, because a practitioner who reads every `spec/status` gap as a fault will spend a lot of time investigating perfectly healthy clusters [*cross-bearing: see Ch 13 — reading status before reading logs*].

4. `apiVersion` selects which version of the Kubernetes API you are using to create this object [source: `k8s-docs-objects-2026-08-23`]: effectively, which schema the rest of the document should be parsed under. It is per-object rather than per-file because a single file can contain several objects of different kinds, and different resource types are versioned independently. A Pod and a NetworkPolicy in one file do not share a version, and there is no sensible file-level answer.

5. [retrieval: ch3] Desired state lives in `spec`; current state lives in `status`. The object is stored by **etcd**, reached through the **kube-apiserver**. Every read and write of cluster state goes through the API server, and etcd is the backing store for all cluster data [source: `k8s-docs-cluster-architecture-2026-08-23`]. If you named the fields but not the components, re-read Chapter 3 §5 before continuing; §3 of this chapter assumes it.

Checkpoint: You've Now Mastered ✓ The four required fields of every Kubernetes manifest ✓ The three techniques for managing an object, and why you pick one and stay with it ✓ Who writes spec, who writes status, and why that asymmetry exists ✓ The full path a declaration takes from your file to a controller's attention □ Where an object's name lives, and what limits its uniqueness (next) □ How to name a set of objects rather than one

You now have the shape of the record. Two questions follow immediately from it, and this chapter answers both. Where does the name in `metadata` have to be unique? And how do you refer to fifty objects at once without listing fifty names?

§3 — Where a Name Lives

In Kubernetes, **namespaces** provide a mechanism for isolating groups of resources within a single cluster. Names of resources need to be unique within a namespace, but not across namespaces [source: k8s-docs-namespaces-2026-08-23].

That is the whole of the core idea: a namespace is a **scope for names**. Two teams can both have a Service called `database` and neither has to rename anything, because the two names never collide. They sit in different scopes. Two ships on two different registries can carry the same name, and neither registry has to care.

The documentation's guidance on when to use them is more restrained than you might expect, and the restraint is itself testable. Namespaces are intended for use in environments with many users spread across multiple teams, or projects. For clusters with a few to tens of users, **you should not need to create or think about namespaces at all**. Start using them when you need the features they provide [source: k8s-docs-namespaces-2026-08-23]. Namespaces are not a maturity badge; they are a tool with a specific job.

Two structural constraints, both short and both testable. **Namespaces cannot be nested inside one another**, and **each Kubernetes resource can only be in one namespace** [source: k8s-docs-namespaces-2026-08-23].

📌 **Snag:** Namespaces do not nest. If you have arrived from cloud IAM (management groups containing subscriptions containing resource groups) or from Linux control-group trees, the instinct to build a hierarchy is strong and the platform will not accommodate it. There is exactly one level.

The correction most guides skip

Here is a piece of official guidance that reliably surprises people: it is **not** necessary to use multiple namespaces to separate slightly different resources, such as different versions of the same software. Use labels to distinguish resources within the same namespace [source: k8s-docs-namespaces-2026-08-23].

Sit with that, because it cuts against a very common habit. A namespace-per-version, or namespace-per-environment-flavor, feels tidy. The documentation says that is the wrong tool, and it names the right one. We will finish this thought in §5, once you have the right tool in hand *[cross-bearing: see Ch 4 §5 — why labels, and not namespaces, partition versions]*.

Not everything lives in a namespace

Now the part that carries real exam weight, and that pays dividends eight chapters from now.

Namespace-based scoping is applicable **only for namespaced objects** (Deployments, Services, and so on) and **not for cluster-wide objects**, such as StorageClasses, Nodes, and PersistentVolumes. Most Kubernetes resources are in some namespace. However, namespace resources are not themselves in a namespace, and low-level resources such as nodes and persistent volumes are not in any namespace [source: k8s-docs-namespaces-2026-08-23]. *[cross-bearing: see Ch 11 — PersistentVolumes and StorageClasses]*



Figure: two namespaces inside one cluster. The identical names in both boxes are legal and unremarkable; that is what "unique within, not across" means. The four resource types in the outer region are not inside either box, and cannot be put inside one.

☆ **Fixed Point: Not everything lives in a namespace.** Nodes, PersistentVolumes, and StorageClasses are cluster-scoped, and so are namespace objects themselves. Namespace scoping applies only to namespaced objects [source: k8s-docs-namespaces-2026-08-23]. Any question about who may act on a resource has to start by asking which side of this boundary the resource is on.

That last sentence is a bearing taken now and plotted later. Chapter 12 is where the two lines cross: Kubernetes' permission model has two role types and two binding types, and the four combinations are not a table to memorize. They are a direct consequence of this boundary. A permission over a namespaced resource can be granted inside one namespace; a permission over a cluster-scoped resource cannot be, because there is no namespace to grant it in [*cross-bearing: see Ch 12 — deriving Role, ClusterRole, RoleBinding, and ClusterRoleBinding from this boundary*].

🔒 **Worth Securing:** You do not have to memorize which resources are namespaced. `kubectl api-resources --namespaced=true` and `kubectl api-resources --namespaced=false` list them [source: k8s-docs-namespaces-2026-08-23], and the answer is authoritative for *your* cluster, including resource types installed by operators that did not exist when this book was printed. Run it twice and you will remember the short exam-relevant list anyway, which is the pleasant thing about lookups: they teach you while you use them.

The four initial namespaces

Kubernetes starts with four namespaces, and each exists for a specific reason [source: k8s-docs-namespaces-2026-08-23]:

Namespace	What it is for
default	So that you can start using a new cluster without first creating a namespace
kube-system	The namespace for objects created by the Kubernetes system
kube-public	Readable by all clients, including those not authenticated. Mostly reserved for cluster usage, for resources that should be visible and readable publicly throughout the whole cluster
kube-node-lease	Holds the Lease objects associated with each node. Node leases allow the kubelet to send heartbeats so that the control plane can detect node failure

Two details in that table are the ones examiners like.

kube-public is a convention, not an enforcement. The documentation is explicit: *the public aspect of this namespace is only a convention, not a requirement* [source: k8s-docs-namespaces-2026-08-23]. It is often repeated as a hard property of the namespace; it is not one. It is not; it is a norm about what people put there, and norms make poor access control.

kube-node-lease is the one people forget, and it connects forward. Those Lease objects are how the control plane knows a node is still alive; when the heartbeats stop, the node controller marks the node's

condition and eventually acts *[cross-bearing: see Ch 8 — node conditions and heartbeats]* [source: k8s-docs-nodes-2026-08-23].

For a production cluster, the documentation suggests not using the default namespace: make other namespaces and use those [source: k8s-docs-namespaces-2026-08-23].

Finally, one plant, deliberately shallow. When you create a Service, it gets a corresponding DNS entry of the form `<service-name>.<namespace-name>.svc.cluster.local`. A container using only `<service-name>` resolves to the Service local to its own namespace; reaching across namespaces requires the fully qualified domain name [source: k8s-docs-namespaces-2026-08-23]. That is one sentence's worth of the topic. The mechanism behind it, what serves those records, what else gets one, how resolution actually proceeds, is Chapter 9's *[cross-bearing: see Ch 9 — cluster DNS, Service records, and FQDNs]*.

Namespaces are also the unit by which cluster resources get divided between multiple users, via resource quota [source: k8s-docs-namespaces-2026-08-23]. Named here; taught in Chapter 8 *[cross-bearing: see Ch 8 — ResourceQuota]*.

§4 — .11 Configuration Kept Outside the Image

Start with the problem, not the object.

You have one container image. It needs to run in development and in production. Chapter 2 established that a running container is not edited in place: you build a new image and recreate the container [source: k8s-docs-containers-2026-08-23]. So if the two environments need different settings, and the image cannot differ, the settings must live somewhere other than the image. One hull, two ports of call, two sets of papers waiting on the quay. You do not build a second vessel to satisfy the second harbor.

Here is the documentation's own framing of the situation. Imagine you are developing an application you can run on your own computer for development and in the cloud to handle real traffic. You write the code to look in an environment variable named `DATABASE_HOST`. Locally you set that variable to `localhost`. In the cloud you set it to refer to a Service that exposes the database component to your cluster [source: k8s-docs-configmap-2026-08-23]. One image. Two values. The values are not in the image.

That is exactly what a ConfigMap is for.

ConfigMap

A **ConfigMap** is an API object used to store **non-confidential** data in key-value pairs. Pods can consume ConfigMaps as environment variables, command-line arguments, or as configuration files in a volume. A ConfigMap allows you to decouple environment-specific configuration from your container images, so that your applications are easily portable [source: k8s-docs-configmap-2026-08-23].

Four facts about ConfigMaps carry weight.

The size ceiling. A ConfigMap is not designed to hold large chunks of data; the data stored in a ConfigMap cannot exceed **1 MiB**. For settings larger than that, the guidance is to mount a volume, or use a separate database or file service [source: k8s-docs-configmap-2026-08-23].

The namespace requirement. The Pod and the ConfigMap must be in the same namespace [source: k8s-docs-configmap-2026-08-23]. That is §3's scope-for-names rule showing up one section later in the place you actually trip over it: a Pod naming a ConfigMap is naming it inside its own scope.

The four consumption paths, and the asymmetry among them. There are four different ways to use a ConfigMap to configure a container inside a Pod: inside a container command and args; as environment variables for a container; as a file in a read-only volume for the application to read; or by writing code that runs inside the Pod and uses the Kubernetes API to read the ConfigMap. **For the first three methods, the kubelet uses the data from the ConfigMap when it launches the container(s) for a Pod. The fourth method lets the application subscribe to updates whenever the ConfigMap changes** [source: k8s-docs-configmap-2026-08-23].

📌 **Snag:** You edited the ConfigMap. The running application did not notice. In my experience this is the most common day-one surprise with ConfigMaps, and it is not a bug: three of the four consumption paths are applied by the kubelet *at container launch*. Only the fourth, where your code reads the Kubernetes API itself, subscribes to changes [source: k8s-docs-configmap-2026-08-23]. If you want the first three to pick up a change, something has to cause the containers to be launched again.

Immutability. Starting from v1.19 you can add an `immutable` field to a ConfigMap definition. Once a ConfigMap is marked as immutable, **it is not possible to revert this change**, nor to mutate the contents of its `data` or `binaryData` fields. You can only delete and recreate the ConfigMap [source: k8s-docs-configmap-2026-08-23]. Notice the shape of that: the same replace-don't-mutate instinct that governs container images, applied to configuration. Notice also that `immutable: true` is a one-way door, and the platform will not ask whether you meant it.

And one caution stated in the documentation with unusual directness, which sets up the rest of this section: **ConfigMap does not provide secrecy or encryption**. If the data you want to store are confidential, use a Secret rather than a ConfigMap, or use additional third-party tools to keep your data private [source: k8s-docs-configmap-2026-08-23].

Which raises the obvious question. What, exactly, does a Secret do about that?

Secret

A **Secret** is an object that contains a small amount of sensitive data such as a password, a token, or a key. Such information might otherwise be put in a Pod specification or in a container image; using a Secret means you don't need to include confidential data in your application code. **Secrets are similar to ConfigMaps but are specifically intended to hold confidential data** [source: k8s-docs-secret-2026-08-23].

Read that last clause carefully: *intended to hold*. Intent is doing a great deal of work in that sentence, and the documentation's very next block explains why.

Kubernetes Secrets are, by default, stored **unencrypted** in the API server's underlying data store (etcd). Anyone with API access can retrieve or modify a Secret, and so can anyone with access to etcd. Additionally, **anyone who is authorized to create a Pod in a namespace can use that access to read any Secret in that namespace** — this includes indirect access such as the ability to create a Deployment [source: k8s-docs-secret-2026-08-23].

Base64 is not a lock

Soundings question 6 asked whether base64 encoding is a form of encryption, and told you to hold the answer. Collect it now, because a Secret is where the misconception does damage.

A Secret's data field holds arbitrary data **encoded using base64**; the serialized form of the secret data is a base64-encoded string [source: k8s-docs-secret-config-file-2026-08-24] [source: k8s-api-ref-secret-v1-2026-08-24]. That is the first thing people notice about a Secret manifest, and it is the first thing people misread. The documentation removes the ambiguity in one sentence:

Base64 encoding is *not* an encryption method, it provides no additional confidentiality over plain text [source: k8s-docs-secrets-good-practices-2026-08-24].

Both halves of the situation now sit in one place, stated by the project itself: *Secret values are encoded as base64 strings and are stored unencrypted by default, but can be configured to be encrypted at rest* [source: k8s-docs-secrets-good-practices-2026-08-24]. Encoded, not encrypted. Unencrypted by default, not unencryptable.

One practical consequence worth carrying: if you configure a Secret through a manifest with the data base64-encoded, sharing that file or checking it into a source repository means the secret is available to everyone who can read the manifest [source: k8s-docs-secrets-good-practices-2026-08-24]. The encoding does not make the file safe to commit. It only makes the file *look* safe to commit, which is worse.

☆ **Fixed Point: Neither object encrypts anything.** A ConfigMap provides no secrecy or encryption. A Secret's values are *base64-encoded*, which is not encryption and adds no confidentiality over plain text, and are stored *unencrypted* by default, readable by anyone with API access, etcd access, or the ability to create a Pod in the namespace. What a Secret adds is *handling*: a distinct object type, a distinct surface for access-control rules, and a defined place to attach encryption at rest. The difference between the two is intent and treatment, not cryptography [source: k8s-docs-configmap-2026-08-23] [source: k8s-docs-secret-2026-08-23] [source: k8s-docs-secrets-good-practices-2026-08-24].

The documentation follows its caution with four steps to take in order to use Secrets safely: enable Encryption at Rest for Secrets; enable or configure RBAC rules with least-privilege access to Secrets; restrict Secret access to specific containers; and consider using external Secret store providers [source: k8s-docs-secret-2026-08-23].

Every one of those four is Chapter 12's, and this section is going to hand them over without teaching a single one. That restraint is deliberate: the material above is alarming enough that the pull toward "and here's how to fix it" is strong, but Chapter 12 is where encryption at rest, least-privilege access rules, and the broader security posture get the room they need [*cross-bearing: see Ch 12 — hardening Secrets, and the access-control model behind it*]. A Secret is a strongbox stowed in the same hold as everything else. The lock is Chapter 12's; this chapter is only telling you the box did not ship with one fitted.

🔑 **Closer Look:** The third item in that caution, *anyone authorized to create a Pod in a namespace can read any Secret in that namespace, including indirectly via a Deployment*, inverts an intuition, and it repays turning over slowly.

You might reasonably assume that "can read Secrets" and "can create workloads" are separate permissions to be granted separately. They are separate permissions, and granting the second effectively grants the first: a Pod you create can mount any Secret in its namespace, and once mounted, its contents are yours to print. There is no way to create a Pod without that being true, because handing Secrets to Pods is what Secrets are for.

The practical consequence is that permission to create Pods in a namespace is, in security terms, a Secrets question, and nothing in the name of the permission says so. This is exactly the thread Chapter 12 picks up.

Types of Secret

Secrets carry a type, which signals what kind of data they hold and, per the API reference, exists to *facilitate programmatic handling of secret data* [source: k8s-api-ref-secret-v1-2026-08-24]. The built-in types [source: k8s-docs-secret-2026-08-23]:

Built-in type	Usage
Opaque	Arbitrary user-defined data — the default type
kubernetes.io/service-account-token	ServiceAccount token (a legacy long-lived credential)
kubernetes.io/dockercfg	Serialized ~/.dockercfg file
kubernetes.io/dockerconfigjson	Serialized ~/.docker/config.json file
kubernetes.io/basic-auth	Credentials for basic authentication
kubernetes.io/ssh-auth	Credentials for SSH authentication
kubernetes.io/tls	Data for a TLS client or server
bootstrap.kubernetes.io/token	Bootstrap token data

Three rows need a sentence each.

opaque is the default, and "arbitrary user-defined data" is exactly what it means. A Secret holding your application's API key, with no type set, is Opaque. The type is a signal to whatever reads the object, not a constraint on what you may store.

kubernetes.io/dockerconfigjson closes a loop Chapter 2 opened. That chapter listed five ways to give a cluster access to a private registry (configuring nodes to authenticate to the registry, a kubelet credential provider that fetches credentials dynamically, pre-pulled images, specifying `imagePullSecrets` on a Pod, and vendor-specific extensions) and deferred the most common one to this section [source: k8s-docs-images-2026-08-23] [*cross-bearing: see Ch 2 §3 — five ways to reach a private registry*]. Here it is: an `imagePullSecret` is a `Secret` of type `kubernetes.io/dockerconfigjson`, and what it holds is a serialized `~/.docker/config.json`, the same credential file your local tooling writes, filed as a cluster object [source: k8s-docs-images-2026-08-23] [source: k8s-docs-secret-2026-08-23]. If a Pod is stuck unable to pull an image from a private registry, this is the object that is missing or wrong.

kubernetes.io/service-account-token is named here and then left alone. It is a legacy long-lived credential; since v1.22 the recommended approach is short-lived, automatically rotating tokens obtained through the `TokenRequest` API [source: k8s-docs-secret-2026-08-23] [source: k8s-docs-service-accounts-2026-08-23]. The identity model those tokens belong to (what a `ServiceAccount` *is*, how a Pod gets one, and what it is allowed to do) is Chapter 5's introduction and Chapter 12's full treatment [*cross-bearing: see Ch 5 §6 — a Pod's identity*].

Side by side

ATTRIBUTE	ConfigMap	Secret
Intended contents	non-confidential key-value data	small amounts of sensitive data (password, token, key)
Consumed by a Pod as	- command and args - environment variables - file in a read-only volume - read via the Kubernetes API	- command and args - environment variables - file in a volume - read via the Kubernetes API
Stored	unencrypted, in etcd	unencrypted, in etcd (by default)
What it adds	nothing — the docs state plainly that it provides no secrecy or encryption	- a distinct object type - a distinct access-control surface - a defined place to attach encryption at rest

LEGEND
 FOCAL — SAME VALUE IN BOTH COLUMNS

Figure: the two objects differ in intent and in treatment, not in cryptography. The third row is identical on both sides, and that is the row people get wrong. [source: k8s-docs-configmap-2026-08-23] [source: k8s-docs-secret-2026-08-23] [source: k8s-docs-volumes-2026-08-23]

⚠ Navigational Hazards

In my experience, this pair produces more confident wrong answers than anything else in this chapter. Six of them, all in one place:

"ConfigMaps are for configuration; Secrets are for *secure* configuration." The first half is fine. The second half is the misconception. A Secret is not encrypted by default and is readable by several categories of principal you may not have thought about [source: k8s-docs-secret-2026-08-23]. It is a *differently handled* object, and the handling is what you must configure.

"The value is base64, so it's protected." Base64 is not an encryption method and provides no additional confidentiality over plain text [source: k8s-docs-secrets-good-practices-2026-08-24]. Anyone who can read the object can decode the value in one command.

"Update the ConfigMap and the running container picks it up." Only if the application reads the Kubernetes API directly. The other three consumption paths are applied by the kubelet at container launch [source: k8s-docs-configmap-2026-08-23].

"Immutable can be turned back off." It cannot. Marking a ConfigMap immutable is irreversible; delete and recreate is the only route back [source: k8s-docs-configmap-2026-08-23].

"A ConfigMap is a fine place for that file." Up to 1 MiB of data. Past that, use a volume, a database, or a file service [source: k8s-docs-configmap-2026-08-23].

"Pods in team-a can reference the ConfigMap in team-b." They cannot. The Pod and the ConfigMap must be in the same namespace [source: k8s-docs-configmap-2026-08-23]. That is §3's scope-for-names rule showing up where you actually trip over it.

One forward pointer before we move on. "Store config in the environment" is the third of the twelve factors [source: twelve-factor-app-2026-08-23], a methodology these two objects implement almost exactly. We will name that connection properly when we look at what cloud native application delivery actually assumes about your application [*cross-bearing: see Ch 15 — the twelve factors, and which ones Kubernetes hands you for free*]. Both objects also appear again as volume types, mounted into a Pod's filesystem [*cross-bearing: see Ch 11 — ConfigMap and Secret volumes*] [source: k8s-docs-volumes-2026-08-23].

🧭 Taking Your Bearings #2: Scoping and Configuration Objects

Four questions, covering §3 and §4 together.

1. ... Name three Kubernetes resources that are **not** namespaced, and say what they have in common.

A) Deployment, Service, Pod B) Node, PersistentVolume, StorageClass C) ConfigMap, Secret, ServiceAccount D) Node, ConfigMap, Namespace

2. ☐ Two versions of one application need to run in one cluster. Do you separate them with two namespaces, or with two label values — and why?

3. ☐ A ConfigMap is updated. Under which consumption path does the running application see the change?

A) Environment variables — the kubelet refreshes them on change
 B) A file in a read-only volume — the file is rewritten and the app re-reads it automatically
 C) Container command and args — the container is re-executed with the new values
 D) Code inside the Pod that reads the ConfigMap through the Kubernetes API

4. ☐ **[retrieval: ch2]** An imagePullSecret is a Secret of type `kubernetes.io/dockerconfigjson`, holding a serialized `~/.docker/config.json`. That is one of the five mechanisms Chapter 2 listed for giving a cluster access to a private image registry. **Name two of the other four**, and say why the Secret is the one this chapter covers.

Answers with Explanations

1. Answer: B. What they have in common is that they are cluster-scoped. They exist outside any namespace and cannot be placed inside one [source: k8s-docs-namespaces-2026-08-23].

- **A is wrong:** all three are namespaced, and they are the documentation's own examples of namespaced objects.
- **C is wrong:** all three are namespaced. ServiceAccounts in particular are explicitly bound to a namespace, with every namespace getting a default one on creation [source: k8s-docs-service-accounts-2026-08-23].
- **D is the interesting trap.** Two of the three are correct: Nodes and Namespaces are both cluster-scoped, and Namespace is the one people most often miss, since namespace resources are not themselves in a namespace. But ConfigMap is namespaced, which is exactly why §4's same-namespace rule exists.

2. Two label values, in one namespace. The documentation is explicit that it is not necessary to use multiple namespaces to separate slightly different resources such as different versions of the same software, and directs you to use labels to distinguish resources within the same namespace [source: k8s-docs-namespaces-2026-08-23]. The *why* is what §5 is about. Hold your answer and check it again in a few pages.

3. Answer: D. Only the API-reading path lets the application subscribe to updates whenever the ConfigMap changes [source: k8s-docs-configmap-2026-08-23].

- **A, B, and C are all wrong for the same reason**, which is why they make a good set: for all three of those methods, the kubelet uses the data from the ConfigMap *when it launches the containers* for a Pod. The data is read once, at launch, and then it is simply what the container has.

- **B is the most seductive** of the three because a volume feels live in a way an environment variable does not, and in some configurations the projected file content does eventually change on disk. The application still has to be written to notice, though, and a container using a ConfigMap as a subPath volume mount will not receive updates at all [source: k8s-docs-volumes-2026-08-23] [cross-bearing: see Ch 11 — ConfigMap and Secret volumes, and the subPath exception]. Treating "it's a volume, so it's live" as a rule will burn you.

4. [retrieval: ch2] Any two of: **configuring the nodes themselves to authenticate to the private registry; a kubelet credential provider that fetches credentials dynamically; pre-pulled images already present on the node; vendor-specific or local extensions** [source: k8s-docs-images-2026-08-23]. The Secret path is the one this chapter covers because it is the only one of the five that is itself a Kubernetes object you write, submit, and version. The other four are properties of the node, the kubelet, or the platform. If you could not produce two, re-read Chapter 2 §3; the list is short and it is exam-relevant.

Checkpoint: You've Now Mastered ✓ Namespaces as a scope for names, and the resources that live outside every scope ✓ The four initial namespaces and what each one is actually for ✓ ConfigMaps and Secrets — what they hold, how they are consumed, and what protection they do not provide ✓ Why base64 in a Secret manifest is an encoding and not a lock □ How to name a set of objects by describing them (next, and it is the one you will use most)

§5 — .11 The Universal Join

Everything so far has been about one object at a time. This section is about how you talk about many.

Labels are key/value pairs that are attached to objects. Labels are intended to be used to specify identifying attributes of objects that are meaningful and relevant to users, but that **do not directly imply semantics to the core system**. They can be used to organize and to select subsets of objects. Labels can be attached to objects at creation time and subsequently added and modified at any time. Each object can have a set of key/value labels defined, and each key must be unique for a given object [source: k8s-docs-labels-selectors-2026-08-23].

Slow down on that middle clause: *do not directly imply semantics to the core system*. Kubernetes does not know what tier: frontend means. It has no built-in concept of a frontend. That ignorance is precisely why labels are useful everywhere: because the system attaches no meaning to them, you are free to attach yours, and the system's grouping machinery works identically regardless of what you meant. A label is closer to a signal flag than to a filing category. The flag means whatever your fleet agreed it means; the machinery that reads it only cares that it is flying. Labels let you map your own organizational structures onto system objects in a loosely coupled fashion, without requiring clients to store those mappings [source: k8s-docs-labels-selectors-2026-08-23].

The documentation's own example labels are the ones practitioners actually use, and they make the abstraction concrete for free [source: k8s-docs-labels-selectors-2026-08-23]:

```
release:      stable | canary
environment:  dev | qa | production
tier:         frontend | backend | cache
partition:    customerA | customerB
track:        daily | weekly
```

The syntax, at the depth this exam tests. Valid label keys have two segments: an optional prefix and a name, separated by a slash. The name segment is required and must be **63 characters or less**, beginning and ending with an alphanumeric character, with dashes, underscores, dots, and alphanumerics between. The prefix is optional; if specified it must be a DNS subdomain no longer than 253 characters, followed by a slash. The **kubernetes.io/ and k8s.io/ prefixes are reserved** for Kubernetes core components. Valid label values must be 63 characters or less, and can be empty [source: k8s-docs-labels-selectors-2026-08-23].

The label selector

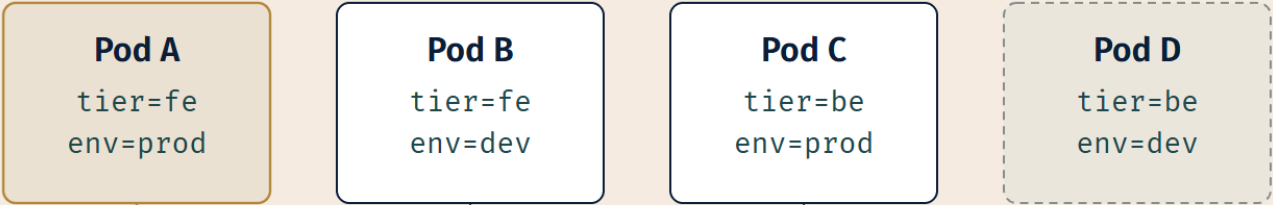
Via a label selector, a client or user can identify a set of objects. The documentation gives it a title, and the title is the one line to memorize verbatim: **the label selector is the core grouping primitive in Kubernetes** [source: k8s-docs-labels-selectors-2026-08-23].

The API supports two types.

Equality-based selectors use `=`, `==`, and `!=`. For example, `environment = production`, Or `tier != frontend` [source: k8s-docs-labels-selectors-2026-08-23].

Set-based selectors use `in`, `notin`, and `exists` (plus its negation). For example, `environment in (production, qa)`, `tier notin (frontend, backend)`, `partition` (meaning: has the key at all), and `!partition` (meaning: does not have the key). Set-based requirements are **more expressive** than equality-based ones, and multiple requirements are **ANDed** together with commas [source: k8s-docs-labels-selectors-2026-08-23].

OBJECTS, EACH CARRYING LABELS



SELECTORS, EACH RESOLVING TO A SET

tier = fe



env = prod



env in (prod, qa)



tier = be , env = prod



LEGEND

○ Pod A – in 3 of 4 sets

○ matches this selector

□ Pod D – matches

Figure: sets, not a taxonomy. Pod A belongs to two selected sets at once and Pod D to none of them, and that overlap is the entire reason this mechanism is useful. If the four Pods had partitioned cleanly into four boxes you would be looking at a folder structure, not a selector.

The bridge to what you will actually see

Some resource types accept only equality-based selectors. Newer resources (**Job, Deployment, ReplicaSet, and DaemonSet**) support set-based requirements through two structured fields, `matchLabels` and `matchExpressions`. And the relationship between them is exact: **`matchLabels` is a map of {key, value} pairs equivalent to a `matchExpressions` entry with operator `In`** [source: k8s-docs-labels-selectors-2026-08-23].

That equivalence is the kind of precise, checkable fact this exam rewards. `matchLabels: {tier: frontend}` and `matchExpressions: [{key: tier, operator: In, values: [frontend]}]` are the same requirement written two ways. `matchLabels` is not a weaker feature; it is shorthand.

☆ **Fixed Point: The label selector is the core grouping primitive in Kubernetes** [source: k8s-docs-labels-selectors-2026-08-23]. Labels are selectable. Annotations are not. Nearly every question in Kubernetes of the form "which objects does this apply to?" is answered by a selector over labels.

Write that one down, because you are about to meet it constantly. A `ReplicaSet` knows which Pods are its Pods by selector [cross-bearing: see Ch 6 — a controller's selector and the Pods it owns]. A `Service` identifies the set of Pods behind it by selector, which is what makes Pod churn survivable [source: k8s-docs-service-2026-08-23] [cross-bearing: see Ch 9 — a Service selects its backends]. Node scheduling constraints use labels on nodes; the recommended approaches all use label selectors to facilitate the selection [source: k8s-docs-assign-pod-node-2026-08-23] [cross-bearing: see Ch 7 — node labels and `nodeSelector`]. A `NetworkPolicy` uses a selector to specify what traffic is allowed to and from the Pods that match [source: k8s-docs-network-policies-2026-08-23] [cross-bearing: see Ch 10 — `NetworkPolicy` selects both its subject and its peers].

🔒 **Worth Securing:** Once you internalize that all of those are the same mechanism pointed at different things, four chapters get considerably easier. `ReplicaSet`, `Service`, `NetworkPolicy`, and `node affinity` are not four mechanisms to learn. They are one mechanism, *describe a set by its attributes*, aimed at four problems. Learn the primitive once and you spend the later chapters learning what each resource *does* with its set, which is the interesting part.

There is one important exception, and it goes in now precisely because you have just been told that everything is a selector. Kubernetes' permission model is not. Role-based access control names its subjects and its resources explicitly — an explicit list, where everything else in this section is a query [source: k8s-docs-rbac-2026-08-23]. A reader who leaves this section assuming otherwise will make a specific, confident, wrong prediction in Chapter 12 [cross-bearing: see Ch 12 — why RBAC names subjects instead of selecting them].

(Kubernetes also has field selectors — the name is worth recognizing, and recognition is all this chapter needs [source: k8s-docs-objects-2026-08-23]. Different thing, different syntax, not a substitute. Noted so that you recognize the name; nothing here depends on it.)

Annotations, by contrast

Annotations are the other half of metadata's user-supplied data. The documentation's opening line is the whole definition: you use annotations to attach **arbitrary non-identifying metadata** to objects, and clients such as tools and libraries can retrieve it [source: k8s-docs-annotations-2026-08-24].

The contrast with labels is stated directly, and it is the sentence to keep:

Labels can be used to select objects and to find collections of objects that satisfy certain conditions. In contrast, annotations are **not used to identify and select objects** [source: k8s-docs-annotations-2026-08-24].

The rule fits in one sentence, and it is the whole distinction:

If you might ever want to find objects by it, it is a label. If you only want to record it, it is an annotation.

A build identifier that a deployment tool wants to select on is a label. A build identifier that exists so a human can read it during an incident is an annotation. Same string, different job.

What people actually put in them. The documentation's own list is the best answer to "like what?": build, release, or image information such as timestamps, release IDs, git branch, PR numbers, image hashes, and registry addresses; pointers to logging, monitoring, analytics, or audit repositories; client library or tool information used for debugging, such as name, version, and build; lightweight rollout-tool metadata such as config or checkpoints; and phone or pager numbers of the people responsible, or a directory entry pointing at the team's web site [source: k8s-docs-annotations-2026-08-24]. Notice the shape of that list. Every item is something a human or a tool reads *after* it has already found the object. None of them is something you would search on.

The syntax difference is the sharper version of the rule. Annotation keys follow exactly the same two-segment rules as label keys, an optional DNS-subdomain prefix and a required name segment of 63 characters or less, and `kubernetes.io/` and `k8s.io/` are reserved for core components in both cases [source: k8s-docs-annotations-2026-08-24]. The *values* are where they part company:

	Label value	Annotation value
Character set	Constrained — values capped at 63 characters; not free-form the way annotation values are [source: k8s-docs-annotations-2026-08-24]	No restrictions — any string, including special characters, whitespace, and structured data such as JSON or YAML
Length	63 characters or less	No per-value cap; all annotations on one object must total ≤ 256 KiB
Selectable	Yes	No

[source: k8s-docs-labels-selectors-2026-08-23] [source: k8s-docs-annotations-2026-08-24]

That table is the distinction restated as engineering. Labels are constrained *because* they are indexed and selected on. Annotations are unconstrained *because* nothing has to search them. For annotations, the keys and the values in the map must be strings — no numbers, booleans, or lists [source: k8s-docs-annotations-2026-08-24]; label values are strings as well, under the tighter syntax above.

⚠ Navigational Hazards

Two mistakes live here, and they are the same mistake twice.

Recording something as an annotation and then trying to select on it. Selectors operate over labels. An annotation cannot be selected on, not because it is a less capable field, but because that is the definition of the two [source: k8s-docs-annotations-2026-08-24]. A controller that needs to find its objects cannot find them by annotation.

Using a namespace to separate two versions of the same software. §3 flagged this and deferred the reason. Here it is: namespaces partition *names*. Labels partition *sets*. Nearly everything in Kubernetes operates over sets, every controller and every Service and every policy, so partitioning by namespace when you meant to partition by set leaves the mechanism that would have grouped your objects unable to see across the line you drew. The documentation's advice to use labels for different versions of the same software is not a stylistic preference [source: k8s-docs-namespaces-2026-08-23]; it is the difference between a distinction the system can act on and one it cannot.

Both errors are the same shape: reaching for the wrong partitioning tool. Learn the shape, not the two facts.

🕒 Taking Your Bearings #3: Labels, Selectors, and Annotations

Four questions. The last one requires two sections at once.

1. ... Write the equality-based form and the set-based form of the same requirement: *objects whose environment label is production*.

2. You are looking at a workload resource with this selector:

```
matchLabels:
  tier: frontend
```

Write the equivalent using `matchExpressions`.

3. Your deployment tooling needs to attach the originating Git commit hash to every object it creates. In one case, a controller will later need to find all objects from a given commit. In the other, the hash exists purely so a human can read it during an incident. Label or annotation, in each case, and why?

4. A selector matches several objects in one namespace, but matches nothing in another namespace where identically labeled objects demonstrably exist. What is happening?

A) Labels are cluster-scoped, so the same label key cannot carry different values in two namespaces B) A namespaced query operates within its namespace scope; it does not cross the boundary [source: k8s-docs-namespaces-2026-08-23] C) Set-based selectors work across namespaces but equality-based ones do not D) The objects in the second namespace must be missing the `kubernetes.io/` prefix on their label keys

Answers with Explanations

1. Equality-based: `environment = production` (or `environment == production`). Set-based: `environment in (production)` [source: k8s-docs-labels-selectors-2026-08-23]. They express the same requirement; set-based syntax is simply more expressive in general, since it can also express membership in several values, key existence, and their negations.

2.

```
matchExpressions:
- key: tier
  operator: In
  values: [frontend]
```

`matchLabels` is a map of `{key, value}` pairs equivalent to `matchExpressions` with operator `In` [source: k8s-docs-labels-selectors-2026-08-23]. If you wrote operator `Equals`, that is not one of the set-based operators. The set-based vocabulary is `In`, `NotIn`, `Exists`, and `DoesNotExist`.

3. The first case is a **label**: a controller needs to *find* objects by it, which is exactly what labels are for and exactly what a selector operates over. The second is an **annotation**: annotations attach arbitrary non-identifying metadata, and are not used to identify and select objects [source: k8s-docs-annotations-2026-08-24]. Note that the *content* is identical in both cases, the same forty-character hash. What determines the answer is not the data, it is whether anything will ever need to select on it. Build and release information sits on the documentation's own list of things people record in annotations [source:

k8s-docs-annotations-2026-08-24], so the default assumption is that a commit hash is *recorded*; it becomes a label only when something has to search on it.

4. Answer: B.

- **A is wrong.** Labels are per-object, not cluster-scoped; the same key can carry different values on every object in the cluster, in any namespace.
- **C is wrong.** Both selector types behave identically with respect to namespace scope; the difference between them is expressiveness [source: k8s-docs-labels-selectors-2026-08-23].
- **D is wrong,** and inverts the rule: `kubernetes.io/` and `k8s.io/` prefixes are *reserved for core components*, not required of yours [source: k8s-docs-labels-selectors-2026-08-23].
- **B is right,** and it is the item to carry forward. Selection is a set operation; namespace is a scope. A query issued inside one namespace considers the objects in that namespace. This combination, a selector plus a namespace boundary, is the exact reasoning Chapter 10 needs for network policy, where the question *which Pods does this apply to?* has to be answered on both sides of a namespace line [source: k8s-docs-network-policies-2026-08-23].

Checkpoint: You've Now Mastered ✓ Labels, their syntax, and why the system's indifference to their meaning is a feature ✓ Both selector syntaxes, and the exact `matchLabels ≡ matchExpressions + In` equivalence ✓ Annotations, what people actually record in them, and the one-sentence rule that separates them from labels ✓ Why versions of one application belong in one namespace under different labels

§6 — .nl A Declaration, Not an Order

Count what you have written in this chapter.

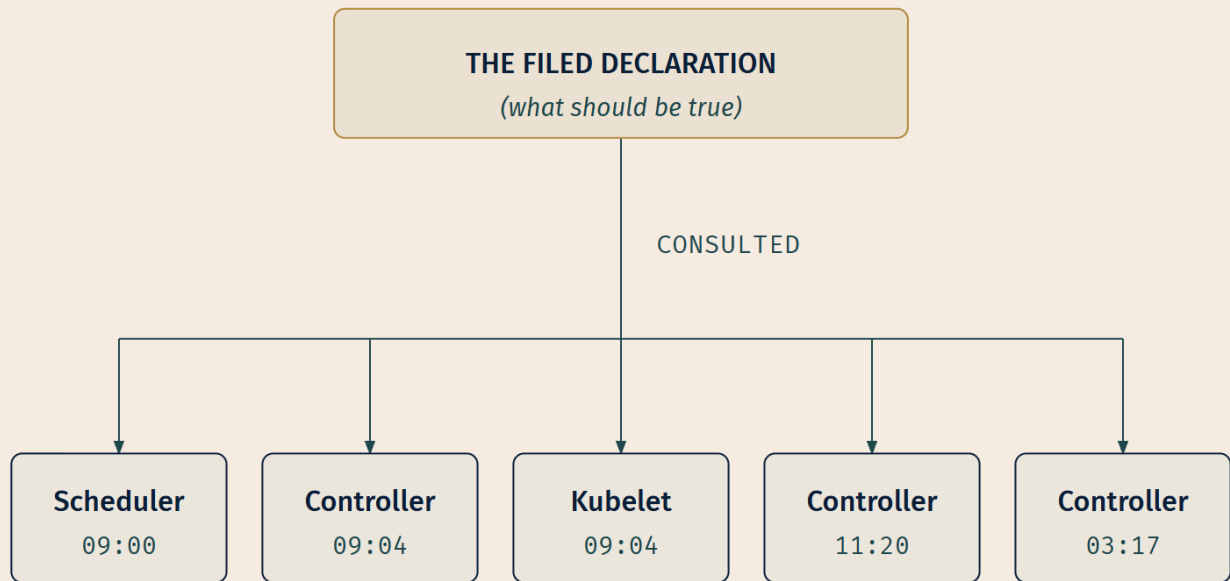
An object, with four fields. A namespace, which scopes a name. A ConfigMap, which holds configuration. A Secret, which holds configuration that is treated differently. A set of labels, and a selector that names a set by describing it.

Not one of them contains a step.

Every object in this chapter is a **noun**. `spec` says what should exist. `status` reports what does. The four fields are how you say it. A namespace says where the name lives. Labels say which sets it belongs to. ConfigMaps and Secrets say what it should be configured with. There is no verb anywhere in the pile: no *then*, no *next*, no *if that failed*, *retry*. You described a state of affairs and filed it.

And yet things happen. Here is why:

The Kubernetes control plane continually and actively manages every object's actual state to match the desired state you supplied [source: k8s-docs-objects-2026-08-23].



*No hand receives an instruction.
Every hand reads the record, observes the world, and closes the gap.*

LEGEND

Reference (focal)

Independent reader

→ Read, own schedule

Figure: the record outranks the moment it was written. Nothing in this figure is a command being passed down a chain; every arrow is a reading.

☀ Zenith

☀ **Zenith:** The loop Chapter 3 showed you as an architecture diagram now has an input you know how to write. That was the promise at the end of Chapter 3 [*cross-bearing: see Ch 3 §6 — the control loop, and the field it compares against*], and this is the shape of it: **you do not participate in the loop. You supply its reference.**

The system reads your declaration and observes the world. Wherever they disagree, it acts. It does this at 09:00 and at 03:17 and on the fourteen-thousandth iteration, with exactly the same logic, without you present, without needing to know what you did last time, without a workflow to resume. Your record is not a message that was delivered. It is the standing answer to a question the cluster asks itself continuously.

Go back to the harbormaster's desk from §1. The declaration you lodged is still sitting there, unchanged, in the same hand you wrote it in. Everything that has happened since happened because someone read it and did not like what they saw out the window.

Name what that architecture buys you, because a distinction without a payoff is just trivia.

A system that takes descriptions rather than commands can accept the same description twice with no harm done: the second submission describes the same world as the first, so there is nothing to correct. It can be told what should be true by someone who does not know what is currently true, because working out the difference is the system's job, not the author's. And it can keep that description in a file, which means the description outlives the terminal session that produced it, the person who typed it, and the incident that prompted it. Chapter 3 put it in the documentation's words and it lands harder now: *it shouldn't matter how you get from A to C* [source: k8s-docs-overview-2026-08-23].

Those three properties compose into something, and this book will spend a later chapter on it. Not yet.

The honest correction

One more time, precisely, because §1 promised it and because slogans deserve auditing.

`kubect1 scale` updates the size of a workload [source: k8s-docs-kubect1-overview-2026-08-23], which is to say it edits a number in a `spec`, and then something else notices. The imperative command did not scale anything. It amended a record, and the loop did the rest.

The objects are declarations, and the imperative commands work by changing declarations. That is the accurate claim, narrower than the chapter subtitle and better. It is the same discipline Chapter 3 applied to "nobody is in charge": both statements are true in the way that matters and false if you press them literally, and knowing which is which is the difference between understanding a system and reciting a slogan about it.

Exam Alert! 🚨

High-Priority Topics

1. **The four required manifest fields:** `apiVersion`, `kind`, `metadata`, `spec`. By name, and what each supplies. The most mechanically checkable fact in this competency.
2. **spec versus status:** which you set, which the system supplies and updates.
3. **Namespaced versus cluster-scoped**, with `Nodes`, `PersistentVolumes`, and `StorageClasses` as the named cluster-scoped examples, and namespace objects themselves as the one that is easy to overlook.
4. **The four initial namespaces** and what each is for. `kube-node-lease` is the forgotten one, and its purpose (Lease objects for node heartbeats) is the detail worth pinning down. `kube-public`'s public aspect is a *convention*.
5. **Labels versus annotations:** identifying, constrained, and selectable, versus non-identifying, unconstrained, and not.
6. **ConfigMap versus Secret:** a difference of intent and handling, not of encryption. Base64 is an encoding.
7. **What a declaration actually is:** the objects are declarations, and the imperative commands work by changing declarations.

Common Traps

Trap	Correct understanding
Using a namespace to separate two versions of the same software	Use labels within one namespace. Namespaces scope names; labels partition sets [source: k8s-docs-namespaces-2026-08-23]
"Everything lives in a namespace"	Nodes, PersistentVolumes, StorageClasses, and namespaces themselves do not [source: k8s-docs-namespaces-2026-08-23]
"kube-public is enforced as publicly readable"	The public aspect is only a convention, not a requirement [source: k8s-docs-namespaces-2026-08-23]
Confusing labels with annotations	Labels identify and can be selected on; annotations record and cannot [source: k8s-docs-annotations-2026-08-24]
"ConfigMaps hold config; Secrets hold <i>secure</i> config"	Secrets are stored unencrypted by default [source: k8s-docs-secret-2026-08-23]. The difference is treatment, not cryptography
"The value is base64, so it's protected"	Base64 is not an encryption method and provides no additional confidentiality over plain text [source: k8s-docs-secrets-good-practices-2026-08-24]
Assuming a ConfigMap change reaches a running container	Only via the API-reading path. The other three are applied by the kubelet at container launch [source: k8s-docs-configmap-2026-08-23]
"An immutable ConfigMap can be un-marked"	It cannot be reverted. Delete and recreate [source: k8s-docs-configmap-2026-08-23]
Forgetting the ConfigMap size ceiling	1 MiB [source: k8s-docs-configmap-2026-08-23]
Referencing a ConfigMap across namespaces	The Pod and the ConfigMap must be in the same namespace [source: k8s-docs-configmap-2026-08-23]

Six of those ten live in §4, which is why that section carries the chapter's densest warning block and its highest attention cost. If you are budgeting revision time, §4 is where it goes.

Practice Questions

Twenty-one questions. Four test material from earlier chapters and are tagged as such. Four require two sections at once, because single-section recall is not what this exam is measuring.

Q1. .. [retrieval: ch3] You run `kubectl apply -f deployment.yaml`. Which component receives the request, and where does the resulting object live?

A) The kube-apiserver receives it and etcd stores it B) The kubelet receives it and writes the object to disk on the node C) The kube-scheduler receives it and holds it in its scheduling queue D) The kube-controller-manager receives it directly and stores it in memory

Q2. .. [retrieval: ch3] You change one number in a manifest's spec and re-apply it. What happens to the control loop as a result?

A) The loop is restarted with the new value as its starting condition B) Nothing until you also run a command telling the controller to reconcile C) The controller now observes a difference between `spec` and `status` and acts to close it D) The previous loop terminates and a new loop is created for the new desired state

Q3. .|| [retrieval: ch3] Chapter 3 used the built-in Job controller as its worked example of the controller pattern. When the Job controller determines that Pods are needed to carry out a task, what does it actually do?

A) It starts the Pods itself, on a node it selects B) It instructs the kubelet on each chosen node to start the containers C) It writes the new Pod objects into etcd directly, since it is a control-plane component D) It tells the API server to create the Pods, and other components in the control plane act on that new information

Q4. .| Which of the following is **not** one of the four fields you must set in a manifest?

A) `apiVersion` B) `kind` C) `status` D) `metadata`

Q5. .|| Which statement about `apiVersion` is correct?

A) It names the version of the Kubernetes cluster the object was created on B) It names the version of the API being used to create this object, and it appears once per object C) It names the version of the API being used, and applies to the whole file D) It is optional for built-in resource types and required only for custom resources

Q6. .|| `kubectl scale deployment/web --replicas=5` is an imperative command. In the terms this chapter has established, what does it actually do?

A) It instructs the kubelet on each node to start the additional containers B) It bypasses the object model and operates directly on the running Pods C) It is rejected, because Kubernetes accepts declarative input only D) It edits a number in the Deployment's `spec`; a controller then observes the difference and acts

Q7. .| Which of these is cluster-scoped?

A) `PersistentVolume` B) `ConfigMap` C) `ServiceAccount` D) `Deployment`

Q8. .|| Your team wants to run v2.3 and v2.4 of the same application side by side in one cluster. What does the Kubernetes documentation direct you to do?

A) Create a namespace per version — this is the documented use of namespaces B) Create a namespace per version and nest both inside a parent namespace C) Create one namespace per version, then use a `ResourceQuota` to link them D) Keep both in one namespace and distinguish them with labels

Q9. .|| Which statement about Kubernetes' four initial namespaces is correct?

A) `kube-public` enforces public readability — unauthenticated readability is a property of the namespace itself B) `kube-system` is where the cluster places objects whose manifests did not specify a namespace C) `kube-node-lease` holds the Lease objects associated with each node, which let the kubelet send heartbeats so the control plane can detect node failure D) `default` is the recommended namespace for production workloads, since using it requires no additional configuration

Q10. Which statement about namespaces is correct?

A) Namespaces can be nested up to three levels deep B) A resource can belong to multiple namespaces if it is referenced from both C) Namespaces cannot be nested, and each resource is in exactly one namespace D) Namespaces cannot be nested, but cluster-scoped resources belong to all of them simultaneously

Q11. What does using a Secret instead of a ConfigMap give you?

A) Encryption of the data at rest, enabled by default B) Encryption in transit between the API server and the kubelet, which ConfigMaps do not get C) Automatic redaction, so that no principal with API access can read the contents D) A distinct object type and access-control surface, with no encryption by default

Q12. An application reads a setting from an environment variable populated by a ConfigMap. An operator edits the ConfigMap. What does the running application see?

A) The old value — the kubelet used the ConfigMap data when it launched the container B) The new value immediately C) The new value after a short kubelet resync interval D) The new value, but only if the application re-reads the environment variable rather than caching it at startup

Q13. [retrieval: ch2] ConfigMaps gained an immutability property in v1.19. Which statement about it is correct?

A) Immutability makes the ConfigMap behave like a container image layer: already-running containers keep the old contents, and newly launched containers get the new ones B) Once marked immutable, the change cannot be reverted and the data cannot be mutated — you can only delete and recreate the ConfigMap C) Immutability can be toggled off if the ConfigMap has no Pods consuming it D) Immutability applies to the data field but the `binaryData` field remains editable

Q14. You need to make a 4 MB reference data file available to a containerized application. What does the documentation direct you to do?

A) Use a volume, a separate database, or a file service — ConfigMap data cannot exceed 1 MiB B) Split it across four ConfigMaps and mount all four C) Store it in a Secret, which has no size limit D) Store it in a ConfigMap — the 1 MiB figure is a recommendation, not a limit

Q15. A Pod in namespace `team-a` needs configuration that already exists in a ConfigMap named `settings` in namespace `team-b`. What is your option?

A) Reference it as `team-b/settings` from the Pod's manifest B) Reference it by its fully qualified name, `settings.team-b.svc.cluster.local` C) Create a ConfigMap named `settings` in `team-a` — the Pod and the ConfigMap must be in the same namespace D) Grant the Pod's ServiceAccount read access to `team-b` and reference it normally

Q16. A Secret is created to hold an application's API key. The manifest sets no `type` field. What type does the Secret carry?

A) `kubernetes.io/basic-auth` — the type is inferred from the key names in data B) `Opaque` — arbitrary user-defined data, which is the default type C) `None` — `type` is a required field, so the object is rejected D) `kubernetes.io/service-account-token` — the default for any Secret not bound to a Pod

Q17. Which of these is a valid **set-based** selector?

A) `environment != production` B) `environment in (production, qa)` C) `environment = production, tier != frontend` D) `environment == production`

Q18. Given `matchLabels: {app: web}`, which `matchExpressions` entry is exactly equivalent?

A) `{key: app, operator: In, values: [web]}` B) `{key: app, operator: Equals, values: [web]}` C) `{key: app, operator: Exists, values: [web]}` D) `{key: app, operator: In, values: []}`

Q19. A controller is written to manage every object carrying a particular attribute. The attribute has been recorded as an annotation. What happens?

A) The controller finds them, but more slowly than if a label had been used B) The controller finds them only if the annotation key uses the `kubernetes.io/` prefix C) The controller finds them only within its own namespace D) The controller cannot select them — selectors operate over labels, and annotations are non-identifying by definition

Q20. Which of the following is a valid **user-defined** label?

A) key `kubernetes.io/tier`, value `frontend` B) key with a 120-character name segment and no prefix, value `frontend` C) key `example.com/tier`, value `frontend` D) key `tier`, value a 200-character description string

Q21. You are looking at a workload manifest that contains both a `spec.replicas` field and a `spec.selector.matchLabels` block. Which field determines *which* objects the workload manages, and which determines *how many* it maintains?

A) `replicas` determines which; `matchLabels` determines how many B) `matchLabels` determines which; `replicas` determines how many C) `matchLabels` determines both — `replicas` is reported in status only D) Neither — both are set by the system and reported in status

Answers and Explanations

Q1. Answer: A. Every read and write of cluster state goes through the kube-apiserver [source: k8s-docs-control-plane-node-communication-2026-08-24], and etcd is the consistent, highly-available key-value store used as Kubernetes' backing store for all cluster data [source: k8s-docs-cluster-architecture-2026-08-23].

- **B** confuses the kubelet's role: it runs containers on a node from PodSpecs provided to it, and it is not a store of record.
- **C** picks a real control-plane component with the wrong job; the scheduler watches for Pods with no assigned node and selects nodes for them [source: k8s-docs-cluster-architecture-2026-08-23]. It receives nothing from kubelet.
- **D** describes the shape of the misconception that controllers are messaged directly. They are not; they watch the API server.

Q2. Answer: C. A controller compares desired state against current state and acts on the difference [source: k8s-docs-controllers-2026-08-23]; the difference is what changed, so the change is what it responds to.

- **A** and **D** both imagine the loop as something with a lifecycle tied to your action. It has neither a start you trigger nor an end. It is described as a non-terminating loop that regulates the state of a system [source: k8s-docs-controllers-2026-08-23].
- **B** is the imperative instinct at its purest, and it is the single most useful wrong answer in this chapter. There is no such command, and needing one would defeat the entire architecture. Nobody tells the controller. The controller is already watching.

Q3. [retrieval: ch3] Answer: D. The documentation is unusually direct about this: *the Job controller does not run any Pods or containers itself. Instead, the Job controller tells the API server to create or remove Pods. Other components in the control plane act on the new information* [source: k8s-docs-controllers-2026-08-23].

- **A** is the intuitive picture of a controller as a worker. Built-in controllers manage state by interacting with the cluster API server; they are not execution engines.
- **B** invents a controller-to-kubelet channel. Chapter 3's hub-and-spoke pattern rules it out: all API usage from nodes (or the Pods they run) terminates at the API server, and none of the other control-plane components is designed to expose remote services [source: k8s-docs-control-plane-node-communication-2026-08-24].
- **C** is the one that survives longest for people who half-remember the architecture. Being a control-plane component does not grant direct access to the store. Ideally only the API server has access to etcd, since access to etcd is equivalent to root permission in the cluster [source: k8s-docs-etcd-access-control-2026-08-24].
- **D** is also the shape of §2 of this chapter: everything, including a controller's own writes, goes through the same door.

Q4. Answer: C. The four fields to set are `apiVersion`, `kind`, `metadata`, and `spec` (for objects that have a `spec` — which a Pod is) [source: [k8s-docs-objects-2026-08-23](#)]. `status` is supplied and updated by the Kubernetes system and its components; writing it accomplishes nothing.

- **A is a required field:** `apiVersion` names which version of the API you are using to create this object, and so selects the schema the rest of the document is parsed under.
- **B is a required field:** `kind` names the resource type, and it is what determines the internal shape of `spec`.
- **D is a required field:** `metadata` carries identity, the name, the UID, and the optional namespace, which is how the cluster tells this object from every other object of the same kind.

Q5. Answer: B. `apiVersion` is which version of the Kubernetes API you're using to create this object [source: [k8s-docs-objects-2026-08-23](#)], and it is one of the fields set per object.

- **A** conflates the API version of a resource type with the version of the cluster software. Those move independently; a resource's API version can remain stable across many cluster releases.
- **C** is the file-level assumption, and it fails because a single file can hold several objects of different kinds, each versioned independently.
- **D** is wrong: the four required fields are required for every object, built-in or custom.

Q6. Answer: D. `kubectl scale` updates the size of a workload [source: [k8s-docs-kubectl-overview-2026-08-23](#)]. It amends a number in the object's `spec`, and the control plane then continually and actively manages the actual state to match the desired state you supplied [source: [k8s-docs-objects-2026-08-23](#)]. The command changed a declaration; the loop did the work.

- **A** invents a direct command channel to the kubelet. Nothing in the object model works that way.
- **B** is the plausible-sounding version of the same error, and it is the one to reason your way out of: there is no "acting on running Pods" that skips the objects, because the objects *are* how the cluster knows what should be running.
- **C** is the over-correction this chapter's §1 precision note exists to prevent. Kubernetes accepts imperative commands all day; the documentation names imperative commands the recommended way to get started or to run a one-off task [source: [k8s-docs-object-management-2026-08-24](#)]. The claim is not "no imperatives." The claim is "the imperatives work by editing declarations."

Q7. Answer: A. Low-level resources such as nodes and persistent volumes are not in any namespace [source: [k8s-docs-namespaces-2026-08-23](#)].

- **B** and **D** are among the documentation's own examples of namespaced objects, and `ConfigMap` in particular is namespaced enough that §4's same-namespace rule exists because of it.
- **C** is namespaced too, and pointedly so. `ServiceAccounts` are bound to a namespace, and every namespace gets a `default` one on creation [source: [k8s-docs-service-accounts-2026-08-23](#)].

Q8. Answer: D. It is not necessary to use multiple namespaces to separate slightly different resources, such as different versions of the same software; use labels to distinguish resources within the same namespace [source: [k8s-docs-namespaces-2026-08-23](#)].

- **A** is the intuitive habit the documentation specifically corrects.
- **B** additionally fails on nesting: namespaces cannot be nested inside one another [source: k8s-docs-namespaces-2026-08-23].
- **C** misuses ResourceQuota, which divides cluster resources between namespaces rather than linking them.

Q9. Answer: C. kube-node-lease holds Lease objects associated with each node; node leases allow the kubelet to send heartbeats so that the control plane can detect node failure [source: k8s-docs-namespaces-2026-08-23].

- **A is the trap this item exists for.** The documentation says plainly that *the public aspect of this namespace is only a convention, not a requirement* [source: k8s-docs-namespaces-2026-08-23]. kube-public is readable by all clients including unauthenticated ones, but that is a fact about how the cluster is configured and what people put there, not an enforcement the namespace performs.
- **B** invents a fallback behavior. kube-system is the namespace for objects created by the Kubernetes system; an object whose manifest omits a namespace lands in whichever namespace the request context supplies — default on a fresh kubeconfig, the ServiceAccount's namespace when kubect1 runs inside the cluster [source: k8s-docs-kubect1-overview-2026-08-23] — and default exists precisely so that you can start using a new cluster without first creating one [source: k8s-docs-namespaces-2026-08-23].
- **D** inverts the documentation's advice: for a production cluster, consider *not* using the default namespace, make other namespaces and use those [source: k8s-docs-namespaces-2026-08-23].

Q10. Answer: C. Namespaces cannot be nested inside one another, and each Kubernetes resource can only be in one namespace [source: k8s-docs-namespaces-2026-08-23].

- **A** and **B** contradict those two rules directly.
- **D** is the subtle one and it is still wrong: cluster-scoped resources do not belong to every namespace. They belong to none. That distinction matters enormously for permissions, because "in all namespaces" and "in no namespace" imply very different things about who can be granted access [cross-bearing: see Ch 12].

Q11. Answer: D. Kubernetes Secrets are, by default, stored unencrypted in the API server's underlying data store; anyone with API access can retrieve or modify one, as can anyone with etcd access or the ability to create a Pod in the namespace [source: k8s-docs-secret-2026-08-23]. What the object type gives you is a place to apply the four recommended protections, not the protections themselves.

- **A** is the misconception this chapter exists to correct. Secret values are stored unencrypted by default, though they *can* be configured to be encrypted at rest [source: k8s-docs-secrets-good-practices-2026-08-24], which is a configuration you perform, not a default you inherit.
- **B** is wrong on the premise: API traffic is TLS-protected as a matter of cluster configuration [source: k8s-docs-cloud-native-security-2026-08-23], and that protection is not specific to Secrets.
- **C** contradicts the documented behavior outright. Anyone with API access can retrieve the Secret; base64 encoding of the stored value provides no additional confidentiality over plain text [source:

k8s-docs-secrets-good-practices-2026-08-24].

Q12. Answer: A. For the environment-variable path, as for command-and-args and read-only volume file paths, the kubelet uses the data from the ConfigMap when it launches the containers for a Pod [source: k8s-docs-configmap-2026-08-23]. Only the API-reading path subscribes to updates.

- **B** and **C** both assume a propagation mechanism that does not exist for this path.
- **D** is the most instructive miss, because half of it is a real belief: applications genuinely do differ in whether they cache configuration at startup. It does not help here. The environment the container was launched with is fixed at launch; re-reading it returns the same value the kubelet supplied. The application's diligence is irrelevant when the source never changes.

Q13. [retrieval: ch2] Answer: B. Once a ConfigMap is marked as immutable, it is not possible to revert this change nor to mutate the contents of the `data` or `binaryData` fields; you can only delete and recreate the ConfigMap [source: k8s-docs-configmap-2026-08-23].

- **A is the ch2 discrimination**, and it fails on Chapter 2's own rule. Image layers are immutable, but the way a change reaches a container is *not* by the object quietly serving new contents to new containers. It is by building a new image and recreating the container [source: k8s-docs-containers-2026-08-23]. An immutable ConfigMap works the same way: you delete and recreate, and consumers are relaunched. There is no dual-contents state in either mechanism.
- **C** is the most tempting wrong answer because it sounds like a reasonable safety valve. There is no such escape hatch.
- **D** is wrong: both fields are covered.

Q14. Answer: A. The data stored in a ConfigMap cannot exceed 1 MiB; for larger settings, consider mounting a volume or using a separate database or file service [source: k8s-docs-configmap-2026-08-23].

- **B** is a workaround for a limit that exists precisely because ConfigMaps are not designed to hold large chunks of data; it fights the design rather than following the guidance.
- **C** is wrong on its premise: a Secret is described as holding *a small amount* of sensitive data [source: k8s-docs-secret-2026-08-23], and is not the escape hatch for size.
- **D** misreads the limit as advisory.

Q15. Answer: C. The Pod and the ConfigMap must be in the same namespace [source: k8s-docs-configmap-2026-08-23]. The configuration has to exist in `team-a`.

- **A** invents a cross-namespace reference syntax that the field does not accept.
- **B** borrows the Service DNS form, which addresses Services over the network [source: k8s-docs-namespaces-2026-08-23]. A ConfigMap is not reached over the network by a Pod's config machinery, and these are two mechanisms that look similar and are not.
- **D** is the closest to plausible and still wrong: permission to read an object is not the same as the ability to reference it as a config source. Access control governs *who may*, not *what the field accepts*.

Q16. Answer: B. Opaque is arbitrary user-defined data and is the default type [source: k8s-docs-secret-2026-08-23]. The type field exists to facilitate programmatic handling of the secret data [source: k8s-api-ref-secret-v1-2026-08-24]: it tells consumers what shape to expect, and Opaque is the honest answer of "no particular shape."

- **A** invents inference from key names. Nothing infers a Secret's type; `kubernetes.io/basic-auth` is a type you choose deliberately when you are storing basic-authentication credentials.
- **C** is wrong twice over. type IS a real top-level Secret field — just not a required one; it defaults to Opaque [source: k8s-docs-secret-2026-08-23]. And a Secret is one of the types that carries data rather than a spec [source: k8s-api-ref-secret-v1-2026-08-24], so "the four required fields" is the wrong frame for this object entirely.
- **D** confuses a specific legacy type with a default. `kubernetes.io/service-account-token` is a ServiceAccount token, a legacy long-lived credential superseded since v1.22 by short-lived tokens from the TokenRequest API [source: k8s-docs-secret-2026-08-23] [source: k8s-docs-service-accounts-2026-08-23].

Q17. Answer: B. Set-based selectors use `in`, `notin`, and `exists` [source: k8s-docs-labels-selectors-2026-08-23].

- **A** and **D** are equality-based: `=`, `==`, and `!=` are the equality-based operators. `!=` in particular reads like a set operation and is not one.
- **C** is the item's best distractor, because the comma makes it *look* set-based. It is two equality-based requirements ANDed together; commas AND multiple requirements regardless of which selector type they use [source: k8s-docs-labels-selectors-2026-08-23]. The comma tells you nothing about the operator; the operators are what you read.

Q18. Answer: A. `matchLabels` is a map of {key, value} pairs equivalent to `matchExpressions` with operator `In` [source: k8s-docs-labels-selectors-2026-08-23].

- **B** uses an operator that does not exist in this vocabulary; the operators are `In`, `NotIn`, `Exists`, and `DoesNotExist`.
- **C** is a real operator with a different meaning: `Exists` tests for the key regardless of value, so it would also match `app: api`.
- **D** is `In` with an empty value list, which matches nothing.

Q19. Answer: D. Annotations attach arbitrary non-identifying metadata, and in contrast to labels are *not used to identify and select objects* [source: k8s-docs-annotations-2026-08-24]; selection is a label operation, and the label selector is the core grouping primitive [source: k8s-docs-labels-selectors-2026-08-23]. The fix is not to work around the selector. It is to record the attribute as a label, because wanting to select on it is what makes it identifying.

- **A** implies a performance difference where there is a capability difference.
- **B** inverts the prefix rule: `kubernetes.io/` and `k8s.io/` are reserved for core components in both labels and annotations, not required of user keys [source: k8s-docs-annotations-2026-08-24].

- **C** describes namespace scoping, which is real but is not what stops the controller here.

Q20. Answer: C. A label key may carry an optional prefix that is a DNS subdomain followed by a slash; `example.com/` is exactly that, and `tier` is a valid name segment [source: [k8s-docs-labels-selectors-2026-08-23](#)].

- **A is invalid for a user-defined label:** the `kubernetes.io/` and `k8s.io/` prefixes are reserved for Kubernetes core components [source: [k8s-docs-labels-selectors-2026-08-23](#)].
- **B is invalid on length:** the name segment must be 63 characters or less [source: [k8s-docs-labels-selectors-2026-08-23](#)]. The 253-character allowance applies to the *prefix*, which is a different segment, and mixing the two limits is the specific slip this option targets.
- **D is invalid on the value:** valid label values must be 63 characters or less [source: [k8s-docs-labels-selectors-2026-08-23](#)]. A 200-character description belongs in an annotation, whose values carry no character-set restriction and are bounded only by a 256 KiB total per object [source: [k8s-docs-annotations-2026-08-24](#)]. That is §5's rule in miniature: if it does not fit in a label value, it was probably never something you wanted to select on.

Q21. Answer: B. `matchLabels` is a selector, and the label selector is the core grouping primitive; it answers *which objects* [source: [k8s-docs-labels-selectors-2026-08-23](#)]. `replicas` sits in `spec` and states desired state: how many should exist, which is exactly the documentation's own worked example of a Deployment spec specifying three replicas [source: [k8s-docs-objects-2026-08-23](#)].

- **A** inverts the two.
 - **C** and **D** both misplace authorship: `replicas` is something you set in `spec`, not something reported to you in `status`. `status` reports how many *currently* exist, which is the other half of the pair and the thing the controller compares against.
-

Chapter Summary

Concept	Remember This
Kubernetes object	A persistent entity, a "record of intent." Create it and the system constantly works to ensure it exists [source: k8s-docs-objects-2026-08-23]
The four fields	apiVersion, kind, metadata, spec. Every object. Every time. Which API, which kind, which one, what it should look like
spec	What you want. You write it
status	What is. The system writes it. Every controller exists to close the distance between the two
Manifest	The file you write the object in. YAML by convention. Submit with <code>kubectl apply -f</code>
Three management techniques	Imperative commands (live objects, no history), imperative object configuration (files, you name the operation), declarative object configuration (directories, kubectl detects the operation). Use only one per object [source: k8s-docs-object-management-2026-08-24]
Name vs UID	A name is reusable after deletion; a UID is distinct across the cluster's whole lifetime and distinguishes historical occurrences of similar entities [source: k8s-docs-names-and-uids-2026-08-24]
Namespace	A scope for names. Unique within, not across. Cannot be nested. One namespace per resource
Initial namespaces	default (start without creating one), kube-system (system objects), kube-public (readable by all — by convention, not enforcement), kube-node-lease (node heartbeat Leases)
Cluster-scoped	Nodes, PersistentVolumes, StorageClasses, and namespace objects themselves. <code>kubectl api-resources --namespaced=false</code> settles it
ConfigMap	Non-confidential key-value data, ≤1 MiB, same namespace as the Pod. Four consumption paths; only the API-reading one sees updates. Immutability is irreversible
Secret	Same shape, different intent and treatment. Values base64-encoded; stored <i>unencrypted</i> by default. Readable by anyone with API access, etcd access, or Pod-creation rights in the namespace
Base64	An encoding, not an encryption method. No additional confidentiality over plain text [source: k8s-docs-secrets-good-practices-2026-08-24]
Opaque	The default Secret type: arbitrary user-defined data
dockerconfig json	The Secret type behind <code>imagePullSecrets</code> , a serialized <code>~/.docker/config.json</code>
Label	Key/value identifying attribute. Selectable, and constrained (values ≤63 chars). Means nothing to the core system, which is why it works everywhere
Label selector	The core grouping primitive in Kubernetes. Equality-based (<code>=</code> , <code>==</code> , <code>!=</code>) and set-based (<code>in</code> , <code>notin</code> , <code>exists</code>), ANDed with commas
matchLabels	Exactly equivalent to <code>matchExpressions</code> with operator <code>In</code>
Annotation	Arbitrary non-identifying metadata. Unconstrained values (≤256 KiB total per object), not selectable. If you might search on it, it's a label

⚓ Safe Harbor

Voyage Progress: 🏠 → 🌊 → ⚓ — Chapter 4 of 20 complete. You have crossed from *watching the system* to *writing to it*, which is the largest single step in this book.

You can now read a manifest you have never seen. You know which of its fields you author and which one is a report. You know where its name has to be unique and which resources have no name-scope at all. You know how to describe a set of objects rather than list one. And you know the exact thing a Secret does and does not do, which is a piece of knowledge something plenty of working practitioners, in my experience, have wrong.

The Voyage Ahead

You have written declarations for five kinds of thing. You have not yet written one for the thing that actually runs.

Chapter 5 is about the Pod: the object that represents a set of one or more running containers on your cluster, and the unit that everything else in Kubernetes ultimately arranges [source: k8s-docs-workloads-2026-08-23]. Everything you learned here applies to it unchanged: four fields, a spec you write, a status you read. But a Pod's status has something the objects in this chapter did not: a **phase**, a small vocabulary of words describing where in its short life the Pod currently is. And that vocabulary is the first thing you look at when something is wrong.

There is a second thing to watch for, and §2 already handed you the tool for it. A Pod is described in the documentation as *relatively ephemeral rather than durable*, and it is never rescheduled to a different node. If the node it is on dies, it is not moved; it is replaced by a new, near-identical Pod with **a different UID** [source: k8s-docs-pod-lifecycle-2026-08-23], which is exactly what UIDs are for: distinguishing between historical occurrences of similar entities [source: k8s-docs-names-and-uids-2026-08-24]. Same name, different object, and the cluster knows the difference even when you don't.

Hold that against what you learned in §1. If the thing that runs your application is designed to be replaced rather than repaired, then a declaration that names a specific Pod is a declaration about something disposable, and something else must be holding the intent that survives it.

Chapter 5 introduces the disposable thing. Chapter 6 introduces what holds the intent.

■ "You have stopped issuing orders. Everything after this is learning what reads them."

Chapter 5: The Smallest Vessel

"A Pod is not a container, and that distinction is worth points"

Exam domain: Kubernetes Fundamentals (44% of the exam) — competency: Kubernetes Core Concepts
[source: cncf-kcna-certification-page-2026-08-23] [source: cncf-kcna-curriculum-pdf-2026-08-23] |

Estimated share of the exam: ~7% (authored allocation — CNCF publishes domain weights, not competency weights [source: cncf-kcna-curriculum-pdf-2026-08-23]; see front matter) | Complexity: Mixed | Novelty: Moderate

Attention Budget

Total time: ~95 minutes | Recommended: Split across 2 sessions

Section	Time	Attention Cost	Best Time to Study
\$1 The Pod as the Unit of Scheduling	10 min	Medium	When alert
\$2 More Than One Container Aboard	5 min	Low	Anytime
\$3 Everything That Must Happen First	8 min	Medium	When alert
🕒 Taking Your Bearings #1	6 min	Medium	After a brief pause
\$4 Scheduled Once, Replaced Never	5 min	Low	Anytime
\$5 Pod Phases and Container States	15 min	High	Peak attention
🕒 Taking Your Bearings #2	7 min	Medium	After a short break
\$6 A Pod's Identity	4 min	Low	Anytime
\$7 Three Probes, Three Jobs	12 min	Medium-high	When alert
\$8 What a Pod Is Owed	12 min	High	Peak attention
🕒 Taking Your Bearings #3	7 min	Medium	After a short break
\$9 The Smallest Deployable Unit	4 min	Low	Anytime

Attention Cost Key:

- **Low:** Concrete, familiar concepts — study anytime
- **Medium:** New concepts requiring focus — study when alert
- **High:** Abstract or dense material — study at peak attention

Recommended split point: after ☹ Taking Your Bearings #2. Sections 1–5 are *what a Pod is and what happens to it*; sections 6–8 are *what the kubelet does for it*. That is the natural seam.

If you only have 15 minutes: read §5 and take Bearings #2. Phase-versus-state is the highest-leverage distinction in this chapter, and Chapter 13's entire troubleshooting method is built on top of it.

“The smallest thing you can name is the smallest thing you can command.” — Lodestar Ledgers

🔊 Soundings

Before reading this chapter, try these questions. Your score determines how to approach the content. No shame in any score; it just points you at a different reading strategy. Six of these test what you already know from outside Kubernetes; two retrieve material from earlier chapters.

1. Two processes need to talk to each other over the network. What does each one minimally need in order to be reachable, and what changes if they happen to be running on the same machine?
2. Two programs must read and write files in the same directory. Outside Kubernetes, how do you arrange that?
3. A service must not begin accepting traffic until a database migration has finished. Using tools you've worked with before — entrypoint scripts, systemd unit ordering, CI job dependencies — how would you enforce that ordering?
4. "The process is running" and "the process can do its job" are not the same claim. Describe a concrete situation where the first is true and the second is false.
5. A load balancer health-checks its backends. One backend stops answering the health check. What does the load balancer do? And, just as importantly, what does it *not* do?
6. In any system you've capacity-planned — a JVM heap, a cgroup, a cloud instance type, a database connection pool — what is the difference between *reserving* capacity and *capping* it?
7. **[retrieval: ch4]** A Kubernetes object has two nested fields that govern its configuration. One you write; one the system maintains. Name both, and say which one reports what is actually true right now.
8. **[retrieval: ch2]** A container will not start because its image could not be pulled. Chapter 2 gave this failure a name. What is that name, and what did Chapter 2 say it would eventually be reported as?

Click for answers + reading strategy

1. **An address and a port.** Reachability requires something to send packets to. On the same machine, they can skip the network entirely and use `localhost` (the loopback interface), because they share one network stack.
2. **A shared filesystem path:** a bind mount, a shared volume, a directory both have permission to open. Both processes must be able to see the same underlying storage.
3. **Any ordering primitive that blocks the second thing until the first succeeds:** an entrypoint script that runs the migration then execs the server; a systemd `After=` plus `Type=oneshot`; a CI job that depends on a prior stage. The common shape: something must *finish* before something else *starts*.
4. **Many valid answers.** A JVM that's alive but stuck in a full GC pause. A web server that's accepting connections but whose database connection pool is exhausted. A process that started but hasn't finished loading a 4 GB model into memory. The pattern: the process exists; it can't serve.
5. **It stops sending new traffic to that backend. It does not kill the backend.** A load balancer removes an unhealthy member from its rotation; it has no authority to restart the process behind it. Those are two different jobs performed by two different systems.
6. **A reservation guarantees you a floor; a cap defines a ceiling.** A reservation is a claim on capacity made in advance so the planner accounts for you. A cap is enforcement at runtime that stops you exceeding an amount. Systems can have one, both, or neither.
7. **spec and status.** You write `spec`, the desired state. The system supplies and updates `status`: the current state, what is actually true [source: `k8s-docs-objects-2026-08-23`]. If you missed this, re-read Chapter 4 §2 before you reach §5 of this chapter.
8. `ImagePullBackOff` — and Chapter 2 said it is reported as a container in the **Waiting** state. If you missed this, re-read Chapter 2 §6 before §5.

If you got 6+ right: Skim this chapter. Focus on the ☆ Fixed Points and the ⚠ Navigational Hazards blocks, then go straight to the 🧭 Taking Your Bearings checkpoints. Your instincts about isolation, ordering, and health checks are correct. What this chapter gives you is Kubernetes' specific vocabulary for them, plus the two or three places where Kubernetes' answer is not the one you'd guess.

If you got 3–5 right: Read at normal pace. The material is well within reach; this chapter is calibrated for you.

If you got 0–2 right: Read carefully, and take the sections in order — several of them depend on the one before. **If questions 7 and 8 were among your misses specifically, go back and re-read Chapter 4 §2 and Chapter 2 §6 before you start §5.** Those two are the published promises this chapter collects, and §5 will read as arbitrary vocabulary without them.

Why This Chapter Matters

You already know this word. You've seen it since Chapter 2, where it arrived with an IOU attached: *containers are not the unit Kubernetes schedules — something wraps them, and we'll name it in Chapter 5.* In the meantime, most readers form the obvious assumption. **Pod is Kubernetes' word for container.**

It isn't, and the way it isn't matters more than a vocabulary quibble. That assumption is wrong about IP addresses. It's wrong about what a Service selects. It's wrong about what `restartPolicy` applies to. It's wrong about what a phase describes, and it's wrong about what gets replaced when something fails. Each of those is a place where the wrong mental model produces a confidently wrong answer: on the exam, and in a terminal at two in the morning with a pager still buzzing. By the end of this chapter you'll have a list of seven things that would each be wrong under the container-equals-Pod assumption, and you'll see that they all come from one design decision.

Here is the shift this chapter delivers. Chapters 3 and 4 gave you a system to look at and something to write. Chapter 5 gives you the first thing you will ever **read under pressure**. Nobody looks up Pod phases when everything is fine. You look at them at the moment something has stopped working, and the reason experienced practitioners are fast in that moment isn't that they've memorized more commands. It's that they know which of two vocabularies they're looking at, and what each one can and cannot tell them. That's the difference between someone who can describe infrastructure and someone who can read what infrastructure is telling them.

This chapter's material is also the most retrieved in the book. Requests and limits come back in scheduling, in troubleshooting, in autoscaling, and in metrics: four separate chapters, each assuming you already have the model. Phase-versus-container-state is Chapter 13's diagnostic method. The shared network namespace is the entire premise of Chapter 9's argument for why Services have to exist. A reader who leaves here able to recite five phase names but unable to say why the Pod has an IP and the container does not will re-learn this chapter four more times, each time under worse conditions. That's not a threat; it's just how the book is built.

Dead Reckoning: A Pod is a Kubernetes object that represents a set of one or more running containers on your cluster [source: k8s-docs-workloads-2026-08-23]. It is the unit Kubernetes schedules: you hand Kubernetes a Pod, not a container [source: k8s-docs-kube-scheduler-2026-08-23]. The containers inside it are co-located and co-scheduled on the same node [source: k8s-docs-containers-2026-08-23]. They share a network namespace [source: k8s-docs-network-model-2026-08-23]. They can share storage [source: k8s-docs-volumes-2026-08-23]. Every other fact in this chapter follows from that.

What You'll Learn

By the end of this chapter, you'll be able to:

- **Explain** why Kubernetes schedules Pods rather than containers, and name the two things every container in a Pod shares.
- **Distinguish** a Pod's phase from a container's state, and say which one a crash-looping application is reported under.
- **Predict** what happens when each of the three probes fails, and identify which one silently removes a Pod from service without restarting anything.
- **Choose** between a request and a limit for a given requirement, and say which component acts on each.
- **Trace** a Pod from creation through failure to replacement, and explain why "rescheduled" is the wrong word for what happens.
- **Identify** what a Pod is to the API server when it has been given no identity at all (it has one anyway).

You'll also stop saying "container" when you mean "Pod," which is a smaller change than it sounds and closes off a whole family of mistakes people make on this exam.

§1 — The Pod as the Unit of Scheduling

Chapter 2 left you with a promise: containers are not the unit Kubernetes schedules; something wraps them [*cross-bearing: see Ch 2 §1 — containers are not the schedulable unit*]. Here is the wrapper.

A Pod represents a set of one or more running containers on your cluster [source: k8s-docs-workloads-2026-08-23] — the documentation's own superlative is that Pods are *the smallest deployable units of computing that you can create and manage in Kubernetes* [source: k8s-docs-pods-2026-08-24]. It is the thing you hand Kubernetes when you want something run: the scheduler watches for newly created **Pods** with no assigned node and finds a node for each one [source: k8s-docs-kube-scheduler-2026-08-23]. You do not hand Kubernetes a container and ask for it to be placed. Chapter 2 already gave you the phrase that follows from this: containers in a Pod are **co-located and co-scheduled** to run on the same node [source: k8s-docs-containers-2026-08-23]. Every node in a cluster runs the containers that form the Pods assigned to that node. The assignment happens at Pod granularity, and the containers come along.

That costs something. Everything in a Pod lands on one machine, scales as one thing, and dies as one thing. So the interesting question isn't *what* a Pod is. It's *why the wrapper exists at all*.

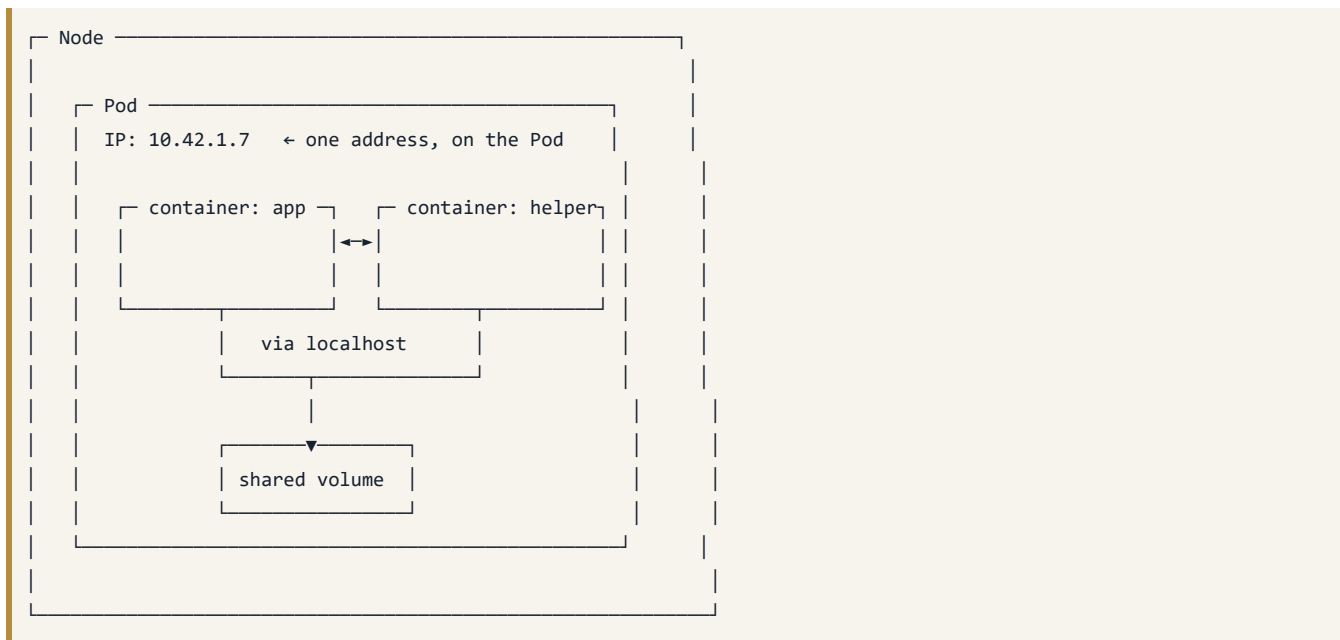
What every container in a Pod shares

Here is the answer, and it's worth reading slowly, because it's the load-bearing fact of the whole chapter.

Each Pod gets its own unique cluster-wide IP address. A Pod has a private network namespace which is shared by all of the containers within it. Processes running in different containers in the same Pod can communicate with each other over localhost [source: k8s-docs-network-model-2026-08-23].

Read that as an explanation rather than a feature list. Suppose you want two containers to behave the way two processes on one host behave: able to reach each other instantly, without service discovery, without a network hop, without either one needing to know the other's address. There is exactly one way to give them that. Put them in the same network namespace. And a network namespace is not something you can hand out per-container-but-shared and still call the container an independent schedulable thing. The moment two containers share a network namespace, they have to be placed together, started together, and torn down together. They have become one unit.

So the Pod is not a container with extra fields. **The Pod is the shared context, and the containers live inside it.** One hull, one address, one berth on the node. Whatever you stow inside travels together or not at all.



Note where the IP address is attached in that figure. It is bound to the Pod boundary, not to either container. That placement is the pedagogy, not decoration.

The second half of the shared context is **storage**. Containers in a Pod can share volumes, which is how two processes in one Pod read and write the same files. The documentation names this as one of the two problems volumes exist to solve, and says plainly that all containers in a Pod can read and write the same files in a shared `emptyDir` volume [source: k8s-docs-volumes-2026-08-23]. We name it here and leave it: the volume types, and what makes a volume outlive the Pod, are Chapter 11's material [cross-bearing: see Ch 11 §1 — volume types and lifetimes].

The PodSpec, finally

Chapter 3 described the kubelet's job in a sentence that contained an unexplained noun: the kubelet "takes a set of PodSpecs that are provided through various mechanisms and ensures that the containers described in those PodSpecs are running and healthy" [source: k8s-docs-cluster-architecture-2026-08-

23]. You were told PodSpecs would get their proper treatment here [*cross-bearing: see Ch 3 §3 — the kubelet and what it ensures*].

There's less to it than the name suggests, because Chapter 4 already taught it. A Pod is an object like any other: `apiVersion`, `kind`, `metadata`, `spec`, plus a `status` the system maintains [source: `k8s-docs-objects-2026-08-23`]. Read "PodSpec" as **the spec field of a Pod**. It's what you write; it lists the containers, their images, and everything else in this chapter. That's the whole connection, and it's worth exactly one sentence. You don't need it developed; you need it *joined up*.

The second half of Chapter 3's sentence — "running **and healthy**" — is a bigger promise, and §7 pays it.

☆ **Fixed Point:** The **Pod**, not the container, is the unit of scheduling. A Pod gets one IP address, shared by every container inside it, and those containers reach each other over `localhost`.

🔖 **Snag:** Each container in a Pod does **not** get its own IP address. The Pod gets one; all its containers share it. This is the easiest carry-over error to make from single-container Docker experience, where "one container, one IP" was a safe assumption. In Kubernetes it is not.

That one fact is the premise of Chapter 9. When you get to Services, the entire argument for why they must exist rests on the Pod having an IP that changes when the Pod is replaced [*cross-bearing: see Ch 9 — why a Service is necessary*].

..1 §2 — More Than One Container Aboard

A Pod can hold more than one container. The question is when it should. Not everything that travels together belongs in the same hold.

Pods are used in two main ways [source: `k8s-docs-pods-2026-08-24`]. Overwhelmingly the most common is **one container per Pod**: the Pod is a thin wrapper around a single container, and that is what you should reach for by default. The other is **multiple tightly-coupled containers** that need to share resources, meaning a main application container plus one or more helpers that supplement or consume it.

The decision rule is short, and it falls straight out of §1. There are exactly two mechanisms that make containers in one Pod tightly coupled:

1. **They reach each other over `localhost`**, because they share the Pod's network namespace [source: `k8s-docs-network-model-2026-08-23`].
2. **They read and write the same files**, because they share a volume [source: `k8s-docs-volumes-2026-08-23`].

🔒 **Worth Securing:** If two containers don't need `localhost` or a shared volume, they don't need to be one Pod. That's the whole test. Everything else — "they belong to the same team," "they're part of the same product," "it's simpler to deploy" — is not a coupling requirement, it's a naming convention.

The helper container in a multi-container Pod has a name you'll meet constantly: the **sidecar**. A log-shipping agent that reads files the app writes to a shared volume; the documented cluster-logging pattern is exactly this, "a sidecar container with a logging agent configured to pick up logs from an application container" [source: [k8s-docs-logging-architecture-2026-08-23](#)]. A proxy that intercepts the app's network traffic on `localhost`, which is what a service mesh does when it "deploys an Envoy proxy alongside each pod" [source: [istio-service-mesh-2026-08-23](#)]. A credential-refreshing helper that rewrites a token file the app reads. In every case the sidecar exists to do something *for* the main container, using one of exactly those two coupling mechanisms.

You'll meet the sidecar again in Chapter 17, where a service mesh deploys a proxy alongside each Pod as its data plane [*cross-bearing: see Ch 17 — the mesh data plane*]. That's Chapter 17's material; here, the word is enough.

Under the hood, modern Kubernetes implements sidecars as a special case of init container — one with `restartPolicy: Always`, which keeps running after Pod startup instead of exiting; the mechanism is on by default since v1.29 [source: [k8s-docs-sidecar-containers-2026-08-24](#)]. That sentence will make full sense a section from now; file it for §3.

🔪 **Snag:** A Pod is not a small virtual machine. The instinct to put a web server, a database, and a cache into one Pod because "they're one application" is exactly what §1's co-scheduling guarantee makes *possible* — and exactly what makes it a mistake. Everything in a Pod scales together, fails together, and is replaced together, whether or not that's what you wanted. Three components with three different scaling profiles want three Pods.

One practical consequence worth banking: once a Pod holds more than one container, several commands stop being unambiguous. Asking for "the logs" of a two-container Pod is an incomplete request. `kubectl logs <pod>` prints a container's logs, and for a multi-container Pod you have to add `-c <container>` [source: [k8s-docs-logging-architecture-2026-08-23](#)]. That's Chapter 13's problem to solve, not this section's [*cross-bearing: see Ch 13 §3 — reading logs from a multi-container Pod*].

..1 §3 — Everything That Must Happen First

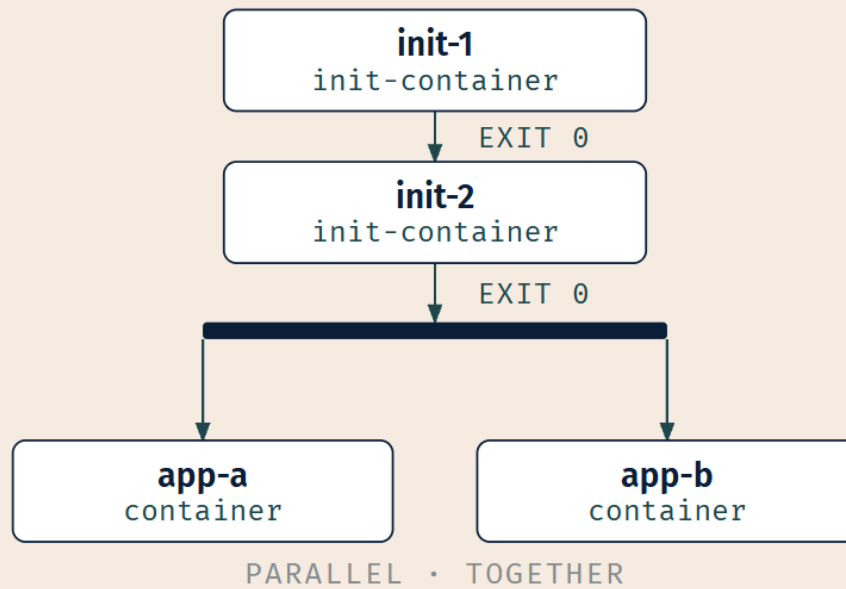
Soundings question 3 asked how you'd stop a service from accepting traffic until a migration had finished. Whatever you answered — entrypoint script, systemd ordering, CI stage dependency — you reached for the same shape: *something must finish before something else starts*. Kubernetes has a first-class object for that shape, and it's called an **init container**.

The mechanics are simple. The semantics are what get tested.

Init containers run before the app containers, in the order they are declared, and each must run to completion successfully before the next one starts [source: k8s-docs-init-containers-2026-08-24]. Only when all of them have succeeded does the kubelet start the Pod's app containers.

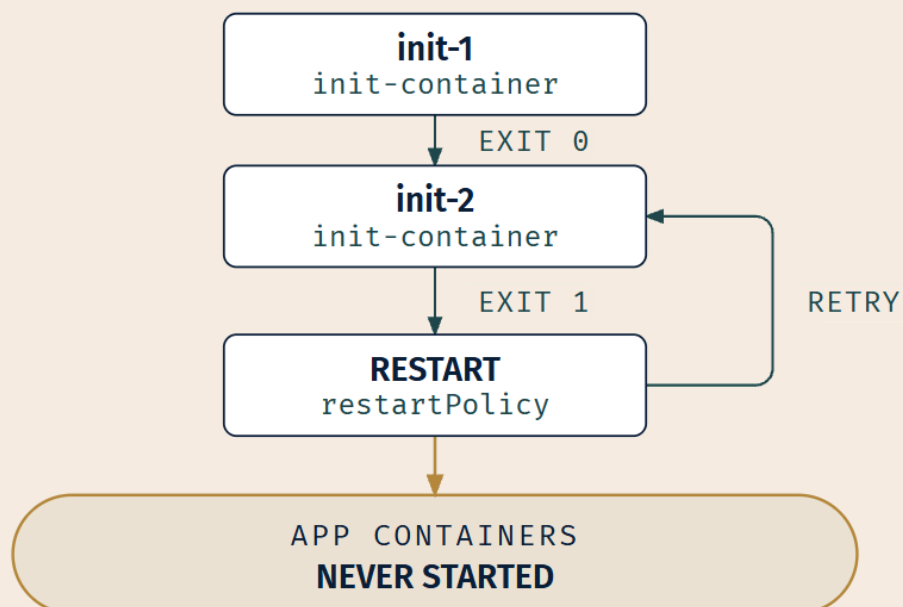
SUCCESS PATH

SEQUENTIAL · ONE AT A TIME



FAILURE PATH

diverges from the success path at init-2's exit code



LEGEND

□ Step (container)

→ Retry loop

▬ Fork · parallel start

▭ Terminal state · focal

Two things in that figure carry the whole section. The init containers are strictly **sequential**: one at a time, each waiting for the previous one's successful exit. The app containers are **parallel**: they all start together once the init sequence completes. That contrast is the fact. One at a time down the gangway, then everyone aboard at once.

When an init container fails

This is the part the exam cares about. If an init container fails, the kubelet restarts it, and the Pod's `restartPolicy` governs whether it gets retried at all. Which means:

- With the default policy, a Pod with a broken init container sits there retrying, and **never progresses to its app containers**.
- With a `restartPolicy` of `Never`, the Pod fails outright.

Those two observable outcomes are what matter here. That sentence also leans on a field this chapter hasn't defined yet, and it does so deliberately. `restartPolicy` is §5's material, and stating the dependency plainly beats pretending the sections are independent [*cross-bearing: see Ch 5 §5 — `restartPolicy` and the restart backoff*]. When you get there, come back and the failure behavior will click into place.

The one axis that generates the rest

Init containers differ from app containers on a single axis, and everything else follows from it: **init containers are expected to exit; app containers are expected to keep running**.

That's why they run in sequence. A thing that exits can be waited on. It's why "success" means exit status 0. And it's why classic init containers don't carry the probes §7 is about: probes answer questions about a long-running process, and an init container is not one.

☆ **Fixed Point:** Init containers run **in the order declared, to successful completion, all of them, before any app container starts**. If one fails, the Pod does not proceed.

🔊 **Mnemonic:** *In order, to completion, all of them, then the app*. Four beats, one line. Say it once and you have the section.

Debugging a Pod stuck behind a failing init container falls under **Troubleshooting**, one of the twelve named KCNA competencies [source: [cncf-kcna-curriculum-pdf-2026-08-23](#)], and the method is Chapter 16's [*cross-bearing: see Ch 16 §2 — debugging init containers*]. What you need here is the model of what *should* happen, so you can recognize when it hasn't.

🕒 Taking Your Bearings #1: What a Pod Is

Five questions covering §1–§3.

1. Two containers are running in the same Pod. How many IP addresses are involved, and how do the two containers reach each other?
2. Someone proposes putting a web server and its database in a single Pod, on the grounds that they are "one application." Give the strongest argument against, in one sentence.
3. An init container exits with status 1. What does the Pod do next, and what has *not* happened yet?
4. A Pod declares three init containers. Under what circumstances does the third one run?
5. **[retrieval: ch3]** Chapter 3 said the kubelet ensures that the containers described in PodSpecs are running and healthy. Now that you know what a Pod is: what object is the kubelet reading, and what is it comparing that object against?

Answers with Explanations:

1. One IP address, attached to the Pod. The two containers reach each other over localhost.

Every container in a Pod shares the Pod's network namespace, and the Pod gets a single unique cluster-wide IP [source: k8s-docs-network-model-2026-08-23].

Why the wrong answers are wrong:

- **"Two IPs, one per container"** is the misconception this chapter exists partly to correct. It comes from single-container Docker, where each container did get its own address, and it survives because nothing in the word "Pod" contradicts it. It will cost you points here and it will make Chapter 9 incomprehensible: the entire reason Services exist is that *the Pod* has an address that changes. If you thought two, stop and re-read §1 before continuing.
- **"They reach each other through a Service"** describes how containers in *different* Pods communicate. Containers in the same Pod need no such thing; a Service in front of your own sidecar is a network hop that buys nothing.

2. They share fate. They scale together, fail together, and are replaced together, and a web server and a database almost never want any of those three to be true at once. You cannot scale the web tier to five replicas without also running five databases.

A weaker but still correct answer: they don't need either coupling mechanism. A web server talks to its database over a network address; it doesn't need localhost or a shared volume. By §2's rule, that alone settles it.

Why two tempting answers are wrong:

- **"They'd collide on ports."** Sometimes true: they share one network namespace, so two processes both wanting port 8080 do conflict. But it's incidental, not structural. Renumber the ports and the

objection evaporates while the real problem (shared fate) is untouched. An objection you can configure your way out of is not the strongest one available.

- **"A Pod should only run one process."** This is Docker-era doctrine, and §2 explicitly does not endorse it. Multi-container Pods are a supported, useful pattern. The test isn't *how many* processes; it's whether `localhost` or a shared volume is genuinely *needed*.

3. The kubelet restarts the init container, subject to the Pod's `restartPolicy`. The app containers have not started, and won't, until every init container has succeeded.

The second half is the part people drop. A failing init container isn't a partial start; it's a full stop. Nothing downstream has begun.

4. Only after the first two have each run to completion successfully.

Why the wrong answers are wrong:

- **"They run in parallel"** confuses init containers with app containers. App containers start together; init containers strictly do not.
- **"In any order"** is the one that feels defensible if you think of them as a set of prerequisites rather than a sequence. They are declared as an ordered list and executed as one. Declaration order is execution order.

5. The kubelet is reading the Pod's `spec` — the `PodSpec` — and comparing it against the actual state of the containers on its node. [source: [k8s-docs-cluster-architecture-2026-08-23](#)] [source: [k8s-docs-objects-2026-08-23](#)]

This is Chapter 3's control-loop pattern operating at Pod scope: read the declared desired state, observe reality, act to close the gap [*cross-bearing: see Ch 3 §6 — controllers and control loops*]. Chapter 3's sentence about the kubelet was, all along, a description of a reconciliation loop with a `PodSpec` as its input.

Why two likely answers are wrong:

- **"It reads the Deployment."** Nothing you've read so far rules this out, which is exactly why it needs ruling out now. The kubelet's input is the Pod. A Deployment is a workload resource that *causes Pods to exist*, and Chapter 6 will show you where it sits. The kubelet never sees it.
- **"It compares the spec against etcd."** The kubelet never talks to etcd. Kubernetes has a "hub-and-spoke" API pattern: all API usage from nodes terminates at the API server, and none of the other control plane components are designed to expose remote services [source: [k8s-docs-control-plane-node-communication-2026-08-24](#)]. Access to etcd is equivalent to root permission in the cluster, so ideally only the API server has it [source: [k8s-docs-etcd-access-control-2026-08-24](#)]. The kubelet compares the spec against *the containers actually running on its own node*.

How'd you do?

- **5/5, or 4 with a clean miss on item 5.** You own §1–§3. Continue; §4 is short and §5 is where the chapter earns its attention budget.
- **3–4 correct.** Solid. Review the items you missed, then continue. If item 1 was one of them, re-read §1's "What every container in a Pod shares" first. That one fact is load-bearing for §5, §7, §8, and all of Chapter 9.
- **0–2 correct.** Don't push on to §5 yet. Go back to §1's "What every container in a Pod shares" subsection and §3's failure behavior, about fifteen minutes of re-reading, then take this checkpoint again. §5 is the densest section in the chapter and it assumes all of this.

Checkpoint: You've Now Mastered

✓ Why the Pod, not the container, is the schedulable unit ✓ What every container in a Pod shares — and why that sharing forces co-scheduling ✓ The two-mechanism test for whether containers belong in one Pod ✓ Init container ordering, completion semantics, and failure behavior □ What happens to a Pod over its lifetime (next) □ How to read what a Pod is telling you (§5 — the chapter's centerpiece)

§4 — Scheduled Once, Replaced Never

Chapter 4 closed by telling you what was coming: a Pod's *status* has a phase, and a Pod is never rescheduled but *replaced*, with a different UID. "*Chapter 5 introduces the disposable thing*" [cross-bearing: see Ch 4 — *The Voyage Ahead*, which pointed here]. Here it is.

The lifetime, in order:

- A Pod is **created** and assigned a unique **UID** [source: k8s-docs-pod-lifecycle-2026-08-23].
- It is **scheduled once in its lifetime** to a specific node, where it remains until termination or deletion [source: k8s-docs-pod-lifecycle-2026-08-23].
- It is **never "rescheduled" to a different node**. Instead, it is replaced by a new, near-identical Pod **with a different UID** [source: k8s-docs-pod-lifecycle-2026-08-23].
- If the node dies, the Pods running on it are **marked for deletion after a timeout** [source: k8s-docs-pod-lifecycle-2026-08-23].
- Pods **do not survive** evictions due to lack of resources or node maintenance [source: k8s-docs-pod-lifecycle-2026-08-23].

The documentation's own summary is that Pods are "relatively ephemeral (rather than durable) entities" [source: k8s-docs-pod-lifecycle-2026-08-23]. That's the word to hold onto.

📌 **Snag:** A Pod on a failed node is **not rescheduled onto a healthy node**. It is deleted, and something creates a new one. The word "rescheduled" implies the same object moving, which will lead you to the wrong answer on questions about UUIDs, about identity, and about why StatefulSets are different [cross-bearing: see Ch 6 — StatefulSets]. Kubernetes does not move Pods. It replaces them.

🔒 **Worth Securing:** The replacement Pod can have the same *name* and still be a different object. Chapter 4 taught that a UUID is "intended to distinguish between historical occurrences of similar entities" [source: k8s-docs-names-and-uuids-2026-08-24] — this is precisely that case. Same name, different UUID, different object, and the cluster knows the difference even when the human reading `kubectl get pods` doesn't.

Why this forces something else to exist

Here's the consequence, and it's the reason this short section exists at all.

If the thing that runs your application is designed to be **replaced rather than repaired**, then something else has to be holding the intent that survives the replacement. The Pod cannot recreate itself; it's gone. Something outside it has to know that three replicas were wanted, notice that only two exist, and create a third.

That something is a **workload resource**. Kubernetes provides several built-in ones, and their job is to manage a set of Pods on your behalf, making sure the right number of the right kind of Pod are running to match the state you specified [source: k8s-docs-workloads-2026-08-23]. Higher-level controllers create the replacement Pods [source: k8s-docs-pod-lifecycle-2026-08-23].

That's Chapter 6, and Chapter 4 already pointed you at it [cross-bearing: see Ch 6 §1 — the resource that holds the surviving intent].

The same instinct, one level up

You've met this idea before. Chapter 2 taught that containers are intended to be immutable: you don't change the code of a running container, you build a new image with the change and recreate the container from it [source: k8s-docs-containers-2026-08-23].

A Pod is that instinct one level up. You don't repair a failed Pod; you get a new one. Replace, don't repair: at the image layer and at the Pod layer both. A hull that cannot be patched underway has to be a hull you are willing to lose, and the whole design follows from accepting that. Noticing that it's the *same* conviction expressed twice is worth more than memorizing either instance.

That replacement story has a mechanical half, and it is exam-relevant: Pods terminate *gracefully*, not instantly. Because Pods represent processes, the design aim is that they get a chance to shut down cleanly rather than being violently killed with no chance to clean up [source: k8s-docs-pod-termination-2026-08-24]. When you delete a Pod, the kubelet runs any `preStop` hook a container defines, then asks the runtime to stop each container with a `TERM` signal. A countdown runs alongside:

terminationGracePeriodSeconds, **30 seconds by default**. If containers are still running when it expires, they get KILL, and only then is the object removed from the API [source: k8s-docs-pod-termination-2026-08-24]. A workload that exits promptly on TERM is what makes replace-don't-repair cheap in practice — Chapter 15 will hand you the methodology's word for it, *disposability* [cross-bearing: see Ch 15 — the twelve factors].

§5 — Pod Phases and Container States

This is the densest section in the chapter and the one the rest of the book leans on hardest. It is also where Chapter 2's second promise comes due.

There are **two** vocabularies here, at **two** different scopes. Two instruments, two scales; read one as though it were the other and you will be confidently lost. Almost every mistake people make with Pod status comes from exactly that. So take them one at a time, then look at the relationship.

Leg one: the Pod's phase

Chapter 4 told you a Pod's status carries a phase. Here it is, with five possible values [source: k8s-docs-pod-lifecycle-2026-08-23]:

- **Pending** — the Pod has been accepted by the cluster, but one or more of its containers has not been set up and made ready to run. This *includes* time spent waiting to be scheduled **and** time spent downloading container images over the network.
- **Running** — the Pod has been bound to a node, and all of the containers have been created. At least one container is still running, **or is in the process of starting or restarting**.
- **Succeeded** — all containers in the Pod have terminated in success, and will not be restarted.
- **Failed** — all containers have terminated, and at least one terminated in failure: it either exited with non-zero status or was terminated by the system, and is not set for automatic restarting.
- **Unknown** — for some reason the state of the Pod could not be obtained. This typically occurs due to an error communicating with the node where the Pod should be running.

Read the definitions of Pending and Running again. Both of them are broader than their names suggest, and both of those breadths are tested.

Leg two: the container's state

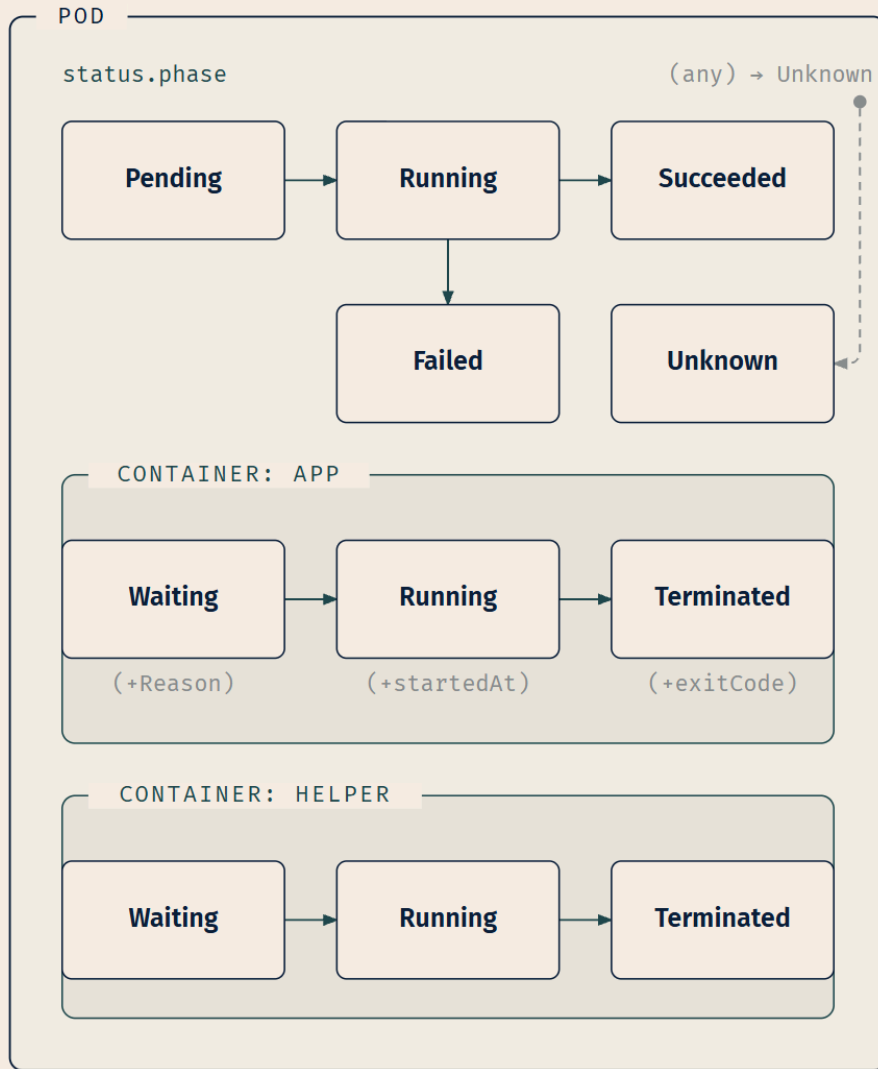
Now the second vocabulary. Each **container** in a Pod is in one of three states [source: k8s-docs-pod-lifecycle-2026-08-23]:

- **Waiting** — the container is still running the operations it requires in order to complete start up: pulling the container image, applying Secret data. A Reason field summarizes *why* it's waiting.
- **Running** — the container is executing without issues. A startedAt timestamp is recorded.

- **Terminated** — the container began execution and then either ran to completion or failed. A reason, an exit code, and start and finish times are recorded.

Notice how much more each container state tells you than a phase does. A phase is one word. A container state comes with a Reason, or an exit code, or a timestamp: the specifics of *what*, not just *which*.

Dead Reckoning: A Pod has exactly one **phase**, which is one of: Pending, Running, Succeeded, Failed, Unknown. Each container in that Pod separately has a **state**, which is one of: Waiting, Running, Terminated. Phase is a Pod-level field in `status`; state is per-container [source: k8s-docs-pod-lifecycle-2026-08-23] [source: k8s-docs-objects-2026-08-23]. A Pod with three containers has one phase and three states. These are different fields with different vocabularies at different scopes. They are not interchangeable.



The nesting in that figure is the point. Container states are drawn *inside* the Pod because that is the actual relationship: one contains the other. If you find yourself picturing them side by side, as two alternative ways of saying the same thing, the figure has failed and so has the model.

Leg three: the distinction, and the two traps that live in it

A Pod can report `Running` while one of its containers sits in `Waiting`, provided that container has already been created and is between runs. That is not a contradiction and it is not a bug. It's two true statements at two scopes.

And it generates the costliest trap in the chapter:

⚠ Navigational Hazards

Three of the easiest facts to get wrong about Pod status share a single root cause: **reading a Pod-scoped signal as though it were container-scoped**. Fix the root cause and you fix all three.

1. Phase is not state. "The Pod is `Waiting`" is not a sentence Kubernetes can produce — `Waiting` is not a phase. "The container is `Pending`" is equally impossible — `Pending` is not a container state. If you find yourself unsure which vocabulary a word belongs to, you're reading at the wrong scope.

2. `Running` does not mean working. Read the definition again: a Pod is `Running` when it's bound to a node, all containers have been created, and at least one container is running **or is in the process of starting or restarting** [source: `k8s-docs-pod-lifecycle-2026-08-23`]. A container that starts, crashes every fifteen seconds, and restarts is, at any given moment, in one of those states. **A crash-looping Pod reports the phase `Running`.** This is the costliest misreading in the chapter, and Chapter 13's entire diagnostic method depends on you not making it.

3. `restartPolicy` is not per-container. It is a Pod-level field and it applies to every container in the Pod. (More on this in a moment.)

If the exam gives you a Pod whose phase is `Running` and asks whether the application is healthy, the answer is: **the phase cannot tell you that**. Phase tells you where the Pod is in its lifecycle. It does not tell you whether anything inside it is doing useful work. `/s7`'s probes exist precisely because phase can't answer that question.

`restartPolicy`, and what turns `Terminated` back into `Running`

A container that reaches `Terminated` doesn't necessarily stay there. What decides is `restartPolicy`.

The spec of a Pod has a `restartPolicy` field with possible values `Always` (the default), `OnFailure`, and `Never`. The `restartPolicy` applies to all containers in the Pod [source: `k8s-docs-pod-lifecycle-2026-08-23`]. It is set once, on the Pod, which is trap #3 above stated as a fact rather than a warning.

When a container does exit and the policy calls for a restart, the kubelet doesn't retry immediately and forever. **After containers in a Pod exit, the kubelet restarts them with an exponential backoff delay —**

10s, 20s, 40s, and so on — capped at five minutes. Once a container has executed for 10 minutes without any problems, the kubelet resets the restart backoff timer for that container [source: k8s-docs-pod-lifecycle-2026-08-23].

🔗 **Closer Look:** The backoff schedule looks like two arbitrary numbers until you see what each one is for. The **five-minute cap** is a floor on how bad things can get: no matter how long a container has been failing, you never wait more than five minutes to find out whether the next attempt works. The **ten-minute reset** is a forgiveness window: a container that has behaved for ten straight minutes has demonstrably recovered, so its history of failures stops counting against it. Cap plus forgiveness. Neither number is magic, but the *shape* — bounded penalty, earned amnesty — is a pattern you'll see again in distributed systems.

The worked example Chapter 2 promised

Chapter 2 named a failure and deferred it here: `ImagePullBackOff` [cross-bearing: see Ch 2 §6 — *ImagePullBackOff* and where its state is defined]. It's the cleanest possible demonstration of the phase/state split.

When a kubelet starts creating containers for a Pod, a container may be in the **Waiting** state because of `ImagePullBackOff`. That status means a container could not start because Kubernetes could not pull a container image, an invalid image name or a private registry with no `imagePullSecret` being the usual causes. The `BackOff` part indicates that Kubernetes will keep trying, with an increasing back-off delay, up to a compiled-in limit of 300 seconds (five minutes) [source: k8s-docs-images-2026-08-23]. That is a separate backoff from the container-restart backoff above — this one governs image pulls, not restarts.

Line that up against both vocabularies:

Signal	Value	Scope
Pod phase	Pending — accepted, but a container isn't set up and running yet	Pod
Container state	Waiting — still doing what it needs in order to start	Container
Container state Reason	<code>ImagePullBackOff</code> — <i>this specific</i> reason it can't start	Container

The phase tells you the Pod hasn't gotten going. The state tells you which container. The Reason tells you why. Three fields, three levels of specificity, and only the third one is actionable.

Note carefully that the phase here is `Pending`, not `Running`. `Running` requires that **all** the containers have been created; a container that cannot pull its image has not been created. That distinction is exactly why the worked overlay in the figure above uses a *post-creation* waiting reason instead of this one.

What this section deliberately does not do is tell you what to *run*. No `kubectl describe` walkthrough, no event stream, no diagnostic sequence. Chapter 5 owns the vocabulary; Chapter 13 owns the method, and Chapter 2 already published that boundary [cross-bearing: see Ch 13 §2 — *diagnosing a Pod that will not start*]. The reason the boundary is worth holding: Chapter 13's method is "*read the phase before you read*

the logs." That instruction is worthless to a reader who doesn't already own the vocabulary. This section is what makes Chapter 13 possible.

Application-scope triage, meaning the app is running but behaving wrongly, is Chapter 16's *[cross-bearing: see Ch 16 §1 — when the Pod is fine and the application isn't]*.

☆ **Fixed Point: Phase is Pod-scoped; state is per-container.** And Running does not mean working — a crash-looping Pod reports the phase Running, because Running includes containers that are starting or restarting.

🕒 Taking Your Bearings #2: Lifetime, Phase, and State

Five questions covering §4–§5. Two of them retrieve material from earlier chapters, because this is where those two threads finally land.

1. A Pod reports the phase Running. One of its containers is in the state Waiting, serving a restart backoff between runs. Is the Pod broken?
2. A colleague sets `restartPolicy` on one container of a two-container Pod, intending only that container to restart on failure. What actually happens?
3. ...ll **Challenge item — this one is meant to be hard.** A container has been crash-looping for twenty minutes. Roughly how long is the kubelet now waiting between restart attempts, and what would reset that interval?
4. **[retrieval: ch2]** Chapter 2 told you to bank a name and said its state was this chapter's material. For a Pod whose only container cannot pull its image: give the Pod's phase, the container's state, and the field on the container status that tells you which specific failure it is.
5. **[retrieval: ch4]** Which of `spec` and `status` carries phase? Who writes it? And what would it mean if you tried to set it yourself?

Answers with Explanations:

1. No — or at least, the two signals don't say so. Both readings are legitimate, because they're at different scopes.

A Pod is Running when it's bound to a node, all containers have been created, and at least one container is running, starting, or restarting [source: k8s-docs-pod-lifecycle-2026-08-23]. The waiting container in this scenario has been created; it is between runs. Nothing about that contradicts the definition. Whether something is *wrong* depends entirely on the Reason on the waiting container, and neither the phase nor the bare state name will tell you.

The discriminating half, which is worth more than the answer itself: change the reason and the answer changes. A container waiting because its **image has not been pulled** has not been created at all, and Running requires that all containers *have* been created. That Pod reports Pending, not Running. Same state name, different reason, different phase. That is the whole game.

Why two tempting answers are wrong — and these are two different misconceptions that happen to produce the same "no":

- **"Yes, because a Pod can't be Running if a container is Waiting"** treats phase and state as one vocabulary that must agree. They are two vocabularies at two scopes, and for post-creation waits they are not required to agree, because they are not describing the same thing.
- **"No, because Running means the application is working"** gets the right answer for a reason that's badly wrong. Running explicitly includes containers that are starting or restarting. A reader holding this belief will look at a crash-looping Pod, see Running, and conclude everything is fine. Chapter 13 will be unusable for them.

2. It cannot be set on one container. restartPolicy is a field on the Pod's spec, and it applies to all containers in the Pod [source: k8s-docs-pod-lifecycle-2026-08-23].

If two workloads genuinely need different restart behavior, that's a signal they should be two Pods, which is §2's rule arriving from an unexpected direction.

3. About five minutes — the backoff has hit its cap. It would reset after the container ran for ten continuous minutes without a problem.

The kubelet restarts with exponential backoff — 10s, 20s, 40s, and onward — capped at five minutes. Once a container has executed for 10 minutes without any problems, the kubelet resets the restart backoff timer for that container [source: k8s-docs-pod-lifecycle-2026-08-23].

This one was labeled a challenge for a reason. If you got it, you're holding a level of detail most candidates skip. If you didn't, notice that you only needed two facts, the cap and the reset, and that twenty minutes of failing is more than enough to reach a cap that's hit within a few minutes. Both outcomes are fine here; the struggle is doing the encoding work.

4. Phase: Pending. Container state: Waiting. The field: Reason, carrying the value ImagePullBackOff.

The phase is Pending because the Pod has been accepted but its container is not yet set up and running [source: k8s-docs-pod-lifecycle-2026-08-23]. The container is Waiting because it is still doing what it needs in order to start, and Waiting carries a Reason field that summarizes why [source: k8s-docs-pod-lifecycle-2026-08-23]. ImagePullBackOff means the container could not start because Kubernetes could not pull the image [source: k8s-docs-images-2026-08-23].

Three fields, three scopes, increasing specificity. That's the shape to remember.

Why two very likely wrong answers are wrong:

- **"State: ImagePullBackOff."** ImagePullBackOff is a **Reason**, not a state. The three container states are Waiting, Running, and Terminated, and nothing else is ever one of them. If you wrote the right string in the wrong slot, you have the fact and not yet the taxonomy, and that is precisely the error §5's hazards block exists to close. It is also the most self-concealing miss in the chapter, because the correct string appears in your answer and it *looks* right.
- **"Phase: Running."** Tempting, because the Pod exists and the kubelet is visibly working on it. But Running requires that all of the containers **have been created**, and this one never was. It's stuck ahead of creation, which is the definition of Pending.

5. status carries phase. The Kubernetes system writes it. Trying to set it yourself would be a category error.

The `status` field describes the current state of the object, supplied and updated by the Kubernetes system and its components; `spec` is where you declare desired state [source: k8s-docs-objects-2026-08-23]. `phase` is an observation, not an instruction. Writing "I would like this Pod's phase to be Running" expresses nothing. The phase is a *report* on what's true, and reports aren't requests. Chapter 4's `spec/status` distinction was abstract when you learned it; this is the first place in the book where it has a concrete sub-field to point at.

How'd you do?

- **5/5.** You have the chapter's centerpiece. §7 and §8 both assume it, and both will go quickly from here.
- **3–4 correct.** Good. Review the misses and continue, but note which one you missed. If it was item 1 or item 4, that's the phase/state scope split specifically, and it's the one thing in this chapter you cannot carry a gap on.
- **0–2 correct.** Stop here. Re-read §5's three legs, the five phases, the three states, and the hazards block, then retake this checkpoint. This is not optional pacing advice: §7's probe behaviors and §8's OOM-kill material are both narrated in this vocabulary, and Chapter 13's entire diagnostic method is *"read the phase before you read the logs."* Twenty minutes here saves you the rest of the book.

Checkpoint: You've Now Mastered

✓ The Pod lifetime — created once, scheduled once, replaced never ✓ Why disposability forces a workload resource to exist (Chapter 6's premise) ✓ Five Pod phases and three container states, at their correct scopes ✓ `restartPolicy` scope, values, and the backoff schedule ✓ The chapter's three highest-value traps, and their shared root cause □ What a Pod is to the API server (§6) □ What "healthy" actually means (§7) □ What a Pod asks for and what it's held to (§8)

§6 — A Pod's Identity

§5 ended with a Pod being destroyed and replaced by a different object with a different UID. That raises a question worth one section: if the instance is disposable, what, if anything, persists about *who* this Pod is?

A service account is a type of non-human account that, in Kubernetes, provides a distinct identity in a Kubernetes cluster [source: k8s-docs-service-accounts-2026-08-23]. Application Pods use one to identify themselves to the API server. Four facts, and then we stop.

One. ServiceAccounts are namespaced, and every namespace gets one named `default` upon creation [source: k8s-docs-service-accounts-2026-08-23]. Chapter 4 taught you the namespaced-versus-cluster-scoped boundary; here it is doing work rather than being recited [*cross-bearing: see Ch 4 §3 — namespaced and cluster-scoped objects*].

Two. If you deploy a Pod in a namespace and don't manually assign a ServiceAccount to it, Kubernetes assigns the `default` ServiceAccount for that namespace to the Pod [source: k8s-docs-service-accounts-2026-08-23]. There is no such thing as a Pod without an identity. Every Pod sails under some flag, including the ones you never bothered to name.

Three. The `default` ServiceAccounts get no permissions by default other than the default API discovery permissions Kubernetes grants to all authenticated principals when RBAC is enabled [source: k8s-docs-service-accounts-2026-08-23]. Having an identity and being able to *do* anything with it are two separate questions, and the default answer to the second one is "essentially nothing."

Four. You assign one via `spec.serviceAccountName` [source: k8s-docs-service-accounts-2026-08-23].

The credential, in one sentence

Chapter 4 cataloged the built-in Secret types and named `kubernetes.io/service-account-token`, then deferred the identity model it belongs to [*cross-bearing: see Ch 4 §4 — the service-account-token Secret type*]. Here is the deferral honored, at the altitude Chapter 4 promised.

In Kubernetes v1.22 and later, Kubernetes gets a **short-lived, automatically rotating token** using the TokenRequest API and mounts it as a **projected volume** [*cross-bearing: see Ch 11 — projected volumes*]. Long-lived ServiceAccount token Secrets, the type Chapter 4 listed, don't expire or rotate and are not recommended [source: k8s-docs-service-accounts-2026-08-23]. The type still exists; it's the legacy form [source: k8s-docs-secret-2026-08-23].

 **Worth Securing: Every Pod has an identity whether or not you gave it one.** Practitioners find this genuinely surprising the first time — the mental model of "I didn't configure authentication, so there isn't any" is wrong. There is an identity, it's the namespace's `default`, it can authenticate to the API server, and it can do almost nothing. That last clause is doing a lot of load-bearing work, and Chapter 12 is where it gets examined.

That's the whole section. Everything else about ServiceAccounts — what one can be *granted*, how RBAC binds permissions to it, how to harden its tokens, and the privilege-escalation path that opens up when the wrong principal can create Pods — is Chapter 12's [*cross-bearing: see Ch 12 §2 — ServiceAccounts as RBAC subjects*]. Chapter 4 told you as much in as many words, and there's no advantage in spending that material seven chapters early.

Identity also shows up once more, in a different guise: the agent that delivers your application to the cluster needs one too [*cross-bearing: see Ch 15 §4 — the delivery agent's identity*].

..1 §7 — Three Probes, Three Jobs

Chapter 3 said the kubelet ensures containers are "running and healthy." §1 handled *running*. This section handles *healthy*, and the answer turns out to be that "healthy" is not one question. It's three.

Soundings question 4 asked you to describe a situation where a process is running but can't do its job. Whatever example you gave — the stuck JVM, the exhausted connection pool, the model still loading — you were identifying a gap that "is the process alive?" cannot detect. Kubernetes fills that gap with probes, and it uses three of them because the gap has three different shapes.

A probe is a diagnostic performed periodically by the kubelet on a container [source: k8s-docs-pod-lifecycle-2026-08-23].

The four mechanisms (how a probe asks)

Take these first, separately, because they're orthogonal to the three types and tangle badly if you learn them together. There are four check mechanisms [source: k8s-docs-pod-lifecycle-2026-08-23]:

Mechanism	What it does	Success means
exec	Executes a command in the container	Exit status 0
httpGet	HTTP GET against the Pod's IP, port, and path	Status code ≥ 200 and < 400
tcpSocket	TCP check against a port	The port is open
grpc	A gRPC health check	The gRPC health check passes

Any probe type can use any mechanism. The mechanism is *how the question is asked*; the type is *what the answer is used for*. Keep them in separate compartments and this section is easy. Merge them and you'll be trying to memorize twelve things instead of seven.

Note that `httpGet` goes to *the Pod's* IP, not the container's: §1's fact turning up somewhere you might not have expected it.

The three types (what the answer is used for)

For each type, the definition matters less than the **consequence of failure**. That's what gets tested, and that's what matters at three in the morning too.

livenessProbe — is the container running? If it fails, **the kubelet kills the container**, and the container is then subject to its restart policy [source: k8s-docs-pod-lifecycle-2026-08-23]. This is §5's `restartPolicy` doing visible work, which is exactly why §5 came first: a liveness probe failure hands the container to the restart machinery you already understand.

readinessProbe — is the container ready to respond to requests? If it fails, **the endpoints controller removes the Pod's IP address from the endpoints of all Services that match the Pod** [source: k8s-docs-pod-lifecycle-2026-08-23]. The container keeps running. Nothing is killed. Nothing is restarted. The Pod is simply taken out of service until it says it's ready again.

Readiness stands a container down from the watch. Liveness relieves it of duty altogether. That behavior is the one Soundings question 5 primed: a load balancer removes an unhealthy backend from rotation; it doesn't kill it. Readiness is Kubernetes' version of exactly that, and it's the probe people most often reverse, because "readiness failed" *sounds* more severe than it is.

startupProbe — has the application within the container started? While a startup probe is configured and has not yet succeeded, **all other probes are disabled** [source: k8s-docs-pod-lifecycle-2026-08-23]. If the startup probe itself fails, the kubelet kills the container and applies the restart policy [source: k8s-docs-pod-lifecycle-2026-08-23].

LIVENESS

ASKS

Is the container running?

ON FAILURE

kubelet **KILLS** the container → restart policy applies

DOES NOT

remove it from Service endpoints

READINESS

ASKS

Can the container respond to requests?

ON FAILURE

Pod IP REMOVED from endpoints
of all matching Services

ONLY PROBE THAT DE-REGISTERS WITHOUT RESTARTING

DOES NOT

kill or restart anything —
the container keeps running

STARTUP

ASKS

Has the application started?

ON FAILURE

kubelet **KILLS** the container → restart policy applies

DOES NOT

run alongside the others —
it **SUPPRESSES** them until it succeeds

The third column is the one doing the teaching. Two probes kill and don't de-register; one de-registers and doesn't kill. Get that asymmetry and the rest is detail.

📌 **Snag:** Configuring a startup probe **disables** the liveness and readiness probes until the startup probe succeeds [source: k8s-docs-pod-lifecycle-2026-08-23]. Readers consistently assume all three run in parallel from the moment the container starts. They don't — and that suppression is the startup probe's entire reason for existing. Without it, a liveness probe would kill a slow-starting application before it ever finished starting, forever.

The parameters

Probes are tuned with five parameters: `initialDelaySeconds`, `periodSeconds`, `timeoutSeconds`, `successThreshold`, and `failureThreshold` [source: k8s-docs-pod-lifecycle-2026-08-23]. Know that they exist and roughly what each governs — how long to wait before the first check, how often to check, how long to wait for an answer, and how many consecutive results it takes to flip the verdict in each direction. Choosing good values is a real engineering skill and it isn't what this exam is asking.

The discrimination this section exists for

Liveness and readiness look almost identical on paper. Both are periodic checks. Both use the same four mechanisms. Both can fail. And they do **opposite** things:

- **Liveness restarts, and does not remove from service.**
- **Readiness removes from service, and does not restart.**

If you remember one sentence from §7, that's the one.

☆ **Fixed Point: Liveness failure → the kubelet kills the container** (restart policy then applies).
Readiness failure → the Pod's IP is removed from the endpoints of all matching Services; nothing is restarted. Startup probe configured → all other probes are disabled until it succeeds.

The readiness behavior is a forward plant. When Chapter 9 explains how a Service knows which Pods to send traffic to, this is the mechanism doing the removing [*cross-bearing: see Ch 9 §4 — readiness and Service endpoint membership*]. And probes are what make a rolling update safe: a new Pod that never reports ready is a new Pod that never receives traffic, which is how Chapter 6 stops a bad release from taking down the service [*cross-bearing: see Ch 6 §4 — what makes a rolling update safe*].

One thing probes are **not**: observability. A probe answers a yes/no question for the kubelet's benefit and produces no history, no trend, and no measurement. That distinction gets its proper treatment in Chapter 18 [*cross-bearing: see Ch 18 §1 — health checking is not observability*].

§8 — What a Pod Is Owed

Soundings question 6 asked you to distinguish reserving capacity from capping it. Kubernetes uses both, calls them by different names, has different components enforce them, and — this is the part that surprises people — enforces the two kinds of cap by completely different mechanisms.

This section has the longest forward reach in the book. Four later chapters retrieve it by name.

Leg one: two words, two components

When you specify the resource request for containers in a Pod, the kube-scheduler uses this information to decide which node to place the Pod on. When you specify a resource limit for a container, the kubelet enforces those limits so that the running container is not allowed to use more of that resource than the limit you set. The kubelet also reserves at least the request amount of that system resource specifically for that container to use [source: k8s-docs-resource-management-2026-08-23].

Two words, two jobs, two components:

	Request	Limit
Who reads it	kube-scheduler — to choose a node	kubelet (with the kernel) — to enforce at runtime
What it means	<i>Reserve at least this much for me</i>	<i>Never let me exceed this</i>
When it acts	At placement time, once	Continuously, while the container runs

And the rule that connects them: **if the node where a Pod is running has enough of a resource available, it's possible — and allowed — for a container to use more of that resource than its request specifies. However, a container is not allowed to use more than its resource limit** [source: k8s-docs-resource-management-2026-08-23].

So a request is a floor, not a ceiling. Exceeding your request on a node with spare capacity is normal, expected behavior, not a violation of anything. One number gets you a berth. The other keeps you inside it.

🧠 **Mnemonic: Requests are about placement. Limits are about containment.** Scheduler places; kubelet contains.

Leg two: the two enforcement mechanisms are not the same

Here is the part that explains a large fraction of real production behavior, and it's the part most people don't know.

CPU limits are enforced by CPU throttling. When a container approaches its cpu limit, the kernel will restrict access to the CPU corresponding to the container's limit. Thus, a cpu limit is a hard limit the kernel enforces [source: k8s-docs-resource-management-2026-08-23].

Memory limits are enforced by the kernel with out of memory (OOM) kills. When a container uses more than its memory limit, the kernel may terminate it. However, terminations only happen when the kernel detects memory pressure. Thus, a container that over allocates memory may not be immediately killed; memory limits are enforced reactively [source: k8s-docs-resource-management-2026-08-23].

Sit with the asymmetry:

- **Exceed your CPU limit and you get slow.** The container keeps running. It's throttled, held to its allocation, and the effect is latency, not death. Neither of \$5's two vocabularies reports it: the phase doesn't change and the container state doesn't change.
- **Exceed your memory limit and you eventually get killed.** But not necessarily *when* you exceed it, only when the kernel detects memory pressure. Which means an over-allocating container can run fine for hours and then die at an apparently unrelated moment, when something else on the node needed memory.

That "reactively" is the word to hold onto. It's why memory problems in Kubernetes have a reputation for being hard to reproduce: the trigger for the kill isn't your container's behavior alone, it's the node's aggregate pressure.

Leg three: resource types and units

The two you will specify constantly [source: k8s-docs-resource-management-2026-08-23]:

Type	What it measures	Base unit
cpu	Compute processing	cpu (core)
memory	RAM	bytes

Two more exist and are specified the same way — `ephemeral-storage` (local ephemeral storage, in bytes) and `hugepages-<size>` (Linux only, in bytes) — and clusters can additionally provide **extended resources**, custom-named resources typically exposed by device plugins [source: k8s-docs-resource-management-2026-08-23]. Know that they exist; you will not be asked to reason about them.

CPU units. In Kubernetes, **1 CPU unit is equivalent to 1 physical CPU core, or 1 virtual core**, depending on whether the node is a physical host or a VM. Fractional requests are allowed: `0.5` requests half as much CPU time as `1.0`, and the quantity expression `0.1` is equivalent to `100m`, "one hundred millicpu" [source: k8s-docs-resource-management-2026-08-23].

Memory units. Measured in bytes. You can express memory as a plain integer or with quantity suffixes, in either decimal form (`k`, `M`, `G`, and `up`) or the power-of-two equivalents (`Ki`, `Mi`, `Gi`, and `up`) [source: k8s-docs-resource-management-2026-08-23]. In practice you will write `Mi` and `Gi` most of the time.

⚠ Navigational Hazards

M means megabytes. m means millibytes. The documentation calls this out explicitly, and for good reason: **a request of 400m of memory is a request for 0.4 bytes** [source: k8s-docs-resource-management-2026-08-23].

This is the most mechanically checkable gotcha in the chapter, which is exactly what makes it so easy to write a question around. 400M and 400m differ by nine orders of magnitude and by one keystroke. When you see a memory quantity on an exam question, read the case of the suffix before you read anything else.

Note that m is perfectly correct — and extremely common — for CPU, where 100m means one tenth of a core. The suffix isn't wrong; it's wrong *for memory*. Habit carries it across, and nothing in the manifest will stop you.

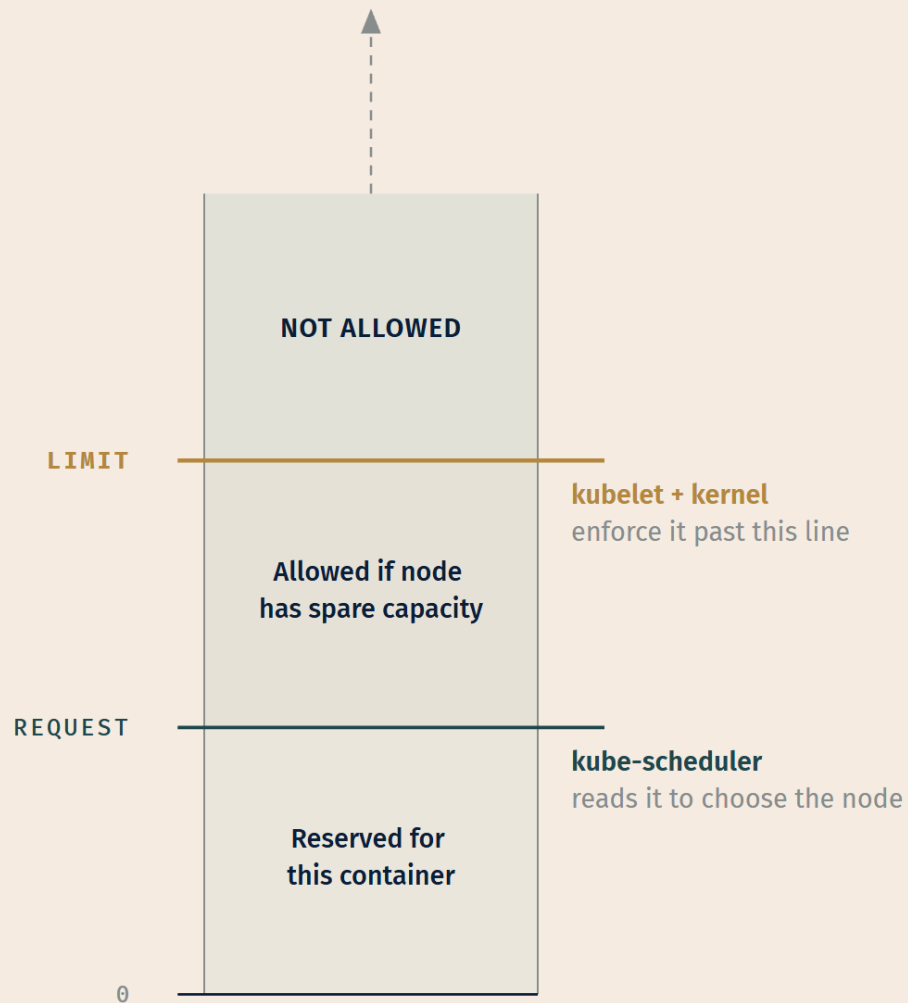
🦋 **Closer Look:** Two details that reward a second look. First, **CPU resource is always specified as an absolute amount, never as a relative amount:** 500m CPU represents roughly the same amount of computing power whether the container runs on a single-core, dual-core, or 48-core machine [source: k8s-docs-resource-management-2026-08-23]. That's more useful than it first appears. Most capacity intuitions are relative ("give this service a quarter of the box"), and they break the moment the box changes size. A CPU request in Kubernetes is portable across node types by construction. Second, there is a floor on precision: **Kubernetes doesn't allow CPU resources finer than 1m** [source: k8s-docs-resource-management-2026-08-23]. One thousandth of a core is as small as the vocabulary goes.

There is a fourth movement to this arithmetic, and it is the one the exam names. **Kubernetes classifies every Pod you run into a quality of service (QoS) class and uses that classification to influence how the Pod is treated when a node comes under resource pressure** [source: k8s-docs-pod-qos-2026-08-24]. You never set the class directly. It is derived entirely from the shape of the requests and limits you just learned to write:

- **Guaranteed** — every container in the Pod has a memory limit and a memory request set equal to each other, and a CPU limit and CPU request set equal to each other. These Pods have the strictest resource bounds and are the least likely to face eviction: guaranteed not to be killed until they exceed their own limits [source: k8s-docs-pod-qos-2026-08-24].
- **Burstable** — the Pod does not meet the Guaranteed criteria, but at least one container has a memory or CPU request or limit. There is a lower-bound guarantee based on the request, with room to use more when the node has spare capacity [source: k8s-docs-pod-qos-2026-08-24].
- **BestEffort** — no container in the Pod has any memory or CPU request or limit at all. These Pods may use whatever node resources are not spoken for by the other classes — and when the node runs short, they are the first over the side [source: k8s-docs-pod-qos-2026-08-24].

Keep the mechanisms separate in your head, because a distractor will happily blur them: CPU overuse is throttled and memory overuse is OOM-killed, per container, as above — while the QoS class governs

eviction under node pressure, a Pod-level decision. Same inputs, different machinery.



ENFORCEMENT DIFFERS BY RESOURCE

cpu → **throttled at the limit** — hard and immediate: you get slow

memory → **OOM-killed past it** — reactive:

under node memory pressure — you get slow, then dead

☆ **Fixed Point: Requests are what the scheduler reads** to place the Pod. **Limits are what the kubelet enforces** on the running container. **CPU limits throttle; memory limits kill** — and the memory kill is reactive, arriving when the kernel detects pressure rather than the instant you cross the line.

Where these two numbers come back

Briefly, because you'll see all of this again: requests are the input to the scheduler's filtering step [*cross-bearing: see Ch 7 §2 — resource requests as a scheduling filter*]. They're what the system is reporting on when a Pod is killed for using too much [*cross-bearing: see Ch 13 §4 — OOMKilled and Evicted*]. They're the baseline autoscalers compare observed usage against [*cross-bearing: see Ch 17 — autoscaling targets*], and they're the denominator when monitoring reports "utilization" [*cross-bearing: see Ch 18 §3 — utilization relative to requests*].

Two numbers in a Pod spec; four later chapters. It's worth getting right the first time.

🕒 Taking Your Bearings #3: Identity, Health, and What a Pod Is Owed

Five questions covering §6–§8, with the last one reaching back into §5.

1. A Pod is created with no ServiceAccount specified. What identity does it have, and what can it do with that identity?
2. A liveness probe and a readiness probe both fail on the same container. Describe **both** consequences.
3. A container takes four minutes to start. Which probe solves this, and what does configuring it do to the other two?
4. A container has a memory request of 256Mi and a memory limit of 512Mi. The node has spare memory. The container is currently using 400Mi. Is anything wrong?
5. Two containers run identical images and identical code. One is exceeding its CPU limit; the other is exceeding its memory limit. Describe what an operator observes in each case, and name the Pod phase and container state each one ends up in.

Answers with Explanations:

1. It has the namespace's default ServiceAccount, and it can do essentially nothing with it.

If you deploy a Pod and don't manually assign a ServiceAccount, Kubernetes assigns the default ServiceAccount for that namespace [source: k8s-docs-service-accounts-2026-08-23]. That account gets no permissions by default other than the API discovery permissions granted to all authenticated principals when RBAC is enabled [source: k8s-docs-service-accounts-2026-08-23].

Both halves matter and they're independent. It *has* an identity: it can authenticate. It has almost no *authorization*: it can't do anything with the authentication.

Why the wrong answers are wrong:

- **"None — it has no identity"** is the intuitive answer and it's wrong. Kubernetes assigns one automatically. There is no such thing as an anonymous Pod in a namespace.
- **"cluster-admin"** or any broad-permissions answer confuses "the system gave it something automatically" with "the system gave it something powerful." The default assignment is deliberately inert.

2. The container is killed and restarted according to restartPolicy, and the Pod's IP is removed from the endpoints of all matching Services.

Both, simultaneously, because the two probes are independent diagnostics with independent consequences [source: k8s-docs-pod-lifecycle-2026-08-23]. A liveness failure kills; a readiness failure de-registers. Two probes failing means both things happen.

This question exists to force both behaviors into one answer, because the failure mode in candidates' heads is treating the two probes as a single "health check" with a single outcome. They're two mechanisms with two different consequences, and they can fire independently or together.

3. A startupProbe. Configuring it disables the liveness and readiness probes until it succeeds.

A startup probe indicates whether the application within the container has started; all other probes are disabled if a startup probe is provided, until it succeeds [source: k8s-docs-pod-lifecycle-2026-08-23].

The second half is where people lose the point. Without the suppression, a four-minute startup would be repeatedly killed by a liveness probe that concluded, correctly given what it can see, that the container wasn't responding. The startup probe's value is not that it checks something new; it's that it *silences* the other two during the window when their answers would be misleading.

4. No. Nothing is wrong.

Exceeding a *request* is allowed when the node has capacity. The documentation is explicit that a container may use more of a resource than its request specifies if the node has enough available. Only the *limit* is a hard boundary that a container is not allowed to cross [source: k8s-docs-resource-management-2026-08-23].

400Mi is above the 256Mi request and below the 512Mi limit. That's the intended operating range, not a problem to be fixed. The request was a floor for the scheduler's benefit, not a promise the container made.

5. The CPU case is throttled and stays Running. The memory case is eventually terminated — reactively, under node memory pressure, not immediately — and its container state becomes Terminated.

The CPU container: when it approaches its cpu limit, the kernel restricts its access to the CPU [source: k8s-docs-resource-management-2026-08-23]. The operator observes **latency**. Requests take longer. Throughput drops. Nothing in the Pod's status changes at all: the phase stays `Running`, the container state stays `Running`. This is the failure mode that hides from every signal §5 taught you.

The memory container: when it uses more than its memory limit, the kernel may terminate it, but terminations only happen when the kernel detects memory pressure, so a container that over-allocates may not be killed immediately [source: k8s-docs-resource-management-2026-08-23]. The operator observes the container **dying**, possibly long after the over-allocation started, and possibly at a moment that correlates with something else entirely on the node. The container reaches the `Terminated` state, with a reason and an exit code recorded [source: k8s-docs-pod-lifecycle-2026-08-23]. The Pod's phase then depends on `restartPolicy` and on what the other containers are doing.

This item required §5 and §8 together, which is the point: it's the direct precursor to Chapter 13's material on diagnosing killed and evicted Pods. What we're **not** doing here is diagnosing it. Naming the mechanism is this chapter's job. Which command to run and which events to read is Chapter 13's [*cross-bearing: see Ch 13 §4 — OOMKilled and Evicted*].

How'd you do?

- **5/5.** You're done with new material. §9 introduces no facts at all; it only rearranges the ones you already have.
- **3–4 correct.** Review the misses, then continue. If item 2 or item 3 was among them, re-read §7's failure-behavior table before the Practice Questions; five of those twenty-three items live in that table.
- **0–2 correct.** Go back to §7's three-probe comparison figure and §8's Leg two (the CPU-throttle / memory-OOM split), roughly twenty minutes, then retake this checkpoint. Both come back in Chapter 13, and §7's readiness behavior is the mechanism Chapter 9's Services are built on.

Checkpoint: You've Now Mastered

✓ What identity a Pod has by default, and what it can do with it ✓ Three probes, four mechanisms, and — most importantly — three distinct failure behaviors ✓ Requests versus limits, and which component acts on each ✓ Why exceeding CPU makes you slow and exceeding memory makes you dead ✓ The `m`-versus-`M` memory suffix trap

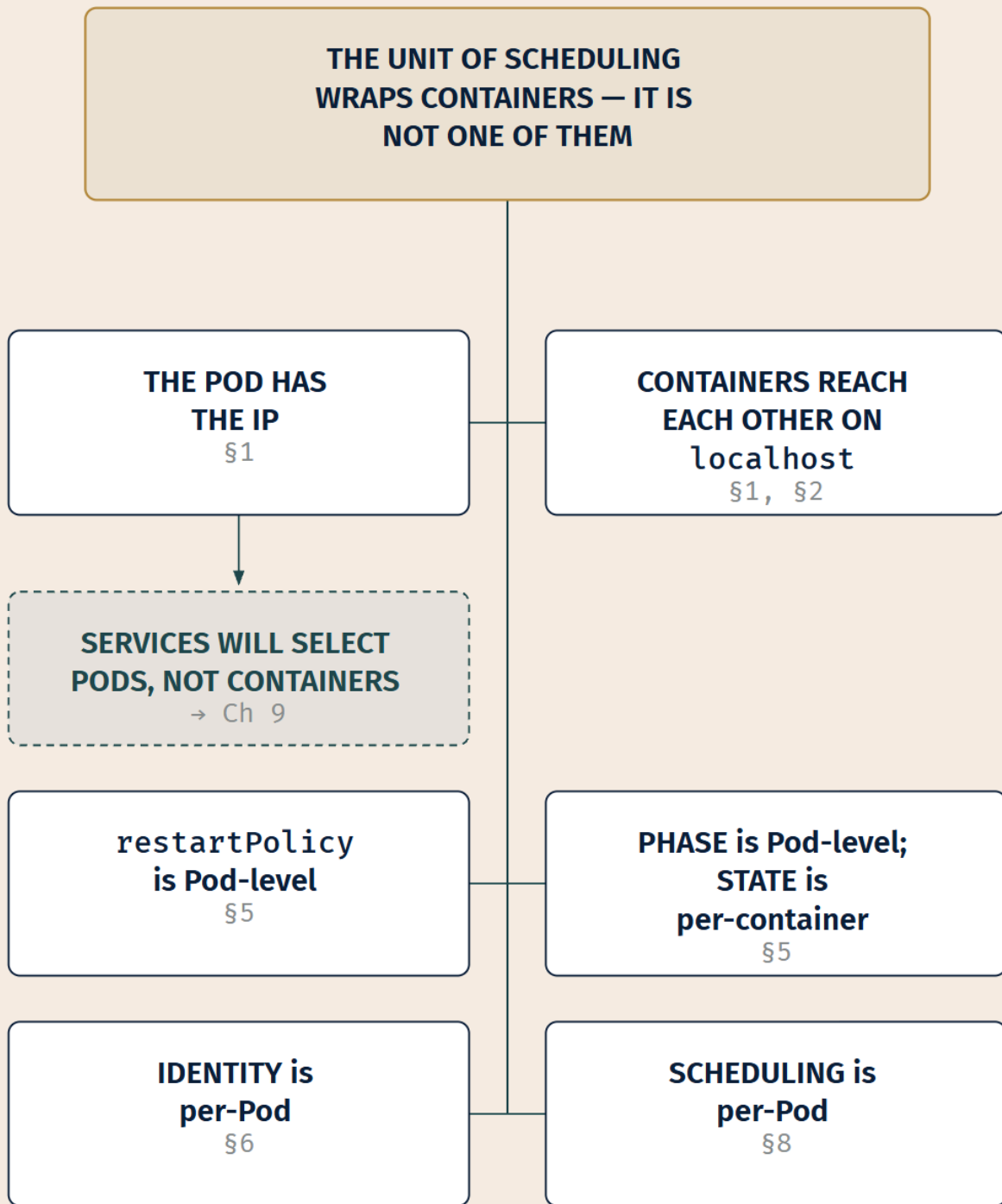
One section left, and it contains no new facts at all.

☀ §9 — The Smallest Deployable Unit

☀ Zenith

Everything in this chapter is a consequence of one decision: **the unit of scheduling wraps containers instead of being one.**

A hull is not cargo. The vessel is the thing that gets a berth, an address, and a name on the manifest; the crates in the hold get none of those, and they go wherever the hull goes. That's it. That's the whole design. Walk back through what you've just learned and watch each fact turn into a consequence of that single choice:

**LEGEND**

CENTRAL CLAIM

CONSEQUENCE

FORWARD REF.

- **The Pod has an IP, not the container** — because a shared network namespace is what the wrapper exists to provide (§1).
- **Containers reach each other on localhost** — same reason, same namespace (§1, §2).
- **restartPolicy is Pod-level and applies to every container** — because the Pod is the unit (§5).
- **The phase is Pod-level while the state is per-container** — because the Pod is the unit, but the containers are what actually run (§5).
- **Identity attaches to the Pod** — one ServiceAccount for the whole thing, not one per container (§6).
- **Requests are declared per container, but the scheduler places the Pod** — because the thing being placed is the wrapper, not any single container inside it (§8).
- **Services will select Pods, not containers** — which is what Chapter 9 is built on (§1, forward).

Now go back to where you started this chapter. If "Pod is Kubernetes' word for container" had been true, **every single one of those seven statements would be wrong**. Not subtly wrong. Wrong in ways that produce confidently incorrect answers about IP addressing, about restart behavior, about what a status field is describing, and about what a Service is pointed at.

That's why the subtitle says the distinction is worth points. It isn't pedantry, and it was never about vocabulary. It's the load-bearing wall. Seven facts you'd otherwise have to memorize separately turn out to be one fact you already understand, applied seven times.

There's a smaller synthesis hiding inside the larger one, too. §4 noted that a Pod is disposable the same way an image is immutable: replace, don't repair. Now add §5's phase-and-state split and §8's request-and-limit split, and a pattern emerges across all three. **Kubernetes consistently separates the thing you declare from the thing that's observed, and separates the scope you're declaring at from the scope where the work happens**. Spec and status. Request and limit. Pod and container. Same instinct, three expressions.

Exam Alert! 🚨

High-priority topics — the seven this chapter treats as load-bearing, in the order we'd study them:

1. **Pod phase versus container state**, with the scopes named correctly. Five phase values at Pod scope; three state values at container scope.
2. **The three probes and their failure behaviors** — specifically that readiness de-registers without restarting, and liveness restarts without de-registering.
3. **restartPolicy is a Pod-level field** with three values (Always, OnFailure, Never) that applies to every container in the Pod.
4. **One IP per Pod**, shared by all its containers, which communicate over localhost.
5. **Requests versus limits** — which component reads which, and that CPU limits throttle while memory limits kill.
6. **A Pod is scheduled once and never rescheduled** — it is replaced, with a new UID.

7. **Every Pod has a ServiceAccount**, defaulting to the namespace's default, which carries no meaningful permissions.

Common traps. Seven recurring misconceptions, and three of them share one root cause:

Trap	The correct understanding
Confusing Pod phase with container state	Two vocabularies, two scopes. A Pod has a phase; each container has a state. Neither word set applies at the other's level.
"Running means the app is working"	Running includes containers that are starting or restarting . A crash-looping Pod reports Running.
"restartPolicy can be set per container"	It's a Pod-level field and applies to all containers.
"A failed Pod is rescheduled to a healthy node"	Pods are never rescheduled. They're deleted and replaced by a new Pod with a different UID.
"Liveness and readiness do the same thing"	Liveness kills and restarts. Readiness de-registers from Services and restarts nothing. Opposite consequences.
Forgetting that a startup probe disables the others	While a startup probe is configured and hasn't succeeded, liveness and readiness are suppressed.
"Each container in a Pod gets its own IP"	The Pod gets one IP, shared by all its containers.

The first three of those aren't three separate things to memorize. They're one rule: **reading a Pod-scoped signal as though it were container-scoped**. Learn the rule and all three traps close at once.

One more, from the documentation's own warning: the memory suffix `m` versus `M`. `400M` is four hundred megabytes; `400m` is four tenths of a byte [source: k8s-docs-resource-management-2026-08-23]. One keystroke, nine orders of magnitude, and a question type that writes itself.

Practice Questions

Twenty-three questions. Five of them retrieve material from Chapters 2–4; four require two sections of this chapter at once. Answers and full explanations follow the last question.

\$1–\$3 — the Pod, multi-container Pods, init containers

1. A Pod contains three containers. How many cluster-wide IP addresses does Kubernetes assign to it?
- A) Three — one per container B) One, assigned to the Pod C) Four — one per container plus one for the Pod D) None; Pods are addressed through Services

2. A team wants to add a helper container — a **sidecar** — alongside an existing application container in the same Pod. Which two mechanisms does the Pod's shared context make available for the two of them to exchange data? (Choose the pair.)

A) A Service and a ConfigMap B) `localhost` networking and a shared volume C) A shared IP and a shared process namespace D) Environment variables and a NetworkPolicy

3. A team proposes a Pod containing an API server, a Redis cache, and a PostgreSQL database, reasoning that they form a single application. What is the strongest technical objection?

A) Kubernetes would schedule each of the three containers separately, which defeats the purpose of grouping them B) The three would need three separate IP addresses C) They would scale, fail, and be replaced as a single unit D) Each would need its own readiness probe, and a Pod can only have one

4. A Pod declares two init containers and two app containers. Which statement correctly describes the startup order?

A) All four start in parallel B) The two init containers start in parallel; when both exit successfully, the app containers start in parallel C) The first init container runs to successful completion, then the second; when both have succeeded, both app containers start D) The init containers run in parallel with the app containers, and the Pod is Ready once all four are running

5. [retrieval: ch4] A Pod is created with the label `app: checkout`. Which statement about that label is correct?

A) It uniquely identifies the Pod, replacing the need for a name B) It is stored in the Pod's `status` and maintained by the system C) It is an identifying attribute in `metadata` that other objects can select on D) It restricts the Pod to nodes carrying the same label

\$4–\$5 — lifetime, phase, container state, restartPolicy

6. A node fails while running a Pod. What happens to that Pod?

A) The scheduler moves it to a healthy node, preserving its UID B) It is marked for deletion after a timeout and replaced by a new Pod with a different UID C) It remains in phase `Running` until the node returns D) It is recreated on the same node once the node recovers, carrying the same UID

7. Which Pod phase describes a Pod that has been accepted by the cluster but whose container image is still downloading?

A) `Waiting` B) `Unknown` C) `Pending` D) `Running`

8. A container in a Pod reports the state `waiting`. Which field tells you specifically why?

A) `phase` B) `Reason` C) `exitCode` D) `startedAt`

9. A Pod's single container crashes and restarts every twenty seconds. What phase does the Pod report?

A) Failed B) Pending C) Unknown D) Running

10. Which statement about `restartPolicy` is correct?

A) It is set per container and defaults to `OnFailure` B) It is set on the Pod and applies to all its containers; it defaults to `Always` C) It is set on the Pod and applies only to app containers, never to init containers D) It is set per container and can be omitted only if a liveness probe is defined

11. A container has been failing continuously for half an hour. Which statement about the kubelet's restart behavior is correct?

A) The kubelet stops retrying after ten failures B) The delay grows without bound C) The delay is capped at five minutes, and resets after the container runs 10 minutes without a problem D) The delay is fixed at 30 seconds regardless of failure count

§6 — identity

12. A Pod is created in the namespace `payments` with no `serviceAccountName` set. Which statement is correct?

A) The Pod has no identity and cannot authenticate to the API server B) The Pod is assigned the default `ServiceAccount` of the `payments` namespace C) The Pod is assigned a cluster-wide `ServiceAccount` shared by all namespaces D) Pod creation fails validation until a `ServiceAccount` is specified

13. [retrieval: ch4] Chapter 4 listed `kubernetes.io/service-account-token` among the built-in `Secret` types. Which statement about it is correct today?

A) It is the current, recommended way to deliver a `ServiceAccount` credential to a Pod B) It is the legacy long-lived form; since v1.22 the recommended approach uses short-lived, automatically rotating tokens via the `TokenRequest` API C) It is a cluster-scoped object, unlike other `Secret` types D) It has been removed from Kubernetes and is no longer a valid `Secret` type

§7 — probes

14. A readiness probe fails on a container that is otherwise running normally. What happens?

A) The kubelet kills the container and applies the restart policy B) The Pod's IP is removed from the endpoints of all matching `Services`; the container keeps running C) The Pod's phase changes to `Failed` D) Both: the container is killed and the Pod is removed from `Service` endpoints

15. A liveness probe fails. What happens?

A) The Pod's IP is removed from Service endpoints; the container keeps running B) The kubelet kills the container, which is then subject to its restart policy C) The Pod's phase changes to Unknown D) The kubelet logs a warning but takes no action until the probe fails ten times

16. A container's startup probe is configured and has not yet succeeded. What is true of its liveness and readiness probes during that window?

A) They run normally alongside the startup probe B) They run, but their failures are logged rather than acted on C) They are disabled until the startup probe succeeds D) They are disabled, and remain disabled for the rest of the container's lifetime

17. Which statement about probe mechanisms is correct?

A) Liveness probes must use `httpGet`; readiness probes must use `exec` B) Any probe type can use `exec`, `httpGet`, `tcpSocket`, or `grpc` C) Only startup probes support `tcpSocket` D) An `exec` probe runs its command in a separate helper container, not in the container being probed

18. An `httpGet` probe returns HTTP 404. Is this a success or a failure, and why?

A) Success — the server responded B) Failure — success requires a status code ≥ 200 and < 400 C) Success — only 5xx responses are treated as failures D) It depends on the value of `successThreshold`

\$8 — requests, limits, resource units

19. A container's manifest requests `memory: 400m`. What has actually been requested?

A) 400 megabytes B) 400 mebibytes C) 0.4 bytes D) 400 millicores of memory bandwidth

20. [retrieval: ch2] Chapter 2 established that containers are immutable — you don't modify a running container, you build a new image and recreate the container. Which statement in this chapter follows the same pattern one level up?

A) A Pod's `spec` can be edited freely while it runs B) A failed Pod is not repaired; it is replaced by a new Pod with a different UID C) A Pod's phase can be reset by the operator D) Container images are re-pulled on every restart

21. [retrieval: ch3] Chapter 3 introduced the control loop: read desired state, observe current state, act to close the gap. How does the kubelet's handling of a Pod exemplify this pattern?

A) It queries `etcd` directly for the cluster's desired state B) It runs a reconciliation loop that keeps the containers described in the Pod's `spec` running, restarting them per the restart policy when they exit C) It schedules Pods onto nodes based on resource requests D) It rewrites the Pod's `spec` when the containers' actual state diverges from it

Two sections at once

22. [retrieval: ch2] A Pod declares three init containers and two app containers. The second init container cannot pull its image and has been retrying for ten minutes. What phase does the Pod report?

A) Running — the init container is starting and restarting, and both count toward Running
 B) Pending
 C) Failed — the image pull has failed repeatedly
 D) Succeeded — the first init container completed successfully

23. A Pod has three containers. One has terminated successfully; the other two are still running normally. What phase does the Pod report?

A) Succeeded — a container terminated in success
 B) Running
 C) Failed — a container is no longer running
 D) Unknown — the containers are in disagreeing states

Answers with Explanations

1. B — One, assigned to the Pod. Each Pod gets its own unique cluster-wide IP address, and the Pod's network namespace is shared by all containers within it [source: k8s-docs-network-model-2026-08-23]. A is the Docker-carry-over misconception, the easiest error to make on this topic. C invents a hybrid that doesn't exist. D confuses addressing with discovery: Pods do have IPs, and Services exist because those IPs are unstable, not because they're absent.

2. B — localhost networking and a shared volume. These are the two coupling mechanisms the Pod's shared context provides [source: k8s-docs-network-model-2026-08-23] [source: k8s-docs-volumes-2026-08-23]. They are also the two-part test from §2: if a sidecar needs neither, it doesn't need to be in the same Pod. A names two objects that work between *any* Pods, requiring no co-location. C is half right, since the IP is shared, but process-namespaces sharing is not the default coupling mechanism the Pod provides. D mixes a configuration mechanism with a network-policy object; neither is Pod-internal.

3. C — They would scale, fail, and be replaced as a single unit. Everything in a Pod shares fate. Three components with three different scaling profiles want three Pods. A inverts the chapter's central fact: Kubernetes does **not** schedule containers separately. The Pod is what gets placed [source: k8s-docs-kube-scheduler-2026-08-23], and grouping them guarantees co-location. If this looked right, re-read §1; it's the misconception the whole chapter is built to correct. B is a non-objection stated as one; they'd share one IP, and that's fine if they were genuinely coupled. D is wrong in its second clause: probes are configured per container, not per Pod. It's a scope error in the opposite direction from the ones §5 warns about, over-generalizing "everything in a Pod is Pod-level" from `restartPolicy` and `phase` to fields that genuinely are per-container.

4. C — Init containers run in sequence, each to successful completion; then the app containers start together. Init containers run before the app containers, in declaration order, each completing successfully before the next begins. App containers then start in parallel. A and B both get the init containers' parallelism wrong: they are strictly sequential, and that ordering guarantee is their entire

purpose. *D* is wrong twice over. It inverts the sequence *and* confuses "running" with "Ready." Readiness is a probe verdict (§7), not a consequence of the container having started.

5. C — It is an identifying attribute in metadata that other objects can select on. Labels are key/value pairs attached to objects, intended to specify identifying attributes, and used to organize and select subsets of objects [source: k8s-docs-labels-selectors-2026-08-23]. *A* confuses labels with names: a Pod's name is unique within a namespace; labels are deliberately non-unique. *B* confuses labels with status; labels live in metadata and you write them. *D* describes `nodeSelector`, which is about *node* labels constraining placement — Kubernetes only schedules the Pod onto nodes that have each of the labels you specify [source: k8s-docs-assign-pod-node-2026-08-23] — not about Pod labels. Hold onto the correct answer: it's the mechanism ReplicaSets use to know which Pods are theirs, and the mechanism a Service uses to know which Pods to route to.

6. B — Marked for deletion after a timeout, and replaced by a new Pod with a different UID. If a node dies, the Pods running on it are marked for deletion after a timeout; a Pod is never rescheduled to a different node but is replaced by a new, near-identical Pod with a different UID [source: k8s-docs-pod-lifecycle-2026-08-23]. *A* is the "rescheduled" misconception; Kubernetes never moves a Pod. *C* misreads phase as a live measurement. A Pod on an unreachable node typically reports `Unknown`, and in any case the control plane acts rather than waiting indefinitely. *D* is the residual half of the same misconception as *A*: it accepts that the Pod doesn't move but still assumes the *object* survives. It doesn't. The UID is what tells you so. A UID distinguishes between historical occurrences of similar entities [source: k8s-docs-names-and-uids-2026-08-24], and the replacement is a different occurrence.

7. C — Pending. Pending explicitly includes the time a Pod spends waiting to be scheduled *and* the time spent downloading container images over the network [source: k8s-docs-pod-lifecycle-2026-08-23]. *A* is the trap: `Waiting` is a **container state**, not a Pod phase. The container in this scenario *is* `Waiting`, but the question asked for the phase, and mixing the vocabularies is exactly the error this chapter warns about. *B* applies when the state can't be obtained at all. *D* requires all containers created and at least one running.

8. B — Reason. The `Waiting` state carries a `Reason` field that summarizes why the container is in that state [source: k8s-docs-pod-lifecycle-2026-08-23]. *A* is Pod-scoped and one word; it can't distinguish an image-pull failure from a missing Secret. *C* and *D* are fields recorded on `Terminated` and `Running` respectively; a `Waiting` container has neither, because it hasn't started.

9. D — Running. A Pod is `Running` when it's bound to a node, all containers have been created, and at least one container is running **or is in the process of starting or restarting** [source: k8s-docs-pod-lifecycle-2026-08-23]. A crash-looping container is always in one of those states. *A* — `Failed` requires that **all** containers have terminated, with at least one having terminated in failure and **not** set for automatic restarting. A crash-looping container is being restarted, so the Pod hasn't reached a terminal phase at all. *B* — `Pending` ends once the Pod is bound to a node and all of its containers have been **created**. This container has been created repeatedly; the Pod left `Pending` on the very first attempt and never returns to it. *C* — `Unknown` reports the control plane's inability to *obtain* the Pod's state, typically because it can't communicate with the node [source: k8s-docs-pod-lifecycle-2026-08-23]. Here the node is reporting fine,

and what it is reporting is a crash loop. "The cluster can't see it" and "the app can't stay up" are different failures. This is the costliest trap in the chapter: the phase is genuinely, correctly `Running`, and it tells you nothing about whether the application works.

10. B — Pod-level, applies to all containers, defaults to `Always`. The Pod's spec has a `restartPolicy` field with values `Always` (default), `OnFailure`, and `Never`, and it applies to all containers in the Pod [source: [k8s-docs-pod-lifecycle-2026-08-23](#)]. A and D both place the field at container scope, which is the documented misconception; D additionally invents a dependency on liveness probes that doesn't exist. C claims a **total exemption** for init containers, and there is none. A Pod-level `Never` policy is precisely what makes a failing init container fatal to the whole Pod, which is §3's failure case.

11. C — Capped at five minutes; resets after 10 minutes of trouble-free execution. The kubelet restarts with exponential backoff (10s, 20s, 40s, ...) capped at five minutes, and resets the backoff timer once a container has executed for 10 minutes without any problems [source: [k8s-docs-pod-lifecycle-2026-08-23](#)]. A invents a retry ceiling; the kubelet keeps trying. B misses the cap, which is the whole point of having one. D describes a fixed-interval retry, which is what exponential backoff exists to avoid.

12. B — The default `ServiceAccount` of the payments namespace. If you deploy a Pod in a namespace and don't manually assign a `ServiceAccount`, Kubernetes assigns that namespace's default [source: [k8s-docs-service-accounts-2026-08-23](#)]. A is the intuitive answer and it's wrong; every Pod gets an identity. C misses the namespaced scope: every namespace gets its *own* default `ServiceAccount` on creation, and they are distinct objects. D invents a validation requirement.

13. B — Legacy long-lived form; short-lived rotating tokens via `TokenRequest` are recommended since v1.22. In Kubernetes v1.22 and later, Kubernetes gets a short-lived, automatically rotating token using the `TokenRequest` API and mounts it as a projected volume; long-lived `ServiceAccount` token Secrets are not recommended [source: [k8s-docs-service-accounts-2026-08-23](#)] [source: [k8s-docs-secret-2026-08-23](#)]. A inverts current guidance. C is false; Secrets are namespaced like other Secret types. D overstates it: the type still exists and is still valid; it's discouraged, not removed.

14. B — The Pod's IP is removed from the endpoints of all matching Services; the container keeps running. If a readiness probe fails, the endpoints controller removes the Pod's IP address from the endpoints of all Services that match the Pod [source: [k8s-docs-pod-lifecycle-2026-08-23](#)]. Nothing is killed. A describes liveness. C invents a phase transition that doesn't occur; readiness has no effect on phase. D is the "they do the same thing" error, combining both probes' consequences into one.

15. B — The kubelet kills the container, which is then subject to its restart policy. That is the documented liveness behavior [source: [k8s-docs-pod-lifecycle-2026-08-23](#)]. A describes readiness. This and question 14 are deliberately mirrored, because reversing them is the easiest probe error to make. C misuses `Unknown`, which is about the control plane's inability to obtain state. D invents a fixed threshold; the number of consecutive failures required is `failureThreshold`, which is configurable.

16. C — They are disabled until the startup probe succeeds. All other probes are disabled if a startup probe is provided, until it succeeds [source: [k8s-docs-pod-lifecycle-2026-08-23](#)]. A is the intuitive default

and it's the trap. *B* invents a softened behavior; the suppression is complete, not partial, and that completeness is the point: a slow-starting application must not be killed by a liveness probe before it has started. *D* over-corrects in the other direction. The suppression **lifts** the moment the startup probe succeeds, at which point liveness and readiness take over for the rest of the container's life. A startup probe that permanently disabled the other two would leave you with no health checking at all.

17. B — Any probe type can use any of the four mechanisms. The four check mechanisms — `exec`, `grpc`, `httpGet`, and `tcpSocket` — are listed independently of the three probe types [source: `k8s-docs-pod-lifecycle-2026-08-23`]. *A* and *C* each invent a constraint pairing a type to a mechanism; keeping "how it asks" and "what the answer does" in separate compartments is what makes §7 manageable. *D* is a different error, a scope error rather than a pairing error: an `exec` probe executes its command **in the container being probed**, which is the only way it could tell you anything about that container's health.

18. B — Failure. Success requires a status code ≥ 200 and < 400 . An `httpGet` probe is considered successful if the response has a status code greater than or equal to 200 and less than 400 [source: `k8s-docs-pod-lifecycle-2026-08-23`]. 404 falls outside that range. *A* confuses "the server is reachable" with "the probe passed"; reachability is what `tcpSocket` tests. *C* invents a 5xx-only rule. *D* misapplies `successThreshold`, which governs how many consecutive *successes* are needed to flip a verdict, not what counts as a success.

19. C — 0.4 bytes. `M` means megabytes while `m` means millibytes; a request of `400m` of memory is a request for 0.4 bytes [source: `k8s-docs-resource-management-2026-08-23`]. *A* would require `400M`. *B* would require `400Mi`; note that the power-of-two suffixes are the ones you actually want most of the time. *D* imports the CPU meaning of `m` into a memory field, which is exactly how this mistake gets made in the wild: for CPU, `m` is correct and idiomatic, `0.1` is equivalent to `100m`, and `1m` is as fine-grained as the vocabulary goes [source: `k8s-docs-resource-management-2026-08-23`]. The habit carries across, and the field doesn't stop you.

20. B — A failed Pod is not repaired; it is replaced by a new Pod with a different UID. Containers are intended to be immutable — build a new image and recreate the container rather than changing a running one [source: `k8s-docs-containers-2026-08-23`] — and Pods follow the same rule at their own level: never rescheduled, always replaced, with a different UID [source: `k8s-docs-pod-lifecycle-2026-08-23`]. *A* contradicts the pattern rather than extending it. *C* misunderstands `status` as writable. *D* describes an `imagePullPolicy` behavior and has nothing to do with immutability as a design principle.

21. B — The kubelet runs a reconciliation loop that keeps the containers described in the Pod's spec running, restarting them per the restart policy when they exit. The kubelet runs a reconciliation loop that keeps the containers described in the Pod spec running [source: `k8s-docs-pod-lifecycle-2026-08-23`]; controllers are control loops that watch state and act to move current state toward desired state [source: `k8s-docs-controllers-2026-08-23`]. *A* is wrong on the architecture: nodes reach the control plane only through the API server, in a hub-and-spoke pattern [source: `k8s-docs-control-plane-node-communication-2026-08-24`], and the kubelet never talks to `etcd`. *C* describes the `kube-scheduler`, not the kubelet [source: `k8s-docs-kube-scheduler-2026-08-23`]. *D* inverts the direction of the loop: controllers

change reality to match spec, never spec to match reality. That inversion is the single most important thing to *not* believe about how Kubernetes works.

22. B — Pending. This one needs §3 and §5 together. Pending means the Pod has been accepted by the cluster but one or more of its containers has not been set up and made ready to run, and it explicitly includes time spent downloading container images over the network [source: k8s-docs-pod-lifecycle-2026-08-23]. A container that cannot pull its image is in the `Waiting` state with the Reason `ImagePullBackOff` [source: k8s-docs-images-2026-08-23], and the app containers behind it have never been created at all. A is the trap, and it's a good one: it borrows the reasoning from question 9, which is correct there and wrong here. Running requires that **all** of the Pod's containers have been created. Question 9's crash-looping container had been created (repeatedly); these app containers never have, because the init sequence hasn't completed. Same rule, opposite outcome, and the difference is the word "created." C — `Failed` requires all containers terminated with at least one having failed and no automatic restarting. This one is still retrying. D — nothing has succeeded; the first init container completing is not the Pod completing, and `Succeeded` requires **all** containers to have terminated in success.

23. B — Running. A Pod is Running when it has been bound to a node, all of its containers have been created, and **at least one** container is still running, starting, or restarting [source: k8s-docs-pod-lifecycle-2026-08-23]. Two of three are running; that satisfies "at least one" comfortably. A is the quantifier trap and the reason this question exists. `Succeeded` requires that **all** containers in the Pod have terminated in success [source: k8s-docs-pod-lifecycle-2026-08-23]. One is not all. Readers who drop the "all" from phase definitions get both this and C wrong for the same reason. C — `Failed` also requires **all** containers terminated, with at least one having terminated in failure. Neither half holds: two are still running, and nothing failed. D — `Unknown` is about the control plane being unable to *obtain* the Pod's state, not about its containers being in different states. Containers in one Pod sitting in different states is the normal case, not an anomaly. It's the whole reason phase and state are separate vocabularies.

Chapter Summary

Concept	Remember This
Pod	The unit Kubernetes actually schedules. A set of one or more containers, co-located and co-scheduled on one node.
Shared context	One IP per Pod, shared by every container; containers talk over localhost; volumes can be shared. This is <i>why</i> the wrapper exists.
PodSpec	Just the <code>spec</code> field of a Pod. The thing the kubelet reads and reconciles against.
Multi-container Pod	Justified only by localhost or a shared volume. Sidecar is the name for the helper. If neither mechanism is needed, use two Pods.
Init containers	Run in declared order, to successful completion, all of them, before any app container starts. A failure stops the Pod cold.
Pod lifetime	Created, assigned a UID, scheduled once. Never rescheduled — replaced, with a new UID.
Pod phase	Pending, Running, Succeeded, Failed, Unknown. One per Pod, in status.
Container state	Waiting (+ Reason), Running (+ startedAt), Terminated (+ exit code). One per container.
The critical distinction	Phase is Pod-scoped; state is per-container. Running includes starting and restarting, so a crash-looping Pod reports Running.
The quantifiers	Running needs all containers created and at least one running. Succeeded and Failed both need all containers terminated. Drop an "all" and you get the phase wrong.
restartPolicy	Pod-level, applies to all containers. Always (default), OnFailure, Never. Backoff 10s/20s/40s..., capped at 5 min, resets after 10 trouble-free minutes.
ServiceAccount	Every Pod has one. Unset means the namespace's default, which has essentially no permissions. Set via <code>spec.serviceAccountName</code> .
Liveness probe	Fails → kubelet kills the container → restart policy applies.
Readiness probe	Fails → Pod IP removed from matching Services' endpoints. Nothing is killed.
Startup probe	While configured and not yet succeeded, disables the other two. Fails → kill + restart policy. Succeeds → the other two resume.
Probe mechanisms	exec, httpGet, tcpSocket, grpc. Independent of the types — any type, any mechanism.
Request	What the kube-scheduler reads to place the Pod. A reserved floor. Exceeding it is allowed if the node has room.
Limit	What the kubelet enforces on the running container. A hard ceiling.
CPU vs memory enforcement	CPU limits throttle (you get slow). Memory limits OOM-kill (you get dead — reactively, under node pressure).
Units	1 cpu = 1 core; 0.1 = 100m, no finer than 1m; CPU is absolute across node sizes. Memory in bytes with M/mi-style suffixes — and m means <i>milli</i> bytes.

Concept	Remember This
The whole chapter, in one line	The unit of scheduling wraps containers instead of being one. Everything else follows.

The Voyage Ahead

You now know what the disposable thing is. You know it gets one IP, one identity, one phase, and one restart policy; that its containers each get their own state; that it is scheduled once and then, when something goes wrong, thrown away rather than fixed.

Which leaves the obvious problem hanging. **If Pods are designed to be replaced, who does the replacing?**

Not the Pod; it's gone. Not you, unless you plan to sit watching a terminal forever. Something has to hold the intent that outlives any individual Pod: the knowledge that three replicas were wanted, the template describing what a replacement should look like, and the loop that notices when reality has fallen short.

Chapter 5 introduced the disposable thing. Chapter 6 introduces what holds the intent [*cross-bearing: see Ch 6 §1 — Deployments, ReplicaSets, and the Pod template*]. It's where the shape of every real Kubernetes workload finally appears, and where you'll find out that the Pod you spent this chapter learning is something you will almost never create directly.

You'll also start seeing this chapter's material used rather than taught. Probes are what make a rolling update safe. Labels are how a controller finds the Pods it owns. And the requests and limits from §8 will come back in Chapter 7 as the thing the scheduler actually filters on.

"A vessel that cannot be repaired at sea must be a vessel you are willing to lose. Build accordingly — and keep the plans."

⚓ **Safe Harbor** — Chapter 5 complete.

You can now read a Pod's status and know what it's telling you, which is the first genuinely diagnostic skill in this book. Nine sections, six figures, and one design decision that turned out to explain all of it.

Voyage Progress: 🏠 → 🌊 → ⚓ — Chapter 5 of 20 complete.

Chapter 6: Fleets, Not Vessels

"Nobody sails one Pod"

Domain: Kubernetes Fundamentals — Kubernetes Core Concepts [source: cncf-kcna-curriculum-pdf-2026-08-23] | Estimated share of the exam: ~6% (authored allocation — CNCF publishes domain weights, not competency weights [source: cncf-kcna-curriculum-pdf-2026-08-23]; see front matter) **Complexity: Mixed | Novelty: Moderate | Prerequisites: Chapters 3, 4, 5

Attention Budget

Total time: ~2 hours | Recommended: split across two sessions

Section	Time	Attention Cost	Best Time to Study
§1 — The Resource That Holds the Intent	10 min	Medium	Mid-session
§2 — A Loop You Can Watch Working	7 min	Low	Anytime
§3 — How a Controller Knows Its Own	12 min	Medium	Mid-session
🕒 Taking Your Bearings #1	5 min	Medium	After a brief pause
§4 — Changing the Fleet Under Way	18 min	High	Peak attention
§5 — Every Rollout Is a Revision	11 min	Medium	Peak attention
🕒 Taking Your Bearings #2	5 min	Medium	After a brief pause
§6 — When Pods Are Not Interchangeable	9 min	Medium	Mid-session
§7 — One Per Node, and Work That Ends	10 min	Low	Anytime
§8 — The Control Loop, Extended	12 min	Medium-high	Peak attention
🕒 Taking Your Bearings #3	5 min	Medium	After a brief pause
§9 — Nobody Sails One Pod (the chapter's ☀ Zenith)	4 min	Low	Anytime
Exam Alert + Practice Questions	25 min	Medium	Separate session

Attention Cost Key:

- **Low:** concrete, familiar concepts. Study anytime.
- **Medium:** new concepts requiring focus. Study when alert.
- **High:** abstract or dense material with arithmetic. Study at peak attention.

Recommended split point: after ☺ Taking Your Bearings #2. Sections 1 through 5 are the Deployment arc from end to end. Sections 6 through 9 are the rest of the family and what the pattern turns into.

If you only have 15 minutes: read §1, jump to the decision tree at the close of §7, then work ☺ Taking Your Bearings #3. That is the highest-leverage route through this chapter.

"A standing order outlives the watch that received it. That is the whole reason you write it down." —
Lodestar Ledgers

☺ Soundings

Before you read this chapter, work these eight. Your score tells you how to read. No shame attached to any score; the three tiers are three reading plans, not three grades. Every question here is answerable from Chapters 3 through 5 or from operational experience you already have. None of them require anything this chapter teaches.

1. You want three copies of a process running on a machine at all times. One of them dies at 3 a.m. What would have to be written down *in advance* for something else to restore it without waking you up?
2. You need to replace a running service with a new version while it stays reachable the entire time. Describe how you have done this, or how you would do it, with tools you already know.
3. There are two ways to identify a group of things: an explicit list of names, or a rule about a property they share. What does each approach cost you when the group changes while you are not looking?
4. A two-member database: one primary, one replica, each with its own data on disk. What actually breaks if you swap which machine is which? Hostnames, storage, and all.
5. A log-collection agent has to run on every machine in your fleet, including the machines that will be added next Tuesday. How would you express that requirement, as opposed to "run six copies"?
6. Your init system supervises two things: a web server that should never exit, and a nightly backup script that should exit. How does it treat them differently, and what does "healthy" mean for each?
7. **[retrieval: ch3]** A control loop compares two things and acts on the difference. Name both, and say what the loop does when they match.
8. **[retrieval: ch5]** A Pod's node dies. Chapter 5 was emphatic that the Pod is not rescheduled. What did it say happens instead, and what did it say was left unanswered?

Answers + reading strategy

1. **The count and the recipe.** Something has to know how many copies you wanted and what a copy looks like, or it cannot tell that one is missing or build a replacement. *Count it right if your answer named both — a quantity and a description.*
2. **Any overlap-based answer is correct.** Add new instances to a load-balancer pool before removing old ones; drain and swap one at a time; run two environments and flip DNS. The common shape is that old and new coexist for a window.
3. **A list is exact but goes stale;** you must maintain it by hand every time membership changes. **A rule stays current** but can silently capture things you did not intend, because you never enumerated what it would match. *Count it right if you named a cost on both sides.*
4. **Nearly everything.** Hostnames the replica uses to find the primary, the on-disk data each one expects to open, replication position, client connection strings. The two machines are not copies of each other. They hold different roles and different bytes. *Count it right if you named storage as well as naming.*
5. **Express it as a property of the machine, not as a number:** "one agent per host" rather than "six agents."
6. **The web server exiting is a failure; the backup script exiting is success.** For the web server, healthy means still running. For the backup, healthy means finished with a zero exit status and gone. *Count it right if you inverted the meaning of "exited" between the two.*
7. **Desired state and current state.** When they match, the loop does nothing, and keeps watching. Doing nothing is a valid output of a control loop, not a pause in it. *Count it right only if you named both states.*
8. **Chapter 5 said the Pod is replaced, not moved:** a new, near-identical Pod with a different UID. What it left unanswered was *who does the replacing*. That is the question this chapter opens on. *Count it right if you named the replacement and the open question.*

If you got 6 or more right: skim. Read the ☆ Fixed Points, the △ Navigational Hazards blocks, and §4 in full. The two rolling-update bounds and their defaults are the easiest pair in this chapter to get backwards. Then work all three ☺ Taking Your Bearings checkpoints.

If you got 3 to 5 right: read at normal pace. This chapter is calibrated for you. Do not skip §3; it is short and everything after it depends on it.

If you got 0 to 2 right: read carefully, and take the recommended split after ☺ Taking Your Bearings #2. **If questions 7 and 8 were among your misses, re-read Chapter 3 §6 and Chapter 5 §4 before you start §2.** Those two sections are the debts this chapter collects, and §2 will read as a vocabulary drill without them.

Why This Chapter Matters

Chapter 5 ended on a question it refused to answer: if Pods are designed to be replaced rather than repaired, who does the replacing? Here is the answer, and it is stranger than "a thing called a Deployment." Kubernetes has no special replacement machinery at all. It has the same control loop you met in Chapter 3, handed a count to hold and a template to copy from. Rolling updates, rollbacks, one-agent-per-node, scheduled batch work, and the operator that runs your database are all that one loop with different desired state plugged into it. This chapter has fewer ideas in it than its table of contents suggests. By §9 you will be able to see that.

What actually changes here is what kind of person you are at a terminal. Chapter 5 made you someone who can read what infrastructure is telling you. This chapter makes you someone who states what should be true and hands the system responsibility for it. That is the real difference between someone new to Kubernetes and someone who isn't. Newcomers reach for the Pod, then for a script that recreates the Pod, then for a cron entry that checks whether the script ran. Practitioners write down the count and the template and go home. That is a standing order: set down once, carried out by whoever has the watch, long after the person who wrote it has gone below. If you have ever written one of those scripts instead, you already know what it costs to maintain.

The stakes, flat: this competency is roughly six percent of the exam, by this book's authored allocation, and that number understates the chapter twice. First, this is where the book's spine passes through. Chapter 3 introduced the control loop, this chapter instantiates it, and Chapter 15 generalizes it to a loop whose desired state lives in a Git repository. A reader who does not feel the shape here will meet Chapter 15's synthesis as a fifth list to memorize instead of as a recognition. Second, the workload-resource decision (Deployment or StatefulSet or DaemonSet or Job) reduces to a handful of questions asked in the right order, and the three misconceptions that surround it can all be defused by one figure and one rule.

What You'll Learn

By the end of this chapter, you'll be able to:

- **Trace** the ownership chain from a Deployment down to a running Pod, and say which layer holds the count and which holds the template.
- **Explain** how a controller knows which Pods belong to it, what the API does when you get that wrong, and what happens to Pods that end up belonging to nobody.
- **Predict** what a cluster does during a rolling update, given `maxSurge` and `maxUnavailable`, and name the thing that makes the update safe rather than merely gradual.
- **Distinguish** the six workload resources by the one property that separates each from its nearest neighbor.
- **State** what actually creates a new revision, what looks like it should but doesn't, and what you give up when you stop retaining revisions.

- **Define** a custom resource, a custom controller, and the pattern that is the two of them together.

You'll also stop reaching for `kubectl run`, which is a smaller change than it sounds and is the point of the whole chapter.

§1 — The Resource That Holds the Intent

Who does the replacing? Something that already knows how many Pods you wanted and what one of them looks like.

Here is the answer in the documentation's own words, and it is worth reading twice: you don't need to manage each Pod directly. Instead, you use workload resources that manage a set of Pods on your behalf, and these resources configure controllers that make sure the right number of the right kind of Pod are running, to match the state you specified [source: k8s-docs-workloads-2026-08-23]. That sentence is this entire chapter in miniature. The rest of §1 is the unpacking.

[cross-bearing: see Ch 5 §4 — a Pod is replaced, never rescheduled] Chapter 5 handed you this question deliberately, and this is where it gets discharged. The Pod that vanished with its node is not recovered. Something notices it is gone and builds another one from a stored description.

Three layers, not two

The workload resource you will reach for most often — and the one the documentation recommends over managing ReplicaSets yourself [source: k8s-docs-replicaset-2026-08-24] — is the **Deployment**. A Deployment manages a set of Pods to run an application workload, usually one that doesn't maintain state, and it provides declarative updates for Pods *and* ReplicaSets [source: k8s-docs-deployment-2026-08-23]. That second half is the part readers skim past, and it is the structural key to the next four sections.

There are three layers between you and a running container, not two. You create a Deployment; the Deployment creates a **ReplicaSet**; the ReplicaSet creates Pods in the background [source: k8s-docs-deployment-2026-08-23]. Usually you define a Deployment and let it manage ReplicaSets automatically. The documentation's own recommendation is that you may never need to manipulate a ReplicaSet object directly [source: k8s-docs-replicaset-2026-08-24].

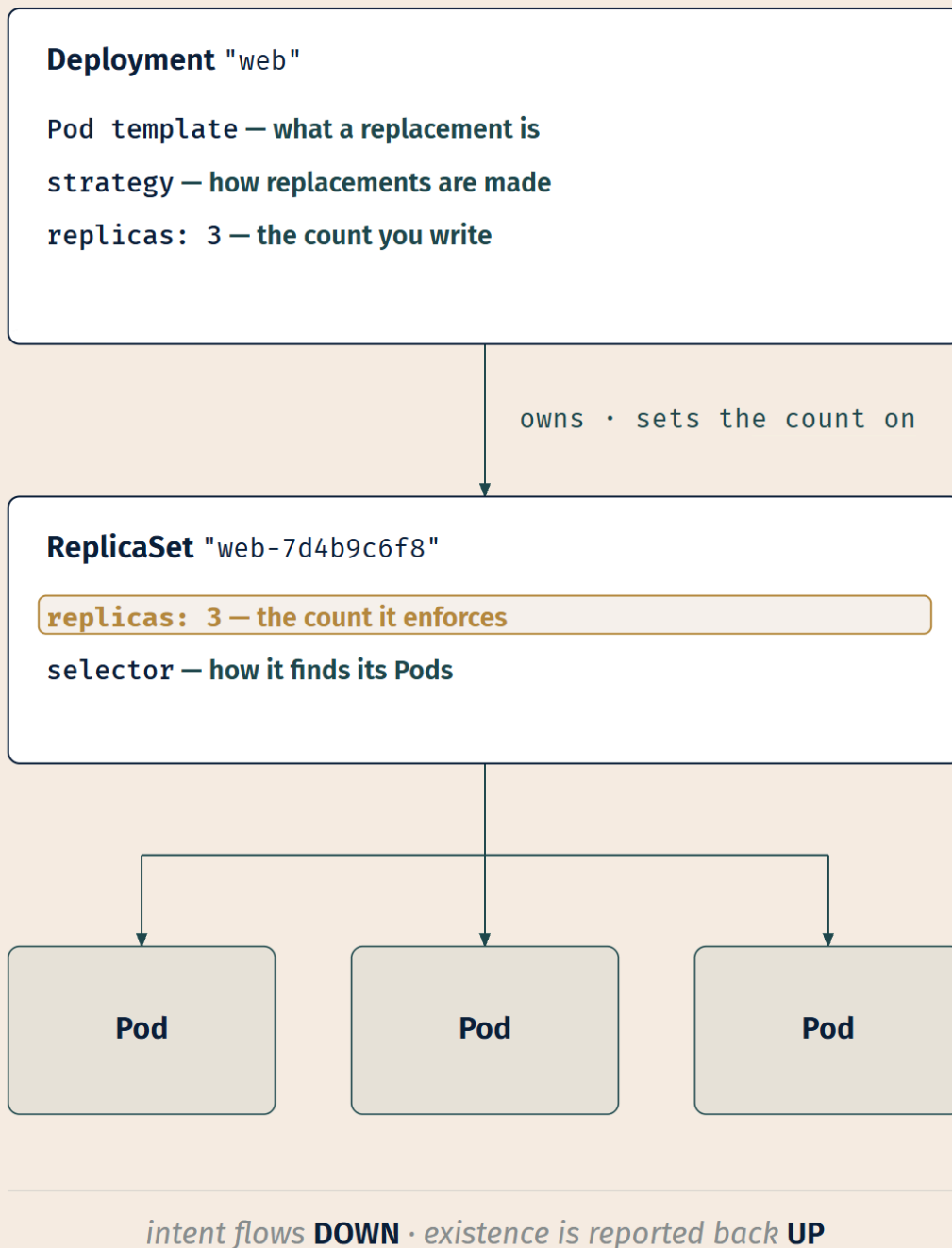


Figure 6.1 — the ownership chain. Notice what lives where. The Deployment is the layer that knows what a Pod should look like and how to replace one; the ReplicaSet is the layer that keeps a specific number of a specific kind alive. When §4 has two ReplicaSets running at once, this division is the reason it works.

☆ Fixed Point:

The chain is Deployment → ReplicaSet → Pod. The Deployment owns the Pod template and the update strategy. The ReplicaSet owns a replica count and enforces it. The Pods run.

You write the count on the Deployment. The Deployment sets it on the ReplicaSet it currently considers current — §4 shows what happens when there are two. Both objects carry a `replicas` field [source: k8s-docs-deployment-spec-fields-2026-08-24] [source: k8s-docs-replicaset-2026-08-24], but only one of them is the layer where the count is *acted on*, and that is the ReplicaSet.

That last clarification matters more than it looks. `.spec.replicas` on a Deployment is an optional field specifying the number of desired Pods, defaulting to 1 [source: k8s-docs-deployment-spec-fields-2026-08-24]. `.spec.replicas` on a ReplicaSet specifies how many Pods should run concurrently, and the ReplicaSet creates or deletes its Pods to match that number, also defaulting to 1 [source: k8s-docs-replicaset-2026-08-24]. Same field name, two altitudes: one is where you express the wish, the other is where the wish is enforced. The simple version, Deployment holds the template and ReplicaSet holds the count, is the version to carry into §4 and §5. The full picture is that the number is written twice, and the Deployment is the author of the second copy.

The template

A ReplicaSet is defined with a selector that specifies how to identify Pods it can acquire, a number of replicas indicating how many Pods it should be maintaining, and a **Pod template** specifying the data of new Pods it should create; when it needs to create a new Pod, it uses that template [source: k8s-docs-replicaset-2026-08-24].

Everything Chapter 4 taught you about an object's four required fields is unchanged. Everything Chapter 5 taught you about a Pod's `spec` is unchanged. It has simply moved one nesting level down: a Pod template has exactly the same schema as a Pod, except that it is nested and carries no `apiVersion` or `kind` of its own — the DaemonSet page states that rule in those words [source: k8s-docs-daemonset-2026-08-24]. You do not need that developed. You need it *located*. [cross-bearing: see Ch 4 §2 — `apiVersion`, `kind`, `metadata`, `spec`]

The reframe

Here is the sentence this section exists to deliver. The Pod you spent all of Chapter 5 learning is an object you will almost never create directly. Not because Pods are unimportant. They are still the only thing that actually runs. But being the thing that gets created *for* you is what a Pod is for. A ReplicaSet replaces Pods that are deleted or terminated for any reason, such as node failure or disruptive node maintenance like a kernel upgrade; a Pod you created by hand does not get that [source: k8s-docs-replicaset-2026-08-24].

🔒 **Worth Securing:** If you find yourself writing a bare Pod manifest for anything other than a one-off experiment, you have picked the wrong object. The question to ask is not "how do I run this container" but "what should stay true about this container."

One piece of vocabulary you may meet in older material: **ReplicationController**. It is the legacy API for managing workloads that can scale horizontally, superseded by the Deployment and ReplicaSet APIs; a Deployment that configures a ReplicaSet is now the recommended way to set up replication [source: k8s-docs-replicationcontroller-2026-08-24]. ReplicaSets are its successors, serving the same purpose and behaving similarly, except that a ReplicationController does not support set-based selector requirements [source: k8s-docs-replicaset-2026-08-24]. Recognize the word, know it is superseded, move on.

[cross-bearing: see Ch 14 — a Helm chart's job is to template this object]

.. §2 — A Loop You Can Watch Working

Chapter 3 promised you a control loop you could watch working in real time. This is it.

A ReplicaSet's purpose is to maintain a stable set of replica Pods running at any given time; it is often used to guarantee the availability of a specified number of identical Pods [source: k8s-docs-replicaset-2026-08-24]. Set that against what Chapter 3 taught: a controller tracks at least one Kubernetes resource type, those objects have a `spec` field representing desired state, and the controller is responsible for making the current state come closer to that desired state [source: k8s-docs-controllers-2026-08-23].

Fill in the blanks and you have a ReplicaSet. Desired state is `.spec.replicas`. Current state is how many matching Pods actually exist. When they differ, the ReplicaSet creates or deletes Pods until they don't [source: k8s-docs-replicaset-2026-08-24]. That is Chapter 3's thermostat with the numbers filled in.

[cross-bearing: see Ch 3 §6–§7 — the control loop, and why nobody is in charge]

The demonstration

Here is the part worth doing rather than reading. Delete a Pod that a ReplicaSet owns:

```
$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
web-7d4b9c6f8-mn4pq	1/1	Running	0	6m
web-7d4b9c6f8-qg8jz	1/1	Running	0	6m
web-7d4b9c6f8-x9k2r	1/1	Running	0	6m

```
$ kubectl delete pod web-7d4b9c6f8-mn4pq
pod "web-7d4b9c6f8-mn4pq" deleted

$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
web-7d4b9c6f8-qg8jz	1/1	Running	0	6m
web-7d4b9c6f8-x9k2r	1/1	Running	0	6m
web-7d4b9c6f8-z7rtc	0/1	ContainerCreating	0	2s

Nobody triggered that. No job ran. No alert fired and no runbook was consulted. A gap appeared between what you asked for and what existed, and a loop that had been watching the whole time closed it. The new Pod is not the old Pod recovered. It is a different Pod with a different name and a different UID [cross-bearing: see Ch 5 §4 — *replacement, not recovery*], built from the same template.

Worth noticing about the mechanics: the controller does not create that Pod itself. More commonly in Kubernetes, a controller sends messages to the API server that have useful side effects [source: k8s-docs-controllers-2026-08-23], and other components act on the new information. The "nobody is in charge" property Chapter 3 established holds at this altitude too. The ReplicaSet controller writes a wish to the API server and something else does the work.

Scaling is the same operation

A ReplicaSet can be scaled up or down by simply updating the `.spec.replicas` field; the controller ensures that the desired number of Pods with a matching label selector are available and operational [source: k8s-docs-replicaset-2026-08-24]. `kubectl scale` is a first-class verb for exactly that, updating the size of the specified deployment [source: k8s-docs-kubectl-overview-2026-08-23]:

```
$ kubectl scale deployment/web --replicas=5
deployment.apps/web scaled
```

Now look at what the loop just did. It saw a gap of two between desired and current, and it closed it. That is precisely what it did sixty seconds ago when you deleted a Pod, except the gap that time was one and it appeared for a different reason. **The loop cannot tell the difference.** "You asked for five and there are three" and "you asked for three and one died" are the same input.

 **Worth Securing:** Self-healing and scaling are not two features. They are one loop, seeing the same kind of gap, taking the same action. If you can hold that, you have most of what this chapter teaches.

Horizontal scaling means that the response to increased load is to deploy more Pods [source: k8s-docs-hpa-2026-08-24]. When it isn't you writing that number, it is usually a **HorizontalPodAutoscaler**: an API resource plus a controller, running in the control plane, that periodically adjusts the desired scale of its target to match observed metrics such as average CPU utilization [source: k8s-docs-hpa-2026-08-24]. Kubernetes implements it as a control loop that runs intermittently rather than continuously [source: k8s-docs-hpa-2026-08-24]. A ReplicaSet can be an HPA target directly, though in practice you point one at a Deployment [source: k8s-docs-replicaset-2026-08-24].

One sentence is all the HPA gets here. Where its metrics come from and what else can autoscale are later problems. *[cross-bearing: see Ch 13 — metrics-server is what an HPA reads] [cross-bearing: see Ch 17 — the autoscaling landscape]*

One last observation from Chapter 3, now with teeth. Your cluster could be changing at any point as work happens and control loops automatically fix failures, which means that potentially your cluster never reaches a stable state; and as long as the controllers are running and able to make useful changes, it doesn't matter whether the overall state is stable [source: k8s-docs-controllers-2026-08-23]. Kubernetes is not trying to arrive anywhere. It is trying to keep closing gaps.

[cross-bearing: see Ch 7 §1 — the Pod this loop just created still has to be placed on a node, and sometimes it can't be] [cross-bearing: see Ch 9 — this churn is exactly why something needs a stable name]

3 — How a Controller Knows Its Own

Chapter 4 taught you label selectors as the universal join and listed the places they get used. This is the first one collected, and it comes with a consequence Chapter 4 did not state.

Membership is a query

A ReplicaSet does not track its Pods by name and does not hold a list. Chapter 4 drew this join once already — figure ch04-fig03-labels-selectors-join — and the picture here is the same one, now with an owner on one side. It has a `.spec.selector`, a label selector, using the labels that identify potential Pods to acquire [source: k8s-docs-replicaset-2026-08-24]. The machinery is exactly what Chapter 4 gave you: set-based requirements are expressed through `matchExpressions`, and `matchLabels` is the equality-shaped shorthand, equivalent to `matchExpressions` with the operator `In`; newer resources such as Job, Deployment, ReplicaSet and DaemonSet support both [source: k8s-docs-labels-selectors-2026-08-23]. Nothing new to learn, one new place to apply it. *[cross-bearing: see Ch 4 §5 — labels and selectors, the universal join]*

Its Pods are whichever Pods match. Not "the Pods it made." The ones that match, right now, whoever made them.

Before reading on: the labels in the Pod template do not match the selector. The controller creates a Pod from the template, and then cannot see it. Work out what happens next before you read the answer. It takes about ten seconds and the answer is worth deriving yourself.

You got a runaway. Create a Pod, fail to see it, notice the count is still short, create another, and keep going, with no condition that ever stops it. That outcome is so obviously bad that Kubernetes refuses to let you build it. In a ReplicaSet, `.spec.template.metadata.labels` **must** match `spec.selector` or the object will be rejected by the API [source: k8s-docs-replicaset-2026-08-24]. Same for a Deployment: `.spec.selector` must match `.spec.template.metadata.labels`, or it will be rejected by the API [source: k8s-docs-deployment-spec-fields-2026-08-24]. Same for a DaemonSet: configuration with those two not matching is rejected [source: k8s-docs-daemonset-2026-08-24].

This is worth sitting with. It is one of the few validations Kubernetes performs by refusing the object outright rather than accepting it and reconciling toward it, and the reason is simple: what you wrote has no reachable steady state at all.

☆ Fixed Point:

A controller's Pods are the Pods matching its selector. Membership is a query, not a list. Because it is a query, the Pod template's labels must satisfy the selector, and the API rejects any workload object where they don't.

Ownership is a separate mechanism

Selectors answer "which Pods are mine right now." Owner references answer "who is responsible for this object." They are not the same thing, and the documentation is explicit that ownership is different from the labels and selectors mechanism [source: k8s-docs-garbage-collection-2026-08-24].

A ReplicaSet is linked to its Pods via the Pods' `metadata.ownerReferences` field, which specifies what resource the current object is owned by; all Pods acquired by a ReplicaSet carry the owning ReplicaSet's identifying information there, and it is through this link that the ReplicaSet knows the state of the Pods it maintains [source: k8s-docs-replicaset-2026-08-24]. Many objects in Kubernetes link to each other this way; owner references tell the control plane which objects are dependent on others, and Kubernetes manages them automatically in most cases [source: k8s-docs-garbage-collection-2026-08-24].

That link is what makes deletion tidy. Kubernetes checks for and deletes objects that no longer have owner references, like the Pods left behind when you delete a ReplicaSet, in a process called **cascading deletion**, and background cascading deletion is the default [source: k8s-docs-garbage-collection-2026-08-24]. In practice: delete the Deployment and its ReplicaSets and their Pods go too. To delete a ReplicaSet and all of its Pods you just use `kubectl delete`; the garbage collector automatically removes the dependent Pods by default [source: k8s-docs-replicaset-2026-08-24].

The opposite outcome has a name too. Dependents that survive their owner are called **orphan** objects, and by default Kubernetes deletes dependents rather than leaving them behind [source: k8s-docs-garbage-collection-2026-08-24]. The way you produce orphans by accident is by changing a selector, so

that Pods a controller used to claim stop matching it and nothing is left holding them. That route is hazardous enough that a DaemonSet's `.spec.selector` cannot be mutated at all once the object is created, precisely because mutating a pod selector can lead to the unintentional orphaning of Pods [source: k8s-docs-daemonset-2026-08-24].

Where the selector actually bites

The API stops you from misconfiguring a single controller. It does not stop two controllers from claiming the same Pods, and it does not stop a controller from claiming a Pod you made yourself.

A ReplicaSet identifies new Pods to acquire using its selector: if a Pod has no owner reference, or an owner reference that is not a controller, and it matches the selector, it will be immediately acquired [source: k8s-docs-replicaset-2026-08-24]. A ReplicaSet is not limited to owning Pods produced by its own template [source: k8s-docs-replicaset-2026-08-24]. So if you hand-create a Pod carrying labels that match some ReplicaSet's selector, that ReplicaSet adopts it; and if adopting it puts the ReplicaSet over its desired count, the newly adopted Pod is immediately terminated [source: k8s-docs-replicaset-2026-08-24].

🔖 **Snag:** You create a debugging Pod with `app: web` on it, because that is what the other Pods have. It is adopted by the ReplicaSet and killed within seconds. Nothing errors. Nothing warns you. Your Pod was simply surplus to a count somebody else was holding.

The documentation's own advice for the template is blunt: be careful not to overlap with the selectors of other controllers, lest they try to adopt this Pod [source: k8s-docs-replicaset-2026-08-24].

Logbook Entry: Overlapping selectors fail in two directions, and neither direction announces itself as a configuration mistake, which is what makes them expensive.

Somebody ships a second workload and copies the first one's manifest as a starting point, including its labels. Neither controller is misconfigured in any way the API can see; each one's template agrees with its own selector, which is all the API validates. But each is now querying for Pods the other created. The documented consequence is adoption: a Pod that matches a selector and has no controller owning it is immediately acquired, and if that acquisition puts the acquirer over its desired count, the Pod is immediately terminated [source: k8s-docs-replicaset-2026-08-24].

Then somebody notices the collision and fixes it the obvious way, by editing one of the selectors. That is the other direction. Pods the edited controller used to claim stop matching it, and nothing is left holding them — the unintentional orphaning the DaemonSet page warns about, and the reason a DaemonSet forbids the edit outright [source: k8s-docs-daemonset-2026-08-24].

The prevention is a single habit, and it costs nothing: give every workload one label that is genuinely unique to it, and never hand-write a selector by copying somebody else's.

[cross-bearing: see Ch 9 — a Service selects its backends with the same mechanism, which is a different controller reading the same labels] [cross-bearing: see Ch 12 — deleting a workload does not delete

everything it referenced]

🕒 Taking Your Bearings #1: Intent, Loops, and Ownership

Five questions on what holds the intent and how it finds its Pods. Two of them reach back to earlier chapters.

1. . Name the three layers between a Deployment and a running container process, and say which layer holds the replica count and which holds the Pod template.

A) Deployment → Pod. The Deployment holds both. B) Deployment → ReplicaSet → Pod. The Deployment holds the template and strategy; the ReplicaSet holds and enforces the count. C) Deployment → ReplicaSet → Pod. The ReplicaSet holds the template; the Deployment holds the count. D) Deployment → ReplicaSet → Pod. The Deployment holds both the count and the template; the ReplicaSet only watches and reports.

2. . You delete a Pod that a ReplicaSet owns. Describe what happens next and what caused it.

3. . You write a Deployment manifest in which the Pod template's labels do not match the Deployment's selector, and you `kubectl apply` it. What happens?

A) The Deployment is created and produces a growing population of Pods it cannot see. B) The Deployment is created but reports zero replicas forever. C) The API rejects the object. D) The selector is silently rewritten to match the template.

4. . **[retrieval: ch3]** Chapter 3 gave you a loop with two states and an action. Fill all three in for a ReplicaSet, using the actual field name for the desired state.

5. . **[retrieval: ch4]** In Chapter 9 you will meet a Service, which finds its backend Pods the same way a ReplicaSet does. What would it mean for one Pod to be selected by both at once?

Answers with Explanations

1. B.

- **A is wrong**, and it is the most consequential wrong answer in this chapter. If you believe a Deployment owns Pods directly, §4 will be incomprehensible, because a rolling update works by having *two* ReplicaSets alive at once, one shrinking and one growing, with the Deployment holding both. Delete the middle layer from your mental model and the mechanism has nowhere to live.
- **C is wrong** because it swaps the layers. The Deployment provides declarative updates for Pods and ReplicaSets [source: k8s-docs-deployment-2026-08-23]; the template and the update strategy are its business. The ReplicaSet is the layer whose whole purpose is maintaining a stable set of replica Pods [source: k8s-docs-replicaset-2026-08-24].

- **D is wrong** because it makes the ReplicaSet passive, which is the half-understanding worth naming. The ReplicaSet is not a bookkeeping record of what the Deployment did; it is the running controller that creates and deletes Pods to reach its own `.spec.replicas` [source: k8s-docs-replicaset-2026-08-24]. The Deployment does not touch Pods at all.
- Note the nuance from §1: `replicas` appears on both the Deployment and the ReplicaSet. B is right because the ReplicaSet is where the count is *enforced*.

2. The ReplicaSet's desired count and its current count no longer agree, so it creates a Pod from its template to close the gap. **The correct answer turns on nothing having been triggered.** No scheduled task ran, no alert fired, no operator intervened. A loop that was already running observed a difference and acted on it, the same thing it does when you scale up, for the same reason [source: k8s-docs-replicaset-2026-08-24].

If your answer was "the Pod restarts" or "Kubernetes brings it back," soften that wording now, because it will cost you in §6. Nothing came back. A *different* Pod, with a different name and a different UID, was built from the same template [source: k8s-docs-pod-lifecycle-2026-08-23]. That distinction is the entire reason StatefulSets exist.

3. **C.** The API rejects it. In a ReplicaSet, `.spec.template.metadata.labels` must match `spec.selector` [source: k8s-docs-replicaset-2026-08-24]; on a Deployment, `.spec.selector` must match `.spec.template.metadata.labels` [source: k8s-docs-deployment-spec-fields-2026-08-24]. Both sources say it in the same words: it will be rejected by the API.

- **A is wrong, but it is the *right* intuition.** The runaway is exactly what would happen if the object were accepted, which is why the API refuses it. If you picked A, you understood the mechanism and did not know about the validation. That is a good place to be missing a point from.
- **B is wrong** because it imagines the controller doing nothing. A selector query returning zero results is not an error condition; the controller would create Pods, not sit still.
- **D is wrong** because Kubernetes does not repair your intent by editing it. It either accepts a record of intent or rejects it.

4. Desired state: `.spec.replicas`. Current state: the number of Pods currently matching the ReplicaSet's selector. Action: create or delete Pods until the two agree [source: k8s-docs-replicaset-2026-08-24]. If your answer for current state was "the number of Pods it created," look again at §3. Membership is a query, and a Pod it never created can count toward the total.

5. Nothing unusual. They are independent queries over the same labels, and the Pod is simply a member of two sets. The ReplicaSet is asking "is this one of the Pods I am responsible for keeping alive"; the Service is asking "is this one of the Pods I should send traffic to." Neither query knows about the other and neither one owns the Pod on the other's behalf. The documentation draws this distinction explicitly for a related case: a Service uses labels to determine which EndpointSlice objects are used for it, and *in addition* each EndpointSlice carries an owner reference, because ownership and selection are different mechanisms doing different jobs [source: k8s-docs-garbage-collection-2026-08-24]. [*cross-bearing: see Ch 9 — EndpointSlice, the object behind a Service's endpoints*]

If you answered that this is a conflict, or that one of them must win, or that the Pod needs to be released by one before the other can have it, go back to §3's closing distinction. **Ownership is exclusive; selection is not.** A Pod can satisfy any number of independent queries, and none of them own it by doing so. Holding the wrong version of this makes Chapter 9's Services look like they are fighting the workload controller, which they never are.

Checkpoint: you've now mastered

✓ The ownership chain, and which layer holds which piece of intent ✓ Why a deleted Pod comes back without anyone doing anything ✓ That a controller finds its Pods by query, and what the API does when the query can't work ✓ That ownership and selection are two mechanisms, not one, and what a Pod belonging to nobody is called

□ How a running fleet gets replaced with a different one (next) □ What gets recorded when it does

..1 §4 — Changing the Fleet Under Way

This is the densest section in the chapter. It is also the one that pays a promise Chapter 5 made in its closing pages.

The mechanism

You declare the new state of the Pods by updating the Pod template on the Deployment. A new ReplicaSet is created, and the Deployment manages moving Pods from the old ReplicaSet to the new one at a controlled rate [source: k8s-docs-deployment-2026-08-23].

Read that again with §1's figure in front of you, because this is where the three-layer chain earns its keep. Two ReplicaSets exist simultaneously. The old one is being scaled down. The new one is being scaled up. Both are owned by the same Deployment, and each is doing nothing more exotic than what §2 showed you: holding a count and closing gaps. The Deployment's entire contribution is deciding, moment to moment, what those two counts should be.

A reader who has the ownership chain finds the rolling update obvious. A reader who does not finds it magic. *[cross-bearing: see Ch 6 §1 — the ownership chain]*

The two bounds

`.spec.strategy` specifies the strategy used to replace old Pods with new ones. `.spec.strategy.type` can be `Recreate` OR `RollingUpdate`, and **RollingUpdate is the default** [source: k8s-docs-deployment-2026-08-23]. A rolling update is shaped by two fields, and these are the two facts in this chapter most easily transposed:

`maxUnavailable` is the maximum number of Pods that can be unavailable during the update. The value can be an absolute number or a percentage of desired Pods. **The absolute number is calculated from a percentage by rounding down. The default value is 25%.** [source: k8s-docs-deployment-spec-fields-2026-08-24]

`maxSurge` is the maximum number of Pods that can be created over the desired number of Pods. The value can be an absolute number or a percentage. **The absolute number is calculated from a percentage by rounding up. The default value is 25%.** [source: k8s-docs-deployment-spec-fields-2026-08-24]

Neither one can be zero if the other is zero [source: k8s-docs-deployment-spec-fields-2026-08-24], which makes sense the moment you say it aloud, because a rollout that may neither exceed the desired count nor drop below it has no legal move to make.

Work one example, once

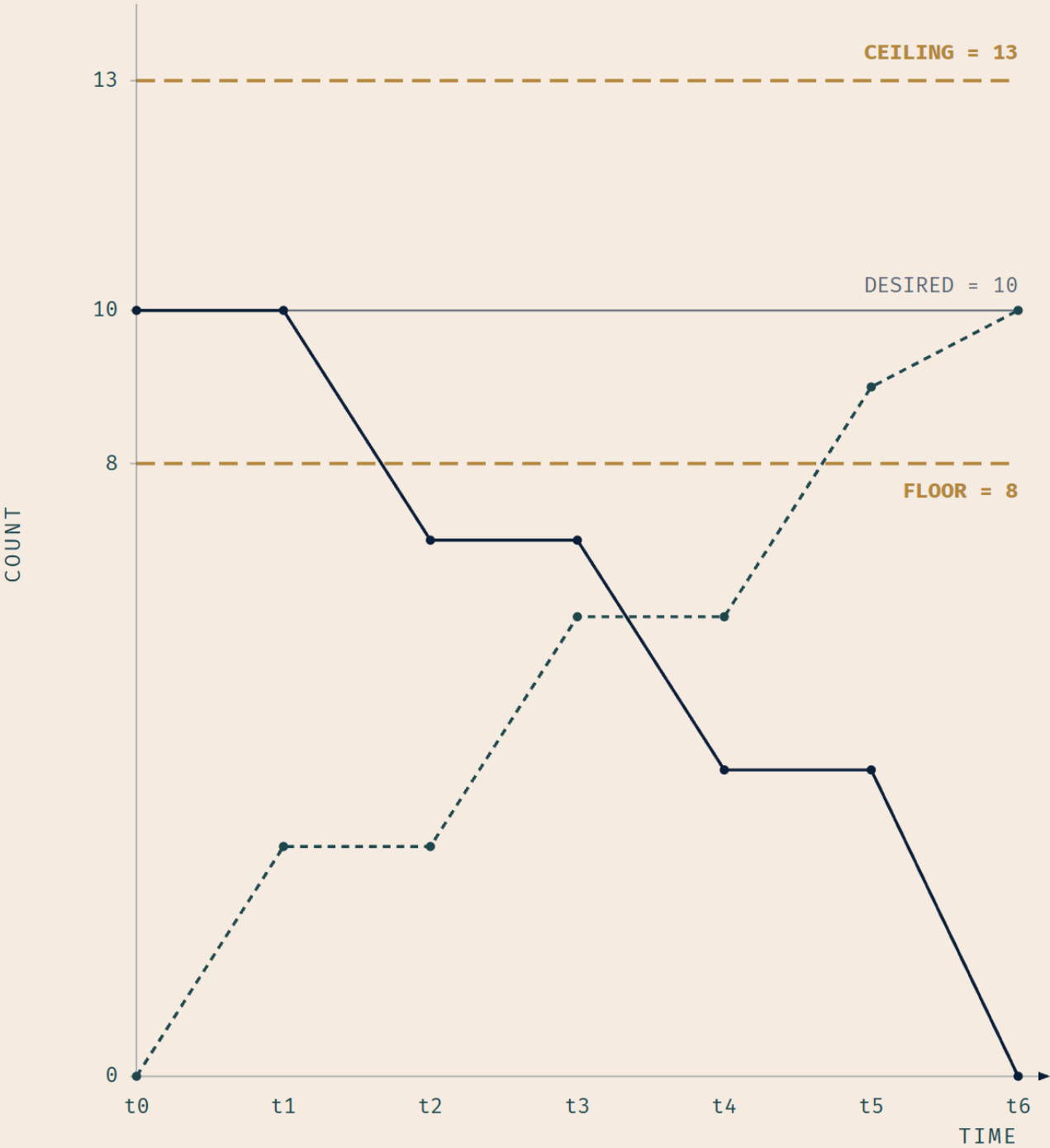
Ten replicas. Both fields left at their defaults.

Field	25% of 10	Rounding direction	Absolute value
<code>maxSurge</code>	2.5	up	3
<code>maxUnavailable</code>	2.5	down	2

Ceiling on total Pods = $10 + 3 = 13$. Floor on available Pods = $10 - 2 = 8$.

At most thirteen Pods exist at any instant during the update, counting old and new together. At least eight are available at any instant. Do that arithmetic once by hand and the two names stop being interchangeable.

Notice the asymmetry in the rounding, because it is not arbitrary. Surge rounds up: Kubernetes will give you *more* headroom above the line than the percentage strictly buys. Unavailable rounds down: it will allow *fewer* Pods to be missing than the percentage strictly permits. Both defaults round in the direction of keeping more capacity online. That is a design decision you can read straight off the field descriptions.



*Old + new never rises above the ceiling ($10 + 3 = 13$).
Available never falls below the floor ($10 - 2 = 8$).*

LEGEND

- Old Pod count
- New Pod count
- Desired replicas (10)
- Ceiling (13) & floor (8)

Figure 6.2 — the two bounds are opposite in kind. `maxSurge` is a ceiling on how many Pods exist. `maxUnavailable` is a floor on how many are usable. They are not two settings of the same knob.

🧠 **Mnemonic:** Surge is above the line. Unavailable is below it.

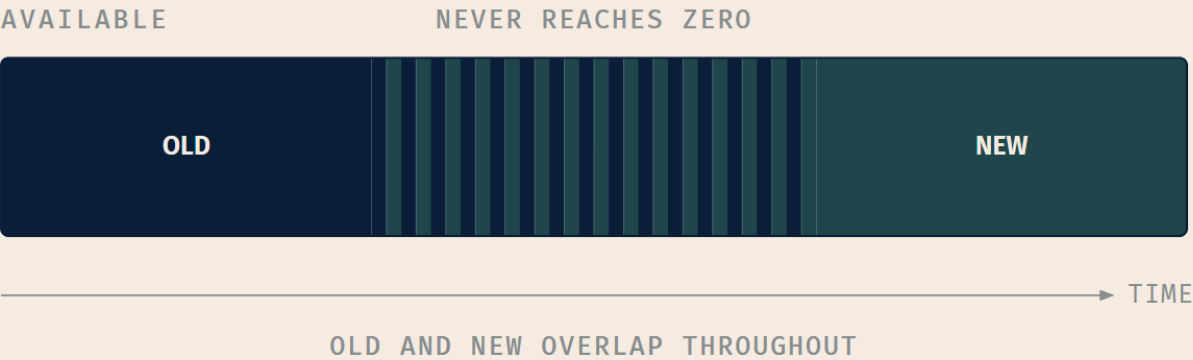
Recreate

The other strategy is the contrast that makes the first one legible. With `Recreate`, all existing Pods are killed before new ones are created [source: [k8s-docs-deployment-2026-08-23](#)].

Recreate



RollingUpdate



LEGEND

- OLD POD
- NEW POD
- OVERLAP
- ZERO AVAILABLE

Figure 6.3 — the gap is the whole point. Recreate has a window in which nothing is serving. That window is not a bug; it is the thing you chose.

And it *is* a legitimate choice, not a mistake. Some applications genuinely cannot have two versions running at once: a schema migration that the old code cannot read, an exclusive lock on a resource, a license that permits one active instance. For those, downtime is not a failure of the deployment strategy; it is the cost of correctness, taken deliberately and scheduled. Treating Recreate as always-wrong is its own trap.

Dead Reckoning: A Deployment's `.spec.strategy.type` is either `Recreate` OR `RollingUpdate`; `RollingUpdate` is the default [source: k8s-docs-deployment-2026-08-23]. With `Recreate`, all existing Pods are terminated before any new Pod is created [source: k8s-docs-deployment-2026-08-23]. With `RollingUpdate`, changing `.spec.template` causes a new `ReplicaSet` to be created and the Deployment to move Pods from the old `ReplicaSet` to the new one at a controlled rate [source: k8s-docs-deployment-2026-08-23]. That rate is bounded by two optional fields under `.spec.strategy.rollingUpdate`. `maxUnavailable` is the maximum number of Pods that may be unavailable during the update; percentages round down; default 25%. `maxSurge` is the maximum number of Pods that may exist over the desired count; percentages round up; default 25%. Neither may be 0 if the other is 0. [source: k8s-docs-deployment-spec-fields-2026-08-24]

☆ Fixed Point:

RollingUpdate is the default strategy. Recreate **kills every old Pod before creating any new one.** `maxSurge` and `maxUnavailable` **both default to 25%.**

The discriminator between the two bounds is the rounding, and it runs in opposite directions: **surge rounds up, unavailable rounds down** [source: k8s-docs-deployment-spec-fields-2026-08-24]. Both defaults therefore err toward keeping more capacity online, which is the sentence to memorize if you can only keep one — you can rebuild both rules from it.

What makes it safe

A gradual replacement is not automatically a safe one. Replacing ten broken Pods two at a time still ends with ten broken Pods; it just takes longer.

What makes it safe is availability. Kubernetes marks a Deployment as progressing when, among other things, new Pods become ready or available, where available means ready for at least `minReadySeconds` [source: k8s-docs-deployment-spec-fields-2026-08-24]. `.spec.minReadySeconds` is the minimum number of seconds for which a newly created Pod should be ready without any of its containers crashing for it to be considered available; it defaults to 0, meaning a Pod counts as available the moment it is ready [source: k8s-docs-deployment-spec-fields-2026-08-24].

And "ready" is Chapter 5's word. A readiness probe indicates whether the container is ready to respond to requests; when it fails, the endpoints controller removes the Pod's IP address from the endpoints of all Services that match the Pod [source: k8s-docs-pod-lifecycle-2026-08-23].

Put those together and you have the property Chapter 5 promised you. The relief comes aboard before the old watch is dismissed, and nobody is dismissed until the relief has actually reported fit. A new Pod that never reports ready never becomes available. A rollout that cannot accumulate available new Pods cannot proceed to remove old ones without breaching the `maxUnavailable` floor. So it doesn't. **The rollout stalls instead of taking down the service.** Readiness probe failures are listed by name among the reasons a Deployment gets stuck trying to deploy its newest ReplicaSet without ever completing [source: `k8s-docs-deployment-spec-fields-2026-08-24`], alongside image pull errors, insufficient quota, insufficient permissions, limit ranges, and application runtime misconfiguration.

This is the moment probes stop being a health-checking feature and become a release-safety mechanism. Chapter 5 told you a Pod that never reports ready never receives traffic. §4 tells you the larger consequence: a *version* that never reports ready never replaces the version that works. *[cross-bearing: see Ch 5 §7 — liveness, readiness, and startup probes]*

🔖 **Snag:** A stalled rollout is not a failed one. The Deployment sits there with both ReplicaSets alive, the old Pods still serving traffic, waiting for a new Pod that will never become ready. Your users see nothing. Your dashboard sees nothing. `kubectl get deployments` shows a ready count that has stopped moving. Nothing will page you unless you asked something to.

The clock on that is `.spec.progressDeadlineSeconds`, the number of seconds to wait for the Deployment to progress before the system reports back that it has failed to progress, surfaced as a condition with type: `Progressing`, status: `"False"` and reason: `ProgressDeadlineExceeded`. It defaults to 600, and the controller keeps retrying regardless [source: `k8s-docs-deployment-spec-fields-2026-08-24`]. Note what it does and does not do: it produces a *signal*. It does not roll anything back. Reading that signal, and finding out which of the six causes you hit, is a diagnosis skill. *[cross-bearing: see Ch 13 — diagnosing a stuck rollout]*

Rolling updates are the mechanism. There is a whole vocabulary of *release strategies* built on top of them: patterns with names, implemented with tooling that sits above the Deployment object. That vocabulary needs machinery this chapter has not introduced yet. *[cross-bearing: see Ch 15 — blue/green, canary, and A/B, and the tooling that implements them]*

..1 §5 — Every Rollout Is a Revision

Every time the Deployment moves Pods from one ReplicaSet to another, something gets written down. That record is what makes going back possible.

The rule, stated exactly

A Deployment's revision is created when a rollout is triggered, and **a new revision is created if and only if the Deployment's Pod template (`.spec.template`) is changed**. Other updates, such as scaling the Deployment, do not create a revision [source: `k8s-docs-deployment-2026-08-23`].

That "if and only if" is the whole content of this section, and it is precisely the kind of boundary that is easy to get backwards, because the intuitive answer is wrong and the correct answer fits in one clause. The intuitive answer is "any change to the Deployment creates a revision." It doesn't. Change the replica count and you have changed the Deployment, changed the cluster, and changed nothing about the revision history.

☆ Fixed Point:

A revision is created if and only if `.spec.template` changes. Scaling does not create a revision.

The corollary catches people: `kubectl rollout undo` will not reverse a scale. There is no revision to go back to, because scaling never made one.

Why does the rule work this way? Because a revision is really a ReplicaSet. Each new ReplicaSet updates the revision of the Deployment [source: k8s-docs-deployment-2026-08-23], and a new ReplicaSet is only created when the template changes: a different template is a different kind of Pod, which needs a different ReplicaSet to hold it. Scaling changes a number on the ReplicaSet you already have. Nothing new was created, so nothing new was recorded.

The verbs

The command surface here is small and closed. `kubectl rollout` manages the rollout of one or many resources, and the valid resource types are deployments, daemonsets and statefulsets [source: k8s-docs-kubectl-rollout-2026-08-24]. Six subcommands:

Command	What it does
<code>kubectl rollout status</code>	Show the status of the rollout [source: k8s-docs-kubectl-rollout-2026-08-24]
<code>kubectl rollout history</code>	View rollout history [source: k8s-docs-kubectl-rollout-2026-08-24]
<code>kubectl rollout undo</code>	Undo a previous rollout [source: k8s-docs-kubectl-rollout-2026-08-24]
<code>kubectl rollout pause</code>	Mark the provided resource as paused [source: k8s-docs-kubectl-rollout-2026-08-24]
<code>kubectl rollout resume</code>	Resume a paused resource [source: k8s-docs-kubectl-rollout-2026-08-24]
<code>kubectl rollout restart</code>	Restart a resource [source: k8s-docs-kubectl-rollout-2026-08-24]

`kubectl rollout history deployment/<name>` shows the revisions. `kubectl rollout undo deployment/<name>` rolls back to the previous revision, and `--to-revision=<n>` goes to a specific one [source: k8s-docs-deployment-2026-08-23].

Pause and resume are the pair that reads as arbitrary until you know why they exist. When you update a Deployment, or plan to, you can pause rollouts before you trigger one or more updates, then resume when you are ready to apply the changes, which lets you apply multiple fixes in between pausing and resuming without triggering unnecessary rollouts [source: k8s-docs-deployment-2026-08-23]. Without pause, three edits to the template are three rollouts, each one moving real Pods and each one interrupting the last. With pause, they are one.

🔒 **Worth Securing:** Pause before a batch of template edits. It is the difference between one rollout and four, and the four are not independent: each one restarts the work of the one before it.

What is kept

By default, all of a Deployment's rollout history is kept in the system so that you can roll back any time you want, and you can change that by modifying the revision history limit [source: k8s-docs-deployment-2026-08-23]. Precisely: `.spec.revisionHistoryLimit` is the number of old ReplicaSets to retain to allow rollback, and it defaults to 10 [source: k8s-docs-deployment-spec-fields-2026-08-24]. Those old ReplicaSets consume space in etcd and crowd the output of `kubectl get rs`, which is why the field exists; and setting it to zero cleans up all old ReplicaSets with zero replicas, with the consequence that a new rollout **cannot be undone**, since its revision history has been cleaned up [source: k8s-docs-deployment-spec-fields-2026-08-24].

"Can I still roll back?" is a real question at a real moment, and the default answer is yes, ten deep.

What a rollback actually is

This matters more than it looks, because you are going to meet the word "rollback" twice more in this book attached to entirely different mechanisms.

Rolling back a Deployment is not undoing an edit, and it is not restoring a backup. It is setting the Pod template to a previous revision's value and letting the same rolling update from §4 run in the other direction: same two ReplicaSets, same `maxSurge` and `maxUnavailable`, same readiness gate, opposite direction of travel. The ReplicaSet holding the old template has been riding at anchor the whole time with nobody aboard, which is exactly why the operation is fast and exactly why deleting the revision history removes the ability to do it.

Same loop. Same mechanics. Opposite direction.

[cross-bearing: see Ch 14 — Helm rollback and Deployment rollback are different mechanisms wearing the same word] [cross-bearing: see Ch 15 — and a third thing, again wearing it]

⚠ Navigational Hazards

"I scaled it and now rollout history shows nothing new." Correct. Scaling is not a template change, so no revision was created [source: k8s-docs-deployment-2026-08-23]. The intuitive model, "the Deployment changed, so the history should record it," is the wrong model. The history records *kinds of Pod*, not *states of Deployment*.

"I'll just rollout undo to get back to three replicas." You cannot. `undo` restores a Pod template. Your replica count is not in a Pod template. If you scaled from three to six and want three back, scale to three.

"I set revisionHistoryLimit: 0 to keep etcd clean." You also removed your ability to roll back; the documentation says so in the same paragraph that describes the field [source: k8s-docs-deployment-spec-fields-2026-08-24]. There is a defensible reason to do this in a GitOps environment where the previous state lives somewhere else entirely. There is no defensible reason to do it because the `kubectl get rs` output was untidy.

"The rollout failed." Usually it stalled. See §4: a Deployment that cannot get new Pods to available does not fail over to the old version. It stops, holding both ReplicaSets, with the old Pods still serving. The distinction matters because "failed" implies something ended and "stalled" means something is still waiting for you.

🕒 Taking Your Bearings #2: Changing the Fleet Under Way

Five questions on changing the fleet and what a change gets recorded as. One reaches back.

1. ☐ A Deployment with ten replicas is updated using the default strategy settings. What is the largest number of Pods that may exist at any instant during the update, and the smallest number that must be available?

A) 12 and 8 B) 13 and 8 C) 13 and 7 D) 11 and 9 — the 25% is a single budget split between surge and unavailability
2. ☐ **This one is meant to be hard. If you miss it, you are far from alone, and the miss is worth more to you than a correct guess would have been.** You scale a Deployment from three replicas to six, then run `kubectl rollout history`. How many new revisions do you see, and why?
3. ☐ Your application cannot run two versions at once, because the new version applies a schema migration the old version cannot read. Which strategy do you choose, and what exactly are you accepting when you choose it?
4. ☐ **[retrieval: ch5]** You push a broken image. The new Pods start, but their readiness probes never succeed. Describe what the Deployment does, and what the users of the service experience.

5. **11** You need to change the image, an environment variable, and the resource limits on one Deployment, and you want one rollout rather than three. How do you do it, and what are the two commands?

Answers with Explanations

1. B — 13 and 8.

Both fields default to 25% [source: k8s-docs-deployment-spec-fields-2026-08-24]. 25% of 10 is 2.5 in both cases, but they round in opposite directions: `maxSurge` rounds **up** to 3, `maxUnavailable` rounds **down** to 2. Ceiling on total = $10 + 3 = 13$. Floor on available = $10 - 2 = 8$.

- **A is wrong** because it rounds `maxSurge` down. This is the most defensible wrong answer and the one worth understanding: if you got here, you had the right structure and the wrong rounding rule.
- **C is wrong** because it rounds `maxUnavailable` up. Same error, other field.
- **D is wrong** because it treats 25% as a single allowance shared between the two fields and splits it — one Pod of surge, one Pod of unavailability, giving 11 and 9. They are independent fields bounding independent quantities: one bounds how many Pods *exist*, the other bounds how many are *usable*, and each gets its own 25%.

2. None. Scaling changes `.spec.replicas`. It does not change `.spec.template`. A new revision is created if and only if the Pod template is changed; other updates, such as scaling, do not create a revision [source: k8s-docs-deployment-2026-08-23].

If you answered "one," you are far from alone, and you now have the rule in a form you will not lose, because you paid for it. The underlying reason is worth carrying: a revision is a `ReplicaSet`, and a new `ReplicaSet` is only needed when the kind of Pod changes. Six copies of the same Pod need the same `ReplicaSet` that three copies needed.

3. Recreate, and you are accepting a window of complete unavailability. All existing Pods are killed before new ones are created [source: k8s-docs-deployment-2026-08-23]. The important half of this answer is the second half. You are not making a mistake; you are trading availability for correctness, because a rolling update would put old code and migrated data in the same room. Schedule it, announce it, and choose it on purpose.

If your answer was "a rolling update with `maxSurge: 0`," that is the near-miss worth naming: it removes the *extra* Pods but not the overlap. With `maxSurge: 0` and `maxUnavailable: 1`, old and new versions still coexist while the update walks through the fleet, which is exactly what a schema migration forbids. Only `Recreate` guarantees no overlap.

4. The rollout stalls. New Pods that never report ready never become available [source: k8s-docs-deployment-spec-fields-2026-08-24]; a rollout that cannot accumulate available new Pods cannot scale the old `ReplicaSet` down past the `maxUnavailable` floor. So both `ReplicaSets` stay alive, the old Pods keep

serving, and, because a failing readiness probe removes a Pod's IP from the endpoints of all Services matching it [source: k8s-docs-pod-lifecycle-2026-08-23], no traffic is ever sent to the broken new Pods.

Users experience nothing. That is the entire value of the mechanism, and it is why Chapter 5 spent as long as it did on probes.

After `progressDeadlineSeconds` (600 by default), the Deployment surfaces a `Progressing` condition with reason: `ProgressDeadlineExceeded` [source: k8s-docs-deployment-spec-fields-2026-08-24]. Naming that signal is this chapter's job. Reading the events, finding which of the six causes fired, and fixing it is Chapter 13's. *[cross-bearing: see Ch 13 §3]*

5. `kubectl rollout pause deployment/<name>`, then make all three edits, then `kubectl rollout resume deployment/<name>`. Pausing before you trigger updates lets you apply multiple fixes in between pausing and resuming without triggering unnecessary rollouts [source: k8s-docs-deployment-2026-08-23]. Without it, each of the three template edits triggers its own rollout, and each one interrupts and supersedes the last: you get three rollouts' worth of Pod churn to arrive at one final state.

Checkpoint: you've now mastered

✓ What a rolling update actually does, and the two bounds that shape it ✓ Why `Recreate` exists and when choosing it is correct ✓ What makes a gradual replacement a safe one ✓ The exact rule for what creates a revision, what a rollback is, and what `revisionHistoryLimit` buys you

📊 Chart → ⌘ **Passage** → 📈 Dawn — you are past the halfway point, and past the hardest section.

☐ The rest of the workload family (next) ☐ What happens when anyone can write a controller

..1 §6 — When Pods Are Not Interchangeable

Everything so far has quietly rested on one assumption, and it is time to name it.

A Deployment is a good fit for managing a stateless application workload where **any Pod in the Deployment is interchangeable** and can be replaced if needed [source: k8s-docs-workloads-2026-08-23]. That is why a count plus a template is a sufficient description of the whole workload. If any Pod can stand in for any other, then "three of these" says everything there is to say. It is also why §2's demonstration was unremarkable: the replacement Pod had a different name and a different UID and nothing cared, because nothing was depending on *which one it was*.

Now ask Soundings question 4 again. What if they are not interchangeable? What if this one is the primary and that one is the replica, each holding data that belongs to it specifically, each reachable at an address the others have written down?

The resource

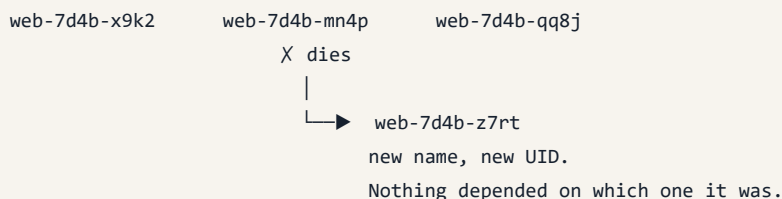
A **StatefulSet** runs a group of Pods and maintains a sticky identity for each of them, which is useful for managing applications that need persistent storage or a stable, unique network identity [source: k8s-docs-statefulset-2026-08-24]. The pivotal sentence: like a Deployment, a StatefulSet manages Pods based on an identical container spec, but unlike a Deployment, it maintains a sticky identity for each Pod, and **these Pods are created from the same spec but are not interchangeable: each has a persistent identifier that it maintains across any rescheduling** [source: k8s-docs-statefulset-2026-08-24].

That identity is concrete, not conceptual. For a StatefulSet with N replicas, each Pod is assigned an integer ordinal unique across the set — by default, from 0 through N-1 — and each Pod derives its hostname from the StatefulSet's name and its own ordinal. The pattern is `$(statefulset name)-$(ordinal)`, so a three-replica StatefulSet named `web` produces `web-0`, `web-1` and `web-2` [source: k8s-docs-statefulset-2026-08-24].

Storage sticks to the identity too. For each volume claim template defined in a StatefulSet, each Pod receives one `PersistentVolumeClaim`, and **the same claim will be bound to that Pod throughout its lifecycle**; when the Pod is rescheduled onto a different node, its mounts follow [source: k8s-docs-statefulset-2026-08-24]. This is what the workloads overview means when it says a StatefulSet matches each Pod with a `PersistentVolume`, and that code running in those Pods can replicate data to other Pods in the same StatefulSet to improve overall resilience [source: k8s-docs-workloads-2026-08-23].

And the ordering is guaranteed rather than incidental: for a StatefulSet with N replicas, Pods are created sequentially in order from 0 to N-1 and terminated in reverse order from N-1 to 0, and before a scaling operation is applied to a Pod, all of its predecessors must be Running and Ready [source: k8s-docs-statefulset-2026-08-24].

Deployment — Pods are interchangeable



StatefulSet — identity is sticky, and the storage belongs to the identity

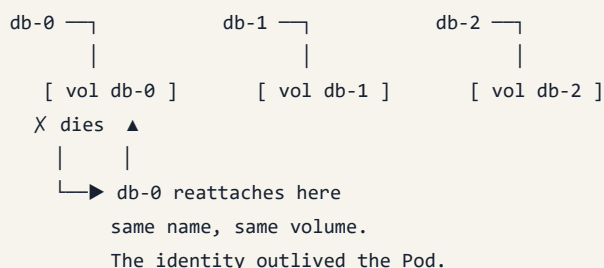


Figure 6.4 — the storage belongs to the identity, not to the Pod. Look at the lower row carefully. The volume is not drawn attached to a Pod; it is drawn attached to db-0, which is a slot that Pods pass through. That is the claim, and Chapter 11 is where it gets completed.

☆ **Fixed Point:**

The property that distinguishes a StatefulSet from a Deployment is whether the Pods are interchangeable, not whether the application writes to disk.

StatefulSet is for related Pods with stable, sticky identities, each typically paired with its own durable storage that follows the identity rather than the Pod.

📌 **Snag:** "It writes to disk, so it needs a StatefulSet." No. A stateless web server can write to disk (caches, temp files, uploaded images on a shared volume) and a Deployment's Pods can mount volumes perfectly well. The question is never *does it write*. The question is: **if I destroyed this Pod and a differently-named Pod appeared in its place, would anything be broken?** If the answer is no, it is a Deployment, disk or no disk.

Extended Analogy: A Deployment's Pods are a watch rotation. Any qualified hand can stand any watch; the roster says "three on deck," not "these three on deck," and when someone goes below you send up whoever is next. The order is a number and a qualification.

A StatefulSet's Pods are a pilot who knows this harbor. The order is not "one pilot." It is *this* pilot, who has the approach notes for this specific entrance, whose knowledge is not transferable by handing someone else the same job title. If that pilot is relieved, the relief has to be given the same approach notes, the same depths, and the same name on the manifest, or the ship goes aground while everyone insists the roster is full.

Both are legitimate ways to staff a task. Confusing them costs you in exactly one direction: treat interchangeable crew as irreplaceable and you carry unnecessary machinery; treat irreplaceable crew as interchangeable and you lose data.

A loop left open on purpose

You have now been told that a StatefulSet's Pods each get their own persistent storage, and you have not been told how that storage is provisioned, requested, sized, reclaimed, or shared. That is deliberate, and you should know it is deliberate rather than wonder what got skipped.

The storage half of this — PersistentVolume, PersistentVolumeClaim, StorageClass, access modes, and the provisioning path — is a chapter of its own, and it needs vocabulary this chapter has not built.

[cross-bearing: see Ch 11 — PersistentVolumes, claims, and how a Pod's storage follows its identity]

One more open thread, smaller: StatefulSets currently require a headless Service to be responsible for the network identity of the Pods, and you are responsible for creating it [source: k8s-docs-statefulset-

2026-08-24]. What a headless Service is belongs with the rest of Services. *[cross-bearing: see Ch 9 — headless Services and stable DNS names]*

[cross-bearing: see Ch 16 — debugging StatefulSets and their claims]

§7 — One Per Node, and Work That Ends

Three more resources, one defining property each, and then the figure that collects all six.

DaemonSet — one per node

A **DaemonSet** ensures that all (or some) nodes run a copy of a Pod. As nodes are added to the cluster, Pods are added to them; as nodes are removed, those Pods are garbage collected. Deleting the DaemonSet cleans up the Pods it created [source: k8s-docs-daemonset-2026-08-24].

Read the first clause again, because the whole resource is in it: *as nodes are added, Pods are added to them*. You never revisit the DaemonSet when the cluster grows. The order was written about the station, not about the number of hands standing it.

The workloads overview puts the purpose well: a DaemonSet defines Pods that provide facilities local to nodes, and each Pod performs a job similar to a system daemon on a classic Unix or POSIX server [source: k8s-docs-workloads-2026-08-23]. The typical uses are all recognizably that shape: running a cluster storage daemon on every node, a log-collection daemon on every node, a node-monitoring daemon on every node [source: k8s-docs-daemonset-2026-08-24]. A DaemonSet might be fundamental to the operation of the cluster, such as a plugin to run cluster networking; it might help you manage the node; or it might provide optional behavior that enhances the platform [source: k8s-docs-workloads-2026-08-23].

Two of those come back later. Cluster networking plugins ship as DaemonSets *[cross-bearing: see Ch 9 — CNI plugins and how Pod networking gets implemented]*, and node-level log agents are the canonical observability example *[cross-bearing: see Ch 18 — node-level log collection]*.

The Pod count is a consequence, not a setting. If you specify a node selector or node affinity in the template, the DaemonSet controller creates Pods on nodes matching it; if you specify neither, it creates Pods on all nodes [source: k8s-docs-daemonset-2026-08-24]. The controller creates a Pod for each eligible node [source: k8s-docs-daemonset-2026-08-24]. Supporting this from another direction: horizontal pod autoscaling does not apply to objects that can't be scaled, and the documentation's own example of such an object is a DaemonSet [source: k8s-docs-hpa-2026-08-24]. *[cross-bearing: see Ch 7 §3 — nodeSelector and node affinity]*

One more thing to bank, because Chapter 7 collects it. DaemonSets keep running on nodes where nothing else will — nodes the cluster has fenced off from ordinary workloads entirely. And you have already met the mechanism that makes this possible, in disguise: it has been holding those networking

and logging agents in place on every node since you first saw them running everywhere in Chapter 3's census. Chapter 7 unmask it [*cross-bearing: see Ch 7 §4 — taints, tolerations, and the fence DaemonSets step over*].

Job — work that ends

Jobs represent one-off tasks that run to completion and then stop. A Job creates one or more Pods and continues to retry execution until a specified number of them successfully terminate; as Pods complete, the Job tracks the successful completions, and when the specified number is reached the Job is complete. Deleting a Job cleans up the Pods it created [source: k8s-docs-job-2026-08-24].

The simple case is one Job object reliably running one Pod to completion, and the Job will start a new Pod if the first one fails or is deleted, for example due to node hardware failure or a node reboot [source: k8s-docs-job-2026-08-24]. Note the shape of that: it is still the control loop. The desired state is just *completion* rather than *a count of running things*.

Chapter 5 taught you five Pod phases and gave you an immediate use for three of them. Here are the other two. *Succeeded* means all containers in the Pod terminated in success and will not be restarted; *Failed* means all containers terminated and at least one terminated in failure [source: k8s-docs-pod-lifecycle-2026-08-23]. For everything in §1 through §6, those two phases were terminal states you hoped never to see. For a Job, *Succeeded* is the *point*. [*cross-bearing: see Ch 5 §5 — the five Pod phases*]

Consistent with that, a Job's Pod template may only use a `restartPolicy` of `Never` or `OnFailure` [source: k8s-docs-job-2026-08-24]. `Always` is not permitted, and once you have the concept, that restriction writes itself: a Pod that is always restarted can never complete.

CronJob — work that ends, repeatedly

A **CronJob** creates Jobs on a repeating schedule, and one CronJob object is like one line of a crontab file on a Unix system [source: k8s-docs-cronjob-2026-08-24]. It is meant for regular scheduled actions such as backups and report generation [source: k8s-docs-cronjob-2026-08-24].

The `.spec.schedule` field is required and follows standard Cron syntax, five fields, minute through day-of-week, so `0 3 * * 1` means weekly on a Monday at 3 a.m. [source: k8s-docs-cronjob-2026-08-24]. You can set `.spec.timeZone` to a valid time zone name to control how the schedule is interpreted [source: k8s-docs-cronjob-2026-08-24]. The `.spec.jobTemplate` defines the Jobs it creates and has exactly the same schema as a Job, nested, without its own `apiVersion` or `kind` [source: k8s-docs-cronjob-2026-08-24]: the same nesting move you saw with Pod templates in §1, one level further up.

One caution that is worth its space because it bites in production: a CronJob creates a Job **approximately** once per execution time of its schedule. The scheduling is approximate because there are circumstances where two Jobs might be created, or none. Kubernetes tries to avoid those situations but does not completely prevent them, therefore the Jobs you define **should be idempotent** [source: k8s-docs-

cronjob-2026-08-24]. Write the nightly report generator so that running it twice produces one report, not two.

The decision

Six resources. Four questions. Get the questions in the right order and the three most common wrong turns disappear.

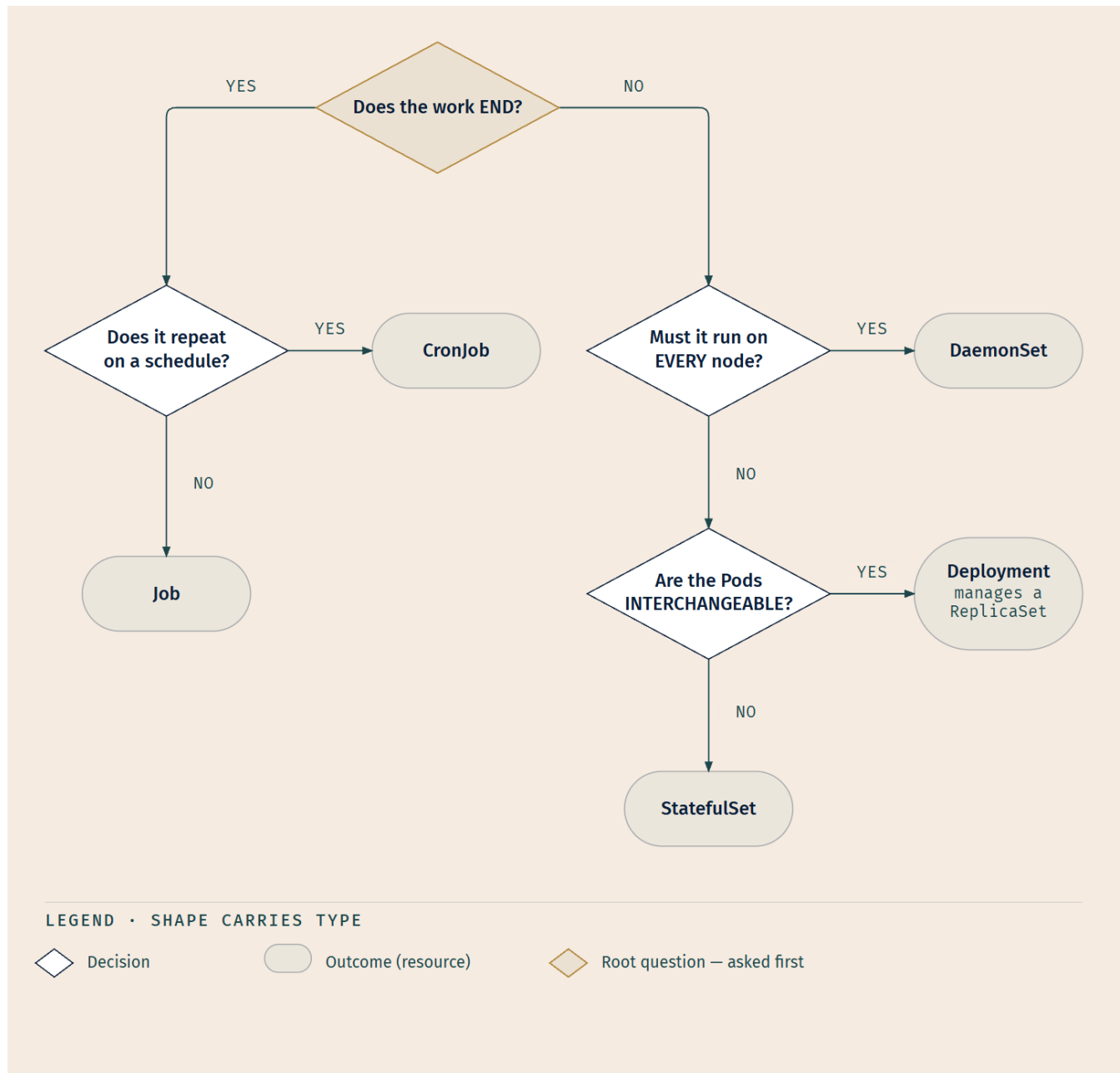


Figure 6.5 — ask about the work before you ask about the application. The first question is about the shape of the work, not the nature of the software. That ordering is deliberate, and the next block explains why.

The documentation offers the same guidance from the ReplicaSet's side, phrased as alternatives: use a Job instead of a ReplicaSet for Pods expected to terminate on their own; use a DaemonSet instead of a ReplicaSet for Pods providing a machine-level function such as machine monitoring or machine logging; and use a Deployment when you want ReplicaSets, since Deployments own and manage them for you [source: k8s-docs-replicaset-2026-08-24].

☆ Fixed Point:

DaemonSet: one Pod per eligible node, added automatically as nodes join. The count is a consequence of the cluster, not a setting. Job: runs a task to completion, once. CronJob: creates the same Job repeatedly, on a schedule.

⚠ Navigational Hazards

Three misconceptions cluster around this section, and they share one root cause, so learn the root cause instead of memorizing three corrections.

The root cause: people choose a workload resource by what the application is, rather than by how its Pods need to be *managed*. Every one of the following is that mistake wearing a different coat.

"It's a database, so it needs a StatefulSet." The application being a database is not the criterion. Interchangeability is. A single-replica read-only cache backed by an ephemeral volume is fine as a Deployment even though it is technically a datastore; a three-member quorum where each member has a distinct identity needs a StatefulSet even if the data is tiny. Ask what happens when one Pod is replaced by a differently-named one, and let the answer decide.

"I need six copies for capacity, so I'll use a DaemonSet." A DaemonSet does not express a number. It creates a Pod for each eligible node [source: k8s-docs-daemonset-2026-08-24], so you get however many nodes are eligible: possibly three, possibly sixty, and different tomorrow. If you want six, you want a Deployment with `replicas: 6`. Use a DaemonSet when the requirement is genuinely *per host*, which usually means the Pod is doing something to or about the machine it is sitting on.

"Job and CronJob both run scheduled work." No. A Job runs a task to completion once [source: k8s-docs-job-2026-08-24]. A CronJob creates Jobs on a repeating schedule [source: k8s-docs-cronjob-2026-08-24]. The CronJob is not a kind of Job. It is a thing that *makes* Jobs, the way a Deployment is not a kind of ReplicaSet but a thing that makes ReplicaSets. Once you notice that both pairs have the same shape, the distinction stops being something to memorize.

Run the tree in Figure 6.5 in order and none of these three can happen, because the tree asks about the work before it asks about the software.

[cross-bearing: see Ch 7 §4 — a DaemonSet's Pods still go through scheduling, and taints are how a node opts out]

§8 — The Control Loop, Extended

Chapter 3 closed by promising you controllers you configure yourself. Chapter 2 named custom resources as the fourth socket in the pattern where Kubernetes defines an interface and lets the ecosystem implement it. Both promises come due here.

Start with the word "resource"

A resource is an endpoint in the Kubernetes API that stores a collection of API objects of a certain kind; the built-in `pods` resource contains a collection of Pod objects [source: k8s-docs-custom-resources-2026-08-23]. You have been using resources all book. `kubectl get pods` talks to one. `kubectl get deployments` talks to another.

A **custom resource** is an extension of the Kubernetes API that is not necessarily available in a default Kubernetes installation; it represents a customization of a particular installation. Custom resources can appear and disappear in a running cluster through **dynamic registration**, and cluster admins can update them independently of the cluster itself [source: k8s-docs-custom-resources-2026-08-23].

Here is the clause that makes it click: once a custom resource is installed, users create and access its objects using `kubectl`, **just as they do for built-in resources like Pods** [source: k8s-docs-custom-resources-2026-08-23]. Nothing about the tooling changes. `kubectl get`, `kubectl describe`, `kubectl apply`, labels, selectors, namespaces, RBAC, all of it works on the new kind, because the new kind lives in the same API. [cross-bearing: see Ch 12 — RBAC, where this permission model is taught]

The object that does the installing

The **CustomResourceDefinition**, the CRD, is the API resource that lets you define custom resources. Defining a CRD object creates a new custom resource with a name and schema that you specify, and the Kubernetes API then serves and handles the storage of it for you. This frees you from writing your own API server, though the generic nature of the implementation means you have less flexibility than with API-server aggregation [source: k8s-docs-custom-resources-2026-08-23].

Many core Kubernetes functions are now built using custom resources, which is part of what makes Kubernetes modular [source: k8s-docs-custom-resources-2026-08-23]. This is not an exotic corner of the platform. It is one of the main ways the platform grows.

The honest limitation

On their own, custom resources let you store and retrieve structured data. That is all [source: k8s-docs-custom-resources-2026-08-23].

Sit with that for a second, because it is the thing that surprises people. A CRD by itself is a shape in a database. You can create objects of that kind, list them, label them, and delete them. Nothing else

happens. No Pods appear. No infrastructure is provisioned. You have defined a noun and taught the API server to store it.

☆ **Fixed Point:**

A custom resource on its own stores and retrieves structured data. A custom resource combined with a custom controller is the operator pattern.

When you combine a custom resource with a custom controller, custom resources provide a true declarative API. The Kubernetes declarative API enforces a separation of responsibilities: you declare the desired state of your resource, and the controller keeps the current state of Kubernetes objects in sync with your declared desired state. This is in contrast to an imperative API, where you instruct a server what to do [source: k8s-docs-custom-resources-2026-08-23].

That is Chapter 3's control loop, written by someone who is not the Kubernetes project. The published description of the controller extension pattern makes the shape explicit: controllers are client programs that read and/or write to the Kubernetes API, following a control loop, reading an object's `.spec`, possibly doing things, and then updating the object's `.status` [source: k8s-docs-extending-kubernetes-2026-08-23]. Read `.spec`, act, write `.status`. Nothing about that sentence is privileged. Anyone can write a program that does it.

🔖 **Snag:** "We installed the CRD, created an object of the new kind, and nothing happened." That is the correct behavior. There is no controller. You gave the cluster a new noun and no verb.

Name this shape, because you are going to meet it again. *An object without its component does nothing.* An Ingress with no ingress controller is the same shape. `kubect1 top` with no metrics-server is the same shape. A vertical autoscaler that isn't shipped by default is the same shape. It is not four gotchas. It is one rule with four instances, and the rule is that Kubernetes will happily accept a record of intent that nothing in the cluster is currently able to act on. [cross-bearing: see Ch 10 — Ingress without an ingress controller] [cross-bearing: see Ch 13 — `kubect1 top` without metrics-server]

The pattern, named

The **operator pattern** combines custom resources and custom controllers [source: k8s-docs-custom-resources-2026-08-23].

What it is for is more interesting than what it is. The pattern aims to capture the key aim of a human operator who is managing a service, someone with deep knowledge of how the system ought to behave, how to deploy it, and how to react if there are problems, and to write that knowledge as code that automates a task beyond what Kubernetes itself provides [source: k8s-docs-operator-pattern-2026-08-23]. The mechanism is unglamorous and precise: operators are clients of the Kubernetes API that act as controllers for a custom resource, and the pattern lets you extend the cluster's behavior without modifying the code of Kubernetes itself [source: k8s-docs-operator-pattern-2026-08-23].

The published list of things people automate this way is the fastest way to make it concrete [source: k8s-docs-operator-pattern-2026-08-23]:

- deploying an application on demand
- taking and restoring backups of that application's state
- handling upgrades of the application code alongside related changes such as database schemas or extra configuration settings
- publishing a Service so applications that don't support Kubernetes APIs can discover them
- simulating failure in all or part of a cluster to test its resilience
- choosing a leader for a distributed application that has no internal election process

Every one of those is somebody's 2 a.m. runbook, turned into a loop that never sleeps.

And now the closing turn, which is the reason §8 sits after §1 rather than before it. The most common way to deploy an operator is to add the CustomResourceDefinition and its associated controller to your cluster, and **the controller will normally run outside of the control plane, much as you would run any containerized application: for example, as a Deployment** [source: k8s-docs-operator-pattern-2026-08-23].

The thing that extends Kubernetes is itself deployed *by* Kubernetes, using the first resource in this chapter. The operator that manages your database is three replicas of a container image, held at a count by a ReplicaSet, held at a template by a Deployment. It is not a plugin. It is not privileged. It sails under the same standing orders as everything else in the fleet, and everything §1 through §5 told you about workloads applies to it unchanged.

The fourth socket

Chapter 2 showed you the move: Kubernetes defines an interface and lets the ecosystem implement it. You met it with the container runtime, with networking, and with storage, and you were told you would meet it once more at the API layer. This is that. The published extension points list **API extensions, Custom Resource Definitions (CRDs) and the API aggregation layer**, as one of six categories, alongside controllers, scheduling extensions, API access extensions, kubectl plugins, and infrastructure extensions [source: k8s-docs-extending-kubernetes-2026-08-23].

Four sockets, one pattern. Collecting them into a single statement about what kind of system Kubernetes is belongs later, when you have met all four in their own contexts. *[cross-bearing: see Ch 17 — CRI, CNI, CSI and CRDs, resolved into one story]*

 **Closer Look:** CRDs are not the only route to a custom API. The other is API-server aggregation, which offers more flexibility at the cost of writing and operating more of the API machinery yourself [source: k8s-docs-custom-resources-2026-08-23]. For this book's purposes: CRD is the common path, aggregation is the rarer one, and knowing that both exist is enough.

[cross-bearing: see Ch 14 — why Helm charts have a crds/ directory] [cross-bearing: see Ch 15 — a delivery tool that is, structurally, a controller acting on custom resources]

🕒 Taking Your Bearings #3: The Rest of the Fleet, and Extending It

Five questions on the rest of the family and the pattern extended.

1.  An application writes to a local disk. Does that mean it needs a StatefulSet? Answer in one sentence, and name the property that actually decides.
2.  You need six copies of a service running for capacity. A colleague suggests a DaemonSet. What is wrong with that suggestion, and what determines a DaemonSet's Pod count?
3.  Nightly at 02:00, a report has to be generated, and the process must exit when it is done. Name the resource you would create, and name the resource it creates each time it fires.
4.  You install a CRD for a resource called Backup, then create a Backup object. `kubectl get backup` shows it. Nothing else happens: no Pods, no snapshots, no events. Is the cluster broken?

A) Yes — the CRD's schema is misconfigured. B) Yes — custom resources have to be created in kube-system. C) No — a custom resource on its own only stores structured data; nothing acts on it without a controller. D) No — CRD objects take up to ten minutes to register.
5.  An operator manages a database. Where does the operator's own controller run, and using which of this chapter's resources?

Answers with Explanations

1. No. The deciding property is whether the Pods are **interchangeable**. A Deployment's Pods can mount volumes and write to them; what a Deployment assumes is that any Pod in it can be replaced by any other [source: k8s-docs-workloads-2026-08-23]. A StatefulSet exists for Pods that are created from the same spec but are *not* interchangeable, each carrying a persistent identifier across rescheduling [source: k8s-docs-statefulset-2026-08-24].

The wrong version of this answer, "yes, disk means StatefulSet," is worth naming because it is an easy one to arrive with, and it fails in both directions. It puts unnecessary machinery around stateless services that happen to cache to disk, and it leaves genuinely identity-dependent workloads in Deployments, where a routine Pod replacement quietly returns the wrong data.

2. A DaemonSet does not express a count. The controller creates a Pod for each eligible node [source: k8s-docs-daemonset-2026-08-24], so the Pod count is a consequence of how many nodes match, which is a property of the cluster and changes as nodes join and leave [source: k8s-docs-daemonset-2026-08-24]. On a nine-node cluster you would get nine, not six, and on a cluster that grew to twenty you would get twenty. Supporting evidence from another angle: horizontal pod autoscaling explicitly does not apply to a DaemonSet, because it is not an object that can be scaled [source: k8s-docs-hpa-2026-08-24].

For six copies, use a Deployment with `replicas: 6`.

3. A CronJob, which creates a **Job** at each scheduled time. A CronJob creates Jobs on a repeating schedule [source: k8s-docs-cronjob-2026-08-24]; the Job is the thing that actually runs to completion and stops [source: k8s-docs-job-2026-08-24]. The schedule for 02:00 daily is `0 2 * * *` in the standard five-field Cron syntax [source: k8s-docs-cronjob-2026-08-24].

If your answer was "a Job with a schedule on it," re-read §7's hazards block. The relationship between CronJob and Job is the same relationship as between Deployment and ReplicaSet: the outer object makes the inner one.

4. C. Nothing is broken. On their own, custom resources let you store and retrieve structured data [source: k8s-docs-custom-resources-2026-08-23]. You have taught the API server a new kind of object and given it nowhere to go. What makes a custom resource *do* something is combining it with a custom controller, which is what turns it into a true declarative API [source: k8s-docs-custom-resources-2026-08-23].

- **A is wrong** because a misconfigured schema would have shown up at CRD creation or at object creation. Your object was accepted and is retrievable; the API layer is working exactly as designed.
- **B is wrong** because namespace has nothing to do with it. Custom resource objects live wherever their scope allows, and `kube-system` confers no special activation.
- **D is wrong** because registration is not delayed; custom resources appear and disappear in a running cluster through dynamic registration [source: k8s-docs-custom-resources-2026-08-23]. Waiting longer will not help.

Name the pattern, because you will retrieve it by name: *an object without its component does nothing.*

You will meet this exact shape at least three more times in this book. Each time, the first instinct will be "something is misconfigured," and each time the answer will be "something is not installed."

5. Outside the control plane, as an ordinary containerized workload, **normally a Deployment** [source: k8s-docs-operator-pattern-2026-08-23]. The operator is a client of the Kubernetes API acting as a controller for a custom resource [source: k8s-docs-operator-pattern-2026-08-23]; it has no privileged position in the architecture and no special deployment mechanism. It is a container image, held at a replica count by a ReplicaSet, held at a template by a Deployment, exactly like everything else you have run this chapter.

If you answered "in the control plane," or "as a static Pod on the control-plane nodes," or "it's installed into the API server," that is the misconception this item exists to catch, and it is worth spending thirty seconds on. An operator is not a component of Kubernetes; it is a *client* of it, with exactly the access its RBAC grants and exactly the deployment story any other container has. Carrying the wrong version of this makes Chapter 15's central move — a delivery tool that is structurally just another controller — read as a special case instead of as the ordinary case.

This item required §1 and §8 together, and it is the direct precursor to a recognition Chapter 15 depends on. Hold onto it.

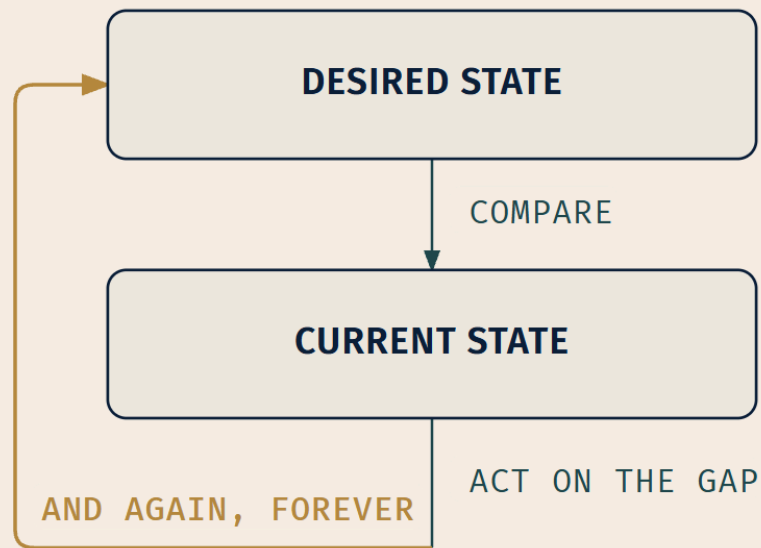
Checkpoint: you've now mastered

✓ The property that separates a StatefulSet from a Deployment ✓ Three sibling resources, each by its one defining property ✓ The decision tree, and why its question order matters ✓ Custom resources, custom controllers, and the pattern that is both

☀ §9 — Nobody Sails One Pod

Nothing new here. Just one thing seen properly.

Take Chapter 3's control loop, the one you retrieved in Soundings question 7 and filled in for a ReplicaSet in ☹ Taking Your Bearings #1, and plug this chapter's controllers into it one at a time.



*Change only what you plug into **DESIRED STATE**, and you have named every controller in this chapter:*

a number	<i>ReplicaSet</i>
a template + an update policy	<i>Deployment</i>
one per matching node	<i>DaemonSet</i>
completion	<i>Job</i>
a Job existing at each scheduled time	<i>CronJob</i>
whatever its author decided	<i>your operator</i>

Figure 6.6 — one loop, six desired states. The loop is drawn once. That is the argument.

☼ **Zenith:** Six resources. One shape. You have been looking at the same diagram for the entire chapter.

The ReplicaSet's desired state is a number. The Deployment's is a template plus a policy for changing it. The DaemonSet's is *one per matching node*: a number, but one the cluster computes rather than one you write. The Job's is *completion*. The CronJob's is *a Job existing at each scheduled time*. The operator's is whatever its author decided a database, or a certificate, or a message queue ought to look like.

Nothing was added to the loop to accommodate any of them. Nothing needed to be.

And now the part that carries forward. That shape is not a Kubernetes implementation detail, a way the maintainers happened to build some controllers. It is what Kubernetes is. §8 already showed you that anyone can write a controller: read an object's `.spec`, do things, write its `.status` [source: k8s-docs-extending-kubernetes-2026-08-23]. There is no line in the architecture separating the loops Kubernetes ships from the loops you write.

Which means the last question left is where the desired state lives. So far it has always lived in the cluster: an object you created with `kubectl apply`, stored by the API server, watched by a controller. There is no rule that says it has to. *[cross-bearing: see Ch 15 — a controller whose desired state lives in a Git repository, and why that is the same technology rather than a new one]*

When you meet that, it will look like a new idea for about ten seconds.

Which returns you to the title. Nobody sails one Pod, not because a single Pod is forbidden, and not because Pods are unimportant. A single Pod is a statement about right now. Every resource in this chapter is a statement about what should keep being true. That is the whole difference, and it is why you will stop reaching for `kubectl run` without ever making a decision to stop.

Exam Alert! ☼

High-priority topics, in descending order of how much of this chapter depends on them.

1. **The workload-resource decision.** Which resource for which shape of work. Figure 6.5 is the single highest-value artifact in this chapter; it collapses three separate misconceptions into one traversal.
2. **Deployment versus StatefulSet is about interchangeability, not disk.** A StatefulSet's Pods are created from the same spec but are not interchangeable [source: k8s-docs-statefulset-2026-08-24]; a Deployment suits workloads where any Pod is interchangeable [source: k8s-docs-workloads-2026-08-23].
3. **DaemonSet is one Pod per eligible node,** added automatically as nodes join [source: k8s-docs-daemonset-2026-08-24]. The count is a consequence of the cluster.

4. **Job runs to completion once; CronJob creates Jobs on a schedule** [source: k8s-docs-job-2026-08-24] [source: k8s-docs-cronjob-2026-08-24].
5. **The ownership chain: Deployment → ReplicaSet → Pod**, and which layer holds the template and which enforces the count [source: k8s-docs-deployment-2026-08-23] [source: k8s-docs-replicaset-2026-08-24].
6. **RollingUpdate is the default strategy; maxSurge and maxUnavailable both default to 25%; Recreate kills all old Pods before creating any new one** [source: k8s-docs-deployment-2026-08-23] [source: k8s-docs-deployment-spec-fields-2026-08-24].
7. **A revision is created if and only if the Pod template changes.** Scaling does not create one [source: k8s-docs-deployment-2026-08-23].
8. **A CRD alone stores structured data; a CRD plus a custom controller is the operator pattern** [source: k8s-docs-custom-resources-2026-08-23].

Common traps.

Trap	The correction
"StatefulSet is for apps that write to disk"	The property is interchangeability. Deployment Pods can write to disk.
"Use a DaemonSet to run several copies"	A DaemonSet expresses <i>per node</i> , not a number. Six copies is a Deployment.
"Job and CronJob are two ways to schedule work"	A Job runs once. A CronJob <i>creates Jobs</i> . Same relationship as Deployment → ReplicaSet.
"Scaling creates a revision"	Only a <code>.spec.template</code> change does.
<code>maxSurge</code> and <code>maxUnavailable</code> transposed	Surge bounds <i>total</i> . Unavailable bounds <i>available</i> . Surge rounds up, unavailable rounds down.
"Recreate is a mistake"	It is a supported strategy with a stated cost. Choosing it deliberately is correct engineering.
"Installing a CRD makes something happen"	It defines a noun. Nothing acts on it until a controller exists.
" <code>revisionHistoryLimit: 0</code> just tidies up"	It also removes the ability to undo the next rollout.

The single most useful thing to internalize: all three of the resource-selection traps are the same error. People choose a workload resource by what the application *is* rather than by how its Pods need to be *managed*. Run Figure 6.5's questions in order (does the work end, must it run on every node, are the Pods interchangeable) and you cannot arrive at any of the three.

Practice Questions

Nineteen questions. Answers and explanations follow the full set, so work them all first.

Q1. You create a Deployment with `replicas: 3`. Which object actually creates the three Pods?

- A) The Deployment B) The ReplicaSet the Deployment creates C) The scheduler D) The kubelet on each node
-

Q2. **[retrieval: ch4]** On a Deployment, where does the number of replicas you *asked for* live, and where does the number *currently running* live?

- A) Both in spec B) Both in status C) The requested count in spec; the running count in status D) The requested count in metadata; the running count in spec
-

Q3. You set `.spec.revisionHistoryLimit: 0` on a Deployment to stop old ReplicaSets accumulating in etcd and crowding `kubectl get rs`. What else have you given up?

- A) Nothing — the field only controls how many entries `kubectl rollout history` prints B) The ability to undo a new rollout, because its revision history is cleaned up C) The ability to pause and resume a rollout D) Rollbacks to revisions older than the previous one, but `rollout undo` with no arguments still works
-

Q4. A ReplicaSet has `replicas: 3` and a selector of `app=web`. You manually create a bare Pod carrying the label `app=web`. What is the most likely outcome?

- A) Nothing; bare Pods are never touched by controllers B) The ReplicaSet adopts the Pod, is now over its desired count, and terminates a Pod C) The API rejects the bare Pod for label conflict D) The ReplicaSet scales up to four to accommodate it
-

Q5. **[retrieval: ch5]** A ReplicaSet replaces a Pod that was deleted. What is true of the replacement, and why does it matter?

- A) It is the same Pod restarted, retaining its UID and IP B) It is a new Pod with a new UID; anything that recorded the old Pod's identity is now stale C) It is the same Pod moved to a different node, retaining its name D) It is a new Pod that reuses the old Pod's name but receives a new UID
-

Q6. You run `kubectl delete deployment/web`. What happens to the ReplicaSets and Pods beneath it, and by what mechanism?

- A) Nothing; they are independent objects and must be deleted individually B) They are deleted by cascading deletion, driven by owner references C) They are deleted because the selector no longer

matches anything D) They remain until the next garbage-collection window, which runs hourly

Q7. A Deployment has eight replicas, with `maxSurge: 0` and `maxUnavailable: 1` set explicitly. What are the ceiling on total Pods and the floor on available Pods, and is this configuration legal?

- A) Ceiling 8, floor 7; legal — the prohibition is only on setting *both* to 0
 - B) Ceiling 8, floor 7; illegal — `maxSurge` may never be 0
 - C) Ceiling 9, floor 7; legal — `maxSurge: 0` still permits one surge Pod for the swap
 - D) Ceiling 8, floor 8; legal — with no surge allowed, nothing may become unavailable either
-

Q8. A Deployment has **six** replicas, with `maxSurge` and `maxUnavailable` both left at their defaults. What are the ceiling on total Pods and the floor on available Pods?

- A) Ceiling 8, floor 5
 - B) Ceiling 7, floor 5
 - C) Ceiling 8, floor 4
 - D) Ceiling 7, floor 4
-

Q9. Which of the following is the correct reason to choose `Recreate` over `RollingUpdate`?

- A) It is faster, because Pods are not replaced one at a time
 - B) The application cannot tolerate two versions running simultaneously
 - C) It needs no spare capacity, because no surge Pods are ever created
 - D) It avoids the need for readiness probes
-

Q10. You make two changes to a Deployment on Monday: you update the container image, and you scale from four replicas to eight. How many new revisions appear in `kubectl rollout history`?

- A) Zero
 - B) One
 - C) Two
 - D) One, but only if the image change is applied second
-

Q11. You run `kubectl rollout undo deployment/web`. Which statement best describes what the cluster does?

- A) It restores a snapshot of the Deployment object taken before the last change
 - B) It sets the Pod template to the previous revision's value and runs the same rolling update in the other direction
 - C) It deletes the current ReplicaSet and recreates Pods from a backup
 - D) It reverses every field change made since the last rollout, including replica count
-

Q12. You deploy a new image. The Pods start, but a bug means their readiness probes never succeed. Sixty seconds later, what is the state of the service and the Deployment?

- A) The service is down; all Pods have been replaced with broken ones
- B) The old Pods are still serving; the rollout has stalled with both ReplicaSets alive
- C) The Deployment has automatically rolled back to

the previous revision D) The Pods are killed and restarted repeatedly until the image is fixed

Q13. Which of these workloads most clearly requires a StatefulSet rather than a Deployment?

A) A web front end that caches rendered pages to a local disk B) A message consumer that must never have two instances processing the queue simultaneously C) A three-member clustered datastore whose members address each other by stable hostname and each own distinct data D) A batch importer that writes its output to object storage

Q14. Your cluster has twelve worker nodes. You create a DaemonSet with no node selector and no affinity. How many Pods will it produce, and what happens when three more nodes join?

A) One Pod; three more nodes changes nothing B) Twelve Pods; three more nodes produces fifteen C) Whatever `replicas` is set to; three more nodes changes nothing D) Twelve Pods; three more nodes requires you to scale the DaemonSet

Q15. A data pipeline must run once, process a file, and exit. A second pipeline must run the same processing every night at midnight. Which resources do you create?

A) Two Jobs, one with a `schedule` field set B) A Job for the first; a CronJob for the second C) A CronJob for both, with the first set to run once D) A Deployment with `replicas: 1` for the first; a CronJob for the second

Q16. **[retrieval: ch5]** Chapter 5 taught five Pod phases. Which phase indicates a Job's Pod has done its work correctly, and what does that phase mean?

A) `Running` — the container is still executing, which for a Job means progress B) `Succeeded` — all containers terminated in success and will not be restarted C) `Completed` — a Job-specific phase not available to other Pods D) `Failed` — all containers terminated, at least one in failure

Q17. A StatefulSet's Pods all match its label selector, exactly as a Deployment's do. A StatefulSet nevertheless guarantees something about its Pods that no selector could express. What is it, and what carries it?

A) That they are all running — carried by the readiness probe B) That each Pod keeps a stable ordinal identity and hostname across rescheduling — carried by the Pod's ordinal and derived name, not by the selector C) That there are exactly `.spec.replicas` of them — carried by the selector's match count D) That they share a namespace — carried by the selector's scope

Q18. You install a CRD defining a resource called `Certificate`, and you create a `Certificate` object. `kubectl get certificate` returns it. No certificate is issued. What is the minimum missing piece?

A) RBAC permissions granting access to the new resource B) A controller that watches `Certificate` objects and acts on them C) API aggregation, which CRDs require to function D) A status subresource, without which nothing can act on the object

Q19. [retrieval: ch3] What does a custom controller written by a third party have in common with a built-in controller such as the Deployment controller, and where does each of them run?

A) Nothing structural — custom controllers use a separate API and run as privileged processes inside the control plane B) Both follow the same control loop — read `.spec`, act, write `.status` — but built-ins ship with the control plane while a custom controller normally runs as an ordinary Deployment C) Both run inside the API server; the only difference is which resources each one watches D) Both are registered with the API server through a `CustomResourceDefinition`, which is what makes a controller discoverable

Answers & Explanations

Q1 — B. The Deployment creates a `ReplicaSet`, and the `ReplicaSet` creates Pods in the background [source: k8s-docs-deployment-2026-08-23]. **A** is the most common wrong answer and it collapses the middle layer; if you hold it, rolling updates become unexplainable, because they work by having two `ReplicaSets` alive at once. **C** is wrong: the scheduler decides *where* an existing Pod runs, not whether it exists. **D** is wrong for the same reason at a lower level: the kubelet runs containers for Pods already assigned to its node.

Q2 — C. Almost every Kubernetes object has a `spec` describing desired state, which you set at creation, and a `status` describing current state, supplied and updated by the system [source: k8s-docs-objects-2026-08-23]. The documentation uses this exact example: set the Deployment `spec` to three replicas, and Kubernetes starts three instances and updates the `status` to match [source: k8s-docs-objects-2026-08-23]. **A** and **B** each collapse the distinction that makes control loops possible; you cannot compare two states stored in one field. **D** is wrong because `metadata` holds identity, not intent.

Q3 — B. `.spec.revisionHistoryLimit` is the number of old `ReplicaSets` retained to allow rollback, defaulting to 10; setting it to zero means all old `ReplicaSets` with 0 replicas are cleaned up, and in that case a new Deployment rollout **cannot be undone**, since its revision history is cleaned up [source: k8s-docs-deployment-spec-fields-2026-08-24]. **A is wrong**, and it is the tempting one: the old `ReplicaSets` are the history. Deleting them removes the mechanism, not merely the display — which is why §5 insists that a revision is a `ReplicaSet`, not a log entry. **C is wrong** because pause and resume act on an in-flight rollout [source: k8s-docs-deployment-2026-08-23] and have nothing to do with retained `ReplicaSets`. **D is**

wrong in the most useful way: `undo` with no arguments goes to the *previous* revision, and that is precisely the revision that no longer exists.

Q4 — B. A ReplicaSet identifies new Pods to acquire using its selector, and a Pod with no controller owner reference that matches will be immediately acquired [source: k8s-docs-replicaset-2026-08-24]. If the acquisition puts the ReplicaSet over its desired count, the newly acquired Pod is immediately terminated [source: k8s-docs-replicaset-2026-08-24]. **A** is wrong and dangerous: a ReplicaSet is explicitly not limited to owning Pods produced by its own template [source: k8s-docs-replicaset-2026-08-24]. **C** is wrong; labels are not unique keys and nothing at the API layer objects. **D** is wrong because the desired count is your declaration, not something the controller revises upward to accommodate surprises.

Q5 — B. A Pod is never rescheduled to a different node; it is replaced by a new, near-identical Pod with a different UID [source: k8s-docs-pod-lifecycle-2026-08-23], and higher-level controllers such as Deployments create those replacements [source: k8s-docs-pod-lifecycle-2026-08-23]. Why it matters: anything that recorded the old Pod's name, UID or IP address is now pointing at something that does not exist. **A** and **C** both preserve identity across replacement, which is exactly the thing that does not happen, and holding either of them is what makes Chapter 9's Services look like unnecessary machinery. **D is the trap worth understanding**, because it is *true of a different resource*: a StatefulSet's Pods do keep their ordinal-derived names across replacement [source: k8s-docs-statefulset-2026-08-24]. The stem says ReplicaSet, whose Pods carry generated names, so the replacement gets a new name as well as a new UID. Which resource is holding the Pods decides which answer is right.

Q6 — B. Owner references tell the control plane which objects depend on others, and Kubernetes deletes objects that no longer have owner references, like the Pods left behind when you delete a ReplicaSet, in a process called cascading deletion, with background cascading deletion as the default [source: k8s-docs-garbage-collection-2026-08-24]. **A** is wrong: the whole point of the owner-reference link is that dependents don't have to be tracked by hand. **C** confuses the two mechanisms; selection finds Pods, ownership governs deletion, and the documentation is explicit that ownership is different from labels and selectors [source: k8s-docs-garbage-collection-2026-08-24]. **D** invents a schedule; cascading deletion is not a periodic sweep.

Q7 — A, ceiling 8 and floor 7, and it is legal. `maxSurge` is the maximum number of Pods that may exist over the desired count, so $8 + 0 = 8$; `maxUnavailable` is the maximum that may be unavailable, so $8 - 1 = 7$ [source: k8s-docs-deployment-spec-fields-2026-08-24]. The legality turns on one clause: neither value may be 0 **if the other is 0** [source: k8s-docs-deployment-spec-fields-2026-08-24]. Here `maxUnavailable` is 1, so `maxSurge: 0` is permitted — and this is the configuration you reach for on a capacity-constrained cluster where there is no room for extra Pods. **B** overstates the rule into a blanket prohibition; the constraint is on the *pair*, because a rollout that may neither exceed nor drop below the desired count has no legal move. **C** invents a floor of one surge Pod; 0 means 0. **D** invents a coupling between the two fields, which is exactly the misconception §4's figure exists to kill: they bound different quantities in different directions.

Q8 — A, ceiling 8, floor 5. `maxSurge` = 25% of 6 = 1.5, rounded **up** to 2, so the ceiling is $6 + 2 = 8$. `maxUnavailable` = 25% of 6 = 1.5, rounded **down** to 1, so the floor is $6 - 1 = 5$ [source: k8s-docs-

deployment-spec-fields-2026-08-24]. **B** rounds surge down. **C** rounds unavailable up. **D** rounds both the wrong way. Both defaults round in the direction of keeping more capacity online; if you can remember that sentence you can rebuild both rules from it.

Q9 — B. Recreate kills all existing Pods before new ones are created [source: k8s-docs-deployment-2026-08-23], which is the correct choice precisely when two versions must not coexist: a schema migration, an exclusive lock, a single-instance license. **A** is a rationalization: speed is not the reason, and the cost is a window of zero availability. **C is the best distractor here, and it is materially true** — Recreate does surge nothing, and people do reach for it on clusters with no headroom. It is still the wrong *reason*, because `maxSurge: 0` (with a non-zero `maxUnavailable`) gets you a zero-headroom rollout without the downtime window [source: k8s-docs-deployment-spec-fields-2026-08-24]. Choose Recreate for correctness, not for capacity. **D** is false and inverts the risk: with Recreate you have less protection, not more, because there is no overlap window in which a failing new version can be caught before the old one is gone.

Q10 — B, one. The image change modifies `.spec.template` and creates a revision. The scale changes `.spec.replicas`, and other updates such as scaling do not create a revision [source: k8s-docs-deployment-2026-08-23]. **A** ignores the image change. **C** is the intuitive answer, two edits and two entries, and it is the one this chapter exists to correct. **D** invents an ordering dependency; the rule is about *which field* changed, not about sequence.

Q11 — B. `kubectl rollout undo` rolls back to the previous revision [source: k8s-docs-deployment-2026-08-23], and a revision is a ReplicaSet holding a previous Pod template. Restoring it sets the template back and lets the same rolling update run in the other direction, bounded by the same `maxSurge` and `maxUnavailable` and gated by the same readiness checks. **A** is wrong because there is no snapshot of the whole object; the old ReplicaSets are what the history consists of, which is why `revisionHistoryLimit: 0` removes the ability to undo [source: k8s-docs-deployment-spec-fields-2026-08-24]. **C** invokes a backup system that does not exist here. **D** is wrong and is the practical trap: `undo` restores a Pod template, so it will not reverse a replica-count change.

Q12 — B. New Pods that never report ready never count as available [source: k8s-docs-deployment-spec-fields-2026-08-24], so the rollout cannot scale the old ReplicaSet down past the `maxUnavailable` floor. Both ReplicaSets stay alive; the old Pods keep serving. Readiness probe failures are named explicitly among the reasons a Deployment gets stuck without completing [source: k8s-docs-deployment-spec-fields-2026-08-24]. Meanwhile the failing readiness probe removes each broken Pod's IP from the endpoints of every matching Service [source: k8s-docs-pod-lifecycle-2026-08-23], so no traffic reaches them. **A** describes what happens *without* readiness probes, which is the point of having them. **C** is wrong: Kubernetes surfaces a `ProgressDeadlineExceeded` condition after `progressDeadlineSeconds` and keeps retrying [source: k8s-docs-deployment-spec-fields-2026-08-24]; it does not roll back for you. **D** describes a liveness-probe failure, which is a different probe doing a different job.

Q13 — C. Members that address each other by stable hostname and own distinct data are, by definition, not interchangeable, which is the criterion [source: k8s-docs-statefulset-2026-08-24]. **A** is the disk misconception: the cache writes to disk and is nonetheless perfectly happy in a Deployment, because replacing one Pod with a differently-named one breaks nothing. **B targets a different error — "singleton,**

therefore StatefulSet. "Only one instance may run" is not the same claim as "the instances are not interchangeable." A Deployment with `replicas: 1` expresses it, and if you must guarantee no overlap even during a rollout, the strategy field is where you say so: `Recreate` [source: k8s-docs-deployment-2026-08-23]. Nothing in the requirement needs a sticky identity. **D** writes durable output but holds no identity; the importer could run as any Pod, or as a Job, and nothing downstream would notice.

Q14 — B. With neither a node selector nor node affinity specified, the DaemonSet controller creates Pods on all nodes [source: k8s-docs-daemonset-2026-08-24], so twelve. As nodes are added to the cluster, Pods are added to them [source: k8s-docs-daemonset-2026-08-24], so three new nodes gives you fifteen, automatically. **A** confuses a DaemonSet with a single-replica workload. **C** and **D** both assume a `replicas` field you would have to manage; the Pod count is a consequence of node eligibility [source: k8s-docs-daemonset-2026-08-24], and horizontal autoscaling explicitly does not apply to a DaemonSet because it is not a scalable object [source: k8s-docs-hpa-2026-08-24].

Q15 — B. A Job defines a task that runs to completion, once [source: k8s-docs-job-2026-08-24]. A CronJob creates Jobs on a repeating schedule [source: k8s-docs-cronjob-2026-08-24]. **A** invents a schedule field on the Job; the schedule lives on the CronJob, which then creates Jobs [source: k8s-docs-cronjob-2026-08-24]. **C** is over-engineering: a CronJob whose schedule fires once is a scheduled thing pretending to be a one-off, and it will surprise whoever inherits it. **D** is wrong in a way worth noticing: a Deployment with `replicas: 1` would treat the process exiting as a failure and restart it forever, which is the precise opposite of what "runs once and exits" means.

Q16 — B. Succeeded means all containers in the Pod have terminated in success and will not be restarted [source: k8s-docs-pod-lifecycle-2026-08-23]. **A** confuses in-progress with done. **C** invents a phase; there are five, and Completed is not among them [source: k8s-docs-pod-lifecycle-2026-08-23] — though Completed is a word you will meet in tooling output, which is exactly why the distractor works. **D** is the failure phase: all containers terminated and at least one terminated in failure [source: k8s-docs-pod-lifecycle-2026-08-23].

Q17 — B. A selector identifies a set. It can answer "is this Pod one of yours"; it cannot answer "which one is this." A StatefulSet's Pods carry identity through their ordinal and derived hostname, `$(statefulset name)-$(ordinal)`, giving `web-0`, `web-1`, `web-2`, and that identity sticks to the Pod across rescheduling [source: k8s-docs-statefulset-2026-08-24]. The identity is a *separate* mechanism layered over selection, which is why the StatefulSet controller also adds a per-Pod name label, `statefulset.kubernetes.io/pod-name`, so you can attach a Service to one specific member [source: k8s-docs-statefulset-2026-08-24]. **A is wrong** because readiness is a property the probe reports on the Pod's status; it is not a guarantee the StatefulSet makes about its members, and it is equally available to a Deployment. **C is wrong** because `.spec.replicas` is the count, and a selector's match count is a *current observation*, not a guarantee — a controller mid-scale has a match count that disagrees with the desired one, which is the whole reason the loop exists. **D is wrong** because namespace scoping comes from the request, not from anything the StatefulSet guarantees, and it is the same for every workload resource.

Q18 — B. On their own, custom resources let you store and retrieve structured data; combining a custom resource with a custom controller is what provides a true declarative API [source: k8s-docs-custom-

resources-2026-08-23]. **A is wrong, and it is the first thing most people check, which is why it is here:** your object was accepted and `kubectl get certificate` returned it, so access is demonstrably not the problem — a permissions failure would have shown up at the request, not as silence afterwards. **C** is backwards: a CRD is the alternative to API aggregation, and defining a CRD means the Kubernetes API serves and stores your resource without you writing an API server [source: k8s-docs-custom-resources-2026-08-23]. **D** is a real feature attached to a false claim; nothing about a `status` subresource is what makes something act on an object. A controller is.

Q19 — B. Controllers are client programs that read and/or write the Kubernetes API, following a control loop: read an object's `.spec`, possibly do things, then update the object's `.status` [source: k8s-docs-extending-kubernetes-2026-08-23]. That description covers both. The difference is deployment, not architecture: an operator's controller normally runs outside the control plane, much as you would run any containerized application, for example as a Deployment [source: k8s-docs-operator-pattern-2026-08-23], while the controllers Kubernetes ships run in the control plane — as the horizontal pod autoscaling controller does [source: k8s-docs-hpa-2026-08-24]. **A** is wrong on both counts: operators are ordinary clients of the Kubernetes API [source: k8s-docs-operator-pattern-2026-08-23], not privileged processes. **C** is wrong; controllers talk to the API server, and a built-in controller sends messages to the API server that have useful side effects [source: k8s-docs-controllers-2026-08-23] rather than running inside it. **D** conflates the noun with the verb, which is the exact confusion §8's Fixed Point exists to correct: a CRD registers a *resource*, not a controller. Nothing about installing a CRD makes any program discoverable or runnable, which is why installing one and creating an object of the new kind produces silence.

Chapter Summary

Concept	Remember this
Workload resource	You don't manage Pods directly. A workload resource holds the intent and configures a controller to keep it true [source: k8s-docs-workloads-2026-08-23].
Ownership chain	Deployment → ReplicaSet → Pod. Template and strategy above, count enforced in the middle, containers at the bottom.
ReplicaSet	Maintains a stable set of replica Pods [source: k8s-docs-replicaset-2026-08-24]. <code>.spec.replicas</code> is the desired state; the loop closes the gap.
Scaling vs self-healing	Same operation. The loop cannot tell why the gap appeared.
HorizontalPodAutoscaler	An API resource plus a controller that periodically adjusts the replica count to match observed metrics [source: k8s-docs-hpa-2026-08-24].
Selector	Membership is a query, not a list. The template's labels must satisfy the selector or the API rejects the object [source: k8s-docs-replicaset-2026-08-24].
Owner references	A separate mechanism from selection. They drive cascading deletion [source: k8s-docs-garbage-collection-2026-08-24].
Orphans	Dependents that outlive their owner. Kubernetes deletes dependents by default; mutating a selector is how you make orphans by accident [source: k8s-docs-garbage-collection-2026-08-24] [source: k8s-docs-daemonset-2026-08-24].
Rolling update	Template change → new ReplicaSet → Pods moved at a controlled rate [source: k8s-docs-deployment-2026-08-23]. Two ReplicaSets alive at once.
maxSurge / maxUnavailable	Both default to 25%. Surge is a ceiling on total, rounding up. Unavailable is a floor on available, rounding down [source: k8s-docs-deployment-spec-fields-2026-08-24]. Neither may be 0 if the other is 0.
Recreate	Kills all old Pods before creating new ones [source: k8s-docs-deployment-2026-08-23]. Downtime chosen deliberately.
What makes it safe	Readiness. A version that never reports ready never becomes available, so the rollout stalls instead of taking the service down.
Revision	Created if and only if <code>.spec.template</code> changes. Scaling does not create one [source: k8s-docs-deployment-2026-08-23].
Rollback	Restore a previous template and run the same rolling update backwards. Ten old ReplicaSets retained by default; <code>revisionHistoryLimit: 0</code> removes the ability to undo [source: k8s-docs-deployment-spec-fields-2026-08-24].
StatefulSet	Sticky identity per Pod; Pods created from the same spec but not interchangeable [source: k8s-docs-statefulset-2026-08-24]. Interchangeability decides, not disk.
DaemonSet	One Pod per eligible node, added as nodes join [source: k8s-docs-daemonset-2026-08-24]. The count is a consequence of the cluster, not a setting.

Concept	Remember this
Job / CronJob	Job runs to completion once [source: k8s-docs-job-2026-08-24]. CronJob creates Jobs on a repeating schedule, approximately, so write them idempotent [source: k8s-docs-cronjob-2026-08-24].
Custom resource	An API endpoint that is not necessarily present in a default install, registered dynamically, accessed with kubectl like anything else [source: k8s-docs-custom-resources-2026-08-23].
CustomResourceDefinition	Defines a custom resource with a name and schema; the API then serves and stores it for you [source: k8s-docs-custom-resources-2026-08-23].
Operator pattern	Custom resource + custom controller [source: k8s-docs-custom-resources-2026-08-23]. The controller normally runs outside the control plane, as a Deployment [source: k8s-docs-operator-pattern-2026-08-23].
The one shape	Six resources, one control loop. Only the desired state differs.

The Voyage Ahead

The loop noticed a gap and created a Pod. That Pod does not yet run anywhere.

Between "this Pod object exists" and "this container is executing on a machine" sits a decision nobody in this chapter made. Something has to look at a Pod with no node assigned to it, look at every node in the cluster, and choose. It has to know how much memory the Pod asked for and how much each node has left. It has to respect that some nodes are reserved, some are draining, and some are deliberately hostile to workloads that have not been told they are welcome.

Most of the time this happens in milliseconds and you never think about it. You think about it on the morning a Pod sits in `Pending`, nothing you do to the Deployment changes anything, and the reason is one layer down: the Deployment's job ended the moment the Pod object existed. That Pod is a record of intent with nowhere to go, and the reason it has nowhere to go is written in the resource requests you set in Chapter 5 and the node properties you have not met yet.

Chapter 7 introduces the scheduler: how a Pod gets placed, what it is actually weighing, and the four mechanisms (`nodeSelector`, `affinity`, `taints`, and `tolerations`) you use to influence a decision you do not make yourself. It also settles a question this chapter raised and walked away from: a `DaemonSet` is supposed to run on every node, so what happens when a node has been marked as one that workloads should stay off?

You know how a fleet is described. Next you find out how a berth is assigned.

"You can write the standing order in an afternoon. Finding the water to carry it out is the other half of the work." — Lodestar Ledgers

⚓ Safe Harbor

Chapter 6 complete. Take the measure of what changed.

You arrived able to read what a Pod was telling you. You leave able to state what should be true about a group of them and hand the maintenance of that statement to something that never sleeps and never forgets. That is not a larger vocabulary. It is a different job.

Specifically, you can now:

✓ Trace intent from a Deployment through a ReplicaSet to a running container, and say which layer owns which piece ✓ Explain why a deleted Pod comes back with nobody having done anything ✓ Predict a rolling update's ceiling and floor from two fields and a replica count, and say when a bound of zero is legal ✓ Name the thing that makes a rollout safe rather than merely gradual, and know it was already in your hands at the end of Chapter 5 ✓ State the revision rule exactly, including the part that catches people, and what retaining no revisions costs you ✓ Pick the right workload resource by asking about the work before asking about the software ✓ Define a custom resource, a custom controller, and the pattern that is both, and know why installing one without the other does nothing

And one thing that is not a bullet point: you have seen the control loop twice now, at two altitudes, and recognized it the second time. Hold onto that recognition. You are going to need it once more, and the third time is the one that matters.

📊 Chart → ⋮ Passage → 📖 **Dawn** — Part II's trunk is behind you.

Chapter 7: Assigning the Berth

"Filter, score, bind — and then a coin flip"

Exam domain: Kubernetes Fundamentals (44% of the exam) — competency: Scheduling [source: cncf-kcna-certification-page-2026-08-23] [source: cncf-kcna-curriculum-pdf-2026-08-23] | **Estimated share of the exam: ~5% (authored allocation — CNCF publishes domain weights, not competency weights** [source: cncf-kcna-curriculum-pdf-2026-08-23]; **see front matter)** | **Complexity: Mixed | Novelty: Moderate**

Attention Budget

Total time: ~130 minutes | Recommended: split across two sessions, with the break after ☹ Taking Your Bearings #2

Section	Time	Attention Cost	Best Time to Study
\$1 — One Decision, Made Once	10 min	Low	Anytime
\$2 — What Makes a Node Feasible	15 min	Medium	When alert
☹ Taking Your Bearings #1	8 min	Medium	Straight after \$2
\$3 — Asking for a Particular Berth	15 min	Medium	When alert
\$4 — When the Berth Refuses You	25 min	High	Peak attention
☹ Taking Your Bearings #2	8 min	Medium	After a short break
\$5 — Placing Pods Relative to Each Other	18 min	Medium-high	When alert
\$6 — Overruling the Scheduler, and Replacing It	14 min	Medium	Anytime
☹ Taking Your Bearings #3	8 min	Medium	After a short break
\$7 — Everything Is a Filter or a Score	6 min	Low	Anytime

Attention Cost Key:

- **Low:** concrete, familiar, or synthesis material — study anytime.
- **Medium:** new concepts requiring focus — study when alert.
- **High:** dense discrimination work — study at peak attention.

If you only have 15 minutes: read \$1, then read \$4's three-effect table, then work ☹ Taking Your Bearings #2. That is, in my judgement, the densest fifteen minutes in the chapter. \$1 is the frame every other fact hangs on, and the three taint effects are the tightest discrete-recall block here.

"The hard part of assigning a berth is not choosing well. It is that you only choose once." — Lodestar Ledgers

🔊 Soundings

Before reading this chapter, try these eight questions. Your score decides how to read, not whether you're allowed to. There's no wrong answer to arrive with, only different starting positions.

Most of these are about assigning work to machines in general. Three of them ask you to retrieve something specific from Chapters 5 and 6.

1. You have eight machines with different amounts of free memory, and a job that needs 8 GB. Describe how you'd pick a machine for it. Then say what you'd have to know **in advance** to pick it automatically, with no human looking.
2. Two candidate machines come out identical on every measure you can name. How do you choose between them, and does the choice matter?
3. A container declares both a *request* and a *limit* for memory. Chapter 5 said one of those two numbers is read by the scheduler and the other is enforced by the kubelet on the running container. Which is which?
4. A Pod is running happily. Its node then fills up with other work. Chapter 5 was emphatic that a Pod is scheduled once in its lifetime. So what does Kubernetes do about the crowded Pod — move it, or something else?
5. Some machines in your fleet are reserved for one team's workloads. There are two ways to express that: mark the machines, or mark the jobs. What does each approach cost you when a job arrives that nobody remembered to mark?
6. A ReplicaSet wants three Pods and two exist, so it creates a third. There is nowhere in the cluster with room for it. What does the loop do next — give up, retry, error, or something else?
7. You run two copies of a service so that one machine failing doesn't take the service down. Both copies land on the same machine. What have you actually lost, and what would you have had to say in advance to prevent it?
8. In any system that assigns work to workers, give one good reason to override the assignment and pin a job to a named worker — and one reason it's a bad habit.

Answers + reading strategy

1. **Pick one with at least 8 GB available, ideally with headroom.** To automate it, the job's requirement has to be *declared* somewhere a machine can read it. You can't discover "this job needs 8 GB" by

looking at a job that hasn't started yet. Hold onto the word *available*; §2 is mostly about what it means.

2. **Any consistent rule works** — lowest hostname, round-robin, least-recently-used. For correctness it doesn't matter. For behavior it does: a predictable rule concentrates load in predictable places. Kubernetes' actual answer is in §1, and it isn't any of the three above.
3. **The request is the scheduler's input; the limit is the kubelet's enforcement ceiling.** If you had them backwards, mark question 3 wrong and read the note at the bottom of this box.
4. **Nothing moves it.** A Pod is scheduled once and is never rescheduled to a different node. If it has to leave, it is *replaced* by a different Pod, not relocated.
5. **Mark the jobs** and unmarked jobs will happily land on the reserved machines, because nothing told those machines to say no. **Mark the machines** and unmarked jobs are excluded by default. But marking the machines only changes what they *refuse*. Whether it also changes where your marked jobs actually land is a separate question, and §4 turns on the answer.
6. **Nothing dramatic.** The loop's job was to create the missing Pod, and it did. The Pod now exists; it simply has nowhere to run. What the Pod's own status says while that's true is §2's material, and it is worth more than it looks.
7. **You lost the redundancy.** One machine failure now takes down both copies, which is exactly the outcome the second copy existed to prevent. To prevent it you'd have had to say, in advance, that the two copies must not share a machine. Nothing about "two copies" implies "two machines."
8. **Good reason:** the work is tied to something only that machine has — a device, a local disk, a license dongle. **Bad habit:** the assignment stops adapting. If the machine is full, down, or renamed, the job has nowhere to fall back to, and you've opted out of every check the assigner was performing for free.

If you got 6+ right: skim. Read §1 and §4 properly, note the ☆ Fixed Points and the △ Navigational Hazards, and go straight to the checkpoints. You have the instincts; this chapter is supplying the vocabulary and one or two facts that will surprise you.

If you got 3–5 right: read at normal pace. This chapter is calibrated for you.

If you got 0–2 right: read carefully, and start with §1 rather than skipping ahead. **If questions 3, 4 and 6 were among your misses, go back and re-read Chapter 5 §8 and Chapter 5 §4 first.** Those two sections are the entire prerequisite base for §1 through §3, and §2 will read as a list of arbitrary rules without them.

Why This Chapter Matters

The last chapter ended on the one thing the control loop cannot do. It creates a Pod. It does not decide where the Pod goes. That gap is what this chapter closes. The interesting part isn't that a component called the scheduler picks a node. It's that the choice is made **once**, by a component that does not run anything, and is then never revisited. Pods are scheduled once in their lifetime to a specific node, and a Pod is never "rescheduled" to a different node [source: k8s-docs-pod-lifecycle-2026-08-23]. The machine a Pod lands on is the machine it dies on. Every label, every affinity rule, every taint in this chapter exists for one reason: that decision is irreversible, and you get exactly one chance to influence it.

Chapters 4 through 6 made you someone who can write down what should exist. This chapter makes you someone who can also write down *where* it should exist, and where it must not. That's a real shift. It's the first point in this book where you're reasoning about the cluster as a heterogeneous place — machines with different hardware, different owners, different health — rather than as a uniform pool of capacity. Practitioners hold both directions in their heads at once: what a workload needs from a machine, and what a machine is willing to accept. Newcomers only ever think about the first, which is why a node refusing their Pod reads as the cluster being broken.

The stakes, stated flat. Five points on this book's allocation, which is not a lot. What that number doesn't capture is that scheduling is where four separate facts live that have no home anywhere else in the curriculum: that the decision is a two-step operation, that ties are broken at random, that binding is a notification rather than an action, and that a Pod which can't be placed sits in `Pending` rather than `erroring`. Each is one sentence. Each is exactly the shape a recognition exam tends to ask about. This chapter is short on volume and dense on returns.

What You'll Learn

By the end of this chapter, you'll be able to:

- **Trace** a Pod from creation to placement through the scheduler's two-step operation, and say what "binding" actually consists of.
- **Name** the factors a scheduling decision takes into account, and say which of them you can influence from a Pod spec or a node label.
- **Explain** why a node with free memory can still be infeasible for a Pod that needs less memory than the node has.
- **Distinguish** the mechanisms for influencing placement by which direction each one asserts — from the Pod, or from the node.
- **Predict** what happens to an already-running Pod when its node gains a taint, changes a label, or fills up.
- **Choose** between `nodeSelector`, node affinity, taints, and inter-Pod rules for a given placement requirement, and say what each one costs.

- **Recognize** every constraint in this chapter as either a filter or a score — which is the only thing you actually have to remember.

You'll also stop reading `Pending` as an error message, which is a smaller change than it sounds, and I think it saves more marks than anything else here.

§1 — One Decision, Made Once

You met the scheduler in Chapter 3 as a control-plane component and were told, in as many words, that *how* it chooses was being held back until later [cross-bearing: see Ch 3 §2 — the control plane components]. This is later.

A scheduler watches for newly created Pods that have no node assigned; for every such Pod it discovers, it becomes responsible for finding a node for that Pod to run on [source: k8s-docs-kube-scheduler-2026-08-23]. `kube-scheduler` is the default one, and it runs as part of the control plane [source: k8s-docs-kube-scheduler-2026-08-23]. The nodes that meet a Pod's scheduling requirements are called **feasible nodes** [source: k8s-docs-kube-scheduler-2026-08-23].

What goes into the decision

Feasibility is not only about free memory. The documented list of factors a scheduling decision takes into account runs: individual and collective resource requirements, hardware / software / policy constraints, affinity and anti-affinity specifications, data locality, inter-workload interference, and so on [source: k8s-docs-kube-scheduler-2026-08-23]. *Deadlines* are named alongside them in the architecture overview [source: k8s-docs-cluster-architecture-2026-08-23].

This chapter teaches the ones you can control from a Pod spec or a node label: resource requirements, hardware and policy constraints, and affinity. The rest are named here so you recognize them on a list, and so that *"the scheduler checks whether the node has enough memory"* doesn't calcify into the whole story.

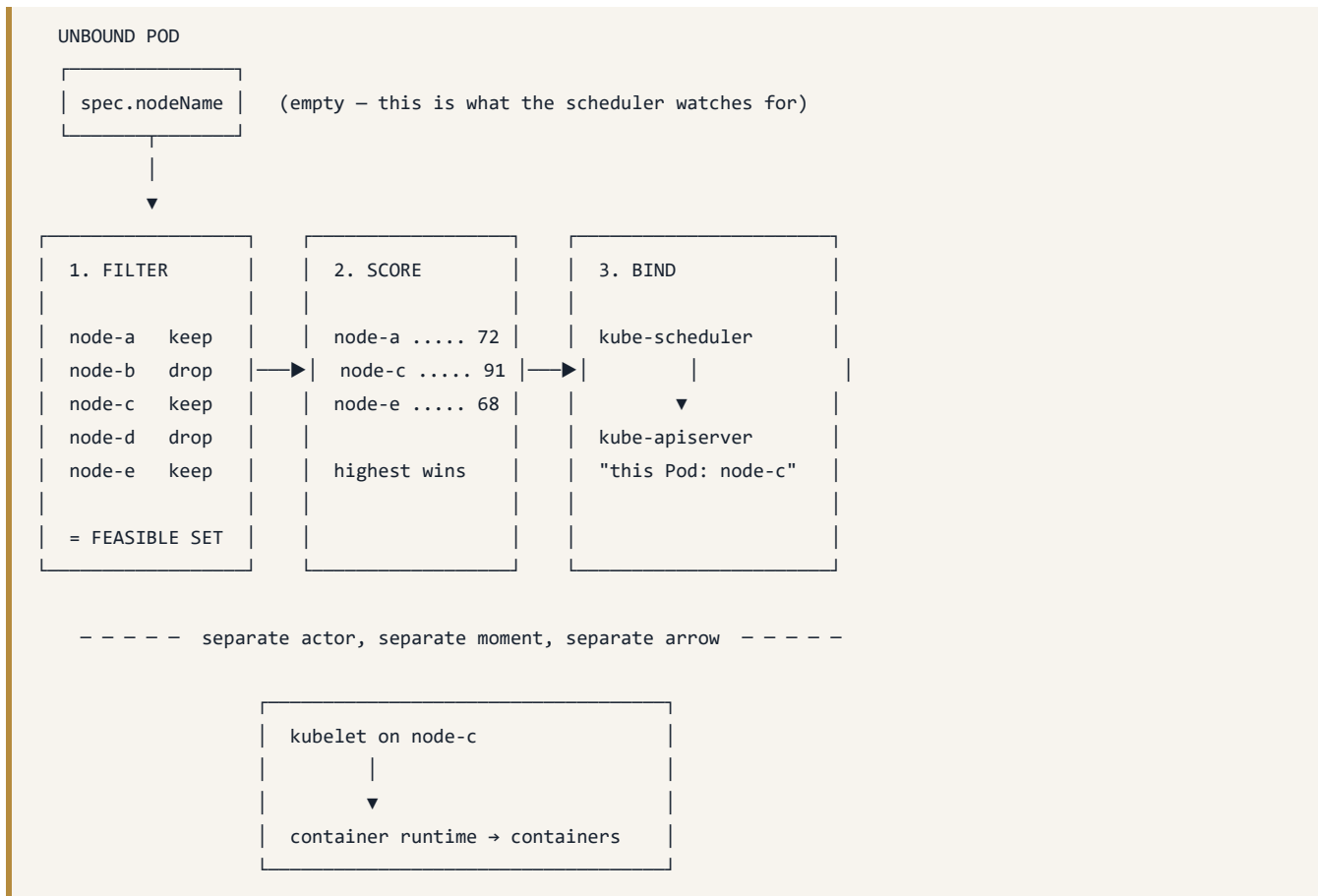
The spine

`kube-scheduler` selects a node in a **2-step operation: filtering, then scoring** [source: k8s-docs-kube-scheduler-2026-08-23].

Filtering finds the set of nodes where it is feasible to schedule the Pod. After this step the node list contains any suitable nodes — often more than one. If the list is empty, that Pod isn't yet schedulable [source: k8s-docs-kube-scheduler-2026-08-23].

Scoring ranks the survivors. The scheduler assigns a score to each node that survived filtering, based on the active scoring rules, and then assigns the Pod to the node with the highest ranking [source: k8s-docs-kube-scheduler-2026-08-23].

Binding is what happens next, and it is not what most people assume. The scheduler notifies the API server about its decision, and *that* is what the word binding names [source: k8s-docs-kube-scheduler-2026-08-23].



Look at where the third arrow goes. The scheduler's arrow lands on the **API server**, not on node-c. Nothing in the scheduler ever touches a container. The kubelet on the chosen node is what starts anything, and it does so because it saw the recorded decision. Not because the scheduler told it to.

☆ Fixed Point:

The scheduler filters, then scores, then binds. Filtering produces the feasible set; scoring ranks it; binding is a notification to the API server. The kubelet on the chosen node is what actually starts the containers.

🧠 **Mnemonic:** *Filter, score, bind* — three words, in the order they happen, and they're printed on the front of this chapter. If you can say the three words you have most of \$1.

And then a coin flip

Here is the fact that surprises people.

If there is more than one node with equal scores, kube-scheduler selects one of them **at random** [source: k8s-docs-kube-scheduler-2026-08-23].

Not the lowest hostname. Not the least loaded. Not round-robin. Random. If you proposed one of the clever answers in Soundings question 2, you were in good company and you were wrong, and being wrong first is exactly why this will stick.

The randomness does real conceptual work, too. It tells you that the scheduler is not trying to be optimal in a way you can reason about or predict. Which means influencing placement is not a matter of understanding the algorithm well enough to out-think it. It's a matter of **saying something**: putting a constraint in the Pod spec, or a mark on the node, that the algorithm is obliged to honor. Everything in §3 through §6 is a way of saying something.

The decision is irreversible

Chapter 3 told you the scheduler selects a node and records that choice, and that it does not start anything. Chapter 5 told you a Pod is scheduled once in its lifetime [*cross-bearing: see Ch 5 §4 — the Pod's lifecycle*]. Put those together and you have the reason this chapter exists.

Pods are created, assigned a unique ID, and scheduled to run on nodes where they remain until termination or deletion. A Pod is never "rescheduled" to a different node; instead, it is replaced by a new, near-identical Pod with a different UID [source: k8s-docs-pod-lifecycle-2026-08-23]. If a node dies, the Pods running on it are marked for deletion after a timeout, and higher-level controllers such as Deployments create the replacements [source: k8s-docs-pod-lifecycle-2026-08-23].

So there is no "move it later." A berth, once allotted, is not renegotiated. There is only "replace it, and hope the replacement lands somewhere better." That's not a design flaw. It's what makes the whole model simple enough to reason about. But it does mean that everything you want to be true about where a Pod runs has to be true *before* the Pod is created.

🔒 **Worth Securing:** If you want a Pod somewhere specific, you have to say so before it exists. Editing a running Pod's placement is not a thing you can do. Every mechanism in this chapter is read at scheduling time, and none of them is enforced continuously afterwards.

..1 §2 — What Makes a Node Feasible

Chapter 5 taught you requests and limits, told you that requests are the input to the scheduler's filtering step, and sent you here for the consequence [*cross-bearing: see Ch 5 §8 — requests and limits*]. Here is the consequence, and it's stranger than the promise made it sound.

The mechanism

The filtering step finds the set of nodes where it is feasible to schedule the Pod. The docs' own example is the `PodFitsResources` filter, which checks whether a candidate node has enough available resources to meet a Pod's specific resource **requests** [source: k8s-docs-kube-scheduler-2026-08-23].

Note the word. Requests. Not limits: the kubelet enforces limits on the running container, and it does that after placement, on a node that has already been chosen [source: k8s-docs-resource-management-2026-08-23]. And not observed usage either. As the next few paragraphs show, a request is spent the moment it is granted, whatever the container does with it afterwards.

🔔 **Closer Look:** `PodFitsResources` is named in the documentation as *an example* of a filter, not as the only one [source: k8s-docs-kube-scheduler-2026-08-23]. Resources are one feasibility test among several; §3 and §4 add more, and §6 covers the fact that the whole filter set is configurable. Don't walk away from this section thinking capacity is the only thing that can make a node infeasible.

Now the part that surprises everyone

When you specify a resource request for a container, the kube-scheduler uses that information to decide which node to place the Pod on, and the kubelet **reserves at least the request amount** of that resource specifically for that container to use [source: k8s-docs-resource-management-2026-08-23].

Read that again with an accountant's eye. A request is a **booking**. Not an estimate, not a hint, not a starting point for measurement. Once a Pod is placed, its requests are taken off the node's available balance and stay off, whether the container ever touches that memory or not.

Before reading on: a node has 16 GiB of memory available to Pods. Four Pods are running on it, and each one requested 4 GiB. Monitoring says the node is using 2 GiB in total. A fifth Pod arrives requesting 1 GiB. Does it schedule on this node? Decide before you keep reading.

It does not. The four Pods booked 16 GiB between them, and the kubelet reserves at least the request amount for each container whether or not the container ever touches it [source: k8s-docs-resource-management-2026-08-23]. Sixteen booked out of sixteen available means zero left, and a Pod asking for 1 GiB is filtered out. The 2 GiB figure describes what the containers are doing with what they booked; it does not hand any of it back.

This is the fact that converts *"the cluster has loads of free memory but my Pod won't schedule"* from a mystery into arithmetic. A node can be busy and empty at the same time, and only one of those two states is the scheduler's business.

☆ Fixed Point:

Filtering fits a Pod's *requests* against a node's *available* capacity, and a request is booked whether or not it is ever used. Ten Pods that each requested 1 GiB and each use 50 MiB have filled a 10 GiB node completely, as far as scheduling is concerned.

The requests / limits / quality-of-service picture is already drawn in `ch05-fig05-requests-limits-qos-classes`. Go back and look at it now that you know which of those numbers the scheduler reads. The QoS classes themselves are Chapter 5's material and matter for eviction rather than for placement [*cross-bearing: see Ch 5 §8 — QoS classes*]; the only thing you need here is which field is the scheduler's input.

What "available" is measured against

A node's status reports both **Capacity** and **Allocatable** [source: `k8s-docs-nodes-2026-08-23`]. Allocatable is the one that matters here: *"Allocatable' on a Kubernetes node is defined as the amount of compute resources that are available for pods", and "the scheduler treats 'Allocatable' as the available capacity for pods"* [source: `k8s-docs-node-allocatable-2026-08-24`]. The scheduler does not over-subscribe Allocatable [source: `k8s-docs-node-allocatable-2026-08-24`].

Practical translation: when you do the arithmetic yourself, do it against Allocatable, not against the machine's total RAM. What makes the two differ, and how it's configured, is Chapter 8's material [*cross-bearing: see Ch 8 — node administration*].

One clause about overhead

Chapter 2 mentioned that a `RuntimeClass` can carry a Pod overhead so the scheduler accounts for the runtime's resource cost, and said the reasoning would arrive later [source: `k8s-docs-runtime-class-2026-08-23`] [*cross-bearing: see Ch 2 §7 — RuntimeClass*]. This is where it arrives: a Pod running under a sandboxed runtime doesn't consume only what its containers asked for, and the overhead exists so the scheduler's arithmetic accounts for the runtime's own cost rather than pretending it's free.

And one clause about making room

There is one circumstance in which the scheduler does something other than wait. If a Pod cannot be scheduled, the scheduler tries to preempt — evict — lower priority Pods to make scheduling of the pending Pod possible [source: `k8s-docs-pod-priority-preemption-2026-08-24`]. Priority and preemption are real, they're configured with a `PriorityClass` [source: `k8s-docs-pod-priority-preemption-2026-08-24`], and they're out of scope here. Register that they exist so that nothing in this chapter reads as a lie later.

The failure mode, which is the exam-critical half

If none of the nodes are suitable, **the Pod remains unscheduled until the scheduler is able to place it** [source: `k8s-docs-kube-scheduler-2026-08-23`]. Its phase is `Pending`, which covers, among other things, *"time a Pod spends waiting to be scheduled"* [source: `k8s-docs-pod-lifecycle-2026-08-23`].

That's it. Nothing errors. Nothing times out. Nothing retries with different parameters or gives up and files a complaint. The Pod waits, indefinitely, and the controller that created it has already done its part and moved on. A Pod can sit in Pending for a week and the cluster will consider this a perfectly ordinary state of affairs: a vessel riding at anchor outside the harbour, on nobody's list.

⚠ **Navigational Hazards:** Pending is a **state**, not an error. It is the honest report of a Pod that has been accepted by the cluster and has nowhere to run yet. No component is quietly retrying it with looser constraints, and no timer will eventually convert it into a failure. If you want to know *why* a Pod is Pending, you have to go and ask, which is Chapter 13's whole opening move [cross-bearing: see Ch 13 — reading Pod failure signatures].

And notice what an unschedulable Pod is, structurally: a standing, machine-readable statement that the cluster is short of somewhere to put work. Something could be watching for exactly that. [cross-bearing: see Ch 17 — reacting to unschedulable Pods]

One related boundary, while you're here: nothing stops you from booking the entire cluster with requests you don't need. The mechanisms that stop *other people* doing that to you are ResourceQuota and LimitRange, and they belong to Chapter 8 [cross-bearing: see Ch 8 — quotas and limit ranges].

🕒 Taking Your Bearings #1 — How the Decision Is Made

Five questions on §1 and §2. Two of them reach back into earlier chapters.

1. Name the scheduler's three steps in order, and say which component actually starts the container.

- A) Score, filter, bind — and the scheduler starts the container.
- B) Filter, score, bind — and the kubelet on the chosen node starts the container.
- C) Filter, score — and the kubelet starts the container; binding is something the kubelet does.
- D) Filter, bind, score — and the container runtime starts the container on the scheduler's instruction.

2. Three nodes survive filtering and all three score identically. What does kube-scheduler do?

- A) Picks the node with the lowest name, for determinism.
- B) Picks the least-loaded node by current CPU usage.
- C) Picks one of them at random.
- D) Leaves the Pod Pending until the tie resolves itself.

3. [retrieval: ch5] A container requests 2 GiB of memory and sets a limit of 4 GiB. Which of those two numbers does the scheduler use when deciding whether a node can take this Pod, and what does the other number do?

4. [retrieval: ch5] A node has 16 GiB allocatable memory and four Pods on it, each of which requested 4 GiB. Monitoring shows the node using 2 GiB in total. A new Pod requesting 1 GiB will not schedule there.

Explain why, in one sentence.

5. [retrieval: ch6] A Deployment wants three replicas. Two are running. The third cannot be placed anywhere in the cluster. Describe the state of the third Pod, and describe what the ReplicaSet controller does about it.

Answers with Explanations:

1 — B. Filtering produces the feasible set, scoring ranks it, and binding notifies the API server [source: k8s-docs-kube-scheduler-2026-08-23]; the kubelet is the agent on each node that makes sure the containers described in the PodSpecs are running [source: k8s-docs-cluster-architecture-2026-08-23].

- **A is wrong** on both counts: the order is inverted (you cannot score nodes you haven't filtered), and the scheduler never starts anything. This is the single most common misconception about the scheduler and it's worth naming as such.
- **C is wrong** because it drops binding, or reassigns it. Binding is the scheduler's third act, not the kubelet's.
- **D is wrong** because binding is not a middle step and the scheduler does not instruct the runtime. It doesn't talk to nodes at all. It talks to the API server.

2 — C. *"If there is more than one node with equal scores, kube-scheduler selects one of these at random"* [source: k8s-docs-kube-scheduler-2026-08-23].

- **A and B are wrong** in the instructive way: they're the answers a competent engineer would design. They are not the answer the system implements, and knowing that is what tells you the scheduler's tie-breaking is not something you can plan around.
- **D is wrong** because a tie is not a failure. A tie means *several* nodes are acceptable, which is the opposite of unschedulable.

3 — The request. The kube-scheduler uses the resource *request* to decide which node to place the Pod on; the *limit* is enforced by the kubelet, which does not allow the running container to use more of the resource than the limit sets [source: k8s-docs-resource-management-2026-08-23]. The kubelet's enforcement happens after placement, on a node already chosen.

4 — The requests are booked whether or not they're used, so the node is already fully allocated. Four Pods × 4 GiB requested = 16 GiB booked against 16 GiB allocatable. The kubelet reserves at least the request amount for each container [source: k8s-docs-resource-management-2026-08-23], and the scheduler does not over-subscribe Allocatable [source: k8s-docs-node-allocatable-2026-08-24]. Usage of 2 GiB describes what the containers are doing with what they booked; it does not return any of it. If you can say this sentence out loud without hesitating, you can diagnose most real Pending Pods.

5 — The Pod is Pending, indefinitely; the controller does nothing further. The Pod remains unscheduled until the scheduler is able to place it [source: k8s-docs-kube-scheduler-2026-08-23], and its phase is Pending, which includes time spent waiting to be scheduled [source: k8s-docs-pod-lifecycle-2026-08-23].

From the controller's point of view, its work is complete — it created the Pod it was missing. Nothing about the Pod's inability to run reflects back on the loop that created it. *Deliberately not covered here:* what you'd run to find out **why**. `kubectl describe`, the event stream, and the scheduler's own message are Chapter 13's material, and the fact that you know exactly what state to look for is what will make that chapter fast.

Checkpoint: You've Now Mastered

✓ The two-step operation, and what binding actually is ✓ The documented list of factors a scheduling decision weighs ✓ Why a tie is broken at random, and what that implies about influencing placement ✓ Why a node can be busy and empty at the same time ✓ Why Pending is a state and not an error

□ How a Pod asks for a particular kind of node (next) □ How a node refuses a Pod (§4)

§3 — Asking for a Particular Berth

Everything so far has been the scheduler's own arithmetic. Now you start putting your thumb on it.

The direction inverts

Chapter 4 taught label selectors as the universal join, and listed node scheduling constraints as one of the four things they're used for [*cross-bearing: see Ch 4 §5 — labels and selectors*]. Watch what changes here.

Every previous use of labels in this book has been **an object selecting a set of Pods** — a Service selecting its backends, a ReplicaSet selecting the Pods it owns. Here it is **a Pod selecting nodes**. Same mechanism, opposite direction. If you've internalized "selectors find Pods," this will read as backwards for a page or two until you say the inversion out loud.

Nodes have labels [source: k8s-docs-assign-pod-node-2026-08-23]. You can attach them manually, and Kubernetes also populates a standard set of labels on all nodes in a cluster [source: k8s-docs-assign-pod-node-2026-08-23]. The kubelet supplies `kubernetes.io/hostname`, which it populates with the hostname of the node, along with `kubernetes.io/os` and `kubernetes.io/arch` [source: k8s-docs-well-known-labels-2026-08-24]. `topology.kubernetes.io/zone` and `topology.kubernetes.io/region` are populated by the kubelet **or** the cloud-controller-manager [source: k8s-docs-well-known-labels-2026-08-24], which means they show up where the cluster has a cloud provider to ask, and may not be there at all on the single-machine cluster you're learning on. Note that now, because §5's examples lean on the zone label.

You can see them and add to them with the commands you already know from Chapter 4, just pointed at a different resource:

```
kubectl get nodes --show-labels
kubectl label nodes <your-node-name> disktype=ssd
```

[source: k8s-docs-assign-pods-nodes-task-2026-08-24]

One caution, more relevant than it looks: if you use labels for node isolation, choose label keys the kubelet cannot modify. The NodeRestriction admission plugin prevents the kubelet from setting labels with a `node-restriction.kubernetes.io/` prefix [source: k8s-docs-assign-pod-node-2026-08-23]. A node that can label itself into a privileged group is not an isolation boundary. The admission gate that enforces this is Chapter 8's *[cross-bearing: see Ch 8 — admission control]*.

nodeSelector

`nodeSelector` is the simplest recommended form of node selection constraint. You add the `nodeSelector` field to your Pod specification and name the node labels you want the target node to have, and **Kubernetes only schedules the Pod onto nodes that have each of the labels you specify** [source: k8s-docs-assign-pod-node-2026-08-23].

Each. Not any. It is an AND of exact matches and it offers nothing else: no "one of these," no "anything except," no "greater than." That bluntness is a feature for the ninety percent of requirements that are genuinely blunt ("this Pod needs an SSD"). The other ten percent are why the next thing exists.

Node affinity

Affinity expands the types of constraint you can define: the language is more expressive, you can indicate that a rule is soft or preferred so the scheduler still schedules the Pod even if it can't find a matching node, and you can constrain a Pod using labels on other Pods [source: k8s-docs-assign-pod-node-2026-08-23]. That third capability is §5. The first two are here.

Node affinity functions like the `nodeSelector` field but is more expressive and allows you to specify soft rules [source: k8s-docs-assign-pod-node-2026-08-23]. Concretely, it adds two things:

A richer operator set. In, NotIn, Exists, DoesNotExist, Gt and Lt [source: k8s-docs-assign-pod-node-2026-08-23]. Compare that to `nodeSelector`'s single implicit "equals."

Two hardness levels, expressed by two field names that everyone resents on first contact [source: k8s-docs-assign-pod-node-2026-08-23]:

- `requiredDuringSchedulingIgnoredDuringExecution` — the scheduler **can't** schedule the Pod unless the rule is met.
- `preferredDuringSchedulingIgnoredDuringExecution` — the scheduler **tries** to find a node that meets the rule, but if a matching node is not available, it still schedules the Pod.

For preferred rules you can attach a weight to each instance, and those weights feed the node's score alongside the other scoring functions; nodes with the highest total score are prioritized [source: k8s-

docs-assign-pod-node-depth-2026-08-24]. You don't need the arithmetic for this exam. You do need to notice where preferred rules land: in the **scoring** step, not the filtering step. Hold that thought until §7.

🔗 **Mnemonic:** Read `requiredDuringSchedulingIgnoredDuringExecution` as two clauses joined at the seam: **required when scheduling, ignored once running**. The word is forty-six characters; the meaning is six. Every one of these field names decomposes the same way: first half is what happens at placement, second half is what happens afterwards.

If you do write affinity by hand, two combination rules are worth knowing because they're easy to get backwards: if you specify multiple terms in `nodeSelectorTerms`, the Pod can be scheduled if **one** of the terms is satisfied — terms are ORed; if you specify multiple expressions in a single `matchExpressions` field, the Pod can be scheduled **only if all** the expressions are satisfied — expressions are ANDed [source: k8s-docs-assign-pod-node-depth-2026-08-24].

The second half of the field name

`IgnoredDuringExecution` means that **if the node's labels change after Kubernetes schedules the Pod, the Pod continues to run** [source: k8s-docs-assign-pod-node-2026-08-23].

You already knew this. You retrieved it in Soundings question 4 and Chapter 5 taught it as a general rule about the Pod's lifetime. What's new is that the rule has a name, and the name is sitting in plain sight inside a field you'll type. A Pod placed on a node labeled `disktype=ssd` does not get evicted when someone strips that label off. The rule was a scheduling-time question and scheduling time is over.

The gradient



That figure is worth thirty seconds. The common misreading is that node affinity is simply "more powerful nodeSelector," a single upgrade path. It isn't. **nodeSelector and required node affinity fail identically**: no matching node, no placement, Pod sits in Pending. What affinity adds is a second, independent axis. Choosing between the three cells is choosing how much you want to say, and separately, how badly you want it *obeyed*.

Reach for `nodeSelector` first. Affinity exists for the requirements `nodeSelector` genuinely cannot express, and a soft rule you didn't need is a rule someone will misread as a guarantee eighteen months from now.

☆ Fixed Point:

`nodeSelector` is an AND of exact node-label matches — nothing more. Node affinity is the same idea with a richer operator set and an optional soft (preferred) mode. `IgnoredDuringExecution` means a label change after binding does not move the Pod.

Chapter 2 also told you a `RuntimeClass` can carry scheduling constraints — a `nodeSelector` and tolerations — so that Pods land on nodes that support the handler [source: [k8s-docs-runtime-class-2026-08-23](#)]. The `nodeSelector` half of that promise is now discharged. The tolerations half is the next section.

§4 — When the Berth Refuses You

This is the densest section in the chapter. Take it at peak attention, and take the three-effect table slowly.

The inversion

Everything in §3 was a property of a **Pod** that draws it toward certain nodes. This section is the other direction: the mark a berth flies to say *not you*.

Node affinity is a property of Pods that attracts them to a set of nodes (either as a preference or a hard requirement). Taints are the opposite — they allow a node to repel a set of pods. [source: [k8s-docs-taints-tolerations-2026-08-23](#)]

Tolerations are applied to Pods, and they allow the scheduler to schedule Pods with matching taints [source: [k8s-docs-taints-tolerations-2026-08-23](#)]. One or more taints are applied to a node; this marks that the node should not accept any Pods that do not tolerate the taints [source: [k8s-docs-taints-tolerations-2026-08-23](#)].

So: the **taint** lives on the node and is a refusal. The **toleration** lives on the Pod and is an exemption from that refusal. Two objects, two directions, and getting them the wrong way round is the source of most confusion in this material.

Taints go on and come off with `kubectl taint`. The removal form is the same command with a trailing minus sign, which is easy to miss and easy to mistype:

```
kubectl taint nodes node1 key1=value1:NoSchedule
kubectl taint nodes node1 key1=value1:NoSchedule-
```

[source: k8s-docs-taints-tolerations-depth-2026-08-24]

The qualification that catches everyone

Tolerations allow scheduling but don't guarantee scheduling: the scheduler also evaluates other parameters as part of its function. [source: k8s-docs-taints-tolerations-2026-08-23]

A toleration removes a veto. That is the entire extent of its power. It does not make a request, it does not raise a node's score, it does not reserve anything, and it does not create capacity. Your GPU workload with a matching toleration is now *allowed* onto the GPU nodes, along with being allowed onto every other node in the cluster, which it was already. Nothing has pulled it toward the GPU nodes at all.

⚠ **Navigational Hazards:** A toleration is not a request. Taints and tolerations exclude; affinity and nodeSelector attract. They are **complementary, not alternative**. If you want a set of nodes dedicated to a workload *and* you want that workload to actually land there, you need both: a taint to keep everyone else off, and a label plus affinity (or nodeSelector) to pull your workload on. The documentation says exactly this — if you want to dedicate nodes to a group *and* ensure they only use the dedicated nodes, *"you should additionally add a label similar to the taint to the same set of nodes... and the admission controller should additionally add a node affinity to require that the pods can only schedule onto nodes labeled with dedicated=groupName"* [source: k8s-docs-taints-tolerations-depth-2026-08-24]. Using only one of the two is the most common real-world mistake in this material, and it produces a cluster that looks like it's misbehaving when it's doing precisely what you asked.

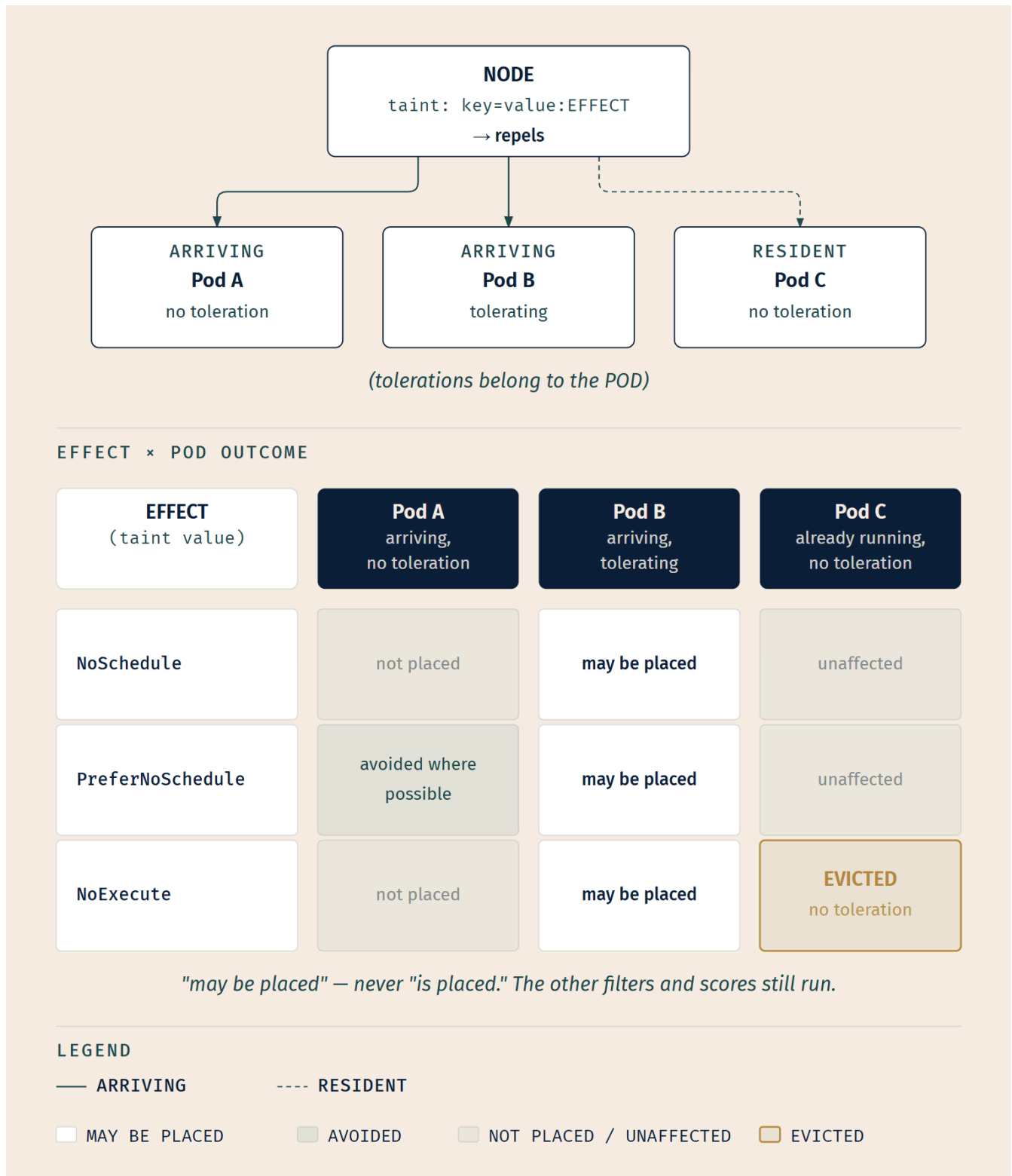
The three effects

Learn these by **when they act:** whether the refusal is posted at the approach, or served on a vessel already tied up. That's what separates them, and it's what a question will turn on.

NoSchedule — no new Pods will be scheduled on the tainted node unless they have a matching toleration. **Pods currently running on the node are not evicted** [source: k8s-docs-taints-tolerations-2026-08-23].

PreferNoSchedule — the soft version of NoSchedule. The control plane will *try* to avoid placing a Pod that does not tolerate the taint on the node, but it is not guaranteed [source: k8s-docs-taints-tolerations-2026-08-23].

NoExecute — the only effect that touches Pods already on the node. Pods that do not tolerate the taint are **evicted immediately**. Pods that tolerate the taint without specifying tolerationSeconds remain bound forever. Pods that tolerate the taint *with* a specified tolerationSeconds remain bound for that long, after which the node lifecycle controller evicts them [source: k8s-docs-taints-tolerations-2026-08-23].



Pod C is the whole point of the figure. Two of the three rows leave it completely alone. Only one row reaches out and touches something that was already moored.

📌 **Snag:** `PreferNoSchedule` is a *preference*, and preferences lose. Pods that don't tolerate the taint will land on a `PreferNoSchedule` node when nothing better is available, and that is correct behavior, not a bug. If you need "never," you need `NoSchedule`.

Matching

Here the metaphors run out. Four rules, no narrative, stated flat.

Dead Reckoning: A toleration matches a taint when the keys are the same and the effects are the same, and one of two operator conditions holds. If the operator is `Exists`, no value should be specified. If the operator is `Equal`, the values must be equal. Two wildcards modify this. If the key is empty, the operator must be `Exists`, which matches all keys and all values — the effect must still match. An empty effect matches all effects with the given key. [source: k8s-docs-taints-tolerations-2026-08-23]

Toleration	Matches	Notes
key + effect + operator: <code>Equal</code> + value	Taints with the same key, same effect, same value	The exact-match case
key + effect + operator: <code>Exists</code>	Taints with the same key and same effect, any value	Do not specify a value with <code>Exists</code>
empty key + operator: <code>Exists</code> + effect	All taints with that effect, any key, any value	Empty key <i>requires</i> <code>Exists</code>
key + operator: <code>Exists</code> + empty effect	All effects for that key	Effect wildcard

And when a node carries more than one taint, the procedure is: *"start with all of a node's taints, then ignore the ones for which the pod has a matching toleration; the remaining un-ignored taints have the indicated effects on the pod"* [source: k8s-docs-taints-tolerations-depth-2026-08-24]. Tolerating three of four taints does not get you onto the node. The fourth still applies.

The taints Kubernetes adds for you

You are not the only party putting taints on nodes.

The control plane, using the node controller, automatically creates taints with a `NoSchedule` effect for node conditions. **The scheduler checks taints, not node conditions, when it makes scheduling decisions.** This ensures that node conditions don't directly affect scheduling. [source: k8s-docs-taints-tolerations-depth-2026-08-24]

That's an elegant piece of design. Node health doesn't get a special channel into the scheduler; it gets translated into the *same* mechanism you'd use by hand.

That paragraph covers one of two families. Separately, the node controller also automatically taints a node when certain conditions are true, including the case where the API server cannot communicate with

a node's kubelet at all [source: k8s-docs-taints-tolerations-depth-2026-08-24], and those taints are the NoExecute pair in the table below. So the built-in family carries both effects, not just NoSchedule, and the table shows them together [source: k8s-docs-daemonset-2026-08-24]:

Taint key	Effect
node.kubernetes.io/not-ready	NoExecute
node.kubernetes.io/unreachable	NoExecute
node.kubernetes.io/disk-pressure	NoSchedule
node.kubernetes.io/memory-pressure	NoSchedule
node.kubernetes.io/pid-pressure	NoSchedule
node.kubernetes.io/unschedulable	NoSchedule
node.kubernetes.io/network-unavailable	NoSchedule

There is also `node.cloudprovider.kubernetes.io/uninitialized`, added when the kubelet starts with an external cloud provider [source: k8s-docs-taints-tolerations-depth-2026-08-24].

Two of these deserve a note each.

Kubernetes automatically adds a toleration for `node.kubernetes.io/not-ready` and `node.kubernetes.io/unreachable` with `tolerationSeconds=300`, unless you or a controller set those tolerations explicitly [source: k8s-docs-taints-tolerations-depth-2026-08-24]. That five-minute grace period is why a briefly-unreachable node doesn't instantly shed everything running on it.

And `node.kubernetes.io/unschedulable` — remember that one. Marking a node unschedulable is a deliberate administrative act: it prevents the scheduler from placing new Pods onto that node but does not affect the Pods already there, and it is the preparatory step before a reboot or other maintenance [source: k8s-docs-nodes-2026-08-23]. That act is Chapter 8's opening move [*cross-bearing: see Ch 8 — taking a node out of service*].

The mechanism you'd already met

Chapter 6 told you that DaemonSets keep running on nodes where nothing else will, and said you'd already met the mechanism in disguise [*cross-bearing: see Ch 6 §7 — DaemonSets*]. Here it is: **the DaemonSet controller automatically adds tolerations for the built-in condition taints** to the Pods it creates — six of the seven unconditionally, with the seventh, `network-unavailable`, added only for DaemonSet Pods that request host networking [source: k8s-docs-daemonset-2026-08-24]. Because the controller sets the `node.kubernetes.io/unschedulable:NoSchedule` toleration automatically, Kubernetes can run DaemonSet Pods on nodes that are marked unschedulable [source: k8s-docs-daemonset-2026-08-24]. And the NoExecute tolerations for `not-ready` and `unreachable` carry no `tolerationSeconds`, so DaemonSet Pods running on such nodes are not evicted [source: k8s-docs-daemonset-2026-08-24].

Which is exactly why your log-shipping agent is still reporting from a node that's under memory pressure and refusing all other work. It isn't privileged. It just tolerates almost everything.

Logbook Entry:

A platform team buys eight GPU nodes. They're expensive, they're scarce, and the machine-learning group has waited four months for them. The obvious first move is to keep everyone else off, so the team taints all eight: `kubectl taint nodes gpu-01 dedicated=ml:NoSchedule`, and so on down the list.

That works immediately and visibly. Batch jobs stop landing on the GPU nodes. The team writes it up as done.

The ML group adds the matching toleration to their training Pods and starts a run. The Pods schedule. The run is slow. Someone eventually checks, and every training Pod is sitting on ordinary CPU nodes — the toleration let them onto the GPU nodes and nothing ever pushed them there. In a cluster of two hundred nodes, eight of which are GPUs, the odds of landing on a GPU node by chance are exactly what you'd expect them to be.

The fix takes one line. Label the same eight nodes `dedicated=ml`, and give the training Pods a `nodeSelector` (or a required node affinity) for that label. Now the taint keeps everyone else off, and the label pulls the right work on.

Keeping a berth clear and being steered into it are two different acts, and only one of them had been performed. The version of this story where the team notices in ten minutes and the version where they notice in three weeks differ only in whether somebody knew that a toleration doesn't attract.

☆ Fixed Point:

A taint is the node's refusal; a toleration is the Pod's exemption from that refusal. Tolerations permit — they never attract. Of the three effects, only `NoExecute` affects Pods that are already running.

Dedicated nodes also show up later as an isolation control rather than a scheduling one [*cross-bearing: see Ch 12 — workload isolation*]. And a Pending Pod whose cause is a taint rather than a shortage of capacity looks identical from the outside until you go and read the events [*cross-bearing: see Ch 13 — diagnosing Pending*].

🕒 Taking Your Bearings #2 — The Two Directions

Five questions on §3 and §4. One reaches back to Chapter 4.

1. [retrieval: ch4] Every previous use of labels in this book has been one object selecting a set of Pods. `nodeSelector` uses the same label-selector mechanism. What is selecting what here, and in which direction?

2. **△ This one is intentionally hard, and the intuitive answer is wrong.** Struggle is the point — missing it is expected and is exactly what makes the correct answer stick.

Your organization has a per-seat license for a CAD package, and only six machines in the cluster are covered by it. You taint those six `license=cad:NoSchedule` and add a matching toleration to the CAD rendering workload. A colleague reports that the rendering Pods are running on ordinary, unlicensed nodes. Is anything broken?

3. A node gains a new taint while Pods are already running on it. For each of the three effects, say whether the running Pods are affected. Then, for `NoExecute` only, say what happens to a Pod that *does* tolerate the taint — with and without a `tolerationSeconds` value.

4. **⚠** A Pod is running on a node. Someone removes the node label that the Pod's `requiredDuringSchedulingIgnoredDuringExecution` node-affinity rule was matching on. What happens to the Pod?

- A) It is evicted immediately, because the rule is required.
- B) It is evicted after a grace period, then rescheduled elsewhere.
- C) Nothing. It keeps running.
- D) It is marked `Pending` and the kubelet stops its containers.

5. Give one placement requirement that `nodeSelector` cannot express but node affinity can, and name the feature of affinity that makes the difference.

Answers with Explanations:

1 — The Pod's spec selects nodes, by the nodes' labels. The direction is inverted relative to every prior use. Nodes have labels, and the recommended approaches to constraining placement all use label selectors [source: k8s-docs-assign-pod-node-2026-08-23]. Same machinery, opposite subject and object: previously a controller or Service selected Pods; here the Pod selects nodes. If this felt backwards while reading §3, that reaction was correct and worth keeping.

2 — No. Nothing is broken, and the cluster is doing exactly what you told it. *"Tolerations allow scheduling but don't guarantee scheduling"* [source: k8s-docs-taints-tolerations-2026-08-23]. The taint keeps *other* Pods off the licensed nodes. The toleration makes your Pods *eligible* for the licensed nodes, and they were already eligible for every untainted node in the cluster, so they land wherever the ordinary scoring puts them.

The fix requires a second, opposite mechanism. Label the licensed nodes (say `license=cad`) and give the workload a `nodeSelector` or a required node affinity for that label. The documentation prescribes exactly

this pairing for dedicated nodes: add a label similar to the taint to the same set of nodes, and add a node affinity requiring that the Pods can only schedule onto nodes carrying it [source: k8s-docs-taints-tolerations-depth-2026-08-24].

Taints exclude. Affinity attracts. A dedicated-node setup needs both. That sentence is the single most transferable operational fact in this chapter.

3 — NoSchedule: unaffected. PreferNoSchedule: unaffected. NoExecute: non-tolerating Pods are evicted immediately. With NoSchedule, Pods currently running on the node are not evicted [source: k8s-docs-taints-tolerations-2026-08-23]. PreferNoSchedule is the soft version of the same thing and likewise concerns placement only [source: k8s-docs-taints-tolerations-2026-08-23]. NoExecute affects Pods already running: non-tolerating Pods are evicted immediately [source: k8s-docs-taints-tolerations-2026-08-23]. The classic wrong answer is that NoSchedule evicts. It doesn't, and the word *Schedule* in the name is telling you so.

For a Pod that does tolerate a NoExecute taint: without a tolerationSeconds value it remains bound forever; with one, it remains bound for that long, after which the node lifecycle controller evicts it [source: k8s-docs-taints-tolerations-2026-08-23]. Kubernetes sets that value for you on exactly two taints — not-ready and unreachable, at tolerationSeconds=300 — unless you or a controller set those tolerations explicitly [source: k8s-docs-taints-tolerations-depth-2026-08-24]. Five minutes of grace, then the node sheds its work.

4 — C. IgnoredDuringExecution means that if the node labels change after Kubernetes schedules the Pod, the Pod continues to run [source: k8s-docs-assign-pod-node-2026-08-23]. The required half applies at scheduling time; the Ignored half applies afterwards, and both halves are in the field name.

- **A is wrong** because it reads only the first half of the name. required describes what the scheduler must honor, not what the kubelet must continuously enforce.
- **B is wrong** on two counts: nothing evicts, and even if something did, a Pod is not rescheduled to a different node — it would be *replaced* [source: k8s-docs-pod-lifecycle-2026-08-23].
- **D is wrong** because Pending describes a Pod that hasn't been placed. This one has been placed and is running.

5 — Any soft requirement, or any rule needing an operator other than equality. nodeSelector is an AND of exact label matches: Kubernetes only schedules the Pod onto nodes that have each of the labels you specify [source: k8s-docs-assign-pod-node-2026-08-23]. Node affinity is more expressive and allows soft rules [source: k8s-docs-assign-pod-node-2026-08-23], and it supports In, NotIn, Exists, DoesNotExist, Gt and Lt [source: k8s-docs-assign-pod-node-2026-08-23]. So: "prefer SSD nodes but run anywhere" needs preferred; "any zone except zone-c" needs NotIn; "a node that has a GPU label at all, whatever its value" needs Exists.

Checkpoint: You've Now Mastered

✓ Which direction each mechanism asserts from — Pod, or node ✓ The three taint effects and which one touches running Pods ✓ What `tolerationSeconds` does, and the one place Kubernetes sets it for you ✓ Why a toleration alone never gets your Pod where you wanted it ✓ How to decode any ...
DuringScheduling...DuringExecution field name on sight

□ Placing Pods relative to *each other* (next) □ Getting out of the scheduler's way entirely (§6)

Good place for a break. §5 changes altitude.

§5 — Placing Pods Relative to Each Other

Everything so far has constrained a Pod against a property of a **node**. This section constrains a Pod against the properties of **other Pods**, and to do that, it needs an abstraction you haven't met.

Start with the failure

Two replicas of a service, created so that one machine failing doesn't take the service down. Both land on the same node. Two hulls, one mooring, and one bad night takes both. The service now has exactly the availability it had with one replica, plus double the resource bill.

Nothing in §2 or §3 prevents this. Here is why. The two Pods have identical requests, identical labels, identical affinity — they came out of one Deployment, so of course they do [*cross-bearing: see Ch 6 §1 — Deployments and ReplicaSets*]. Nothing in their specs distinguishes one node from another, and nothing tells the scheduler that these two Pods are related, so it is entirely free to pick the same node twice. It is not being careless. You never told it these two were connected.

That's the gap. Redundancy is a property of a **set**, and none of the mechanisms so far can express a property of a set. Every rule you've written has been about one Pod and one node, evaluated in isolation.

Inter-Pod affinity and anti-affinity

Inter-pod affinity and anti-affinity let you constrain Pods against **labels on other Pods** — "only schedule on nodes in the same zone as a Pod with this label," or "spread these Pods across nodes" [source: k8s-docs-assign-pod-node-2026-08-23].

- **Pod affinity attracts.** Schedule this Pod where a Pod carrying that label already is. Useful for co-locating things that talk to each other constantly.
- **Pod anti-affinity repels.** Do not schedule this Pod where a Pod carrying that label already is. This is the availability tool.

Both come in the same required and preferred flavours as node affinity. This is genuinely the same machinery pointed at a different set of labels, not a second system to learn. One exception, since §3 just handed you six operators: `Gt` and `Lt` are node-affinity-only and are not available for `podAffinity` [source: k8s-docs-assign-pod-node-depth-2026-08-24]. `podAntiAffinity` in

requiredDuringSchedulingIgnoredDuringExecution mode means only a single Pod can be scheduled into a single topology domain; in preferredDuringSchedulingIgnoredDuringExecution mode you lose the ability to enforce the constraint [source: k8s-docs-topology-spread-constraints-2026-08-24].

Notice the phrase that keeps appearing: *topology domain*.

The domain is a variable

This is the hard idea in the section, and it's the one that makes §5 .all rather than .all .

The domain is not always "the node." You express the topology domain using a `topologyKey`, **which is the key for the node label that the system uses to denote the domain** [source: k8s-docs-assign-pod-node-depth-2026-08-24]. Nodes that have a label with that key and identical values are considered to be in the same topology [source: k8s-docs-topology-spread-constraints-2026-08-24].

So the same rule, over the same cluster, means different things depending on which label you name.

Same cluster. Same rule: no two Pods labelled `app=web` in one domain.

The only difference is the topologyKey.

TOPOLOGYKEY: KUBERNETES.IO/HOSTNAME

a domain = one node

ZONE-A

n1

[web]

n2

[web]

n3

[web]

ZONE-B

n4

[web]

n5

[web]

n6

[web]

6 domains → up to 6 Pods placed

TOPOLOGYKEY: TOPOLOGY.KUBERNETES.IO/ZONE

a domain = one zone

ZONE-A

n1

[web]

n2

n3

ZONE-B

n4

[web]

n5

n6

2 domains → at most 2 Pods placed

One label key changed. The rule's meaning changed with it.

Same cluster, same six nodes, same two zones, same rule text. Change one label key and the rule goes from "spread across machines" to "spread across failure zones," which is dramatically stricter, and, if you only have two zones, dramatically more likely to leave Pods Pending.

☆ Fixed Point:

Inter-Pod rules are evaluated against the labels of Pods that are *already placed*, within a topology domain defined by a node label (topologyKey). The domain is the part people forget — it is a variable, not a synonym for "node."

🔍 **Closer Look:** The documentation states the cost of these rules without explaining it: *"Inter-pod affinity and anti-affinity require substantial amounts of processing which can slow down scheduling in large clusters significantly. We do not recommend using them in clusters larger than several hundred nodes"* [source: k8s-docs-assign-pod-node-depth-2026-08-24]. The explanation below is mine rather than theirs, and it is the obvious one. To decide whether a single node is feasible, the scheduler now has to know what's already running everywhere else in the domain: the answer for node-a depends on the contents of node-b. That's a fundamentally different cost shape from "does this node have 4 GiB free." This is a sharp tool. It is not free.

Topology spread constraints

Anti-affinity can say "not in the same domain." What people usually *want* is "distributed fairly evenly, and tell me how much unevenness you'll tolerate." That's a different requirement, and it has a purpose-built mechanism.

You can use *topology spread constraints* to control how Pods are spread across your cluster among failure-domains such as regions, zones, nodes, and other user-defined topology domains. [source: k8s-docs-topology-spread-constraints-2026-08-24]

This can help to achieve high availability as well as efficient resource utilization. [source: k8s-docs-topology-spread-constraints-2026-08-24]

Like everything else in this section, they rely on node labels to identify the topology domain each node is in [source: k8s-docs-topology-spread-constraints-2026-08-24]. Four fields carry the meaning, and for this exam you want to recognize them rather than compose them:

Field	What it says
topology Key	The key of node labels. Nodes that have a label with this key and identical values are considered to be in the same topology. [source: k8s-docs-topology-spread-constraints-2026-08-24]
labelSel ector	Finds the matching Pods. Pods matching this selector are counted to determine the number of Pods in their corresponding topology domain. [source: k8s-docs-topology-spread-constraints-2026-08-24]
maxSkew	Describes the degree to which Pods may be unevenly distributed. Must be specified and must be greater than zero. [source: k8s-docs-topology-spread-constraints-2026-08-24]
whenUnsa tisfiabl e	DoNotSchedule (the default) tells the scheduler not to schedule the Pod; ScheduleAnyway tells the scheduler to still schedule it while prioritizing nodes that minimize the skew. [source: k8s-docs-topology-spread-constraints-2026-08-24]

Read `whenUnsatisfiable` again and you'll see something you have already met three times. `DoNotSchedule` is a hard rule; `ScheduleAnyway` is a soft one. Required and preferred, once more, under new names. That's the fourth appearance of the same pair in this chapter, and §7 is about to tell you why.

🔖 **Snag:** "Spread my replicas evenly across nodes" and "never put two of these together" are different requirements. Anti-affinity states the second and only approximates the first. If you find yourself writing anti-affinity rules and then reasoning about how many replicas you're allowed to have, you wanted spread constraints.

One honest limitation, because it's the kind of thing that bites in production and it's cheap to know now: *"There's no guarantee that the constraints remain satisfied when Pods are removed. For example, scaling down a Deployment may result in imbalanced Pods distribution"* [source: k8s-docs-topology-spread-constraints-2026-08-24]. These are scheduling-time constraints. Like everything else in this chapter, they describe a decision, not a standing invariant.

Distribution matters downstream too: a Service's backends being on distinct nodes is what makes the Service resilient rather than merely load-balanced [*cross-bearing: see Ch 9 — Services and endpoints*].

§6 — Overruling the Scheduler, and Replacing It

Two escape hatches at two different altitudes, held together by one frame. Everything so far has been about *influencing* a decision the scheduler makes. This section is about not letting it make the decision at all.

nodeName — the Pod-level hatch

`nodeName` is a more direct form of node selection than affinity or `nodeSelector`. It's a field in the Pod spec, and **if it is not empty, the scheduler ignores the Pod** and the kubelet on the named node tries to place the Pod on that node [source: k8s-docs-assign-pod-node-2026-08-23]. Using `nodeName` **overrules** using `nodeSelector` or affinity and anti-affinity rules [source: k8s-docs-assign-pod-node-depth-2026-08-24].

Overrules — not "takes precedence within the same evaluation." The scheduler doesn't run. Nothing you wrote in §2 through §5 is consulted, because the component that would have consulted it was skipped.

Which produces the one failure mode in this chapter that isn't Pending [source: k8s-docs-assign-pod-node-depth-2026-08-24]:

- If the named node does not exist, the Pod will not run, and in some cases may be automatically deleted.
- If the named node does not have the resources to accommodate the Pod, the Pod will fail and its reason will indicate why, for example OutOfmemory or OutOfcpu.
- Node names in cloud environments are not always predictable or stable.

Every other placement failure in this chapter leaves a Pod waiting patiently for conditions to improve. This one fails outright, because the feasibility check that would have caught the problem in advance never happened. You took responsibility for it, and nobody is going to tell you when you get it wrong.

So the framing matters: `nodeName` is **not the most forceful way of asking**. It is the absence of asking — mooring where you like, and telling no one. The API does let you specify a node for a Pod when you create it, *"but this is unusual and is only done in special cases"* [source: k8s-docs-kube-scheduler-2026-08-23], which, coming from reference documentation, is a stronger discouragement than it looks.

The case that makes binding concrete

Here's the thing that reframes the whole section, and it comes from a resource you already know.

The DaemonSet controller does not use `nodeName` to place its Pods. It creates a Pod for each eligible node and adds the `spec.affinity.nodeAffinity` field of the Pod to match the target host. After the Pod is created, **the default scheduler typically takes over and then binds the Pod to the target host by setting the `.spec.nodeName` field** [source: k8s-docs-daemonset-2026-08-24].

Read that last clause again. Binding is writing `.spec.nodeName`. That's the whole physical content of the operation the scheduler performs in step three. It tells the API server which node, the API server records it in that field, and the kubelet on that node notices its own name.

So `nodeName` is not a special user-facing shortcut at all. It is **the field that binding writes to**, and setting it by hand means filling in the scheduler's answer before it was asked the question.

🔒 **Worth Securing:** `nodeName` is the scheduler's output, not a separate API. When you set it yourself you aren't overriding a decision — you're pre-writing it, and skipping every check that would have validated it. That's why the failure is immediate rather than patient.

`schedulerName` — the cluster-level hatch

`kube-scheduler` is designed so that, if you want and need to, **you can write your own scheduling component and use that instead** [source: k8s-docs-kube-scheduler-2026-08-23]. Pods can name which

scheduler should handle them; a DaemonSet, for example, exposes `.spec.template.spec.schedulerName` for exactly this purpose [source: k8s-docs-daemonset-2026-08-24].

You do not need to know how to build a scheduler. You need to know that the seat is pluggable: the default scheduler is a default, not a fixture. That fact gets collected with several of its siblings much later [cross-bearing: see Ch 17 — the cluster's extension points].

The vocabulary you'll meet in older material

There are two documented ways to configure the filtering and scoring behavior of the scheduler [source: k8s-docs-kube-scheduler-2026-08-23]:

- **Scheduling Policies** — **Predicates** for filtering, and **Priorities** for scoring.
- **Scheduling Profiles** — **plugins** that implement different scheduling stages, including QueueSort, Filter, Score, Bind, Reserve and Permit. kube-scheduler can run different profiles.

Treat this as a **vocabulary mapping onto §1's spine**, not as two configuration systems you might choose between. Predicates are filtering under an older name. Priorities are scoring under an older name. The profile plugin stages are the same pipeline with more seats exposed, and two of those seat names, Filter and Score, are just the steps you already know.

Older name	What it is
Predicate	A filter — decides whether a node is feasible
Priority	A score — ranks the nodes that survived filtering

If you read an older blog post that says "the PodFitsResources predicate," you now know that's a filter, and you can carry on reading. That's what this material is worth on this exam.

☆ Fixed Point:

`nodeName` **bypasses the scheduler entirely. It overrules `nodeSelector` and every affinity rule, and because it skips the feasibility check, a Pod that doesn't fit *fails* rather than waiting in Pending. Predicates are filters; Priorities are scores.**

Where profile configuration actually lives — which is to say, in the control plane's own component configuration — is a question about running a cluster rather than using one [cross-bearing: see Ch 8 — cluster administration].

🕒 Taking Your Bearings #3 — Relative Placement, and Opting Out

Five questions on §5 and §6, with one that needs three sections at once.

1. Your three replicas all land on one node. Nothing in their spec is wrong. Explain why the scheduler was entitled to do that, and name the kind of rule that would have prevented it.
2.  An inter-Pod anti-affinity rule uses a topologyKey of `topology.kubernetes.io/zone` rather than `kubernetes.io/hostname`. What changes about where the Pods may land?
3. A Pod spec sets both a `nodeSelector` and a `nodeName`. Which wins, and what happens if the named node cannot fit the Pod?
 - A) The `nodeSelector` wins; `nodeName` is only a hint.
 - B) `nodeName` wins and the scheduler is bypassed; if the node can't fit the Pod, the Pod stays Pending.
 - C) `nodeName` wins and the scheduler is bypassed; if the node can't fit the Pod, the Pod fails.
 - D) The API rejects the Pod, because the two fields conflict.
4. Map the older Predicates-and-Priorities vocabulary onto the two steps you learned in §1.
5.  A DaemonSet's Pod is running on a node that is under memory pressure and is refusing all other work. Explain how it got there, and name the component that placed it.

Answers with Explanations:

1 — Because nothing in the spec expressed a relationship between the three Pods; inter-Pod anti-affinity (or a topology spread constraint) is the kind of rule that would have. Three replicas from one Deployment are identical: same requests, same labels, same node constraints. Nothing in their specs distinguishes one node from another, and nothing expresses a relationship between the three, so the scheduler placing all three in one place is a legal outcome of a decision made three times independently. Inter-pod anti-affinity constrains Pods against labels on other Pods [source: k8s-docs-assign-pod-node-2026-08-23], which is the only kind of rule in this chapter that can say anything about a *set*.

2 — The exclusion now applies per zone rather than per node, which is strictly stronger. Nodes with the same value for the `topologyKey` label are considered to be in the same topology [source: k8s-docs-topology-spread-constraints-2026-08-24]. With `kubernetes.io/hostname`, each node is its own domain, so two Pods may not share a node but two Pods in different nodes of the same zone are fine. With `topology.kubernetes.io/zone`, each zone is one domain, so two Pods may not share a *zone* at all. In a two-zone cluster, a required anti-affinity rule on the zone key caps you at two placeable Pods regardless of how many nodes you own; the rest sit Pending. The domain is the variable, and changing it changes the answer.

3 — C. Using `nodeName` overrides using `nodeSelector` or affinity and anti-affinity rules [source: k8s-docs-assign-pod-node-depth-2026-08-24], and if the `nodeName` field is not empty the scheduler ignores the Pod [source: k8s-docs-assign-pod-node-2026-08-23]. If the named node does not have the resources to

accommodate the Pod, the Pod **fails**, with a reason such as `OutOfMemory` or `OutOfCPU` [source: k8s-docs-assign-pod-node-depth-2026-08-24].

- **A is wrong:** `nodeName` is not a hint, it's a bypass.
- **B is the good distractor**, because `Pending` is the correct answer for every *other* failure in this chapter. It's wrong here precisely because nothing ran the feasibility check — there is no scheduler waiting for conditions to improve.
- **D is wrong:** the API accepts the Pod. Nothing about the combination is invalid; `nodeSelector` simply never gets read.

4 — Predicates are filtering; Priorities are scoring. Scheduling Policies configure the scheduler with Predicates for filtering and Priorities for scoring [source: k8s-docs-kube-scheduler-2026-08-23]. Same two steps, older names. The Profiles model exposes the same stages as plugin extension points, two of which are literally called `Filter` and `Score` [source: k8s-docs-kube-scheduler-2026-08-23].

5 — The DaemonSet controller set node affinity for that node and added tolerations for the built-in condition taints; the ordinary default scheduler then typically placed it. The `DaemonSet` controller creates a Pod for each eligible node and adds `spec.affinity.nodeAffinity` matching the target host; after the Pod is created, the default scheduler typically takes over and binds the Pod to the target host by setting `.spec.nodeName` [source: k8s-docs-daemonset-2026-08-24]. Separately, the controller automatically adds tolerations for the built-in node-condition taints, including `node.kubernetes.io/memory-pressure:NoSchedule` [source: k8s-docs-daemonset-2026-08-24].

The point worth taking from this: nothing special happened. This is the ordinary scheduler doing its ordinary job, on a Pod that was constructed to be feasible on exactly one node and to tolerate what that node is complaining about. There's no privileged path and no bypass. And the binding step you learned as a vocabulary word in §1 turns out to be a field write you can point at.

Checkpoint: You've Now Mastered

✓ Why identical replicas are entitled to land in identical places ✓ That the topology domain is a variable, and what changing it costs ✓ What `nodeName` actually is, and the one failure that isn't `Pending` ✓
Predicates → filter, Priorities → score

□ The one thing you actually have to remember (next, and it's short)

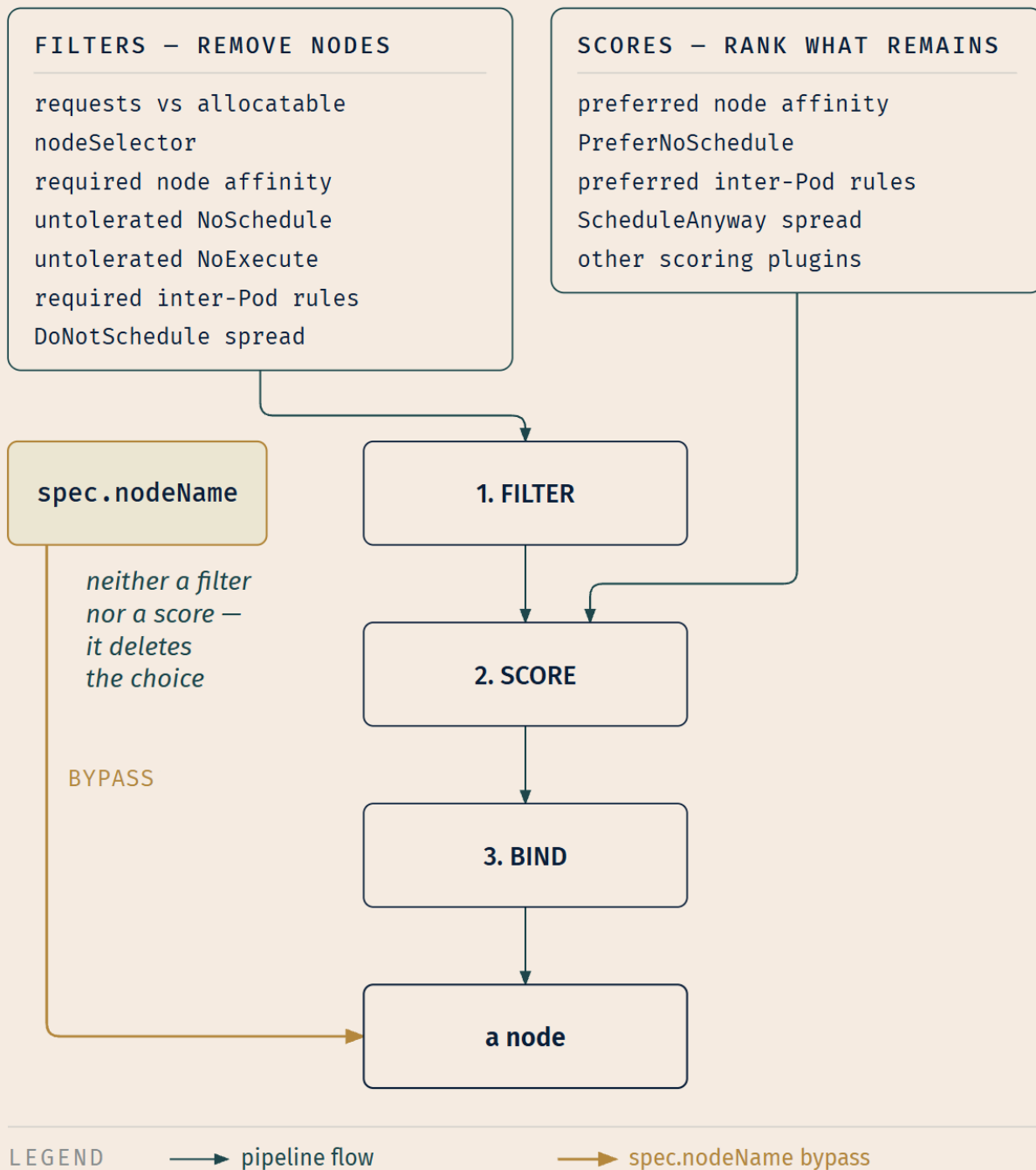
☀ §7 — Everything Is a Filter or a Score

You've now learned six vocabularies for influencing placement. They have six different syntaxes, they live in different places in the spec, half of them are on Pods and half on nodes, and one of them has a field name forty-six characters long. They look like six separate systems.

They are not.

☀ Zenith

Every mechanism in this chapter plugs into one of exactly two slots in \$1's pipeline. It either removes nodes from consideration — a filter — or it changes the ranking of the nodes that survived — a score. Six vocabularies. Two slots. That's the chapter.



Same three stages you saw on the first page of this chapter. Nothing new has been added to the pipeline. Everything you learned in §2 through §5 contributed to step one or step two, and the *required / preferred* distinction that kept recurring under four different names — *required vs preferred node affinity*, *NoSchedule VS PreferNoSchedule*, *required vs preferred inter-Pod rules*, *DoNotSchedule vs ScheduleAnyway* — was never four distinctions. It was always the same one: *is this a filter, or is this a score?*

Hard rules filter. Soft rules score. That is the whole taxonomy.

And there is exactly one thing in this chapter that fits neither slot, which is why it's drawn outside the pipeline. `nodeName` does not narrow the feasible set and it does not adjust a ranking. It removes the decision. That is precisely why it's dangerous, and precisely why the documentation calls it unusual [source: k8s-docs-kube-scheduler-2026-08-23].

🔒 **Worth Securing:** The diagnostic question for any placement problem you'll ever meet: *is this a filter that excluded every node, or a score that ranked the wrong one first?* The two have completely different symptoms. A filter problem leaves a Pod Pending forever; a score problem puts the Pod somewhere you didn't want, immediately and silently. Knowing which one you're looking at is most of the diagnosis.

One last thing to notice, and then we'll stop. The scheduler watches for newly created Pods that have no node assigned [source: k8s-docs-kube-scheduler-2026-08-23], compares what exists against what has been placed, and acts on the difference. That shape should be familiar by now. It's the same shape as every controller in Chapter 6. The scheduler is not an exception to the cluster's architecture; it's another instance of it, wearing a specialized hat. Keep that in your peripheral vision; the book comes back to it properly later.

Exam Alert! 🚨

High-Priority Topics

1. **Filter → score → bind**, in that order. Binding is a notification to the API server.
2. **The scheduler does not start the container.** The kubelet on the chosen node does.
3. **Equal scores break at random.**
4. **An unschedulable Pod stays Pending.** Not an error. Nothing times out.
5. **Filtering fits requests** against Allocatable — not limits, and a request stays booked whether or not it is used.
6. **A Pod is scheduled once and is never rescheduled.** It gets replaced, not moved.
7. **The three taint effects and their timing** — only NoExecute touches running Pods.
8. **A toleration permits; it does not attract.**
9. **`nodeName` bypasses the scheduler**, overrules `nodeSelector` and affinity, and fails rather than waiting.

Common Traps

You met the first of these in Chapter 3, where it was defused explicitly and you were told it would come back *[cross-bearing: see Ch 3 §2 — the control plane]*. Here's the piece Chapter 3 held back.

Trap	The correction
"The scheduler places the Pod on the node."	It notifies the API server. The kubelet on the chosen node is what starts containers.
"The scheduler picks the single best node, deterministically."	Ties are broken at random [source: k8s-docs-kube-scheduler-2026-08-23].
"An unschedulable Pod errors out."	It remains unscheduled until the scheduler is able to place it [source: k8s-docs-kube-scheduler-2026-08-23].
"A failed Pod is rescheduled to a healthy node."	A Pod is never rescheduled to a different node; it is replaced by a new Pod with a different UID [source: k8s-docs-pod-lifecycle-2026-08-23].
"A toleration schedules the Pod onto the tainted node."	Tolerations allow scheduling but don't guarantee it — the scheduler still evaluates every other parameter [source: k8s-docs-taints-tolerations-2026-08-23]. The most durable error in this material.
"NoSchedule evicts the Pods already running."	It doesn't. Pods currently running are not evicted [source: k8s-docs-taints-tolerations-2026-08-23]. Only NoExecute does.
"Node affinity moves a Pod when the node's labels change."	IgnoredDuringExecution — the Pod continues to run [source: k8s-docs-assign-pod-node-2026-08-23].
"PreferNoSchedule means never."	It's a preference; the control plane will try to avoid the node but it is not guaranteed [source: k8s-docs-taints-tolerations-2026-08-23].
"nodeSelector and affinity are two syntaxes for the same power."	Affinity adds soft rules and five operators nodeSelector cannot express [source: k8s-docs-assign-pod-node-2026-08-23].
"A Pod with nodeName set that doesn't fit will wait."	It fails, with a reason such as OutOfmemory [source: k8s-docs-assign-pod-node-depth-2026-08-24]. The one failure here that isn't Pending.

Practice Questions

Eighteen questions. Four reach back into earlier chapters, and four require two sections of this chapter at once. Answers and explanations follow the full set.

1. Which statement correctly describes what kube-scheduler does with an unbound Pod?

A) Filter, score, bind — and the scheduler then instructs the kubelet on the chosen node directly. B) Filter, score, bind — and the scheduler notifies the API server of its choice. C) Score, filter, bind — the nodes are

ranked first, then the unsuitable ones are dropped. D) Filter, bind, score — the Pod is bound, then ranked against the alternatives.

2. After filtering, four nodes remain and all four receive the same score. What happens?

A) The scheduler re-runs scoring with additional plugins until a winner emerges. B) The scheduler selects one of the four at random. C) The scheduler selects the node with the earliest creation timestamp. D) The Pod is marked unschedulable because the result is ambiguous.

3. A node reports 32 GiB of RAM, of which 30 GiB is allocatable. Three Pods run on it, having requested 8 GiB each; between them they are currently using 3 GiB. A new Pod requests 12 GiB of memory. Assuming memory is the only constraint, will the new Pod be filtered onto this node?

A) Yes — 30 GiB minus 3 GiB used leaves 27 GiB available. B) Yes — the scheduler over-subscribes memory by design. C) No — 24 GiB is already booked against 30 GiB allocatable, leaving 6 GiB, which is less than 12 GiB. D) No — the three Pods' *limits* sum to more than the node's allocatable memory, which blocks new placements.

4. A Pod has been sitting in `Pending` for six hours. Which statement is accurate?

A) The scheduler has failed and will emit an error after a configurable timeout. B) The Pod is retried with progressively relaxed constraints until it fits. C) The Pod is in a normal state; it remains unscheduled until the scheduler is able to place it. D) The controller that created the Pod will delete it and create a replacement.

5. [retrieval: ch5] Your Pod was placed on a modestly-specced node an hour ago. A much better node has since joined the cluster and is sitting nearly idle. What happens to the running Pod?

A) The scheduler re-evaluates placement periodically and migrates it. B) Nothing. It stays where it is for the rest of its life. C) The kubelet on the new node claims it. D) It is evicted and rescheduled at the next scoring cycle.

6. [retrieval: ch3] Which pair correctly assigns responsibility?

A) The scheduler records the placement decision with the API server; the kubelet on that node starts the containers. B) The scheduler starts the containers directly; the kubelet reports their status back to the control plane. C) The controller manager records the placement decision; the scheduler then starts the containers on that node. D) The scheduler selects the node and notifies that node's kubelet directly; the kubelet starts the containers.

7. A Pod's `nodeSelector` matches exactly one node in the cluster. That node does not have enough allocatable memory for the Pod's request. What happens?

A) The Pod schedules onto the next-best node, since `nodeSelector` is advisory rather than binding. B) The Pod fails immediately with `OutOfMemory`, because its placement was fully determined. C) The Pod stays

Pending — both the label filter and the resource filter must pass, and one of them doesn't. D) The scheduler evicts a Pod from the matched node to make room, since a `nodeSelector` was specified.

8. A Pod is running on a node it reached via `requiredDuringSchedulingIgnoredDuringExecution` node affinity on the label `disktype=ssd`. An operator removes that label from the node. What happens to the Pod?

A) It is evicted, because the rule was `required` and is enforced continuously. B) It continues to run, because the rule was a scheduling-time question only. C) It is evicted once `tolerationSeconds` elapses, as with a `NoExecute` taint. D) It moves to a different node that still carries the label.

9. A platform team taints eight GPU nodes with `dedicated=gpu:NoSchedule` and adds a matching toleration to the ML team's training Pods. The training Pods are landing on ordinary CPU nodes. Which single change makes them land on the GPU nodes?

A) Add a second toleration for the same taint, so that the match is unambiguous to the scheduler. B) Change the taint's effect to `NoExecute`, which is the stronger form and will pull the Pods across. C) Label the eight GPU nodes and give the training Pods a `nodeSelector` or a `required` node affinity for that label. D) Remove the taint, since the taint is what is currently keeping the training Pods off the GPU nodes.

10. Which of the scheduler's two steps does an *untoleration* `NoSchedule` taint participate in, and which does `PreferNoSchedule` participate in?

A) Both participate in filtering. B) `NoSchedule` filters; `PreferNoSchedule` scores. C) `NoSchedule` scores; `PreferNoSchedule` filters. D) Neither participates in either — taints are evaluated after binding.

11. A node that is already running Pods gains a new taint. Which statement about the running Pods is correct?

A) `NoSchedule` and `NoExecute` both evict non-tolerating Pods; `PreferNoSchedule` does not. B) All three effects evict non-tolerating Pods, differing only in speed. C) Only `NoExecute` evicts non-tolerating Pods; `NoSchedule` and `PreferNoSchedule` affect new placements only. D) None of the three affects running Pods; taints are a placement-time mechanism only.

12. [retrieval: ch4] Chapter 4 introduced set-based label selectors with operators such as `in`, `notin` and `exists`. Which statement about node affinity's operators is correct?

A) Node affinity supports only equality, exactly like `nodeSelector`. B) Node affinity supports `In`, `NotIn`, `Exists`, `DoesNotExist`, `Gt` and `Lt`. C) Node affinity supports the same implicit equality as `nodeSelector`, plus `Gt` and `Lt` for numeric comparison. D) Node affinity has no operators; its expressiveness comes solely from the soft/hard distinction.

13. A node carries a `NoSchedule` taint. A new Pod has no toleration for it, but does carry a `required` inter-Pod affinity rule saying it must be co-located with a Pod that happens to be running on that node. What happens?

A) The Pod schedules there, because pod affinity is evaluated after taints and therefore overrides them. B) The Pod does not schedule there — the untolerated taint removes the node during filtering, and pod affinity cannot restore a node that has been filtered out. C) The Pod schedules there, because a required pod-affinity rule is a filter and a NoSchedule taint only affects scoring. D) The Pod schedules on a neighboring node in the same topology domain, which satisfies the co-location rule.

14. A team's inter-Pod anti-affinity rule uses `topologyKey: kubernetes.io/hostname`. They change it to `topology.kubernetes.io/zone` in a cluster with two zones and twelve nodes. What is the effect on a Deployment with six replicas carrying the matching label?

A) No change — the rule already spread the Pods across nodes. B) The rule becomes weaker; more Pods can share a zone. C) The rule becomes stricter; with a required rule and two domains, at most two replicas can be placed and the rest stay Pending. D) The rule becomes stricter, and the scheduler places the extra replicas on whichever nodes minimize the skew rather than leaving them Pending.

15. [retrieval: ch6] A DaemonSet achieves perfect one-per-node distribution without any anti-affinity rule or topology spread constraint. Why doesn't it need one?

A) The DaemonSet controller silently injects anti-affinity into every Pod it creates. B) Distribution is the resource's definition — the controller creates one Pod per eligible node, so there is no set-level constraint for the scheduler to enforce. C) DaemonSet Pods bypass the scheduler entirely via `nodeName`. D) The scheduler applies a built-in default spread constraint to all DaemonSet Pods.

16. A Pod spec sets `nodeSelector: {disktype: ssd}`, a required node affinity for zone `zone-a`, and `nodeName: worker-07`. `worker-07` has neither an SSD label nor a `zone-a` label, and lacks the memory for the Pod. What happens?

A) The API server rejects the Pod for contradictory placement fields. B) The scheduler resolves the conflict in favor of the most specific constraint and places the Pod in `zone-a`. C) The scheduler ignores the Pod entirely; the kubelet on `worker-07` tries to place it and the Pod fails, with a reason such as `OutOfmemory`. D) The Pod stays Pending until an SSD node in `zone-a` named `worker-07` appears.

17. You read an older article describing "the PodFitsResources predicate" and "the LeastRequestedPriority priority." Onto which parts of the modern two-step operation do those map?

A) Predicates map to binding; priorities map to filtering. B) Predicates map to filtering; priorities map to scoring. C) Both map to scoring; filtering had no configurable component. D) Neither maps onto the modern operation; they describe a different algorithm.

18. A Pod carries a toleration with `key: dedicated`, `operator: Exists`, and no effect field. Which of a node's taints does that single toleration cover?

A) Only `dedicated` taints carrying the `NoSchedule` effect, since that is the default effect. B) Every taint on the node whose key is `dedicated`, whatever its value and whatever its effect. C) Every taint on the node

regardless of key, because `Exists` with no value is the universal wildcard. **D**) `None` — a toleration that omits effect is incomplete and is rejected when the Pod is created.

Answers with Explanations:

1 — B. Filtering finds the feasible set, scoring ranks it, and the scheduler then notifies the API server in a process called binding [source: [k8s-docs-kube-scheduler-2026-08-23](#)]. **A** gets the sequence right and the actor wrong: the scheduler talks to the API server, not to nodes, and the kubelet acts on a decision it *observed* rather than one it was told. **C** inverts the two steps — scoring runs on the nodes that survived filtering, so it cannot precede it. **D** puts binding before the decision has been made, which is incoherent: binding is the decision being recorded.

2 — B. *"If there is more than one node with equal scores, kube-scheduler selects one of these at random"* [source: [k8s-docs-kube-scheduler-2026-08-23](#)]. **A** invents a re-scoring loop that doesn't exist; nothing escalates a tie. **C** is the tidy deterministic answer a reasonable engineer would design, and it's wrong, which is what makes it a good distractor. **D** confuses a tie (several nodes are acceptable) with an empty feasible set (none are).

3 — C. The kubelet reserves at least the request amount of a resource for the container [source: [k8s-docs-resource-management-2026-08-23](#)], and the scheduler does not over-subscribe Allocatable [source: [k8s-docs-node-allocatable-2026-08-24](#)] — where Allocatable, not total RAM, is what the scheduler treats as available capacity for Pods [source: [k8s-docs-node-allocatable-2026-08-24](#)]. Three Pods × 8 GiB = 24 GiB booked; 30 - 24 = 6 GiB left; 12 > 6. **A** is the trap: it does the arithmetic against *usage*, and it also uses the wrong baseline. The reservation stands whether or not the container touches the memory [source: [k8s-docs-resource-management-2026-08-23](#)], so used-versus-booked is not the subtraction the filter performs. **B** contradicts the source directly — the scheduler does not over-subscribe Allocatable. **D** puts the limit in the scheduler's hands; limits are enforced by the kubelet on the running container, after placement [source: [k8s-docs-resource-management-2026-08-23](#)], and never enter the filter.

4 — C. If none of the nodes are suitable, the Pod remains unscheduled until the scheduler is able to place it [source: [k8s-docs-kube-scheduler-2026-08-23](#)], and `Pending` explicitly covers time spent waiting to be scheduled [source: [k8s-docs-pod-lifecycle-2026-08-23](#)]. **A** invents a timeout; nothing in the documented behavior converts waiting into failure. **B** invents constraint relaxation — nothing rewrites your spec on your behalf. **D** describes a controller behavior that isn't triggered by a Pod being unplaceable; the Pod exists, so the controller's count is already satisfied.

5 — B. Pods are scheduled once in their lifetime to a specific node, and a Pod is never "rescheduled" to a different node [source: [k8s-docs-pod-lifecycle-2026-08-23](#)]. This is the same rule you learned in Chapter 5 with a *node-failure* motivation; here it applies to a strictly better opportunity, and the answer is identical. Kubernetes does not optimize placement continuously. **A** and **D** both describe a rebalancing loop the platform does not have — nothing revisits a placement after binding. **C** has the wrong actor and the wrong direction: a kubelet never claims Pods. It acts on Pods the API server has already recorded as bound to its own node.

6 — A. The scheduler selects a node and notifies the API server [source: k8s-docs-kube-scheduler-2026-08-23]; the kubelet is an agent on each node that makes sure containers described in the PodSpecs are running [source: k8s-docs-cluster-architecture-2026-08-23]. **B** puts container-starting in the scheduler's hands, which is the most common misconception about this component and the reason `ch07-fig01` draws two separate arrows. **C** compounds that error with a second one: the controller manager runs controllers, not placement bookkeeping. **D** concedes that the kubelet starts containers and still gets the topology wrong — the scheduler does not talk to nodes at all. Its arrow lands on the API server, and the kubelet reads from there.

7 — C. Both are filters. `nodeSelector` restricts the Pod to nodes carrying each specified label [source: k8s-docs-assign-pod-node-2026-08-23]; `PodFitsResources` checks whether a candidate node has enough available resources for the Pod's requests [source: k8s-docs-kube-scheduler-2026-08-23]. The single label-matching node fails the resource filter, the feasible list is empty, and the Pod isn't yet schedulable [source: k8s-docs-kube-scheduler-2026-08-23]. **A** treats `nodeSelector` as soft — it isn't; Kubernetes only schedules the Pod onto nodes carrying *each* label. **B** is the `nodeName` failure mode borrowed into the wrong question: `nodeName` skips the feasibility check, but `nodeSelector` doesn't, so this Pod waits. **D** invents preemption-on-demand triggered by a selector; preemption is a priority mechanism, not a selector one.

8 — B. `IgnoredDuringExecution` means that if the node labels change after Kubernetes schedules the Pod, the Pod continues to run [source: k8s-docs-assign-pod-node-2026-08-23]. **A** reads only the first half of the field name; `required` governs the scheduler, not a continuous enforcement loop. **C** confuses affinity with tolerations — `tolerationSeconds` belongs to `NoExecute` taints and has no meaning for node affinity. **D** contradicts the once-only rule: a Pod is never rescheduled to a different node [source: k8s-docs-pod-lifecycle-2026-08-23].

9 — C. *"Tolerations allow scheduling but don't guarantee scheduling"* [source: k8s-docs-taints-tolerations-2026-08-23], and the documented dedicated-node pattern requires *both* a taint and a matching label plus node affinity if the Pods must only use the dedicated nodes [source: k8s-docs-taints-tolerations-depth-2026-08-24]. **A** treats the problem as a matching failure, but the toleration is already matching correctly; a duplicate changes nothing, because a toleration's only power is to remove a veto. **B** confuses strength of refusal with attraction: `NoExecute` would additionally evict every non-tolerating Pod already on those nodes [source: k8s-docs-taints-tolerations-2026-08-23], and the training Pods would still land wherever ordinary scoring puts them. **D** deletes the half of the setup that was working — the exclusion — and still supplies no pull, so the GPU nodes would fill with everyone else's work and the training Pods would be no likelier to be there.

10 — B. An untolerated `NoSchedule` taint means no new Pods will be scheduled on the node [source: k8s-docs-taints-tolerations-2026-08-23] — the node is taken out of consideration entirely, which is what filtering does. `PreferNoSchedule` means the control plane will *try* to avoid the node but it is not guaranteed [source: k8s-docs-taints-tolerations-2026-08-23] — the node stays in the running with a worse standing, which is what scoring does. **A** makes the soft effect hard, which would mean a `PreferNoSchedule` node could never take a non-tolerating Pod; the source says otherwise. **C** inverts both,

and would make `NoSchedule` merely a discouragement. **D** is the answer of someone who thinks a toleration raises a node's score: taints are weighed before binding, not after, because binding is the recording of a decision already made.

11 — C. `NoSchedule` does not evict Pods currently running on the node; `PreferNoSchedule` is its soft form; `NoExecute` evicts non-tolerating Pods immediately [source: [k8s-docs-taints-tolerations-2026-08-23](#)]. **A** lets `NoSchedule` evict, which is the classic error, and the word *Schedule* in its name is the clue against it. **B** flattens all three into a single behavior differing only in timing, which erases the one distinction the effects exist to draw. **D** overcorrects: `NoExecute` genuinely does reach Pods already running, and that's precisely what separates it from the other two.

12 — B. Node affinity supports `In`, `NotIn`, `Exists`, `DoesNotExist`, `Gt` and `Lt` [source: [k8s-docs-assign-pod-node-2026-08-23](#)], a richer vocabulary than `nodeSelector`'s implicit equality. **A** describes `nodeSelector`, not affinity, and misses both of affinity's additions. **C** is the half-remembered version: `Gt` and `Lt` are indeed the unusual pair (they're the ones unavailable to `podAffinity` [source: [k8s-docs-assign-pod-node-depth-2026-08-24](#)]), but affinity adds four more besides — the set-membership and existence operators are the ones you'll reach for daily. **D** drops the operator set entirely; expressiveness and hardness are two *independent* additions, and this option keeps only one of them.

13 — B. An untoleraed `NoSchedule` taint marks the node as not accepting Pods that do not tolerate it [source: [k8s-docs-taints-tolerations-2026-08-23](#)], so the node is out of consideration before any ranking begins. Pod affinity is also a feasibility question — it can only ever *narrow* the surviving set, never restore a node that has already been excluded. **A** invents an evaluation ordering in which a later rule can readmit a node an earlier one rejected; nothing works that way, and filters compose by intersection, not by precedence. **C** gets the direction of both mechanisms wrong at once: it correctly files pod affinity as a filter but misfiles `NoSchedule` as a mere ranking penalty, when an untoleraed `NoSchedule` node will not accept the Pod at all. **D** sounds plausible and isn't: a *required* co-location rule constrains the Pod to the topology domain containing the target Pod, and with a hostname-scoped domain that means the target's node specifically, not its neighborhood.

14 — C. Nodes with identical values for the `topologyKey` label are in the same topology domain [source: [k8s-docs-topology-spread-constraints-2026-08-24](#)], and `required-mode` pod anti-affinity means only a single Pod can be scheduled into a single topology domain [source: [k8s-docs-topology-spread-constraints-2026-08-24](#)]. Twelve nodes gave twelve domains; two zones give two. Four of the six replicas have nowhere legal to go. **A** misses that the domain itself changed — the rule text is identical, which is exactly the trap. **B** has the direction backwards: fewer, larger domains is stricter, not looser. **D** borrows the behavior of a *topology spread constraint* set to `ScheduleAnyway`, which does tell the scheduler to place the Pod anyway while prioritizing nodes that minimize the skew [source: [k8s-docs-topology-spread-constraints-2026-08-24](#)]. A required anti-affinity rule has no such soft mode — the preferred form exists, but choosing it means you lose the ability to enforce the constraint [source: [k8s-docs-topology-spread-constraints-2026-08-24](#)], which is a different thing from placing to minimize skew.

15 — B. The `DaemonSet` controller creates a Pod for each eligible node and sets that Pod's node affinity to match the target host [source: [k8s-docs-daemonset-2026-08-24](#)]. Each Pod is constructed for one

specific node, so there is no set-level relationship left for a scheduling constraint to enforce. **A** describes a mechanism the controller doesn't use — it uses node *affinity* per Pod, not anti-affinity across the set. **C** is wrong: the default scheduler typically takes over and binds the Pod [source: k8s-docs-daemonset-2026-08-24], so the ordinary pipeline runs. **D** invents a built-in default that isn't documented, and would have to apply to every DaemonSet in every cluster. *This is a discrimination question as much as a retrieval one: "the Pods end up spread out" and "a spreading constraint was enforced" are not the same claim.*

16 — C. If the `nodeName` field is not empty, the scheduler ignores the Pod and the kubelet on the named node tries to place it [source: k8s-docs-assign-pod-node-2026-08-23]; `nodeName` overrules `nodeSelector` and affinity and anti-affinity rules [source: k8s-docs-assign-pod-node-depth-2026-08-24]; and if the named node lacks the resources, the Pod fails with a reason such as `OutOfMemory` [source: k8s-docs-assign-pod-node-depth-2026-08-24]. **A** invents API validation that doesn't happen — the fields are not checked against one another. **B** invents conflict resolution, but there is no conflict to resolve, because the component that would have read the other two fields never ran. **D** is the trap, because `Pending` is the right answer everywhere else in this chapter. Here nothing performed the feasibility check, so there is nothing waiting for conditions to improve.

17 — B. Scheduling Policies configure the scheduler with Predicates for filtering and Priorities for scoring [source: k8s-docs-kube-scheduler-2026-08-23]. The names changed; the two steps didn't. **A** inverts the mapping, which would put a *fits-resources* test in the ranking stage. **C** is wrong on the specific point that filtering had no configurable component: Predicates were exactly that component. **D** denies a correspondence the source states directly in the same sentence that names both terms.

18 — B. A toleration matches a taint when the keys are the same and the effects are the same, with the operator condition satisfied — and two wildcards modify that. An **empty effect matches all effects with the given key** [source: k8s-docs-taints-tolerations-2026-08-23]. So this toleration covers dedicated taints at `NoSchedule`, `PreferNoSchedule` and `NoExecute` alike, and `Exists` with no value means the taint's value is irrelevant too. **A** reads the missing effect as a default rather than as a wildcard; there is no "default effect" in the matching rules, only a wildcard. **C** confuses the two wildcards. The all-keys wildcard requires an **empty key**, and in that case the operator must be `Exists` [source: k8s-docs-taints-tolerations-2026-08-23]; here the key is dedicated, so taints with other keys are untouched — and untouched matters, because you start with all of a node's taints, ignore the ones with a matching toleration, and the remainder still have their effects on the Pod [source: k8s-docs-taints-tolerations-depth-2026-08-24]. **D** invents a validation rule; the empty effect is a documented wildcard, not an omission the API rejects.

Chapter Summary

Concept	Remember This
The two-step operation	Filter, then score. Filtering produces the feasible set; scoring ranks it.
Scheduling factors	Resource requirements, hardware / software / policy constraints, affinity and anti-affinity, data locality, inter-workload interference, deadlines [source: k8s-docs-kube-scheduler-2026-08-23] [source: k8s-docs-cluster-architecture-2026-08-23].
Binding	A notification to the API server [source: k8s-docs-kube-scheduler-2026-08-23] — concretely, writing <code>.spec.nodeName</code> [source: k8s-docs-daemonset-2026-08-24]. Not an act of starting anything.
Who starts the container	The kubelet on the chosen node. Never the scheduler.
Tie-break	Equal scores are broken at random . Not deterministically, not by load.
Feasibility	Requests fitted against Allocatable. A request stays booked whether or not it is used — and limits never enter the filter.
Pending	A state, not an error. Nothing times out, nothing retries with looser rules, nothing gives up.
Scheduled once	A Pod is never rescheduled to a different node. It is replaced, with a new UID.
Node labels	Same selector mechanism as Chapter 4, pointed the other way: the Pod selects nodes.
nodeSelector	An AND of exact label matches. Blunt, readable, hard.
Node affinity	Same idea, richer operators (In, NotIn, Exists, DoesNotExist, Gt, Lt), plus a soft mode.
IgnoredDuringExecution	A node's labels changing after binding does not move the Pod.
Taints	Live on the node . A refusal. Three effects.
Tolerations	Live on the Pod . An exemption from the refusal. They permit; they never attract.
Effect timing	Only NoExecute touches Pods that are already running.
tolerationSeconds	Only meaningful with NoExecute. Kubernetes sets it to 300 for not-ready and unreachable unless you override it.
Dedicated nodes	Taint to exclude, label + affinity to attract. Both. Always.
Inter-Pod rules	Constrain a Pod against the labels of Pods already placed, within a topology domain.

Concept	Remember This
<code>topologyKey</code>	The node label whose values define the domain boundaries. It's a variable, not "the node."
Topology spread	The purpose-built mechanism for even distribution. <code>DoNotSchedule</code> filters; <code>ScheduleAnyway</code> scores.
<code>nodeName</code>	Bypasses the scheduler. Overrides everything. Fails instead of waiting.
Predicates / Priorities	Older names for filtering and scoring.
The whole chapter	Every mechanism is a filter or a score. <code>nodeName</code> is neither — it deletes the choice.

The Voyage Ahead

You can now say where a Pod should go, where it must not go, and what it should be near. What you cannot yet say is anything about the machine itself.

You have no way to take a node out of service before you reboot it. No way to stop one team booking the entire cluster's memory with requests they'll never use. No idea which versions of which components are allowed to disagree with each other, or what happens when they do. And no vocabulary for the three gates every request passes through on its way into the API server — which is the same API server this whole chapter has been quietly writing to.

There's a clue already in your hands. In §4 you met a built-in taint called `node.kubernetes.io/unschedulable`, sitting in the family table with a `NoSchedule` effect, and it wasn't put there by a failing disk or an exhausted process table. Marking a node unschedulable is a deliberate administrative act: it stops new Pods arriving without disturbing the ones already there, and it is the preparatory step before a reboot or other maintenance [source: k8s-docs-nodes-2026-08-23]. The command that does it, the command that clears the node out afterwards, and everything else in the `kubectl` surface that a cluster administrator actually uses — that's Chapter 8, and it's the last chapter of Part II.

Chapter 8 is where the rules you've been learning turn into consequences.

Safe Harbor

You have finished the hardest five points in Part II. Scheduling is easy to meet as a catalog of six unrelated features. You have it as one pipeline with two slots.

The next time you see a Pod stuck in `Pending`, you will not wonder whether something is broken. You will ask which filter emptied the list, and that is the question a practitioner asks.

📖 → 🌐 → 📡 · **Part II • Chapter 7.** One chapter left before Part III.

"You cannot move a berth once it is assigned. You can only be careful about what you said before it was."

Chapter 8: Standing the Watch

"The commands you'll actually type, and the versions that bite"

Domain: Kubernetes Fundamentals — 44% of the exam [source: cncf-kcna-curriculum-pdf-2026-08-23] •

Competency: Cluster Administration — ~5% (authored allocation) • Complexity: Mixed • Novelty: Moderate

The 44% is CNCF's published weight for one of its four domains [source: cncf-kcna-certification-page-2026-08-23]. The ~5% is this book's own allocation across the competencies inside that domain: the published curriculum carries a percentage against each of the four domains and none against the competencies listed within them [source: cncf-kcna-curriculum-pdf-2026-08-23]. The front matter explains how these allocations were derived.

Attention Budget

Total time: ~150 minutes | Recommended: split across two sessions, breaking after ☹ Taking Your Bearings #2

Section	Time	Attention Cost	Best Time to Study
\$1 — The Grammar of a Command	12 min	Low	Anytime
\$2 — Three Gates and a Logbook	22 min	High	Peak attention
\$3 — Dividing a Shared Cluster	16 min	Medium	When alert
☹ Taking Your Bearings #1	8 min	Medium	After a brief break
\$4 — Taking a Node Out of Service	18 min	Medium	When alert
\$5 — Who Owns the Control Plane	10 min	Low	Anytime
☹ Taking Your Bearings #2	8 min	Medium	After a brief break
\$6 — Versions That Are Allowed to Disagree	20 min	High	Peak attention — start of a fresh session
\$7 — The One Backup That Matters	10 min	Medium	When alert
☹ Taking Your Bearings #3	8 min	Medium	After a brief break
\$8 — Rules, or Consequences	8 min	Low	Anytime
Practice Questions	25 min	Medium	When alert

Attention Cost Key:

- **Low:** concrete, familiar concepts — study anytime

- **Medium:** new concepts requiring focus — study when alert
- **High:** abstract or dense material — study at peak attention

If you only have 15 minutes: read §2's gate sequence and §6's skew table, then work ☹ Taking Your Bearings #3. That is where this chapter's exam points actually live. §1, §4 and §5 are recognition material you will mostly get right on instinct.

On the session split: the recommended break after Bearings #2 puts the request path and the node in one session, and versions, disaster and synthesis in the other. It also isolates §6, the densest pure-recall block in this book, at the start of a fresh session, which is where it belongs.

"A watch is not a task you finish. It is a stretch of time during which the ship is your responsibility, and the log records what you did about it." — Lodestar Ledgers

☹ Soundings

Before reading this chapter, try these eight questions. Your score determines how to approach the content; there is no shame in any score, only different reading strategies. Several of these ask you to reason rather than recall. Answer in your own words. One sentence is enough.

1. You have a command-line tool on your laptop that manages a server somewhere else. Before it can do anything at all, what does it need to know, and what changes when you become responsible for two of those servers instead of one?
2. Chapter 3 was emphatic that one component is the only way into a Kubernetes cluster — nothing else is designed to expose a remote service. Name that component, and say what that architecture implies about where you would put a security check.
3. A server receives a request to do something expensive. List the distinct questions it must answer before it does the work. Then say whether any of those questions could result in the request being *changed*, rather than simply accepted or refused.
4. Chapter 4 said namespaces are intended for environments with many users spread across teams or projects, and it named the mechanism by which cluster resources get divided between them. What was that mechanism called, and what do you think it constrains?
5. Chapter 7's built-in taint table included `node.kubernetes.io/unschedulable`, with a `NoSchedule` effect: a taint applied deliberately by an operator rather than automatically by a failing disk. Using Chapter 7's rule about that effect, what did it do to the Pods *already running* on the node?
6. Chapter 4 pointed at the `kube-node-lease` namespace and said the Lease objects in it are how the control plane knows a node is still alive. If those Leases stop being renewed, what *should* the

control plane conclude, and what should it be careful not to conclude?

7. Two components of one system are running at different versions. One is a client, one is a server. Which direction of mismatch worries you more — a newer client talking to an older server, or an older client talking to a newer server — and why?
8. You run a service on a cloud provider's managed platform. A colleague runs the same service on hardware in a rack they own. Name one operational duty that sits with whoever operates the control plane in each case.

Answers + reading strategy

1. **An address and a credential, at minimum.** With two servers, *which* server becomes a piece of state you have to carry somewhere, which means the tool needs a notion of *current* target as well as *possible* targets.
2. **The kube-apiserver.** If every request in the cluster must pass through one component, that component is the natural and sufficient place to put access control. One door means one set of locks.
3. **Score this one correct if you produced three or more distinct questions, or if you answered "yes" to the "changed" clause.** Most readers produce two — *who are you*, and *are you allowed* — and say no. If that was you, score it incorrect and read §2 slowly: the third question is the one this chapter exists to give you, and the "changed" clause is the property that makes it a different kind of check rather than a third variation on "no."
4. **Resource quota.** Most readers will say it constrains "how much a team can use," which is right as far as it goes. Hold on to what you guessed; §3 will sharpen it.
5. **Nothing.** NoSchedule means no new Pods will be scheduled on the tainted node unless they have a matching toleration, and Pods currently running on the node are not evicted [source: k8s-docs-taints-tolerations-2026-08-23]. That raises an obvious follow-up question, which §4 answers.
6. **It should conclude that it cannot tell.** An absent heartbeat is evidence of a *communication* failure, which could be a dead node or could be a network partition. Concluding "the node is broken" claims more than the evidence supports.
7. **A newer client talking to an older server is the more dangerous direction:** the client can ask for things the server has never heard of. That intuition is correct in general and, for exactly one component in Kubernetes, wrong. §6 names it.
8. **Any duty that moves when somebody else runs the control plane counts.** Upgrade timing and etcd backup are the two this chapter can defend; which further duties move is a per-provider question rather than a documented split, so a broad answer is a pass here.

If you got 6+ right: skim. Read §2 and §6 properly — they carry this chapter's exam points — and work all three Taking Your Bearings checkpoints. The rest you can move through quickly.

If you got 3–5 right: read at normal pace. This chapter is calibrated for you.

If you got 0–2 right: read carefully, and if questions 2, 4, 5 and 6 were among your misses, re-read [*cross-bearing: see Ch 3 §5 — the API server and the only door in*] and [*cross-bearing: see Ch 4 §3 — namespaces and cluster-scoped objects*] before you start §2. Those two sections are the prerequisite base for more than half this chapter. Without Chapter 3's single-door architecture in place, §2 will read as an arbitrary list of three words.

Why This Chapter Matters

```
kubectl cordon node-7.
```

Three words, and a machine goes out of service without disturbing a single running process. Chapter 7 ended by naming the *act*, telling you the command was this chapter's opening move, and declining to give it. Here it is.

The shape is the more interesting thing, though. You have been typing commands in exactly this form since Chapter 3 (`kubectl apply -f`, `kubectl get pods`, `kubectl scale`) and nobody has told you what the form *is*. It worked anyway. That is precisely the condition under which a candidate walks into the exam room confident and then loses five points to a question about which component is permitted to be one minor version newer than which.

Chapters 2 through 7 made you someone who can describe what should run and where. This chapter makes you someone responsible for the thing it runs on. That is a different posture, and the vocabulary shifts with it: on watch, you think in three questions — what can I take out of service safely, what can I stop other people doing, and what can I not get back. You are about to acquire all three.

And here is the doubt worth carrying through the next eight sections. By the end of this chapter you will not have learned a single new mechanism. Every administrative act in it — taking a node out of service, capping a team's resource consumption, registering a machine, refusing a malformed request — is a write through a door you already know, reconciled by a controller you already met. That claim should be hard to believe right now, because what follows looks like four unrelated subjects wearing one chapter number. §8 will make the case. Until then, keep score.

The stakes, stated plainly: about five points on this book's allocation, which is not many. What the number understates is the *shape* of those points. §6's version-skew material is the single most mechanically checkable block in the entire curriculum. The rules are exact, the numbers are small, and there is no partial credit for nearly remembering them, which makes it the easiest place in the exam to lose points you had every opportunity to keep. That is the whole case for reading this chapter carefully, and it does not need inflating.

What You'll Learn

By the end of this chapter, you'll be able to:

- **Decompose** any `kubectl` command into its four slots, and say where the tool found the cluster it just talked to.
- **Trace** a request through the three gates it passes before anything is written down, and say what each gate can and cannot do about it.
- **Distinguish** a `ResourceQuota` from a `LimitRange` by what each constrains and at what scope, and say which of the two can make a previously valid Pod stop being accepted.
- **Take** a node out of service and put it back, predicting what happens to the Pods on it at each step.
- **State** which Kubernetes components are permitted to disagree about their version, by how much, and in which direction.
- **Recognize** every administrative act in this chapter as a write through one door, reconciled by a controller you already know, which is the only thing you actually have to remember.

You'll also stop reading the version-skew table as arbitrary trivia. That is a smaller change than it sounds, and it is worth more exam points than anything else in Part II.

§1 — The Grammar of a Command

You have been typing these for four chapters.

```
kubectl apply -f deployment.yaml.kubectl get pods.kubectl scale deployment web --replicas=5.
```

Every one of them worked. Nobody told you what the shape was, and you did not need to be told, which is exactly why it earns ten minutes now.

Here is the shape. Every `kubectl` invocation takes the form `kubectl [command] [TYPE] [NAME] [flags]` [source: k8s-docs-kubectl-overview-2026-08-23]. Four slots, and only the first is mandatory:

- **command** — the operation you want performed on one or more resources: `create`, `get`, `describe`, `delete` [source: k8s-docs-kubectl-overview-2026-08-23].
- **TYPE** — the resource type [source: k8s-docs-kubectl-overview-2026-08-23].
- **NAME** — the name of the specific resource. If the name is omitted, details for all resources are displayed [source: k8s-docs-kubectl-overview-2026-08-23].
- **flags** — optional. Flags you specify on the command line override default values and any corresponding environment variables [source: k8s-docs-kubectl-overview-2026-08-23].

Put five commands you have already run through those same four slots, and the shape appears retroactively.

kubectl	[command]	[TYPE]	[NAME]	[flags]
kubectl	cordon	—	node-7	—
kubectl	get	Pods	—	—
kubectl	apply	—	—	-f deploy.yaml
kubectl	scale	deployment	web	--replicas=5
kubectl	describe	node	worker-3	—

case-INsensitive

pod = pods = po

case-SENSITIVE

node-7 ≠ Node-7

Figure 8.1 — Five commands you already know, aligned on the same four slots. What to notice is the empty columns: `kubectl get pods` omits NAME and flags, and gets every Pod as a result. What to notice second is the asymmetry at the bottom, which is the examinable half of this section.

That asymmetry earns its own sentence, because it is the exact shape of an exam distractor. Resource types are case-insensitive, and you may use the singular, plural, or abbreviated form [source: k8s-docs-kubectl-overview-2026-08-23]. Resource *names* are case-sensitive [source: k8s-docs-kubectl-overview-2026-08-23]. The tool is relaxed about what kind of thing you meant and exacting about which one.

☆ **Fixed Point:** One grammar, four slots — `kubectl [command] [TYPE] [NAME] [flags]`. **Resource types are case-insensitive and abbreviable. Resource names are case-sensitive.**

The verb surface

Here is the operations table. Read it as an inventory rather than a list of new things; you have met a third of it already.

Verb	What it does	Where it lives in this book
get	List one or more resources	Ch 3, then throughout
describe	Display the detailed state of one or more resources	Ch 5, then throughout
apply	Apply a configuration change to a resource from a file or stdin	Ch 4
create	Create one or more resources from a file or stdin	Ch 4
delete	Delete resources from a file, stdin, or by label selector, name, or resource	Ch 4
logs	Print the logs for a container in a Pod	ahead, in Ch 13
exec	Execute a command against a container in a Pod	ahead, in Ch 16
scale	Update the size of the specified replication controller / deployment	Ch 6
rollout	Manage the rollout of a resource (deployments, daemonsets, statefulsets)	Ch 6
explain	Get documentation of various resources	here
config	Modify kubeconfig files	here

(All verbs and descriptions: [source: *k8s-docs-kubectl-overview-2026-08-23*].)

One entry deserves a sentence of its own. `explain` gets documentation of various resources [source: *k8s-docs-kubectl-overview-2026-08-23*], which makes it the only verb in the table that answers a question about *the resource type* rather than a question about *your cluster*.

🔒 **Worth Securing:** `kubectl explain` is the entry in this table that pays off longest. Because it returns documentation for a resource type rather than the contents of your cluster, it works on types you have never seen, including the CustomResourceDefinitions of Chapter 6 §8, installed by tools that did not exist when this book was written. Two years from now it will still be the fastest way to find out what a field does.

[cross-bearing: see Ch 4 §1 — *apply*, and *the declarative model*]. Chapter 4 gave `apply` a single sentence and sent you here for the rest. This table is that payoff, and Chapter 4's larger point stands unchanged: the objects are declarations, and the imperative verbs work by changing declarations. [cross-bearing: see Ch 6 §2 — *kubectl scale as a write to .spec.replicas*].

Where the cluster came from

The second real idea in this section is a question you have never had to ask, because the answer was already on your machine.

For configuration, `kubectl` looks for a file named `config` in the `$HOME/.kube` directory. You can specify other kubeconfig files by setting the `KUBECONFIG` environment variable or by setting the `--kubeconfig` flag [source: k8s-docs-kubectl-overview-2026-08-23]. That is the precedence, stated flat: a default location, an environment variable, and a flag. Per the general rule above — flags specified on the command line override default values and any corresponding environment variables — the flag wins over the environment variable [source: k8s-docs-kubectl-overview-2026-08-23].

If you answered Soundings question 1 with "an address and a credential," this is where that instinct lands. The file holds both, plus the answer to the two-server problem: which one you are currently talking to. That last part has a name — a **context** is one named bundle of cluster, user and namespace, and the **current context** is the one `kubectl` uses when you do not say otherwise. A kubeconfig can hold many; you are always working inside exactly one.

The surprising case: kubectl inside a Pod

By default `kubectl` first determines whether it is running within a Pod, and thus inside a cluster. It starts by checking for the `KUBERNETES_SERVICE_HOST` and `KUBERNETES_SERVICE_PORT` environment variables, and for the existence of a ServiceAccount token file at `/var/run/secrets/kubernetes.io/serviceaccount/token`. If all three are found, in-cluster authentication is assumed [source: k8s-docs-kubectl-overview-2026-08-23]. And when `kubectl` runs in a cluster it acts against the namespace of the ServiceAccount, unless `--namespace` is given [source: k8s-docs-kubectl-overview-2026-08-23].

Chapter 4 used the second half of that fact once, in passing, in an answer key. Here it is in full: three environment checks, an assumed identity, and a different default namespace.

🔒 **Snag:** `kubectl` inside a Pod does not use your kubeconfig, is not you, and does not default to the default namespace. It authenticates as the Pod's ServiceAccount and looks in that ServiceAccount's namespace. Every practitioner who has ever run a debugging shell inside a cluster has been surprised by this at least once, usually by an empty `kubectl get pods` that they were certain should have returned something.

The command that opens the chapter

```
kubectl cordon node-7.
```

Verb, resource name. The grammar, instantiated: the project's own reference gives the usage as `kubectl cordon NODE` with the synopsis "Mark node as unschedulable" [source: k8s-docs-kubectl-cordon-2026-08-24], which is the four-slot shape with `TYPE` and flags both empty. This is the thing Chapter 7 promised would be Chapter 8's opening move [cross-bearing: see Ch 7 §4 — the built-in taint

`node.kubernetes.io/unschedulable`]. What it does is what Chapter 7 already told you: it marks a node unschedulable, preventing the scheduler from placing new Pods onto that Node, without affecting the existing Pods on the Node [source: [k8s-docs-nodes-2026-08-23](#)].

That is all this section will say about it. Everything else belongs to §4: what it writes, what it does *not* do, what the second command is, and what happens if you skip that second command [*cross-bearing: see Ch 8 §4 — taking a node out of service*]. Carry the question with you; three sections from now it will have a better answer than it would have here.

..1 §2 — Three Gates and a Logbook

Soundings question 3 asked you to list the distinct questions a server must answer before doing expensive work. If you produced two — *who are you*, and *are you allowed* — you produced the answer nearly everyone produces, and you have just located a hole in your own model. That is a good place to start reading from, and it deserves naming rather than skipping past: a reader who has just discovered their model is incomplete is the most receptive reader this chapter gets.

There are three.

Chapter 3 named them in passing, at the point the API server was introduced, and pointed here for the treatment [*cross-bearing: see Ch 3 §5 — the API server and the only door in*]. The project's documentation is unambiguous that these are stages of one request path rather than three parallel checks: "When a request reaches the API, it goes through several stages" [source: [k8s-docs-controlling-access-2026-08-24](#)], and the page presents them as sequential sections — transport security, authentication, authorization, admission control, auditing [source: [k8s-docs-controlling-access-2026-08-24](#)]. The cluster-administration guidance on securing a cluster lists the same material in the same order — Controlling Access to the Kubernetes API, Authenticating, Authorization, Using Admission Controllers, and Auditing [source: [k8s-docs-cluster-administration-2026-08-23](#)] — and the project's extension-point taxonomy names three of them as the API access extensions: authentication, authorization, and dynamic admission control via webhooks [source: [k8s-docs-extending-kubernetes-2026-08-23](#)].

This chapter compresses that to three gates and a logbook. The project's own page opens with a fourth stage before any of them: transport security, in which the API server listens on port 6443 on the first non-localhost network interface, protected by TLS [source: [k8s-docs-controlling-access-2026-08-24](#)]. That is a real stage, and it is covered inside gate one below rather than given a box of its own.

Gate one: authentication — *who are you?*

Authentication establishes the identity behind the request. The cluster creation script or cluster admin configures the API server to run one or more Authenticator modules [source: [k8s-docs-controlling-access-2026-08-24](#)], and the input to the authentication step is the entire HTTP request, though it typically examines the headers and/or client certificate [source: [k8s-docs-controlling-access-2026-08-24](#)]. The API server is configured to listen for remote connections on a secure HTTPS port, typically 443, with

one or more forms of client authentication enabled [source: k8s-docs-control-plane-node-communication-2026-08-24].

Two facts about this gate are worth carrying. First, its failure mode is specific: if the request cannot be authenticated, it is rejected with HTTP status code 401 [source: k8s-docs-controlling-access-2026-08-24]. Second, a small surprise — while Kubernetes uses usernames for access control decisions and in request logging, it does not have a `User` object [source: k8s-docs-controlling-access-2026-08-24]. Identity here is a property of a request, not a record in the datastore.

The two identities you already have in the cluster arrive at this gate by different routes. Nodes should be provisioned with the public root certificate for the cluster, so that they can connect securely to the API server, along with valid client credentials; a good approach is for the kubelet's client credentials to take the form of a client certificate [source: k8s-docs-control-plane-node-communication-2026-08-24]. Automating the provisioning of those certificates is what kubelet TLS bootstrapping is for [source: k8s-docs-control-plane-node-communication-2026-08-24]. Pods, meanwhile, connect by leveraging a `ServiceAccount`: Kubernetes automatically injects the public root certificate and a valid bearer token into the Pod when it is instantiated [source: k8s-docs-control-plane-node-communication-2026-08-24]. That injected token is the same file `kubect1` looks for in §1 when it decides it is running inside a cluster.

Gate two: authorization — *may you do this?*

Authorization decides whether the identity established at gate one is permitted to perform *this action* on *this object*. A request must include the username of the requester, the requested action, and the object affected by the action [source: k8s-docs-controlling-access-2026-08-24]. One or more forms of authorization should be enabled, especially if anonymous requests or `ServiceAccount` tokens are allowed [source: k8s-docs-control-plane-node-communication-2026-08-24]. Securing your cluster means implementing effective authentication *and* authorization for API access: the pair, not either alone [source: k8s-docs-cloud-native-security-2026-08-23].

RBAC is one authorizer among several — Kubernetes supports multiple authorization modules, such as ABAC mode, RBAC Mode, and Webhook mode [source: k8s-docs-controlling-access-2026-08-24] — and RBAC in full is Chapter 12's material [*cross-bearing: see Ch 12 §3 — Role, ClusterRole, and the binding model*]. For now, one fact about it is enough: it is a mechanism that lives at this gate, and this gate's inputs are an identity, a verb, and an object — not the contents of your YAML.

Note the gate's quorum rule, because §3 and the third gate both turn on the contrast: if any module authorizes the request, then the request can proceed; if all of the modules deny the request, then the request is denied, with HTTP status code 403 [source: k8s-docs-controlling-access-2026-08-24].

Gate three: admission control — *should this, exactly as written, be allowed?*

This is the gate you did not have.

Admission control modules are software modules that can modify or reject requests [source: k8s-docs-controlling-access-2026-08-24], and unlike the first two gates they can access the contents of the object that is being created or modified [source: k8s-docs-controlling-access-2026-08-24]. They sit before persistence: once a request passes all admission controllers, it is validated using the validation routines for the corresponding API object, and then written to the object store [source: k8s-docs-controlling-access-2026-08-24].

And here is the property that makes them a genuinely different kind of thing rather than a third variation on "no." Authentication and authorization answer yes or no. Admission may answer yes, no, or *yes — but not as you wrote it*: admission controllers can also set complex defaults for fields [source: k8s-docs-controlling-access-2026-08-24].

☆ **Fixed Point:** Authentication, then authorization, then admission. Authentication asks **who**. Authorization asks **may you**. Admission asks **should this, exactly as written, be allowed to happen** — and it is the only one of the three that can change your request instead of refusing it [source: k8s-docs-controlling-access-2026-08-24].

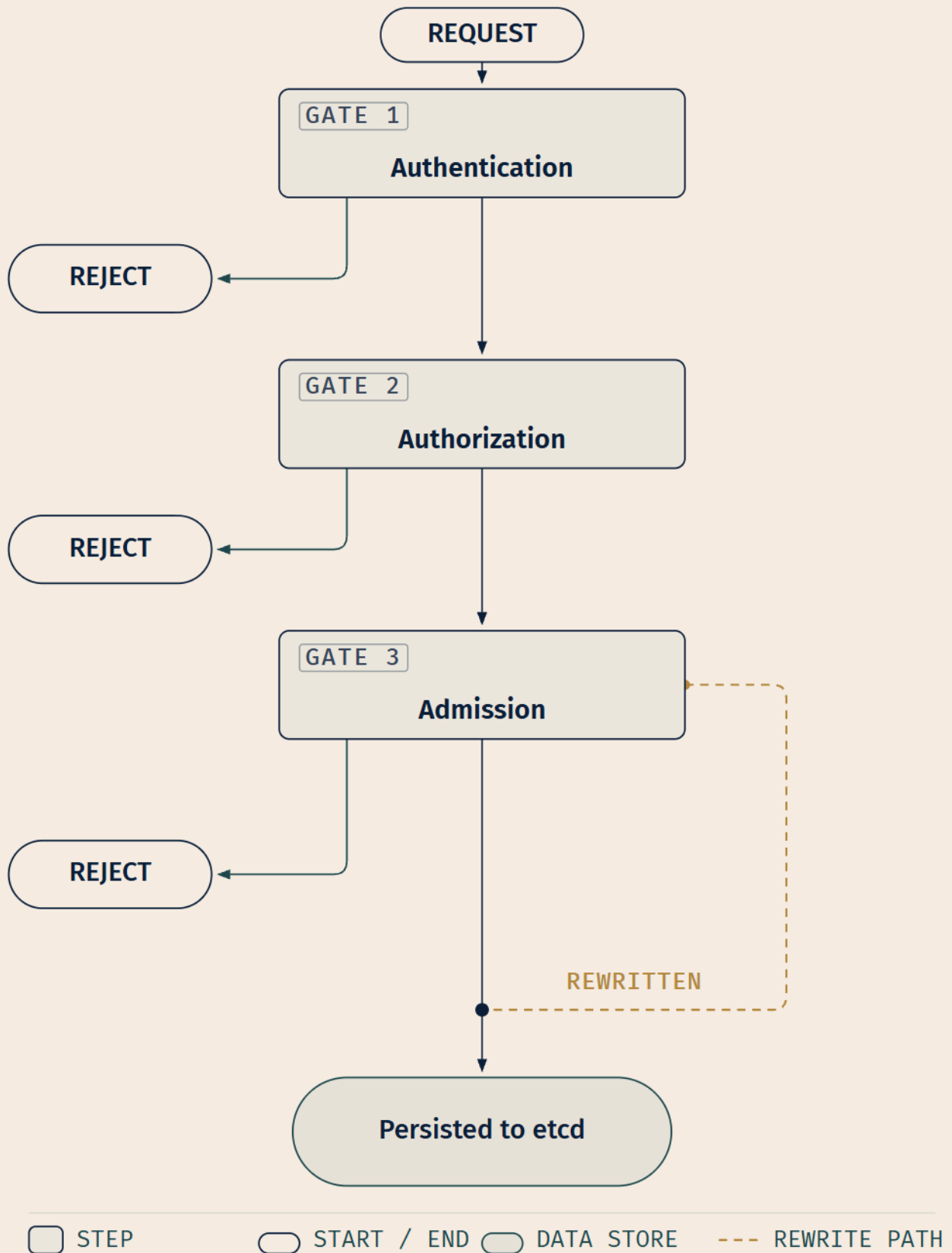


Figure 8.2 — What to notice is the fourth arrow. Gates one and two have one way out other than forward: refusal. Gate three has two. It can refuse, or it can alter the request and let the altered version continue. The three questions, in order, are: *who are you*, *may you do this*, and *should this, as written, be allowed*.

🔑 **Mnemonic:** *Who, may, and how*. Three words, in order, one per gate. The third is the odd one because "how" is a question about the request rather than about you.

⚠️ **Navigational Hazards:** Authorization and admission are not two names for the same check, and the sharpest proof is that they disagree about what counts as a decision. **Authorization is any-module-approves:** if any module authorizes the request, it proceeds; only if all modules deny is it refused [source: k8s-docs-controlling-access-2026-08-24]. **Admission is any-module-rejects:** unlike authentication and authorization modules, if any admission controller module rejects, the request is immediately rejected [source: k8s-docs-controlling-access-2026-08-24]. One gate is a vote you can win with a single supporter; the next is a veto any participant can exercise. A request can be fully authenticated, entirely authorized, and still be refused — or quietly rewritten — at the third gate.

🔍 **Closer Look:** Admission controllers act on requests that create, modify, delete, or connect to (proxy) an object. They do not act on requests that merely read objects [source: k8s-docs-controlling-access-2026-08-24] — reads bypass the admission control layer entirely [source: k8s-docs-admission-controllers-2026-08-24]. So the whole of this gate is invisible to `kubectl get`, which is deeper than the exam requires and explains a great deal of otherwise-baffling behavior the first time you install a policy engine.

Extended Analogy: Think of a working commercial harbour rather than a locked building. A vessel arriving is met first by a pilot boat, whose only question is *which vessel is this*: papers, registration, identity. That is authentication. It has no view on your business here.

Once identified, the harbourmaster consults the berth allocations: is this vessel entitled to a berth in this harbour today? That is authorization. The harbourmaster does not open a single crate. The question is about standing, not about cargo.

Then, and only then, the customs officer comes aboard. This one *does* open crates. She may find something prohibited and turn the whole vessel around. But she has a third option the other two lack: she may say the vessel can dock provided a particular container stays sealed, or provided a declaration is completed and attached before unloading. The vessel proceeds — altered. That is admission control, and the third option is the entire reason it is a separate office rather than one more line on the harbourmaster's form.

Two admission controllers you have already met

This is not an abstraction you have to take on faith. You have seen it work twice.

Chapter 7 introduced the **NodeRestriction** admission plugin, which prevents the kubelet from setting labels with a `node-restriction.kubernetes.io/` prefix [source: k8s-docs-assign-pod-node-2026-08-23]: the reason you can trust node-isolation labels not to have been forged by the node they describe. It is

an admission controller by name in the project's own plugin reference — `NodeRestriction` limits the `Node` and `Pod` objects a kubelet can modify [source: [k8s-docs-admission-controllers-2026-08-24](#)]. Chapter 7 stated the rule and pointed here for the enforcement [*cross-bearing: see Ch 7 §3 — node labels and the NodeRestriction plugin*]. This gate is the enforcement.

And the Pod Security Standards are enforced by the built-in Pod Security Admission controller [source: [k8s-docs-pod-security-standards-2026-08-23](#)]. That is one clause and no more; Chapter 12 owns the three profiles and the three modes [*cross-bearing: see Ch 12 §6 — Pod Security Standards and Pod Security Admission*]. What matters here is the derivation: when you meet Pod Security Admission four chapters from now, you will not be learning a new kind of thing. You will be learning one instance of the third gate.

The same is true of §3's material, arriving next. `ResourceQuota` is an admission controller that observes the incoming request and ensures that it does not violate any of the constraints enumerated in the `ResourceQuota` object [source: [k8s-docs-admission-controllers-2026-08-24](#)], and `LimitRanger` does the same for the constraints in a `LimitRange` object [source: [k8s-docs-admission-controllers-2026-08-24](#)]. Neither is a separate subsystem with its own enforcement path; both take effect at this gate.

🔗 **Closer Look:** Dynamic admission control means the cluster calls out to a webhook you supplied. Kubernetes makes synchronous HTTP requests to a remote service, a webhook backend, and the documentation is candid that this adds a potential point of failure [source: [k8s-docs-extending-kubernetes-2026-08-23](#)]. Which is to say: once you install a validating webhook, your webhook being down is a thing that can stop your cluster accepting requests.

And a logbook

Alongside the three gates, the cluster-administration guidance on securing a cluster lists **Auditing** [source: [k8s-docs-cluster-administration-2026-08-23](#)]. It sits in the same list, at the same level, as the three access-control pages.

Kubernetes auditing provides a security-relevant, chronological set of records documenting the sequence of actions in a cluster [source: [k8s-docs-audit-2026-08-24](#)]. It allows cluster administrators to answer what happened, when it happened, who initiated it, on what it happened, where it was observed, from where it was initiated, and to where it was going [source: [k8s-docs-audit-2026-08-24](#)].

And one detail restates this section's architecture a fourth time: audit records begin their lifecycle inside the `kube-apiserver` component, and each request on each stage of its execution generates an audit event [source: [k8s-docs-audit-2026-08-24](#)]. The logbook is not a separate observer watching the door. It is kept by the door.

Why three gates at one door is a complete story

Close on the architecture, because it is the reason this section is short enough to be learnable.

Kubernetes has a hub-and-spoke API pattern. All API usage from nodes, or from the Pods they run, terminates at the API server. None of the other control plane components are designed to expose remote services [source: k8s-docs-control-plane-node-communication-2026-08-24].

That is the load-bearing sentence. Three gates on one door would be an incomplete access-control story if there were other doors; you would be securing one entrance out of several. There are not. Chapter 3's single-door architecture is what makes a three-gate model *sufficient* rather than merely *present*, and it is why this chapter has a spine at all [cross-bearing: see Ch 8 §8 — one door, and controllers behind it].

..1 §3 — Dividing a Shared Cluster

Chapter 7 §2 left you with a complaint and an IOU. The complaint: nothing in what you had learned so far stops you booking the entire cluster with resource requests you will never actually use. The IOU: the mechanisms that stop *other people* doing that to *you* live here [cross-bearing: see Ch 7 §2 — requests, and the cluster you could book by accident].

There are two of them, and the only mistake worth guarding against is swapping them.

ResourceQuota — a ceiling on a namespace

When several users or teams share a cluster with a fixed number of nodes, there is a concern that one team could use more than its fair share of resources [source: k8s-docs-resource-quotas-2026-08-24]. A resource quota, defined by a ResourceQuota object, provides constraints that limit **aggregate resource consumption per namespace** [source: k8s-docs-resource-quotas-2026-08-24]. That is the sentence Chapter 4 gave you in outline — namespaces are a way to divide cluster resources between multiple users, via resource quota [source: k8s-docs-namespaces-2026-08-23] — now stated at full strength [cross-bearing: see Ch 4 §3 — namespaces, and what they are for].

A cluster administrator creates at least one ResourceQuota for each namespace; users create resources in the namespace, and the quota system tracks usage to ensure it does not exceed hard resource limits [source: k8s-docs-resource-quotas-2026-08-24]. If creating or updating a resource violates a quota constraint, the control plane rejects that request with HTTP status code 403 Forbidden [source: k8s-docs-resource-quotas-2026-08-24].

What can a quota count? Three families, and you need the shape rather than the roster:

- **Compute totals** — requests.cpu, requests.memory, limits.cpu, limits.memory, and hugepages-<size>, with cpu and memory as aliases for the two request forms [source: k8s-docs-resource-quotas-2026-08-24].
- **Storage** — requests.storage and persistentvolumeclaims, optionally scoped per StorageClass [source: k8s-docs-resource-quotas-2026-08-24].
- **Object counts** — written count/<resource> for core API group resources and count/<resource>.<group> otherwise; countable resources include count/pods, count/services, count/secrets,

`count/configmaps` and `count/deployments.apps` [source: k8s-docs-resource-quotas-2026-08-24].

One quota rule is worth more exam points than the rest of the section combined, because it changes what a valid manifest is:

☆ **Fixed Point: If you enforce a resource quota in a namespace for either `cpu` or `memory`, you and other clients must specify either `requests` or `limits` for that resource, for every new Pod you submit. If you don't, the control plane may reject admission for that Pod** [source: k8s-docs-resource-quotas-2026-08-24].

Read that as a consequence rather than a rule to memorize. A quota is a ceiling on a total; a total can only be computed if every contributor declares its share. A Pod that declares nothing is not a small Pod — it is an *uncountable* one, and the quota system cannot let it through.

Note also what a quota is not: ResourceQuotas are independent of the cluster capacity, so if you add nodes to your cluster, this does not automatically give each namespace the ability to consume more resources [source: k8s-docs-resource-quotas-2026-08-24].

LimitRange — a rule about an object

A LimitRange is a policy to constrain the resource allocations — limits and requests — that you can specify for **each applicable object kind**, such as Pod or PersistentVolumeClaim, in a namespace [source: k8s-docs-limit-range-2026-08-24]. Four things it provides [source: k8s-docs-limit-range-2026-08-24]:

- enforce minimum and maximum compute resource usage per Pod or Container in a namespace;
- enforce minimum and maximum storage request per PersistentVolumeClaim;
- enforce a ratio between request and limit for a resource;
- **set default request/limit values for compute resources in a namespace and automatically inject them into Containers at runtime.**

The administrator creates a LimitRange in a namespace; users then create objects in that namespace [source: k8s-docs-limit-range-2026-08-24]. Two things happen in order: first, the LimitRange admission controller applies default request and limit values for all Pods (and their containers) that do not set compute resource requirements; second, the LimitRange tracks usage to ensure it does not exceed the resource minimum, maximum and ratio defined in any LimitRange present in the namespace [source: k8s-docs-limit-range-2026-08-24]. A violation fails with the same `403 Forbidden` a quota violation produces, plus a message explaining the constraint that has been violated [source: k8s-docs-limit-range-2026-08-24].

The timing is worth one sentence, because it is the mutate-versus-reject distinction of §2 arriving one gate later in the same request's life: **LimitRange validations occur only at Pod admission stage, not on running Pods** — if you add or modify a LimitRange, the Pods that already exist in that namespace continue unchanged [source: k8s-docs-limit-range-2026-08-24].

The discrimination, made structural

Definitions are easy to swap. Questions are harder. If you can answer these two about a mechanism, you will never confuse them again:

What is being counted? The quota counts the namespace's aggregate total [source: k8s-docs-resource-quotas-2026-08-24]. The LimitRange counts one object's numbers [source: k8s-docs-limit-range-2026-08-24].

What happens to a manifest that says nothing about resources? In a quota'd namespace, the control plane may reject it [source: k8s-docs-resource-quotas-2026-08-24]. Under a LimitRange, the admission controller fills it in [source: k8s-docs-limit-range-2026-08-24].

☆ **Fixed Point: ResourceQuota counts the namespace. LimitRange constrains the object.** One is a ceiling on a team; the other is a rule about a manifest.

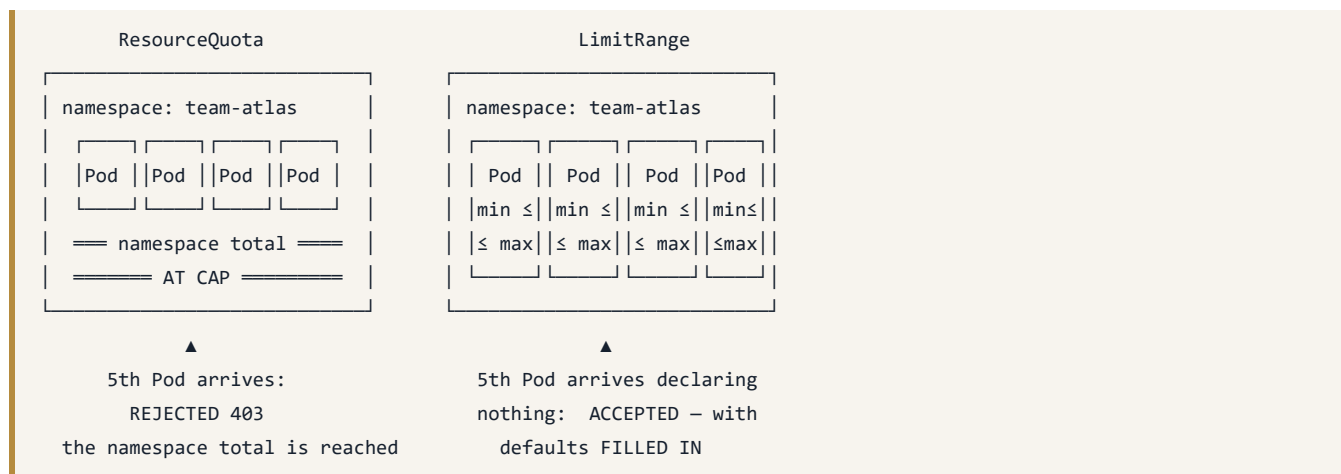


Figure 8.3 — Both mechanisms live inside one namespace; the boundary is the same on each side and is not the discrimination. What differs is what is being counted and how it fails. On the left, one aggregate total for the namespace, and a fifth Pod rejected at the cap. On the right, per-object bounds on each Pod, and a fifth Pod that declares nothing is not refused but modified.

📌 **Snag:** A LimitRange that supplies a default request changes what your manifest means without changing your manifest. The Pod you get is not the Pod you wrote, and `kubectl get pod <name> -o yaml` is where you find out. Two aggravating details: a LimitRange does not check the consistency of the default values it applies [source: k8s-docs-limit-range-2026-08-24], and if two or more LimitRange objects exist in the namespace, it is not deterministic which default value will be applied [source: k8s-docs-limit-range-2026-08-24].

🔒 **Worth Securing:** The two are usually deployed together, and the sources make the reason explicit rather than leaving it to intuition. A quota'd namespace *forces* Pods to declare requests or limits [source: k8s-docs-resource-quotas-2026-08-24] — and you can use a LimitRange to automatically set a default request for these resources [source: k8s-docs-resource-quotas-2026-08-24]. The quota sets the ceiling and makes declaration compulsory; the LimitRange makes sure a developer who forgets still ends up with a Pod that is countable. That is why the Kubernetes security guidance names them in one breath: define ResourceQuotas to fairly allocate shared resources, and use LimitRanges to ensure that Pods specify their resource requirements [source: k8s-docs-cloud-native-security-2026-08-23].

[cross-bearing: see Ch 5 §8 — requests and limits, the numbers a LimitRange defaults]

The hinge, and it is worth thirty seconds

ResourceQuota and LimitRange are namespaced objects. The Nodes that §4 is about to discuss are not.

Chapter 4 established this and you may already feel where it goes: namespace-based scoping is applicable only for namespaced objects — Deployments, Services and so on — and not for cluster-wide objects such as StorageClass, Nodes and PersistentVolumes [source: k8s-docs-namespaces-2026-08-23].

So the two halves of "stop people using too much" sit on opposite sides of a boundary you already know. **You can quota a team. You cannot quota a machine.** A quota is a statement about a namespace, and every resource it can count is a namespaced one; a Node is not in a namespace, so no ResourceQuota reaches it.

Say that out loud once, because Chapter 12 is going to *derive* the RBAC four-way matrix from exactly this boundary rather than asking you to memorize four combinations *[cross-bearing: see Ch 12 §3 — namespaces and cluster-scoped permissions]*.

And one closing observation, offered rather than promised: both of these mechanisms are admission controllers [source: k8s-docs-admission-controllers-2026-08-24]. Neither is a separate subsystem with its own enforcement path. That is the first installment of a claim §8 will finish.

🕒 Taking Your Bearings #1 — The Path a Command Takes In

Five questions on §1 through §3. Answer in your own words before checking; the effort of retrieval is what makes this stick.

1. 📄 Decompose `kubectl describe node worker-3` into its four slots and name each one. Then say which of the four you could change the capitalization of without breaking the command.
2. 📄 You run `kubectl get pods` from a shell on your laptop, and again from inside a running Pod in the same cluster. Both succeed and return different results. Explain *both* differences — what identity each invocation used, and what namespace each looked in.

3. **..** Name the three gates a request passes, in order. Then say which of them can result in the request being *changed* rather than accepted or rejected.
4. **..** A request is refused. You are told that the identity was valid, and that the identity had permission to perform this action on this resource. Which gate refused it, and give one plausible reason.
5. **..** **[retrieval: ch4]** Chapter 4 said namespaces are the unit by which cluster resources get divided between users, and it named the mechanism. Name it, say what scope it constrains — and then say what the *other* mechanism in §3 constrains instead.

Answers + explanations

1. `kubectl / describe (command) / node (TYPE) / worker-3 (NAME)`; there are no flags. **You could capitalize the TYPE freely**, since resource types are case-insensitive and accept singular, plural or abbreviated forms. You could not capitalize the name; names are case-sensitive.

Common wrong turns: "both are case-insensitive" and "both are case-sensitive" are the two tempting symmetrical answers, and the asymmetry is the whole point. `node worker-3` works. `node Worker-3` does not.

2. From your laptop: `kubectl` used the `kubeconfig` at `$HOME/.kube/config` (or whatever `KUBECONFIG` / `--kubeconfig` pointed at), authenticated as *you*, and looked in the namespace of the current context. From inside the Pod: `kubectl` found `KUBERNETES_SERVICE_HOST`, `KUBERNETES_SERVICE_PORT` and the ServiceAccount token file, assumed in-cluster authentication, authenticated as the **ServiceAccount**, and looked in the **ServiceAccount's namespace**. Two different identities, two different namespaces, one command.

3. **Authentication, then authorization, then admission control.** Only admission can change the request.

The reason this matters is not that the third gate has an extra feature. It is the *reason there are three gates rather than one*. Two of them are asking about you and answering yes or no; the third is asking about the request itself and has a third answer available: yes, in modified form. When you meet Pod Security Admission in Chapter 12, you will be meeting an instance of this gate, and you will not have to learn a new kind of thing.

Common wrong turns: a two-gate answer (the most common incomplete model, and the one Soundings question 3 was built to expose); the order reversed; and attributing the mutation power to authorization.

4. **Admission.** Both earlier gates already said yes — the identity was established and the action was permitted — so the refusal came from the gate that looks at the request's *contents*. A plausible reason: the Pod would have exceeded its namespace's ResourceQuota, or a policy plugin rejected something about the object as written.

One plausible cause is enough. There is no need to enumerate admission controllers; the full plugin surface is out of scope here and Chapter 12 owns the policy landscape.

5. ResourceQuota, which constrains **aggregate resource consumption per namespace**. The other mechanism is **LimitRange**, which constrains **each applicable object kind** in a namespace and supplies defaults so that Pods specify their resource requirements at all.

Common wrong turn: swapping them, which is the only real error available here. If you find the two blurring, hold the failure modes rather than the definitions: a quota's answer to an undeclared Pod is 403; a LimitRange's answer is to fill the numbers in.

How'd you do?

5/5: You have the request path. Move on to §4 with confidence. **3–4:** Solid. Review the ones you missed, particularly if question 3 was among them, since §4 through §8 all lean on the gate model. **0–2:** No shame in it, but do not continue yet. Re-read §2 before §4, about ten minutes. If question 5 was a miss as well, spend five more on §3's hinge. Everything from here builds on the idea that administrative acts are ordinary writes that pass ordinary gates.

Checkpoint: you've now mastered ✓ The four-slot grammar, and the case-sensitivity asymmetry inside it
 ✓ Where `kubectl` finds its cluster — on a laptop and inside a Pod ✓ The three gates, in order, and which one can rewrite you ✓ ResourceQuota versus LimitRange, by scope and by failure mode □ Taking a node out of service (next)

..1 §4 — Taking a Node Out of Service

You have been carrying an open question since §1. Here is the answer, and it takes three commands rather than one, which is the entire reason it was worth waiting for.

Where nodes come from

First, two sentences you have never been given, because they are the node-side instance of a pattern §8 is going to lean on.

There are two main ways to have Nodes added to the API server: the kubelet on a node self-registers to the control plane, which is the default, or you (or another human user) manually add a Node object [source: k8s-docs-nodes-2026-08-23]. After you create a Node object, the control plane checks whether it is valid — whether a kubelet has registered with the API server matching the `metadata.name` field of the Node — and if the node is healthy, meaning the necessary services are running, then it is eligible to run a Pod [source: k8s-docs-nodes-2026-08-23]. The name of a Node object must be a valid DNS subdomain name and must be unique [source: k8s-docs-nodes-2026-08-23].

Note what a kubelet does when it joins a cluster: it writes an object through the API server. It does not open a private channel. It does the same thing you do.

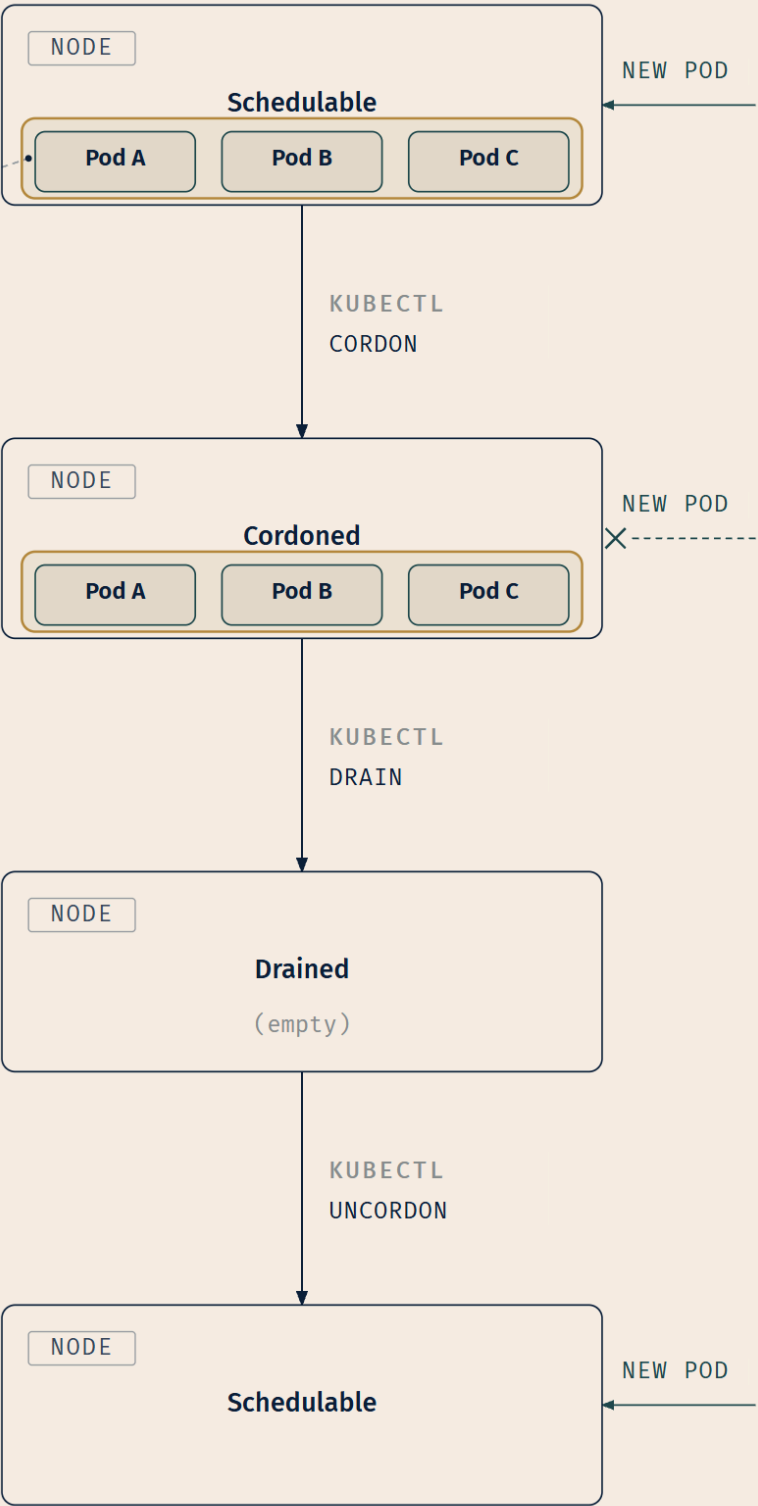
The three commands

Marking a node as unschedulable with `kubectl cordon $NODENAME` prevents the scheduler from placing new Pods onto that Node, but does not affect existing Pods on the Node; this is useful as a preparatory step before a node reboot or other maintenance [source: k8s-docs-nodes-2026-08-23]. You can use `kubectl drain` to safely evict all of your Pods from a node before you perform maintenance on it, such as a kernel upgrade or hardware maintenance [source: k8s-docs-safely-drain-node-2026-08-24]. `kubectl uncordon` restores scheduling [source: k8s-docs-nodes-2026-08-23] — and the drain documentation is explicit that this is a separate step you must take: you need to run `kubectl uncordon <node name>` afterwards to tell Kubernetes that it can resume scheduling new Pods onto the node [source: k8s-docs-safely-drain-node-2026-08-24].

Read the middle clause of that first sentence again. It is doing more work than its length suggests. `cordon` **deliberately leaves the running Pods alone**. That is not an oversight in the tool; it is the point of having a separate step. And it means the phrase "take a node out of service," which sounds like one action, is two.

☆ **Fixed Point:** `cordon` stops arrivals and touches nothing already aboard. `drain` clears what is aboard. `uncordon` reopens. Three commands, three jobs — and **the maintenance sequence needs the first two**.

A, B, C: unchanged, still running in both panels. Cordon blocks NEW Pods only – it does not evict.



LEGEND

- PODS UNCHANGED
- NEW POD ADMITTED
- NEW POD TURNED AWAY

Figure 8.4 — What to notice is what does *not* change between the first two panels. The three Pods aboard the cordoned node are still running, still serving, still entirely unaffected. Only the arriving Pod's fate differs. The node does not empty until `drain`.

⚠ **Navigational Hazards: A cordoned node is not an empty node.** If you cordon a node and then reboot it for maintenance, every Pod still on that node goes down with the machine. This is the single most consequential confusion in this chapter, and unlike most exam traps it has a real operational cost attached. The instinct that "take out of service" means "empty" is a reasonable instinct. It is also wrong, and the fix is one more command.

🔗 **Mnemonic:** A cordon is a rope across a doorway. It stops people coming in. It does not remove the people already inside.

🔍 **Closer Look:** `drain` is not a special maintenance channel either. You can request eviction by calling the Eviction API directly, or programmatically using a client of the API server, like the `kubectl drain` command; this creates an `Eviction` object, which causes the API server to terminate the Pod [source: [k8s-docs-api-eviction-2026-08-24](#)]. Using the API to create an `Eviction` object for a Pod is like performing a policy-controlled `DELETE` operation on the Pod [source: [k8s-docs-api-eviction-2026-08-24](#)]. So `drain` is a client that writes objects through the one door — which is §8's whole claim, arriving early.

One exception, and Chapter 7 §4 already joined these two for you. Pods that are part of a `DaemonSet` tolerate being run on an unschedulable Node [source: [k8s-docs-nodes-2026-08-23](#)], because the `DaemonSet` controller automatically adds a `node.kubernetes.io/unschedulable` toleration with a `NoSchedule` effect to `DaemonSet` Pods [source: [k8s-docs-daemonset-2026-08-24](#)]. Chapter 6 taught `DaemonSet` as one-Pod-per-eligible-node; Chapter 7 taught the built-in condition tolerations and joined them [cross-bearing: see Ch 7 §4 — *the DaemonSet controller's automatic tolerations*] [cross-bearing: see Ch 6 §7 — *DaemonSet and node-local facilities*]. Here is what that join buys you during maintenance.

Node conditions

A Node's status contains, among other fields: `Addresses`; `Conditions`; `Capacity and Allocatable`; and `Info` such as kernel version, Kubernetes version, container runtime details and the operating system [source: [k8s-docs-node-status-2026-08-24](#)]. `kubectl describe node <name>` shows them [source: [k8s-docs-nodes-2026-08-23](#)].

The `conditions` field describes the status of all `Running` nodes [source: [k8s-docs-node-status-2026-08-24](#)]:

Condition	True when
Ready	The node is healthy and ready to accept Pods. False if the node is not healthy and is not accepting Pods. Unknown if the node controller has not heard from the node in the last <code>node-monitor-grace-period</code> (default is 50 seconds)
DiskPressure	Pressure exists on the disk size — that is, if the disk capacity is low
MemoryPressure	Pressure exists on the node memory — that is, if the node memory is low
PIDPressure	Pressure exists on the processes — that is, if there are too many processes on the node
NetworkUnavailable	The network for the node is not correctly configured

(All five conditions, the three Ready values, and the grace-period default: [source: *k8s-docs-node-status-2026-08-24*].)

Four of those are two-valued. Ready is three-valued, and the third value is the interesting one.

Unknown is not a fourth failure mode. It is the control plane declining to guess. False means the node reported itself unhealthy: the node is *talking to you* and telling you something is wrong. Unknown means nobody has heard from it, which could equally be a dead machine or a network partition between the machine and the control plane. Those two situations call for different interventions, which is why the distinction is preserved rather than collapsed. If you answered Soundings question 6 with "it should conclude that it cannot tell," you reasoned your way to the shape of this answer without the vocabulary.

Treat the 50-second default as an illustration of the parameter rather than as the examinable fact. The parameter name is the durable thing; the number is configuration, and a cluster you meet may have been given a different one.

🔖 **Snag:** Print details of a cordoned node with a command-line tool and the Condition list includes `SchedulingDisabled`. **`SchedulingDisabled` is not a Condition in the Kubernetes API; instead, cordoned nodes are marked `Unschedulable` in their spec** [source: *k8s-docs-node-status-2026-08-24*]. The thing `kubectl` shows you is not always a thing the API has — and the field that actually changed is one you write, not one the system reports. Hold that; §8 needs it.

Heartbeats, and a control loop you already know

For nodes there are two forms of heartbeat: updates to the `.status` of a Node, and Lease objects within the `kube-node-lease` namespace, with each Node having an associated Lease object [source: *k8s-docs-nodes-2026-08-23*].

Chapter 4 pointed at exactly those Leases and said they were how the control plane detects node failure [source: *k8s-docs-namespaces-2026-08-23*] [cross-bearing: see Ch 4 §3 — *the four initial namespaces*].

That IOU is now settled: two heartbeat forms, one of them an object in a namespace you have already listed.

The node controller is a Kubernetes control plane component that manages several aspects of nodes: assigning a CIDR block to the node when it is registered; keeping its internal list of nodes up to date with the cloud provider's list of available machines; and monitoring the nodes' health — updating the Ready condition to Unknown when a node becomes unreachable, and triggering API-initiated eviction of all the Pods from the node if it stays unreachable [source: k8s-docs-nodes-2026-08-23].

Read that third job as a shape rather than as a list. It observes (heartbeats), it compares against what it expects, and it acts (condition update, then eviction). **The node controller is a control loop.** You met the pattern in Chapter 3 and you have met it in every chapter since [*cross-bearing: see Ch 3 §6 — the control loop*]. Noticing that costs one sentence and buys \$8 half its argument.

Capacity and Allocatable

Chapter 7 §2 told you that what makes Capacity and Allocatable differ, and how that is configured, is this chapter's material [*cross-bearing: see Ch 7 §2 — Capacity, Allocatable, and what the scheduler counts*]. Here is that promise, paid.

The fields in the capacity block indicate the total amount of resources that a Node has; the allocatable block indicates the amount of resources on a Node that is available to be consumed by normal Pods [source: k8s-docs-node-status-2026-08-24]. 'Allocatable' on a Kubernetes node is defined as the amount of compute resources that are available for pods [source: k8s-docs-node-allocatable-2026-08-24]. The scheduler treats 'Allocatable' as the available capacity for Pods, and does not over-subscribe it [source: k8s-docs-node-allocatable-2026-08-24].

Why are they two different numbers? Because Kubernetes nodes can be scheduled to Capacity, and Pods can consume all the available capacity on a node by default — which is an issue, because nodes typically run quite a few system daemons that power the OS and Kubernetes itself [source: k8s-docs-reserve-compute-resources-2026-08-24]. Two reservations account for those daemons: kubeReserved captures resource reservation for Kubernetes system daemons like the kubelet and the container runtime, and systemReserved captures reservation for OS system daemons such as sshd and udev [source: k8s-docs-reserve-compute-resources-2026-08-24].

The configuration detail — how much is reserved, and the flags that set it — is above the associate tier and this book does not cover it. What the exam wants from you is the distinction: **Capacity is the machine's total; Allocatable is what the scheduler does arithmetic against.**

§5 — Who Owns the Control Plane

Everything so far in this chapter has been something you *do*. This section is about something you *own*, and the two are not the same question.

It is also, deliberately, the easiest reading in Part II. You have just come through the chapter's densest run and you have another one waiting in §6. Take this one slowly and without effort.

The planning questions

Before choosing how to build a cluster, the documentation asks you to consider [source: k8s-docs-cluster-administration-2026-08-23]:

- Do you want to try out Kubernetes on your computer, or do you want to build a high-availability, multi-node cluster?
- Will you be using a hosted Kubernetes cluster, or hosting your own?
- Will your cluster be on-premises, or in the cloud?
- Will you be running Kubernetes on bare-metal hardware, or on virtual machines?
- Do you want to run a cluster, or do you expect to do active development of Kubernetes project code?

Five questions, and the honest observation is that in most working lives the answers arrive already decided: by budget, by a compliance requirement, by what the platform team standardized on two years ago. Knowing the axes still buys you something, because it tells you what was traded away.

The tools, split by what they are for

For learning. If you are learning Kubernetes, use tools supported by the Kubernetes community or in the ecosystem to set up a cluster on a local machine: **minikube**, which runs a single- or multi-node local Kubernetes cluster, and **kind** — Kubernetes IN Docker — which runs local clusters using Docker containers as nodes [source: k8s-docs-setup-tooling-2026-08-23].

For production. When evaluating a solution for a production environment, consider which aspects of operating a cluster you want to manage yourself and which you prefer to hand off to a provider [source: k8s-docs-setup-tooling-2026-08-23]. The options are managed and turnkey certified Kubernetes services from cloud providers, and self-managed clusters bootstrapped with **kubeadm**, the officially supported tool for creating clusters, used to install the control plane and join nodes [source: k8s-docs-setup-tooling-2026-08-23]. Other ecosystem tools include **k3s**, a lightweight distribution [source: k8s-docs-setup-tooling-2026-08-23].

🔒 **Worth Securing:** kind and minikube are not two names for the same thing. kind runs its nodes as Docker containers, which makes clusters fast to create and destroy and makes it the usual choice inside CI pipelines. minikube runs a local cluster with a broader set of conveniences around it, which makes it the usual choice when a human is sitting in front of it. Choosing between them is really a question of whether a person or a pipeline is the user.

One requirement none of these removes

Whichever route you take, a container runtime — containerd or CRI-O — must be installed on every node, and kubectl is the command-line tool for managing any cluster [source: k8s-docs-setup-tooling-2026-08-23].

23].

That first clause is worth six chapters of your attention for one moment. Chapter 2 taught you the boundary between Kubernetes and the thing that actually starts containers, and taught it as an interface question [*cross-bearing: see Ch 2 §4 — the CRI, and what Kubernetes does not do itself*]. This is the first time in this book that boundary has had an operational consequence. `kubeadm` will build you a control plane and join your nodes to it, and a container runtime must be on those nodes regardless, because a container runtime is on the other side of a line the project deliberately drew.

And the second clause is quietly the reason this book works. However the cluster was built — laptop, rack, or a provider's console — the tool is the same one, and the grammar is the one from §1.

What ownership actually means

Also collect one small debt here: scheduler profile configuration lives in the control plane's own component configuration, which is a thing you can reach only if you own the control plane [*cross-bearing: see Ch 7 §6 — scheduling profiles*].

That is the general shape of the answer to Soundings question 8. Whoever operates the control plane holds the upgrade calendar and holds the `etcd` backup. If that is a provider, those are theirs. If it is you, they are yours.

Which is precisely why the next two sections exist. **§6 is which versions are allowed to disagree. §7 is what you cannot get back.** Those are the two duties that move.

Logbook Entry: The managed-versus-self-hosted decision is usually argued as a cost question, and cost is rarely what decides it in practice.

A control plane is not expensive. Three modest machines will run one. What is expensive is the *calendar* attached to it: three minor releases a year, each with its own upgrade window, its own compatibility matrix, its own regression to discover in staging on a Thursday. Plus certificate expiries. Plus `etcd` backups you have to prove you can actually restore from, which is a different exercise from taking them. Plus the person who has to be reachable while all of that happens.

Crews that self-host successfully are almost always crews that budgeted for that calendar deliberately, as a named piece of somebody's job. Crews that regret it are usually crews that priced the machines and not the Thursdays. Neither choice is the right one in general; plenty of organizations have excellent reasons to own the whole thing, and regulatory ones are only the most obvious. But the question worth asking out loud before you decide is not *can we run this*, it is *whose watch does this stand on*.

🕒 Taking Your Bearings #2 — The Machine, and Whose It Is

Five questions on §4 and §5. Two of them reach back into earlier chapters.

1. **[retrieval: ch7]** You cordon a node. Chapter 7 taught you a built-in taint whose effect matches what cordoning does: name that taint and its effect, and then, using Chapter 7's rule about that effect, say what happens to the Pods already running on the node.
2. **Challenge item — this one is meant to be uncomfortable, and the intuition it breaks is a reasonable intuition.** An engineer cordons a node and immediately reboots it for maintenance. Three services go down. What did they skip, and why did the cordon not prevent it?
3. A node stops responding. Its Ready condition changes. To what value — and what does that value mean that False would not? Then name the other four conditions a Node's status carries.
4. **[retrieval: ch2]** You bootstrap a cluster with kubeadm. Before any node in it can run a Pod, one piece of software must be installed on that node which kubeadm does not provide. Name it, name the two implementations the documentation names, and say which interface Kubernetes uses to talk to it.
5. Two engineers describe their clusters. One says "we upgraded to 1.36 last Tuesday." The other says "we got upgraded to 1.36 last Tuesday." What is different about their situations, and name one further duty that differs between them.

Answers + explanations

1. The taint is `node.kubernetes.io/unschedulable`, with a `NoSchedule` effect. **Nothing happens to the running Pods:** `NoSchedule` means no new Pods will be scheduled on the tainted node unless they have a matching toleration, and Pods currently running on the node are not evicted [source: [k8s-docs-taints-tolerations-2026-08-23](#)].

Note what the question asks and does not ask. It asks you to name a taint whose *effect matches*, which you can do from Chapter 7 alone. What `cordons` writes is a spec field — cordoned nodes are marked `Unschedulable` in their spec [source: [k8s-docs-node-status-2026-08-24](#)] — and the relationship between that field and the built-in taint is left where the documentation leaves it.

Common wrong turn: answering that the Pods are evicted, which is `NoExecute`'s behavior, not `NoSchedule`'s.

2. They skipped `kubectl drain`. `cordons` stops new Pods arriving and deliberately leaves running Pods alone; the three services were still on that node when the machine went down.

If you got this wrong, you got it wrong for a sensible reason. "Take a node out of service" reads as one action, and in most operational vocabularies it is one action. Kubernetes splits it in two because there are real situations where you want arrivals stopped and the current occupants left in place: `cordons` is documented as a preparatory step before a node reboot or other maintenance, and draining is disruptive, so you may want to `cordons` now and `drain` during a window. The split is a feature. It is also a trap, and the fix is one more command before the reboot.

3. **Unknown.** The node controller has not heard from the node within the `node-monitor-grace-period`. `False` would mean something different and more specific: the node reported *itself* unhealthy and not

accepting Pods. Unknown is the control plane saying it has no current information, which is consistent with a dead machine and equally consistent with a network partition.

The other four conditions: **DiskPressure**, **MemoryPressure**, **PIDPressure**, **NetworkUnavailable**. Three of those four are pressure conditions — disk, memory, processes — which is the easiest way to hold them; the fourth is about configuration rather than exhaustion.

Common wrong turns: `False` (the intuitive answer, and the wrong one) and `NotReady` (which is how it is often displayed in summary output, but is not one of the three condition values). If you offered `SchedulingDisabled` among the four, re-read §4's Snag: that string is displayed by command-line tools and is not a Condition in the API.

4. A container runtime — containerd or CRI-O — and Kubernetes talks to it through the **CRI**, the Container Runtime Interface.

Six chapters back, Chapter 2 drew this as an interface boundary and it was, at the time, an architectural point. This is the moment it becomes an operational one: the officially supported bootstrapper will build you a control plane, and the thing that runs containers still has to be there, because it lives on the other side of the line.

5. The first engineer operates their own control plane (self-hosted); the second is on a managed platform where the provider holds the upgrade calendar. The further duty this chapter can defend: **etcd backup**, which sits with whoever operates the control plane.

Notice that both engineers are now running 1.36. The version is not what differs. The *ownership of the decision* is.

A note on scope: it is tempting to extend the list — certificates, machine maintenance, capacity. Those may or may not move depending on the provider, and no Kubernetes documentation defines a shared-responsibility split, so this book asserts only the two duties the architecture itself decides.

How'd you do?

5/5: You own the node lifecycle. Take the recommended break here; §6 deserves a fresh start. **3–4:** Good. If item 2 was a miss, re-read §4's Navigational Hazards before continuing; it is the one item in this chapter with an operational cost attached. **0–2:** Re-read §4, about fifteen minutes, with Figure 8.4 open. Then take the break. §6 is the densest section in this book and it is not the place to arrive tired.

Checkpoint: you've now mastered ✓ How Node objects come into existence, and that a kubelet joins by writing one ✓ `cordon` / `drain` / `uncordon`, and which two the maintenance sequence needs ✓ The five node conditions, and why Ready has three values ✓ Two heartbeat forms, and the control loop watching them ✓ Capacity, Allocatable, and the two reservations that separate them □ Which versions are permitted to disagree (next)

§6 — Versions That Are Allowed to Disagree

This section is a table. There is no honest way around that, and printing the table is the wrong way to teach it: a table you memorized in August is a table you have half-lost by October.

So take the rule first, before any numbers.

Nothing in the cluster may be newer than the API server it talks to.

Every component in a Kubernetes cluster is a client of one door. Chapter 3 established that; §2 built three gates on it. A client that is *newer* than its server is a client that may ask for things the server has never heard of: new fields, new resources, new behavior that does not exist on the other end of the connection. That single sentence generates three of the five rows below. The numbers are then not five unrelated facts; they are the *sizes of the windows*.

The vocabulary

Kubernetes versions are expressed as $x.y.z$, where x is the major version, y is the minor version, and z is the patch version, following Semantic Versioning terminology [source: k8s-version-skew-policy-2026-08-23].

The Kubernetes project maintains release branches for the most recent **three** minor releases [source: k8s-version-skew-policy-2026-08-23]. Kubernetes 1.19 and newer receive approximately one year of patch support; 1.18 and older received approximately nine months [source: k8s-releases-cadence-2026-08-23]. Applicable fixes, including security fixes, may be backported to those three release branches depending on severity and feasibility [source: k8s-version-skew-policy-2026-08-23].

The cadence, which makes the branch count make sense

Since 2021 the project ships **three minor releases per year**, approximately every fifteen weeks, each following a release cycle led by a SIG Release team; patch releases are cut monthly from the supported branches [source: k8s-releases-cadence-2026-08-23].

Now put the two facts beside each other, because neither is memorable alone and together they are almost self-evident: three releases a year, across three supported branches, is roughly a year of coverage, which is exactly the patch-support figure. The three-branch rule is not an arbitrary number somebody picked. It is what "about a year of support" costs at this release cadence.

[cross-bearing: see Ch 17 §8 — SIG Release, KEPs, and how a release gets made]. You will meet the cadence again there, inside the governance material that explains why it is what it is. Make the connection now; it is the cheapest insurance this chapter offers against forgetting the number.

📌 **Snag:** "Kubernetes supports the last two releases" is a common half-memory and it is wrong. It is **three**.

The rules

Dead Reckoning: The supported version skew, stated flat.

Component	Rule
kube-apiserver	In highly-available clusters, the newest and oldest kube-apiserver instances must be within one minor version of each other
kubelet	Must not be newer than kube-apiserver. May be up to three minor versions older. (A kubelet older than 1.25 may only be up to two minor versions older.) With kube-apiserver at 1.36: kubelet at 1.36, 1.35, 1.34 or 1.33
kube-proxy	Must not be newer than kube-apiserver. May be up to three minor versions older than kube-apiserver, and up to three minor versions older <i>or newer</i> than the kubelet instance it runs alongside
kube-controller-manager, kube-scheduler, cloud-controller-manager	Must not be newer than the kube-apiserver instances they communicate with. Expected to match the kube-apiserver minor version, but may be up to one minor version older, to allow live upgrades
kubectrl	Supported within one minor version, older or newer , of kube-apiserver. With kube-apiserver at 1.36: kubectrl at 1.37, 1.36 or 1.35

(All five rules and both worked examples: [source: k8s-version-skew-policy-2026-08-23].)

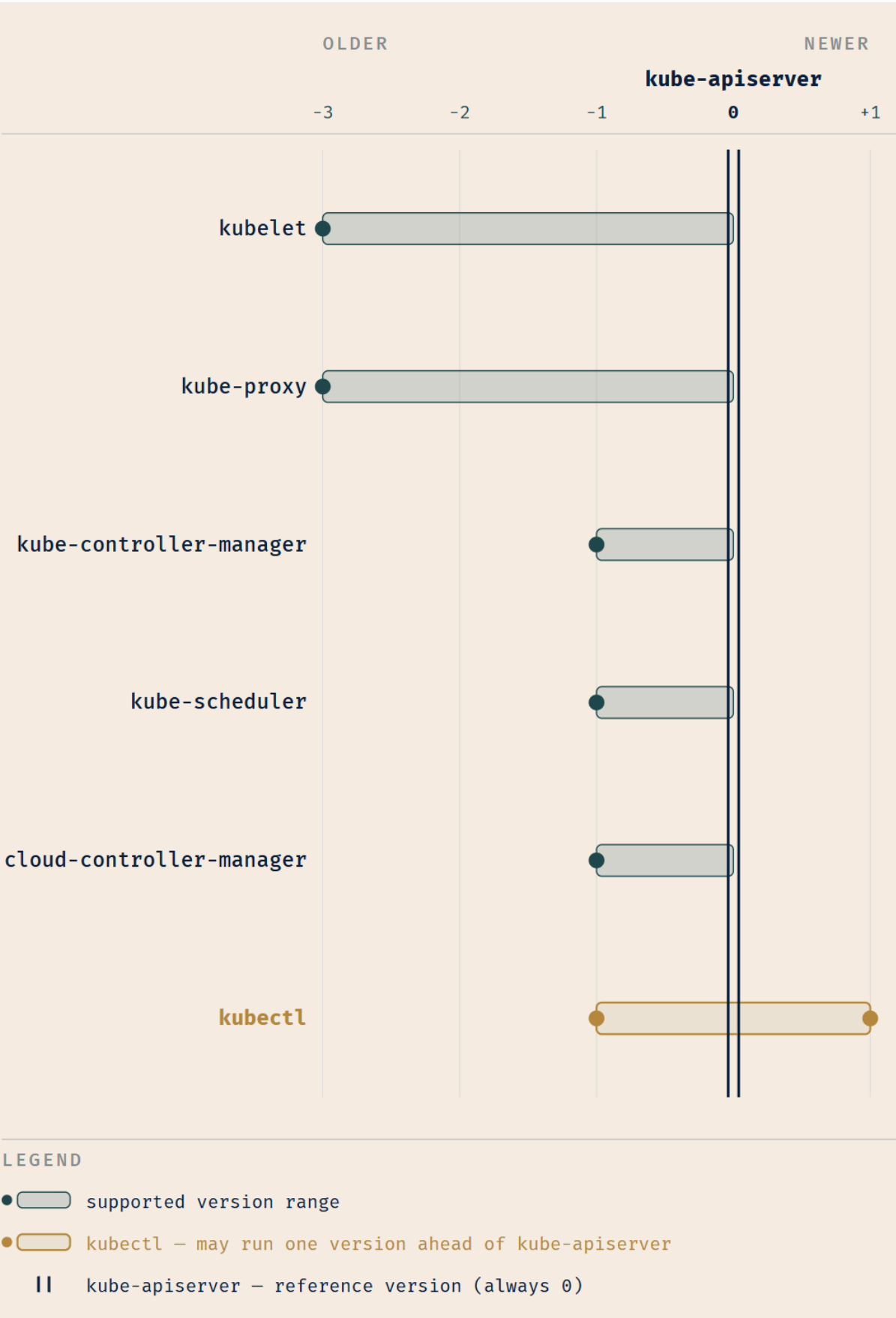


Figure 8.5 — The double line at 0 is the API server's minor version. For every component but one it is a wall: bars extend to the left and stop dead. `kubect1` is the single bar that crosses to the right. The HA kube-apiserver rule is deliberately absent — it is a mutual bound *between* API servers, not a bound relative to one, so it has no bar to draw. The axis is relative; the rules do not change when the version numbers do.

The exception, which is where the exam points are

`kubect1` is the only entry in that table permitted to be *newer* than the API server.

Everything in the first half of this section built an intuition — never newer, never newer, never newer — and for exactly one component that intuition is wrong. Worse, it is wrong in a way that looks like carelessness rather than like a rule, so candidates who half-remember the kubelet's generous three-minor window reach for it when the question is about `kubect1`, and lose the point twice over: wrong number, wrong direction.

There is an explanation, and this book offers it as its own reasoning rather than as documented rationale — the version-skew policy states the rules without saying why. `kubect1` is a **user tool that addresses the cluster from outside**, not a component running inside it. It is the thing on your laptop. It is not participating in the cluster's internal consistency, so its compatibility window is about human convenience: you should be able to run one `kubect1` against a fleet of clusters that are not all on the same release. That is a good reason for the only symmetric window in the table, and it is worth holding onto — but hold it as a memory aid, not as a fact the exam can ask you to reproduce.

☆ **Fixed Point:** Nothing may be newer than the API server. kubelet may be up to **three** minors older. `kubect1` is the single exception, in both directions, at one minor.

⚠ **Navigational Hazards:** The kubelet rule and the `kubect1` rule are different rules, and candidates routinely apply the first to the second. **kubelet: three minors, older only. `kubect1`: one minor, either direction.** They are not the same number and they are not the same shape. If a question gives you a `kubect1` version, the number you want is one, and the direction is both.

🧠 **Mnemonic:** *Three back, three a year, three branches.* The kubelet's window, the release cadence, and the number of supported minor releases are all three. That is a coincidence, and it is a useful one: three of this chapter's numbers collapse into a single digit, which leaves you only one other number to hold — `kubect1`'s one, in both directions.

What the rule implies about upgrade order

The order falls out of the rule and does not need memorizing separately. If nothing may be newer than the API server, then the API server must be upgraded first; everything else follows behind it, within its permitted window. No cached documentation states an upgrade order in those words — this is a derivation from the tagged rule above, and it is offered as one.

That is as far as this book goes. Writing an upgrade runbook is not in the curriculum, and the procedure differs by how the cluster was built anyway — which, as §5 noted, may not be a decision that was ever yours [*cross-bearing: see Ch 8 §5 — whose watch the upgrade stands on*].

A note on the numbers in this section. The rules above are stable. The specific releases are not: the roster supported at the time this book's sources were captured was 1.36, 1.35 and 1.34 [source: k8s-releases-cadence-2026-08-23], and by the time you sit the exam it will be a different three. Learn the rule and treat the numbers as an illustration of it. Nothing in this book's practice questions turns on which minor version is current, and nothing in the exam should either.

[*cross-bearing: see Ch 13 §6 — version skew as a cause of failures you will otherwise misdiagnose*]. This material returns there, deliberately, in a form where you have to use it rather than recite it.

..1 §7 — The One Backup That Matters

Short section. It is also the only material in this chapter where the consequence of getting it wrong cannot be undone afterwards, so read it at half speed.

Chapter 3 introduced etcd as a consistent and highly-available key value store used as Kubernetes' backing store for all cluster data [source: k8s-docs-etcd-access-control-2026-08-24], and pointed here for what to do about that [*cross-bearing: see Ch 3 §2 — etcd, the cluster's memory*].

All Kubernetes objects are stored in etcd [source: k8s-docs-etcd-backup-2026-08-23]. Every Deployment you have written in this book. Every ConfigMap and Secret. Every Service. Every Node object, including the ones a kubelet wrote itself in §4. All of it is one datastore's contents.

Which is why: periodically backing up the etcd cluster data is important to recover Kubernetes clusters under disaster scenarios, such as **losing all control plane nodes** [source: k8s-docs-etcd-backup-2026-08-23].

Sit with that scenario for a second, because it is more specific and more survivable than "the cluster died." Losing every control-plane node does not stop your worker nodes. The kubelets keep running the containers they were last told to run; traffic keeps being served. What you have lost is not the workloads. It is the entire record of *intent*: every declaration that says what should be running, which is the only thing that lets the cluster put itself back together when something changes.

The mechanics

Backing up an etcd cluster can be accomplished in two ways: a built-in snapshot, `etcdctl snapshot save backup.db`, or a volume snapshot of etcd's storage [source: k8s-docs-etcd-backup-2026-08-23]. The `etcdctl` form takes optional `--endpoints`, `--cacert`, `--cert` and `--key` flags for a TLS-protected cluster [source: k8s-docs-etcd-backup-2026-08-23].

Restoring uses `etcdctl snapshot restore`, which operates directly on the etcd data files; after a restore, the control plane components are restarted against the restored data directory [source: k8s-docs-etcd-backup-2026-08-23].

🔧 **Closer Look:** Restore is not a command you run against a running cluster. `etcdctl snapshot restore` operates on the data files directly, and the control-plane components come back up against the restored directory afterwards. That is why restoring from a snapshot is a maintenance *event*, with a window, a plan, and somebody watching, rather than an operation you slip in between meetings.

The fact that matters more than the commands

Two sentences, and read them together rather than separately.

The snapshot file contains all the Kubernetes state and critical information; keep it encrypted and store it outside the control plane nodes [source: k8s-docs-etcd-backup-2026-08-23].

Access to etcd is equivalent to root permission in the cluster, so ideally only the API server should have access to it [source: k8s-docs-etcd-access-control-2026-08-24].

Put them side by side and they say something you should feel rather than merely file away: **an unencrypted etcd snapshot sitting on a control-plane node is simultaneously your only disaster recovery and a complete compromise of the cluster, waiting for someone to copy it.** Not a credential *for* the cluster. Root *in* the cluster, in one file, at rest.

☆ **Fixed Point:** Every object you have ever created lives in etcd. **Access to etcd is equivalent to root permission in the cluster.** A snapshot is therefore both your only route back from disaster and the most dangerous single file you own.

📦 **Worth Securing:** "Store it outside the control plane nodes" is the entire point of that sentence, and it is the half people skip. A snapshot that lives only on the machines it exists to protect you against losing is not a backup. It is a copy that goes down with the original — the maritime word for which is *ballast*, not *lifeboat*.

Notice also that the second sentence is the single-door architecture stated from the other side. §2 said every request terminates at the API server. This says only the API server should reach the store behind it. Same claim, different direction [*cross-bearing: see Ch 8 §2 — one door, three gates*].

And whose job is this? §5's answer applies: whoever operates the control plane holds it. On a managed platform you may not be able to reach etcd at all. On a self-hosted cluster it is yours, and it is the one duty in this chapter where doing it late is not the same as doing it.

[*cross-bearing: see Ch 12 §4 — Secrets, and encryption at rest*]. Chapter 12 covers why "keep it encrypted" is not paranoia. It is the reason that clause is in the sentence.

🕒 Taking Your Bearings #3 — The Two Things You Cannot Improvise

Five questions on §6 and §7. These carry the highest concentration of directly examinable material in the chapter.

1. ... Your cluster's API servers are at 1.36. For each of the following, say whether it is supported: (a) a kubelet at 1.33; (b) a kubelet at 1.37; (c) a kubect1 at 1.37; (d) a kube-scheduler at 1.34.
2. ... State the one rule that generates three of the five rows of the skew table. Then name the two rows it does *not* generate, and say why each is different.
3. ... How many minor releases does the project maintain release branches for? Roughly how long is patch support for 1.19 and newer? Roughly how many minor releases ship per year? Then say why those three numbers are consistent with each other.
4. ... You lose every control-plane node. Your worker nodes are untouched and your applications are still serving traffic. What have you actually lost — and what single artifact would let you get it back?
5. ... An etcd snapshot is sitting, unencrypted, on one of your control-plane nodes. Give two *separate* reasons this is wrong.

Answers + explanations

1. (a) **Supported** — kubelet may be up to three minor versions older than kube-apiserver. (b) **Not supported** — kubelet must never be newer than kube-apiserver. (c) **Supported** — kubect1 is permitted within one minor version in either direction, and this is the only component for which "newer" is allowed. (d) **Not supported** — kube-scheduler may be at most one minor version older than the API servers it talks to, and 1.34 is two behind 1.36.

Common wrong turn: marking (c) unsupported. That is the kubelet's never-newer rule applied to kubect1, and it is this chapter's most durable error — it costs you twice, because it is the wrong number *and* the wrong direction. If you missed only (c), spend five minutes with Figure 8.5; the picture holds the exception better than the table does.

Note also what this item is testing. It is not the version numbers; the roster will be different by the time you sit the exam. It is four rules, applied. Redo the question with the API servers at any minor version you like and the four answers stay in the same places.

2. The rule: **nothing may be newer than the API server**. It generates the kubelet row, the kube-proxy row, and the controller-manager/scheduler/cloud-controller-manager row — three of five.

The two rows it does not generate, and they are different for different reasons:

- **kubect1**, because it is a user tool addressing the cluster from outside rather than a component running inside it. It is not participating in the cluster's internal consistency, so its window is symmetric — one release either way.

- **The HA kube-apiserver row**, because it is not a bound *relative to* the API server at all. It is a mutual bound *between* API servers: the newest and oldest instances must be within one minor version of each other. There is no "the API server" to be measured against when the rule is about several of them.

Common wrong turn: naming kubect1 and stopping. The HA row is the one most readers miss, precisely because it does not look like an exception — it looks like another row, until you notice it has no fixed point to measure from. If you could answer both halves, you derived the table rather than memorizing it, and that is the version of this knowledge that survives to exam day.

3. Three release branches. **Approximately one year** of patch support for 1.19 and newer. **Approximately three** minor releases per year, roughly every fifteen weeks.

They are consistent because three releases a year across three maintained branches is about twelve months of coverage: the branch count is the support window, expressed in releases instead of months.

Common wrong turn: "the last two releases." That is the standard half-memory for this fact and it is wrong in the one direction that costs a point — it is three, and the three-year-three-branch coincidence in §6's Mnemonic exists to keep it there.

You will meet the cadence again in Chapter 17, inside the project's release-governance material, where SIG Release and the KEP process explain where those fifteen weeks go [*cross-bearing: see Ch 17 §8 — SIG Release and the release cycle*]. That is not a throwaway pointer: this trio of numbers is among the most forgettable material in this book, and the Chapter 17 pass is when it gets its second look.

4. You have lost **the cluster's entire record of intent**, every object, since all Kubernetes objects are stored in etcd. Not the running workloads: kubelets keep running what they were last told to run, and traffic keeps flowing. What is gone is every declaration of what *should* be running, which means nothing can be reconciled, changed, scheduled, healed or scaled.

The artifact that brings it back is **an etcd snapshot**.

The scenario is constructed so you have to notice that "running workloads" and "cluster state" are different things. Plenty of people's first answer to "we lost the control plane" is "the applications are down," and for a little while, they are not.

5. First: it is not stored outside the nodes it protects against losing. The documented instruction is to store it outside the control plane nodes, precisely because a snapshot that lives only on the machines whose loss it insures against does not survive the event it exists for.

Second: access to etcd is equivalent to root permission in the cluster, and the snapshot contains all the Kubernetes state and critical information. An unencrypted snapshot is therefore a complete compromise of the cluster in a single readable file — every Secret, every configuration, everything — available to anyone who can read that disk.

Two reasons, two different failure modes: one about availability, one about confidentiality. A snapshot can fail you by not being there, or by being read.

How'd you do?

5/5: You have the two duties that cannot be improvised. §8 is a short read and then you are done with Part II. **3–4:** Review your misses now rather than later. If item 1 or 2 was among them, spend ten minutes with Figure 8.5; the picture holds better than the table. **0–2:** Re-read §6 with the derivation in mind: one rule, three rows, two exceptions for two different reasons. Do not try to memorize five rows. Then re-read §7, which is five minutes and has no numbers in it at all.

Checkpoint: you've now mastered ✓ The rule that generates the skew table, and the two rows that sit outside it ✓ Three branches, one year, three releases a year — and why those agree ✓ What losing every control-plane node does and does not cost you ✓ Why an etcd snapshot is both your recovery and your largest single risk □ Why none of this was new (next, and it is the point of the chapter)

§8 — Rules, or Consequences

You were asked to keep score. Here is the claim.

You did not learn a single new mechanism in this chapter.

Every administrative act in it is a write through the one door you met in Chapter 3, reconciled by a controller you had already been introduced to. Take them in order.

§1. `kubectl` is a client of the API server. Not a management console, not a privileged back channel: a command-line client that assembles an HTTP request and sends it to the same endpoint your Pods use. Chapter 3's single door, addressed from a terminal.

§2. The three gates are not a subsystem bolted onto the side. They are what the single door *does* before it writes anything down — and the documentation says so in one sentence: once a request passes all admission controllers, it is validated and then written to the object store [source: k8s-docs-controlling-access-2026-08-24]. Chapter 3 told you every request terminates at the API server; §2 is the "and then what" of that sentence. Even the audit log is kept inside the same component [source: k8s-docs-audit-2026-08-24].

§3. A ResourceQuota is an object. You apply it exactly as you apply a Deployment. It takes effect because an admission controller reads it when other requests arrive [source: k8s-docs-admission-controllers-2026-08-24], which means a quota is a **declaration that changes what other declarations are permitted to say**. Nothing about it is a special-case enforcement engine.

§4, and this is the one that should land. `kubectl cordon` has no private channel to the node. It does not connect to the machine. It writes a field on a Node object through the API server — cordoned nodes are marked Unschedulable in their spec [source: k8s-docs-node-status-2026-08-24] — and spec is the half of

an object that *you* declare, as against *status*, which the system reports back. Chapter 4 taught you that distinction [*cross-bearing: see Ch 4 §2 — spec and status*], and here it does real work: cordoning is an act of *declaring intent about a machine*, which is why it lives in `spec` and why `SchedulingDisabled`, the thing your terminal prints, is not a `Condition` in the API at all.

The scheduler then does what the scheduler always does: it checks taints, not node conditions, when it makes scheduling decisions [source: `k8s-docs-taints-tolerations-depth-2026-08-24`], and Chapter 7 already showed you a built-in `node.kubernetes.io/unschedulable` taint with a `NoSchedule` effect sitting in that table [source: `k8s-docs-daemonset-2026-08-24`]. Nothing here is a new mechanism. It is the object model and the scheduler you already have, with an operator's hand on the `spec` field.

§4 again. The node controller observes heartbeats, compares them against what it expects, and acts: updating a condition, then evicting. It is a control loop — the same loop, at the same altitude as Chapter 6's — and you could have predicted its structure without being told.

§6. The skew rules are the compatibility contract of that same one door. Read the table again with that in mind and three of the five rows are answering one question: *which clients will this door accept?*

§7. `etcd` is what is behind the door, and the reason only the API server should reach it is the same reason there is only one door in the first place.

☼ **Zenith:** One door, and behind it controllers you have already met. Everything in this chapter is a write to the first, reconciled by the second.

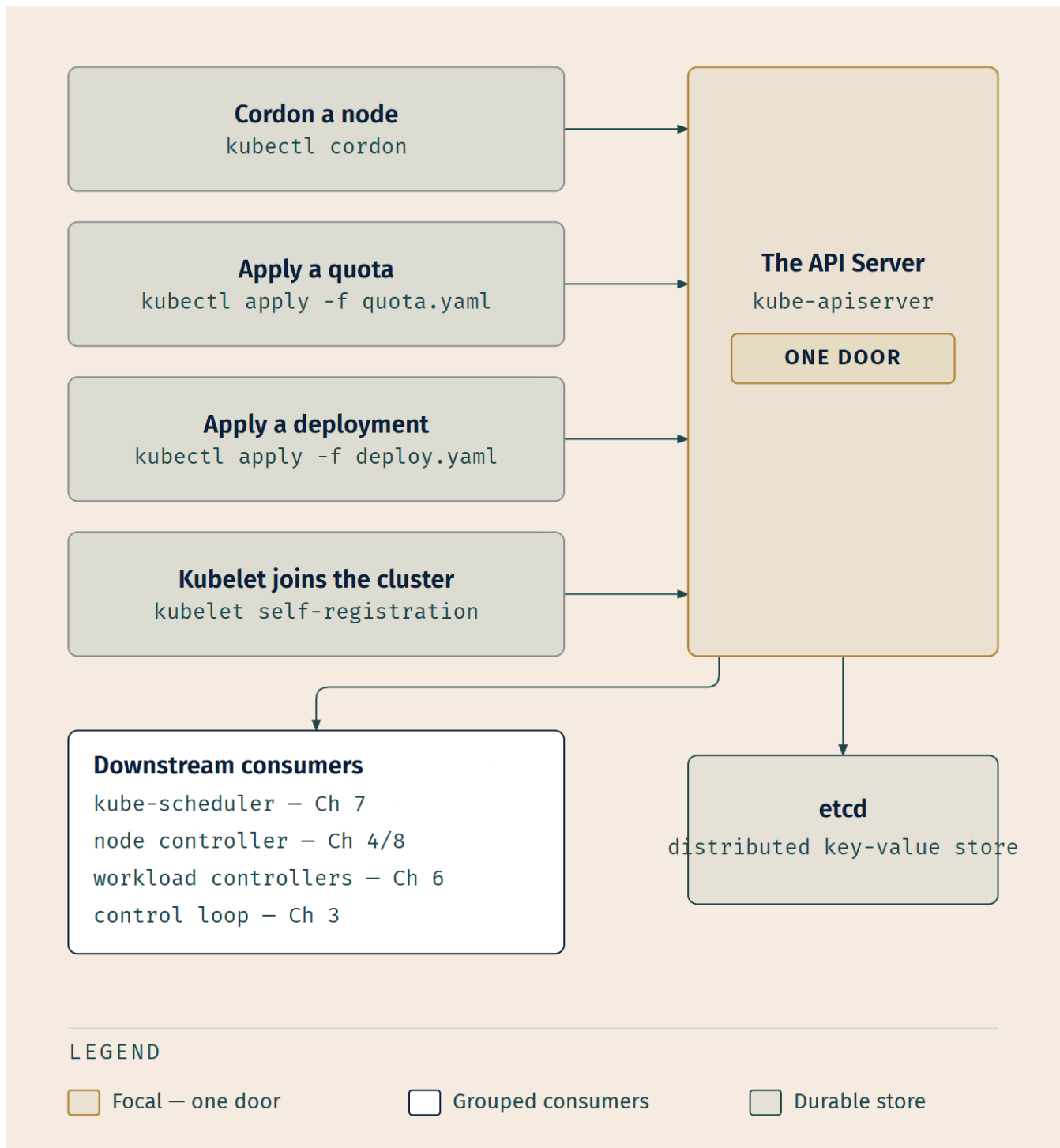


Figure 8.6 — What to notice is the chapter number beside each controller. This is Chapter 3's architecture diagram, unchanged, except that you have now put your own hands on the left-hand side of it, and every mechanism on the right-hand side is one you were taught somewhere in Part II. What to notice second is that no arrow on the left reaches a controller directly. There are no side channels.

Rules, or consequences

Chapter 7 closed by saying this chapter is where the rules turn into consequences. That is the sentence to land on.

A list of administrative rules is among the least memorable material any study guide can put in front of you, and this chapter contained a great many of them. A set of consequences of a single architecture is a different thing entirely. If you can hold **one door, and controllers behind it**, you can regenerate most of this chapter without ever having memorized it, and you can go further than that. When you meet a Kubernetes administrative feature this book does not cover, the two questions that will get you most of the way are already in your hands.

🔒 **Worth Securing:** Faced with an unfamiliar Kubernetes administrative feature, ask two questions in this order: **what object does it write**, and **what controller is watching that object?** Those two questions answer most of them, and where they do not, they at least tell you what kind of thing you are looking at.

Where the claim overreaches

One honest correction, because the claim as stated is slightly too neat and you would notice.

§5 and §6 are not consequences of the architecture. Which bootstrap tool is officially supported, how many release branches the project maintains, and how far a kubelet may lag: these are facts about a *project*, made by people in meetings, and no amount of understanding the single-door model will let you derive them. They have to be learned. That is exactly why §6 flagged them as memorization and gave you a derivation for three of the five rows anyway. The parts that can be reasoned about should be reasoned about, and the residue should be admitted as residue.

Chapter 4 §6 established this book's habit of narrowing a claim until it is true rather than leaving it broad and impressive [*cross-bearing: see Ch 4 §6 — the habit of narrowing a claim until it is true*]. So: **every administrative act in this chapter is a write through one door, reconciled by a controller you already know. The project's policies are not, and those are the parts you memorize.** That version survives contact with the exam.

Exam Alert! 🚨

High-priority topics, in descending order of confidence:

1. **The three gates, in order** — authentication, then authorization, then admission control.
2. **Only admission can change the request.** The other two accept or refuse.
3. **kubelet may be up to three minor versions older than kube-apiserver, and must never be newer.**
4. **kubect1 is the only component permitted to be newer** — within one minor version, in either direction.

5. **Three supported minor releases**, approximately one year of patch support (1.19+), approximately three minor releases per year.
6. **cordon stops arrivals; drain evicts**. A cordoned node is not an empty node.
7. **Ready is three-valued**. `Unknown` means the control plane has not heard from the node, which is not the same claim as `False`.
8. **ResourceQuota is namespace-aggregate; LimitRange is per-object** — and a quota'd namespace forces every new Pod to declare requests or limits.
9. **Upgrade the API server first**, because nothing may be newer than it.
10. `kubectl [command] [TYPE] [NAME] [flags]` — types case-insensitive and abbreviable, names case-sensitive.
11. **All objects live in etcd, and access to etcd is equivalent to root permission in the cluster**.

Common traps. Each of these catches real candidates. None of them is dressed up with an invented statistic, because nobody publishes one.

Trap	The correct understanding
"kubelet must match the API server version"	It must not be <i>newer</i> . It may be up to three minors older. Matching is not required
Applying the kubelet skew rule to kubect1	Different rule, different number, different shape. kubelet: three, older only. kubect1: one, either direction
"Kubernetes supports the last two minor releases"	Three
"Everything lives in a namespace"	Nodes, PersistentVolumes and StorageClasses do not. This is why no quota can cap a team's Node consumption
"cordon takes the node out of service"	It stops new Pods. It does not move the ones already there. That is <i>drain</i>
"Authorization and admission are two words for the same check"	Authorization is any-module-approves and looks at identity, verb and object; admission is any-module-rejects and looks at the object's contents
"Ready: False is what an unreachable node reports"	An unreachable node shows Unknown. False is a node that reported <i>itself</i> unhealthy
"SchedulingDisabled is a node Condition"	It is a display string. Cordoned nodes are marked Unschedulable in their spec
"A ResourceQuota limits how big any one Pod can be"	That is LimitRange. A quota is an aggregate ceiling on the namespace
"A Pod with no resource fields is always valid"	Not in a namespace with a cpu or memory quota. Declare requests or limits, or let a LimitRange declare them for you
"kubect1 inside a Pod behaves as it does on your laptop"	It detects in-cluster conditions, authenticates as the ServiceAccount, and defaults to that ServiceAccount's namespace
"Resource names are case-insensitive, because resource types are"	Types are. Names are not
"An etcd snapshot on the control-plane node is a backup"	It is a copy that goes down with the thing it was protecting — and, unencrypted, a root-equivalent credential

Practice Questions

Eighteen questions. Several require two sections at once; that is deliberate, because the exam does not organize itself by chapter section either. Answers and full explanations follow the set.

1. Which statement about `kubect1` command syntax is correct?

A) Both resource types and resource names are case-insensitive B) Resource types are case-insensitive and may be given in singular, plural or abbreviated form; resource names are case-sensitive C) Resource

names are case-insensitive; resource types must be given in the plural D) Both resource types and resource names are case-sensitive

2. In what order does a request pass the API server's access-control gates, and which of them can result in the request being modified rather than accepted or rejected?

A) Authorization → authentication → admission; only admission can modify B) Authentication → authorization → admission; only authorization can modify C) Authentication → authorization → admission; only admission can modify D) Authentication → authorization → admission; any of the three can modify

3. A request to create a Deployment is refused. Investigation confirms that the client's identity was valid and that the identity was permitted to create Deployments in that namespace. Which gate refused it?

A) Authentication — the credential must have expired mid-request B) Authorization — permission to create a Deployment does not imply permission to create this one C) Admission control — the request's contents were evaluated and found unacceptable D) None; a request that passes authentication and authorization is always persisted

4. A developer submits a Pod whose resource requests would push their namespace past its ResourceQuota. The Pod is rejected. Which gate rejected it, and would the outcome be different if a cluster administrator had submitted the identical manifest to the same namespace?

A) Authorization; yes — an administrator has broader permissions, so the quota would not apply B) Admission control; no — the quota constrains the namespace, not the submitter C) Admission control; yes — the quota applies to the namespace's owner, and an administrator's requests are attributed to the cluster rather than the namespace D) Admission control; yes — quota enforcement is skipped for cluster-admin identities

5. Which pairing correctly describes ResourceQuota and LimitRange?

A) ResourceQuota bounds an individual Pod's resources; LimitRange caps a namespace's total B) Both cap a namespace's aggregate consumption; LimitRange additionally supplies defaults C) ResourceQuota caps a namespace's aggregate consumption; LimitRange constrains individual objects and supplies defaults D) ResourceQuota applies to cluster-scoped objects; LimitRange applies to namespaced objects

6. [retrieval: ch4] A platform team wants to cap the number of Nodes that a particular application team may consume. Why can they not do this with a ResourceQuota?

A) ResourceQuota cannot count objects, only compute resources B) Nodes are not namespaced objects, and a ResourceQuota constrains a namespace C) ResourceQuota is evaluated at the authorization gate, which has no visibility into Nodes D) They can; a ResourceQuota in the kube-system namespace applies cluster-wide

7. [retrieval: ch5] A namespace has a LimitRange that supplies a default CPU request. A developer submits a Pod manifest that declares no resource fields at all. The Pod is accepted, with the default filled

in. What has changed about where this Pod can be placed, compared with the manifest as written?

- A) Nothing — defaults are recorded for reporting but are not used in placement decisions
- B) It can now be placed on fewer nodes, because it now books capacity against Allocatable that it did not book before
- C) It can now be placed on more nodes, because a declared request lets the scheduler relax its filtering
- D) Nothing — placement is decided from limits, not requests

8. You are told: "take node worker-3 out of service and clear it before the maintenance window." Using only the four-slot grammar, which commands accomplish this, and in what order?

- A) `kubectl drain worker-3`, then `kubectl cordon worker-3`
- B) `kubectl cordon worker-3`, then `kubectl drain worker-3`
- C) `kubectl cordon worker-3` alone — draining is implied
- D) `kubectl uncordon worker-3`, then `kubectl drain worker-3`

9. A node stops responding to the control plane. What value does its Ready condition take, and what does that value assert?

- A) `False` — the node has reported that it is unhealthy and not accepting Pods
- B) `NotReady` — a distinct value used only for unreachable nodes
- C) `Unknown` — the node controller has not heard from the node within the grace period
- D) `SchedulingDisabled` — the node controller has marked it ineligible for new Pods

10. [retrieval: ch4] When you run `kubectl cordon node-7`, which part of the Node object is written, and how can you tell?

- A) `status` — cordoning reports an observed condition of the machine
- B) `spec` — it is a statement of desired state, and `status` is written by the system rather than by you
- C) `metadata` — cordoning applies a label to the Node object
- D) Neither; `cordon` bypasses the object model and signals the kubelet directly

11. [retrieval: ch3] The node controller notices that a node has stopped reporting, updates a condition, and eventually evicts that node's Pods. Name the pattern, and name two earlier components in this book that work the same way.

- A) A webhook; the Pod Security Admission controller and the NodeRestriction plugin
- B) A control loop; the Deployment controller and the scheduler's watch on unscheduled Pods
- C) A daemon; the kubelet and kube-proxy
- D) A reconciliation batch job; the DaemonSet controller and etcd's compaction

12. Which statement about cluster bootstrap tooling is correct?

- A) `kubeadm` is intended for local learning environments; `kind` is the officially supported production bootstrapper
- B) `kubeadm` is the officially supported tool for creating clusters, installing the control plane and joining nodes; `kind` and `minikube` are for local learning environments
- C) `k3s` is the officially supported tool for creating clusters; `kubeadm` is a lightweight distribution
- D) `kubeadm` is the officially supported tool for creating clusters; it installs a container runtime on each node as part of joining them

13. Which pairing correctly separates a duty that sits with whoever operates the control plane from one that does not?

A) Taking etcd backups does not sit with the control-plane operator; installing a container runtime on each node does B) Choosing which container images an application runs sits with the control-plane operator; taking etcd backups does not C) Taking etcd backups sits with the control-plane operator; choosing which container images an application runs does not D) Installing a container runtime on each node sits with the control-plane operator; taking etcd backups does not

14. Your cluster's API servers are at 1.36. Which of the following combinations is **not** supported?

A) kubelet at 1.33 B) kubect1 at 1.37 C) kube-scheduler at 1.35 D) kube-proxy at 1.37

15. Which component is permitted to run at a *newer* minor version than kube-apiserver, and within what window?

A) kubelet, up to three minor versions B) kube-proxy, up to three minor versions C) kubect1, within one minor version D) None; no component may ever be newer than kube-apiserver

16. In what order must a cluster's components be upgraded, and why?

A) kubelet first, because the nodes must be ready before the control plane restarts B) kube-apiserver first, because nothing may be newer than it C) etcd first, because the API server depends on it for storage D) All components simultaneously, because no version skew is permitted between them

17. [retrieval: ch6] You drain a node for maintenance. Every Pod is evicted except one, which is still running afterwards. Chapter 6 introduced the controller responsible. Name it, and say what its Pods carry that lets them stay.

A) A StatefulSet; its Pods hold a stable ordinal identity that the eviction path preserves B) A Deployment with a node selector; a placement constraint makes its Pods ineligible for eviction C) A DaemonSet; the controller automatically adds a `node.kubernetes.io/unschedulable` toleration with a `NoSchedule` effect to its Pods D) A Job; Pods running to completion are exempt from eviction until they finish

18. An operations team stores its nightly etcd snapshots, unencrypted, on the primary control-plane node. Which statement best describes the problem?

A) There is no problem, provided the node's filesystem permissions are correct B) The snapshots should be encrypted, but their location is fine because that is where etcd runs C) The snapshots should be stored outside the control-plane nodes and kept encrypted — otherwise they will not survive the disaster they exist for, and access to etcd data is equivalent to root permission in the cluster D) The snapshots should be taken with `etcdctl snapshot save`; `etcdctl` is the restore utility

Answers and Explanations

1. **B.** Resource types are case-insensitive and accept singular, plural or abbreviated forms; names are case-sensitive. **A is wrong** because it flattens the asymmetry in the permissive direction, the single most common form of this error, because the tool's tolerance about types encourages the assumption. **C is wrong** on both counts and additionally invents a plural requirement that does not exist. **D is wrong** in the strict direction: `kubectl get PODS web-01` works perfectly well.
2. **C.** Authentication, then authorization, then admission; only admission can modify. **A is wrong** on order alone — it names the correct modifying gate, so a reader who knows only that admission mutates cannot eliminate it, which is the point of the option. **B is wrong** on the modifying gate: authorization's inputs are the username, the requested action and the object affected, not the object's contents, so it has nothing to modify with. **D** gets the order right and then gives all three gates a power only the third has, which is the error that collapses three distinct gates into one undifferentiated "security check."
3. **C.** Both earlier gates already returned yes, so the refusal came from the one that examines contents — and admission is the only gate that can access the contents of the object being created or modified. **A is wrong**: an unauthenticated request fails at gate one with a 401, and the question states the identity was valid. **B is wrong** because it misdescribes authorization; authorization decides whether an identity may perform an action on a resource, and the question stipulates that it did. **D is wrong** and is the trap for readers still working from a two-gate model: passing authentication and authorization is necessary, not sufficient.
4. **B.** Quota enforcement happens at the admission gate — ResourceQuota is an admission controller that observes the incoming request — and a quota constrains aggregate consumption *per namespace*, regardless of who submitted the request. **A is wrong** twice: wrong gate, and it imports a rule that does not exist, since quota is not an identity-scoped check. **C is wrong** on a subtler misreading: it applies §3's scope hinge backwards, treating a namespaced constraint as if the submitter's identity could relocate the request's scope. Nothing about quota is attributed to a submitter. **D** gets the gate right and then invents an administrator exemption; the appeal of this distractor is that it *feels* like how permissions usually work, which is exactly why it is worth defusing.
5. **C.** Quota provides constraints that limit aggregate resource consumption per namespace; LimitRange constrains the resource allocations you can specify for each applicable object kind, and supplies defaults so Pods declare their requirements. **A is wrong** because it swaps them, which is the only real mistake available in this section. **B is wrong** because it gives both mechanisms namespace-aggregate scope, losing the scope distinction that is the actual content. **D is wrong**: both are namespaced objects, and the scope difference between them is *within* a namespace, not across the namespaced/cluster-scoped boundary.
6. **B. [retrieval: ch4]** Nodes are cluster-scoped, and a ResourceQuota is a statement about a namespace. Chapter 4 established the boundary; §3 turned it into an operational consequence. **A is wrong** as a claim about quota's capabilities — a quota counts objects perfectly well, in the `count/<resource>` form — and, more importantly, misses the point: the obstacle is scope, not counting. **C is wrong** on the gate: quota is

enforced at admission, and authorization has no view of object scope in any case. **D is wrong:** kube-system is a namespace like any other, not a privileged scope from which cluster-wide policy can be issued.

7. B. [retrieval: ch5] Chapter 5 established that when you specify the resource request for containers in a Pod, the kube-scheduler uses this information to decide which node to place the Pod on [source: k8s-docs-resource-management-2026-08-23], and §4 of this chapter established that the scheduler treats Allocatable as the available capacity for Pods and does not over-subscribe it. A Pod that previously declared nothing now books capacity, so nodes that would have accepted it may now fail to fit it. **A is wrong:** the defaulted value is a real field on a real object and is used exactly as if you had written it. **C inverts the effect;** declaring a request adds a constraint, it does not relax one. **D is wrong** on which of requests and limits the scheduler reads.

8. B. cordon first to stop new arrivals, then drain to evict what is already there — cordon is documented as a preparatory step before a node reboot or other maintenance, which is precisely this sequence. Note the grammar: both take the node's name directly, without a preceding TYPE, because the verb already implies the resource type. **A reverses the documented order,** putting the preparatory step after the operation it prepares for. **C is the chapter's headline trap:** cordon does not empty anything, so rebooting after a bare cordon takes down every Pod still aboard — which is a real outage, not just a lost mark. **D begins by making the node schedulable again,** which is the opposite of the instruction; uncordon is the third command, run after maintenance.

9. C. Ready becomes Unknown when the node controller has not heard from the node within the node-monitor-grace-period. **A is the intuitive wrong answer** and misstates the evidence: False means the node *told you* it is unhealthy, which requires the node to be talking. **B is wrong:** NotReady is a display convention in summary output, not one of the condition's three values. **D is wrong** on two counts — SchedulingDisabled is not a Condition in the Kubernetes API at all, and it describes a deliberately cordoned node rather than an unreachable one.

10. B. [retrieval: ch4] Cordoned nodes are marked Unschedulable in their spec, which is exactly what Chapter 4's rule predicts: spec is what you declare, status is what the system reports back, and unschedulability is a decision you made about the machine rather than an observation of it. **A is wrong** for that reason. **C is wrong:** the mechanism is a spec field, not a label. **D is wrong** and is worth rejecting emphatically, because it is precisely the belief §8 exists to dismantle — and note that SchedulingDisabled, the string your terminal prints, is not in the API either, so even the output is not what it appears to be. There are no side channels.

11. B. [retrieval: ch3] Observe, compare, act: it is a control loop, the same pattern as the ReplicaSet controller holding a replica count and the Job controller driving Pods to completion, both in Chapter 6. **A is wrong:** admission plugins act synchronously on inbound requests rather than reconciling observed state against declared state. **C is wrong:** the kubelet and kube-proxy are node agents, and "daemon" describes where they run rather than how they work. **D is wrong** on both the pattern and the examples; nothing here is batched, and etcd compaction is storage maintenance, not reconciliation.

12. B. kubeadm is the officially supported tool for creating clusters, installing the control plane and joining nodes; kind and minikube are the documented local learning tools. **A swaps the two categories.** **C swaps kubeadm and k3s:** k3s is a lightweight distribution, not the official bootstrapper. **D** names the right bootstrapper and then attributes a duty to it that the CRI boundary explicitly excludes: a container runtime (containerd or CRI-O) must be installed on every node, and kubeadm does not supply it. That is the Chapter 2 interface boundary showing up as an operational requirement — the most attractive wrong answer here precisely because it *sounds* like something a complete bootstrapper would do.

13. C. etcd backup sits with whoever operates the control plane; choosing container images is a workload-side decision made by whoever runs the workloads. **A inverts both halves** — it moves the backup off the control plane and puts node-level runtime installation on it. **B is wrong** on the first half: image selection is a Pod-spec author's decision, made by the users who create resources in a namespace, not by the control-plane operator. **D is the sharpest distractor**, because installing a runtime on each node *sounds* like platform work — but it is a node-provisioning duty, and it does not become the control-plane operator's just because both sound infrastructural. The half that gives it away is the second: etcd backup unambiguously belongs to whoever runs the control plane.

14. D. kube-proxy must not be newer than kube-apiserver, so 1.37 against a 1.36 API server is unsupported. **A is supported:** kubelet may be up to three minor versions older. **B is supported:** kubect1 is the one component permitted to be newer, within one minor. **C is supported:** the scheduler may be up to one minor older. Note that B and D differ only in which component is one version ahead, which is the whole of the distinction being tested.

15. C. kubect1, within one minor version in either direction. This book's explanation for the exception — that kubect1 is a user tool addressing the cluster from outside rather than a component inside it — is offered as reasoning rather than as documented rationale; the rule itself is what the policy states. **A is the chapter's most durable error:** the kubelet's three-minor window is generous, but it runs in one direction only. **B applies the same mistake to kube-proxy**, which is also older-only relative to the API server. **D states the general rule correctly and forgets that there is exactly one exception**, which is the answer of a candidate who learned the principle and not the table.

16. B. kube-apiserver first, because nothing may be newer than it, and every other component's window is expressed relative to it. **A inverts the dependency:** a kubelet upgraded ahead of the API server would be newer than its server, which is the one thing the policy forbids outright. **C confuses a runtime dependency with an upgrade constraint** — the API server does store its data in etcd, but the skew policy governs Kubernetes components, and etcd's own upgrade is a separate procedure. **D is wrong** because skew is not merely permitted but *required* by the upgrade process: the one-minor allowance for the controller manager and scheduler exists specifically to allow live upgrades.

17. C. [retrieval: ch6] A DaemonSet. Its Pods carry a `node.kubernetes.io/unschedulable` toleration with a `NoSchedule` effect, added automatically by the DaemonSet controller — the toleration half of this is Chapter 7 §4's [*cross-bearing: see Ch 7 §4 — the DaemonSet controller's automatic tolerations*] — which is why Kubernetes can run DaemonSet Pods on nodes marked unschedulable. **A confuses identity with immunity:** a StatefulSet's ordinal identity governs naming and ordering, not eviction. **B is the most**

plausible wrong answer — it is true that placement constraints affect where a Pod *goes*, and false that they affect whether it can be *evicted*; those are different mechanisms, and Chapter 7's affinity material does nothing here. **D is wrong**: a Job's Pods are evicted like any others, and the Job controller's response is to create replacements elsewhere.

18. C. Two independent failures. A backup stored only on the machines it insures against losing does not survive the event; and access to etcd data is equivalent to root permission in the cluster, so an unencrypted snapshot is a complete compromise in one readable file. **A is wrong** because filesystem permissions do not address either failure; they do not make the file survive the node's loss, and they are a thin defense for something this valuable. **B accepts the location** on a reasoning error: etcd running there is the reason the snapshot must *not* live there. **D inverts the two utilities**: `etcdctl snapshot save` takes the backup and `etcdctl snapshot restore` restores it, and the two names differ by two letters, which is exactly why the confusion is worth having a distractor for.

Chapter Summary

Concept	Remember this
kubectl grammar	kubectl [command] [TYPE] [NAME] [flags]. Types case-insensitive and abbreviable; names case-sensitive
kubeconfig	\$HOME/.kube/config by default; KUBECONFIG or --kubeconfig override it, and the flag wins
kubectl in a Pod	Detects the two env vars plus the ServiceAccount token file; authenticates as the ServiceAccount; defaults to that ServiceAccount's namespace
The three gates	Authentication → authorization → admission. Who, may, and how
Admission's distinction	The only gate that can change the request instead of refusing it — and the only one where <i>any</i> module's rejection is final
Reads	Do not pass admission at all
Auditing	A chronological record of the sequence of actions in a cluster, kept inside the kube-apiserver
ResourceQuota	A ceiling on aggregate consumption per namespace ; violation is 403
The quota consequence	In a namespace with a cpu or memory quota, every new Pod must declare requests or limits
LimitRange	Constrains each applicable object kind , enforces min/max and ratio, and injects defaults at admission — never on running Pods
The scope hinge	You can quota a team. You cannot quota a machine — Nodes are not namespaced
Node registration	kubelet self-registration is the default; a human may also create the Node object
cordon	Stops new Pods. Leaves running Pods entirely alone. Writes .spec.unschedulable
drain	Evicts the Pods cordon left running, via the Eviction API
uncordon	Restores scheduling — a separate step you must run yourself
DaemonSet exception	DaemonSet Pods carry an automatic unschedulable toleration, so they tolerate a cordoned node
Node conditions	Ready, DiskPressure, MemoryPressure, PIDPressure, NetworkUnavailable
Ready: Unknown	The control plane has not heard from the node. Not "the node is broken" — "we cannot tell"
SchedulingDisabled	A display string, not a Condition in the API
Heartbeats	Two forms: .status updates, and Lease objects in kube-node-lease

Concept	Remember this
node controller	Assigns a CIDR at registration, syncs with the cloud provider's machine list, monitors health. A control loop
Capacity vs Allocatable	Capacity is the machine's total; Allocatable is what is available for Pods, after kubeReserved and systemReserved account for the daemons. The scheduler does arithmetic against Allocatable
Bootstrap tooling	kubeadm officially supported for creating clusters; minikube and kind for learning; k3s lightweight
Everywhere	A container runtime — containerd or CRI-O — must be on every node
The generating rule	Nothing may be newer than the API server — three of the five rows
kubelet skew	Never newer; up to three minors older
kubectl skew	One minor, either direction — the sole component-level exception
HA API servers	Newest and oldest within one minor of each other — a mutual bound, not a bound relative to one
Supported releases	Three branches, ~1 year of patch support (1.19+), **~3** minor releases per year
Upgrade order	API server first, because nothing may be newer than it
etcd	Holds all Kubernetes objects. Access to it is equivalent to root in the cluster
Backup	etcdctl snapshot save or a volume snapshot. Keep it encrypted; store it outside the control-plane nodes
Restore	etcdctl snapshot restore, operating on the data files; control-plane components restart against the restored directory
The chapter's claim	One door, and controllers behind it. Every administrative <i>act</i> here is a write through the first, reconciled by the second — though the project's <i>policies</i> still have to be learned

⚓ Safe Harbor

You started this chapter able to describe what should run and where. You end it able to take a machine out of service without dropping a request, stop a team consuming a cluster they share, say which of five components may lag the API server and by how much, and name the one file whose loss you cannot work around.

More usefully, you end it with a method. The two questions from §8 — *what object does it write, and what controller is watching* — will carry you through administrative features this book never mentions, which is a better return than any of the individual facts above.

Part II is complete. Ship, cargo, and company: the container, the cluster, the objects, the Pod, the controllers, the placement, and now the watch. That is the whole of what runs and how it is looked after.

The Voyage Ahead

There is a question Part II has been carefully not asking.

You have spent seven chapters putting workloads onto machines and one chapter standing over the result, and in all of it, nothing has had to *reach* anything else. The Pods were placed. The nodes were managed. The quotas were enforced. Not once did a Pod on worker-3 have to find a Pod on worker-11, or hold an address either of them could rely on, or survive that address changing when the Pod is rescheduled somewhere else forty seconds from now — which, given everything Chapter 6 taught you about controllers, it very well might be.

That last part is the difficulty, and it is worth feeling the shape of it before Chapter 9 answers it. Every Pod gets an address. Pods are also, as Chapter 6 established, disposable: a controller may replace one at any moment, and the replacement is a different Pod. So the cluster contains a large number of network endpoints, each perfectly valid, none of them stable, and applications that need to talk to each other anyway.

Part III is how that works: addresses, the abstraction that makes them survivable, how names resolve, and how anything outside the cluster gets in at all. You have just spent a chapter learning that everything administrative is a write through one door. Networking is where you find out what the cluster does with everything *behind* that door, and it is, by some distance, the part of Kubernetes that most rewards understanding over memorization.

Chapter 9 opens by giving every Pod an address, and then explaining why that is not enough.

"The chart tells you where the harbours are. It does not tell you how the water moves between them — and the water is what you are actually sailing on."

Chapter 9: Every Pod Has an Address

"Flat networks, stable names, and the abstraction that survives churn"

Domain: Container Orchestration (28% of exam) [source: cncf-kcna-curriculum-pdf-2026-08-23] |
Competency: Networking | **Authored allocation:** ~7% — CNCF publishes domain weights only, not competency weights; see front matter **Complexity:** Mixed | **Novelty:** Moderate

Attention Budget

Total time: ~100 minutes | **Recommended:** Split across 2 sessions

Section	Time	Attention Cost	Best Time to Study
\$1 — Four Rules and a Plugin	12 min	High	Peak attention
\$2 — The Address That Doesn't Last	6 min	Low	Anytime
\$3 — Four Ways to Be Reachable	18 min	High	Peak attention
🕒 Taking Your Bearings #1	6 min	Medium	After brief break
\$4 — The List Behind the Name	12 min	Medium	Mid-session
\$5 — When You Don't Want a Single Address	9 min	Medium	Mid-session
🕒 Taking Your Bearings #2	6 min	Medium	After brief break
\$6 — The Component That Makes It Real	7 min	Low	Anytime
\$7 — Names, and Where They Resolve	14 min	High	Peak attention
🕒 Taking Your Bearings #3	6 min	Medium	After brief break
\$8 — A Query With a Name	5 min	Low	Anytime

Attention Cost Key:

- **Low:** Concrete, familiar concepts—study anytime
- **Medium:** New concepts requiring focus—study when alert
- **High:** Abstract or complex material—study at peak attention (morning for most people)

Recommended session split: stop after 🕒 Taking Your Bearings #2. That gives you the model, the abstraction, and the backends in one sitting; the implementation, the names, and the synthesis in the next. It also puts \$3 and \$7, the chapter's two densest recall blocks, in different sessions. Two hard passages, taken on separate watches. That matters more than it sounds like it should.

If you only have 15 minutes: read §3's type ladder and §7's record shapes, then take ☹ Taking Your Bearings #3. On this book's judgment, that is where this chapter's exam points concentrate. §1 is the most important section for understanding what Kubernetes networking actually is, and it is the least likely to be tested directly. Those two facts do not coincide, and there is no point pretending they do.

"A harbor is rebuilt piece by piece until nothing original remains. The name on the chart never moves — and the name is how anyone finds it." — Lodestar Ledgers

☹ Soundings

Before reading this chapter, try these questions. Your score determines how to approach the content — no shame in any score, just different reading strategies.

1. You are running three interchangeable copies of a service. One of them dies and is replaced by a copy at a different address. What does a client need so that nobody has to go and tell it the new address? Name two mechanisms you have seen do that job.
2. Chapter 5 said a Pod has one network namespace shared by all of its containers. Two containers in one Pod both want to listen on port 8080. What happens? And how does either one reach the other?
3. A Deployment performs a rolling update and every Pod is replaced. A client somewhere was holding the IP address of one of the old Pods. What happens to that client, and what would have to be true for it not to care?
4. Chapter 4 said a selector is a query over labels, and Chapter 6 said a ReplicaSet finds its Pods that way. Suppose a *different* controller needed to find the same set of Pods, for a different reason. What mechanism would you expect it to use — and what would break if someone edited one Pod's labels?
5. Chapter 4 gave you, in a single sentence, the DNS name form a Service gets. Write it out. Then: a container in namespace payments uses only the bare name database. Which Service does it reach, if there is a database Service in both payments and billing?
6. On an ordinary network, a request crosses a NAT boundary. What does the receiving process see as the source address, and name one thing that becomes harder because of it.
7. A platform that manages containers ships with no networking implementation of its own, and expects you to install one before anything works. Give one reason a system would be designed that way, and one cost of the design.

8. Something inside a private network has to be reachable from outside it. Name two mechanisms you have used or seen. For each, say who supplies the box that terminates the outside connection — you, or someone else.

Click for answers + reading strategy

1. **An indirection — something whose address doesn't change while the things behind it do.** A load balancer with a fixed VIP and a health-checked backend pool; a DNS name re-pointed as backends move; a service registry the client queries. Any two of those count.
2. **The second container cannot bind 8080 — they share one port space.** They reach each other over `localhost`, because they share one network namespace. The Pod has one IP address; the containers share it. (*Chapter 5 §1.*)
3. **The client breaks** — it holds a valid-looking address for something that no longer exists. It would not care if what it held were something that doesn't move. (*Chapter 6 §4.*)
4. **A selector — a query over the same labels.** Editing a Pod's labels can drop it out of one controller's set, out of the other's, or out of both. Nothing arbitrates between them: each evaluates its own query against the same field, and neither is told what the other decided. (*Chapter 4 §5, Chapter 6.*)
5. `<service-name>.<namespace-name>.svc.cluster.local` [source: k8s-docs-namespaces-2026-08-23]. A bare database from inside `payments` resolves to the database Service **in** `payments` — the caller's own namespace. It never sees the one in `billing`.
6. **The receiving process sees the NAT device's address, not the original sender's.** That makes source-based authorization, rate-limiting, audit logging, and debugging harder — anything that wanted to know *who* actually called.
7. **Reason:** pluggability. Different environments have genuinely different networking requirements, and a single built-in implementation would be wrong for most of them. **Cost:** nothing works out of the box; you must choose and install something before the platform functions at all.
8. Port forwarding on a firewall or router (**you** supply the box); a cloud load balancer with a public address (**the provider** supplies it); a reverse proxy on a bastion host (**you**); a tunnel service (**the provider**). Any two, with the ownership answered.

If you got 6+ right: Skim this chapter. Read §3 and §7 properly — they carry the memorizable material — and read every ☆ Fixed Point and △ Navigational Hazards callout. Then take all three ☺ Taking Your Bearings checkpoints, because the traps in this chapter are traps for people who already know some networking.

If you got 3–5 right: Read at normal pace. This chapter is calibrated for you.

If you got 0–2 right: Read carefully. And specifically: **if questions 2, 3 and 4 were among your misses, go back to Chapter 5 §1 and Chapter 6 §4 before you start §2 of this chapter.** The first half of this chapter is

built directly on the Pod's network namespace and on controller churn. Without both of those, §2 reads as a solution to a problem you have not yet felt.

Why This Chapter Matters

Every Pod in the cluster gets its own IP address, unique across the whole cluster. Any Pod can reach any other Pod at that address directly, with no proxies and no address translation, whether the two are on the same machine or on machines in different racks [source: k8s-docs-network-model-2026-08-23]. Chapter 8 pointedly did not tell you that. It had reason not to: the claim wasn't sourceable in that chapter's material, and it belongs here, where it can be stated properly and then immediately complicated.

Because here is the complication. That address is worth almost nothing. Chapter 6 taught you that a controller may replace any Pod at any moment, and that the replacement is not a repaired version of the old Pod — it is a *different Pod*, with a different identity and, now that you know §1's rule, a different address. So the system hands every workload a perfectly good address and then guarantees the address will expire. A bearing on a vessel already under way: true when you take it, wrong by the time you act on it. **The whole subject of this chapter is the gap between those two sentences.**

This is also the first chapter in which two things have to find each other. Everything from Chapter 2 through Chapter 8 was about getting a workload *onto* something: into a container, onto a node, under a controller, past an admission gate. Placement. This chapter is about reachability, and practitioners will tell you it is the point where Kubernetes stops feeling like an unusually opinionated scheduler and starts feeling like a platform. The reason is specific rather than sentimental. The flat network model is an unusually strong constraint, and once you have it, a whole category of problems that dominate ordinary infrastructure work simply stops existing. Port collisions between unrelated applications. Coordinating who gets which host port. Address translation that hides the caller. Maintaining a registry of where things currently live. None of those are solved here. They are *absent*.

Part III of this book opens with this chapter. Chapter 8 closed by naming what Part II had carefully avoided asking, and by describing what comes next as addresses, the abstraction that makes them survivable, how names resolve, and how anything outside the cluster gets in at all. The first three of those are this chapter; the fourth is Chapter 10.

One thing to hold as you read, because it will not make sense until the end. **There is exactly one object in this chapter, and it does not do anything.** That is a strange claim to make about the object that the entirety of Kubernetes networking rests on, and you should not accept it yet. §8 is where it gets paid off.

Dead Reckoning: The Kubernetes network model requires that every Pod have a unique cluster-wide IP, that all Pods be able to reach all other Pods directly without NAT or proxies, and that node agents be able to reach the Pods on their node [source: k8s-docs-network-model-2026-08-23]. Kubernetes defines that model. A CNI network plugin implements it [source: k8s-docs-extending-kubernetes-2026-08-23]. A Service is an API object that provides a stable, long-lived IP address or hostname for a set of Pods that changes over time [source: k8s-docs-network-model-2026-08-23]. Its default type is ClusterIP [source: k8s-docs-service-2026-08-23]. Its current backends are recorded in EndpointSlice objects, maintained by a controller [source: k8s-docs-network-model-2026-08-23]. kube-proxy programs each node to intercept traffic to the Service's address [source: k8s-docs-virtual-ips-kube-proxy-2026-08-23]. Cluster DNS publishes the address under a name [source: k8s-docs-dns-pod-service-2026-08-23].

Stakes, stated plainly. This is roughly seven points on our authored allocation, inside a domain worth 28% [source: cncf-kcna-curriculum-pdf-2026-08-23]. What that number understates is structural: within this book, Networking is the competency the others lean on. Chapter 10 cannot explain Ingress without the Service-type ladder from §3. Chapter 13 cannot explain Service troubleshooting without the endpoint list from §4. You will be back here.

What You'll Learn

By the end of this chapter, you'll be able to:

- **State** the four rules of the Kubernetes network model, and name the thing that implements them — which is not Kubernetes.
- **Explain** why a Pod's own IP address is insufficient for anything a client needs to keep talking to, using the churn a controller creates.
- **Choose** between ClusterIP, NodePort, LoadBalancer and ExternalName for a given exposure requirement — and say which three are layers of the same mechanism and which one is not a layer at all.
- **Trace** the path from a Service's selector to the set of addresses traffic actually reaches, and name the condition a Pod must satisfy to be on that list.
- **Write** the DNS name of a Service in another namespace, and say what a bare name would have resolved to instead.
- **Recognize** the whole apparatus — the virtual IP, the endpoint list, the DNS record — as one control loop reconciling the answer to a label query, which is the only thing you actually have to remember.

You'll also stop thinking of a Service as a load balancer, which is the single most useful correction in this chapter and the one that makes Chapter 10 straightforward instead of confusing.

Throughout this chapter, one example: a **frontend** Pod that needs to reach a **database**, where the database runs as three replicas managed by a Deployment. Nothing exotic. It is the most ordinary shape in the catalog, and each section will hand it to the next.

§1 — Four Rules and a Plugin

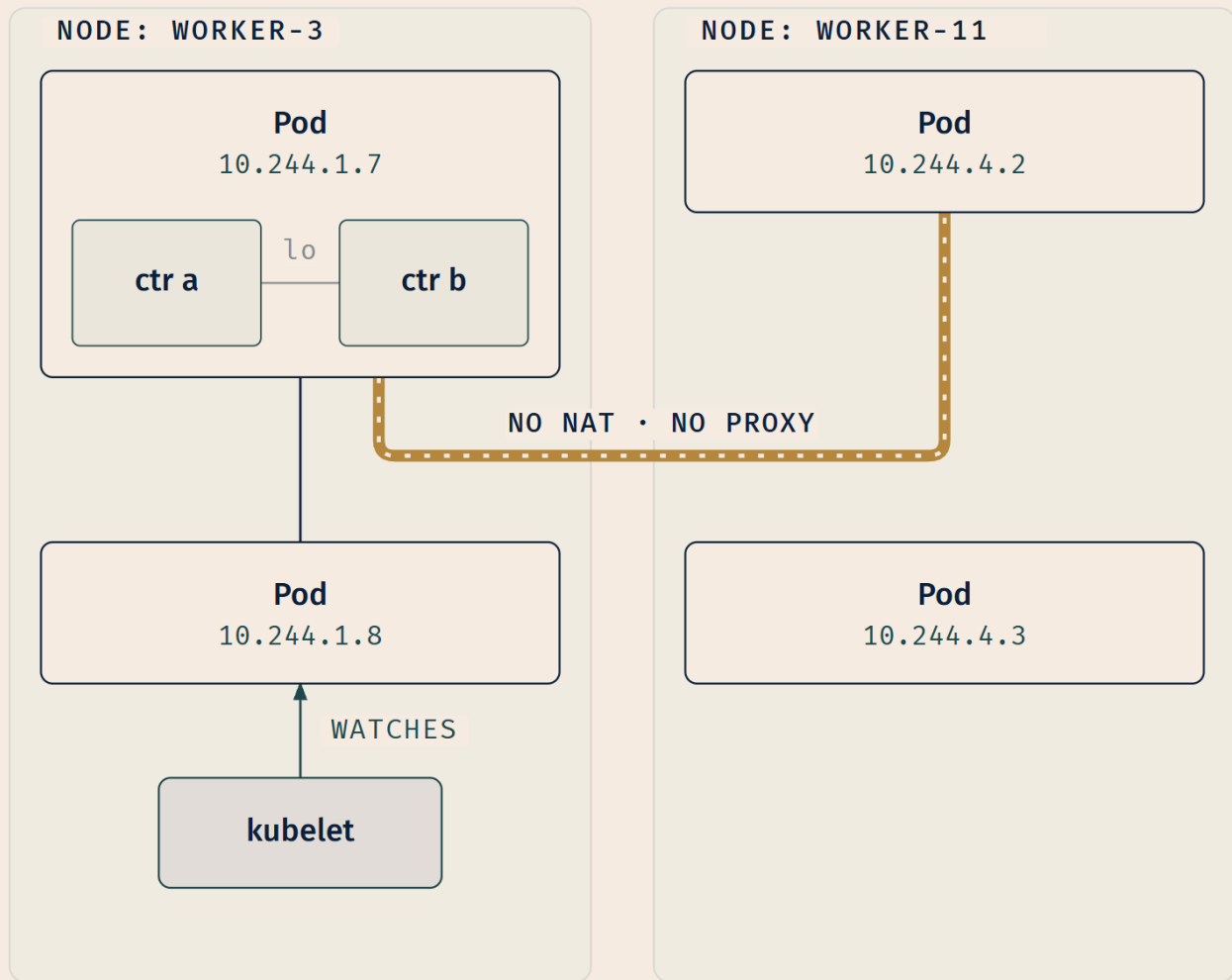
Kubernetes networking is specified as a small set of requirements. Not a mechanism, not a topology — requirements. Here they are, as a numbered set, because that is how they are examinable and how §8 will retrieve them.

Rule 1. Each Pod in a cluster gets its own **unique cluster-wide IP address**. A Pod has its own private network namespace, shared by all of the containers within it; processes running in different containers in the same Pod communicate with each other over `localhost` [source: k8s-docs-network-model-2026-08-23].

Rule 2. The **pod network** — also called the cluster network — handles communication between Pods. Barring intentional network segmentation, all Pods can communicate with all other Pods, **whether they are on the same node or on different nodes** [source: k8s-docs-network-model-2026-08-23].

Rule 3. Pods communicate with each other **directly, without the use of proxies or address translation (NAT)** [source: k8s-docs-network-model-2026-08-23]. (One documented exception: on Windows, this rule does not apply to host-network Pods [source: k8s-docs-network-model-2026-08-23]. Note it; nothing in this book is built on it.)

Rule 4. Agents on a node — system daemons, the kubelet — can communicate with all Pods on that node [source: k8s-docs-network-model-2026-08-23].



Every connection drawn is direct — nothing sits between the Pods.

LEGEND

— POD ↔ POD • SAME NODE

- - - - - POD ↔ POD • CROSS NODE

→ KUBELET WATCHES POD

— CONTAINER ↔ CONTAINER (LO)

Rule 3 deserves a paragraph rather than a line. Pause on it if you answered Soundings question 6.

In ordinary infrastructure, reaching a process means reaching a *host* and then a *port on that host*, and the address the receiver sees is frequently not the address the sender used. You accept a certain amount of pain as the price of that arrangement: coordinating which application gets which port on which machine, maintaining the translation tables, losing the caller's real identity somewhere in the middle,

explaining at eight the next morning why the logs say every request came from the same three addresses.

Kubernetes forbids all of it. The receiving Pod sees the sending Pod's actual address. Two different applications can both listen on port 8080 forever, because they are not sharing a port space; each has its own. An application inside the cluster can be written as though every other application in the cluster is on the same flat network, because it is.

That is not an aesthetic preference. It is the reason the rest of this chapter can be as simple as it is.

📌 **Snag:** "Each container gets its own IP" is the most common form of this mistake, and it usually arrives with readers whose habits were formed one container at a time, where container and address amounted to the same thing. Inside a Pod they do not. The address belongs to the **Pod**. Every container in a Pod shares the network namespace, including the IP address and network ports; inside a Pod, and *only* then, the containers can communicate using `localhost` [source: k8s-docs-pods-2026-08-24]. Containers in *different* Pods have distinct IP addresses and must use IP networking to reach each other [source: k8s-docs-pods-2026-08-24].

[*cross-bearing: see Ch 5 §1 — the Pod's shared network namespace*]. That chapter told you the containers share an address. This chapter tells you what the address is *worth*: it is routable from anywhere in the cluster.

☆ **Fixed Point:** Every Pod gets a unique **cluster-wide** IP address, and **all Pods can reach all Pods without NAT and without proxies** — same node or different nodes. The Pod holds the address; its containers share it and reach each other over `localhost`.

Kubernetes defines it. Something else implements it.

Everything above is a specification. It describes what must be true, and says nothing about how.

Pod networking is implemented by a **network plugin**, and the interface it plugs into is **CNI, the Container Network Interface** — listed among Kubernetes' infrastructure extension points alongside CSI for storage, CRI for container runtimes, and device plugins for custom hardware [source: k8s-docs-extending-kubernetes-2026-08-23]. CNI plugins are **binary plugins**: Kubernetes executes them as external binaries rather than linking them in [source: k8s-docs-extending-kubernetes-2026-08-23].

That published list runs longer than four [source: k8s-docs-extending-kubernetes-2026-08-23], and this book does not follow it item for item. What it follows is **the four pluggable interfaces**: CRI for runtimes, CNI for networking, CSI for storage, and CRDs for object types Kubernetes does not ship. [*cross-bearing: see Ch 2 §4 — CRI, CNI, CSI and CRDs as the four pluggable interfaces*]. That section named CNI and pointed you here. This is the second of the four; Chapter 17 collects all of them [*cross-bearing: see Ch 17 §4 — the four pluggable interfaces, collected*].

Which network plugin? That is a genuine choice with genuine consequences. **Calico** is a networking and network policy provider supporting overlay and non-overlay networks, with or without BGP. **Cilium**

provides a flat Layer 3 network with an eBPF-based data plane (see the glossary), in either native routing or overlay mode, and is a CNCF project at the Graduated level. **Flannel** is an overlay network provider [source: k8s-docs-cluster-addons-2026-08-24]. There are many more.

☆ **Fixed Point:** Kubernetes **defines** the network model. A **CNI network plugin is required to implement** it. The four rules above are requirements the plugin satisfies, not machinery Kubernetes provides [source: k8s-docs-network-plugins-2026-08-24] [source: k8s-docs-extending-kubernetes-2026-08-23].

Chapter 6 noted in passing that cluster networking plugins commonly ship as DaemonSets, one Pod on every node. At the time that was an isolated fact about DaemonSets; now it is a satisfying one. A thing that must configure networking on every node is exactly a thing that wants one copy per node. *[cross-bearing: see Ch 6 §7 — DaemonSets and per-node infrastructure]*

🔗 **Closer Look:** The model is a *requirement*, not a description of a mechanism, and that is why implementations differ so widely. The three plugins named above already span overlay and non-overlay networks, with or without BGP, native routing or encapsulation, and an eBPF-based data plane [source: k8s-docs-cluster-addons-2026-08-24]. Those are genuinely different pieces of engineering, and every one of them satisfies the same four rules. An application inside the cluster cannot tell which one it is running on: the model is the contract, and it is the same contract underneath all of them. That last part is the point. Deeper than the exam requires.

One practical consequence, and it retrieves last chapter at exactly one chapter's distance: a node whose network is not correctly configured reports the `NetworkUnavailable` condition — `True` if the network for the node is not correctly configured [source: k8s-docs-nodes-2026-08-23]. You met the node condition list in Chapter 8 *[cross-bearing: see Ch 8 §4 — node conditions]*. This is the shortest demonstration available that the plugin is not a footnote.

Rule 2's hedge, *barring intentional network segmentation*, is doing quiet work. Kubernetes has an API for segmenting Pod-to-Pod traffic on purpose, and it is a Chapter 10 subject *[cross-bearing: see Ch 10 §6 — NetworkPolicy]*.

So: every Pod has an address, and any Pod can use it. Which would settle the matter, except that those addresses belong to Pods, and you already know what happens to Pods.

.. §2 — The Address That Doesn't Last

Your frontend Pod has the database Pod's address. It works. Requests go out, responses come back, everything is fine.

Then the database Deployment gets a new image tag and performs a rolling update. Every database Pod is replaced. And Chapter 5 was precise about what "replaced" means: Kubernetes does not repair a Pod and hand it back — a Pod is never rescheduled to a different node; it is replaced by a new, near-identical

Pod with a different UID [source: k8s-docs-pod-lifecycle-2026-08-23]. A different Pod. Which, per §1's first rule, has a different address.

The frontend is now holding a valid-looking address for something that does not exist. Nothing malfunctioned. The system did precisely what it was designed to do, and the design broke the client.

[cross-bearing: see Ch 6 §4 — rolling updates and Pod replacement], and [cross-bearing: see Ch 5 §4 — a Pod is replaced, never rescheduled]. Chapter 5 told you in as many words that this fact is the premise of Chapter 9 *[cross-bearing: see Ch 5 §1 — the Pod, not the container, holds the address]*. Here it is, being used.

Generalize it, in the documentation's own framing, because the exam's framing of Services descends from this paragraph:

If some set of Pods (call them "backends") provides functionality to other Pods (call them "frontends") inside your cluster, how do the frontends find out and keep track of which IP address to connect to? [source: k8s-docs-service-2026-08-23]

The set of Pods running at one moment can be different from the set running a moment later [source: k8s-docs-service-2026-08-23]. That is the problem. Notice what it is *not*: not a failure mode, not a bug, not a degraded state to be recovered from. It is the normal condition of a system that replaces workloads freely.

A chart names the harbour, never the ships riding in it. That is exactly why the chart is still good next season.

The answer is an object.

A **Service** is a method for exposing a network application that is running as one or more Pods in your cluster. Each Service object defines a **logical set of endpoints** — usually those endpoints are Pods — along with a policy about how to make those Pods accessible [source: k8s-docs-service-2026-08-23]. The Service API lets you provide a **stable, long-lived** IP address or hostname for a service implemented by one or more backend Pods, where the individual Pods making up the service can change over time [source: k8s-docs-network-model-2026-08-23].

Read that last clause again. *Where the individual Pods making up the service can change over time.* The churn is not an inconvenience the Service copes with. The churn is the condition the Service exists for.

☆ **Fixed Point:** A Service is a stable, long-lived address for a set of Pods **that is expected to change**. It is not a workaround for churn; it is the abstraction that makes churn a non-event.

How does a Service know which Pods? By a **selector**. Services most commonly abstract access to Kubernetes Pods thanks to the selector [source: k8s-docs-service-2026-08-23] — a query over labels, which is exactly what Chapter 4 told you a selector is, and exactly the mechanism Chapter 6 said a ReplicaSet uses to find its Pods. *[cross-bearing: see Ch 4 §5 — labels and selectors]*. Naming it now and

leaving the machinery for §4 is the honest split, because Chapter 4 already told you a Service selects its backends this way. What §4 adds is where the query's answer gets *written down*.

And one fact about defaults, because it is cheap and it is examinable: **ClusterIP** exposes the Service on a cluster-internal IP, making it reachable only from within the cluster, and **it is the default that is used if you don't explicitly specify a type** [source: k8s-docs-service-2026-08-23]. §3 opens there and builds outward.

Before you go on, the correction that is worth more than anything else in this section:

🔒 **Worth Securing:** "A Service is a load balancer" is the most durable wrong model in Kubernetes networking, and it is wrong in a specific and useful way. A load balancer is *a thing that runs*: a process, on a machine, receiving your traffic and forwarding it. A Service is a **declaration that gets reconciled** — an object, in exactly the sense Chapter 4 established, stating that a set of Pods should be reachable at a stable address. Whether anything is listening, whether any Pod matches, whether traffic goes anywhere at all: none of that is the Service's doing, and none of it changes whether the Service exists. Almost every confusing thing in the next four sections follows from that distinction.

[cross-bearing: see Ch 4 §2 — *spec and status*; a Service is an object like any other]

..1 §3 — Four Ways to Be Reachable

There are four Service types. Three of them are layers of the same mechanism. One of them is not a layer at all, and confusing it for one is the most reliable way to lose a point in this competency.

The three that stack

ClusterIP exposes the Service on a cluster-internal IP. Choosing this value makes the Service only reachable from within the cluster. This is the default that is used if you don't explicitly specify a type for a Service [source: k8s-docs-service-2026-08-23].

NodePort exposes the Service on each node's IP at a static port — the node port. And here is the part candidates miss: to make the node port available, **Kubernetes sets up a cluster IP address, the same as if you had requested a Service of type: ClusterIP** [source: k8s-docs-service-2026-08-23]. NodePort does not replace ClusterIP. It adds to it.

LoadBalancer exposes the Service externally using an external load balancer [source: k8s-docs-service-2026-08-23]. It is the outermost rung, the one that puts an address in *front* of the cluster rather than inside it.

LOADBALANCER**LoadBalancer**

reachable from: the internet

(external LB — supplied by your cloud provider, not Kubernetes)

NODEPORT**NodePort**

reachable from: anything that can reach a node

<node-ip>:<static-port> — every node

CLUSTERIP**ClusterIP**

reachable from: inside the cluster only
(the default type)

*Each ring adds to the ones inside it.
Asking for an outer ring does not remove the inner ones.*

NOT ON THE LADDER

NOT A FOURTH RING • SEPARATE MECHANISM

EXTERNALNAME**ExternalName**

my-svc CNAME → api.foo.bar.example

no cluster IP • no endpoints • no proxying of any kind

LEGEND

 Ring — additive layer

Boundary — not on the ladder

☆ **Fixed Point:** The ladder types are **additive**. NodePort and LoadBalancer are both supersets of ClusterIP [source: k8s-docs-service-ports-2026-08-24], and the case the documentation spells out is the one to memorize: **a NodePort Service also has a cluster IP — Kubernetes sets one up, exactly as if you had requested type: ClusterIP** [source: k8s-docs-service-2026-08-23]. Asking for a higher rung never removes the rungs below it.

The one that is not on the ladder

ExternalName maps the Service to the contents of the `externalName` field — for example, to the hostname `api.foo.bar.example`. The mapping configures your cluster's DNS server to return a **CNAME record** with that external hostname value. **No proxying of any kind is set up** [source: k8s-docs-service-2026-08-23].

Read that last sentence as the definition rather than as a footnote. An ExternalName Service has no cluster IP. It has no endpoint list. Nothing intercepts a packet on its behalf, because there is no address to intercept traffic to. When a client resolves its name, DNS hands back a different name, and the client connects to whatever that resolves to — directly, with Kubernetes entirely out of the path.

It is a DNS alias flying a Service's colors.

⚠ **Navigational Hazards:** ExternalName is not the fourth rung of the ladder. It allocates no address, selects no Pods, and proxies nothing — it is a CNAME record and nothing more. Every exam question that presents it as "the type you use for external things" is testing exactly this. The word "External" in two type names (ExternalName, and LoadBalancer's external load balancer) is doing you no favors; they have nothing mechanically in common.

The other fact that catches people

Choosing type: LoadBalancer does not cause Kubernetes to load-balance anything.

Kubernetes does not directly offer a load balancing component; you must provide one, or you can integrate your Kubernetes cluster with a cloud provider [source: k8s-docs-service-2026-08-23].

What type: LoadBalancer does is *signal* that an external load balancer should be provisioned. On a managed cluster with cloud-provider integration, something notices that signal, provisions a load balancer, and reports its address back. On a bare-metal cluster with no such integration and no add-on that provides one, a type: LoadBalancer Service sits there with no external address indefinitely. Not for thirty seconds. Indefinitely: the signal is raised correctly, and there is nobody ashore to answer it.

And nothing is broken. The object is correct; the declaration is valid; the piece that would act on it is not present. Hold on to that shape — you will see it again in §4, and Chapter 10 will give it a name.

☆ **Fixed Point: Kubernetes provides no load balancer.** type: LoadBalancer is a request for one from somewhere else — your cloud provider, or a component you install.

🔑 **Mnemonic:** *Inside · on every node · out in the world · somewhere else entirely.* ClusterIP, NodePort, LoadBalancer, ExternalName — in the order you will meet them, with the last one deliberately phrased so it doesn't sound like a continuation.

Choosing

Four lines, not a flowchart. Definitions and additivity are more testable than the decision, and this book weights them accordingly.

- Traffic stays inside the cluster → **ClusterIP**.
- You need a fixed port on every node, usually because something in front of the cluster will target it → **NodePort**.
- You have a cloud provider that will hand you an external address → **LoadBalancer**.
- You want an in-cluster name for something that isn't in the cluster at all → **ExternalName**.

The three port numbers

A Service's port entry can carry three different numbers, and they answer three different questions. `port` is the port the Service itself exposes. `targetPort` is the port on the container that traffic is delivered to. `nodePort` — present only on the types that have one — is the port opened on every node.

A Service can map **any** incoming port to a `targetPort`, and by default, for convenience, `targetPort` is set to the same value as `port` [source: k8s-docs-service-ports-2026-08-24]. That default is exactly why the distinction is easy to miss: in most manifests you will ever read, the two numbers are identical, and it is natural to conclude they are one field wearing two names. They are not. The moment a container listens on 8080 and you want the Service reachable on 80, they separate — and the manifest that separates them is the one a question will put in front of you.

`targetPort` does not have to be a number at all. Port definitions in Pods can have names, and a Service can reference those names in `targetPort`, which works even when the Service selects a mixture of Pods that expose the same named port on different port numbers [source: k8s-docs-service-ports-2026-08-24]. The Service says which port it wants by name; each Pod says where that name lives. It is the same move as a selector, one layer down: name the thing you want, let the other side say where it is.

For type: `NodePort`, the control plane allocates the node port from a range set by the `--service-node-port-range` flag — **default 30000–32767** — and every node proxies that same port number into the Service [source: k8s-docs-service-ports-2026-08-24]. You may name a `nodePort` yourself, in which case Kubernetes either allocates it or fails the API request because the port is already taken, which makes collision-avoidance your problem rather than the cluster's. Leave it out and one is allocated for you [source: k8s-docs-service-ports-2026-08-24].

☆ **Fixed Point:** `port` is on the **Service**. `targetPort` is on the **container**. `nodePort` is on the **node**. Default node-port range: **30000–32767**.

⚠ **Navigational Hazards:** Two traps live in this one field set. The first is reading `targetPort` as "the port the Service listens on" — it is the opposite end of the hop. The second is assuming a node port is the same number as `port`; it is drawn from the 30000–32767 range and, unless you asked for a specific one, it will not resemble either of the other two numbers. A question that shows you `port: 80, targetPort: 8080` and `nodePort: 31234` is testing whether you can say which end of the connection each one describes.

[cross-bearing: see Ch 8 §1 — a Service is created by the same apply as every other object] [cross-bearing: see Ch 3 §5 — the API server as the only door in]

The ceiling

One thing before you leave this section, because Chapter 10 is going to need it.

A LoadBalancer Service gives you one external address per Service. That is fine for one Service. It is expensive and awkward for fifty.

And none of these four types knows anything about HTTP. They move packets. They cannot route on a hostname, or a URL path, or a header, because at the layer they operate at those things are just bytes in a payload. If you want `shop.example.com/checkout` and `shop.example.com/catalog` to reach different backends behind one address, no Service type in this list can do it.

Protocol awareness is precisely what the next layer up exists to supply: **the Gateway API, or its predecessor Ingress, makes Services accessible to clients outside the cluster, with protocol-aware HTTP/HTTPS routing using URIs, hostnames, and paths** [source: k8s-docs-network-model-2026-08-23]. The address arithmetic — one per Service, fifty Services — is this book's own argument for getting there sooner rather than later. Chapter 10 opens exactly there. *[cross-bearing: see Ch 10 §1 — the exposure ceiling and what crosses it]*

🧭 Taking Your Bearings #1

Five questions on the model and the abstraction — §1 through §3. Two of them reach back to earlier chapters.

1. .. A Pod on `worker-3` sends a request to a Pod on `worker-11`. What does the receiving Pod see as the source address, and what two things does the network model guarantee are *not* involved?
2. .. **[retrieval: ch5]** A Pod runs two containers. One listens on 8080. Can the other also listen on 8080? How does each reach the other? And how many IP addresses does the Pod have?
3. .. You create a Service with `type: NodePort`. A colleague says, "so now it's only reachable from outside — we lost the internal address." What is wrong with that, and why?

4. **[retrieval: ch6]** Chapter 6 walked a Deployment through a rolling update. State what happens to the Pod IP addresses during that update, and then say what a Service gives a client that the Pod IPs cannot.
5. **Two Services.** One has type: `LoadBalancer` and, twenty minutes after creation, still has no external address. One has type: `ExternalName` pointing at `api.vendor.example`. For each: what is Kubernetes doing, and is anything broken?

Answers with Explanations:

1. The sending Pod's own IP address. Not involved: NAT (address translation), and proxies.

The model states it directly — Pods can communicate with each other directly, without the use of proxies or address translation [source: [k8s-docs-network-model-2026-08-23](#)] — and that the guarantee holds whether the Pods are on the same node or different nodes [source: [k8s-docs-network-model-2026-08-23](#)].

Why the tempting wrong answers are wrong:

- **"The sending node's IP"** would be correct in almost any non-Kubernetes cluster, which is exactly why it's the trap. Node-to-node forwarding with source rewriting is the normal arrangement elsewhere. It is forbidden here.
- **"A translated address from the pod network's NAT pool"** assumes cross-node traffic must be translated. It must not be. §1's third rule forbids address translation between Pods outright [source: [k8s-docs-network-model-2026-08-23](#)], and that prohibition binds every plugin equally: whatever a plugin does internally to carry a packet from one node to another, the receiver still sees the sender's own address. Reason from the rule, not from a guess about how any particular plugin moves the bytes.

2. No — they share one port space. Over localhost. One address.

Every container in a Pod shares the network namespace, including the IP address and network ports; within a Pod, containers share an IP address and port space and can find each other via `localhost` [source: [k8s-docs-pods-2026-08-24](#)].

Why the wrong answers are wrong:

- **"Yes, they each get their own port space"** is the single-container mental model most people arrive with. Two containers in one Pod compete for ports exactly as two processes on one host do.
- **"Two addresses, one per container"** is the single most common Kubernetes networking misconception. One Pod, one address per address family.

3. Nothing was lost. Requesting `NodePort` also allocates a cluster IP.

To make the node port available, Kubernetes sets up a cluster IP address, the same as if you had requested a Service of type: `ClusterIP` [source: k8s-docs-service-2026-08-23].

Say the rule at the level of the ladder rather than as a fact about NodePort, because the shape is what makes the types easy to hold: **a type that adds reachability keeps what the type beneath it already provided**. The documentation states that shape explicitly for NodePort over ClusterIP [source: k8s-docs-service-2026-08-23], and records NodePort and LoadBalancer alike as supersets of ClusterIP [source: k8s-docs-service-ports-2026-08-24] — so the full ladder reads LoadBalancer over NodePort over ClusterIP. Hold it in that form and you carry one rule instead of three separate facts — and Chapter 10's argument for Ingress, "one external address per Service, and that does not scale," becomes available to you without further work.

4. Every Pod is replaced, and each replacement has a different address. A Service gives the client one address that does not change while the set behind it does.

A Pod is replaced by a new, near-identical Pod with a different UID [source: k8s-docs-pod-lifecycle-2026-08-23]; §1's first rule gives that new Pod its own address. The Service API provides a stable, long-lived IP address or hostname for a service implemented by one or more backend Pods, where the individual Pods making up the service can change over time [source: k8s-docs-network-model-2026-08-23].

If you answered something like "the client breaks, and it wouldn't have if it were using something that doesn't move" — you have written §2's opening for yourself, which is the ideal outcome for this item.

5. LoadBalancer: waiting for an external load balancer that nothing is providing. ExternalName: working exactly as designed, as a CNAME with no proxying. Neither is broken.

Kubernetes does not directly offer a load balancing component; you must provide one, or integrate with a cloud provider [source: k8s-docs-service-2026-08-23]. A cluster with no such integration has nothing to act on the request, so the Service waits. Indefinitely, and correctly.

ExternalName maps the Service to the contents of the `externalName` field by configuring cluster DNS to return a CNAME record; no proxying of any kind is set up [source: k8s-docs-service-2026-08-23]. There is nothing further for it to do.

Why the tempting wrong answers are wrong:

- **"The LoadBalancer Service failed to provision"** treats absence of a provider as failure. The declaration is correct; the actor is missing.
- **"The ExternalName Service is proxying traffic to the vendor"** is the trap. It is not in the path at all.

This is a genuinely reasonable thing to be surprised by. Most people meet type: `LoadBalancer` on a managed cluster where it just works, and the first bare-metal cluster is where the assumption surfaces. That's a normal way to learn it.

Checkpoint: You've Now Mastered

✓ The four rules of the Kubernetes network model, and who implements them ✓ Why a Pod IP is not something a client can hold on to ✓ Four Service types, how the ladder is built, and which one isn't a rung on it at all ✓ That Kubernetes ships no load balancer

Next: the Service has a name and an address, a mark that holds position while everything behind it is replaced. Now find out what *is* behind it, and what a Pod has to do to get on the list.

§4 — The List Behind the Name

A Service has a selector. The selector is a query. Somebody has to run the query, and somebody has to write down the answer, and the answer has to stay current while Pods appear and vanish underneath it.

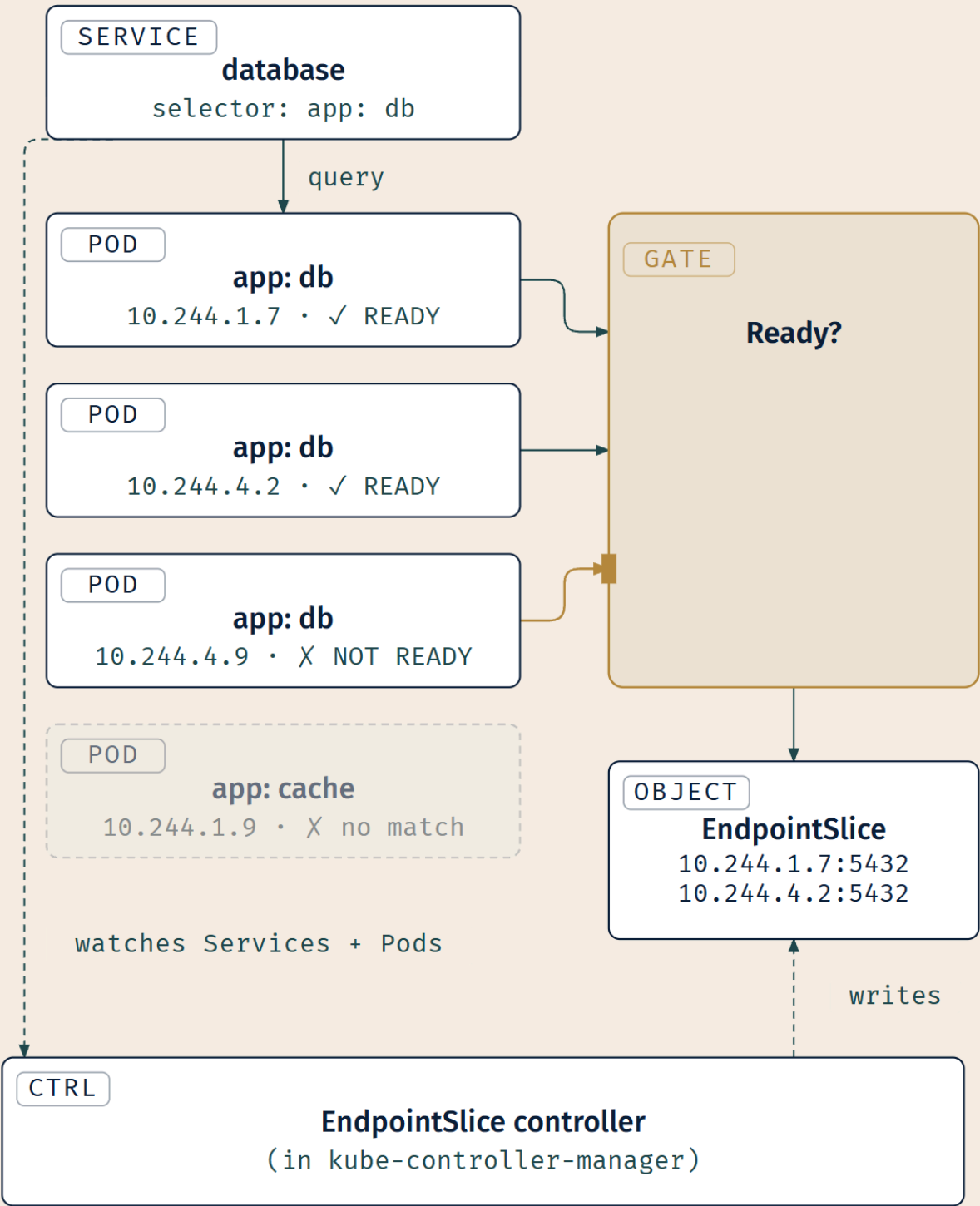
The path, in four steps

One. The Service carries a **selector** over Pod labels [source: k8s-docs-service-2026-08-23].

Two. Kubernetes automatically manages **EndpointSlice** objects to provide information about the Pods currently backing a Service [source: k8s-docs-network-model-2026-08-23].

Three. The thing doing the managing is the **EndpointSlice controller**, one of the controllers running inside the kube-controller-manager, whose job is described as populating EndpointSlice objects **to provide a link between Services and Pods** [source: k8s-docs-cluster-architecture-2026-08-23]. You met the controller-manager's controller list in Chapter 3 [*cross-bearing: see Ch 3 §2 — the controllers inside kube-controller-manager*]; this is one of the names on it. You will also meet it under an older name, **endpoints controller**, including in a quotation later in this section. One job, two names in the documentation. Not two components.

Four. Anything that needs to know a Service's current backends reads the **EndpointSlices**, not the selector. The selector is the question. The EndpointSlice is the written-down answer.



- LEGEND
- ADMITTED (READY) → STOPPED AT GATE --→ WATCH / WRITE
 - EXCLUDED — NO MATCH □ FOCAL — READINESS GATE

Selection is not ownership

Chapter 6 already did half of this section's work, in a context where it was surprising. Its answer key established that a Service uses **labels** to allow the control plane to determine which EndpointSlice objects are used for that Service, and that in addition to the labels, each EndpointSlice managed on behalf of a Service carries an **owner reference** [source: k8s-docs-garbage-collection-2026-08-24].

Two mechanisms, doing two jobs. Labels answer *which slices belong to this Service*. Owner references answer *what should be cleaned up when this Service is deleted*, and help different parts of Kubernetes avoid interfering with objects they don't control [source: k8s-docs-garbage-collection-2026-08-24]. A fact that was a curiosity in Chapter 6 turns out to be structural here. *[cross-bearing: see Ch 6 §3 — selection versus ownership]*

Readiness gates membership

Here is the part you were promised.

A Pod that matches the selector is **not automatically a destination**. It has to be **Ready**.

Chapter 5 taught you readiness probes and told you plainly that this is the mechanism doing the removing. The probe's documented behavior says it outright: `readinessProbe` indicates whether the container is ready to respond to requests, and **if the readiness probe fails, the endpoints controller removes the Pod's IP address from the endpoints of all Services that match the Pod** [source: k8s-docs-pod-lifecycle-2026-08-23]. And the Pod's `Ready` condition is documented as meaning the Pod is able to serve requests and **should be added to the load balancing pools of all matching Services** [source: k8s-docs-pod-termination-2026-08-24].

That quotation is where the older name surfaces. The *endpoints controller* removing an address and the *EndpointSlice controller* writing the slice are the same job [source: k8s-docs-cluster-architecture-2026-08-23]; if you read them as two components, the path in this section grows a step it does not have.

[cross-bearing: see Ch 5 §7 — readiness probes]. That section told you what a readiness probe does. This one tells you where it does it: in the EndpointSlice.

☆ **Fixed Point: selector → EndpointSlice → traffic**. The selector proposes; readiness disposes. Matching the selector is what puts a Pod **in** the EndpointSlice — the slice references *all* the Pods the selector matches [source: k8s-docs-endpointslices-2026-08-24]. Being **Ready** is what makes its endpoint eligible to **receive traffic**. Readiness does not remove the endpoint; it disqualifies it *[cross-bearing: see Ch 16 §4 — is anything even selected]*.

Three chapters converge on that sentence, and it is worth naming the convergence. Chapter 5 gave you the probe. Chapter 6 relied on the mechanism without explaining it: the reason a bad release cannot take a Service down mid-rollout is that a new Pod which never reports `Ready` never joins the endpoint list, so traffic never reaches it, so the rollout stalls instead of the service failing. Chapter 9 is where the wiring becomes visible. *[cross-bearing: see Ch 6 §4 — why a failed rollout does not take the Service down]*

The same fact, running backwards

Termination is the mirror image, and it is subtler than you'd guess.

At the same time as the kubelet is starting graceful shutdown of a Pod, **the control plane evaluates whether to remove that shutting-down Pod from EndpointSlice objects**, where those objects represent a Service with a configured selector. ReplicaSets and other workload resources no longer treat the shutting-down Pod as a valid, in-service replica [source: k8s-docs-pod-termination-2026-08-24].

But — and this is the interesting half — **any endpoints that represent the terminating Pods are not immediately removed from EndpointSlices**, and a status indicating terminating state is exposed from the EndpointSlice API. **Terminating endpoints always have their ready status as `false`**, so load balancers will not use them for regular traffic [source: k8s-docs-pod-termination-2026-08-24].

So the list is not a boolean membership test. It carries state, and note where that state lives. The Pod has a Ready condition; the Pod's *entry in the slice* has a ready status of its own [source: k8s-docs-pod-termination-2026-08-24]. A terminating Pod stays on the list, marked, so that anything watching can distinguish "gone" from "going," which is what lets in-flight connections drain rather than being severed.

🔍 **Closer Look:** If traffic draining on a terminating Pod is needed, actual readiness can be checked as a condition called `serving` [source: k8s-docs-pod-termination-2026-08-24]. That is the distinction between "should get new traffic" and "can still handle traffic it already has." Deeper than the exam requires, and the reason graceful shutdown works at all rather than merely being promised.

Looking at the list

You can inspect it directly:

```
kubectl get endpointslices -l kubernetes.io/service-name=<service-name>
```

Make sure the endpoints in the EndpointSlices match up with the number of Pods you expect to be members of your Service. **If they don't, the Service's selector probably does not match the Pods' labels, or the Pods are not Ready** [source: k8s-docs-debug-pods-2026-08-23].

Two causes. That is the whole diagnostic content of this section, and it is stated as a fact rather than a procedure on purpose: Chapters 13 and 16 own the troubleshooting workflow, and they will retrieve this by name. *[cross-bearing: see Ch 16 §4 — telling an empty endpoint list from a list that serves nothing]*

📌 **Snag:** A Service with no endpoints is not a broken Service. It is a correct Service whose selector currently matches nothing — or, in the case the diagnostic pages describe, one whose matching Pods are not Ready and so are serving nothing. Those are **two different bugs, and they live in two different files** — one is a mismatch between the Service's selector and the Pod template's labels; the other is an application that isn't passing its own health check. Confusing them costs you an afternoon, because you spend it editing the wrong YAML.

One more clause, closing a Chapter 7 loop. That chapter argued that a Service's backends landing on distinct nodes is what makes it resilient rather than merely load-balanced. Now you can see why the argument was necessary: the endpoint list is *just a list*. It has no opinion about where its endpoints are. Topology is the scheduler's problem, which is where you solved it. *[cross-bearing: see Ch 7 §5 — spreading replicas across failure domains]*

The case that closes the section

A Service whose selector matches nothing is a completely valid object.

It has a cluster IP. It has a DNS record. Its EndpointSlice is empty, and traffic sent to it goes nowhere at all. Nothing is broken; nothing has failed; the declaration is being reconciled correctly against a set that happens to be empty. The control loop is doing exactly its job, and its job produces nothing, because there is nothing to produce.

You met the same shape in §3, with a LoadBalancer Service waiting on a provider that doesn't exist. Two instances now. Chapter 10 will meet a third and give the pattern a name.

§5 — When You Don't Want a Single Address

Two deliberate exceptions. They only make sense now — a reader meeting either of these before §3 and §4 has no idea what is being subtracted from what.

Headless: no single address, on purpose

Sometimes you don't need load-balancing and a single Service IP. In that case, you can create what are termed **headless Services**, by explicitly specifying "None" for the cluster IP address (`.spec.clusterIP`) [source: k8s-docs-service-2026-08-23].

What changes depends on whether the Service has a selector:

- For headless Services that **define selectors**, the endpoints controller creates EndpointSlices in the Kubernetes API, and modifies the DNS configuration to return **A or AAAA records that point directly to the Pods** backing the Service [source: k8s-docs-service-2026-08-23].
- For headless Services **without selectors**, no EndpointSlices are created automatically [source: k8s-docs-service-2026-08-23].

That first bullet carries the Kubernetes documentation's own phrasing, *the endpoints controller*. §4 called the same component **the EndpointSlice controller**. These are two names for one job, not two controllers: whichever name you meet, the object being written is an EndpointSlice.

☆ **Fixed Point:** `clusterIP: None` is a **configuration, not a failure**. It says: *do not give me one address — give me all of them*. DNS returns the Pod addresses directly instead of a single virtual IP.

🔒 **Snag:** "Headless" sounds like something went wrong, and a teammate seeing `<none>` in the CLUSTER-IP column will often say so. It means the Service has no *head* — no single virtual IP standing in front of the set, no flagship to hail on the whole fleet's behalf. The Pods are all still there. You just get all of them, instead of one of them.

Why anyone would want that

Chapter 6 already handed you the answer and then deferred it.

StatefulSets currently require a headless Service to be responsible for the network identity of the Pods. You are responsible for creating this Service [source: k8s-docs-statefulset-2026-08-24].

Think about what a StatefulSet is for. Its Pods are created from the same spec, but **are not interchangeable**: each has a persistent identifier that it maintains across any rescheduling [source: k8s-docs-statefulset-2026-08-24]. A database with one primary and two replicas. A quorum of peers that need to know each other by name. In those workloads, "send this to any one of them, I don't care which" is not a convenience; it is *precisely wrong*. If you need to reach the primary, an address that might land you on a read replica is not a stable endpoint. It is a signal sent to the whole fleet when what you needed was to raise one ship by name.

So the headless Service subtracts the one feature the ordinary Service exists to provide, and that subtraction is the feature. [*cross-bearing: see Ch 6 §6 — StatefulSets and stable identity*], where you were told this and told to wait. §7 shows you what the resulting DNS looks like, and Chapter 11 closes the storage half of the same identity story [*cross-bearing: see Ch 11 §6 — StatefulSets and their per-replica volume claims*].

Services without selectors

The second exception is not an exception to the address; it is an exception to the *backends*.

Services most commonly abstract access to Kubernetes Pods thanks to the selector — but when used with a corresponding set of EndpointSlice objects and **without a selector**, the Service can abstract other kinds of backends, including ones that run **outside the cluster**: an external database, a service in another namespace or cluster, or a workload being migrated [source: k8s-docs-service-2026-08-23].

The selector is how a Service *usually* finds its endpoints. It is not what a Service *is*. Remove it, supply the endpoint addresses yourself, and everything downstream — the cluster IP, the DNS record, kube-proxy's interception, the client's connection — proceeds exactly as before.

🔒 **Worth Securing:** The selectorless Service is the fixed light a migration steers by. Your application inside the cluster talks to `database.production.svc.cluster.local` from day one, whether that name currently resolves to a managed database in a data center three hundred kilometers away, or to Pods you moved in last night. The client never learns the difference, and never has to be redeployed to find out. The migration stops being the client's problem, which is usually the hardest part of making it happen at all.

Two binaries, four cases

Headless-or-not and selector-or-not are independent, which means there are four combinations, and people routinely conflate two of them.

	Has a selector	No selector
Normal (has a cluster IP)	The ordinary case from §3 and §4. Cluster IP; EndpointSlices populated automatically from the selector; DNS resolves to the cluster IP.	A cluster IP in front of endpoints you manage yourself. DNS resolves to the cluster IP; traffic is proxied to whatever addresses you supplied [source: k8s-docs-service-2026-08-23].
Headless (clusterIP: None)	No cluster IP. The endpoints controller creates EndpointSlices; DNS returns A/AAAA records pointing directly at the Pods [source: k8s-docs-service-2026-08-23].	No cluster IP, and no EndpointSlices are created automatically [source: k8s-docs-service-2026-08-23].

Four cells, and all four are supported configurations. None of them is an error state.

🕒 Taking Your Bearings #2

Five questions on what's actually behind the name, and the two times you deliberately don't want one — §4 and §5. A name on a chart is only as good as the survey behind it. These five ask what the survey found, and who keeps it current.

1. `❏` **[retrieval: ch6]** Chapter 6 said a ReplicaSet finds its Pods by selector, and that a Service asking the same question about the same Pods is a *different controller reading the same labels*. Name the object where the Service's answer gets written down, and name the controller that writes it.
2. `❏` A Service's selector matches four Pods. Three are Ready; one is failing its readiness probe. How many endpoints are listed, how many are receiving traffic, and what happens to the fourth Pod when its probe starts passing?
3. `❏` `kubectl get endpointslices` for your Service returns no endpoints. The Pods are running. Name the two usual causes, and say how you would tell them apart.
4. `❏` A teammate calls a headless Service "broken" because `kubectl` shows no cluster IP. Explain what `clusterIP: None` does, and give one workload for which it is the correct choice.
5. `❏` You need cluster-internal clients to reach a database that runs on hardware outside the cluster, using an ordinary Service name. Two Service configurations could do this. Name both, and say what is different about what the client experiences.

⚠ **Question 3 is intentionally challenging.** Your instinct will be to look at the network. The answer is in two YAML files. If you struggle with it, that struggle is the point — this is the item most likely to save you real time later, and it encodes better for having been hard.

Answers with Explanations:

1. The EndpointSlice. Written by the EndpointSlice controller.

Kubernetes automatically manages EndpointSlice objects to provide information about the Pods currently backing a Service [source: k8s-docs-network-model-2026-08-23]; the EndpointSlice controller populates EndpointSlice objects to provide a link between Services and Pods [source: k8s-docs-cluster-architecture-2026-08-23].

Worth noticing what Chapter 6's framing bought you here [*cross-bearing: see Ch 6 §3 — how a controller knows its own*]. Two controllers ask the same question — *which Pods carry these labels* — for completely different reasons, and they do not coordinate. The ReplicaSet controller asks in order to count and to create replacements. The EndpointSlice controller asks in order to write down addresses. Neither knows the other exists. Both read the same field.

Why a wrong answer is wrong: "The Service itself stores them" is the natural guess and it's wrong in an instructive way. The Service stores the *query*. A separate object stores the *answer*, and a separate controller keeps that answer current.

2. Four endpoints, but only three of them serving. When the fourth Pod's probe starts passing, its endpoint's condition flips to ready and it begins receiving traffic.

A Pod's Ready condition means the Pod is able to serve requests and should be added to the load balancing pools of all matching Services [source: k8s-docs-pod-termination-2026-08-24]; if a readiness probe fails, the endpoints controller removes the Pod's IP address from the endpoints of all Services that match the Pod [source: k8s-docs-pod-lifecycle-2026-08-23].

Two documentation pages describe that removal differently and they are not in conflict. The Pod-lifecycle page speaks of removing the address *from the endpoints* — the set that gets served. The EndpointSlice API models the same event as a **condition on an endpoint that stays listed** [source: k8s-docs-endpointslices-2026-08-24], which is why §5's terminating Pods keep their entries with `ready: false`. Removed from the serving set, still present in the slice. At this exam's level the operative half is the first: a Pod that fails its readiness probe stops receiving traffic through the Service.

One naming note before you carry that sentence forward. The documentation calls this component *the endpoints controller* on the Pod-lifecycle page and *the EndpointSlice controller* in the control-plane component list. That is one controller doing one job under two names: nothing additional is running, and question 1's answer and this one are describing the same thing.

Now connect it to Chapter 6, because this is the cheapest place to close that loop [*cross-bearing: see Ch 6 §4 — rolling-update mechanics*]. During a rolling update, a new Pod that never reports Ready **never receives traffic**. That is *why* a bad release cannot take your Service down: the broken replicas are excluded from the endpoint list by the same mechanism that admits the healthy ones. Chapter 6

depended on that behavior to explain safe rollouts without ever naming the object it happens in. This is the object.

Why a wrong answer is wrong: **"Three, and there is no fourth endpoint"** is the intuitive answer and it is half right. The slice references every Pod the selector matches [source: k8s-docs-endpointslices-2026-08-24], so the fourth endpoint is listed — carrying a condition that says it must not be used as a target for Service traffic. That distinction is worth holding, because it is what makes the empty-slice diagnostic in Chapter 16 work: an endpoint list that is *not* empty but serves nothing is a readiness problem, and an actually empty list is a selector problem [cross-bearing: see Ch 16 §4 — *is anything even selected*]. The readiness gate is a second, independent condition, and it is evaluated continuously rather than once at creation.

3. Either the Service's selector doesn't match the Pods' labels, or the Pods are not Ready. Tell them apart by comparing the Service's selector against the Pods' labels, and by checking the Pods' Ready condition.

If the endpoints don't match the Pods you expect, the Service's selector probably does not match the Pods' labels, or the Pods are not Ready [source: k8s-docs-debug-pods-2026-08-23].

Why the tempting wrong answers are wrong:

- **"The network plugin is misconfigured"** would produce a much broader failure than one Service having no endpoints. Endpoint membership is computed by a controller reading the API; it does not touch the data plane at all.
- **"kube-proxy isn't running"** would break traffic to Services that *do* have endpoints. It cannot empty an endpoint list — kube-proxy reads that list, it doesn't write it.
- **"The Pods are on the wrong nodes"** — the endpoint list has no opinion about node placement whatsoever.

If you found this hard: good. It is genuinely counterintuitive that a *networking* symptom has two *non-networking* causes, and knowing that in advance is worth more than most of the recall material in this chapter.

4. It means: don't allocate one virtual IP for this Service. DNS returns the Pod addresses directly instead. Correct choice for a StatefulSet.

You create headless Services by explicitly specifying "None" for `.spec.clusterIP`; for headless Services that define selectors, DNS is configured to return A or AAAA records pointing directly to the Pods backing the Service [source: k8s-docs-service-2026-08-23]. StatefulSets currently require a headless Service to be responsible for the network identity of the Pods, and you are responsible for creating it [source: k8s-docs-statefulset-2026-08-24]. That requirement is the direct consequence of what a StatefulSet is for [cross-bearing: see Ch 6 §6 — *StatefulSet and stable identity*]: members that are not interchangeable need names that are not interchangeable either, and a single shared virtual IP cannot supply those.

Why the wrong answer is wrong: **"The cluster IP allocation failed"** treats None as an absence rather than a value. It is a value. Somebody typed it.

5. A Service without a selector, backed by manually managed EndpointSlices; or type: ExternalName. The difference is proxying: the first puts kube-proxy in the path, the second does not.

A Service used with a corresponding set of EndpointSlice objects and without a selector can abstract backends that run outside the cluster, including an external database [source: k8s-docs-service-2026-08-23]. ExternalName maps the Service to an external hostname via a CNAME record, and no proxying of any kind is set up [source: k8s-docs-service-2026-08-23].

The proxying difference is the discriminator, and it is the reason this question exists. Both are correct answers to the stated requirement, and they behave differently in ways that matter:

- With the **selectorless Service**, the client connects to a cluster IP inside the cluster. Traffic is intercepted on the node and redirected to the external address. The client's connection target is a cluster-internal address; DNS is ordinary Service DNS.
- With **ExternalName**, the client resolves the Service name, gets a CNAME to the external hostname, and connects to it directly. Kubernetes is not in the path at all. Which means TLS certificate names, source addresses seen by the external database, and anything that inspects the connection all behave differently.

This is the chapter's best interleaving item because it needs §3 and §5 simultaneously, and neither section alone gets you there.

Checkpoint: You've Now Mastered

✓ The path from selector to EndpointSlice to traffic ✓ Readiness as a gate on endpoint membership, not a step in a chain ✓ The two causes of an empty endpoint list, and why neither is a network problem ✓ Headless and selectorless Services as deliberate configurations

This is the natural end of session one, if you're splitting the chapter — a reasonable place to put in for the night. What remains is the component that actually moves the packet, the names you type, and the observation that ties all of it together.

..1 §6 — The Component That Makes It Real

You now know what a Service declares and where its backends are recorded. Nothing so far has moved a packet.

One component, one job

Every node in a Kubernetes cluster runs a kube-proxy — unless you have deployed your own alternative component in its place. The kube-proxy component is responsible for implementing a **virtual IP mechanism for Services of type other than ExternalName**. Each instance of kube-proxy watches the

Kubernetes control plane for the addition and removal of **Service and EndpointSlice objects**. For each Service, kube-proxy calls appropriate APIs — depending on the kube-proxy mode — to configure the node to **capture traffic to the Service's clusterIP and port, and redirect that traffic to one of the Service's endpoints** (usually a Pod, but possibly an arbitrary user-provided IP address) [source: k8s-docs-virtual-ips-kube-proxy-2026-08-23].

Chapter 3 introduced kube-proxy as a node component, a network proxy that runs on each node in your cluster, implementing part of the Kubernetes Service concept [source: k8s-docs-cluster-architecture-2026-08-23], and left the "how" for here. [*cross-bearing: see Ch 3 §3 — kube-proxy as a node component*]

And then there is this sentence, which you should read twice:

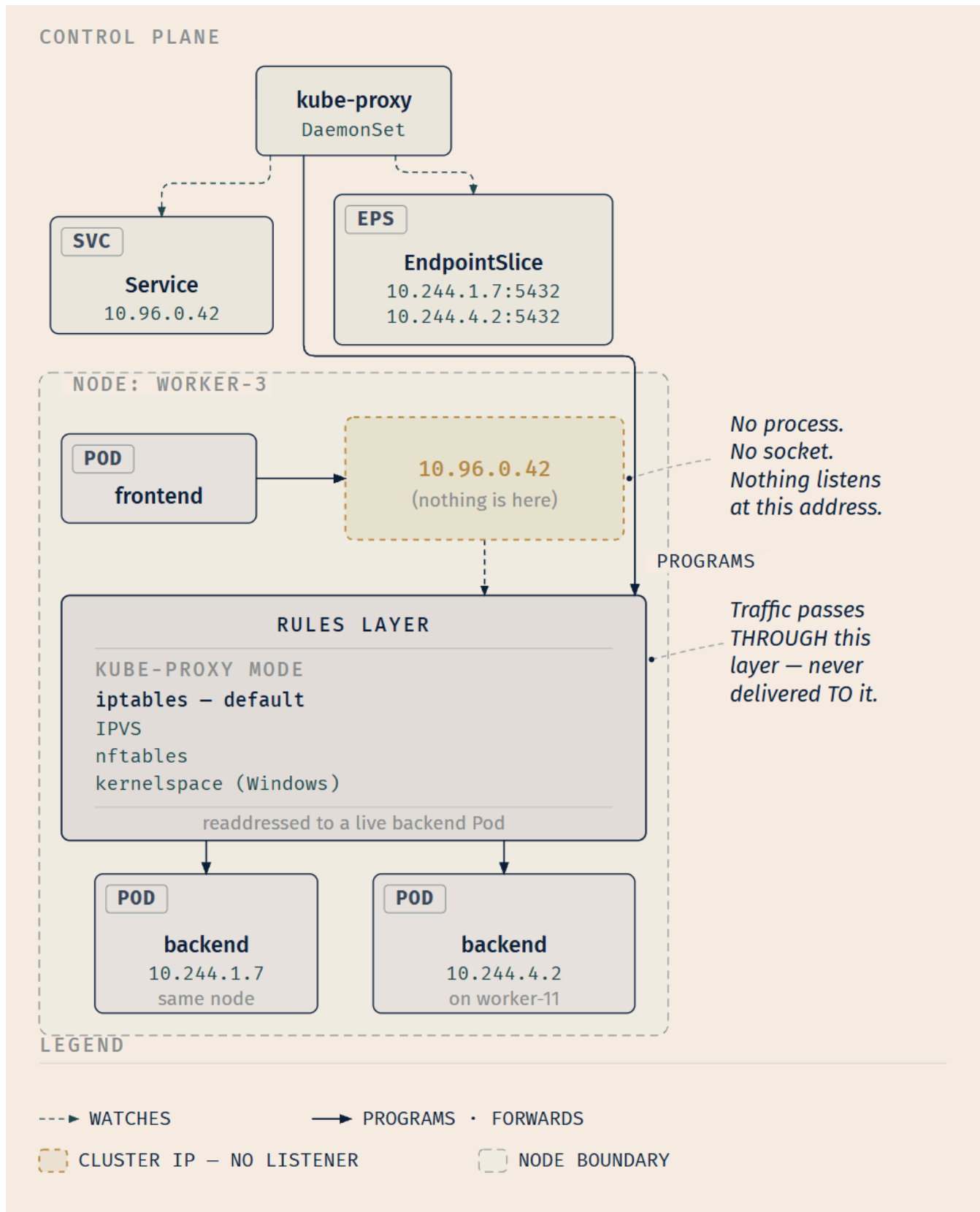
A control loop ensures that the rules on each node are reliably synchronized with the Service and EndpointSlice state as indicated by the API server. [source: k8s-docs-virtual-ips-kube-proxy-2026-08-23]

A control loop. Watching objects, comparing to actual state, reconciling. You have met that shape in Chapters 3, 4, 6, 7 and 8, and here it is again in a reference page about packet forwarding. File that; §8 collects it.

The cluster IP is virtual

Here is the consequence that reframes everything before it.

Nothing is listening on a cluster IP. No process is bound to it. There is no host at that address, no socket, no server. It exists only as a rule, replicated on every node, that says: *packets addressed here go to one of those addresses instead*. That is not a separate fact to memorise. It follows directly from what the documentation says kube-proxy does, which is **capture traffic to the Service's clusterIP and port, and redirect that traffic to one of the Service's endpoints** [source: k8s-docs-virtual-ips-kube-proxy-2026-08-23]. Capture and redirect. Nothing in that sentence binds a socket.



This is also why §3's ExternalName exclusion makes sense from the other side. An ExternalName Service has **no address to intercept** — it produces a CNAME and nothing else — so there is nothing for kube-

proxy to program. The exclusion isn't a special case; it falls straight out of what kube-proxy does.

🔒 **Worth Securing:** Nothing is listening on a cluster IP. It is not a host, not a process, not a port on a machine. It is a rule on every node that **captures** traffic to that address and **redirects** it elsewhere [source: k8s-docs-virtual-ips-kube-proxy-2026-08-23]. Once that lands properly, roughly half of the confusing behavior in Kubernetes networking stops being confusing — including why the node-level tools you would ordinarily reach for show you nothing bound at that address, and why "a Service is a load balancer" was never quite right.

☆ **Fixed Point:** kube-proxy implements the virtual IP mechanism for **every Service type except ExternalName**, by watching **Service and EndpointSlice** objects and programming each node. Modes: **iptables (the default)**, IPVS, nftables on Linux; **kernel-space** on Windows.

The modes

On Linux nodes, the available modes for kube-proxy are: **iptables** — configures packet forwarding rules using iptables, and is **the default**; **ipvs** — configures packet forwarding rules using IPVS; **nftables** — configures packet forwarding rules using nftables, GA since Kubernetes 1.33. On Windows there is exactly one mode: **kernel-space** — configures packet forwarding rules in the Windows kernel [source: k8s-docs-virtual-ips-kube-proxy-2026-08-23].

Mode	Platform	Notes
iptables	Linux	The default
ipvs	Linux	
nftables	Linux	GA since v1.33
kernel-space	Windows	The only mode on Windows

That is the entire requirement at this level: recognize the four names, and know which one is the default. You do not need to understand how iptables chains differ from IPVS virtual servers, and this book is not going to pretend otherwise to fill a page. A candidate who can name the four and identify iptables as the default has everything this material is likely to ask for.

kube-proxy is optional

One closing beat, and it is a nice one because it reaches back to §1.

If you use a network plugin that implements packet forwarding for Services by itself, and provides equivalent behavior to kube-proxy, then you do not need to run kube-proxy on the nodes in your cluster [source: k8s-docs-cluster-architecture-2026-08-23]. Cilium is a named example: it **can act as a replacement for kube-proxy** [source: k8s-docs-cluster-addons-2026-08-24].

So the plugin that implements the network model can also implement the Service data plane. Which is a good inoculation against reading kube-proxy as load-bearing architecture. It is one implementation of

one job, and that job can be done elsewhere.

🔧 **Closer Look:** kube-proxy is optional. A plugin like Cilium can do the same work in its own eBPF data plane (see the glossary) [source: k8s-docs-cluster-addons-2026-08-24]. If you meet a cluster running no kube-proxy at all, nothing is missing. Deeper than the exam requires, but useful the first time you see it and assume something is wrong.

[cross-bearing: see Ch 17 §5 — a service mesh moves this interception into a sidecar or an ambient layer]

..1 §7 — Names, and Where They Resolve

Nothing so far has involved a name. The frontend still needs the database's cluster IP from somewhere, and hard-coding a cluster IP is barely better than hard-coding a Pod IP — you'd have to know it before the Service existed.

This section is where you stop needing to know any addresses at all. From here on the frontend steers by name rather than by position, and something else keeps the position current.

What serves the records

Kubernetes creates DNS records for Services and Pods. You can contact Services with consistent DNS names instead of IP addresses [source: k8s-docs-dns-pod-service-2026-08-23]. Kubernetes publishes information about Pods and Services which is used to program DNS, and **the kubelet configures Pods' DNS so that running containers can look up Services by name rather than IP** [source: k8s-docs-dns-pod-service-2026-08-23].

The server doing the answering is **CoreDNS**, the cluster DNS addon that serves these records [source: k8s-docs-dns-pod-service-2026-08-23], listed under Service Discovery in the cluster addons as a flexible, extensible DNS server which can be installed as the in-cluster DNS for Pods [source: k8s-docs-cluster-addons-2026-08-24].

And the reason you have never installed it, never configured it, and possibly never thought about it: **DNS is a built-in Kubernetes service launched automatically using the addon manager cluster add-on** [source: k8s-docs-dns-cluster-addon-2026-08-24].

Chapter 3 promised you CoreDNS by name *[cross-bearing: see Ch 3 §4 — cluster addons and CoreDNS]*, and Chapter 4 gave you a Service's DNS name in a single sentence and explicitly deferred four things: the mechanism, what serves the records, what else gets one, and how resolution actually proceeds *[cross-bearing: see Ch 4 §3 — namespaces and Service DNS names]*. All four are discharged in this section.

The Service record

Normal (not headless) Services are assigned DNS A and/or AAAA records, depending on the IP family of the Service, with a name of the form `my-svc.my-namespace.svc.cluster-domain.example`. This resolves to

the cluster IP of the Service [source: k8s-docs-dns-pod-service-2026-08-23].

Four labels, in a fixed order:

Position	Part	What it is
1	my-svc	The Service's name
2	my-namespace	The namespace the Service lives in
3	svc	Literal. Marks this as a Service record
4	cluster-domain.example	The cluster domain — commonly <code>cluster.local</code>

So the frontend types `database.production.svc.cluster.local`, and gets back a cluster IP that will not change for the life of the Service, regardless of what happens to the Pods behind it.

The same name, a different answer

Here is the section's best fact, and it is the one people are most often surprised by.

Headless Services (without a cluster IP) are also assigned DNS A and/or AAAA records with the same name form; unlike normal Services, this resolves to the set of IPs of all of the Pods selected by the Service. Clients are expected to consume the set or else use standard round-robin selection from the set [source: k8s-docs-dns-pod-service-2026-08-23].

The name did not change shape. Same four labels, same order, same everything. What changed is the *number of answers*: one cluster IP for a normal Service, the whole Pod set for a headless one.

\$5 told you this happens. This is where you can see that it happens **without a different name**, which is why it catches people — the client's configuration file looks identical in both cases.

☆ **Fixed Point:** The **same name form** — `<service>.<namespace>.svc.<cluster-domain>` — gives you the **cluster IP** for a normal Service and **the set of Pod IPs** for a headless one.

Why a bare name works, and where it stops

Chapter 4 gave you the rule flat: a bare `<service-name>` resolves to the Service local to your namespace, and reaching across namespaces requires the fully qualified domain name [source: k8s-docs-namespaces-2026-08-23].

Here is why, and it is more satisfying than a rule.

By default, a client Pod's DNS search list includes the Pod's own namespace and the cluster's default domain [source: k8s-docs-dns-pod-service-2026-08-23].

That is it. That is the entire mechanism. It is not special-case Kubernetes behavior; it is ordinary DNS search-domain resolution, the same thing that has let you type a short hostname on a corporate

network for thirty years. A Pod in payments searching for ledger tries `ledger.payments.svc.cluster.local`, because `payments.svc.cluster.local` is in its search list. It finds something. It stops.

☆ **Fixed Point:** A bare name resolves in the client Pod's **own namespace only** — **because that namespace is in the Pod's DNS search list**. This is not a Kubernetes special case; it is ordinary DNS search-domain resolution, which is also why crossing a namespace boundary requires the full `<service>`.

`<namespace>.svc.<cluster-domain>`.

⚠ **Navigational Hazards:** The bare name is the single most common cross-namespace mistake, and it fails in the worst available way. If **no** Service of that name exists in the caller's namespace, you get a resolution failure — annoying, but obvious, and you'll fix it in a minute. If a Service of that **same name** exists in the caller's namespace, the lookup **succeeds**, and your application connects to the wrong thing. No error. No timeout. No log line. Just the wrong database, answering questions it shouldn't have been asked. This is why the FQDN rule is worth memorising rather than looking up.

🧠 **Mnemonic:** *service, namespace, svc, cluster.local*. Four labels, always in that order. The two in the middle are the ones people reverse under pressure — and note that `svc` is a literal, not an abbreviation of anything in your cluster.

The other record shapes

Two more, compactly, plus one that closes §5's loop.

SRV records are created for **named ports** that are part of normal or headless Services, with the form `_port-name._port-protocol.my-svc.my-namespace.svc.cluster-domain.example` [source: k8s-docs-dns-pod-service-2026-08-23]. Note that the Service's own four labels are intact at the end; the port-name and protocol are prefixed on.

Pods get records too. In general a Pod has the DNS resolution of the form `pod-ipv4-address.my-namespace.pod.cluster-domain.example` — for example, `172-17-0-3.default.pod.cluster.local` [source: k8s-docs-dns-pod-service-2026-08-23]. The dots in the address become dashes, because dots are label separators in DNS and cannot appear inside a label. Note also that position 3 is `pod`, not `svc`.

And the one that finishes the StatefulSet story: the Pod spec has optional **hostname** and **subdomain** fields; **when both are set and a headless Service exists with the same name as the subdomain, the Pod's FQDN is** `hostname.subdomain.namespace.svc.cluster-domain.example` [source: k8s-docs-dns-pod-service-2026-08-23].

Note the shape, because it is the same shape by which an individual StatefulSet member is individually addressable. §5 told you a StatefulSet requires a headless Service to be responsible for the network identity of its Pods; a stable per-member name of this shape, surviving rescheduling, is what "network identity" cashes out to. [cross-bearing: see Ch 6 §6 — *StatefulSet Pod naming*]

NAME COMPONENTS

RESOLVES TO

1

2

3

4

SAME NAME → DIFFERENT ANSWER

Service (normal)

→ the cluster IP

my-svc

•

my-namespace

•

svc

•

cluster.local

Service (headless)

→ ALL the Pod IPs

my-svc

•

my-namespace

•

svc

•

cluster.local

SRV (named port)

→ the named port

_http._tcp.my-svc

•

my-namespace

•

svc

•

cluster.local

port prefix

Pod (by address)

→ that Pod

172-17-0-3

•

my-namespace

•

pod

•

cluster.local

dots → dashes

Pod (hostname + subdomain)→ that Pod, by a
stable name

hostname.subdomain

•

namespace

•

svc

•

cluster.local

Columns 2 and 4 are identical in every row.

Column 3 is svc, except Pod-by-address: pod.

Rows one and two share one name — the answer differs.**LEGEND**

dashed border — breaks the svc pattern

accent frame — same name, different answer

Dead Reckoning: The five record shapes, stated flat.

Name shape	Resolves to
<code>my-svc.my-namespace.svc.cluster-domain.example</code> (normal Service)	The cluster IP of the Service
<code>my-svc.my-namespace.svc.cluster-domain.example</code> (headless Service)	The set of IPs of all Pods selected by the Service
<code>_port-name._port-protocol.my-svc.my-namespace.svc.cluster-domain.example</code>	The named port on that Service
<code>pod-ipv4-address.my-namespace.pod.cluster-domain.example</code> (e.g. <code>172-17-0-3.default.pod.cluster.local</code>)	That Pod
<code>hostname.subdomain.namespace.svc.cluster-domain.example</code> (requires <code>hostname</code> + <code>subdomain</code> set, and a headless Service named for the subdomain)	That Pod, by a stable name

Record types are A and/or AAAA depending on the IP family, except SRV records, which are SRV. Cluster DNS is served by CoreDNS, launched automatically by the addon manager. Pod DNS configuration is written by the kubelet. A bare service name resolves within the client Pod's own namespace because that namespace is in the Pod's default DNS search list. [source: [k8s-docs-dns-pod-service-2026-08-23](#)] [source: [k8s-docs-dns-cluster-addon-2026-08-24](#)]

🦋 **Closer Look:** A Pod's `dnsPolicy` field controls how its DNS is configured. `ClusterFirst` — any query that does not match the configured cluster domain suffix is forwarded to an upstream nameserver — **is the default policy if `dnsPolicy` is not explicitly specified** [source: [k8s-docs-dns-pod-service-2026-08-23](#)]. Also available: `Default` (inherit resolution config from the node), `ClusterFirstWithHostNet` (for Pods running with `hostNetwork`), and `None`, where the Pod ignores DNS settings from the Kubernetes environment and all settings come from the `dnsConfig` field [source: [k8s-docs-dns-pod-service-2026-08-23](#)]. Note the trap in the naming: the value called `Default` is *not* the default. Deeper than the exam requires — but if you ever meet a Pod that can't resolve cluster names at all, this field is where to look.

One boundary worth marking before Chapter 10. What you have just learned is **DNS-based service discovery**: mapping a name to an address, in the ordinary DNS sense. Chapter 10 introduces **name-based virtual hosting**: routing HTTP traffic to multiple host names at the same IP address [source: [k8s-docs-ingress-2026-08-23](#)]. Both involve hostnames, and they sit on opposite sides of the connection. One turns a name into an address before any traffic moves; the other sorts traffic that has already arrived at a single address. Conflating them makes Chapter 10 considerably harder than it needs to be. [cross-bearing: see *Ch 10 §2 — name-based virtual hosting*]

🕒 Taking Your Bearings #3

Five questions on what moves the packet and what you actually type — §6 and §7.

1. `..||` Name the component that implements the virtual IP mechanism for Services, say which Service type it does **not** implement, and name the two object types it watches.
 2. `..||` A colleague says, "I'll SSH to the node and see what's listening on the cluster IP." What will they find, and why?
 3. `..||` Name kube-proxy's modes and identify the default on Linux. Then say what it would mean to find a cluster running no kube-proxy at all.
 4. `..||` A container in namespace `payments` needs to reach a Service named `ledger` in namespace `billing`. Write the name it should use, and say what would happen if it used just `ledger`.
 5. `..||` Two Services in the same namespace: `db-a` is a normal Service, `db-b` is headless. A client resolves `db-a.prod.svc.cluster.local` and `db-b.prod.svc.cluster.local`. Describe what each lookup returns, and how many answers the client gets.
-

Answers with Explanations:

1. kube-proxy. It does not implement ExternalName. It watches Service and EndpointSlice objects.

The kube-proxy component is responsible for implementing a virtual IP mechanism for Services of type other than `ExternalName`; each instance watches the control plane for the addition and removal of Service and `EndpointSlice` objects [source: `k8s-docs-virtual-ips-kube-proxy-2026-08-23`].

Why a wrong answer is wrong: **"It watches Services and Pods"** is close and consequential. kube-proxy reads the *computed* answer, the `EndpointSlice`, not the raw Pods. The `EndpointSlice` controller is what reads Pods. Two controllers, two inputs, one chain.

2. Nothing. The cluster IP is virtual — no process is bound to it.

kube-proxy configures the node to capture traffic to the Service's `clusterIP` and port, and redirect that traffic to one of the Service's endpoints [source: `k8s-docs-virtual-ips-kube-proxy-2026-08-23`]. That is the whole documented mechanism, and everything else in this answer follows from it rather than from any separate statement in the documentation: if traffic is *captured and redirected*, then it passes *through* a rule and is never delivered to the cluster IP. Nothing accepts a connection there, so there is no socket to find, and `netstat` or `ss` on the node has nothing bound to show you.

Why the wrong answers are wrong:

- **"kube-proxy is listening on it"** — kube-proxy programs rules; in the default iptables mode it does not sit in the data path receiving your packets.
- **"One of the backend Pods"** — the backend Pods listen on their *own* addresses. The cluster IP is not one of them.

If you got this one, \$6 landed. If you didn't, it's worth a re-read of the Worth Securing callout before you move on, because this idea is load-bearing for everything in Chapter 10.

3. iptables (the default), IPVS, and nftables on Linux; kernelspace on Windows. A cluster with no kube-proxy means the network plugin implements Service packet forwarding itself.

The mode list and the default are documented directly [source: k8s-docs-virtual-ips-kube-proxy-2026-08-23]. And: if you use a network plugin that implements packet forwarding for Services by itself, providing equivalent behavior to kube-proxy, you do not need to run kube-proxy on the nodes in your cluster [source: k8s-docs-cluster-architecture-2026-08-23] — Cilium being a named example of a plugin that can act as a replacement [source: k8s-docs-cluster-addons-2026-08-24].

A note on what this answer does not say: it makes no claim about which mode is faster, more scalable, or better, and none about whether replacing kube-proxy performs better than keeping it. The documentation states the list, the default, and the substitution, and says nothing about relative performance — so neither does this book. If you have read that IPVS scales better than iptables for very large Services, that may well be true, but it isn't a claim this material supports, and no exam question you meet should turn on it.

4. ledger.billing.svc.cluster.local. A bare ledger would resolve within payments only — failing, or worse, silently reaching a different ledger.

A client Pod's DNS search list includes the Pod's own namespace and the cluster's default domain [source: k8s-docs-dns-pod-service-2026-08-23]; a container using only <service-name> resolves to the Service local to its namespace, and reaching across namespaces requires the FQDN [source: k8s-docs-namespaces-2026-08-23].

"It fails" is the incomplete answer, and the incompleteness matters. If payments has no ledger Service, you get NXDOMAIN, your application logs a connection error, and you fix it in five minutes. If payments *does* have a ledger Service — perhaps a test fixture, perhaps a stub someone left behind, perhaps a genuinely different service that happens to share a name — the lookup **succeeds against the wrong Service**. Your application connects. It writes. Nothing anywhere reports an error.

That second case is the one that costs people afternoons, and it's the reason to memorise the FQDN form rather than plan to look it up.

5. db-a returns one address — the Service's cluster IP. db-b returns the set of IPs of all the Pods it selects.

Normal Services are assigned A and/or AAAA records resolving to the cluster IP of the Service; headless Services are assigned records with **the same name form**, but resolving to the set of IPs of all of the Pods selected by the Service, and clients are expected to consume the set or use standard round-robin selection from it [source: k8s-docs-dns-pod-service-2026-08-23].

Both answers come from the same place. Cluster DNS is a built-in service launched automatically as a cluster addon [source: k8s-docs-dns-cluster-addon-2026-08-24], and CoreDNS is what serves it [source:

k8s-docs-cluster-addons-2026-08-24]. It is not behaving differently for db-b than for db-a; the difference is in what each Service's records are defined to contain, not in what publishes them.

The point of the item is that the two names are structurally identical. Nothing in db-b.prod.svc.cluster.local announces that it is headless. A client library that assumes one answer per lookup will take the first address it gets and never speak to the other Pods — which is fine for some workloads and quietly wrong for others.

Checkpoint: You've Now Mastered

✓ kube-proxy's job, its four modes, and its optionality ✓ That the cluster IP is a rule, not a socket ✓ Five DNS record shapes and what each resolves to ✓ Why a bare name is namespace-local, and what that costs when it goes wrong

One section left. It teaches nothing new.

☀ §8 — A Query With a Name

Back to the claim from the opening. **There is one object in this chapter, and it does not do anything.**

A Service is a **label query with a name attached**. Everything else in this chapter is a control loop reconciling that query's current answer against some piece of the network.

Walk it back through, in the chapter's own order.

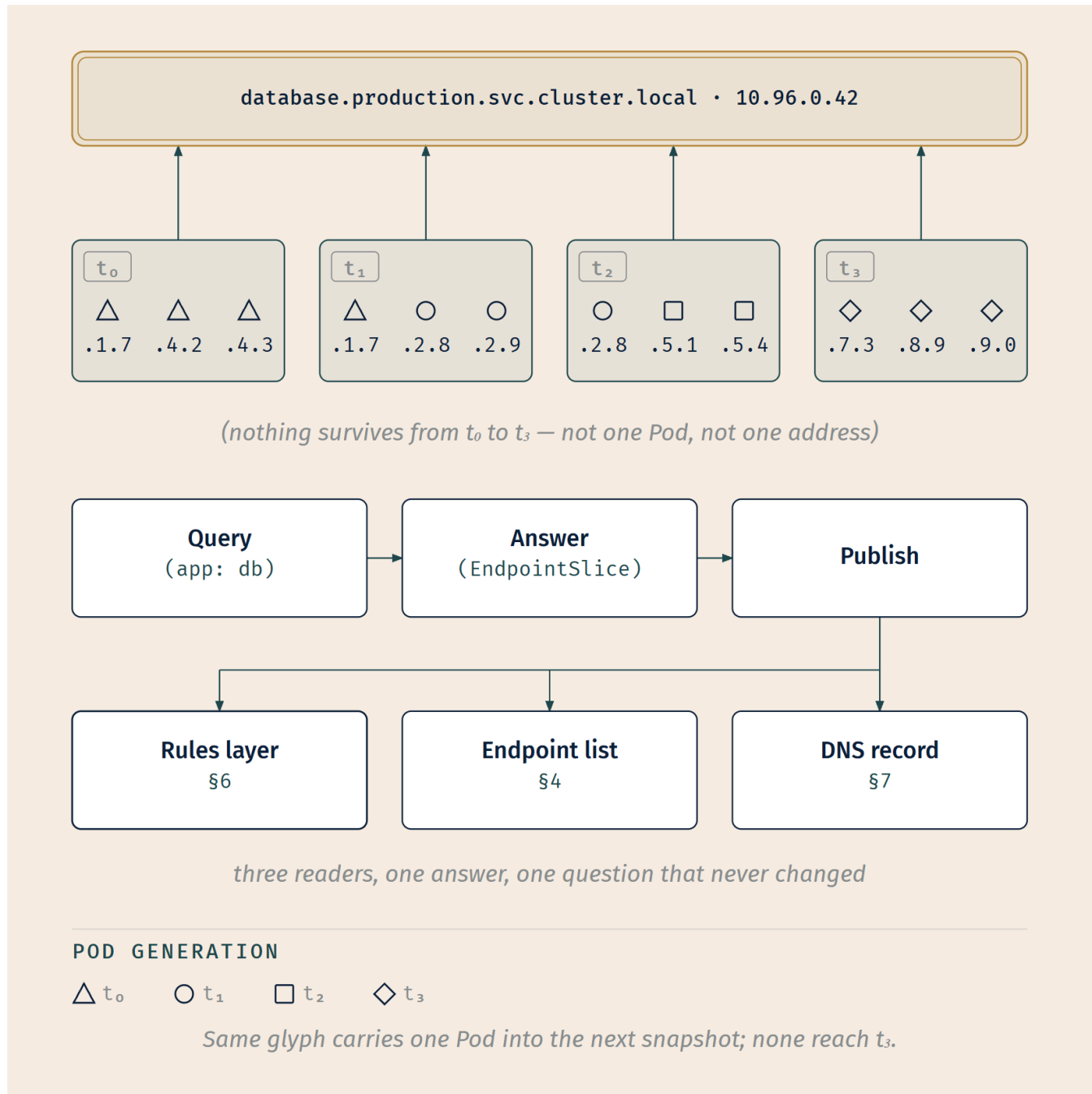
From §2. The Service is an object. You apply it through the same API server door as everything else, and its spec is a statement of desired state in exactly the sense Chapter 4 defined: *these Pods should be reachable at a stable address*. It does not run. It does not receive traffic. It states a condition that should hold.

From §4. The EndpointSlice controller watches Services and Pods, evaluates the selector, and writes down the answer [source: k8s-docs-cluster-architecture-2026-08-23]. Desired state, observed state, reconciliation. **And this is the same shape as the ReplicaSet controller running over the same labels for an entirely different purpose** — which is exactly what Chapter 6 was telling you when it distinguished selection from ownership. Two loops, one label set, no coordination, no conflict. [*cross-bearing: see Ch 6 §3 — selection versus ownership*]

From §6. kube-proxy watches Services and EndpointSlices and programs each node. The documentation says so in as many words: *a control loop ensures that the rules on each node are reliably synchronized with the Service and EndpointSlice state as indicated by the API server* [source: k8s-docs-virtual-ips-kube-proxy-2026-08-23]. That is another instance of the control loop, this time in a reference page about packet forwarding rather than in a chapter about controllers.

From §7. Cluster DNS publishes the answer as a name. Same input, third consumer, third format.

From §1. And underneath all of it, none of the actual packet-moving is Kubernetes' work at all. Kubernetes defines the network model; a CNI plugin implements it [source: k8s-docs-extending-kubernetes-2026-08-23] — which is the second instance of an arrangement you first met in Chapter 2, where CRI does the same thing for container runtimes. [cross-bearing: see Ch 2 §4 — CRI as the first pluggable interface]



☼ **Zenith:** A Service is a **label query with a name**. The virtual IP, the endpoint list, and the DNS record are three control loops publishing that one query's current answer in three different formats. The Pods were

never the stable thing. **The question was.**

That is the chapter's title, made good on. The stable thing was never a Pod — you knew that after §2. But it was never really an IP address either: the cluster IP belongs to nothing, nothing listens on it, and it exists only as a rule that other machinery maintains. What is actually stable is "*which Pods carry the label `app: db` and are Ready*" — a question whose answer is different every few minutes and whose **meaning** is the same forever. Pods churn beneath it the way water churns beneath a fixed star. The question doesn't move.

That is what makes churn survivable. And it is why the abstraction is a declaration rather than a device: a device would have to be somewhere, and would fail when that somewhere failed. A question does not have a location.

Where the claim overreaches

The claim is a slogan, and slogans need narrowing until they're true.

§3's type ladder and §7's record shapes are **not** consequences of this architecture. They are facts about an API, decisions somebody made about what to call things and how many layers to stack. You cannot derive "NodePort also allocates a cluster IP" from "a Service is a label query," and you cannot derive `_port-name._port-protocol`. from anything at all. Those you memorise, the same as anyone.

Everything else in this chapter, you can rebuild from §1's model plus the observation above. That is a genuinely useful ratio, and it is worth knowing which side of the line each fact sits on before you spend study time on it.

🔒 **Worth Securing:** The practical form of the Zenith, for when Kubernetes networking does something you didn't expect. Ask two questions: **what does the selector currently match?** and **which loop hasn't caught up yet?** Between them they cover most of it — and both are answerable with `kubectl` in under a minute, which is more than can be said for most debugging heuristics.

[cross-bearing: see Ch 15 §7 — the control loop, generalized, pointed at a Git repository]. This chapter observes that Kubernetes networking is *made of* control loops. Chapter 15 makes the larger argument, and it is the structural claim this whole book is building toward. Don't get ahead of it.

The boundary

Everything in this chapter has been about the inside of the cluster.

Every name you learned resolves to something with a Pod behind it — or, in `ExternalName`'s case, to a name that isn't Kubernetes' business at all. Every address you learned is reachable only by things that are already in. Even `NodePort` and `LoadBalancer`, the two types that reach outward, do it by *offering a door*, not by describing what comes through it or where it should go.

Chapter 10 crosses that boundary properly: one address serving many services, routing decisions made on the contents of an HTTP request rather than on a destination address, and a rule about what happens when you create the object and nobody has installed the thing that acts on it.

Exam Alert! 🚨

High-Priority Topics

1. **Every Pod gets a unique cluster-wide IP**, and all Pods reach all Pods **without NAT and without proxies** — same node or across nodes [source: k8s-docs-network-model-2026-08-23].
 2. **The Pod holds the IP; its containers share it** and communicate over `localhost` [source: k8s-docs-pods-2026-08-24].
 3. **ClusterIP is the default Service type**, and is reachable only from inside the cluster [source: k8s-docs-service-2026-08-23].
 4. **NodePort also allocates a cluster IP** — the documentation is explicit that Kubernetes sets one up "the same as if you had requested a Service of type: ClusterIP" [source: k8s-docs-service-2026-08-23].
 5. **Kubernetes does not provide a load balancer**. You supply one, or integrate with a cloud provider [source: k8s-docs-service-2026-08-23].
 6. **ExternalName is a CNAME with no proxying of any kind** [source: k8s-docs-service-2026-08-23].
 7. **clusterIP: None is deliberate** — DNS returns the Pod addresses directly [source: k8s-docs-service-2026-08-23].
 8. **A Service without a selector is a supported pattern**, backed by manually managed EndpointSlices [source: k8s-docs-service-2026-08-23].
 9. `<service>.<namespace>.svc.<cluster-domain>`, and a **bare name resolves in the local namespace only** [source: k8s-docs-dns-pod-service-2026-08-23] [source: k8s-docs-namespaces-2026-08-23].
 10. **kube-proxy implements the virtual IP for every type except ExternalName; iptables is the default mode** [source: k8s-docs-virtual-ips-kube-proxy-2026-08-23].
 11. **A Pod must be Ready to be an endpoint** [source: k8s-docs-pod-lifecycle-2026-08-23].
 12. **Kubernetes defines the network model; a CNI network plugin is required to implement it** [source: k8s-docs-network-plugins-2026-08-24].
-

Common Traps — every one of these is a documented mistake with a specific correct answer, which is to say every one is a hazard somebody has already run onto and marked. Don't let the wrong version be the one that's fresh when you sit down.

The trap	The correction	Where it's defused
"Pods need NAT or a proxy to reach Pods on other nodes"	Direct, no NAT, no proxy — that's rule 3	\$1, Bearings #1 item 1
"Each container in a Pod gets its own IP"	The Pod has the address; containers share it	\$1 Snag, Bearings #1 item 2
"NodePort replaces ClusterIP"	NodePort also allocates a cluster IP	\$3 Fixed Point, Bearings #1 item 3
"LoadBalancer means Kubernetes provides a load balancer"	It provides none. You supply one	\$3 Fixed Point, Bearings #1 item 5
"ExternalName proxies traffic"	CNAME only. No proxying of any kind	\$3 Navigational Hazards, Bearings #1 item 5
"A headless Service is a broken Service"	clusterIP: None is a value somebody typed	\$5 Fixed Point, Bearings #2 item 4
"A Service without a selector is invalid"	Supported pattern, used for external backends	\$5, Bearings #2 item 5
"A bare service name works across namespaces"	Local namespace only — and it may succeed <i>wrongly</i>	\$7 Navigational Hazards, Bearings #3 item 4
"Something is listening on the cluster IP"	Nothing is. It's a forwarding rule, not a socket	\$6 Worth Securing, Bearings #3 item 2
"kube-proxy is required"	Optional, if the plugin does equivalent forwarding	\$6 Closer Look, Bearings #3 item 3
"Headless Services have no DNS record"	Same name form; different answer	\$7, Bearings #3 item 5
"A Service with no endpoints is broken"	Correct Service; selector matches nothing, or Pods not Ready	\$4 Snag, Bearings #2 item 3
"A terminating Pod leaves the Service instantly"	Retained with ready: <code>false</code> , so traffic drains	\$4 Closer Look
"Kubernetes ships the network"	It defines the <i>requirements</i> ; a CNI plugin is required to implement them	\$1 Fixed Point

The most valuable one on that list is "something is listening on the cluster IP." It has one clean correct answer, and that answer follows directly from what kube-proxy is documented to do: configure the node to *capture* traffic addressed to the Service's cluster IP and port, and *redirect* that traffic to one of the Service's endpoints [source: k8s-docs-virtual-ips-kube-proxy-2026-08-23]. Capture-and-redirect requires nothing bound to that address, which is exactly why nothing is bound to it. Believing the wrong version makes roughly half of Kubernetes networking incomprehensible, including several things Chapter 10 will assume you understand.

Practice Questions

Twenty-three questions. Four of them draw on earlier chapters and are tagged; one is deliberately not multiple choice. Answers and explanations follow the full set — attempt them all before looking.

1. Which of the following is required by the Kubernetes network model?

A) Pods communicate across nodes through a NAT gateway maintained by the kubelet B) All Pods can communicate with all other Pods, on the same node or on different nodes, without proxies or address translation C) Each container in a Pod receives its own cluster-wide IP address D) Pods on different nodes communicate via the node's external IP and a mapped port

2. A Pod is running three containers. **[retrieval: ch5]** How many IP addresses does the cluster allocate for that Pod, and how many entries would that Pod contribute to a matching Service's EndpointSlice?

A) Three addresses, three entries B) Three addresses, one entry C) One address, three entries D) One address, one entry

3. Which statement about CNI is correct?

A) CNI is the Kubernetes-built-in networking implementation, enabled by default B) CNI is the interface through which a network plugin implements Pod networking; the plugin is an external binary rather than a Kubernetes component C) CNI is a Kubernetes API object you create to define a Pod network D) CNI is the protocol kube-proxy uses to program iptables rules

4. A Service is created with no type field specified. What type is it, and from where is it reachable?

A) NodePort; from any node's IP at a static port B) ClusterIP; from inside the cluster only C) ClusterIP; from inside the cluster and from any node's IP D) LoadBalancer; from wherever the provider's load balancer is reachable

5. You change a Service from type: ClusterIP to type: NodePort. What happens to the cluster IP?

A) It is released, since NodePort replaces it B) It is retained — Kubernetes sets up a cluster IP for NodePort Services as well C) It is retained but becomes unreachable from inside the cluster D) It is replaced by the node's IP address

6. Your team runs Kubernetes on bare metal with no cloud-provider integration and no load-balancer add-on installed. You create a type: `LoadBalancer` Service. What happens?

- A) Kubernetes provisions a software load balancer on one of the nodes
 - B) The Service is rejected at admission with a validation error
 - C) The Service is created and waits indefinitely for an external address; nothing is broken
 - D) The Service silently falls back to type: `NodePort`
-

7. A Service resolves to `api.vendor.example`, and a client Pod connects to it successfully. An engineer runs a packet capture on the node, expecting to find the connection matched by a kube-proxy rule, and finds no rule for that Service at all. Which type is the Service, and why is there no rule to capture?

- A) `NodePort` — a `NodePort` Service surrenders its cluster IP, so kube-proxy has no address left to program a rule against
 - B) `ClusterIP` — kube-proxy programs rules only for Services that are reachable from outside the cluster
 - C) `ExternalName` — it is implemented purely as a DNS CNAME, and kube-proxy's virtual-IP mechanism covers every type except this one
 - D) `LoadBalancer` — the provider's load balancer handles the traffic, so no node-level rule is needed
-

8. You need clients inside the cluster to reach `analytics.vendor.example`, a SaaS endpoint outside the cluster, using an in-cluster name. Which Service configuration produces a CNAME with Kubernetes entirely out of the traffic path?

- A) type: `ClusterIP` with no selector and manually created `EndpointSlices`
 - B) type: `ExternalName` with `externalName: analytics.vendor.example`
 - C) type: `LoadBalancer` with an annotation naming the vendor
 - D) A headless Service with `externalName` set
-

9. [retrieval: ch4] A Pod carries labels that a `ReplicaSet` selects on and labels that a Service selects on. Someone edits one of those labels. Name both consequences, and say whether the two controllers coordinate.

- A) The Pod leaves both sets simultaneously; the controllers coordinate through the owner reference
 - B) Only the `ReplicaSet` is affected; Services select on owner references, not labels
 - C) Each selector is evaluated independently, so the Pod may drop out of either set, both, or neither
 - D) Nothing happens until the Pod is restarted, at which point both selectors re-evaluate
-

10. Where does Kubernetes record the set of Pods currently backing a Service, and what writes it?

- A) In the Service's `status` field, written by kube-proxy
 - B) In `EndpointSlice` objects, written by the `EndpointSlice` controller
 - C) In the Pods' annotations, written by the kubelet
 - D) In cluster DNS, written by CoreDNS
-

11. [retrieval: ch5] Chapter 5 said a readiness probe controls whether a Pod receives traffic. Name the object that fact is actually implemented in, and say what a failing probe changes about it.

A) The Pod's `status.phase`; a failing probe moves the Pod to `Failed` B) The Service's `spec.selector`; a failing probe removes the Pod's labels from it C) The `EndpointSlice`; a failing probe removes the Pod's address from the Service's endpoints D) The node's iptables rules; a failing probe causes kube-proxy to drop packets to the Pod

12. `kubectl get endpointslices -l kubernetes.io/service-name=web` returns a slice with no endpoints. The Pods you expect are Running. What are the two usual causes?

A) kube-proxy is not running, or the CNI plugin is misconfigured B) The Service's selector does not match the Pods' labels, or the Pods are not Ready C) The Service's DNS record has not propagated yet, or the slice you fetched is a stale cached read D) The Pods are in a different namespace from the Service, or the Service was created before the Pods existed

13. A colleague reports: "I created the Service, it has a cluster IP and a DNS record, but nothing responds and there are zero endpoints." What is the correct characterisation?

A) The Service failed to create properly and should be deleted and recreated B) Nothing is wrong with the Service — it is being reconciled correctly against a set that is currently empty C) The API server rejected the selector and defaulted it to empty D) The DNS record proves the Service works; the problem must be in the client

14. What does setting `.spec.clusterIP` to `None` do, and which workload type requires it?

A) Reserves the Service's name in DNS without allocating any endpoints; required by CronJobs B) Makes the Service reachable only from inside the cluster, with no external path; required by StatefulSets for Pod network identity C) Creates a headless Service — DNS returns the Pod addresses directly instead of one virtual IP; required by StatefulSets for Pod network identity D) Removes the Service's selector, so its endpoints must be maintained by hand; required by Deployments with more than one replica

15. You are migrating a database into the cluster. Today it runs on external hardware; next month it will run as Pods. You want in-cluster clients to use one unchanging Service name throughout, with traffic proxied by the cluster in both phases. What do you configure today?

A) A type: `ExternalName` Service pointing at the database's hostname B) A headless Service with no selector C) A Service with no selector, plus manually managed `EndpointSlices` holding the external address D) A type: `ExternalName` Service now, swapped for a selector-based Service on migration day

16. kube-proxy's modes on Linux are:

A) userspace (default), iptables, IPVS B) iptables (default), IPVS, nftables C) iptables, IPVS (default), CNI D) nftables (default), iptables, kernelspace

17. An engineer SSHes to a worker node and runs `ss -tlnp` looking for a process bound to a Service's cluster IP. What will they find, and why?

A) kube-proxy, because it terminates connections to cluster IPs and forwards them B) One of the backend Pods, because kube-proxy binds the cluster IP to a chosen endpoint C) Nothing — the cluster IP is virtual, existing only as a forwarding rule that captures and readdresses traffic D) CoreDNS, because it owns all cluster-internal addresses

18. Your cluster is running normally, serving traffic to Services, and there are no kube-proxy Pods anywhere. What is happening, and which of §1's four rules is the same component also satisfying?

A) The cluster is broken and Services are failing silently; no rule is being satisfied B) The network plugin implements Service packet forwarding itself — and the same plugin implements the Pod network, satisfying the rule that all Pods can reach all Pods directly C) The control plane is proxying Service traffic through the API server; rule 4, node-agent reachability D) Services are being resolved by DNS to Pod IPs directly, bypassing the need for a proxy; rule 1, unique Pod addresses

19. *Not multiple choice — write the two names out before checking.* A Pod in namespace `billing` must reach a Service named `ledger` in namespace `payments`. The Service exposes a port named `http` over TCP, and the cluster uses the default cluster domain. Write (a) the name the Pod would use for an ordinary address lookup, and (b) the SRV name that would return the port number and hostname for that named port.

20. A StatefulSet's three Pods sit behind a headless Service. A client resolves the Service's name and receives three addresses rather than one. What served that answer, and what name form reaches one specific member?

A) kube-proxy served it from its endpoint cache; individual members are reached at `<pod-ip-dashed>.<namespace>.pod.cluster.local` B) CoreDNS, the cluster's DNS add-on, served it from the Service's endpoints; individual members are reached at `<pod-name>.<service-name>.<namespace>.svc.cluster.local` C) The API server served it directly; individual members are reached by appending an ordinal to the Service's own name D) CoreDNS served it, but only the set is addressable — a headless Service deliberately gives up per-member names

21. [retrieval: ch4] Chapter 4 told you a bare service name resolves within the local namespace. What makes that true — what is configured, and by which component?

A) CoreDNS rewrites bare queries to the caller's namespace, using the caller's source IP B) The Pod's DNS search list contains its own namespace and the cluster's default domain, configured by the kubelet C) The API server rejects cross-namespace DNS queries unless an FQDN is used D) kube-proxy scopes DNS traffic per namespace using iptables rules

22. You meet a Kubernetes networking behavior this chapter never covered. Applying the chapter's method rather than recalling a fact, which pair of questions will locate it?

A) Which node is the traffic on, and which iptables rule matches it B) Which object holds the declaration of intent, and which controller is reconciling that declaration into a published answer C) Which CNI plugin is installed, and which of its features is enabled D) Which DNS record exists for it, and what address that record returns

23. A Service manifest reads `port: 80`, `targetPort: 8080`, `type: NodePort`, and no `nodePort` value. A client inside the cluster connects to the Service's cluster IP on port 80. Which statement is correct?

A) The connection fails, because `port` and `targetPort` must match for in-cluster traffic B) Traffic arrives at the container on port 8080, and a node port from the 30000–32767 range has also been allocated automatically C) Traffic arrives at the container on port 80, and `targetPort` applies only to traffic entering through the node port D) No node port exists, because none was specified in the manifest

Answers with Explanations

1. B.

All Pods can communicate with all other Pods, whether on the same node or on different nodes, and Pods can communicate with each other directly, without the use of proxies or address translation [source: k8s-docs-network-model-2026-08-23].

A describes a NAT gateway, explicitly forbidden by rule 3. **C** is the each-container-gets-an-IP misconception; every container in a Pod shares the network namespace including the IP address and network ports [source: k8s-docs-pods-2026-08-24]. **D** describes host-port mapping, which is the model Kubernetes replaces; it would be the correct answer for almost any non-Kubernetes container platform, which is precisely why it's attractive.

2. D. One address, one entry.

Each Pod is assigned a unique IP address for each address family, and every container in a Pod shares the network namespace including the IP address [source: k8s-docs-pods-2026-08-24]. An EndpointSlice records the Pods currently backing a Service [source: k8s-docs-network-model-2026-08-23] — Pods, not containers.

This is the each-container-gets-an-IP misconception tested at its downstream consequence rather than at its definition. **A** and **B** both assume per-container addressing; **C** assumes per-container endpoints. If you can see that container count and endpoint count are unrelated numbers, the trap has nothing left to catch you with.

3. B.

Network plugins use CNI, the Container Network Interface, to implement pod networking, and CNI plugins are external binaries that Kubernetes executes rather than in-process Kubernetes components [source: k8s-docs-extending-kubernetes-2026-08-23].

A inverts the relationship — CNI is an interface, not an implementation. **C** invents an API object. **D** confuses two unrelated components: kube-proxy programs node rules via OS APIs, and has nothing to do with CNI.

4. B.

ClusterIP exposes the Service on a cluster-internal IP, makes it reachable only from within the cluster, and is the default used if you don't explicitly specify a type [source: k8s-docs-service-2026-08-23].

A fails on its first half before the second one matters: NodePort is never a default, it must be asked for, and asking for it would additionally allocate the cluster IP that B describes [source: k8s-docs-service-2026-08-23]. **C** is the near-miss worth naming: ClusterIP alone does *not* expose anything on node IPs. That is what NodePort adds — and the additivity runs one way only. Getting a cluster IP from NodePort is guaranteed; getting node reachability from ClusterIP is not. **D** is not a default either, and it is the one type on this list that cannot function at all without a component Kubernetes does not supply [source: k8s-docs-service-2026-08-23], which is the subject of question 6.

5. B.

To make the node port available, Kubernetes sets up a cluster IP address, the same as if you had requested a Service of type: `ClusterIP` [source: k8s-docs-service-2026-08-23].

A is the trap in its purest form — the belief that types are alternatives rather than layers. **C** invents a restriction. **D** confuses an address with a reachability mechanism: the node IPs are *additional* paths, not a substitute address.

6. C.

Kubernetes does not directly offer a load balancing component; you must provide one, or you can integrate your Kubernetes cluster with a cloud provider [source: k8s-docs-service-2026-08-23]. With

neither present, nothing acts on the request.

A is the trap. **B** is wrong because the object is entirely valid — validity and effect are different questions. **D** invents a fallback that would be surprising and undocumented behavior.

Nothing here is broken and nothing needs fixing. The Service exists and is reconciled; only its external address is absent.

7. C.

ExternalName maps the Service to the contents of the `externalName` field by configuring the cluster's DNS to return a CNAME record with that external hostname value; **no proxying of any kind is set up** [source: k8s-docs-service-2026-08-23]. kube-proxy provides the virtual IP mechanism for Services of type *other than* ExternalName [source: k8s-docs-virtual-ips-kube-proxy-2026-08-23], so there is no rule to find, because ExternalName is the one type outside its scope.

A is the types-are-alternatives trap wearing a diagnostic disguise: a NodePort Service keeps its cluster IP, because Kubernetes sets one up for NodePort Services as well [source: k8s-docs-service-2026-08-23]. There would be rules for both paths, not none. **B** inverts kube-proxy's scope — ClusterIP is the type it most obviously serves. **D** assumes an external load balancer displaces the node-level rules; it does not, because LoadBalancer is inside the "other than ExternalName" set too.

8. B.

ExternalName produces a CNAME and no proxying [source: k8s-docs-service-2026-08-23], so the client connects directly to the vendor.

A would also work as a way to reach the vendor, but it puts kube-proxy in the path — traffic goes to a cluster IP and is redirected — which is exactly what the question excludes. **C** and **D** are invented configurations; `externalName` is meaningful only with `type: ExternalName`.

9. C.

A Service uses labels to allow the control plane to determine which EndpointSlice objects are used for it [source: k8s-docs-garbage-collection-2026-08-24], and a selector is a query over labels. Two controllers evaluating two selectors against one Pod produce two independent results.

A confuses ownership with selection — the two mechanisms are explicitly distinguished, and owner references exist so different parts of Kubernetes avoid interfering with objects they don't control [source: k8s-docs-garbage-collection-2026-08-24]. **B** is wrong twice: a Service selects on labels, not on owner references, and the ReplicaSet holds no privileged position over the Service — both are ordinary selector evaluations by ordinary controllers, and neither is told about the other. **D** invents a restart trigger; selectors are evaluated continuously by watching controllers, not at Pod startup.

10. B.

Kubernetes automatically manages EndpointSlice objects to provide information about the Pods currently backing a Service [source: k8s-docs-network-model-2026-08-23]; the EndpointSlice controller populates them to provide a link between Services and Pods [source: k8s-docs-cluster-architecture-2026-08-23].

A puts the answer in the wrong object and names the wrong writer — kube-proxy *reads* EndpointSlices [source: k8s-docs-virtual-ips-kube-proxy-2026-08-23]. **C** puts the answer on the wrong object too, and names a component that never participates: the kubelet reports the status of Pods on its own node, whereas endpoint membership is computed centrally by a controller reading the API [source: k8s-docs-cluster-architecture-2026-08-23], not written per-Pod by each node. **D** confuses a publisher with a source of truth: DNS publishes the answer; it does not compute it.

11. C.

If a readiness probe fails, the endpoints controller removes the Pod's IP address from the endpoints of all Services that match the Pod [source: k8s-docs-pod-lifecycle-2026-08-23]; a Pod's Ready condition means it should be added to the load balancing pools of all matching Services [source: k8s-docs-pod-termination-2026-08-24].

A is a real and important distinction: a failing readiness probe does *not* change the Pod's phase, kill it, or restart it. That's the liveness probe's job. The Pod keeps running, keeps being counted by its controller, and simply stops receiving traffic. **B** would mean readiness edits your Service, which it does not. **D** describes a downstream effect (kube-proxy reprograms rules because the slice changed) as though it were the mechanism.

12. B.

If the endpoints don't match the Pods you expect, the Service's selector probably does not match the Pods' labels, or the Pods are not Ready [source: k8s-docs-debug-pods-2026-08-23].

A names data-plane components, and it is the most tempting option because the symptom presents as a networking failure. But endpoint membership is computed entirely in the control plane, by a controller reading the API [source: k8s-docs-cluster-architecture-2026-08-23]. Neither kube-proxy nor the CNI plugin can empty a slice; both are downstream readers of it.

C pairs two instincts imported from outside Kubernetes. DNS propagation delay is a real phenomenon in the public DNS and has no bearing on an EndpointSlice, which you read from the API server directly. And kubect1 reads through to the API server rather than serving you a cached copy — a stale-read explanation would let you off the hook for a diagnosis you have not made yet.

D is the closest of the three, and worth taking seriously. Its first half is real: a Service's selector is namespace-scoped, so Pods in another namespace are never selected — but that is a *special case of the first documented cause*, not a separate one, which is why the option's framing is wrong even where its fact is right. Its second half is flatly wrong and is the reason to reject the pair: creation order does not

matter to a control loop. The controller reconciles continuously, so a Service created before its Pods picks them up the moment they appear.

13. B.

The Service has an address and a DNS record because those are allocated on creation, independent of whether any Pod matches. The EndpointSlice is empty because the selector's current answer is the empty set, which the controller has computed correctly.

A is the instinct to fix by recreation, and it will produce an identical Service with an identical empty slice. **C** invents an API server behavior. **D** is the reasoning error worth naming: the presence of a name and an address proves that *reconciliation is happening*, not that it found anything.

This is the second instance in the chapter of the same shape — a valid object whose effect depends on something that isn't there. Chapter 10 gives it a name.

14. C.

You create headless Services by explicitly specifying "None" for `.spec.clusterIP`; for headless Services with selectors, DNS returns A or AAAA records pointing directly to the Pods [source: k8s-docs-service-2026-08-23]. StatefulSets currently require a headless Service to be responsible for the network identity of the Pods [source: k8s-docs-statefulset-2026-08-24].

B is the option to dwell on, because half of it is correct. Its workload clause is right — StatefulSets do require a headless Service — and two options here name StatefulSets precisely so that knowing the workload does not hand you the answer. Its definition clause is the confusion this question exists to catch: "reachable only from inside the cluster" is what *ClusterIP* means, a rung on the type ladder. Headless sits on a different axis entirely. It is about whether a virtual IP is allocated at all, not about who may reach one.

A invents a behavior and attaches it to a workload with no Service requirement of any kind. A headless Service with selectors most certainly has endpoints — returning them directly is the entire point.

D crosses the two axes of §5's four-cell table. Headless (`clusterIP: None`) and selectorless are independent choices; a Service can be either, both, or neither. Setting `.spec.clusterIP` to None does not touch the selector, and Deployments require neither.

15. C.

A Service used with a corresponding set of EndpointSlice objects and without a selector can abstract other kinds of backends, including ones that run outside the cluster — an external database, or a workload being migrated [source: k8s-docs-service-2026-08-23].

A would give clients the vendor hostname directly, meaning that when you migrate you must change what clients resolve *and* the connection semantics change — no proxying today, proxying tomorrow. The

question specifically asks for proxying in both phases. **B** removes the single address, which the clients want.

D is the plan most teams actually propose, and it is worth understanding why it loses to **C** rather than merely being wrong. Today it behaves exactly like **A**: a CNAME with no proxying [source: k8s-docs-service-2026-08-23], so the connection semantics still change on migration day. And the swap is a visible cutover, which is the thing the requirement was written to avoid. **C** keeps one name, one cluster IP, and one proxying path throughout; the only thing that changes is the contents of a slice.

The migration itself follows from the two facts above rather than from any documented procedure: add the selector, and the controller takes over the slice you had been writing by hand. Clients notice nothing.

16. B.

On Linux, the available modes are iptables (the default), IPVS, and nftables (GA since Kubernetes 1.33); on Windows there is only kernelspace [source: k8s-docs-virtual-ips-kube-proxy-2026-08-23].

A names a mode that is not among the documented Linux modes. **C** mislabels the default and lists CNI, which is the interface a network plugin implements pod networking through — not a kube-proxy mode, and not even the same layer of the stack. **D** mislabels the default and places the Windows-only mode in the Linux list.

17. C.

kube-proxy configures the node to capture traffic to the Service's `clusterIP` and port and redirect it to one of the Service's endpoints [source: k8s-docs-virtual-ips-kube-proxy-2026-08-23]. Capture-and-redirect is not the same as listen-and-forward. Nothing binds the address.

A and **B** both assume something terminates the connection at the cluster IP. **D** confuses DNS with addressing entirely — CoreDNS answers name queries; it owns no addresses.

18. B.

If you use a network plugin that implements packet forwarding for Services by itself, providing equivalent behavior to kube-proxy, you do not need to run kube-proxy [source: k8s-docs-cluster-architecture-2026-08-23]; Cilium can act as a replacement [source: k8s-docs-cluster-addons-2026-08-24]. And the plugin implementing Service forwarding is the same plugin implementing Pod networking via CNI [source: k8s-docs-extending-kubernetes-2026-08-23], so it is also satisfying the model's requirement that all Pods reach all Pods directly [source: k8s-docs-network-model-2026-08-23].

A assumes kube-proxy's absence is a defect. **C** invents API-server proxying. **D** describes headless-Service behavior and generalizes it to every Service, which is wrong for anything with a cluster IP.

19.

(a) `ledger.payments.svc.cluster.local`

A normal Service is assigned an A or AAAA record of the form `my-svc.my-namespace.svc.cluster-domain.example`, resolving to the Service's cluster IP [source: k8s-docs-dns-pod-service-2026-08-23]. With the default cluster domain that is `<service-name>.<namespace-name>.svc.cluster.local` [source: k8s-docs-namespaces-2026-08-23].

(b) `_http._tcp.ledger.payments.svc.cluster.local`

Named ports get an SRV record of the form `_port-name._port-protocol.my-svc.my-namespace.svc.cluster-domain.example` [source: k8s-docs-dns-pod-service-2026-08-23]. The two leading underscore labels are the port's name and its protocol, in that order, prefixed onto the ordinary Service name from part (a).

Two things to check yourself on. First, the shape is stable: everything to the right of the Service name is the same in both answers, and the SRV form only prepends. If you wrote something with `payments` and `ledger` in the other order, that is the error to fix before exam day, because it will produce a name that resolves to nothing rather than an error you can read.

Second — and this is the part that costs points — the bare name `ledger` is not an answer to either half. From a Pod in `billing`, a bare name is tried against that Pod's own search list, which begins with its own namespace [source: k8s-docs-dns-pod-service-2026-08-23]. It will not reach `payments`. If a `ledger` Service also happens to exist in `billing`, the bare name will resolve — to the wrong one, silently.

20. B.

DNS is a built-in Kubernetes service launched automatically as a cluster addon [source: k8s-docs-dns-cluster-addon-2026-08-24], and CoreDNS is what serves it [source: k8s-docs-cluster-addons-2026-08-24]. For headless Services with selectors, DNS returns A or AAAA records pointing directly to the Pods [source: k8s-docs-service-2026-08-23] — three addresses for three Pods, which is exactly what the client saw. Individual members of a StatefulSet are addressable at `$(podname).$(governing service domain)` [source: k8s-docs-statefulset-2026-08-24], which expands to the form in B.

Note that both halves of the answer come from the same place. The set and the member are two questions put to one component.

A is wrong on both halves. kube-proxy has no role in name resolution at all — and in a headless Service there is no virtual IP for it to program in the first place. The `pod.cluster.local` form it names is real, but it is the Pod record, keyed on a dashed rendering of the Pod's IP address [source: k8s-docs-dns-pod-service-2026-08-23], which is exactly the unstable identifier a StatefulSet exists to avoid.

C puts the API server in the DNS path, which it never occupies, and grafts the ordinal onto the wrong label. The ordinal is already inside the Pod's own name; the member's name is the Pod name under the governing Service's domain, not the Service name with a number attached.

D inverts the reason headless exists. Per-member addressability is the point, and it is why a StatefulSet requires a headless Service to be responsible for the network identity of its Pods [source: k8s-docs-

statefulset-2026-08-24].

21. B.

By default, a client Pod's DNS search list includes the Pod's own namespace and the cluster's default domain [source: k8s-docs-dns-pod-service-2026-08-23], and the kubelet configures Pods' DNS so that running containers can look up Services by name [source: k8s-docs-dns-pod-service-2026-08-23].

A would be a plausible design and is not the one used; the behavior is ordinary DNS search-domain resolution on the *client* side, not rewriting on the server side. **C** invents an API server role in DNS. **D** invents a kube-proxy role in DNS.

Converting the rule into a mechanism is the point of this item. Once you know it's a search list, the cross-namespace failure mode stops being arbitrary: the caller's own namespace is tried *first*, so a same-named local Service wins, silently.

22. B.

This is the only question in the set that tests the method rather than a fact, and it is the one worth carrying out of the chapter. Every mechanism in §1 through §7 had the same two-part shape. A Service declares a selector; the EndpointSlice controller publishes the current answer [source: k8s-docs-cluster-architecture-2026-08-23]. An EndpointSlice declares a set of endpoints; kube-proxy watches the Service and EndpointSlice objects and maintains the node's rules accordingly [source: k8s-docs-virtual-ips-kube-proxy-2026-08-23]. A Service's name declares an identity; cluster DNS publishes what it currently resolves to. Find the object and find the loop, and you have located a behavior you have never read about.

A is a node-level answer to a cluster-level question. It describes one implementation kube-proxy happens to use on Linux today, and the mode is configurable and the component itself optional [source: k8s-docs-cluster-architecture-2026-08-23], so the method fails on the first cluster that does it differently.

C assumes the plugin is where every networking answer lives. It is where the *model* is implemented, but Services, endpoints and names are API-level and are the same whichever plugin you installed.

D stops one step short, and stops in the same place question 10's option D did: DNS publishes answers, it does not compute them. Naming the publisher without naming the loop behind it leaves you able to read the current state and unable to explain why it is what it is.

23. B.

A Service maps any incoming port to a `targetPort`, so a client connecting on 80 reaches the container on 8080 [source: k8s-docs-service-ports-2026-08-24]. And because no `nodePort` was specified, one is allocated from the `--service-node-port-range`, default 30000–32767 [source: k8s-docs-service-ports-2026-08-24].

A invents a constraint; the whole point of the two fields is that they may differ. **C** reverses the hop — `targetPort` describes the container end of every connection to the Service, not just the ones arriving through a node port. **D** is the trap worth naming: omitting `nodePort` does not mean no node port. It means you did not choose which one, so the control plane chose for you — the same shape as omitting type and getting `ClusterIP`, and as omitting `targetPort` and getting port's value. Kubernetes fills in defaults; it does not skip the field.

Chapter Summary

Concept	Remember This
Network model	Four rules: unique cluster-wide Pod IP; all Pods reach all Pods; no NAT, no proxies ; node agents reach local Pods
Pod IP	Belongs to the Pod . Containers share it and use <code>localhost</code>
CNI	Kubernetes defines the model; a plugin implements it. Second of the four pluggable interfaces this book tracks
Service	A stable, long-lived address for a set of Pods that is expected to change . An object, not a running thing
ClusterIP	The default type. Inside the cluster only
NodePort	Every node's IP at a static port — and also a cluster IP
LoadBalancer	Exposed externally by a load balancer the provider supplies . Kubernetes supplies none
ExternalName	A CNAME . Not on the ladder. No proxying of any kind
Selector	The question . A query over Pod labels
EndpointSlice	The written-down answer , maintained by the EndpointSlice controller
Readiness	The gate . Matching the selector is necessary and not sufficient
Empty endpoints	Selector mismatch, or Pods not Ready. Two bugs, two files
Headless	<code>clusterIP: None</code> — deliberate. DNS returns the Pod addresses. StatefulSets need one
No selector	Supported. Manually managed EndpointSlices, for backends outside the cluster
kube-proxy	Virtual IP for every type but ExternalName . Watches Service + EndpointSlice. iptables is the default
Cluster IP	Virtual . kube-proxy captures traffic addressed to it and redirects; nothing is listening. A rule, not a socket
Cluster DNS	CoreDNS , a built-in addon, serves every record below. It publishes the answer; it does not decide it
Service DNS	<code><service>.<namespace>.svc.<cluster-domain></code> — cluster IP for normal, all Pod IPs for headless
Bare name	Local namespace only . May succeed against the wrong Service
The Zenith	A Service is a label query with a name . Three loops publish its answer in three formats

⚓ **Safe Harbor** — the cluster's own network, and every object that names it.

You arrived at this chapter able to run workloads and unable to connect them. You now know what guarantees the cluster network makes, what a Service actually is, how its backends are computed and gated, what programs the node, and what the reader — sorry, the *client* — types. A Service is a name on the chart: what sits under it changes with the tide, and the name does not.

That is the inside of the cluster accounted for. Domain 2's Networking competency runs wider than one chapter — traffic arriving from outside, and the policy that decides which of it is allowed, is Chapter 10's work — but Chapters 10, 13 and 16 all stand on what you just finished.

Voyage Progress: 📖 → ⏏ **Ch 9 of 20** → 🔍

The Voyage Ahead

Everything you just learned works inside a boundary: water the cluster itself controls, where every address in play is one the cluster handed out.

Every name resolved to something already on this side of it. Every address was reachable by things already inside. NodePort and LoadBalancer reach outward, but only by *offering a door* — a port on every node, an address from a provider. Neither of them knows or cares what comes through it. They move packets to a Service, and the Service moves them to a Pod, and at no point does anything open the crate to see what the cargo is.

That's the ceiling §3 named. One external address per Service, which does not scale past a handful. No protocol awareness at all, so `shop.example.com/checkout` and `shop.example.com/catalog` are indistinguishable to every mechanism in this chapter.

Chapter 10 crosses the boundary. It introduces the objects that route on the *contents* of an HTTP request — hostnames, paths, headers — so that one address can serve many services, and something standing at the entrance finally reads the writing on the crate before deciding where it goes. It explains why the Kubernetes project now recommends one API over the one most people learned first, and what "frozen" means in that recommendation, which is a more precise word than it looks and worth exactly the attention you'd give a definition on an exam. And it introduces NetworkPolicy, which is what rule 2's "*barring intentional network segmentation*" was quietly gesturing at all along [*cross-bearing: see Ch 10 §6 — NetworkPolicy*].

It also opens with an object that you create, and that does nothing at all, because the component that acts on it has not been installed. You have seen that shape twice in this chapter now. Chapter 10 gives it a name, and once it has a name you will start seeing it everywhere.

"A light is known by its character, not its position — and only because someone ashore keeps it burning to the same pattern." — Lodestar Ledgers

Chapter 10: Traffic from Beyond the Cluster

"Frozen, superseded, and inert without a controller"

Domain: Container Orchestration (competency: Networking) | Domain weight: 28% [source: cncf-kcna-curriculum-pdf-2026-08-23] | Complexity: Mixed | Novelty: Moderate

The 28% figure is CNCF's published weight for the whole Container Orchestration domain. The published curriculum gives four domain-level percentages and nothing finer — no weight is attached to Networking, or to any other competency [source: cncf-kcna-curriculum-pdf-2026-08-23]. This book's allocation of that 28% across its Part III chapters is therefore the author's, derived from curriculum breadth rather than from any published figure. See the front matter's weight-allocation disclosure.

Attention Budget

Total time: ~125 minutes | Recommended: Split across 2 sessions

Section	Time	Attention Cost	Best Time to Study
📍 Soundings	10 min	Low	Anytime
\$1 Where LoadBalancer Runs Out	6 min	Low	Anytime
\$2 Routing by Host and Path	12 min	Medium	Mid-session
\$3 The Object Is Not the Implementation	8 min	Medium	Mid-session
🕒 Taking Your Bearings #1	6 min	Medium	After a brief break
\$4 Frozen, Not Deprecated	6 min	Medium	Peak attention
\$5 Roles, Not Just Routes	12 min	High	Peak attention
🕒 Taking Your Bearings #2	6 min	Medium	After a brief break
\$6 Allowing, Never Denying	14 min	High	Start of session 2
\$7 What NetworkPolicy Cannot Do	8 min	Medium	Mid-session
🕒 Taking Your Bearings #3	6 min	High	After a brief break
\$8 Nothing Happens Without a Controller	5 min	Low	Anytime
Exam Alert + Practice Questions	25 min	High	Fresh session

Attention Cost Key:

- **Low:** Concrete, familiar concepts — study anytime
- **Medium:** New concepts requiring focus — study when alert
- **High:** Abstract or complex material — study at peak attention

The session split goes between \$5 and \$6. This chapter is two chapters wearing one number: an API-generations arc (\$1–\$5) and a policy arc (\$6–\$7), joined at \$8. \$6 asks you to unlearn a firewall instinct you have probably held for a decade, and it will land better on a fresh head than on a tired one. The split puts roughly 66 minutes before the break and 58 after it, the last 25 of which are the Exam Alert and the practice set. Take those as a third sitting if the second runs long.

If you only have 15 minutes: read \$6. It is the section most likely to overturn something you currently believe, and believing it costs you points. If you have 35, add \$3 and \$4 and take Taking Your Bearings #3 — the controller rule, the freeze, and the policy defaults, plus the checkpoint that tests the hardest of the three.

"An object is a record of intent. Intent does not act." — Lodestar Ledgers

🔊 Soundings

Before reading this chapter, try these eight questions. Your score determines how to approach the content; no shame in any score, just different reading strategies. Four of these test priors you arrive with; four are deliberate retrieval from Chapters 3 and 9.

1. You run one web server on one IP address, and it serves `shop.example.com` and `blog.example.com` differently. What does the server have to look at to tell the two apart, and at what point in the connection does that information become available?
2. You have fifty services that all need to be reachable from outside the cluster. Chapter 9 gave you a Service type that does exactly that. Name it, then say what fifty of them costs you: addresses, and anything else you can think of.
3. Which of Chapter 9's four Service types can send `/checkout` to one set of Pods and `/catalog` to another? Say why.
4. On the firewalls you have configured or worked behind: if a packet matches no rule at all, is it allowed or dropped? And if one rule permits something a later rule forbids, which one wins?
5. Two APIs. One is announced as **deprecated**. The other is announced as **no longer being developed, with no further changes**. For each, say whether you would expect it to be removed, and whether you would start a new project on it today.

6. Chapter 3 asked you to remember a one-sentence rule about objects and the components that act on them. Write it down. Then name one place you have already met it.
7. Chapter 9's second network-model rule said all Pods can reach all Pods — "*barring intentional network segmentation*." What do you think that hedge was pointing at, and at what layer would something have to sit to enforce it?
8. A client connects to `https://shop.example.com` and the request eventually reaches an application server. Where does the TLS connection actually end? Who holds the certificate and private key, and what does the application server see arriving?

Click for answers + reading strategy

1. **The Host header, available only after the TCP connection is established and the request has been sent** [source: `k8s-docs-gateway-api-depth-2026-08-24`]. The client's DNS lookup resolved the name to an address before any traffic moved; the name itself travels inside the request. (HTTPS carries an earlier tell in the **SNI (Server Name Indication)** field of the TLS handshake — traffic for several hostnames can be multiplexed on a single port that way, where the proxy terminating TLS supports SNI [source: `k8s-docs-ingress-depth-2026-08-24`] — but the routing decision is conventionally made on the Host header.)
2. **Service.Type=LoadBalancer** [cross-bearing: see *Ch 9 §3 — the Service type ladder*]. Fifty of them costs fifty external addresses, fifty provisioned load balancers, fifty line items on a cloud bill, and fifty things to configure, monitor, and eventually decommission.
3. **None of them.** None of them reads HTTP. Three of the four operate on addresses and ports; the fourth, `ExternalName`, is a DNS alias with no proxying set up at all [source: `k8s-docs-service-2026-08-23`]. `/checkout` and `/catalog` are bytes inside an HTTP request, and nothing in Chapter 9 opens the request to look.
4. Most likely **dropped**, and **the deny wins**. That is how ordinary packet-filtering firewalls behave, and it is a sound instinct nearly everywhere.
5. **Deprecated** implies eventual removal, so you would avoid starting new work on it. **No longer developed** says nothing about removal; it says the thing is finished. It may well be permanent. You might still use it, knowing it will never gain a feature.
6. **An object without its component does nothing.** You have met it at least twice: a type: `LoadBalancer Service` on a cluster with no load balancer to provision, and a `Service` whose selector matched no Pods.
7. Something has to be able to restrict Pod-to-Pod reachability. Since the CNI plugin is what actually moves the packets, enforcement would have to live down there, wherever the packets themselves are handled.

8. **At the reverse proxy or load balancer at the edge**, which holds the certificate and private key. The application server behind it receives plain HTTP [source: k8s-docs-ingress-depth-2026-08-24].
-

If you got 6+ right: Skim §1 and §2 — you have the priors. Read §4 carefully anyway; it is a word-level distinction and skimming is exactly how people lose that point. Then read §6 and §7 at full attention regardless of your score.

If you got 3–5 right: Read at normal pace. The material is in reach and this chapter is calibrated for you.

If you got 0–2 right: Read carefully. And if questions 2, 3, or 7 were among your misses, **go back to Chapter 9 first**. Not "review" — go back. This chapter re-teaches no part of the Service model. §1's argument is arithmetic on Chapter 9's ladder, §2's backends are Chapter 9's Services, and §6's subjects are Chapter 9's Pod IPs. A re-read of Chapter 9 will buy you more than a careful read of this chapter will.

Why This Chapter Matters

Chapter 9 ended by naming a ceiling. One external address per Service is reasonable when you have one Service. It is absurd when you have fifty. And no Service type in that chapter can tell

`shop.example.com/checkout` from `shop.example.com/catalog`, because at the layer those mechanisms operate on, the difference between those two requests is bytes in a payload that nothing is opening.

This chapter is what sits above that ceiling [*cross-bearing: see Ch 9 §3 — the Service-type ladder and its limits*].

But the more valuable thing here is not the objects. It is a question.

Every chapter so far has rewarded the same professional instinct: get the object right, and the cluster does the rest. Write a correct Deployment and Pods appear. Write a correct Service and a stable name appears. That instinct has been reliable for nine chapters, and this is the chapter where it stops being enough. This is where you learn the question that separates people who have run Kubernetes from people who have read about it.

The question is not *did I write the object correctly*. It is **is anything watching this object**.

Orders written out in a fair hand, with every heading correct, change nothing at all if nobody has been detailed to read them.

A well-formed Ingress on a cluster with no Ingress controller is a correct object that does nothing. A well-formed NetworkPolicy on a network plugin that does not implement NetworkPolicy is a correct object that does nothing. Unlike the Ingress, it does nothing *quietly*. Unreachability is loud: a pager goes off, a customer complains, somebody is looking at it within minutes. Unrestricted reachability is silent; everything works exactly as it did, which is indistinguishable from a policy working perfectly against

traffic nobody happens to be sending. That asymmetry is the most valuable thing in this chapter, and you should have it in hand before you meet either object.

Chapter 9 told you that Chapter 10 would give a name to a shape you had already met twice. Here is the part Chapter 9 did not say: you will meet it **twice more in this chapter alone**, in two objects that have nothing to do with each other. This is where it stops being a curiosity and becomes a rule you can apply to things this book never mentions.

The stakes, stated flat. This is the smallest allocation in Part III, and the material still deserves full attention for two specific reasons. First, *frozen* versus *deprecated* is the most precise word-level distinction in the curriculum, and precise distinctions are what multiple-choice exams are built out of. Second, *NetworkPolicy* carries more cataloged traps than any other single topic in this book, and every one of them is a case where your existing firewall intuition hands you the wrong answer with complete confidence.

Two reasons. The chapter does not need a third.

What You'll Learn

By the end of this chapter, you'll be able to:

- **Place** any exposure mechanism on the correct side of the layer boundary — what operates on addresses and ports, and what opens the request and reads it.
- **State** what Ingress does, what it explicitly does not do, and which two protocols are the whole of its remit.
- **Distinguish** a simple fanout from name-based virtual hosting, and both from the DNS-based service discovery you learned last chapter.
- **Explain** why a correctly written Ingress can have no effect whatsoever, and name the thing whose absence causes it.
- **Say what frozen means** — precisely, in both of its halves — and why it is not the same word as *deprecated*.
- **Name** the three role-mapped Gateway API resources and the organizational role each one belongs to.
- **Predict** whether a given connection is allowed under a given set of *NetworkPolicies*, using rules that are additive, allow-only, and enforced at both ends of the connection — and starting from the posture a Pod holds before any policy has selected it.
- **Describe** what becomes of a *NetworkPolicy* that the cluster's network plugin does not implement, and why that failure is the harder of this chapter's two to notice.

You'll also acquire one rule that outlives this chapter: an object without its component does nothing. You'll use it in Chapter 13, in Chapter 17, and on things this book never gets to.

§1 — Where LoadBalancer Runs Out

You already have the argument. Chapter 9 made it, and made it recently.

One external address per Service. Fifty Services, fifty addresses, fifty provisioned load balancers, fifty bills, fifty things to manage. And no Service type in that chapter reads HTTP, so a single address cannot serve two paths. A Service knows an address and a port, and stops there *[cross-bearing: see Ch 9 §3 — the Service-type ladder and the exposure ceiling]*.

That is the ceiling. This section does not re-derive it. It names the vocabulary the rest of the chapter runs on, and gets out of the way.

The layer boundary

Everything in Chapter 9 operates at what practitioners call **layer 4**. A Service moves packets to an address and a port and has no opinion about what those packets contain. Everything in §2 through §5 operates at **layer 7**: it reads the request — the host it was addressed to, the path it asked for, sometimes the headers it carries — and decides where to send it on that basis.

The documentation sorts the same two groups without putting layer numbers on them. It describes Ingress as protocol-aware HTTP/HTTPS routing using URIs, hostnames, and paths; Gateway API as dynamic infrastructure provisioning and advanced traffic routing; and type: LoadBalancer as the simpler but less configurable mechanism for getting traffic into a cluster [source: k8s-docs-network-model-2026-08-23]. The one place it does reach for **OSI (Open Systems Interconnection)** layer numbers is NetworkPolicy, which it places at the IP address or port level — layer 3 or 4 [source: k8s-docs-network-model-2026-08-23]. That is not a coincidence, and §6 is where it pays.

Keep this boundary marked. It is not decoration and it is not only about §2. §6 goes back *down* to layers 3 and 4, and a reader who has lost the ladder will experience NetworkPolicy as an unrelated topic that happened to land in the same chapter.

☆ **Fixed Point:** Everything in Chapter 9 moves **packets** to an address. Everything in §2 through §5 reads **requests**. Which side of that boundary a mechanism sits on determines what it can know, and therefore what it can decide.

North-south and east-west

Two words from ordinary industry vocabulary that this book has not used yet. They make the shape of this chapter sayable in one sentence.

North-south traffic enters the cluster from outside. **East-west** traffic moves between Pods inside it. Those two definitions are the industry's, not the project's. What the Kubernetes project supplies is the pairing, in a blog post on Gateway API rather than in the reference documentation, describing the API's initial focus as ingress, "north-south," traffic, and service mesh as the "east-west" case [source: k8s-blog-gateway-api-north-south-east-west-2026-08-24].

This chapter does one of each. §1 through §5 are about north-south. §6 and §7 are about east-west.

🔑 **Mnemonic:** *North-south goes through the wall; east-west stays inside it. §1–§5 is the wall. §6–§7 is inside.*

The edge router

One piece of scaffolding Chapter 9 did not supply, and §3 will need.

The Kubernetes documentation is explicit that in most common deployments, **the nodes in your cluster are not part of the public internet** [source: k8s-docs-ingress-depth-2026-08-24]. Something sits between the two: the **edge router**, a router that enforces the firewall policy for your cluster, whether that is a gateway managed by a cloud provider or a physical piece of hardware [source: k8s-docs-ingress-depth-2026-08-24].

Naming it here means that when §3 tells you an Ingress controller may fulfill an Ingress "usually with a load balancer, though it may also configure your edge router," you already know what that clause is about.

The same terminology block gives the vocabulary for everything else in this chapter, and it should all be familiar: a **node** is a worker machine, part of a cluster; the **cluster network** is the set of links, logical or physical, that carry communication within a cluster according to the Kubernetes networking model; a **Service** identifies a set of Pods using label selectors and is assumed to have a virtual IP routable only within the cluster network [source: k8s-docs-ingress-depth-2026-08-24]. That last clause is the whole problem in one line. The addresses Chapter 9 gave you work beautifully inside the harbour wall, and mean nothing beyond it.

What comes next, and the first thing worth knowing about it

There is an object for the layer-7 job. It is called Ingress.

And the first thing to know about it is that writing one may accomplish nothing at all. We will get to why in §3. Carry the expectation into §2 [*cross-bearing: see Ch 10 §3 — the Ingress controller and what its absence costs*].

[*cross-bearing: see Ch 17 §5 — a service mesh does at layer 7 for east-west traffic roughly what §2 does for north-south*]

..1 §2 — Routing by Host and Path

Chapter 9 gave you a warning attached to a promise. It told you, by name, that conflating **DNS-based service discovery** with **name-based virtual hosting** would make this chapter considerably harder than it needs to be, and it pointed here [*cross-bearing: see Ch 9 §7 — DNS-based service discovery, and what it is not*]. We will settle that distinction properly, a few beats in.

First, the object.

What Ingress is

Ingress exposes HTTP and HTTPS routes from outside the cluster to Services within the cluster. Traffic routing is controlled by rules defined on the Ingress resource [source: k8s-docs-ingress-depth-2026-08-24].

Read that twice, because both halves matter. *Routes from outside to Services within* — the things an Ingress routes to, in every case this chapter covers, are ordinary Services. The ClusterIP Services you built in Chapter 9, unchanged, unaware that anything new is in front of them. And *rules defined on the resource*: the routing decisions live in the object you write, not in the controller's configuration file.

An Ingress may be configured to give Services externally-reachable URLs, load balance traffic, terminate SSL/TLS, and offer name-based virtual hosting [source: k8s-docs-ingress-depth-2026-08-24]. Four capabilities, as the documentation enumerates them.

And what it refuses to do

Immediately after the capabilities, and not buried at the bottom of the section where you would skim past it:

An Ingress does not expose arbitrary ports or protocols. Exposing services other than HTTP and HTTPS to the internet typically uses a service of type `Service.Type=NodePort` or `Service.Type=LoadBalancer` [source: k8s-docs-ingress-depth-2026-08-24].

This is more satisfying than a caveat. It means the ladder you learned in Chapter 9 is not superseded by this chapter. It is *specialized past*. Ingress takes over one class of traffic, the HTTP class, and everything else goes back down to §1's layer. A PostgreSQL database, a message broker speaking its own binary protocol, a game server, an SMTP relay: none of those go through an Ingress. They go through NodePort or LoadBalancer, exactly as they did last chapter.

☆ **Fixed Point:** Ingress is **HTTP and HTTPS only**. It exposes no arbitrary ports and no other protocols. Anything else goes back down to `Service.Type=NodePort` or `Service.Type=LoadBalancer`.

The shapes an Ingress takes

Four, and they are best learned as a progression rather than as a list.

Ingress backed by a single Service. The degenerate case: no rules at all, one default backend, everything goes to one place [source: k8s-docs-ingress-depth-2026-08-24].

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: test-ingress
spec:
  defaultBackend:
    service:
      name: test
      port:
        number: 80

```

Worth thirty seconds, mostly so the more interesting cases have a baseline to differ from.

Simple fanout. A fanout configuration **routes traffic from a single IP address to more than one Service, based on the HTTP URI being requested** — which, as the documentation dryly notes, allows you to keep the number of load balancers down to a minimum [source: k8s-docs-ingress-depth-2026-08-24]. This is our running example. One host, two paths, two backends:

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: simple-fanout-example
spec:
  rules:
    - host: foo.bar.com
      http:
        paths:
          - path: /foo
            pathType: Prefix
            backend:
              service:
                name: service1
                port:
                  number: 4200
          - path: /bar
            pathType: Prefix
            backend:
              service:
                name: service2
                port:
                  number: 8080

```

One host. Two entries under paths. Each names a backend Service and a port. That is the whole shape.

Name-based virtual hosting. Name-based virtual hosts **support routing HTTP traffic to multiple host names at the same IP address** [source: k8s-docs-ingress-depth-2026-08-24]. Note what changed: the list is now at the level of host, and the paths beneath each are just /.

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: name-virtual-host-ingress
spec:
  rules:
  - host: foo.bar.com
    http:
      paths:
      - pathType: Prefix
        path: "/"
        backend:
          service:
            name: service1
            port:
              number: 80
  - host: bar.foo.com
    http:
      paths:
      - pathType: Prefix
        path: "/"
        backend:
          service:
            name: service2
            port:
              number: 80

```

Compare the two manifests side by side. They put the same number of Services behind the same single address. They differ in **which part of the request the rule reads**: the path in one, the host in the other. That is the entire distinction, and it is the one an exam will ask you to make.

☆ **Fixed Point: Simple fanout** splits by *path* at one host. **Name-based virtual hosting** splits by *host* at one address. Both put many Services behind one IP; they differ only in what the rule reads.

SIMPLE FANOUT – THE INGRESS RULE READS THE PATH



NAME-BASED VIRTUAL HOSTING – THE INGRESS RULE READS THE HOST



*Same client address, same number of Services, in both panels.
Only the underline – the fragment the Ingress rule reads – differs.*

TLS. You can secure an Ingress by specifying a **Secret that contains a TLS private key and certificate** [source: k8s-docs-ingress-depth-2026-08-24]. The Ingress resource supports a single TLS port, 443, and **assumes TLS termination at the ingress point – traffic to the Service and its Pods is in cleartext** [source: k8s-docs-ingress-depth-2026-08-24]. The TLS Secret must contain keys named `tls.crt` and `tls.key` [source: k8s-docs-ingress-depth-2026-08-24], and the documentation's example Secret is of type `kubernetes.io/tls`, the Secret type Chapter 4 cataloged [cross-bearing: see Ch 4 §4 – Secret types, including `kubernetes.io/tls`].

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: tls-example-ingress
spec:
  tls:
  - hosts:
    - https-example.foo.com
    secretName: testsecret-tls
  rules:
  - host: https-example.foo.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: service1
            port:
              number: 80

```

This is the chapter's cheapest retrieval and one of its most satisfying. You have known what a Secret is since Chapter 4; here is one doing a visible job.

🔒 **Worth Securing:** TLS termination at the Ingress means the private key lives in a Secret in the cluster and the backend Services can speak plain HTTP. Convenient, and worth knowing precisely, because the documentation says outright that traffic onward to the Service and its Pods is in cleartext [source: k8s-docs-ingress-depth-2026-08-24]. Encrypting *that* leg is a different problem with a different answer [*cross-bearing: see Ch 17 §5 — what a service mesh adds inside the cluster*].

The distinction Chapter 9 asked for

Both DNS-based service discovery and name-based virtual hosting involve hostnames. They sit on **opposite sides of the connection**.

DNS turns a name into an address **before any traffic moves**. A client that wants `shop.example.com` asks a resolver, gets back `203.0.113.10`, and only then opens a connection. The name has done its work and is, as far as the network is concerned, finished.

Virtual hosting sorts traffic that has **already arrived** at a single address, by reading the name back out of the request. The client that resolved `shop.example.com` and the client that resolved `blog.example.com` got the *same* address from DNS. They are both now talking to the same socket on the same machine. The only thing distinguishing them is the `Host` header they each sent, and something at that address has to open the request and read it.

⚠ **Navigational Hazards:** DNS gave you the address. Virtual hosting decides what happens *after* you have used it. Same hostname, two different jobs, on opposite sides of the connection. This is the confusion Chapter 9 warned you about by name, and the reason it warned you is that a reader who has them merged will spend \$5's request flow wondering why the resolver appears twice. Once you are inside, virtual hosting is the harbourmaster deciding which berth you take.

The fields, briefly

An Ingress needs `apiVersion`, `kind`, `metadata` and `spec`, like every object you have written since Chapter 4. The spec **contains a list of rules matched against all incoming requests**, and supports rules for directing HTTP(S) traffic only [source: k8s-docs-ingress-depth-2026-08-24].

Each HTTP rule contains an **optional host** — if no host is specified the rule applies to all inbound HTTP traffic through that IP address; if a host is given, the rules apply to that host — and **a list of paths, each with an associated backend defined with a `service.name` and a `service.port.name` or `service.port.number`**. Both the host and the path must match the content of an incoming request before traffic is directed to the referenced Service [source: k8s-docs-ingress-depth-2026-08-24].

Path types. Each path in an Ingress is **required to have a corresponding path type**, and paths without an explicit `pathType` are not validated [source: k8s-docs-ingress-depth-2026-08-24]. There are three:

pathType	Behavior
Exact	Matches the URL path exactly, case-sensitively [source: k8s-docs-ingress-depth-2026-08-24]
Prefix	Matches on a URL path prefix split by <code>/</code> , case-sensitively, element by element [source: k8s-docs-ingress-depth-2026-08-24]
Implementation Specific	Matching is up to the IngressClass; implementations may treat it as its own type or identically to Prefix or Exact [source: k8s-docs-ingress-depth-2026-08-24]

The element-by-element part of `Prefix` is where people get caught. Matching is done on **path elements** — the labels between `/` separators — not on raw string prefixes. A path of `/foo/bar` does **not** match a request path of `/foo/barbaz`, because the last element `bar` is only a substring of `barbaz`, not equal to it [source: k8s-docs-ingress-depth-2026-08-24].

📌 **Snag:** `Prefix` is not a string prefix. `/aaa/bb` does not match `/aaa/bbb` [source: k8s-docs-ingress-depth-2026-08-24]. The comparison happens element by element, and `bb ≠ bbb`. If you have carried over an instinct from string-prefix path matching anywhere else, this is *almost* the semantics you expect, and "almost" is what makes it expensive.

The default backend. An Ingress with no rules sends all traffic to a single default backend, and `.spec.defaultBackend` is what handles requests in that case [source: k8s-docs-ingress-depth-2026-08-24]. Conventionally the default backend is a configuration option of the Ingress controller rather than something you specify per-Ingress [source: k8s-docs-ingress-depth-2026-08-24], but note the rule that binds them: **if no `.spec.rules` are specified, `.spec.defaultBackend` must be specified** [source: k8s-docs-

ingress-depth-2026-08-24]. And if none of the hosts or paths in any Ingress match an incoming request, the traffic is routed to the default backend [source: k8s-docs-ingress-depth-2026-08-24].

That is the Ingress object. It is a small, legible API, and everything in it does exactly what it says.

None of it has happened yet.

§3 — The Object Is Not the Implementation

Here is the sentence:

You must have an Ingress controller to satisfy an Ingress. Only creating an Ingress resource has no effect [source: k8s-docs-ingress-depth-2026-08-24].

Not "less effect." Not "reduced functionality." None. The manifests in §2 are correct, well-formed, and accepted by the API server, and on a cluster with no Ingress controller they route nothing, because nothing is reading them.

☆ **Fixed Point: You must have an Ingress controller to satisfy an Ingress. Only creating an Ingress resource has no effect** [source: k8s-docs-ingress-depth-2026-08-24].

What an Ingress controller is

An Ingress controller is responsible for fulfilling the Ingress, usually with a load balancer, though it may also configure your edge router or additional frontends to help handle the traffic [source: k8s-docs-ingress-depth-2026-08-24]. There it is: §1's edge router, doing its job in a sentence you can now read without stopping. For an Ingress to work in your cluster, **there must be an ingress controller running**, and you need to select at least one and make sure it is set up [source: k8s-docs-ingress-controllers-2026-08-24].

Notice the structure of what you just read. The Ingress object is a *description of desired routing*. The controller is a control loop that reads that description and makes something in the real world match it. That is Chapter 3's control loop, unchanged and by now familiar: desired state in an object, a controller watching, reality dragged toward the description [*cross-bearing: see Ch 3 §6 — the control loop and the controller pattern*]. Recognizing it here should cost you nothing, which is the point of having learned it once properly.

The rule, retrieved

Chapter 3 gave you a sentence and asked you to keep it:

An object without its component does nothing.

It also told you that you would meet it four more times. This is the first of those four, and Chapter 3 published the pointer to this exact paragraph [*cross-bearing: see Ch 3 §4 — addons, and what else is optional*].

Two counts run through this chapter, and they are not the same count. Chapter 3's four are the instances it lined up ahead of you: this one, one more in §7, and two in chapters still to come. The other count is your own, the instances you have personally watched fail. That one started last chapter, and it currently stands at two.

You do not have to take the rule on faith, because you have already collected that evidence. Last chapter, twice:

- A type: `LoadBalancer Service` on a bare-metal cluster with no provider integration. A real object. A real cluster IP. An external address field that stays `<pending>` forever, because Kubernetes does not directly offer a load balancing component; you must provide one, or integrate with a cloud provider [source: `k8s-docs-service-2026-08-23`] [*cross-bearing: see Ch 9 §3 — LoadBalancer and what has to exist beneath it*].
- A Service whose selector matched nothing. A real object, a real cluster IP, a real DNS record, an empty `EndpointSlice`, and traffic that goes nowhere at all [*cross-bearing: see Ch 9 §4 — selectors, EndpointSlices, and the empty case*].

Now a third: an Ingress with no controller.

🔒 **Worth Securing:** Chapter 3's phrase, verbatim, and worth writing on something you will see again: **an object without its component does nothing**. You have now personally met three instances: the `LoadBalancer` with no provider, the `Service` with no matching Pods, and this. Three sightings of the same light, and you stop calling it a coincidence and start calling it a landmark.

Naming which controller: `IngressClass`

"You must have a controller" raises an obvious follow-up: *which one, if there are two?*

You may deploy any number of ingress controllers in a cluster, using **ingress class** to tell them apart [source: `k8s-docs-ingress-controllers-2026-08-24`]. Ingresses can be implemented by different controllers, often with different configuration, so **each Ingress should specify which controller it is intended to use** — done with the `ingressClassName` field on the Ingress, which references an **IngressClass** resource that carries additional configuration including the name of the controller that should implement the class [source: `k8s-docs-ingress-depth-2026-08-24`].

```

apiVersion: networking.k8s.io/v1
kind: IngressClass
metadata:
  name: external-lb
spec:
  controller: example.com/ingress-controller
  parameters:
    apiGroup: k8s.example.com
    kind: IngressParameters
    name: external-lb

```

If you do not specify an IngressClass on an Ingress and your cluster has **exactly one** IngressClass marked as default, Kubernetes applies that default [source: k8s-docs-ingress-controllers-2026-08-24]. You mark one as default by setting the `ingressclass.kubernetes.io/is-default-class` annotation to the string `"true"` [source: k8s-docs-ingress-controllers-2026-08-24]. If more than one IngressClass carries that marking, an Ingress that omits `ingressClassName` cannot be created at all; the resolution is to ensure at most one is marked default [source: k8s-docs-ingress-depth-2026-08-24]. Some ingress controllers work even without a default IngressClass defined; even so, the Kubernetes project still recommends that you define one [source: k8s-docs-ingress-depth-2026-08-24].

The honest note

One more fact, and it is the one that keeps this from being a tidy story:

Ideally, all Ingress controllers should fit the reference specification. In reality, the various Ingress controllers operate slightly differently [source: k8s-docs-ingress-depth-2026-08-24].

That is the documentation's own phrasing, twice over: the Ingress page and the Ingress Controllers page say the same thing in nearly the same words [source: k8s-docs-ingress-controllers-2026-08-24]. It matters because the promise of a portable object is undercut, precisely, by the gap between a reference specification and any particular implementation of it. The documentation's own advice is to review your controller's documentation to understand the caveats of choosing it [source: k8s-docs-ingress-depth-2026-08-24].

📌 **Snag:** Two clusters, the same Ingress manifest, different behavior. Same chart, different pilot. The object is portable; the controllers are only *ideally* identical. The gap between the reference specification and a particular implementation is where a configuration that worked for a year stops working after a migration, and where the failure looks like a bug in your manifest rather than a difference in the thing reading it.

You now have a rule and three instances of it — three on your own count, one on Chapter 3's. Do not close the pattern here. §7 has a fourth, and it behaves differently from the first three in a way that turns out to matter more than all of them.

[cross-bearing: see Ch 6 §8 — a custom controller acting on a custom resource is this same shape, met as the operator pattern] *[cross-bearing: see Ch 13 §7 — `kubectl top` on a cluster with no metrics-server]*

[cross-bearing: see Ch 17 §7 — VPA, which is an addon and is not there by default]

🕒 Taking Your Bearings #1

Five questions on §1 through §3 — the ceiling, the object, and the component that has to exist. Two of them reach back to earlier chapters.

1. . [retrieval: ch9] You need to expose a PostgreSQL database and a web application to clients outside the cluster. Which one can an Ingress handle, and what do you use for the other?
 2. . One IP address serves `shop.example.com` and `blog.example.com`, routing each to a different Service. Name the Ingress capability. Then: one host, with `/catalog` and `/checkout` going to different Services. Name that one.
 3. . A colleague applies a correct Ingress manifest to a fresh cluster. `kubectl get ingress` shows it. No traffic reaches the application. Name the most likely cause, and say what `kubectl get` actually proves.
 4. . An Ingress is applied with no `ingressClassName` set. Name the one thing in the cluster that decides whether it is assigned a controller anyway — and say what a *second* IngressClass carrying the same marking would do to that Ingress.
 5. . [retrieval: ch3] Chapter 3 gave you a one-sentence rule about objects and components. State it, then name the two things from Chapter 9 that were instances of it.
-

Answers with Explanations:

1. The web application, over HTTP/HTTPS. The database needs `Service.Type=NodePort` or `Service.Type=LoadBalancer`.

An Ingress exposes HTTP and HTTPS routes and nothing else; exposing services other than HTTP and HTTPS typically uses `NodePort` or `LoadBalancer` [source: [k8s-docs-ingress-depth-2026-08-24](#)]. PostgreSQL speaks its own wire protocol over TCP, so there is nothing for a layer-7 router to read.

The framing that matters here is **specialization, not replacement**. Ingress does not sit *instead of* the Service-type ladder. It sits *above* it, for one class of traffic. The ladder is still the correct answer for everything else, which is why §4's news about the Ingress API being frozen is survivable rather than alarming: nothing you learned in Chapter 9 is going anywhere.

Why a wrong answer is wrong: "use an Ingress for both, with different ports" fails because an Ingress does not expose arbitrary ports at all [source: [k8s-docs-ingress-depth-2026-08-24](#)]. The port fields in an Ingress rule name the *backend Service's* port, not a port the Ingress listens on.

2. Name-based virtual hosting; simple fanout.

Name-based virtual hosting routes HTTP traffic to multiple host names at the same IP address [source: k8s-docs-ingress-depth-2026-08-24]. Simple fanout routes traffic from a single IP address to more than one Service based on the HTTP URI [source: k8s-docs-ingress-depth-2026-08-24]. Asked as a pair on purpose: the thing to retrieve is not either definition but the **discriminator**, host versus path. Both put many Services behind one address.

The error to watch for is swapping them. The tell is in the manifest: several entries under `paths` is fanout; several entries under `rules`, each with its own `host`, is virtual hosting.

3. No Ingress controller is installed. `kubectl get` proves only that the object exists.

Only creating an Ingress resource has no effect; a controller must be present to satisfy it [source: k8s-docs-ingress-depth-2026-08-24]. `kubectl get ingress` returning your object tells you a record is in etcd and the API server will serve it back. It is a fact about storage. It is not a fact about routing, and nothing in that output is evidence that any component has ever looked at the object.

Nothing here is a mistake on your colleague's part. The manifest is correct, the expectation was reasonable, and the missing piece is invisible from the object. That is precisely what makes this pattern worth learning as a *first* question rather than a last resort.

4. An IngressClass marked as the cluster default. A second one carrying the same marking does not widen the net — it removes it, and the Ingress can no longer be created at all.

Setting the `ingressclass.kubernetes.io/is-default-class` annotation to the string "true" on an IngressClass makes it the cluster default, and new Ingresses that do not specify an `ingressClassName` are assigned that class [source: k8s-docs-ingress-depth-2026-08-24]. The condition is **exactly one**. If more than one IngressClass is marked default, an Ingress that omits `ingressClassName` cannot be created; the resolution is to ensure at most one carries the marking [source: k8s-docs-ingress-depth-2026-08-24].

The instinct to correct: more defaults feels like more coverage. It is the opposite. Two defaults do not give an unclassified Ingress two chances to be adopted. They take away the one chance it had, because the cluster now has no way to choose.

Worth holding next to question 3. Both are the same failure in the end, an object that never reaches a controller. But this one fails at the moment you apply it, and that one applies cleanly and then quietly does nothing. Only one of them tells you.

5. [retrieval: ch3] An object without its component does nothing. The two Chapter 9 instances: a `type: LoadBalancer` Service on a cluster with no load balancer to provision one, and a Service whose selector matches no Pods.

The wrong answer to watch for is stopping at one. `type: LoadBalancer` is the instance people keep, because the absent piece has a name and a price attached to it. The Service whose selector matches nothing is the one they drop — nothing there is *un-installed*, and the object is complete and correct on

its own terms. Same shape all the same: a record the cluster will serve back to you, and nothing behind it. And the Ingress controller from §3 is not one of the two. It is the third, and the one you found yourself.

If you can list the instances now, §8 will land as a count you already made rather than as an assertion the book handed you. That is the difference between remembering a rule and holding one.

Checkpoint: You've Now Mastered

✓ Where the Service-type ladder runs out, and the layer boundary that explains why ✓ What Ingress does, what it refuses to do, and the two shapes it takes ✓ Why a perfectly correct object can accomplish nothing ✓ Which controller an Ingress belongs to, and what happens when nothing says ✓ The rule, with three of your own instances behind it

Two sections on the API you just learned, and then we go somewhere completely different.

§4 — Frozen, Not Deprecated

Chapter 9 told you this section was coming and told you it was worth exam-level attention. Everything here is definition, and precision is the only thing it has to deliver.

The statement

The Kubernetes documentation carries a note directly beneath its description of Ingress. Here it is, in full:

The Kubernetes project recommends using Gateway instead of Ingress. The Ingress API has been frozen.

This means that:

- The Ingress API is generally available, and is subject to the stability guarantees for generally available APIs. The Kubernetes project has no plans to remove Ingress from Kubernetes.
- The Ingress API is no longer being developed, and will have no further changes or updates made to it.

[source: k8s-docs-ingress-depth-2026-08-24]

Two bullets. Both load-bearing. Nearly everyone drops one.

The split

The half readers drop	What it actually says	What it means for you
Still GA, still guaranteed, no removal plans	It is not going away, and it is not going to break under you	Your existing Ingress configurations are not a migration emergency
No further development, no changes or updates	Nothing new will ever be added	Anything Ingress cannot do today, it will never do

Collapse the first bullet and you get *"Ingress is deprecated,"* which is wrong, and which will have you planning a migration you do not need. Collapse the second and you get *"Ingress is fine, ignore the note,"* which is also wrong, and which will have you designing new systems around an API that has stopped growing.

☆ **Fixed Point: Frozen ≠ deprecated.** Ingress is generally available, subject to GA stability guarantees, with **no plans for removal** — **and** no longer developed, with **no further changes or updates** [source: k8s-docs-ingress-depth-2026-08-24]. Both halves, or you have the wrong fact.

Why "deprecated" is a different word

The two words point in different directions, and the difference is about *what* is being announced.

Deprecation is a statement about future removal. Kubernetes has a formal, published deprecation policy, and it exists precisely so that removals are predictable: the policy governs the removal of API objects, fields, annotations, enumerated values and component config structures, and its stated purpose is to avoid breaking existing users when a feature must be removed [source: k8s-docs-deprecation-policy-2026-08-24]. Under it, GA API versions **may be marked as deprecated, but must not be removed within a major version of Kubernetes** [source: k8s-docs-deprecation-policy-2026-08-24]. Deprecation is the first step on a defined path toward an eventual exit.

A freeze is a statement about future development. It says the thing is finished. Nothing more will be added. It says nothing whatsoever about removal, and a frozen API can be permanent.

Kubernetes has said one of these about Ingress and not the other, and the choice was deliberate. When the Ingress note says the API is "subject to the stability guarantees for generally available APIs," it links directly to that deprecation policy: the guarantees are the same ones any GA API enjoys [source: k8s-docs-ingress-depth-2026-08-24].

⚠ **Navigational Hazards:** A question offering *"Ingress is deprecated and will be removed in a future release"* is testing exactly one thing: whether you kept both halves. So is a question offering *"Ingress is unaffected and fully supported for new development."* Neither is the answer. Candidates get this wrong in both directions, which is why an exam can build a clean four-option question out of it. The two most attractive distractors write themselves.

🧠 **Mnemonic:** Frozen things keep. They just don't grow.

What "recommends" obliges

Recommends is not *requires*.

Here is the recommendation as the documentation states it, with nothing attached: **the Kubernetes project recommends using Gateway instead of Ingress** [source: k8s-docs-ingress-depth-2026-08-24]. Not *for new work*. Not *by some deadline*. Not *unless you have a reason*. One unqualified sentence. What sits beside it is equally unqualified: the project has not deprecated Ingress, has announced no removal, and continues to extend it the stability guarantees that GA APIs carry. A light has been hung on the new channel; the old one has not been closed.

Where that leaves you in practice is a second question, and the answer to it is this book's reading rather than the project's wording: **do not panic about what you are running, and think hard before building something new on an API that will never gain a feature again**. If you have heard the recommendation summarized as "*use Gateway for new work*" — including elsewhere in this chapter — that framing is a practitioner's gloss on the second bullet, not a scope the documentation supplies. The documentation says *instead of*, and stops there.

Keep the two apart. On an exam, you are being asked what the project said, not what a sensible engineer does about it.

The project said what it said, precisely. The reasoning behind the decision is not this book's to supply.

[cross-bearing: see Ch 8 §6 — semantic versioning, which is half the vocabulary this section spends]

The obvious next question is what "use Gateway instead" actually means. §5.

§5 — Roles, Not Just Routes

The temptation is to open with three resource names. Resist it. The resource names are a *consequence*; taught in the wrong order they become three arbitrary things to memorise instead of one idea with three parts.

The idea

Gateway API is a family of API kinds that provide dynamic infrastructure provisioning and advanced traffic routing. It makes network services available using an **extensible, role-oriented, protocol-aware** configuration mechanism [source: k8s-docs-gateway-api-depth-2026-08-24].

Role-oriented is the word that matters. Everything else in the API falls out of it.

The three roles

Gateway API kinds are modeled after the organizational roles responsible for managing Kubernetes service networking [source: k8s-docs-gateway-api-depth-2026-08-24]:

- **Infrastructure Provider** — manages infrastructure that allows multiple isolated clusters to serve multiple tenants; a cloud provider, for example [source: k8s-docs-gateway-api-depth-2026-08-24].
- **Cluster Operator** — manages clusters, and is typically concerned with policies, network access, and application permissions [source: k8s-docs-gateway-api-depth-2026-08-24].
- **Application Developer** — manages an application running in a cluster, and is typically concerned with application-level configuration and Service composition [source: k8s-docs-gateway-api-depth-2026-08-24].

A note on vocabulary before we go further. **"Cluster operator" here is a role name — a job a person or team holds — not the operator pattern.** This book has otherwise reserved "operator" for the custom-resource-plus-custom-controller shape you met in Chapter 6 [cross-bearing: see Ch 6 §8 — the operator pattern]. Two words, two unrelated meanings, and this is the only place in the book they sit near each other. When Gateway API says *cluster operator*, it means the humans who run the cluster.

The resources

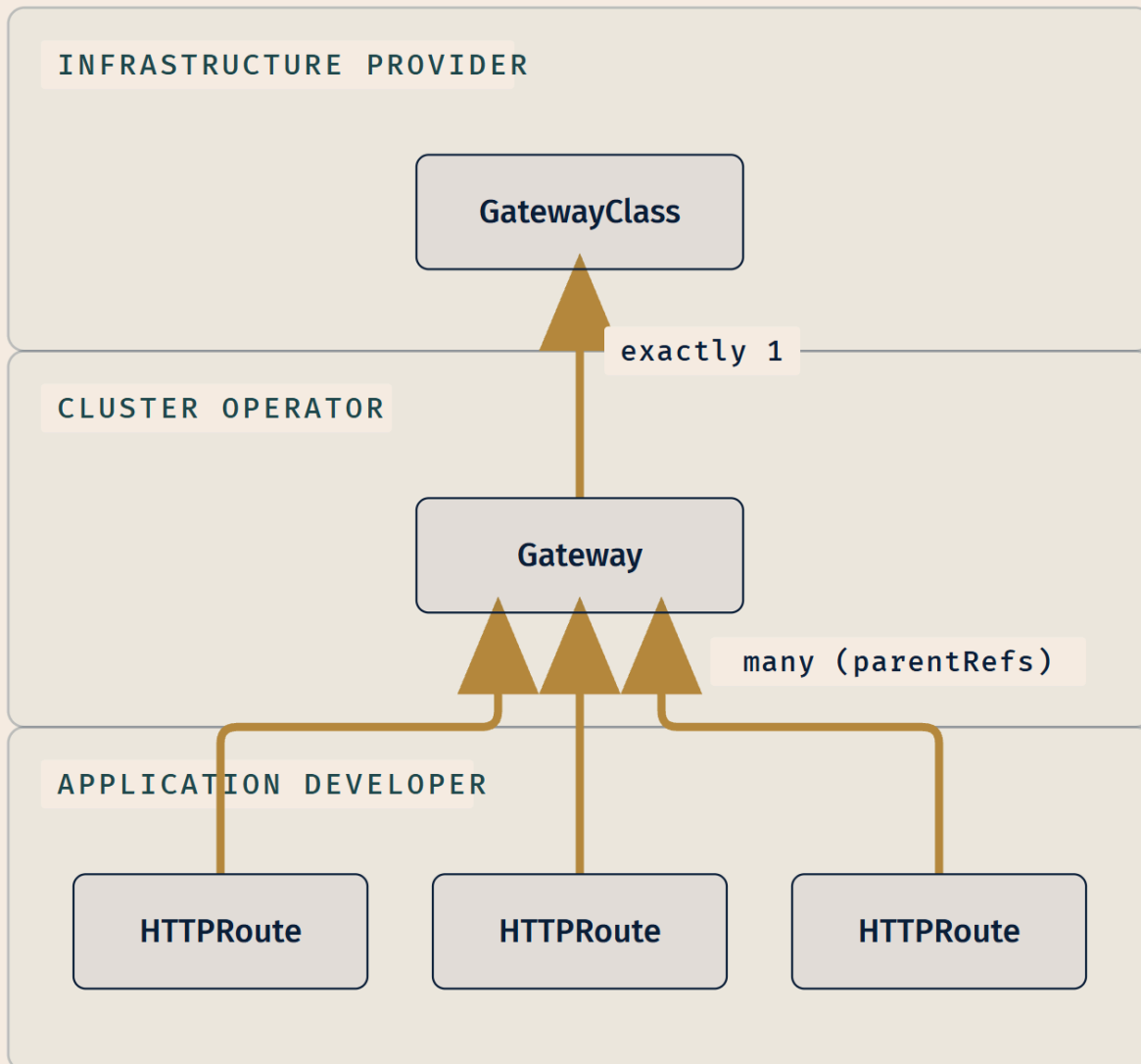
Now the resource model reads as a consequence rather than a list. Gateway API has four stable API kinds [source: k8s-docs-gateway-api-depth-2026-08-24]; three of them carry the role split, and those three are the ones this chapter uses:

- **GatewayClass** — defines a set of gateways with common configuration, managed by a controller that implements the class [source: k8s-docs-gateway-api-depth-2026-08-24].
- **Gateway** — defines an instance of traffic-handling infrastructure, such as a cloud load balancer [source: k8s-docs-gateway-api-depth-2026-08-24].
- **HTTPRoute** — defines HTTP-specific rules for mapping traffic from a Gateway listener to a representation of backend network endpoints, which are often represented as a Service [source: k8s-docs-gateway-api-depth-2026-08-24]; GRPCRoute does the same for gRPC-specific rules [source: k8s-docs-gateway-api-depth-2026-08-24].

The cardinality is examinable, so state it as such. A Gateway object is associated with **exactly one** GatewayClass, and the GatewayClass describes the gateway controller responsible for managing Gateways of that class. One or more route kinds are then associated to Gateways: **many** Routes may attach to a Gateway. A Gateway can filter the routes that may attach to its listeners, forming a bidirectional trust model with routes [source: k8s-docs-gateway-api-depth-2026-08-24].

☆ **Fixed Point:** The three resources this chapter uses — **GatewayClass, Gateway, HTTPRoute** — and the role each belongs to. A Gateway is associated with **exactly one** GatewayClass; **many** Routes may attach to one Gateway [source: k8s-docs-gateway-api-depth-2026-08-24].

The mapping, which is the whole design



Bands are ownership boundaries, not runtime layers.

The infrastructure provider supplies the **GatewayClass**. The cluster operator creates the **Gateway**. The application developer writes the **HTTPRoute**. Each concern gets its own resource, so each role can hold its own without needing edit rights on anyone else's.

Here is what a Gateway looks like — the cluster operator's object:

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: example-gateway
  namespace: example-namespace
spec:
  gatewayClassName: example-class
  listeners:
  - name: http
    protocol: HTTP
    port: 80
    hostname: "www.example.com"
    allowedRoutes:
      namespaces:
        from: Same
```

And an HTTPRoute — the application developer's:

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: example-httproute
spec:
  parentRefs:
  - name: example-gateway
  hostnames:
  - "www.example.com"
  rules:
  - matches:
    - path:
        type: PathPrefix
        value: /login
    backendRefs:
    - name: example-svc
      port: 8080
```

[source: [k8s-docs-gateway-api-depth-2026-08-24](#)]

Look at what those two manifests do and do not contain. The Gateway names its class and declares its listeners and which namespaces may attach routes; it says nothing about `/login`. The HTTPRoute names its parent Gateway under `parentRefs`, declares its hostnames and path matches and backends; it says nothing about ports, protocols, or which load balancer implementation is underneath. The seam between them is exactly the seam between the two roles.

And compare that HTTPRoute against §2's fanout Ingress. Same requirement, expressed in a different vocabulary: one host, a path match, a backend Service and port. This is not a new routing model to learn

from scratch; it is the shapes you already know, redistributed across resources that belong to different owners.

🔒 **Worth Securing:** The role split is the innovation. Two things are documented. Ingress controllers frequently use annotations to configure behavior [source: k8s-docs-ingress-depth-2026-08-24], and Gateway API kinds support common cases such as header-based matching and traffic weighting "that were only possible in Ingress by using custom annotations" [source: k8s-docs-gateway-api-depth-2026-08-24]. The reading that joins them — that so much real-world configuration ended up in annotations *because* Ingress put infrastructure choice, cluster policy, and application routing into one object that one team had to own — is this book's, not the documentation's.

The request flow

Concrete, end to end, for a Gateway implemented as a reverse proxy [source: k8s-docs-gateway-api-depth-2026-08-24]:

1. The client starts to prepare an HTTP request for `http://www.example.com`.
2. The client's DNS resolver queries for the destination name and learns a mapping to one or more IP addresses associated with the Gateway.
3. The client sends a request to the Gateway IP address; the reverse proxy receives the HTTP request and uses the `Host`: **header** to match a configuration derived from the Gateway and attached `HTTPRoute`.
4. Optionally, the reverse proxy performs request header and/or path matching based on the `HTTPRoute`'s match rules.
5. Optionally, the reverse proxy modifies the request — adding or removing headers, say — based on the `HTTPRoute`'s filter rules.
6. Lastly, the reverse proxy forwards the request to one or more backends.

Step 2 and step 3 are the §2 distinction, drawn in the specification's own hand. DNS does its work and finishes; the `Host` header does its work afterward, on a connection that has already arrived. One question is asked across open water, before anything moves; the other is asked at the quayside, of something already tied up alongside.

And step 3 is Soundings question 1's answer, seven sections later. You worked out that a server distinguishing two hostnames on one address has to read the `Host` header, from ordinary web experience, before this chapter started. Here is the same mechanism in a Kubernetes specification, doing the same job under a different name. That is usually how this material goes: the priors were right, the vocabulary is new.

The other design principles

Three more, briefly [source: k8s-docs-gateway-api-depth-2026-08-24]:

- **Portable** — Gateway API specifications are defined as custom resources and are supported by many implementations.
- **Expressive** — the kinds support common traffic routing cases such as header-based matching and traffic weighting, which in Ingress were only possible through custom annotations. That is the concrete answer to what §4's freeze costs you.
- **Extensible** — custom resources can be linked at various layers of the API, making granular customization possible at the appropriate places within the structure.

Is it there?

Having just been told to prefer Gateway API, the obvious next question is whether it is in your cluster. It is not.

Instead of Gateway API resources being natively implemented by Kubernetes, the specifications are defined as Custom Resources supported by a wide range of implementations. You install the Gateway API CRDs, or follow the installation instructions of your selected implementation [source: k8s-docs-gateway-api-depth-2026-08-24]. The docs describe Gateway API as "an add-on containing API kinds" [source: k8s-docs-gateway-api-depth-2026-08-24], and the cluster addon list carries it among the networking entries [source: k8s-docs-cluster-addons-2026-08-24].

🦋 **Closer Look:** The API the project names as Ingress's successor [source: k8s-docs-network-model-2026-08-23] is not built into the API server the way Ingress is. It arrives as custom resources [*cross-bearing: see Ch 6 §8 — custom resources and CustomResourceDefinitions*]. That is deeper than the exam requires, and it is a rather good demonstration of Chapter 6's claim that the extension mechanism is powerful enough to build first-class-looking APIs on top of. The successor to a built-in API is, structurally, an extension.

[*cross-bearing: see Ch 9 §7 — the client's resolver, which appears here as one step in a flow rather than as a topic*] [*cross-bearing: see Ch 17 §4 — CRDs as one of the four pluggable interfaces, of which this is a conspicuous instance*]

🕒 Taking Your Bearings #2

Five questions on §4 and §5 — one API frozen, one API recommended, and the difference that word makes.

1. 📄 State both halves of what the Kubernetes project has said about the Ingress API.
2. 📄 True or false, with justification: *Ingress is deprecated and will be removed in a future release.*
3. 📄 Name the three role-mapped Gateway API resources and say which organizational role each belongs to.

4. ❌ How many GatewayClasses is a Gateway associated with, and how many Routes can attach to one Gateway?
5. ❌ A request arrives at a Gateway's IP address. Name the header the reverse proxy uses to match a configuration, and name the two optional things the HTTPRoute may do before the request is forwarded.

Answers with Explanations:

1. It is frozen: generally available and subject to the stability guarantees for GA APIs, with no plans for removal — and no longer being developed, with no further changes or updates [source: k8s-docs-ingress-depth-2026-08-24].

An answer carrying only one half is wrong, because one half is precisely the error being tested.

And it is worth saying *why* both halves matter rather than just repeating them. The stability half means there is **no migration emergency**: what you are running keeps working, keeps being supported, and is not scheduled for removal. The no-development half means there is **no future capability**: whatever gap you find in Ingress today is permanent. Those two facts drive different decisions, one about existing systems, one about new ones, and that is exactly why the project stated both.

2. False, on both counts. Ingress has not been deprecated, and there are no plans to remove it [source: k8s-docs-ingress-depth-2026-08-24]. What has been said is that it will not be developed further, and that the Kubernetes project recommends using Gateway instead of Ingress [source: k8s-docs-ingress-depth-2026-08-24]. Take that recommendation in the project's own words rather than a softened paraphrase of them. §4 gave the operational reading of what a recommendation does and does not oblige you to do, and the reading is the book's, not the documentation's.

The wrong answer to watch for is the one that reasons from feel: "no longer developed" *feels* like deprecation, so a reader who has absorbed the second half of question 1 answers this one wrong. In Kubernetes, deprecation is a formally defined process with published rules about removal timelines [source: k8s-docs-deprecation-policy-2026-08-24], and the project did not invoke it here.

3. GatewayClass — infrastructure provider. Gateway — cluster operator. HTTPRoute — application developer [source: k8s-docs-gateway-api-depth-2026-08-24].

Asked as a mapping rather than a list because the mapping *is* the design. If you can produce the three names but not the three roles, you have memorised the consequence and missed the cause.

One point of precision the answer key must make: **"cluster operator" is a role — a team or a person who runs the cluster — not the operator pattern from Chapter 6.** The word does double duty in Kubernetes vocabulary, and this is the one place in this book where both senses are in play.

4. Exactly one GatewayClass. Many Routes [source: k8s-docs-gateway-api-depth-2026-08-24].

Pure recall, and exactly the kind of cardinality detail multiple-choice exams reach for, because it is unambiguous and either known or not.

The wrong answer to watch for is the Ingress-shaped one. In Ingress, a single object carries the entry point *and* every routing rule, so it is natural to expect a Gateway to carry its routes the same way. It does not. The routes attach to it from outside, and — per question 3 — they belong to a different role than the Gateway does.

5. The Host: header. Then, optionally: header and/or path matching based on the HTTPRoute's match rules, and optional modification of the request based on its filter rules [source: k8s-docs-gateway-api-depth-2026-08-24].

The wrong answer to watch for is *the path*. §2 spent considerably longer on paths than on hosts, and recency does what recency does: readers reach for path matching as the first thing a proxy performs. It is not first. The Host: header selects the configuration; path and header matching are among the optional steps that happen after that selection has already been made.

This also closes Soundings question 1. You named the Host header at the start of this chapter, from ordinary web experience and before reading a word of Kubernetes networking. The specification agrees with you. Notice that: a fair amount of this material is your existing knowledge wearing new vocabulary, and recognizing which parts are genuinely new is how you spend study time well.

Checkpoint: You've Now Mastered

✓ *Frozen and deprecated*, precisely, in both halves ✓ Gateway API's role-oriented design, and the resources that fall out of it ✓ The cardinality, and the request flow end to end

That closes the API arc. What follows shares a chapter with it and shares nothing else: different layer, different direction, different problem. Everything from §1 to §5 has been about getting a request through the harbour wall; §6 is about what is permitted to move once it is already inside. Take the break here if you are taking one.

..1 §6 — Allowing, Never Denying

One piece of housekeeping before we get under way, and it is not politeness.

The word *ingress* is about to mean something completely different, and capitalisation is the tell. For four sections, Ingress has meant an API object and the controller that fulfills it. From here to the end of §7, lowercase *ingress* means **a direction of traffic**: inbound, as opposed to *egress*, outbound. NetworkPolicy has nothing to do with the Ingress object. If you carry the old meaning into this section, you will spend §7 trying to work out how the Ingress controller fits in, and it does not.

Good. Now the object.

What a NetworkPolicy controls

NetworkPolicies let you specify rules for traffic flow at the IP address or port level — OSI layer 3 or 4 — within your cluster, and also between Pods and the outside world [source: k8s-docs-network-policies-depth-2026-08-24]. They are an **application-centric construct**, which lets you specify how a Pod is allowed to communicate with various network *entities* — a word the documentation chose deliberately to avoid overloading "endpoints" and "services," which already have specific Kubernetes meanings [source: k8s-docs-network-policies-depth-2026-08-24]. And they **apply to a connection with a Pod on one or both ends, and are not relevant to other connections** [source: k8s-docs-network-policies-depth-2026-08-24].

Note what that rules out. This is **network reachability**: who may open a connection to whom, at layer 3 or 4, and nothing else. It is not the boundary between a workload and the host it runs on. Chapter 2 already told you those were different axes, and it told you on a graded question [*cross-bearing: see Ch 2 §7 — RuntimeClass, and workload-to-host isolation as a separate concern*]. The other axis of Pod security has its own chapter [*cross-bearing: see Ch 12 §5 — what a Pod may do to its node*].

And note the layer. §1 spent five sections climbing to layer 7 to read hostnames and paths. This section is back down at 3 and 4, reading addresses and ports. Different problem, different altitude.

Three identifiers, and two selectors doing different jobs

The entities a Pod can communicate with are identified through a combination of three identifiers [source: k8s-docs-network-policies-depth-2026-08-24]:

1. **Other Pods** that are allowed — with the exception that a Pod cannot block access to itself.
2. **Namespaces** that are allowed.
3. **IP blocks** — with the exception that traffic to and from the node where a Pod is running is always allowed, regardless of the IP address of the Pod or the node.

For Pod- and namespace-based policies, **you use a selector to specify what traffic is allowed to and from the Pods that match the selector**. For IP-based policies, you define the rule on **IP blocks (CIDR ranges)** [source: k8s-docs-network-policies-depth-2026-08-24]. (*CIDR notation is a way of writing a range of IP addresses as an address plus a prefix length: 172.17.0.0/16 means "the addresses whose first sixteen bits are those of 172.17.0.0." An except list carves ranges back out of the block — in the manifest below, everything in 172.17.0.0/16 apart from the addresses in 172.17.1.0/24. The glossary carries the expansion.*)

Chapter 4 saw this coming. It told you, six chapters ago, that **a NetworkPolicy selects both its subject and its peers** [*cross-bearing: see Ch 4 §5 — labels and selectors as the universal join*]. Pause on that, because it is the structurally most interesting thing in this section. Here is the shape:

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: test-network-policy
  namespace: default
spec:
  podSelector:           # <-- chooses the SUBJECT: who this policy governs
    matchLabels:
      role: db
  policyTypes:
  - Ingress
  - Egress
  ingress:
  - from:
    - ipBlock:
        cidr: 172.17.0.0/16
        except:
        - 172.17.1.0/24
    - namespaceSelector: # <-- chooses PEERS: who may connect
        matchLabels:
          project: myproject
    - podSelector:       # <-- also chooses PEERS
        matchLabels:
          role: frontend
    ports:
    - protocol: TCP
      port: 6379
  egress:
  - to:
    - ipBlock:
        cidr: 10.0.0.0/24
    ports:
    - protocol: TCP
      port: 5978

```

[source: k8s-docs-network-policies-depth-2026-08-24]

Each NetworkPolicy includes a podSelector which selects the grouping of Pods to which the policy applies — here, Pods labeled `role: db` [source: k8s-docs-network-policies-depth-2026-08-24]. That is the subject. Inside the rules, a different set of selectors chooses who may connect: `podSelector` selects particular Pods in the **same namespace as the NetworkPolicy** as allowed sources or destinations, and `namespaceSelector` selects particular namespaces for which **all** Pods are allowed [source: k8s-docs-network-policies-depth-2026-08-24].

One mechanism, the label selector you learned in Chapter 4, doing two entirely different jobs in one object, at two different depths of the same YAML.

📌 **Snag:** Whether `namespaceSelector` and `podSelector` appear as one `from` entry or two changes the meaning completely. A single entry specifying **both** selects particular Pods *within* particular namespaces, an AND. Two entries in the `from` array is an OR: connections from Pods in the local namespace with the peer label, **or** from any Pod at all in the matching namespaces [source: k8s-docs-network-policies-depth-2026-08-24]. One YAML hyphen is the difference. The documentation's own advice: when in doubt, use `kubectl describe` to see how Kubernetes has interpreted the policy [source: k8s-docs-network-policies-depth-2026-08-24].

The two sorts of isolation

This is the center of the section, and the place your firewall instinct gets corrected.

There are **two sorts of isolation for a Pod: isolation for egress, and isolation for ingress**. They concern what connections may be established. "Isolation" here is **not absolute** — it means *some restrictions apply*. The alternative, "non-isolated for a direction," means that **no restrictions apply** in that direction. The two are declared independently, and both are relevant for a connection from one Pod to another [source: k8s-docs-network-policies-depth-2026-08-24].

Egress. By default, a Pod is **non-isolated for egress; all outbound connections are allowed**. A Pod becomes isolated for egress if there is **any** NetworkPolicy that both **selects the Pod** and has Egress in its `policyTypes`. When it is isolated, the only allowed outbound connections are those permitted by the egress list of some policy that applies to it [source: k8s-docs-network-policies-depth-2026-08-24].

Ingress. By default, a Pod is **non-isolated for ingress; all inbound connections are allowed**. It becomes isolated for ingress on exactly the same terms, with Ingress in `policyTypes`. When it is isolated, the only allowed inbound connections are **those from the Pod's node** and those permitted by the ingress list of some applicable policy [source: k8s-docs-network-policies-depth-2026-08-24].

Reply traffic for allowed connections is implicitly allowed in both directions [source: k8s-docs-network-policies-depth-2026-08-24], which is to say the mechanism is connection-aware, not packet-by-packet, and you do not need a return rule.

Now collect the debt from Soundings question 4. You almost certainly answered *dropped* and *the deny wins*, because that is how ordinary firewalls work and it is a good instinct nearly everywhere else. Kubernetes is the other way around on both counts. **A Pod starts fully open in both directions, and becomes restricted only because some policy went looking for it and found it**. Nothing is closed until something selects it. This is open water rather than a walled harbour: it stays open until somebody declares a restricted zone and puts you inside it.

☆ **Fixed Point:** By default a Pod is **non-isolated in both directions**. It becomes isolated for a direction only when some NetworkPolicy both **selects it** and names that direction in `policyTypes` [source: k8s-docs-network-policies-depth-2026-08-24]. No policy means no restriction.

A note on `policyTypes`, because it has a default that catches people. Each policy includes a `policyTypes` list which may include Ingress, Egress, or both. **If no `policyTypes` are specified, Ingress will always be**

set, and Egress will be set if the policy has any egress rules [source: k8s-docs-network-policies-depth-2026-08-24]. So an omitted `policyTypes` is not "neither." It is at minimum `Ingress`.

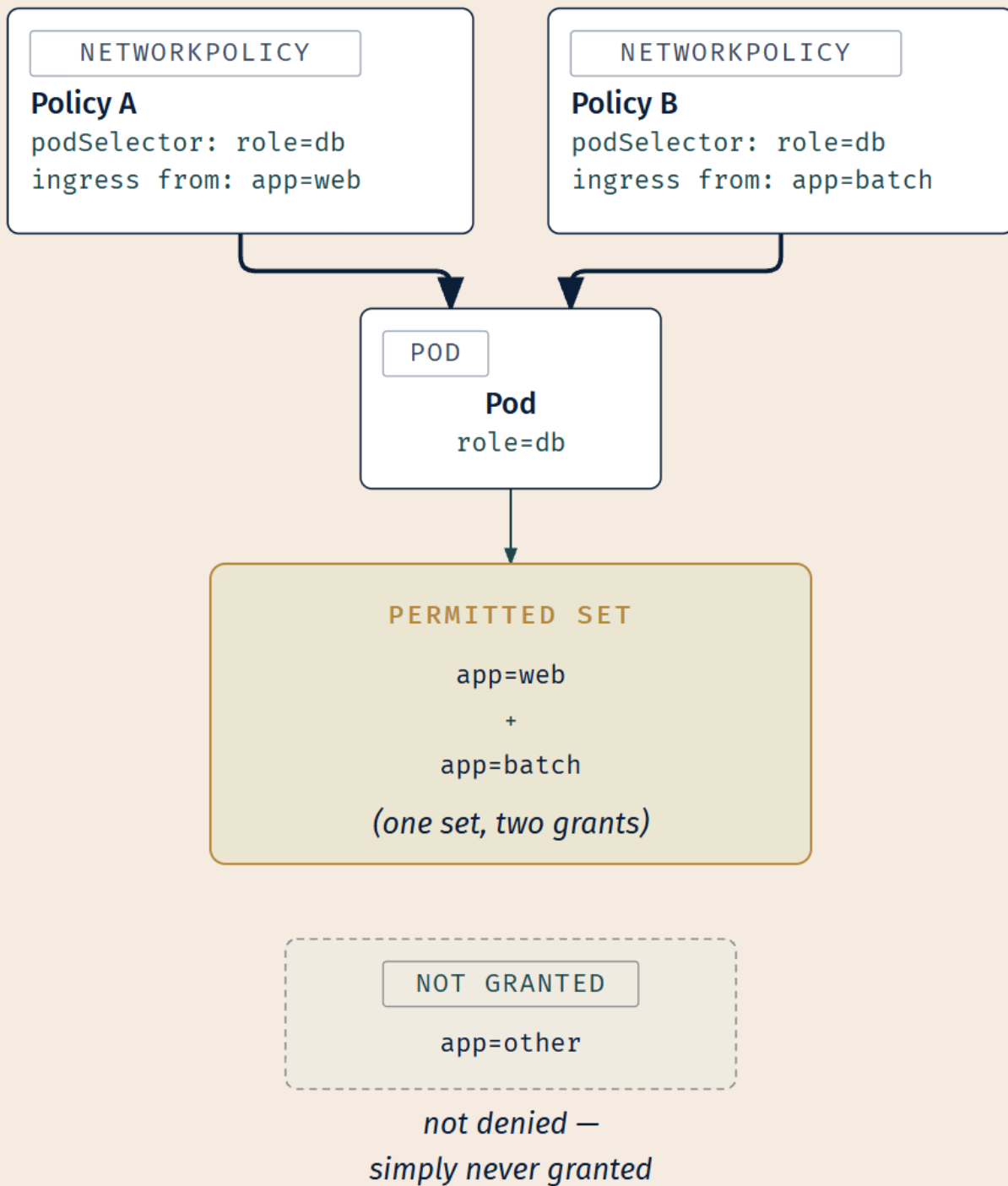
Additive, and there is no deny

The effects of the ingress lists combine additively. The effects of the egress lists combine additively. Network policies do not conflict; they are additive. If any policy or policies apply to a given Pod for a given direction, the connections allowed in that direction are **the union of what the applicable policies allow**. Thus **order of evaluation does not affect the policy result** [source: k8s-docs-network-policies-depth-2026-08-24].

Sit with that for a moment, because it removes something you have relied on everywhere else.

There is **no deny rule**. None. The API has no syntax for one. Two policies selecting the same Pod produce the union of what they permit, and there is no third policy you can write that subtracts from that union. If a Pod can currently reach something and you want it not to, you do not add a denial. **You remove the grant.**

☆ **Fixed Point: Policies are additive and never conflict. There is no deny rule** [source: k8s-docs-network-policies-depth-2026-08-24]. Two policies produce the union of what they permit. Removing access means removing the grant, not adding a denial.

**LEGEND**

- ➡ podSelector — chooses the SUBJECT
- ➡ peer selector — chooses WHO MAY CONNECT

Note what is *not* in that figure: any mark of denial. No barrier, no crossed-out arrow, no red X. The excluded Pod is excluded by the **absence of a grant**, which is a different thing from being blocked, and drawing it as blocking would contradict the Fixed Point above.

Both ends must allow it

For a connection from a source Pod to a destination Pod to be allowed, both the egress policy on the source Pod and the ingress policy on the destination Pod need to allow the connection. If either side does not allow the connection, it will not happen [source: k8s-docs-network-policies-depth-2026-08-24].

This is the rule that costs practitioners the most time, because a policy that is perfectly correct in isolation is only ever half of a working configuration. You write an egress policy on `frontend` permitting traffic to `api`, you verify the YAML, you apply it, and nothing connects, because `api` has an ingress policy that never heard of `frontend`. Two harbours, two authorities: clearance to depart is not permission to enter, and a vessel holding only one of them stays at anchor.

☆ **Fixed Point: Both ends must allow it.** The source Pod's egress policy *and* the destination Pod's ingress policy [source: k8s-docs-network-policies-depth-2026-08-24].

⚠ **Navigational Hazards:** If your firewall instinct says "*unlisted traffic is dropped*" and "*the more restrictive rule wins*," both instincts are wrong here. And they are wrong in the direction that makes a cluster **more open than you expect**, not less. Candidates get both of these wrong, reliably, and they get them wrong *confidently*, which is worse. The default is open. Nothing denies. The union permits.

🗝 **Mnemonic:** *Nothing is closed until something selects it; nothing selected can be re-closed by another rule; and both ends have to agree.*

Getting default-deny with no deny rule

The obvious objection: if there is no deny rule, how does anyone ever lock anything down?

Follow the two facts you already have. A Pod becomes isolated for a direction when a policy selects it and names that direction. Once isolated, the only permitted connections are the ones some policy's list allows. So: **select the Pods, name the direction, and permit nothing**. Isolation without permission is denial, arrived at by construction rather than by a deny keyword.

The mechanism for "select every Pod in the namespace" is an **empty podSelector**, which **selects all Pods in the namespace** [source: k8s-docs-network-policies-depth-2026-08-24]:

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
spec:
  podSelector: {}
  policyTypes:
    - Ingress
    - Egress

```

No ingress list. No egress list. Every Pod in the namespace selected, both directions named, nothing permitted. **This ensures that even Pods without any other NetworkPolicy selected will not be allowed any ingress or egress traffic** [source: k8s-docs-network-policies-depth-2026-08-24]. The same shape with only Ingress in policyTypes gives you default-deny inbound and leaves outbound alone [source: k8s-docs-network-policies-depth-2026-08-24].

And because the model is additive, the reverse also works: an explicit allow-all is a policy with podSelector: {} and a single empty rule, ingress: [{}], which permits everything to everything even if other policies have caused some Pods to be treated as isolated [source: k8s-docs-network-policies-depth-2026-08-24]. Union semantics cut both ways: you cannot subtract, and neither can anybody else.

The documentation puts the whole model in one sentence worth memorising: **a Pod will accept all traffic by default; however, once a NetworkPolicy is created for a Pod, the Pod will reject any traffic that is not allowed by any NetworkPolicy — and other Pods in the namespace that are not selected by any NetworkPolicy will continue to accept all traffic** [source: k8s-docs-network-policies-depth-2026-08-24].

The two exceptions

Both were in the three-identifiers list above, and both are unconditional:

- **A Pod cannot block access to itself** [source: k8s-docs-network-policies-depth-2026-08-24].
- **Traffic to and from the node where a Pod is running is always allowed, regardless of the IP address of the Pod or the node** [source: k8s-docs-network-policies-depth-2026-08-24].

The second one is why the ingress isolation rule says the allowed inbound connections are "those from the Pod's node **and** those allowed by the ingress list": node-local traffic is not something a policy grants, it is something no policy can take away.

📌 **Snag:** These two exceptions get rediscovered regularly by someone testing a policy from the wrong place. `kubectl exec` into the Pod and `curl` itself: allowed, always, and it proves nothing. Test from the node: allowed, always, and it proves nothing either. If you want to know whether a restriction works, the traffic has to originate somewhere the policy could actually govern.

[cross-bearing: see Ch 9 §1 — the network model's second rule, and the "barring intentional network segmentation" hedge that pointed here] [cross-bearing: see Ch 4 §3 — namespaces, which are the second

of the three identifiers] [cross-bearing: see Ch 5 §1 — the Pod IP, which is ultimately what a policy is about] [cross-bearing: see Ch 12 §9 — RBAC and NetworkPolicy as one shared semantic]

§7 — What NetworkPolicy Cannot Do

Two facts. This section teaches two facts and nothing else, and the first one is the highest-consequence sentence in the chapter.

The prerequisite

Network policies are implemented by the network plugin. To use network policies, you must be using a networking solution which supports NetworkPolicy. Creating a NetworkPolicy resource without a controller that implements it will have no effect [source: k8s-docs-network-policies-depth-2026-08-24].

Read that last clause against §3's. *Only creating an Ingress resource has no effect. Creating a NetworkPolicy resource without a controller that implements it will have no effect.* The same sentence, twice, four sections apart, about two objects with nothing else in common.

☆ **Fixed Point: NetworkPolicies are implemented by the network plugin.** On a plugin that does not implement NetworkPolicy, the resource has no effect [source: k8s-docs-network-policies-depth-2026-08-24].

Why this one is worse

Here is the asymmetry, and it is the reason §7 exists as its own section.

When an Ingress does nothing, **requests fail**. The site is down, somebody's monitoring fires, a user complains, and within minutes someone is looking at it. The failure announces itself.

When a NetworkPolicy does nothing, **traffic flows exactly as it did before**. `kubectl get networkpolicy` shows the object. `kubectl describe` shows the rules, correctly parsed and neatly formatted. Everything you can observe about the object says it is fine, and the observable behavior of an unenforced policy is *identical* to the observable behavior of a perfectly enforced policy against traffic nobody happens to be sending. There is no signal. There is nothing to notice. One failure fires a flare; the other is an uncharted rock, and nothing on the surface says it is there.

That is not a documented claim; the source states the plugin dependency and the no-effect consequence and stops there. The characterisation of the failure as *silent*, and as harder to detect than a broken Ingress, is this book's reasoning about what those two documented facts imply. Hold the two apart: the dependency is sourced, the inference about detectability is ours. We think it is sound, it is the most valuable thing in this chapter, and it is still an inference.

⚠ **Navigational Hazards:** *"I applied a NetworkPolicy, so that traffic is blocked"* is only true if something is enforcing it. Verify that your network plugin supports NetworkPolicy before you rely on one, and test the restriction from somewhere the policy could actually govern rather than trusting the object's existence. The object existing is a fact about etcd.

Nothing about this is careless. You wrote a correct policy, the API accepted it, `kubectl` showed it back to you, and every signal available said the thing was working. The expectation is entirely reasonable. It is the *mechanism* that offers no feedback, and that is worth knowing about in advance precisely because you will not discover it in the moment.

Where else could it possibly live?

You can reason your way to this dependency rather than memorising it.

Chapter 9 taught that Kubernetes **defines** the network model but provides none of the machinery that satisfies it: a CNI network plugin is required to implement the model, and it does the actual work of wiring Pods onto a network [*cross-bearing: see Ch 9 §1 — CNI and the Kubernetes network model*]. CNI is one of the interfaces where Kubernetes hands off to an implementation — and it is the container runtime, not the kubelet, that loads the plugin: CNI management was removed from the kubelet in Kubernetes 1.24 [source: [k8s-docs-network-plugins-2026-08-24](#)].

So if the plugin is what moves the packets, **where else could enforcement possibly live?** Nowhere. The dependency is not an oversight or an inconvenience. It is the only place in the stack where the machinery to enforce a layer-3/4 rule exists.

If you reasoned to something like this in Soundings question 7, you derived the *dependency* before the chapter stated it. Not the consequence — the silence is the part you had no way to predict — but the dependency itself, which is the half that makes the other half inevitable. Notice that.

What it cannot do, stated flat

Dead Reckoning: The source states this list as current "as of" whichever release you happen to be reading — a version-templated claim with no fixed version behind it, so there is no release number to pin here without asserting more than the documentation does. What follows is the list as it stood at this book's source snapshot, 24 August 2026. Treat it as a list that shrinks over time; check the current page before concluding that an item on it is still impossible [source: k8s-docs-network-policies-depth-2026-08-24].

The following functionality does not exist in the NetworkPolicy API [source: k8s-docs-network-policies-depth-2026-08-24]:

- Forcing internal cluster traffic to go through a common gateway.
- Anything TLS related.
- Node specific policies. You can use CIDR notation, but you cannot target nodes by their Kubernetes identities specifically.
- Targeting of services by name. You can target Pods or namespaces by their labels, which is often a viable workaround.
- Creation or management of "Policy requests" that are fulfilled by a third party.
- Default policies which are applied to all namespaces or Pods.
- Advanced policy querying and reachability tooling.
- The ability to log network security events, for example connections that are blocked or accepted.
- The ability to explicitly deny policies.
- The ability to prevent loopback or incoming host traffic. Pods cannot block localhost access, nor can they block access from their resident node.

The documentation notes that some of these may be achievable through operating-system components such as SELinux, OpenVSwitch or IPTables, through layer-7 technologies such as ingress controllers and service mesh implementations, or through admission controllers [source: k8s-docs-network-policies-depth-2026-08-24].

Ten items. Three of them earn a sentence each, because they are the ones you will actually reach for.

No TLS. Anything TLS related is out of scope, and the documentation says outright to use a service mesh or ingress controller for it [source: k8s-docs-network-policies-depth-2026-08-24]. §2 already told you that terminating TLS at the Ingress leaves the leg from Ingress to Pod in cleartext. NetworkPolicy will not encrypt it either. That gap has an owner [*cross-bearing: see Ch 17 §5 — service mesh, mTLS, and what a mesh adds inside the cluster*].

No targeting Services by name. Policies select Pods. You can target Pods or namespaces by label, which the documentation calls a viable workaround [source: k8s-docs-network-policies-depth-2026-08-24], but you cannot write allow traffic to the checkout Service. This is surprising after nine chapters in which

nearly everything has been Service-shaped, and it is the item on this list a reader is most likely to reach for by reflex.

No explicit deny. §6 taught additivity as a *property* of the model: policies grant, and the model has no subtraction operator. The out-of-scope list states that same architectural fact as a *limitation* — the ability to explicitly deny is simply not in the API [source: k8s-docs-network-policies-depth-2026-08-24]. Same fact, met from the side you will actually encounter it on. You go looking for a deny rule, and there is not one.

🦋 **Closer Look:** "No targeting of services by name" is stranger than it looks, and it follows directly from §6. A policy selects Pods. A Service is a stable name in front of a set of Pods that *changes*; that is the entire reason Chapter 9 gave you Services. Selecting the Service would mean selecting a moving target through an indirection that the policy layer, sitting at layer 3/4 on Pod IPs, does not have access to. The restriction is a consequence of the architecture, not an omission from the API. Deeper than the exam requires.

Two objects. Four sections apart. Nothing in common except a failure mode.

🕒 Taking Your Bearings #3

Five questions on §6 and §7 — what is permitted, how permissions add up, and what the mechanism cannot reach.

1. 📖 [retrieval: ch4] Chapter 4 said a NetworkPolicy selects both its subject and its peers. Point at the two selectors in that sentence and say what each one is choosing.
2. 📖 A Pod in namespace `prod` has no NetworkPolicy selecting it anywhere in the cluster. What inbound and outbound traffic is permitted?
3. 📖 ⚠️ **This one is intentionally hard. Struggle is the point.** Two NetworkPolicies select the same Pod. Policy A permits inbound traffic from `app: web`. Policy B permits inbound traffic from `app: batch`. What is permitted, and could a third policy be written to forbid `app: web`? If it could not, what *would* you write to close every Pod in that namespace to all inbound traffic?
4. 📖 Pod `frontend` has an egress policy permitting traffic to `app: api`. Pod `api` has an ingress policy permitting traffic only from `app: admin`. Can `frontend` reach `api`?
5. 📖 You apply a NetworkPolicy intended to block traffic from a specific Pod. Traffic still flows. Name two distinct explanations, one of which is not a mistake in the policy.

Answers with Explanations:

1. [retrieval: ch4] The policy's own top-level podSelector chooses which Pods the policy applies to. The selectors inside its rules — podSelector and namespaceSelector under from/to — choose which Pods and namespaces those Pods may talk to.

Each NetworkPolicy includes a podSelector which selects the grouping of Pods to which the policy applies [source: k8s-docs-network-policies-depth-2026-08-24]; separately, the selectors in an ingress from or egress to section select Pods and namespaces as allowed sources or destinations [source: k8s-docs-network-policies-depth-2026-08-24].

Six chapters ago, learning about labels, you were told this object would use one mechanism for two jobs. This is the structural insight §6 is built on, and you should be able to point at both selectors in a manifest without hesitating.

2. All of both — the Pod is non-isolated for ingress and for egress.

By default a Pod is non-isolated in both directions, and becomes isolated only when some policy both selects it and names that direction in policyTypes [source: k8s-docs-network-policies-depth-2026-08-24].

Wrong answer to reject explicitly: *"no policy means no traffic."* That is the firewall instinct, and it is exactly backwards. It is also the single most consequential wrong belief a reader can bring to this material, because someone holding it will look at a cluster with no NetworkPolicies and conclude it is locked down.

3. Inbound from both app: web and app: batch — the lists combine additively. No: there is no deny rule, so nothing can subtract a permission; removing that access means removing the grant. And to close the namespace, you write a policy that selects every Pod in it and grants nothing.

Network policies do not conflict; they are additive, and the connections allowed in a direction are the union of what the applicable policies allow [source: k8s-docs-network-policies-depth-2026-08-24]. The ability to explicitly deny is on the published out-of-scope list [source: k8s-docs-network-policies-depth-2026-08-24].

If you spent a while looking for the deny rule before accepting there isn't one, that is the correct experience of this question. Every firewall you have configured had one. The absence is genuinely strange, and it is worth stating as a *semantic* rather than a NetworkPolicy quirk: **the model has no subtraction operator.** Permissions compose by union and only by union, order of evaluation is irrelevant, and the only way to reduce what is permitted is to change what grants it.

The third part is the constructive half, and it follows from the first two rather than being a separate technique. An empty podSelector selects every Pod in the namespace [source: k8s-docs-network-policies-depth-2026-08-24]; a policy that selects them all, names Ingress, and offers no from entries isolates every one of them for ingress while permitting nothing. The union of an empty set of grants is empty. **Denial is reached by selecting broadly and granting nothing — never by forbidding.** If you derived that before reading it, you have the model.

Hold on to the phrasing about subtraction. Chapter 12 retrieves this exact semantic by name and builds an argument on it *[cross-bearing: see Ch 12 §9 — additive, never deny]*.

4. No. For a connection from a source Pod to a destination Pod to be allowed, both the egress policy on the source and the ingress policy on the destination must allow it; if either side does not, the connection will not happen [source: k8s-docs-network-policies-depth-2026-08-24].

frontend's egress policy is correct and permits exactly what it should. It is also irrelevant on its own, because api never granted frontend anything. A clearance to depart is not a clearance to enter: the far harbour issues its own, and this one did not. Two objects, both individually sensible, and the connection still fails, which is why this rule costs practitioners so much time.

5. Either: the network plugin does not implement NetworkPolicy, so the resource has no effect at all; or the traffic falls under one of the unconditional exceptions — a Pod cannot block access to itself, and traffic to and from its own node is always allowed [source: k8s-docs-network-policies-depth-2026-08-24].

The question deliberately requires you to entertain that **the object may be perfect and still inert**. That is a move most troubleshooting instincts do not make; the reflex is to re-read the YAML, and the YAML is fine.

Say the detection problem out loud, because it is the part worth carrying: **a policy that is not enforced looks exactly like a policy that is enforced against traffic nobody is sending**. There is no observable difference. That is a property of the mechanism, not of anyone's attention.

Checkpoint: You've Now Mastered

✓ Non-isolated by default, in both directions, until something selects ✓ Additive, allow-only, no deny rule, order-independent ✓ Denial by construction — select everything, grant nothing ✓ Both ends must allow it ✓ The plugin dependency, and the ten things the API cannot do

One section left, and it is about something you already noticed.

☼ §8 — Nothing Happens Without a Controller

This chapter taught two objects that have nothing to do with each other.

One routes HTTP by hostname and path at the edge of the cluster, at layer 7, for traffic coming in from outside. The other permits and forbids TCP connections between Pods inside it, at layers 3 and 4, for traffic that never leaves. Different layers. Different directions. Different problems. In most organizations, different teams.

And they fail the same way.

Write either one perfectly. Apply it successfully. Watch `kubect1 get` return it. And nothing happens, because in one case no Ingress controller is installed, and in the other the network plugin does not implement NetworkPolicy.

☀ **Zenith:**

You have now seen this four times, and two of the four were your own from last chapter.

A type: `LoadBalancer` Service with no provider to fulfill it. A Service whose selector matched no Pods. An Ingress with no controller. A NetworkPolicy on a plugin that does not implement one.

Chapter 3 gave you the sentence — *an object without its component does nothing* — and told you that you would meet it four more times. This is where that debt comes due in full.

But the rule is not a fact about Ingress, and it is not a fact about NetworkPolicy. It is a fact about **what a Kubernetes object is**, which you have held since Chapter 4 without necessarily seeing where it led.

An object is a record of intent. Intent does not act.

Something has to be watching, and willing, and *present*. Every object in this book works this way: the Deployment that produced Pods, the Service that produced a stable name, the Job that ran to completion. In all of those cases the watcher happened to be there, so the object appeared to do the work itself. These four are simply the cases where the watcher is missing, and the appearance falls away.

A chart drawn perfectly is still a chart. Somebody has to stand the watch.

You did not need the book to tell you this, incidentally. Soundings question 6 asked you to write down Chapter 3's rule and name a place you had met it, and if you answered that question you had already assembled most of the argument. The four instances are yours; the sentence was handed to you seven chapters ago.

[cross-bearing: see Ch 3 §6 — the control loop, which is why this is true rather than merely that it is]

[cross-bearing: see Ch 4 §1 — the declarative model, and the object as an artifact of intent]

🔒 **Worth Securing:** You now own a question you can ask about anything: **what is watching this, and is it installed?**

Chapter 13 will hand you a cluster where `kubect1 top` returns an error, and the answer is metrics-server *[cross-bearing: see Ch 13 §7 — the resource metrics pipeline]*. Chapter 17 will hand you VPA, which is an addon and is not shipped by default *[cross-bearing: see Ch 17 §7 — the autoscaling landscape]*. Those two are the instances §3 was counting toward when it called this the first of four; the count above runs the other way, backward over the four already behind you. Chapter 3 published both of those pointers before you had any of the evidence. You have the evidence now.

It also works on objects this book never mentions, which is the actual return on this chapter.



An object without its component does nothing.

☐ VALID

☐ ABSENT

☐ SILENT

Look at the fourth panel one more time. Three of these announce themselves. One does not.

⚓ **Safe Harbor** — Chapter 10 complete. You crossed the boundary from moving packets to reading requests, met the API that does it and the one that supersedes it, went back down two layers to restrict traffic inside the cluster, and collected a rule that will outlast every object in this chapter.

🔐 → 🔒 → 🔑 — *Part III: passage. Two chapters of the network behind you.*

Exam Alert! 🚨

High-Priority Topics

1. **Ingress exposes HTTP and HTTPS only.** No arbitrary ports, no other protocols; anything else uses NodePort or LoadBalancer.
2. **You must have an Ingress controller.** Creating an Ingress resource alone has no effect.
3. **Frozen, not deprecated** — GA, stability guarantees, no removal plans, **and** no further development. Both halves or nothing.
4. **The project recommends using Gateway instead of Ingress.** That is the recommendation as written — it carries no qualifier about new work or existing work. Reading it as *use Gateway for new work* is this book's operational gloss, not the project's wording; §4 sets out why that reading is fair and what it does not license.
5. **Simple fanout routes by URI; name-based virtual hosting routes by host.** Both put many Services behind one address.
6. **An Ingress may terminate TLS**, using a Secret that contains a private key and certificate; traffic onward to the Pods is cleartext.
7. **A Pod is non-isolated in both directions by default.** All ingress and all egress allowed.
8. **Policies are additive and never conflict. There is no deny rule.**
9. **Both ends must allow the connection** — the source's egress and the destination's ingress.
10. **NetworkPolicies are implemented by the network plugin.** No supporting plugin, no effect.
11. **GatewayClass / Gateway / HTTPRoute**, mapped to infrastructure provider / cluster operator / application developer. Exactly one GatewayClass per Gateway; many Routes.
12. **Node-local traffic is always allowed, and a Pod cannot block access to itself.**

Common Traps — each one has a specific wrong belief behind it, and the correction is the thing to carry into the exam room.

The trap	The correction
"Creating an Ingress object exposes the app"	Only creating an Ingress resource has no effect. A controller must be running.
"Ingress can expose any protocol"	HTTP and HTTPS only. Everything else goes back to NodePort or LoadBalancer.
"Ingress is deprecated and will be removed"	Frozen. GA, guaranteed, no removal plans — and no further development.
"All Ingress controllers behave identically"	Ideally they fit the reference specification. In reality they operate slightly differently.
"Creating a NetworkPolicy secures the cluster"	Only if the network plugin implements NetworkPolicy. Otherwise: no effect, no signal.
"A Pod with no NetworkPolicy is closed by default"	Backwards. Non-isolated in both directions until something selects it.
"One NetworkPolicy can deny what another allows"	There is no deny rule. Policies combine by union.
"Only one end needs to permit the connection"	Both. Source's egress <i>and</i> destination's ingress.
"NetworkPolicy can block node-local or self traffic"	Neither. Both exceptions are unconditional.
"NetworkPolicy can do TLS / name targeting / logging / explicit deny"	All four are on the published out-of-scope list.
"Virtual hosting is just DNS"	Opposite sides of the connection. DNS resolves before traffic moves; virtual hosting sorts traffic that has arrived.
"Gateway API is a rename of Ingress"	Different API, different resource model, built around a different organizing principle.
"NetworkPolicy can target a Service"	It selects Pods. Targeting Services by name is explicitly out of scope.
"An Ingress controller and a NetworkPolicy plugin are unrelated concerns"	Functionally unrelated. Structurally identical — which is \$8.

A note on frequency. Every trap above is a real point of confusion, drawn from the documentation's own emphases and from what the material makes easy to get wrong. What this book will not tell you is how often any of them appears on the exam. The published curriculum gives four domain weights and nothing finer [source: cncf-kcna-curriculum-pdf-2026-08-23] — no question counts, no per-competency split, nothing that would let anyone honestly attach a number to a single trap. Inventing one would be worse than saying nothing.

Practice Questions

Nineteen questions, four options each. Four draw on earlier chapters, and they are tagged. Answers follow the full set — attempt them all before scrolling.

1. . Three mechanisms, three altitudes. At which OSI layer does each operate: the Service types from Chapter 9, an Ingress, and a NetworkPolicy?

A) Services at layer 4; Ingress at layer 7; NetworkPolicy at layer 7. B) Services at layer 4; Ingress at layer 7; NetworkPolicy at layer 3 or 4. C) Services at layer 3; Ingress at layer 4; NetworkPolicy at layer 3. D) All three at layer 7, differing only in what they are permitted to configure.

2. . [retrieval: ch9] You need to expose an HTTP application and a message broker speaking its own binary protocol, both to clients outside the cluster. What do you need?

A) One Ingress carrying both, with a ClusterIP Service behind each. B) One Ingress for the HTTP application with a ClusterIP Service behind it; a NodePort or LoadBalancer Service for the broker. C) Two Ingresses, one per workload, each with a ClusterIP Service behind it. D) No Ingress — a LoadBalancer Service for each, since an Ingress routes to Pods and both workloads have several.

3. . Which set correctly lists what an Ingress may be configured to provide?

A) Externally-reachable URLs for Services; load balancing; SSL/TLS termination; name-based virtual hosting. B) Externally-reachable URLs for Services; load balancing; SSL/TLS termination; exposure of arbitrary TCP and UDP ports. C) Externally-reachable URLs for Services; enforcement of which Pods may receive the traffic; SSL/TLS termination; name-based virtual hosting. D) Externally-reachable URLs for Services; load balancing; end-to-end TLS all the way to the Pods; name-based virtual hosting.

4. . An Ingress rule has `host: shop.example.com` and two path entries, `/catalog` and `/checkout`, each naming a different backend Service. Which shape is this, and what is the rule reading in order to decide?

A) Name-based virtual hosting; the rule is reading the `Host:` header. B) Simple fanout; the rule is reading the HTTP URI — the path. C) A single-service Ingress; the rule reads nothing, because `defaultBackend` sends everything to one Service. D) Name-based virtual hosting; the rule is reading the path, because a host is present.

5. . An Ingress path is configured with `pathType: Prefix` and `path: /aaa/bb`. A request arrives for `/aaa/bbb`. Does it match?

A) Yes — `bb` is a prefix of `bbb`, and that is what `Prefix` means. B) No — `Prefix` matches element by element, and `bb` and `bbb` are different elements. C) No — `Prefix` matches only the configured path itself and nothing longer. D) Undeterminable from the manifest, because `pathType` semantics are left to the controller.

6. . A cluster runs one Ingress controller. An engineer applies an Ingress manifest with no `ingressClassName` field. Under what condition does it still get handled?

A) Whenever at least one IngressClass exists in the cluster. B) When exactly one IngressClass is marked as default, by the `ingressclass.kubernetes.io/is-default-class` annotation set to `"true"`. C) Always — with one controller installed, there is nothing else the Ingress could mean. D) Never — `ingressClassName` is a required field and the manifest will be rejected.

7. ... [retrieval: ch3] Chapter 3 said a controller watches for objects and drives reality toward what they describe. An Ingress controller is one. What does it watch, and what does it change?

A) It watches Ingress objects and the Services they reference, and changes a load balancer — or the edge router, or additional frontends — to match. B) It watches Ingress objects and changes the Services they reference. C) The API server watches Ingress objects and configures the load balancer; the controller only validates the manifest. D) It watches Pods and changes the Ingress object's status to match.

8. ... Two objects, both correct as written, both doing nothing: an Ingress on a cluster with no Ingress controller, and a NetworkPolicy on a cluster whose network plugin does not implement NetworkPolicy. What is absent in each, and in what one respect do the two situations differ?

A) The controller and the plugin. They differ in nothing that matters — both fail identically. B) The controller and the plugin. They differ in what you can see: one produces traffic that visibly fails to arrive, the other produces no signal at all. C) A `defaultBackend` and an `ipBlock`. Both objects are incomplete as written. D) The controller and the plugin. They differ in that the Ingress is rejected at admission and the NetworkPolicy is accepted.

9. ... What has the Kubernetes project said about the Ingress API, in both of its halves?

A) That it is deprecated, and scheduled for removal in a future release. B) That it is generally available and subject to the stability guarantees for GA APIs with no plans for removal — and that it is no longer being developed, with no further changes or updates. C) That it is generally available, fully supported, and actively developed for new work. D) That it is no longer being developed, and therefore deprecated by definition.

10. ... You are choosing an API for external HTTP routing on a system being designed today. Which does the Kubernetes project recommend, and does that recommendation mean the other one will stop working?

A) Gateway. Yes — Ingress will be removed once Gateway API reaches GA. B) Gateway. No — Ingress is GA, carries the GA stability guarantees, and has no removal plans. C) Ingress, because it is GA and Gateway API is not present in a default cluster. D) Neither; the project is explicitly neutral between the two.

11. ... Express one requirement twice — one host, two paths, two backend Services — first in the Ingress vocabulary, then in the Gateway API vocabulary, with the owning role for each Gateway API resource.

A) Ingress: one Ingress with one rule and two paths. Gateway API: one Gateway with two paths — the same object renamed, owned by the application developer. B) Ingress: one Ingress with one rule and two paths, plus the two Services and a controller to fulfill it. Gateway API: a GatewayClass (infrastructure

provider), a Gateway (cluster operator), and an HTTPRoute with two path matches and two backendRefs (application developer). C) Ingress: one Ingress per path, so two. Gateway API: one HTTPRoute per path, so two, both owned by the cluster operator. D) Ingress: one Ingress and one IngressClass. Gateway API: one Gateway and one GatewayClass — HTTPRoute being the Gateway API name for an IngressClass.

12. .u In Gateway API, how many GatewayClasses does a Gateway reference, and how many Routes may attach to one Gateway?

A) Exactly one GatewayClass; many Routes. B) Many GatewayClasses; exactly one Route. C) Exactly one of each. D) Many of each.

13. .u A Pod is selected by a NetworkPolicy whose `policyTypes` lists only `Ingress`. What is permitted outbound from that Pod?

A) All outbound traffic. B) No outbound traffic — declaring one direction isolates both. C) Outbound traffic only to Pods in the same namespace. D) Outbound traffic only to the peers named in the policy's `ingress` list, applied in reverse.

14. .u Namespace `prod` contains Pods `web`, `api`, and `db`. One NetworkPolicy exists: it selects `db` and permits ingress from `app: api`. It declares no egress rules. Can `web` reach `db`? Can `web` reach `api`? Can `db` reach an external address?

A) No · Yes · Yes B) No · No · Yes C) No · No · No D) Yes · Yes · No

15. .u An engineer wants to stop a Pod labeled `app: legacy` from reaching the `payments` Pods. Two existing policies currently permit that traffic. What is the approach?

A) Write a more restrictive policy selecting `payments` that excludes `app: legacy`; the more restrictive policy wins. B) Find and remove or narrow the two policies that currently permit it. There is no deny policy to write. C) Add a deny rule naming `app: legacy` to one of the two existing policies. D) Apply a policy selecting `app: legacy` with `policyTypes: [Egress]` and an empty egress list, which removes only its access to `payments`.

16. .u [retrieval: ch4] A single NetworkPolicy carries a selector at the top of its `spec` and further selectors underneath `ingress.from`. Which chooses what — and what happens to the policy's effect on a Pod if someone relabels that Pod out from under the top-level selector?

A) The top-level `podSelector` chooses the Pods the policy governs; the selectors under `ingress.from` choose peers. Relabelling drops the Pod out of the policy's subjects, leaving it non-isolated — less restricted, not more. B) The top-level `podSelector` chooses the Pods the policy governs; the selectors under `ingress.from` choose peers. Relabelling drops the Pod out of every policy, and a Pod outside every policy receives nothing. C) The selectors under `ingress.from` choose the Pods the policy governs; the top-level `podSelector` chooses peers. Relabelling changes nothing about the policy's subjects. D) All the selectors choose peers; the policy governs the whole namespace. Relabelling narrows the peer set.

17.  A NetworkPolicy has been applied. The policy is correct as written, and traffic it should be blocking is still flowing. Three of the four explanations below are ones this chapter has given you. Which is the one that **cannot** be the explanation?

A) The cluster's network plugin does not implement NetworkPolicy, so the resource has no effect. B) The traffic is node-local, or the Pod is reaching itself — neither can be blocked by any policy. C) Another policy in the namespace permits the same traffic, and policies combine by union. D) The policy targets the destination Service rather than the Pods behind it, so it selects nothing.

18.  [retrieval: ch3] Name the rule Chapter 3 gave you about objects and components, and every instance of it you have met in this book so far.

A) *An object without its component does nothing.* Instances: a type: LoadBalancer Service with no provider to fulfill it (Ch 9 §3); a Service whose selector matches no Pods (Ch 9 §4); an Ingress with no Ingress controller (Ch 10 §3); a NetworkPolicy on a plugin that does not implement NetworkPolicy (Ch 10 §7). B) *An object without its component does nothing.* Instances: an Ingress with no Ingress controller, and a NetworkPolicy on a plugin that does not implement NetworkPolicy. C) *An object takes effect once the API server has admitted it.* Instances: the four above. D) *An object without its component does nothing.* Instances: a type: LoadBalancer Service with no provider; an Ingress with no controller; a NetworkPolicy on an unsupported plugin; an Ingress whose pathType does not match the request path.

19.  A cluster runs a frontend Pod that receives requests from users on the internet and, to serve them, calls a backend Pod in the same cluster. Which describes the two flows, and which of this chapter's mechanisms governs each?

A) Both are north-south; Ingress governs the first, NetworkPolicy the second B) The user-to-frontend flow is north-south and is governed by Ingress; the frontend-to-backend flow is east-west and is governed by NetworkPolicy C) The user-to-frontend flow is east-west and is governed by Ingress; the frontend-to-backend flow is north-south and is governed by NetworkPolicy D) Both are east-west, because both flows terminate inside the cluster

Answers with Explanations

1. B.

The documentation attaches OSI layer numbers to exactly one of these three: network policies control traffic flow at the IP address or port level — OSI layer 3 or 4 [source: k8s-docs-network-policies-depth-2026-08-24]. The layer numbering for Services and Ingress is ordinary practitioner vocabulary rather than a documented label; what is documented is the capability difference. Ingress is described as protocol-aware HTTP/HTTPS routing using URIs, hostnames and paths, set against type: LoadBalancer as the simpler, less-configurable path [source: k8s-docs-network-model-2026-08-23].

The shape worth carrying out of the chapter is the round trip. §2 through §5 climb to where the request itself is readable. §6 descends again, and the descent is why §7's limits are the limits they are.

Why A is wrong: it puts NetworkPolicy at layer 7. That is the error behind every expectation that a policy can match a hostname, a URL path, or a Service name — and every one of those is on the published out-of-scope list precisely because the mechanism is not up there.

Why C is wrong: it demotes Ingress to layer 4, which would make it a variant of the Service types rather than a different altitude. If that were true, the broker in question 2 could go behind it.

Why D is wrong: it collapses the distinction the whole chapter is built on. A mechanism that cannot read the request cannot route on its contents, no matter what it is permitted to configure.

2. B. [retrieval: ch9]

An Ingress does not expose arbitrary ports or protocols; exposing services other than HTTP and HTTPS typically uses NodePort or LoadBalancer [source: k8s-docs-ingress-depth-2026-08-24]. The HTTP application still needs a Service behind the Ingress — an Ingress backend is a combination of Service and port, which is the case this chapter covers, rather than a Pod [source: k8s-docs-ingress-depth-2026-08-24] — and a ClusterIP is sufficient, because the Ingress is what makes it externally reachable.

The point of the item is **specialization, not replacement**. A candidate who believes Ingress supersedes the Service ladder goes looking for a way to put the broker behind the Ingress, and there is not one.

Why A is wrong: it is the specialization error in its purest form. The Ingress cannot carry the broker at all, so no arrangement of Services behind it helps.

Why C is wrong: two Ingresses do not solve a protocol problem. Adding a second one that also cannot carry binary traffic changes nothing.

Why D is wrong: the premise is false. An Ingress routes to a Service and port, not to individual Pods, so "several Pods" is not an obstacle — that is what the Service is for.

3. A.

An Ingress may be configured to give Services externally-reachable URLs, load balance traffic, terminate SSL/TLS, and offer name-based virtual hosting [source: k8s-docs-ingress-depth-2026-08-24].

Why B is wrong: arbitrary TCP and UDP ports are the fifth thing people assume is on the list and the one thing explicitly excluded — exposing services other than HTTP and HTTPS typically uses NodePort or LoadBalancer instead [source: k8s-docs-ingress-depth-2026-08-24]. This is the single most common wrong answer about what an Ingress is for.

Why C is wrong: deciding which Pods may receive traffic is NetworkPolicy's job, and it lives at a different layer entirely. An Ingress routes; it does not authorize.

Why D is wrong: the Ingress resource assumes TLS termination at the ingress point — traffic onward to the Service and its Pods is in cleartext [source: k8s-docs-ingress-depth-2026-08-24]. "End-to-end TLS to the Pods" is the opposite of what terminating TLS means, and the distinction matters the moment anyone asks what is on the wire inside the cluster.

4. B.

A fanout configuration routes traffic from a single IP address to more than one Service based on the HTTP URI being requested [source: k8s-docs-ingress-depth-2026-08-24].

Why A is wrong: virtual hosting splits on **host**, and here there is only one host to split on. The tell is where the list lives in the manifest: several entries under `paths` is fanout; several entries under `rules`, each with its own `host`, is virtual hosting.

Why C is wrong: `defaultBackend` handles requests that match none of the rules, and if no `.spec.rules` are specified then `.spec.defaultBackend` must be [source: k8s-docs-ingress-depth-2026-08-24]. This manifest has rules and paths, so the rules do the deciding; the default backend is a fallback, not the mechanism.

Why D is wrong: it gets the mechanism right and the name wrong, which is the more dangerous half of the error. The presence of a `host` does not make something virtual hosting — *several* hosts do.

5. B.

Prefix matching is done on a **path element by path element** basis, and the documentation gives this exact case: Prefix with path `/aaa/bb` against request path `/aaa/bbb` is **not** a match [source: k8s-docs-ingress-depth-2026-08-24]. The elements are compared as whole labels. That `bb` happens to be a string prefix of `bbb` is irrelevant.

Why A is wrong: this is the misconception the rule exists to prevent, and the documentation heads it off explicitly — if the last element of the path is a substring of the last element in the request path, it is not a match [source: k8s-docs-ingress-depth-2026-08-24].

Why C is wrong: it over-corrects. Prefix does match longer request paths — `/aaa/bb/cc` matches — provided every configured element matches a whole element of the request.

Why D is wrong: controllers do vary in documented ways, but this is not one of them. Prefix semantics are pinned by the specification with a worked example. Reaching for "it depends on the controller" where the spec is explicit is the caveat from §3 applied somewhere it does not belong.

6. B.

If `ingressClassName` is omitted and exactly one default IngressClass exists, Kubernetes applies it, and an IngressClass is marked as default by setting the `ingressclass.kubernetes.io/is-default-class` annotation to "true" [source: k8s-docs-ingress-controllers-2026-08-24].

Note the "exactly one." The condition is not "at least one."

Why A is wrong: an IngressClass existing is not the same as an IngressClass being marked default. The annotation is the whole mechanism.

Why C is wrong: the number of controllers installed is not what the rule is written against. The cluster resolves the class from the IngressClass resources, not by inferring that there is only one candidate.

Why D is wrong: `ingressClassName` is optional, which is exactly why the default-class mechanism exists.

7. A.

An Ingress controller is responsible for fulfilling the Ingress, usually with a load balancer, though it may also configure your edge router or additional frontends [source: k8s-docs-ingress-depth-2026-08-24]. That is Chapter 3's control-loop shape with the nouns filled in: desired state recorded in an object, a controller watching, external reality reconciled toward the description.

The value of the item is in converting a memorised component name into a recognized instance of a pattern.

Why B is wrong: the controller reads the Services to learn where to send traffic. It does not modify them. Reversing this makes the Ingress sound like a mutation of the Service layer rather than a layer above it.

Why C is wrong — and this is the misconception worth naming: the API server stores the object and serves it back. It changes nothing outside the cluster. That distinction is the difference between a record and a control loop, and it is the reason an Ingress with no controller sits there looking perfectly healthy.

Why D is wrong: it inverts the direction of the loop. The object is the input, not the output.

8. B.

Both are the same structural failure. Only creating an Ingress resource has no effect [source: k8s-docs-ingress-depth-2026-08-24]; network policies are implemented by the network plugin, and creating a NetworkPolicy resource without a controller that implements it will have no effect [source: k8s-docs-network-policies-depth-2026-08-24]. In both cases nothing is wrong with the object at all, which is why re-reading the YAML produces nothing, indefinitely.

The difference is what reaches you. A website that does not load is a report; nobody has to go looking. Traffic that flows when a policy says it should not produces no report from anyone, because the only party who would notice is the traffic you were trying to stop.

One clarification on provenance: the two "no effect" facts are documented. The characterisation of the second failure as the harder one to detect is this book's reasoning about what those two facts imply, not a documented claim.

Why A is wrong: structurally identical, operationally not. Treating them as the same failure is what leaves the second one running for months.

Why C is wrong: neither object is incomplete. A `defaultBackend` is optional when rules are present, and `ipBlock` is one of three peer identifiers, not a requirement. The whole point is that both manifests review clean.

Why D is wrong: the API server admits both. It stores objects; it does not check that something exists to act on them. If it rejected an Ingress on a controller-less cluster, this chapter's central pattern would not exist.

9. B.

Both halves, in the project's own words: generally available and subject to the stability guarantees for GA APIs, with no plans for removal; and no longer being developed, with no further changes or updates [source: [k8s-docs-ingress-depth-2026-08-24](#)].

Operationally, the first half means there is no migration emergency for what you run today. The second means there is no future capability — whatever Ingress cannot do now, it never will.

Why A is wrong: it drops the stability half and replaces it with the wrong status. `deprecated` is a formal state with published consequences, and Ingress has not been given it.

Why C is wrong: it drops the no-development half. "Fully supported" is true; "actively developed" is the part the announcement specifically denies.

Why D is wrong: it fuses the two halves into a conclusion neither supports. No longer being developed says nothing about removal. GA APIs may be marked deprecated, but must not be removed within a major version of Kubernetes [source: [k8s-docs-ingress-depth-2026-08-24](#)] — and Ingress has not been marked deprecated in the first place.

Offering only one of A or C would teach the other, which is why a well-built item offers both.

10. B.

The Kubernetes project recommends using Gateway instead of Ingress [source: [k8s-docs-ingress-depth-2026-08-24](#)]. It simultaneously states that Ingress is GA, carries the GA stability guarantees, and has no removal plans [source: [k8s-docs-ingress-depth-2026-08-24](#)]. Both facts are true at once and point in different directions, and holding both is the skill this section tests. The practical reading — that the recommendation bites hardest on work being designed today, and least on work already running — is this book's gloss, not the project's wording.

Why A is wrong: Ingress is already GA and already guaranteed. There is no removal event waiting on Gateway API's maturity.

Why C is wrong: GA status is not a recommendation, and installation friction is not either. Gateway API arrives as custom resources rather than in a default cluster [source: [k8s-docs-gateway-api-depth-2026-08-24](#)], which is a real operational cost — and it is a separate question from what the project recommends.

Why D is wrong: the recommendation is explicit and unqualified. Reading neutrality into it is the mirror error of reading a deprecation into it.

11. B.

Ingress vocabulary: one Ingress object, one rule containing two paths, each naming a backend Service and port — plus the two Services and an Ingress controller to fulfill it [source: k8s-docs-ingress-depth-2026-08-24].

Gateway API vocabulary: a GatewayClass belonging to the infrastructure provider, a Gateway belonging to the cluster operator, and an HTTPRoute attached to that Gateway via `parentRefs`, carrying two path matches and two `backendRefs`, belonging to the application developer [source: k8s-docs-gateway-api-depth-2026-08-24].

The exercise is worth doing because it shows Gateway API is not a new routing model to learn from scratch. It is the shapes you already know, redistributed across resources that belong to different owners. The routing requirement did not change. The ownership boundaries did.

Why A is wrong — and it is the tempting one: it reads Gateway API as a rename of Ingress. It is not. Ingress is one object with one owner; Gateway API is three objects with three owners, and that split is the reason the API exists at all. A candidate who believes it is a rename will not be able to say why anyone bothered.

Why C is wrong: neither API needs one object per path. An Ingress rule holds many paths; an HTTPRoute holds many matches. And routes belong to the application developer, not the cluster operator — collapsing the roles is the same error as A wearing different clothes.

Why D is wrong: HTTPRoute is not the Gateway API analogue of IngressClass. GatewayClass is. HTTPRoute is where the routing rules live, which in the Ingress vocabulary is the Ingress object itself.

12. A.

A Gateway references exactly one GatewayClass, and Routes attach to Gateways rather than the other way round, so a single Gateway carries many [source: k8s-docs-gateway-api-depth-2026-08-24].

The cardinality is the kind of detail multiple-choice exams reach for, and it also encodes the role split. One GatewayClass per Gateway, because the infrastructure provider defines one kind of thing and the cluster operator instantiates it. Many Routes per Gateway, because many application teams share one entry point without asking each other's permission — which is the problem the three-role design was drawn to solve.

Why B is wrong: it inverts both. A Gateway that could reference many classes would have no defined infrastructure behind it, and a Gateway limited to one Route would reproduce exactly the per-Service cost problem §1 opened with.

Why C is wrong: one Route per Gateway is the same cost problem again, and it would make the application-developer role meaningless — every new route would require a cluster operator.

Why D is wrong: many GatewayClasses per Gateway would leave the infrastructure ambiguous. The class is what determines the implementation.

13. A.

A Pod is isolated for egress only if some NetworkPolicy both selects it and has Egress in its `policyTypes` [source: k8s-docs-network-policies-depth-2026-08-24]. This policy names only Ingress, so it does not isolate the Pod for egress at all, and the default — non-isolated, all outbound connections allowed [source: k8s-docs-network-policies-depth-2026-08-24] — still stands.

Why B is wrong: the two directions are declared independently. Restricting one says nothing about the other, and this is the most reliably mis-answered fact in the section.

Why C is wrong: namespace boundaries are not a default. Nothing about declaring ingress isolation creates an egress rule of any shape, namespace-scoped or otherwise.

Why D is wrong: ingress rules are not applied in reverse to egress. They govern one direction, and they govern it only because Ingress is named in `policyTypes`.

14. A.

Take them one at a time.

web → db: **no**. db is selected by a policy naming Ingress, so it is isolated for ingress, and the only inbound connections permitted are those from its node and those the policy's ingress list allows — which is `api` only [source: k8s-docs-network-policies-depth-2026-08-24].

web → api: **yes**. api is selected by no policy at all, so it remains non-isolated for ingress and all inbound is allowed [source: k8s-docs-network-policies-depth-2026-08-24]. web is itself unselected, so it is non-isolated for egress. Both ends allow it, which is what a connection needs.

db → external: **yes**. The policy declares no egress rules, and if no `policyTypes` are specified then Ingress is set by default [source: k8s-docs-network-policies-depth-2026-08-24]. db is not isolated for egress, so all outbound connections are allowed.

Why B is wrong: it gets web → api backwards. This is the trap — assuming one policy in a namespace changes the *namespace's* posture. It changes exactly one Pod's posture, in exactly one direction.

Why C is wrong: it extends the same error to db's outbound traffic, treating the policy as a wall around the namespace rather than a statement about one Pod's inbound connections.

Why D is wrong: it swaps the directions — permitting the inbound connection the policy restricts and restricting the outbound one it never mentions.

15. B.

Policies are additive and combine by union [source: k8s-docs-network-policies-depth-2026-08-24], and the ability to explicitly deny is on the published out-of-scope list. If two policies permit the traffic, the only way to stop permitting it is to stop permitting it — remove the grants, or narrow them so app: legacy falls outside.

Why A is wrong: this is the specific shape a firewall-experienced engineer's mistake takes, and it deserves an explicit answer rather than a shrug. There is no precedence between policies. A new policy does not override an old one, no matter how narrow it is; it adds its allowances to theirs. The correction is not "you wrote it wrong." It is "there is no such rule to write."

Why C is wrong: the API has no deny verb to add. A policy expresses what is permitted; there is nowhere in the schema to say what is not.

Why D is wrong, and this one is close enough to be worth slowing down for: selecting app: legacy with policyTypes: [Egress] and an empty egress list *does* work — it isolates the Pod for egress and permits nothing outbound. That is the default-deny idiom, applied correctly. What it does not do is remove *only* its access to payments. It removes access to everything, including DNS. The mechanism is right; the claim about its scope is wrong, and the scope is the whole question.

16. A. [retrieval: ch4]

The top-level podSelector in a policy's spec chooses which Pods the policy governs. The selectors underneath ingress.from — podSelector and namespaceSelector there — choose which peers are permitted [source: k8s-docs-network-policies-depth-2026-08-24]. Same selector grammar from Chapter 4, two entirely different jobs, distinguished only by where they sit in the document. (The third peer identifier, ipBlock, is not a selector at all — it names CIDR ranges, which have no labels to select on.)

Relabel a Pod out from under the top-level selector and the policy stops selecting it. It is not partially governed or governed by a narrower rule. It is outside the policy, and since a Pod is isolated only because some policy selects it, it reverts to non-isolated: all inbound allowed [source: k8s-docs-network-policies-depth-2026-08-24].

That consequence is worth sitting with, because it runs against the instinct. Editing a label — usually the safest change anyone makes — can make a Pod **less** restricted, and nothing about the operation announces that it has done so.

Why B is wrong: it has the mechanism right and the consequence exactly backwards. A Pod selected by no policy is not closed; it is open. This is trap #48 arriving through a side door, and if you chose B, that is the reflex to name.

Why C is wrong: it inverts the two positions. Under that reading, the peers would be governed and the governed Pods would be permitted — which would make every policy in the chapter mean the opposite of what it says.

Why D is wrong: a policy governs the Pods its top-level selector matches, never the namespace wholesale. An empty `podSelector` selects every Pod in the namespace, but that is a selector doing its job, not a default.

17. D.

A `NetworkPolicy` selects Pods. It has no field that names a Service, and targeting Services by name is on the published out-of-scope list [source: [k8s-docs-network-policies-depth-2026-08-24](#)]. So D is not a policy that is failing — it is a policy that cannot be written. If someone reports having written it, they have written something else.

The other three are all live:

A is §7's case. Network policies are implemented by the network plugin, and creating a `NetworkPolicy` resource without a controller that implements it will have no effect [source: [k8s-docs-network-policies-depth-2026-08-24](#)].

B is §6's pair of unconditional exceptions — traffic from a Pod's own node, and a Pod reaching itself, are permitted regardless of any policy [source: [k8s-docs-network-policies-depth-2026-08-24](#)].

C is union semantics doing what union semantics do. Policies are additive [source: [k8s-docs-network-policies-depth-2026-08-24](#)], so a second policy permitting the traffic permits it, and the order the two were written in does not matter.

The discriminating move here is rejecting a candidate that sounds like a policy problem and is not one. Three plausible explanations and a mechanism that does not exist is a harder set than three wrong ones, and it is closer to what a real hour of debugging feels like.

18. A. [retrieval: ch3]

An object without its component does nothing. Four instances so far:

- A type: `LoadBalancer Service` on a cluster with no provider to fulfill it (Ch 9 §3).
- A Service whose selector matches no Pods (Ch 9 §4).
- An Ingress with no Ingress controller (Ch 10 §3) [source: [k8s-docs-ingress-depth-2026-08-24](#)].
- A `NetworkPolicy` on a plugin that does not implement `NetworkPolicy` (Ch 10 §7) [source: [k8s-docs-network-policies-depth-2026-08-24](#)].

If you produced all four, §8's argument is one you assembled rather than one you were handed, and the question you take forward from it — *what is watching this, and is it installed?* — is a tool rather than a slogan.

Why B is wrong, and it is the likeliest miss: the common error here is undercounting, not misnaming. Recalling only this chapter's two instances leaves the pattern looking like an Ingress quirk. It is the two from Chapter 9 that make it a pattern, and it is the pattern that transfers to Chapter 13's metrics-server and Chapter 17's VPA.

Why C is wrong: admission is not the rule. The API server admitted every one of these four objects and stored them faithfully; that is precisely why none of them announced a problem. Mistaking storage for effect is the same error as question 7's option C.

Why D is wrong: it states the rule correctly and then supplies an instance that is not one. An Ingress with a mismatched `pathType` is a broken object with its component present — a manifest bug, findable by reading the manifest. Every genuine instance of this pattern is an object with nothing wrong with it. If re-reading the YAML can fix it, it is not this.

19. B.

North-south traffic enters the cluster from outside; east-west traffic moves between Pods inside it. The user's request crosses the boundary, so it is north-south, and §1–§5 — Ingress and Gateway API — are what govern that crossing. The frontend's call to the backend never leaves the cluster, so it is east-west, and §6–§7 — NetworkPolicy — are what govern that.

A collapses the distinction by calling both flows north-south, which would leave NetworkPolicy governing traffic that crossed the cluster boundary — not what it does. **C** inverts the two terms; this is the miss worth checking yourself against, because the words carry no intrinsic direction and are easy to swap. **D** mistakes *where the traffic ends* for *where it came from*: a request that terminates on a Pod inside the cluster is still north-south if it originated outside.

The pairing is also the chapter's own map: §1–§5 is one direction, §6–§7 the other. If you can place the two halves of this chapter, you can place the two terms.

Chapter Summary

Concept	Remember This
The exposure ceiling	One external address per Service [<i>cross-bearing: see Ch 9 §3 — the four Service types</i>]. Fine for one; expensive for fifty. No Service type reads HTTP.
The layer boundary	Chapter 9 moves packets to an address. This chapter reads requests. Which side you are on determines what you can know.
North-south / east-west	Into the cluster / between Pods. §1–§5 is one; §6–§7 is the other.
Edge router	The router enforcing the cluster's firewall policy. A cloud gateway or physical hardware. An Ingress controller may configure it.
Ingress	HTTP and HTTPS routes from outside to Services within, controlled by rules on the resource. Nothing else.
The four capabilities	Externally-reachable URLs, load balancing, TLS termination, name-based virtual hosting.
Simple fanout	One address, one host, many paths → many Services. The rule reads the URI .
Name-based virtual hosting	One address, many hosts → many Services. The rule reads the host .
DNS vs virtual hosting	DNS resolves a name <i>before</i> traffic moves. Virtual hosting sorts traffic that has <i>already arrived</i> . Opposite sides of the connection.
pathType	Exact, Prefix, ImplementationSpecific. Prefix matches element by element, not by string prefix.
Ingress controller	Required. Creating an Ingress alone has no effect. Ideally all fit the reference spec; in reality they differ.
IngressClass	ingressClassName names which controller should fulfill an Ingress. One default class applies when the field is omitted.
Frozen ≠ deprecated	GA + stability guarantees + no removal plans and no further development. Both halves.
Gateway API	Extensible, role-oriented, protocol-aware. Not built in — installed as custom resources.
The three roles	Infrastructure provider → GatewayClass. Cluster operator → Gateway. Application developer → HTTPRoute.
Gateway cardinality	Exactly one GatewayClass per Gateway. Many Routes per Gateway.
NetworkPolicy scope	Layer 3/4, application-centric, applies to connections with a Pod on one or both ends. Not host isolation.
The three identifiers	Pods, namespaces, IP blocks. Selectors for the first two; CIDR for the third.
Non-isolated by default	A Pod is open in both directions until some policy selects it <i>and</i> names that direction.

Concept	Remember This
Additive, no deny	Policies never conflict; the permitted set is the union. Removing access means removing the grant.
Both ends	Source's egress <i>and</i> destination's ingress. Either one refusing kills the connection.
Default-deny by construction	Empty podSelector, both policyTypes, no rules. Isolation without permission is denial.
The two exceptions	A Pod cannot block access to itself. Node-local traffic is always allowed.
Plugin dependency	NetworkPolicies are implemented by the network plugin. No supporting plugin, no effect, no signal.
Out of scope	No TLS, no Service-name targeting, no logging, no explicit deny, no loopback blocking, and five more.
The rule	An object without its component does nothing. Four instances of the pattern so far. Ask: <i>what is watching this, and is it installed?</i>

The Voyage Ahead

You have spent two chapters on the network, and you have been treating one thing as a given the entire time: that a Pod which restarts is a Pod that starts over. Chapter 5 said it directly — Pods are cattle, replaced rather than repaired — and Chapters 6 through 10 have quietly depended on it. A Service can point at an interchangeable set of backends precisely because they *are* interchangeable.

Chapter 11 is where that assumption runs out.

Some workloads write things down. A database has a disk, and the contents of that disk are not a detail of the Pod. They are the entire point of running it. Chapter 11 works out what happens to data when the Pod that produced it is deleted, and the answer turns out to be a ladder of three different lifetimes, only one of which survives the thing that created it.

It also closes a loop this book left open on purpose. Chapter 6 introduced StatefulSet and told you it was about stable *identity*, not about writing to disk, and then admitted the explanation was incomplete and would stay that way until storage arrived. Storage arrives in Chapter 11, and the second half of that answer arrives with it: what a per-replica volume claim is, and why it outlives not just the Pod but the rescheduling.

And you will meet the last of the four pluggable interfaces. You have three of them already, collected one chapter at a time: CRI at the container runtime in Chapter 2, CRDs at the API itself in Chapter 6, CNI at the network last chapter. If you counted along with Chapter 9 and arrived at two, you were counting something narrower and counting it correctly — that chapter was tallying the times Kubernetes *hands the work to somebody else*, which CRI and CNI both do and CRDs do not. This is the wider set: the four places the project publishes an interface and lets you supply what sits behind it. Chapter 11 brings CSI, at

storage, and that closes the set. By the time Chapter 17 gathers all four in one place, the shape should be familiar enough that the gathering feels like recognition rather than instruction.

One caution about that number, since this chapter has made a point of separating what a source says from what we have concluded from it. *The four pluggable interfaces* is this book's phrase and this book's grouping. The documentation's own map of where Kubernetes can be extended is larger than ours and cut differently: six extension points, five plugin types under infrastructure alone, and custom resources filed under a different heading entirely [source: k8s-docs-extending-kubernetes-2026-08-23]. Chapter 17 sets both maps side by side [cross-bearing: see Ch 17 §4 — *the four pluggable interfaces, collected*]. What makes our four a set is a judgement rather than a heading: at each of them, Kubernetes defines an interface and hands the implementation to somebody else. The judgement is ours. The four interfaces are real, and each one is sourced where you met it.

One last thing to carry across the chapter boundary. You will meet several objects in Chapter 11 that describe storage without providing any, and at least one arrangement where a claim sits unbound because the thing that would satisfy it has not been installed.

You know what question to ask about that now.

■ *"An object is a record of intent. Intent does not act. Something has to be watching."*

Chapter 11: Below the Waterline

"Storage outlives the Pod that asked for it"

Exam Domain: Container Orchestration — Storage (D2.4) | Domain Weight: 28% [source: cncf-kcna-curriculum-pdf-2026-08-23] Authored chapter allocation: ~5% of the total exam (CNCf publishes four domain weights and no sub-competency weights; the D2.4 identifier is this book's own numbering, not CNCf's) | Complexity: Mixed | Novelty: Moderate

Attention Budget

Total time: ~135 minutes | Recommended: Split across 2 sessions — roughly 70 minutes up to the break after §4, and 65 after it

Section	Time	Attention Cost	Best Time to Study
📍 Soundings	8 min	Low	Before you read anything else
Why This Chapter Matters · What You'll Learn	4 min	Low	Anytime
§1 Three Lifetimes, and the Volumes That Have Them	15 min	Low	Anytime
§2 The Claim and the Supply	12 min	Medium	Mid-session
🕒 Taking Your Bearings #1	6 min	Medium	After a brief pause
§3 Provisioning on Demand	12 min	Medium	Mid-session
§4 Access Modes and What Happens After	14 min	Medium	Peak attention — the densest concentration of this chapter's named exam traps
🕒 Taking Your Bearings #2	8 min	High	After a real break; this one is hard on purpose
§5 Who Actually Provides the Storage	8 min	Low	Anytime
§6 Pods That Are Not Interchangeable, Revisited	10 min	Medium	Mid-session
🕒 Taking Your Bearings #3	6 min	Medium	After brief break
§7 Outliving the Pod That Asked	4 min	Low	Anytime
Exam Alert	4 min	Low	Anytime
Practice Questions (17, with answer keys)	20 min	High	Its own sitting — the questions interleave sections, and the answer keys teach
Chapter Summary	3 min	Low	Anytime
The Voyage Ahead	2 min	Low	Anytime

Attention Cost Key:

- **Low:** Concrete, familiar concepts — study anytime
- **Medium:** New concepts requiring focus — study when alert
- **High:** Abstract, complex, or effortful retrieval — study at peak attention

If you only have 15 minutes: read §4. More of this chapter's named exam traps sit there than anywhere else.

If you have thirty-five: read §2 and §4, then take ☹ Taking Your Bearings #2.

"The cargo does not belong to the crew. It was aboard before this watch, and it will be aboard after."
— Lodestar Ledgers

🔍 Soundings

Before reading this chapter, try these eight questions. Your score determines how to approach the content. No score is a bad score; each one points at a different reading strategy.

1. A container writes a file to `/tmp/report.json`, then the process inside it crashes. The kubelet restarts the container. Is the file still there? Which layer was it written to?
2. Chapter 5 said that every container in a single Pod shares two things. What are they? One of them is what this entire chapter is about.
3. Is a `PersistentVolume` a namespaced resource or a cluster-scoped one? How would you settle the question from the cluster itself, without looking it up?
4. What makes a `StatefulSet`'s Pods non-interchangeable in a way a `Deployment`'s Pods are not? And what did Chapter 6 say, in as many words, that it was deliberately *not* explaining yet?
5. In a traditional storage array, who creates a **LUN (Logical Unit Number)** and who requests one? Does the person requesting need to know the array's vendor?
6. Chapter 10 asked you to hold a count. Name the pluggable interfaces you have met so far in this book, and say what each one hands to somebody outside the Kubernetes project.
7. You delete a virtual machine. Does its attached disk go with it? Does your answer depend on how the disk came to exist in the first place?
8. Can two separate machines mount the same block device read-write at the same time, without a clustered filesystem underneath? What goes wrong if they try?

Click for answers + reading strategy

1. **No — the file is gone.** It was written to the container's writable layer, which is discarded and rebuilt from the image when the container restarts. *[cross-bearing: see Ch 2 §2 — the writable container layer]*

2. **A network namespace (and therefore an IP address) and volumes.** Volumes are this chapter's whole subject. *[cross-bearing: see Ch 5 §1 — the Pod as the unit of scheduling]*
3. **Cluster-scoped.** You settle it from the cluster with `kubectl api-resources --namespaced=false`, which lists every resource kind that does not live inside a Namespace. *[cross-bearing: see Ch 4 §3 — namespaced vs cluster-scoped]*
4. **A stable ordinal identity that survives rescheduling.** `web-0` stays `web-0` on whatever node it lands on. Chapter 6 said outright that it was deferring how that Pod's storage is *provisioned, requested, sized, reclaimed, or shared*. This chapter is where that debt comes due.
5. **The storage administrator creates the LUN; the application team requests one.** The requester specifies a size and a performance need, and does not have to know whether the array is NetApp, Pure, or a pile of disks in a closet.
6. **Three so far.** CRI at the container runtime (Chapter 2), CRDs at the API itself (Chapter 6), CNI at the network (Chapter 9). Each hands a published contract to somebody outside the Kubernetes project so they can plug in an implementation without editing Kubernetes' source. The fourth arrives in §5 of this chapter.
7. **It depends, and the "depends" is the interesting part.** A boot disk created automatically with the VM usually dies with it; a disk you provisioned separately and attached usually survives. Provisioning history determines deletion behavior.
8. **No, not safely.** This is general storage background rather than a Kubernetes rule: two independent filesystem drivers each believing they own the block device will corrupt it, because each caches metadata the other does not know it changed. A clustered filesystem exists to coordinate exactly that.

Scoring. Seven of the eight questions above ask for two things. Count a question right only if both halves are right — a half-answer is a gap, and the rubric below is calibrated on that basis.

If you got 6+ right: Skim this chapter. Focus on the ☆ Fixed Points and the ⚠ Navigational Hazards callouts, and go straight to ☺ Taking Your Bearings #2. That checkpoint is where this chapter's domain analysis puts the heaviest exam yield, and where a confident reader is still most likely to lose points.

If you got 3–5 right: Read at normal pace. The material is in reach and this chapter is calibrated for you.

If you got 0–2 right: Read carefully, but do not go back and re-read whole chapters. Check three *sections*: Ch 5 §1 (what a Pod shares), Ch 4 §3 (cluster-scoped resources), and Ch 6 §6 (StatefulSet identity). That is three sections, not three chapters. This material assumes narrow, specific things rather than everything that came before it.

Why This Chapter Matters

For ten chapters, everything you have learned has rested on a single assumption: a Pod is disposable. It can be deleted and replaced. It can be rescheduled onto a different node. It can be scaled from three to thirty and back to one, and no individual Pod's disappearance is an event worth naming.

That assumption is what makes Kubernetes work. It is also exactly what a database cannot tolerate.

So here is the question this chapter opens and does not close until §4: **when the Pod is gone, who decides whether the data is gone too, and when did they decide it?** Not *whether* you can attach storage; you can, and it is not difficult. The question is who holds the authority over the data's survival, and at what moment they exercised it. The answer is more interesting than "you do," and it arrives later than you would expect.

The move this chapter teaches is the one that separates people who *use* a cluster from people who *run* one: separating the request for storage from the supply of it. A developer who understands that they write a claim and never a volume can be handed a cluster whose backing store they have never heard of — Ceph, **EBS (Elastic Block Store)**, a NetApp filer, a single **NFS (Network File System)** export in a rack somewhere — and be productive on it that same afternoon. That is not an exam trick. It is the actual division of labor on every platform team you will ever join.

State this plainly, once: the failure modes in this chapter destroy data rather than merely stop traffic. A Service misconfigured badly is an outage, and outages end. A reclaim policy misunderstood is a deleted volume, and that does not end. This is the chapter where reading carefully has a different payoff than it does elsewhere.

Dead Reckoning: On-disk files in a container are ephemeral. When a container crashes or is stopped, the container state is not saved, so all files created or modified during that container's lifetime are lost; the kubelet restarts the container with a clean state [source: k8s-docs-volumes-2026-08-23]. Volumes exist to solve two problems: surviving container crashes, and sharing files between containers running in one Pod [source: k8s-docs-volumes-2026-08-23]. Persistent volumes exist to solve a third: surviving the Pod itself. Ephemeral volume types have a lifetime linked to a specific Pod; persistent volumes exist beyond the lifetime of any individual Pod [source: k8s-docs-volumes-2026-08-23].

Extended Analogy: Think of the ship's hold, below the waterline.

The cargo is not part of the crew. The crew is relieved and replaced on a schedule that has nothing to do with what is in the hold. Watches change, hands sign off at the end of a voyage, a new complement comes aboard for the next one. Through all of it, the cargo sits where it was stowed.

The hold is inventoried, claimed, and released by a process that runs on entirely different rails from the watch rotation. A shipper files a claim against space in the hold. Somebody working from the stowage plan decides which part of the hold satisfies it. When the claim is released, what happens next — is the cargo landed, is it held for the next voyage, is it destroyed — was settled by the terms of the arrangement long before anyone came to collect it.

That last sentence is this chapter's whole argument. Hold onto it.

What You'll Learn

By the end of this chapter, you'll be able to:

- **Trace** a file from the container filesystem outward, and say at which of three boundaries it stops surviving.
- **Distinguish** a PersistentVolume from a PersistentVolumeClaim from a StorageClass — the three-way distinction this book's domain analysis puts at the front of the Storage competency — and say which one a Pod actually references.
- **Predict** whether a claim will bind, sit unbound, or trigger a volume into existence, given a cluster's provisioning setup.
- **Read** an access mode as a statement about how many *nodes*, not how many Pods (a distinction with an entire fourth mode devoted to it, which should tell you something about how often it gets missed).
- **State** what happens to the data after the claim is deleted, and where that decision was actually made.
- **Explain** why a StatefulSet's storage survives not just a Pod restart but a reschedule onto a different node, closing the loop Chapter 6 opened on purpose.

You will also collect the last of the four pluggable interfaces, and with it a rule about interfaces that Chapter 17 will ask you to state without help.

§1 — Three Lifetimes, and the Volumes That Have Them

Chapter 10 told you, in its closing pages, that this chapter opens with a ladder of three different lifetimes, only one of which survives the thing that created it. Here is that ladder. The count is exactly three.

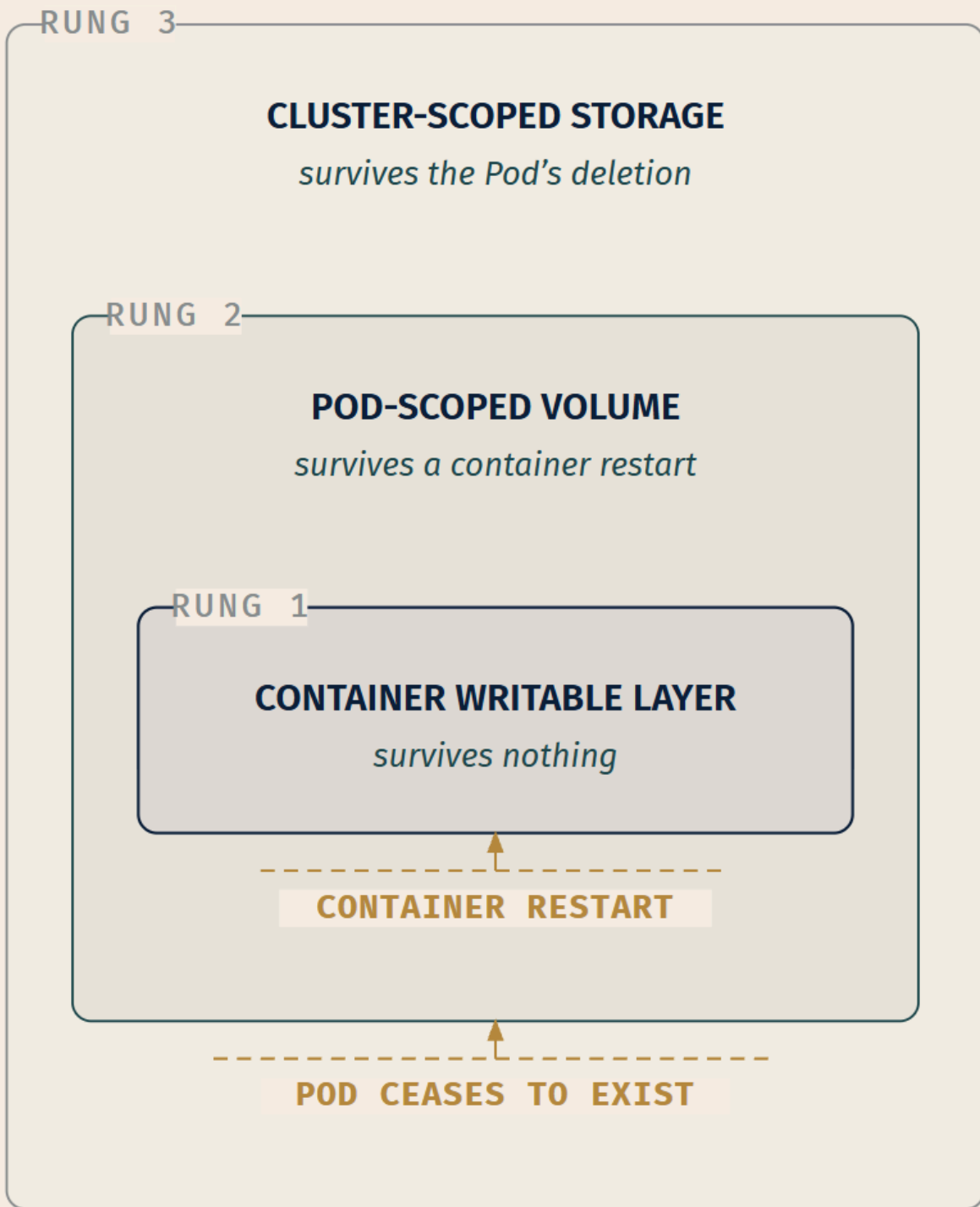
Start at the bottom, where you already are.

A container's filesystem is assembled from the image's read-only layers with a single writable layer on top [*cross-bearing: see Ch 2 §2 — the writable container layer*]. Everything the process writes goes into that writable layer. When the container stops — crash, OOM kill, a `SIGTERM` it handled badly — the kubelet restarts it, and the restarted container gets a clean state assembled fresh from the image [source: k8s-docs-volumes-2026-08-23]. The writable layer is discarded. That is rung one, and the boundary that ends it is **a container restart**.

Now attach a volume. A volume declared in the PodSpec and mounted into a container is not part of the container's filesystem stack; it is mounted into it. When the container restarts, the volume is still there, because the volume belongs to the Pod, not to the container. The documentation states this as a flat rule: *for any kind of volume in a given Pod, data is preserved across container restarts* [source: k8s-docs-volumes-2026-08-23]. That is rung two. It survives the boundary that killed rung one.

But rung two has a boundary of its own. **When a Pod ceases to exist, Kubernetes destroys ephemeral volumes** [source: k8s-docs-volumes-2026-08-23]. Not when the Pod is unhealthy, not when a container inside it dies: when the Pod object itself is gone. Delete the Pod, and its `emptyDir` goes with it.

Rung three is what is left when you ask for storage whose lifetime is not tied to any Pod at all. **Kubernetes does not destroy persistent volumes** [source: k8s-docs-volumes-2026-08-23]. That is the entire distinction, and it is the hold the epigraph was describing: aboard before this watch, aboard after. §2 onward is about how you get one.



LEGEND - - - - DATA ABOVE THE LINE IS DISCARDED

☆ Fixed Point

Three lifetimes, two boundaries. The container's writable layer is destroyed by a **container restart**. A Pod-scoped (ephemeral) volume survives that, and is destroyed when **the Pod ceases to exist**. Cluster-scoped persistent storage survives both, because nothing in a Pod's lifecycle reaches it. Every storage question in Kubernetes is a question about which rung you are standing on.

The volumes that live on rung two

With the ladder in place, the volume types hang off it cleanly. Nearly all of the ones you will meet on the exam are rung-two types: their lifetime is the Pod's.

`emptyDir` is the plain one. The volume is created when the Pod is assigned to a node, and as the name says, it is initially empty [source: k8s-docs-volume-types-depth-2026-08-25]. Every container in the Pod can read and write the same files in it, which is the shared-filesystem half of what Chapter 5 told you a Pod's containers have in common [cross-bearing: see Ch 5 §1 — the Pod's shared context]. Chapter 5 named it and left it; here it is.

The lifetime rules for `emptyDir` are the ladder restated in one type. *When a Pod is removed from a node for any reason, the data in the `emptyDir` is deleted permanently.* And immediately after: *A container crashing does not remove a Pod from a node, therefore the data in an `emptyDir` volume is safe across container crashes* [source: k8s-docs-volume-types-depth-2026-08-25].

Two knobs matter. Setting `emptyDir.medium` to "Memory" makes Kubernetes mount a `tmpfs` — a RAM-backed filesystem — instead of using disk [source: k8s-docs-volume-types-depth-2026-08-25]. Fast, and with a catch the documentation states directly: *while `tmpfs` is very fast, be aware that, unlike disks, files you write count against the memory limit of the container that wrote them* [source: k8s-docs-volume-types-depth-2026-08-25]. A process that fills a memory-backed `emptyDir` gets OOM-killed for it [cross-bearing: see Ch 5 §8 — requests, limits, and what a Pod is owed]. The second knob is `sizeLimit`, which caps the volume's capacity on the default medium [source: k8s-docs-volume-types-depth-2026-08-25].

📌 **Snag:** By default there is *no* cap. The documentation is blunt: *there is no limit on how much space an `emptyDir` or `hostPath` volume can consume, and no isolation between containers or Pods* [source: k8s-docs-volumes-csi-and-subpath-2026-08-25]. A runaway log writer in one Pod can fill the node's disk and put every other Pod on that node into disk pressure. `sizeLimit` exists because the default is unbounded.

`hostPath` mounts a file or directory from the host node's filesystem into your Pod [source: k8s-docs-volume-types-depth-2026-08-25]. It has an optional `type` field alongside the required path [source: k8s-docs-volume-types-depth-2026-08-25]. That field controls what Kubernetes checks before mounting: whether the path must already exist, must be a directory, may be created, must be a socket, and so on [source: k8s-docs-volumes-2026-08-23]. The documentation's own framing of `hostPath` is exactly right: *this is not something that most Pods will need, but it offers a powerful escape hatch for some applications* [source: k8s-docs-volume-types-depth-2026-08-25].

An escape hatch is precisely what it is, and escape hatches open both ways.

⚠ **Navigational Hazards: `hostPath` is a hole in the wall**

The documentation opens its warning with a sentence that does not hedge: *using the `hostPath` volume type presents many security risks. If you can avoid using a `hostPath` volume, you should.* [source: k8s-docs-volume-types-depth-2026-08-25]

Why it is dangerous is more instructive than that it is. *Access to the host filesystem can expose privileged system credentials (such as for the kubelet) or privileged APIs (such as the container runtime socket) that can be used for container escape or to attack other parts of the cluster* [source: k8s-docs-volume-types-depth-2026-08-25]. A Pod that can read `/var/lib/kubelet` can read the node's credentials. A Pod that can write to the container runtime socket can start a privileged container of its own choosing. The Pod boundary you have spent ten chapters trusting is only as strong as the mounts you allow through it.

Restricting access to specific host directories through admission-time validation only holds if those mounts are additionally required to be read-only; give an untrusted Pod a read-write mount and its containers may be able to subvert the restriction [source: k8s-docs-volume-types-depth-2026-08-25].

There is a second-order problem too, and it is the kind that bites in production rather than on an exam: *Pods with identical configuration (such as created from a PodTemplate) may behave differently on different nodes due to different files on the nodes* [source: k8s-docs-volume-types-depth-2026-08-25]. A `hostPath` mount silently makes your Pods node-dependent. The replica that works and the replica that doesn't are running the same image.

This is the workload-to-host boundary problem, and it is why an entire security apparatus exists to police it. Named here, and left here: [cross-bearing: see Ch 12 §5 — *what a Pod may do to its node*].

`configMap` and `secret` as volume sources. Chapter 4 taught you both objects and then told you they would reappear here as volume types mounted into a filesystem [cross-bearing: see Ch 4 §4 — *configuration kept outside the image*]. This is that reappearance. A `ConfigMap`'s data can be referenced in a volume of type `configMap` and consumed as files by the containerized application [source: k8s-docs-volumes-2026-08-23]. A `secret` volume does the same for a `Secret`: *you can store secrets in the Kubernetes API and mount them as files for use by Pods without coupling to Kubernetes directly* [source: k8s-docs-volume-types-depth-2026-08-25].

Both are always mounted read-only [source: k8s-docs-volume-types-depth-2026-08-25]. And one detail about `secret` volumes matters more than it looks: *secret volumes are backed by tmpfs (a RAM-backed filesystem), so they are never written to non-volatile storage* [source: k8s-docs-volume-types-depth-2026-08-25]. A `Secret` mounted as a volume does not land on the node's disk. A `Secret` injected as an environment variable is a different proposition entirely, for reasons Chapter 12 will develop [cross-bearing: see Ch 12 §4 — *Secrets are not encrypted*]. File over environment variable is half an argument already, and you now hold that half.

projected is the multiplexer. A *projected volume maps several existing volume sources into the same directory* [source: k8s-docs-projected-volumes-2026-08-25]. The list of projectable sources runs longer than this chapter needs; the four that matter to you are `secret`, `configMap`, `serviceAccountToken`, and `downwardAPI` [source: k8s-docs-projected-volumes-2026-08-25].

You have already used one of these without being told what it was. In Chapter 5, a Pod's ServiceAccount token arrived in the container's filesystem via a projected token volume [*cross-bearing: see Ch 5 §6 — a Pod's identity*]. That was `serviceAccountToken`, one entry in the list above. The mechanism you met carrying an identity token is the same mechanism, generalized: assemble several distinct sources into one directory so the application sees a single coherent config tree instead of four mount points.

downwardAPI makes a Pod's own metadata available to the application running inside it. *Within the volume, you can find the exposed data as read-only files in plain text format* [source: k8s-docs-volume-types-depth-2026-08-25]. A container that needs to know its own Pod name, namespace, or labels reads them from a file rather than being told at build time.

Generic ephemeral volumes are the interesting hybrid, and they are the type most likely to make you re-read the ladder. They are *similar to `emptyDir` volumes in the sense that they provide a per-pod directory for scratch data that is usually empty after provisioning*, but with capabilities `emptyDir` does not have: *storage can be local or network-attached, volumes can have a fixed size that Pods are not able to exceed*, and typical volume operations like snapshotting, cloning, and resizing are supported if the driver supports them [source: k8s-docs-ephemeral-volumes-2026-08-25].

Here is the mechanism. Read it twice; it prefigures §6. The Pod spec carries a full `PersistentVolumeClaim` template inline. When such a Pod is created, *the ephemeral volume controller then creates an actual `PersistentVolumeClaim` object in the same namespace as the Pod and ensures that the `PersistentVolumeClaim` gets deleted when the Pod gets deleted* [source: k8s-docs-ephemeral-volumes-2026-08-25]. A real claim, created for you, garbage-collected with the Pod. Rung two behavior, built out of rung three machinery.

Naming is deterministic: *the name is a combination of the Pod name and volume name, with a hyphen (-) in the middle* [source: k8s-docs-ephemeral-volumes-2026-08-25]. A Pod named `my-app` with a volume named `scratch-volume` produces a PVC named `my-app-scratch-volume`.

🔍 **Closer Look:** That deterministic naming has a sharp edge the documentation calls out explicitly. A Pod named `pod-a` with volume `scratch` and a Pod named `pod` with volume `a-scratch` both compute to the PVC name `pod-a-scratch` [source: k8s-docs-ephemeral-volumes-2026-08-25]. Kubernetes detects the conflict: a PVC is only used for an ephemeral volume if it was created for that Pod, checked via the ownership relationship, and *an existing PVC is not overwritten or modified* [source: k8s-docs-ephemeral-volumes-2026-08-25]. But detection is not resolution: *without the right PVC, the Pod cannot start* [source: k8s-docs-ephemeral-volumes-2026-08-25]. Below KCNA depth; recorded because it is the kind of thing that costs someone an afternoon.

`subPath` is not a volume type at all. It is a mount modifier, and it earns its place here because of one exception you were promised by name in Chapter 4.

The general mechanism first: *sometimes, it is useful to share one volume for multiple uses in a single Pod. The `volumeMounts[*].subPath` property specifies a sub-path inside the referenced volume instead of its root* [source: k8s-docs-volumes-csi-and-subpath-2026-08-25]. One PVC, mounted into a MySQL container at its `mysql` subdirectory and into a PHP container at its `html` subdirectory: two containers, two mount points, one underlying volume.

Now the exception. Chapter 4 told you flatly that a mounted ConfigMap picks up changes when the ConfigMap is updated, and told you the exception had a name and would arrive here *[cross-bearing: see Ch 4 §4 — configuration kept outside the image]*. Here it is, a flat rule with no conditions attached: *a container using a ConfigMap as a `subPath` volume mount will not receive updates when the ConfigMap changes* [source: k8s-docs-volume-types-depth-2026-08-25].

The same applies to the two neighbors: *a container using a Secret as a `subPath` volume mount will not receive Secret updates* [source: k8s-docs-volume-types-depth-2026-08-25], and *a container using the downward API as a `subPath` volume mount does not receive updates when field values change* [source: k8s-docs-volume-types-depth-2026-08-25].

 **Mnemonic: `subPath` cuts the wire.** A whole-volume mount stays connected to the object that feeds it. A `subPath` mount takes a snapshot of one path and stops listening. If you mount config with `subPath` and then wonder why your rolling ConfigMap update did nothing, this is why.

The rung-three teasers

Two volume types on the list belong to rung three and are named here only so you recognize the shape when §2 formalizes it.

`nfs` allows an existing NFS share to be mounted into a Pod, and the contrast with `emptyDir` is exactly the ladder: *unlike `emptyDir`, which is erased when a Pod is removed, the contents of an `nfs` volume are preserved, and the volume is merely unmounted* [source: k8s-docs-volume-types-depth-2026-08-25]. Preserved, not deleted. Merely unmounted. That sentence is rung three in miniature. It also means the data can be pre-populated before any Pod exists, and shared between Pods. NFS in particular *can be mounted by multiple writers simultaneously* [source: k8s-docs-volume-types-depth-2026-08-25], which will matter a great deal in §4.

`local` represents a mounted local storage device: a disk, a partition, a directory [source: k8s-docs-volume-types-depth-2026-08-25]. Unlike `hostPath`, `local` volumes are used *in a durable and portable manner without manually scheduling Pods to nodes, because the system is aware of the volume's node constraints by looking at the node affinity on the PersistentVolume* [source: k8s-docs-volume-types-depth-2026-08-25]. It has a hard restriction: *local volumes can only be used as a statically created PersistentVolume. Dynamic provisioning is not supported* [source: k8s-docs-volume-types-depth-2026-08-25]. And an honest limitation: *if a node becomes unhealthy, then the `Local` volume becomes*

inaccessible to the Pod. The Pod using this volume is unable to run [source: k8s-docs-volume-types-depth-2026-08-25].

Both of those types name a `PersistentVolume`. You have not been told what one is. That is next.

§2 — The Claim and the Supply

Before anything else, dispose of a collision that the documentation itself sets up.

You have just spent a section learning that a **volume** is a field in a `PodSpec`: a thing declared inside a Pod, mounted into its containers, scoped to its lifetime. Now you are going to meet an object called a **PersistentVolume**, and the natural reading is that it is the same noun with a modifier — a volume, but persistent.

It is not. The documentation describes a `PersistentVolume` as being *volume plugins like Volumes, but with a lifecycle independent of any individual Pod that uses the PV* [source: k8s-docs-persistent-volumes-2026-08-23]. That "like Volumes" is precisely what invites the confusion. A `volume` is a field in a `PodSpec`. A `PersistentVolume` is a cluster-scoped API object with its own name, its own lifecycle, and its own existence independent of every Pod in the cluster. Different things. Similar words. Keep them apart.

Here is the arrangement. The cleanest way in is the documentation's own proportion:

Pods consume node resources and PVCs consume PV resources. [source: k8s-docs-persistent-volumes-2026-08-23]

Read that as an analogy with four terms. A Pod does not create CPU; it requests some, and the scheduler finds a node that has it [*cross-bearing: see Ch 7 §2 — what makes a node feasible*]. A `PersistentVolumeClaim` does not create storage; it requests some, and a control loop finds a `PersistentVolume` that has it. The parallel is exact, which is why the documentation reaches for it.

A `PersistentVolume` (PV) is a piece of storage in the cluster that has been provisioned by an administrator or dynamically provisioned using `StorageClasses`. *It is a resource in the cluster just like a node is a cluster resource. This API object captures the details of the implementation of the storage, be that NFS, iSCSI, or a cloud-provider-specific storage system* [source: k8s-docs-persistent-volumes-2026-08-23]. That is the supply side: a real piece of storage, described to Kubernetes, sitting in the cluster waiting to be used.

A `PersistentVolumeClaim` (PVC) is a request for storage by a user [source: k8s-docs-persistent-volumes-2026-08-23]. The glossary is even blunter about what it does and does not contain: a claim *specifies the amount of storage, how the storage will be accessed (read-only, read-write and/or exclusive) and how it is reclaimed (retained, recycled or deleted)*, while *details of the storage itself are described in the `PersistentVolume` object* [source: k8s-glossary-storage-terms-2026-08-25]. A claim says *how much* and *how*. It does not say *which array*.

That split explains something you learned seven chapters ago and probably filed as a fact to memorize. Chapter 4 taught you that a `PersistentVolume` is cluster-scoped, naming it alongside `Nodes` and `StorageClasses` [*cross-bearing: see Ch 4 §3 — where a name lives*]. What it did not tell you — because it had no reason to yet — is that the **claim** is namespaced. You now have the reason for both halves rather than a rule for one. Supply is a cluster-wide resource, like a node: it belongs to no team, and the administrator who provisioned it did so for the cluster. Demand comes from a specific application in a specific namespace, and belongs to whoever owns that namespace. The scoping is not an arbitrary API decision; it is the supply/demand split expressed in the API's own vocabulary. The hold belongs to the ship. The claim belongs to whoever is shipping.

And the claim's namespace is load-bearing at consumption time too: *claims must exist in the same namespace as the Pod using the claim. The cluster finds the claim in the Pod's namespace and uses it to get the PersistentVolume backing the claim* [source: [k8s-docs-persistent-volumes-depth-2026-08-25](#)].

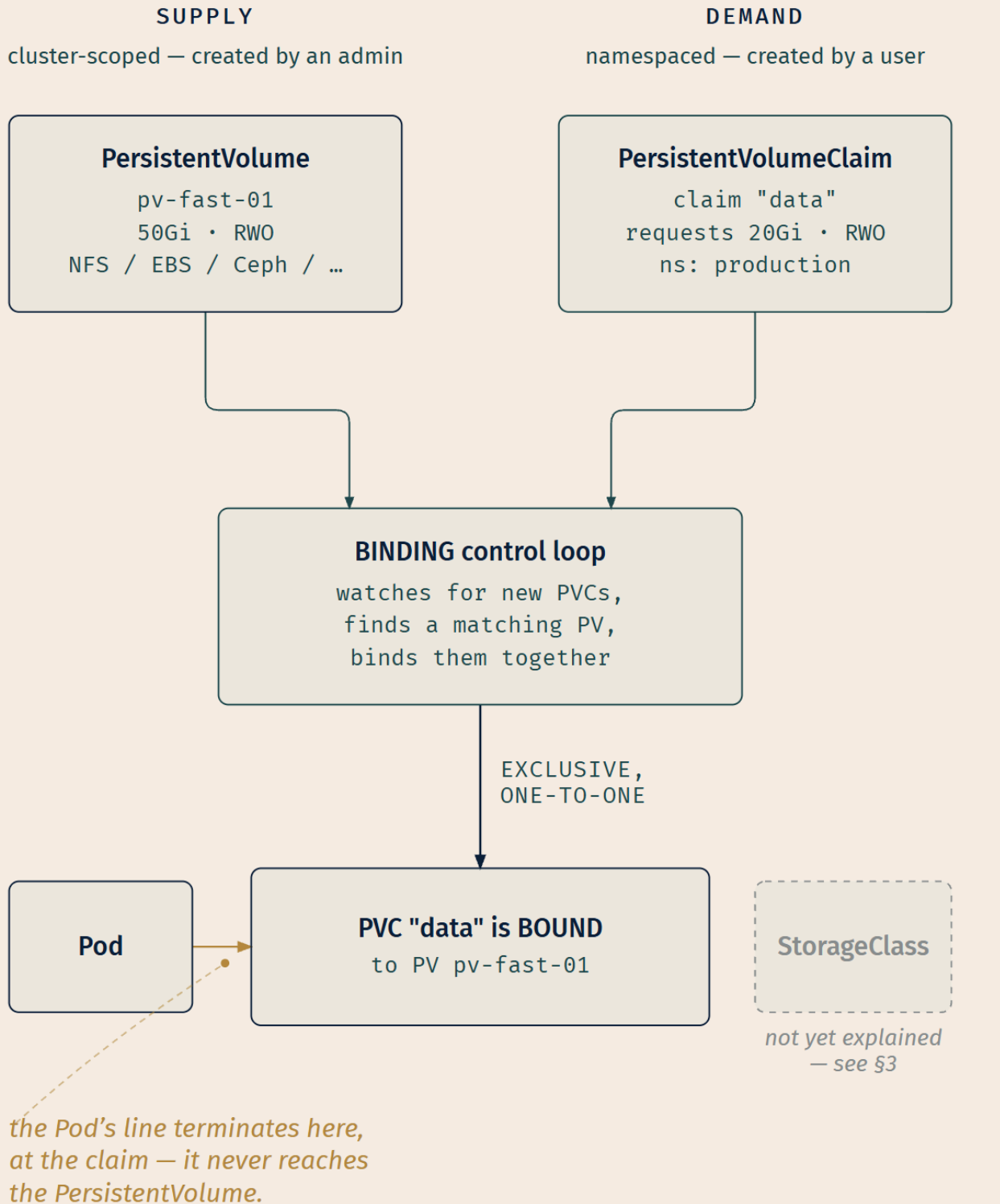
That leaves the routing detail this whole section exists to fix.

☆ Fixed Point

PV is supply. PVC is demand. A Pod references the claim — never the volume.

Pods use claims as volumes. The cluster inspects the claim to find the bound volume and mounts that volume for a Pod. Users schedule Pods and access their claimed PVs by including a `persistentVolumeClaim` section in a Pod's `volumes` block [source: [k8s-docs-persistent-volumes-2026-08-23](#)].

The Pod's line terminates at the PVC. It never touches the PV. That single routing fact is what lets a developer write a manifest that works on a cluster backed by EBS and on a cluster backed by Ceph without changing a character.

**LEGEND**

— Pod references the claim

StorageClass — not yet covered

Binding

The matching is done by a control loop, and naming it as one costs nothing and buys a great deal: *a control loop in the control plane watches for new PVCs, finds a matching PV (if possible), and binds them together* [source: k8s-docs-persistent-volumes-2026-08-23]. Watch, compare desired against actual, act, repeat. That is the same machinery that runs Deployments, Jobs, and every other controller in the cluster [cross-bearing: see Ch 3 §6 — controllers and the control loop]. There is no separate storage subsystem with its own logic. There is a controller, doing what controllers do.

Two properties of binding are exam material and both are counter-intuitive in the same direction: readers expect binding to be looser than it is.

Binding is exclusive and one-to-one. *Once bound, PersistentVolumeClaim binds are exclusive, regardless of how they were bound. A PVC to PV binding is a one-to-one mapping* [source: k8s-docs-persistent-volumes-2026-08-23]. A 50Gi PV satisfying a 20Gi claim does not have 30Gi left over for someone else. It is spoken for, entirely, by that claim.

An unmatched claim waits forever. *Claims will remain unbound indefinitely if a matching volume does not exist. Claims will be bound as matching volumes become available* [source: k8s-docs-persistent-volumes-2026-08-23]. Not an error. Not a timeout. Not a failure event you can alert on cleanly. The claim simply sits, and a Pod that references it does not start.

📌 **Snag:** "The claim is 20Gi and the PV is 50Gi, so it will fit" is a reasonable guess and a wrong one, because *fits* is not the only criterion. A claim can also specify a `storageClassName`, and *only PVs of the requested class, ones with the same storageClassName as the PVC, can be bound to the claim* [source: k8s-docs-persistent-volumes-depth-2026-08-25]. It can specify a label selector, where *only the volumes whose labels match the selector can be bound to the claim* [source: k8s-docs-persistent-volumes-depth-2026-08-25]. All requirements from both `matchLabels` and `matchExpressions` are ANDed together [source: k8s-docs-persistent-volumes-depth-2026-08-25], reusing the selector grammar you already know [cross-bearing: see Ch 4 §5 — labels and selectors]. It can even name a specific volume: *a PVC can specify a particular PersistentVolume by name using the `volumeName` field* [source: k8s-docs-persistent-volumes-depth-2026-08-25]. Capacity is one filter among several.

Phases

A PersistentVolume reports where it stands in its own lifecycle through a phase. The concept documentation names four, and the third one is the one that catches people.

Phase	Meaning
Available	Provisioned, and no claim has taken it
Bound	Matched to a claim, and spoken for by that claim alone
Released	The claim is gone; the cluster has not yet finished with the volume
Failed	Automatic reclamation was attempted and did not succeed

[source: k8s-docs-persistent-volumes-depth-2026-08-25]

Released is not Available. A volume whose claim has been deleted has *left* the bound state without *entering* the free state. Why that gap exists, and what closes it, is §4's business.

📌 **Snag:** Two Kubernetes-project sources count the phases differently, and you should know it before an exam option makes the point for you. The concept documentation names the four above. The API reference for `PersistentVolume v1` additionally documents a `Pending` phase, *used for PersistentVolumes that are not available* [source: k8s-api-ref-persistentvolume-v1-2026-08-25]. Learn the four — they describe the arc from free to bound to released that this chapter is built on. But if `Pending` turns up as an option, do not eliminate it on the grounds that no such phase exists. It exists. It is simply not one of the four the teaching documentation walks you through.

🔒 **Worth Securing:** Kubernetes will not let you pull storage out from under a running Pod by accident. *The purpose of the Storage Object in Use Protection feature is to ensure that PersistentVolumeClaims in active use by a Pod and PersistentVolumes that are bound to PVCs are not removed from the system, as this may result in data loss* [source: k8s-docs-persistent-volumes-depth-2026-08-25]. Delete a PVC that a Pod is using and *the PVC is not removed immediately. PVC removal is postponed until the PVC is no longer actively used by any Pods* [source: k8s-docs-persistent-volumes-depth-2026-08-25]. You will see it sitting in `Terminating` with `kubernetes.io/pvc-protection` in its finalizers list. If you have ever run `kubectl delete pvc` and watched it hang, this is why. It is protecting you.

A claim that never binds is, from the Pod's point of view, indistinguishable from a scheduling failure: the Pod sits in `Pending` and nothing happens. Chapter 13 will teach you to tell those two apart from the symptoms [*cross-bearing: see Ch 13 §2 — Pods that never start*]. This chapter supplies the mechanism; that one supplies the diagnosis.

🕒 Taking Your Bearings #1

Five questions on the lifetime ladder and the supply/demand split. Answers and explanations follow; attempt them first.

1. A Pod runs a single container that writes `/data/cache.db`, where `/data` is an `emptyDir` volume. The container hits a null pointer and exits with code 1. The kubelet restarts it. Then, ten minutes later, you

run `kubectl delete pod` and the Deployment's ReplicaSet creates a replacement. What is the state of `cache.db` after each event?

A) Gone after the crash; gone after the delete B) Present after the crash; gone after the delete C) Present after the crash; present after the delete D) Gone after the crash; present after the delete

2. *[retrieval: ch2]* A different container in a different Pod writes `/tmp/scratch.log`, with no volume mounted anywhere. The process crashes and the kubelet restarts the container. Where was that file written, and is it still there?

A) To the image's read-only layers; still there, because image layers are immutable B) To the container's writable layer; still there, because the layer persists across restarts C) To the container's writable layer; gone, because the restarted container is assembled fresh from the image D) To the container's writable layer; gone, because the Pod was replaced

3. Which statement correctly describes what a Pod references in order to use persistent storage?

A) The Pod's `volumes` block names a `PersistentVolume` directly, and Kubernetes finds a matching claim B) The Pod's `volumes` block names a `PersistentVolumeClaim`, and the cluster uses the claim to find the bound `PersistentVolume` C) The Pod's `volumes` block names a `StorageClass`, which selects a `PersistentVolume` at mount time D) The Pod names both the `PersistentVolume` and the `PersistentVolumeClaim`, so the binding can be verified

4. A cluster has one `PersistentVolume`: 100Gi, `Available`, no `StorageClass`, no labels. A user creates a 10Gi PVC with no class and no selector; it binds. A second user then creates a 5Gi PVC with no class and no selector. What happens to the second claim?

A) It binds to the same PV, since 90Gi of capacity remains unused B) It binds to the same PV and the first claim is evicted, since binding is exclusive C) It remains unbound indefinitely, and will bind only if a matching volume becomes available D) It fails immediately with an error, since no `Available` PV exists

5. An administrator deletes a `PersistentVolumeClaim` whose PV has the `Retain` reclaim policy. Immediately afterward, what phase is the `PersistentVolume` in, and what does that phase mean?

A) `Available` — the volume is free and the next matching claim will bind to it B) `Released` — the claim is gone, but the volume has not been reclaimed and is not yet reusable C) `Failed` — deleting a bound claim is an error condition D) `Bound` — the reclaim policy `Retain` preserves the binding until an admin intervenes

Answers with Explanations:

1 — B. Present after the crash. Gone after the delete. If you reached for C, you reached for the trap this question was built around, and you are in large company.

- **A is wrong** because a container crash does not remove the Pod from the node, and *the data in an emptyDir volume is safe across container crashes* [source: k8s-docs-volume-types-depth-2026-08-25]. The file survives the restart.
- **B is correct.** Crash → survives (rung two beats a container restart). Delete → gone, because *when a Pod is removed from a node for any reason, the data in the emptyDir is deleted permanently* [source: k8s-docs-volume-types-depth-2026-08-25]. And the replacement Pod is a *different Pod*; its emptyDir is created empty when it is assigned to a node.
- **C is wrong** and is the single most common emptyDir misconception: assuming that because a volume survived a container restart, it must be persistent. It survives one boundary and not the other.
- **D is wrong** in both halves, and each half is a belief worth naming. "Gone after the crash" is *restart means fresh everything* — true of the container filesystem, false of the volume. "Present after the delete" is *the controller restores the volume along with the Pod* — the ReplicaSet restores the Pod, and an emptyDir in a new Pod is new and empty. D has the ladder exactly inverted.

2 — C. The file went to the container's writable layer, and the restarted container gets a clean state assembled from the image [source: k8s-docs-volumes-2026-08-23]. Rung one survives nothing.

- **A is wrong** because image layers are read-only; a write to a path backed by an image layer is copied up into the writable layer, not written into the image.
- **B is wrong** because it confuses the writable layer's *location* (correct) with its *lifetime* (wrong). This is the misconception the whole ladder exists to correct.
- **D is wrong** on the event, and this is the near-miss to sit with. The location is right and the outcome is right — the file is gone — but nothing replaced the Pod. The container exited and the kubelet restarted it inside the same Pod, on the same node. Rung one is erased by the *container* boundary, several rungs below the one D names. Arriving at the right outcome through the wrong boundary is still arriving wrongly, and the exam will offer you that option.

3 — B. *Pods use claims as volumes. The cluster inspects the claim to find the bound volume and mounts that volume for a Pod* [source: k8s-docs-persistent-volumes-2026-08-23].

- **A is wrong**, and it is the reversal to watch for. It has the direction of the relationship backwards: claims find volumes, not the other way around, and the Pod's reference terminates at the claim.
- **C is wrong**: a StorageClass is not something a Pod mounts. §3 covers what it actually does.
- **D is wrong**: the Pod names only the claim. It has no field for a PV.

4 — C. *Claims will remain unbound indefinitely if a matching volume does not exist. Claims will be bound as matching volumes become available* [source: k8s-docs-persistent-volumes-2026-08-23]. The only PV is Bound, so there is nothing to match.

- **A is wrong** because a PVC to PV binding is a *one-to-one mapping* and binds are exclusive [source: k8s-docs-persistent-volumes-2026-08-23]. Leftover capacity is not shareable. This is the "big enough, so it fits" trap.
- **B is wrong**: binding exclusivity protects the existing bind; it does not preempt it.

- **D is wrong** in an instructive way. There is no immediate error. The claim sits, quietly, and the Pod that references it sits in Pending. "Fails immediately" would at least be visible; "waits forever" is what actually happens.

5 — B. *When the PersistentVolumeClaim is deleted, the PersistentVolume still exists and the volume is considered 'released'. But it is not yet available for another claim because the previous claimant's data remains on the volume* [source: k8s-docs-persistent-volumes-depth-2026-08-25].

- **A is wrong**, and this specific wrong answer teaches more than the right one. Released ≠ Available. The volume is not reusable. §4 covers what an administrator has to do to make it so.
- **C is wrong**: the Failed phase records a volume whose automatic reclamation did not succeed [source: k8s-docs-persistent-volumes-depth-2026-08-25]. That is a later event and a different one. Deleting a bound claim is routine, not an error.
- **D is wrong**: the binding does not survive the claim's deletion. The claim is gone.

If you scored 0–2: Go back to §1's ladder figure and §2's Fixed Point specifically, about eight minutes of re-reading. The rest of the chapter builds directly on those two.

If you scored 3–4: Solid. Review the ones you missed, understand *why*, and continue.

If you scored 5: You have the foundation. §3 and §4 are where the exam yield concentrates, and you are in good shape to absorb them.

Checkpoint: You've Now Mastered ✓ The three-rung lifetime ladder and the two boundaries that define it
 ✓ Which volume types live on which rung, and what subPath cuts off ✓ PV as supply, PVC as demand, and why a Pod references only the claim ✓ Binding as an exclusive, one-to-one control-loop match that waits indefinitely ✓ The PV phases, and that Released is not Available

Two questions remain open, and they are the ones that matter: where does a PV come from if nobody made one, and what happens to the data when the claim is gone?

📖 → 🔄 → 📌 · **Voyage Progress: two sections of seven. The vocabulary is set; the mechanics start now.**

..1 §3 — Provisioning on Demand

Everything in §2 assumed a PersistentVolume already existed. Somebody made one, it sat there Available, and a claim found it.

That model works, and it has a name. It does not scale past a certain point, for a reason that becomes obvious the moment you try it. An administrator pre-creating PVs has to guess: how many, what sizes, what performance tiers, for which teams. Guess low and claims sit unbound. Guess high and you have

provisioned storage nobody asked for. Guess wrong about the size distribution and you have forty 10Gi volumes and a team that needs one 400Gi volume. It is stocking the hold for a manifest nobody has written yet.

The object that fixes this is the `StorageClass`, and it is the third element that has been sitting off to the side of the figure since §2.

What a `StorageClass` is

A `StorageClass` provides a way for administrators to describe the classes of storage they offer. Different classes might map to quality-of-service levels, or to backup policies, or to arbitrary policies determined by the cluster administrators. Kubernetes itself is unopinionated about what classes represent [source: k8s-docs-storage-classes-2026-08-25].

That last sentence is doing real work. Kubernetes does not know what fast means, or bulk, or archive. It knows those are names an administrator chose and attached to a provisioning configuration. The documentation offers a comparison that lands well for anyone with a storage background: *the Kubernetes concept of a storage class is similar to "profiles" in some other storage system designs* [source: k8s-docs-storage-classes-2026-08-25].

The problem it solves is stated back in the `PersistentVolumes` documentation, and it reads as a problem statement rather than a feature description: *cluster administrators need to be able to offer a variety of `PersistentVolumes` that differ in more ways than size and access modes, without exposing users to the details of how those volumes are implemented. For these needs, there is the `StorageClass` resource* [source: k8s-docs-persistent-volumes-2026-08-23].

Without exposing users to the details. That is the supply/demand split from §2, extended one level: the user asks for fast, and never learns what fast is made of.

Each `StorageClass` contains the fields `provisioner`, `parameters`, and `reclaimPolicy`, which are used when a `PersistentVolume` belonging to the class needs to be dynamically provisioned to satisfy a `PersistentVolumeClaim` [source: k8s-docs-storage-classes-2026-08-25]. And *the name of a `StorageClass` object is significant, and is how users can request a particular class* [source: k8s-docs-storage-classes-2026-08-25]. The name is the API. When a developer writes `storageClassName: low-latency`, they are using that name as the entire interface to a provisioning system they know nothing else about.

Here is what one actually looks like:


```

apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: low-latency
  annotations:
    storageclass.kubernetes.io/is-default-class: "false"
provisioner: csi-driver.example-vendor.example
reclaimPolicy: Retain # default value is Delete
allowVolumeExpansion: true
mountOptions:
  - discard # this might enable UNMAP / TRIM at the block storage layer
volumeBindingMode: WaitForFirstConsumer
parameters:
  guaranteedReadWriteLatency: "true" # provider-specific

```

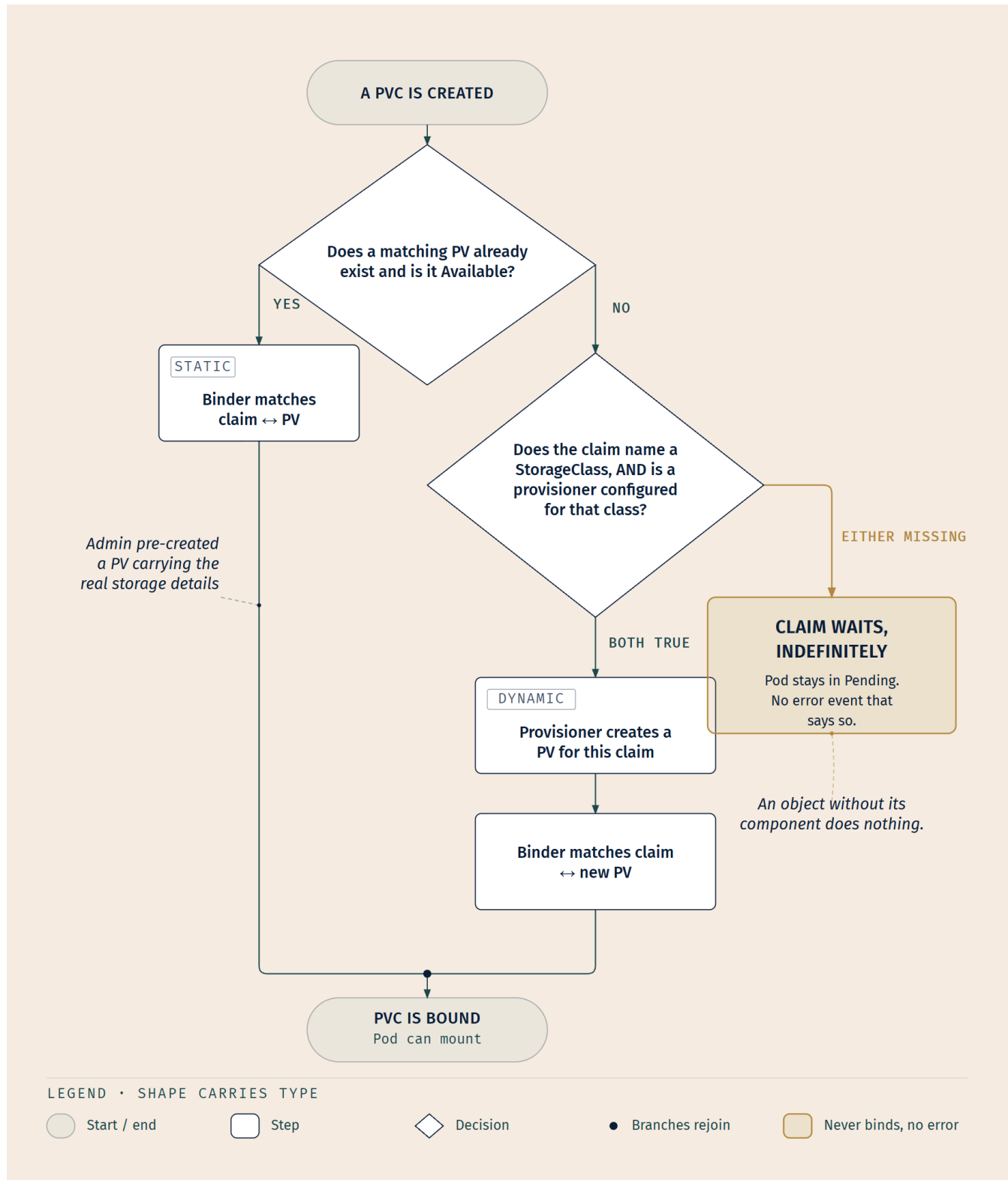
[source: k8s-docs-storage-classes-2026-08-25]

Note parameters. That field is a passthrough: its contents are meaningful to the provisioner and opaque to Kubernetes. `guaranteedReadWriteLatency: "true"` means something to that vendor's driver and nothing at all to the API server.

Two fields in that example, `allowVolumeExpansion` and `mountOptions`, get no treatment in this chapter, and that is deliberate rather than an oversight. They are real provisioning-behavior knobs, and they are reproduced here so you are looking at an actual `StorageClass` rather than a trimmed one. They sit outside this chapter's scope; nothing later depends on them.

The `provisioner` field is the one that makes the class do anything: *each StorageClass has a provisioner that determines what volume plugin is used for provisioning PVs. This field must be specified* [source: k8s-docs-storage-classes-2026-08-25]. Provisioners come in two flavors: internal ones shipped alongside Kubernetes with `kubernetes.io`-prefixed names, and *external provisioners, which are independent programs that follow a specification defined by Kubernetes, whose authors have full discretion over where their code lives, how the provisioner is shipped, how it needs to be run* [source: k8s-docs-storage-classes-2026-08-25]. Hold that phrase — *independent programs* — because §5 is about what happens when nobody runs one.

Two ways a volume comes to exist



Static provisioning is the §2 model, named. A cluster administrator creates a number of PVs. They carry the details of the real storage, which is available for use by cluster users [source: k8s-docs-persistent-

volumes-2026-08-23]. Supply exists first; demand arrives later and matches against it.

Dynamic provisioning inverts the order. *When none of the static PVs the administrator created match a user's PersistentVolumeClaim, the cluster may try to dynamically provision a volume specially for the PVC* [source: k8s-docs-persistent-volumes-2026-08-23]. Demand arrives first, and supply is manufactured to fit it.

Note the word *may*. It is doing precise work, and the next sentence explains why.

☆ Fixed Point

Dynamic provisioning requires two conditions, not one.

This provisioning is based on StorageClasses: the PVC must request a storage class and the administrator must have created and configured that class for dynamic provisioning to occur [source: k8s-docs-persistent-volumes-2026-08-23].

The claim must name a class **and** that class must be configured to provision. One without the other yields a claim that waits forever, which, per §2, is not an error, not a timeout, and not a failure event. It is silence.

Defaults, and one opt-out that surprises people

A cluster does not have to make every developer name a class explicitly. *You can mark a StorageClass as the default for your cluster. When a PVC does not specify a storageClassName, the default StorageClass is used* [source: k8s-docs-storage-classes-2026-08-25]. The mark is an annotation:

`storageclass.kubernetes.io/is-default-class: "true"` [source: k8s-docs-persistent-volumes-depth-2026-08-25].

Where a default exists, that is what lets a developer write a five-line PVC and get a working disk without knowing any of this. Whether a given cluster has one is a question to ask rather than assume; the ☞ below covers what happens when the answer is no.

Now the case that catches people. There are two ways a PVC can appear to "not ask for a class," and they behave differently:

What the PVC has	What happens
No <code>storageClassName</code> field at all	The default StorageClass is used, if one exists
<code>storageClassName: ""</code> (empty string)	Dynamic provisioning is disabled for that claim

A PVC with its `storageClassName` set to "" is explicitly stating that it is bound with a PV with no class, hence the PV's `storageClassName` must also be empty. A PVC with no `storageClassName` is not quite the same and is treated differently by the cluster [source: k8s-docs-persistent-volumes-depth-2026-08-25]. And from the lifecycle section, stated as a consequence: *claims that request the class "" effectively disable dynamic provisioning for themselves* [source: k8s-docs-persistent-volumes-2026-08-23].

📌 **Snag:** `storageClassName: ""` is not "use whatever the cluster's default is." It is the opposite: "do not use a class at all, and do not provision anything for me." A reader who writes it expecting the default gets a claim that will only ever bind to a classless PV, and on a cluster where every PV comes from a StorageClass, that means it never binds. Empty string is an opt-out, not a shrug.

🔒 **Worth Securing:** A cluster can have no default at all: *if you don't mark any StorageClass as default (and one hasn't been set for you by, for example, a cloud provider), then Kubernetes cannot apply that defaulting for PersistentVolumeClaims that need it* [source: k8s-docs-storage-classes-2026-08-25]. Such a PVC is not rejected: *the new PVC creates as you defined it, and the storageClassName of that PVC remains unset until a default becomes available* [source: k8s-docs-storage-classes-2026-08-25]. And if a default appears later, the control plane retroactively updates those waiting PVCs to point at it [source: k8s-docs-storage-classes-2026-08-25]. This is a control loop doing exactly what control loops do: the desired state was underspecified, and the moment the information arrived, it reconciled [cross-bearing: see Ch 3 §6 — controllers and the control loop].

The claim that waits because nobody installed the thing

You were set up for this two chapters ago. Chapter 10's closing said: *you will meet several objects in Chapter 11 that describe storage without providing any, and at least one arrangement where a claim sits unbound because the thing that would satisfy it has not been installed. You know what question to ask about that now.*

You do. The phrase is Chapter 3's, and Chapter 10 §3 named it as a pattern: **an object without its component does nothing** [cross-bearing: see Ch 10 §3 — the object is not the implementation].

A StorageClass is a description of a provisioning capability. It is not the provisioning capability. Create a StorageClass whose provisioner names a driver that nobody deployed, and you get a perfectly valid API object that provisions exactly nothing. Claims naming that class sit unbound. Pods referencing those claims sit Pending.

This is the third sighting of the same light. An Ingress with no Ingress controller routes nothing. A NetworkPolicy on a CNI plugin that doesn't implement policy blocks nothing. A StorageClass with no running provisioner creates nothing. Each time, the object exists, `kubectl get` shows it, `kubectl describe` shows a sensible spec, and nothing happens, because the object was always only ever a request addressed to a component that has to be separately installed and running.

You will see it a fourth time in §5, and by then the recognition should be automatic.

When binding waits for the scheduler

One more StorageClass field, and it connects storage to something you learned four chapters ago in a way that is genuinely load-bearing rather than decorative.

The volumeBindingMode field controls when volume binding and dynamic provisioning should occur. When unset, Immediate mode is used by default [source: k8s-docs-storage-classes-2026-08-25]. Immediate

means what it says: *volume binding and dynamic provisioning occurs once the PersistentVolumeClaim is created* [source: k8s-docs-storage-classes-2026-08-25].

That sounds fine until you think about *where* the volume gets created. Consider a cloud with three availability zones and block storage that can only be attached to instances in the same zone. The claim is created; Immediate binding provisions a volume in zone A; then the scheduler tries to place the Pod and discovers the only nodes with enough memory are in zone C. The documentation states the outcome without softening it: *for storage backends that are topology-constrained and not globally accessible from all Nodes in the cluster, PersistentVolumes will be bound or provisioned without knowledge of the Pod's scheduling requirements. This may result in unschedulable Pods* [source: k8s-docs-storage-classes-2026-08-25].

The fix is to make binding wait: *a cluster administrator can address this issue by specifying the WaitForFirstConsumer mode which will delay the binding and provisioning of a PersistentVolume until a Pod using the PersistentVolumeClaim is created. PersistentVolumes will be selected or provisioned conforming to the topology that is specified by the Pod's scheduling constraints. These include, but are not limited to, resource requirements, node selectors, pod affinity and anti-affinity, and taints and tolerations* [source: k8s-docs-storage-classes-2026-08-25].

Read that list of constraints again: resource requirements, node selectors, affinity, anti-affinity, taints and tolerations. That is Chapter 7's filter phase, item for item [*cross-bearing: see Ch 7 §2 — what makes a node feasible*]. WaitForFirstConsumer exists because scheduling and storage placement are not two decisions that happen to interact. They are one decision, and making them separately produces a Pod that cannot run.

The local volume documentation makes the same argument independently, which is a good sign that it is the real reason rather than a rationalization: *when using local volumes, it is recommended to create a StorageClass with volumeBindingMode set to WaitForFirstConsumer. Delaying volume binding ensures that the PersistentVolumeClaim binding decision will also be evaluated with any other node constraints the Pod may have, such as node resource requirements, node selectors, Pod affinity, and Pod anti-affinity* [source: k8s-docs-volume-types-depth-2026-08-25].

🔍 **Closer Look:** `WaitForFirstConsumer` sits above the depth this book judges KCNA to require. CNCF publishes the `Storage` competency as a single word and nothing more; this book's domain analysis reads that word as centring on the three-way distinction between `PersistentVolume`, `PersistentVolumeClaim`, and `StorageClass`, not on tuning binding modes. What you should carry forward from this subsection is the consequence, not the field name: **volume binding can wait on scheduling, because binding a volume before the scheduler has picked a node can bind it somewhere the Pod cannot go**. If you remember that sentence and forget `volumeBindingMode` entirely, you have taken the right thing.

Two operational notes for the day you meet this in a real cluster. First, the mode is not universally supported: *the following plugins support `WaitForFirstConsumer` with dynamic provisioning: CSI volumes, provided that the specific CSI driver supports this* [source: `k8s-docs-storage-classes-2026-08-25`]. Second, it interacts badly with one particular shortcut: *if you choose to use `WaitForFirstConsumer`, do not use `nodeName` in the Pod spec to specify node affinity. If `nodeName` is used in this case, the scheduler will be bypassed and PVC will remain in pending state* [source: `k8s-docs-storage-classes-2026-08-25`]. Bypass the scheduler [cross-bearing: see Ch 7 §6 — *overruling the scheduler*] and you have removed the very consumer the binding was waiting for.

..1 §4 — Access Modes and What Happens After

Calibrate on the exam yield rather than the difficulty glyph, because on this section the two point in opposite directions.

This section is marked ..1 Standard, and that rating is honest. Access modes and reclaim policies are enumerable facts with no conceptual depth to speak of. There is no hard idea here. But five of the seven exam traps this chapter's domain analysis identified live in this section and the last one, and four of them are here. **This material is not hard, and it is where the points are.** Read it as though it were ..111, and take the checkpoint that follows seriously.

Two questions, and they are the two halves of one question. *What may you do with this volume?* That is access modes. *What happens when you stop?* That is reclaim policy. Anyone who has managed storage before will recognize that these are always asked together.

Dead Reckoning: There are four access modes. `ReadWriteOnce` (RWO): the volume can be mounted as read-write by a single node. `ReadOnlyMany` (ROX): the volume can be mounted as read-only by many nodes. `ReadWriteMany` (RWX): the volume can be mounted as read-write by many nodes.

`ReadWriteOncePod` (RWOP): the volume can be mounted as read-write by a single Pod [source: k8s-docs-persistent-volumes-depth-2026-08-25]. In the CLI they are abbreviated RWO, ROX, RWX, and RWOP [source: k8s-docs-persistent-volumes-depth-2026-08-25]. A volume can only be mounted using one access mode at a time, even if it supports many [source: k8s-docs-persistent-volumes-depth-2026-08-25].

There are three reclaim policies, one of which is deprecated. `Retain`: when the claim is deleted the PV still exists, the volume is considered released, and an administrator must manually reclaim it. `Delete`: removes both the `PersistentVolume` object from Kubernetes and the associated storage asset in the external infrastructure. `Recycle`: deprecated [source: k8s-docs-persistent-volumes-2026-08-23].

The defaults differ by how the volume came to exist: `Retain` is the default for manually created `PersistentVolumes`, and `Delete` is the default for dynamically provisioned `PersistentVolumes` [source: k8s-api-ref-persistentvolume-v1-2026-08-25].

Now the parts that need explaining.

Access modes count nodes

The Soundings asked you whether two machines can safely mount one block device read-write at the same time. They cannot, not without a clustered filesystem coordinating them.

That is not a Kubernetes fact, and no Kubernetes document will tell you so. It is ordinary storage knowledge that the platform inherits from the hardware beneath it. Two independent filesystem drivers, each caching metadata for the same device, will eventually write over each other's bookkeeping and destroy the filesystem. Two crews restowing the same hold from two different manifests lose the cargo, and neither crew notices until somebody goes looking for it.

That physical constraint is what access modes encode. The unit is the **node**, because the node is where the mount happens and the kernel's filesystem driver is what would be doing the corrupting.

The documentation puts the constraint's origin plainly: *a `PersistentVolume` can have any access mode supported by the resource provider. For example, NFS can support multiple read/write clients, but an iSCSI volume can only be used by one client at a time. Each PV gets its own set of access modes describing that specific PV's capabilities* [source: k8s-docs-persistent-volumes-depth-2026-08-25]. Access mode is not a policy you choose. It is a capability the underlying storage either has or does not.

And here is the sentence that generates the single most common storage mistake in Kubernetes, followed immediately by its own remedy: *`ReadWriteOnce` access mode still can allow multiple Pods to access the volume when the Pods are running on the same node. For single Pod access, use `ReadWriteOncePod`* [source: k8s-docs-persistent-volumes-depth-2026-08-25].

☆ Fixed Point

RWO counts nodes. RWOP is the one that counts Pods.

Three of the four modes are statements about how many *nodes* may mount the volume. Only `ReadWriteOncePod` changes the unit: *use ReadWriteOncePod access mode if you want to ensure that only one Pod across whole cluster can read and write that PVC* [source: k8s-docs-persistent-volumes-depth-2026-08-25].

The existence of a fourth mode devoted entirely to the Pod/node distinction is itself the evidence that the distinction is hard to hold. Kubernetes shipped a whole access mode because people kept assuming RWO already meant it.

WHAT YOU MAY DO

Access Modes

unit = nodes (except RWOP)

RWO

☒ ☐ ☐

1 node · read-write

ROX

☒ ☒ ☒

many nodes · read-only

RWX

☒ ☒ ☒

many nodes · read-write

RWOP

☒ ☐ ☐

1 **POD** · read-write

the unit changes HERE, and only here

WHAT HAPPENS AFTER

Reclaim Policies

when the claim is deleted

	PV	OBJ	ASSET	DATA
Retain	kept	kept	kept	kept
Delete	gone	gone	gone	gone
Recycle (deprecated)	—	—	—	—

DEFAULT FOR DYNAMIC PROVISIONING

Delete

LEGEND

☒ NODE = COUNTED UNIT
 ☐ UNCOUNTED
 ☒ POD = COUNTED UNIT (RWOP)

Two more facts about access modes that are easy to state and easy to forget:

A volume can only be mounted using one access mode at a time, even if it supports many. For example, a NFS volume can be mounted as `ReadWriteOnce` on one node and `read-only` on another node at the same time, but not on the same node for both read-write and read-only [source: k8s-docs-persistent-volumes-depth-2026-08-25].

And a limit on what the mode actually enforces: a volume access mode does not enforce write protection. For example, if you provision a `ReadOnlyMany` volume, it does not prevent an application from writing to the mounted volume if the Pod's `securityContext` allows write access [source: k8s-docs-persistent-

volumes-depth-2026-08-25]. `ReadOnlyMany` describes what the storage supports. It is not a permission system [cross-bearing: see Ch 12 §5 — what a Pod may do to its node].

🔑 **Mnemonic: Read the last word as the unit.** `ReadWriteOnce`: once *per node*. `ReadOnlyMany`: many *nodes*. `ReadWriteMany`: many *nodes*. `ReadWriteOncePod`: the unit is in the name, because it is the exception. Three modes leave the unit implicit and one spells it out; the one that spells it out is the one that differs.

Reclaim: what happens after

Delete a `PersistentVolumeClaim` and the storage does not simply vanish, nor does it simply persist. What happens is determined by a policy attached to the `PersistentVolume`, and, critically, that policy was usually chosen by someone else, earlier.

When a user is done with their volume, they can delete the PVC objects from the API that allows reclamation of the resource. The reclaim policy for a `PersistentVolume` tells the cluster what to do with the volume after it has been released of its claim [source: k8s-docs-persistent-volumes-2026-08-23].

Retain. *The `Retain` reclaim policy allows for manual reclamation of the resource. When the `PersistentVolumeClaim` is deleted, the `PersistentVolume` still exists and the volume is considered 'released'. But it is not yet available for another claim because the previous claimant's data remains on the volume* [source: k8s-docs-persistent-volumes-depth-2026-08-25].

The word doing the work there is *manual*. Retain does not mean "kept and reusable." It means "kept and stuck." The administrator's steps are enumerated:

1. Delete the `PersistentVolume`. The associated storage asset in external infrastructure still exists after the PV is deleted.
2. Manually clean up the data on the associated storage asset accordingly.
3. Manually delete the associated storage asset.

[source: k8s-docs-persistent-volumes-depth-2026-08-25]

And if you want the storage back in service: *if you want to reuse the same storage asset, create a new `PersistentVolume` with the same storage asset definition* [source: k8s-docs-persistent-volumes-depth-2026-08-25]. A new `PersistentVolume` object. The released one does not go back to `Available` on its own, ever.

Delete. *For volume plugins that support the `Delete` reclaim policy, deletion removes both the `PersistentVolume` object from Kubernetes, as well as the associated storage asset in the external infrastructure* [source: k8s-docs-persistent-volumes-depth-2026-08-25]. Both. The API object and the actual disk in the actual cloud.

Recycle is deprecated. *The recommended approach is to use dynamic provisioning* [source: k8s-docs-persistent-volumes-depth-2026-08-25]. For completeness: where it is still supported, it *performs a basic*

scrub (rm -rf /thevolume/) on the volume and makes it available again for a new claim* [source: k8s-docs-persistent-volumes-depth-2026-08-25]. It exists on the exam as a name you should recognize and correctly identify as retired.

Where the decision was actually made

Now the part that answers the question this chapter opened with.

Volumes that were dynamically provisioned inherit the reclaim policy of their StorageClass, which defaults to Delete [source: k8s-docs-persistent-volumes-depth-2026-08-25].

And the StorageClass's own default, from the class's side: *if no reclaimPolicy is specified when a StorageClass object is created, it will default to Delete* [source: k8s-docs-storage-classes-2026-08-25].

Follow the chain. A developer writes a PVC. It names no class, so the cluster's default class applies. That class's author did not set `reclaimPolicy`, so it is `Delete`. The volume is provisioned and inherits `Delete`. Some months later the developer deletes the PVC — perhaps while cleaning up a namespace, perhaps as part of tearing down a test environment that turned out to be sharing a class with production — and the disk is destroyed.

Nobody in that story made a decision about the data's survival. The terms of the arrangement were set once, in a StorageClass manifest, by an administrator who was configuring a cluster and not thinking about this specific developer or this specific volume: standing orders written before this cargo, this shipper, or this voyage existed. The documentation says as much, in the mildest possible terms: *the administrator should configure the StorageClass according to users' expectations; otherwise, the PV must be edited or patched after it is created* [source: k8s-docs-persistent-volumes-depth-2026-08-25].

☆ Fixed Point

Dynamically provisioned volumes inherit the StorageClass's reclaim policy, and that policy defaults to Delete.

The decision about whether your data survives the deletion of your claim was made by whoever wrote the StorageClass: before your namespace existed, before your application was written, and without reference to either. If you never looked at your cluster's default StorageClass, you do not know what happens when you delete a PVC.

`kubectl get storageclass` takes two seconds. Run it on a cluster you care about.

⚠ Navigational Hazards: the reclaim cluster

Three closely-related mistakes, all of which produce the same category of outcome: the one where the data is gone.

"Deleting a PVC keeps the data." Only if the reclaim policy says so, and for a dynamically provisioned volume the inherited default is `Delete` [source: k8s-docs-persistent-volumes-depth-2026-08-25]. The safe assumption is the opposite of the intuitive one.

"Retain means the volume is reusable." It means `Released`, not `Available`. *It is not yet available for another claim because the previous claimant's data remains on the volume* [source: k8s-docs-persistent-volumes-depth-2026-08-25]. Reclaiming it takes three manual steps and produces a *new* PV object.

"Recycle will scrub it and hand it back." `Recycle` is deprecated [source: k8s-docs-persistent-volumes-depth-2026-08-25]. Do not plan around it, and recognize it on an exam as the retired option.

The through-line: every one of these is a wrong guess about a default someone else set. That is what makes them dangerous rather than merely wrong. You cannot catch them by reading your own manifest. They were settled in somebody else's.

🔒 **Worth Securing:** Manually created `PersistentVolumes` default to `Retain` [source: k8s-api-ref-persistentvolume-v1-2026-08-25], the opposite of the dynamic default. This is not an inconsistency; it is the API being sensible about intent. If an administrator hand-built a PV pointing at a real NFS export, deleting the API object should not scrub the export. If a provisioner created a volume on demand for one claim, cleaning it up when the claim is gone is the point of having it created on demand. The defaults differ because the situations differ.

Chapter 6's five verbs, settled

Chapter 6 §6 made an unusually blunt deferral. It told you about `StatefulSets` and stable identity, and then said: *you have not been told how that storage is provisioned, requested, sized, reclaimed, or shared. That is deliberate.*

Five verbs. Here they are, closed:

Verb	Where it was answered
provisioned	§3 — statically by an administrator, or dynamically by a provisioner named in a <code>StorageClass</code>
requested	§2 — by a <code>PersistentVolumeClaim</code> , which is a request for storage by a user
sized	§2 — the claim requests a capacity; binding matches it against a PV that satisfies it
reclaimed	§4 — by the reclaim policy, <code>Retain</code> or <code>Delete</code> , inherited from the class for dynamic volumes
shared	§4 — by the access mode, which states how many nodes (or, for RWOP, how many Pods) may mount it

Chapter 6 was right that it was deliberate. Every one of those verbs needed the supply/demand split before it could be answered honestly, and that split is the whole content of §2. Explaining "reclaimed" in Chapter 6 would have required explaining StorageClasses, which would have required explaining provisioning, which would have required the PV/PVC distinction: an entire chapter, arriving five chapters early, in the middle of a discussion about workload controllers.

What Chapter 6 kept back for itself, and still owns, is **identity** [cross-bearing: see Ch 6 §6 — when Pods are not interchangeable]. §6 of this chapter is where the two halves meet.

🕒 Taking Your Bearings #2

⚠️ **These questions are intentionally challenging.** Struggle is the point here. Four of the exam traps this chapter's domain analysis identified are tested in this checkpoint alone, and that concentration is not accidental. It is where the material is genuinely counter-intuitive.

Two of the four turn on a decision made by someone who was gone before you came aboard: a reclaim policy inherited from a class nobody named, set by an administrator who is not going to be consulted when the claim is deleted. That is the shape of most storage surprises.

If you find these easy, you have genuinely mastered the highest-yield section of the chapter. If you struggle, you are encoding deeply, and §5 immediately afterward is a straightforward win.

1. A PersistentVolume is bound with access mode `ReadWriteOnce`. Three Pods are scheduled: `pod-a` and `pod-b` on node-1, and `pod-c` on node-2. All three reference the same PVC. Which Pods can mount the volume read-write?

A) Only `pod-a` — RWO means exactly one Pod B) `pod-a` and `pod-b` — RWO permits multiple Pods on the same node C) All three — RWO restricts writes but not mounts D) None — RWO is invalid for multi-Pod configurations and the PVC will fail to bind

2. A team's cluster has a single StorageClass marked default, whose manifest specifies no `reclaimPolicy`. A developer creates a PVC with no `storageClassName`, gets a bound 50Gi volume, runs a database on it for six months, then deletes the PVC while decommissioning the namespace. What happens to the data, and where was that decided?

A) Data is retained; decided by the PVC, which defaults to `Retain` B) Data is retained; decided by the PersistentVolume, since manually created PVs default to `Retain` C) Data is destroyed; decided by the StorageClass, whose unspecified `reclaimPolicy` defaults to `Delete` and is inherited by the dynamically provisioned volume D) Data is destroyed; decided at deletion time by the user, since deleting a PVC always deletes its backing storage

3. [retrieval: ch7] A StorageClass sets `volumeBindingMode: WaitForFirstConsumer`. A developer creates a PVC using that class, then creates a Pod that references it. The Pod requests 64Gi of memory and no

node in the cluster has that much allocatable. What is the state of the PVC and the Pod?

A) PVC Bound, Pod Pending — the volume provisions immediately and the Pod waits for a node B) PVC Pending, Pod Pending — binding waits for the scheduler to pick a node, and the scheduler never does C) PVC Bound, Pod Pending — the volume provisions in the zone with the most free capacity, and the Pod waits for a node there D) PVC Failed, Pod Failed — a claim that cannot be satisfied is marked failed, and a Pod whose volume failed is marked failed with it

4. An administrator wants to be certain that a PVC's data survives when the PVC is deleted, and that the storage can be handed to a different application afterward with the old data cleaned off. The volume was dynamically provisioned from a class with `reclaimPolicy: Retain`. Which sequence is correct after the PVC is deleted?

A) The PV returns to `Available` and the next matching claim binds to it, inheriting the previous data B) The PV becomes `Released`; the admin deletes the PV, cleans the storage asset, deletes the asset, and creates a new PV if the asset is to be reused C) The PV becomes `Released` and Kubernetes scrubs it automatically before returning it to `Available` D) The PV becomes `Failed` because `Retain` cannot complete automatic reclamation

5. A developer writes a PVC with `storageClassName: ""`, on a cluster that has one default `StorageClass` backed by a working CSI provisioner and no classless `PersistentVolumes`. What happens?

A) The default `StorageClass` is used, since an empty string means "unspecified" B) The PVC is rejected by the API server as invalid C) The PVC is created, dynamic provisioning is disabled for it, and it remains unbound because no PV without a class exists D) The PVC binds to the first `Available` PV of any class, since an empty class matches all classes

Answers with Explanations:

1 — **B.** *ReadWriteOnce access mode still can allow multiple Pods to access the volume when the Pods are running on the same node* [source: [k8s-docs-persistent-volumes-depth-2026-08-25](#)]. `pod-a` and `pod-b` share `node-1` and can share the volume. `pod-c` is on a different node and cannot.

- **A is wrong**, and it is *the* access-mode trap. RWO counts nodes. The mode that means exactly one Pod is `ReadWriteOncePod`, and its existence is the proof that RWO does not mean that [source: [k8s-docs-persistent-volumes-depth-2026-08-25](#)].
- **C is wrong**: the "Once" is a real constraint on nodes, and `pod-c` on `node-2` is blocked. Its Pod will not start.
- **D is wrong**: RWO is entirely valid and extremely common; nothing about it prevents binding.

2 — **C.** Follow the chain: no `storageClassName` → the default class applies → that class specifies no `reclaimPolicy` → *if no `reclaimPolicy` is specified when a `StorageClass` object is created, it will default to `Delete`* [source: [k8s-docs-storage-classes-2026-08-25](#)] → *volumes that were dynamically provisioned*

inherit the reclaim policy of their StorageClass [source: k8s-docs-persistent-volumes-depth-2026-08-25]. Deleting the claim destroys the PV object and the backing asset.

- **A is wrong:** a PVC has no reclaim policy of its own to default. Reclaim policy lives on the PV.
- **B is wrong** in a subtle and instructive way. *Retain* is the default for manually created PVs [source: k8s-api-ref-persistentvolume-v1-2026-08-25], but this volume was dynamically provisioned, and the dynamic path has the opposite default. Choosing B means you knew a real fact and applied it to the wrong provisioning path.
- **D is wrong:** deleting a PVC does not *always* delete the storage. It does what the reclaim policy says. Getting the outcome right for the wrong reason is still getting it wrong, and the exam will offer you that option.

3 — B. *WaitForFirstConsumer* will delay the binding and provisioning of a *PersistentVolume* until a *Pod* using the *PersistentVolumeClaim* is created, and the volume is then selected or provisioned conforming to the topology that is specified by the *Pod*'s scheduling constraints [source: k8s-docs-storage-classes-2026-08-25]. The *Pod* is unschedulable, because no node satisfies its memory request during the filter phase [cross-bearing: see Ch 7 §2 — what makes a node feasible], so the scheduler never picks a node, so binding never proceeds. Both objects sit *Pending*.

- **A is wrong:** that is *Immediate* mode's behavior, which is precisely what *WaitForFirstConsumer* exists to avoid.
- **C is wrong**, and it is the most reasonable of the three, because it assumes the provisioner has some sensible policy of its own — free capacity, spare headroom, whichever zone looks emptiest. It does not. The mode provisions to the topology the *Pod*'s scheduling constraints specify, and there are no such constraints to conform to until the scheduler has chosen a node [source: k8s-docs-storage-classes-2026-08-25]. Zone capacity is not an input here. The *Pod*'s node is the input, and there is not going to be one.
- **D is wrong.** *Failed* is a *PersistentVolume* phase, describing a volume whose automatic reclamation did not succeed [source: k8s-docs-persistent-volumes-depth-2026-08-25]: a storage-lifecycle outcome, not a scheduling one. Nothing here has been reclaimed, or attempted. A *Pod* that no node can hold waits in *Pending*; it is not marked failed for waiting, and neither is its claim.

4 — B. Chapter 4's Fixed Point named *Nodes*, *PersistentVolumes*, and *StorageClasses* as cluster-scoped, alongside namespace objects themselves [cross-bearing: see Ch 4 §3 — where a name lives]. This chapter supplied the reason for two of them: a PV is supply the cluster owns, and a *StorageClass* is a template the cluster offers. Neither belongs to a tenant, so neither belongs to a namespace.

- **A and C** both include the *PersistentVolumeClaim*, and that is the half Chapter 4 never taught. The claim is namespaced — it is the tenant's request, and *claims must exist in the same namespace as the Pod using the claim* [source: k8s-docs-persistent-volumes-depth-2026-08-25]. **C** is the more reasonable of the two: having correctly placed the PV at cluster scope, it assumes the claim follows its supply. It does not.
- **D** swaps the *Node* for the *Namespace* object. *Namespace* objects genuinely are cluster-scoped — they are the example Chapter 4 said people most often miss, since a namespace is not itself in a

namespace — so the trap is a true statement sitting beside a false one. The set still fails on the claim.

5 — C. A PVC with its `storageClassName` set to "" is explicitly stating that it is bound with a PV with no class, hence the PV's `storageClassName` must also be empty [source: k8s-docs-persistent-volumes-depth-2026-08-25], and claims that request the class "" effectively disable dynamic provisioning for themselves [source: k8s-docs-persistent-volumes-2026-08-23]. No classless PVs exist, so nothing matches, and nothing will be provisioned.

- **A is wrong** and is the whole point of the question. Omitting the field and setting it to "" are different acts with different meanings. Only the omission gets the default.
- **B is wrong:** the empty string is entirely valid API input with a defined meaning.
- **D is wrong:** a claim requesting class "" binds only to PVs whose class is also empty. It matches nothing else.

If you scored 0–2: This is the hard checkpoint and a low score here is common. Re-read §4 in full, about fourteen minutes, with particular attention to the two ☆ Fixed Points and the ⚠ Navigational Hazards block. If only question 3 went wrong, the gap is §3's `WaitForFirstConsumer` subsection rather than §4.

If you scored 3–4: Good. This checkpoint was built to be hard. Work out *why* the ones you missed were wrong before continuing; the wrong answers here are all real misconceptions, not filler.

If you scored 5: You have the highest-yield material in the chapter and you have it cold. §5 is short and §6 is a payoff.

Checkpoint: You've Now Mastered ✓ Access modes as node-count semantics, and RWOP as the one exception ✓ The three reclaim policies, including which one is retired ✓ That dynamically provisioned volumes inherit `Delete` by default, from the class ✓ That `Released` requires three manual steps and a new PV object to become reusable ✓ Chapter 6's five deferred verbs, all five closed

One question left, and it is the one that has been implicit since §3: when a provisioner creates a volume, what is actually doing the creating? Something below the waterline has been doing that work for four sections now, and you have not been introduced to it yet.

..1 §5 — Who Actually Provides the Storage

Chapter 10 closed by telling you that this chapter brings the last of the four pluggable interfaces, and that with it the set closes. Here it is.

You have three already, collected one chapter at a time. CRI at the container runtime [*cross-bearing: see Ch 2 §4 — the Container Runtime Interface*]. CRDs at the API itself [*cross-bearing: see Ch 6 §8 — the control*]

loop, extended]. CNI at the network [*cross-bearing: see Ch 9 §1 — four rules and a plugin*]. The fourth is CSI, at storage.

The Container Storage Interface (CSI) defines a standard interface for container orchestration systems (like Kubernetes) to expose arbitrary storage systems to their container workloads [source: k8s-docs-volumes-csi-and-subpath-2026-08-25].

The glossary states the consequence: *CSI allows vendors to create custom storage plugins for Kubernetes without adding them to the Kubernetes repository (out-of-tree plugins)* [source: k8s-glossary-storage-terms-2026-08-25]. And from the volumes page, the same claim in the sharpest available form: *vendors can implement `csi` volumes to introduce new storage systems into Kubernetes without ever having to edit the core Kubernetes code* [source: k8s-docs-volumes-2026-08-23].

Without ever having to edit the core Kubernetes code. If that phrasing feels familiar, it should. It is structurally identical to what CRI bought at the runtime boundary and what CNI bought at the network boundary. Same move, fourth socket.

Why it exists: the world before

The reason CSI was necessary takes one paragraph, and it makes the argument concrete rather than architectural.

Previously, all volume plugins were "in-tree". The "in-tree" plugins were built, linked, compiled, and shipped with the core Kubernetes binaries. This meant that adding a new storage system to Kubernetes (a volume plugin) required checking code into the core Kubernetes code repository [source: k8s-docs-volumes-csi-and-subpath-2026-08-25].

Sit with the consequences of that. The documentation records the arrangement, not what it cost the people living inside it, so what follows is this book's reading rather than a sourced claim. A storage vendor wanting Kubernetes support had to submit code to the Kubernetes project, have it reviewed by Kubernetes maintainers, and wait for a Kubernetes release. Their bug fixes shipped on Kubernetes' schedule, not their own. Meanwhile, Kubernetes maintainers were reviewing and carrying storage code for hardware they did not own and could not test.

Both CSI and FlexVolume allow volume plugins to be developed independently of the Kubernetes code base, and deployed (installed) on Kubernetes clusters as extensions [source: k8s-docs-volumes-csi-and-subpath-2026-08-25]. FlexVolume was the earlier attempt and is now deprecated: *using an out-of-tree CSI driver is the recommended way to integrate external storage with Kubernetes* [source: k8s-docs-volumes-csi-and-subpath-2026-08-25].

☆ Fixed Point

CSI is a published contract between two parties, and one of them is not Kubernetes.

The specification states its own purpose in a sentence that says more than any Kubernetes documentation can: CSI exists to *"define an industry standard 'Container Storage Interface' (CSI) that will enable storage vendors (SP) to develop a plugin once and have it work across a number of container orchestration (CO) systems"* [source: csi-spec-objective-2026-08-25].

Read *"across a number of container orchestration systems."* CSI is not a Kubernetes feature that vendors happen to use. It is a cross-orchestrator standard that Kubernetes happens to implement, the same shape as OCI at the image and runtime boundary [*cross-bearing: see Ch 2 §5 — the Open Container Initiative*]. Storage stops being Kubernetes' problem and becomes a vendor's, on terms both parties agreed to in public.

With this you hold all four: **CRI, CNI, CSI, CRDs**. Chapter 17 will ask you to state the shape they have in common without help [*cross-bearing: see Ch 17 §4 — every place Kubernetes lets you in*].

What a driver is, and what installing one puts in the cluster

Chapter 2 promised you "CSI and storage drivers," with *drivers* in the promise. So: what is a driver, concretely?

A CSI driver is software you deploy into the cluster, and it deploys as ordinary Kubernetes workloads. A *CSI driver is typically deployed in Kubernetes as two components: a controller component and a per-node component* [source: kubernetes-csi-docs-deployment-2026-08-25]. The controller component *can be deployed as a Deployment or StatefulSet on any node in the cluster*, and the node component *should be deployed on every node in the cluster through a DaemonSet* [source: kubernetes-csi-docs-deployment-2026-08-25].

That shape should be recognizable without further explanation. One controller, running somewhere, handling the cluster-wide operations: create this volume, delete that one [*cross-bearing: see Ch 6 §1 — the resource that holds the intent*]. One agent per node, running everywhere, handling the node-local operations: mount this volume into this path on this machine. DaemonSet is exactly the workload type for "one per node" [*cross-bearing: see Ch 6 §7 — one per node, and work that ends*]. A CSI driver is not exotic infrastructure. It is a Deployment and a DaemonSet, running software somebody else wrote.

Installing one also puts an object in the cluster: *CSIDriver captures information about a Container Storage Interface (CSI) volume driver deployed on the cluster* [source: k8s-api-ref-csidriver-v1-2026-08-25]. Which makes it the fourth object in this chapter that describes storage without providing any — PV, PVC, StorageClass, CSIDriver — and the one where the gap is most literal, since its whole documented job is to *capture information about a driver that exists elsewhere*.

Once installed, the driver's volumes are usable in three ways: *through a reference to a PersistentVolumeClaim, with a generic ephemeral volume, or with a CSI ephemeral volume if the driver*

supports that [source: k8s-docs-volumes-csi-and-subpath-2026-08-25]. The PersistentVolumeClaim path is the one §2 through §4 described.

The fourth sighting

You have now seen the pattern three times in three sections: an Ingress without a controller, a StorageClass without a provisioner, and now this. Here is the fourth, and the sources state it not as a warning but as a plain ordering requirement:

To use a CSI driver from a storage provider, you must first deploy it to your cluster. You will then be able to create a Storage Class that uses that CSI driver [source: k8s-glossary-storage-terms-2026-08-25].

*First. Then. And from the migration documentation, in case the point needed to be blunter: as part of that migration, you — or another cluster administrator — must have installed and configured the appropriate CSI driver for that storage. **The core of Kubernetes does not install that software for you.*** [source: k8s-docs-volumes-csi-and-subpath-2026-08-25]

An object without its component does nothing [cross-bearing: see Ch 10 §3 — the object is not the implementation]. A StorageClass naming a CSI driver nobody deployed is not an exotic failure you have to imagine. It is the documented prerequisite, unmet. The claim sits unbound. The Pod sits Pending. `kubect1 get storageclass` shows the class, healthy and correct, describing a capability that does not exist.

That is the same light, the fourth time. And once you have seen it four times, you stop asking "is the object there?" and start asking "is the *component* there?", which is the question Chapter 10 said you would know to ask.

🔍 Closer Look: CSI migration

CNCF publishes the Storage competency as a single word. This book's reading of it puts CSI at recall depth: name the storage interface, say what it is for, and stop there. What follows goes deeper than that reading requires, and is here for the day you meet it in a cluster rather than on a test.

The in-tree plugins did not vanish when CSI arrived; they were migrated behind it. *The CSIMigration feature directs operations against existing in-tree plugins to corresponding CSI plugins (which are expected to be installed and configured). As a result, operators do not have to make any configuration changes to existing Storage Classes, PersistentVolumes, or PersistentVolumeClaims (referring to in-tree plugins) when transitioning to a CSI driver that supersedes an in-tree plugin* [source: k8s-docs-volumes-csi-and-subpath-2026-08-25].

The compatibility promise is unusually strong: *existing PVs created by an in-tree volume plugin can still be used in the future without any configuration changes, even after the migration to CSI is completed for that volume type, and even after you upgrade to a version of Kubernetes that doesn't have compiled-in support for that kind of storage* [source: k8s-docs-volumes-csi-and-subpath-2026-08-25]. Your manifests keep working; the machinery underneath them was replaced. *The actual storage management now happens through the CSI driver* [source: k8s-docs-volumes-csi-and-subpath-2026-08-25].

The operations CSI covers: *provisioning/delete, attach/detach, mount/unmount, and resizing of volumes* [source: k8s-docs-volumes-csi-and-subpath-2026-08-25]. That list is the storage lifecycle, and it maps cleanly onto everything §2 through §4 described.

📌 **Snag:** CSI is an interface, not a product. There is no "CSI storage" you can buy or deploy, any more than there is "CNI networking." What you deploy is a *driver*, written by whoever owns the storage it speaks to — one for a cloud provider's block store, one for a software-defined storage project, one for an on-premises array — each implementing the same contract against different hardware. A question that treats CSI as a storage system rather than as the interface storage systems implement is testing exactly this confusion.

..1 §6 — Pods That Are Not Interchangeable, Revisited

In Chapter 6, you were told that this explanation was incomplete. Not implied. Told, in as many words, with the deferred material enumerated and the deferral labeled deliberate.

This is the missing half, arriving on schedule.

Chapter 6 §6 gave you StatefulSet identity: Pods with stable ordinals, `web-0` and `web-1` and `web-2`, names that stick across rescheduling rather than being regenerated with each replacement [*cross-bearing: see Ch 6 §6 — when Pods are not interchangeable*]. Chapter 9 gave you the network half of that identity: the headless Service [*cross-bearing: see Ch 9 §5 — when you don't want a single address*] and the per-Pod

DNS names it produces *[cross-bearing: see Ch 9 §7 — names, and where they resolve]*. What was missing both times was storage, and storage needed the whole of §2 through §4 before it could be explained without hand-waving.

One claim per Pod, from a template

The mechanism is a field on the StatefulSet: `volumeClaimTemplates`. It looks like this, in the documentation's own nginx example:

```
volumeClaimTemplates:
- metadata:
  name: www
  spec:
    accessModes: [ "ReadWriteOnce" ]
    storageClassName: "my-storage-class"
    resources:
      requests:
        storage: 1Gi
```

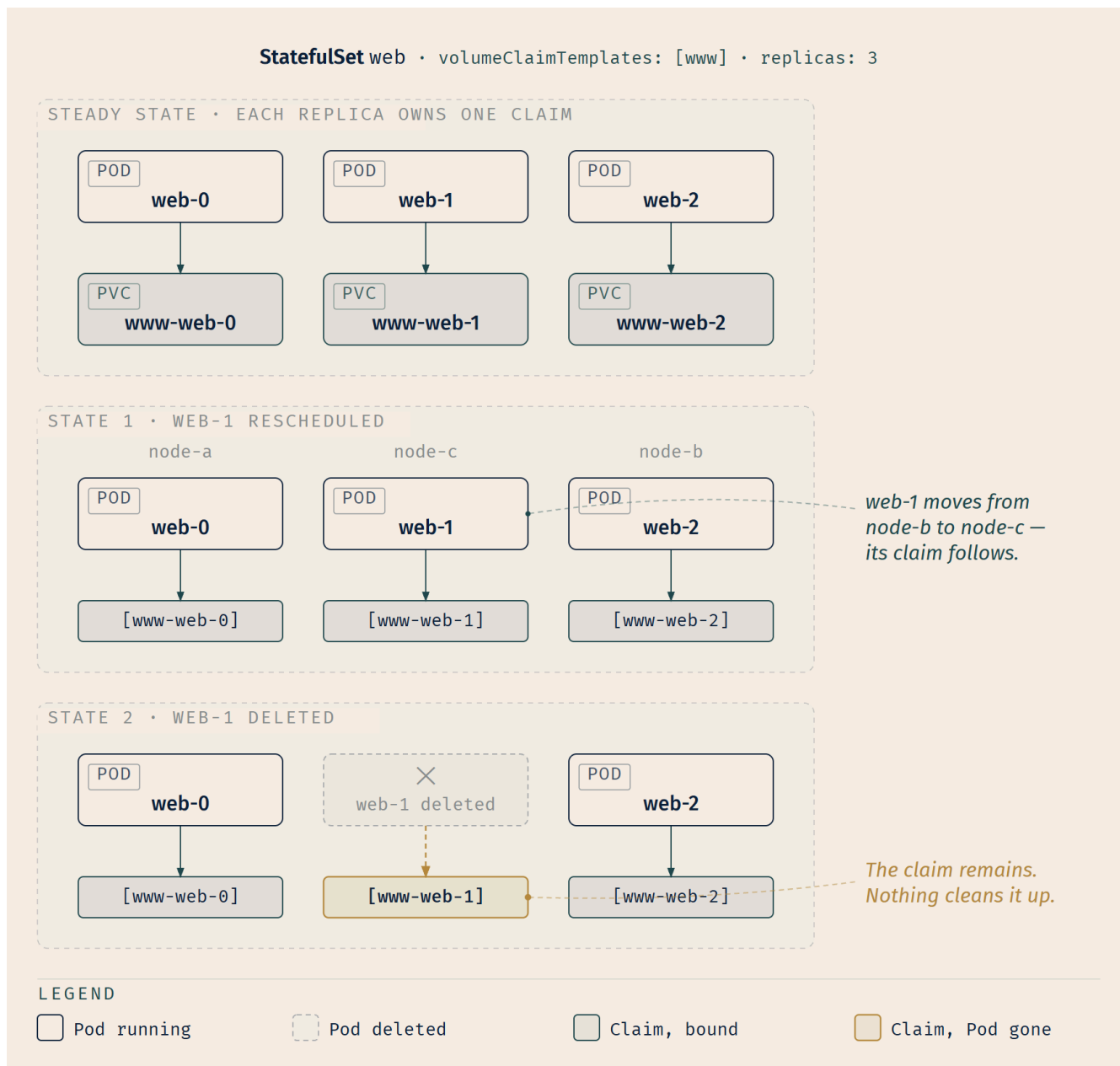
[source: k8s-docs-statefulset-storage-2026-08-25]

That is a `PersistentVolumeClaim` spec — the same fields §2 taught you, in the same shapes — sitting inside a workload controller as a template rather than as an object. And the rule for what the controller does with it is one sentence:

For each `VolumeClaimTemplate` entry defined in a `StatefulSet`, each Pod receives one `PersistentVolumeClaim` [source: k8s-docs-statefulset-storage-2026-08-25].

Not one claim for the set. One claim *per Pod*, minted from the template as each Pod is created. A three-replica StatefulSet named `web` with a template named `www` yields three claims, and the naming follows the same ordinal identity Chapter 6 taught, so you can read a cluster's storage from its Pod names and vice versa.

Where those claims get their storage is §3, arriving as a consequence rather than as a rule: *the storage for a given Pod must either be provisioned by a PersistentVolume Provisioner based on the requested storage class, or pre-provisioned by an admin* [source: k8s-docs-statefulset-2026-08-24]. Dynamic or static. The two paths from §3, with nothing special added for StatefulSets.



The reschedule, which is the under-weighted half

Chapter 10 promised you two things about per-replica claims: that they outlive the Pod, and that they outlive *the rescheduling*. Most readers weight the first and skim the second. The second is the more interesting one.

A StatefulSet's Pod keeps the same PersistentVolumeClaim for the whole of its lifecycle, and when that Pod is rescheduled onto a different node, its `volumeMounts` mount that same claim wherever it lands [source: k8s-docs-statefulset-2026-08-24].

Note what that does *not* say. It does not say the storage moves. It does not say the data is copied. It says the mount happens wherever the Pod lands, because the Pod's storage is attached to *the claim*, and the

claim is not attached to a node at all. The claim is a namespaced API object with no node in it, lashed to no particular deck. The volume behind it is cluster-scoped. Neither one cares which machine `web-1` is running on today.

That is why a `StatefulSet` can survive a node failure with its data intact [source: [k8s-docs-statefulset-storage-2026-08-25](#)], and stating it explicitly explains something otherwise mysterious: the identity you learned in Chapter 6 is not just a naming convention. `web-1` is a name that carries a claim with it. Reschedule the Pod and the claim comes along, because the claim was never anywhere else.

🔒 **Worth Securing:** This also explains the constraint you may have wondered about back in Chapter 6: why `StatefulSets` recreate a Pod with the *same* ordinal rather than just adding a new one. A Deployment's replacement Pod is a new Pod with a new name and no history. A `StatefulSet`'s replacement for `web-1` must be *named* `web-1`, because the name is how it finds `www-web-1`. Identity and storage are one mechanism seen from two sides.

The claims survive the workload

Now the part that costs people real money.

☆ Fixed Point

A `StatefulSet`'s `PersistentVolumeClaims` are not deleted when the Pod is deleted, or when the `StatefulSet` is deleted. This must be done manually.

The claims are governed by a retention policy, and that policy's default is to keep them: *Retain (default): PVCs from the `volumeClaimTemplate` are not affected when their Pod is deleted* [source: [k8s-docs-statefulset-storage-2026-08-25](#)], and *the default for policies is Retain* [source: [k8s-docs-statefulset-storage-2026-08-25](#)].

The volumes behind those claims persist alongside them: *the `PersistentVolumes` associated with the Pods' `PersistentVolume Claims` are not deleted when the Pods, or `StatefulSet` are deleted. This must be done manually* [source: [k8s-docs-statefulset-storage-2026-08-25](#)]. Read that second sentence carefully and notice which object it is about — the `PersistentVolumes`, not the claims. Both survive, for different reasons stated in different places. The claims are the ones you will find still sitting in the namespace afterward.

`kubectl delete statefulset web` removes the workload and leaves `www-web-0`, `www-web-1`, and `www-web-2` sitting in the namespace, bound, billing, and invisible to anyone who thinks deleting a workload cleans up after itself.

The reason is stated as a deliberate choice rather than an oversight: *deleting and/or scaling a `StatefulSet` down will not delete the volumes associated with the `StatefulSet`. This is done to ensure data safety, which is generally more valuable than an automatic purge of all related `StatefulSet` resources* [source: [k8s-docs-statefulset-2026-08-24](#)].

Read that as a judgment call, because that is what it is. Kubernetes chose the failure mode where the hold stays full of cargo nobody remembers loading over the failure mode where one command puts a database over the side. Given the two, it chose correctly. But it means the cleanup is yours, and nobody will remind you.

🔍 Closer Look: the retention policy field

That default is configurable. *The optional `.spec.persistentVolumeClaimRetentionPolicy` field controls if and how PVCs are deleted during the lifecycle of a StatefulSet* [source: k8s-docs-statefulset-storage-2026-08-25], with two independently settable policies: `whenDeleted`, which configures the volume retention behavior that applies when the StatefulSet is deleted, and `whenScaled`, which configures the volume retention behavior that applies when the replica count of the StatefulSet is reduced [source: k8s-docs-statefulset-storage-2026-08-25]. Each takes `Delete` or `Retain`, and `Retain` is the default for both [source: k8s-docs-statefulset-storage-2026-08-25].

One boundary to know: these policies apply only to *voluntary* removal. *If a Pod associated with a StatefulSet fails due to node failure, and the control plane creates a replacement Pod, the StatefulSet retains the existing PVC. The existing volume is unaffected, and the cluster will attach it to the node where the new Pod is about to launch* [source: k8s-docs-statefulset-storage-2026-08-25]. A node dying is not a scale-down. Your data does not get cleaned up because a machine crashed.

📌 **Snag:** There is a mechanism elsewhere in this chapter that does the *opposite*, and the contrast is worth holding. A **generic ephemeral volume** also creates a PVC per Pod, and *when the Pod is deleted, the Kubernetes garbage collector deletes the PVC, which then usually triggers deletion of the volume because the default reclaim policy of storage classes is to delete volumes* [source: k8s-docs-ephemeral-volumes-2026-08-25]. Two mechanisms, both creating one claim per Pod, with exactly opposite deletion behavior. The difference is intent: an ephemeral volume is scratch space that happens to be provisioned like real storage, and a StatefulSet's claim is real storage that happens to be created by a controller.

🕒 Taking Your Bearings #3

Five questions on CSI, the driver behind it, and the StatefulSet pairing.

1. Which statement most accurately describes what CSI is?

- A) A storage system maintained by the CNCF that Kubernetes clusters can deploy for persistent volumes
- B) A standard interface allowing storage vendors to write one plugin that works across container orchestration systems, without editing core Kubernetes code
- C) A Kubernetes-internal API used by the kubelet to mount volumes, not exposed to external vendors
- D) The successor to StorageClass, replacing dynamic provisioning with vendor-managed volumes

2. An administrator creates a StorageClass whose provisioner field names `blockstore.example.com`. The corresponding CSI driver has never been deployed to the cluster. A developer creates a PVC requesting that class and a Pod that mounts it. What is observed?

A) The API server rejects the StorageClass at creation time, since the provisioner does not exist B) The PVC binds to an existing Available PersistentVolume of a different class, since the requested class cannot be satisfied C) The StorageClass exists and looks correct, the PVC remains unbound, and the Pod stays Pending D) Kubernetes automatically installs the named CSI driver on first use

3. [retrieval: ch6] You run `kubectl get statefulset` in a namespace and get "No resources found." You then run `kubectl get pvc` in the same namespace and see `www-web-0`, `www-web-1`, and `www-web-2`, all Bound. What is the most likely explanation, and what does it tell you about identity and storage?

A) The claims are orphaned by a bug; StatefulSet deletion is supposed to remove them B) A StatefulSet named `web` was deleted; its per-replica PVCs survive by design and must be removed manually C) The claims belong to a Deployment, since Deployments also generate per-replica claims D) The StatefulSet is in a different namespace; PVCs are cluster-scoped and appear everywhere

4. A StatefulSet has three replicas and one `volumeClaimTemplate`. `web-1` is running on `node-b`. `node-b` fails and the control plane schedules the replacement `web-1` onto `node-e`. What happens to `web-1`'s storage?

A) A new PVC is created for the replacement Pod on `node-e`, and the old data is lost with the failed node B) The existing PVC is retained, and the cluster attaches the existing volume to the node where the new Pod launches C) The data is copied from `node-b` to `node-e` by the StatefulSet controller before the Pod starts D) The Pod cannot be rescheduled, because a StatefulSet's storage is pinned to its original node

5. A StatefulSet named `cache` declares three replicas and two entries in `volumeClaimTemplates`, named `data` and `wal`. Once all three Pods are running, how many PersistentVolumeClaims exist in the namespace, and what are they named?

A) Three — `cache-0`, `cache-1`, and `cache-2`; each Pod receives one claim regardless of how many templates the set declares B) Six — `data-cache-0` through `data-cache-2`, and `wal-cache-0` through `wal-cache-2` C) Two — `data-cache` and `wal-cache`; each template produces one claim, which all three Pods mount D) Six — `cache-data-0` through `cache-data-2`, and `cache-wal-0` through `cache-wal-2`

Answers with Explanations:

1 — **B**. The specification's own objective: to *"enable storage vendors (SP) to develop a plugin once and have it work across a number of container orchestration (CO) systems"* [source: `csi-spec-objective-2026-08-25`], and vendors can introduce new storage systems *without ever having to edit the core Kubernetes code* [source: `k8s-docs-volumes-2026-08-23`].

- **A is wrong** and is the most common CSI misconception: treating an interface as a product. There is no "CSI storage." There are CSI *drivers*, one per storage system.

- **C is wrong** in the crucial respect: CSI is explicitly out-of-tree and vendor-facing. That is its entire purpose.
- **D is wrong:** CSI and StorageClass are complementary. A StorageClass names a provisioner; a CSI driver is what that provisioner often is.

2 — C. To use a CSI driver from a storage provider, you must first deploy it to your cluster [source: k8s-glossary-storage-terms-2026-08-25], and the core of Kubernetes does not install that software for you [source: k8s-docs-volumes-csi-and-subpath-2026-08-25]. The object exists; the component does not; nothing happens.

- **A is wrong:** the API server does not validate that a provisioner is running. The StorageClass is a perfectly valid object.
- **B is wrong,** and it is the one to linger on, because it is the reasonable answer rather than the careless one. Binding is not best-effort matching. A claim that names a class is filtered against that class, and the filter is a requirement, not a preference — alongside capacity, access mode, label selector, and `volumeName`, all ANDed together [source: k8s-docs-persistent-volumes-depth-2026-08-25]. A claim that no volume satisfies stays unbound indefinitely rather than settling for a near miss [source: k8s-docs-persistent-volumes-2026-08-23].
- **D is wrong,** and stating why matters more than the right answer: nothing in Kubernetes installs drivers. This is the fourth instance of the pattern named at Ch 10 §3.

3 — B. The citation that speaks about *claims* is the retention policy. A StatefulSet's `persistentVolumeClaimRetentionPolicy` has two settings — `whenDeleted`, for deletion of the StatefulSet itself, and `whenScaled`, for a scale-down — and the default for policies is *Retain* [source: k8s-docs-statefulset-storage-2026-08-25]. Unless somebody changed it, deleting `web` left `www-web-0`, `www-web-1`, and `www-web-2` exactly where they were. The documentation adds that the PersistentVolumes behind those claims are not deleted either, and that removing them is manual [source: k8s-docs-statefulset-storage-2026-08-25]. The naming, `<template>-<statefulset>-<ordinal>`, tells you the set was called `web` and had a template called `www`. What it says about identity and storage: the claim is attached to the *name*, not to the workload object, which is why deleting the workload does not disturb it.

- **A is wrong:** this is documented, deliberate behavior, chosen because *data safety... is generally more valuable than an automatic purge* [source: k8s-docs-statefulset-2026-08-24].
- **C is wrong:** Deployments have no `volumeClaimTemplates` field. A Deployment's replicas share whatever claim the PodSpec names, if any.
- **D is wrong** twice over: PVCs are namespaced, not cluster-scoped [*cross-bearing: see Ch 4 §3 — where a name lives*], and `kubectl get` was run in the namespace.

4 — B. If a Pod associated with a StatefulSet fails due to node failure, and the control plane creates a replacement Pod, the StatefulSet retains the existing PVC. The existing volume is unaffected, and the cluster will attach it to the node where the new Pod is about to launch [source: k8s-docs-statefulset-storage-2026-08-25].

- **A is wrong:** the replacement `web-1` reuses `www-web-1`, which is the entire point of stable identity.

- **C is wrong:** nothing is copied. The volume is cluster-scoped storage attached at mount time; it was never *on node-b* in the sense this answer implies.
- **D is wrong:** this inverts the mechanism. The claim's independence from any node is what *permits* the reschedule.

5 — B. For each `VolumeClaimTemplate` entry defined in a `StatefulSet`, each Pod receives one `PersistentVolumeClaim` [source: k8s-docs-statefulset-storage-2026-08-25]. Three Pods against two templates is six claims, and each name is assembled `<template>-<statefulset>-<ordinal>` — template first.

- **A is wrong**, and it is the most reasonable wrong answer here: it remembers "each Pod receives one `PersistentVolumeClaim`" and drops the clause standing in front of it. The rule is one claim per Pod *per template*.
- **C is wrong**, and it is Deployment-shaped thinking. A single shared claim is what a Deployment's replicas get when the `PodSpec` names a PVC directly. `volumeClaimTemplates` exists precisely because that is not what a `StatefulSet` wants [cross-bearing: see Ch 6 §6 — *when Pods are not interchangeable*].
- **D is wrong** on ordering alone, which is what makes it dangerous. It gets the count right and the components right, and still names six claims that do not exist. `cache-data-0` looks entirely plausible and matches nothing.

If you scored 0–2: Re-read §5 and §6, about eighteen minutes. If the misses clustered in questions 3 through 5, the gap is §6 alone, and it repays a re-reading with one question held in mind: what is the claim actually attached to? Every answer in this checkpoint follows from that.

If you scored 3–4: Good. You have the interface pattern and the `StatefulSet` pairing.

If you scored 5: You have closed the book's one deliberate forward reference, and with §5 behind you, you hold all four interfaces. Chapter 17 will be a collection exercise rather than a learning one.

Checkpoint: You've Now Mastered ✓ CSI as the fourth pluggable interface, and as a cross-orchestrator standard rather than a Kubernetes feature ✓ What a CSI driver is — a Deployment plus a `DaemonSet`, written by somebody outside the project ✓ `volumeClaimTemplates`, the one-claim-per-Pod-per-template rule, and the `<template>-<set>-<ordinal>` name it produces ✓ Why a `StatefulSet`'s storage follows a reschedule, and why identity is the mechanism that makes it possible ✓ That the claims outlive the workload, deliberately, and that cleanup is yours

☀ §7 — Outliving the Pod That Asked

Look back at what you did in this chapter, and notice that you only ever asked one question.

Does the file survive the container restart? Does the volume survive the Pod? Does the claim survive the workload? Does the data survive the claim? Four questions, one shape: **what survives what?** Every section widened the scope by exactly one step, and the ladder in §1 was the whole chapter drawn small.

Here is what makes it worth the walk. At every rung, the thing that would seem to have the most at stake in the answer is not the thing that decides it.

The container does not decide whether its files survive a restart. The image layout decided that, before the container existed. The Pod does not decide whether its volume survives deletion; the volume's *type* decided it, chosen in a manifest by whoever wrote the PodSpec. And the claim does not decide whether the data survives the claim's deletion. The StorageClass decided that, in a `reclaimPolicy` field, in a manifest written by an administrator who was configuring a cluster and had never heard of your application.

The workload never gets a vote. That is not a defect. It is the same separation that runs through everything you have learned:

A Pod does not decide which node it lands on; the scheduler does, by filtering and scoring [*cross-bearing: see Ch 7 §1 — one decision, made once*]. A Deployment does not decide how many replicas exist right now; a control loop does, reconciling toward the number in the spec [*cross-bearing: see Ch 3 §6 — controllers and the control loop*]. And an object does not decide whether anything happens to it; the component that watches for it does, if somebody installed one [*cross-bearing: see Ch 10 §3 — the object is not the implementation*].

Storage is that pattern applied to durability. And the reason for naming it is practical rather than aesthetic: it is why a developer can be handed a cluster they have never seen and be productive on it the same afternoon. They write a claim. Somebody else — possibly years ago, possibly a vendor they will never meet, possibly a CSI driver running as a DaemonSet on nodes they will never log into — arranged for that claim to be satisfiable. The developer does not need to know how. That ignorance is not a gap in their knowledge. It is the interface working.

Go back to the epigraph. *The cargo does not belong to the crew. It was aboard before this watch, and it will be aboard after.* Watches change. What sits below the waterline is not consulted about the handover, and that is exactly why the handover works: a ship that renegotiated its cargo at every change of watch would not be carrying anything, it would be holding a meeting. The Pod is a watch. The claim is the entry in the papers. What becomes of the cargo when the papers are torn up was settled in a `reclaimPolicy` field, long before this watch came aboard.

☼ **Zenith:** Chapter 4 taught you that a Kubernetes object is a record of intent, and that the record outlives the thing that acts on it [*cross-bearing: see Ch 4 §1 — you file a declaration*]. That is the same sentence as this chapter's title.

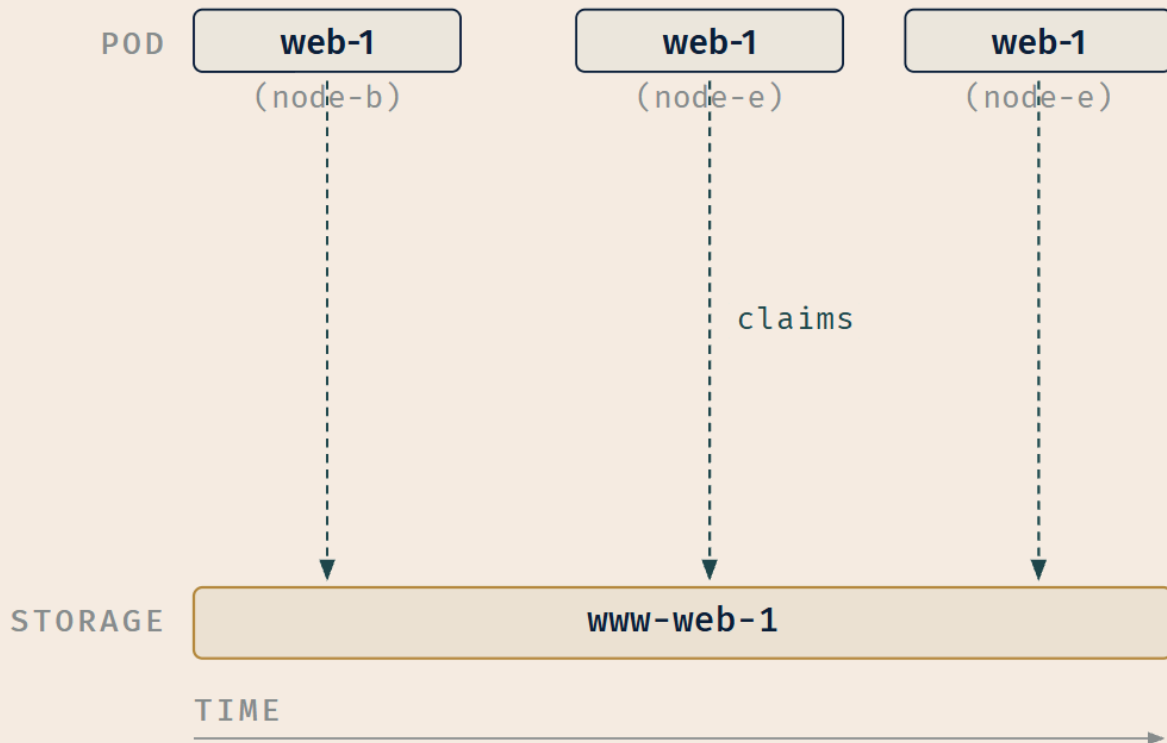
The claim is a record of intent about storage. It outlives the Pod that asked for it, because it was never the Pod's to begin with. It was filed on the Pod's behalf, against a supply the Pod knows nothing about, under terms settled before the Pod was scheduled. **Storage outlives the Pod that asked for it**, not because storage is special, but because *records of intent outlive the things that act on them*, and a claim is a record of intent.

Ten chapters of Kubernetes, and it is one idea wearing different clothes.

PERSISTENTVOLUMECLAIM • CH.11

Storage Outlives the Pod

A claim outlives the Pod that made it.



www-web-1 — one continuous line, never broken.

LEGEND

☐ Pod — bounded

☐ Storage — persists

--- Claims

Exam Alert! 🚨

High-Priority Topics:

1. **PersistentVolume vs PersistentVolumeClaim vs StorageClass.** CNCF publishes the Storage competency as a single word under Container Orchestration. This book's reading of it puts the three-way PV/PVC/StorageClass distinction at the center, which is why it leads this list. Know: PV is supply and cluster-scoped; PVC is demand and namespaced; StorageClass describes *classes* of storage and enables dynamic provisioning. And know that a Pod references the claim [source: k8s-docs-persistent-volumes-2026-08-23].
2. **Access modes, and what unit each counts.** RWO, ROX, RWX count **nodes**. RWOP counts **Pods** [source: k8s-docs-persistent-volumes-depth-2026-08-25]. If you memorize one sentence from this chapter, make it that one.
3. **Reclaim policy, and where the decision was made.** Retain / Delete / Recycle(deprecated). Dynamically provisioned volumes inherit the class's policy, which defaults to Delete [source: k8s-docs-persistent-volumes-depth-2026-08-25]; manually created PVs default to Retain [source: k8s-api-ref-persistentvolume-v1-2026-08-25].
4. **Static vs dynamic provisioning, and the two conditions dynamic requires.** The claim must request a class **and** the administrator must have created and configured that class for provisioning [source: k8s-docs-persistent-volumes-2026-08-23].

Common Traps — these are documented targets in this book's domain analysis, not merely things that are easy to confuse:

Trap	Correction
"A PVC binds to any PV that's big enough"	Binding is exclusive and one-to-one, and an unmatched claim stays unbound indefinitely [source: k8s-docs-persistent-volumes-2026-08-23]
Reversing PV and PVC	PV is supply, PVC is demand, Pods reference claims [source: k8s-docs-persistent-volumes-2026-08-23]
"ReadWriteOnce means one Pod"	It means one node ; multiple Pods on that node may share it. RWOP is the one-Pod mode [source: k8s-docs-persistent-volumes-depth-2026-08-25]
"Deleting a PVC always keeps the data"	Dynamic volumes inherit the class's policy, defaulting to Delete [source: k8s-docs-persistent-volumes-depth-2026-08-25]
"Retain means the PV is immediately reusable"	It becomes Released, not Available; manual reclamation takes three steps and a new PV object [source: k8s-docs-persistent-volumes-depth-2026-08-25]
Expecting Recycle to be live	Deprecated; use dynamic provisioning [source: k8s-docs-persistent-volumes-depth-2026-08-25]
"Class \"\" uses the default class"	It disables dynamic provisioning for that claim [source: k8s-docs-persistent-volumes-2026-08-23]

Two more that are worth knowing — both are sourced facts and both are common mistakes, though they have not been assessed for exam frequency the way the seven above have:

- **emptyDir survives container crashes but not Pod removal.** *A container crashing does not remove a Pod from a node, so the data is safe across crashes, but when a Pod is removed from a node for any reason, the data in the emptyDir is deleted permanently* [source: k8s-docs-volume-types-depth-2026-08-25].
 - **A StatefulSet's PVCs survive deletion of the Pod and of the StatefulSet,** and must be deleted manually. The retention policy that governs this defaults to Retain, which leaves PVCs created from a volumeClaimTemplate unaffected when their Pod is deleted [source: k8s-docs-statefulset-storage-2026-08-25].
-

Practice Questions

Seventeen questions, interleaved rather than grouped by section.

1. [retrieval: ch5] In Chapter 5, a Pod's ServiceAccount token arrived inside the container's filesystem without any configMap or secret volume being declared. Which volume type delivered it, and what else can that type carry?

A) A secret volume; it can carry Secret data and nothing else B) A projected volume; it maps several existing volume sources — among them secret, configMap, downwardAPI, and serviceAccountToken — into a single directory C) A hostPath volume mounting the node's kubelet credentials directory D) A generic ephemeral volume provisioned by the ServiceAccount controller

2. Which object does an application developer write in order to obtain persistent storage, on a cluster where they have no administrative access?

A) A PersistentVolume, specifying the backing storage system B) A StorageClass, specifying a provisioner C) A PersistentVolumeClaim, specifying a size and access mode D) A CSIDriver, specifying the vendor's driver name

3. A cluster has a working default StorageClass with reclaimPolicy unspecified. A PVC is created without a storageClassName, binds successfully, and is later deleted. What happens to the PV and the backing storage asset?

A) Both are retained; the PV becomes Released B) Both are deleted, because the class defaults to Delete and the volume inherited it C) The PV is deleted but the backing asset is retained D) The PV becomes Available and the asset is scrubbed

4. [retrieval: ch4] Chapter 4 named three cluster-scoped resources, two of which this chapter has now given you a reason for. Which set is correct?

A) Node, PersistentVolume, PersistentVolumeClaim B) Node, PersistentVolume, StorageClass C) PersistentVolume, PersistentVolumeClaim, StorageClass D) Namespace, PersistentVolume, PersistentVolumeClaim

5. A ConfigMap named `app-config` is mounted into a container using `subPath: settings.yaml`. An operator updates the ConfigMap with `kubectl apply`. What does the running container see?

A) The updated content, after the kubelet's sync period B) The updated content immediately, since ConfigMap volumes are watched C) The original content — a `subPath` mount does not receive updates D) An error, since ConfigMaps mounted with `subPath` become invalid on update

6. Which of the following is required for dynamic provisioning to occur when a PVC is created?

A) The PVC must request a storage class, and the administrator must have created and configured that class for provisioning B) A matching PersistentVolume must already exist in the `Available` phase C) The PVC must specify `volumeName` naming the PV to be created D) The cluster must have exactly one StorageClass marked default, so that the provisioner is unambiguous

7. An `nfs` volume is used by three Pods spread across three different nodes, all writing to it. Which access mode does this arrangement require the PersistentVolume to support?

A) `ReadWriteOnce` B) `ReadOnlyMany` C) `ReadWriteMany` D) `ReadWriteOncePod`

8. *[retrieval: ch10]* A cluster administrator creates a StorageClass whose `provisioner` field references a CSI driver that has not been installed. Which previously-named pattern does this instantiate, and what will a claim requesting that class do?

A) The absent-component pattern — "an object without its component does nothing"; the claim remains unbound B) The eventual-consistency pattern; the claim binds after a reconciliation delay C) The admission-rejection pattern; the StorageClass is rejected at creation D) The default-fallback pattern; the claim falls back to the default StorageClass

9. Which statement about `hostPath` volumes is accurate?

A) They are the recommended way to provide node-local durable storage, superseding `local` volumes B) They present many security risks, and should be avoided where possible — a `local` PersistentVolume is the suggested alternative C) They are automatically read-only, which mitigates the security concern D) They provide the same node-affinity awareness as `local` volumes

10. A StatefulSet named `db` has two replicas and one `volumeClaimTemplate` named `data`. What PersistentVolumeClaims exist once both Pods are running, and what happens to them if the StatefulSet is deleted?

A) One claim, `data-db`, shared by both Pods; deleted with the StatefulSet B) Two claims, `data-db-0` and `data-db-1`; both deleted with the StatefulSet C) Two claims, `data-db-0` and `data-db-1`; both survive and

must be deleted manually D) Two claims, `data-db-0` and `data-db-1`; both are garbage-collected when their Pods are deleted

11. A PersistentVolume supports both `ReadWriteOnce` and `ReadOnlyMany`. Can it be mounted read-write on node-1 and read-only on node-1 at the same time?

A) Yes — a PV may use all of its supported access modes concurrently B) No — a volume can only be mounted using one access mode at a time C) Yes, but only if the two mounts are in different namespaces D) No — supporting two access modes is invalid and the PV will be rejected

12. *[retrieval: ch2]* CSI, CRI, CNI, and CRDs are described in this book as the four pluggable interfaces. What do all four have in common?

A) All four are implemented by DaemonSets running on every node B) All four are cluster-scoped API objects C) All four publish a contract that lets someone outside the Kubernetes project supply an implementation without editing core Kubernetes code D) All four were introduced in the same Kubernetes release and are versioned together

13. `kubectl get pv` shows exactly one PersistentVolume on a cluster: 100Gi, phase `Available`, `storageClassName: fast`, carrying the label `tier=production`. A user then creates a 10Gi PersistentVolumeClaim requesting `storageClassName: fast` with a selector of `matchLabels: {tier: staging}`. What happens?

A) It binds — the PV satisfies both the capacity request and the class request B) It binds — label selectors filter Pods onto nodes, not claims onto volumes C) It remains unbound — a selector on a claim is an additional requirement, and every requirement must be satisfied D) It binds, and the PV's `tier` label is updated to `staging` to reflect its new claimant

14. An `emptyDir` volume is configured with `medium: Memory`. A container writes 2GiB of data into it. The container has a memory limit of 1GiB and is currently using 200MiB of heap. What happens?

A) Nothing — `tmpfs` usage is accounted to the node, not the container B) The write fails with `ENOSPC` once the node's RAM is exhausted C) The container is OOM-killed, because files written to a memory-backed `emptyDir` count against the writing container's memory limit D) The volume automatically spills to disk when the memory limit is approached

15. After a PVC bound to a `Retain`-policy PV is deleted, which sequence returns the underlying storage asset to service for a *different* application, with the old data removed?

A) Wait for the PV to return to `Available`, then bind a new claim to it B) Delete the PV, manually clean the storage asset, manually delete the asset, and create a new PV if reusing the same asset definition C) Patch the PV's `claimRef` to null, which returns it to `Available` with data intact D) Set the PV's reclaim policy to `Recycle`, which scrubs and republishes it

16. A `StatefulSet` Pod named `cache-2` is running on `node-x` with claim `store-cache-2`. `node-x` is cordoned and drained. What is true of the replacement Pod?

A) It will be named `cache-3` and will receive a newly provisioned claim B) It will be named `cache-2` and will mount `store-cache-2` on whichever node it is scheduled to C) It cannot be scheduled elsewhere, since its storage is bound to `node-x` D) It will be named `cache-2` but will start with an empty volume until the data is replicated

17. Which statement about generic ephemeral volumes is correct?

A) They are identical to `emptyDir` in every respect except that they can be network-attached B) They cause a real `PersistentVolumeClaim` to be created in the Pod's namespace, which is deleted when the Pod is deleted C) They cannot be provisioned by CSI drivers, only by in-tree plugins D) Their PVCs survive Pod deletion and must be cleaned up manually, like a `StatefulSet`'s

Answers with Explanations:

1 — B. A projected volume maps several existing volume sources into the same directory [source: [k8s-docs-projected-volumes-2026-08-25](#)], and `serviceAccountToken` is one of those sources. That is how a token reaches a container whose `PodSpec` declares no `Secret` at all — the mechanism you met in Chapter 5 as a special case is the general one [*cross-bearing: see Ch 5 §6 — a Pod's identity*].

- **A** names a real volume type with the wrong reach. A `secret` volume carries `Secret` data — and carries it on `tmpfs`, so it is *never written to non-volatile storage* [source: [k8s-docs-volume-types-depth-2026-08-25](#)] — but it composes nothing, and the Pod in the question declared no `Secret` to compose from. **C** is this chapter's hazard wearing a plausible costume: a Pod reading the node's `kubelet` credentials directory is precisely the escape the `hostPath` warning exists to prevent [source: [k8s-docs-volume-types-depth-2026-08-25](#)], and it is not how any token is delivered. **D** invents a provisioner. A generic ephemeral volume is requested in the `PodSpec` and satisfied by a storage driver; the `ServiceAccount` controller issues no volumes.

2 — C. A *`PersistentVolumeClaim` (PVC)* is a request for storage by a user [source: [k8s-docs-persistent-volumes-2026-08-23](#)], specifying size and access mode while *details of the storage itself are described in the `PersistentVolume` object* [source: [k8s-glossary-storage-terms-2026-08-25](#)].

- **A** is the administrator's object and is cluster-scoped. **B** is also the administrator's, and requires knowing a provisioner. **D** describes a driver installation, not a storage request.

3 — B. If no *`reclaimPolicy`* is specified when a *`StorageClass` object* is created, it will default to *`Delete`* [source: [k8s-docs-storage-classes-2026-08-25](#)], and *volumes that were dynamically provisioned inherit the reclaim policy of their `StorageClass`* [source: [k8s-docs-persistent-volumes-depth-2026-08-25](#)]. *`Delete` removes both the `PersistentVolume` object from Kubernetes, as well as the associated storage asset* [source: [k8s-docs-persistent-volumes-depth-2026-08-25](#)].

- **A** describes `Retain`, which is the manual-creation default, not the dynamic one. **C** splits the two deletions, which `Delete` does not do. **D** describes `Recycle`, deprecated.

4 — B. Chapter 4 named `PersistentVolume` as a canonical cluster-scoped resource; claims are namespaced, and *claims must exist in the same namespace as the Pod using the claim* [source: k8s-docs-persistent-volumes-depth-2026-08-25]. The reason is the supply/demand split: supply belongs to the cluster, demand belongs to a team.

- **A** is wrong on the `PersistentVolume`. A PV is a cluster resource in the same sense a node is a cluster resource — it belongs to no namespace because it belongs to no team, and it was very likely created before the namespace that will consume it existed.
- **C** is wrong on the `PersistentVolumeClaim`, and it is the most reasonable wrong answer here: having correctly placed the PV at cluster scope, it assumes the claim follows its supply. It does not. The claim is the tenant's half of the arrangement, and it lives where the tenant lives, in the same namespace as the Pod that mounts it.
- **D** is wrong on both halves; it inverts the relationship entirely, putting supply inside a namespace and demand outside every namespace.

5 — C. A container using a `ConfigMap` as a `subPath` volume mount will not receive updates when the `ConfigMap` changes [source: k8s-docs-volume-types-depth-2026-08-25].

- **A** and **B** describe whole-volume mount behavior, which is the rule this is the exception to. **D** invents a failure mode. Nothing errors; it simply does not update, which is worse, because it is silent.

6 — A. The PVC must request a storage class and the administrator must have created and configured that class for dynamic provisioning to occur [source: k8s-docs-persistent-volumes-2026-08-23]. Two conditions.

- **B** describes static provisioning; in fact dynamic provisioning is what happens when *no* matching PV exists. **C** invents a field usage; `volumeName` binds to an existing PV. **D** attaches the mechanism to the wrong thing. A default class decides what happens to a claim that names *no* class; it is not a precondition for provisioning. A claim that names `fast` provisions against `fast` whether or not any class is marked default — and *you can have a cluster without any default StorageClass* [source: k8s-docs-storage-classes-2026-08-25], on which every classless claim simply waits.

7 — C. `ReadWriteMany` — the volume can be mounted as read-write by many nodes [source: k8s-docs-persistent-volumes-depth-2026-08-25]. Three nodes, all writing.

- **A** would permit only one node. **B** would forbid the writes. **D** would permit only one Pod cluster-wide, which is even more restrictive than A.

8 — A. To use a CSI driver from a storage provider, you must first deploy it to your cluster [source: k8s-glossary-storage-terms-2026-08-25], and the core of Kubernetes does not install that software for you [source: k8s-docs-volumes-csi-and-subpath-2026-08-25]. The claim stays unbound indefinitely.

- **B** is wrong because there is no retry that ends in a bind. *Claims will remain unbound indefinitely if a matching volume does not exist* [source: k8s-docs-persistent-volumes-2026-08-23] — the binder

is a control loop, and a loop reconciling toward a volume that nothing will ever create converges on waiting. This is not a delay before success; it is the terminal state.

- **C** deserves an explicit rejection: the API server validates schema, not the existence of running components. A StorageClass naming a provisioner nobody deployed is a valid object.
- **D** is wrong because a claim that names a class does not fall back to another one. The default class applies only to a claim with *no* storageClassName field; naming a class is a commitment to that class, and there is nothing in the binding path that quietly substitutes a different one.

9 — B. Using the *hostPath* volume type presents many security risks. If you can avoid using a *hostPath* volume, you should. For example, define a local *PersistentVolume*, and use that instead [source: k8s-docs-volume-types-depth-2026-08-25].

- **A** inverts the recommendation. **C** is false; read-only must be *required* by admission-time validation to be effective [source: k8s-docs-volume-types-depth-2026-08-25]. **D** is the distinction that makes *local* preferable: *the system is aware of the volume's node constraints by looking at the node affinity on the PersistentVolume* [source: k8s-docs-volume-types-depth-2026-08-25], which *hostPath* does not provide.

10 — C. For each *volumeClaimTemplate* entry defined in a *StatefulSet*, each Pod receives one *PersistentVolumeClaim* [source: k8s-docs-statefulset-storage-2026-08-25] — hence two, named *data-db-0* and *data-db-1*. Their survival is governed by *persistentVolumeClaimRetentionPolicy*, whose *whenDeleted* and *whenScaled* fields both default to *Retain* [source: k8s-docs-statefulset-storage-2026-08-25]. Deleting the *StatefulSet* therefore leaves both claims standing, and removing them is a deliberate manual act.

- **A** describes a shared claim, which is what a Deployment referencing a single PVC would give you. **B** gets the naming right and the survival wrong, which is the more expensive half — a reader holding **B** will delete a *StatefulSet* expecting the storage to go with it, and be billed for volumes they believe are gone. **D** describes generic ephemeral volumes, not *volumeClaimTemplates*. The two mechanisms both produce one claim per Pod and differ precisely on deletion: the ephemeral-volume controller *ensures that the PersistentVolumeClaim gets deleted when the Pod gets deleted* [source: k8s-docs-ephemeral-volumes-2026-08-25], while a *StatefulSet*'s claims are retained by default. Same shape, opposite ending.

11 — B. A volume can only be mounted using one access mode at a time, even if it supports many. For example, a *NFS* volume can be mounted as *ReadWriteOnce* on one node and *read-only* on another node at the same time, but not on the same node for both *read-write* and *read-only* [source: k8s-docs-persistent-volumes-depth-2026-08-25]. The question's scenario is the documentation's exact counterexample.

- **A** ignores the "one at a time" constraint. **C** invents a namespace dimension. **D** is false; a PV listing several supported modes is entirely normal.

12 — C. Each of the four publishes a contract so an implementation can be supplied from outside. CSI states it directly — vendors can *introduce new storage systems into Kubernetes without ever having to edit the core Kubernetes code* [source: k8s-docs-volumes-2026-08-23] — and the extension-points

documentation groups CSI, CNI, and CRI together as infrastructure extensions alongside CRDs as API extensions [source: k8s-docs-extending-kubernetes-2026-08-23]. Grouping these four as *the* four pluggable interfaces is this book's framing, not a Kubernetes ranking.

- **A** is true of some node components but not of CRDs or the CSI controller component. **B** is false; CRI is not an API object at all. **D** is fabricated; they arrived at different times and version independently.

13 — C. Capacity is one filter among several, not the whole test. A claim's `storageClassName` must match the volume's, and where a claim carries a `selector`, the volume must satisfy every requirement in it — `matchLabels` and `matchExpressions` are ANDed together, so a candidate volume has to clear all of them [source: k8s-docs-persistent-volumes-depth-2026-08-25]. The single PV here clears the size and clears the class, and fails on `tier`. That is enough. *Claims will remain unbound indefinitely if a matching volume does not exist* [source: k8s-docs-persistent-volumes-2026-08-23], and nothing in the cluster will report this as an error.

- **A** is the trap this question exists for: it treats "big enough, right class" as sufficient. It is necessary, not sufficient. **B** misplaces the mechanism. Label selectors are the book's universal join [*cross-bearing: see Ch 4 §5 — the universal join*]; they attach controllers to Pods, Services to endpoints, and — here — claims to volumes. **D** inverts the direction of the match. Binding selects a volume that already satisfies the claim; it never edits the volume to make it fit.

14 — C. While `tmpfs` is very fast, be aware that, unlike disks, files you write count against the memory limit of the container that wrote them [source: k8s-docs-volume-types-depth-2026-08-25]. 2GiB written into `tmpfs` plus 200MiB of heap, against a 1GiB limit, is an OOM kill [*cross-bearing: see Ch 5 §8 — what a Pod is owed*].

- **A** is the misconception the documentation exists to correct. **B** describes what would happen with a `sizeLimit` on the default medium, not with a memory limit. **D** invents a spill mechanism.

15 — B. The documented steps: delete the PV, manually clean up the data on the associated storage asset, manually delete the asset. And *if you want to reuse the same storage asset, create a new PersistentVolume with the same storage asset definition* [source: k8s-docs-persistent-volumes-depth-2026-08-25].

- **A** never happens; `Released` does not become `Available` on its own [source: k8s-docs-persistent-volumes-depth-2026-08-25]. **C** describes an undocumented manual hack and, critically, leaves the old data in place, which the question explicitly required removing. **D** relies on a deprecated policy.

16 — B. A `StatefulSet` Pod keeps its ordinal identity across rescheduling [*cross-bearing: see Ch 6 §6 — when Pods are not interchangeable*], so the replacement comes back as `cache-2`. Its claim is independent of any node: when the replacement is scheduled, its volume mounts follow the `PersistentVolumeClaims` associated with that ordinal, wherever the scheduler puts it [source: k8s-docs-statefulset-storage-2026-08-25]. Kubernetes documents the same behavior for the harsher, involuntary case — when a Pod is lost to node failure, *the existing volume is unaffected, and the cluster will attach it to the node where the new*

Pod is about to launch [source: k8s-docs-statefulset-storage-2026-08-25]. A drain is the gentler version of the same story.

- **A** describes Deployment-style replacement, where replicas are interchangeable and names are regenerated. **C** inverts the mechanism; claim independence from nodes is what *enables* the reschedule. **D** invents replication that Kubernetes does not perform.

17 — B. *The ephemeral volume controller then creates an actual PersistentVolumeClaim object in the same namespace as the Pod and ensures that the PersistentVolumeClaim gets deleted when the Pod gets deleted* [source: k8s-docs-ephemeral-volumes-2026-08-25].

- **A** understates the differences: generic ephemeral volumes also support fixed size caps, snapshotting, cloning, and resizing where the driver supports them [source: k8s-docs-ephemeral-volumes-2026-08-25]. **C** is backwards; they *can* be provided by CSI drivers and by *any* driver supporting dynamic provisioning [source: k8s-docs-ephemeral-volumes-2026-08-25]. **D** describes StatefulSet behavior, which is the exact opposite of this mechanism and is the discrimination the question is testing.
-

Chapter Summary

Concept	Remember This
The lifetime ladder	Three rungs, two boundaries. Container restart kills rung one. Pod deletion kills rung two. Nothing in a Pod's lifecycle reaches rung three.
<code>emptyDir</code>	Safe across container crashes, deleted permanently when the Pod is removed. <code>medium: Memory</code> makes it tmpfs and charges it to the writing container's memory limit.
<code>hostPath</code>	A powerful escape hatch with many security risks. Avoid it; prefer a <code>local PersistentVolume</code> . Makes Pods silently node-dependent.
<code>subPath</code>	Cuts the update wire. <code>ConfigMap</code> , <code>Secret</code> , and <code>downwardAPI</code> mounts via <code>subPath</code> do not receive updates.
Persistent Volume	Supply. Cluster-scoped. Created by an administrator or by a provisioner. Captures the real storage's implementation details.
Persistent VolumeClaim	Demand. Namespaced. Requests a size and an access mode. A Pod references this, never the PV.
Binding	A control loop match. Exclusive, one-to-one, and filtered on more than size — class, selector, and <code>volumeName</code> each narrow the field, and every requirement is ANDed. An unmatched claim stays unbound indefinitely: not an error, just silence.
PV phases	The concept documentation names four: Available → Bound → Released → (Failed). Released is not Available. The API reference documents a fifth, Pending; if you meet it on an answer sheet, recognize it rather than rule it out.
StorageClasses	Describes classes of storage. Names a provisioner, carries opaque parameters, sets <code>reclaimPolicy</code> . The name is the interface.
Dynamic provisioning	Two conditions: the claim names a class and the class is configured to provision. One without the other waits forever.
<code>storageClassName: ""</code>	Opts <i>out</i> of dynamic provisioning. Not the same as omitting the field, which gets the default.
Access modes	RWO / ROX / RWX count nodes . RWOP counts Pods , and exists because people assume RWO already does. One mode at a time.
Reclaim policies	Retain (kept, released, manual three-step reclamation), Delete (PV object and backing asset both destroyed), Recycle (deprecated).
The inherited default	Dynamic volumes inherit the class's policy, defaulting to Delete. Manually created PVs default to Retain.
CSI	The fourth pluggable interface. A cross-orchestrator standard, not a Kubernetes feature. A driver is a Deployment plus a DaemonSet, and Kubernetes will not install it for you.
<code>volumeClaimTemplates</code>	One PVC per Pod, named <code><template>-<set>-<ordinal></code> . Follows the Pod across reschedules because it was never on a node.

Concept	Remember This
PVC survival	A StatefulSet's claims outlive the Pod <i>and</i> the StatefulSet, by design. Cleanup is manual, and nobody will remind you.

The Voyage Ahead

You now hold all four pluggable interfaces, and with them a rule you can state without help: an object without its component does nothing. You have watched that rule fire at the Ingress controller, at the network plugin, at the provisioner, and at the CSI driver — four sightings, one light. Chapter 17 will collect all four interfaces in one place and ask you what they have in common; you already know.

But this chapter also left something exposed, and the next one is about it.

Twice in these pages, storage handed a workload something it should probably not have had. `hostPath` mounts the node's filesystem into a Pod, and the documentation's warning was not about disk space. It was about *privileged system credentials* and *container escape*. Secrets mount as files backed by tmpfs, never written to non-volatile storage, which is a security property so specific it can only exist because someone was worried about the alternative.

Both of those are the same question wearing different clothes: **what is a workload allowed to do, and who decided?** You have spent this chapter learning that storage decisions were made elsewhere, by someone else, before you arrived — reading the manifest, not holding the keys. Chapter 12 asks the harder version. Not *what happens to your data* but *who is allowed to touch it*, and it turns out that Kubernetes' answer has a shape you have already met twice in this chapter without noticing.

Here is the tell. Carry it with you: the permission system you are about to learn has no way to say *no*. None. There is no deny rule, anywhere in it. You have already seen one other Kubernetes system with exactly that property [*cross-bearing: see Ch 10 §6 — allowing, never denying*], and by the end of the next chapter you will understand why two systems built for entirely different purposes arrived at the same design, and why that design is a feature rather than an omission.

Bring the `secret` volume with you. Chapter 12 §4 has an argument to make about file mounts versus environment variables, and you already hold half of it.

⚓ **Safe Harbor** — Domain 2's storage competency is complete. You can trace a file from the container filesystem out to a cluster-scoped volume and name what stops it at each boundary; you can distinguish a `PersistentVolume` from a claim from a class and say which one a Pod actually references; you can predict whether a claim binds, waits, or provisions; you can read an access mode as a node count; and you can say where the decision about your data's survival was actually made. Chapter 6's five deferred verbs are all settled, and the book's one deliberate forward reference is closed.

"The hold is inventoried by people who never sail. That is not a failure of the arrangement. That is the arrangement."

Chapter 12: Locks, Keys, and Watchstanders

"RBAC has no deny rule, and Secrets aren't encrypted"

Domain: Container Orchestration (D2) — 28% of the exam [source: cncf-kcna-curriculum-pdf-2026-08-23]
| Chapter allocation: ~7% | Complexity: Mixed | Novelty: Moderate

A note on that 7%. The CNCF publishes weights for the four domains and for nothing below them [source: cncf-kcna-curriculum-pdf-2026-08-23]. There is no published figure for how much of Domain 2 is security, and anyone who gives you one is guessing. The ~7% above is this book's own allocation, derived from the competency list and the shape of the published objectives. It is a planning number for your study time, not a fact about the exam. Treat it the way you would treat any estimate: useful for deciding what to read next, useless for arguing with.

Attention Budget

Total time: ~155 minutes of reading, plus about 35 more for the exam alert, the practice questions and the summary | Recommended: split the reading across two sessions

The natural seam is after Checkpoint #2. Sections 1 through 6 are the Kubernetes API and the workload; sections 7 through 9 leave the cluster, go looking at what you shipped, and come back. They are genuinely different material and a break between them costs you nothing.

Block	Time	Attention Cost	Best Time to Study
🕒 Soundings	10 min	Medium	Before you start reading
Why This Chapter Matters / What You'll Learn	6 min	Low	Anytime
§1 Four Layers and Four Phases	10 min	Low	Anytime
§2 Who You Are	12 min	Medium	When alert
§3 What You May Do	22 min	High	Peak attention
🕒 Taking Your Bearings #1	8 min	Medium	After a brief break
§4 Secrets Are Not Encrypted	14 min	Medium	When alert
§5 What a Pod May Do to Its Node	16 min	High	Peak attention
§6 Three Levels, Three Modes	12 min	Medium	When alert
🕒 Taking Your Bearings #2	8 min	Medium	After a brief break
— session break —			
§7 Trusting What You Ship	14 min	Medium	Anytime
§8 Rules That Watch	8 min	Low	Anytime
🕒 Taking Your Bearings #3	8 min	Medium	After a brief break
§9 Additive, Never Deny	6 min	Low	When you can think, not when you can grind
Exam Alert	4 min	Low	Anytime
Practice Questions (23)	28 min	High	A separate sitting from the reading
Chapter Summary / The Voyage Ahead	4 min	Low	Anytime

Attention Cost Key:

- **Low:** concrete, familiar concepts — study anytime.
- **Medium:** new concepts requiring focus — study when alert.
- **High:** abstract or dense material — study at peak attention.

If you only have fifteen minutes: read §3 and take Checkpoint #1. It is the densest section in the chapter and the highest-yield material in the domain.

"The RBAC API prevents users from escalating privileges by editing roles or role bindings. Because this is enforced at the API level, it applies even when the RBAC authorizer is not in use." — Kubernetes documentation, Using RBAC Authorization

🔊 Soundings

Before reading this chapter, try these eight questions. Your score sets your reading strategy. There is no wrong score, only different approaches.

Six of these test material from earlier chapters. Two of them, questions 6 and 7, ask you to reason from professional intuition you may have built outside Kubernetes entirely. Answer both kinds honestly; the second kind is doing more work than it looks like.

1. A resource is cluster-scoped rather than namespaced. Can a permission over it be granted "inside one namespace"? Is your answer a matter of policy, or is it forced by something structural?
2. A Pod's manifest names no ServiceAccount. What identity does the Pod run as, and where did that identity come from?
3. A Secret's values are base64-encoded. What does base64 do to those values, and — stated as plainly as Chapter 4 stated it — what does it *not* do?
4. Name the three gates an API request passes through, in order. Which of the three runs last?
5. Can one NetworkPolicy deny traffic that a different NetworkPolicy allows?
6. Think of a permission system you have actually used: Unix file modes, a cloud IAM policy, a Windows ACL, a firewall ruleset. In that system, can one rule *subtract* access that another rule grants?
7. A software release is signed. What does that signature prove, and what does it not prove? Would a signature that covers a *tag* mean the same thing as one that covers a *digest*?
8. A process is running as root inside a container. Is that the same root as the node's? What does Chapter 2 §1 tell you that bears on the answer?

Answers + reading strategy

1. No — and the answer is forced, not chosen. A cluster-scoped resource belongs to no namespace at all, so there is no namespace inside which the grant could be made. *[cross-bearing: see Ch 4 §3 — namespaces and cluster-scoped]*
2. The default ServiceAccount for its namespace. Every namespace gets one at creation, and Kubernetes assigns it to any Pod that does not name another. *[cross-bearing: see Ch 5 §6 — a Pod's identity]*
3. Base64 makes the value transport-safe as text. It does not encrypt, obscure, or protect it: anyone who can read the encoded string can decode it in one command. *[cross-bearing: see Ch 4 §4 — configuration kept outside the image]*
4. Authentication, then authorization, then admission. Admission runs last. *[cross-bearing: see Ch 8 §2 — three gates and a logbook]*

5. No. NetworkPolicies are additive: each one allows, and the effect of several is the union of what they allow. There is no way to write a policy that takes away traffic another policy permits. *[cross-bearing: see Ch 10 §6 — allowing, never denying]*
6. Almost certainly yes. Unix has no explicit deny, but cloud IAM does, Windows ACLs do, and firewall rulesets are built out of it, with the order-dependence that comes along for the ride. Hold on to that expectation. This chapter is going to violate it twice.
7. A signature proves that some specific bytes were signed by the holder of some specific key, and nothing more. It does not prove the software is safe, correct, or free of vulnerabilities. On the second half: if your instinct was that a signature over a tag and a signature over a digest are not the same statement, hold that answer — §7 shows what signing tools actually do about it. *[cross-bearing: see Ch 2 §3 — registries, tags, and digests]*
8. Chapter 2 §1 gives you the material: a container is a process in a set of Linux namespaces and cgroups, sharing the host's kernel. That points both ways at once. The process is *isolated* from the host's view of things, but it is running against the same kernel. Which of those two facts dominates is exactly what §5 is about.

If you got 6 or more right: skim. Read the ☆ Fixed Points, the △ Navigational Hazards, and §3 and §9 in full: §3 because it is where most of this chapter's traps live, §9 because it makes an argument you cannot get from a summary. Then go straight to the Practice Questions.

If you got 3 to 5 right: read at normal pace. The chapter is calibrated for you.

If you got 0 to 2 right: read carefully, and re-read **Chapter 4 §3 before you start this chapter** — not alongside it, before it. Section 3 below is built as a derivation from that boundary, and two earlier chapters have already promised you it would be. The others can wait until you reach the section that needs them: Ch 5 §6 (a Pod's identity) before §2, Ch 8 §2 (the three gates) before §6, Ch 10 §6 (allowing, never denying) before §9.

Why This Chapter Matters

Chapter 11 left you holding a question and refused to answer it: **what is a workload allowed to do, and who decided?**

That question stays open for the next eight sections. Chapter 11 also gave you something deliberately, as a tell: the permission system you are about to learn has no way to say no. None. You now know one other system with the same property, because you met it in Chapter 10 and it surprised you then. What you have not been told is *why*, and this chapter owes you that. Sections 2 through 8 are the evidence. Section 9 is the argument.

Something changes for you along the way. There is a line between an engineer who can *use* a cluster and an engineer who can be trusted with somebody else's, and this chapter is most of it. Every control in here answers a question a platform team gets asked in a room with an auditor in it: *Who can read the database password? What stops a compromised container from reading the node's kubelet credentials? How do you know the image running in production is the one your build system produced?* Naming which control answers which question is the job. Not reciting the controls: mapping them onto questions.

That is also why this chapter carries two framings rather than one, and why §1 spends its time on maps instead of features. The two maps answer different questions, and neither answer implies the other. A cluster with immaculate RBAC and an unsigned image from an unverified registry is not a secure cluster. It is a cluster with immaculate RBAC.

Two facts in this chapter are things a competent engineer will get wrong on a real cluster, today, without noticing. The first is that a Secret is protected because its values are base64-encoded. The second is that a namespace where somebody holds only a `view` RoleBinding is a safe place to let them look around. Both are wrong, both are wrong quietly, and both are wrong in a way that produces no error message and no alert. That is the honest stake here: not a story about a breach, just the plain observation that these two mistakes are invisible right up until they are not.

Dead Reckoning: Security in Kubernetes is not one system. It is five systems that answer five different questions: *who are you, what may you do, what is stored where, what may your workload do to the machine it runs on, and what did you actually ship*. They were built at different times by different people. They do not check each other's work. None of them substitutes for another, and the failure of any one of them is not detected by the other four.

Extended Analogy: A ship carries locks, keys, and watchstanders, and the reason all three exist is that no one of them does the others' work.

The **key** is not the permission. A key is a credential, a piece of metal that proves you are the person the quartermaster issued it to. What *opens* a particular door is the ship's manifest of who holds which key, kept somewhere else entirely, revisable without touching the key itself. Kubernetes keeps these in two different objects on purpose, and confusing them is the most common conceptual error in this chapter: a ServiceAccount is a key, and it opens nothing at all until something else says it does.

The **strongbox is not the safe**. A strongbox keeps papers together, dry, and in one identifiable place. It does not make them unreadable to anyone who opens it. Kubernetes Secrets are a strongbox: a distinct object type with its own handling, its own access rules, and its own storage path. The mistake is assuming the box came with a lock fitted. It did not, and Chapter 4 told you so.

And the **watch does not prevent anything**. A lookout at the masthead has no authority to alter course. The watch's entire function is to see, and to report what it saw to someone who can act. That distinction is the whole of §8: some tools in this chapter refuse things at the gate, and one of them stands on deck and tells you what already happened. Confusing those two is how people end up believing they have a control when what they have is a log entry.

What You'll Learn

By the end of this chapter, you'll be able to:

- **Place** any Kubernetes security control on two maps at once — which layer it protects, and which lifecycle phase it acts in — and tell which of the two an exam question is actually asking about.
- **Distinguish** an identity from a permission, and name the two separate objects Kubernetes keeps them in.
- **Derive** which of Role, ClusterRole, RoleBinding and ClusterRoleBinding a situation calls for, from the namespace/cluster-scoped boundary you already have, rather than from a memorized table.
- **State** what protects a Secret, what does not, and the three ways somebody reads one anyway — one of which is simply being allowed to create a Pod, which nobody thinks of as a permission to read secrets.
- **Read** a securityContext and a namespace's Pod Security labels together, and predict whether a given Pod is admitted, warned about, or refused outright.
- **Trace** an image from build to running container and name the checkpoint at each handoff, including which one a signature actually covers.

You will also finish an argument Chapter 10 started and Chapter 11 refused to settle: why two Kubernetes systems built for entirely unrelated purposes both refuse to let you say no.

§1 — Four Layers and Four Phases

Kubernetes security has two standard maps, and the reason to learn both is that they answer different questions. One tells you *where* a control sits. The other tells you *when* it acts.

The phases: when

The Kubernetes documentation now frames cloud native security around the lifecycle phases defined in the CNCF's cloud native security whitepaper: **develop, distribute, deploy, and runtime** [source: k8s-docs-cloud-native-security-2026-08-23].

Develop is about the integrity of the development environment and the design of the application. The documentation names adopting an architecture such as zero trust that minimizes attack surface even against internal threats, defining a code review process that considers security, and building a threat model that identifies trust boundaries [source: k8s-docs-cloud-native-security-2026-08-23]. Nothing in this phase is a Kubernetes object. That is the point of having the phase.

Distribute is the supply chain, for your container images and for the cluster components themselves. The recommendations are concrete: scan container images and other artifacts for known vulnerabilities; ensure software distribution uses encryption in transit with a chain of trust for the software source; use validation mechanisms such as digital certificates for supply chain assurance; and restrict access to artifacts by placing container images in a private registry that only allows authorized clients to pull [source: k8s-docs-cloud-native-security-2026-08-23]. That list is §7's outline.

Deploy is about restrictions on *what* can be deployed, *who* can deploy it, and *where* it can go [source: k8s-docs-cloud-native-security-2026-08-23]. Read that sentence again with a slight squint and you have described GitOps before anyone has given you the word: a system whose entire job is to constrain what reaches the cluster, from where, on whose authority. [*cross-bearing: see Ch 15 — push, or pull*]

Runtime is where most of this chapter lives, and the documentation splits it into three areas: **access, compute, and storage** [source: k8s-docs-cloud-native-security-2026-08-23]. That split is this chapter's table of contents.

- **Runtime protection: access.** The Kubernetes API is what makes the cluster work, so protecting it is the core of cluster security — effective authentication and authorization for API access, ServiceAccounts to provide and manage security identities for workloads and cluster components, and TLS protecting API traffic including between nodes and the control plane [source: k8s-docs-cloud-native-security-2026-08-23]. That is §2 and §3.
- **Runtime protection: compute.** Enforce Pod Security Standards so applications run with only the privileges they need; run a specialized, typically read-only immutable node operating system; define ResourceQuotas and LimitRanges; **partition workloads across different nodes to improve isolation**; use a container runtime that provides security restrictions; and on Linux nodes use a security module such as AppArmor or seccomp [source: k8s-docs-cloud-native-security-2026-08-23]. That is §5 and §6, and note two familiar objects showing up in a new role. ResourceQuota and

LimitRange were fairness controls when you met them; here they are security controls, because a workload that can exhaust a node is a workload that can affect every other workload on it. *[cross-bearing: see Ch 8 §3 — dividing a shared cluster]* And "partition workloads across different nodes" is the isolation use of node selection that Chapter 7 told you would come back. *[cross-bearing: see Ch 7 §4 — when the berth refuses you]*

- **Runtime protection: storage.** Integrate an external storage plugin providing encryption at rest for volumes; enable encryption at rest for API objects; protect durability with backups you have verified you can restore; authenticate connections between nodes and network storage; and encrypt data within the application itself [source: k8s-docs-cloud-native-security-2026-08-23]. That is §4.

Sandboxed runtimes sit in runtime/compute. The "container runtime that provides security restrictions" clause [source: k8s-docs-cloud-native-security-2026-08-23] is where RuntimeClass and the sandboxed runtimes belong on this map. *[cross-bearing: see Ch 2 §7 — not all isolation is equal]*

The layers: where

The other framing is older. It is the one the Kubernetes project's own security overview carried in the v1.22 documentation, and it is still the framing most third-party material uses. **The 4Cs of Cloud Native security are Cloud, Clusters, Containers, and Code** [source: k8s-docs-4cs-cloud-native-security-v1-22-archived-2026-08-31], and they nest: each layer of the model builds upon the next outermost layer, so the Code layer benefits from strong Cloud, Cluster and Container security beneath it. That page states the consequence bluntly — "You cannot safeguard against poor security standards in the base layers by addressing security at the Code level" [source: k8s-docs-4cs-cloud-native-security-v1-22-archived-2026-08-31].

Cloud is the trusted computing base. The page is direct: "If the Cloud layer is vulnerable (or configured in a vulnerable way) then there is no guarantee that the components built on top of this base are secure" [source: k8s-docs-4cs-cloud-native-security-v1-22-archived-2026-08-31]. Its concerns are network access to the API server and the nodes, the permissions the cluster holds against the cloud provider's own API, and access to etcd, including the recommendation that etcd's disk especially should be encrypted at rest, *since etcd holds the state of the entire cluster (including Secrets)* [source: k8s-docs-4cs-cloud-native-security-v1-22-archived-2026-08-31]. Hold that clause. §4 is going to need it.

Cluster splits into securing the configurable cluster components and securing the applications that run in the cluster, and the workload-security list reads like this chapter's index: RBAC authorization, authentication, application secrets management including encryption at rest, Pod Security Standards, resource management, network policies, and TLS for Ingress [source: k8s-docs-4cs-cloud-native-security-v1-22-archived-2026-08-31].

Container covers vulnerability scanning as part of the image build, image signing and enforcement to maintain a system of trust, creating users inside containers with the least OS privilege necessary, and selecting container runtime classes that provide stronger isolation [source: k8s-docs-4cs-cloud-native-security-v1-22-archived-2026-08-31].

Code is described as "*one of the primary attack surfaces over which you have the most control*" — TLS for everything in transit, limiting exposed port ranges, scanning third-party dependencies, static analysis, and dynamic probing against your own service [source: k8s-docs-4cs-cloud-native-security-v1-22-archived-2026-08-31].

Why both

A reasonable question at this point is why a book would teach you two maps of the same territory. The honest situation:

The current kubernetes.io security overview presents the lifecycle phases. It *replaced* the 4Cs framing on that page [source: k8s-docs-cloud-native-security-2026-08-23]. So the phases are the current framing from the primary authority, and if the exam draws from the current documentation, the phases are what it is drawing from.

But the 4Cs have not gone anywhere else. The framing predates the current page, and you will still meet it in third-party material and in internal security documentation written before the change. It is also genuinely useful, because it answers a question the phases do not.

Which brings us to the actual distinction, and it is worth more than either list.

☆ Fixed Point

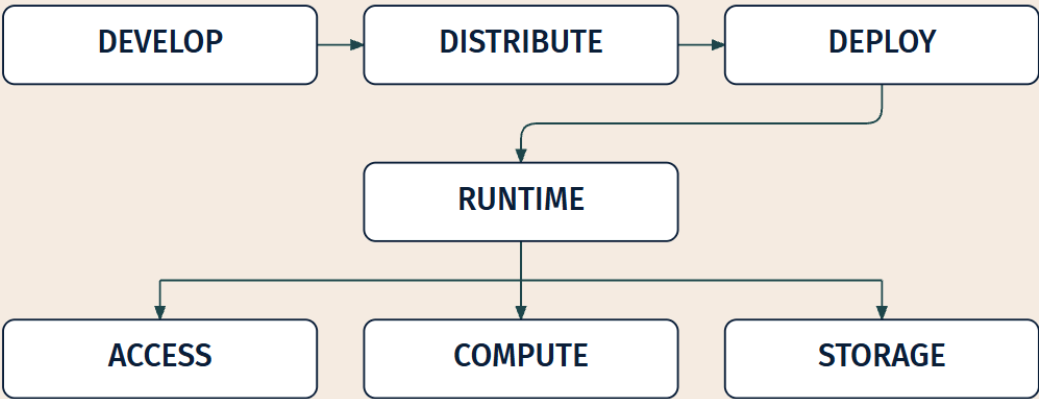
The phases answer *when* a control acts. The layers answer *where* it acts. Every control in this chapter has a position on both maps. A question that appears to be about one is frequently about the other.

Image signing sits in the **distribute** phase: it happens after the build, before anything reaches the cluster. It also sits at the **Container** layer, because what it protects is the container image. Neither of those statements is more true than the other and neither implies the other. RBAC is **runtime/access** on the phase map and **Cluster** on the layer map. `securityContext` is **runtime/compute** and **Container**. Encryption at rest is **runtime/storage** and **Cloud** (the etcd disk) shading into **Cluster** (the API objects).

The whole thing at once, then, with a few controls plotted on both.

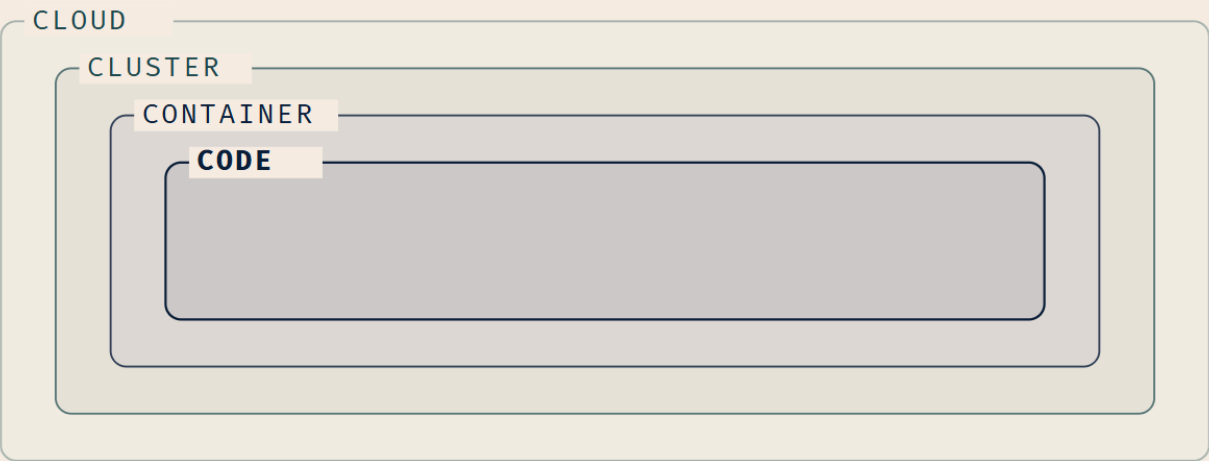
THE PHASES

when a control acts



THE LAYERS

where a control acts



FIVE CONTROLS, PLOTTED ON BOTH

CONTROL	PHASE	LAYER
RBAC	runtime / access	Cluster
ServiceAccount	runtime / access	Cluster
securityContext	runtime / compute	Container
encryption at rest	runtime / storage	Cloud → Cluster

image signing

distribute

Container

Do not try to hold twelve controls on this map yet. Hold the *shape*: four phases in sequence, four layers nested, and the runtime phase split three ways. The sections that follow fill it in.

§2 — Who You Are

Every request that reaches the Kubernetes API arrives claiming to be somebody. This section is about who that somebody can be, and, as importantly, about what that claim is *not*.

The object

A **ServiceAccount** is a type of non-human account that provides a distinct identity in a Kubernetes cluster. Application Pods, system components, and entities both inside and outside the cluster can use a ServiceAccount's credentials to identify as that ServiceAccount [source: k8s-docs-service-accounts-2026-08-23]. They exist as ServiceAccount objects in the API server. They are **namespaced**, bound to a Kubernetes namespace, with every namespace getting a default ServiceAccount on creation, and they are lightweight and portable [source: k8s-docs-service-accounts-2026-08-23].

You met the object in Chapter 5, where it was introduced as the identity a Pod runs as. [*cross-bearing: see Ch 5 §6 — a Pod's identity*] That chapter deliberately stopped there and deferred everything about permissions to here. This section collects that plant; it does not re-teach it.

What Chapter 5 could not tell you is the contrast that makes ServiceAccounts interesting. Service accounts are **different from user accounts**, which are authenticated human users in the cluster — and “by default, user accounts don't exist in the Kubernetes API server” [source: k8s-docs-service-accounts-2026-08-23].

Read that twice. Kubernetes has no User object. There is no `kubectl create user`. A human identity arrives at the API server from *outside the cluster*, and Kubernetes' job is to validate the claim, extract a username and a set of groups, and hand those strings to the next gate. It stores nothing. Usernames are just strings, and “it is up to you as a cluster administrator to configure the authentication modules so that authentication produces usernames in the format you want” [source: k8s-docs-rbac-depth-2026-08-31]. Groups likewise are strings, supplied by the authenticator, with the `system:` prefix reserved [source: k8s-docs-rbac-depth-2026-08-31].

That asymmetry surprises people: in-cluster identities are objects, human identities are not. It is the reason ServiceAccounts get their own section while users get a paragraph. Human identity is somebody else's system. Workload identity is Kubernetes'.

🔗 **Closer Look:** ServiceAccounts get names prefixed with `system:serviceaccount:` and belong to groups prefixed with `system:serviceaccounts:` [source: k8s-docs-rbac-depth-2026-08-31]. Singular for the account, plural for the group. That one-character difference is real, and it is exactly the sort of thing that is unpleasant to debug at 3 a.m.

The default account, and what it can do

When you create a cluster, Kubernetes automatically creates a ServiceAccount named `default` for every namespace. If you delete it, the control plane replaces it. And if you deploy a Pod in a namespace without manually assigning it a ServiceAccount, Kubernetes assigns the namespace's `default` [source: k8s-docs-service-accounts-2026-08-23].

Now the part that matters:

The default service accounts in each namespace get no permissions by default other than the default API discovery permissions that Kubernetes grants to all authenticated principals if RBAC is enabled [source: k8s-docs-service-accounts-2026-08-23].

API discovery means the endpoints that tell a client what API groups and versions the server supports. It is the read-only self-description every client needs before it can do anything at all. That is the entire budget.

☆ Fixed Point

An identity and a permission are two different things, kept in two different objects. The `default` ServiceAccount is the proof: every Pod in the cluster has an identity, and almost none of them can do anything with it.

RBAC has a word for the thing a grant is made to — a **subject**. A ServiceAccount is one; so is a user and so is a group. Holding the word now is what lets §3 talk about grants without having to keep saying "whatever the permission is being given to."

The sidebar's first panel was about exactly this. A key is stamped, issued, and unmistakably yours; what it opens is written down somewhere else entirely, and until somebody writes it there, the key turns nothing.

This is the single most useful sentence in the chapter for reasoning about the rest of it, and it is why §2 comes before §3 rather than merging with it. Creating a ServiceAccount grants nothing. Assigning it to a Pod grants nothing. The Pod authenticates successfully, arrives at the second gate, and is turned away: a completely different failure from not being recognized at all, and one that produces a different message. *[cross-bearing: see Ch 8 §2 — three gates and a logbook]*

You assign one with `spec.serviceAccountName` in the Pod spec. The documented sequence is: create a ServiceAccount object; grant permissions to it using an authorization mechanism such as RBAC; and

assign it to Pods at creation time via `spec.serviceAccountName` [source: k8s-docs-service-accounts-2026-08-23]. Three steps, and the middle one is §3.

The documented use cases read as a list of "why would a workload need to be somebody" [source: k8s-docs-service-accounts-2026-08-23]:

- Pods that need to talk to the Kubernetes API server — reading Secrets, cross-namespace access.
- Pods that need to talk to an external service that requires an identity.
- Authenticating to a private image registry using an `imagePullSecret`.
- An external service that needs to talk to the API server — a CI/CD pipeline, for instance.
- Third-party security software that relies on the ServiceAccount identity of different Pods to group them into contexts.

The fourth of those is worth pausing on. An agent running outside the cluster, or an agent running inside it whose job is to reconcile the cluster's contents against something, is a subject exactly like any Pod is a subject. It needs an identity, it needs grants, and because its job is broad its grants tend to be broad. That is a shape you will meet again. *[cross-bearing: see Ch 15 §4 — an agent that watches a repository]*

The credential

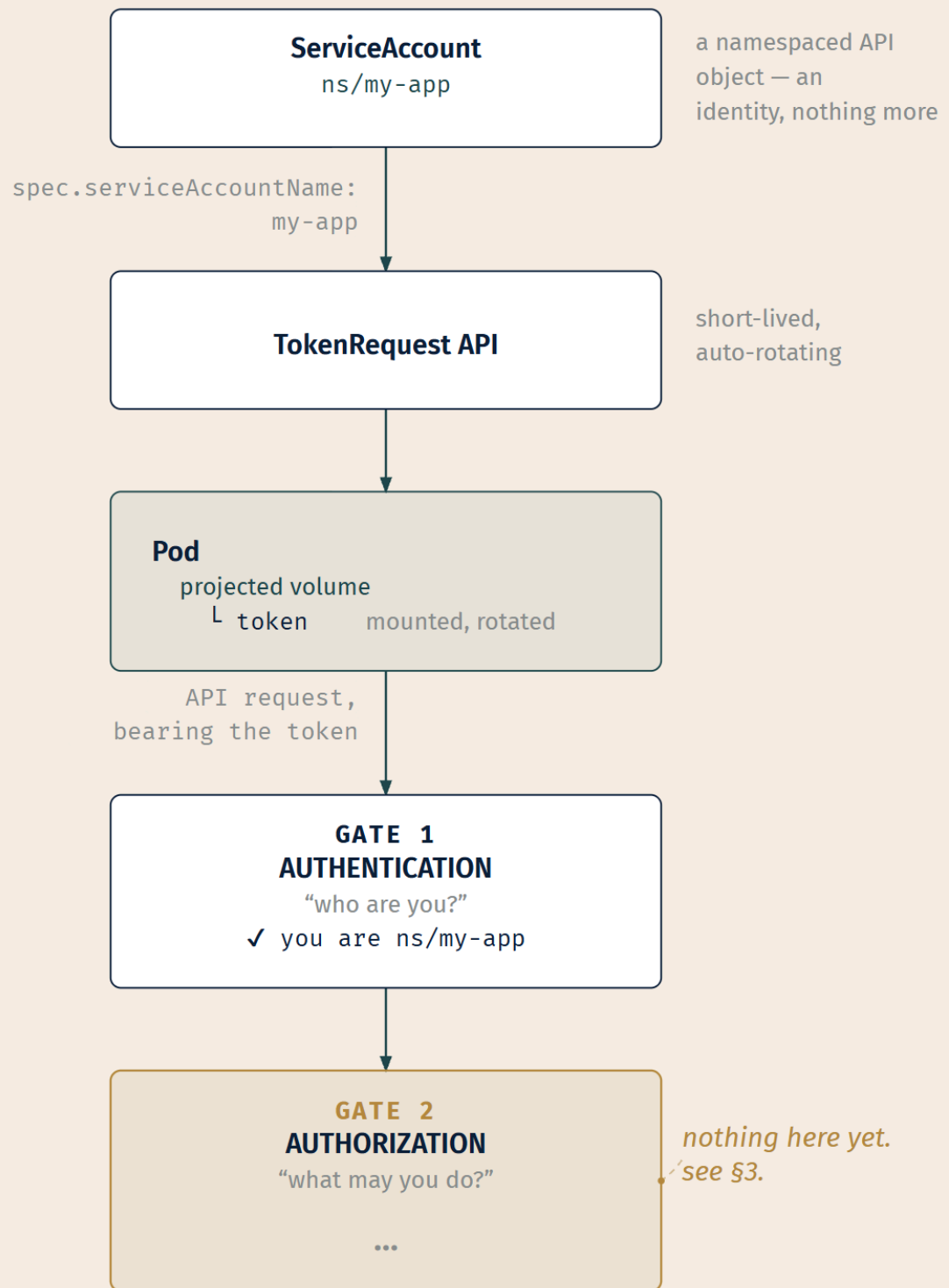
An identity is not much use without something to prove it with. That something is a token, and the recommended way of getting one changed.

In Kubernetes v1.22 and later, Kubernetes gets a short-lived, automatically rotating token using the **TokenRequest** API and mounts the token as a **projected volume**. The recommended approach is the TokenRequest API or token volume projection. **Not** recommended: long-lived ServiceAccount token Secrets, which don't expire or rotate and pose a security risk [source: k8s-docs-service-accounts-2026-08-23].

You have already seen the delivery mechanism. When Chapter 11 walked the volume types, the projected volume was there, and what it was projecting into the Pod was precisely this. *[cross-bearing: see Ch 11 §1 — three lifetimes, and the volumes that have them]*

🔖 **Snag:** There is a Secret type called `kubernetes.io/service-account-token` [source: k8s-docs-secret-2026-08-23], and its existence makes it look like the current mechanism. It is the legacy one. A Secret of that type is a credential written to etcd that neither expires nor rotates, which means a copy of it taken today still works next year. If a question offers you "create a ServiceAccount token Secret" as the way to give a Pod credentials, that is the distractor. The token arrives by projection, and nothing durable is created.

The whole flow, then, and note what is deliberately missing from it.



LEGEND

Step

Stateful

Focal (gap)

Note

The emptiness on the right is the Fixed Point drawn. The identity is complete. The token is valid. Authentication succeeds. And the account can do nothing, because nothing has been said about permissions yet.

Identity outlives the workload

One more thing, easy to miss and directly consequential.

Kubernetes checks for and deletes objects that no longer have owner references, and owner references are what drive that garbage collection [source: k8s-docs-garbage-collection-2026-08-24]. When you delete a Deployment, cascading deletion removes what the Deployment owns. It does **not** remove the ServiceAccount those Pods ran as, because a ServiceAccount you created alongside a Deployment carries no owner reference back to it. Neither do the Secrets it referenced. Neither do the RoleBindings that granted it anything.

Chapter 6 told you that deleting a workload does not delete everything it referenced, and pointed here. *[cross-bearing: see Ch 6 §3 — how a controller knows its own]* This is the security consequence: **identity and grants outlive the workload that used them**. A cluster that has been in production for three years accumulates ServiceAccounts nobody remembers creating, still holding grants nobody remembers making, ready to be used by anything that can name them. Nobody deletes them, because nobody is certain what would break.

That is not an exotic attack. It is entropy, and cleaning it up is one clause of what least privilege means in practice — which brings us to the object that does the granting.

..1 §3 — What You May Do

Before anything else, a word about a word.

This is the third distinct sense of **binding** you have met in six chapters, and you met the second one *last chapter*. The scheduler binds a Pod to a node: a one-time decision, filter then score then bind *[cross-bearing: see Ch 7 §1 — one decision, made once]*. A PersistentVolumeClaim binds to a PersistentVolume, exclusive and one-to-one *[cross-bearing: see Ch 11 §2 — the claim and the supply]*. Neither of those is what this section means. **Inside this section, a bare "binding" means the RBAC object and nothing else**, and the objects themselves are always written out in full: RoleBinding, ClusterRoleBinding. If you catch yourself expecting an RBAC binding to be exclusive or one-to-one because the last one you met was, it is not. A subject may hold any number of them and they all apply at once.

With that cleared: authorization.

RBAC is *an* authorization mode

The Kubernetes API server does not have one authorization mechanism. It has a chain of them, and the documentation is precise about how the chain behaves: **"All parts of an API request must be allowed by some authorization mechanism in order to proceed"** and *"[each authorizer] is checked in sequence. If any authorizer approves or denies a request, that decision is immediately returned and no other authorizer is consulted. If all modules have no opinion on the request, then the request is denied"* [source: k8s-docs-authorization-2026-08-31].

The modules include **Node**, a special-purpose mode granting permissions to kubelets based on the pods they are scheduled to run; **ABAC**, an access control paradigm whereby access rights are granted to users through policies which combine attributes together; **RBAC** — role-based access control — which regulates access based on the roles of individual users within an enterprise; and **Webhook**, which makes a synchronous HTTP callout, blocking the request until the remote service responds [source: k8s-docs-authorization-2026-08-31]. Chapter 8 quoted that list at you and pointed here. *[cross-bearing: see Ch 8 §2 — three gates and a logbook]*

So: RBAC is what essentially every cluster uses and what the curriculum teaches, but it is one mode among several rather than the mechanism. That one clause is all ABAC gets in this book, and it is enough. The fact worth holding is that other modes exist, not how they work.

RBAC uses the `rbac.authorization.k8s.io` API group to drive authorization decisions, allowing policies to be configured dynamically through the Kubernetes API [source: k8s-docs-rbac-2026-08-23]. Four object kinds: **Role**, **ClusterRole**, **RoleBinding** and **ClusterRoleBinding** [source: k8s-docs-rbac-2026-08-23].

The two role objects

Dead Reckoning: A Role or ClusterRole contains rules representing a set of permissions. A Role always sets permissions within a particular namespace, and you must specify that namespace when you create it. A ClusterRole is a non-namespaced resource. A RoleBinding grants a role's permissions to a list of subjects within a specific namespace; a ClusterRoleBinding grants that access cluster-wide. A RoleBinding may reference any Role in the same namespace, or may reference a ClusterRole and bind it into the RoleBinding's namespace. After you create a binding, you cannot change the Role or ClusterRole it refers to. The four default user-facing roles are `cluster-admin`, `admin`, `edit`, and `view`. [source: k8s-docs-rbac-2026-08-23]

That paragraph is the whole object model stated flat, and if you memorize nothing else from this section, memorize it. Now take it apart, because there is a derivation hiding in it that two earlier chapters have already promised you.

Role is namespaced. When you create one, you have to say which namespace it belongs in [source: k8s-docs-rbac-2026-08-23]. **ClusterRole**, by contrast, is a non-namespaced resource, and the documentation lists three distinct uses for one [source: k8s-docs-rbac-2026-08-23]:

1. Define permissions on **namespaced** resources and be granted access **within individual namespaces**.
2. Define permissions on **namespaced** resources and be granted access **across all namespaces**.
3. Define permissions on **cluster-scoped** resources.

Notice that only the third of those is about cluster-scoped resources. Two of the three uses of a ClusterRole concern namespaced resources, which means the object's name is actively misleading about two thirds of its job.

The rule the documentation gives for choosing is simple: *"If you want to define a role within a namespace, use a Role; if you want to define a role cluster-wide, use a ClusterRole"* [source: k8s-docs-rbac-2026-08-23].

The two binding objects, and the derivation

A binding grants the permissions defined in a role to a user or set of users. It holds a list of **subjects** — users, groups, or service accounts — and a reference to the role being granted [source: k8s-docs-rbac-2026-08-23]. A RoleBinding grants permissions within a specific namespace; a ClusterRoleBinding grants that access cluster-wide.

And then the sentence that does the work:

"A RoleBinding may reference any Role in the same namespace. Alternatively, a RoleBinding can reference a ClusterRole and bind that ClusterRole to the namespace of the RoleBinding." [source: k8s-docs-rbac-2026-08-23]

Four objects, and the natural reflex is to make a two-by-two table and memorize the cells. Chapter 4 told you not to. [cross-bearing: see Ch 4 §3 — where a name lives] Chapter 8 told you the same thing in nearly the same words. So: the derivation instead, and it needs exactly one thing you already have.

Chapter 4 established that some resources are namespaced and some are cluster-scoped. Put that beside two sentences you already have from the RBAC reference: *"A Role always sets permissions within a particular namespace"*, and *"ClusterRole, by contrast, is a non-namespaced resource"* [source: k8s-docs-rbac-2026-08-23]. A cluster-scoped resource belongs to no namespace, not to all of them — so **there is no namespace inside which a Role could hold a rule about it**. Not "should not." *Could not*. The difference between "no namespace" and "all namespaces" is the entire derivation.

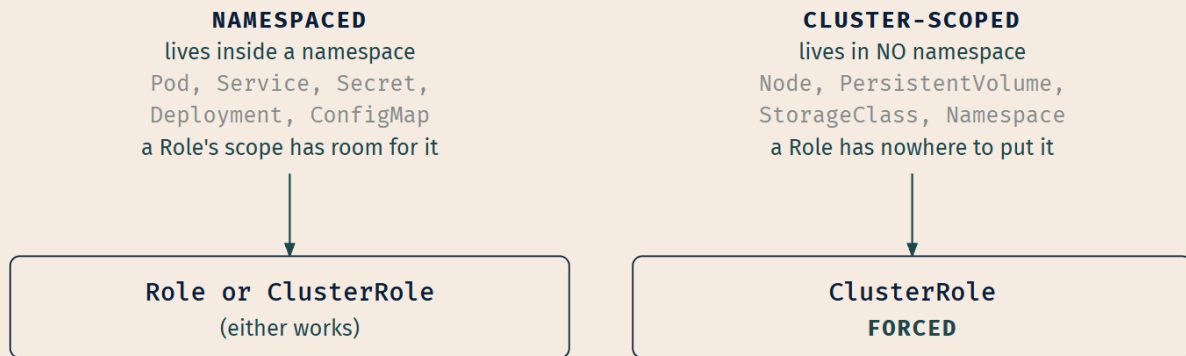
Start from there and ask two independent questions.

Question one: what does this permission cover? If the resources are namespaced, a Role can hold the rules, because a Role's scope has room for them. If any resource is cluster-scoped, a Role cannot hold the rules, because a Role is defined within a namespace and the resource is not in one. So cluster-scoped resources force a **ClusterRole**. Not by convention, but by the structure of what a Role is.

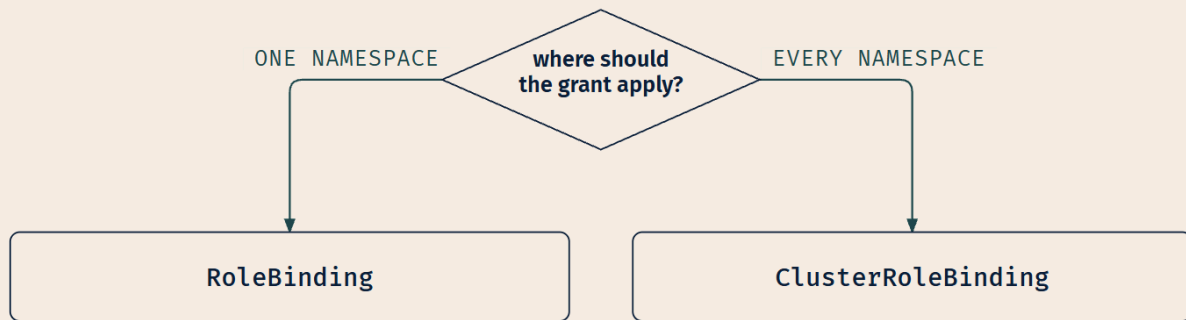
Question two: where should the grant apply? If it should apply in one namespace, a **RoleBinding** in that namespace. If it should apply everywhere, a **ClusterRoleBinding**.

Those two questions are independent, which is why there are four objects rather than two. And the combination that surprises people, a ClusterRole bound by a RoleBinding, falls out immediately: you wrote the rules in a ClusterRole because you wanted them reusable, or because you needed cluster-scoped resources; you bound them with a RoleBinding because you wanted the grant to land in one namespace.

THE BOUNDARY YOU ALREADY HAVE (CH 4 §3)



THE SECOND, INDEPENDENT QUESTION



THE FOUR COMBINATIONS FALL OUT

Role + RoleBinding those rules, in that namespace

ClusterRole + RoleBinding those rules, in that ONE namespace

ClusterRole + ClusterRoleBinding those rules, everywhere

~~Role + ClusterRoleBinding~~
X DOES NOT EXIST

A Role's rules are namespace-local;
there is nothing cluster-wide to grant.

THE BINDING DETERMINES THE SCOPE OF THE GRANT.

LEGEND

Object or binding

Not a valid combination

Key takeaway

If you cover the bottom of that figure, you should be able to rebuild it from the top. That is the test of whether you have the derivation or just the table.

☆ Fixed Point

The binding determines the scope of the grant. A ClusterRole bound by a RoleBinding grants that ClusterRole's permissions *inside that one namespace only* — including `cluster-admin`, which is where this trips people.

That last clause is not hypothetical. The documentation says of `cluster-admin`: *"When used in a ClusterRoleBinding, it gives full control over every resource in the cluster and in all namespaces. When used in a RoleBinding, it gives full control over every resource in the role binding's namespace, including the namespace itself"* [source: k8s-docs-rbac-2026-08-23]. Same ClusterRole. Two completely different amounts of power, decided entirely by which binding object you reached for.

Rules: verbs over resources

Inside a Role or ClusterRole, the rules name verbs and resources. The verbs the API recognizes are `get`, `list`, `create`, `update`, `patch`, `watch`, `delete`, and `deletecollection` [source: k8s-docs-authorization-2026-08-31]. Authorization decisions also consider the resource name, the subresource, the namespace, and the API group [source: k8s-docs-authorization-2026-08-31].

You can narrow further. *"You can also refer to resources by name for certain requests through the resourceNames list. When specified, requests can be restricted to individual instances of a resource"* [source: k8s-docs-rbac-depth-2026-08-31], so "may read the Secret named `db-password`, and no other Secret" is expressible. With limits: *"You cannot restrict deletecollection or top-level create requests by resource name. For create, this limitation is because the name of the new object may not be known at authorization time"* [source: k8s-docs-rbac-depth-2026-08-31]. Which is exactly right when you think about it: you cannot filter on a name that does not exist yet.

And a fact that costs one sentence and saves a lot of confusion later: **RBAC rules name custom resources exactly as they name built-in ones.** The documentation says so directly — aggregation *"lets you, as a cluster administrator, include rules for custom resources, such as those served by CustomResourceDefinitions or aggregated API servers, to extend the default roles"*, and *"You can assume that CronTab objects are named `crontabs` in URLs as seen by the API server"* [source: k8s-docs-rbac-depth-2026-08-31]. A CRD-backed resource lives in the same API, gets addressed the same way, and is granted the same way. Chapter 6 promised you that custom resources work with `kubectl get`, labels, selectors, namespaces, RBAC, all of it. [cross-bearing: see Ch 6 §8 — the control loop, extended] Nothing special is required here, and that is the whole point of putting custom resources in the API in the first place.

Aggregated ClusterRoles

"You can aggregate several ClusterRoles into one combined ClusterRole. A controller, running as part of the cluster control plane, watches for ClusterRole objects with an aggregationRule set. The aggregationRule defines a label selector that the controller uses to match other ClusterRole objects that should be combined into the rules field of this one" [source: k8s-docs-rbac-depth-2026-08-31]. Create a new ClusterRole matching the selector and its rules get added to the aggregate automatically.

Which is a control loop — desired state expressed as a selector, a controller reconciling toward it — doing a job you have watched it do elsewhere. *[cross-bearing: see Ch 3 §6 — controllers and the control loop]*

The default user-facing roles use aggregation, which is how a cluster administrator can extend admin, edit and view to cover custom resources [source: k8s-docs-rbac-depth-2026-08-31]. Define a ClusterRole with the right labels and the built-in roles quietly gain the rules.

Subjects are named, not selected

Something here should stop you.

Chapter 4 taught you that Kubernetes joins things with **selectors**. A Deployment finds its Pods by selector. A Service finds its endpoints by selector. A NetworkPolicy finds the Pods it applies to by selector. An aggregated ClusterRole, two paragraphs ago, finds its member ClusterRoles by selector. The pattern is so consistent that Chapter 4 warned you specifically that assuming it holds here would produce *a specific, confident, wrong prediction in Chapter 12*. *[cross-bearing: see Ch 4 §5 — the universal join]*

This is that prediction. RBAC does not select its subjects. It **names** them. *"A RoleBinding or ClusterRoleBinding binds a role to subjects. Subjects can be group, users or ServiceAccounts"* [source: k8s-docs-rbac-depth-2026-08-31], and each is identified by a literal string: a username, a group name, a ServiceAccount name. There is no subjectSelector. You cannot write "everything labeled team=payments."

Why is the interesting question, and the documentation does not answer it, so here is the reading that makes sense of it.

A selector is a *query*, evaluated continuously against a set that changes. That is a feature everywhere else in Kubernetes: a Deployment should pick up a Pod that starts matching, a Service should route to an endpoint that becomes ready. The set moving underneath you is the point.

Privilege is the one place where the set moving underneath you is a catastrophe. A grant defined by selector would silently expand the moment somebody added a label, and adding a label is one of the lowest-privilege operations in the entire system, something dozens of people and every controller in the cluster can do. You would have a permission system in which the answer to "who can read the production Secrets?" changed without anyone editing a permission, and could not be answered by reading the permission objects. Naming subjects means the list of who holds a grant is written down in the grant, and it changes only when somebody changes it.

🔒 **Worth Securing:** The general form of that argument shows up throughout security design: *dynamic membership is a feature for routing and a liability for authorization*. Cloud IAM systems that do support attribute-based group membership generally pair it with heavy audit tooling for exactly this reason, because the question "who currently has this?" stops being answerable by reading the policy. [inferred]

The four default roles, and what each cannot do

Every cluster ships with four user-facing roles. Learning what each one *can* do is less useful than learning what each one *cannot*, because that is where the questions are.

cluster-admin — "Allows super-user access to perform any action on any resource" [source: k8s-docs-rbac-2026-08-23]. In a ClusterRoleBinding: everything, everywhere. In a RoleBinding: everything in that namespace, including the namespace object itself.

admin — intended to be granted within a namespace using a RoleBinding. "If used in a RoleBinding, allows read/write access to most resources in a namespace, including the ability to create roles and role bindings within the namespace. This role does not allow write access to resource quota or to the namespace itself" [source: k8s-docs-rbac-depth-2026-08-31]. So **admin** can delegate — it can make new Roles and bindings — but it cannot raise its own ceiling by editing the quota that constrains it.

edit — "Allows read/write access to most objects in a namespace. This role does not allow viewing or modifying roles or role bindings" [source: k8s-docs-rbac-2026-08-23]. That is the exam-relevant boundary: **edit** can run things, **edit** cannot re-grant. But read the rest of the documentation's own sentence, because it is doing something uncomfortable: "However, this role allows accessing Secrets and running Pods as any ServiceAccount in the namespace, so it can be used to gain the API access levels of any ServiceAccount in the namespace" [source: k8s-docs-rbac-depth-2026-08-31].

Sit with that. **edit** cannot edit RBAC, and it does not need to, because it can run a Pod as any ServiceAccount in the namespace, which means it can act with that account's permissions. The formal restriction is real and the practical ceiling is much higher. That is not a bug in **edit**; it is a fact about namespaces, and §4 is going to make it worse.

view — "Allows read-only access to see most objects in a namespace. It does not allow viewing roles or role bindings. This role does not allow viewing Secrets, since reading the contents of Secrets enables access to ServiceAccount credentials in the namespace, which would allow API access as any ServiceAccount in the namespace (a form of privilege escalation)" [source: k8s-docs-rbac-depth-2026-08-31].

view cannot read Secrets, and the documentation tells you exactly why: a Secret can be a credential, so reading Secrets is transitively the ability to *become* somebody. Remember this one specifically. §4 needs it in about ten minutes.

Additive, with no deny

Chapter 11 gave this to you already, in its closing pages, as the tell for this chapter: the permission system you were about to learn has no way to say no. So this is a confirmation, not a reveal.

"Permissions are purely additive (there are no 'deny' rules)." [source: k8s-docs-rbac-2026-08-23]

That is the whole rule, and it is one parenthetical in the documentation.

☆ Fixed Point

RBAC permissions are purely additive. There is no deny rule. The effect of a subject's grants is the union of every rule in every role bound to them. Removing access means removing a grant — there is nothing else to do, and no rule you can write that subtracts.

⚠ Navigational Hazards

Two things follow from the no-deny rule and both catch people.

You cannot carve an exception out of a grant. If somebody holds `edit` in a namespace and you decide they should have everything `edit` gives except the ability to delete Deployments, there is no rule you can add to accomplish that. The only path is to stop granting `edit` and grant a narrower role you construct yourself. Any exam option offering a "deny" rule, a "deny" verb, or a rule that removes a permission is wrong on its face.

And a binding cannot be retargeted. *"After you create a binding, you cannot change the Role or ClusterRole that it refers to"* [source: k8s-docs-rbac-2026-08-23]. If a RoleBinding points at the wrong role, you delete it and create a new one. That is a different operation with different consequences: a window during which the subject holds nothing, and, under a system that reconciles a cluster against a repository, a delete-and-create rather than an update. *[cross-bearing: see Ch 15 §5 — ordering the sync]*

Escalation prevention, and least privilege

One more mechanism, because it explains a class of error message.

"The RBAC API prevents users from escalating privileges by editing roles or role bindings. Because this is enforced at the API level, it applies even when the RBAC authorizer is not in use" [source: k8s-docs-rbac-depth-2026-08-31].

Concretely: you can only create or update a role if you already have all the permissions it contains, at the same scope, or you have been granted the `escalate` verb on roles [source: k8s-docs-rbac-depth-2026-08-31]. And you can only create or update a binding if you already hold all the permissions in the referenced role, or you hold the `bind` verb on it [source: k8s-docs-rbac-depth-2026-08-31]. The example the documentation gives is exact: *"if user-1 does not have the ability to list Secrets cluster-wide, they cannot create a ClusterRole containing that permission"* [source: k8s-docs-rbac-depth-2026-08-31].

Which means `admin`'s ability to create roles in its namespace is bounded by what `admin` itself holds. You cannot bootstrap yourself upward by writing a more generous role.

All of which is in service of one practice. *"Kubernetes RBAC is a key security control to ensure that cluster users and workloads have only the access to resources required to execute their roles"* [source: [k8s-docs-rbac-good-practices-2026-08-31](#)], and the documentation's general rules belong in one place [source: [k8s-docs-rbac-good-practices-2026-08-31](#)]:

- **Assign permissions at the namespace level where possible.** Use `RoleBindings` rather than `ClusterRoleBindings` to give rights only within a specific namespace.
- **Avoid wildcard permissions**, especially to all resources — *"providing wildcard access gives rights not just to all object types that currently exist in the cluster, but also to all object types which are created in the future."*
- **Do not use `cluster-admin` accounts except where specifically needed.**
- **Avoid adding users to the `system:masters` group.** Any member *"bypasses all RBAC rights checks and will always have unrestricted superuser access, which cannot be revoked by removing `RoleBindings` or `ClusterRoleBindings`."*

That last one deserves its own beat. `system:masters` membership is not an RBAC grant, so removing RBAC objects does not remove it. It is also invisible to any audit that reads RBAC objects. And if the cluster uses an authorization webhook, membership in that group bypasses the webhook too: requests from its members are never sent [source: [k8s-docs-rbac-good-practices-2026-08-31](#)].

 **Mnemonic: Role says what. Binding says where. Subject says who.** Three objects, three questions, and every RBAC question you will be asked is one of those three in disguise.

You can check any of this from the command line: `kubectl auth can-i` queries the authorization layer directly to determine whether an action is permitted, and `--as` impersonates another principal, so an administrator can check what somebody else is allowed to do [source: [k8s-docs-authorization-2026-08-31](#)]. It is the fastest way to settle an argument about a grant, and it consults the real authorizer rather than your reading of the YAML.

Taking Your Bearings #1

Five questions covering §1 through §3. Take them before continuing — the sections that follow assume you have this.

1. A cluster administrator enables encryption at rest for API objects stored in `etcd`. On the lifecycle-phase map and the 4Cs layer map, where does this control sit?

A) Distribute phase; Container layer B) Runtime phase (storage); Cloud shading into Cluster layer C) Runtime phase (storage); Container layer D) Deploy phase; Cluster layer

2. A Pod is created in the `payments` namespace with no `serviceAccountName` field. The Pod starts successfully and its application immediately tries to list the Pods in its own namespace. What happens, and why?

A) The request succeeds — a Pod's own namespace is always readable by its ServiceAccount
 B) The Pod fails to start, because no ServiceAccount was named
 C) Authentication fails, because the default ServiceAccount has no token
 D) Authentication succeeds and authorization fails, because the default ServiceAccount holds only API discovery permissions

3. *[retrieval: ch4]* Your team needs a group of engineers to be able to read `PersistentVolume` objects — which are cluster-scoped — as part of a read-only permission set you want to reuse. Which combination of objects does this call for, and what forces the choice?

A) A Role in each namespace where the engineers work, bound by a RoleBinding in each — Roles are the correct object for a team-scoped grant
 B) A ClusterRole, bound by a RoleBinding in each namespace where the engineers work
 C) A ClusterRole, bound by a ClusterRoleBinding — because the cluster-scoped resource forces the ClusterRole, and a permission over a resource that is in no namespace cannot be scoped to one
 D) Either A or B, depending on how many namespaces are involved

4. A RoleBinding in the `staging` namespace currently references the ClusterRole `edit`. You need it to reference `view` instead. What do you do?

A) Update the RoleBinding's `roleRef` field with `kubectl edit`
 B) Delete the RoleBinding and create a new one referencing `view`
 C) Add a second RoleBinding referencing `view`; the more restrictive one wins
 D) Patch the ClusterRole `edit` to remove write verbs

5. Which of the following is true of the default roles?

A) `view` can read Secrets but cannot modify them; `edit` cannot read Secrets at all
 B) `edit` can create Roles and RoleBindings in its namespace; `admin` cannot
 C) `view` cannot read Secrets, and `edit` cannot view or modify roles or role bindings
 D) `cluster-admin` always confers cluster-wide power regardless of which binding object references it

Answers with Explanations:

1. **B.** Encryption at rest for API objects is squarely in the runtime phase's **storage** area; the documentation's runtime/storage list names "enable encryption at rest for API objects" directly [source: k8s-docs-cloud-native-security-2026-08-23]. On the layer map it starts at Cloud, where the 4Cs page recommends the etcd disk be encrypted at rest "*since etcd holds the state of the entire cluster (including Secrets)*" [source: k8s-docs-4cs-cloud-native-security-v1-22-archived-2026-08-31], and extends into Cluster, where the API-object encryption is configured.

A is the position of image signing — both coordinates wrong, and worth recognizing as a pair. **C** has the phase right and the layer wrong. The Container layer covers the image and what runs from it; etcd is

neither. This is the error you make if you reason "it's a Kubernetes thing, so it must be Container." **D** has the layer half-right and the phase wrong. The deploy phase is about restrictions on what can be deployed, who can deploy it, and where [source: k8s-docs-cloud-native-security-2026-08-23]; encryption at rest governs data already in the datastore, which is runtime. This is the error you make if you reason "an operator configures it, so it must be a deploy-time act."

The point of the question is that both coordinates are required and neither implies the other.

2. D. Every namespace gets a default ServiceAccount, and Kubernetes assigns it to any Pod that does not name one [source: k8s-docs-service-accounts-2026-08-23]. The Pod gets a valid, projected, rotating token, so **authentication succeeds**. But the default account gets no permissions beyond API discovery [source: k8s-docs-service-accounts-2026-08-23], so **authorization fails**. **A** is the widespread and wrong assumption that a Pod can read its own namespace. **B** confuses identity assignment with a scheduling or admission failure; the Pod runs fine. **C** is the common misdiagnosis: people see a 403 and go looking at the token. The token is fine. This is the Fixed Point in question form.

3. C. Work it from the boundary. PersistentVolume is cluster-scoped, so it lives in no namespace, so a Role — which *"always sets permissions within a particular namespace"* [source: k8s-docs-rbac-2026-08-23] — has nowhere to put the rule. The **ClusterRole is forced**. Then the second, independent question: where should the grant apply? A resource that is in no namespace cannot have a permission over it scoped to one, so the binding must be a ClusterRoleBinding.

A fails at the first question — it reaches for the object whose scope cannot hold the rule. **B** is the trap, and it is the most instructive wrong answer here. A RoleBinding *can* reference a ClusterRole, and doing so is correct and common [source: k8s-docs-rbac-2026-08-23] — but it binds that ClusterRole into the RoleBinding's namespace, and the PersistentVolume rule has no namespace to land in. The result looks configured and grants nothing. This is the misconception that a ClusterRole bound by a RoleBinding does everything a ClusterRoleBinding would, just in one place. **D** treats a structural constraint as a matter of scale. The number of namespaces is irrelevant; a cluster-scoped resource is not in any of them.

4. B. *"After you create a binding, you cannot change the Role or ClusterRole that it refers to"* [source: k8s-docs-rbac-2026-08-23]. **A** fails; the API rejects the change to roleRef. **C** is the deny-rule misconception wearing a different hat: bindings are additive, so a second binding referencing view would add nothing that edit did not already include and would certainly not restrict anything. **D** would change the meaning of edit for every subject in the cluster, which is a much larger blast radius than the problem calls for, and the API's escalation-prevention rules may well refuse it anyway.

5. C. Both halves are documented: view *"does not allow viewing Secrets"* [source: k8s-docs-rbac-depth-2026-08-31], and edit *"does not allow viewing or modifying roles or role bindings"* [source: k8s-docs-rbac-2026-08-23]. **A** inverts both, and its second half is specifically wrong in a way worth remembering: edit *does* have access to Secrets [source: k8s-docs-rbac-depth-2026-08-31]. **B** inverts the delegation boundary; admin can create roles and bindings in its namespace, edit cannot. **D** is the cluster-admin trap: in a RoleBinding it is scoped to that binding's namespace [source: k8s-docs-rbac-2026-08-23].

How'd You Do?

5/5: move on. §4 will land cleanly.

3–4: re-read the section behind each miss before continuing. This chapter's sections are near-independent arcs, so a gap here does not close itself later — it just stays a gap.

0–2: stop and re-read §3 in full before starting §4. Not a skim: the four objects, the derivation, and the negative space of the four default roles. §4's central argument is built directly on the default roles and the additive rule, and it will read as a list of unrelated cautions without them.

Checkpoint: You've Now Mastered

✓ Placing a control on both the phase map and the layer map ✓ The separation of identity from permission, and what the `default ServiceAccount` proves ✓ Deriving the four RBAC objects from the namespaced/cluster-scoped boundary ✓ Additive permissions, no deny rule, and binding immutability ✓ The negative space of the four default roles

Three sections down. Next: the object you were told, eight chapters ago, did not ship with a lock fitted.

..1 §4 — Secrets Are Not Encrypted

Chapter 4 introduced Secrets and did something unusual. It told you the object was not what it appeared to be, and then explicitly declined to fix it: *the lock is Chapter 12's; this chapter is only telling you the box did not ship with one fitted. [cross-bearing: see Ch 4 §4 — configuration kept outside the image]*

So this section is not going to re-alarm you. You have been appropriately alarmed since Chapter 4. This is the part where we say what to do about it.

What you already hold

Two things, both from earlier chapters, both correct, both worth restating in one line each.

Base64 is encoding. *"Base64 encoding is not an encryption method, it provides no additional confidentiality over plain text"* [source: k8s-docs-secrets-good-practices-2026-08-24]. That is the documentation's own sentence and Chapter 4 gave you the same fact. Nothing has changed. Anyone who can read the encoded string can decode it, and there is no key involved because there is no cipher involved.

Secrets get some handling ConfigMaps do not. A Secret is only sent to a node if a Pod on that node requires it; the kubelet stores its copy in a temporary filesystem (tmpfs) rather than on durable storage; and once the Pods that depend on it are deleted, the local copies are removed [source: k8s-docs-secret-risks-2026-08-31]. Chapter 11 told you about the tmpfs part when it walked volume types, and said

explicitly that you were being handed half an argument. *[cross-bearing: see Ch 11 §1 — three lifetimes, and the volumes that have them]* We will finish that argument shortly.

The claim, and the three ways in

The whole problem, in the documentation's own words:

"Kubernetes Secrets are, by default, stored unencrypted in the API server's underlying data store (etcd). Anyone with API access can retrieve or modify a Secret, and so can anyone with access to etcd. Additionally, anyone who is authorized to create a Pod in a namespace can use that access to read any Secret in that namespace; this includes indirect access such as the ability to create a Deployment." [source: k8s-docs-secret-2026-08-23]

Three sentences, three routes to the same object.

Route one: API access. Anyone the authorization layer permits to get a Secret reads it. That is the obvious one and the one an RBAC review catches. It is broader than `get`, though, and the documentation is explicit: *"List and watch access also effectively allow for users to reveal the Secret contents. For example, when a List response is returned (for example, via `kubectl get secrets -A -o yaml`), the response includes the contents of all Secrets"* [source: k8s-docs-rbac-good-practices-2026-08-31]. A grant of `list` on Secrets is a grant of `read` on every Secret in scope, and it does not look like one.

Route two: etcd access. Anyone who can read the etcd data store reads every Secret in the cluster, because that is where they are, unencrypted. This is where Chapter 8's backup guidance comes due. When Chapter 8 told you to back up etcd and to keep the backup encrypted, the encryption clause was not general caution — *[cross-bearing: see Ch 8 §7 — the one backup that matters]* **a backup of etcd is a copy of every Secret in the cluster, in the clear, sitting in whatever storage you put backups in.** Every password, every token, every TLS private key, in one file, protected by exactly whatever protects your backups. That is why the clause is in the sentence.

Route three: the ability to create a Pod. And this is the one nobody counts as a permission to read secrets.

The RBAC good-practices page states it in full: *"Permission to create workloads (either Pods, or workload resources that manage Pods) in a namespace implicitly grants access to many other resources in that namespace, such as Secrets, ConfigMaps, and PersistentVolumes that can be mounted in Pods. Additionally, since Pods can run as any ServiceAccount, granting permission to create workloads also implicitly grants the API access levels of any service account in that namespace"* [source: k8s-docs-rbac-good-practices-2026-08-31].

The mechanism is not subtle once you see it. A Pod spec can mount any Secret in its namespace. If you can create a Pod, you can create a Pod that mounts the Secret and prints it. You never asked for `get secrets`. You do not have `get secrets`. You have `create pods`, which is a permission everybody who deploys anything holds, and it is transitively equivalent.

And the second half is worse: because a Pod can name any ServiceAccount in the namespace, whoever can create Pods can act as any identity in that namespace. That is where §3's uncomfortable sentence about `edit` came from, and now you can see the full shape of it.

☆ Fixed Point

A Secret is stored unencrypted in etcd by default. Encryption at rest is opt-in, and it is a cluster-operator decision made in the API server's configuration — not a field you can set on a manifest.

☆ Fixed Point

Anyone authorized to create a Pod in a namespace can read every Secret in that namespace — including indirectly, by creating a Deployment, and including by running as any ServiceAccount that namespace holds.

⚠ Navigational Hazards

This is the most consequential practical fact in the chapter, and the most invisible.

An RBAC audit that greps for `get secrets` and finds nobody holding it will report the Secrets as safe. It is reading the wrong permission. In a namespace where a dozen engineers hold permission to deploy, a dozen engineers can read every Secret, and no line in any Role says so.

The documentation draws the only correct conclusion: *"namespaces should be used to separate resources requiring different levels of trust or tenancy. It is still considered best practice to follow least privilege principles and assign the minimum set of permissions, but **boundaries within a namespace should be considered weak**"* [source: k8s-docs-rbac-good-practices-2026-08-31].

That last clause is the design guidance. If two things must not read each other's Secrets, they go in different namespaces. There is no arrangement of Roles inside a single namespace that achieves it, because the escalation path does not run through the Secret permissions at all.

Recall from §3, one section ago, that `view` cannot read Secrets precisely because reading a Secret can mean becoming a ServiceAccount [source: k8s-docs-rbac-depth-2026-08-31]. Put that next to what you just read and the design intent is coherent throughout: Kubernetes treats "can read Secrets" and "can act as anybody in this namespace" as nearly the same permission, and it is right to.

Encryption at rest, and what it buys

So enable encryption at rest. This is the lock finally being fitted to the strongbox, and it pays to be exact about which door it closes.

"All of the APIs in Kubernetes that let you write persistent API resource data support at-rest encryption. For example, you can enable at-rest encryption for Secrets. This at-rest encryption is additional to any system-level encryption for the etcd cluster or for the filesystem(s) on hosts where you are running the kube-apiserver" [source: k8s-docs-encrypt-data-2026-08-31]. And the default is unambiguous: "By

default, the API server stores plain-text representations of resources into etcd, with no at-rest encryption" [source: k8s-docs-encrypt-data-2026-08-31].

The mechanism is a configuration file. *"The kube-apiserver process accepts an argument --encryption-provider-config that specifies a path to a configuration file. The contents of that file, if you specify one, control how Kubernetes API data is encrypted in etcd. If you are running the kube-apiserver without the --encryption-provider-config command line argument, you do not have encryption at rest enabled"* [source: k8s-docs-encrypt-data-2026-08-31]. The file holds an `EncryptionConfiguration` object in the `apiserver.config.k8s.io/v1` API group, naming the API kinds to encrypt and an ordered list of providers [source: k8s-docs-encrypt-data-2026-08-31].

One trap in the checking: even with the flag set, if the referenced file lists the identity provider first, you still do not have encryption enabled, because *"the default identity provider does not provide any confidentiality protection"* [source: k8s-docs-encrypt-data-2026-08-31]. A configuration file that looks configured is not the same as encryption that is on.

Note the scope carefully. *"This task covers encryption for resource data stored using the Kubernetes API. If you want to encrypt data in filesystems that are mounted into containers, you instead need to either: use a storage integration that provides encrypted volumes [or] encrypt the data within your own application"* [source: k8s-docs-encrypt-data-2026-08-31].

So the honest accounting of what encryption at rest gives you. It protects the object **as written to etcd**, which closes route two, the etcd-access and etcd-backup route, and closes it well. It does nothing whatsoever about routes one and three, because a caller the API server has authorized gets the object decrypted, in full, as normal. The lock is on the box; it says nothing about who is handed the box. That is not a shortcoming; it is the definition of the control. But an engineer who enables encryption at rest and believes the Secrets problem is now solved has closed one of three doors and stopped counting.

🔒 **Closer Look:** This is also the moment to draw a line you will need in a later chapter. **Encryption at rest and encryption in transit are separate decisions with separate mechanisms.** At rest is about bytes sitting in storage. In transit is about bytes moving across a network: TLS between components, and, at the workload layer, mutual TLS between services. Enabling one says nothing about the other. [cross-bearing: see Ch 17 §5 — a network that knows what it's carrying]

The four hardening steps

Chapter 4 enumerated four things and deferred all of them. Here they are, in the documentation's own ordering:

In order to safely use Secrets, take at least the following steps: **enable Encryption at Rest for Secrets; enable or configure RBAC rules with least-privilege access to Secrets; restrict Secret access to specific containers; consider using external Secret store providers.** [source: k8s-docs-secret-2026-08-23]

Encryption at rest we have just covered.

Least-privilege RBAC on Secrets specifically. The good-practices page gets granular: *"Components: Restrict `watch` or `List` access to only the most privileged, system-level components. Only grant `get` access for Secrets if the component's normal behavior requires it. Humans: Restrict `get`, `watch`, or `List` access to Secrets. Only allow cluster administrators to access `etcd`. This includes read-only access"* [source: k8s-docs-secrets-good-practices-2026-08-24]. And it adds an `etcd` management practice: consider wiping or shredding the durable storage `etcd` used once it is no longer in use, and configure encrypted TLS between `etcd` instances to protect Secret data in transit between them [source: k8s-docs-secrets-good-practices-2026-08-24].

Restrict access to specific containers. *"If you are defining multiple containers in a Pod, and only one of those containers needs access to a Secret, define the volume mount or environment variable configuration so that the other containers do not have access to that Secret"* [source: k8s-docs-secrets-good-practices-2026-08-24]. A Pod is not a trust boundary between its own containers unless you make it one. A sidecar you did not write and do not fully control sits inside the Pod's boundary; whether it can read the database password is a choice you make in the manifest.

External secret store providers. *"You can use third-party Secrets store providers to keep your confidential data outside your cluster and then configure Pods to access that information. The Kubernetes Secrets Store CSI Driver is a DaemonSet that lets the kubelet retrieve Secrets from external stores, and mount the Secrets as a volume into specific Pods that you authorize to access the data"* [source: k8s-docs-secrets-good-practices-2026-08-24]. Note the shape of that: a CSI driver [cross-bearing: see Ch 11 §5 — who actually provides the storage] delivered as a DaemonSet [cross-bearing: see Ch 6 §7 — one per node, and work that ends], doing a job through two mechanisms you already know.

And one more, aimed at you rather than the cluster: *"If you configure a Secret through a manifest, with the secret data encoded as base64, sharing this file or checking it in to a source repository means the secret is available to everyone who can read the manifest"* [source: k8s-docs-secrets-good-practices-2026-08-24]. Base64 in a Git repository is a plaintext password in a Git repository with a costume on.

File over environment variable

Chapter 11 handed you half an argument and told you it was half. Time to say which half and supply the rest.

The half you hold: the kubelet stores a Secret's local copy in tmpfs, memory-backed rather than durable storage, and removes it when the Pods depending on it are deleted [source: k8s-docs-secret-risks-2026-08-31]. That is a real property of the mount path.

The half you were owed: a mounted Secret stays current, and an environment variable does not. When a Secret is mounted as a volume, updates propagate to the Pod automatically, with one documented exception: *"A container using a Secret as a subPath volume mount does not receive automated Secret updates"* [source: k8s-docs-secret-risks-2026-08-31]. An environment variable, by contrast, is read at container start and is then a fixed string in the process's environment for the life of that process. Rotate the Secret and the running container keeps using the old value until something restarts it.

Add the third-container point from the hardening list — per-container mount control is expressible either way, but a volume mount is the more natural place to express it [source: k8s-docs-secrets-good-practices-2026-08-24] — and you have the argument.

What the argument is *not*: a claim that environment variables leak somewhere volumes do not. You will read that in a great deal of prep material and it is plausible, but the Kubernetes documentation does not say it. What the documentation does say is symmetrical between the two mechanisms: *"Applications still need to protect the value of confidential information after reading it from an environment variable or volume. For example, your application must avoid logging the secret data in the clear or transmitting it to an untrusted party"* [source: k8s-docs-secrets-good-practices-2026-08-24]. That is a warning about your application's handling, and it applies to both.

So: prefer the file mount, on the grounds of rotation and tmpfs, and be precise about why. It is a better recommendation for being accurately argued.

Secret types

Secrets carry a type, and the type tells consumers what shape of data to expect [source: k8s-docs-secret-2026-08-23]:

Built-in type	Usage
Opaque	arbitrary user-defined data (the default)
kubernetes.io/service-account-token	ServiceAccount token — the legacy long-lived credential
kubernetes.io/dockercfg	serialized ~/.dockercfg file
kubernetes.io/dockerconfigjson	serialized ~/.docker/config.json file
kubernetes.io/basic-auth	credentials for basic authentication
kubernetes.io/ssh-auth	credentials for SSH authentication
kubernetes.io/tls	data for a TLS client or server
bootstrap.kubernetes.io/token	bootstrap token data

Two of those connect to material you already have. `kubernetes.io/dockerconfigjson` holds a serialized `~/.docker/config.json` [source: k8s-docs-secret-2026-08-23], which is precisely what a workload needs for *"authenticating to a private image registry using an imagePullSecret"* [source: k8s-docs-service-accounts-2026-08-23]. Chapter 2 introduced pull secrets and flagged that they are a genuine security boundary rather than a convenience feature [cross-bearing: see Ch 2 §6 — *when Kubernetes pulls, and when it doesn't*], and §7 will pick that up as the last checkpoint in the supply chain. And `kubernetes.io/service-account-token` is the legacy credential §2 warned you about.

A closing note for the next chapter: a Pod that references a Secret which does not exist does not get a running container. The kubelet cannot assemble the container's configuration, so the container sits waiting rather than starting and crashing — a recognizably different shape from a Pod that runs and then

dies, and one of the first things to check when a Pod never comes up. *[cross-bearing: see Ch 13 §2 — pods that never start]*

..1 §5 — What a Pod May Do to Its Node

Chapter 10 drew you two axes and filled in one of them. NetworkPolicy governs which Pod may open a connection to which Pod: reachability, workload to workload. The other axis, it said, was what a Pod may do to the machine it is running on, and it was Chapter 12's. Chapter 11 pointed at the same place twice: once when it warned you about `hostPath` and called this *the workload-to-host boundary problem, and why an entire security apparatus exists to police it*, and once when it insisted that a volume access mode is not a permission system.

This is that axis, and that apparatus. It is also the boundary the chapter's title is actually about. Everything else here arranges who may ask the API for what; this section is about what a workload can do to the deck it is standing on.

You have in fact already seen the word. Chapter 11 quoted the storage documentation verbatim: *"it does not prevent an application from writing to the mounted volume if the Pod's securityContext allows write access."* You met `securityContext` inside a sourced quotation, doing exactly the job this section is about, with a pointer attached. Now it gets defined.

The default position

Start with the uncomfortable baseline, because everything here is a departure from it.

A container is a process in a set of Linux namespaces and cgroups, sharing the host's kernel *[cross-bearing: see Ch 2 §1 — what a container actually is]*. That gives it an isolated *view*: its own filesystem, its own process table, its own network stack. It does not give it a different kernel or, by default, a different user database.

The Kubernetes documentation's own recommendation says so by implication: *"When you deploy a workload in Kubernetes, use the Pod specification to restrict that workload from running as the root user on the node"* [source: k8s-docs-linux-kernel-security-constraints-2026-08-31]. Note the phrasing — *root user on the node*, not "root in the container." The documentation treats those as the same thing, and the feature that separates them exists precisely because they otherwise are not separate: `hostUsers: false` lets you *"run containers as root users in the user namespace, but as non-root users in the host namespace on the node"* [source: k8s-docs-linux-kernel-security-constraints-2026-08-31]. Absent that, a process running as root inside a container is running as **UID 0 against the host's kernel**. The isolation is real and it is not the same thing as being unprivileged.

The field

"A security context defines privilege and access control settings for a Pod or Container" [source: k8s-docs-security-context-2026-08-31]. The settings include, per the documentation's own list [source: k8s-docs-security-context-2026-08-31]:

- **Discretionary Access Control** — permission to access an object, like a file, based on user ID (UID) and group ID (GID).
- **SELinux** — objects assigned security labels.
- **Running as privileged or unprivileged.**
- **Linux Capabilities** — "Give a process some privileges, but not all the privileges of the root user."
- **AppArmor** — "Use program profiles to restrict the capabilities of individual programs."
- **Seccomp** — "Filter a process's system calls."
- **allowPrivilegeEscalation** — "Controls whether a process can gain more privileges than its parent process. This bool directly controls whether the `no_new_privs` flag gets set on the container process."
- **readOnlyRootFilesystem** — "Mounts the container's root filesystem as read-only."

Two scopes, and which wins

`securityContext` appears in two places, and the relationship between them is exactly the sort of detail an exam is built on.

At Pod scope: "To specify security settings for a Pod, include the `securityContext` field in the Pod specification. The `securityContext` field is a `PodSecurityContext` object. The security settings that you specify for a Pod apply to all Containers in the Pod" [source: k8s-docs-security-context-2026-08-31].

At container scope: "Security settings that you specify for a Container apply only to the individual Container, and they override settings made at the Pod level when there is overlap. Container settings do not affect the Pod's Volumes" [source: k8s-docs-security-context-2026-08-31].

📌 **Snag: The container's setting wins where both are set.** Pod scope is the default for everything in the Pod; container scope is a per-container override. And the tail of that sentence matters too: container settings do not affect the Pod's Volumes, so volume ownership behavior (`fsGroup`, for instance) is a Pod-level concern only.

Concretely at Pod scope: `runAsUser` specifies that all processes in any container run with a given user ID; `runAsGroup` specifies the primary group ID, and "if this field is omitted, the primary group ID of the containers will be `root(0)`"; `fsGroup` makes all container processes part of a supplementary group and sets the owner group for mounted volumes and files created in them [source: k8s-docs-security-context-2026-08-31]. Notice what omitting `runAsGroup` does: it does not mean "no group," it means group 0.

Capabilities

"With Linux capabilities, you can grant certain privileges to a process without granting all the privileges of the root user" [source: k8s-docs-security-context-2026-08-31]. Linux divides root's authority into categories, so a process can hold the specific privilege it needs — binding a low port, changing file ownership — without holding the rest.

You add or drop them via the `capabilities` field in a container's `securityContext`. *"Without specifying a capabilities field, containers receive default capabilities"*, which the container runtime decides [source: k8s-docs-security-context-2026-08-31]. And a syntax detail that catches everyone once: *"Linux capability constants have the form CAP_XXX. But when you list capabilities in your container manifest, you must omit the CAP_ portion of the constant. For example, to add CAP_SYS_TIME, include SYS_TIME in your list of capabilities"* [source: k8s-docs-security-context-2026-08-31].

seccomp and AppArmor

Two Linux security modules, doing related but distinct jobs. The documentation names three common kernel features Kubernetes lets you configure: **seccomp** (filter which system calls a process can make), **AppArmor** (restrict the access privileges of individual programs), and **SELinux** (assign security labels to objects for more manageable policy enforcement) [source: k8s-docs-linux-kernel-security-constraints-2026-08-31]. Whether a feature is available depends on the node's operating system enabling it in the kernel [source: k8s-docs-linux-kernel-security-constraints-2026-08-31].

seccomp operates at the syscall layer. *"Each capability has a set of system calls (syscalls) that a process can make. seccomp lets you restrict these individual syscalls. It can be used to sandbox the privileges of a process, restricting the calls it is able to make from userspace into the kernel"* [source: k8s-docs-linux-kernel-security-constraints-2026-08-31]. Container runtimes usually include a default seccomp profile, and Kubernetes can apply profiles loaded onto a node to your Pods and containers [source: k8s-docs-linux-kernel-security-constraints-2026-08-31]. In the manifest: `seccompProfile` with a type of `RuntimeDefault`, `Unconfined`, or `Localhost` [source: k8s-docs-security-context-2026-08-31].

The documentation's own advice about writing custom profiles is refreshingly blunt: *"seccomp is a low-level security configuration that you should only configure yourself if you require fine-grained control over Linux syscalls"*, with named risks — configurations breaking during application updates, attackers still exploiting vulnerabilities through allowed syscalls, and profile management becoming challenging at scale. The recommendation: *"Use the default seccomp profile that's bundled with your container runtime. If you need a more isolated environment, consider using a sandbox, such as gVisor"* [source: k8s-docs-linux-kernel-security-constraints-2026-08-31].

AppArmor operates on programs and resources rather than syscalls. *"AppArmor is a Linux kernel security module that supplements the standard Linux user and group based permissions to confine programs to a limited set of resources... It is configured through profiles tuned to allow the access needed by a specific program or container, such as Linux capabilities, network access, and file permissions. Each profile can be*

run in either enforcing mode, which blocks access to disallowed resources, or complain mode, which only reports violations" [source: k8s-docs-linux-kernel-security-constraints-2026-08-31].

Enforcing versus complain. Hold that shape. §6 is about to do the same thing at a different layer, and recognizing the pattern will save you memorizing it twice.

The two modules differ in how they identify what they are protecting: *"AppArmor uses profiles to define access to resources. SELinux uses policies that apply to specific labels", and "In AppArmor, you define resources using file paths. SELinux uses the index node (inode) of a resource to identify the resource"* [source: k8s-docs-linux-kernel-security-constraints-2026-08-31]. A node's OS typically ships one or the other [source: k8s-docs-linux-kernel-security-constraints-2026-08-31].

Privileged containers

And now the field that undoes all of it.

"Any container in a Pod can enable Privileged mode if you set the `privileged: true` field in the `securityContext` field for the container" [source: k8s-docs-linux-kernel-security-constraints-2026-08-31]. What that does:

"Privileged containers override or undo many other hardening settings such as the applied seccomp profile, AppArmor profile, or SELinux constraints. Privileged containers are given all Linux capabilities, including capabilities that they don't require. For example, a root user in a privileged container might be able to use the `CAP_SYS_ADMIN` and `CAP_NET_ADMIN` capabilities on the node, bypassing the runtime seccomp configuration and other restrictions." [source: k8s-docs-linux-kernel-security-constraints-2026-08-31]

Specifically: privileged containers run as the `Unconfined` seccomp profile, overriding any profile you specified; they ignore any applied AppArmor profiles; and they run as the `unconfined_t` SELinux domain [source: k8s-docs-linux-kernel-security-constraints-2026-08-31].

⚠ Navigational Hazards

`privileged: true` is not one more setting alongside the others. It is the setting that turns the others off.

The Pod Security Standards define their most permissive level, `privileged`, as *"an absence of restrictions"* — a policy that *"is defined by an absence of restrictions"* and under which a Pod *"is able to bypass typical container isolation mechanisms (for example, access the node's host network)"* [source: k8s-docs-pod-security-standards-2026-08-23]. That is the documentation's own characterization and it needs no editorializing.

The guidance is correspondingly direct: *"In most cases, you should avoid using privileged containers, and instead grant the specific capabilities required by your container using the `capabilities` field in the `securityContext` field. Only use privileged mode if you have a capability that you can't grant with the `securityContext`"* [source: k8s-docs-linux-kernel-security-constraints-2026-08-31].

And one consequence that belongs to §4 as much as here: **a container running with `privileged: true` can access all Secrets on that node** [source: k8s-docs-secret-risks-2026-08-31]. Not its namespace's Secrets. The node's. Every Secret that the kubelet has mounted for any Pod scheduled there.

🔒 **Worth Securing:** The RBAC good-practices page connects this back to who is allowed to create workloads at all: *"Users who can run privileged Pods can use that access to gain node access and potentially to further elevate their privileges"* [source: k8s-docs-rbac-good-practices-2026-08-31]. And it names the mitigation: where you do not fully trust a principal to create suitably secure Pods, enforce the Baseline or Restricted Pod Security Standard [source: k8s-docs-rbac-good-practices-2026-08-31]. Which is §6, and is why §6 comes next.

The apparatus, not the field

The section's promise was an apparatus, so the rest of it: the controls that sit beside `securityContext` rather than inside it.

Stronger isolation at the runtime layer. If the shared-kernel model is not acceptable for a workload, you change the runtime rather than tuning the fields. The 4Cs page's Container-layer guidance is to *"select container runtime classes that provide stronger isolation"* [source: k8s-docs-4cs-cloud-native-security-v1-22-archived-2026-08-31], and the seccomp guidance points the same way: *"If you need a more isolated environment, consider using a sandbox, such as gVisor"* [source: k8s-docs-linux-kernel-security-constraints-2026-08-31]. That is Chapter 2's material and it is not re-taught here. [cross-bearing: see Ch 2 §7 — not all isolation is equal]

Isolation at the placement layer. Where a workload runs is a security control. The runtime/compute list says to *"partition workloads across different nodes to improve isolation"* [source: k8s-docs-cloud-native-security-2026-08-23], and the RBAC good-practices page is more specific: limit the number of nodes running powerful Pods, and *"avoid running powerful pods alongside untrusted or publicly-exposed ones."*

Consider using *Taints and Toleration*, *NodeAffinity*, or *PodAntiAffinity* to ensure pods don't run alongside *untrusted or less-trusted Pods*" [source: k8s-docs-rbac-good-practices-2026-08-31]. Every mechanism in that sentence is Chapter 7's. [cross-bearing: see Ch 7 §4 — when the berth refuses you] Chapter 7 taught them as scheduling controls and told you they would come back as isolation controls. Here they are, unchanged, doing a different job.

Avoid needing root at all. "Configuring the kernel security features on this page provides fine-grained control over the actions that processes in your cluster can take, but managing these configurations can be challenging at scale. Running containers as non-root, or in user namespaces if you need root privileges, helps to reduce the chance that you'll need to enforce your configured kernel security capabilities" [source: k8s-docs-linux-kernel-security-constraints-2026-08-31]. The documentation's summarized recommendation: "Unless necessary, run Linux workloads as non-root by setting specific user and group IDs in your Pod manifest and by specifying `runAsNonRoot: true`" [source: k8s-docs-linux-kernel-security-constraints-2026-08-31]. There is also `hostUsers: false`, though the documentation notes it "is still in early stages of development and might not have the level of support that you need" [source: k8s-docs-linux-kernel-security-constraints-2026-08-31].

Watch what you assign. One practical warning: "Ensure that the user or group that you assign to the workload has the permissions required for the application to function correctly. Changing the user or group to one that doesn't have the correct permissions could lead to file access issues or failed operations" [source: k8s-docs-linux-kernel-security-constraints-2026-08-31]. Setting `runAsUser: 1000` on an image built expecting root is a fast way to produce a container that starts and immediately fails on a permission error.

☆ Fixed Point

securityContext governs the workload-to-host axis. NetworkPolicy governs the workload-to-workload axis. They are separate systems on separate objects at separate layers. They fail independently, and neither substitutes for the other: a Pod perfectly isolated by NetworkPolicy can still be `privileged: true` and read every Secret on its node.

Chapter 10 told you those were two axes. This section is the proof.

..1 §6 — Three Levels, Three Modes

Chapter 8 made you an unusually specific promise. It said that when you met Pod Security Admission four chapters later, *you will not be learning a new kind of thing. You will be learning one instance of the third gate.* [cross-bearing: see Ch 8 §2 — three gates and a logbook]

So discharge that before anything else, because the derivation is the whole of what makes this section easy.

You already know that every request to the API server passes three gates in order: authentication, then authorization, then **admission**. Admission is the gate that inspects the object itself: after we know who you are and that you are allowed to do this in general, does this specific object pass the rules? Admission controllers are what run there, some compiled in and some reached by webhook.

Pod Security Admission (PSA) is a built-in admission controller. That is the entire architectural statement. *"Kubernetes offers a built-in Pod Security admission controller to enforce the Pod Security Standards"* [source: k8s-docs-pod-security-admission-2026-08-31], stable since v1.25 [source: k8s-docs-pod-security-admission-2026-08-31]. Nothing new is happening. A Pod object arrives at the third gate, a controller examines it against a policy, and the gate does something about it.

What you need beyond that is the policy — three of them — and what "does something" means — three options. Hence the section title.

The three levels

The Pod Security Standards (PSS) define three policies covering the security spectrum. They are **cumulative** and range from highly permissive to highly restrictive [source: k8s-docs-pod-security-standards-2026-08-23].

Profile	Description
Privileged	<i>"Unrestricted policy, providing the widest possible level of permissions. This policy allows for known privilege escalations."</i>
Baseline	<i>"Minimally restrictive policy which prevents known privilege escalations. Allows the default (minimally specified) Pod configuration."</i>
Restricted	<i>"Heavily restricted policy, following current Pod hardening best practices."</i>

[source: k8s-docs-pod-security-standards-2026-08-23]

Privileged is *"purposely-open, and entirely unrestricted"*, aimed at system- and infrastructure-level workloads managed by privileged, trusted users [source: k8s-docs-pod-security-standards-2026-08-23]. Your CNI plugin's DaemonSet lives here, and legitimately so; it needs to configure the node's network.

Baseline is *"aimed at ease of adoption for common containerized workloads while preventing known privilege escalations"*, targeted at application operators and developers of non-critical applications [source: k8s-docs-pod-security-standards-2026-08-23]. Crucially, it allows the default minimally-specified Pod: a Pod spec with no securityContext at all passes Baseline.

Restricted is *"aimed at enforcing current Pod hardening best practices, at the expense of some compatibility"*, targeted at operators and developers of security-critical applications and at lower-trust users [source: k8s-docs-pod-security-standards-2026-08-23]. The "expense of some compatibility" is not a hedge: Restricted requires `runAsNonRoot: true` [source: k8s-docs-pod-security-standards-profiles-2026-08-31], and an image built to run as root cannot satisfy that without being changed.

The levels are the §5 fields with names on them. What each actually checks:

THE LEVELS – what gets checked

securityContext field	privileged	baseline	restricted
privileged: true	allowed	X forbidden	X forbidden
hostNetwork/hostPID/hostIPC	allowed	X forbidden	X forbidden
hostPath volumes	allowed	X forbidden	X forbidden
hostPort	allowed	restricted	restricted
capabilities.add	allowed	known list	only NET_BIND_SERVICE
capabilities.drop	–	–	must include ALL
seccompProfile.type	allowed	not Unconfined	RuntimeDefault or Localhost
appArmorProfile.type	allowed	Runtime Default or Localhost	Runtime Default or Localhost
allowPrivilegeEscalation	allowed	–	must be false
runAsNonRoot	–	–	must be true
runAsUser	any	any	must be non-zero or unset
volume types	any	any	safe list only (configMap, csi, emptyDir, secret, projected, pvc, ...)

cumulative – PERMITTED PODS: privileged $\supset$ baseline $\supset$ restricted
(restricted imposes every baseline requirement,
plus the rows below the line, so it admits fewer Pods)

THE MODES – what happens when a check fails

enforce $\rightarrow$ the Pod is REJECTED
audit $\rightarrow$ recorded in the audit log; the Pod runs
warn $\rightarrow$ user-facing warning; the Pod runs

APPLIED PER NAMESPACE, BY LABEL

pod-security.kubernetes.io/<MODE>: <LEVEL>

e.g. pod-security.kubernetes.io/enforce: baseline
pod-security.kubernetes.io/warn: restricted
pod-security.kubernetes.io/audit: restricted

▲ all three, on one namespace, at two different levels.
This is not a contradiction. It is a migration.

The rows in that figure are the Baseline and Restricted controls this chapter grades on, not the complete published list. Baseline forbids privileged, host namespaces (hostNetwork, hostPID, hostIPC), hostPath volumes, and unconfined seccomp; restricts hostPort, added capabilities to a known list, AppArmor and SELinux to approved values, /proc mount type to Default, and sysctls to a safe list. It also forbids Windows host access via hostProcess and, from v1.34, disallows direct host connections from probes and lifecycle hooks — two controls the figure omits because neither is Linux-workload material. Restricted adds every Baseline requirement plus: volumes limited to a safe list; allowPrivilegeEscalation: false; runAsNonRoot: true; runAsUser non-zero or unset; seccomp explicitly RuntimeDefault Or Localhost; and capabilities dropping ALL with only NET_BIND_SERVICE addable back [source: k8s-docs-pod-security-standards-profiles-2026-08-31].

🔍 **Closer Look:** The Restricted capability rule rewards a precise reading: drop must include ALL, and add is limited to undefined/nil or NET_BIND_SERVICE [source: k8s-docs-pod-security-standards-profiles-2026-08-31]. NET_BIND_SERVICE is the exception because binding a port below 1024 is a common legitimate reason a container wants a scrap of root's authority, and refusing it would break otherwise-conformant images for no security gain.

There is no fourth level between Privileged and Baseline, and the FAQ explains why: *"The three profiles defined here have a clear linear progression from most secure (Restricted) to least secure (Privileged), and cover a broad set of workloads. Privileges required above the Baseline policy are typically very application specific, so we do not offer a standard profile in this niche"* [source: k8s-docs-pod-security-standards-profiles-2026-08-31].

The three modes

The Standards are enforced by the built-in Pod Security Admission controller, which applies a policy **per namespace** via labels of the form pod-security.kubernetes.io/<MODE>: <LEVEL>, where MODE is **enforce** (violations are rejected), **audit** (violations are recorded in the audit log), or **warn** (violations trigger a user-facing warning), and LEVEL is privileged, baseline, or restricted [source: k8s-docs-pod-security-standards-2026-08-23].

💀 **Dead Reckoning:** Three levels: privileged, baseline, restricted. Three modes: enforce, audit, warn. They are applied to a namespace as labels of the form pod-security.kubernetes.io/<MODE>: <LEVEL>. A namespace may carry all three modes at once, at three different levels. The level names the policy to check against. The mode names what happens if the check fails.

☆ Fixed Point

Three levels × three modes, applied per namespace by label. The level says *what* is checked. The mode says *what happens* when the check fails. They are independent axes. Confusing them is the most likely wrong answer in this chapter.

That independence is not a curiosity. It is the entire operational design. A cluster that wants to move to restricted sets warn: restricted and audit: restricted while keeping enforce: baseline. Nothing

breaks. Developers see warnings when they submit non-conformant Pods and the audit log accumulates a list of exactly what would fail. When the list empties, you flip `enforce` to `restricted` and you already know it will hold. That is a migration path, expressible only because level and mode are orthogonal.

🧠 **Mnemonic: Level = what. Mode = when it hurts.** If an exam option describes `enforce` as a level or `restricted` as a mode, it is wrong at the grammar, before you get to the meaning.

The namespace as a control surface

Notice what the label form implies. The unit of Pod Security policy is the **Namespace object**, which you have known since Chapter 4 as a scope for names and a boundary for namespaced resources. *[cross-bearing: see Ch 4 §3 — where a name lives]* Here it acquires a new job: it is the thing policy attaches to.

Which means the ability to label a namespace is a security-relevant permission. The RBAC good-practices page says so directly: *"Users who can perform patch operations on Namespace objects (through a namespaced RoleBinding to a Role with that access) can modify labels on that namespace. In clusters where Pod Security Admission is used, this may allow a user to configure the namespace for a more permissive policy than intended by the administrators"* [source: k8s-docs-rbac-good-practices-2026-08-31].

Patching a label is about as innocuous-looking as an operation gets. Here it is the ability to lower the security policy of everything deployed into that namespace. And the same paragraph notes the parallel case for NetworkPolicy: labels indirectly granting access an administrator did not intend [source: k8s-docs-rbac-good-practices-2026-08-31]. Two systems, same lever.

PodSecurityPolicy

One clause, because you will meet it.

PodSecurityPolicy was the predecessor, and Pod Security Admission is what supersedes it. It is not the mechanism a current cluster uses, which makes its *absence* the load-bearing fact. If a question offers PodSecurityPolicy as a current mechanism, that is the distractor.

Two failure shapes, for later

Two things to carry forward.

A Pod refused by `enforce` **never starts**. It is not a Pod that starts and crashes; it is an object the API server declined to accept. That is a different diagnostic shape from anything in the crash-loop family, and it shows up at a different point in the triage flow. *[cross-bearing: see Ch 13 §2 — pods that never start]*

And a debug container is a container. In a namespace enforcing `restricted`, a debugging tool that tries to inject a container with elevated privileges can be refused by admission, which is a surprising thing to

discover in the middle of an incident. *[cross-bearing: see Ch 16 §3 — getting inside, and adding what isn't there]*

🕒 Taking Your Bearings #2

Five questions covering §4 through §6.

1. ⚠ This one is intentionally difficult. Struggle is the point — it is the single most consequential practical question in the chapter, and getting it wrong here is much cheaper than getting it wrong on a cluster.

You are auditing a namespace called `payments`. You enumerate every `Role`, `ClusterRole`, `RoleBinding` and `ClusterRoleBinding` that touches it. No subject holds `get`, `list`, or `watch` on secrets. Eight engineers hold a custom `deployer` `Role` — `create`, `update`, `patch` and `delete` on `deployments`, `Pods` and `services`, and nothing at all on secrets — bound by `RoleBindings` so they can ship. The Secrets in the namespace include the production database password.

Who can read that password, and what is the correct remediation?

A) Nobody — the audit is correct, since no subject holds any read verb on Secrets B) Only cluster administrators, via `etcd`; enable encryption at rest to close it C) All eight engineers, because permission to create workloads implicitly grants access to Secrets in the namespace; the remediation is to move the Secret and its consumers into a separate namespace D) All eight engineers, but only because `deployer` grants `delete` on `Pods`; removing that one verb closes the path

2. A cluster operator enables encryption at rest for Secrets with an `EncryptionConfiguration`. Which of the following is now true?

A) A user with `get secrets` permission receives an encrypted blob they must decrypt client-side B) An `etcd` backup no longer contains readable Secret values, but an authorized API caller still receives the Secret in the clear C) Secrets mounted into containers are now encrypted on the container filesystem D) The kubelet's `tmpfs` copy on each node is now encrypted too, since it is derived from the encrypted object

3. A Pod spec sets `securityContext.runAsUser: 2000` at Pod scope. One of its two containers sets `securityContext.runAsUser: 3000` at container scope. What UID does each container's processes run as?

A) Both run as 2000; container-level settings are ignored for user IDs B) Both run as 3000; the most restrictive setting applies to the whole Pod C) The container with the override runs as 3000; the other runs as 2000 D) The Pod fails admission, because conflicting `runAsUser` values are rejected

4. A namespace carries these three labels:

```
pod-security.kubernetes.io/enforce: baseline
pod-security.kubernetes.io/warn: restricted
pod-security.kubernetes.io/audit: restricted
```

A developer submits a Pod that sets no securityContext at all and mounts no host paths. What happens?

A) The Pod is rejected, because it does not meet restricted B) The Pod is rejected, because enforce: baseline requires an explicit securityContext C) The Pod is created, and the developer sees a warning that it does not meet restricted; a violation is also recorded in the audit log D) The Pod is created silently, because enforce is the only mode that produces any output

5. [retrieval: ch8] Explain, in terms of a mechanism you already know, where Pod Security Admission sits in the processing of an API request — and name one consequence of that position.

A) It runs before authorization, so a Pod can be refused for its contents before the API server has decided whether the requester may create Pods at all B) It runs during authorization, alongside RBAC, so a Pod can be refused either for who submitted it or for what it contains C) It runs at the admission gate — the third and last of the three gates — so it inspects the Pod object itself, and a rejected Pod never enters the cluster at all D) It runs after the Pod is scheduled but before the kubelet starts the container, so a rejected Pod appears briefly in Pending

Answers with Explanations:

1. **C.** All eight, and not through any Secret permission. *"Permission to create workloads (either Pods, or workload resources that manage Pods) in a namespace implicitly grants access to many other resources in that namespace, such as Secrets, ConfigMaps, and PersistentVolumes that can be mounted in Pods"* [source: k8s-docs-rbac-good-practices-2026-08-31]. Anyone who can create a Pod can create one that mounts the Secret and prints it. The documentation's own remediation is the one in C: *"namespaces should be used to separate resources requiring different levels of trust or tenancy... boundaries within a namespace should be considered weak"* [source: k8s-docs-rbac-good-practices-2026-08-31].

A is the audit finding, and it is the whole point of the question. The audit is not sloppy — it enumerated every object correctly and read the permission it was looking for accurately. It is reading a permission that is not the relevant one. **B** identifies a real route (etcd) and a real control (encryption at rest) but misses the route that is actually open here. Encryption at rest does nothing about an authorized API caller [source: k8s-docs-encrypt-data-2026-08-31]. **D** gets the population right and the mechanism exactly backwards. The escalation runs through **create**, not **delete**: a Pod spec can mount any Secret in its namespace, so the operative permission is the one that lets you bring a new Pod into existence. Strip **delete** from **deployer** and all eight engineers can still read the password by tomorrow morning. This distractor exists because people instinctively police the destructive verbs.

2. **B.** Encryption at rest protects the object as written to etcd [source: k8s-docs-encrypt-data-2026-08-31]. It closes the etcd and etcd-backup route completely and does nothing to the API route. **A** inverts the

boundary; decryption happens server-side, transparently. **C** is the scope error the documentation explicitly heads off: encrypting data in filesystems mounted into containers requires a storage integration providing encrypted volumes, or application-level encryption [source: k8s-docs-encrypt-data-2026-08-31]. **D** invents a chain of custody. The kubelet receives the decrypted Secret and places its copy in tmpfs [source: k8s-docs-secret-risks-2026-08-31]; tmpfs is a separate protection with a separate rationale (it is memory-backed and removed with the Pod), not a downstream effect of at-rest encryption.

3. C. *"Security settings that you specify for a Container apply only to the individual Container, and they override settings made at the Pod level when there is overlap"* [source: k8s-docs-security-context-2026-08-31]. Pod scope is the default; container scope overrides it, per container. **A** inverts precedence. **B** invents a "most restrictive wins" rule that does not exist, and note 3000 is not obviously more restrictive than 2000 anyway. **D** invents a validation that does not exist; this configuration is entirely ordinary.

4. C. Level and mode are independent axes, and a namespace may carry all three modes at different levels [source: k8s-docs-pod-security-standards-2026-08-23]. enforce: baseline is the gate that can reject, and *"Baseline... allows the default (minimally specified) Pod configuration"* [source: k8s-docs-pod-security-standards-2026-08-23], so a Pod with no securityContext passes it. warn: restricted and audit: restricted both fire, because the Pod does not meet Restricted (which requires runAsNonRoot: true, allowPrivilegeEscalation: false, capabilities dropping ALL, and an explicit seccomp profile [source: k8s-docs-pod-security-standards-profiles-2026-08-31]), but neither of those modes rejects anything. **A** reads warn's level as though warn enforced. **B** inverts Baseline's defining property — allowing the minimally specified Pod is precisely what distinguishes it from Restricted; it is the one level that asks for nothing. **D** is wrong about warn and audit being silent, which is precisely their purpose.

5. C. Pod Security Admission is *"a built-in Pod Security admission controller"* [source: k8s-docs-pod-security-admission-2026-08-31], and admission is the third and last of the three gates. The consequence that matters: it inspects the object, so it can make decisions no authorization rule could. RBAC can say "this subject may create Pods," but only admission can say "not *this* Pod." And a rejected Pod is never created, so it has no phase, no events, and no logs.

A puts admission before authorization. It is a coherent-sounding design and it is not this one — the order is authentication, authorization, admission, and it matters: admission is only ever reached for a request that has already been authorized, so a subject with no create pods grant never gets far enough for a Pod Security check to be relevant. **B** collapses two gates into one. RBAC and PSA answer different questions at different stages; authorization reasons about verb, resource and namespace, and cannot see the object's field values at all. **D** is the plausible-sounding wrong answer: it describes something that would look like a Pod stuck in Pending, which is a genuinely different failure and one you will diagnose in a later chapter.

How'd You Do?

5/5: take the session break and start §7 fresh.

3–4: re-read the section behind each miss now, while the material is warm. Question 1 is the one to be least comfortable missing.

0–2: re-read §5 before you go on to re-read §6. The three levels are a list of `securityContext` fields with names attached, and they are unmemorable — genuinely unmemorable, not just difficult — if the fields themselves are not yet familiar. Fields first, then levels.

Checkpoint: You've Now Mastered

✓ The three exposure paths to a Secret, and which control closes which ✓ What encryption at rest protects and what it leaves open ✓ `securityContext` at two scopes, and which one wins ✓ Three levels against three modes, and why they are independent ✓ Pod Security Admission as one instance of a gate you already knew

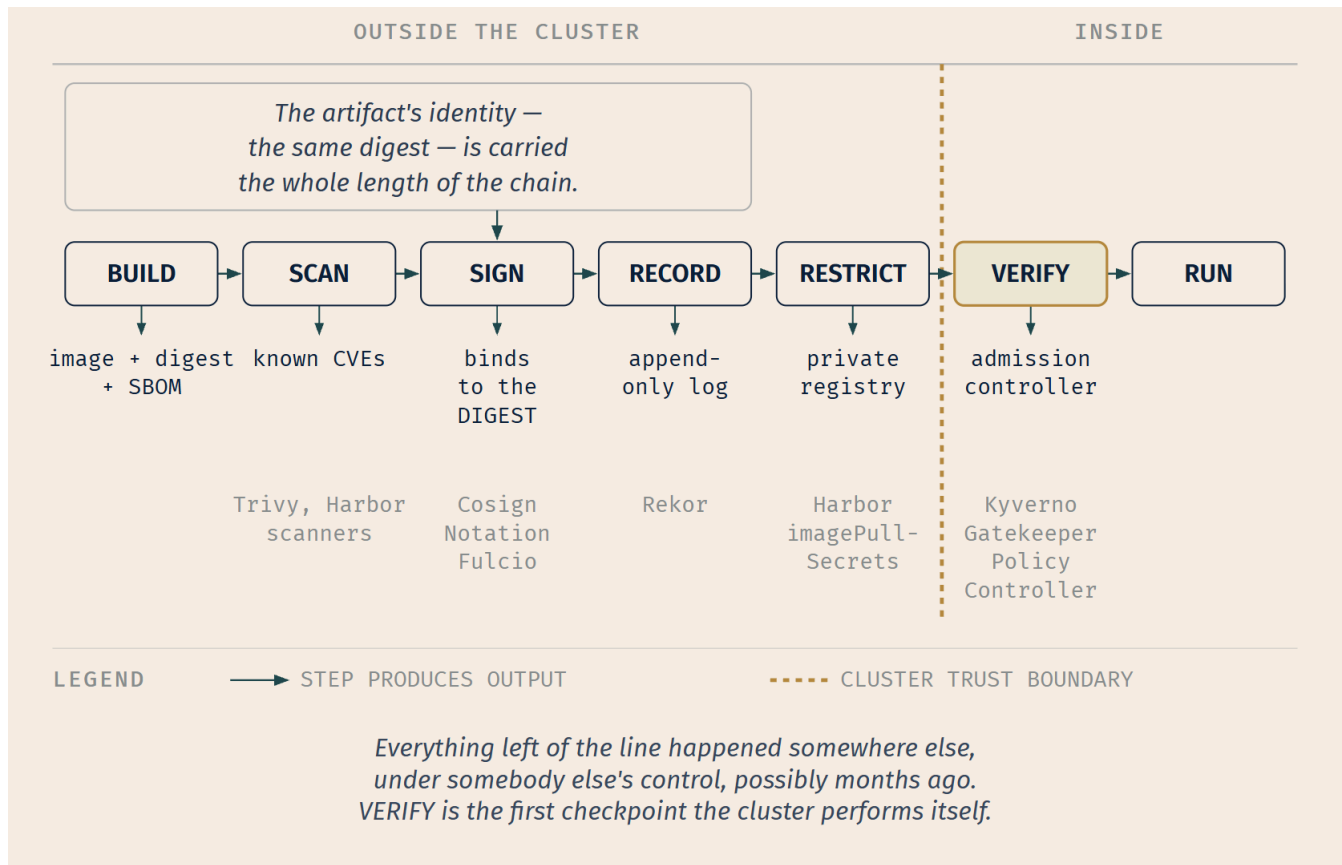
That is the API and the workload. **This is the natural place to stop if you are splitting this chapter across two sessions.** What follows leaves the cluster entirely.

§7 — Trusting What You Ship

Everything so far has been about a running cluster: who may call the API, what a workload may do once it is there. This section is about the question that comes first and gets asked least: **is the thing you are running the thing you built?**

Chapter 2 raised this twice and deferred both times. It called reproducible layers *the hinge on which supply-chain verification swings*, and it flagged `imagePullSecrets` as *a genuine security boundary rather than a convenience feature*. Both arguments were made there. This section completes them rather than repeating them.

There is a way to organize this material as a roster of projects, and it is the wrong way. There are five or six well-known tools here and they are not alternatives to each other; they occupy different positions in a sequence. So we will walk the sequence and name the tools where they sit.



That vertical line is the point of the figure. Every step but one happens outside the cluster, in a build system you may not administer, at a time that is not now. Admission is the cluster's gangway: the one place it gets to inspect the cargo before it comes aboard. Which is why §8 follows this section rather than preceding it.

Scan

"As part of an image build step, you should scan your containers for known vulnerabilities" [source: k8s-docs-4cs-cloud-native-security-v1-22-archived-2026-08-31], and the distribute-phase guidance says the same: scan container images and other artifacts for known vulnerabilities [source: k8s-docs-cloud-native-security-2026-08-23]. A **CVE** — Common Vulnerabilities and Exposures — is an identifier attached to a publicly disclosed vulnerability, and a scanner's job is to enumerate what an image contains and report which of those components have known vulnerabilities against them.

The Code layer's parallel recommendation belongs alongside it: *"It is a good practice to regularly scan your application's third party libraries for known security vulnerabilities"* [source: k8s-docs-4cs-cloud-native-security-v1-22-archived-2026-08-31]. Same activity, different layer. Your dependencies and the image's base OS packages are both other people's code.

Sign

A scan tells you what is *in* an image. A signature tells you *who produced it*, and the two are independent. A signed image can be full of known vulnerabilities; a clean image can come from anywhere.

"Sign container images to maintain a system of trust for the content of your containers" [source: k8s-docs-4cs-cloud-native-security-v1-22-archived-2026-08-31].

Sigstore is the project most of this ecosystem now runs through. It *"empowers software developers and consumers to securely sign and verify software artifacts such as release files, container images, binaries, software bills of materials (SBOMs), and more"*, operating as a free, public-good service under the Open Source Security Foundation [source: sigstore-overview-2026-08-23]. Its components divide the job [source: sigstore-overview-2026-08-23]:

- **Cosign** — the client tool for signing and verifying artifacts, including container images.
- **Fulcio** — the code-signing certificate authority, which issues **short-lived certificates bound to a verified identity** rather than long-lived keys.
- **Rekor** — an **immutable, append-only transparency log** that records signing events for public audit.
- **Policy Controller** — enforces signature verification policies within Kubernetes, as an admission controller.

Keyless signing is what ties them together, and it removes the worst problem in signing. Conventionally, signing means holding a private key, which means protecting a private key forever, which means that the compromise of that key retroactively invalidates everything it ever signed. Sigstore's flow instead: a Cosign client creates an **ephemeral** key pair and requests a certificate from Fulcio using an OpenID Connect identity token; Fulcio validates the token and issues a short-lived certificate linking the public key to the verified identity; the artifact is signed; **the private key is discarded after a single signing**; and the signature and certificate are recorded in Rekor [source: sigstore-overview-2026-08-23].

The key exists for seconds and is then gone. What persists is the certificate binding a public key to an identity, and the log entry proving when. There is no long-lived secret to steal.

Notary Project is the other signing standard you will meet: *"a set of specifications and tools intended to provide a cross-industry standard for securing software supply chains"*, focused on *"signing and validating software artifacts, ensure they have not been tampered with and provide security policies to determine which validated artifacts are allowed to be used in your systems"*, with **Notation** as its CLI [source: notary-project-signing-digest-2026-08-31]. It is CNCF incubating [source: notary-project-signing-digest-2026-08-31].

What a signature covers — and this is the one

Chapter 2 taught you that a tag is a mutable pointer and a digest is identity, and it taught it as a matter of build hygiene: pin your digests so you know what you deployed.

It was not hygiene. Here is the Notary Project's own statement of what happens at signing time:

"Notation resolves the tag to the digest before signing if a tag is used to identify the container image."

"Always reference and use the image digest instead of a tag since digest is immutable." [source: notary-project-signing-digest-2026-08-31]

Read the first sentence carefully. Even if you hand the tool a tag, it does not sign the tag. It resolves the tag to a digest and signs *that*. The signature has nothing to say about the tag at all.

☆ Fixed Point

A signature binds to a digest, not a tag. Chapter 2 taught you that a tag is a mutable pointer and a digest is content identity, and framed it as build hygiene. It was not hygiene. It is the reason a signature means anything at all.

Think about what a tag-covering signature would even assert. "I attest that whatever `myapp:v2` happens to point at is trustworthy" — a statement about a name, which anyone with push access can repoint at any moment, retroactively extending your attestation to bytes you have never seen. It would be worse than no signature, because it would look like one.

A digest is a cryptographic hash of content. Signing it says: *these exact bytes, and no others*. That is the only form of the statement that survives contact with a mutable registry. [cross-bearing: see Ch 2 §3 — registries, tags, and digests]

🗝 **Mnemonic: You sign the bytes, not the label on the box.**

Record

"Use validation mechanisms such as digital certificates for supply chain assurance" [source: k8s-docs-cloud-native-security-2026-08-23], and the transparency log is what makes those mechanisms auditable rather than merely present. Rekor is *"an immutable, append-only transparency log that records signing events for public audit and verification"* [source: sigstore-overview-2026-08-23]. Append-only means entries cannot be removed or altered after the fact, so "was this artifact signed, by whom, and when?" is a question with a durable answer, including for artifacts signed with keys that no longer exist — which under keyless signing is all of them.

Attestation, provenance, and SBOMs

Attestation generalizes signing. A signature says "I vouch for these bytes." An attestation says "I vouch for this *claim* about these bytes": that they were built by a particular pipeline from a particular commit, that they passed a particular test suite, that they contain a particular set of components.

in-toto is the framework for the build-process half. It *"is designed to ensure the integrity of a software product from initiation to end-user installation. It does so by making it transparent to the user what steps were performed, by whom and in what order"* [source: in-toto-overview-2026-08-31]. That sentence —

what steps, by whom, in what order — is in substance what **provenance** means: not just what the artifact is, but the verifiable record of how it came to be. SPDX names the same idea directly, listing *"Provenance and Integrity: Tracking the origin and history of components, including checksums and cryptographic hashes"* among what its standard covers [source: sbom-standards-spdx-cyclonedx-2026-08-31]. in-toto is a CNCF graduated project [source: in-toto-overview-2026-08-31].

An **SBOM** — Software Bill of Materials — is the components half. A bill of materials is a standardized record of what a software artifact is made of. The two dominant standards are **SPDX (Software Package Data Exchange)**, from the Linux Foundation, *"an open standard designed to facilitate the communication of Bill of Materials (BOM) information across diverse domains, including software, artificial intelligence (AI), datasets, and system components"*, covering metadata for packages, files and snippets, licensing information, and provenance and integrity [source: sbom-standards-spdx-cyclonedx-2026-08-31]; and **CycloneDX**, from OWASP (the Open Worldwide Application Security Project), *"a full-stack Bill of Materials (BOM) standard that provides advanced supply chain capabilities for cyber risk reduction"* [source: sbom-standards-spdx-cyclonedx-2026-08-31].

And an SBOM is itself an artifact that can be signed; Sigstore names SBOMs explicitly among the artifact types it handles [source: sigstore-overview-2026-08-23]. Which closes a loop: a signed SBOM is a verifiable claim about what is inside a verifiable image, and the two together are what lets you answer "are we affected by this newly disclosed vulnerability?" without rebuilding anything.

TUF — The Update Framework — belongs here too, though it operates one level up. It is *"a framework for securing software update systems"* that *"maintains the security of software update systems, providing protection even against attackers that compromise the repository or signing keys"* [source: tuf-overview-2026-08-31]. The attacks it names are the ones a naive signature check misses entirely [source: tuf-overview-2026-08-31]:

- *"An attacker keeps giving you the same file, so you never realize there is an update."*
- *"An attacker gives you an older, insecure version of a file that you already have and tricks you into thinking it's newer."*
- *"An attacker gives you a newer version of a file you have but it's still not the newest one."*
- *"An attacker compromises the key used to sign these files. Now you download a file that is properly signed, but is still malicious."*

Every one of those involves a *correctly signed* artifact. That is what makes TUF interesting: signing answers "did this come from who I think?" and answers nothing about freshness, ordering, or key compromise. TUF is CNCF graduated [source: tuf-overview-2026-08-31].

Restrict

The distribute-phase list ends with a step that is not cryptographic at all: *"restrict access to artifacts — place container images in a private registry that only allows authorized clients to pull images"* [source: k8s-docs-cloud-native-security-2026-08-23].

Harbor is the CNCF graduated registry built for this. Its stated mission is *"to be the trusted cloud native repository for Kubernetes"*, and it *"is an open source registry that secures artifacts with policies and role-based access control, ensures images are scanned and free from vulnerabilities, and signs images as trusted"* [source: harbor-overview-2026-08-31]. Its feature list names security and vulnerability analysis, content signing and validation, and identity integration with role-based access control [source: harbor-overview-2026-08-31]. Which is to say it does scan, sign and restrict in one place — a useful thing to know when a question offers it as an answer.

And the cluster's side of the restriction is a Secret. An `imagePullSecrets` entry holds registry credentials in a Secret of type `kubernetes.io/dockerconfigjson`, which is a serialized `~/.docker/config.json` [source: k8s-docs-secret-2026-08-23], and one of the documented `ServiceAccount` use cases is *"authenticating to a private image registry using an imagePullSecret"* [source: k8s-docs-service-accounts-2026-08-23]. Chapter 2 told you this was a security boundary rather than a convenience. Now you can see the whole boundary: the registry refuses unauthorized pulls, the credential to pull is a Secret, and the rules about who can read Secrets from §4 apply to it in full.

Verify

The last checkpoint, and the first one the cluster performs itself. The deploy phase says: *"You can enforce measures from the distribute phase, such as verifying the cryptographic identity of container image artifacts"* [source: k8s-docs-cloud-native-security-2026-08-23]. The 4Cs Container layer says: *"Image Signing and Enforcement"* — signing and enforcement, two words, two activities [source: k8s-docs-4cs-cloud-native-security-v1-22-archived-2026-08-31].

Signing without verification is theater. A signature that nothing checks is a file in a registry. The check happens at admission: Sigstore's Policy Controller *"enforces signature verification policies within Kubernetes as an admission controller"* [source: sigstore-overview-2026-08-23], and Kyverno can *"verify container images and metadata for software supply chain security"* [source: kyverno-overview-2026-08-23].

Which is the third gate again, doing another job. Same position, different question. And it hands us straight into §8.

🔒 **Worth Securing:** The one-sentence version of this whole section: **a signature tells you where something came from, a scan tells you what is wrong with it, an SBOM tells you what is in it, provenance tells you how it was made, and none of the four does any of the others' work.** Every real supply-chain question is asking which of those you need.

..1 §8 — Rules That Watch

Section 6 gave you a controller that answers one question extremely well. Pod Security Admission checks a Pod against three fixed policies, at the admission gate, and does so without configuration beyond a namespace label. That is its strength and its limit: it is not extensible. If your organization's rule is "every

Pod must carry a cost-center label," or "no image may come from outside our registry," or "every Namespace gets a default-deny NetworkPolicy on creation," PSS has nothing to say about any of it.

A **policy engine** answers arbitrary questions at that same gate.

The CNCF glossary gives the underlying idea a name: "**Policy as Code is the practice of storing the definition of policies as one or more files in machine-readable and processable form**" [source: cncf-glossary-policy-as-code-2026-08-31]. Codified policy gets consistent automated enforcement instead of manual review, and, because the files live in version control, a change history you can audit and revert [source: cncf-glossary-policy-as-code-2026-08-31].

The organizing question for this section is not *which engine* but **when does the rule run?**

At admission

Kyverno — Greek for "govern" — is *"a cloud native policy engine"* originally built for Kubernetes and now usable outside clusters as a unified policy language. It *"allows platform engineers to automate security, compliance, and best practices validation and deliver secure self-service to application teams"* [source: kyverno-overview-2026-08-23].

Its policies can *"validate, mutate, generate, or clean up Kubernetes resources; verify container images and metadata for software supply chain security; and be applied as a Kubernetes admission controller (webhook) or as a CLI-based scanner"* [source: kyverno-overview-2026-08-23]. Policies are written in YAML using declarative rules and **CEL (Common Expression Language)**, managed as Kubernetes resources, and version-controlled with Git [source: kyverno-overview-2026-08-23].

Those four verbs are worth separating, because they are not variations on one thing:

- **Validate** — refuse an object that breaks a rule. This is what PSS does, generalized.
- **Mutate** — change an object as it passes. Add a missing label, inject a sidecar, set a default.
- **Generate** — create *other* objects in response. A new Namespace appears; a default NetworkPolicy and a ResourceQuota appear with it.
- **Clean up** — remove objects meeting a condition.

Validate and mutate are the two answers Chapter 8 told you the third gate has [*cross-bearing: see Ch 8 §2 — three gates and a logbook*]: two gates ask about you and answer yes or no, while the third asks about the request itself and can also answer *yes, in modified form*. Those two answers have names — a **validating** admission webhook and a **mutating** one. **A policy engine is one.** It is what people actually install into that extension point, and it is the cleanest possible payoff for having learned the distinction. [*cross-bearing: see Ch 17 §4 — every place Kubernetes lets you in*]

OPA — Open Policy Agent — is the other widely used engine, a CNCF graduated project, with its **Gatekeeper** admission controller for Kubernetes, expressing policy in the **Rego** language [source: kyverno-overview-2026-08-23]. The Pod Security Standards page itself names the third-party enforcement options: Kubewarden, Kyverno, and OPA Gatekeeper [source: k8s-docs-pod-security-

standards-profiles-2026-08-31], alongside a framing worth keeping — *"Decoupling policy definition from policy instantiation allows for a common understanding and consistent language of policies across clusters, independent of the underlying enforcement mechanism"* [source: k8s-docs-pod-security-standards-profiles-2026-08-31]. The Standards are a *definition*; PSA and the policy engines are *instantiations* of it. That is why a third-party engine can enforce the same three levels PSA does.

The practical difference between Kyverno and OPA Gatekeeper, at the level a KCNA candidate needs: Kyverno's policies are Kubernetes resources written in YAML and CEL; OPA's are written in Rego, a purpose-built policy language. Both sit at the admission gate. Choosing between them is a team decision about language and ecosystem, not a security-model decision.

At runtime

Everything above happens before an object exists. **Falco** answers a different question at a different time.

Falco *"is a cloud native security tool that provides runtime security across hosts, containers, Kubernetes, and cloud environments. It is designed to detect and alert on abnormal behavior and potential security threats in real-time"*, and it is a CNCF graduated project, originally created by Sysdig [source: falco-overview-2026-08-23].

How it works: *"Falco observes Linux kernel events (system calls) and data from plugins, enriches them with metadata from the container runtime and Kubernetes, evaluates the event stream against a rules engine, and emits real-time alerts when rules detect violations"* [source: falco-overview-2026-08-23].

Its default detections read like a list of things you now know to worry about [source: falco-overview-2026-08-23]:

- privilege escalation via privileged containers
- namespace manipulation
- unauthorized modifications to sensitive directories such as `/etc` or `/usr/bin`
- suspicious network connections
- shell or SSH binary execution inside containers
- unauthorized file ownership or permission changes
- symlink creation

Look at the first one against §5. A `privileged: true` Pod that got past admission — because the namespace was `enforce: privileged`, or because it predates the policy, or because someone granted an exception — is invisible to every control in §6. Falco sees the behavior after the fact.

⚠ Navigational Hazards

Falco detects. It does not prevent. It observes, evaluates, and emits alerts [source: falco-overview-2026-08-23]. There is no step in that sequence that blocks anything.

This is the watch at the masthead, and it pays to be precise about what that is worth. A control that reports is not a lesser version of a control that refuses; it answers a question refusal cannot. Admission-time controls can only reason about the object as submitted; they know nothing about what the process does at hour six. Runtime detection knows nothing about the object and everything about the behavior. You want both, and an exam question distinguishing them is asking whether you know they are different questions rather than different qualities of answer.

The two positions, side by side

	admission time	runtime
What it examines	the object, as submitted	process behavior, as it happens
When	before the object exists	after it is running
What it can do	refuse, or change	observe, and report
Fixed question	Pod Security Admission	—
Arbitrary question	Kyverno, OPA/Gatekeeper	Falco

Three ways of covering the same territory: PSS answers a fixed question at the gate; a policy engine answers an arbitrary question at the same gate; Falco answers a different question entirely, later, and answers it by telling you rather than by stopping you.

🧭 Taking Your Bearings #3

Five questions covering §7 and §8, plus one deliberate long-spacing item back to §3.

1. [retrieval: ch2] A build pipeline signs the image reference `registry.example.com/myapp:v2`. Three weeks later, an attacker with push access to the registry pushes different content to that same tag. A cluster verifies signatures at admission. What happens when the new content is pulled, and why?

A) Verification passes, because the tag `v2` was signed and it is still `v2` B) Verification fails, because the signature bound to the digest of the original content, and the new content has a different digest C) Verification passes only if the attacker also re-signed; otherwise the image is pulled unverified D) Verification is not possible, because signatures cannot be applied to tagged references at all

2. Put these supply-chain checkpoints in the order they occur, and identify which is the first one performed by the cluster itself: scan, verify, sign, restrict, record.

A) sign → scan → record → verify → restrict; the cluster performs *restrict* B) scan → sign → record → restrict → verify; the cluster performs *verify* C) scan → record → sign → verify → restrict; the cluster performs *verify* D) scan → sign → record → restrict → verify; the cluster performs *restrict*

3. Your cluster already enforces the `baseline` Pod Security Standard in every namespace. Your organization now requires that every Pod carry a `cost-center` label, and that any Pod missing one be rejected. What do you reach for, and why can't Pod Security Admission do it?

A) Add a fourth Pod Security level; PSA supports custom levels via ConfigMap B) A policy engine such as Kyverno or OPA Gatekeeper, because PSA evaluates only the fixed Pod Security Standards and cannot express an arbitrary rule C) Falco, because it evaluates arbitrary rules against cluster objects D) An RBAC rule denying `create` on Pods without the label; PSA is for `securityContext` only

4. A container in your cluster has been running for six hours. During hour six, a process inside it spawns a shell and writes to `/etc`. Which tool in this chapter is positioned to tell you, and what specifically will it do about it?

A) Pod Security Admission — it will terminate the Pod B) A Kyverno validating policy — it will refuse the write, because the policy is evaluated against every action in the cluster C) Falco — it will observe the kernel events, evaluate them against its rules, and emit an alert; it will not stop the process D) The restricted Pod Security Standard — it will prevent the write, because `readOnlyRootFilesystem` is required

5. A team lead asks you to remove one specific permission from an engineer who currently holds the `edit` ClusterRole in a namespace: they should keep everything `edit` provides, except the ability to delete Deployments. What is your answer?

A) Add a Role with a deny rule for `delete` on `deployments`, bound in the same namespace B) Patch the existing RoleBinding to exclude that verb C) There is no way to subtract from a grant; stop granting `edit` and grant a custom Role built from the permissions they should have D) Add a second RoleBinding to a more restrictive role; the intersection of the two applies

Answers with Explanations:

1. **B.** *"Notation resolves the tag to the digest before signing if a tag is used to identify the container image"* [source: `notary-project-signing-digest-2026-08-31`] — the signature never covered the tag, only the digest of the content the tag pointed at. New content has a new digest, so the existing signature does not verify against it, and admission refuses. **A** is the misconception the Fixed Point exists to kill, and it is the reason the documentation says *"Always reference and use the image digest instead of a tag since digest is immutable"* [source: `notary-project-signing-digest-2026-08-31`]. **C** gets the outcome accidentally right for the wrong reason and then adds a false claim: an unverified image is not "pulled unverified" under an enforcing policy, it is refused. **D** contradicts the quoted sentence; you may hand the tool a tag, it just does not sign one.

2. B. Scan and sign happen at build time; the signature is recorded in a transparency log; the artifact goes to a registry that restricts who may pull it; and the cluster verifies at admission before running it [source: k8s-docs-cloud-native-security-2026-08-23; sigstore-overview-2026-08-23]. **Verify** is the first step the cluster itself performs; everything before it happened elsewhere, possibly months earlier, under someone else's control.

A signs before scanning, which would attest to content you have not examined, and then names the wrong cluster-performed step. **C** records before signing, which is backwards — the log records signing events [source: sigstore-overview-2026-08-23], so there is nothing to record yet. **D** is the one to take slowly, because **the order is correct** and only the second half is wrong. Restricting who may pull is the registry's job, and the registry is not the cluster; the cluster's own first act on this artifact is verification at the admission gate. If you picked D, your model of the sequence is fine and your model of the boundary is not — which is exactly the distinction the figure's vertical line draws.

3. B. PSA enforces the Pod Security Standards and nothing else [source: k8s-docs-pod-security-admission-2026-08-31]; its three levels are fixed definitions about `securityContext` and related fields, with no mechanism for expressing an organizational rule about labels. Kyverno validates arbitrary conditions at the admission gate [source: kyverno-overview-2026-08-23], as does OPA Gatekeeper. **A** invents an extension mechanism PSA does not have. **C** puts a runtime detection tool at the admission gate; Falco emits alerts, it does not refuse objects [source: falco-overview-2026-08-23]. **D** misunderstands both systems: RBAC has no deny rules, and it authorizes on verb and resource, not on an object's field values. RBAC cannot see the label.

4. C. Falco *"observes Linux kernel events (system calls)... enriches them with metadata from the container runtime and Kubernetes, evaluates the event stream against a rules engine, and emits real-time alerts"*, and its default detections explicitly include unauthorized modifications to sensitive directories such as `/etc` and shell execution inside containers [source: falco-overview-2026-08-23]. It reports; it does not block.

A reaches six hours past admission's window and then invents a capability admission does not have. Admission accepts or refuses objects; it does not terminate running Pods. **B** is the sharper version of the same error, and worth naming precisely: a policy engine's validating rules are evaluated against **API requests at the admission gate**, not against process behavior. A process writing a file generates no API request, so there is no admission event for the policy to fire on. Kyverno can also run as a CLI-based scanner [source: kyverno-overview-2026-08-23], but that scans manifests, not syscalls. **D** is the closest to defensible and still wrong on two counts: `readOnlyRootFilesystem` is a `securityContext` field the Restricted level does not in fact mandate [source: k8s-docs-pod-security-standards-profiles-2026-08-31], and in any case a level constrains a Pod *at admission*, so it could have prevented the Pod from being created but has no role once it is running.

5. C. *"Permissions are purely additive (there are no 'deny' rules)"* [source: k8s-docs-rbac-2026-08-23]. There is no rule that subtracts, so the only path is to stop granting the role that includes the unwanted permission and grant something narrower. **A** invents deny rules. **B** would not work even if `roleRef` were mutable, which it is not [source: k8s-docs-rbac-2026-08-23], because a binding grants a role wholesale; it

does not filter it. **D** is the most seductive because it sounds like how firewalls work: additive means *union*, not intersection, so a second binding can only add.

How'd You Do?

5/5: §9 will be a confirmation rather than a surprise, which is how it is meant to read.

3–4: review the miss and continue. §9 introduces no new object and depends on no single fact from this checkpoint.

0–2: re-read §7 before §8 — §8's whole framing is "the same gate, a different question," and it needs §7's picture of where that gate sits. If the miss was question 5, re-read §3's closing pages on the additive rule before you start §9, because §9 is an argument about that rule and nothing else.

Checkpoint: You've Now Mastered

✓ The supply chain as a sequence of checkpoints, and what each one buys ✓ What a signature binds to, and why a tag would be worthless ✓ Where a policy engine sits and what it does that PSS cannot ✓ Admission-time versus runtime, and why detection is a different question rather than a weaker answer

One section left, and it is the one Chapter 10 and Chapter 11 have both been pointing at.

☀ §9 — Additive, Never Deny

Nothing new here. This section introduces no object, no field, and no tool. It makes an argument, and the argument is the thing the last two chapters have been building toward.

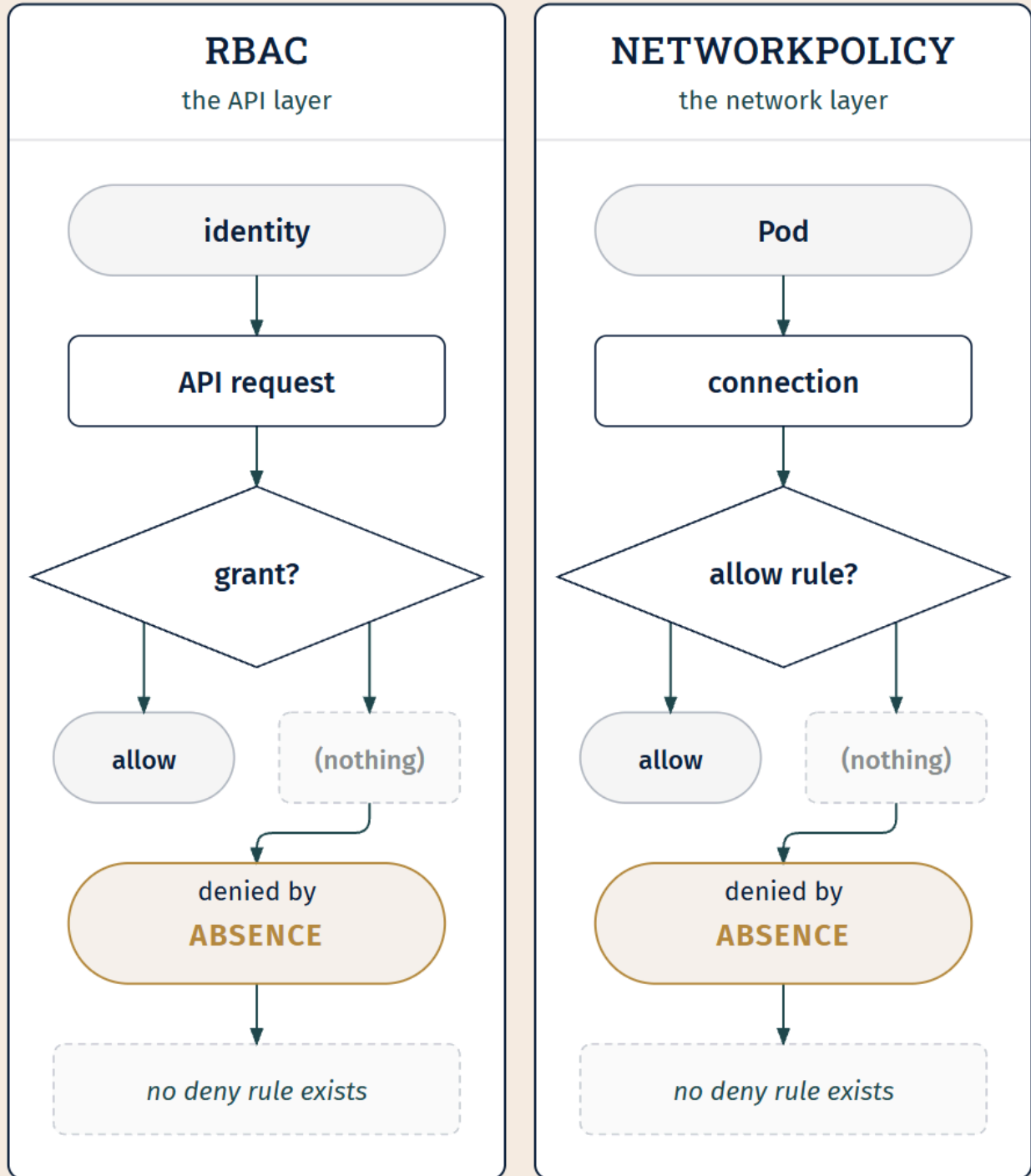
The two systems

RBAC governs what an identity may ask the API server for. Its objects are Roles and bindings. It sits at the second of the three gates. It was designed for the problem of multi-tenant API access. *"Permissions are purely additive (there are no 'deny' rules)"* [source: k8s-docs-rbac-2026-08-23].

NetworkPolicy governs which Pod may open a connection to which Pod. Its objects are policies with selectors. It is implemented by a CNI plugin, not the API server. It was designed for the problem of lateral movement inside a flat network. It is additive: policies allow, and the effect of several is the union of what they allow. *[cross-bearing: see Ch 10 §6 — allowing, never denying]*

Those are two systems with essentially nothing in common. Different layers, different objects, different implementers, different problems, different decades. Chapter 10 told you NetworkPolicy allows and

never denies, and told you to hold on to the phrasing about **subtraction**. Chapter 11 told you the permission system in this chapter would have no way to say no. Section 3 confirmed it.



*different layer. different object. different decade.
the same shape.*

The claim, and its status

☼ **Zenith:** Two systems, built by different people for unrelated problems at different layers of the stack, arrived independently at the same design: rules that only ever grant, and the removal of access accomplished only by the removal of a grant. Neither has a deny rule. In both, subtraction is not an operation you can perform.

Chapter 11 promised you would understand *why*, and why it is a feature rather than an omission. So here is the honest situation, stated before the argument rather than after it.

The documentation states the property and does not explain it. The RBAC reference says permissions are purely additive with no deny rules, in a parenthetical [source: k8s-docs-rbac-2026-08-23]. Chapter 10's sources said the equivalent about NetworkPolicy. Neither offers a rationale, and no source in this book's corpus does either.

What follows is therefore **a reading, not a citation**. It is the interpretation that makes the best sense of what both systems do. Hold it more loosely than you hold the facts around it.

The reading

A deny rule makes the effect of a grant **non-local**.

That is the whole argument, and everything else follows from it.

Consider a system with deny. You are handed a rule: *this subject may delete Deployments in this namespace*. What does the rule do? You cannot say. Somewhere else there may be a rule that denies it, and until you have read every rule that could possibly apply, you do not know the effect of the one in front of you. A deny rule is a correction pencilled onto somebody else's chart: the sheet in your hands is accurate and still does not tell you where you are, because the correction is on a sheet you have not seen.

Which forces a second thing: **evaluation order becomes semantics**. Once both allow and deny exist, "which one wins?" must be answered, and every real system answers it differently. Deny-overrides. First-match. Most-specific-wins. Explicit-beats-inherited. Each is defensible and none is obvious, and the choice is now part of what a policy *means*. Anyone reasoning about the system has to hold not just the rules but the resolution procedure.

Now add the two conditions that describe an actual Kubernetes cluster.

Rules are written by many authors. Platform engineering writes cluster-wide roles. Application teams write namespace-scoped ones. Every operator you install ships bindings for its own controllers. Helm charts arrive carrying RBAC objects nobody reads. A Kubernetes cluster's policy is not authored; it accretes, from a dozen sources, none of which coordinated with the others.

The objects being governed change constantly, without anyone's involvement. Pods appear and disappear. Namespaces get created. CRDs add resource types that did not exist when the roles were

written, which is exactly why the documentation warns that wildcard access grants rights *"not just to all object types that currently exist in the cluster, but also to all object types which are created in the future"* [source: k8s-docs-rbac-good-practices-2026-08-31].

Under those two conditions, a deny rule is close to unusable. You would have a policy whose meaning could not be determined by reading any bounded subset of it, assembled by parties who never spoke, against a resource set that changes while you read.

Additive-only is what makes a single rule readable in isolation. With no deny, a grant means exactly what it says: it adds these permissions, unconditionally, and no rule anywhere can take them back. To know whether a subject can do something, you take the union of their grants — an operation that is order-independent, composable, and answerable without global knowledge. That property is not an accident of either system. It is the only property that survives the way these systems are actually used.

That, I think, is why two unrelated teams landed in the same place. They were not copying each other. They were solving the same shape of problem — distributed authorship, dynamic membership, composability required — and it has one clean answer.

The cost, which is real

"A feature rather than an omission" should not be oversold, so name what it costs, because it is not nothing.

You cannot carve an exception out of a grant. Section 3's hazard was not a footnote. When somebody holds `edit` and should have everything except one verb on one resource, there is no rule to write. You stop granting `edit` and construct a role by hand from the permissions they should have, and you now maintain that role forever, updating it as the cluster acquires resource types the original author never anticipated. The clean composability comes out of the same property that makes exceptions expensive.

Sometimes an exception is exactly what you want. Additive-only says: then build the grant you actually mean, from the ground up. That is more work, and it is honestly more work, and the trade is that anyone can read the result and know what it does.

What this is worth

You now have a thing that is more useful than either fact.

Faced with a Kubernetes access-control system you have never seen — one that ships in a future release, or in a project you meet next year — you can predict its shape. Additive. No deny. Removal by removing the grant. And you can say why: because that is what composability requires when many hands write the policy and the objects will not hold still.

That is not memorized. It is derived, and it will survive the next curriculum change.

Exam Alert! 🚨

High-Priority Topics

1. **The four-way matrix, stated as a rule rather than a table.** *The binding determines the scope of the grant.* The highest-yield fact in the chapter, and the one most likely to be tested through a scenario rather than a definition. If you can derive the four combinations from the namespace/cluster-scoped boundary, you do not need to remember them.
2. **RBAC is additive with no deny rule.** Every "how do you remove this one permission?" question resolves to *remove the grant*. There is no other answer.
3. **The four default roles and their negative space.** What each *cannot* do: view cannot read Secrets; edit cannot view or modify roles or bindings; admin cannot write the namespace's resource quota or the namespace itself; and cluster-admin in a RoleBinding is limited to that binding's namespace.
4. **Secrets are unencrypted in etcd by default**, and there are three exposure paths — API access, etcd access, and the ability to create a Pod.
5. **TokenRequest, not token Secrets.** Since v1.22 the recommended credential is a short-lived, automatically rotating projected token. Long-lived ServiceAccount token Secrets are the legacy risk.
6. **Three PSS levels × three PSA modes, applied per namespace by label.** Level = what is checked. Mode = what happens.

Common Traps

These are the specific wrong answers this material produces. Every one of them is a wrong answer somebody has confidently written down.

The trap	The correct understanding
RBAC has deny rules	Purely additive. Removing access means removing the grant. [source: k8s-docs-rbac-2026-08-23]
ClusterRole is only for cluster-scoped resources	Two of its three documented uses are about namespaced resources — bound into one namespace, or across all. [source: k8s-docs-rbac-2026-08-23]
The four combinations must be memorized	They derive from one boundary. The binding sets the scope.
A binding can be retargeted	It cannot, after creation. Delete and recreate. [source: k8s-docs-rbac-2026-08-23]
view can read Secrets	It cannot — nor roles nor bindings. Reading Secrets is transitively becoming a ServiceAccount. [source: k8s-docs-rbac-depth-2026-08-31]
edit can manage RBAC in its namespace	It cannot; admin can. [source: k8s-docs-rbac-2026-08-23]
cluster-admin always means the whole cluster	In a RoleBinding it is scoped to that namespace. [source: k8s-docs-rbac-2026-08-23]
Removing every binding revokes anyone's access	Not for <code>system:masters</code> — that membership bypasses RBAC entirely and cannot be revoked by removing bindings. [source: k8s-docs-rbac-good-practices-2026-08-31]
Only get on Secrets reveals them	<code>list</code> and <code>watch</code> reveal the contents of every Secret in scope. [source: k8s-docs-rbac-good-practices-2026-08-31]
Secrets are encrypted	Unencrypted in etcd by default; encryption at rest is opt-in and is a control-plane configuration. [source: k8s-docs-secret-2026-08-23]
An RBAC audit showing no get secrets means Secrets are safe	Pod creation in the namespace reads any Secret in it, including via a Deployment. [source: k8s-docs-secret-2026-08-23]
Token Secrets are current best practice	TokenRequest, short-lived and rotating, since v1.22. [source: k8s-docs-service-accounts-2026-08-23]
Pod Security levels and admission modes are the same axis	Levels say what is checked; modes say what happens. Independent. [inferred]
PodSecurityPolicy is a current control	Superseded by Pod Security Admission; not the mechanism a current cluster uses.
A signature covers the tag you signed	It resolves to the digest. Tags are mutable; signatures are not. [source: notary-project-signing-digest-2026-08-31]
A valid signature means the artifact is current	It means the bytes came from the signer. Freshness, ordering and key compromise are what TUF addresses. [source: tuf-overview-2026-08-31]
Falco prevents the behavior it detects	It observes and alerts. It does not block. [source: falco-overview-2026-08-23]

One row above carries an [inferred] marker: the level/mode confusion. That mark means the row describes something *easy to confuse*, not something anyone has published as *frequently tested*. The

distinction matters. This book will tell you when material is high-yield by reasoning about the published objectives, and it will not manufacture a statistic to make the point land harder.

Practice Questions

Twenty-three questions. Five of them require material from earlier chapters, and several cannot be answered from any single section of this one.

1. Which statement correctly distinguishes the cloud native security lifecycle phases from the 4Cs?

A) The phases describe cloud environments; the layers describe on-premises environments B) The phases answer *when* a control acts; the layers answer *where* it acts C) The phases replaced the layers, which are now incorrect D) The layers apply to Kubernetes; the phases apply to the application only

2. According to the runtime protection guidance, which of the following belongs to the **compute** area rather than access or storage?

A) Using ServiceAccounts to provide security identities for workloads B) Enabling encryption at rest for API objects C) Enforcing Pod Security Standards so applications run with only necessary privileges D) Protecting the encryption keys used for API traffic

3. A cluster administrator wants to remove an engineer's superuser access. The engineer's credential places them in the `system:masters` group. The administrator deletes every `ClusterRoleBinding` and `RoleBinding` that names that engineer. What is the result?

A) Access is revoked — `system:masters` is bound to `cluster-admin` by a `ClusterRoleBinding`, which the administrator has now deleted B) Access is unchanged; membership of `system:masters` bypasses all RBAC rights checks and cannot be revoked by removing bindings C) Access is reduced to the default API discovery permissions granted to all authenticated principals D) Access is unchanged until the API server next restarts, at which point the default bindings are reconciled and the bypass is removed

4. Which is the current recommended way for a Pod to obtain ServiceAccount credentials?

A) Create a Secret of type `kubernetes.io/service-account-token` and mount it B) A short-lived, automatically rotating token from the `TokenRequest` API, mounted as a projected volume C) Store the token in a `ConfigMap` and reference it as an environment variable D) Have the application call the `TokenReview` API at startup to obtain a token for its own ServiceAccount

5. Your team needs a set of permissions over Node objects — a cluster-scoped resource. Which pair of objects can express this grant, and why?

- A) Role + RoleBinding, in the kube-system namespace, since that is where node components live
 - B) ClusterRole + ClusterRoleBinding, because a Role cannot hold rules over a resource that is in no namespace
 - C) ClusterRole + RoleBinding, scoping the node access to one namespace
 - D) Either Role + RoleBinding or ClusterRole + ClusterRoleBinding, depending on preference
-

6. A ClusterRole named secret-reader grants get and list on secrets. It is referenced by a RoleBinding in the staging namespace. Which statement is true?

- A) The subjects can read Secrets in every namespace, because the role is a ClusterRole
 - B) The RoleBinding is invalid; a RoleBinding cannot reference a ClusterRole
 - C) The subjects can read Secrets in staging only
 - D) Nothing is granted; a ClusterRole's rules take effect only when a ClusterRoleBinding references them
-

7. You need to allow a subject to read the ConfigMap named app-config in a namespace, and no other ConfigMap. Which mechanism expresses this?

- A) A label selector on the Role's rule
 - B) The resourceNames list on the rule
 - C) A separate Role per ConfigMap, since RBAC cannot filter by name
 - D) A fieldSelector on the RoleBinding
-

8. *[retrieval: ch4]* A colleague proposes granting permissions to "every ServiceAccount labeled tier=frontend" so that new frontend workloads pick up the grant automatically. What is wrong with this proposal?

- A) Nothing — this is what aggregated ClusterRoles are for
 - B) RBAC names subjects as literal strings; there is no subject selector, and a grant that expanded when someone added a label would not be auditable
 - C) It would work, but only for ServiceAccounts in the same namespace as the binding
 - D) It would work if the subjects were users rather than ServiceAccounts; only ServiceAccounts must be named literally
-

9. Which of these operations does the admin default role permit within its namespace?

- A) Writing the namespace's ResourceQuota
 - B) Deleting the namespace object itself
 - C) Creating Roles and RoleBindings
 - D) Modifying cluster-scoped resources referenced from the namespace
-

10. A Secret's data field contains cGFzc3dvcmQxMjM=. What protection does this encoding provide?

A) It is encrypted with the cluster's default key B) It provides no additional confidentiality over plain text
C) It is hashed, so the original value cannot be recovered D) It is protected only while the Secret is at rest in etcd

11. Which single control closes the exposure path created by an unencrypted etcd backup?

A) A NetworkPolicy restricting access to the etcd Pods B) Removing `get secrets` from all Roles and ClusterRoles
C) Encryption at rest, configured via `EncryptionConfiguration` on the API server D) Mounting Secrets as files rather than environment variables

12. Which statement about mounting a Secret is correct?

A) A Secret mounted as a volume receives updates automatically, except when mounted as a `subPath` B) A Secret consumed as an environment variable updates automatically when the Secret changes
C) Both mechanisms update automatically; the difference is only in file permissions D) Neither mechanism receives updates; the Pod must be recreated in both cases

13. A Pod sets `securityContext.runAsNonRoot: true` at Pod scope. What does this accomplish?

A) It drops all Linux capabilities from every container B) It requires that the containers do not run as the root user
C) It mounts the root filesystem read-only D) It applies the `RuntimeDefault` seccomp profile

14. Which statement about `privileged: true` is correct?

A) It grants a single additional capability, `CAP_SYS_ADMIN` B) It grants all Linux capabilities, but any seccomp or AppArmor profile you specified in the manifest remains in force
C) It overrides or undoes many other hardening settings, including the applied seccomp and AppArmor profiles, and grants all Linux capabilities D) It is required in order to set `runAsUser: 0`

15. *[retrieval: ch10]* Which statement correctly describes the relationship between `securityContext` and `NetworkPolicy`?

A) `NetworkPolicy` governs the workload-to-host axis; `securityContext` governs workload-to-workload B) They are two views of the same underlying enforcement mechanism
C) `securityContext` governs the workload-to-host axis; `NetworkPolicy` governs the workload-to-workload axis; they fail independently and neither substitutes for the other D) `NetworkPolicy` supersedes `securityContext` on clusters using a CNI plugin that supports it

16. A namespace is labeled `pod-security.kubernetes.io/enforce: restricted`. A developer submits a Pod with no `securityContext`. What happens?

- A) It is admitted, because Restricted allows the default minimally-specified Pod
 - B) It is rejected, because Restricted requires `runAsNonRoot: true`, `allowPrivilegeEscalation: false`, capabilities dropping ALL, and an explicit seccomp profile
 - C) It is admitted with a warning
 - D) It is admitted, and the missing fields are populated with Restricted-conformant defaults
-

17. Which pairing is correct?

- A) `enforce` is a level; `baseline` is a mode
 - B) `warn` is a level; `restricted` is a mode
 - C) `restricted` is a level; `audit` is a mode
 - D) `privileged` is a mode; `enforce` is a level
-

18. *[retrieval: ch8]* Pod Security Admission and a Kyverno validating policy have something structural in common. What is it, and what distinguishes them?

- A) Both run at the authorization gate; PSA is built in and Kyverno is not
 - B) Both run at the admission gate; PSA answers a fixed question about the Pod Security Standards, while a policy engine answers an arbitrary question
 - C) Both run at runtime; PSA inspects Pods and Kyverno inspects all objects
 - D) Both are authorization modes configured with `--authorization-mode`
-

19. *[retrieval: ch2]* What does a container image signature bind to?

- A) The image tag, since that is what the signer typed
 - B) The image's digest — the tool resolves a tag to a digest before signing
 - C) The registry hostname and repository path
 - D) The image's SBOM, which in turn references the layers
-

20. A cluster verifies image signatures at admission. Which component performs the verification, and where does it sit relative to the rest of the supply chain?

- A) The container runtime, at pull time, after everything else in the chain
 - B) An admission controller — such as Sigstore's Policy Controller or Kyverno — and it is the first checkpoint the cluster itself performs
 - C) The registry, before serving the image
 - D) The kubelet, immediately before starting the container
-

21. *[retrieval: ch10]* Which statement most accurately characterizes what RBAC and NetworkPolicy have in common?

- A) Both are implemented by the API server and evaluated at the same gate
- B) Both use label selectors to identify the subjects they govern
- C) Both are purely additive: rules only grant, and removing access

means removing a grant D) Both were retrofitted with explicit deny rules in later API versions, which is why the additive behavior is now the default rather than the only option

22. A namespace carries `pod-security.kubernetes.io/enforce: restricted`. A developer submits a Pod whose single container sets exactly this and nothing else:

```
securityContext:
  runAsNonRoot: true
  runAsUser: 1000
  allowPrivilegeEscalation: false
  capabilities:
    drop: ["ALL"]
```

What happens, and why?

A) Admitted — every Restricted requirement is satisfied by that block B) Rejected — Restricted also requires an explicit seccomp profile of `RuntimeDefault` or `Localhost`, and none is set C) Rejected — `runAsUser: 1000` conflicts with `runAsNonRoot: true` D) Admitted with a warning, because `enforce` applies leniently to capability rules

23. An organization verifies image signatures at admission and rejects anything unsigned. An attacker with control of a registry serves a client an older image that is correctly signed by the organization's own build pipeline and that contains a vulnerability fixed two releases ago. Does signature verification catch this, and what addresses it?

A) Yes — the signature will not verify, because the older image's digest differs from the current one B) No — the artifact is correctly signed, so verification passes; this is the class of attack The Update Framework exists to address C) No — but an SBOM attached to the image would refuse to load, because its component list no longer matches the deployed release D) Yes — Rekor will reject the lookup, because the older signing event has been superseded by a newer one

Answers with Explanations

1. B. The two framings answer different questions, and a control has a position on both: image signing is `distribute/Container`, RBAC is `runtime/Cluster`. **A** invents a distinction neither framing makes. **C** overstates the situation. The current `kubernetes.io` page does present the phases in place of the 4Cs [source: `k8s-docs-cloud-native-security-2026-08-23`], but the 4Cs are not wrong, they are a different map, and they remain the framing most third-party material uses. **D** invents a scope split that neither framing has.

2. C. The runtime/compute list names enforcing Pod Security Standards *"for applications to help ensure they run with only the necessary privileges"* [source: k8s-docs-cloud-native-security-2026-08-23]. **A** and **D** are runtime/access; ServiceAccounts and TLS for API traffic both appear in that list. **B** is runtime/storage. The point of the question is that all three areas are runtime; naming the phase is not enough.

3. B. *"Avoid adding users to the system:masters group. Any user who is a member of this group bypasses all RBAC rights checks and will always have unrestricted superuser access, which cannot be revoked by removing RoleBindings or ClusterRoleBindings"* [source: k8s-docs-rbac-good-practices-2026-08-31]. Group membership is not an RBAC grant, so removing RBAC objects removes nothing.

A is the intuitive and wrong model — it treats group membership as though it were expressed through a binding that could be deleted. **C** describes what would happen to an ordinary subject whose bindings were all removed: identity intact, permissions down to API discovery [source: k8s-docs-service-accounts-2026-08-23]. It is the right outcome for the wrong principal. **D** invents a reconciliation that runs the wrong direction. The API server does reconcile default bindings at startup, but the bypass does not depend on any binding, so a restart restores nothing and removes nothing.

Two consequences worth carrying: this is invisible to any audit that reads RBAC objects, and if the cluster uses an authorization webhook, requests from members of that group are never sent to it at all [source: k8s-docs-rbac-good-practices-2026-08-31].

4. B. *"In Kubernetes v1.22 and later, Kubernetes gets a short-lived, automatically rotating token using the TokenRequest API and mounts the token as a projected volume"* [source: k8s-docs-service-accounts-2026-08-23]. **A** is the legacy mechanism; long-lived token Secrets *"don't expire or rotate and pose a security risk"* [source: k8s-docs-service-accounts-2026-08-23]. **C** puts a credential in the wrong object type entirely. **D** is the near-miss worth naming: TokenRequest and TokenReview are different APIs doing opposite jobs. TokenRequest *issues* a token; TokenReview *validates* one somebody has presented. And a workload that had to authenticate in order to fetch its own credential would have nothing to authenticate with — which is exactly the bootstrap problem the projected volume solves by delivering the token before the process starts.

5. B. Node is cluster-scoped, so a Role — which *"always sets permissions within a particular namespace"* [source: k8s-docs-rbac-2026-08-23] — has nowhere to put the rule, and the ClusterRole is forced. The grant must then be cluster-wide, because the resource is in no namespace to scope it to. **A** misunderstands what kube-system is: a namespace containing objects, not a namespace that contains the nodes. **C** is the seductive one. A ClusterRole *can* be bound by a RoleBinding, and doing so binds it into that namespace [source: k8s-docs-rbac-2026-08-23] — but the Node rule has no namespace to land in. **D** treats a structural constraint as a stylistic preference.

6. C. *"A RoleBinding can reference a ClusterRole and bind that ClusterRole to the namespace of the RoleBinding"* [source: k8s-docs-rbac-2026-08-23]. The binding determines the scope. **A** is the most common version of this trap and is the reason `cluster-admin` in a RoleBinding surprises people. **B** contradicts the documented behavior. **D** is A's mirror image: where A over-scopes the grant, D refuses it

entirely. Both errors come from the same wrong premise — that a ClusterRole's scope is fixed by the role object rather than by the binding.

7. B. *"You can also refer to resources by name for certain requests through the `resourceNames` list. When specified, requests can be restricted to individual instances of a resource"* [source: k8s-docs-rbac-depth-2026-08-31]. **A** is the selector reflex again; there is no label selector on an RBAC rule. **C** is wrong about the capability existing, though it correctly identifies that RBAC's naming is literal rather than queried. **D** invents a field.

8. B. RBAC names subjects — users, groups, ServiceAccounts — as literal strings [source: k8s-docs-rbac-depth-2026-08-31]. There is no subject selector, and Chapter 4 specifically warned that assuming the selector pattern held here would produce a confident wrong prediction. **A** confuses two different uses of selectors: an aggregated ClusterRole uses a label selector to combine *ClusterRoles*, not to select subjects [source: k8s-docs-rbac-depth-2026-08-31]. **C** invents a namespace constraint on a mechanism that does not exist. **D** is a false symmetry, and worth being explicit about: all three subject kinds are named literally. There is no subject kind that RBAC selects rather than names.

9. C. *"If used in a RoleBinding, allows read/write access to most resources in a namespace, including the ability to create roles and role bindings within the namespace. This role does not allow write access to resource quota or to the namespace itself"* [source: k8s-docs-rbac-depth-2026-08-31]. **A** and **B** are the two things that sentence explicitly excludes, and the exclusions are principled: writing your own quota would let you raise your own ceiling. **D** confuses a namespaced role with cluster-scoped access.

10. B. *"Base64 encoding is not an encryption method, it provides no additional confidentiality over plain text"* [source: k8s-docs-secrets-good-practices-2026-08-24]. **A** invents a cluster key. **C** confuses encoding with hashing; base64 is trivially reversible, which is the entire point of an encoding. **D** confuses base64 with encryption at rest, a separate, opt-in mechanism.

11. C. Encryption at rest protects the object as written to etcd [source: k8s-docs-encrypt-data-2026-08-31], which is exactly the content of an etcd backup. **A** protects access to the running etcd over the network and does nothing about a backup file sitting in object storage. **B** addresses the API route, not the etcd route. **D** is a good practice for a different reason and does not touch etcd at all.

12. A. When a Secret is mounted as a volume, updates propagate automatically; *"A container using a Secret as a subPath volume mount does not receive automated Secret updates"* [source: k8s-docs-secret-risks-2026-08-31].

B inverts the asymmetry: an environment variable is read at container start and is then a fixed string in the process's environment, so a rotated Secret does not reach a running container. **C** is wrong about where the difference lies. The two mechanisms differ in *update behavior*; file permissions are a volume-only concern that has no environment-variable counterpart to differ from. **D** generalizes the environment-variable behavior to both, which erases the whole rotation half of the file-over-env-var argument.

13. B. `runAsNonRoot` requires that containers do not run as root — the Restricted level mandates `runAsNonRoot: true` for exactly this reason [source: k8s-docs-pod-security-standards-profiles-2026-08-31], and the documentation recommends it generally [source: k8s-docs-linux-kernel-security-constraints-2026-08-31]. **A** describes `capabilities.drop`. **C** describes `readOnlyRootFilesystem`. **D** describes `seccompProfile`. All four are `securityContext` fields, which is why this question is worth taking slowly.

14. C. *"Privileged containers override or undo many other hardening settings such as the applied seccomp profile, AppArmor profile, or SELinux constraints. Privileged containers are given all Linux capabilities, including capabilities that they don't require"* [source: k8s-docs-linux-kernel-security-constraints-2026-08-31].

A dramatically understates it; the point is *all* capabilities, and the recommended alternative is to grant specific ones via the `capabilities` field [source: k8s-docs-linux-kernel-security-constraints-2026-08-31]. **B** is the version of this misconception most worth killing: it assumes hardening layers stack independently, so that a seccomp profile you wrote would survive a privileged flag somebody else set. The documented behavior is the opposite — privileged containers run as the Unconfined seccomp profile, overriding any profile specified in the manifest, and ignore any applied AppArmor profiles [source: k8s-docs-linux-kernel-security-constraints-2026-08-31]. **D** invents a dependency. Running as UID 0 inside a container needs no privileged flag at all; `privileged: true` is about capabilities and kernel-constraint bypass, not about which UID the process runs as.

15. C. Chapter 10 drew the two axes; this chapter filled in the second. `securityContext` constrains what a workload may do to its node; `NetworkPolicy` constrains which workloads may reach which. Different objects, different implementers, independent failure. **A** inverts them. **B** is false: one is a Pod spec field enforced by the runtime and kubelet, the other is an object enforced by a CNI plugin. **D** invents a supersession that would make no sense, since a `NetworkPolicy` has nothing to say about `privileged: true`.

16. B. Restricted requires `runAsNonRoot: true`, `allowPrivilegeEscalation: false`, capabilities dropping ALL, and an explicit `RuntimeDefault` or `Localhost` seccomp profile [source: k8s-docs-pod-security-standards-profiles-2026-08-31]. A Pod with no `securityContext` satisfies none of those, and `enforce` rejects. **A** describes **Baseline**, which *"Allows the default (minimally specified) Pod configuration"* [source: k8s-docs-pod-security-standards-2026-08-23]; that is the specific confusion this question tests. **C** describes `warn` behavior. **D** invents a mutation; PSA validates, it does not mutate.

17. C. `restricted` is one of the three levels; `audit` is one of the three modes [source: k8s-docs-pod-security-standards-2026-08-23]. Every other option crosses the axes, which is the single most likely wrong answer this chapter produces. Check the grammar before the meaning: `enforce/audit/warn` are modes, `privileged/baseline/restricted` are levels, and no term is both.

18. B. PSA is *"a built-in Pod Security admission controller"* [source: k8s-docs-pod-security-admission-2026-08-31], and Kyverno can be *"applied as a Kubernetes admission controller (webhook)"* [source: kyverno-overview-2026-08-23]. Same gate, and Chapter 8 promised you would recognize it. The distinction is the question each answers: PSA's is fixed to the Pod Security Standards, a policy engine's is

arbitrary. **A** puts both at the wrong gate. **C** confuses admission with runtime. **D** confuses admission controllers with authorization modes, a real distinction the chapter draws in §3.

19. B. *"Notation resolves the tag to the digest before signing if a tag is used to identify the container image"* [source: notary-project-signing-digest-2026-08-31]. **A** is the misconception, and it matters because a tag-covering signature would attest to content the signer has never seen — Chapter 2's mutable-pointer lesson, arriving as a security consequence. **C** would attest to a location rather than to content. **D** inverts the relationship: an SBOM is a separate signable artifact that describes an image, not the thing a signature on the image binds to.

20. B. Sigstore's Policy Controller *"enforces signature verification policies within Kubernetes as an admission controller"* [source: sigstore-overview-2026-08-23], and Kyverno can *"verify container images and metadata"* [source: kyverno-overview-2026-08-23]. Verification at admission is the first checkpoint the cluster itself performs; everything before it (scan, sign, record, restrict) happened outside, under someone else's control.

A places verification at the runtime, at pull time. That is late by one gate and wrong about the actor: by the time anything is pulled, the object has already been accepted, so a refusal there would leave a Pod the API server has already admitted. **C** places it at the registry. A registry can restrict *who may pull*, which is a real control [source: k8s-docs-cloud-native-security-2026-08-23], but it is not the cluster deciding whether to accept the object, and a registry you do not administer is exactly what verification exists to be independent of. **D** is the most tempting: the kubelet does pull the image, and it is inside the cluster. But the enforcement decision is made at admission, before the object is accepted and long before any kubelet is involved.

21. C. Both are purely additive; in both, the only way to remove access is to remove a grant [source: k8s-docs-rbac-2026-08-23]. That is the shared semantic, and §9 argues it is not a coincidence. **A** is false on both halves: NetworkPolicy is enforced by a CNI plugin, not the API server, and not at the authorization gate. **B** is half true and therefore the best distractor here — NetworkPolicy does use selectors, and RBAC pointedly does not. **D** invents a version history. Neither system has ever had a deny rule, and §9's argument is that the additive design is load-bearing rather than a default somebody happened to pick.

22. B. Restricted's seccomp control requires the profile type to be `RuntimeDefault` or `Localhost` explicitly [source: k8s-docs-pod-security-standards-profiles-2026-08-31], and no `seccompProfile` is set. Everything else in that block conforms: `runAsNonRoot: true`, `runAsUser non-zero`, `allowPrivilegeEscalation: false`, and capabilities dropping `ALL`.

A is the near miss this question exists to produce — four of five requirements met, and the missing one is the least visible because it is the only one that is not obviously about root. **C** invents a conflict. A non-zero `runAsUser` is precisely what Restricted's "Running as Non-root user" control asks for [source: k8s-docs-pod-security-standards-profiles-2026-08-31]; the two fields agree rather than collide. **D** crosses the axes. `enforce` rejects, full stop — it is a mode, not a severity, and it does not apply differently to different controls. If you wanted the warning behavior, that is a different label.

23. B. TUF names exactly this attack: *"An attacker gives you an older, insecure version of a file that you already have and tricks you into thinking it's newer"* [source: tuf-overview-2026-08-31]. Every attack in TUF's list involves a *correctly signed* artifact, which is what distinguishes freshness and ordering from authenticity — and why TUF *"maintains the security of software update systems, providing protection even against attackers that compromise the repository or signing keys"* [source: tuf-overview-2026-08-31].

A misunderstands what verification checks. A signature over the older digest is entirely valid for that older digest. Verification asks "did these bytes come from who I think?" and answers nothing about which version they are. **C** invents an enforcement behavior. An SBOM is a record of what an artifact contains, not a gate, and the older image's SBOM correctly describes the older image. **D** inverts the transparency log's function. Rekor is append-only and records signing events for public audit [source: sigstore-overview-2026-08-23]; entries are never superseded or expired, and nothing about it is a freshness check performed at pull time.

Chapter Summary

Concept	Remember This
Phases vs layers	Phases = <i>when</i> a control acts (develop, distribute, deploy, runtime). Layers = <i>where</i> (Cloud $\supset$ Cluster $\supset$ Container $\supset$ Code). Every control has both coordinates.
Runtime's three areas	Access (the API), compute (the workload on the node), storage (data at rest). This chapter's table of contents.
ServiceAccount	A namespaced identity object. Every namespace has a default; every Pod that names nothing gets it.
Identity $\neq$ permission	Two things, two objects. The default ServiceAccount has an identity and can do essentially nothing.
Credentials	TokenRequest, short-lived, rotating, projected. Long-lived token Secrets are legacy.
Users and groups	Not Kubernetes objects. They arrive from outside as strings the authenticator produces.
The four RBAC objects	Role (namespaced) / ClusterRole (not) hold rules. RoleBinding (one namespace) / ClusterRoleBinding (everywhere) grant them.
The derivation	Cluster-scoped resource $\rightarrow$ ClusterRole is forced. Where the grant applies $\rightarrow$ the binding decides. The binding determines the scope of the grant.
Additive, no deny	Permissions are the union of grants. To remove access, remove the grant. There is no other operation.
Binding immutability	roleRef cannot change after creation. Delete and recreate.
Subjects	Named as literal strings — users, groups and ServiceAccounts alike. There is no subject selector, and that is deliberate.
Escalation prevention	You cannot create a role holding permissions you do not have, absent the <code>escalate</code> verb; you cannot bind one, absent <code>bind</code> .
system:masters	Not an RBAC grant. Bypasses every rights check, cannot be revoked by deleting bindings, invisible to an RBAC audit.
Default roles	<code>view</code> cannot read Secrets. <code>edit</code> cannot touch RBAC. <code>admin</code> cannot write its quota or namespace. <code>cluster-admin</code> in a RoleBinding is namespace-scoped.
base64	Encoding. Provides no confidentiality over plain text.
Secret storage	Unencrypted in etcd by default. Encryption at rest is opt-in, via EncryptionConfiguration on the API server.
Three exposure	API access (including <code>list</code> and <code>watch</code>) $\cdot$ etcd access, including backups $\cdot$ the ability to create a Pod.

Concept	Remember This
paths	
The weak boundary	Boundaries within a namespace are weak. Separate trust levels with separate namespaces.
File over env var	A mounted Secret updates automatically (except via <code>subPath</code>); an environment variable is fixed at container start.
securityContext	Pod scope sets the default; container scope overrides it. Governs the workload-to-host axis.
privileged : true	Not one more setting — the setting that turns the others off. All capabilities; seccomp, AppArmor and SELinux constraints bypassed; every Secret on the node readable.
The three PSS levels	privileged (no restrictions) · baseline (prevents known escalations, allows the default Pod) · restricted (current hardening best practice). Cumulative in permitted Pods.
The three PSA modes	enforce (reject) · audit (log) · warn (tell the user).
Levels × modes	Independent axes, applied per namespace by label <code>pod-security.kubernetes.io/<MODE>: <LEVEL></code> . Level = what. Mode = what happens.
The supply chain	build → scan → sign → record → restrict → verify → run. Verify is the first step the cluster performs itself.
Signatures	Bind to the digest . A tool handed a tag signs the digest it resolves to.
Sigstore	Cosign (client) · Fulcio (short-lived certs from a verified identity) · Rekor (append-only transparency log) · Policy Controller (admission-time verification). Keyless: the private key is discarded after one signing.
TUF	Freshness, ordering and key compromise — the attacks that use a <i>correctly signed</i> artifact. Signing alone does not address them.
Policy engines	Kyverno (YAML/CEL, validate/mutate/generate/clean up) and OPA Gatekeeper (Rego), both at admission. An arbitrary question where PSA answers a fixed one.
Falco	Runtime. Observes kernel events, evaluates rules, emits alerts . Detects; does not prevent.
The shared semantic	RBAC and NetworkPolicy both allow and never deny, from different layers for different reasons — because a deny rule makes a grant's effect non-local, and these policies are written by many hands against objects that will not hold still.

The Voyage Ahead

You have spent this chapter answering a question by refusing things. What is a workload allowed to do? Whatever a grant says, and nothing else. What may it do to its node? Whatever a securityContext permits and a Pod Security level tolerates. What may it run? Whatever survived verification at the gate.

Every one of those controls is a way of saying no by not saying yes. Which means every one of them can produce a workload that does not work, and none of them will tell you that is why.

That is the next chapter's problem. A Pod stuck in Pending. A Pod that starts and immediately dies. A Pod that does not appear at all because an admission controller you forgot about declined the object, which is a failure with no phase, no events, and no logs, because there is no Pod. A container that runs perfectly and cannot read the file it needs, because somebody set `runAsUser` on an image built expecting root. A Deployment that will not roll out because the Secret it references was renamed.

Chapter 13 is about reading those signatures. Not guessing at them — reading them, in a fixed order, starting with a question most people skip: *whose problem is this?* There is a triage flow, it has a specific sequence, and the sequence exists because the cheap checks eliminate whole categories of cause before you spend an hour reading logs that were never going to contain the answer.

Bring three things with you from this chapter. The first is the shape of a Pod that was *rejected* rather than one that *failed*: you now know that admission can refuse an object outright, and that a refused object leaves nothing behind to inspect. The second is the `securityContext` fields, because a container that cannot write where it expects to write is a permissions failure wearing an application error's clothing. The third is Secrets: a Pod that references one that does not exist never gets a running container, and it fails in a recognizable way rather than a mysterious one.

You have spent twelve chapters learning what the cluster does when everything works. The next two are about the other case, and the honest truth is that the second kind of knowledge is what people actually get paid for.

⚓ Safe Harbor

That is Domain 2's security competency, complete. Nine sections, five independent systems, and one argument that took two chapters to set up.

Take stock of what changed. You can now look at any Kubernetes security control and say which of five questions it answers — *who are you, what may you do, what is stored where, what may your workload do to the machine, what did you ship* — and you can say which of the other four it does not answer, which is the harder and more useful half. You can derive the RBAC matrix rather than recalling it. You can look at a namespace's labels and a Pod's spec together and predict the outcome. And you can name the exposure path that no RBAC audit will show you.

The last one is worth sitting with for a moment. Most of what separates a competent Kubernetes engineer from a trusted one is not knowing more controls. It is knowing what each control does *not* cover, and refusing to be reassured by a green check mark that was measuring the wrong thing.

Chart, passage, dawn: ☞ → ≍ → ⚓

▮ "A grant says what it says. That is the whole design, and it is worth what it costs."

Chapter 13: When the Cluster Won't Answer

"Read the phase before you read the logs"

Domain: Container Orchestration (Troubleshooting) | **Published domain weight:** 28% [source: cncf-kcna-curriculum-pdf-2026-08-23] **Complexity:** Mixed | **Novelty:** Moderate | **Prerequisites:** Heavy

Container Orchestration is 28% of the exam. How that 28% divides among its four competencies is not published — the allocation of chapters across this Part is the author's, derived from the competency list, not from CNCF data.

Attention Budget

Total time: ~95 minutes | **Recommended:** Split across 2 sessions

Section	Time	Attention Cost	Best Time to Study
§1 Whose Problem Is This	8 min	Low	Anytime
§2 Pods That Never Start	18 min	High	Peak attention
§3 Looking Inside	12 min	Medium	Mid-session
🕒 Taking Your Bearings 1	8 min	Medium	After a brief break
§4 Pods That Start and Don't Stay	15 min	High	Peak attention
§5 When the Node Is the Problem	10 min	Medium	Mid-session
🕒 Taking Your Bearings 2	8 min	Medium	After a brief break
§6 Versions That Don't Agree	7 min	Medium	Anytime
§7 Numbers Nobody Collects	8 min	Low	Anytime
🕒 Taking Your Bearings 3	7 min	Medium	After a brief break
🌟 §8 Read the Phase First	5 min	Low	End of session

Attention Cost Key:

- **Low:** Concrete, familiar concepts — study anytime
- **Medium:** New concepts requiring focus — study when alert
- **High:** Abstract or complex material — study at peak attention

If you only have 15 minutes: read §1 and §2, then take the first checkpoint. Those two sections carry the chapter's method and its highest-value material.

Natural split point: after ☹ Taking Your Bearings 1. §1–§3 teach the method and the never-started family; §4–§8 apply it to everything else.

"Ninety percent of debugging is figuring out where to look. The other ten percent is realizing you were looking at the wrong thing." — Lodestar Ledgers

🕒 Soundings

Before reading this chapter, try these eight questions. Every one is answerable from Chapters 1–12 or from ordinary professional experience; none requires anything this chapter teaches. Your score tells you how to read.

Two of them deliberately probe material you met a while ago and may have let slip. Missing those two is information, not failure: it tells you which sections of this chapter are doing the most work for you.

1. A Pod's status reports a Reason string of `ContainerCreating`. Does that string belong to the Pod's **phase** taxonomy or the **container state** taxonomy?
2. A Pod has been sitting in `Pending` for ten minutes. Is some component of the cluster periodically retrying it with relaxed constraints, hoping to eventually place it?
3. A Pod spec names the image `myapp` with no tag at all. What `imagePullPolicy` does Kubernetes apply by default, and when does the kubelet consult the registry?
4. A Pod has two containers. Container A requests 100m CPU and 128Mi memory, with limits of 200m and 256Mi. Container B requests 50m CPU and 64Mi memory, with no limits at all. What QoS class does this Pod receive?
5. A node reports `Ready=False`. What is that condition actually telling you?
6. Your control plane runs Kubernetes 1.37. What is the oldest kubelet minor version that is supported against it?
7. A Pod runs three containers: `app`, `cache`, and `log-shipper`. How do you ask for the logs of `cache` specifically?
8. Someone creates an Ingress object on a cluster where no Ingress controller has ever been installed. Does the object get created? Does anything route traffic?

Answers + reading strategy

1. Container state. Reason is a field on the container state, not on the Pod phase. The five Pod phases are `Pending`, `Running`, `Succeeded`, `Failed`, and `Unknown`, and none of them is called `ContainerCreating`. [source:

k8s-docs-pod-failure-signatures-2026-08-31] *[cross-bearing: see Ch 5 §5 — Pod phases and container states]*

2. No. Nothing is retrying it with looser constraints. Pending is a stable report that no feasible node was found. The scheduler will place the Pod the moment a node becomes feasible, but it will not relax the Pod's own requirements to make that happen. *[cross-bearing: see Ch 7 §2 — feasible nodes]*

3. With no tag, Kubernetes assumes :latest, and the default imagePullPolicy for :latest is Always. The kubelet queries the registry every time it launches a container. [source: k8s-docs-images-2026-08-23] *[cross-bearing: see Ch 2 §6 — imagePullPolicy]*

4. Burstable. It is not Guaranteed, because container B has no limits and container A's limits do not equal its requests. It is not BestEffort, because at least one container has requests. [source: k8s-docs-pod-qos-2026-08-24] *[cross-bearing: see Ch 5 §8 — QoS classes]*

5. Ready=False means the node is not healthy and is not accepting Pods. [source: k8s-docs-node-status-2026-08-24] The kubelet is what reports a node's status, in the form of updates to the Node's .status. [source: k8s-docs-node-controller-heartbeats-2026-08-31] *[cross-bearing: see Ch 8 §4 — node conditions]*

6. 1.34. The kubelet may be up to three minor versions older than the API server, and must never be newer. [source: k8s-version-skew-policy-2026-08-31] *[cross-bearing: see Ch 8 §6 — the version-skew window]*

7. kubectl logs <pod> -c cache. For a multi-container Pod, -c <container> is how you name the one you mean. [source: k8s-docs-logging-architecture-2026-08-23] *[cross-bearing: see Ch 5 §2 — multi-container Pods]*

8. The object is created and nothing routes traffic. This is exactly the pattern Chapter 10 named: the API server accepts and stores the object, and no controller is watching for it, so it is a record of intent with nobody to honor it. *[cross-bearing: see Ch 10 §3 — an object without its component does nothing]*

If you got 6+ right: Skim §1 through §4 for the signature names and the method, but read **§6 and §7 properly**. Those two sections are where earlier chapters deliberately left debts for this one to pay, and a high Soundings score does not mean you have already collected them.

If you got 3–5 right: Read at normal pace. This chapter is calibrated for you, and it applies rather than introduces, so the reading will feel faster than its length suggests.

If you got 0–2 right: Before you start §1, **re-read Chapter 5 §5**, on Pod phases and container states. Not alongside this chapter; before it. Every section here is a lookup keyed on the phase, and if that taxonomy has faded, the material degrades into a list of strings to memorize, which is precisely the failure this chapter exists to prevent.

Why This Chapter Matters

A Pod is broken. What do you do first?

Almost everyone reaches for the same thing: `kubectl logs`. It is the obvious move. The application is misbehaving, applications write logs, so read the logs. Sometimes that instinct is right. In the three most common failure shapes, it is wrong, and it costs you the first ten minutes of the investigation before handing you a confident wrong answer.

A Pod stuck in Pending has no logs, because it has no container. There is nothing running anywhere for a log to come out of. `kubectl logs` will tell you so, and if you read that message as "the application isn't logging anything," you have just started debugging the wrong thing.

A Pod in `CrashLoopBackOff` does have logs, but the container `kubectl logs` reads by default is the *current* one, which has not started yet, because the kubelet is waiting out a backoff delay before trying again. So the command returns nothing. Meanwhile the application has already started and died five times, and every one of those deaths wrote a stack trace to a log you did not ask for.

A Pod that was `Evicted` is gone from the node entirely. Its containers were terminated to reclaim resources. Whatever it wrote is subject to the node's log rotation and the kubelet's cleanup, and by the time you go looking, the thing you want may not be there.

Three of the most common failure shapes, and in all three the first thing everyone reaches for is the wrong thing. Reading the logs first is like opening a ship's log to find out whether the ship ever left harbor: the book will be silent, and the silence will tell you nothing, because a vessel still at the quay writes no entries either.

So here is the question this chapter opens and does not answer until the end: **what do you read first, and why is it always the same thing?**

You are not going to learn nine error strings. You could get nine error strings from a blog post, and you would forget them within a week, because a list of strings has no structure to hang on. What you are going to learn is a method, the same one practitioners actually use, which is not recognition but narrowing. When an experienced operator meets a Pod they have never seen fail in this particular way, they do not consult a mental glossary. They take a bearing, then another, and read the answer off the intersection — each question they ask the cluster eliminates a whole category of cause, and they arrive at the diagnosis without ever having seen this exact failure before.

That is transferable. Error-string recall is not. Kubernetes will ship a failure signature next year that nobody has written a blog post about yet, and the method will find it anyway.

Dead Reckoning: A Pod's phase tells you which stage of the platform's own start-up sequence stopped. Each stage is owned by a different component. Knowing the stage tells you which component to interrogate, and interrogating the right component is the whole of diagnosis. `kubectl logs` interrogates the application, which is the last component in the sequence and therefore the last one worth asking.

A word about how this chapter treats the exam. CNCF publishes four domain weights and a list of competency names — no sub-competency weights, no item counts, no published statement of which question shapes it favors. So when this chapter calls something high-value, that is the author's judgement from the competency list and the material's own structure, not a published figure, and it is stated as judgement everywhere it appears.

On that judgement: failure signatures are the material most study guides reduce to a two-column glossary — signature on the left, one-line cause on the right. A glossary will get you through a question that names a string. It will not get you through a question that describes a symptom and asks what you would check, and the second shape is the one this chapter is built for, because it is the one the method survives.

What You'll Learn

By the end of this chapter, you'll be able to:

- **Separate** a platform-scope failure from an application-scope one before you spend time on either
- **Read** a Pod's phase and its container-state Reason as the first two coordinates of a diagnosis, rather than as error messages to be recognized
- **Distinguish** the failure signatures that mean *never started* from those that mean *started, and did not stay*
- **Run** `kubectl describe`, `kubectl events`, and `kubectl logs --previous` in the order that actually narrows the problem
- **Recognize** when the node, not the workload, is the broken thing, and when to drop below the Kubernetes API to `crictl`
- **Explain** why `kubectl top` returns an error on a cluster that nobody finished building

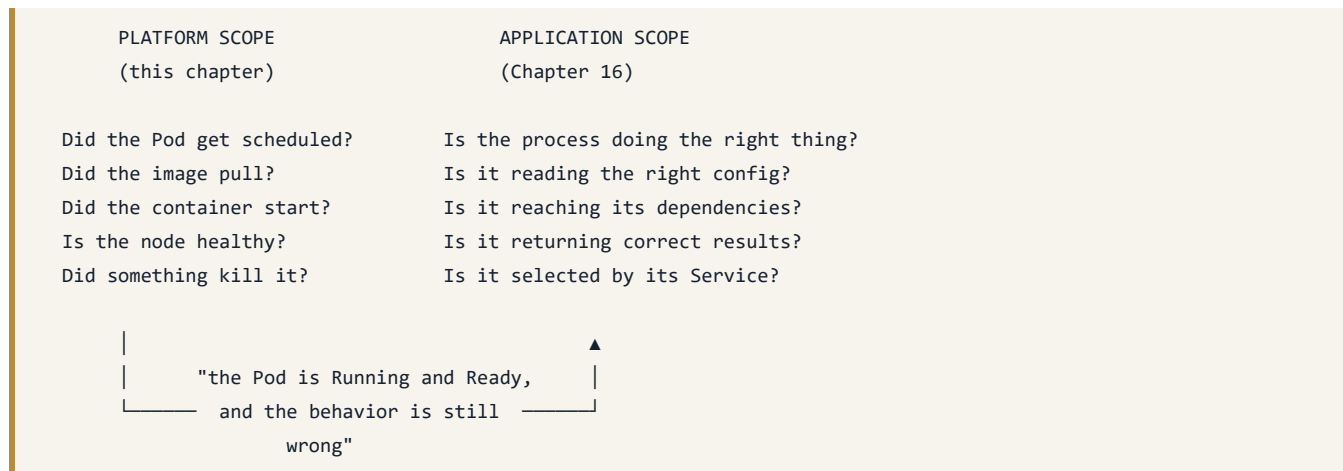
You'll also stop reading logs first, which is the single habit that most reliably separates a practitioner from someone who has read about Kubernetes.

1. \$1 — Whose Problem Is This, and What to Read First

Before you diagnose anything, decide whose problem it is.

The Kubernetes documentation splits its own troubleshooting material along exactly this line. The debugging guide for applications opens by saying it "is to help users debug applications that are deployed into Kubernetes and not behaving correctly. This is *not* a guide for people who want to debug their cluster." [source: k8s-docs-debug-pods-2026-08-31] The cluster guide opens with the mirror image: "This doc is about cluster troubleshooting; we assume you have already ruled out your application as the root cause of the problem you are experiencing." [source: k8s-docs-debug-cluster-2026-08-31]

Each guide assumes you have already done the other one's work. That is not an oversight; it is the split made explicit. There are two audiences, and the first move in any investigation is deciding which one you are.



The handoff runs one way. Platform scope asks whether Kubernetes did its job; application scope asks whether your code is doing its job. You cross the line exactly once per investigation, and only in that direction.

The boundary is sharper than it sounds, and it has a mechanical test. **If the Pod is Running and Ready, the failure is confined to that one workload, and the behavior is still wrong, you have crossed into application scope.** Everything before that (scheduling, pulling, configuring, starting, staying alive, being killed) is the platform doing or failing to do its job, and that is this chapter.

That middle clause is load-bearing, and it is the one people drop. A cluster-wide network fault — DNS not resolving, the CNI plugin not wiring Pods up, the service proxy not programming its rules — produces Pods that are Running and Ready and behave wrongly, and it is unambiguously the platform's problem, not your code's. The tell is that it is not confined to one workload. If several unrelated applications go wrong at once, do not cross the line; go to the network. [cross-bearing: see Ch 9 §1 — CNI and pod networking]

[cross-bearing: see Ch 16 §1 — when the Pod is fine and the application isn't]

That is also why you will not find `kubectl exec`, `kubectl debug`, or `kubectl port-forward` taught here. They are real tools, they are on the KCNA competency list, and they belong to the other chapter. They exist to get you inside a container that is already running, which is a question you only ask once the

platform has succeeded. *[cross-bearing: see Ch 16 §3 — getting inside a container]* and *[cross-bearing: see Ch 16 §5 — bypassing the Service on purpose]*

The order, and why it is that order

Once you know you are in platform scope, the sequence is fixed:

Scope → phase → conditions → events → logs.

Logs come last. That is the part everyone resists, so it gets an argument rather than an assertion.

Go back to the three failures from the chapter opening. A Pending Pod has no container, so a log read returns nothing, and "nothing" is indistinguishable from "the app is quiet." A CrashLoopBackOff Pod has a container that has not started yet, so a log read returns nothing, and again "nothing" looks like silence rather than a container that has died five times. An Evicted Pod is not on the node at all, so a log read may find nothing. Once more: "nothing."

Three completely different causes. One identical output. **The logs cannot distinguish between them, and the phase distinguishes between them instantly.**

That is the whole argument. Logs are not useless; logs are the *last* signal to become trustworthy, because they only mean what you think they mean once you already know the container ran. The phase tells you whether the container ran. So the phase goes first.

The order is not arbitrary and it is not a ritual. Each reading narrows the water the next one has to search. Scope tells you which cluster you are looking at; the phase tells you which shore; the conditions tell you which rock; the events tell you what the lookout actually called. By the time you reach the logs, there is very little water left, which is exactly why the logs can finally be read for what they say rather than for what you hope they say.

 **Mnemonic: S-P-C-E-L** — Scope, Phase, Conditions, Events, Logs. If you find yourself typing `kubect1 logs` and you cannot say out loud what phase the Pod is in, you have skipped four steps.

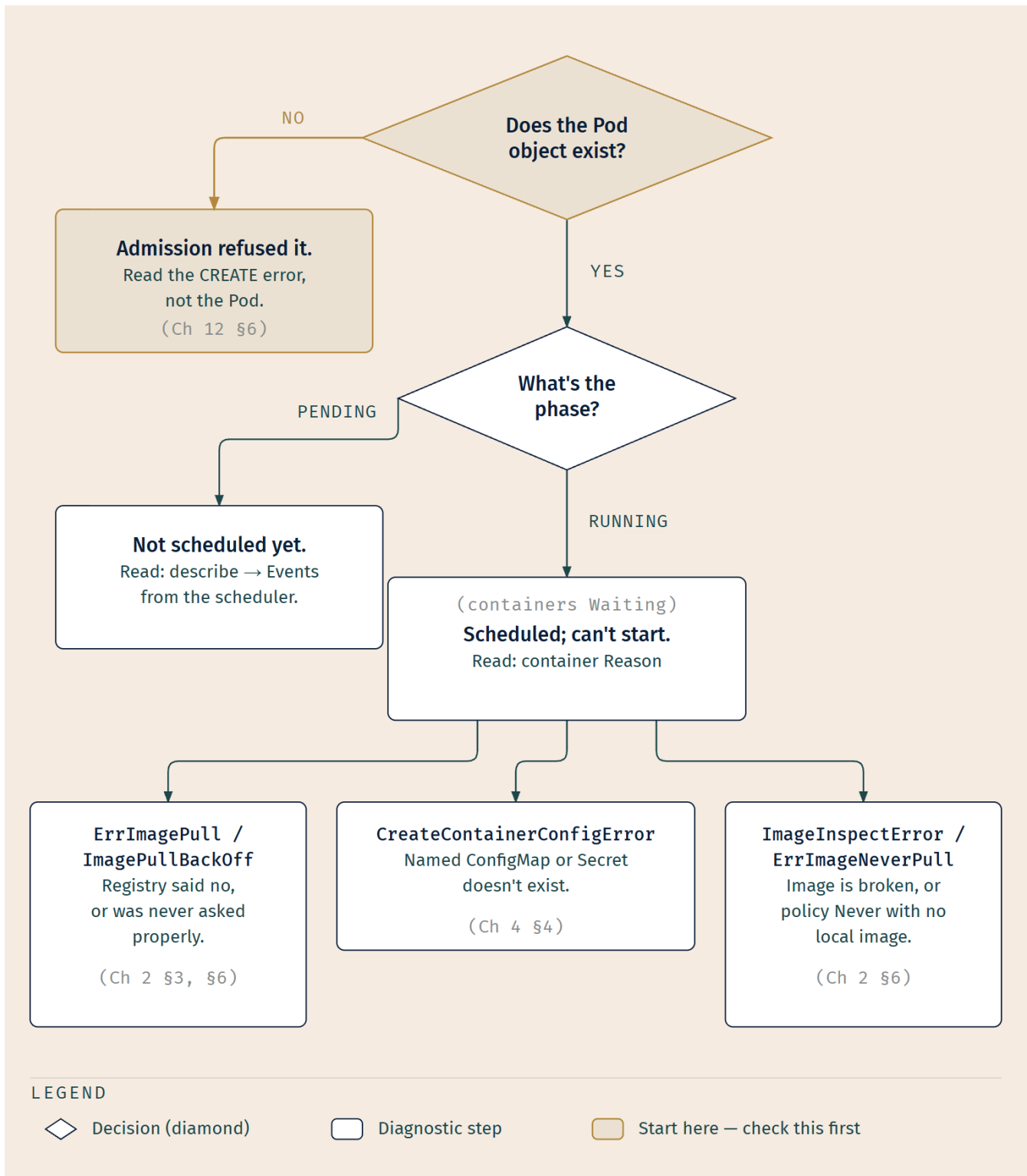
The phase, meanwhile, is not an error message. It is a position in the platform's own start-up sequence, and each position has a component that owns it: the scheduler owns placement, the kubelet owns pulling and starting, the container runtime owns running, and the node controller owns whether the node is answering at all. Reading the phase is not recognizing a string. It is taking your first bearing, and it is the bearing that tells you which coast the rest of the fix is measured against.

[cross-bearing: see Ch 5 §5 — Pod phase and container state]

..1 §2 — Pods That Never Start

This is the largest failure family, and the one where the method pays off hardest. Almost every signature in it produces the same visible symptom, a Pod that is not running, from a completely different cause.

Every failure in this section shares one property: **no container ever executed**. Nothing ran, so nothing logged, so no amount of log-reading will help you. What separates them is *how far into the start-up sequence the platform got before it stopped*.



The first question is not "what's the error" — it is "is there a Pod at all." That branch alone eliminates an entire category of confusion.

The case where there is no Pod

Start at the top of the tree, because this is the branch people skip.

If a Pod was rejected at admission — by Pod Security Admission [source: k8s-docs-pod-security-admission-2026-08-31], by a validating webhook, by a quota — **there is no Pod object to describe**. The refusal happened at the harbor entrance, and there is no vessel inside to inspect. Chapter 12 already flagged this to you and said it "shows up at a different point in the triage flow"; here is that point.

The consequence is practical. `kubectl get pod myapp` returns `NotFound`, and the natural reading of `NotFound` is "something deleted it" or "I'm in the wrong namespace." Neither is true. The object never existed, because the request to create it was rejected, and the rejection message went to whoever issued the create: your terminal, if you ran `kubectl apply`; the Deployment's ReplicaSet status, if a controller was creating it on your behalf.

📌 **Snag:** When a Deployment's Pods are refused at admission, the Deployment does not fail loudly. It sits at zero available replicas, and the reason is on the **ReplicaSet**, not on any Pod. `kubectl describe replicaset <name>` is where the refusal message is. Chasing the missing Pod will find you nothing, because there is nothing to find.

[cross-bearing: see Ch 12 §6 — Pod Security Admission]

Pending: scheduled nowhere

If the Pod exists and its phase is `Pending`, the platform has accepted it and not placed it. The documentation is direct: "If a Pod is stuck in Pending it means that it can not be scheduled onto a node. Generally this is because there are insufficient resources of one type or another that prevent scheduling." [source: k8s-docs-debug-pods-2026-08-23]

The instruction that follows is the important one: "Look at the output of the `kubectl describe` command; there should be messages from the scheduler about why it can not schedule your pod." [source: k8s-docs-debug-pods-2026-08-23]

Note what that sentence assumes. The scheduler *tells you*. It writes an event explaining which nodes it considered and why each one failed. You do not have to deduce anything. You have to go and read it.

This is the point Chapter 7 made and this chapter now cashes in. A Pod in `Pending` is not drifting and it is not broken. It is riding at anchor, waiting for a berth that has not opened, and nothing in the cluster is quietly slackening the lines to make it fit. If the Pod requests 8 CPUs and no node has 8 CPUs free, it will sit there indefinitely, and it will sit there just as patiently if the reason is a taint it does not tolerate.

⚠ Navigational Hazards

A capacity shortage and a taint look identical from the outside. Both produce a Pod in Pending, indefinitely, with no containers. `kubectl get pods` shows you exactly the same line for both.

They are completely different problems. A capacity shortage means the cluster is full and you need to reduce requests, delete Pods, or add nodes. A taint means the nodes have capacity and are refusing this particular Pod on purpose; you need a toleration, or you need to accept that the refusal is correct.

The only thing that distinguishes them is the scheduler's own event text, which reports how many nodes failed and on what grounds. This is why "read the events" is not a formality. It is the single step that separates two problems with opposite fixes.

[cross-bearing: see Ch 7 §2 — feasible nodes, and why a Pod stays Pending] and [cross-bearing: see Ch 7 §4 — taints and tolerations]

Two other Pending causes surprise people often enough to name. Exhausting CPU or memory across the cluster is the obvious one: "you may have exhausted the supply of CPU or Memory in your cluster, in this case you need to delete Pods, adjust resource requests, or add new nodes to your cluster." [source: k8s-docs-debug-pods-2026-08-23] The less obvious one is `hostPort`: "when you bind a Pod to a `hostPort` there are a limited number of places that pod can be scheduled; in most cases `hostPort` is unnecessary, try using a Service object to expose your Pod." [source: k8s-docs-debug-pods-2026-08-23] A `hostPort` silently reduces your feasible-node count to "nodes where that port is free," which on a small cluster can be one node, or zero.

An unbound PersistentVolumeClaim belongs in this family too: a storage problem can arrive disguised as a scheduling problem, with a Pod sitting in Pending for reasons that have nothing to do with CPU, memory, or taints *[cross-bearing: see Ch 11 §2 — PV and PVC binding]*. Chapter 11 promised you would be able to tell the two apart from the symptoms, so here is how — and the answer is that you have to look at the claim, because the Pod cannot tell you.

Run `kubectl get pvc` beside `kubectl get pods`. A claim reading Pending means it has not found a volume, and **claims remain unbound indefinitely if a matching volume does not exist** [source: k8s-docs-persistent-volumes-depth-2026-08-25] — there is no timeout and no eventual failure, which is why this looks so much like a Pod that simply will not schedule.

Then read the claim's StorageClass, because **the binding mode inverts the direction of cause**:

- Under **Immediate**, binding happens as soon as the claim is created, without knowledge of any Pod's scheduling requirements — which the documentation notes "may result in unschedulable Pods" [source: k8s-docs-storage-classes-2026-08-25]. A Pending claim here is a genuine storage problem, and it is upstream of the Pod. Fix the storage and the Pod schedules.
- Under **WaitForFirstConsumer**, binding is *deliberately* delayed until a Pod using the claim is created, so that the volume can be provisioned to match that Pod's scheduling constraints [source: k8s-

docs-storage-classes-2026-08-25]. A Pending claim here may be entirely normal — it is waiting on the scheduler. The Pod is not blocked by the claim; the claim is waiting for the Pod. Diagnose the Pod's scheduling constraints, not the storage.

☆ **Fixed Point:** A Pending Pod with a Pending claim is a **storage** problem under Immediate binding and a **scheduling** problem under WaitForFirstConsumer. Same two symptoms, opposite direction of cause, and the StorageClass is the only thing that tells you which.

Waiting: scheduled, and unable to start

If the Pod has been placed on a node but its containers are not running, the containers are in the Waiting state, and Waiting carries a Reason field that names the specific problem. The documentation frames the whole category: "If a Pod is stuck in the Waiting state, then it has been scheduled to a worker node, but it can't run on that machine. The most common cause of Waiting pods is a failure to pull the image." [source: k8s-docs-debug-pods-2026-08-23]

This is where the phase-versus-state distinction earns its keep. The *phase* may still read Pending, because the containers have not been made ready to run, or the Pod may show Running with containers not yet up. Either way, the string that tells you what is actually wrong lives on the **container state**, not the phase. You get it from `kubectl describe pod`, which "shows the state for each container within that Pod." [source: k8s-docs-pod-failure-signatures-2026-08-31]

Here are the Waiting reasons that matter for the never-started family, from the documented reason table [source: k8s-docs-pod-failure-signatures-2026-08-31]:

Reason	What it means	Where the cause lives
ContainerCreating	The container is being created.	Nowhere — this is normal, briefly.
ErrImagePull	"There was a general error pulling the image."	Image name, registry, or credentials
ImagePullBackOff	"The container image pull has failed, and kubelet will keep trying."	Same as above — this is ErrImagePull after retries began
ImageInspectError	"There was an error inspecting the container image."	The image itself, or the runtime's view of it
ErrImageNeverPull	"The image pull policy is set to Never, but the image is not present locally."	The Pod spec's imagePullPolicy
PodInitializing	The Pod is being initialized.	An init container that has not finished

One more reason belongs to this family and is **not** in that table: CreateContainerConfigError, covered below.

The image-pull family

`ErrImagePull` and `ImagePullBackOff` are the same problem at two moments in time. The first pull attempt fails and you get `ErrImagePull`. The kubelet then starts retrying with an increasing delay, and the signature becomes `ImagePullBackOff`: "The BackOff part indicates that Kubernetes will keep trying to pull the image, with an increasing back-off delay, up to a compiled-in limit of 300 seconds (5 minutes)." [source: k8s-docs-images-2026-08-23]

So `ImagePullBackOff` is not a different diagnosis from `ErrImagePull`. It is the *same* diagnosis, observed a little later. If you see `ErrImagePull` and refresh, you will usually watch it become `ImagePullBackOff`, and nothing has changed except the elapsed time.

The documented cause list is short: "invalid image name, or pulling from a private registry without an `imagePullSecret`." [source: k8s-docs-images-2026-08-23] The debugging guide gives you three checks in order: "make sure that you have the name of the image correct; have you pushed the image to the registry?; try to manually pull the image to see if the image can be pulled." [source: k8s-docs-debug-pods-2026-08-23]

That third check is underrated. Pulling the image by hand from a shell on the node, or from your laptop if the registry is reachable from there, collapses the question "is this a Kubernetes problem or a registry problem?" in about five seconds.

The subtle version of this failure involves the pull policy. Recall that if you specify no tag, or the tag `:latest`, the default policy is `Always`, and the kubelet "queries the container image registry to resolve the name to an image digest" on every container launch. [source: k8s-docs-images-2026-08-23] A cached image is provisions already aboard, but with `Always` the kubelet still has to hail the registry before it will use them. That means a Pod which has been running happily for weeks on a cached image will fail to restart the moment the registry becomes unreachable. A Pod pinned to a specific tag other than `:latest` gets `IfNotPresent` by default and rides out the same registry outage without noticing.

🔍 **Closer Look:** `Always` does not mean "download the layers every time." The kubelet resolves the tag to a digest, and "if the kubelet has a container image with that exact digest cached locally, it uses its cached image." [source: k8s-docs-images-2026-08-23] The bandwidth cost is small. The *availability* cost is not: the registry has to answer, and if it doesn't, the container doesn't start.

[cross-bearing: see Ch 2 §3 — tags and digests] and *[cross-bearing: see Ch 2 §6 — imagePullPolicy and image pull secrets]*

`ErrImageNeverPull` is the mirror-image mistake. Somebody set `imagePullPolicy: Never`, usually to force use of an image they built locally on the node, a common pattern in `minikube` and `kind` development, and the image is not actually there. Kubernetes does exactly what it was told: it does not fetch, so the container cannot start. The signature is unambiguous and the fix is either "put the image on that node" or "stop saying `Never`."

`ImageInspectError` is the odd sibling, and it is the one every treatment of this family skips. Here the fetch is not the problem — the runtime has the image and cannot read it. "There was an error inspecting the container image." [source: k8s-docs-pod-failure-signatures-2026-08-31] Diagnostically, that points you at the image itself or at the runtime's view of it — a corrupt or truncated layer, a manifest the runtime cannot parse, a partly-populated image cache — and away from the registry, the image name, and the pull credentials, which are the first three things anyone checks. It is rare. It is also unmistakable once you know what it rules out.

CreateContainerConfigError: the configuration cannot be assembled

This one is worth dwelling on because the symptom is so far from the cause.

A container's configuration includes everything the kubelet has to gather before it can hand a container definition to the runtime: environment variables, mounted volumes, and, crucially, the contents of any `ConfigMap` or `Secret` the Pod references. If a referenced `ConfigMap` does not exist, or a referenced `Secret` does not exist, or a named key inside one of them is missing, the kubelet cannot finish assembling the container. It stops, and reports `CreateContainerConfigError`.

Notice what has *not* gone wrong. The Pod scheduled successfully; the node was feasible. The image pulled successfully; the registry answered. Every earlier stage worked. The failure is at the last step before the container runs, and it is caused by an object in a completely different part of your manifests.

Chapter 12 pointed a reader here for exactly this case: a Pod that references a `Secret` which does not exist. It will not start, it will never start, and nothing about the message mentions `Secrets` unless you read the events.

📌 **Snag:** A missing `ConfigMap` and a *misspelled key inside an existing ConfigMap* produce the same reason string. `describe` will name which one it could not resolve. Read the message, not just the reason.

[cross-bearing: see Ch 4 §4 — *ConfigMaps and Secrets*]

Init containers, from the platform side

If a Pod's containers show `PodInitializing`, an init container is still running, or is failing. [source: k8s-docs-pod-failure-signatures-2026-08-31] Because init containers run before the app containers do, a single init container that keeps failing will hold the entire Pod at the gate indefinitely. [cross-bearing: see Ch 5 §3 — *init containers*]

From platform scope, the diagnosis is: identify *which* init container is stuck, and confirm whether it is running or looping. That is a `describe` question. What the init container is actually doing wrong — the logic inside it, the dependency it is waiting on that never appears — is application scope.

[cross-bearing: see Ch 16 §2 — *debugging init containers*]

☆ Fixed Point

Every failure in this section means no container ever executed. Pending, the image-pull family, CreateContainerConfigError, ErrImageNeverPull — in each case the platform stopped before running your code. Reading application logs cannot help, because there are no application logs. The diagnosis lives in `kubectl describe` and in the events, always.

§3 — Looking Inside

You have now seen the method twice and been told twice to "read the events." Here is what that actually means, and what the three commands genuinely do.

First: confirm you are talking to the cluster you think you are

Before you trust a single line of output, confirm which cluster answered. The official troubleshooting guide for `kubectl` itself opens on this: it exists for people who "encounter issues accessing `kubectl` or connecting to your cluster," and its first instruction is to "make sure you have installed and configured `kubectl` correctly on your local machine. Check the `kubectl` version to ensure it is up-to-date and compatible with your cluster." [source: k8s-docs-troubleshoot-kubectl-2026-08-31]

The failure this prevents is not exotic. You are looking at a staging cluster while debugging a production incident; you are in the `default` namespace while your workload lives in `payments`; the Pod "does not exist" because you are asking the wrong API server. Every one of those produces confident, well-formatted, entirely irrelevant output.

`kubectl config current-context` costs you one second and eliminates the whole category.

[cross-bearing: see Ch 8 §1 — kubeconfig and contexts]

`kubectl describe`: the first real question

"The first step in debugging a Pod is taking a look at it. Check the current state of the Pod and recent events." [source: k8s-docs-debug-pods-2026-08-31]

```
kubectl describe pod <pod-name>
```

`describe` gives you, in one screen, the phase, every container's state and Reason, the node it landed on (or the absence of one), the volumes it wants, and, at the bottom, a list of recent events concerning this object.

The guidance on reading it is deliberately unglamorous: "Look at the state of the containers in the pod. Are they all Running? Have there been recent restarts? Continue debugging depending on the state of the pods." [source: k8s-docs-debug-pods-2026-08-23]

Two of those three questions are the ones this chapter is organized around. *Are they all running* separates §2 from §4. *Have there been recent restarts* is the tell for §4's entire family: a restart count above zero means the container ran, which rules out everything in §2 instantly.

🔒 **Worth Securing:** The restart count in `kubectl get pods` output is a diagnosis all by itself. Zero restarts and not running means never started; you are in §2. Non-zero restarts means it started and something ended it; you are in §4. One column, and it halves your search space before you have described anything.

Events: a first-class surface, not a footer

Most people meet events as the bottom third of `describe` output and treat them as supplementary detail. They are not. Events are how the components of the cluster tell you what they did and why, and for several failure modes they are the *only* place the reason is written.

The scheduler's explanation of why no node was feasible: an event. The kubelet's report that the image pull failed and what the registry said: an event. The Deployment controller's report that a rollout exceeded its progress deadline: an event.

```
kubectl events --for pod/<pod-name>
kubectl get events --sort-by=.metadata.creationTimestamp
```

The second form is the one to reach for when you do not yet know which object is at fault. Events are not sorted usefully by default, and a namespace's event stream read in creation order is a chronology of everything the cluster tried to do recently.

Chapter 6 left you a specific promise here. A Deployment that stalls reports `ProgressDeadlineExceeded`, a condition which says the rollout did not finish in time and says nothing at all about *why*. Several quite different underlying causes produce that identical condition, and Chapter 6 told you there were six of them. Here they are, from the same page Chapter 6 drew on: **insufficient quota, readiness probe failures, image pull errors, insufficient permissions, limit ranges, and application runtime misconfiguration** [source: `k8s-docs-deployment-spec-fields-2026-08-24`]. Notice how little they have in common. Two are about permission, one is about the registry, one is about the application's own health reporting, and one is about a quota you may not have known existed. A single condition string covers all six, which is exactly why the condition is a starting point and not a diagnosis. The events on the ReplicaSet and on its Pods are where the actual reason lives.

Worked through: a Deployment shows `ProgressDeadlineExceeded`. You describe the Deployment and learn only that it gave up waiting. You then find the new ReplicaSet it created (`kubectl describe deployment` names it), describe that, and find it created three Pods. You describe one of those Pods and find its container `Waiting With Reason: ImagePullBackOff`, and its events carry the registry's actual refusal: an authentication failure. The rollout stalled because the new image tag was pushed to a registry the cluster has no pull secret for.

ProgressDeadlineExceeded never mentioned images, registries, or credentials. Three describe calls down the ownership chain did.

[cross-bearing: see Ch 6 §4 — rolling updates and progress deadlines]

Events expire, and this matters more than it sounds

An Event is a Kubernetes object like any other. It lives in etcd, it appears in the API, and, unlike most objects, **it is deleted automatically after a retention window**. The API server's `--event-ttl` flag sets that window. [source: k8s-apiserver-event-ttl-and-toleration-defaults-2026-08-31]

The window is bounded and it is short — short enough that a failure investigated the next working day will have no events left. Read the flag on your own cluster rather than assuming a figure. That has a consequence people learn the hard way:

⚠ Navigational Hazards

The absence of an event is not evidence. If a Pod failed overnight and you investigate in the morning, its events are gone. `kubectl describe` will show `Events: <none>`, which reads exactly like "nothing happened."

Something happened. The record of it expired. A log that nobody keeps is a passage nobody can reconstruct.

The practical rule: an empty event list on an object that is currently healthy tells you nothing at all, and an empty event list on an object that failed some time ago tells you only that you are too late. Never conclude "there was no error" from an absent event. Conclude "I need a different source": a monitoring system, a log aggregator, or a reproduction.

[cross-bearing: see Ch 8 §2 — auditing] — the audit log answers a different question (who called the API, and what did they change) and has its own retention, but when events have expired it is sometimes the only surviving record that a change was made at all.

kubectl logs, and the three flags that matter

Now, finally, logs.

```
kubectl logs <pod-name>
kubectl logs <pod-name> -c <container-name>
kubectl logs <pod-name> --all-containers
kubectl logs <pod-name> --previous
```

Kubernetes "captures logs from each container in a running Pod" [source: k8s-docs-logging-architecture-2026-08-31], and by default `kubectl logs` reads the current instantiation of the container in a single-container Pod.

-c names one container. For a multi-container Pod, `-c <container>` is how you say which one you mean. [source: k8s-docs-logging-architecture-2026-08-23] Chapter 5 handed this case to this section by name. On a Pod with `app`, `cache`, and `log-shipper`, a bare `kubectl logs` cannot know which you intended, and reading the wrong container's silence and concluding the application is broken is a real and common misdiagnosis that costs people entire afternoons.

`--all-containers` returns all of them at once, which is what you usually want when you do not yet know which container is at fault.

`--previous` is the one that matters most in this chapter. It "retrieves logs from a previous instantiation of a container." [source: k8s-docs-logging-architecture-2026-08-23] This is the flag that reads the container that *died*, and it is the answer to the second of the three failures from this chapter's opening.

☆ Fixed Point

On a crash-looping Pod, `kubectl logs` reads the container that has not started yet, and returns nothing. `kubectl logs --previous` reads the container that died, and returns the reason.

If a Pod is restarting and its logs are empty, you have used the wrong command, not discovered a silent application.

One constraint on all of this: the kubelet keeps a bounded amount. "By default, if a container restarts, the kubelet keeps one terminated container with its logs." [source: k8s-docs-logging-architecture-2026-08-23] One. Not a history. `--previous` gets you the most recent death, and the one before that is gone.

[cross-bearing: see Ch 5 §2 — multi-container Pods]

01 SCOPE

```
kubectl config current-context
```

Right cluster? Right namespace?

02 PHASE

```
kubectl get pods
```

Pending? Running? Restarts > 0?

```
kubectl get pod <name> -o wide
```

Which node? Which IP?

03 CONDITIONS

```
kubectl describe pod <name>
```

Container state + Reason.

```
kubectl describe node <node>
```

Node conditions, if suspect.

04 EVENTS

```
kubectl events --for pod/<name>
```

What did the components SAY?

```
kubectl get events --sort-by=...
```

Chronology, when object unknown.

05 LOGS

```
kubectl logs <name> -c <container>
```

```
kubectl logs <name> --previous
```

*Only meaningful once you know
the container actually ran.*

LEGEND

Accent = read this step last

kubect1 = exact command to run

Each step narrows the next one. Skipping to the bottom does not save time — it produces output you cannot interpret, because interpretation depends on everything above it.

Taking Your Bearings: Pods That Never Start, and the Commands That Tell You Why

Six questions. Take them before reading on.

1. A Pod's containers show Reason: CreateContainerConfigError. Which of the following is definitely true?

A) The image could not be pulled from the registry B) The Pod could not be scheduled to any node C) The Pod was scheduled and the image is available, but a referenced object could not be resolved D) The container started and then exited with a configuration error

2. You run `kubect1 get pods` and see a Pod with STATUS: ImagePullBackOff and RESTARTS: 0. What does the restart count of zero tell you?

A) The container is being created and the count has not incremented yet B) The container has never executed, which is consistent with a pull failure C) The container ran once and was not restarted D) restartPolicy must be set to Never

3. A Pod has been in Pending for twenty minutes. Which single command output most directly tells you why?

A) `kubect1 logs <pod>` B) `kubect1 describe pod <pod>` — specifically the events from the scheduler C) `kubect1 get pod <pod> -o yaml` — specifically the .spec section D) `kubect1 top pod <pod>`

4. Your `kubect1 apply` of a Deployment succeeded, but no Pods exist and the Deployment shows zero available replicas. Where is the explanation most likely to be?

A) In the Pods' events — you need to find the Pod name first B) In the kubelet's logs on the target node C) On the ReplicaSet, because the Pod creation was refused before any Pod object existed D) In the scheduler's logs

5. A Pod spec sets `imagePullPolicy: Never` and names an image that exists in your registry but not on the node. What happens?

A) The kubelet pulls the image anyway, because the image exists B) The container starts using a cached copy from another node C) `ErrImageNeverPull` — the kubelet does not fetch, and the image is not local D) The Pod stays in `Pending` until the image is manually pushed to the node

6. *[retrieval: ch2]* You want a Pod to run one exact, unchangeable build of an image, with no possibility that a moved pointer silently gives you different content. Do you reference the image by tag or by digest, and why?

A) By tag, because tags are validated by the registry on every pull B) By digest, because a digest is a hash of the image's content and is immutable, while a tag can be moved to point at a different image C) Either — tags and digests are two spellings of the same identifier D) By tag with `imagePullPolicy: Always`, which makes tags immutable

Answers with explanations

1. C.

`CreateContainerConfigError` occurs at the last step before the container runs. The kubelet is assembling the container's configuration and cannot resolve something the Pod references, typically a missing `ConfigMap` or `Secret`, or a missing key inside one.

- **A is wrong** because a pull failure produces `ErrImagePull` or `ImagePullBackOff`. If you are seeing `CreateContainerConfigError`, the image pull already succeeded.
- **B is wrong** for the same shape of reason: an unscheduled Pod is `Pending` with no container state at all. Container state exists only for Pods that have been placed on a node.
- **D is wrong**, and it is the most instructive distractor. Nothing executed. "Started and then exited" is the `S4` family, and it always shows a non-zero restart count. This one has zero.

2. B.

Restart counts are the cheapest signal in the whole chapter. A restart implies a container ran and ended. Zero restarts alongside a `Waiting` reason means the container has not run even once, which is exactly what a pull failure means.

- **A is wrong**, and it targets a genuine confusion between two `Waiting` reasons. `ContainerCreating` is the normal, brief state of a container being created; `ImagePullBackOff` is a container that cannot be created because the image has not arrived. The stem names the second, not the first, and there is no pending increment to wait for.
- **C is wrong** because a container that ran once and was not restarted would be `Terminated`, not `Waiting` with a pull-failure reason.
- **D is wrong**. `restartPolicy` governs what happens *after* a container exits, and nothing has exited here. `Always` is the default and would be equally consistent with this output.

3. B.

The scheduler writes an event explaining why it could not place the Pod, reporting how many nodes failed and on what grounds. That event is at the bottom of `describe` output.

- **A is wrong**, and it is the chapter's opening mistake. A Pending Pod has no container, therefore no logs.
- **C is wrong**. `.spec` tells you what you asked for, which you already know. The scheduler's explanation of why the request could not be satisfied is not in `.spec`.
- **D is wrong** on two counts: `kubectl top` reports usage for running workloads, and on most clusters it fails outright for the reason §7 covers.

4. C.

If Pod creation was refused at admission, there is no Pod object to describe, and the refusal message goes to the controller that attempted the create: the ReplicaSet.

- **A is wrong** because there is no Pod name to find. This is precisely the trap: the reflex to hunt for the Pod produces `NotFound` and a false conclusion.
- **B is wrong** because the kubelet was never involved. The request never got past the API server.
- **D is wrong** for the same reason. The scheduler only sees Pods that exist.

5. C.

`ErrImageNeverPull` is documented as exactly this case: "The image pull policy is set to `Never`, but the image is not present locally." [source: [k8s-docs-pod-failure-signatures-2026-08-31](#)]

- **A is wrong**. `Never` means never. The kubelet "does not try fetching the image." [source: [k8s-docs-images-2026-08-23](#)]
- **B is wrong**. There is no cross-node image sharing. Each node's image cache is its own.
- **D is wrong**. The Pod is not `Pending`; it has already been scheduled to a node. The failure is at the container-start stage, not the scheduling stage. This distractor tests whether you can place the failure in the sequence.

6. B. [retrieval: ch2]

Digests are "a unique identifier for a specific version of an image — a hash of the image's content — and are immutable; tags can be moved to point to different images." [source: [k8s-docs-images-2026-08-23](#)]

- **A is wrong**. Registries do not validate that a tag still points where it once did, because moving tags is a supported operation.
- **C is wrong**, and this is the confusion the question exists to catch. They are different kinds of identifier: one is a mutable pointer, one is content-derived and fixed.
- **D is wrong**. `Always` changes *when* the kubelet resolves the tag, not whether the tag can move. It resolves more often, to whatever the tag currently names.

If you got 5–6: You have the never-started family. §4 is the other half of the split, and it is easier once this half is solid.

If you got 3–4: Solid. Re-read the signature map figure in §2 and check which branch you lost.

If you got 0–2: Go back to **§2** before continuing, specifically the distinction between "there is no Pod object" and "there is a Pod that has not started." Everything in §4 assumes you can tell never-started from started-and-gone, and that judgment is built in §2.

Checkpoint: You've Now Mastered

✓ The never-started signature family and what separates its members ✓ Why Pending is a report rather than an error ✓ The triage order, and the argument for why logs come last ✓ Reading describe, events, and logs in a sequence that narrows

Next: the failures where the container definitely ran — and something ended it.

..1 §4 — Pods That Start and Then Don't Stay

Every signature in §2 meant no container executed. Every signature here means one did.

That single distinction is the most valuable thing in this chapter, and the restart count gives it to you for free. A container that ran left evidence: an exit code, a termination reason, and logs you can read with --previous. The diagnostic problem is no longer "why won't it start" but "what ended it, and what triggered the ending."

CrashLoopBackOff: the most misread string in Kubernetes

Say what it means out loud, because the name works against you:

The container started. It ran. It exited. Kubernetes restarted it. It exited again. Kubernetes is now waiting before trying a third time.

Every one of those steps is a success from the platform's point of view. The image pulled. The config resolved. The process launched. The platform did its entire job correctly, and your process ended.

The documented sequence is precise [source: k8s-docs-container-restart-backoff-2026-08-31]:

1. **Initial crash:** Kubernetes attempts an immediate restart based on the Pod `restartPolicy`.
2. **Repeated crashes:** After the initial crash Kubernetes applies an exponential backoff delay for subsequent restarts. This prevents rapid, repeated restart attempts from overloading the system.
3. **CrashLoopBackOff state:** This indicates that the backoff delay mechanism is currently in effect for a container in a crash loop.
4. **Backoff reset:** If a container runs successfully for a certain duration, Kubernetes resets the backoff delay.

Point three is the one to internalize. `CrashLoopBackOff` does not name the crash. It names **the waiting between crashes**. When you see it, the container is not running and is not failing either; it is sitting out a delay.

The curve is documented: "After containers exit, the kubelet restarts them with an exponential backoff delay: 10s, 20s, 40s, ..., capped at 300 seconds (5 minutes). Once a container executes successfully for 10 minutes without problems, the kubelet resets the restart backoff timer." [source: [k8s-docs-container-restart-backoff-2026-08-31](#)]

Two practical consequences. First, a Pod that has been crash-looping for a while will appear to do nothing for up to five minutes at a stretch. That is the cap, not a hang. Second, the reset requires **ten minutes of successful running**, which means a container that dies every eight minutes never resets its backoff and will keep escalating to the cap forever, even though it is technically "working" most of the time.

The documented causes are broad: "application errors, configuration errors, resource constraints, failing health checks, or probe failures." [source: [k8s-docs-container-restart-backoff-2026-08-31](#)] That breadth is the point. `CrashLoopBackOff` tells you the container is dying, not why. The why is in `kubectl logs --previous`, in the container's exit code, and in the events.

⚠ Navigational Hazards

`CrashLoopBackOff` is not an image problem. It is the strongest available evidence that the image is *fine*: the pull succeeded, the config resolved, and the process started. The word "Loop" is the tell. You cannot loop something that never ran once.

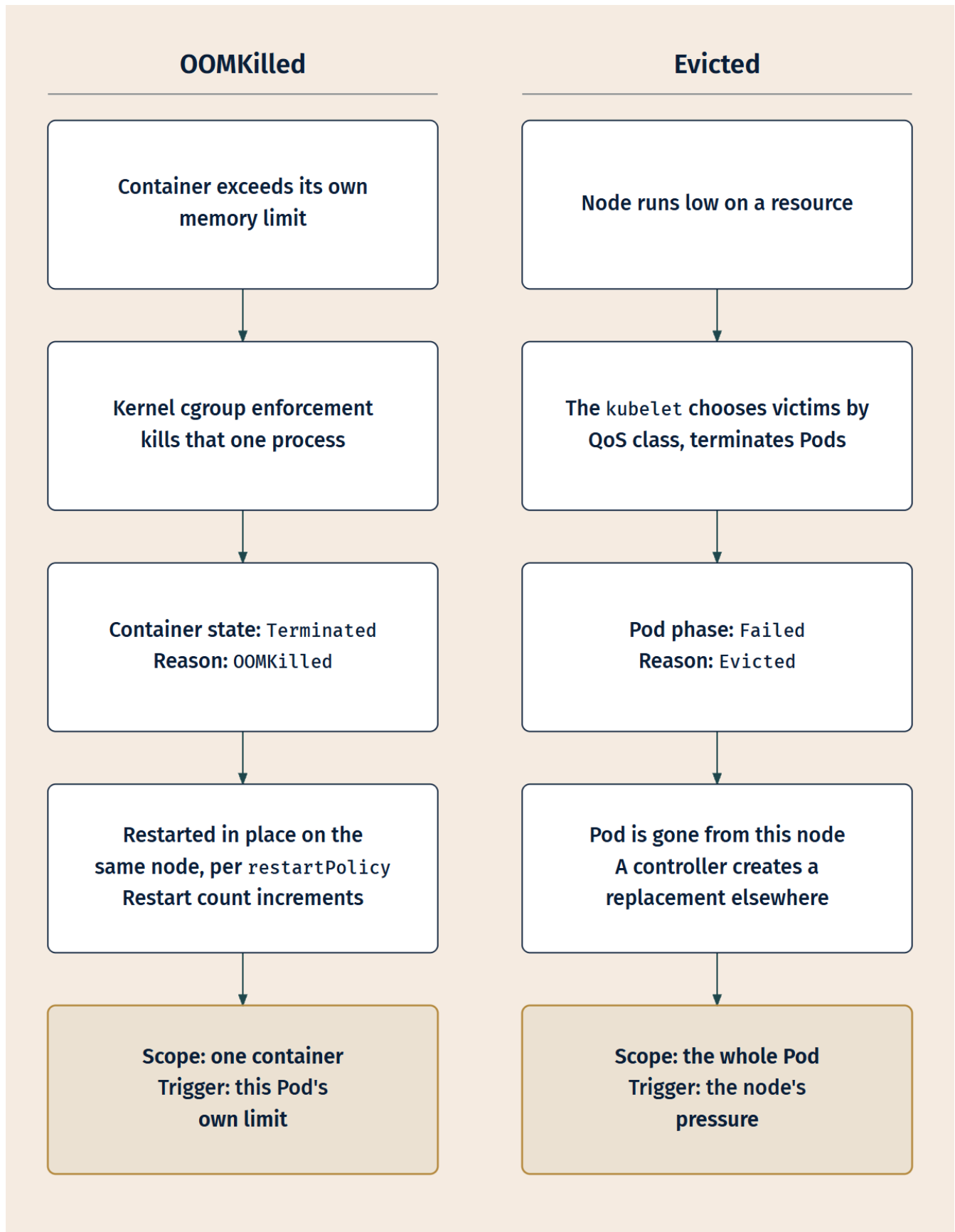
This trap catches people because "BackOff" also appears in `ImagePullBackOff`, and the two strings look like siblings. They are opposites. `ImagePullBackOff` means the kubelet is retrying a *pull*. `CrashLoopBackOff` means the kubelet is retrying a *start*, having already pulled successfully.

Also recall that `restartPolicy` is what makes a loop possible at all. The default is `Always`, and with `Always` a container that exits, even with code 0, even successfully, is restarted. [source: [k8s-docs-container-restart-backoff-2026-08-31](#)] A container that runs a task and exits cleanly, deployed under the default policy, will loop forever. That is not a bug; it is a Deployment being used where a Job belonged.

[cross-bearing: see Ch 5 §4 — restartPolicy and restart backoff]

OOMKilled versus Evicted: the chapter's most confusable pair

Both mean "something ended your workload over memory." Everything else about them is different, and this is the discrimination most worth being able to make cold.



Different trigger, different scope, different outcome. Three axes, and they disagree on all three.

OOMKilled is the container exceeding its own memory limit. The container is ended and the reason is recorded as **OOMKilled** — documented as "The container ran out of memory." [source: k8s-docs-pod-failure-signatures-2026-08-31] The Pod survives. The other containers in it survive. The killed container is restarted in place per its `restartPolicy`, and the restart count goes up.

This is scoped tightly: "Any Container exceeding a resource limit will be killed and restarted by the kubelet without affecting other Containers in that Pod." [source: k8s-docs-pod-qos-2026-08-24]

Chapter 5 put the same event a layer lower, saying that when a container exceeds its memory limit **the kernel** may terminate it [source: k8s-docs-resource-management-2026-08-23]. Both are true and they are not in competition: the kernel does the killing, and the kubelet observes the dead container and applies the Pod's `restartPolicy`. When a question asks who *restarts* the container, the answer is the kubelet. When it asks what *killed* it, the answer is the kernel enforcing a limit the kubelet set for it.

If a container is OOM-killed repeatedly, the visible signature in `kubectl get pods` becomes `CrashLoopBackOff` — that follows from two sourced facts (a container exceeding its limit is killed and restarted; repeated restarts enter backoff) rather than from a source that states it outright. The layering trips people up: `CrashLoopBackOff` and **OOMKilled** are not alternatives, they are two altitudes of the same event, and `describe` shows you the lower one under the container's last state.

🧠 **Mnemonic: OOM** is **O**ne container, **O**ver its own limit, **O**n the same node afterward. Eviction is the node's decision about the whole Pod, and the Pod does not come back to that node.

Out Of Memory, since the acronym has now appeared twice without being spelled out.

Evicted is the kubelet reclaiming resources on a node under pressure. "Node-pressure eviction is the process by which the kubelet proactively terminates pods to reclaim resource on nodes." [source: k8s-docs-node-pressure-eviction-2026-08-31]

The kubelet is watching more than memory: it "monitors resources like memory, disk space, and filesystem inodes on your cluster's nodes. When one or more of these resources reach specific consumption levels, the kubelet can proactively fail one or more pods on the node to reclaim resources and prevent starvation." [source: k8s-docs-node-pressure-eviction-2026-08-31]

The outcome is at Pod scope: "During a node-pressure eviction, the kubelet sets the phase for the selected pods to `Failed`, and terminates the Pod." [source: k8s-docs-node-pressure-eviction-2026-08-31] The Pod is finished. It does not restart in place. If it belongs to a controller, that controller notices and creates a replacement: "If the pods are managed by a workload management object (such as `StatefulSet` or `Deployment`) that replaces failed pods, the control plane (`kube-controller-manager`) creates new pods in place of the evicted pods." [source: k8s-docs-node-pressure-eviction-2026-08-31] A bare Pod with no controller is simply gone.

Before the kubelet touches your workloads it tries to help itself: "The kubelet attempts to reclaim node-level resources before it terminates end-user pods. For example, it removes unused container images when disk resources are starved." [source: k8s-docs-node-pressure-eviction-2026-08-31] So an eviction wave usually means the node has already exhausted its own cheap options.

🔗 **Closer Look:** Node-pressure eviction and API-initiated eviction are explicitly different things. "Node-pressure eviction is not the same as API-initiated eviction." [source: k8s-docs-node-pressure-eviction-2026-08-31] One is the kubelet acting on local pressure; the other is a request through the API server. It matters here because they behave differently under disruption constraints: the kubelet under pressure "does not respect your configured PodDisruptionBudget or the pod's terminationGracePeriodSeconds." [source: k8s-docs-node-pressure-eviction-2026-08-31] A node in trouble stops negotiating.

[cross-bearing: see Ch 8 §4 — cordon, drain, and taking a node out of service]

Eviction order, and the trap in it

When the kubelet has to choose victims, it chooses by QoS class. "When a Node runs out of resources, Kubernetes will first evict BestEffort Pods running on that Node, followed by Burstable and finally Guaranteed Pods." [source: k8s-docs-pod-qos-2026-08-24]

BestEffort first. Burstable second. Guaranteed last.

There is a refinement worth carrying: "When this eviction is due to resource pressure, only Pods exceeding resource requests are candidates for eviction." [source: k8s-docs-pod-qos-2026-08-24] A Pod living within its requests is not a candidate at all. That is what a request buys you: standing, when the kubelet is choosing.

⚠ Navigational Hazards

The request and the limit answer two different questions, and confusing them is the easiest error to make in this material.

The **limit** is what gets your container OOM-killed. Exceed it and the container is ended. Your requests are irrelevant to that event.

The **request**, via the QoS class it helps determine and via the "exceeding requests" rule above, is what determines your standing when the *node* is under pressure and something has to go. Your limits are largely irrelevant to that decision.

Two triggers — your own limit, and the node's pressure. Two scopes — one container, and the whole Pod. An exam question that hands you a Pod spec and asks "what protects this from eviction" is testing whether you reach for the limit when you should reach for the request.

And the trap under the trap:

📌 **Snag:** "BestEffort asks for nothing, so it's the humblest and therefore the safest" is exactly backwards. **BestEffort is evicted first.** Specifying no requests does not make you a good citizen; it makes you the first thing thrown overboard, because the kubelet has no reason to believe you need anything.

[cross-bearing: see Ch 5 §8 — requests, limits, and QoS classes]

Probe failures: two shapes, and one of them is silent

Probes are a deliberate source of the failures in this section, and the two probe types fail in visibly different ways.

A failing liveness probe produces a restart loop. The kubelet kills the container and applies the restart policy. [source: k8s-docs-pod-lifecycle-2026-08-23] From the outside this is indistinguishable from an application that crashes on its own: restart count climbing, eventually `CrashLoopBackOff`. The distinguishing evidence is in the events, where the kubelet records the probe failure, and in the logs, where a self-crashing application usually leaves a stack trace and a probe-killed one usually does not.

That distinction matters because the fixes are opposite. If the application is crashing, fix the application. If the probe is wrong (too short a timeout, too aggressive an `initialDelaySeconds`, a health endpoint that is slow under load) then the application is *fine* and the probe is manufacturing an outage.

A failing readiness probe produces something much quieter. The Pod stays `Running`. It does not restart. It reports `0/1 Ready`. And "the endpoints controller removes the Pod's IP address from the endpoints of all Services that match the Pod." [source: k8s-docs-pod-lifecycle-2026-08-23]

Read that consequence carefully. The Pod is alive, it is consuming its node's resources, `kubectl get pods` shows it as `Running`, and it is receiving **no traffic at all**, because it has been silently removed from its Service's endpoints.

☆ Fixed Point

A liveness probe failure restarts the container. A readiness probe failure removes the Pod from Service endpoints and changes nothing else.

The second is the chapter's quietest failure: `Running`, `0/1 Ready`, zero restarts, no events after the first few, and no traffic. Anyone who reads only the `STATUS` column will conclude the Pod is healthy.

The `READY` column in `kubectl get pods` is the tell. `1/1` and `0/1` both print `Running` next to them.

[cross-bearing: see Ch 5 §7 — liveness, readiness, and startup probes] and [cross-bearing: see Ch 9 §4 — readiness and Service endpoint membership]. From the application side, "is anything even selected" is [cross-bearing: see Ch 16 §4 — is anything even selected].

Logbook Entry: A team once spent most of a day on an intermittent 502 that appeared under load and vanished when they went looking for it. Every Pod was Running. The Deployment showed the expected replica count. Nothing had restarted.

What was happening: under load, one replica's health endpoint took longer than its readiness timeout, failed, and was pulled from the Service. Load redistributed to the remaining replicas, which made *them* slower, so a second one failed readiness, and so on. As soon as the load dropped, every replica recovered, readiness passed again, and all of them rejoined the Service, leaving a system that looked completely healthy by the time anyone finished typing `kubectl get pods`.

The READY column had been telling the story the whole time. Nobody was reading that column, because STATUS said Running and Running reads like *fine*.

The lesson is small and repeats constantly: on a Pod that is running, READY is the more informative column, and a readiness failure leaves no restart count and no crash to find.

§5 — When the Node Is the Problem

Everything so far has assumed the node was working and the workload was not. Sometimes it is the other way around, and the symptoms mislead, because a broken node makes its Pods look broken.

"The first thing to debug in your cluster is if your nodes are all registered correctly." [source: k8s-docs-debug-cluster-2026-08-31]

```
kubectl get nodes
kubectl describe node <node-name>
```

Node conditions, as a diagnostic

You met the node conditions in Chapter 8 — Ready, MemoryPressure, DiskPressure, PIDPressure, NetworkUnavailable — as part of what a node reports about itself. This section does not restate that table [cross-bearing: see Ch 8 §4 — node conditions, and what each one means]. It asks a different question: **given a condition, what do you do next?** The middle column below is a reminder, not a definition; Chapter 8 owns those.

Condition seen	What it is telling you	Your next move
Ready=True	Healthy, accepting Pods	The node is not your problem. Return to the workload.
Ready=False	Unhealthy, and saying so	The kubelet is reporting .status and reporting a problem [source: k8s-docs-node-controller-heartbeats-2026-08-31]. Go to the node and read the kubelet's own logs.
Ready=Unknown	Not saying anything at all	Nobody is reporting at all. Suspect the kubelet process, the machine, or the network to the control plane.
MemoryPressure=True	Memory low	Expect evictions. Your Pod's failure is probably §4's Evicted, not its own fault.
DiskPressure=True	Disk low	Expect image-garbage-collection and evictions. Also a plausible cause of pull failures.
PIDPressure=True	Too many processes	Container starts will fail in confusing ways. Look for a process-leaking workload.

The distinction between `False` and `Unknown` is the one to take to the exam. `False` means somebody is talking to you and telling you they are unwell. `Unknown` means nobody is talking to you at all. Those lead to completely different investigations.

 **Worth Securing:** When several unrelated Pods across the same node fail in different ways at the same time, stop diagnosing the Pods. Describe the node. Correlated, heterogeneous failures on one node are almost never a coincidence of workloads.

[cross-bearing: see Ch 8 §4 — node conditions, cordon and drain]

Heartbeats, and what happens when they stop

A node proves it is alive through two channels. "For nodes there are two forms of heartbeats: Updates to the .status of a Node. Lease objects within the kube-node-lease namespace. Each Node has an associated Lease object." [source: k8s-docs-node-controller-heartbeats-2026-08-31]

Watching those heartbeats is the node controller's job. One of its roles is "monitoring the nodes' health," and specifically: "In the case that a node becomes unreachable, updating the Ready condition in the Node's .status field. In this case the node controller sets the Ready condition to Unknown." [source: k8s-docs-node-controller-heartbeats-2026-08-31]

That answers a question people ask about `Ready=Unknown`: who wrote it, if the node is not talking? The node controller did, on the node's behalf, because the node stopped.

What follows is deliberately unhurried. "If a node remains unreachable: triggering API-initiated eviction for all of the Pods on the unreachable node. **By default, the node controller waits 5 minutes between**

marking the node as Unknown and submitting the first eviction request." [source: k8s-docs-node-controller-heartbeats-2026-08-31]

Five minutes of apparent inaction, on purpose. A node that vanishes for ninety seconds because of a network blip should not have its entire workload torn down and rescheduled. The delay buys time for a transient problem to resolve itself.

The mechanism is taint-based rather than a dedicated timer. "The node controller is also responsible for evicting pods running on nodes with NoExecute taints, unless those pods tolerate that taint. The node controller also adds taints corresponding to node problems like node unreachable or not ready. This means that the scheduler won't place Pods onto unhealthy nodes." [source: k8s-docs-node-controller-heartbeats-2026-08-31] And Pods carry a default toleration for exactly these: "Kubernetes automatically adds a toleration for node.kubernetes.io/not-ready and node.kubernetes.io/unreachable with tolerationSeconds=300, unless you, or a controller, set those tolerations explicitly." [source: k8s-docs-taints-tolerations-depth-2026-08-24]

So the five-minute wait is not a special case bolted on for node failure. It is the same machinery from Chapter 7, doing a job you would not have guessed it was doing.

The node controller also declines to act at scale. If a large share of a zone's nodes go unhealthy at once, "the eviction rate is reduced," and on small clusters "evictions are stopped" entirely; and in the worst case, "when all zones are completely unhealthy (none of the nodes in the cluster are healthy)... the node controller assumes that there is some problem with connectivity between the control plane and the nodes, and doesn't perform any evictions." [source: k8s-docs-node-controller-heartbeats-2026-08-31]

That last one is a piece of engineering humility worth noticing. A compass that reads north in every direction is telling you about the compass. When every node looks dead, the most likely explanation is that the observer is wrong, not that every machine died simultaneously. So the controller stops acting.

Finally, the Pod-level consequence: "If a Node dies, the Pods running on (or scheduled to run on) that node are marked for deletion. The control plane marks the Pods for removal after a timeout period." [source: k8s-docs-pod-failure-signatures-2026-08-31] And they do not come back as themselves: "A given Pod (as defined by a UID) is never 'rescheduled' to a different node; instead, that Pod can be replaced by a new, near-identical Pod." [source: k8s-docs-pod-failure-signatures-2026-08-31]

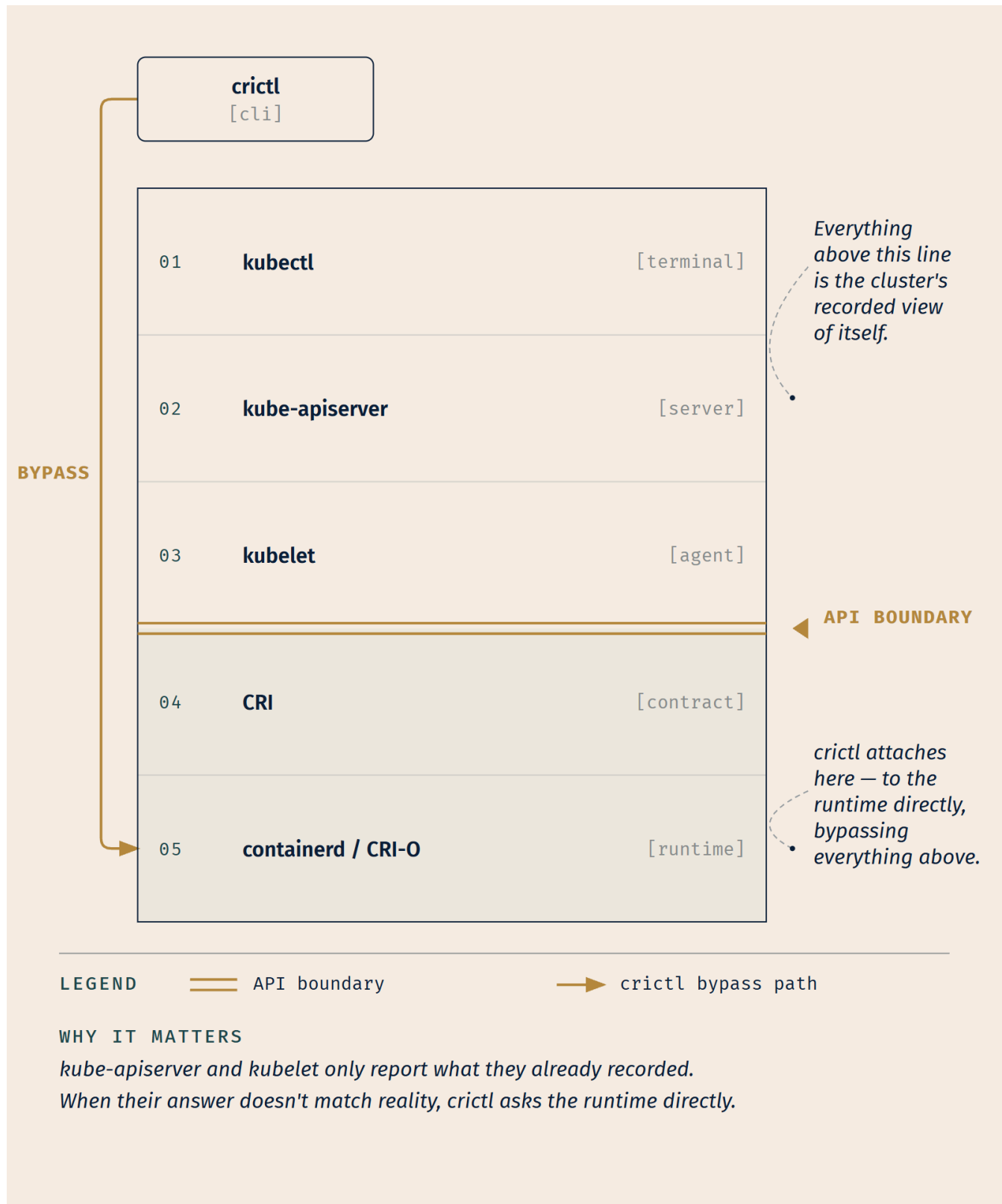
crictl, and why a tool below the API exists

Chapter 3 promised you this framing, so here it is.

Every command in this chapter so far has gone through the API server. That is by design: the API server is the only door in, and everything you have read has been the cluster's own recorded view of itself. *[cross-bearing: see Ch 3 §5 — the only door in]*

Which raises a question. What do you do when the failure is *in* that path? If the kubelet cannot register a container with the API server, then from kubectl's point of view the container does not exist. You will

look at an empty result and conclude nothing is running, while a container may be running perfectly well on the node in front of you.



crictl is not a better kubectL. It answers a different question: "what does the runtime on this machine think is happening," as opposed to "what does the cluster believe."

crictl "is a command-line interface for CRI-compatible container runtimes. You can use it to inspect and debug container runtimes and applications on a Kubernetes node." [source: k8s-docs-crictl-2026-08-31] It has been stable since Kubernetes v1.11 [source: k8s-docs-crictl-2026-08-31], and it runs on the node, not from your laptop; it "requires a Linux operating system with a CRI runtime." [source: k8s-docs-crictl-2026-08-31]

For KCNA purposes, two commands and one argument are what matter.

```
crictl ps          # containers the runtime is actually running, on this node
crictl logs <id> # that container's logs, read from the runtime directly
```

The argument is the whole point: **when the cluster's view and the node's view disagree, crictl is how you see the node's view.** A container that kubectl cannot account for but crictl ps lists is a container the kubelet failed to register, which localizes your problem to the kubelet, not to the workload. A container that neither can see was never started, which is a different problem entirely.

🔍 **Closer Look:** crictl connects to the runtime through an endpoint, configurable via `--runtime-endpoint`, the `CONTAINER_RUNTIME_ENDPOINT` environment variable, or `/etc/crictl.yaml`. [source: k8s-docs-crictl-2026-08-31] If you skip that configuration, "crictl attempts to connect to a list of known endpoints, which might result in an impact to performance." [source: k8s-docs-crictl-2026-08-31]

[cross-bearing: see Ch 2 §4 — the Container Runtime Interface] and [cross-bearing: see Ch 3 §3 — the kubelet]

One step further down, and then we stop. If the kubelet itself is the suspect, either not reporting or reporting `Ready=False` with no useful detail, the next honest move is to read the kubelet's own service logs on that machine (`journalctl -u kubelet` on a systemd host). That is a Linux administration step rather than a Kubernetes one, and it is past what KCNA asks of you. It is named here because pretending the trail ends at crictl would be dishonest, and because knowing where the trail goes is part of knowing where your own scope ends.

Extended Analogy: Think of the three layers as three accounts of the same night.

kubect1 is the harbormaster's ledger: authoritative, complete, and assembled from what each vessel reported. It is the right book to consult, and almost always sufficient.

cric1 is the individual ship's own log, kept aboard, written by the crew. It agrees with the harbormaster's ledger except when the ship's report never arrived, and *those* are exactly the nights you need it.

The kubelet's service logs are the account of the officer who was supposed to send the report and did not. You go to that account only when the first two disagree, and what you are looking for is not what the ship did, but why nobody heard about it.

You do not read all three every time. You read down the stack only as far as the disagreement takes you.

🕒 Taking Your Bearings: Started, Then Gone — and the Node Beneath

Six questions.

1. A container's last state shows `Terminated with Reason: OOMKilled`, and the Pod is `Running` with a restart count of 4. What happened, and where is the Pod now?

A) The node ran out of memory; the Pod has been rescheduled to a different node
 B) The container exceeded its own memory limit; it was killed and restarted in place on the same node
 C) The kubelet evicted the Pod under memory pressure and a controller recreated it
 D) The Pod was refused at admission for requesting too much memory

2. A Pod shows `CrashLoopBackOff`. Which of the following has *already succeeded*?

A) Nothing — `CrashLoopBackOff` means the container could not be created
 B) Scheduling only; the image pull is still being retried
 C) Scheduling, the image pull, the configuration assembly, and at least one container start
 D) Scheduling and the image pull, but not the configuration assembly

3. A Pod shows `Running`, 0/1 in the `READY` column, zero restarts, and receives no traffic. What is the most likely cause?

A) The container is crash-looping
 B) The Service selector does not match the Pod's labels
 C) The readiness probe is failing, so the Pod has been removed from its Service's endpoints
 D) The node is `NotReady`

4. A node has shown `Ready=Unknown` for the last two minutes. Its Pods are still listed as `Running`. Why has nothing been rescheduled yet?

A) Unknown is not a failure state, so no action is ever taken B) The node controller waits a period after marking a node Unknown before submitting eviction requests, so that a transient problem can resolve C) Pods on an Unknown node cannot be evicted at all without manual intervention D) The scheduler has not yet run its next scheduling cycle

5. You suspect a node's kubelet is faulty. You run `kubectl get pods -o wide --all-namespaces` and confirm that no Pod in the cluster is recorded as running on that node. You then run `crictl ps` on the node itself and find a running container whose Pod appears nowhere in the API. What does this tell you?

A) `crictl` is showing stale data and should be ignored B) The container is registered with the API server but hidden by a namespace filter C) The runtime is running a container the cluster has no record of — which points at the kubelet's reporting path, not at the workload D) The Pod exists but is in Pending, so `kubectl get pods` did not list it

6. [retrieval: ch5] A Pod has one container that requests 500m CPU and 512Mi memory, with limits of 500m CPU and 512Mi memory. What QoS class is it, and why?

A) Burstable, because the Pod declares no Pod-level request B) Guaranteed, because every container has both a request and a limit for CPU and for memory, and in each case the limit equals the request C) BestEffort, because no other Pod on the node has declared anything D) Burstable, because a single-container Pod cannot reach Guaranteed

Answers with explanations

1. B.

OOMKilled is container-scoped: the container exceeded its own limit, and "Any Container exceeding a resource limit will be killed and restarted by the kubelet without affecting other Containers in that Pod." [source: k8s-docs-pod-qos-2026-08-24] The Pod stays where it is; the restart count is the evidence.

- **A is wrong.** That describes eviction, which sets the Pod phase to Failed and results in a replacement Pod elsewhere, not a restart in place.
- **C is wrong** for the same reason. An evicted Pod is not Running with a restart count; it is Failed, and a *different* Pod exists elsewhere.
- **D is wrong.** Admission refusal means no Pod object at all. This Pod exists and has run four times.

2. C.

CrashLoopBackOff names the backoff delay in effect for a container in a crash loop [source: k8s-docs-container-restart-backoff-2026-08-31], and a loop requires at least one completed run. Everything upstream of that run therefore worked: the Pod scheduled, the image pulled, the configuration assembled, and the process launched.

- **A is wrong**, and it is the inversion the chapter's [△ Navigational Hazard](#) warns about. You cannot loop something that never ran once.
- **B is wrong**. A pull still being retried is `ImagePullBackOff`. The shared "BackOff" is exactly what makes these two look like siblings; one retries a *pull*, the other retries a *start*.
- **D is wrong**. A container whose configuration could not be assembled reports `CreateContainerConfigError` and never starts, so it has nothing to loop.

3. C.

A readiness failure is defined by this exact outcome: the Pod keeps running and "the endpoints controller removes the Pod's IP address from the endpoints of all Services that match the Pod." [source: [k8s-docs-pod-lifecycle-2026-08-23](#)]

- **A is wrong**. A crash loop shows a climbing restart count. This one is zero.
- **B** is a genuinely plausible cause of "no traffic," which is why it is here. But a selector mismatch does not produce `0/1` in the `READY` column. `READY` reports probe status, not Service membership. The `0/1` is what makes readiness the better answer.
- **D is wrong**. A `NotReady` node would affect all Pods on it and would show in `kubectl get nodes`, and it would not selectively produce `0/1` readiness on one Pod.

4. B.

"By default, the node controller waits 5 minutes between marking the node as `Unknown` and submitting the first eviction request." [source: [k8s-docs-node-controller-heartbeats-2026-08-31](#)] Two minutes in, the wait has not expired.

- **A is wrong**. Action is taken, just not yet. `Unknown` absolutely leads to eviction if it persists.
- **C is wrong**. No manual intervention is required; the mechanism is automatic once the delay elapses.
- **D is wrong**. The scheduler places new Pods and has no role in removing Pods from a failed node. That is the node controller's work.

5. C.

This is exactly the case `crictl` exists for. The stem does the comparison correctly — matching Pod identities across all namespaces rather than counting rows — so the runtime's view and the cluster's view have genuinely diverged, which localizes the fault to the layer between them: the kubelet's registration and reporting path.

- **A is wrong**. `crictl` queries the runtime live. If anything, it is the *more* immediate of the two views.
- **B is wrong** given the stem, though it is always the first thing to rule out in practice — which is why the stem specifies `--all-namespaces`. A namespace filter would hide a container the API server knows about; here, the API server does not know about it.
- **D is wrong**, and it is the sharpest distractor. `kubectl get pods` absolutely lists Pending Pods — that is how every failure in §2 is observed — and in any case a Pending Pod has no running container

for `crictl ps` to show.

6. B. [retrieval: ch5]

The Guaranteed criteria are stated per container: every container must have a memory limit and request, both greater than zero and equal to each other, and likewise for CPU. [source: k8s-docs-pod-qos-2026-08-24] This Pod's single container meets all four.

- **A is wrong.** The criteria are evaluated container by container; a Pod-level request is not required to reach Guaranteed.
- **C is wrong.** BestEffort requires that no container declare any request or limit at all, and it has nothing to do with what other Pods on the node declared.
- **D is wrong.** Nothing in the criteria refers to how many containers a Pod has.

If you got 5–6: Strong. You can separate the two families by trigger and scope, which is the hardest discrimination in the chapter.

If you got 3–4: Re-read the OOMKilled versus Evicted figure in §4 and say the three axes out loud: trigger, scope, outcome.

If you got 0–2: Go back to §4, specifically the  Navigational Hazards on requests versus limits. That single distinction accounts for most errors in this material.

Checkpoint: You've Now Mastered

✓ CrashLoopBackOff as a restart-throttling state, not a start failure ✓ OOMKilled and Evicted as three-way-different events ✓ Which of requests and limits answers which question ✓ The two probe-failure shapes, including the silent one ✓ Node conditions as instructions, and when to drop below the API to `crictl`

§6 — Versions That Don't Agree

Chapter 8 taught you the version-skew rules and told you they would come back "in a form where you have to use it rather than recite it." This is that form. This section will not restate the skew table; go and re-read it if you need it [cross-bearing: see Ch 8 §6 — the version-skew window and the three-minor rule]. What it does is show you what skew *looks like* when you meet it as a symptom, and how you would rule it out.

Skew is a diagnosis of last resort, and it is on the list precisely because nobody thinks of it. The symptoms it produces impersonate other problems convincingly.

The shape of a skewed client

Symptom: `kubectl` reports that a resource type does not exist, or a field you are certain is valid is rejected, or output is missing columns another engineer sees.

Why skew explains it: "kubectl is supported within one minor version (older or newer) of kube-apiserver." [source: k8s-version-skew-policy-2026-08-31] A client too far behind does not know about API groups the cluster has since added; a client too far ahead sends fields the server does not understand. Neither produces an error saying "your client is old." It produces an error about your *resource*, which sends you to your YAML.

How to rule it out: run `kubectl version`. Compare client and server minor versions. If they are more than one apart, stop investigating your manifest. You are debugging your instrument, not the water.

This is worth a specific warning because of who it happens to. An engineer with several clusters, one at 1.35 and one at 1.37, has one `kubectl` binary. It is inside the window for one cluster and outside it for the other, and the same command produces different results depending on which context is active.

📌 **Snag:** When a manifest applies cleanly against one cluster and is rejected by another, check `kubectl version` against both before you change a single line of YAML. The manifest is usually innocent.

The shape of a skewed kubelet

Symptom: one node behaves differently from the others. A feature works everywhere except there. Pods scheduled to it fail in ways that make no sense against Pods with identical specs elsewhere.

Why skew explains it: "kubelet may be up to three minor versions older than kube-apiserver" and "must not be newer than kube-apiserver." [source: k8s-version-skew-policy-2026-08-31] An old kubelet is *supported*; it is not an error state. But it does not implement API fields that were added after its release. It will accept a Pod spec containing a field it has never heard of and simply not act on it. No error. No event. The field is silently ignored on that one node.

That silence is the whole danger. A misconfiguration produces a message; an unimplemented field produces nothing, which reads as "it worked."

How to rule it out: `kubectl get nodes -o wide` prints each node's kubelet version. One node out of line with the others, on a cluster where a feature works inconsistently, is a strong lead. The general instruction from the project supports this as routine practice: when reporting a problem, include the "Kubernetes version: `kubectl version`" and the "container runtime version." [source: k8s-docs-troubleshooting-overview-2026-08-31] It is the first thing the maintainers ask for, which tells you how often it is the answer.

There is a forward risk too. "Running a cluster with kubelet instances that are persistently three minor versions behind kube-apiserver means they must be upgraded before the control plane can be

upgraded." [source: k8s-version-skew-policy-2026-08-31] A node at the far edge of the window is not merely a diagnostic curiosity; it is blocking the next control-plane upgrade.

Known issues as a triage step

There is a step most people skip, and the same troubleshooting page that lists the four debugging guides closes by adding it: **"You should also check the known issues for the release you're using."** [source: k8s-docs-troubleshooting-overview-2026-08-31], pointing at the release notes on GitHub.

That step feels like an admission of defeat, and it should not. The Kubernetes project "maintains release branches for the most recent three minor releases," with each getting "approximately 1 year of patch support." [source: k8s-version-skew-policy-2026-08-31] Fixes land in those branches continuously. A behavior that is genuinely a bug, and genuinely already known, will be described in the release notes of the version you are running, and hours of careful, correct investigation will arrive at a conclusion someone already wrote down.

Reading the known issues first is not giving up. It is the cheapest step available and it is on the official list.

🔒 **Worth Securing:** When a failure survives the whole triage flow (phase, conditions, events, logs, node) and still makes no sense, your next two moves are `kubectl version` across every component you can reach, and the release notes for the version you are on. Both are five-minute checks. Both are on the official troubleshooting path. Neither is where anyone thinks to look on the second hour of an incident.

[cross-bearing: see Ch 17 §8 — SIG Release and the release cadence]

..1 §7 — Numbers Nobody Collects by Default

You have a Pod you suspect is memory-hungry. You type the obvious command:

```
kubectl top pod myapp
```

And you get an error.

Not a Pod-specific error, but an error saying the metrics API is unavailable, or the server could not find the requested resource. The same command fails identically for every Pod, every node, and every namespace on the cluster.

Nothing is broken. **A stock Kubernetes cluster publishes no usage metrics at all** — not because nobody is measuring, but because nobody installed the component that would gather the measurements into an API you can query. The soundings are being taken on every node. There is simply nobody amidsthips writing them into a book you can read.

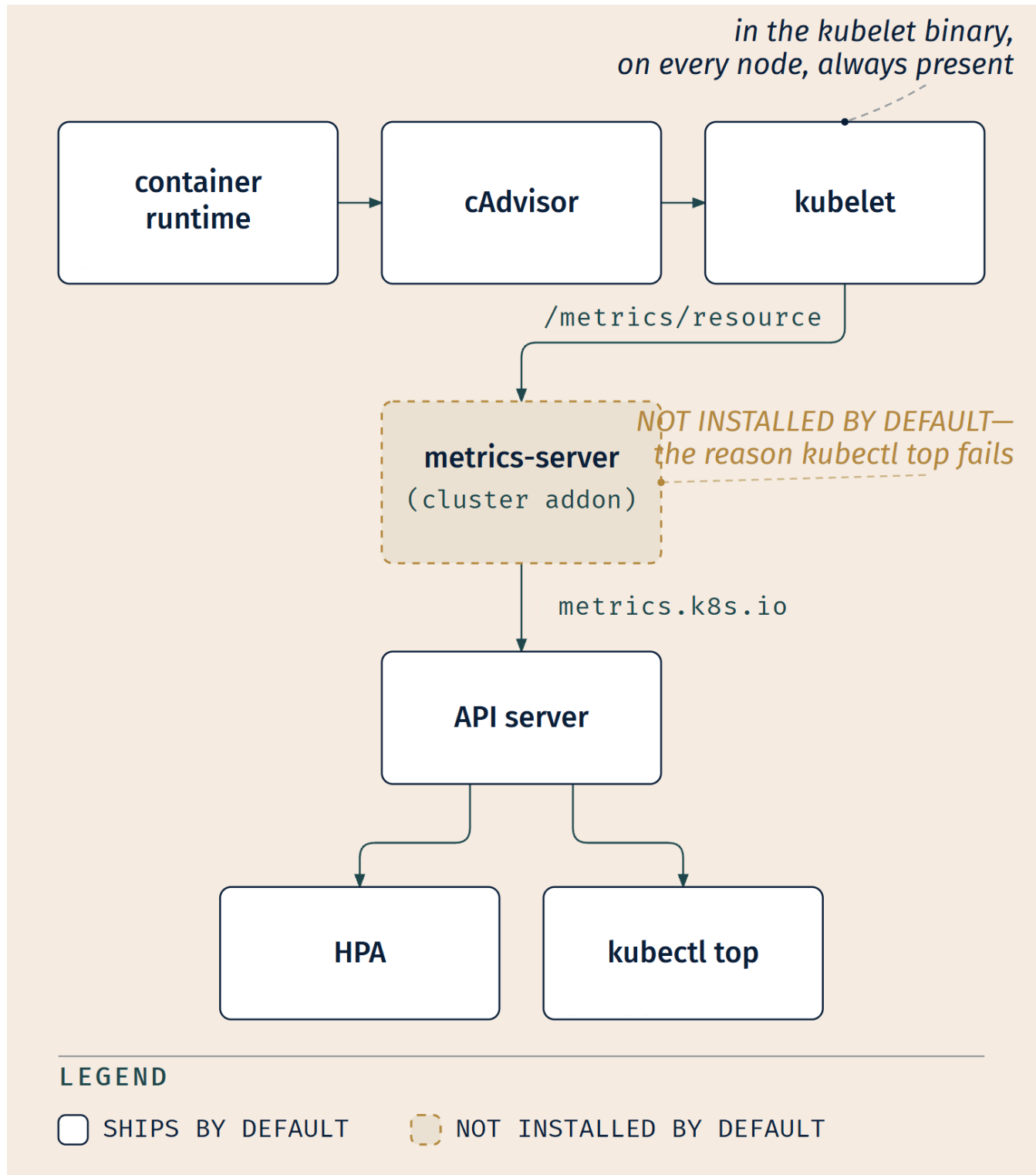
The pattern you already own

You have met this exact shape before. Chapter 3 gave you the sentence and Chapter 10 §3 named it as a pattern: **an object without its component does nothing**. An Ingress object on a cluster with no Ingress controller is accepted by the API server, stored in etcd, retrievable with `kubect1 get`, and completely inert. The API is the contract. The controller is the implementation. Kubernetes ships the contract; somebody has to install the implementation.

[cross-bearing: see Ch 10 §3 — an object without its component does nothing]

`kubect1 top` is the same pattern with one twist: here it is not even the object that is missing, it is the **API itself**. The Metrics API is not part of the core API server. It is served by an extension, and if nobody deployed that extension, the API server has no such endpoint to route your request to. Hence the shape of the error: not "no data," but "no such resource."

What the pipeline actually is



The dashed box is the whole lesson. Everything solid is already running on your cluster right now. The measurements exist; nothing is publishing them.

Working from the bottom of the stack up [source: k8s-docs-resource-metrics-pipeline-2026-08-31]:

cAdvisor is a "Daemon for collecting, aggregating and exposing container metrics included in Kubelet." Note *included in*: it is part of the kubelet binary, on every node, always. Nobody installs it.

The kubelet exposes what cAdvisor gathered: "Resource metrics are accessible using the `/metrics/resource` and `/stats` kubelet API endpoints."

metrics-server is the missing piece: a "Cluster addon component that collects and aggregates resource metrics pulled from each kubelet. The API server serves Metrics API for use by HPA, VPA, and by the `kubectl top` command. Metrics Server is a reference implementation of the Metrics API."

The Metrics API is what consumers actually query: a "Kubernetes API supporting access to CPU and memory used for workload autoscaling. To make this work in your cluster, you need an API extension server that provides the Metrics API."

That last sentence is the fact. The API exists as a specification; it does not exist on your cluster until something serves it.

The 2026-08-23 snapshot is blunter still: metrics-server "is a cluster addon component (not deployed by default in all distributions)." [source: k8s-docs-resource-metrics-pipeline-2026-08-23] Some distributions install it for you; many do not. Which is why "does `kubectl top` work?" is a genuinely useful first question about an unfamiliar cluster. It tells you something about how thoroughly the cluster was built.

☆ Fixed Point

`kubectl top` **requires metrics-server, and many distributions do not install it.** An error from `kubectl top` is a statement about what is installed on the cluster, not about the workload you asked about.

The same absence has a second consequence people meet separately and never connect: **a HorizontalPodAutoscaler reads the same Metrics API.** [source: k8s-docs-resource-metrics-pipeline-2026-08-31] An HPA created on a cluster without metrics-server is accepted like any other object, has no metric source to read, and never scales anything: the same pattern, one layer up. [cross-bearing: see Ch 6 §2 — the HPA in one sentence] and [cross-bearing: see Ch 17 §7 — the autoscaling landscape]

One scope note worth carrying: metrics-server "is meant only for autoscaling purposes — for example, don't use it to forward metrics to monitoring solutions, or as a source of monitoring solution metrics." [source: k8s-docs-resource-metrics-pipeline-2026-08-23] It holds current values for autoscaling decisions. It is not a monitoring system, it keeps no history, and you cannot query it for what happened an hour ago. [cross-bearing: see Ch 18 §3 — metrics-server versus a monitoring system]

[cross-bearing: see Ch 3 §4 — addons, and what else is optional]

Where logs actually live, and why `kubectl logs` is not an archive

The same "nobody built this for you" argument applies to logs, and it is the one that costs people evidence.

When you run `kubectl logs`, the request goes to the API server, which routes it to the kubelet on the node, and "the kubelet on that node handles the request and reads directly from the log file; the kubelet returns the content of the log file; only the latest log file's contents are available." [source: k8s-docs-logging-architecture-2026-08-23]

There is no log database. There is a file on a node's disk, and a kubelet willing to read it to you.

That file is rotated: the kubelet "is responsible for rotating container logs and managing the logging directory structure, configured via `containerLogMaxSize` (default 10Mi) and `containerLogMaxFiles` (default 5)." [source: k8s-docs-logging-architecture-2026-08-23] And it is not durable against the events this chapter is about: "if a container restarts, the kubelet keeps one terminated container with its logs. If a pod is evicted from the node, all corresponding containers are also evicted, along with their logs." [source: k8s-docs-logging-architecture-2026-08-23]

Read that last clause against §4. **An evicted Pod takes its logs with it.** The failure you most want to investigate is the one whose evidence is most likely gone.

The Kubernetes project states the gap plainly: "In a cluster, logs should have a separate storage and lifecycle independent of nodes, pods, or containers. This concept is called cluster-level logging." And: **"Cluster-level logging architectures require a separate backend to store, analyze, and query logs. Kubernetes does not provide a native storage solution for log data. Instead, there are many logging solutions that integrate with Kubernetes."** [source: k8s-docs-logging-architecture-2026-08-31]

The most common answer is to "use a node-level logging agent that runs on every node (typically a DaemonSet) and pushes logs to a backend." [source: k8s-docs-logging-architecture-2026-08-23] That is the whole gloss you need here; the agents themselves, and the architecture around them, are Chapter 18's. [cross-bearing: see Ch 18 §6 — node-level logging agents] and [cross-bearing: see Ch 6 §7 — DaemonSets]

⚠ Navigational Hazards

`kubectl logs` is a live read, not an archive. It reads a file on a node, and that file is rotated, capped, and destroyed along with the Pod.

The reader most likely to be caught by this is the one investigating the most interesting failure: an eviction, a node that died, a Pod deleted and recreated by a controller. In every one of those cases the evidence is gone by definition, and the absence of logs means nothing about what the application did.

This is the same lesson as the event retention window in §3, arriving from a second direction. Two of the platform's diagnostic surfaces are ephemeral by design. Neither one's silence is evidence.

🕒 Taking Your Bearings: Skew, Metrics, and What Isn't There

Five questions. Two of them reach back to earlier chapters, because §6 and §7 are where two earlier chapters' debts come due.

1. `kubectl top nodes` returns an error on a cluster where every node is Ready and every workload is healthy. What is the most likely explanation?

A) The nodes have insufficient permissions to report metrics B) metrics-server is not installed, so the Metrics API is not served C) metrics-server is installed but has not completed its first scrape, so the API is not answering yet D) The cluster has no HorizontalPodAutoscaler, so the Metrics API is not activated

2. *[retrieval: ch8]* Your control plane is at 1.37. You are asked whether a node running kubelet 1.33 is supported. What is the answer, and what is the rule?

A) Supported — the kubelet may be any version older than the API server B) Not supported — the kubelet may be at most three minor versions older, and 1.33 is four behind 1.37 C) Supported — 1.33 is within the three-version window D) Not supported — the kubelet must exactly match the API server's minor version

3. *[retrieval: ch10]* Which of the following best describes what an HPA does on a cluster with no metrics-server installed?

A) The HPA object is rejected at creation, because its dependency is missing B) The HPA is created and scales using each kubelet's metrics endpoint directly C) The HPA is created and never scales — the object exists, but the component it depends on does not D) The HPA falls back to scaling on a fixed schedule

4. A container was evicted from a node four hours ago. You want to read what it logged just before it went. What should you expect?

A) `kubectl logs --previous` will return them, because the kubelet keeps one terminated container B) The logs went with the Pod when it was evicted from the node; without a cluster-level logging backend they are gone C) The API server retains Pod logs independently of the node D) The logs are available for the duration of the event retention window

5. A manifest applies cleanly against your development cluster and is rejected against production with an error about an unrecognized field. Both clusters run the same workload. What should you check before editing the manifest?

A) The namespace's ResourceQuota B) The `kubectl` client version against each cluster's API server version C) The node count on each cluster D) Whether the production cluster has metrics-server installed

Answers with explanations

1. B.

metrics-server is a "Cluster addon component" and the Metrics API requires "an API extension server that provides the Metrics API." [source: k8s-docs-resource-metrics-pipeline-2026-08-31] Without it, there is no endpoint to answer the request, which is why the error is about the resource, not about any particular node.

- **A is wrong.** This is not a permissions problem, and healthy nodes reporting normally would not produce it.
- **C is wrong,** and it targets the natural hope that the error is transient. A metrics-server that was installed but had not finished a scrape would return empty or partial *data*; the failure here is that the API resource does not exist at all. Those are different errors, and the difference is the diagnosis.
- **D is wrong,** and it inverts the dependency. The HPA is a *consumer* of the Metrics API — it "use[s] data from the metrics API" [source: k8s-docs-resource-metrics-pipeline-2026-08-31] — so its presence or absence cannot activate anything.

2. B. [retrieval: ch8]

The rule is that "kubelet may be up to three minor versions older than kube-apiserver," and the documented example for this exact control-plane version is explicit: "kube-apiserver is at **1.37**; kubelet is supported at **1.37, 1.36, 1.35, and 1.34.**" [source: k8s-version-skew-policy-2026-08-31] Three minors below 1.37 is 1.34, which is the floor. 1.33 is four behind and falls outside the window.

- **A is wrong.** The window is bounded, and the bound is three.
- **C is wrong,** and it is the trap the question exists for. It states the conclusion without doing the arithmetic, and the arithmetic is where people slip: it is tempting to count 1.36, 1.35, 1.34, 1.33 as "three older" when that is four. The off-by-one is the single most common error in applying this rule.
- **D is wrong.** Exact matching is not required — the skew window exists precisely so that a cluster can be upgraded one component at a time.

3. C. [retrieval: ch10]

The HPA "use[s] data from the metrics API" [source: k8s-docs-resource-metrics-pipeline-2026-08-31], and the Metrics API requires an extension server to provide it. This is Chapter 10's named pattern applied one layer up: the object exists, the API server accepts it, and nothing happens because the component that would act is absent.

- **A is wrong,** and it is the informative distractor. Kubernetes does not validate that a controller or a dependency exists before accepting an object. That is precisely what makes this failure mode quiet.
- **B is wrong.** The HPA reads the aggregated Metrics API through the API server, not each kubelet directly. That aggregation is exactly what metrics-server does.
- **D is wrong.** No such fallback exists.

4. B.

"If a pod is evicted from the node, all corresponding containers are also evicted, along with their logs." [source: k8s-docs-logging-architecture-2026-08-23] Without a cluster-level logging backend, which "Kubernetes does not provide" [source: k8s-docs-logging-architecture-2026-08-31], there is nowhere else for them to be.

- **A is wrong.** The "one terminated container" retention applies to a container that *restarted* within a surviving Pod. The Pod here is gone from the node.
- **C is wrong.** The API server stores no logs. It proxies the request to a kubelet that reads a file.
- **D is wrong.** The event retention window governs Event objects, not logs. Two different ephemeral surfaces with two different lifetimes; conflating them is the trap.

5. B.

"kubect1 is supported within one minor version (older or newer) of kube-apiserver." [source: k8s-version-skew-policy-2026-08-31] One client binary against two clusters at different versions can be inside the window for one and outside it for the other, producing exactly this asymmetry.

- **A is wrong.** A quota rejection names quota, not an unrecognized field.
- **C is wrong.** Node count has no bearing on API field validation.
- **D is wrong.** metrics-server has nothing to do with validating a manifest.

If you got 4–5: Both debts collected. The skew rule and the named-component pattern are the two pieces of this chapter that came from elsewhere, and you have them.

If you got 2–3: Look specifically at which one you lost. If it was skew, re-read Chapter 8 §6's table; this chapter deliberately did not restate it. If it was the component pattern, re-read §7's opening.

If you got 0–1: Re-read §7 first, then Chapter 8 §6. §7 is short and its argument is a pattern you already own from Chapter 10; getting it back is quick.

Checkpoint: You've Now Mastered

✓ Version skew as a symptom shape, not just a policy table ✓ Known issues as a legitimate, cheap triage step ✓ The resource metrics pipeline, and why kubect1 top fails on a stock cluster ✓ Why kubect1 logs is not an archive, and which failures destroy their own evidence

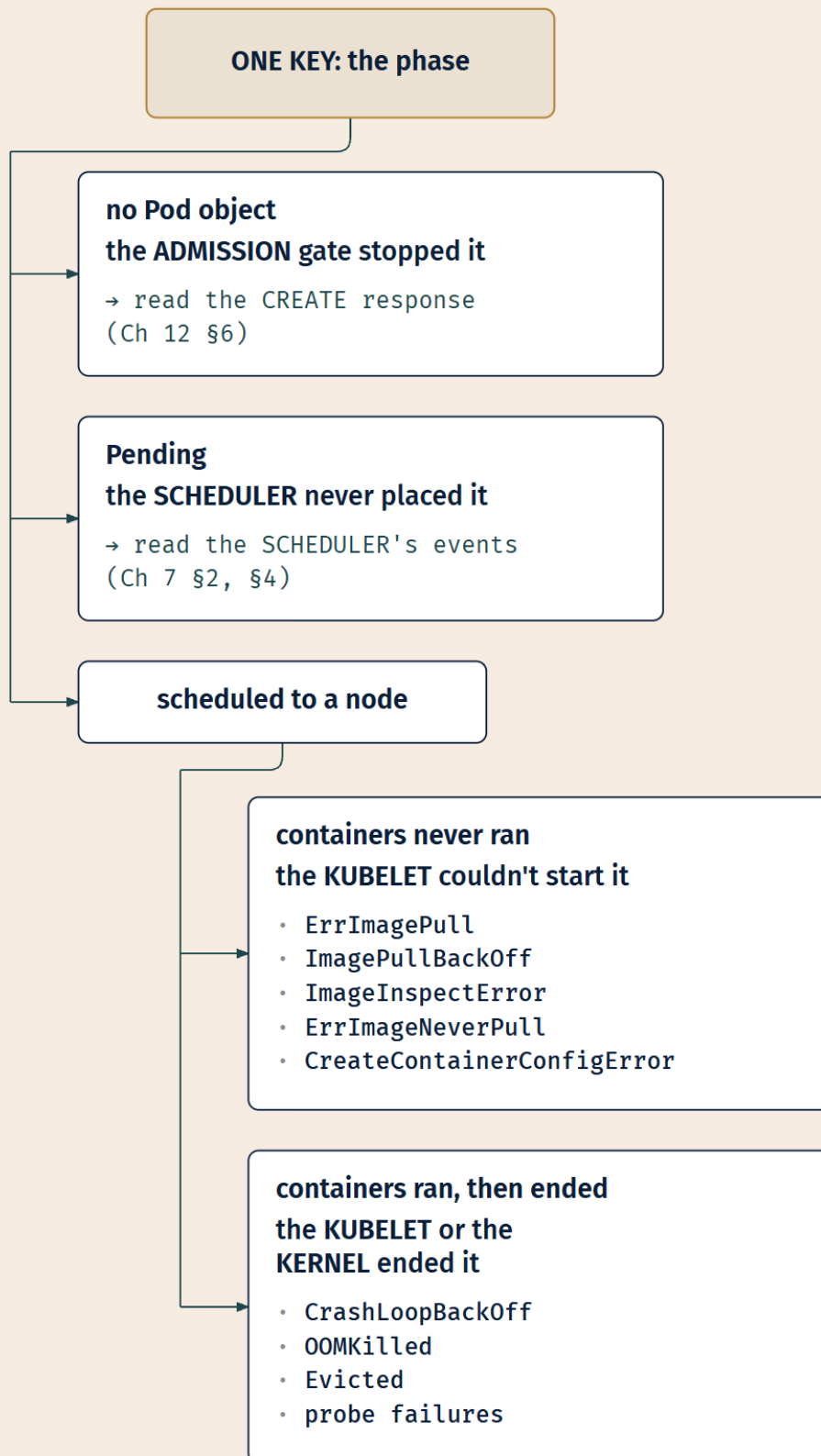
☼ §8 — Read the Phase First

Look back at what you have actually learned.

Pending. ErrImagePull. ImagePullBackOff. ImageInspectError. ErrImageNeverPull.
CreateContainerConfigError. CrashLoopBackOff. OOMKilled. Evicted.

Nine strings. That is what a glossary would have given you, and if that is what you had come away with, you would forget six of them within a month, because nine unrelated facts have nothing to hold on to.

But that is not the shape of what you learned.



Nine signatures. One lookup. The key is always the phase.

Not nine facts on a flat list. One axis, and a position on it.

Every signature in this chapter is a **position in the platform's start-up sequence**. The sequence is the same every time: admitted, scheduled, pulled, configured, started, running, killed. The phase tells you which of those steps stopped. The step tells you which component owned it. And the component tells you what to read, because each component has a place where it writes down what it did.

The scheduler writes events. The kubelet writes container states, reasons, and events. The node controller writes conditions. The admission gate writes a response to whoever called it.

That is the whole method: **the phase names a stage, the stage names a component, and the component names a source**. Three steps from symptom to the right place to look, and none of them requires you to have seen this particular failure before.

Which is why it survives contact with the unfamiliar. When Kubernetes adds a signature next year that no study guide has covered, you will not recognize it, and you will not need to. You will read the phase, identify the stage, ask the component, and it will tell you. Instruments change; the way you fix a position does not. The method is older than any particular string and will outlive all of them.

And now the turn.

This chapter opened by asking what you read first. It has been answering a second question the entire time without saying so. Because the phase does not only tell you *what happened*. It tells you *whose problem this is*. A Pod that never started is not your application's failure; your application has not run. A Pod that started and was killed by the node is not your application's failure either; it did nothing wrong except exist somewhere that ran out of memory.

Only when the Pod is Running and Ready, the trouble is confined to that one workload, and the behavior is still wrong, has the platform finished its work and handed the problem to you. When several unrelated workloads go wrong at once with every Pod healthy, the platform has not finished — it has failed somewhere the phase cannot see, and the network is the first place to look. *[cross-bearing: see Ch 9 §1 — CNI and pod networking]*

So §1's question and §8's answer are the same question. *What do you read first* and *whose problem is this* have one answer, and it is the phase. That is why the order is what it is, and why it is not arbitrary: the first thing you read is the thing that tells you whether to keep reading at all.

☆ Fixed Point

Read the phase before you read the logs.

The phase tells you which stage of the platform's sequence stopped, which component owns that stage, and where that component writes down what it did. The logs tell you what your application said — which only means anything once you know your application ran.

There is a familiar shape underneath this, if you want it. A Pod sitting in `Pending` is not a broken thing. It is a control loop that has not converged: a declaration of intent that no node has yet been able to satisfy, being patiently re-evaluated by a component that will act the instant it can. It is not an error. It is the system telling you, accurately, that the world has not yet caught up with what you asked for. *[cross-bearing: see Ch 3 §6 — the control loop]*

Diagnosis, in this system, is mostly reading a loop's report and believing it.

Exam Alert! ⚠️

High-Priority Topics:

1. **The never-started / started-then-died split, and which signature belongs to which.** This is the chapter in one line. `Pending`, the image-pull family, and `CreateContainerConfigError` are never-started. `CrashLoopBackOff`, `OOMKilled`, and `Evicted` are started-then-died. The restart count tells you which side you are on.
2. **`CrashLoopBackOff` means the container ran.** The image pulled, the config resolved, the process started, and exited. It is a restart-throttling state, not a start failure.
3. **`OOMKilled` versus `Evicted`.** Different trigger (this container's own limit versus the node's pressure), different scope (one container versus the whole Pod), different outcome (restarted in place on the same node versus replaced elsewhere).
4. **`kubectl top` requires metrics-server, which many distributions do not install.** The error is about the cluster's build, not about the workload.

Common Traps — ⚠️ Navigational Hazards, with what each one costs you:

Trap	The correct understanding
Reading <code>CrashLoopBackOff</code> as an image problem	It is the opposite signal. The image was fine — that is what "Loop" implies.
<code>kubect1 logs</code> on a crash-looping Pod returns nothing, so the app is silent	You are reading a container that has not started. <code>--previous</code> reads the one that died.
Treating the absence of an event as evidence	Events expire. An empty event list on a Pod that failed hours ago means nothing.
Assuming a <i>request</i> protects a container from being killed	The limit is what gets you OOM-killed. The QoS class — which requests help determine — is what orders eviction. Two different questions.
"BestEffort is safest, because it asks for nothing"	BestEffort is evicted first .
Pending means something is retrying with looser constraints	Nothing is. Pending is a stable, honest report. Go and read the scheduler's events.
Diagnosing an omitted <code>-c</code> as an application failure on a multi-container Pod	Confirm which container you read before concluding anything about the application.
Expecting <code>kubect1</code> to account for a container the kubelet never registered	That is precisely the case <code>crictl</code> exists for.
Assuming a Pod that "won't start" always exists as an object	An admission refusal means there is no Pod at all — the reason is on whatever tried to create it.
Expecting <code>kubect1 logs</code> to be an archive	It is a live read of a rotating file on a node. An evicted Pod takes its logs with it.

Practice Questions

Sixteen questions, weighted toward the two signature families. Several present a symptom and ask what you would *do*, rather than naming a string — the shape a glossary cannot answer.

1. A Pod is Running with 1/1 in the READY column and a restart count of 0, on a node reporting `Ready=True`. Users report that it returns the wrong data. No other workload on the cluster is affected. What is your next move?

A) `kubect1 describe pod` and read the events for a platform fault B) `kubect1 describe node` and check for pressure conditions C) Stop investigating the platform — every platform signal is good and the trouble is confined to this workload, so this is application scope D) `kubect1 logs --previous`, to read the container that died

2. *[interleaved: D1.3 scheduling]* A Pod has been in Pending for fifteen minutes. `kubect1 describe pod` shows a scheduler event reporting that all six nodes in the cluster were rejected because each carried a taint the Pod does not tolerate. What is the correct interpretation?

A) The cluster is out of capacity and needs more nodes B) Six nodes have capacity but are refusing this Pod on purpose; the Pod needs a matching toleration, or it needs to not want those nodes C) The Pod's resource requests exceed every node's allocatable capacity D) The Pod will be scheduled once the scheduler retries it with the taint ignored

3. Which single column of `kubectl get pods` output most quickly separates a never-started failure from a started-then-died failure?

A) STATUS B) READY C) RESTARTS D) AGE

4. A Pod's container shows `Waiting with Reason: ImagePullBackOff`. Which of the following is *not* a plausible cause?

A) The image name is misspelled B) The image is in a private registry and no image pull secret is configured C) The image has not been pushed to the registry D) The application inside the image exits immediately on startup

5. A Deployment's Pods all show `CreateContainerConfigError`. The image is correct and pullable, and the nodes have ample capacity. What should you look for?

A) A missing or misnamed ConfigMap or Secret referenced by the Pod spec B) A taint on every node C) An `imagePullPolicy` of `Never` on a node without the image D) A `restartPolicy` set to `Never`

6. A Pod runs a container whose memory limit is 256Mi. Under load, the process allocates 300Mi. What happens, and what is the resulting signature?

A) The whole Pod is evicted; the phase becomes `Failed` and a replacement appears elsewhere B) The container is killed for exceeding its own limit, its reason is recorded as `OOMKilled`, and it is restarted in place C) The scheduler moves the Pod to a node with more memory D) The container is throttled to 256Mi and continues running

7. A node reports `MemoryPressure=True`. Three Pods on it are terminated. Which is terminated first, and what happens to it afterwards?

A) The Pod with the highest memory limit; it is restarted in place B) The oldest Pod; it is restarted in place C) The `BestEffort` Pod; its phase becomes `Failed` and a controller creates a replacement elsewhere D) The `Guaranteed` Pod, because it is holding the most reserved memory

8. *[interleaved: D1.1 workloads]* A Deployment shows `ProgressDeadlineExceeded` and has zero available replicas. `kubectl describe deployment` gives you the condition but no cause. What is the correct next step?

A) Increase `progressDeadlineSeconds` until the rollout completes B) Delete and recreate the Deployment C) Describe the Deployment's new `ReplicaSet` and then its Pods, reading the events at each level D) Read the Deployment's logs with `kubectl logs deployment/<name>`

9. *[interleaved: D2.2 security]* A Pod will not start. Its container reports `CreateContainerConfigError`, and the Pod spec mounts a Secret named `db-credentials` as a volume. `kubectl get secret db-credentials` returns `NotFound`. What is the diagnosis, and at what stage did it fail?

A) The Pod failed at scheduling; no node could satisfy the volume request B) The Pod failed at image pull; Secrets are needed to authenticate to the registry C) The Pod scheduled and pulled successfully, and failed at container configuration because the kubelet could not resolve a referenced object D) The Pod was refused at admission because the Secret did not exist

10. A Pod shows `Running` with `1/1` in the `READY` column, and its restart count is 7. What can you conclude?

A) The Pod is currently unhealthy and receiving no traffic B) The container has died and been restarted seven times, but is currently running and passing its readiness probe C) The Pod has been evicted seven times D) The Pod is Ready now, so the seven restarts must have been caused by the platform rather than by the application

11. You want to read the logs of the container instance that died, not the one currently starting. Which command?

A) `kubectl logs <pod> --all-containers` B) `kubectl logs <pod> --previous` C) `kubectl logs <pod> --tail=100` D) `kubectl describe pod <pod>`

12. A node has shown `Ready=Unknown` for the last ten minutes. What has the node controller done, and what has happened to the Pods?

A) Nothing — `Unknown` is informational only B) The node controller set the condition and, after its wait period elapsed, began evicting the Pods; controllers will create replacements elsewhere C) The kubelet on that node evicted its own Pods under pressure D) The scheduler has marked the node unschedulable but left the Pods alone permanently

13. On a node you suspect of a kubelet fault, `kubectl get pods -o wide` shows two Pods on that node. `crictl ps` on the node shows four running containers belonging to Kubernetes workloads. What does the discrepancy indicate?

A) `crictl` is counting containers, and each Pod has multiple containers, so the numbers may be entirely consistent B) The runtime is definitively running workloads the cluster does not know about C) `kubectl` output is stale and should be refreshed D) Some of those containers belong to the node's static Pods, which `kubectl` never displays

14. A cluster-wide feature works on every node except one. That node's kubelet is four minor versions behind the API server. Which statement is correct?

A) This is supported; the kubelet may be any number of minor versions behind B) This is outside the supported skew window, and the old kubelet may silently ignore API fields it does not implement C) The API server will refuse to schedule any Pod to that node D) The kubelet will report an error event for each unimplemented field

15. `kubectl top pod` fails on a cluster. Every workload is healthy. Which of the following would resolve it?

A) Restarting the kubelet on each node B) Installing metrics-server, which aggregates the kubelets' metrics into the Metrics API that `kubectl top` queries C) Increasing the API server's `--event-ttl` D) Enabling the kubelet's `/metrics/resource` endpoint, which is off by default

16. An engineer says: "The Pod failed overnight and `kubectl describe` shows Events: <none>, so nothing went wrong with the platform — it must be an application bug." What is the flaw in that reasoning?

A) There is no flaw; an empty event list is conclusive B) Events expire after a retention window, so their absence hours later is not evidence of anything C) `describe` never shows events for failed Pods D) The Pod's events were deleted along with the Pod, so a surviving Pod always retains all of its events

Answers with explanations

1. C. Every platform signal in the stem is good: the Pod scheduled, pulled, configured, started, is passing its readiness probe, and sits on a healthy node. And the trouble is confined to one workload, which rules out the cluster-wide network faults that also present as `Running` and `Ready`. The platform has finished its work.

- **A is wrong.** `describe` reports platform state, and the stem has already given you every platform signal in good order. The events will be empty or uninformative, and reading them is the habit of continuing to interrogate a component that has already answered.
- **B is wrong** for the same reason, with the node's own condition already stated as `Ready=True`. Node conditions earn attention when *several* Pods on one node fail together, not when one workload returns wrong results on a healthy node.
- **D is wrong.** Nothing died — the restart count is zero, so there is no previous instantiation for `--previous` to read.

2. B. The event names the reason: an untolerated taint. Six nodes were considered and all six refused this Pod because it does not tolerate what they carry. Capacity is not the issue.

- **A is wrong**, and it is the trap. A Pod stuck in `Pending` reads like "the cluster is full" if you do not read the event text. It is not.
- **C is wrong.** A request-versus-capacity failure produces a different predicate in the scheduler's message — a shortfall of a named resource, not a refusal by a taint.
- **D is wrong**, and it is the Chapter 7 misconception this chapter exists to close. Nothing retries with relaxed constraints. `Pending` is a stable report, and the scheduler will not ignore a taint to satisfy it.

3. C. A non-zero restart count means a container ran and ended. Zero means it has not run yet. One column, and it halves the search space.

- **A is wrong.** STATUS is informative but it is a display field that mixes phase and container reason, and it does not by itself distinguish "never ran" from "ran and stopped." CrashLoopBackOff and ImagePullBackOff both print in this column.
- **B is wrong.** READY reports probe status on a running Pod. It is the right column for §4's silent readiness failure, but it does not separate the two families.
- **D is wrong.** AGE tells you how long the object has existed, not what happened to it.

4. D. An application that exits immediately would produce CrashLoopBackOff; the image pulled and the container ran. ImagePullBackOff means the image never arrived, so nothing inside it has executed and its behavior is irrelevant.

- **A is plausible,** and therefore wrong as the answer to a NOT-question: "invalid image name" is a documented cause. [source: k8s-docs-images-2026-08-23]
- **B is plausible:** "pulling from a private registry without an imagePullSecret" is the other documented cause. [source: k8s-docs-images-2026-08-23]
- **C is plausible:** the debugging guide's own checklist asks "have you pushed the image to the registry?" [source: k8s-docs-debug-pods-2026-08-23]

5. A. CreateContainerConfigError means the kubelet could not assemble the container's configuration, typically because a referenced ConfigMap or Secret, or a key inside one, does not exist.

- **B is wrong.** A universal taint produces Pending with no container state at all.
- **C is wrong,** and it targets a real conflation of policy problems with configuration problems. imagePullPolicy: Never with no local image produces its own reason string, ErrImageNeverPull [source: k8s-docs-pod-failure-signatures-2026-08-31], and in any case the stem stipulates the image is correct and pullable.
- **D is wrong.** restartPolicy governs behavior after a container exits. Nothing has exited.

6. B. Exceeding a memory *limit* is scoped to the container: "Any Container exceeding a resource limit will be killed and restarted by the kubelet without affecting other Containers in that Pod." [source: k8s-docs-pod-qos-2026-08-24] The reason recorded is OOMKilled — "The container ran out of memory." [source: k8s-docs-pod-failure-signatures-2026-08-31]

- **A is wrong.** Eviction is triggered by *node* pressure, not by one container exceeding its own limit. Different trigger, different scope.
- **C is wrong.** Pods are never moved. "A given Pod (as defined by a UID) is never 'rescheduled' to a different node." [source: k8s-docs-pod-failure-signatures-2026-08-31]
- **D is wrong.** Exceeding a memory limit ends the container; it is not capped and allowed to continue. [cross-bearing: see Ch 5 §8 — requests, limits, and CPU throttling], where the contrasting behavior of the two resource types is covered.

7. C. "Kubernetes will first evict BestEffort Pods running on that Node, followed by Burstable and finally Guaranteed Pods." [source: k8s-docs-pod-qos-2026-08-24] And an eviction is Pod-scoped, not container-scoped: "the kubelet sets the phase for the selected pods to Failed, and terminates the Pod," after which

a controller "creates new pods in place of the evicted pods." [source: k8s-docs-node-pressure-eviction-2026-08-31]

- **A is wrong** twice over: the limit does not determine eviction order, and an evicted Pod is not restarted in place.
- **B is wrong** on both counts as well. Age is not a factor, and the outcome is a replacement elsewhere, not a restart.
- **D is wrong**, and inverted: Guaranteed is evicted *last*, not first.

8. C. ProgressDeadlineExceeded says the rollout did not finish in time and nothing about why. The cause is one level down, on the new ReplicaSet's Pods, in their events.

- **A is wrong.** Extending the deadline does not fix a rollout that is genuinely blocked; it just makes you wait longer for the same failure.
- **B is wrong.** Recreating discards the evidence and, if the cause is unchanged, reproduces the failure.
- **D is wrong.** A Deployment has no logs. `kubectl logs deployment/<name>` proxies to a Pod, and here the new Pods are the thing that is failing.

9. C. Scheduling succeeded (a node was chosen), the image pulled (or the Pod would show a pull reason), and the failure is at the last step before the container runs: the kubelet cannot resolve the referenced Secret.

- **A is wrong.** A Secret volume does not affect scheduling. The Pod is on a node.
- **B is wrong.** Image pull secrets are a different mechanism, and a pull failure produces a pull reason.
- **D is wrong**, and it is the important distractor. Admission refusal means **no Pod object exists**. Here the Pod exists and is describable, which rules admission out entirely — a referenced Secret's absence is discovered by the kubelet at container-configuration time, not at the admission gate.

10. B. A restart count of 7 means the container has died and been restarted seven times. `1/1 Ready` means it is currently passing its readiness probe and is in its Service's endpoints. Both facts are true at once.

- **A is wrong.** `1/1` means it is ready and is receiving traffic.
- **C is wrong.** Eviction terminates a Pod and produces a replacement elsewhere; it does not increment a restart count on a surviving Pod.
- **D is wrong**, and it is a real inversion. `READY` reports the probe's *current* verdict and says nothing about what caused earlier terminations. The restart count alone attributes no cause — the causes documented for repeated restarts include "application errors, configuration errors, resource constraints, failing health checks, or probe failures." [source: k8s-docs-container-restart-backoff-2026-08-31]

11. B. `--previous` "retrieves logs from a previous instantiation of a container." [source: k8s-docs-logging-architecture-2026-08-23]

- **A is wrong.** `--all-containers` selects across containers in the Pod, not across instantiations in time.
- **C is wrong.** `--tail` limits how many lines you get from the current container.
- **D is wrong.** `describe` shows state and events, not log content.

12. B. The node controller "sets the Ready condition to Unknown" when a node becomes unreachable, and after its wait — "By default, the node controller waits 5 minutes between marking the node as Unknown and submitting the first eviction request" — evictions begin. [source: k8s-docs-node-controller-heartbeats-2026-08-31] Ten minutes is past that. Controller-managed Pods get replacements elsewhere.

- **A is wrong.** Action follows, just not immediately.
- **C is wrong.** The kubelet is not answering; that is what Unknown means. It cannot be the one acting.
- **D is wrong.** The scheduler does not evict, and the outcome is not permanent inaction.

13. A. This is a counting question disguised as a diagnostic one. `crictl ps` lists **containers**; `kubect1 get pods` lists **Pods**. Two Pods with two containers each is four containers, and everything is consistent.

- **B is wrong** without further checking. You must compare like with like before concluding there is a discrepancy at all. The genuine `crictl` signal is a container the cluster has *no record of*, which requires matching Pod and container identities, not counting rows.
- **C is wrong.** There is no reason to believe the output is stale.
- **D is wrong**, and it targets a real belief. Static Pods are "managed directly by the kubelet and represented by mirror Pods" in the API [source: k8s-docs-pod-failure-signatures-2026-08-31] — so `kubect1` does show them, and they are not a source of hidden containers.

14. B. "kubelet may be up to three minor versions older than kube-apiserver." [source: k8s-version-skew-policy-2026-08-31] Four is outside that window. The characteristic symptom is silence: an old kubelet accepts a spec containing fields it does not implement and does not act on them.

- **A is wrong.** The window is bounded at three.
- **C is wrong.** The scheduler will happily place Pods on it. The node is Ready and looks normal, which is what makes this hard to diagnose.
- **D is wrong**, and this is the danger. It does *not* error on unimplemented fields. If it did, you would have found the problem in five minutes.

15. B. `metrics-server` is the "Cluster addon component that collects and aggregates resource metrics pulled from each kubelet," and the API server then "serves Metrics API for use by HPA, VPA, and by the `kubect1 top` command." [source: k8s-docs-resource-metrics-pipeline-2026-08-31] It is not installed by default in all distributions.

- **A is wrong.** The kubelets are already exposing metrics. Nothing is aggregating them.
- **C is wrong.** `--event-ttl` governs event retention and has nothing to do with metrics.
- **D is wrong**, and it is the misreading of §7's figure that the question exists to catch. Nothing needs enabling: "Resource metrics are accessible using the `/metrics/resource` and `/stats kubelet` API

endpoints." [source: k8s-docs-resource-metrics-pipeline-2026-08-31] The measurements are already being published by every kubelet; the gap is one layer up.

16. B. Events are objects with a retention window governed by the API server's `--event-ttl`. [source: k8s-apiserver-event-ttl-and-toleration-defaults-2026-08-31] Hours after a failure, their absence tells you only that you are too late.

- **A is wrong.** That is precisely the flawed reasoning the question asks you to identify.
 - **C is wrong.** `describe` shows events for any object that has current ones.
 - **D is wrong,** and it conflates two lifetimes. Events have their own retention window and expire independently of the object they describe — which is exactly why a Pod that is still present can show `Events: <none>`.
-

Chapter Summary

Concept	Remember This
Triage order	Scope → phase → conditions → events → logs. Logs last, because they only mean something once you know the container ran.
The two-audience split	Platform scope asks whether Kubernetes did its job. Application scope asks whether your code did. Running + Ready + confined to one workload + still wrong = you have crossed the line. Several workloads wrong at once = look at the network, not the phase.
Pending	No feasible node. A stable, honest report, not a transient error. The scheduler's event names the reason — capacity and taints look identical without it.
No Pod object at all	An admission refusal leaves nothing to describe. The reason is on whoever tried to create it — your terminal, or the ReplicaSet.
ErrImagePull / ImagePullBackOff	The same problem at two moments. Bad name, missing image, or missing pull credentials. The container has never run.
ImageInspectError	The image arrived and the runtime cannot read it. Points at the image or the runtime, not at the registry or the credentials.
CreateContainerConfigError	Scheduled and pulled fine; a referenced ConfigMap or Secret could not be resolved. The cause lives in a different manifest.
ErrImageNeverPull	imagePullPolicy: Never, and the image is not on the node. Kubernetes did exactly what it was told.
CrashLoopBackOff	The container ran . Image fine, config fine, process started and exited. This names the wait between restarts: 10s, 20s, 40s, capped at 5 minutes, reset after 10 minutes of success.
OOMKilled	One container exceeded its own limit . Restarted in place, same node, restart count up.
Evicted	The node ran out of something. The kubelet ended the whole Pod (phase <code>Failed</code>), and a controller creates a replacement elsewhere.
Eviction order	BestEffort first, Burstable second, Guaranteed last. Asking for nothing does not make you safe — it makes you first.
Requests vs limits	The limit is the threshold that gets your container OOM-killed. The request and its QoS class govern your standing when the node is under pressure and something has to go.
Probe failures	Liveness fails → restart loop. Readiness fails → Running, 0/1 Ready, silently dropped from Service endpoints, no restarts. The quiet one.
--previous	Reads the container that died. On a crash-looping Pod, the default log read returns nothing because it reads the one that has not started.
Events expire	Retention is bounded and short — read <code>--event-ttl</code> on your own cluster. An absent event is never evidence that nothing happened.
Node conditions	<code>False</code> = somebody is reporting a problem. <code>Unknown</code> = nobody is reporting at all. Two different investigations.

Concept	Remember This
Node death	The node controller waits 5 minutes after Unknown before submitting the first eviction request, and Pods carry a default 300-second toleration for the not-ready and unreachable taints.
<code>crictl</code>	Below the API, on the node, talking to the runtime. Use it when the cluster's view and the node's view disagree.
Version skew	kubelet up to three minors behind the API server, never ahead; kubectl within one either way. An out-of-window kubelet ignores unknown fields silently .
<code>kubectl top</code>	Requires metrics-server, an addon many distributions do not install. The error is about the cluster's build, not the workload.
<code>kubectl logs</code> is not an archive	A live read of a rotating file on a node. An evicted Pod takes its logs with it.

⚓ **Safe Harbor** — you have finished the platform half of troubleshooting. Nine signatures, one lookup, one key.

The Voyage Ahead

You can now tell whether Kubernetes did its job.

That is a genuinely bounded skill, and its boundary is the interesting part. Everything in this chapter answered a question about the platform: did it schedule, pull, configure, start, keep alive. When the answer to all of those is yes, when the Pod is Running, it is Ready, its restart count is stable, its node is healthy, and no other workload is in trouble, the platform has finished and it hands you back your own problem.

That is where the next troubleshooting chapter picks up. The Pod is fine and the application is not. The request goes in and the wrong thing comes out; the Service exists and selects nothing; the config was mounted exactly as written and the value in it is wrong. None of those are visible from the phase, because the phase has stopped having anything to say.

The tools change too, and now you know why they were withheld. `kubectl exec` to get inside a container that is already running. Ephemeral containers and `kubectl debug` for images too minimal to contain a shell. `kubectl port-forward` to bypass the Service on purpose and prove where the break is. Every one of those requires a running container, which is why none of them belonged here.

[cross-bearing: see Ch 16 §1 — when the Pod is fine and the application isn't]

Before that, though, there is a different kind of question waiting. This chapter kept saying "somebody has to install that": metrics-server, a logging backend, an Ingress controller. The next chapters are about how anything gets installed on a cluster at all, and about the moment a folder full of YAML stops being a workable answer.

"The phase is the cluster telling you where it stopped. Every diagnosis you will ever make starts by believing it."

Chapter 14: Packing for the Voyage

"A chart is not a release, and templates are not the point"

Domain: Cloud Native Application Delivery — Application Delivery | **Domain Weight:** 16% [source: lf-kcna-exam-page-2026-08-23] **Complexity:** Mixed | **Novelty:** Moderate | **Prerequisites:** Standard

The 16% figure is the published weight for the whole Cloud Native Application Delivery domain, which this book divides across Chapters 14, 15, and 16. That division is an authored allocation, not a published one. CNCF publishes the domain weight and the competency names, and nothing finer [source: cncf-kcna-curriculum-pdf-2026-08-23].

Attention Budget

Total time: ~90 minutes | **Recommended:** Single session, or split after §4

Section	Time	Attention Cost	Best Time to Study
§1 Why a Folder of YAML Stops Working	10 min	Low	Anytime
§2 What a Chart Contains	15 min	Medium	Mid-session
§3 Chart, Release, Revision	18 min	High	Peak attention
§4 Where Charts Come From	8 min	Low	Anytime
🕒 Taking Your Bearings (1)	6 min	Medium	After a brief break
§5 Patching Instead of Templating	14 min	Medium	Mid-session
§6 Which One, When	8 min	Medium	Mid-session
🕒 Taking Your Bearings (2)	6 min	Medium	After a brief break
§7 A Package, Not a Template	5 min	Low	Anytime

Attention Cost Key:

- **Low:** Concrete, familiar concepts — study anytime
- **Medium:** New concepts requiring focus — study when alert
- **High:** Abstract or complex material — study at peak attention

If you only have 15 minutes: read §3. It is the chapter, and the other six sections are its supporting cast.

"Charts are easy to create, version, share, and publish — so start using Helm and stop the copy-and-paste." — The Helm project [source: helm-homepage-2026-08-31]

🔊 Soundings

Before reading this chapter, try these eight questions. Your score tells you how to read what follows; there is no bad score here, only different reading strategies. Every question is answerable from Chapters 1 through 13 or from professional experience you already have.

1. You run `kubectl apply -f manifests/` against a directory holding twelve YAML files. What does that command do, and what does it promise you about the order in which those twelve files are applied?
2. You have a cluster with no metrics-server, and `kubectl top nodes` is failing. In a declarative system, what does "install metrics-server" actually consist of?
3. You run `kubectl rollout undo deployment/api`. What changes in the cluster?
4. You apply a manifest whose `kind` is `PostgresCluster`. The API server rejects it because it does not recognize that kind. What has to happen first, and who is responsible for making it happen?
5. Your application needs a different database hostname in staging than in production. Where does that hostname belong, and why is baking it into the container image the wrong answer?
6. What is the difference between an image tag and an image digest, and what, precisely, is a registry?
7. From your own professional experience with any package manager at all (apt, npm, pip, NuGet, Homebrew, whatever you have used), what does a package manager give you that a folder of files does not?
8. Two clusters need to run the same application. One needs three replicas and the hostname `app.staging.example.com`; the other needs twelve replicas and `app.example.com`. Using only the tools this book has taught you so far, what are your options, and what is wrong with each of them?

Answers + reading strategy

1. It applies every manifest in the directory, creating what does not exist and updating what does [*cross-bearing: see Ch 4 §1 — declarative object configuration and kubectl apply*]. It promises nothing about ordering. The files go in; the API server accepts them; whether object A exists before object B is not something the command undertakes to arrange.
2. It consists of applying a file of Kubernetes objects that somebody else wrote — literally `kubectl apply -f ../components.yaml` [source: metrics-server-install-2026-08-31]. There is no installer. There is only

YAML and `kubectl apply` [*cross-bearing: see Ch 13 §7 — metrics-server and the resource metrics pipeline*].

3. It sets the Deployment's Pod template back to a previous revision's template, which causes the Deployment controller to roll the workload back [*cross-bearing: see Ch 6 §5 — every rollout is a revision*].

4. A CustomResourceDefinition naming that kind has to be registered with the API server first, and somebody has to do that registering: a cluster administrator, or an operator's installation procedure. Custom kinds do not exist until their definitions do [*cross-bearing: see Ch 6 §8 — the control loop, extended*].

5. In a ConfigMap, or a Secret if it is sensitive, consumed by the Pod as an environment variable or a mounted file. Baking it into the image is wrong because it makes the image environment-specific: you would need one image per environment, which destroys the property that made images useful in the first place [*cross-bearing: see Ch 4 §4 — configuration kept outside the image*].

6. A tag is a mutable, human-friendly pointer at an image; a digest is an immutable cryptographic hash of the image's content, and is therefore its identity. A registry is a service that stores and serves images, addressed by digest [*cross-bearing: see Ch 2 §3 — registries, tags, and digests*].

7. Most people's answer includes some subset of: a name, a version, a place to get it from, the ability to install without reading the contents, the ability to uninstall cleanly, and the ability to find out what is currently installed. Hold onto whichever ones you named. You will need them shortly.

8. Honest answers: copy the directory and edit the copy, and now you maintain two directories that will drift; run `sed` over the manifests in a shell script, and now your configuration is a text-substitution program with no validation; hand-edit before each apply, and now the process is a human with a good memory. Every one of these is a real thing people do. Every one of them has a specific failure mode, and this chapter names all four.

If you got 6+ right: Skim, but read §3 and §6 properly. Those two are what this chapter is actually about; the rest is vocabulary you can move through quickly.

If you got 3–5 right: Read at normal pace. This chapter is calibrated for you.

If you got 0–2 right: Re-read **Ch 4 §1** before starting this chapter — not alongside it. That section is the foundation the whole first half rests on. If question 3 specifically was one you missed, add **Ch 6 §5** to that list; §3 of this chapter is unreadable without it.

Why This Chapter Matters

Thirteen chapters have now told you to install something.

Install a CNI plugin, because the network model requires one and Kubernetes does not ship one [*cross-bearing: see Ch 9 §1 — the network model and CNI*]. Install an Ingress controller, because the Ingress object does nothing without one [*cross-bearing: see Ch 10 §3 — the object is not the implementation*]. Install a CSI driver. Install metrics-server, or `kubect1 top` will keep failing [*cross-bearing: see Ch 13 §7 — metrics-server and the resource metrics pipeline*]. Chapter 13 closed by naming this debt out loud: *this chapter kept saying "somebody has to install that."*

Not one of those chapters said how.

That was deliberate, and it is now due. Here is the shape of the answer, and the shape of the problem. In a declarative system there is no installer; there is only a set of objects that somebody wrote, which you apply. "Install metrics-server" means: fetch a set of YAML files somebody else wrote, read enough of them to find the values that need to be different on your cluster, change those, apply the whole set in roughly the right order, and hope. That is not a distribution mechanism. That is a directory with instructions attached, and the instructions are a note somebody left for the next watch.

So the question this chapter opens on is not "which tool should I use." It is prior to that. **When somebody hands you a directory of YAML and a README, what have they actually handed you, and why does it not have a name yet?**

Everything you have built in this book so far has been a single object. A Pod. A Deployment. A Service. A PersistentVolumeClaim. Practitioners do not hand each other objects. They hand each other *units*: a named, versioned thing that installs as one act, uninstalls as one act, can be given to somebody who will never read it, and can be undone when it turns out to have been a mistake. This chapter is where you stop writing manifests and start shipping software, and the shift is larger than the tooling makes it look.

Dead Reckoning: Helm is a package manager for Kubernetes [source: helm-homepage-2026-08-31]. Its packaging format is a chart: a collection of files describing a related set of Kubernetes resources, laid out in a particular directory tree, and packageable into versioned archives for deployment [source: helm-charts-2026-08-31]. Installing a chart creates a release — an instance of that chart running in a cluster [source: helm-architecture-2026-08-31] — and a sequential counter tracks releases as they change [source: helm-glossary-2026-08-31]. Kustomize is a standalone tool for customizing Kubernetes objects through a kustomization file [source: k8s-docs-kustomization-2026-08-31]; it is template-free and built into `kubectl` as `apply -k` [source: kustomize-overview-2026-08-23]. Kubernetes requires neither. Both exist because a directory of manifests has no name, no version, and no declared surface of variation.

The stakes are the ones Chapter 1 already put on the table. Cloud Native Application Delivery carries 16% of the current blueprint [source: lf-kcna-exam-page-2026-08-23], and this is the first chapter that cashes that.

One disclosure before we start, because this book has been consistent about them and this is the chapter where consistency costs something.

CNCF publishes the competency name, "Application Delivery," and publishes nothing beneath it [source: cncf-kcna-curriculum-pdf-2026-08-23]. No sub-topic list. No named tools. The words *Helm* and *Kustomize* appear nowhere in the published curriculum [source: cncf-kcna-curriculum-pdf-2026-08-23], nowhere in the domain and competency list on the Linux Foundation exam page [source: lf-kcna-exam-page-2026-08-23], and nowhere on the public course outline for LFS250, the training course the Linux Foundation bundles with the exam [source: lf-lfs250-course-outline-2026-08-31]. The published outline resolves only to a chapter title.

So the topic list in this chapter is authored inference. It comes from what the ecosystem actually uses to solve the problem the competency names, and from the fact that a domain called Application Delivery with no coverage of how applications are packaged would be a strange thing indeed. It is, I think, clearly the right call. But you should know it is a call. In practice this means one rule holds throughout: nothing in this chapter is described as "frequently tested" or "commonly appears," because nobody outside the exam authority knows that. Where two things are genuinely easy to confuse, I will say so, and that claim stands on its own.

Logbook Entry: Every operations team eventually accumulates the directory. You know the one. It is called `k8s/` or `deploy/` or, in the most honest cases, `yaml/`. Inside it there is `deployment.yaml`, and beside that `deployment-staging.yaml`, and beside *that* `deployment-prod-final-v2-USE-THIS-ONE.yaml`, whose name is a small monument to the moment somebody realized the naming scheme had stopped working and did not have time to fix it.

There is usually a `README` too, and the `README` usually contains a numbered list, and step 4 of that list usually says something like "change the image tag and the ingress host, then apply in this order." Step 4 is the whole problem, written down by somebody who could see it clearly and had no vocabulary for it.

This is not incompetence. It is what happens when a perfectly good technique, declarative object configuration, is asked to do a job one size larger than the job it was designed for. The directory is the correct answer to "how do I describe what should be running." It is the wrong answer to "how do I give this to someone else."

What You'll Learn

By the end of this chapter, you'll be able to:

- **Name** the four things a folder of manifests cannot do, and recognize each one the moment you hit it
- **Read** a Helm chart's directory layout and say what each entry is *for*
- **Separate** package from installation from installed-state — chart, Helm release, release revision — and stop collapsing three things into one word

- **Distinguish** `helm rollback` from `kubectl rollout undo`: different unit, different scope, same English word
- **Explain** what Kustomize does instead of templating, and why it needs no engine you have to install
- **Choose** between the two for a given situation, and say what the choice actually turns on

You'll also stop reading this as a chapter about YAML syntax, which is the misreading it exists to prevent.

.. §1 — Why a Folder of YAML Stops Working

Start where Chapter 13 left you: with a cluster that needs metrics-server, and no idea how it gets there.

You already have a technique for this, and it is a good one. Write the objects you want. Put them in a directory. Run `kubectl apply -f manifests/`. The API server records your intent and the controllers reconcile toward it [*cross-bearing: see Ch 4 §6 — a declaration, not an order*]. For a single application on a single cluster maintained by a single person, this is not a compromise or a stepping stone. It is correct, and you should keep doing it.

It stops working in four specific places. Not "gets awkward" — stops. Each of the four is a thing the technique structurally cannot do, and naming them precisely matters, because the rest of this chapter answers exactly these four and nothing else.

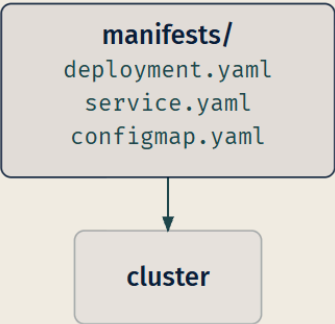
Failure one: environment variation

The same application runs in three places. Staging wants two replicas, production wants twelve, and the developer's laptop cluster wants one. Staging points at `db.staging.internal`, production at `db.prod.internal`. Production has resource limits; the laptop cannot afford them.

You know where the *configuration* goes: ConfigMaps and Secrets, kept outside the image [*cross-bearing: see Ch 4 §4 — configuration kept outside the image*]. But the replica count is not configuration the application reads. It is a field in the Deployment's spec. So is the resource limit. So is the image tag. These are differences in the manifests themselves, and a directory of manifests has exactly one answer to that: make another directory.

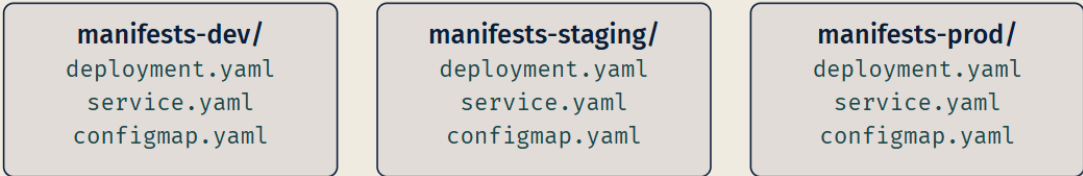
Now you have two directories that are 95% identical, and the 5% is not marked. Six weeks later somebody fixes a readiness probe in staging and does not fix it in production, because nothing told them there was a second copy. This is drift, and it is not a discipline problem: it is what happens when two files are supposed to stay in sync and nothing in the system knows that they are.

ONE CLUSTER



Correct. Keep doing this.

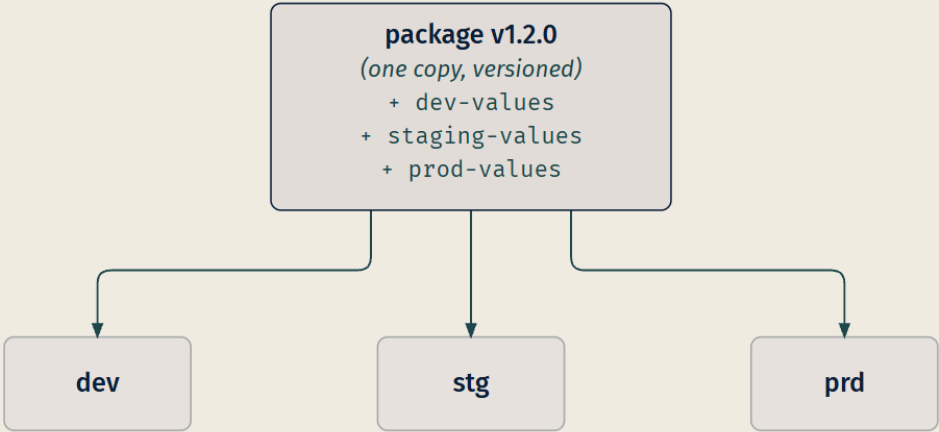
THREE CLUSTERS



95% IDENTICAL
and nothing marks the 5%

THIS IS WHERE MOST TEAMS LIVE

WHAT YOU WANT



LEGEND CONFIG FILES K8S CLUSTER DUPLICATION RISK

Failure two: apply ordering

`kubectl apply -f manifests/` applies everything in the directory. It does not undertake to apply things in an order that makes sense.

Mostly this does not matter, because Kubernetes is a reconciling system: a Deployment whose ConfigMap does not exist yet will produce Pods that cannot start until the ConfigMap shows up, and then it will recover on its own [*cross-bearing: see Ch 13 §2 — diagnosing a Pod that will not start*]. Eventual consistency covers a great many ordering sins.

It does not cover all of them. The sharpest case is the one you pre-tested in the Soundings: a manifest whose `kind` is a custom resource. Custom kinds do not exist until a CustomResourceDefinition registers them with the API server [*cross-bearing: see Ch 6 §8 — the control loop, extended*]. Apply a `PostgresCluster` before the CRD that defines `PostgresCluster`, and the API server does not queue your object for later. It rejects it, because as far as the API server is concerned you have asked it to create an object of a kind that does not exist. There is no reconciliation to wait for. There is only an error.

So the README says "apply the CRDs first, then everything else." Which is to say: the ordering information exists, and it lives in prose, in a file that no software reads.

Failure three: versioning

A directory does not have a version.

This sounds like a small thing and it is not. Ask the question that operations teams ask at three in the morning: *what is currently installed on this cluster?* With a directory, the honest answer is "whatever was in that directory the last time somebody applied it, assuming nobody has applied anything since, and assuming the directory has not changed." You can check what objects exist. You cannot ask the cluster what *version of the thing* is installed, because the thing has no version and, strictly speaking, is not a thing. It is a set of objects that happen to have arrived together.

And because there is no version, there is no meaningful undo. You can roll back a Deployment [*cross-bearing: see Ch 6 §5 — every rollout is a revision*], and that is genuinely useful, but it rolls back one Deployment. It does not roll back "the release that also changed the ConfigMap and added a Service and modified the Ingress rule." Those are five objects with five independent histories, and nothing in the cluster knows they were once one act.

Failure four: distribution

The one that matters most, and the one that is easiest to miss because it does not bite you. It bites the person you hand the directory to.

You cannot give somebody a directory of manifests and expect them to install it without reading it. They have to read it, because the parts they need to change are not marked, and the order is in the README,

and the README might be out of date. Every installation is a small act of comprehension — which is, again, a poor thing to leave for the next watch. And this time the note *is* the delivery mechanism.

This is why "install metrics-server" is an *apply a file of objects* operation rather than a command. The project's own instruction is exactly that: `kubectl apply -f` against a published `components.yaml` [source: metrics-server-install-2026-08-31]. It needs the aggregation layer enabled on the API server to serve the Metrics API through it [source: metrics-server-install-2026-08-31], which is why the file contains registration and not just a workload. It is one indivisible apply because there is no unit smaller than "all of those files, plus the knowledge of which parts are yours to change."

📌 **Snag:** It is tempting to read these four as one problem, "manifests are messy." They are not one problem, and the difference shows up on exam-shaped questions. Environment variation is about *what varies*. Ordering is about *sequence*. Versioning is about *identity over time*. Distribution is about *handing it to a stranger*. A tool can solve some and not others, and §6 is entirely about which solves which.

Two tools in the ecosystem answer these four, from opposite directions. **Helm** is a package manager for Kubernetes [source: helm-homepage-2026-08-31], and is the whole of §2, §3, and §4. **Kustomize** takes a different route entirely and is §5 [cross-bearing: see Ch 14 §5 — *patching instead of templating*].

That is all you get about the answers for now. Sit with the problem a moment longer; §2 will not mean much unless the problem is clear.

§2 — What a Chart Contains

Helm calls its packaging format a **chart**, which is a convenient word for a book like this one.

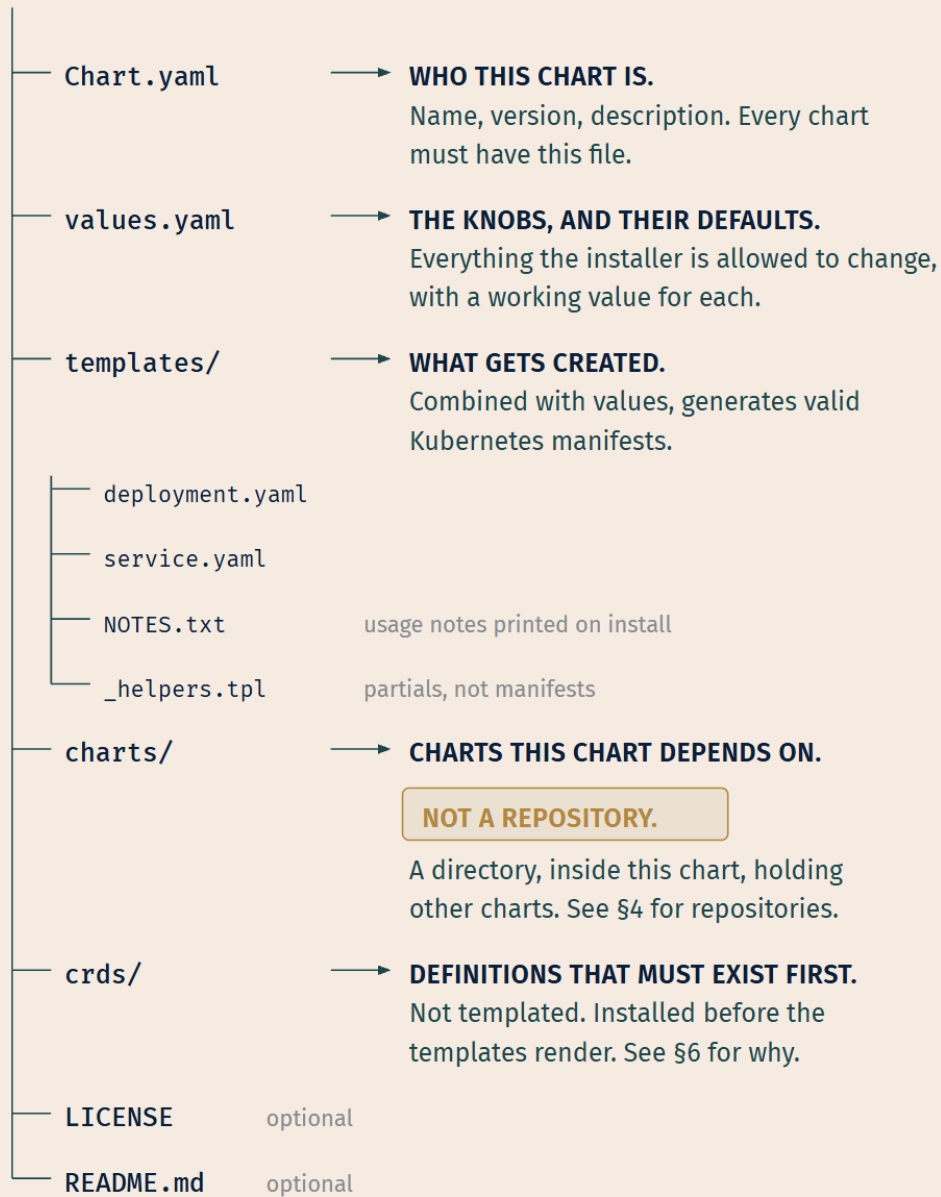
☆ Fixed Point

A chart is a collection of files that describe a related set of Kubernetes resources, laid out in a particular directory tree, and packageable into versioned archives for deployment [source: helm-charts-2026-08-31]. A single chart might deploy something as simple as one memcached Pod, or as complex as a full web-app stack with HTTP servers, databases, and caches [source: helm-charts-2026-08-31].

Note what that definition contains and what it does not. It says *collection of files, versioned, deployable*. It does not say *templated*. Templating is in there, and we will get to it, but the definition of a chart does not require it, and that is not an accident of phrasing. Hold onto it; §7 is built on it.

The directory name is the name of the chart, without version information: a chart describing WordPress lives in a `wordpress/` directory [source: helm-charts-2026-08-31].

wordpress/



LEGEND → ENTRY → PURPOSE

 CRITICAL DISTINCTION • OPTIONAL DETAIL

Take the five annotated entries one at a time, and notice that the useful question about each is not *what is it* but *what is it for*.

Chart.yaml — the chart's identity

Every chart must have this file [source: helm-glossary-2026-08-31]. It carries the chart's name, its version, and its description. Charts are versioned according to the SemVer 2 specification, and **a version number is required on every chart** [source: helm-glossary-2026-08-31].

That requirement is failure three from §1, closed by fiat. A directory has no version because nothing makes it have one; a chart has a version because Helm refuses to work with a chart that lacks one. Chart.yaml is also where dependencies are declared, having absorbed the older requirements.yaml file in Helm 3 [source: helm-changes-since-helm2-2026-08-31] — which is why the file's own apiVersion was bumped from v1 to v2 for Helm 3 [source: helm-changes-since-helm2-2026-08-31]. If you ever find yourself looking at a chart in the wild and wondering how old it is, that field is the tell.

values.yaml — the declared surface of variation

values.yaml holds the default configuration values for the chart [source: helm-charts-2026-08-23].

This is failure one, and "it holds the defaults" undersells what the file accomplishes. values.yaml is where the chart author states, in one place, **which things an installer is allowed to change**, and gives each of them a value that works. Everything in that file is a knob. Everything not in that file is a decision the chart author made on your behalf.

That is the same move as ConfigMaps, one level up. A ConfigMap externalizes configuration out of the image so the image can be environment-independent [*cross-bearing: see Ch 4 §4 — configuration kept outside the image*]. values.yaml externalizes variation out of the manifests so the *manifests* can be environment-independent. Same principle, different layer.

At install time you override those defaults. --values (or -f) points at a YAML file of overrides, and can be given more than once, with the rightmost file taking precedence; --set supplies overrides on the command line, and where both are used, --set values are merged into --values with higher precedence [source: helm-using-helm-2026-08-31]. Worth committing to memory in that order — rightmost file wins, and --set wins over any file.

templates/ — what gets created

A directory of templates that, **when combined with values, generate valid Kubernetes manifest files** [source: helm-charts-2026-08-23].

That sentence is the whole mechanism. A template is a Kubernetes manifest with holes in it; values fill the holes; the result is an ordinary manifest of the kind you have been writing since Chapter 4. Here is a fragment of a Deployment template beside what it renders to:

```
Template (in templates/deployment.yaml):
```

```
spec:
  replicas: {{ .Values.replicaCount }}
```

With values.yaml containing `replicaCount: 3`, this renders to:

```
spec:
  replicas: 3
```

That is as far as this book goes into Helm's template language. It supports named partials, which is why `_helpers.tpl` exists [source: helm-named-templates-2026-08-31], and a good deal besides; you could spend a week on it. You do not need to for this exam, and, more to the point, spending pages on template syntax would argue against this chapter's own conclusion. The syntax is not what a chart *is*.

Two entries in `templates/` are not manifests. `NOTES.txt` is an optional plain-text file containing short usage notes [source: helm-charts-2026-08-23]: the "here is how to reach your new installation" text a chart author leaves for whoever installs it. And files whose names begin with an underscore, conventionally `_helpers.tpl`, are assumed *not* to have a manifest inside; they are not rendered to Kubernetes object definitions but are available everywhere within other chart templates for use [source: helm-named-templates-2026-08-31]. They hold reusable partials. Neither becomes an object in your cluster.

charts/ — a directory of dependency charts

A directory containing any charts upon which this chart depends [source: helm-charts-2026-08-23]. A chart nested inside another chart this way is a **subchart**.

This solves a real problem: your application chart needs a Redis, and somebody has already written a good Redis chart, so you depend on theirs instead of writing your own. A chart in there is still a chart, which means it still has templates, which means it is still a source of objects.

📌 **Snag:** `charts/` is not a chart repository. It is a directory *inside a chart* holding the charts that chart depends on. A chart repository is something else entirely: an HTTP server, out on the network, which §4 covers. The two share a word and share nothing else, and this is one of the genuinely easy confusions in this material. If you take one thing from this figure, take that annotation.

crds/ — Custom Resource Definitions

A special directory you can create in your chart to hold your CRDs [source: helm-crd-best-practices-2026-08-31]. These CRDs are **not templated**, and are installed by default when you run `helm install` for the chart [source: helm-crd-best-practices-2026-08-31].

That is failure two, ordering, and §6 explains exactly why it needs its own directory rather than being one more file in `templates/`. For now, note the two sourced properties: not templated, and installed by default rather than by instruction. Both will matter.

The section's thesis, stated plainly now that you have seen the parts: **the chart is the unit, and templating is how the unit absorbs variation**. Templating is not what makes it a chart. Versioning, a declared variation surface, and packageability are what make it a chart. That distinction is going to do a great deal of work in §7.

A worked instance, to make the abstraction concrete. Take the Deployment from Chapter 6, the one with the ownership chain down to ReplicaSets and Pods [*cross-bearing: see Ch 6 §1 — the resource that holds the intent*]. Chapter 6 promised that a Helm chart's job is to template that object, and here is what that means: the Deployment goes in `templates/deployment.yaml` with its replica count, image tag, and resource limits replaced by value references; the working defaults for those go in `values.yaml`; and the whole thing gets a name and a version in `Chart.yaml`. The Deployment did not change. It acquired a wrapper that knows what varies about it.

Everything in a chart's `templates/` becomes objects in a cluster the same way your hand-written manifests do, through the API server, as records of intent [*cross-bearing: see Ch 4 §2 — the anatomy of a record*]. Charts do not have a private channel into the cluster. They generate manifests and the manifests go the ordinary way.

One thing charts are *not*, and it is worth saying now so you are not surprised in Chapter 15: a chart is a source of manifests, not a thing that applies them on a schedule. A delivery agent that watches a repository and keeps a cluster in sync can use a chart as its manifest source, and that arrangement is where a great deal of production delivery actually happens [*cross-bearing: see Ch 15 §4 — an agent that watches a repository*].

..1 §3 — Chart, Release, Revision

This is the section the subtitle is about, and the one thing in this chapter to get exactly right.

Four words are in play, and readers routinely collapse them into two. You already own two of them. All four side by side, then the rest of the section goes to the two that are new.

Word	What it is	Where you met it
Package	The chart. A named, versioned collection of files.	§2, just now
Manifest	What templates render to. An ordinary Kubernetes object description.	Ch 4 §2
Helm release	One installation of a chart into a cluster.	Here
Release revision	One numbered state of that release over time.	Here

A release is an installation, not a package

☆ Fixed Point

A release is an instance of a chart running in a Kubernetes cluster [source: helm-architecture-2026-08-31]. When a chart is installed, the Helm library creates a release to track that installation [source: helm-glossary-2026-08-31]. **The same chart can be installed many times, each creating a separately named release that can be upgraded and rolled back independently** [source: helm-charts-2026-08-23].

Read that last clause twice. One chart, many releases. Independently upgradable. Independently rollbackable.

The chart is the thing on the shelf. The release is the thing you installed. If you install the same WordPress chart three times, one for the marketing site, one for the docs site, one for a staging copy, you have one chart and three releases, each with its own name, its own values, and its own history. Upgrading one does not touch the others.

Naming is now mandatory. In Helm 2, omitting a name produced an auto-generated one; in production that proved to be more of a nuisance than a helpful feature, so **Helm 3 throws an error if no name is provided with `helm install`** [source: helm-changes-since-helm2-2026-08-31]. If you want one generated anyway, `--generate-name` will do it [source: helm-changes-since-helm2-2026-08-31].

Releases live in namespaces. In Helm 3, information about a release is stored in the same namespace as the release itself, which means you can `helm install wordpress stable/wordpress` in two separate namespaces and refer to each by changing your namespace context [source: helm-changes-since-helm2-2026-08-31]. A release name is scoped to a namespace, exactly as most Kubernetes object names are [*cross-bearing; see Ch 4 §3 — where a name lives*].

`helm list` follows the same scoping: it lists the releases in your current context's namespace, and `--all-namespaces` widens it to the cluster [source: helm-changes-since-helm2-2026-08-31].

Which answers the question §1 left open and never closed. *What is currently installed on this cluster?* `helm list` names the releases in your namespace; `--all-namespaces` names them everywhere. A directory has no such command, and not because nobody wrote one — because a directory is not a thing you can ask about.

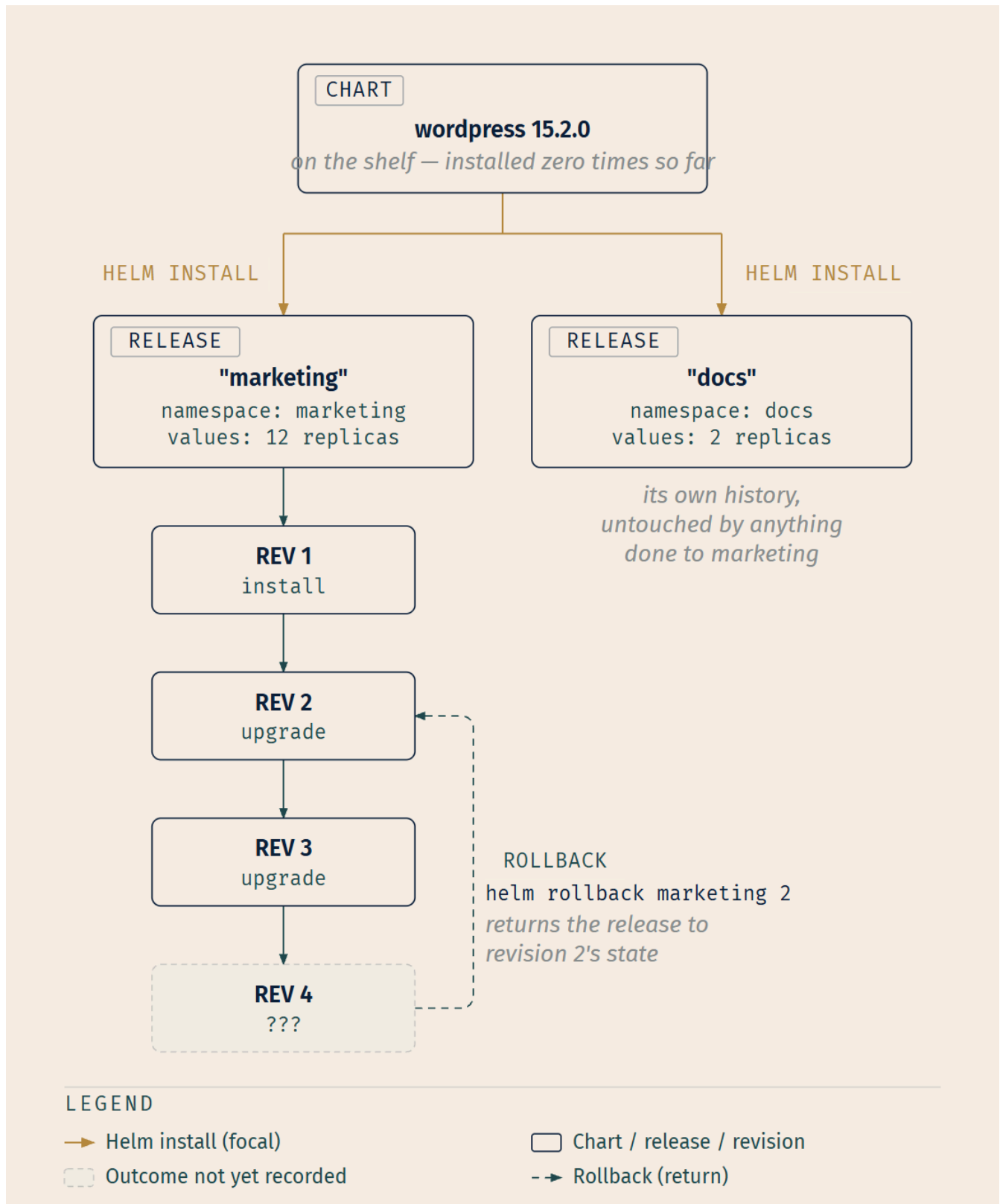
🔒 **Worth Securing:** Where does a Helm release's state actually live? **In Secrets, in the namespace of the release, by default** [source: helm-storage-backends-2026-08-31]. Not in a Helm server component — there isn't one — and not on your laptop. The `HELM_DRIVER` environment variable selects the backend and accepts `configmap`, `secret`, or `sql` [source: helm-storage-backends-2026-08-31].

This is a small fact with a large implication: Helm's record of what it installed is an ordinary Kubernetes object, subject to the ordinary rules [*cross-bearing: see Ch 4 §4 — configuration kept outside the image*]. Whoever can read Secrets in that namespace can read Helm's bookkeeping. Whoever can delete them can make Helm forget.

A revision is a numbered state of a release

Installing a chart creates a new release object [source: helm-using-helm-2026-08-31]. Upgrading changes it. **A sequential counter is used to track releases as they change** [source: helm-glossary-2026-08-31].

Upgrades are not wholesale replacements. `helm upgrade` will only update things that have changed since the last release [source: helm-using-helm-2026-08-31]. The unchanged objects stay as they are.



That question mark is deliberate, and it is worth a sentence. **Whether a rollback is itself recorded as a new numbered revision — a rev 4 whose contents match rev 2 — or simply returns the release to revision 2, is not something Helm's own documentation settles in the material this book cites.** What is certain is

the effect: after the command, the release is running revision 2's state. Do not memorize a counter behavior for this; memorize that a rollback is an *ordinary release operation* that takes you to a numbered revision you name.

The rollback command's argument shape is straightforward: **the first argument is the name of a release and the second is a revision (version) number; if that second argument is omitted or set to 0, it rolls back to the previous release** [source: helm-rollback-cli-2026-08-31].

The word that means two things

Now the payoff, and Chapter 6 owes you this one. It told you that you would meet the word "rollback" twice more in this book, attached to entirely different mechanisms, and pointed at this chapter and at Chapter 15 [cross-bearing: see Ch 6 §5 — *every rollout is a revision*]. This is the first of those two. The second is Chapter 15's rollback-by-revert [cross-bearing: see Ch 15 §4 — *an agent that watches a repository*], and it is a third thing again.

A bearing is only meaningful with its landmark attached. So is this word.

☆ Fixed Point

helm rollback and kubectl rollout undo are different mechanisms wearing the same English word. They differ in three ways:

- **Unit.** `helm rollback` operates on a *Helm release* — everything the chart installed. `kubectl rollout undo` operates on *one workload object*: a Deployment, DaemonSet, or StatefulSet [source: k8s-docs-kubectl-rollout-2026-08-24].
- **Scope.** A Helm rollback returns the whole release to a previous revision: the Deployment, the Service, the ConfigMap, the Ingress rule, all of it. `kubectl rollout undo` changes one workload's Pod template and nothing else.
- **Bookkeeping.** Helm's history lives in the release record — Secrets in the release's namespace [source: helm-storage-backends-2026-08-31]. A Deployment's revision history lives in its ReplicaSets [cross-bearing: see Ch 6 §1 — *the Deployment to ReplicaSet to Pod ownership chain*].

Neither calls the other. `helm rollback` does not run `kubectl rollout undo` underneath.

That last line is the trap to name outright, because the two mechanisms *look* like they should be related and are not. Helm's own account of how it works has no Kubernetes-client shell-out in it: it fetches information from the Kubernetes API server, renders the charts client-side, and stores a record of the installation in Kubernetes [source: helm-changes-since-helm2-2026-08-31]. So when Helm rolls a release back, it computes the objects the target revision described and applies them. If that changes a Deployment's Pod template, the Deployment controller does what it always does and starts a rolling update [cross-bearing: see Ch 6 §4 — *changing the fleet under way*]. The rolling update is a *consequence*, in the ordinary reconciling way. Helm did not ask for it. Helm changed a record of intent, and a controller noticed.

🧠 **Mnemonic: The scope is in the noun.** `helm rollback` takes a *release* name. `kubectl rollout undo` takes a *Deployment* name. Whatever noun the command takes is the unit it moves, and since one of those nouns contains many of the other, the difference in blast radius follows from the grammar. If you can remember which command takes which noun, you cannot get the scope question wrong.

⚠️ Navigational Hazards

Three collapses to avoid, in increasing order of how often they catch people:

Chart and release used interchangeably. "I rolled back the chart." You did not; a chart is a package and packages do not have installed state. You rolled back a release. This one matters because the correction is not pedantry: the whole point of the distinction is that *one chart installs many times*, and a sentence that conflates them cannot express that.

"Revision" left unqualified. Chapter 6 owns the unqualified word for Deployment revisions. In this chapter and the next, say **release revision** or **Helm revision** whenever there is any chance of ambiguity.

"Rollback" left unqualified. There are three rollbacks in this book — Deployment, Helm release, and GitOps revert — and the bare noun distinguishes none of them. Say which.

Extended Analogy: Think about how you already reason about installed software on a machine you administer.

There is a *package*: a named, versioned artifact sitting in a repository, identical for everyone who fetches it. Nobody's machine changes it. It is a thing you can name and a thing you can pin.

There is an *installation*: what happened when you ran the install on this particular machine, with these particular options. Two machines can install the same package and end up meaningfully different, because the options differed. The installation has a location, a configuration, and a state.

And there is the *history* of that installation, the record of what version was installed when, and what it was before, which is what lets you say "put it back the way it was on Tuesday."

Chart, release, revision. You have had this mental model since the first time you administered anything. Helm did not invent it; Helm brought it to Kubernetes, where the native tooling had objects but no packages. The reason the vocabulary is worth learning precisely is not that the concepts are hard, since you already hold them, but that Kubernetes had no word for any of it until Helm supplied three.

§4 — Where Charts Come From

Failure four was distribution: you cannot hand somebody a directory. So where do charts live, and how do they travel?

The chart repository

☆ Fixed Point

A chart repository is an HTTP server that houses an `index.yaml` file and optionally some packaged charts [source: helm-chart-repository-2026-08-31]. It is managed with the `helm repo` commands [source: helm-charts-2026-08-23].

That is the whole definition, and its plainness is the interesting part. Not a service with an API surface. Not a daemon. An HTTP server, holding an index file and some tarballs. Because a chart repository can be any HTTP server that can serve YAML and tar files and answer GET requests, you have a plethora of options for hosting your own [source: helm-chart-repository-2026-08-31].

The index is a YAML file called `index.yaml`, containing metadata about the packages, including the contents of each chart's `Chart.yaml` file [source: helm-chart-repository-2026-08-31]. That is what the index is *for*: one file that tells a client what the server has and at what versions, without downloading anything else.

A packaged chart is a **chart archive**: a tarred and gzipped, and optionally signed, chart [source: helm-glossary-2026-08-31].

📌 **Snag**: Second half of the trap from §2, and this is the sentence to hold: `charts/` is a **directory inside a chart** containing the charts it depends on. A **chart repository** is an **HTTP server on the network** housing packaged charts. Same word, opposite ends of the pipeline: one is a component of a package, the other is a place packages are kept. If a question puts both in play, that is the distinction it is testing.

Registries hold charts too

Here is the part that pays off Chapter 2 in a way you would not have predicted.

You learned about registries as the place container images live [*cross-bearing: see Ch 2 §3 — registries, tags, and digests*]. Re-read the OCI's own definition of a registry with fresh eyes: **"a service that handles the required APIs defined in this specification"** [source: oci-distribution-spec-2026-08-24]. Not "a service that stores images." A service that implements an API for distributing *content*.

That distinction was doing quiet work all along. The OCI Distribution Specification defines "an API protocol to facilitate and standardize the distribution of content" [source: oci-distribution-spec-2026-08-24]: content, addressed by digest, with no requirement that the content be an image.

So: **as of Helm 3.8.0, Helm is able to store and work with charts in container registries, as an alternative to Helm repositories** [source: helm-blog-oci-ga-2026-08-31]. Since OCI artifacts make it possible to store more than container images, you can store charts, images, and other artifacts in a single OCI registry [source: helm-blog-oci-ga-2026-08-31]. The Helm documentation now recommends using container registries with OCI support to store and share chart packages [source: helm-oci-registries-2026-08-31].

An OCI-based registry can contain zero or more Helm repositories, and each of those repositories can contain zero or more packaged Helm charts [source: helm-oci-registries-2026-08-31]. When pushing, the reference must be prefixed with `oci://` [source: helm-oci-registries-2026-08-31]; the registry reference basename is inferred from the chart's name, and the tag from the chart's semantic version [source: helm-oci-registries-2026-08-31].

The reasoning behind the shift is instructive, and the Helm project wrote it down. Chart repositories had "a very hard time abstracting most of the security implementations required in a production environment," a standard authentication and authorization API being important in production; chart provenance signing was optional rather than integral; the same chart uploaded by two tenants cost twice the storage; and a single index file serving search, metadata, and fetching was clunky to design around securely [source: helm-changes-since-helm2-2026-08-31]. Meanwhile the Distribution project, the successor to the original Docker Registry, had many years of hardening, security best practices, and battle-testing behind it, offered as a product by many major cloud vendors [source: helm-changes-since-helm2-2026-08-31].

Which is to say: rather than reinvent a hardened content-distribution service, Helm moved onto the one the industry had already built. Standardization at the OCI layer [*cross-bearing: see Ch 2 §5 — the Open Container Initiative*] turned out to buy something nobody was specifically aiming at when the distribution spec was written.

Chart version is not application version

One quiet error to close on.

Charts are versioned according to SemVer 2, and a version number is required on every chart [source: helm-glossary-2026-08-31]. That version is **the chart's**, the packaging's. It moves when the packaging changes: a template fixed, a default adjusted, a new value exposed.

`Chart.yaml` separately carries `appVersion`, documented as "the version of the app that this contains" [source: helm-charts-2026-08-31] — the version of the *application the chart installs*. These move independently, and Helm says so outright: **"Note that the `appVersion` field is not related to the version field"** [source: helm-charts-2026-08-31]. It is informational, and has no impact on chart version calculations [source: helm-charts-2026-08-31]. You can ship chart 4.1.2 and 4.1.3 that both install nginx 1.25.3, because what changed between them was the chart. You can ship chart 5.0.0 that installs nginx 1.26.0, where both changed at once.

🧠 **Mnemonic: Version is the box; `appVersion` is what's in the box.** You can redesign the box without changing the contents, and you can change the contents without redesigning the box. If a question gives you two numbers and asks which one the chart maintainer bumps when they fix a template typo, it is the box.

🕒 Taking Your Bearings: The Package and the Thing You Installed

Five questions on §1 through §4. Take them before reading the answers.

1. You run `helm install marketing wordpress/wordpress` and then, in a different namespace, `helm install docs wordpress/wordpress`. Later you run `helm rollback marketing` 2. What happens to the docs installation?

A) It also rolls back to revision 2, because both come from the same chart B) Nothing — releases are upgraded and rolled back independently C) It rolls back only if it shares a namespace with `marketing` D) `helm list` will now show `docs` as out of date, since the two releases track the same chart version
2. A colleague says: "I looked in the chart's `charts/` directory to find the version of Redis available in our chart repository." What has gone wrong in that sentence?

A) Nothing — `charts/` is the local cache of the configured chart repository B) `charts/` holds the charts this chart depends on; a chart repository is an HTTP server elsewhere on the network C) `charts/` is where `helm repo add` writes its index, so it holds only metadata, not versions D) `charts/` is only present after `helm install`, so it would be empty before then
3. Which of these is required to be present in every Helm chart?

A) `crds/` B) `templates/NOTES.txt` C) `Chart.yaml`, including a version number D) `_helpers.tpl`
4. [retrieval: ch6] You run `kubectl rollout undo deployment/api`. Which statement is correct?

A) It reverts the Deployment's Pod template to that of a previous revision, and the Deployment controller then rolls the workload back accordingly B) It reverts the Deployment's `.spec.replicas` to its previous value, leaving the Pod template alone C) It reverts the entire set of objects that were applied at the same time as the Deployment D) It pauses the rollout so that no further changes take effect until resumed
5. Your team publishes a chart that installs `nginx`. You fix a typo in a template label — no change to which `nginx` is installed. Which field in `Chart.yaml` should move?

A) `appVersion`, because the chart's output changed B) The chart version, because the packaging changed C) Both, always, since they are required to stay in step D) Neither; template fixes are not versioned changes

Answers with Explanations

1. **B.** One chart, many releases, each upgraded and rolled back independently [source: [helm-charts-2026-08-23](#)]. **A is wrong** and is the single most common collapse in this material: it treats the chart as if it had installed state, when the chart is a package and only the release is installed. **C is wrong**, and it inverts the reason namespaces matter here: release names are scoped to their namespace in Helm 3 [source: [helm-changes-since-helm2-2026-08-31](#)], which is what makes two installations of one chart possible in

the first place — it has nothing to do with whether one release's rollback reaches the other. **D is wrong** on both halves: `helm list` reports the releases in your namespace context [source: helm-changes-since-helm2-2026-08-31], and `docs` is not out of date — it is installed at the chart version and values it was installed with, and nothing done to marketing changes that [source: helm-charts-2026-08-23].

2. B. `charts/` is a directory containing any charts upon which this chart depends [source: helm-charts-2026-08-23]; a chart repository is an HTTP server housing an `index.yaml` and packaged charts [source: helm-chart-repository-2026-08-31]. **A is wrong:** it is not a cache of anything; it is part of the chart's own content. **C is wrong:** `helm repo add` does not write into a chart's `charts/` directory. **D is wrong;** `charts/` is authored content and exists in the chart as shipped.

3. C. Every chart must have `Chart.yaml` [source: helm-glossary-2026-08-31], and a version number is required on every chart [source: helm-glossary-2026-08-31]. **A is wrong:** `crds/` is a special directory you can create [source: helm-crd-best-practices-2026-08-31], not one you must. **B is wrong:** `NOTES.txt` is explicitly optional [source: helm-charts-2026-08-23]. **D is wrong;** `_helpers.tpl` is a convention for partials [source: helm-named-templates-2026-08-31], useful but not required.

4. A. `kubectl rollout undo` undoes a previous rollout [source: k8s-docs-kubectl-rollout-2026-08-24], and it does so by reverting the Pod template; the controller then reconciles toward it in the ordinary way [cross-bearing: see Ch 6 §5 — every rollout is a revision]. **B is wrong:** rolling back a Deployment restores a previous *template*, not a previous replica count — scaling is a separate operation, `kubectl scale` [cross-bearing: see Ch 6 §2 — a loop you can watch working]. **C is wrong**, and this is the distractor worth dwelling on: the multi-object scope belongs to `helm rollback`, not to `kubectl rollout undo`, and mixing them up is exactly the confusion §3 exists to prevent. **D describes** `kubectl rollout pause`, a different subcommand entirely [source: k8s-docs-kubectl-rollout-2026-08-24].

5. B. The chart's own version tracks the packaging; `appVersion` tracks the application the chart installs, and they move independently — Helm states that `appVersion` "is not related to the version field" [source: helm-charts-2026-08-31]. **A inverts the two.** **C is wrong:** nothing requires them to move together, and requiring it would force meaningless application-version bumps for template fixes. **D is wrong:** charts are versioned according to SemVer 2 and a version is required [source: helm-glossary-2026-08-31], so any published change gets one.

If you scored 0–2: Re-read §3 before continuing. Not the whole chapter — §3. The chart/release/revision split is what §5 and §6 are built on, and if it has not landed yet, the Kustomize contrast will read as a second set of vocabulary rather than as a contrast.

Checkpoint: You've Now Mastered ✓ The four things a folder of manifests structurally cannot do ✓ What each entry in a chart directory is for ✓ Chart vs Helm release vs release revision — three words, three things ✓ Why `helm rollback` and `kubectl rollout undo` are not the same mechanism ✓ Where charts live, and why registries turned out to hold more than images

One more tool to meet, and it disagrees with Helm about almost everything except the goal.

..1 §5 — Patching Instead of Templating

Everything so far has come from one idea: describe the manifest with holes in it, and fill the holes.

There is another answer, and it starts by refusing that idea outright.

☆ Fixed Point

Kustomize introduces a template-free way to customize application configuration [source: kustomize-overview-2026-08-23]. It is a standalone tool for customizing Kubernetes objects through a kustomization file [source: k8s-docs-kustomization-2026-08-31], and **it is built into kubectl as `apply -k`** [source: kustomize-overview-2026-08-23]. Since Kubernetes 1.14, kubectl has supported management of objects using a kustomization file [source: k8s-docs-kustomization-2026-08-31].

Two things in that block do independent work and should be held separately.

Template-free. There are no placeholders. Each manifest in a Kustomize setup is an ordinary, valid, applicable Kubernetes object — `kubectl apply -f base/deployment.yaml` works with no rendering step, because there is nothing to render. (The `kustomization.yaml` sitting beside it is Kustomize's own object, and is the one file in that directory the API server will not take. Which is the same fact that made the CRD case in §1 sharp: the API server serves the kinds it knows about, and nothing else.)

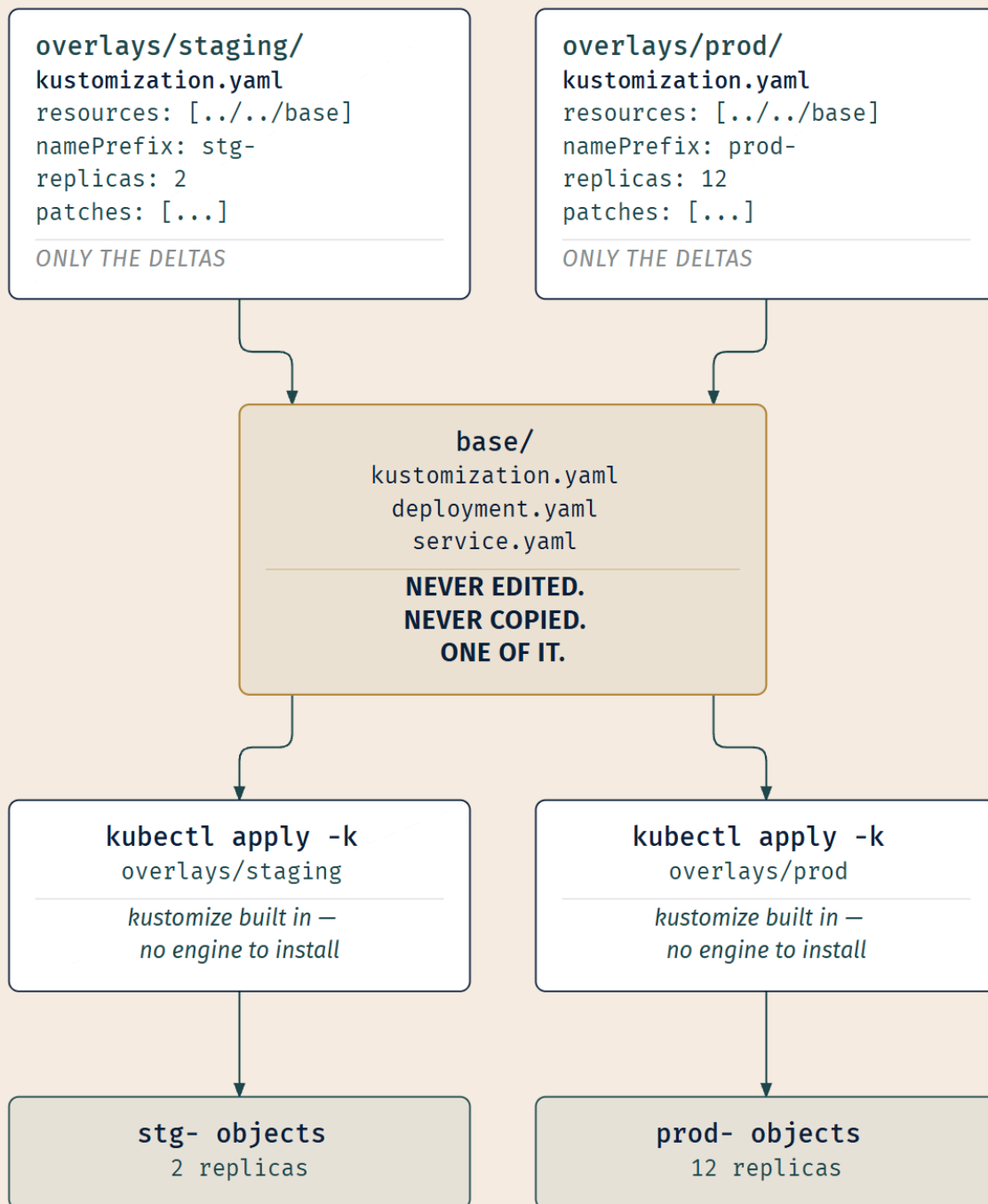
Built into kubectl. There is no engine to install, no client to distribute to your team, no version to keep in step. If a machine has kubectl, it has Kustomize. That practical fact shapes when the tool is the right answer.

Base and overlay

The organizing idea is a fork/modify/rebase workflow: a **base** directory holds the upstream manifests and a `kustomization.yaml`, and **overlays** (dev, staging, prod, say) reference the base and layer patches, name prefixes and suffixes, labels, images, and generated ConfigMaps and Secrets on top [source: kustomize-overview-2026-08-23]. This manages any number of distinctly customized configurations without forking the originals [source: kustomize-overview-2026-08-23].

That last clause is the claim, and it is the one to hold onto: **without forking the originals**. The base is not edited. The base is not copied. Each overlay declares only its own differences and points at the base it differs from.

A fixed reference, and a declared offset from it. Navigators have been doing arithmetic of that shape for a very long time; only the notation is new.



The kustomization file is itself a YAML specification of a Kubernetes Resource Model object called a *Kustomization*, and a kustomization describes how to generate or transform other objects [source: [kubectl-book-kustomization-fields-2026-08-31](#)]. That framing is worth noticing: the customization instructions are themselves an object of a declared kind, not a script.

What an overlay can declare

The kustomization file's field list is long; the useful ones for our purposes cluster into four groups [source: kubectl-book-kustomization-fields-2026-08-31].

What to include. `resources` names the resources to include: the base, plus any manifests belonging to this overlay alone.

Blanket transformations. `namespace` adds a namespace to all resources. `namePrefix` and `nameSuffix` prepend or append to the names of all resources *and references*. `labels` adds labels and optionally selectors to all resources, `commonAnnotations` adds annotations. `images` modifies the name, tags, and/or digest for images. `replicas` changes the replica count for a resource.

Two of these deserve a note. `namePrefix` updating *references* as well as names is why a prefixed Deployment still finds its prefixed ConfigMap: Kustomize understands the relationships, not just the strings. And the label transformer is the same machinery you learned in Chapter 4, labels applied across every object in the overlay, with selectors updated to match [*cross-bearing: see Ch 4 §5 — the universal join*].

Targeted patches. `patches` patches resources; `patchesStrategicMerge` patches using the strategic merge patch standard; `patchesJson6902` patches using the JSON 6902 standard [source: kubectl-book-kustomization-fields-2026-08-31].

Two patch styles, and the difference is one clause each. A **strategic merge patch** is a fragment of the object that looks like the object: you write the piece of the Deployment you want to change, in the same shape, and Kustomize merges it in. A **JSON patch** is a list of explicit operations on paths: replace this path, add to that array, remove this element. Strategic merge reads more naturally and handles most cases; JSON patch is what you reach for when you need surgical precision, particularly on list elements where merge semantics get ambiguous.

Either way, the patch targets an object by identity: its `apiVersion`, `kind`, and `metadata.name` [*cross-bearing: see Ch 4 §2 — the anatomy of a record*]. That is how Kustomize knows which base object a delta belongs to.

Generators. `configMapGenerator` generates ConfigMap resources, `secretGenerator` generates Secret resources, and `generatorOptions` controls their behavior [source: kubectl-book-kustomization-fields-2026-08-31].

These build ConfigMaps and Secrets from literal values or from files on disk, rather than requiring you to hand-author the object with its data map. Given that ConfigMaps and Secrets are precisely the objects that vary most between environments [*cross-bearing: see Ch 4 §4 — configuration kept outside the image*], having overlays generate them is more than a convenience: it puts the most environment-specific objects in the layer that is *about* environment specificity.

🔗 **Closer Look:** The field list also contains `helmCharts`, described as a Helm chart inflation generator [source: `kubectl-book-kustomization-fields-2026-08-31`]. Kustomize can take a Helm chart, render it, and treat the rendered output as resources to patch. The two tools are not mutually exclusive at the mechanical level, whatever the arguments online suggest; §6 returns to this. Well past what the exam requires, but a useful thing to know exists the first time you see it in somebody's repository.

The list also carries both `bases` and `resources`, which overlap — `bases` is described as "add resources from a kustomization dir" [source: `kubectl-book-kustomization-fields-2026-08-31`]. Kustomizations you meet in the wild will use either, and modern ones tend to list the base under `resources`.

The move that is actually different

Step back from the field list, because the field list is not the point and it is easy to read this section as "a second syntax for the same idea."

Helm's answer to environment variation is: *make the manifest incomplete, and complete it differently each time*. The artifact in your repository is not a valid Kubernetes object. It is a template that becomes one.

Kustomize's answer is: *keep the manifest complete and correct, and describe the differences separately*. The artifact in your repository is a valid Kubernetes object throughout. Nothing is ever rendered from a non-object into an object; the transformation is object-to-object.

That difference has real consequences. You can read a base directory without a rendering engine, because it is manifests. You can validate those manifests with any tool that validates manifests. A newcomer can open `base/deployment.yaml` and understand it without knowing anything about Kustomize at all. Against that: there is no `values.yaml`, no single file where the author declares "these, and only these, are the things you may change." An overlay can patch anything, which is more powerful and less guided.

Neither of these is a defect. They are the same trade-off seen from two sides, and §6 is about which side you want to be standing on.

..1 §6 — Which One, When

Two tools. Same problem. Opposite mechanisms. Which do you use?

The question is asked badly most of the time: as a preference poll, or a maturity ladder, or a tribal marker. It is none of those. It has an answer, and the answer turns on one thing.

The distinction that decides it

Are you distributing software to people who will not read it, or adapting software you already have for environments you control?

If you are **distributing**, you want Helm, and the reasons are precisely the four failures from §1. A stranger needs a *name* to ask for and a *version* to pin, and `Chart.yaml` supplies both, and requires them [source: helm-glossary-2026-08-31]. They need a place to fetch it from without cloning your repository, and a chart repository or an OCI registry supplies that [source: helm-chart-repository-2026-08-31] [source: helm-oci-registries-2026-08-31]. They need to know which parts are theirs to change without reading the manifests, and `values.yaml` supplies exactly that, as a declared surface. And they need to install, upgrade, and undo as single acts against a named thing, which the release model supplies [source: helm-architecture-2026-08-31].

Which is why, when you go looking for an operator, a controller, an Ingress controller, or a monitoring stack, what you tend to find is a chart. Not because templating is superior. Because the recipient is a stranger, and everything a stranger needs is packaging.

If you are **adapting**, you want Kustomize. There is nobody to distribute to: the manifests are yours, they live in your repository, and the people changing them can read them. In that setting a template engine is overhead, making your artifacts un-readable-as-manifests to buy you a distribution capability you are not using. Kustomize's proposition is that your manifests stay ordinary manifests and the differences live in small overlay files that say only what differs.

CRITERIA	HELM	KUSTOMIZE
What varies	values, filling holes in templates	patches, applied to a complete base
The unit	a versioned chart (version REQUIRED)	a directory. no version (nothing requires one)
Distribution	chart repository, or an OCI registry	none. it's your repo.
Lifecycle	releases and revisions; install/upgrade/rollback as single acts	none. apply is apply. no installed-state record of its own
Where the engine lives	a CLI you install	in kubectl. <code>`apply -k`</code> Nothing to install.

WHAT THE CHOICE ACTUALLY TURNS ON:

Distributing to strangers who won't read it



HELM

Adapting what you already have, for yourself



KUSTOMIZE

And the two combine. The commonest production shape is: take somebody's chart for the third-party components, use Kustomize for your own applications, and, if you need to adjust a chart's rendered output in a way its values do not expose, let Kustomize inflate the chart and patch the result [source: [kubectl-book-kustomization-fields-2026-08-31](#)]. This is not a compromise position. It is what "package" and "adapt" look like when both are happening in the same repository, which is most of the time.

Why charts have a `crds/` directory

Now the piece we deferred, and it closes the chapter's loop.

Chapter 6 promised you an answer to this *[cross-bearing: see Ch 6 §8 — the control loop, extended]*, and the answer is failure two from §1, ordering, returning at the end of the chapter as something you can now diagnose yourself.

Recall the rule, in the Helm project's own words: **"For a CRD, the declaration must be registered before any resources of that CRDs kind(s) can be used"** [source: helm-crd-best-practices-2026-08-31]. There is a declaration of a CRD, the YAML with kind `CustomResourceDefinition`, and then there are resources that use the CRD [source: helm-crd-best-practices-2026-08-31]. The declaration must come first, and this is not a preference. Until the CRD is registered, the custom kind does not exist as far as the API server is concerned, and an object of that kind is not "not ready yet." It is invalid.

Now apply the chart model to that. A chart's templates render to a stream of manifests. If the CRD were merely one more file in `templates/`, it would render alongside everything else and be applied alongside everything else, and whether it landed first would be a matter of luck. That is exactly failure two: no ordering guarantee, and one case where the absence of a guarantee is fatal rather than merely slow.

So Helm carved out a directory. `crds/` **is a special directory you can create in your chart to hold your CRDs; these CRDs are not templated, but will be installed by default when running a `helm install` for the chart** [source: helm-crd-best-practices-2026-08-31]. Not templated, so there is nothing to render and nothing to sequence within. Installed by default, so getting them in is the tool's responsibility rather than the README's.

The mechanism has documented limits, and they are the kind of thing that catches people in production:

- `--skip-crds` will skip the CRD installation step [source: helm-crd-best-practices-2026-08-31].
- **There is no support at this time for upgrading or deleting CRDs using Helm** [source: helm-crd-best-practices-2026-08-31].
- The `--dry-run` flag of `helm install` and `helm upgrade` is not currently supported for CRDs [source: helm-crd-best-practices-2026-08-31].

There is a second approach: put the CRD definition in one chart, and any resources that use that CRD in *another* chart, a workflow that may be more useful for cluster operators who have admin access to a cluster [source: helm-crd-best-practices-2026-08-31].

⚠ Navigational Hazards

The no-upgrade limitation is the one to internalize. A chart with a `crds/` directory installs its CRDs on first install and does not update them on subsequent upgrades [source: [helm-crd-best-practices-2026-08-31](#)]. If the chart's next version ships a CRD with new fields, upgrading the release will not bring you those fields. Operators shipped as charts run into this constantly, and the symptom is confusing: the chart upgraded successfully and the new feature does not work, because the API server is still serving the old schema.

The restriction is defensible on its face. A CRD's removal or downgrade is not a local change — it reaches every object of that kind in the cluster. Helm declining to automate that is a decision, not an oversight.

Chapter 17 collects the pluggable interfaces this book has been accumulating, and CRDs shipped as chart content is one of the places where the extension mechanism and the packaging mechanism visibly interact [cross-bearing: see *Ch 17 §4 — the four pluggable interfaces, collected*].

The four, closed

Look back at §1 and check the arithmetic:

Failure	Helm's answer	Kustomize's answer
Environment variation	<code>values.yaml</code> — a declared surface of what may change	Overlays — declared deltas against an unmodified base
Apply ordering	<code>crds/</code> for the one case that is fatal	— (<code>apply -k</code> is still an <code>apply</code> ; same eventual consistency, same CRD problem)
Versioning	Chart version, required on every chart; <code>helm list</code> answers what is installed	— (a directory still has no version; Git supplies one, which is Ch 15's subject)
Distribution	Chart repositories and OCI registries	— (nothing to distribute; that is the premise)

Note the shape of that column. Kustomize answers *one* of the four cleanly, declines two of them by design, and shares the fourth's limitation. That is not a deficiency; it is a smaller tool solving the subset of the problem that occurs when there is nobody to distribute to. Working out which column your situation is in is the entire skill — ten seconds with the chart in front of you, once you know which two marks to sight on.

🕒 Taking Your Bearings: Patching, and Choosing

Five questions on §5 and §6.

1. What does Kustomize do to the base directory when you apply an overlay?

A) Copies it into the overlay directory, then patches the copy B) Edits the base files in place to reflect the overlay's values C) Nothing — the base is neither modified nor duplicated; the overlay declares deltas against it D) Renders the base through a template engine using the overlay as values

2. A colleague says they cannot use Kustomize because they are not allowed to install additional tooling on the build agents. What is the correct response?

A) They are right; Kustomize requires a standalone binary B) Kustomize is built into `kubectl` as `apply -k`, so any machine with `kubectl` already has it C) They can use Helm instead, which has no separate binary D) Kustomize runs as a controller in the cluster, so nothing is installed on the agents

3. A chart ships a CRD in its `crds/` directory. You upgrade the release to a chart version whose CRD adds a new field. What happens?

A) The CRD is updated as part of the upgrade, and the new field becomes available B) The upgrade fails, because Helm detects the CRD change and refuses C) The CRD is not upgraded — Helm has no support for upgrading CRDs — so the new field is not available D) The CRD is deleted and recreated, taking existing custom resources with it

4. You are publishing a monitoring stack for other teams to install on their own clusters. They should not need to read your manifests. Which approach fits, and why?

A) Kustomize, because a base plus overlays is simpler to understand B) Helm, because a chart supplies a name, a required version, a fetchable location, and a declared set of values C) Either — the choice is purely stylistic D) Neither; cross-team distribution requires a GitOps agent

5. [retrieval: ch13] A cluster has no metrics-server and `kubectl top nodes` fails. Now that you have vocabulary for it, what does "install metrics-server" consist of?

A) Enabling a built-in Kubernetes feature gate on the API server B) Applying a published file of Kubernetes objects that somebody else wrote, with `kubectl apply -f` C) Installing a binary on each node, since metrics collection is a node-level concern D) Nothing; `kubectl top` works on any conformant cluster and the failure indicates a different problem

Answers with Explanations

1. **C.** Overlays reference the base and layer patches on top, managing customized configurations *without forking the originals* [source: kustomize-overview-2026-08-23]. **A is wrong** and is the most common wrong model: if it copied, you would be back at failure one from §1, with two directories drifting apart. **B is wrong**: an overlay that edited its base would make the base environment-specific, defeating the arrangement entirely. **D is wrong**, and it is the trap this whole section exists to disarm. Kustomize is explicitly a *template-free* way to customize configuration [source: kustomize-overview-2026-08-23]. There is no rendering step and no template engine.

2. B. Kustomize is built into kubectl as `apply -k` [source: kustomize-overview-2026-08-23], and kubectl has supported kustomization files since 1.14 [source: k8s-docs-kustomization-2026-08-31]. **A is half-true and wrong where it counts:** a standalone Kustomize binary does exist [source: k8s-docs-kustomization-2026-08-31], but it is not required for the kubectl-integrated path. **C inverts reality:** Helm is the one that needs a separate client. **D is wrong;** Kustomize is client-side, and nothing about it runs in the cluster.

3. C. There is no support at this time for upgrading or deleting CRDs using Helm [source: helm-crd-best-practices-2026-08-31]. **A is what most people expect and is the reason this trips teams up in production:** the release upgrades cleanly and the new capability silently is not there. **B is wrong:** the upgrade succeeds; it just skips the CRD. **D is wrong,** and it describes the outcome the restriction exists to prevent.

4. B. Distribution to people who will not read your manifests is exactly what packaging is for: a required version [source: helm-glossary-2026-08-31], a fetchable location [source: helm-chart-repository-2026-08-31], and a declared surface of what they may change. **A is wrong:** simplicity is not the axis, and a Kustomize base gives a stranger no version to pin and no repository to fetch from. **C is wrong;** the choice turns on distribute-vs-adapt and has a defensible answer here. **D is wrong:** a delivery agent addresses *who applies it and when*, which is a different question and Chapter 15's [cross-bearing: see Ch 15 §3 — push, or pull].

5. B. In a declarative system there is no installer; installation is applying objects somebody wrote — the project publishes a `components.yaml` and tells you to `kubectl apply -f it` [source: metrics-server-install-2026-08-31] [cross-bearing: see Ch 13 §7 — metrics-server and the resource metrics pipeline]. What this chapter added is the word for the packaged form of "objects somebody wrote" — in practice you will most often meet metrics-server as a chart, which is failure four answered. **A is wrong:** metrics-server is a separate component, not a feature gate, which is why `kubectl top` fails on a bare cluster in the first place. **C is wrong:** metrics-server is a Deployment that reads from the kubelets, not a per-node binary you install. **D is wrong,** and it is the misconception Chapter 13 named. `kubectl top` failing on a cluster without metrics-server is expected behavior, not a fault.

If you scored 0–2: Re-read §5, then the decision table in §6. The most common cause of a low score here is reading Kustomize as "Helm with different syntax," which makes every comparison question a coin flip.

Checkpoint: You've Now Mastered ✓ What an overlay does, and what it conspicuously does not do to its base ✓ Why Kustomize needs no engine installed ✓ What the Helm-vs-Kustomize choice actually turns on ✓ Why `crds/` exists, and the three limits on how it works

☀ §7 — A Package, Not a Template

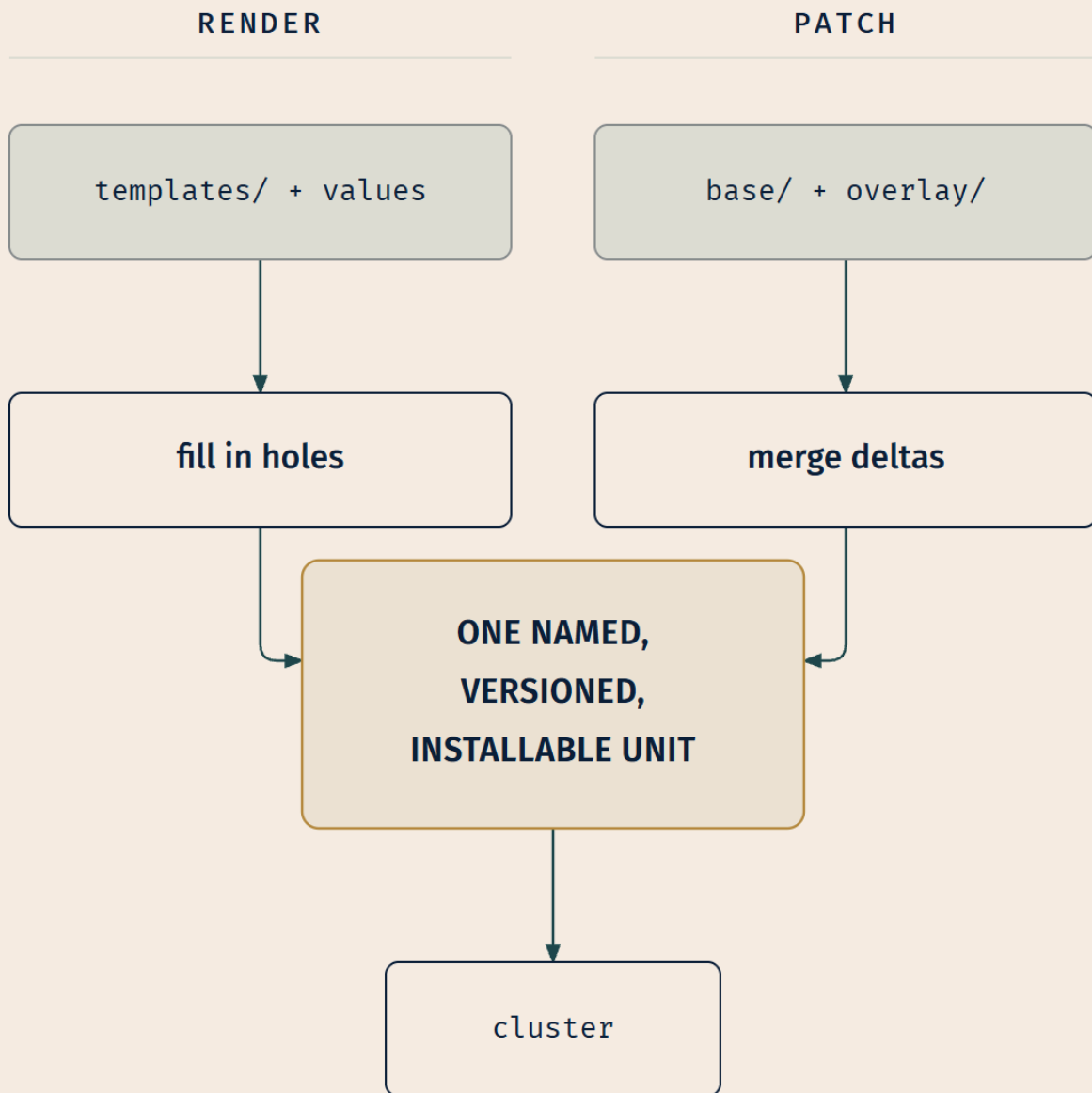
Here is the thing that has been true for the whole chapter, and is easiest to see now that both tools are in view.

You have been taking the sight from too low.

Helm and Kustomize disagree about mechanism as completely as two tools can. One makes the manifest incomplete and completes it from values. The other keeps the manifest complete and describes differences against it. One has a template engine; the other advertises the absence of a template engine as its headline feature [source: kustomize-overview-2026-08-23]. If mechanism were the axis, they would be opposites.

They agree about everything that matters. Both exist to take a directory of loose files and turn it into **one addressable unit**: a thing with a name, a thing that installs as a single act, a thing whose differences across environments are declared in one place rather than scattered through copies. They arrived at that goal from opposite directions and neither of them was aiming at templating. Templating is a thing Helm happens to use on the way.

Which means the argument the ecosystem spends so much energy on — templating versus not-templating, engine versus no-engine, Go templates versus patches — is an argument about *how*, conducted between two tools that had already agreed on *what*. And the *what* is the part that changes how you work.



The mechanisms could not be more different.

The destination is the same one.

The destination is the point.

That is what the subtitle meant. *A chart is not a release*: the package and the installation are different things, and collapsing them costs you the ability to say what is running where. *Templates are not the point*: they are one way of absorbing variation, and a tool exists that absorbs variation without them and solves the same problem anyway.

There is a nice symmetry in it with something you already know. Chapter 4 taught that a Kubernetes object is a *record of intent*, a durable declaration of what should be true, which controllers then work to make true [*cross-bearing: see Ch 4 §6 — a declaration, not an order*]. A package is that same move, one level up: a durable declaration of what a whole application should be, with a version number on it so you can say *which* declaration, and a name so you can ask what happened to it. Not a bigger object. The same idea applied to the set.

And now the question you cannot avoid, which this chapter has no way to answer.

You have a unit. It has a name, a version, and a place to be fetched from. It installs as one act and can be undone as one act.

Who runs the install? When? Triggered by what? From where?

Because so far the answer is: a person, at a keyboard, running a command. That person has cluster credentials on their laptop. They remember to do it, or they do not. They apply the version they think is current. Nothing checks whether the cluster still matches what was installed, and nothing notices when somebody else changes it by hand.

Packaging solved *what you hand over*. It solved nothing at all about *how it gets applied and stays applied*. That gap is where Chapter 15 lives, and it turns out the answer reaches back into the oldest idea in this book [*cross-bearing: see Ch 15 §3 — push, or pull*].

Exam Alert! 🚨

High-priority *for this chapter's material*, on the reasoning given in Why This Chapter Matters — nobody outside the exam authority publishes a priority order beneath this competency, and nothing below is a frequency claim.

High-Priority Topics

1. **Chart, Helm release, release revision — three words, three things.** A chart is the package. A release is one installation of it [source: helm-architecture-2026-08-31]. A revision is one numbered state of that release [source: helm-glossary-2026-08-31]. One chart, many releases; one release, many revisions.
2. **charts/ is not a chart repository.** charts/ is a directory inside a chart holding its dependencies [source: helm-charts-2026-08-23]. A chart repository is an HTTP server housing packaged charts [source: helm-chart-repository-2026-08-31].

3. Helm is a package manager, not a template engine [source: helm-homepage-2026-08-31].

Templating is one mechanism inside it. Kustomize solves the same problem with no templating at all [source: kustomize-overview-2026-08-23], which is the cleanest available proof that templating was never the definition.

4. Kustomize is built into kubectl as `apply -k` [source: kustomize-overview-2026-08-23], and is template-free. Nothing to install; nothing rendered.

Common Traps

The trap	The correct understanding
"Helm is a templating engine"	It is a package manager [source: helm-homepage-2026-08-31]. Chart → values → templates → Helm release . Templating is a means; the unit is the point.
Using "chart" and "release" interchangeably	One chart installs many times, each creating a separately named release, upgradable and rolled back independently [source: helm-charts-2026-08-23].
Reading charts/ as a chart repository	charts/ holds dependency charts <i>inside</i> a chart [source: helm-charts-2026-08-23]. A repository is an HTTP server managed with <code>helm repo</code> [source: helm-chart-repository-2026-08-31].
Assuming helm rollback runs kubectl rollout undo underneath	Different unit, different scope. <code>helm rollback</code> takes a release name [source: helm-rollback-cli-2026-08-31]; <code>kubectl rollout undo</code> takes one workload [source: k8s-docs-kubectl-rollout-2026-08-24].
Reading chart version as the version of the software	The chart has its own SemVer version, required on every chart [source: helm-glossary-2026-08-31]. <code>appVersion</code> is the application's. They move independently.
Assuming Kustomize needs an engine installed	It is in kubectl [source: kustomize-overview-2026-08-23]. <code>kubectl apply -k</code> works on a stock installation.
Assuming an overlay edits or copies its base	It does neither — customization happens <i>without forking the originals</i> [source: kustomize-overview-2026-08-23].
Expecting a chart upgrade to upgrade its CRDs	There is no support for upgrading or deleting CRDs with Helm [source: helm-crd-best-practices-2026-08-31]. The release upgrades; the CRD does not.
"You have to run Tiller"	Helm 3 removed Tiller entirely — "one of the first decisions we made regarding Helm 3 was to completely remove Tiller" [source: helm-changes-since-helm2-2026-08-31]. Material that describes installing or securing Tiller is describing a Helm that no longer exists.

That last row deserves a sentence of its own, because it connects to something Chapter 1 warned you about. Tiller was Helm 2's in-cluster component, introduced so that multiple people could interact with the same set of releases; with RBAC enabled by default from Kubernetes 1.6, locking it down for production became difficult to manage, and the permissive default configuration could grant users a far broader range of permissions than intended [source: helm-changes-since-helm2-2026-08-31]. Helm 3

removed it, storing release records in Kubernetes directly and evaluating permissions through your kubeconfig instead [source: helm-changes-since-helm2-2026-08-31].

Treat Tiller as a dating stamp. This domain carries 16% of the current blueprint [source: lf-kcna-exam-page-2026-08-23], and a study resource that explains how to secure Tiller is describing a Helm that Helm 3 removed [source: helm-changes-since-helm2-2026-08-31] — which is reason enough to read carefully whatever else it tells you about this domain.

Practice Questions

1. [interleaved: D1.1] You run `kubectl apply -f manifests/` where the directory contains a CustomResourceDefinition and a custom resource that uses it. The custom resource is rejected. Why?

A) `kubectl apply` applies files alphabetically, and the resource sorted before the definition B) The CRD declaration must be registered before any resources of its kind can be used, and applying a directory guarantees no ordering C) Custom resources cannot be applied with `kubectl apply`; they require a controller D) The CRD needs to be in a `crds/` directory for `kubectl` to recognize it

2. Which statement about `values.yaml` is correct?

A) It contains the rendered manifests that will be applied to the cluster B) It holds the default configuration values for the chart, and defines what an installer may change C) It is generated by Helm at install time from the `--set` flags D) It is the file an installer edits in place before running `helm install`

3. Which entry in a chart directory does **not** contribute Kubernetes objects to your cluster?

A) `templates/` B) `charts/` C) `values.yaml` D) `crds/`

4. In `templates/`, a file named `_helpers.tpl` is present. What is it for?

A) It contains helper Kubernetes objects created before the main templates B) Files beginning with an underscore are not rendered to object definitions but are available for use within other templates C) It stores the default values that `values.yaml` inherits from D) It is rendered last, after all other templates, so its output can reference them

5. You run `helm install app ./chart -f base-values.yaml -f prod-values.yaml --set replicaCount=12`. All three sources set `replicaCount`. Which value takes effect?

A) The one in `base-values.yaml`, because the leftmost `--values` file is authoritative B) The one in `prod-values.yaml`, because a file always beats a command-line flag C) 12, because `--set` values are merged into `--values` with higher precedence D) The install fails, because `--values` may be given only once

6. In Helm 3, where is a release's state stored by default?

A) In a Tiller Pod in kube-system B) In Secrets in the namespace of the release C) In a local database on the machine running the Helm client D) In etcd, under a Helm-specific prefix, bypassing the API server

7. `helm rollback my-app` is run with no revision argument. What happens?

A) The command errors, because a revision number is required B) It rolls back to revision 1, the original installation C) It rolls back to the previous release D) It rolls back to the most recent revision created by `helm install` rather than by `helm upgrade`

8. Which is the clearest statement of how `helm rollback` differs from `kubectl rollout undo`?

A) `helm rollback` is faster because it does not perform a rolling update B) `helm rollback` acts on a release — everything the chart installed — while `kubectl rollout undo` acts on a single workload object C) They are equivalent; `helm rollback` is a wrapper around `kubectl rollout undo` D) `kubectl rollout undo` acts on the whole namespace, while `helm rollback` acts on one Deployment

9. [interleaved: D1.1] A team has a Deployment, a Service, a ConfigMap, and an Ingress, all installed together as one Helm release. A bad upgrade changed the Deployment's image and the ConfigMap's contents. Which single action returns all four objects to their prior state?

A) `kubectl rollout undo` on the Deployment B) `helm rollback` on the release C) `kubectl apply -f` with the old manifests D) `kubectl rollout undo` on the Deployment plus a manual ConfigMap edit

10. `helm install` is run with no release name and no flags. What happens in Helm 3?

A) A name is auto-generated from the chart name B) Helm throws an error, because a name is required C) The chart name is used as the release name D) The release is created in the default namespace with a random suffix

11. What is a chart repository?

A) A Git repository containing chart source files B) An HTTP server housing an `index.yaml` file and optionally some packaged charts C) A directory inside a chart holding its dependencies D) A cluster-side component that serves charts to `helm install`

12. [interleaved: D1.4] Why can an OCI registry store Helm charts?

A) Helm charts are internally structured as container images B) The OCI Distribution Specification defines an API for distributing *content*, not specifically images, so registries can hold other artifacts C) Helm converts charts to images at push time and back at pull time D) It cannot; charts require a dedicated chart repository

13. An overlay's `kustomization.yaml` declares a `configMapGenerator`. What does that field do, and why does it belong in an overlay rather than in the base?

A) It generates ConfigMap resources; ConfigMaps are among the objects that vary most between environments, which is what an overlay is for B) It copies the base's ConfigMaps into the overlay so they

can be edited without affecting the base C) It reads ConfigMaps that already exist in the cluster and merges them into the overlay's output D) It converts a chart's `values.yaml` into a ConfigMap so that Helm charts can be used as Kustomize bases

14. What does the `namePrefix` field in a `kustomization.yaml` do?

A) Prepends a value to the names of all resources and references B) Renames only the kustomization file's own metadata C) Filters which resources from the base are included D) Prepends a value to container image names

15. Which pair correctly describes Kustomize's two patch styles?

A) Strategic merge patch writes a fragment shaped like the object; JSON patch lists explicit operations on paths B) A strategic merge patch must restate the whole object; a JSON patch names only the changed fields C) Strategic merge patch is server-side; JSON patch is client-side D) Strategic merge patch requires a template engine; JSON patch does not

16. A team keeps their application manifests in their own Git repository, deploys to three clusters they administer, and has no external consumers. Which approach fits best, and why?

A) Helm, because versioning is always required B) Kustomize, because there is nothing to distribute and the manifests stay readable as ordinary manifests C) Helm, because three environments exceed what overlays can express D) Kustomize, because overlays remove the need to keep the base under version control

17. You push a Helm chart to an OCI registry. Which statement about the reference is correct?

A) The reference must be prefixed with `oci://` and must not contain the basename or tag on push B) The reference must include the chart's digest, since tags are not supported for charts C) The reference is identical in form to a Helm repository URL D) Charts pushed to OCI registries lose their version, which must be supplied at install time

Answers with Explanations

1. B. The declaration must be registered before any resources of that CRD's kinds can be used [source: helm-crd-best-practices-2026-08-31], and applying a directory makes no ordering promise. Failure two from §1, in its sharpest form. **A is wrong:** even if the ordering happened to be favorable, that would be luck, not a guarantee, and alphabetical ordering is not something `kubectl apply` undertakes. **C is wrong;** custom resources are applied exactly like any other object once their kind exists [*cross-bearing: see Ch 6 §8 — the control loop, extended*]. **D is wrong:** `crds/` is a Helm chart convention [source: helm-crd-best-practices-2026-08-31], meaningless to bare `kubectl`.

2. B. `values.yaml` holds the default configuration values for the chart [source: helm-charts-2026-08-23], which is also the declared surface of what an installer may change. **A confuses values with rendered**

output; the rendering happens from `templates/` combined with values [source: helm-charts-2026-08-23]. **C inverts the direction**: `--set` overrides `values.yaml`, merged with higher precedence [source: helm-using-helm-2026-08-31], not the reverse. **D is the beginner's move and is wrong for the reason that matters most in this chapter**: overrides are supplied with `--values/-f` or `--set` [source: helm-using-helm-2026-08-31], which leaves the chart itself unmodified — and a chart you have edited is no longer the versioned artifact you fetched, which is the whole property packaging exists to give you.

3. C. `values.yaml` holds defaults [source: helm-charts-2026-08-23]; it is input to rendering, and nothing in it becomes an object. **A is wrong**: `templates/` combined with values generates valid manifests [source: helm-charts-2026-08-23], which is precisely how objects get created. **B is wrong**, and it is the interesting distractor: `charts/` holds charts the chart depends on [source: helm-charts-2026-08-23], and a chart is a source of manifests like any other, so objects do come from there. **D is wrong**: CRDs in `crds/` are installed by default when running `helm install` [source: helm-crd-best-practices-2026-08-31] — not templated, but very much created.

4. B. Files whose names begin with an underscore are assumed not to have a manifest inside; they are not rendered to Kubernetes object definitions but are available everywhere within other chart templates [source: helm-named-templates-2026-08-31], and `_helpers.tpl` is the default location for template partials [source: helm-named-templates-2026-08-31]. **A is wrong**: nothing in `_helpers.tpl` becomes an object at all, let alone an early one. **C inverts the relationship**; values come from `values.yaml` and the command line [source: helm-using-helm-2026-08-31]. **D is wrong for the same reason as A, from the other side**: there is no render order to place it in, because underscore-prefixed files are not rendered to object definitions in the first place [source: helm-named-templates-2026-08-31].

5. C. Where both are used, `--set` values are merged into `--values` with higher precedence [source: helm-using-helm-2026-08-31]. **A inverts the file rule**: `--values` can be specified multiple times and the *rightmost* file takes precedence [source: helm-using-helm-2026-08-31], so `prod-values.yaml` beats `base-values.yaml` — and then `--set` beats both. **B inverts the flag rule**, and gets the intuition exactly backwards: the command line is the most specific thing you said, so it wins. **D is wrong**: `--values` is explicitly repeatable [source: helm-using-helm-2026-08-31], which is what makes a base-plus-environment file arrangement possible.

6. B. In Helm 3, Secrets are the default storage driver [source: helm-changes-since-helm2-2026-08-31], and release information is stored in Secrets in the namespace of the release [source: helm-storage-backends-2026-08-31]. **A describes Helm 2**; Tiller was removed completely in Helm 3 [source: helm-changes-since-helm2-2026-08-31] and this is the single best marker of outdated material. **C is wrong**: state lives in the cluster, which is why any Helm client with the right kubeconfig sees the same releases. **D is wrong**; nothing writes to etcd directly, and the API server is the only door in [*cross-bearing: see Ch 3 §5 — the only door in*].

7. C. If the revision argument is omitted or set to 0, it rolls back to the previous release [source: helm-rollback-cli-2026-08-31]. **A is wrong**; the argument is optional, with a documented default. **B is wrong**: revision 1 is a specific revision you would have to name. **D is wrong**, and it invents a distinction the command does not draw: "previous release" means the immediately preceding revision, whatever

produced it — a sequential counter tracks releases as they change [source: helm-glossary-2026-08-31], without sorting them by which command incremented it.

8. B. `helm rollback`'s first argument is a release name [source: helm-rollback-cli-2026-08-31]; `kubectl rollout undo` manages the rollout of a Deployment, DaemonSet, or StatefulSet [source: k8s-docs-kubectl-rollout-2026-08-24]. Different unit, different scope. **A is wrong on the mechanism:** a Helm rollback that changes a Pod template produces a rolling update as an ordinary consequence of reconciliation. **C is the trap this section exists to prevent;** neither calls the other — Helm renders charts client-side and stores a record of the installation in Kubernetes [source: helm-changes-since-helm2-2026-08-31], with no `kubectl` in the path. **D inverts both scopes.**

9. B. The release is the unit that contains all four objects, and rolling it back returns them together [source: helm-rollback-cli-2026-08-31]. **A is wrong** and is the single-object trap: it fixes the Deployment and leaves the ConfigMap wrong. **C might work** if you still have the old manifests and apply them correctly, but it is not a single action and it does not update Helm's release record, leaving the cluster and Helm's bookkeeping disagreeing. **D is A's problem plus manual work**, which is the state Helm exists to eliminate.

10. B. Helm 3 throws an error if no name is provided with `helm install` [source: helm-changes-since-helm2-2026-08-31]. **A describes Helm 2**, which auto-generated names — behavior that proved to be more of a nuisance than a helpful feature in production [source: helm-changes-since-helm2-2026-08-31]. You can still opt into it with `--generate-name` [source: helm-changes-since-helm2-2026-08-31]. **C is wrong, though it describes what most chart READMEs look like:** the conventional `helm install wordpress bitnami/wordpress` passes the chart name *explicitly* as the release name, which is easy to misread as inference. Helm does not infer it. **D is wrong on both halves:** no name is generated without `--generate-name` [source: helm-changes-since-helm2-2026-08-31], and which namespace you land in is a matter of your `kubeconfig` context [source: helm-changes-since-helm2-2026-08-31], not something `helm install` invents.

11. B. A chart repository is an HTTP server that houses an `index.yaml` file and optionally some packaged charts [source: helm-chart-repository-2026-08-31]. **A is wrong:** a repository serves packaged charts over HTTP, not source files over Git, though Git is where chart source often lives. **C describes charts/** [source: helm-charts-2026-08-23], the confusion this chapter names twice. **D is wrong;** nothing cluster-side is involved. Any HTTP server that serves YAML and tar files and answers GET requests will do [source: helm-chart-repository-2026-08-31].

12. B. The OCI Distribution Specification defines an API protocol to facilitate and standardize the distribution of *content* [source: oci-distribution-spec-2026-08-24], and since OCI artifacts make it possible to store more than container images, a single registry can hold charts, images, and other artifacts [source: helm-blog-oci-ga-2026-08-31]. **A is wrong:** a chart is a tarred and gzipped chart directory [source: helm-glossary-2026-08-31], not an image. **C is wrong;** no conversion happens, and the registry stores the artifact as an artifact. **D is wrong:** Helm has supported this since 3.8.0 [source: helm-blog-oci-ga-2026-08-31] and now recommends it [source: helm-oci-registries-2026-08-31].

13. A. `configMapGenerator` generates `ConfigMap` resources [source: kubectrl-book-kustomization-fields-2026-08-31], and `ConfigMaps` and `Secrets` are exactly the objects that differ most between environments [*cross-bearing: see Ch 4 §4 — configuration kept outside the image*] — which is why generating them in the overlay puts the most environment-specific objects in the layer that is *about* environment specificity. **B is wrong** and reproduces the wrong model of Kustomize generally: nothing copies the base, because customization happens without forking the originals [source: kustomize-overview-2026-08-23]; a generator builds new objects rather than duplicating existing ones. **C is wrong:** Kustomize transforms files, not cluster state — it has no installed-state record of its own, which is one of the entries in §6's comparison table. **D describes something adjacent to `helmCharts`, the Helm chart inflation generator** [source: kubectrl-book-kustomization-fields-2026-08-31], and inverts it besides: that field consumes a chart, while `configMapGenerator` produces `ConfigMaps`.

14. A. `namePrefix` prepends the value to the names of all resources *and references* [source: kubectrl-book-kustomization-fields-2026-08-31]. That "and references" clause is the load-bearing part: a prefixed `Deployment` still finds its prefixed `ConfigMap`. **B is wrong;** the transformation applies to the resources, not to the kustomization's own metadata. **C describes resources.** **D describes images**, which modifies name, tags, and/or digest for images [source: kubectrl-book-kustomization-fields-2026-08-31].

15. A. `patchesStrategicMerge` patches using the strategic merge patch standard and `patchesJson6902` using the JSON 6902 standard [source: kubectrl-book-kustomization-fields-2026-08-31]; a strategic merge patch is a fragment shaped like the object, and a JSON patch is a list of operations on paths. **B inverts the relationship exactly**, and it is the plausible error: the merge patch is the *fragment* form — you write only the piece you are changing — while the operation-list form is the one that names paths explicitly. **C is wrong;** the two are not a client/server split, and both describe a change to a manifest before it is applied. **D is wrong**, and it contradicts the tool's headline property: Kustomize is template-free [source: kustomize-overview-2026-08-23].

16. B. Nothing is being distributed, the manifests are the team's own, and a Kustomize base stays readable as ordinary manifests with overlays declaring only deltas [source: kustomize-overview-2026-08-23]. **A is wrong;** a required version is valuable *for a stranger who must pin something*, and Git supplies identity for a team's own repository. **C is wrong;** overlays are explicitly designed for managing any number of distinctly customized configurations [source: kustomize-overview-2026-08-23]. **D is wrong and inverts the arrangement:** an overlay declares deltas *against* a base [source: kustomize-overview-2026-08-23], so the base is the thing that most needs a version — which is exactly the job Git does here, and Chapter 15's subject [*cross-bearing: see Ch 15 §3 — push, or pull*].

17. A. When using `helm push`, the reference must be prefixed with `oci://` and must not contain the basename or tag [source: helm-oci-registries-2026-08-31]. **B is wrong:** the tag is inferred from the chart's semantic version [source: helm-oci-registries-2026-08-31], so tags very much apply. **C is wrong;** the `oci://` prefix is exactly what distinguishes the two forms. **D is wrong:** the version is what the tag is derived from [source: helm-oci-registries-2026-08-31], so it travels with the chart rather than being supplied later.

Chapter Summary

Concept	Remember This
The four failures	A folder of manifests cannot handle environment variation, cannot guarantee apply ordering, has no version, and cannot be handed to a stranger. Everything in this chapter answers one of these four.
Chart	A collection of files describing a related set of Kubernetes resources, laid out in a directory tree, packageable into versioned archives. The unit.
<code>Chart.yaml</code>	The chart's identity. Required on every chart, and a version number is required in it.
<code>values.yaml</code>	The declared surface of variation — what an installer may change, with a working default for each. Overrides come from <code>-f</code> (rightmost wins) and <code>--set</code> (beats every file).
<code>templates/</code>	Combined with values, generates valid Kubernetes manifests. Files starting with <code>_</code> are partials, not objects.
<code>charts/</code>	Dependency charts <i>inside</i> this chart. Not a repository.
<code>crds/</code>	Not templated, installed by default. Solves the one ordering failure that is fatal rather than slow. Cannot be upgraded by Helm.
Helm release	One installed instance of a chart, named, scoped to a namespace. One chart installs many times; each release upgrades and rolls back independently.
Release revision	One numbered state of a release. <code>helm rollback <release> <revision></code> ; omit the revision to go back one.
<code>helm list</code>	Answers "what is installed here" — the releases in your namespace context, or <code>--all-namespaces</code> for the cluster. A directory has no equivalent.
<code>helm rollback</code> vs <code>kubectl rollout undo</code>	Different unit (release vs one workload), different scope (everything the chart installed vs one Pod template), different bookkeeping. Neither calls the other.
Chart repository	An HTTP server housing <code>index.yaml</code> and packaged charts. Managed with <code>helm repo</code> .
OCI registry as chart store	The distribution spec standardized <i>content</i> , not images. Charts, images, and other artifacts in one registry.
version vs appVersion	The box vs what's in the box. They move independently.
Kustomize	Template-free customization, built into <code>kubectl</code> as <code>apply -k</code> . Nothing to install.
Base and overlay	The base is never edited and never copied. Overlays declare only their deltas and point at the base.
The decision	Distributing to strangers who will not read it → Helm. Adapting what you already have → Kustomize.
Tiller	Helm 2's in-cluster component, removed entirely in Helm 3. Material that explains securing it is describing a Helm that no longer exists.

Concept	Remember This
☼ The Zenith	Two tools that disagree completely about mechanism and agree completely about goal: turn a directory into one named, versioned, installable unit. The unit is the point; the templating argument is about <i>how</i> .

⚓ Safe Harbor

Part IV opened with a debt thirteen chapters deep, and you have now collected on it. When a chapter told you to install a CNI plugin, an Ingress controller, a CSI driver, metrics-server, you now know exactly what that sentence was hiding, and you have the vocabulary to say what it should have said instead.

More than that: you can now read a stranger's `deploy/` directory and take its depth. Which of the four failures is this directory living with? Is anything marked as variable? Is there a version? Could you hand this to a colleague on another team without a phone call? Those are not academic questions. They are the questions that decide whether an application is shippable, and until this chapter you did not have a way to ask them.

The Voyage Ahead

You have a unit now. The hard part, turning a pile of files into one named, versioned, installable thing, is behind you.

And it has bought you less than it should have, because the unit still gets applied by a person. Somebody with cluster credentials on their laptop, running a command, from a machine nobody audits, at a moment nobody records. They apply the version they believe is current. Afterward, nothing keeps watch. If someone edits an object by hand next Tuesday, the cluster and the package quietly stop agreeing, and no one finds out until the next deploy overwrites the fix or the fix overwrites the deploy.

The next chapter is about closing that gap, and about a question that sounds procedural and is not: **should the pipeline push changes into the cluster, or should something inside the cluster pull them?** The answer reshapes where your credentials live, how large a mistake can get before something catches it, and what "the truth" even means about a running system.

It also brings you back, one last time, to the oldest idea in this book, the one from Chapter 3 that has been quietly underneath everything since. You have seen the control loop reconcile a Deployment toward its spec, a scheduler toward a placement, a claim toward a volume. Chapter 15 points it at a Git repository, and the recognition when that lands is the one this book has been saving.

"The chart is what you meant. The cluster is what happened. The interesting question is who keeps them the same."

Chapter 15: The Chart Is the Truth

"GitOps is the control loop you already learned, pointed at a repository"

Domain Weight: 16% (Cloud Native Application Delivery) [source: cncf-kcna-curriculum-pdf-2026-08-23] | Complexity: Mixed | Novelty: Paradigm-shifting

This domain **doubled** in the 2025-11-24 revision, from 8% to 16% [source: lf-kcna-program-changes-2026-08-23] — the largest proportional change on the blueprint, and the reason a great deal of third-party prep under-serves it. CNCF publishes two competencies under this domain — Application Delivery and Debugging — and no topic list beneath either [source: cncf-kcna-curriculum-pdf-2026-08-23]. The allocation of that 16% across Chapters 14, 15, and 16 is this book's authored judgment, not a published split.

Attention Budget

Total time: ~100 minutes | Recommended: Split across 2 sessions

Section	Time	Attention Cost	Best Time to Study
📍 Soundings	8 min	Medium	Before you start
\$1 Twelve Factors, and the Ones Kubernetes Already Solved	10 min	Low	Anytime
\$2 Ways to Replace What's Running	12 min	Medium	Mid-session
🕒 Taking Your Bearings 1	6 min	Medium	After a short break
\$3 Push, or Pull	15 min	High	Peak attention
\$4 An Agent That Watches a Repository	15 min	High	Peak attention
🕒 Taking Your Bearings 2	5 min	Medium	After a short break
\$5 Ordering the Sync	8 min	Medium	Anytime
\$6 The Other Agent, and More Than One Cluster	8 min	Low	Anytime
🕒 Taking Your Bearings 3	5 min	Medium	Immediately before \$7
\$7 The Control Loop, Pointed at a Repository	8 min	Medium	Peak attention

Attention Cost Key:

- **Low:** Concrete, familiar concepts — study anytime
- **Medium:** New concepts requiring focus — study when alert
- **High:** Abstract or complex material — study at peak attention

Session break suggested after ☹ Taking Your Bearings 1. Sections 3 and 4 carry the chapter's argument and deserve a fresh head.

If you only have 15 minutes: read §3, then read §7. Those two sections are the chapter.

"Every deploy is a claim about what should be true. Most of them are never checked again." —
Lodestar Ledgers

🔗 Soundings

Before reading this chapter, try these questions. Your score determines how to approach the content — no shame in any score, just different reading strategies.

This chapter leans harder on what came before than any other in the book. Two of these questions are load-bearing: miss them and the last section will not land, and no amount of careful reading here will fix that. Better to find out now.

1. A controller holds a recorded target and observes what actually exists. What does it do when the two differ, and how often does it check?
2. A Kubernetes object has one field you write and one the system writes. Which is which, and what does it mean when the two disagree?
3. What does a Deployment's default update strategy actually do, and what do `maxSurge` and `maxUnavailable` control?
4. You install a CustomResourceDefinition on a cluster. What can now exist that could not exist before, and what still has to be true before anything actually happens?
5. A process running inside the cluster needs to create and delete objects across several namespaces. What does it need in order to be allowed to, and what decides how much it may do?
6. Where does environment-specific configuration belong, and why is baking it into the container image the wrong answer?
7. General professional knowledge, no Kubernetes required: somebody makes a change directly on a production system, outside whatever process the team normally uses. What goes wrong afterward, and when do you find out?
8. General professional knowledge: you have a repository whose commit history records what was intended. What can you do with that history that you cannot do with the current state of a running server?

Click for answers + reading strategy

1. It acts to close the gap. It takes whatever action moves current state toward desired state, and it does this continuously, in a loop that never ends, not once at creation time. (*Ch 3 §6*)
2. You write *spec*; the system writes *status*. A disagreement between them means the system has not yet reached, or can no longer reach, what you asked for. It is a report, not necessarily a fault. (*Ch 4 §2*)
3. *RollingUpdate* replaces Pods gradually, standing up new ones while taking old ones down, so the application stays available throughout. *maxSurge* caps how many Pods may exist above the desired replica count during the update; *maxUnavailable* caps how many may be missing below it. (*Ch 6 §4*)
4. A new *kind* of object can now be created and stored through the API server: the API is extended. But storage is all you get. Nothing acts on those objects unless a controller is running and watching for them. (*Ch 6 §8, and the pattern named at Ch 10 §3*)
5. It needs an identity, a *ServiceAccount*, and it needs RBAC grants bound to that identity. The *Role* or *ClusterRole* defines what verbs on what resources are permitted; the binding attaches that permission set to the *ServiceAccount*. Permissions are additive, and there is no deny rule. (*Ch 5 §6, Ch 12 §2–§3*)
6. Outside the image, in *ConfigMaps* and *Secrets*, injected at runtime. Baking it in means one image per environment, which destroys the property that makes images useful: the artifact you tested is not the artifact you ship. (*Ch 4 §4, Ch 2 §2*)
7. The system now differs from whatever the team's records say it contains, and nothing announces this. You find out later, from the wrong direction: at the next deployment, when the change is silently overwritten, or when a new person reads the records and acts on a description that stopped being true weeks ago.
8. You can see what was intended and when, who intended it, what it replaced, and — critically — you can return to any previous intent exactly, because the history is complete and each entry is fixed. A running server tells you only what is true right now. It does not tell you what anybody meant.

If you got 6+ right: Skim §1 and §2. They name things you already do. Read §3, §4, and §7 properly; that is where the chapter's argument lives, and it is an argument rather than a vocabulary list.

If you got 3–5 right: Read at normal pace. This chapter is calibrated for you.

If you got 0–2 right: Before you start, re-read **Ch 3 §6** (controllers and the control loop) and **Ch 4 §2** (*spec* versus *status*). Not alongside this chapter. Before it. Those two sections are the foundation this entire chapter stands on, and §7's payoff is a retrieval of Ch 3 §6 specifically. If you missed **question 1** in

particular, that re-read is not optional. Everything else here will make sense without it. The last section will not.

Why This Chapter Matters

The previous chapter left you holding a package and a complaint.

You have a unit now: a chart, a set of overlays, something with a version number that installs the same way twice. It has bought you less than it should have, because the unit still gets applied by a person. Somebody with cluster credentials on a laptop, running a command from a machine nobody audits, at a moment nobody records. They apply the version they believe is current. Afterward, nothing keeps watch.

That complaint has two halves, and this chapter answers them in order.

The first half is **who applies it**. That sounds procedural, the sort of thing settled in a runbook and forgotten. It isn't. The answer determines where cluster credentials physically live, who can reach them, and how large a single mistake gets before something catches it. Change the answer and you change the security posture of the whole delivery path. We take that up in §3.

The second half is **what happens afterward**. "Afterward, nothing keeps watch" is the actual defect, and it should strike you as strange, because you have spent thirteen chapters inside a system whose entire architecture is things that keep watch. A ReplicaSet keeps watch. The scheduler keeps watch. The node controller keeps watch. Kubernetes is, structurally, a collection of processes that notice a gap and close it, forever.

So you already own the answer to the second half. You have not yet been shown that you own it. That is what §7 is for, and it is why this chapter's subtitle gives the answer away on the second line: the recognition is worth more than the surprise.

A shift in what you are. Up to now you have been someone who *makes changes to a cluster*. This chapter asks you to become someone who *maintains a claim about what a cluster should contain*, and then never touches the cluster again.

That is genuinely uncomfortable, and it should be said plainly rather than sold. Thirteen chapters of your growing competence have been measured in `kubectl` fluency: knowing the verb, knowing the flag, knowing which object to reach for. This chapter asks you to give up the terminal as the instrument of change. Not as an instrument of *inspection*; you will read the cluster constantly, and Chapters 13 and 16 are built on it. But as the thing you use to make something different, the terminal goes away. What replaces it is a file, a commit, and a review.

In my experience, practitioners who make this shift describe the same two feelings in sequence: first that it is slower, then that they cannot go back.

The stakes here were banked in Chapter 1, and one clause will do: this domain doubled in the 2025-11-24 blueprint revision. Chapter 14 cashed the first half. This is the second.

About what CNCF actually publishes. Chapter 14 made this statement at length and it covers this chapter too, so one back-bearing rather than a repetition: the published curriculum gives two competency names under this domain and no list of topics beneath either [*cross-bearing: see Ch 14 — Why This Chapter Matters*]. What supports the inference that GitOps belongs here is positive rather than speculative. Argo and Flux are both CNCF **graduated** projects [source: cncf-project-maturity-levels-2026-08-23], and OpenGitOps is a CNCF project [source: opengitops-principles-v1-2026-08-31]. A CNCF exam asking about application delivery is asking about the delivery model CNCF's own graduated projects implement. That is the basis. It is a good one, and it is honest about being an inference.

One consequence runs through the rest of the chapter without further comment: nothing here is described as "frequently tested" or "commonly appears." Those claims would require a published sub-topic list, and there isn't one. What you will see instead is "easy to confuse" and "this is the distinction the material rewards," which are claims this book can actually stand behind.

Dead Reckoning: GitOps is a set of four principles about how desired state is stored and applied. The desired state is declarative; it lives in a version-controlled store that keeps a complete history; software agents pull it from that store rather than having it pushed to them; and those agents continuously compare actual state against it and act to close the difference [source: opengitops-principles-v1-2026-08-31]. Argo CD and Flux are two implementations. Both are Kubernetes controllers: Argo CD's application controller "is a Kubernetes controller" [source: argocd-architecture-2026-08-31], and Flux ships as a set of "Flux Controllers" [source: flux-concepts-2026-08-31]. Continuous integration is not part of the definition.

What You'll Learn

By the end of this chapter, you'll be able to:

- **Name** the twelve factors, and say which ones Kubernetes hands you and which remain your application's problem
- **Distinguish** the two update strategies a Deployment implements from the release patterns that need tooling above it — blue/green and canary
- **State** the four OpenGitOps principles, and explain why a pipeline that pushes to a cluster satisfies none of the last two
- **Read** a delivery agent as what it structurally is: a controller, with its desired state in an unusual place
- **Explain** what OutOfSync reports, why it is a signal rather than an error, and why a person can cause one without anything failing
- **Say** what Argo CD does by default when it detects drift, and why that default is the right one

- **Recognize** the control loop from Chapter 3 wearing different clothes, and say exactly what changed and what did not

You'll also stop thinking of GitOps as a deployment tool, which is the misreading this chapter exists to prevent.

.. §1 — Twelve Factors, and the Ones Kubernetes Already Solved

Two chapters ago you were promised a methodology's word for a thing you already do. Chapter 4 promised it about configuration [*cross-bearing: see Ch 4 §4 — configuration kept outside the image*], and Chapter 5 promised it about shutdown [*cross-bearing: see Ch 5 §4 — scheduled once, replaced never*]. Both promises point here.

The **twelve-factor app** is a methodology for building software delivered as a service. Its own summary is that a twelve-factor app uses declarative formats for setup automation, keeps a clean contract with the underlying operating system for portability, is suitable for deployment on modern cloud platforms, minimizes divergence between development and production, and can scale up without significant changes to tooling or architecture [source: twelve-factor-app-2026-08-23].

Read that list again and notice something: it predates Kubernetes and describes Kubernetes exactly.

That is not coincidence and it is not prophecy. Both came out of the same problem, running many applications reliably on shared infrastructure, and arrived at the same conclusions about what an application must do to be run that way.

Here are the twelve:

	Factor	In one line
I	Codebase	One codebase tracked in revision control, many deploys
II	Dependencies	Explicitly declare and isolate dependencies
III	Config	Store config in the environment
IV	Backing services	Treat backing services as attached resources
V	Build, release, run	Strictly separate build and run stages
VI	Processes	Execute the app as one or more stateless processes
VII	Port binding	Export services via port binding
VIII	Concurrency	Scale out via the process model
IX	Disposability	Maximize robustness with fast startup and graceful shutdown
X	Dev/prod parity	Keep development, staging, and production as similar as possible
XI	Logs	Treat logs as event streams
XII	Admin processes	Run admin/management tasks as one-off processes

[source: twelve-factor-app-2026-08-23]

Twelve numbered rules is a poor thing to memorize and a good thing to sort. The useful question is not "what is factor VII" but **who solves this**: the platform, or you.

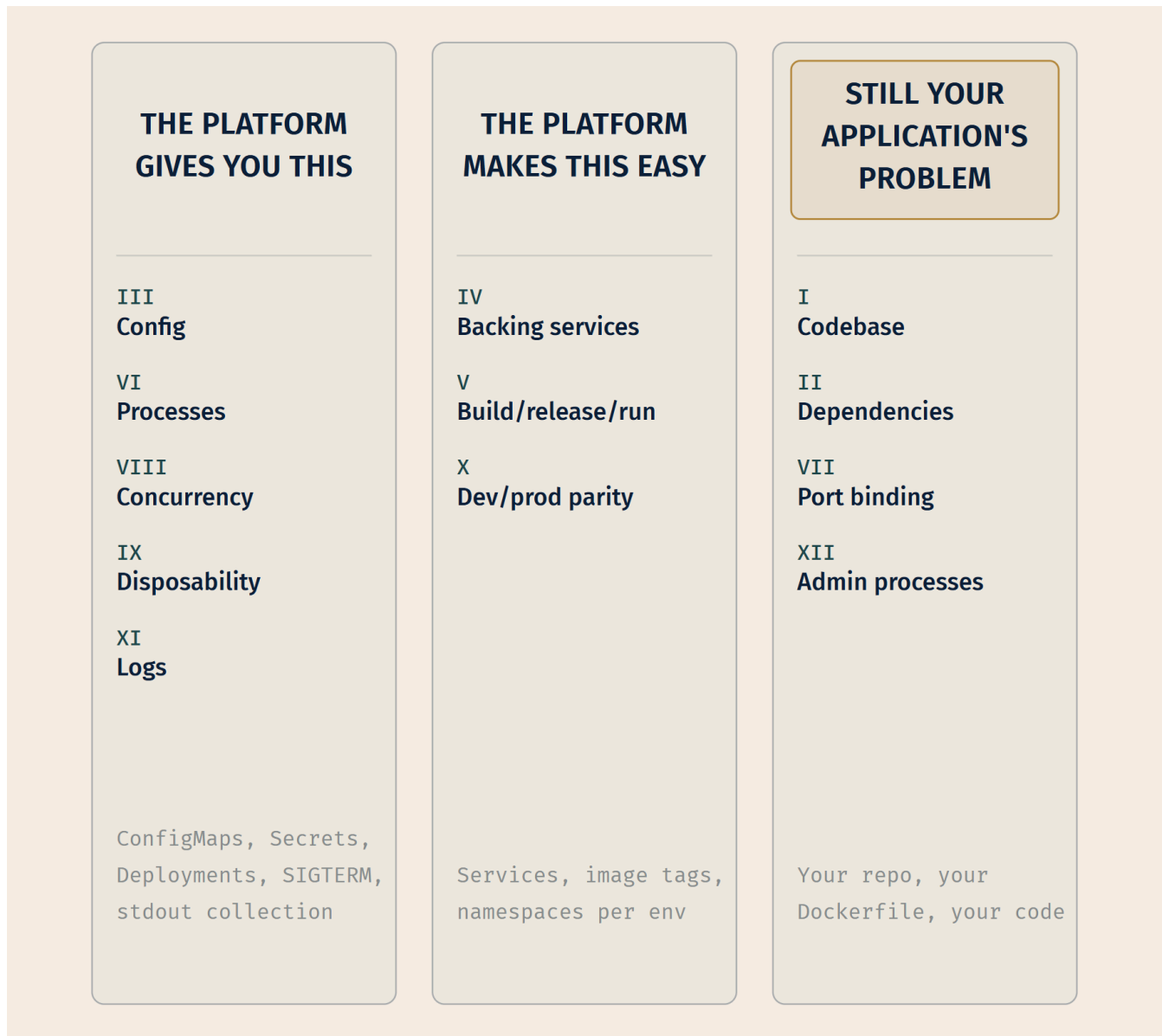


Figure 15.1 — The twelve factors, sorted by who solves them. The three columns are the argument: twelve unrelated-looking rules resolve into things the platform already does for you, things it removes the friction from, and things that remain entirely yours. The middle column is where most of the disappointment lives — Kubernetes makes dev/prod parity *achievable*, not automatic.

Four of these deserve development, because each one is a name for something you have already been taught.

Factor III — Config

"An app's config is everything that is likely to vary between deploys (staging, production, developer environments, etc)." [source: twelve-factor-iii-config-2026-08-31]

The methodology's test for whether you have done this correctly is unusually sharp: *"A litmus test for whether an app has all config correctly factored out of the code is whether the codebase could be made open source at any moment, without compromising any credentials"* [source: twelve-factor-iii-config-2026-08-31]. That is a test you can run against your own repository this afternoon, and it is more honest than most compliance checklists.

The instruction is "store config in the environment," and this is where readers slip. It does **not** mean "put it in a config file." The document is explicit that a config file is an improvement over hard-coded constants but *"still has weaknesses: it's easy to mistakenly check in a config file to the repo"* [source: twelve-factor-iii-config-2026-08-31]. The prescription is environment variables, precisely because they live outside the codebase and are far less likely to be committed by accident. The methodology's own phrasing is that there is *"little chance of them being checked into the code repo accidentally"* [source: twelve-factor-iii-config-2026-08-31].

Kubernetes gives you this in the form you already know: ConfigMaps and Secrets, mounted as environment variables or as files [cross-bearing: see Ch 4 §4 — configuration kept outside the image]. The object model was designed for factor III.

📌 **Snag:** "Store config in the environment" is a claim about *where config lives relative to your code*, not about the literal `env:` block. A ConfigMap mounted as a file satisfies factor III perfectly well: the file is supplied by the deploy environment, not checked into the repository. Readers who take the phrase literally conclude that mounted-file config violates the methodology. It doesn't.

One more piece of factor III catches people, and it is worth carrying. The methodology objects to grouping variables into named environments: *"In a twelve-factor app, env vars are granular controls, each fully orthogonal to other env vars. They are never grouped together as 'environments', but instead are independently managed for each deploy"* [source: twelve-factor-iii-config-2026-08-31]. If you have ever watched a staging config bundle acquire a permanent, load-bearing difference from production that nobody remembers introducing, you have seen why.

Factors VI and VIII — Processes and Concurrency

"Twelve-factor processes are stateless and share-nothing. Any data that needs to persist must be stored in a stateful backing service, typically a database" [source: twelve-factor-vi-processes-2026-08-31].

This is the assumption underneath Deployments. A Deployment can replace any Pod with any other Pod because the methodology's constraint holds: no Pod is carrying anything the next one will need. Scale out by adding processes, which is factor VIII, and any process can serve any request.

The methodology bans one thing by name here: *"Sticky sessions are a violation of twelve-factor and should never be used or relied upon"* [source: twelve-factor-vi-processes-2026-08-31].

And when the constraint genuinely does not hold, when the replicas are *not* interchangeable, Kubernetes has a different object for you. It is a different object precisely because it is stepping outside this factor [cross-bearing: see Ch 6 §6 — when Pods are not interchangeable].

Factor IX — Disposability

Chapter 5 taught you `terminationGracePeriodSeconds` and the SIGTERM-then-SIGKILL sequence, and told you the word was coming. Here it is.

"The twelve-factor app's processes are disposable, meaning they can be started or stopped at a moment's notice" [source: twelve-factor-ix-disposability-2026-08-31]. The two halves are startup and shutdown. On startup: *"Processes should strive to minimize startup time. Ideally, a process takes a few seconds from the time the launch command is executed until the process is up and ready to receive requests or jobs."* On shutdown: *"Processes shut down gracefully when they receive a SIGTERM signal from the process manager" [source: twelve-factor-ix-disposability-2026-08-31].*

And a third clause, which is the one that matters most on a real cluster: *"Processes should also be robust against sudden death, in the case of a failure in the underlying hardware" [source: twelve-factor-ix-disposability-2026-08-31].* Graceful shutdown is a courtesy the platform extends when it can. A node that loses power extends nothing.

🌀 **Mnemonic:** Disposability has two speeds and a floor. **Fast up** (seconds, not minutes). **Clean down** (handle SIGTERM). **Survive the floor dropping** (be correct even when neither happens).

Factor XI — Logs

"Logs are the stream of aggregated, time-ordered events collected from the output streams of all running processes and backing services" [source: twelve-factor-xi-logs-2026-08-31].

The rule is that the application does not participate in log management at all: *"A twelve-factor app never concerns itself with routing or storage of its output stream. Each running process writes its event stream, unbuffered, to stdout" [source: twelve-factor-xi-logs-2026-08-31].* The execution environment captures the stream, collates it with every other stream from the app, and routes it onward [source: twelve-factor-xi-logs-2026-08-31].

That is exactly why `kubectl logs` works on every container without any container being configured for it, and it is why cluster-wide log collection can be a node-level concern rather than an application concern [cross-bearing: see Ch 18 §6 — lines from everywhere].

☆ Fixed Point

The twelve-factor app is a set of constraints an application accepts in exchange for being run by a platform. Kubernetes implements the platform side — config injection, stateless replication, graceful termination, log collection. It cannot implement the application side. An application that writes its own log files, keeps session state in memory, or takes ninety seconds to start is not made twelve-factor by being containerized.

Twelve-factor is also a predecessor of vocabulary you will meet later. When Chapter 17 lists the characteristics of cloud native systems, several of them are these factors under newer names [cross-

bearing: see Ch 17 §3 — small pieces, replaced whole].

§2 — Ways to Replace What's Running

Chapter 6 taught you the mechanics of replacing a running fleet: `RollingUpdate` and `Recreate`, `maxSurge` and `maxUnavailable`, pause and resume [cross-bearing: see Ch 6 §4 — changing the fleet under way]. This section does not restate any of it. What it adds is the vocabulary: the names practitioners use for release patterns, the trade-off each one makes, and one line that readers persistently get wrong.

The line is this.

☆ Fixed Point

`RollingUpdate` and `Recreate` are values of a field on a `Deployment`. `Blue/green` and `canary` are patterns that require tooling above the `Deployment` object. A `Deployment` cannot express either one on its own.

Chapter 6 named the second group and deferred them. They arrive now.

The umbrella: progressive delivery

The term that covers the whole family is **progressive delivery**: *"the process of releasing updates of a product in a controlled and gradual manner, thereby reducing the risk of the release, typically coupling automation and metric analysis to drive the automated promotion or rollback of the update"* [source: argo-rollouts-strategies-2026-08-23].

Note the two halves of that definition. *Gradual* is the visible part. *Metric analysis driving automated promotion or rollback* is the part that makes it more than a slow deploy: the release watches itself.

The four, and what each costs

Recreate. *"A `Recreate` deployment deletes the old version of the application before bringing up the new version. As a result, this ensures that two versions of the application never run at the same time, but there is downtime during the deployment"* [source: argo-rollouts-strategies-2026-08-23].

The trade is stated in the definition: you accept downtime and you get the guarantee that two versions never coexist. That guarantee is not a consolation prize. If the new version runs a schema migration the old version cannot read, coexistence is the failure mode, and `Recreate` is the correct choice rather than the lazy one.

RollingUpdate. *"A `RollingUpdate` slowly replaces the old version with the new version. As the new version comes up, the old version is scaled down in order to maintain the overall count of the application. This is the default strategy of the `Deployment` object"* [source: argo-rollouts-strategies-2026-08-23].

No downtime, and the exact inverse guarantee: both versions serve traffic simultaneously, for the duration. Your application must tolerate that.

Blue/green. Two complete environments. CNCF's glossary describes the operator maintaining two environments, "blue" and "green," with one serving live production traffic while the other is updated; after testing on the inactive environment, traffic switches via load balancer [source: [cncf-glossary-blue-green-deployment-2026-08-31](#)]. Argo's description agrees and adds the reason: *"During this time, only the old version of the application will receive production traffic. This allows the developers to run tests against the new version before switching the live traffic to the new version"* [source: [argo-rollouts-strategies-2026-08-23](#)].

The cost is capacity. You run two full copies. What you buy is the ability to test the new version *in production, with production configuration and production backing services*, before a single user reaches it. The cutover is one moment rather than a gradual mix, which means the rollback is also one moment.

CNCF's glossary adds an assessment worth reading, because it is more critical than most vendor material: blue/green *"is an appropriate strategy for non-cloud native software that needs to be updated with minimal downtime. However, its use is normally a 'smell' that legacy software needs to be re-engineered so that components can be updated individually"* [source: [cncf-glossary-blue-green-deployment-2026-08-31](#)]. The glossary notes the term is typically applied to whole systems with multiple services updated in lockstep, and is sometimes misapplied to individual services [source: [cncf-glossary-blue-green-deployment-2026-08-31](#)].

Canary. *"A Canary deployment exposes a subset of users to the new version of the application while serving the rest of the traffic to the old version. Once the new version is verified to be correct, the new version can gradually replace the old version"* [source: [argo-rollouts-strategies-2026-08-23](#)]. Argo Rollouts states the same idea from the operator's side: *"a canary rollout is a deployment strategy where the operator releases a new version of their application to a small percentage of the production traffic"* [source: [argo-rollouts-canary-2026-08-31](#)].

CNCF's glossary is blunt about why anyone bothers: *"No matter how thorough the testing strategy, there are always some bugs discovered in production. Shifting 100% of traffic from one version of an app to another can lead to more impactful failures"* [source: [cncf-glossary-canary-deployment-2026-08-31](#)].

The cost is infrastructure. Canary needs something that can split traffic by proportion, and something that can judge whether the small proportion is going well. Argo's comparison states this directly: canary strategies *"offer greater flexibility but demand more infrastructure (traffic-splitting via a service mesh or ingress controller) and metric analysis; Blue/Green needs no traffic provider and suits workloads such as queue workers"* [source: [argo-rollouts-strategies-2026-08-23](#)].

That last clause is the practical answer to "which one." A queue worker has no inbound traffic to split. Canary is not a better blue/green; it is a different tool that needs a request path to work on [*cross-bearing: see Ch 10 §3 — the object is not the implementation*] [*cross-bearing: see Ch 17 §5 — a network that knows what it's carrying*].

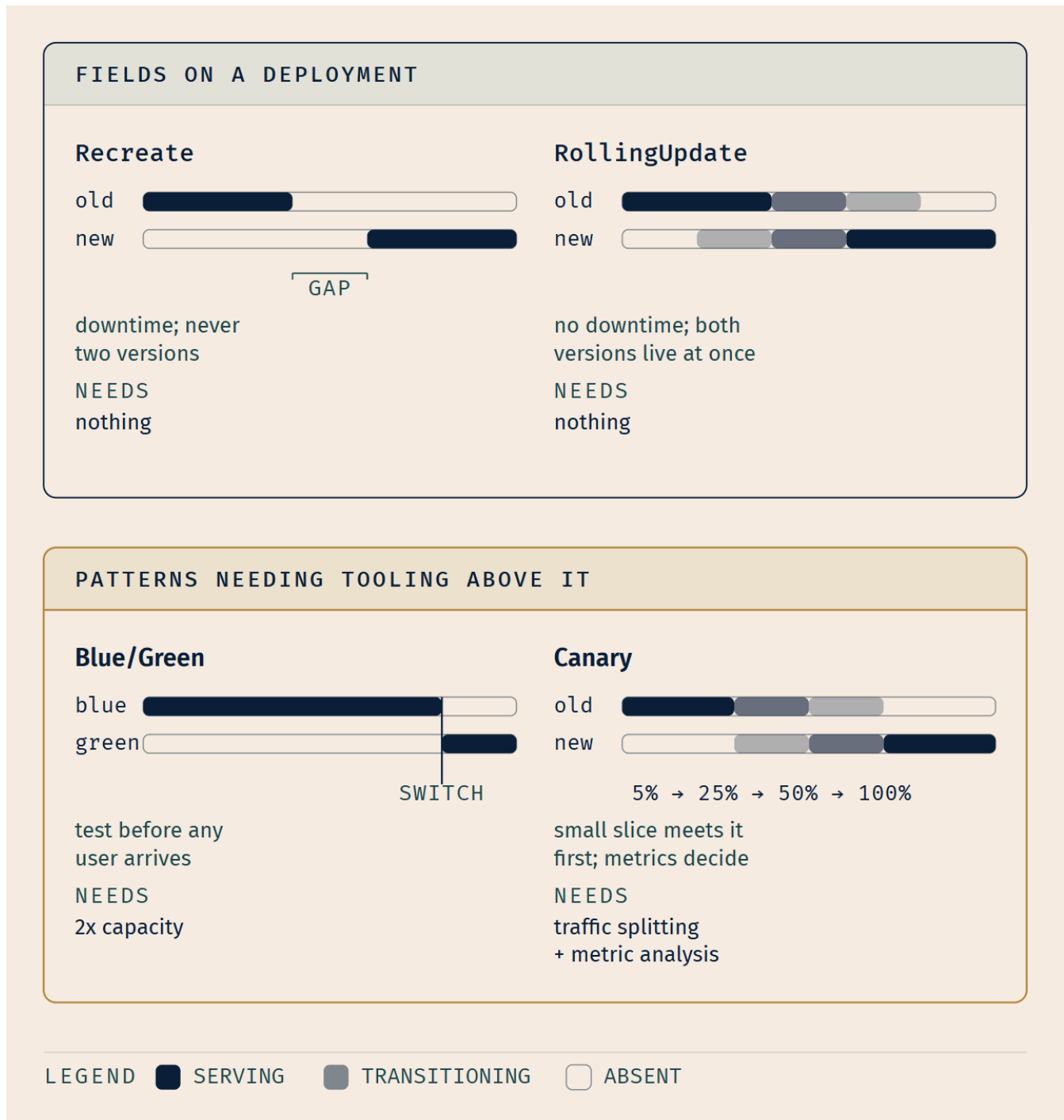


Figure 15.2 — Four ways to replace what's running. The two enclosures carry the point: the top pair are values you set on a Deployment, and the bottom pair are patterns something else has to implement for you. The footer rows are the actual decision criteria — what each strategy demands before you can use it at all.

📖 **Worth Securing:** My own read, from watching teams choose: blue/green usually wins over canary for infrastructure reasons rather than risk reasons. And the mechanical half of that is not opinion. If there is no service mesh and no metrics pipeline wired to the release, canary is not on the menu at all, because canary *"demand[s] more infrastructure (traffic-splitting via a service mesh or ingress controller) and metric analysis"* [source: argo-rollouts-strategies-2026-08-23]. Knowing *why* a team runs blue/green tells you more about their platform than about their risk appetite.

One term this book will not teach you

You may have heard **A/B testing** listed alongside these four. This book leaves it out deliberately, and the reason is a real distinction rather than an omission.

A/B testing appears in Argo's documentation not as a rollout strategy but as a use of a separate resource: *"A user can use experiments to enable A/B/C testing by launching multiple experiments with a different version of their application for a long duration"* [source: argo-rollouts-experiments-2026-08-31]. It is product experimentation. You are measuring which version users prefer, over a long duration, on purpose. The four strategies above are release mechanics: you are measuring whether the new version is *broken*, for as short a duration as you can manage. Delivery tooling can implement both, which is why they end up in the same lists, but they answer different questions.

Nothing in this chapter's questions turns on A/B testing.

🔍 **Closer Look:** One vocabulary note, for readers with a good memory. Chapter 6 called these "release strategies" [cross-bearing: see Ch 6 §4 — *changing the fleet under way*]. This book's headword is **deployment strategy**, and the reason is crowding: "release" already carries two other meanings here, a Kubernetes minor version and a Helm release [cross-bearing: see Ch 14 §3 — *chart, release, revision*]. Three senses of one word across two adjacent chapters is one too many. Both phrases name the same thing; if you meet "release strategy" in the field, it is this.

🕒 Taking Your Bearings 1: The Application, and the Shapes of a Release

Six questions. Two of them reach back to earlier chapters. Those are marked, and they are marked because they are the point.

1. An application stores uploaded files on the local filesystem of whichever Pod handled the upload. Which twelve-factor factor does this violate, and what does it break in Kubernetes specifically?
2. A team stores its production database password in a file called `config/production.yml`, committed to the application repository, and reads it at startup. Apply the twelve-factor litmus test. What does it say?
3. [retrieval: ch4] Your application needs a different API endpoint URL in staging than in production, and the same container image must run in both. Name two Kubernetes mechanisms that deliver that URL to the running container, and say where the value physically lives in each case.

4. [retrieval: ch6] During a RollingUpdate on a Deployment with 10 replicas, maxSurge: 2 and maxUnavailable: 1. What is the maximum number of Pods that may exist at any moment during the update, and the minimum number that may be available?
5. A team wants to release a new version to 5% of users, watch error rates for twenty minutes, and automatically abort if errors rise. Their workload is an HTTP API behind an Ingress. Which strategy is this, and what must already exist in the cluster for it to be possible?
6. Which of these is a value you can set on a Deployment's update-strategy field, and which requires additional tooling: Recreate, blue/green, RollingUpdate, canary?

Answers with explanations

1. Factor VI (Processes — stateless and share-nothing). The twelve-factor rule is that *"any data that needs to persist must be stored in a stateful backing service"* [source: twelve-factor-vi-processes-2026-08-31]. What it breaks in Kubernetes: a Deployment replaces any Pod with any other Pod, and a request that retrieves the file may land on a replica that never had it. Worse, the Pod's writable layer is destroyed when the Pod is, so the data is not merely misplaced. It is gone at the next rollout, node drain, or eviction.

Why not "factor IX, disposability"? Tempting, and adjacent. A Pod holding unique data is certainly not disposable. But IX is about *startup and shutdown behavior*; the root violation is holding state at all, which is VI.

2. It fails. The test is whether *"the codebase could be made open source at any moment, without compromising any credentials"* [source: twelve-factor-iii-config-2026-08-31]. Publishing this repository publishes the production database password. The methodology anticipates exactly this: a config file is better than a hard-coded constant, but *"it's easy to mistakenly check in a config file to the repo"* [source: twelve-factor-iii-config-2026-08-31] — and here it wasn't even a mistake, it was the design.

3. [retrieval: ch4] ConfigMap and Secret. Either can be surfaced to the container as environment variables or mounted as files. Where the value lives: in the cluster, as an object in etcd, entirely outside the image. The image is identical in both environments; the object differs. That separation is what allows one tested artifact to run everywhere [cross-bearing: see Ch 4 §4 — configuration kept outside the image].

Common wrong answer: "build two images with different baked-in values." This is the failure factor III exists to name. You are now shipping an artifact you did not test, because the thing you tested was the other image.

4. Maximum 12 Pods; minimum 9 available. maxSurge: 2 permits two Pods above the desired count of 10. maxUnavailable: 1 permits one below it. The two are independent caps, not a single budget: the controller may be simultaneously two over and one unavailable, which is what makes the update proceed at all [cross-bearing: see Ch 6 §4 — changing the fleet under way].

Why "11 and 10" is wrong: that reads `maxSurge` and `maxUnavailable` as if only one could be in effect. Both apply throughout.

5. Canary. The tell is the *proportion of traffic plus automated abort on a metric*. That is progressive delivery's definition, "*coupling automation and metric analysis to drive the automated promotion or rollback of the update*" [source: argo-rollouts-strategies-2026-08-23].

What must already exist: something that can split traffic by weight, a service mesh or an ingress controller, and a metrics system the release logic can query [source: argo-rollouts-strategies-2026-08-23]. Without traffic splitting there is no way to send 5% anywhere; without metrics there is nothing to abort on.

6. Recreate and RollingUpdate are Deployment strategy values. Blue/green and canary require tooling above the Deployment. This is the section's Fixed Point and the distinction most likely to be tested from this material. `RollingUpdate` "*is the default strategy of the Deployment object*" [source: argo-rollouts-strategies-2026-08-23]; the other two are patterns implemented by controllers that manage Deployments or replace them.

If you got 5–6: You have the application layer. §3 changes subject entirely: it stops asking what to deploy and starts asking who does it.

If you got 3–4: Solid. Re-read the answer to whichever you missed; the reasoning matters more than the fact.

If you got 0–2: Re-read **§1's factor III** and **§2's Fixed Point** before continuing. Those two ideas get used again in §3 and §4 without reintroduction.

..l §3 — Push, or Pull

This is the chapter's central argument, and it begins with a fact that clears the ground.

Kubernetes "*does not deploy source code and does not build your application. Continuous Integration, Delivery, and Deployment (CI/CD) workflows are determined by organization cultures and preferences as well as technical requirements*" [source: k8s-docs-overview-2026-08-23].

That is the documentation's own list of what Kubernetes is not, and it settles something. The cluster is indifferent to who builds your artifact and how. It receives objects through its API and reconciles them. Everything before that — compiling, testing, packaging, tagging — happens elsewhere, by arrangement. Chapter 4 told you this in passing [*cross-bearing: see Ch 4 §1 — you file a declaration*]; here it becomes load-bearing.

The three letters, expanded

The abbreviation bundles three practices that are worth separating, since the exam-relevant distinctions live in the gaps between them.

CI — continuous integration. *"The practice of integrating code changes as regularly as possible"* [source: cncf-glossary-continuous-integration-2026-08-31]. In practice this means a server that checks every commit merges cleanly, runs quality checks and tests, and, as CNCF puts it, *"allows software teams to turn every code commit into either a concrete failure or a viable release candidate"* [source: cncf-glossary-continuous-integration-2026-08-31].

CD — continuous delivery. *"A set of practices in which code changes are automatically deployed into an acceptance environment (or, in the case of continuous deployment, into production)"* [source: cncf-glossary-continuous-delivery-2026-08-31]. It includes procedures ensuring software is adequately tested before deployment, and a way to roll changes back if needed [source: cncf-glossary-continuous-delivery-2026-08-31].

CD — continuous deployment. *"Goes a step further than continuous delivery by deploying finished software directly to production"* [source: cncf-glossary-continuous-deployment-2026-08-31].

📌 **Snag:** CNCF abbreviates *both* continuous delivery and continuous deployment as "CD" [source: cncf-glossary-continuous-delivery-2026-08-31] [source: cncf-glossary-continuous-deployment-2026-08-31]. This is genuinely ambiguous in the field, not just on paper. When the distinction matters, and it matters when the question is whether a human approves the production step, spell the word out.

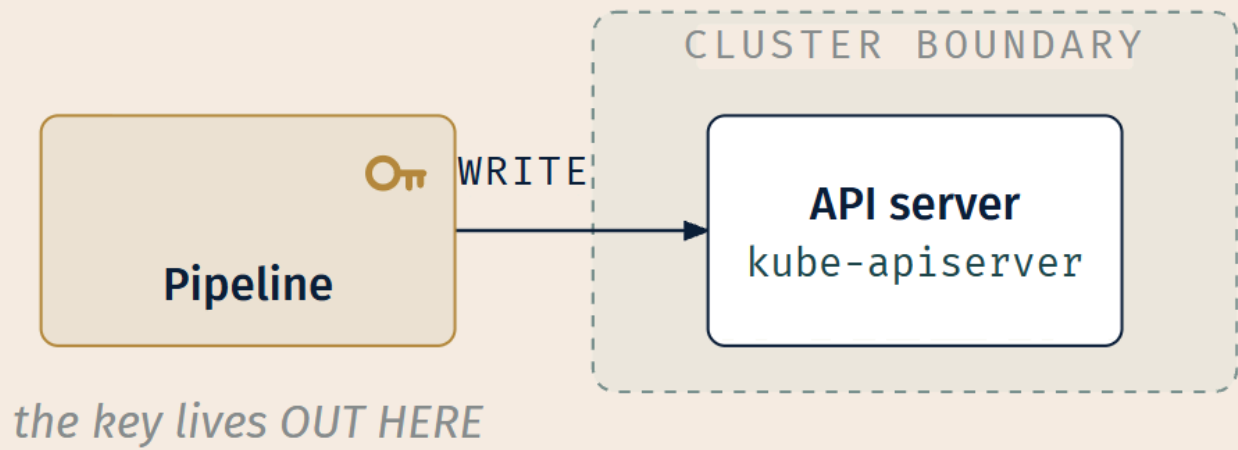
The architectural question

Now the question this section exists for. An artifact has been built. Something must apply it to a cluster. There are two arrangements, and they differ in a way that is not a matter of taste.

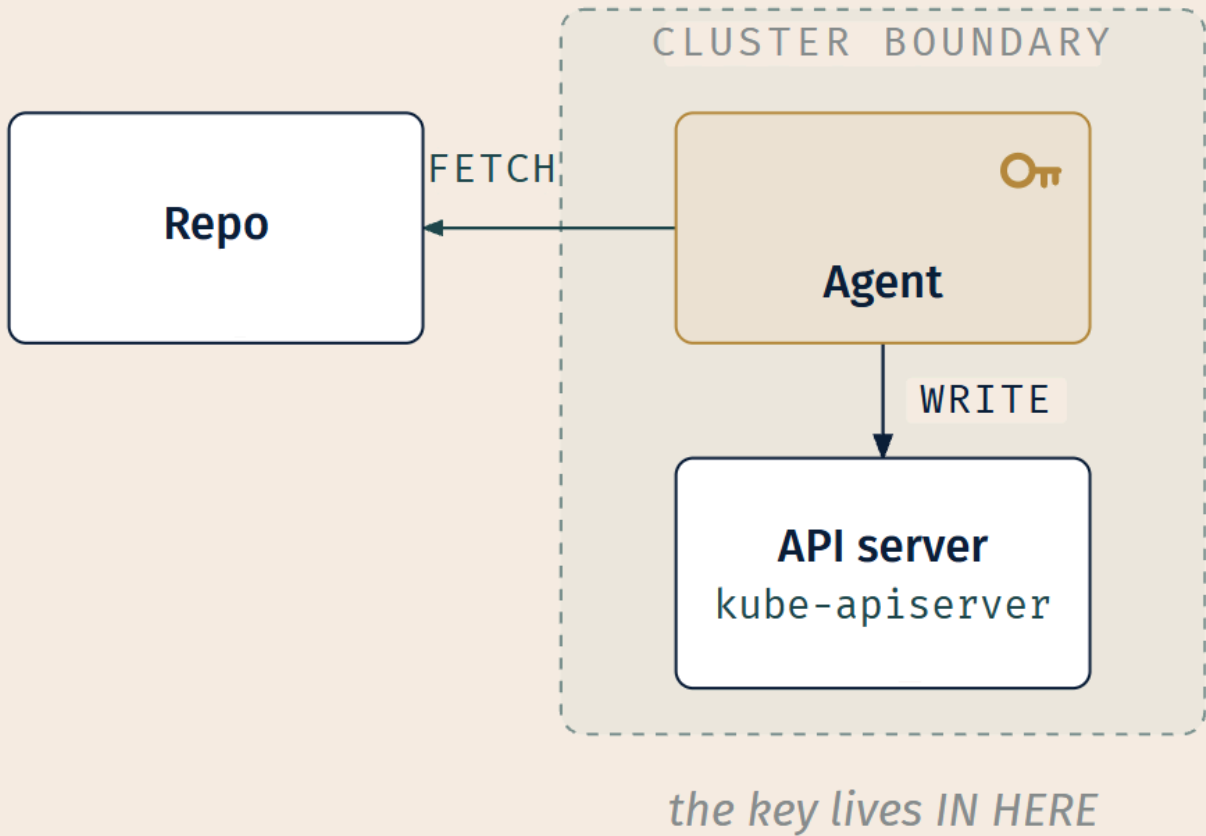
Push. A pipeline runs outside the cluster. It holds credentials *to the cluster*. When it finishes building, it reaches inward across the cluster boundary and applies the manifests. This is what most teams do first, because it is the obvious extension of a build pipeline: the pipeline already has the artifact, so give it the ability to install the artifact.

Pull. An agent runs *inside* the cluster. It holds credentials *to a repository*. It reaches outward, fetches the desired state, and applies it locally. Nothing outside the cluster holds cluster credentials, because nothing outside the cluster needs them.

PUSH



PULL



LEGEND

 Cluster-write credential

 Cluster boundary

 Privileged write

 External fetch

Figure 15.3 — Push and pull, mirrored. The cluster boundary is the same line in both panels and the arrow is the only thing that reverses. Watch the key: in push it sits outside the boundary, in a system the cluster does not control. In pull it sits inside, in a Pod the cluster does control. Everything else in this section is a consequence of that one difference.

Four consequences follow, and none of them is subtle once you look.

A note on what follows. The four consequences below are **this book's reading** of principle 3 combined with the two projects' documented credential storage, not a comparison any cached source makes in these terms. No CNCF or vendor document in this chapter's corpus argues push-versus-pull as a security question. Where a consequence can be anchored to a source, it is tagged; where it cannot, it is the book's argument and you should weigh it as such.

Where the credentials sit. In push, a set of cluster-write credentials lives in your CI system: in its secret store, readable by its jobs, and often by anyone who can edit a pipeline definition. In pull, they live in a Kubernetes Secret in the cluster they apply to. Argo CD, for instance, *"stores the credentials of the external cluster as a Kubernetes Secret in the argocd namespace"* [source: argocd-security-cluster-credentials-2026-08-31] [cross-bearing: see Ch 12 §4 — secrets are not encrypted].

There is one thing the corpus does say in this neighborhood, and it is worth having. Argo CD's best-practices page argues for separating the config repository from the source repository partly on access grounds: *"The developers who are developing the application, may not necessarily be the same people who can/should push to production environments"* [source: argocd-best-practices-2026-08-31]. That is the access-separation instinct that push-based delivery tends to collapse and pull-based delivery tends to preserve.

What a compromise gets. This is the sharper version of the same point, and it is entirely the book's inference. Compromise a push pipeline and you have write access to every cluster it deploys to, which for a shared CI system may be all of them. Compromise a pull agent and you have one cluster, the one you were already inside. This book calls that reach the **blast radius**: how far the damage from a single compromise extends. Pull does not prevent compromise. It bounds it.

What happens between deploys. In push, nothing. The pipeline runs, exits, and the cluster is on its own until the next commit. In pull, the agent is still running. It is still comparing. CNCF's glossary names exactly this as the problem GitOps addresses, listing configuration drift first among them and observing that *"configuration drift can be hard to detect and resolve without a source of truth governing it"* [source: cncf-glossary-gitops-2026-08-31]. This is the difference that turns out to matter most, and §4 is about what an agent does with the comparison.

What "the truth" means. In push, the authoritative answer to "what is supposed to be running" is whatever the last pipeline applied, a fact you must reconstruct from build logs. In pull, it is a file, in a repository, that anyone can read, with a history showing every previous answer.

Logbook Entry: There is a version of this story on every platform team.

The incident is small. A service is misbehaving on Friday afternoon, somebody with cluster access finds the cause, and fixes it directly: one field, one `kubectl edit`, thirty seconds. The service recovers. It is genuinely the right call in the moment; the alternative is a code review at 6pm.

The fix is correct and it is also invisible. It exists nowhere except in the cluster's own state. Nobody writes it down, because writing it down was the slow path they were avoiding.

Six weeks later somebody deploys a routine change to the same service. The deployment applies the manifests from the repository, which have never known about the Friday fix. The field reverts. The original problem returns, now with six weeks of distance between cause and effect, and a change log pointing at an unrelated commit.

Push-based delivery has no mechanism that would have caught this. The pipeline was not running for those six weeks and had nothing to compare against if it had been. The gap between what the repository said and what the cluster contained was real, persistent, and completely silent.

Keep this story in mind through §4. The mechanism that catches it has a name, and it is not an alarm.

GitOps

GitOps is the name for the pull arrangement done rigorously. The definition is not a vendor's; it comes from OpenGitOps, a CNCF project, and it is four principles rather than a description of tooling.

"GitOps is a set of principles for operating and managing software systems." The desired state of a GitOps-managed system must be:

1. **Declarative** — *"A system managed by GitOps must have its desired state expressed declaratively."*
2. **Versioned and Immutable** — *"Desired state is stored in a way that enforces immutability, versioning and retains a complete version history."*
3. **Pulled Automatically** — *"Software agents automatically pull the desired state declarations from the source."*
4. **Continuously Reconciled** — *"Software agents continuously observe actual system state and attempt to apply the desired state."*

[source: opengitops-principles-v1-2026-08-31]

1 • DECLARATIVE

desired state expressed
declaratively

◀ *you know this: Ch 4 §1*

2 • VERSIONED & IMMUTABLE

stored so as to enforce
immutability, versioning,
complete history

◀ **NEW HERE**

3 • PULLED AUTOMATICALLY

agents automatically pull
desired state from source

◀ *you know this: Ch 3 §5*

4 • CONTINUOUSLY RECONCILED

agents continuously observe
actual state and apply
desired state

◀ *you know this: Ch 3 §6*

Figure 15.4 — The four principles, and where you already met three of them. The markers in each block are the figure's real content. Only principle 2 is new to you. The rest is Chapter 4's declarative model and Chapter 3's watch-and-reconcile architecture, restated with the desired state living somewhere else. Hold that thought; §7 collects on it.

Two of the four carry the weight, and both are places readers go wrong.

Principle 3 is why push-based CD is not GitOps. The word is *pulled*. Agents fetch the declarations; nothing hands the declarations to them. OpenGitOps states it as an active property of the agent: *"Software agents automatically pull the desired state declarations from the source"* [source: opengitops-1-0-announcement-2026-08-31]. A pipeline that stores manifests in Git and then pushes them into a cluster satisfies principles 1 and 2 and fails 3 and 4 completely. It is a perfectly reasonable thing to build. It is not GitOps, and calling it GitOps loses the distinction that the term exists to make.

The project is unusually explicit that this precision was deliberate: *"The wording of each principle and linked glossary item was very carefully chosen"* [source: opengitops-1-0-announcement-2026-08-31].

Principle 4 is why GitOps is not a deploy-time event. *Continuously* observe. Not "observe at deploy time," not "observe when triggered." The agent is described as having an ongoing obligation: *"The GitOps software agents have to be aware of the actual state of a system under management and attempt to apply the desired state"* [source: opengitops-1-0-announcement-2026-08-31].

OpenGitOps names the thing this catches. **Drift** is *"when a system's actual state has moved or is in the process of moving away from the desired state..."* [source: opengitops-glossary-2026-08-31] — the ellipsis is the snapshot's, and the definition it carries is partial. **Reconciliation** is *"the process of ensuring the actual state of a system matches its desired state"* [source: opengitops-glossary-2026-08-31].

CNCF's own glossary entry for GitOps names drift first among the problems the practice addresses, alongside failed deployments, inconsistent environments, and difficulty tracking historical changes [source: cncf-glossary-gitops-2026-08-31].

🔗 **Closer Look:** "GitOps" names Git, but the principles do not require it. OpenGitOps says so: *"many version control systems can be used in GitOps as long as they meet those two basic requirements and teams use them in a conformant manner"* [source: opengitops-1-0-announcement-2026-08-31]. The announcement does not restate which two requirements it means; on this book's reading they are principle 2's immutability and complete version history, which is the only pair the principles state about the store itself. Git is, in practice, the overwhelmingly common choice. The definition is about the properties, not the tool.

One thing GitOps does not do

Before §4 makes this concrete, kill a misreading that the phrase "the repository is the source of truth" invites.

⚠ Navigational Hazards

A GitOps agent does **not** write to the cluster's datastore directly, and does not bypass the API server.

Chapter 3 made a claim about this architecture that has held for twelve chapters: the API server is the only thing that mutates cluster state, and every actor — kubectl, the scheduler, every controller — goes through it [*cross-bearing: see Ch 3 §5 — the only door in*]. A delivery agent is an API client like any other. Its requests pass through authentication, then authorization, then admission, in that order, exactly as yours do [*cross-bearing: see Ch 8 §2 — three gates and a logbook*].

Where this goes wrong: readers hear "the repository is the source of truth" and picture the repository *replacing* etcd, as though desired state now lives in Git instead of in the cluster. It does not. etcd still holds every object. What Git holds is the *authored* desired state, upstream of the cluster, and the agent's job is to keep the two in agreement by making ordinary API calls.

The consequence you can act on: an agent that lacks RBAC permission to create a resource will fail to create it, with an ordinary authorization error. GitOps grants no exemption from any cluster control. If anything it is *more* constrained than a human operator, because its permissions are written down.

Which raises the question §4 opens with: what is this agent, exactly?

..1 §4 — An Agent That Watches a Repository

Argo CD is a "*declarative, GitOps continuous delivery tool for Kubernetes*" [source: argocd-overview-2026-08-23]. You have met the name once already, in Chapter 3, where it appeared as a promise about a sentence this chapter would retrieve [*cross-bearing: see Ch 3 §5 — the only door in*].

Here is that sentence, and it is the most important one in the section.

The Argo CD application controller "*is a Kubernetes controller which continuously monitors running applications and compares the current, live state against the desired target state (as specified in the repo)*" [source: argocd-architecture-2026-08-31].

Read it as three separate claims:

- *is a Kubernetes controller* — the thing Chapter 3 §6 defined
- *continuously monitors and compares current against desired* — the thing Chapter 3 §6 said controllers do
- *desired target state as specified in the repo* — the only part that is new

Chapter 6 told you that anyone can write a controller acting on custom resources [*cross-bearing: see Ch 6 §8 — the control loop, extended*]. Somebody did. This is what it looks like.

☆ Fixed Point

A GitOps delivery agent is a controller. Its desired state lives in a repository instead of in etcd. Nothing else about the architecture is different — not the API server's position, not the watch-based coordination, not the shape of the loop.

The object model

Argo CD's own glossary defines the pieces, and the vocabulary is worth getting exactly right because the words are ordinary English used precisely.

Application — "A group of Kubernetes resources as defined by a manifest. This is a Custom Resource Definition (CRD)" [source: argocd-core-concepts-2026-08-31]. More precisely: "The Application CRD is the Kubernetes resource object representing a deployed application instance in an environment" [source: argocd-declarative-setup-2026-08-31].

An Application is defined by two pieces of information: a "source reference to the desired state in Git (repository, revision, path, environment)" and a "destination reference to the target cluster and namespace" [source: argocd-declarative-setup-2026-08-31]. That is the whole contract: *this content, from there, goes there.*

Target state — "The desired state of an application, as represented by files in a Git repository." **Live state** — "The live state of that application. What pods etc are deployed." **Sync status** — "Whether or not the live state matches the target state. Is the deployed application the same as Git says it should be?" **Sync** — "The process of making an application move to its target state. E.g. by applying changes to a Kubernetes cluster." **Refresh** — "Compare the latest code in Git with the live state. Figure out what is different."

[source: argocd-core-concepts-2026-08-31]

Notice that refresh and sync are separate words. Refresh compares. Sync acts. A system can know it is out of agreement without doing anything about it, and whether it acts is a policy decision, which, as you are about to see, has a documented default that surprises people.

🔒 **Worth Securing:** If you find yourself confusing *live state* and *target state* under pressure, anchor them to Chapter 4. **Target state is spec, kept outside the cluster. Live state is what status reports.** Argo CD's central comparison is the one you have been reading since Chapter 4, with one operand relocated [*cross-bearing: see Ch 4 §2 — the anatomy of a record*].

Where the manifests come from

An Application's source is a repository path, but the content at that path need not be plain YAML. Argo CD accepts manifests specified "in several ways: *kustomize applications; helm charts; jsonnet files; plain directory of YAML/json manifests; any custom config management tool configured as a config management plugin*" [source: argocd-overview-2026-08-23].

Argo CD's glossary has a word for this choice: the **application source type** is *"which Tool is used to build the application,"* where a **tool** is *"a tool to create manifests from a directory of files. E.g. Kustomize,"* and a **configuration management plugin** is simply *"a custom tool"* [source: argocd-core-concepts-2026-08-31].

This is where the previous chapter's work gets collected. A Helm chart is a manifest source for a GitOps agent [cross-bearing: see Ch 14 §2 — *what a chart contains*]. So is a Kustomize overlay [cross-bearing: see Ch 14 §5 — *patching instead of templating*]. Chapter 14 gave you packaging; this chapter gives packaging somewhere to go. The agent renders the chart or builds the overlay, and compares the *result* against the cluster.

🔖 **Snag:** "Argo CD only deploys plain YAML" is a common and confident mistake, and it usually comes from having seen one tutorial. The tool's whole design assumes a rendering step: a dedicated component, the repository server, *"maintains a local cache of the Git repository holding the application manifests"* and is responsible for *"generating and returning the Kubernetes manifests"* given a repository URL, a revision, a path, and template-specific configuration [source: argocd-architecture-2026-08-31]. Rendering is not an add-on. It is a whole component.

What it tracks

An Application names a revision as well as a repository, and there are three kinds of thing that revision can be. Argo CD's tracking documentation is precise about the difference, and the difference is about stability.

A **branch or symbolic reference** (including HEAD): *"Argo CD will continually compare live state against the resource manifests defined at the tip of the specified branch or the resolved commit of the symbolic reference"* [source: argocd-tracking-strategies-2026-08-31]. The tip moves; so does your target.

A **tag**: *"If a tag is specified, the manifests at the specified Git tag will be used to perform the sync comparison."* Tags are *"generally considered more stable, and less frequently updated"* than branches [source: argocd-tracking-strategies-2026-08-31].

A **pinned commit**: *"If a Git commit SHA is specified, the app is effectively pinned to the manifests defined at the specified commit."* And the consequence: *"Since commit SHAs cannot change meaning, the only way to change the live state of an app which is pinned to a commit, is by updating the tracking revision in the application to a different commit containing the new manifests"* [source: argocd-tracking-strategies-2026-08-31].

That last sentence is principle 2, versioned and immutable, showing up as an operational property rather than an abstraction. Argo CD's own best-practices documentation makes the same argument against tracking an unstable revision: *"Since this is not a stable target, the manifests for this kustomize application can suddenly change meaning, even without any changes to your own Git repository. A better version would be to use a Git tag or commit SHA"* [source: argocd-best-practices-2026-08-31].

🧠 **Mnemonic: Branch moves. Tag rarely moves. Commit never moves.** Pick according to how much you want the deck under your production cluster to shift without a commit of your own.

Synced, and OutOfSync

Now the section's second Fixed Point, and the one most likely to be tested from this material.

"A deployed application whose live state deviates from the target state is considered OutOfSync" [source: argocd-overview-2026-08-23]. Argo CD "reports and visualizes the differences, while providing facilities to automatically or manually sync the live state back to the desired target state" [source: argocd-overview-2026-08-23].

☆ Fixed Point

OutOfSync means live state deviates from the target state in Git. It is a drift signal, not an error. Nothing has necessarily failed. A person editing an object by hand produces an OutOfSync application, and so does a commit that has not been applied yet.

Argo CD's glossary keeps these on separate lines for exactly this reason. **Sync status** answers *"is the deployed application the same as Git says it should be?"* A separate item, **sync operation status**, answers *"whether or not a sync succeeded"* [source: argocd-core-concepts-2026-08-31]. Two questions. Two answers. A sync can succeed and leave the application OutOfSync; the documentation says so outright: *"It is possible for an application to be OutOfSync even immediately after a successful Sync operation"* [source: argocd-diffing-outofsync-2026-08-31].

Return to the Logbook Entry in §3. Friday afternoon, one field, thirty seconds, invisible for six weeks. Under a GitOps agent, that edit produces an OutOfSync status within one reconciliation interval. Not an alert, not a page, not a failure. A status field, changed, saying *these two things no longer agree*. The mechanism that catches the story is not an alarm. It is a comparison that never stops running.

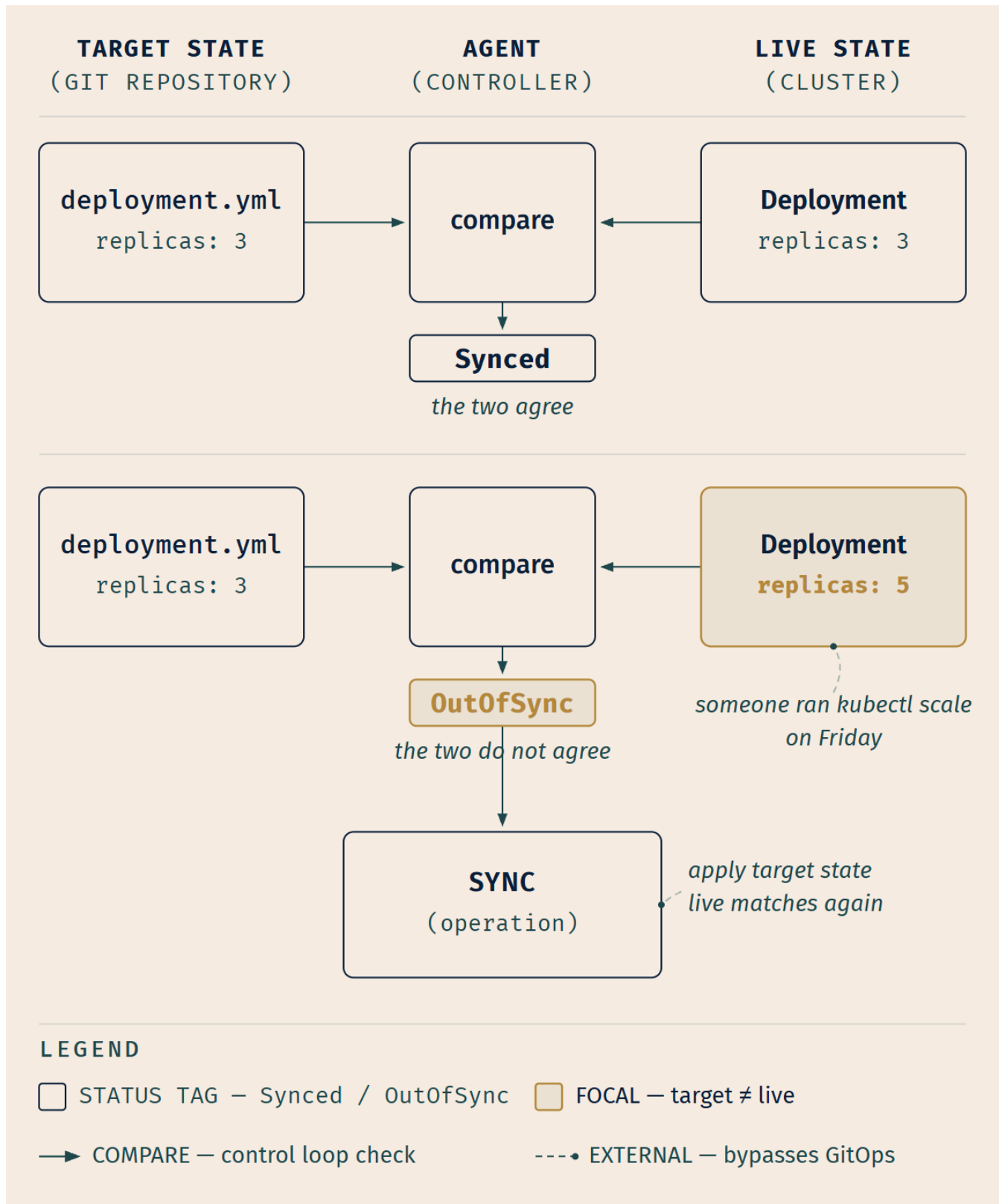


Figure 15.5 — The comparison, and the two answers it produces. The lower panel shows the cause readers do not expect: nothing failed, no deploy went wrong, a person changed the cluster. OutOfSync

reports the disagreement. Sync is the separate operation that closes it — and, by default, a separate *decision*.

Doing something about it

Reporting a difference and correcting it are separate decisions, and Argo CD keeps them separate, including in what it does when you say nothing.

"Argo CD has the ability to automatically sync an application when it detects differences between the desired manifests in Git, and the live state in the cluster" [source: argocd-auto-sync-policy-2026-08-31]. That ability is configured declaratively, which is itself in keeping, since the agent's own behavior is a field on an object in the repository.

Now the part that catches people, and it is the strongest evidence in the chapter for the Fixed Point you just read.

⚠ Navigational Hazards

Out of the box, Argo CD reports drift. It does not revert it.

Two behaviors that readers assume are automatic are documented as opt-in:

- **Self-healing is off by default.** *"By default, changes that are made to the live cluster will not trigger automated sync"* [source: argocd-auto-sync-policy-2026-08-31]. **Self-heal** is the name for the agent correcting drift it detects in the cluster, rather than only responding to new commits, and until you enable it, a hand-edited object stays hand-edited.
- **Pruning is off by default.** *"By default (and as a safety mechanism), automated sync will not delete resources when Argo CD detects the resource is no longer defined in Git"* [source: argocd-auto-sync-policy-2026-08-31]. Delete a manifest from the repository and the object stays on the cluster until pruning is enabled.

Two further facts worth carrying. Automated sync fires only on a difference: *"An automated sync will only be performed if the application is OutOfSync"* [source: argocd-auto-sync-policy-2026-08-31]. And the cadence is not instantaneous: the automated sync interval *"defaults to 120s with added jitter of 60s for a maximum period of 3 minutes"* [source: argocd-auto-sync-policy-2026-08-31].

Where people get confused: §6 will tell you that Flux *"promptly reverts"* a manual `kubectl` change [source: flux-concepts-2026-08-31], and it is tempting to generalize that into "GitOps agents revert manual edits." Two graduated implementations of the same four principles ship with different defaults here. Principle 4 requires continuous *observation*; what the agent then *does* about a difference is configuration, and the two projects choose oppositely out of the box.

That default is worth pausing on rather than merely noting, because it is the Fixed Point's best proof. If `OutOfSync` were an error, a tool would not sit there reporting it. Argo CD's out-of-the-box behavior treats

it as exactly what the Fixed Point says it is: a report that two things differ, handed to a human who decides what that means.

Beyond automated sync sit two related behaviors, both named in the feature list: **automated configuration drift detection and visualization**, and **automated or manual syncing of applications to its desired state** [source: argocd-overview-2026-08-23]. CNCF's glossary lists self-healing among GitOps's characteristic benefits, alongside *"transparency and traceability of changes, reliability and security through declarative states, and rollback, revert"* [source: cncf-glossary-gitops-2026-08-31], which brings us to the third thing in this book to wear a familiar word.

Rollback, for the third time

Chapter 6 promised you this. When it taught `kubectl rollout undo`, it said the word would appear twice more, and that the second occasion would be a delivery tool [*cross-bearing: see Ch 6 §5 — every rollout is a revision*]. Chapter 14 spent the first [*cross-bearing: see Ch 14 §3 — chart, release, revision*]. This is the second and last.

Argo CD offers *"rollback/roll-anywhere to any application configuration committed in Git repository"* [source: argocd-overview-2026-08-23].

Look at what is actually happening. You do not invoke a rollback subsystem. You change what the repository says, typically with `git revert`, producing a new commit whose content is the old content, and the agent does what it has been doing continuously since it started: it notices the target moved, and it closes the gap. The rollback mechanism *is* the sync mechanism. There is no second code path.

	What moves	Where the previous state was kept
<code>kubectl rollout undo</code> (Ch 6 §5)	The Deployment's Pod template	In the old ReplicaSet, on the cluster
<code>helm rollback</code> (Ch 14 §3)	The Helm release to a prior revision	In Helm's release history, on the cluster
Rollback by revert (here)	A commit in the repository	In the repository's history

Three mechanisms, one English word. This book writes the third as **rollback by revert**, always in that three-word form, so that it cannot be confused with the other two.

📌 **Snag:** Do not call the thing you revert a "revision." In this book, *revision* has two owners already: a Deployment revision [*cross-bearing: see Ch 6 §5 — every rollout is a revision*] and a Helm release revision [*cross-bearing: see Ch 14 §3 — chart, release, revision*]. A Git revision is a **commit**. Argo CD's own API field happens to be called *revision*, which is why this needs saying rather than assuming.

The agent's own identity

One thing remains, and two earlier chapters pointed at it explicitly [*cross-bearing: see Ch 5 §6 — a Pod's identity*] [*cross-bearing: see Ch 12 §2 — who you are*].

A delivery agent is a Pod. A Pod that talks to the API server needs an identity, and in Kubernetes that identity is a ServiceAccount. The Kubernetes documentation lists this among the reasons non-human identities exist at all: *"an external service needs to communicate with the Kubernetes API server (CI/CD pipelines)"* [source: k8s-docs-service-accounts-2026-08-23].

Then ask what this particular Pod does for a living. It creates, updates, and deletes objects — Deployments, Services, ConfigMaps, RBAC objects, custom resources — across many namespaces, on behalf of whatever is committed to a repository. Its grants must be broad, because "apply whatever the repository says" is a broad job description.

Argo CD's security documentation states the default plainly: *"By default, Argo CD uses a clusteradmin level role in order to: 1. watch & operate on cluster state 2. deploy resources to the cluster"* [source: argocd-security-cluster-credentials-2026-08-31].

The same documentation immediately supplies the reduction: *"Although Argo CD requires cluster-wide read privileges to resources in the managed cluster to function properly, it does not necessarily need full write privileges to the cluster."* Operators may edit the ClusterRole of argocd-manager-role *"such that write privileges are limited to only the namespaces and resources that you wish Argo CD to manage"* [source: argocd-security-cluster-credentials-2026-08-31].

Note the asymmetry, because it is the interesting part: cluster-wide **read** is structural, since the agent cannot detect drift in something it cannot see, while broad **write** is a default, not a requirement.

For clusters other than the one it runs in, the credentials are ordinary Kubernetes objects: *"To manage external clusters, Argo CD stores the credentials of the external cluster as a Kubernetes Secret in the argocd namespace,"* comprising *"the K8s API bearer token associated with the argocd-manager ServiceAccount created during argocd cluster add, along with connection options to that API server"* [source: argocd-security-cluster-credentials-2026-08-31].

⚠ Navigational Hazards

A delivery agent is not exempt infrastructure. It is a Pod with a ServiceAccount and, by default, cluster-admin-equivalent grants [source: argocd-security-cluster-credentials-2026-08-31], which makes it one of the highest-value subjects in the entire cluster [cross-bearing: see Ch 12 §3 — *what you may do*].

Where people get confused: the reasoning goes "GitOps is more secure than push, therefore the agent is a security improvement, therefore I do not need to think about it." The first clause is this book's blast-radius argument from §3, and it is an argument rather than a documented finding. The conclusion does not follow from it in any case. Pull moves the credentials *inside* the cluster; it does not make them smaller. Anyone who can commit to the tracked repository can, transitively, do whatever the agent may do.

Which means: in a mature GitOps setup, the repository's branch-protection rules *are* an access-control mechanism, and they are exactly as load-bearing as your RBAC policy. That is not a metaphor. Commit access to the tracked branch is cluster access, mediated by an agent that will faithfully apply whatever it finds.

Two structural points before moving on, both of which discharge promises.

First: Argo CD's `Application` is a custom resource, and installing its CRD extends the API so that `Application` objects can *exist*. Whether anything *happens* to them depends entirely on whether the application controller is running, which is the pattern Chapter 10 named, and it has no cleaner instance in this book [cross-bearing: see Ch 10 §3 — *the object is not the implementation*]. An `Application` on a cluster with no Argo CD controller is a stored document. It describes a deployment that will never occur.

Second: this is a controller acting on custom resources, which is precisely the thing Chapter 6 said you could build [cross-bearing: see Ch 6 §8 — *the control loop, extended*]. It is not a new category of technology. It is the category you were taught, aimed somewhere unexpected.

🕒 Taking Your Bearings 2: Push, Pull, and the Agent

Five questions. One reaches back.

1. A team stores all manifests in Git. Their CI pipeline, running on a hosted service, builds an image and then runs `kubectl apply -f manifests/` against the production cluster using credentials stored in the CI system's secret store. Which of the four OpenGitOps principles does this satisfy, and which does it fail?
2. An Argo CD application shows `OutOfSync`. The most recent sync operation completed successfully. Give two explanations that require nothing to have failed, and say where a *failure* would have been reported instead.

3. Two teams run identical workloads. Team A uses push-based CD from a shared CI system that also deploys to eleven other clusters. Team B runs an in-cluster agent per cluster. An attacker obtains full control of the CI system. Describe the difference in what they can now reach, using the term this book gives it.
4. [retrieval: ch12] A delivery agent needs to create Deployments and Services in namespaces it does not own, and — where pruning is enabled — delete resources that have been removed from the repository. Name the two kinds of object that must exist before any of that is permitted, and say which one determines the *scope* across namespaces.
5. An Application tracks the main branch. A colleague argues the team should track a Git tag instead. What changes about the system's behavior, and which OpenGitOps principle is the argument appealing to?

Answers with explanations

1. Satisfies 1 (Declarative) and 2 (Versioned and Immutable). Fails 3 (Pulled Automatically) and 4 (Continuously Reconciled).

Manifests are declarative and stored in Git with history, so the first two hold. Principle 3 requires that *"software agents automatically pull the desired state declarations from the source"* [source: opengitops-principles-v1-2026-08-31]; here the pipeline pushes, from outside. Principle 4 requires agents to *continuously* observe and apply; this pipeline runs once per commit and then exits, leaving nothing observing between runs.

This is a perfectly good delivery system. It is not GitOps. Storing manifests in Git is necessary and nowhere near sufficient, which is exactly the confusion the term is meant to prevent.

2. Two benign explanations:

A person changed the cluster: somebody edited a live object directly with `kubectl`. Live state now deviates from target state, so the status is `OutOfSync` [source: argocd-overview-2026-08-23]. The last sync succeeded; it just happened before the edit. And by default this state persists, because *"changes that are made to the live cluster will not trigger automated sync"* [source: argocd-auto-sync-policy-2026-08-31].

A commit has not landed yet: a new commit arrived in the tracked path and has not been applied. The target moved; live state has not caught up. Nothing has failed — this is the normal transient state between commit and sync.

Also benign, and worth knowing: the documentation states outright that *"it is possible for an application to be `OutOfSync` even immediately after a successful `Sync` operation"* [source: argocd-diffing-outofsync-2026-08-31].

Where a failure would show: in **sync operation status**, which is a separate glossary entry answering a separate question — *"whether or not a sync succeeded"* — as against sync status, which answers *"is the deployed application the same as Git says it should be?"* [source: argocd-core-concepts-2026-08-31]. Reading `OutOfSync` as a failure report is the misconception this item targets.

3. Blast radius, which is this book's term for how far one compromise reaches. Compromising Team A's CI system yields write credentials to twelve clusters, because that is what the system holds in order to do its job. Compromising the same system for Team B yields the ability to build and publish images, which is bad and a real supply-chain problem, but not cluster-write credentials, because Team B's clusters hold their own credentials internally and the CI system never had any.

Pull does not prevent the compromise. It bounds what one compromise reaches. Note that this is the book's architectural argument rather than a claim any cached source makes; what the sources do establish is where each arrangement stores its credentials.

4. [retrieval: ch12] A ServiceAccount and an RBAC binding (with its Role or ClusterRole). The ClusterRoleBinding determines cross-namespace scope.

The ServiceAccount is the identity; the binding attaches a permission set to it. Because the work spans namespaces the agent does not own, the grant must be cluster-scoped, a ClusterRole bound by a ClusterRoleBinding, rather than a Role in each namespace [*cross-bearing: see Ch 12 §3 — what you may do*]. Argo CD's own documentation confirms the shape: it uses a `argocd-manager` ServiceAccount, and reduction is done by editing the ClusterRole `argocd-manager-role` [source: argocd-security-cluster-credentials-2026-08-31].

Why "just a ServiceAccount" is wrong: a ServiceAccount alone gets almost nothing. Identity without a binding is a name with no permissions attached.

Why the pruning qualifier is in the stem: permission to delete and *configuration* to delete are different things. The agent must be permitted before it can be configured, but permission alone does not make it prune — *"by default (and as a safety mechanism), automated sync will not delete resources when Argo CD detects the resource is no longer defined in Git"* [source: argocd-auto-sync-policy-2026-08-31].

5. Behavior change: the target stops moving on its own. With a branch, Argo CD compares against *"the tip of the specified branch"*, which advances with every merge [source: argocd-tracking-strategies-2026-08-31]. With a tag, the target is fixed until someone deliberately moves the tag or changes what the Application tracks; tags are *"generally considered more stable, and less frequently updated"* [source: argocd-tracking-strategies-2026-08-31].

The principle: 2, Versioned and Immutable — *"Desired state is stored in a way that enforces immutability, versioning and retains a complete version history"* [source: opengitops-principles-v1-2026-08-31]. Argo CD's best-practices page makes exactly this argument, noting that an unstable revision means manifests *"can suddenly change meaning, even without any changes to your own Git repository"* [source: argocd-best-practices-2026-08-31].

If you got 4–5: You have the chapter's core. What remains is ordering, alternatives, and the synthesis.

If you got 2–3: Re-read §3's four principles and §4's OutOfSync Fixed Point.

If you got 0–1: Go back to **§3** and read it properly before continuing. §5 and §6 assume the push/pull distinction as settled ground.

§5 — Ordering the Sync

This section is marked Advanced, and the marking is honest: it goes somewhat deeper than an associate credential is likely to reach. It is here because the chapter raises a question it would be unsatisfying to leave hanging, and because a promise from Chapter 12 lands precisely on it.

The question is ordering.

Chapter 14 opened by listing what a folder of YAML fails to give you, and one of the four was apply ordering: `kubectl apply -f` over a directory offers no guarantee about which object lands first [*cross-bearing: see Ch 14 §1 — why a folder of YAML stops working*]. Helm's `crds/` directory exists partly to address one instance of this [*cross-bearing: see Ch 14 §6 — which one, when*].

GitOps does not inherit that fix. It inherits the problem, at a larger scale. An agent reconciling an entire repository against an entire cluster faces the same ordering question about far more objects: a namespace before the things inside it, a CustomResourceDefinition before any custom resource that uses it, a database migration before the version of the application that expects the new schema.

Argo CD's answer has two levels, and the two are independent.

Phases

A sync runs in phases, and hooks attach to them:

- **PreSync** — hooks run *"prior to the application of the manifests"*
- **Sync** — hooks run *"after all PreSync hooks completed and were successful, at the same time as the application of the manifests"*
- **PostSync** — hooks run *"after all Sync hooks completed and were successful, a successful application, and all resources in a Healthy state"*
- **SyncFail** — hooks run *"when the sync operation fails"*

[source: argocd-sync-phases-and-waves-2026-08-31]

Phase execution is strictly ordered and gated on success. PreSync-marked resources are applied first, and if any fail the process stops. Sync-marked resources are applied next; a failure marks the operation failed and also triggers SyncFail hooks. PostSync hooks run last, and their failure marks the deployment failed [source: argocd-sync-phases-and-waves-2026-08-31].

Read PreSync as *"this must be finished before anything else changes"*: the database migration, the schema check. Read PostSync as *"this runs only if everything else worked and is healthy"*: the smoke test, the notification, the traffic cutover.

That last one connects back. Argo CD's feature list ties hooks directly to §2's vocabulary: *"PreSync, Sync, PostSync hooks to support complex application rollouts (e.g. blue/green and canary upgrades)"* [source: [argocd-overview-2026-08-23](#)]. Section 2 told you blue/green and canary need tooling above the Deployment. Hooks are part of how that tooling is built; a PostSync hook is a natural place for "now switch the traffic."

Waves

Phases are coarse. Within a phase you frequently need finer ordering, and that is what waves provide.

Resources carry an integer that determines their order within a phase, and Argo CD's rule for it is short: *"Hooks and resources are assigned to wave 0 by default. The wave can be negative, so you can create a wave that runs before all other resources"* [source: [argocd-sync-phases-and-waves-2026-08-31](#)]. Lower values sync first. The mechanism is an annotation, `argocd.argoproj.io/sync-wave`, taking an integer value [source: [argocd-sync-phases-and-waves-2026-08-31](#)], noted for concreteness, not for memorization.

The ordering algorithm, in order of precedence, begins: *"1. The phase 2. The wave they are in (lower values first)"*, with two further deterministic tie-breaks after those [source: [argocd-sync-phases-and-waves-2026-08-31](#)]. Phase first, then wave. Everything you can control is in those two lines.

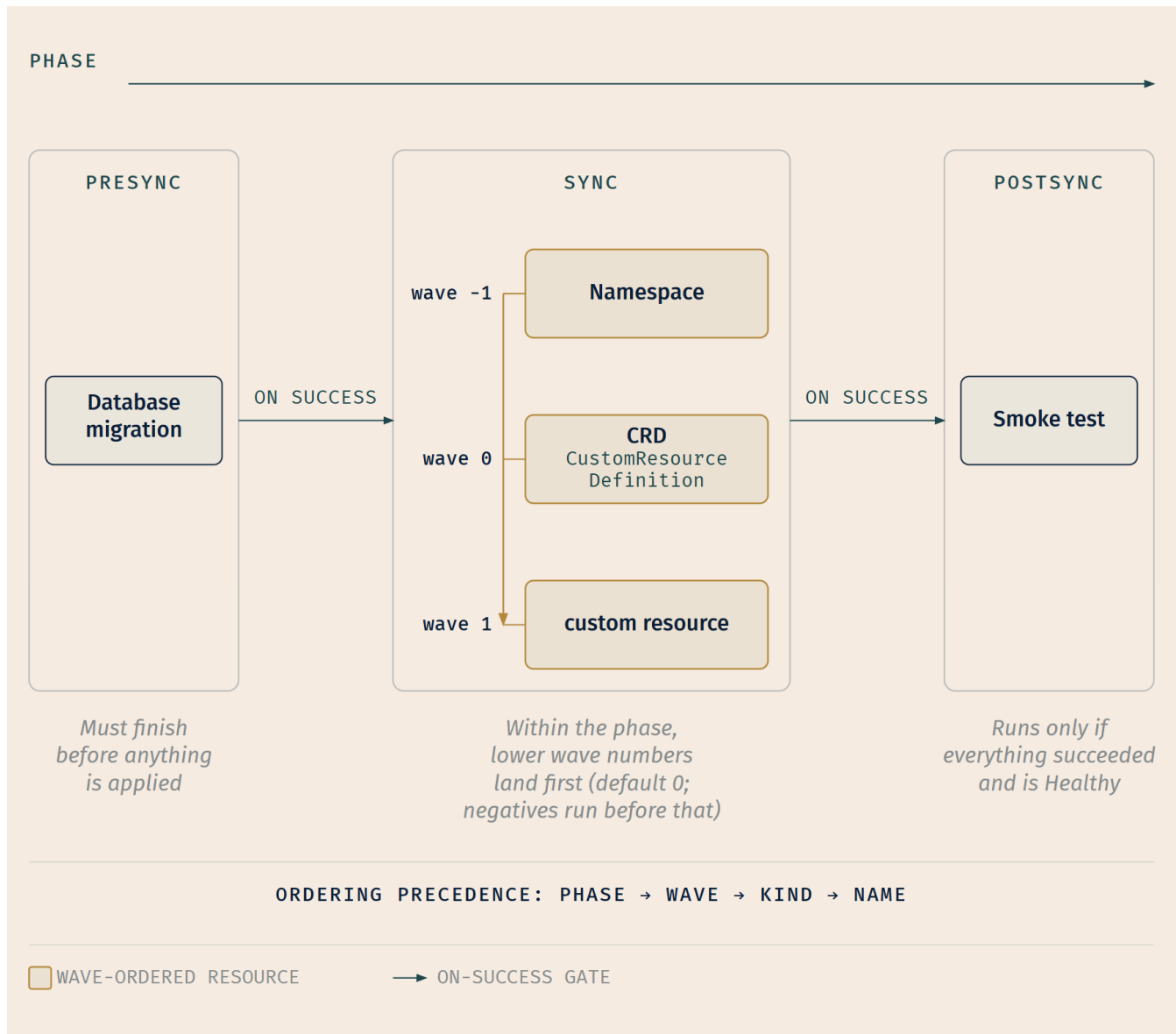


Figure 15.6 — Two orderings, nested. The horizontal axis is the phase; the vertical axis inside sync is the wave. The example is the ordering problem this section opened with: a namespace must exist before the CustomResourceDefinition, which must exist before any custom resource that uses it.

Chapter 12 pointed here for a specific reason, and it is a good illustration of why ordering is not a theoretical concern.

Chapter 12 taught that **"After you create a binding, you cannot change the Role or ClusterRole that it refers to"** [source: k8s-docs-rbac-2026-08-23] — the binding's *role reference* is fixed, though its list of subjects is not [*cross-bearing; see Ch 12 §3 — a binding cannot be retargeted*]. So retargeting a RoleBinding is not an update at all. It is a delete and a create, with a window in between during which the subject holds nothing.

That is the general shape, and it is why ordering stops being theoretical. Some objects cannot simply be updated in place: they must be deleted and recreated. Under a system that reconciles a whole repository against a whole cluster in one operation, that is a real ordering constraint. The delete must precede the create, and anything depending on the result must come after both. Waves are how you say so.

What to take from this section. Phases run in a fixed order and are gated on success. Waves order resources within a phase, lower numbers first, defaulting to zero and permitting negatives. Ordering is a problem GitOps has and a single `kubectl apply` merely ignores.

That is the whole of what this section owes you. The annotation is here for concreteness, not for memorization; you do not need to be able to write one from memory, and nothing in this chapter's questions asks you to.

..1 §6 — The Other Agent, and More Than One Cluster

Argo CD is not the only implementation, and the alternative is worth meeting. Not because you need to operate it, but because the two make genuinely different design choices and the contrast sharpens what "GitOps agent" means.

Flux is a GitOps Toolkit: *"a collection of specialized tools, Flux Controllers, composable APIs, and reusable Go packages"* [source: flux-concepts-2026-08-31]. The earlier phrasing was blunter: *"Flux is a GitOps Toolkit: a set of composable APIs and specialized tools that can be used to build Continuous Delivery on top of Kubernetes"* [source: flux-concepts-2026-08-23].

That word *composable* is the whole contrast. Argo CD presents as one integrated product, with a single Application resource binding source to destination and one UI over everything, though "integrated" does not mean "one process," since §4 already showed you three components behind it [source: argocd-architecture-2026-08-31]. Flux presents as a set of controllers you assemble, each owning its own custom resources:

Controller	Its custom resources
Source	GitRepository, OCIRepository, HelmRepository, HelmChart, Bucket, and others
Kustomize	Kustomization
Helm	HelmRelease
Notification	Provider, Alert, Receiver
Image Reflector / Image Automation	ImageRepository, ImagePolicy, ImageUpdateAutomation

[source: flux-components-2026-08-31]

The table names representative resources per controller rather than a complete inventory; the source controller alone carries seven [source: flux-components-2026-08-31]. What is worth carrying is the shape —

one controller per concern, each with its own API — not the roster.

Neither posture is better. Argo CD's integration gives you one thing to learn and a UI that shows you everything at once. Flux's composition gives you pieces you can adopt separately and replace individually. Teams pick on organizational grounds more than technical ones.

Sources as a first-class concept. Flux elevates the idea of where-state-comes-from into its own API: "A *Source defines the origin of a repository containing the desired state of the system and the requirements to obtain it (e.g. credentials, version selectors)*" [source: flux-concepts-2026-08-31]. The source controller produces an artifact; other controllers consume it.

Notice `OCIRepository` and `HelmRepository` in that list. Chapter 14 taught you that OCI registries can hold charts [cross-bearing: see Ch 14 §4 — *where charts come from*], and Flux treats that as one source kind among several. Its `Kustomize` controller consumes overlays [cross-bearing: see Ch 14 §5 — *patching instead of templating*], and its `Kustomization` API "defines a pipeline for fetching, decrypting, building, validating and applying `Kustomize` overlays or plain Kubernetes manifests" [source: flux-kustomization-api-2026-08-31].

The most concrete statement of principle 4 in this chapter. Flux's own documentation gives reconciliation a number and a consequence:

"The reconciliation runs every five minutes by default, but this can be changed with `.spec.interval`." And: "If you make any changes to the cluster using `kubectl edit/patch/delete`, they will be promptly reverted" [source: flux-concepts-2026-08-31].

One qualification belongs with that figure. The `Kustomization` API reference states that `.spec.interval` is "a required field that specifies the interval at which the `Kustomization` is reconciled" with a minimum value of 60 seconds [source: flux-kustomization-api-2026-08-31], and declares no API-level default. So "five minutes by default" describes the `Kustomization` that Flux's own bootstrap generates, not a default the API supplies. The interval is always explicitly configured; five minutes is what it is usually configured to.

🔒 **Worth Securing:** *"If you make any changes to the cluster using `kubectl edit/patch/delete`, they will be promptly reverted"* [source: flux-concepts-2026-08-31]. Read that as principle 4 with the abstraction removed. Continuous reconciliation is not a scheduling detail. It means your manual change has a shelf life measured in minutes. This is the same property that makes a `ReplicaSet` recreate a Pod you deleted, and note that Argo CD, out of the box, does not do this [source: argocd-auto-sync-policy-2026-08-31]. Two graduated projects, four shared principles, opposite defaults.

Bootstrap. Flux installs itself the way it installs everything else. *"The process of installing the Flux components in a GitOps manner is called a bootstrap. The manifests are applied to the cluster, a `GitRepository` and `Kustomization` are created for the Flux components, then the manifests are pushed to an existing Git repository (or a new one is created)"* [source: flux-concepts-2026-08-31]. The 2026-08-23 capture puts the consequence plainly: *"Flux manages itself like any other resource"* [source: flux-concepts-2026-08-23].

Sit with that for a moment, because it is the sort of thing that either seems trivial or seems remarkable depending on how carefully you read it. Upgrading Flux is a commit. Reconfiguring Flux is a commit. The agent's own desired state lives in the repository the agent watches, and the agent applies it to itself.

Where the credentials live in Flux. The security model differs from Argo CD's in an interesting way. Flux installs a `crd-controller` ClusterRole with *"full access to all the Custom Resource Definitions defined by Flux controllers,"* and a `cluster-reconciler` ClusterRoleBinding referencing the `cluster-admin` ClusterRole, *"bound to service accounts for only kustomize-controller and helm-controller,"* because those two *"are the only two controllers that manage resources in the cluster"* [source: flux-security-2026-08-31].

The reduction path is impersonation rather than narrowing: *"In a soft multi-tenancy setup, Flux does not reconcile a tenant's repo under the cluster-admin role. Instead, you specify a different service account in your manifest, and the Flux controllers will use the Kubernetes Impersonation API under cluster-admin to impersonate that service account"* [source: flux-security-2026-08-31]. The Kustomization API exposes this directly: `.spec.serviceAccountName` specifies *"the ServiceAccount to be impersonated while reconciling"* [source: flux-kustomization-api-2026-08-31].

Same problem as §4's, a broadly privileged agent, solved by a different route. Argo CD narrows the ClusterRole; Flux keeps the broad role and impersonates a narrower identity per workload.

More than one cluster. Which brings us to the question that shows the two designs most clearly. Where does desired state live when there are twenty clusters?

Argo CD's answer is documented and is a control point: among its features is the *"ability to manage and deploy to multiple clusters"* [source: argocd-overview-2026-08-23], with each external cluster's credentials stored as a Secret in the `argocd` namespace of the managing cluster [source: argocd-security-cluster-credentials-2026-08-31]. One Argo CD, many destinations, one place to look, and one place holding credentials to everywhere.

Flux's documented position is narrower, and the honest statement is the narrow one: Flux's reconciling controllers run *in* the cluster they reconcile [source: flux-security-2026-08-31], and bootstrap installs Flux into a cluster against a Git repository [source: flux-concepts-2026-08-31]. There is no equivalent documented mechanism for one Flux storing credentials to other clusters.

The trade is §3's blast-radius argument at a larger scale, and it remains this book's argument rather than a documented finding: a single control point gives you one console to reason about and one component whose compromise reaches every destination it holds credentials for. Per-cluster agents give you isolation and no unified view.

🔍 **Closer Look:** Argo and Flux are both CNCF **graduated** projects [source: cncf-project-maturity-levels-2026-08-23], the maturity tier CNCF describes as stable, widely adopted, and production ready [source: cncf-project-maturity-levels-2026-08-23]. What the levels *mean* is Chapter 17's subject, and that is the durable thing to know [*cross-bearing: see Ch 17 §2 — sandbox, incubating, graduated, and who decides*]. The roster of which projects currently hold which level is dated data that changes; do not memorize it.

🕒 Taking Your Bearings 3: Ordering, and the Other Agent

Five questions. The last one is the most important retrieval item in this chapter. Read it carefully, because the next section is about to depend on your answer.

1. A repository contains a CustomResourceDefinition and a custom resource that uses it. Applied together with no ordering, the custom resource is rejected. Which object must land first, what mechanism expresses that, and what ordering value does a resource with no ordering annotation receive?
2. A team needs a database schema migration to complete before any new application Pod starts, and a smoke test to run only after everything is up and healthy. Which hook phase does each belong to, and what does the second one additionally require before it runs?
3. Describe the structural difference between Argo CD's and Flux's design posture in one sentence each, and name one consequence of that difference for a team adopting either.
4. A colleague fixes a production incident with `kubect1 patch` on a cluster managed by Flux. They do not commit the change. What happens, and which OpenGitOps principle is responsible? Then say why the same edit on an out-of-the-box Argo CD installation behaves differently.
5. [retrieval: ch3] In your own words: what does a controller do, what two things does it compare, and how often does it do it? Answer without mentioning Git, repositories, or delivery.

Answers with explanations

1. The CustomResourceDefinition must land first, because a custom resource of a kind the API server does not yet recognize is rejected. The mechanism is **sync waves** — resources sync "*lower values first*" within a phase, and negative waves exist precisely so you can run something before everything else [source: argocd-sync-phases-and-waves-2026-08-31]. Give the CRD a lower wave than the custom resource; the absolute numbers do not matter, only the relative order.

A resource with no ordering annotation lands in **wave 0**: "*Hooks and resources are assigned to wave 0 by default*" [source: argocd-sync-phases-and-waves-2026-08-31].

A common wrong answer: that waves are absolute positions rather than relative ordering. They are not slots to be filled, and there is no requirement that you use consecutive integers or start anywhere in particular. -5 and 0 order exactly as 0 and 1 do. Readers who treat the number as a position rather than a sort key end up inventing constraints the mechanism does not have.

2. Migration → PreSync. Smoke test → PostSync.

PreSync hooks run *"prior to the application of the manifests"*, before any new Pod exists [source: argocd-sync-phases-and-waves-2026-08-31].

PostSync additionally requires health, not merely completion. It runs *"after all Sync hooks completed and were successful, a successful application, and all resources in a Healthy state"* [source: argocd-sync-phases-and-waves-2026-08-31]. That health gate is the part readers miss: PostSync is not "after the apply," it is "after the apply worked and the result is healthy."

3. Argo CD is integrated — one product with one Application resource binding source to destination, and a UI over the whole thing. **Flux is composable** — *"a collection of specialized tools, Flux Controllers, composable APIs"* [source: flux-concepts-2026-08-31], each with its own custom resources [source: flux-components-2026-08-31].

Any of these consequences earns credit: Flux lets you adopt or replace pieces independently while Argo CD is more nearly all-or-nothing; Argo CD gives one console showing every application while Flux's state is spread across controllers; Flux's per-controller APIs mean more objects to learn, Argo CD's integration means fewer.

A common wrong answer: reading "composable" as "less capable," or "integrated" as "one process."

Neither follows. Composability is about how the pieces are packaged and adopted, not about what they can do, and Argo CD's integration sits on top of three distinct components, an API server, a repository server, and an application controller [source: argocd-architecture-2026-08-31]. "Integrated" describes the product surface, not the process count.

4. The change is reverted, by principle 4 (Continuously Reconciled).

Flux states the behavior directly: *"if you make any changes to the cluster using kubectl edit/patch/delete, they will be promptly reverted"* [source: flux-concepts-2026-08-31]. The interval is typically five minutes [source: flux-concepts-2026-08-31] and is always explicitly configured, with a 60-second minimum and no API-level default [source: flux-kustomization-api-2026-08-31], so the durable answer is the behavior, not the number. Principle 4 requires that *"software agents continuously observe actual system state and attempt to apply the desired state"* [source: opengitops-principles-v1-2026-08-31]. The uncommitted patch is not the desired state, so it is not what the agent applies.

Why Argo CD differs out of the box: self-healing is opt-in. *"By default, changes that are made to the live cluster will not trigger automated sync"* [source: argocd-auto-sync-policy-2026-08-31]. The edit produces an OutOfSync status and stays put until somebody acts or self-heal is enabled. Both projects implement principle 4's *observation*; they differ on what they do about what they observe.

Practical consequence worth stating: under continuous reconciliation with self-heal on, an emergency fix must be committed to survive. The tool is not being obstructive; it is doing exactly the job it was installed for. A team that finds this intolerable during incidents needs a documented way to suspend reconciliation, not a workaround.

5. [retrieval: ch3] A controller compares desired state against current state, and acts to close the gap. It does this continuously, in a loop, indefinitely — not once at creation, and not only when something triggers it. If the two disagree it takes whatever action moves current toward desired, then it checks again *[cross-bearing: see Ch 3 §6 — controllers and the control loop]*.

A common wrong answer: describing the loop as event-driven — "it runs when something changes." That inverts the architecture. The controller is not woken by a notification that work is waiting; it observes state and decides for itself, on its own schedule, whether there is a gap. The difference matters in the next section, where the thing being observed lives somewhere unexpected and the loop does not care.

If that came back clean, keep it loaded. The next section is one substitution away from it.

If it did not come back clean, **re-read Ch 3 §6 now, before §7**. This is not a formality. The final section owns no new material; its entire content is a change made to the thing you just tried to state, and if the original is fuzzy, the change is invisible.

☼ §7 — The Control Loop, Pointed at a Repository

This section teaches nothing new.

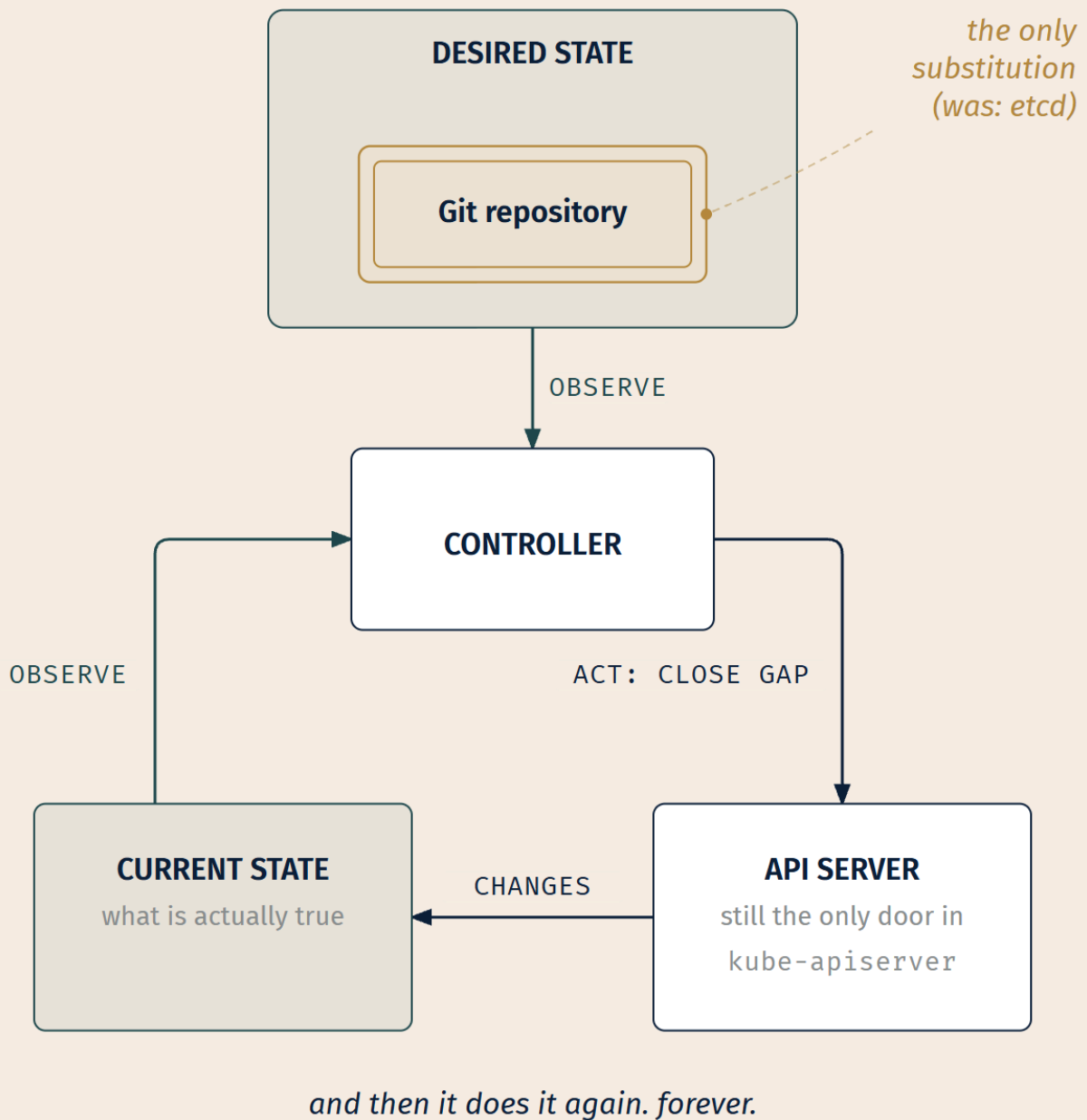
That is not modesty and it is not a warning; it is the design. Everything below has already been taught, most of it in Chapter 3, some of it as recently as four pages ago. What happens here is a single substitution, performed slowly, in front of you.

Here is the loop, as Chapter 3 gave it to you.

A controller holds a **desired state**, a record of what should be true. It observes the **current state**, what is actually true. When the two differ, it takes action to close the gap. Then it checks again. It does not stop. There is no completion condition, no final state, no moment at which the controller decides its work is finished and exits.

In Chapter 3, the desired state was in etcd, reached through the API server.

Now move it. Take the desired state out of etcd and put it in a Git repository.



LEGEND



ONLY SUBSTITUTION

— OBSERVE

— ACT / WRITE

Figure 15.7 — The control loop, pointed at a repository. Lay this beside Chapter 3's control-loop figure and the point is that it is the same loop. The controller sits in the same place, the API server is still the only door in, and the arrows run in the same directions. One box changed contents.

Look at what did *not* move.

The API server is still the only mutator. The agent writes to the cluster the way everything else does: as an API client, through authentication, authorization, and admission [*cross-bearing: see Ch 3 §5 — the only door in*].

The coordination is still watching, not telling. Nothing pushes work to the controller. It observes and decides. Chapter 3 called this out as the architecture's central property, and said it would be retrieved: the same architecture, with a different thing in the hub position.

The controller is still shaped the same way. Compare, act, repeat. No new lifecycle, no new mechanism, no new category of component.

Reconciliation is still endless. A ReplicaSet does not finish. Neither does this.

One box changed contents. That is the entire technical delta between "Kubernetes" and "GitOps."

The four principles, re-read

Now the four principles from §3, which looked like a list when you met them and are not one.

1. Declarative. This is Chapter 4. You file a declaration; the system's job is to make it true [*cross-bearing: see Ch 4 §1 — you file a declaration*]. Nothing added.

2. Versioned and immutable. This is the only genuinely new thing in the whole model, and it is a property of the *store*, not of Kubernetes. etcd holds the current desired state. A repository holds every desired state there has ever been, in order, each one fixed. That difference is what makes rollback by revert possible at all: you cannot return to a state your store did not keep.

3. Pulled automatically. This is Chapter 3's watch [*cross-bearing: see Ch 3 §5 — the only door in*]. Agents observe a source and fetch from it. The direction of travel was never inward.

4. Continuously reconciled. This is the loop itself. Word for word, it is what Chapter 3 §6 told you a controller does.

Three of the four were already yours. You have been running GitOps-shaped machinery since Chapter 3; the principles simply name what happens when the desired-state store is one a human can read, review, and revert.

Which kills the misreading this chapter was built to prevent.

☆ Fixed Point

GitOps is not "running CI from Git." None of the four principles mentions integration, building, testing, or a pipeline. All four are about desired state — how it is expressed, how it is stored, how it is obtained, and how it is applied. The build is somebody else's job, and Kubernetes said so in its own documentation: it *"does not deploy source code and does not build your application"* [source: k8s-docs-overview-2026-08-23].

Why the chapter is called what it is

The chart is the truth, but not because a file is inherently authoritative. Files are only ever claims. A YAML manifest on somebody's laptop is a claim nothing enforces; a repository full of manifests that nothing watches is a folder of hopeful documents, which is precisely what Chapter 14 left you holding.

The chart is the truth because **something is continuously making it true**. The authority is not in the file. It is in the loop that never stops comparing the file to the world and acting on the difference.

That is what you have been building toward since Chapter 3, whether or not it looked like it. And it is why Chapter 6 said what it said when it finished teaching the control loop: that when you met this, it would look like a new idea for about ten seconds [*cross-bearing: see Ch 6 §9 — nobody sails one pod*].

Ten seconds is about right.

⚓ Safe Harbor

Checkpoint: You've Now Mastered

✓ The twelve factors, sorted by who solves them — and the four that matter most in a cluster ✓ Deployment strategy vocabulary, and the line between a Deployment field and a pattern needing tooling ✓ Push versus pull as an architectural question about where credentials sit and how far one compromise reaches ✓ The four OpenGitOps principles, and why "pulled" and "continuously" carry the definition ✓ Argo CD as a controller: Application, manifest sources, tracking targets, Synced and OutOfSync ✓ What Argo CD does by default when it sees drift — report it, not revert it — and why that is the Fixed Point's best proof ✓ Rollback by revert, and why it is the third mechanism to wear that word ✓ The delivery agent's identity, its default grants, and why commit access is cluster access ✓ Sync phases and waves, and the ordering problem they exist for ✓ Flux's composable posture, self-bootstrap, and the opposite default it ships with ✓ The control loop, pointed at a repository — and everything about it that did not change

Exam Alert! 🚨

High-Priority Topics

- 1. The four OpenGitOps principles, in order, with the words that matter.** Declarative; versioned and immutable; **pulled** automatically; continuously reconciled [source: opengitops-principles-v1-2026-08-31]. *Pulled* and *continuously* carry the distinction. A definition missing either one describes something that is not GitOps.
 - 2. OutOfSync is a drift signal, not an error.** It reports that live state deviates from the target state in Git [source: argocd-overview-2026-08-23]. A person editing the cluster produces it. Nothing failed, and out of the box Argo CD reports it rather than reverting it [source: argocd-auto-sync-policy-2026-08-31].
 - 3. Argo CD is a Kubernetes controller.** It *"continuously monitors running applications and compares the current, live state against the desired target state"* [source: argocd-architecture-2026-08-31]. It does not bypass the API server, and it is not a new category of technology.
 - 4. Deployment strategy vocabulary versus Deployment fields.** RollingUpdate and Recreate are values on a Deployment [source: argo-rollouts-strategies-2026-08-23]. Blue/green and canary are patterns implemented by tooling above it.
-

Common Traps — these are distinctions that are easy to confuse, and they are the ones this material rewards getting right.

The trap	The correct understanding
"GitOps means running CI from Git"	GitOps is four principles about <i>desired state</i> . Continuous integration is not one of them, and the cluster does not care who built the artifact.
Assuming a pipeline pushes to the cluster	Principle 3 is explicit: agents pull desired-state declarations from the source [source: opengitops-principles-v1-2026-08-31]. Push-based CD is not GitOps, whatever its manifests are stored in.
Treating reconciliation as a deploy-time event	Principle 4 is continuous and indefinite, the same property that makes a ReplicaSet recreate a Pod you deleted last Tuesday.
"Every GitOps agent reverts manual changes"	Flux does, promptly [source: flux-concepts-2026-08-31]. Argo CD, by default, does not — <i>"changes that are made to the live cluster will not trigger automated sync"</i> [source: argocd-auto-sync-policy-2026-08-31]. Same principles, opposite defaults.
"OutOfSync means the sync failed"	Sync status and sync <i>operation</i> status are two different fields answering two different questions [source: argocd-core-concepts-2026-08-31]. An application can be OutOfSync immediately after a successful sync [source: argocd-diffing-outofsync-2026-08-31].
"Argo CD only deploys plain YAML"	Kustomize applications, Helm charts, Jsonnet, plain YAML/JSON directories, and custom config-management plugins [source: argocd-overview-2026-08-23].
"Argo CD can only track a branch"	Branches, tags, or a pinned Git commit [source: argocd-tracking-strategies-2026-08-31].
Assuming a GitOps agent writes to the datastore directly	It is an API client like any other, subject to authentication, authorization, and admission. Chapter 3's claim about the only door in is not suspended [<i>cross-bearing: see Ch 3 §5 — the only door in</i>].
Assuming a delivery agent needs no identity because it is "infrastructure"	It is a Pod with a ServiceAccount and, by default, cluster-admin-level grants [source: argocd-security-cluster-credentials-2026-08-31] — one of the highest-value subjects in the cluster.
Collapsing the third rollback into one of the first two	rollout undo restores a Pod template. helm rollback returns a release to a prior revision. Rollback by revert changes a commit and lets the agent reconcile. Three mechanisms, one word.
Assuming a Deployment can express blue/green or canary by itself	Both need something above the Deployment; canary additionally needs traffic splitting and metric analysis [source: argo-rollouts-strategies-2026-08-23].

Practice Questions

Twenty-one questions. Several present a situation rather than asking for a definition; those are the ones worth slowing down for, because a glossary cannot answer them.

Q1. An application writes its logs to `/var/log/app.log` inside the container and rotates them itself with a background thread. Which twelve-factor factor does this violate, and what capability does it break?

Q2. According to the twelve-factor methodology, which of the following is the *strongest* evidence that an application has correctly factored out its config?

A) It has no configuration at all; every value is a compile-time constant B) Its codebase could be made open source at any moment without compromising credentials C) It has separate configuration bundles named `staging` and `production` D) It reads all settings from a `config.yaml` file at startup

Q3. A team argues that because their application is containerized and deployed by a Deployment, it is twelve-factor compliant. The application holds user session state in process memory. Evaluate the claim.

Q4. Which strategy guarantees that two versions of an application never run simultaneously, and what does it cost?

A) RollingUpdate — costs additional capacity B) Canary — costs traffic-splitting infrastructure C) Recreate — costs downtime D) Blue/green — costs double capacity

Q5. A team runs a queue-consuming worker with no inbound HTTP traffic. They want to validate a new version against production configuration before it processes real work. Which strategy fits, and why is canary a poor choice here?

Q6. A team tells you they have configured blue/green deployment "in the Deployment spec." Without seeing their manifests, what do you already know is wrong with that description, and what must actually exist for them to be running blue/green?

Q7. Progressive delivery is defined as releasing updates in a controlled and gradual manner, *"typically coupling automation and metric analysis to drive the automated promotion or rollback of the update"* [source: argo-rollouts-strategies-2026-08-23]. Which half of that definition distinguishes progressive delivery from simply deploying slowly, and why?

Q8. A colleague who has just installed Argo CD says: "so this is a new kind of thing — a deployment engine that sits outside the normal Kubernetes machinery and pushes state in." Correct them in structural terms. What *category* of component is the Argo CD application controller, what two things does it compare, and what single element of the Chapter 3 architecture is different?

Q9. Which statement about push-based delivery is accurate?

A) It requires the cluster's API server to be reachable from the public internet B) It stores cluster-write credentials outside the cluster C) It is incompatible with storing manifests in Git D) It bypasses the Kubernetes API server

Q10. Explain "blast radius" as this book uses it, using a compromised CI system as the example. Say what pull-based delivery does and does not change about it, and note what part of the argument is the book's rather than a documented finding.

Q11. Which principle is violated by a system that stores declarative manifests in Git, has an in-cluster agent fetch them, applies them on each new commit, and then does nothing until the next commit?

A) Declarative B) Versioned and immutable C) Pulled automatically D) Continuously reconciled

Q12. Kubernetes documentation states that it *"does not deploy source code and does not build your application"* [source: k8s-docs-overview-2026-08-23]. What does this establish about the relationship between CI and GitOps?

Q13. An Argo CD Application reports `OutOfSync`, and the most recent sync operation reports success. Which is true?

A) The report is contradictory; one of the two must be stale B) Sync status and sync operation status answer different questions, and both readings are valid simultaneously C) `OutOfSync` overrides the operation status; the sync did not actually complete D) The application will self-heal automatically, so the report can be disregarded

Q14. An Argo CD Application points at a repository path containing a Helm chart. A colleague says this cannot work because "Argo CD is a YAML tool, and Helm is a separate deployment mechanism." Correct them, naming the component involved.

Q15. A team pins an Application to a Git commit SHA. Their build pipeline pushes new commits to the tracked branch daily. What happens to the cluster, and what would have to change for it to update?

Q16. [cross-domain: D2.2] A GitOps agent must create Deployments and Services in twelve namespaces it does not own, and — with pruning enabled — delete resources removed from the repository. What must exist for this to be permitted, and why is a Role in each namespace a poor answer?

Q17. [cross-domain: D1.1] An engineer describes `OutOfSync` as "the status field not matching the spec field." Is this a good analogy? Explain precisely what is being compared and where each operand lives.

Q18. Three things in this book are called rollback. Name the mechanism each one uses and where each keeps the state it returns to.

Q19. A repository contains a namespace, a CustomResourceDefinition, and a custom resource of the type that CRD defines. Applied simultaneously, this fails. What must land in what order, what mechanism expresses that ordering, and what happens to an object with no ordering annotation?

Q20. Which hook phase runs *"after all Sync hooks completed and were successful, a successful application, and all resources in a Healthy state"* [source: argocd-sync-phases-and-waves-2026-08-31], and what does that make it suitable for?

Q21. Flux describes itself as a GitOps Toolkit — *"a collection of specialized tools, Flux Controllers, composable APIs"* [source: flux-concepts-2026-08-31] — while Argo CD presents as an integrated product with a single Application resource. Name one practical consequence of this difference for a team adopting either; name the thing Flux does at install time that Argo CD does not; and describe how their documented positions on drift correction differ.

Answers with full explanations

Q1. Factor XI (Logs — treat logs as event streams). The rule is that *"a twelve-factor app never concerns itself with routing or storage of its output stream"*; each process writes unbuffered to stdout and the execution environment captures it [source: twelve-factor-xi-logs-2026-08-31].

What it breaks: `kubect1 logs` returns nothing useful, because the container's stdout is empty. Node-level log collection sees nothing. The logs exist only in the container's writable layer and are destroyed with the Pod, which means the logs from the crash you are investigating died with the thing that crashed.

Q2. B. *"A litmus test for whether an app has all config correctly factored out of the code is whether the codebase could be made open source at any moment, without compromising any credentials"* [source: twelve-factor-iii-config-2026-08-31].

A is wrong, and it targets a real confusion: factor III is about *where config lives*, not about having less of it. An application with no configuration cannot vary between deploys at all, which fails factor III's own definition of config as *"everything that is likely to vary between deploys"* [source: twelve-factor-iii-config-2026-08-31], and it fails factor X, dev/prod parity, in the other direction. **C is wrong**, and is in fact a named anti-pattern: env vars *"are never grouped together as 'environments', but instead are independently managed for each deploy"* [source: twelve-factor-iii-config-2026-08-31]. **D is wrong** — a config file is explicitly called out as an improvement that *"still has weaknesses: it's easy to mistakenly check in a config file to the repo"* [source: twelve-factor-iii-config-2026-08-31].

Q3. The claim fails. In-memory session state violates factor VI: *"Twelve-factor processes are stateless and share-nothing. Any data that needs to persist must be stored in a stateful backing service"* [source: twelve-factor-vi-processes-2026-08-31]. The methodology bans the usual workaround by name — *"Sticky sessions are a violation of twelve-factor and should never be used or relied upon"* — and suggests a time-expiring datastore instead [source: twelve-factor-vi-processes-2026-08-31].

The general point matters more than this instance: containerization and Deployments implement the *platform* side of twelve-factor. They cannot implement the *application* side. Packaging a stateful process in a container produces a containerized stateful process.

Q4. C. *"A Recreate deployment deletes the old version of the application before bringing up the new version. As a result, this ensures that two versions of the application never run at the same time, but there is downtime during the deployment"* [source: argo-rollouts-strategies-2026-08-23].

A is wrong — `RollingUpdate` guarantees the opposite: both versions run concurrently by design, which is what avoids the downtime. **D is wrong** — blue/green does run both versions simultaneously; only one receives production traffic [source: argo-rollouts-strategies-2026-08-23], which is a different guarantee and does not help with, say, an incompatible schema migration. **B is wrong** — canary deliberately runs both against real traffic.

Q5. Blue/green. Both versions run, only the old receives production traffic, and *"this allows the developers to run tests against the new version before switching the live traffic"* [source: argo-rollouts-

strategies-2026-08-23], against production configuration and production backing services.

Canary is a poor fit for a specific mechanical reason: it works by proportioning *traffic*, and a queue worker has no inbound traffic to proportion. Argo's own comparison makes the pairing: canary demands traffic-splitting via a service mesh or ingress controller, while blue/green *"needs no traffic provider and suits workloads such as queue workers"* [source: argo-rollouts-strategies-2026-08-23].

Q6. What is wrong: a Deployment's update-strategy field takes only two values, Recreate and RollingUpdate [source: argo-rollouts-strategies-2026-08-23]. Blue/green is not among them and cannot be expressed there. Whatever they have configured, it is not blue/green in the Deployment spec, most likely a RollingUpdate they are describing loosely, or a second Deployment they are switching between by hand.

What must actually exist: **two complete environments running simultaneously**, and **something that controls which one receives production traffic** — a Service selector, a load balancer, or an ingress rule — plus something that flips it. CNCF's description turns on exactly that: two environments, one live, traffic switched via load balancer after testing the inactive one [source: cncf-glossary-blue-green-deployment-2026-08-31]. A Deployment object has no concept of a second environment and no control over traffic routing, which is why the pattern needs tooling above it. This is §2's Fixed Point in its consequential form.

Q7. The metric-analysis half. *Gradual* alone is just a slower deployment: a RollingUpdate with a small maxSurge is gradual and is not progressive delivery. What the definition adds is *"automation and metric analysis to drive the automated promotion or rollback of the update"* [source: argo-rollouts-strategies-2026-08-23]. The release evaluates itself against measurements and decides whether to continue or reverse. Gradual buys time; metric analysis is what uses the time.

Q8. It is a Kubernetes controller — not a new category of component, and not outside the normal machinery. The documentation says so in the terms Chapter 3 taught: the application controller *"is a Kubernetes controller which continuously monitors running applications and compares the current, live state against the desired target state (as specified in the repo)"* [source: argocd-architecture-2026-08-31].

What it compares: live state (what is actually deployed) against target state (what the repository says should be) [source: argocd-core-concepts-2026-08-31].

What is different: one thing only — where the desired state is kept. In Chapter 3, desired state lived in etcd, reached through the API server. Here it lives in a repository. Everything else holds: the loop is the same loop, the controller occupies the same position, and it writes through the API server as an ordinary API client subject to authentication, authorization, and admission [*cross-bearing: see Ch 3 §5 — the only door in*].

The "pushes state in" half of the colleague's description is wrong twice over. The agent runs inside the cluster and *pulls* from the repository, which is principle 3 [source: opengitops-principles-v1-2026-08-31]; and it does not push into anything, it makes ordinary API calls like every other controller.

Q9. B. In push-based delivery the pipeline lives outside the cluster and must hold credentials to it, which is the arrangement this chapter contrasts with pull.

A is wrong, and it is the confusion most worth clearing: push versus pull is about *where the credentials live*, not about network topology. A self-hosted runner inside the network, a VPN, or a private CI instance all push to an API server that is not publicly reachable. Nothing about push requires exposure. **C is wrong** — many push pipelines apply manifests from Git; that is precisely the arrangement Q8 and TYB 2 Q1 describe, and it is what makes the distinction subtle. **D is wrong** — nothing bypasses the API server, in either model [*cross-bearing: see Ch 3 §5 — the only door in*].

Q10. Blast radius is this book's term for how far the damage from a single compromise reaches. A shared CI system holding write credentials for a dozen clusters has a blast radius of a dozen clusters: whoever controls the pipeline controls every cluster it deploys to. Under pull-based delivery, each cluster's agent holds credentials only to its own cluster, so compromising the CI system yields the ability to build and publish artifacts, a serious supply-chain problem, but not direct cluster-write access anywhere.

What pull does not change: it does not prevent compromise, and it does not make the agent's own credentials smaller. Argo CD's default is *"a clusteradmin level role"* [source: argocd-security-cluster-credentials-2026-08-31]. Anyone who can commit to the tracked branch can, transitively, do whatever the agent may do.

What is sourced and what is not: the credential *locations* are documented — Argo CD stores external-cluster credentials as a Secret in the argocd namespace [source: argocd-security-cluster-credentials-2026-08-31], and its best-practices page argues for access separation on the grounds that *"the developers who are developing the application, may not necessarily be the same people who can/should push to production environments"* [source: argocd-best-practices-2026-08-31]. The *comparison* between push and pull as a security posture is this book's reading of principle 3 and those documented placements. No CNCF or vendor source in this chapter's corpus makes it in these terms.

Q11. D. Principle 4 requires that agents *"continuously observe actual system state and attempt to apply the desired state"* [source: opengitops-principles-v1-2026-08-31]. A system that acts only on new commits satisfies principle 3, since it does pull, but has no answer to drift introduced any other way. A manual `kubect1 edit` would persist indefinitely, which is exactly the failure GitOps exists to close.

A, B, and C are all satisfied by the described system: the manifests are declarative, they are in a versioned store, and the agent pulls them. C is the closest call and the most useful distractor — readers who conflate "pulls" with "keeps checking" pick it. Pulling on a trigger is still pulling; principle 3 is about *direction*, principle 4 is about *cadence*.

Q12. It establishes that CI and GitOps are orthogonal concerns, not stages of one thing. The cluster does not build your application and does not care who did; CI/CD workflows are *"determined by organization cultures and preferences as well as technical requirements"* [source: k8s-docs-overview-2026-08-23]. GitOps starts after a deployable artifact exists, and concerns itself only with how desired state is stored

and applied. A team can have excellent CI and no GitOps, or GitOps with a build process that is entirely manual.

This is why "GitOps means running CI from Git" is wrong at the level of category, not just detail.

Q13. B. Sync status answers *"is the deployed application the same as Git says it should be?"* while sync **operation** status answers *"whether or not a sync succeeded"* — two separate glossary entries, two separate questions [source: argocd-core-concepts-2026-08-31]. The documentation states the combination outright: *"it is possible for an application to be OutOfSync even immediately after a successful Sync operation"* [source: argocd-diffing-outofsync-2026-08-31].

A is wrong — it assumes the two fields report one fact, which is exactly the collapse the glossary's separate entries exist to prevent. **C is wrong**, and it is the misconception Exam Alert #2 names: reading OutOfSync as a failure report. It is a drift signal; a person editing the cluster produces one with nothing having failed [source: argocd-overview-2026-08-23]. **D is wrong on two counts.** First, self-healing is not on by default: *"by default, changes that are made to the live cluster will not trigger automated sync"* [source: argocd-auto-sync-policy-2026-08-31], so nothing is guaranteed to happen on its own. Second, a drift report is information, not noise — the whole point of the status is that somebody reads it.

Q14. The colleague is wrong on both counts. Argo CD accepts manifests *"in several ways: kustomize applications; helm charts; jsonnet files; plain directory of YAML/json manifests; any custom config management tool configured as a config management plugin"* [source: argocd-overview-2026-08-23].

The component: the **repository server**, which *"maintains a local cache of the Git repository holding the application manifests"* and is responsible for *"generating and returning the Kubernetes manifests"* given a repository URL, revision, path, and template-specific configuration [source: argocd-architecture-2026-08-31]. Rendering is a dedicated component, not an afterthought. Argo CD's glossary even names the choice: the **application source type** is *"which Tool is used to build the application"* [source: argocd-core-concepts-2026-08-31].

Q15. Nothing happens to the cluster. *"If a Git commit SHA is specified, the app is effectively pinned to the manifests defined at the specified commit"* [source: argocd-tracking-strategies-2026-08-31]. New commits on the branch are irrelevant; the Application is not looking at the branch.

To update: *"the only way to change the live state of an app which is pinned to a commit, is by updating the tracking revision in the application to a different commit containing the new manifests"* [source: argocd-tracking-strategies-2026-08-31]. Somebody must change the Application's own revision field, which is itself typically a commit in a repository, and therefore itself reviewable. That is the point of pinning.

Q16. [cross-domain: D2.2] A ServiceAccount, and a ClusterRole bound by a ClusterRoleBinding. The ServiceAccount is the identity the agent's Pod runs as; the ClusterRole enumerates permitted verbs on resources; the ClusterRoleBinding attaches the permission set to the identity across the whole cluster [cross-bearing: see Ch 12 §3 — what you may do].

Why per-namespace Roles are a poor answer: they work, and they break. Every new namespace the repository introduces requires a new Role and RoleBinding before the agent can act in it, which means the agent cannot create a namespace and populate it in one sync, and the failure appears as a permissions error at exactly the moment someone adds a namespace. Argo CD's own model uses a cluster-scoped role for this reason, noting that it *"requires cluster-wide read privileges to resources in the managed cluster to function properly"* even when write is narrowed [source: argocd-security-cluster-credentials-2026-08-31].

On the pruning qualifier in the stem: permission and configuration are separate. Deleting requires the delete verb in the grant, but the grant does not cause deletion — *"by default (and as a safety mechanism), automated sync will not delete resources when Argo CD detects the resource is no longer defined in Git"* [source: argocd-auto-sync-policy-2026-08-31].

Q17. [cross-domain: D1.1] It is a good analogy and worth being precise about. Argo CD compares **target state** — *"the desired state of an application, as represented by files in a Git repository"* — against **live state** — *"the live state of that application. What pods etc are deployed"* [source: argocd-core-concepts-2026-08-31].

The mapping to Chapter 4: target state plays the role of `spec` (what was asked for), and live state is observed from the cluster, which is what `status` reports on [cross-bearing: see Ch 4 §2 — the anatomy of a record].

The one refinement: the operands live in different places. In an ordinary object, `spec` and `status` are two fields of one record in etcd. Under GitOps, the authored `spec` lives outside the cluster entirely, and the cluster's copy is downstream of it. That relocation is the whole substitution, and it is why an object's own `spec` can be perfectly satisfied while the application is `OutOfSync`, if somebody changed the `spec` itself by hand.

Q18. Three mechanisms:

- `kubectl rollout undo` (Ch 6 §5) — restores a Deployment's Pod template. Prior state lives in the old ReplicaSet, on the cluster.
- `helm rollback` (Ch 14 §3) — returns a Helm release to a prior revision. Prior state lives in Helm's release history, on the cluster.
- **Rollback by revert** (this chapter) — changes a commit in the repository; the agent reconciles as it always does. Prior state lives in the repository's history. Argo CD describes the capability as *"rollback/roll-anywhere to any application configuration committed in Git repository"* [source: argocd-overview-2026-08-23].

The structural point worth carrying: the third has no dedicated rollback code path. You move the target and the loop does the rest, the same loop that handles every ordinary change.

Q19. Order: namespace, then CustomResourceDefinition, then the custom resource. The namespace must exist before namespaced objects can be placed in it, and the CRD must be registered before the API server will accept a custom resource of that kind.

The mechanism is sync waves, which order resources within a phase, *"lower values first"* [source: argocd-sync-phases-and-waves-2026-08-31]. What matters is the relative order, not the absolute numbers: -2, -1, 0 and 0, 1, 2 behave identically. The full precedence puts the phase first and the wave second, with deterministic tie-breaks after those [source: argocd-sync-phases-and-waves-2026-08-31].

An object with no ordering annotation lands in wave 0: *"Hooks and resources are assigned to wave 0 by default. The wave can be negative, so you can create a wave that runs before all other resources"* [source: argocd-sync-phases-and-waves-2026-08-31]. That negative-wave capability is what makes waves an ordering system rather than a queue — you can always insert something ahead of the default without renumbering everything else.

Q20. PostSync, and the quoted condition is what makes it distinctive: it requires not just completion but *"all resources in a Healthy state"* [source: argocd-sync-phases-and-waves-2026-08-31].

Suitable for: smoke tests, notifications, and traffic cutover, anything that should happen only once the new state is both applied and demonstrably working. Argo CD ties the hook mechanism to release patterns explicitly, describing hooks as supporting *"complex application rollouts (e.g. blue/green and canary upgrades)"* [source: argocd-overview-2026-08-23].

Why not PreSync: PreSync runs *"prior to the application of the manifests"* [source: argocd-sync-phases-and-waves-2026-08-31], before the thing you would be testing exists.

Q21. Practical consequence (any one earns credit): Flux's composability means a team can adopt the source and Kustomize controllers without the Helm or image-automation ones, and can replace pieces independently; Argo CD's integration means fewer objects to learn and a single UI showing every application's state, at the cost of being more nearly all-or-nothing. Flux's design surfaces as many custom resources across several controllers [source: flux-concepts-2026-08-31]; Argo CD's surfaces mainly as Application objects [source: argocd-core-concepts-2026-08-31].

What Flux does at install time that Argo CD does not: it installs itself in a GitOps manner. *"The process of installing the Flux components in a GitOps manner is called a bootstrap. The manifests are applied to the cluster, a GitRepository and Kustomization are created for the Flux components, then the manifests are pushed to an existing Git repository"* [source: flux-concepts-2026-08-31] — so that, as the earlier capture puts it, *"Flux manages itself like any other resource"* [source: flux-concepts-2026-08-23]. Upgrading Flux is a commit.

Drift correction — the documented positions differ. Flux states that manual `kubectl edit/patch/delete` changes *"will be promptly reverted"* [source: flux-concepts-2026-08-31]. Argo CD's automated sync policy states the opposite default: *"by default, changes that are made to the live cluster will not trigger automated sync"* [source: argocd-auto-sync-policy-2026-08-31], and pruning is likewise opt-in — *"by default (and as a safety mechanism), automated sync will not delete resources when Argo CD detects the resource is no longer defined in Git"* [source: argocd-auto-sync-policy-2026-08-31]. Two graduated projects implementing the same four principles, shipping with opposite answers to "what do I do about drift I detect."

On multi-cluster: Argo CD documents managing multiple clusters from one control point — the feature list includes the *"ability to manage and deploy to multiple clusters"* [source: argocd-overview-2026-08-23], with each remote cluster's credentials stored as a Secret in the argocd namespace [source: argocd-security-cluster-credentials-2026-08-31]. Flux's documented position is narrower: its reconciling controllers run in the cluster they reconcile [source: flux-security-2026-08-31] and bootstrap installs it into a cluster against a repository [source: flux-concepts-2026-08-31]. Credit an answer that states the Argo CD side accurately and does not over-claim the Flux side.

Chapter Summary

Concept	Remember This
Twelve-factor app	Constraints an application accepts so a platform can run it. Kubernetes implements the platform half; the application half is still yours.
Factor III (Config)	Config is what varies between deploys. The test: could you open-source the repo today without leaking a credential?
Factor IX (Disposability)	Fast startup, graceful SIGTERM shutdown, correct even on sudden death.
Recreate / RollingUpdate	Values of a field on a Deployment. Downtime with no version overlap, versus no downtime with overlap.
Blue/green	Two full environments; only one takes traffic; test before the switch. Costs capacity, needs no traffic provider.
Canary	A slice of real traffic meets the new version first. Needs traffic splitting and metric analysis.
Progressive delivery	Gradual <i>plus</i> metric analysis driving automated promotion or rollback. Gradual alone is just slow.
Push	Pipeline outside the cluster holds cluster credentials and reaches in. Nothing watches between runs.
Pull	Agent inside the cluster holds repository credentials and reaches out. Never stops watching.
Blast radius	This book's term for how far one compromise reaches. Pull bounds it; it does not shrink the agent's own grants.
The four principles	Declarative · versioned and immutable · pulled automatically · continuously reconciled.
Argo CD	A Kubernetes controller comparing live state against target state in a repository.
Application	The custom resource: a source (repo, revision, path) and a destination (cluster, namespace).
Manifest sources	Kustomize, Helm charts, Jsonnet, plain YAML/JSON, config-management plugins.
Tracking	Branch (moves), tag (rarely moves), pinned commit (never moves).
OutOfSync	Live state deviates from target state. A drift signal, not an error. A person can cause one.
Self-heal and prune	Both opt-in in Argo CD. Out of the box it reports drift and does not revert or delete.
Rollback by revert	Change the commit; the loop does the rest. No second code path. Third mechanism to wear the word.
Agent identity	A Pod, a ServiceAccount, broad grants by default. Commit access to the tracked branch is cluster access.
Sync phases	PreSync → Sync → PostSync, each gated on the previous succeeding; SyncFail runs when a sync fails.
Sync waves	Integer ordering within a phase, lower first, default 0, negatives allowed.

Concept	Remember This
Flux	A composable toolkit of controllers. Bootstraps itself. Reverts manual <code>kubectl</code> changes — the opposite default from Argo CD.
The Zenith	The control loop, with a Git repository where <code>etcd</code> sat. Nothing else moved.

The Voyage Ahead

You now hold a delivery model in which nobody touches the cluster and something never stops watching it.

Which is exactly when you find out that the thing being watched is not the thing you thought.

The agent reports `Synced`. Every object matches the repository, every replica is present, the rollout completed and the health checks pass. And the application does not work. Users get errors, or timeouts, or the wrong data, and nothing in the delivery path has anything to say about it, because the delivery path did its job perfectly. It applied what you asked for. You asked for the wrong thing, or the right thing configured wrongly, and no amount of reconciliation can tell the difference.

Chapter 13 taught you to diagnose a cluster that will not run your Pod. It ended by handing something back: the case where the platform is fine and the problem is yours. That handoff comes due next.

The next chapter is about the Pod that is running, healthy, `Synced`, and wrong, and about the tools that let you get inside a container that was built with nothing in it to help you.

"Reconciliation will make the cluster match your intention exactly. It has no opinion about your intention."

Chapter 16: Your Application, Their Cluster

"Four questions that separate your bug from theirs"

Domain Weight: 16% (Cloud Native Application Delivery) [source: cncf-kcna-curriculum-pdf-2026-08-23] | **Competency:** Debugging | **Authored allocation for this chapter:** ~4% **Complexity:** Mixed | **Novelty:** Moderate | **Prerequisites:** Heavy

Attention Budget

Total time: ~82 minutes | **Recommended:** Single session, or split after §5

Section	Time	Attention Cost	Best Time to Study
§1 Handed Back	6 min	Low	Anytime
§2 When It Never Got Started	9 min	Medium	Mid-session
§3 Getting Inside, and Adding What Isn't There	17 min	High	Peak attention
🕒 Taking Your Bearings (1)	6 min	Medium	After a brief break
§4 Is Anything Even Selected	12 min	High	Peak attention
§5 Bypassing the Service on Purpose	7 min	Low	Anytime
🕒 Taking Your Bearings (2)	5 min	Medium	After a brief break
§6 When Each Replica Is Its Own	7 min	Medium	Mid-session
§7 Before You Ship It	5 min	Low	Anytime
🕒 Taking Your Bearings (3)	5 min	Medium	After a brief break
§8 Mine, or the Platform's	3 min	Low	Anytime

Attention Cost Key:

- **Low:** Concrete, familiar concepts — study anytime
- **Medium:** New concepts requiring focus — study when alert
- **High:** Abstract or complex material — study at peak attention

If you only have 15 minutes: read §1 and §4, then take the second Taking Your Bearings checkpoint.

"The first step in troubleshooting is triage. What is the problem? ..." — Kubernetes documentation, *Debug Pods* [source: k8s-docs-debug-pods-2026-08-31]

🔊 Soundings

Before reading this chapter, try these eight questions. Every one of them is answerable from Chapters 1–15 or from general professional knowledge; none requires anything this chapter is about to teach you. Your score determines how to approach the content. No shame in any score; different scores just call for different reading strategies.

1. A Pod is Running. It reports 1/1 Ready. Its restart count is zero, its node is healthy, and no other workload in the namespace is affected. The response it returns is still wrong. Whose problem is this — the platform's or the application's?
2. A Pod declares three init containers. The second one exits non-zero. What happens to the third init container, and what happens to the app containers?
3. A Pod is Running but reports 0/1 READY. What has happened to its standing as a target for traffic through its Service?
4. Of port and targetPort on a Service, which one names the port the container is actually listening on?
5. Write out the in-cluster DNS name a Pod would use to reach a Service named `api` in the namespace `payments`.
6. A StatefulSet Pod is rescheduled onto a different node. Name one thing about it that does not change.
7. You delete the Pod `web-2` from a three-replica StatefulSet. What happens to its `PersistentVolumeClaim`?
8. An image is built from a minimal base — no package manager, no `/bin/sh`, nothing but the application binary and its libraries. What happens when you run `kubectl exec <pod> -- sh` against it?

Answers + reading strategy

1. **The application's.** The mechanical test from Chapter 13 is: if the Pod is running and ready, and the fault is confined to one workload, the platform has done its job and the problem is yours. [*cross-bearing: see Ch 13 §1 — whose problem is this, and what to read first*]
2. **The second one is retried; the third does not start; the app containers do not start.** Init containers run in order and each must complete successfully before the next begins, and "if a Pod's init container fails, the kubelet repeatedly restarts that init container until it succeeds" [source: `k8s-docs-init-containers-2026-08-24`]. The Pod stays Pending with the `Initialized` condition false

[source: k8s-docs-init-containers-2026-08-24]. *[cross-bearing: see Ch 5 §3 — everything that must happen first]*

3. **It is still listed behind the Service, but marked as not ready.** Readiness maps onto the endpoint's own condition, and an endpoint that is not serving is not one that should be used as a target for Service traffic [source: k8s-docs-endpointslices-2026-08-24]. Readiness does not delete the endpoint; it disqualifies it. *[cross-bearing: see Ch 9 §4 — the list behind the name]*
4. **targetPort.** port is the port the Service itself is reachable on; targetPort is the port on the Pod that traffic is delivered to. *[cross-bearing: see Ch 9 §3 — four ways to be reachable]*
5. **api.payments.svc.cluster.local** — the general form is <service>.<namespace>.svc.<cluster-domain> [source: k8s-docs-dns-pod-service-2026-08-23]. A Pod in the payments namespace could also just say api. *[cross-bearing: see Ch 9 §7 — names, and where they resolve]*
6. **Any of: its hostname, its ordinal, its DNS name, or its PersistentVolumeClaim.** All four are part of the sticky identity a StatefulSet maintains — *"each has a persistent identifier that it maintains across any rescheduling"* [source: k8s-docs-statefulset-2026-08-24]. *[cross-bearing: see Ch 6 §6 — when Pods are not interchangeable]*
7. **Nothing. The PVC survives.** *"Retain (default): PVCs from the volumeClaimTemplate are not affected when their Pod is deleted", and "the default for policies is Retain" for both the scale-down and the delete case* [source: k8s-docs-statefulset-storage-2026-08-25]. *[cross-bearing: see Ch 11 §6 — Pods that are not interchangeable, revisited]*
8. **It fails.** There is no shell in the image to execute. An image contains exactly what was put into it and nothing else. There is no ambient operating system underneath waiting to supply the missing pieces. *[cross-bearing: see Ch 2 §2 — what's inside an image]*

If you got 6+ right: Skim, but read §3 and §6 properly. Those two carry material with no analog anywhere earlier in the book, and skimming them will cost you.

If you got 3–5 right: Read at normal pace. This chapter is calibrated for you.

If you got 0–2 right: Before you start this chapter, go back and re-read **Chapter 13 §1** — not the whole chapter, that one section. This chapter's opening move is receiving a handoff that Chapter 13 §1 made. Without it, everything here reads as a list of commands instead of a method.

Why This Chapter Matters

Three chapters ago, the platform finished its work and handed you back your own problem.

That is where this chapter starts. Not with an introduction — with the far side of a handoff you were given in Chapter 13 and have been carrying ever since. The Pod is Running. It is 1/1 Ready. The restart

count is zero, the node is healthy, the events are quiet, and the thing your users are complaining about is still happening.

Chapter 13 taught you to stop reaching for the logs and read the phase first. The phase, here, has nothing left to say. Every instrument you have trusted for two hundred pages reads fair, and the system is broken anyway.

So what do you actually look at?

The answer is a different set of four questions and a different set of tools, and there is a reason those tools were withheld until now: every one of them requires a running container. `exec` needs a process to enter. An ephemeral container needs a Pod to attach to. `port-forward` needs something listening on the other end. All of them assume the condition Chapter 13 spent its whole length helping you establish.

There is also a shift in who you are while you use them. Chapter 13's reader stood outside somebody else's platform and interrogated it, asking whether the cluster had done its job. This chapter's reader is the person who shipped the thing, working inside a cluster they do not own, with permissions they did not grant themselves, on an image that may not contain a shell. That is the actual posture of an application engineer on Kubernetes, and it is what the chapter title names.

Dead Reckoning: When a Pod is Running and Ready and the application is still wrong, the platform's own signals keep reporting "fine," because from the platform's point of view everything is. The diagnostic surface you need is inside the container, in the Service that routes to it, and in the configuration the process actually read. The platform inspects none of those on your behalf. The tools for reaching them are `kubectl exec`, `kubectl debug`, `kubectl port-forward`, and `kubectl get endpointslices`. Each answers a different question. Knowing which question you are asking is most of the work.

The stakes are specific. Cloud Native Application Delivery is 16% of the KCNA exam [source: [cncf-kcna-curriculum-pdf-2026-08-23](#)], and Debugging is one of its two competencies. The competency is not flag syntax. It is which verb answers which question, and no glossary can teach you that: the verbs are only distinguishable by the question each one is for.

What You'll Learn

By the end of this chapter, you'll be able to:

- **Take** the handoff from platform scope and state, mechanically, which side of the boundary a failure is on
- **Ask** the four questions that localize an application fault — is it running, is it healthy, is it reachable, is it configured — in the order that clears the most water first
- **Read** a failing init container from the application side, and recognize the ordering and re-run assumptions that break one

- **Get inside** a running container with `exec`, and get inside one with no shell at all using an ephemeral container and `kubectl debug`
- **Diagnose** a Service that exists, selects nothing, and is therefore working exactly as written
- **Prove** where a break is by deliberately bypassing the Service path with `port-forward`
- **Ask** all four questions of a single StatefulSet replica, when the replicas are not interchangeable

You'll also stop asking "is Kubernetes broken?" as your first question, which is the habit that costs application engineers the most time on somebody else's cluster.

13 §1 — Handed Back

Here is the sentence Chapter 13 ended on, restated from this side: **the Pod is fine and the application isn't.**

That is not a failure of the platform's diagnostics. It is what a successful platform diagnosis looks like. The kubelet started your container. The scheduler placed it. The probes passed. Nothing crashed. The cluster has taken its own bearings and found nothing out of position. The fault is somewhere the cluster does not look.

The Kubernetes documentation draws the same line at the top of its own application-debugging guide: *"This guide is to help users debug applications that are deployed into Kubernetes and not behaving correctly. This is not a guide for people who want to debug their cluster."* [source: k8s-docs-debug-pods-2026-08-31] Two audiences, two sets of tools, one boundary between them.

You already have the mechanical test for which side you are on: scope first, then phase, then conditions, then events, then logs, and a fault that is *not* confined to a single workload is still the platform's [*cross-bearing: see Ch 13 §1 — whose problem is this, and what to read first*]. What is new here is the direction of travel, and one honest addition to the test.

The addition is this: on somebody else's cluster, you will frequently be *unable* to check the platform side yourself. You may not have permission to read node conditions, or to list Pods in `kube-system`, or to see the events on a node you don't own. The boundary is therefore not only a statement about where the fault is. It is also a statement about what you are allowed to see. The practical consequence: make your case for "this is platform-side" from evidence you *can* gather, because the person who can check the other half is going to ask you for it.

None of which makes the platform team an adversary. "Their cluster" in this chapter's title is a statement of scope, not of blame. They own the machinery; you own the workload; the boundary is where those two responsibilities meet, and the entire value of being able to place a fault on the correct side is that neither of you spends an afternoon on the other's half.

The four questions

Everything after this section is one of four questions, asked in an order chosen to clear the most water first:

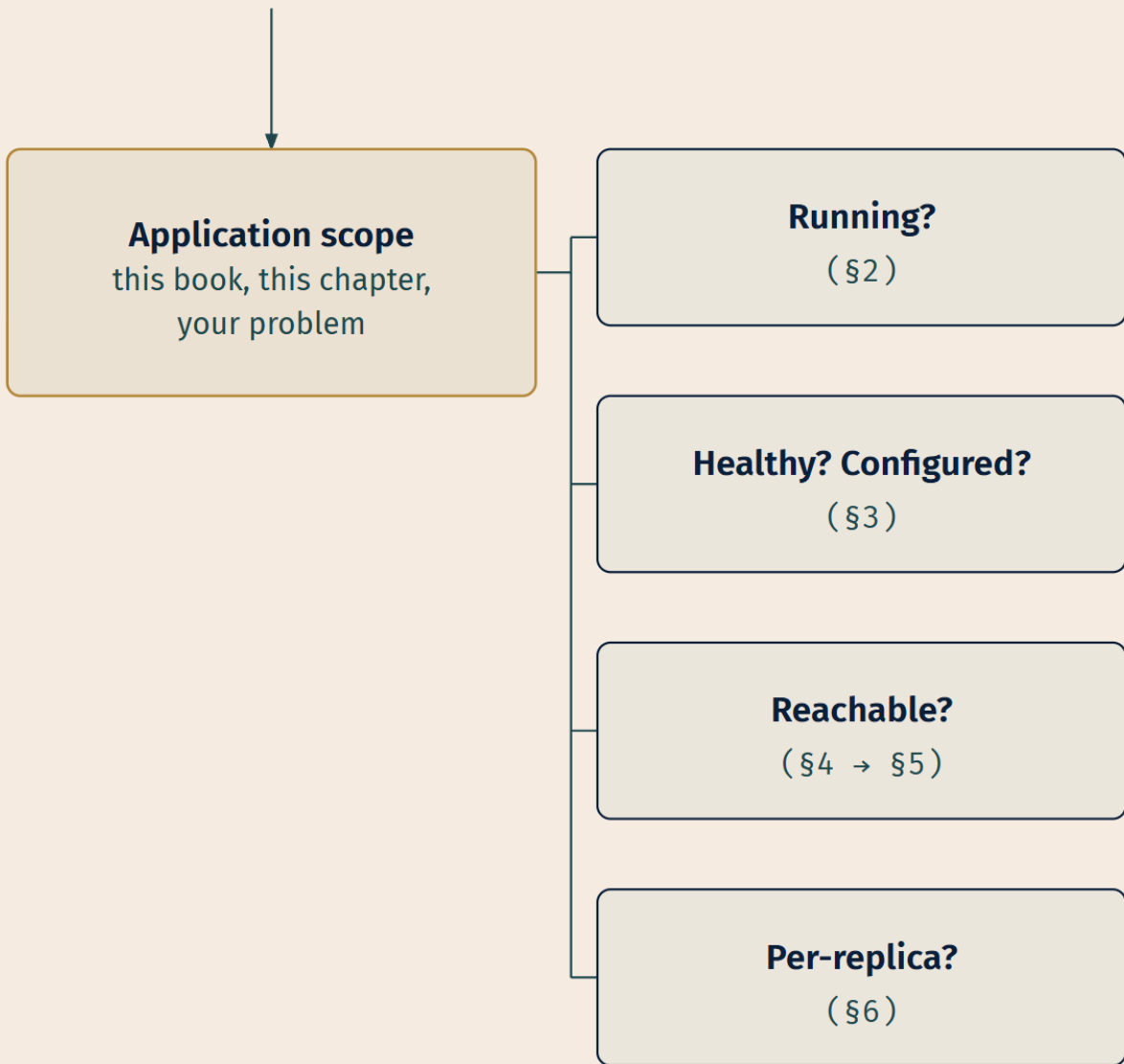
Is it running? — Not "does the Deployment exist," but did this container's process actually get to the point of executing your code. A Pod that never cleared its init sequence has never run a line of your application.

Is it healthy — and is it configured the way you think? — The process is up. Is it doing what you told it, with the values you believe you gave it? These two are one question because the answer to both lives in the same place: inside the running container.

Is it reachable? — Something is listening. Can the traffic get to it, through the Service that is supposed to route it?

Is it configured? — Which appears twice, deliberately. Read on.

FROM CH 13
PLATFORM SCOPE DISCHARGED



LEGEND

 Root — application scope

 Triage question

Top to bottom — each question eliminates ground before the next.

The mapping to sections is direct, with one wrinkle:

Question	Answered in
Is it running?	§2
Is it healthy — and is it configured the way you think?	§3
Is it reachable?	§4, then §5
(all four, for workloads whose replicas are not interchangeable)	§6

Note the doubling. "Is it configured" gets answered in two places, because a configuration fault surfaces at two different times. A missing ConfigMap key can stop a Pod before it ever starts, and that shows up at init (§2). A *present* key with the wrong value gets read cleanly by a running process that then does the wrong thing, and that shows up only when you go in and look (§3). Same class of bug, two entirely different signatures, two different sections. Four questions, eight sections: the arithmetic works out because the questions overlap, not because anything is missing.

🔒 **Worth Securing:** Ask the questions in order. Each one, answered, deletes a large region of the search area. The temptation is to jump to whichever tool you learned most recently, usually `exec`, and start poking. That is how a fifteen-minute diagnosis becomes an afternoon: you can spend a very long time inside a container that was never going to work because its init container failed forty minutes ago.

..l §2 — When It Never Got Started

A Pod that is stuck before its first application container has run is the easiest fault in this chapter to misdiagnose, because the symptom — "nothing is happening" — is the least informative reading any instrument can give you. It looks identical to a dozen other things.

You have the model already. Init containers run before app containers, in the order written, one at a time, each to completion [*cross-bearing: see Ch 5 §3 — everything that must happen first*]. What Chapter 5 deliberately did not give you was the method for reading one when it goes wrong. And Chapter 13 owned the platform-side half of this failure: an init container whose *image* cannot be pulled is a platform-scope problem with a platform-scope signature [*cross-bearing: see Ch 13 §2 — pods that never start*]. What is left, and what this section owns, is the case where the init container runs perfectly well and does the wrong thing.

Reading the state

Start with the Pod's status, which tells you which init container you are looking at:

```
kubect1 get pod <pod-name>
```

The STATUS column carries a specific vocabulary for this case [source: k8s-docs-debug-init-containers-2026-08-31]. A Pod mid-sequence reports `Init:N/M` — N init containers complete out of M total. A Pod

whose init container is failing reports `Init:Error` or `Init:CrashLoopBackOff`. That is already a lot of information: `Init:1/3` tells you the first one succeeded and you should be looking at the second.

The Pod's phase is `Pending` throughout, with the `Initialized` condition false — *"a Pod that is initializing is in the Pending state but should have a condition `Initialized` set to false"* [source: k8s-docs-init-containers-2026-08-24] [cross-bearing: see Ch 5 §5 — Pod phase and container state]. This is why "read the phase first" alone does not finish the job here. Every Pod in this state has the same phase, and the discriminating detail is in the status string and the container list.

One more thing the status is telling you: a failing init container is not simply abandoned. *"If a Pod's init container fails, the kubelet repeatedly restarts that init container until it succeeds. However, if the Pod has a restartPolicy of Never, and an init container fails during startup of that Pod, Kubernetes treats the overall Pod as failed."* [source: k8s-docs-init-containers-2026-08-24] That is why `Init:CrashLoopBackOff` exists as a status at all, and it is why a Pod can sit at this station indefinitely rather than resolving to `Failed`.

Reading the logs

Here is where people lose time:

```
kubectl logs <pod-name>
```

This returns nothing useful. The plain form targets the Pod's app container, and the app container has not started, so there is nothing to print. What you need is the `-c` flag naming the specific init container [source: k8s-docs-debug-init-containers-2026-08-31]:

```
kubectl logs <pod-name> -c <init-container-name>
```

The `-c` flag is not new to you [cross-bearing: see Ch 13 §3 — reading logs from a multi-container Pod]; what is new is that here it is not optional. A multi-container Pod at least gives you a log stream when you omit `-c`. An initializing Pod gives you an error or an empty result, and the reader who reads that as "the init container isn't logging anything" has just concluded something false about a container that is loudly complaining into a stream nobody asked for.

🔖 **Snag:** `kubectl logs <pod>` on a Pod stuck in init tells you nothing, and the nothing is misleading. Always name the init container with `-c`. If you don't know its name, `kubectl describe pod <pod>` is where the init containers and their states are listed.

The three ways an init container is wrong

Ordering. Init containers are the mechanism Kubernetes gives you for holding an application back until a precondition is met: *"init containers offer a mechanism to block or delay app container startup until a set of preconditions are met"* [source: k8s-docs-init-containers-2026-08-24]. The most common failure is an init container waiting for something that cannot arrive until this Pod is up.

A classic shape, which follows directly from the sequencing rules above: an init container that blocks until a Service has endpoints, for a Service whose only backend is *this* Pod. The init container waits. The app container cannot start until the init container exits. The Pod cannot become ready until the app container starts. The Service cannot gain a ready endpoint until the Pod is ready. Two lines, each waiting for the other to take up the slack, and the deadlock is perfectly stable. Nothing in the platform will tell you this; the platform is doing exactly what you asked.

The tell is an `Init:0/1` Pod that has sat there for twenty minutes with no error, no restarts, and a log stream saying something calm and patient like "waiting for dependency."

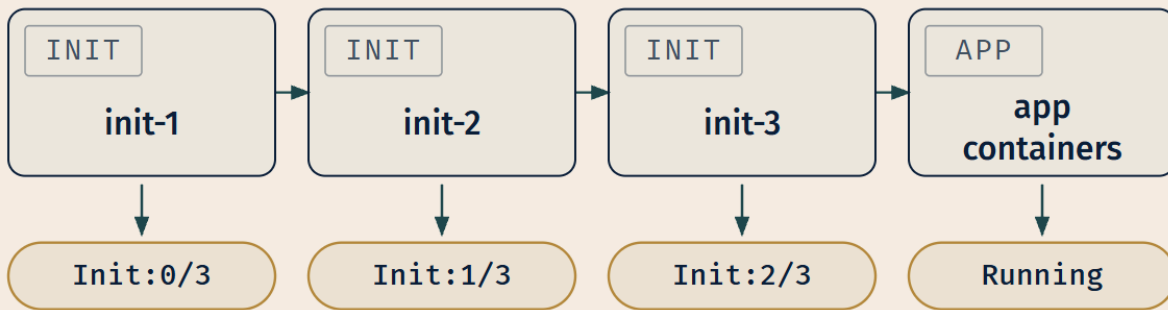
Non-idempotency. An init container will be re-run. Not might — will. If the Pod restarts, every init container executes again from the beginning [source: k8s-docs-init-containers-2026-08-24]. An init container that assumes a clean slate — that creates a directory without checking, that runs a migration without a guard, that appends to a file that should have one line — works perfectly the first time and fails on every restart after. This produces one of the more maddening signatures in Kubernetes: a workload that comes up fine on a fresh deploy and refuses to come back after any disruption at all.

🔒 **Worth Securing:** Write init containers as though they will run five times, because eventually they will. Idempotency is not a nicety here; it is a correctness requirement imposed by the restart semantics. If your init container's job is "create X," its actual job is "ensure X exists."

Configuration errors visible at init. Init containers are frequently where a config problem first becomes visible, because the init container is usually the first thing that reads the mounted configuration. A `ConfigMap` key that does not exist, a `Secret` whose value is base64-decoded into something the wrong shape, a mount path that collides with something in the image: these surface as an init container exiting non-zero with a message that names the actual problem [cross-bearing: see Ch 4 §4 — *ConfigMaps and Secrets*].

That last point is worth dwelling on, because it is the good news in this section. A config error caught at init gives you a clean, specific, non-mysterious failure with the reason printed in a log. The *same* config error that gets past init, because the value is present but wrong, becomes §3's problem, and §3's problem is much harder.

POD STARTUP – INIT CONTAINER SEQUENCE



DEBUG COMMANDS

READ WITH `kubectl logs <pod> -c <init-name>`

ALSO READ `kubectl describe pod <pod>`
(exit codes, order, state)

FAILURE SIGNATURES

STUCK, NO ERROR

→ ordering deadlock — what is it waiting for?

FAILS ONLY ON RESTART

→ non-idempotent — it assumed a clean slate

EXITS NON-ZERO, LOUDLY

→ config error — read the message, it's telling you

LEGEND

 Init / app container

 Pod status

→ Startup order

There is one more diagnostic surface here worth knowing. A container can write to a **termination message** file at `/dev/termination-log`, and Kubernetes surfaces the contents in the Pod's status [source: [k8s-docs-determine-reason-pod-failure-2026-08-31](#)]. The docs describe the purpose plainly:

"Termination messages provide a way for containers to write information about fatal events to a location where it can be easily retrieved and surfaced by tools like dashboards and monitoring software." [source:

k8s-docs-determine-reason-pod-failure-2026-08-31] For an init container that fails in a way that logs don't capture well, writing a one-line reason to the termination log makes the failure legible from `kubectl describe` alone.

🔗 **Closer Look:** The same source notes that in most cases, information you put in a termination message should also be written to the general Kubernetes logs [source: k8s-docs-determine-reason-pod-failure-2026-08-31]. The termination message is a summary surfaced in status, not a replacement for logging.

3 — Getting Inside, and Adding What Isn't There

The Pod is running. Your code is executing. And you have run out of what can be read from outside the hull.

This section is about getting inside a container, and about what to do when there is no inside to get into. It is the densest material in the chapter, and it has no analog anywhere earlier in the book. Chapter 8's `kubectl` verb table pointed here explicitly for `exec` [*cross-bearing: see Ch 8 §1 — the grammar of a command*], and Chapter 12 pointed here for the debug-container case [*cross-bearing: see Ch 12 §6 — three levels, three modes*]. This is where both debts come due. If you are following one of those pointers and it said "getting inside a container" while another said "getting inside, and adding what isn't there" — same section, both phrasings, you are in the right place.

`kubectl exec`

The direct move is to run a command inside a container that is already running:

```
kubectl exec -it <pod-name> -- /bin/bash
```

The docs describe it exactly this way: *"This page shows how to use `kubectl exec` to get a shell to a running container"* [source: k8s-docs-get-shell-running-container-2026-08-31]. For a multi-container Pod, name the container with `-c`, the same flag you have been using with `logs`:

```
kubectl exec -it <pod-name> -c <container-name> -- /bin/sh
```

The double dash matters. Everything after `--` is the command run inside the container; everything before it belongs to `kubectl` [source: k8s-docs-get-shell-running-container-2026-08-31]. Omit it and `kubectl` will try to interpret your command's flags as its own.

What you are actually doing in there, most of the time, is answering the second half of the second question: **is it configured the way you think?** Not "does the ConfigMap say what I meant"; you can read the ConfigMap from outside. The question is what the *process* got. Environment variables can be shadowed, mounted files can be masked by another mount, a default in the application code can quietly

win over an empty string, and a value can be correct in the manifest and absent in the container because a mount path was one character off.

```
kubectl exec <pod-name> -- env | grep DATABASE
kubectl exec <pod-name> -- cat /etc/config/app.yaml
kubectl exec <pod-name> -- ls -la /etc/config/
```

That third one catches more bugs than the other two combined. A directory that is empty, or that contains a file named something you did not expect, is a mount that did not land the way the manifest reads.

📌 **Snag:** The gap between "what the manifest says" and "what the process read" is where a large share of application-scope bugs live, and it is invisible from `kubectl get -o yaml`. The manifest is a statement of intent; the container's filesystem and environment are the outcome. When they disagree, `exec` is the only place you find out.

📌 **Snag:** There is a second way the manifest and reality disagree, and `exec` cannot see this one at all, because the field never reached the server. *"Often a section of the pod description is nested incorrectly, or a key name is typed incorrectly, and so the key is ignored"* [source: k8s-docs-debug-pods-2026-08-23]. The apply succeeded. Nothing errored. The field is simply absent from what the API server stored, and the container is faithfully running the spec that actually exists rather than the one you wrote. The move is a round trip: apply, then read back what the server kept and compare it against your file. The docs prescribe exactly that, comparing `kubectl get pods/mypod -o yaml` against the original [source: k8s-docs-debug-pods-2026-08-23].

The image with nothing in it

Now the problem.

```
kubectl exec -it <pod-name> -- /bin/sh
```

```
OCI runtime exec failed: exec failed: unable to start container process:
exec: "/bin/sh": stat /bin/sh: no such file or directory
```

There is no shell. There is no `cat`, no `ls`, no `env` binary, no package manager to install one with. The image contains your application binary, the libraries it links against, and nothing else. This is a **distroless** image, and it is not a mistake. It is a deliberate hardening choice. The Kubernetes documentation is direct about the trade: *"...distroless images enable you to deploy minimal container images that reduce attack surface and exposure to bugs and vulnerabilities. Since distroless images do not include a shell or any debugging utilities, it's difficult to troubleshoot distroless images using `kubectl exec` alone."* [source: k8s-docs-ephemeral-containers-concept-2026-08-31]

Your first instinct is probably right and also unavailable: add a second container to the Pod with the tools in it. You cannot.

☆ Fixed Point

You cannot add a container to a Pod once the Pod has been created. The Pod's container list is fixed at creation. That single fact is the entire reason ephemeral containers exist as a separate mechanism, and it is the fact worth carrying into a question about them.

The documentation states the constraint and the consequence together: *"Since Pods are intended to be disposable and replaceable, you cannot add a container to a Pod once it has been created. Instead, you usually delete and replace Pods in a controlled fashion using deployments."* [source: k8s-docs-ephemeral-containers-concept-2026-08-31] And then the exception: *"Sometimes it's necessary to inspect the state of an existing Pod, however, for example to troubleshoot a hard-to-reproduce bug. In these cases you can run an ephemeral container in an existing Pod to inspect its state and run arbitrary commands."* [source: k8s-docs-ephemeral-containers-concept-2026-08-31]

Ephemeral containers

An ephemeral container is a container that runs temporarily inside an existing Pod, added through a dedicated API path rather than by editing the Pod's spec. The documentation is explicit that it is not editable through the normal route: *"Ephemeral containers are created using a special `ephemeralcontainers` handler in the API rather than by adding them directly to `pod.spec`, so it's not possible to add an ephemeral container using `kubectl edit`."* [source: k8s-docs-ephemeral-containers-concept-2026-08-31]

They are deliberately constrained, and the constraints are exam material [source: k8s-docs-ephemeral-containers-concept-2026-08-31]:

- **No ports, no probes.** *"Ephemeral containers may not have ports, so fields such as `ports`, `livenessProbe`, `readinessProbe` are disallowed."*
- **No resource requests or limits.** *"Pod resource allocations are immutable, so setting resources is disallowed."*
- **No removal, no change.** *"Like regular containers, you may not change or remove an ephemeral container after you have added it to a Pod."*
- **No restarts.** *"Ephemeral containers differ from other containers in that they lack guarantees for resources or execution, and they will never be automatically restarted, so they are not appropriate for building applications."*
- **Not on static Pods.** *"Note: Ephemeral containers are not supported by static pods." [cross-bearing: see Ch 13 §5 — when the node is the problem]*

Read those five together and the design intent is obvious: this is an instrument, not a workload. It gets no guarantees because it is not supposed to be part of your application, and it cannot be removed because a Pod's container list, even the ephemeral part of it, is append-only.

🔑 **Mnemonic:** An ephemeral container is a **tool passed through the hatch**, not a compartment added to the hull. No resources reserved for it, no probes watching it, no restart if it dies, and once it's through the hatch you cannot take it back.

kubectl debug

`kubectl debug` is the verb that puts one there. In its simplest form it attaches an ephemeral container to a running Pod, using an image you choose, one that has the tools the target image lacks:

```
kubectl debug -it <pod-name> --image=busybox:1.28 --target=<container-name>
```

`--target` names the container you want the debug container to be able to see into. The principle behind it is sourced: *"When using ephemeral containers, it's helpful to enable process namespace sharing so you can view processes in other containers."* [source: k8s-docs-ephemeral-containers-concept-2026-08-31] Without a shared view of the target's processes, an ephemeral container is a separate process tree in the same Pod, and most of what you wanted to inspect is invisible from it.

That is one of three shapes. The other two answer different questions.

`--copy-to`: a copy, not a repair

The second shape makes a **copy** of the Pod and modifies the copy:

```
kubectl debug <pod-name> -it --image=ubuntu --share-processes --copy-to=<new-pod-name>
```

The documentation's framing: *"Sometimes Pod configuration options make it difficult to troubleshoot in certain situations. For example, you can't run `kubectl exec` to troubleshoot your container if your container image does not include a shell or if your application crashes on startup. In these situations you can use `kubectl debug` to create a copy of the Pod with configuration values changed to aid debugging."* [source: k8s-docs-kubectl-debug-copy-node-profiles-2026-08-31]

☆ Fixed Point

`--copy-to` makes a copy. The original Pod is not touched — and that is the feature, not a limitation.

You get a new Pod with whatever changes you need (a different command, an extra container, a shell as the entrypoint) while the broken Pod keeps running exactly as it was, still serving traffic, still available for the platform team to look at, still in the state that produced the bug.

This is the part most readers get backwards on first encounter. The mental model people arrive with is that a debugging tool operates on the thing being debugged: you attach a debugger to the process, you modify the running system. `--copy-to` does the opposite, and on a cluster you do not own, running a workload you did not deploy, that inversion is the whole point. You can experiment freely on a copy. You cannot experiment freely on production.

The copy is also the answer to a case `exec` can never handle: a container that crashes on startup. There is no running process to enter, and by the time you type the command it is gone again. Copy the Pod, change the entrypoint to a shell, and now you have a container built from the same image, with the same config, that sits still while you look at it.

🔒 **Worth Securing:** A copy is a real Pod. It consumes resources and it will sit there until you delete it. Clean up after yourself — `kubectl delete pod <copy-name>`.

`kubectl debug node/`: stepping back over the line

The third shape targets a node rather than a Pod:

```
kubectl debug node/<node-name> -it --image=ubuntu
```

This creates a Pod on the target node with access to the node's filesystem and host namespaces [source: k8s-docs-kubectl-debug-copy-node-profiles-2026-08-31]. It is genuinely useful and it is, unmistakably, the moment you step back across the boundary §1 drew. A node is not application scope. If you find yourself reaching for `debug node/`, you have either concluded that the fault is platform-side — in which case the right move is usually to hand it to whoever owns the platform, with the evidence you gathered — or you *are* the person who owns the platform, wearing a different hat.

Which makes it an argument for the boundary rather than an exception to it. The tool exists, it is on the same documentation page as the others, and the reason it feels out of place here is that it *is* out of place here. Notice the feeling. It is the boundary doing its job. Node-level diagnosis, including the node-local tooling below the Kubernetes API, belongs to the platform-scope chapter [*cross-bearing: see Ch 13 §5 — when the node is the problem*].

⚠️ Navigational Hazards

A debug container is a container, and admission can refuse it.

You have RBAC permission to run `kubectl debug`. You run it. It fails — not with a permissions error on the verb, but with a rejection from admission control. This is not a bug and it is not RBAC.

The ephemeral container you are injecting is a container in that namespace, and Pod Security Admission enforces the namespace's standard against containers [*cross-bearing: see Ch 12 §6 — three levels, three modes*]. In a namespace enforcing the `restricted` standard, a debug image that wants to run as root, or wants elevated capabilities, or wants host namespace access, has asked for more than the namespace permits. A container that would not be allowed to exist there does not become allowed by being ephemeral.

This is an easy failure to misread as "I don't have permission to debug." You do. The *container you asked for* doesn't meet the namespace's standard. Try a debug image that runs as non-root, or ask the namespace's owner what the enforcement level is.

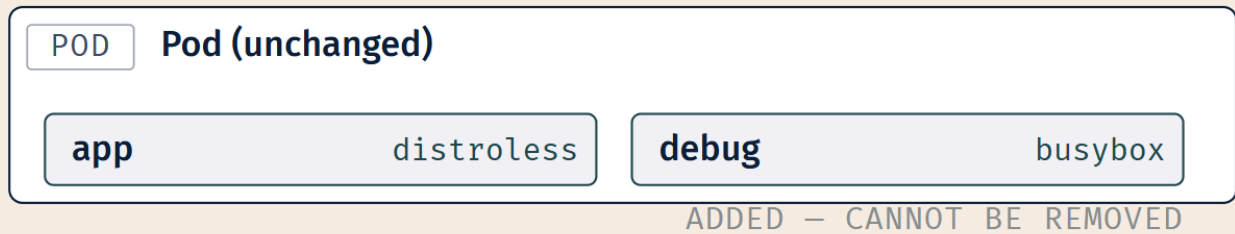
Debug profiles

`kubect1 debug` also accepts a `--profile` flag that sets a bundle of security-related properties on the debug container: whether it runs privileged, whether it gets host namespaces, and so on. The available profile names include `general`, `baseline`, `restricted`, `netadmin`, and `sysadmin`, with `general` as the default in the generated CLI reference [source: [k8s-docs-kubect1-debug-reference-2026-08-31](#)].

The shape to remember, rather than the list: a profile is a preset for how much privilege the debug container asks for, and asking for more than the namespace allows is what triggers the admission refusal in the Hazard above. The `restricted` profile exists as the low-privilege end of that range, which is the end you want in a namespace enforcing the restricted standard.

(A) EPHEMERAL CONTAINER

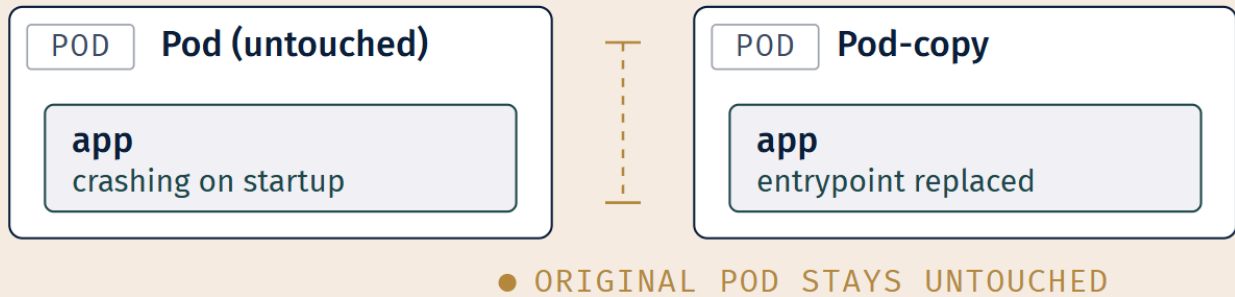
into the running Pod



ASKS: "what does the running process see right now?"

(B) --COPY-TO

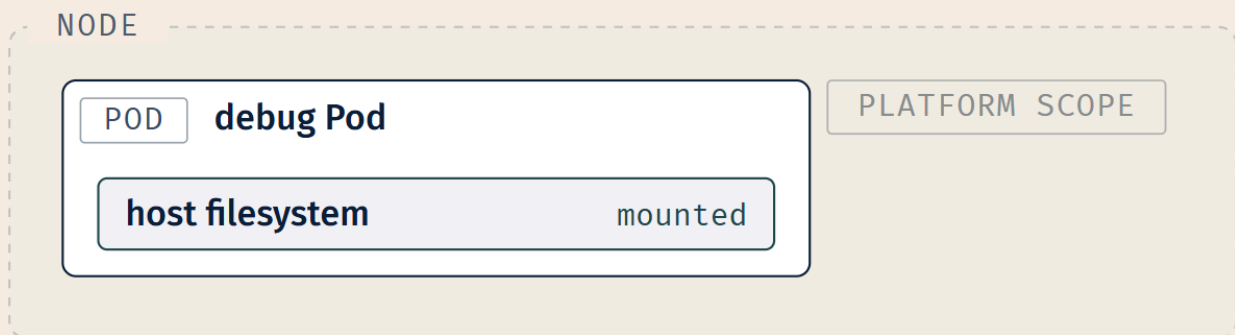
a new Pod, original untouched



ASKS: "what would happen if I changed something?"

(C) NODE/

the host, not the workload



ASKS: "is the machine itself the problem?"

(see Ch 13 §5)

LEGEND
☐ POD — CONTAINER GROUP

☐ NODE — HOST BOUNDARY

NO CONNECTION — FOCAL

☐ PLATFORM-SCOPE TAG

Three shapes, three questions. They are not interchangeable, and choosing the wrong one is how you spend twenty minutes in a copy of a Pod investigating a problem that only exists in the original.

🕒 Taking Your Bearings: Taking the Handoff, and Getting Inside

Six questions on §1 through §3. One of them tests material from an earlier chapter.

1. A Pod is Running, reports 1/1 Ready, has zero restarts, and its node shows no adverse conditions. Two other workloads in the same namespace are behaving normally. The application returns HTTP 500 on one endpoint. What does the scope test tell you?

- A) Platform scope — a 500 indicates a runtime failure the kubelet should have caught
- B) Application scope — the fault is confined to one workload and every platform signal is clean
- C) Indeterminate — you need node-level access before you can decide
- D) Platform scope — readiness probes passing means the platform is asserting the app is correct

2. A Pod shows STATUS: Init:1/3. Which command gives you the most useful next piece of information?

- A) `kubectl logs <pod>`
- B) `kubectl logs <pod> -c <second-init-container-name>`
- C) `kubectl exec -it <pod> -- sh`
- D) `kubectl port-forward <pod> 8080:8080`

3. Why do ephemeral containers exist as a separate API mechanism rather than as an ordinary edit to a Pod's spec?

- A) Because ephemeral containers must be declared before the Pod is scheduled
- B) Because a Pod's container list cannot be added to once the Pod exists
- C) Because ephemeral containers are scheduled to a different node
- D) Because RBAC cannot grant update on Pods

4. You run `kubectl debug <pod> --copy-to=<pod>-debug --image=ubuntu`. Immediately afterward, what is the state of the original Pod?

- A) Terminated and replaced by the copy
- B) Running, but with the ubuntu container now attached to it
- C) Running, entirely unchanged
- D) Paused until the debug session ends

5. An init container's job is `mkdir /data/cache`. The workload deploys cleanly. Three weeks later the node is drained and the Pod is rescheduled; the Pod now fails to start. What is the most likely explanation?

- A) The PersistentVolume failed to reattach on the new node

- B) The init container is not idempotent and fails when the directory already exists
- C) The init container image was garbage-collected from the new node
- D) Init containers do not run on rescheduled Pods

6. [retrieval: ch2] You run `kubectl exec -it <pod> -- /bin/sh` and `get stat /bin/sh: no such file or directory`. What does this tell you about the image?

- A) The image is corrupted and needs to be repulled
- B) The container runtime does not support `exec`
- C) The image was built without a shell — it contains only what was put into it
- D) `/bin/sh` exists but the container is running as a user without execute permission on it

Answers with Explanations:

1 — **B**. Every element of the mechanical test points the same way: the workload is running, ready, and stable; the fault does not extend beyond it; and no platform signal is adverse.

- **A is wrong** because HTTP status codes are application output. The kubelet has no opinion about your response bodies, and a 500 is not a container failure. The container is working fine, returning exactly what your code told it to return.
- **C is wrong** and this is the important distractor. It is true that you may lack node-level access. It is not true that this makes the diagnosis indeterminate; §1's addition to the test says the opposite. You make the case from what you *can* see, and here what you can see is conclusive.
- **D is wrong** because a readiness probe asserts that the container is willing to receive traffic, not that it produces correct answers. A probe that hits `/healthz` and gets a 200 will pass happily while every other endpoint returns garbage [*cross-bearing: see Ch 5 §7 — liveness, readiness, and startup probes*].

2 — **B**. `Init:1/3` means the first init container completed and the second is where you are. Name it with `-c` and read its output.

- **A is wrong** — the most common time-waster in this whole section. The plain form targets an app container that has not started; you get nothing useful, and the nothing looks like silence rather than like a misdirected question.
- **C is wrong** because there is no app container running to `exec` into. You would need `-c` here too, and even then, `exec` into a *running* init container is a narrow move that only helps if the init container is hanging rather than exiting.
- **D is wrong** because nothing is listening. `port-forward` requires a running process on the target port; this Pod has not reached its application yet.

3 — **B**. This is the Fixed Point stated as a question. The container list is fixed at Pod creation, so a mechanism to add a *temporary* container had to be built outside the normal spec-editing path — hence

the dedicated `ephemeralcontainers` API handler [source: [k8s-docs-ephemeral-containers-concept-2026-08-31](#)].

- **A is wrong, and it inverts the mechanism.** The entire point of an ephemeral container is that it is added *after* the Pod exists: *"you can run an ephemeral container in an existing Pod to inspect its state"* [source: [k8s-docs-ephemeral-containers-concept-2026-08-31](#)]. A container declared before scheduling is just an ordinary container.
- **C is wrong** — they run in the existing Pod, on the node the Pod is already on. That is the entire point; a container on another node would see none of the state you are trying to inspect.
- **D is wrong** because the constraint is architectural, not authorization-based. Even with unlimited permissions on Pods, you could not add a container to one that already exists, which is exactly what the Fixed Point says.

4 — C. Unchanged, still running, still serving. The copy is a separate Pod.

- **A is wrong** and is the misconception this option exists to catch. Nothing is deleted. If `--copy-to` terminated the original, it would be useless for its main purpose: debugging something you cannot afford to disturb.
- **B is wrong** — that describes the plain ephemeral-container form (`kubectl debug` without `--copy-to`), which is a different shape answering a different question.
- **D is wrong** — Kubernetes has no notion of pausing a Pod for inspection.

5 — B. A fresh deploy hits an empty volume and `mkdir` succeeds. A rescheduled Pod re-runs every `init` container [source: [k8s-docs-init-containers-2026-08-24](#)] against a volume where `/data/cache` already exists, `mkdir` exits non-zero, and the Pod is stuck. Classic non-idempotency.

- **A is possible in principle but is the wrong first answer** — and it is the wrong first answer in a specific, instructive way. A PV reattachment failure is platform scope, would produce a different signature (the Pod stuck on volume mounting, with events saying so), and would not be preceded by three weeks of clean operation followed by an `init` failure specifically. Reach for the application-scope explanation when the application-scope signature is what you're looking at.
- **C is wrong** — a garbage-collected image is repulled, and a pull failure has its own distinctive signature [*cross-bearing: see Ch 13 §2 — pods that never start*].
- **D is wrong** and is the outright false statement in the set. *"If the Pod restarts, or is restarted, all init containers must execute again"* [source: [k8s-docs-init-containers-2026-08-24](#)], which is exactly what makes idempotency mandatory.

6 — C. An image contains exactly what was put into it. No shell in the image means no shell in the container; there is no host filesystem underneath supplying the gaps [*cross-bearing: see Ch 2 §2 — what's inside an image*].

- **A is wrong** — a corrupt image fails at pull or at container creation, not with a clean "this specific path does not exist."

- **B is wrong** — the runtime supports `exec` fine. It attempted the `exec` and reported, accurately, that the binary you named is not there.
- **D is wrong** — a permission problem produces a permission error. no such file or directory is a statement about existence.

How'd You Do?

If you scored 0–2: Stop here rather than pressing on. §4 assumes the boundary is settled and the tools are in hand, and it will read as a list of commands if either is shaky. Re-read **§2** and **§3**. If question 1 was among the misses, re-read **Chapter 13 §1** first — the scope test is the thing everything after it rests on.

If you scored 3–4: Solid footing. Read the why-wrong explanation for each one you missed — most of the value in this checkpoint is in the wrong answers — then continue.

Checkpoint: You've Now Mastered

✓ Placing a fault on the correct side of the scope boundary, including when you cannot see the other side ✓ The four triage questions and why "is it configured" appears twice ✓ Reading a failing init container, and the three ways one goes wrong ✓ `exec` for the container that has a shell, and the two places the manifest and reality diverge ✓ Ephemeral containers, `kubect1 debug`, and the three shapes it takes

Two questions remain: is it reachable, and what changes when the replicas aren't interchangeable. The next section is the one where a perfectly correct Service does nothing at all.

..1 §4 — Is Anything Even Selected

You have a Service. It exists. `kubect1 get service` returns it, complete with a ClusterIP. It is a name on the chart pointing at a position, and requests to it fail.

Chapter 9 gave you the model: a Service is a name and a selector, and behind the name is a list of endpoints assembled by matching that selector against Pod labels [*cross-bearing: see Ch 9 §4 — the list behind the name*]. Chapter 9 also told you, in as many words, that this chapter owns the troubleshooting workflow and would come back for those facts by name. Here we are.

The single most useful command in this section is the one that reads the list:

```
kubect1 get endpointslices -l kubernetes.io/service-name=<service-name>
```

That label is not arbitrary. Every `EndpointSlice` the control plane creates for a Service carries a `kubernetes.io/service-name` label, and the docs say it exists precisely to enable "simple lookups of all

EndpointSlices belonging to a Service" [source: k8s-docs-endpointslices-2026-08-24] [cross-bearing: see Ch 9 §4 — the list behind the name]. And the interpretation is direct, from the docs: *"Make sure that the endpoints in the EndpointSlices match up with the number of pods that you expect to be members of your service. If they don't, the Service's selector probably does not match the Pods' labels, or the Pods are not Ready."* [source: k8s-docs-debug-pods-2026-08-23]

☆ Fixed Point

A Service with no ready endpoints is not broken. It is working exactly as written, and finding nothing it is willing to send traffic to. There are two causes, and they live in two different files: the selector does not match the Pod labels, or the Pods match but are not Ready.

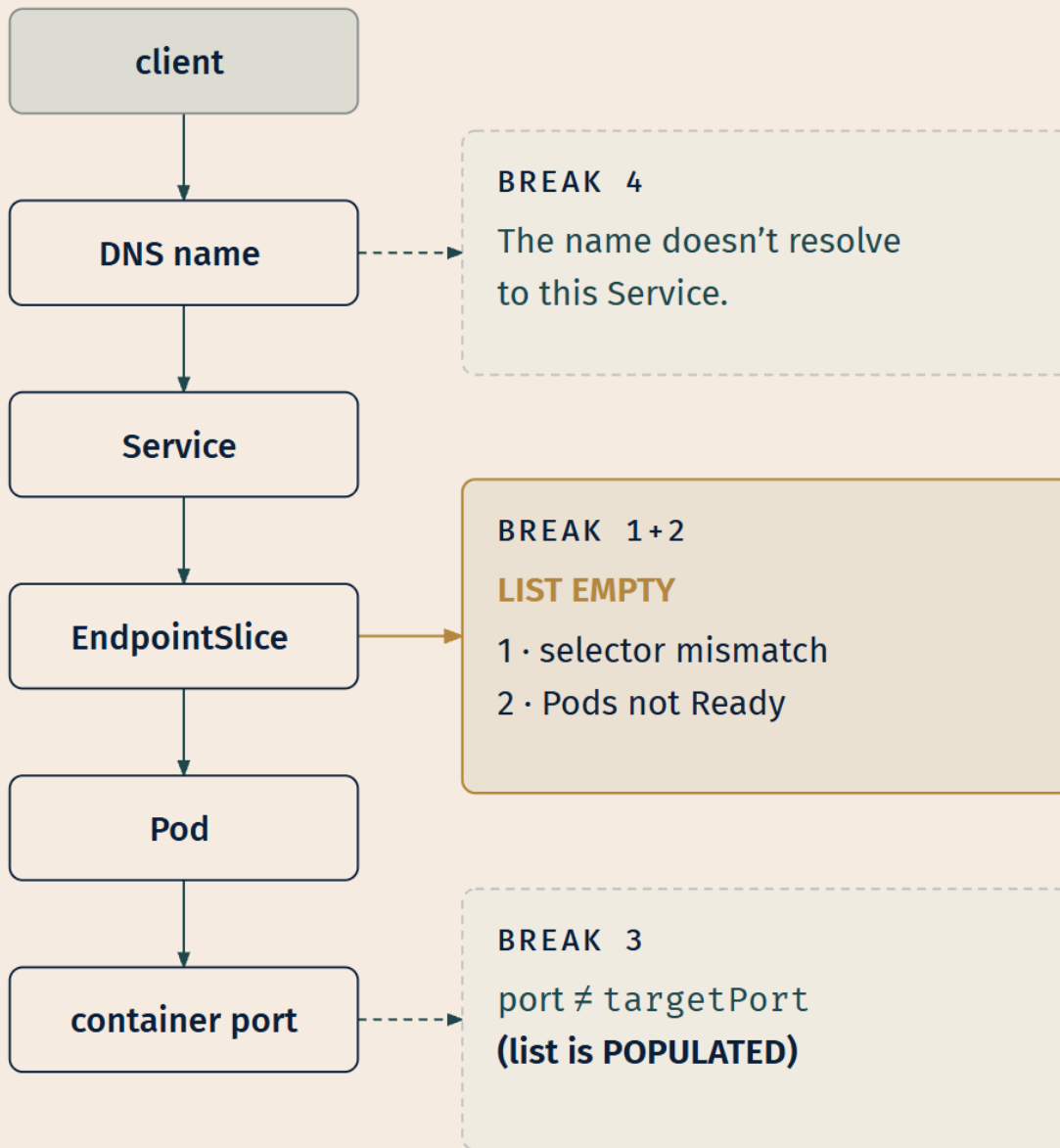
That distinction is the whole diagnostic value of the reading. The Service object is fine. The controller is fine. Nothing has failed. Something is *mismatched*, and the mismatch is either in the label/selector relationship or in the readiness state, and knowing which of those two you are looking at tells you which file to open.

And the slice itself tells you which. This is worth getting exactly right, because the two causes leave different traces. The control plane *"automatically creates EndpointSlices for any Kubernetes Service that has a selector specified,"* and those slices *"include references to all the Pods that match the Service selector"* [source: k8s-docs-endpointslices-2026-08-24]. **All** the matching Pods — readiness is not a filter on membership. What readiness controls is a condition on each endpoint: the *serving* condition *"indicates that the endpoint is currently serving responses, and so it should be used as a target for Service traffic,"* and for endpoints backed by a Pod, *"this maps to the Pod's Ready condition"* [source: k8s-docs-endpointslices-2026-08-24].

So: a selector that matches nothing leaves the slice with no endpoints at all. Readiness that is failing leaves the endpoints exactly where they are, marked not ready, and service proxies skip them. Same practical outcome for your traffic; two different readings on the screen, and the difference is the diagnosis.

Four break points, and two of them are not about the list

Here is where the section has to be careful, because four things can break a request to a Service and only two of them are about whether anything is selected.



LEGEND

UPSTREAM of the list — breaks 1 and 2 → EMPTY LIST

DOWNSTREAM of the list — break 3 →
POPULATED LIST, request still fails

BESIDE the whole path — break 4 →
you never reached this Service at all

Break 1 — selector/label mismatch. The Service's `spec.selector` and the Pod template's `metadata.labels` are written in two different places, frequently in two different files, and they drift. Someone renames `app: web` to `app: web-frontend` in the Deployment and does not touch the Service. Everything applies cleanly. Nothing errors. The slice goes empty and stays that way.

The docs are blunt about how ordinary this is: *"Make sure that the Pods you ran are actually selected by the Service"* [source: k8s-docs-debug-service-2026-08-31]. Check both sides:

```
kubectl describe service <service-name>      # what does it select?
kubectl get pods -l <selector-from-above>     # does anything match?
kubectl get pods --show-labels                # what do the Pods actually carry?
```

Break 2 — not Ready. The Pods match perfectly. The selector is correct. They are all in the slice. And every one of them is `0/1 READY`, so every endpoint in that slice is marked not ready and no service proxy will send it anything. Readiness does not remove an endpoint; it disqualifies it [cross-bearing: see Ch 9 §4 — the list behind the name].

This is the quiet one, and it is quiet because the Pods look alive. `kubectl get pods` shows `Running`. The restart count is zero. Nothing is crashing. The `READY` column is the only place the truth is printed, and it is a column people's eyes slide past [cross-bearing: see Ch 13 §4 — pods that start and then don't stay]. If nothing is receiving traffic and the selector checks out, look at the readiness state before you look at anything else.

🔖 **Snag:** No ready endpoints has exactly two causes, and the slice itself separates them. **Zero endpoints in the slice** means nothing matched the selector, a label problem, and the labels live in the Deployment's Pod template. **Endpoints present but not ready** means readiness is holding them back, and that lives in the probe definition. If your tooling only shows you a ready count, `kubectl get pods -l <the-service-selector>` settles it the same way in one command: nothing returned is a mismatch, Pods returned is a readiness problem.

Break 3 — port vs targetPort. Now a different failure shape entirely. The selector matches. The Pods are ready. The endpoints are ready. And requests still fail, because the Service is delivering traffic to a port nothing is listening on.

`port` is the port the Service is reachable at; `targetPort` is the port on the Pod that traffic is forwarded to, and it is the one that has to match what the container actually binds [cross-bearing: see Ch 9 §3 — four ways to be reachable]. The docs put the question directly: *"Is the Service correct?"* and *"Is the Service defined correctly?"* [source: k8s-docs-debug-service-2026-08-31], with the port pairing as one of the things being asked about.

```
spec:
  ports:
    - port: 80          # clients connect here
      targetPort: 8080  # the container must be listening HERE
```

If the container listens on 8080 and `targetPort` says 80, everything selects correctly and every request lands on a closed port.

Break 4 — the name. And the fourth one is not about this Service at all: the DNS name the client is using does not resolve to the Service you have been staring at. A typo in the namespace, a name that resolves in the client's own namespace to something else, a hardcoded name from a different environment. Normal Services get a record *"of the form `my-svc.my-namespace.svc.cluster-domain.example`", and "by default, a client Pod's DNS search list includes the Pod's own namespace"*, so a short name resolves relative to the *client's* namespace, not the service's [source: k8s-docs-dns-pod-service-2026-08-23] [cross-bearing: see Ch 9 §7 — names, and where they resolve].

The docs' first debugging move for this is to run a Pod in the cluster and look from where the client looks: *"For many steps here you will want to see what a Pod running in the cluster sees. The simplest way is to run an interactive busybox Pod:"* [source: k8s-docs-debug-service-2026-08-31] From inside that Pod, resolve the name and see what comes back, or whether anything does.

🔒 **Worth Securing:** Keep breaks 1–2 and breaks 3–4 on separate pages of the log. Breaks 1 and 2 produce **no ready endpoints**. Break 3 produces **ready endpoints and a failed request**. Break 4 means you never reached this Service in the first place. If you conflate them, you will go hunting for a label mismatch behind a Service whose endpoints are all ready, and the slice was telling you the answer the whole time.

§5 — Bypassing the Service on Purpose

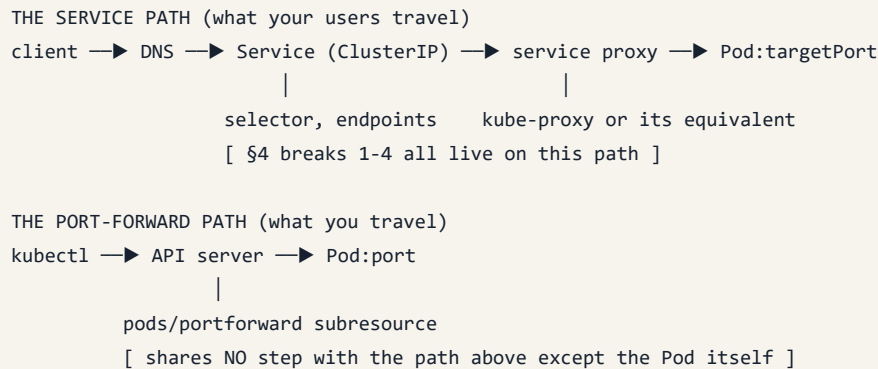
There is a move available at this point that is more useful than it looks.

```
kubectl port-forward <pod-name> 8080:80
```

This opens a tunnel from a port on your local machine to a port on a Pod in the cluster. The docs describe the mechanism plainly: *"`kubectl port-forward` allows using resource name, such as a pod name, to select a matching pod to port forward to."* [source: k8s-docs-port-forward-2026-08-31] It is commonly used for convenience — reaching a database or an admin interface from a laptop without exposing it — and that use is real and fine and not what this section is about.

What this section is about is what happens when you use it as an experiment.

Two paths that share almost nothing



The port-forward path is an API-server operation, not a networking one. The authorization requirements make this concrete: *"To use `kubectl port-forward`, a user must have permission to access the target resource (for example, a Pod or Service) and the `portforward` subresource. Typical required permissions include `get` on `pods` and `create` on `pods/portforward`."* [source: k8s-docs-port-forward-authorization-2026-08-31] You are not routing traffic through the cluster's Service machinery. You are asking the API server to open a channel to a Pod, and it does.

The same documentation notes the security consequence, which is also the diagnostic consequence: *"Cluster administrators should carefully restrict these permissions, as port-forwarding can provide direct network access to workloads and may bypass network-level controls."* [source: k8s-docs-port-forward-authorization-2026-08-31]

"May bypass network-level controls" is, from the diagnostic side, the entire point. If the path skipped nothing, the experiment would prove nothing.

The inference, and the trap inside it

So: your Service call fails. You port-forward straight to a Pod behind that Service. It works. The application responds correctly.

The instinct at this moment is relief — *the app is fine*. And that instinct, left alone, is where the diagnosis stops being useful, because "the app is fine" is not a conclusion. It is half of one.

☆ Fixed Point

A working port-forward beside a failing Service call does not mean the application is fine. It means the application is fine and the Service path is not. It is a narrowing step, not a clean bill of health — and the thing it narrows to is exactly the four break points in §4.

Read it as an elimination. The port-forward path shares nothing with the Service path except the Pod itself. If traffic arriving directly at the Pod produces a correct response, then the process is running,

listening, and serving correctly, and every remaining candidate lives on the path you just skipped: the DNS name, the selector, the endpoints, the service proxy, the port pairing.

🔑 **Mnemonic:** Port-forward is a **second approach to the same anchorage**. If the boat can be reached from seaward but not up the channel, the boat is fine and the channel is the problem. You have not fixed anything. You have halved the map.

And the negative case is just as informative. If the port-forward *also* fails — connection refused, or a hang, or the same wrong response — then the Service path is exonerated and the fault is inside the container. That sends you back to §3: exec in, check what the process is actually bound to, check what config it actually read.

🔒 **Worth Securing:** Note which port you forwarded to. `kubectl port-forward pod/x 8080:80` reaches container port 80. If the container is listening on 8080 and you happened to forward to 8080, you have accidentally routed around break 3 as well, and a successful port-forward on the *right* port while the Service points at the *wrong* one is precisely the `port/targetPort` signature. Forward to the port the container claims to use, then compare that number against the `targetPort` the Service declares — `kubectl describe service <name>` prints it.

One clarifying note on scope: port-forward is a diagnostic here, and only a diagnostic. It is not how applications reach each other in a cluster and it is not an exposure mechanism; the Service and Ingress machinery for that belongs to earlier chapters [*cross-bearing: see Ch 9 §6 — the component that makes it real*]. Also: it is TCP only, and it terminates when you stop the command [source: k8s-docs-port-forward-2026-08-31].

🕒 Taking Your Bearings: Reachability — What Selects, What Routes, What Proves

Five questions on §4 and §5. Two of them test material from earlier chapters. This is the checkpoint where the reachability material leans hardest on Chapter 9 and Chapter 5, so it is where that dependency gets verified.

1. `kubectl get endpointslices -l kubernetes.io/service-name=api` returns a slice with three endpoints, every one of them showing `ready: false`. `kubectl get pods -l app=api` returns three Pods, all Running, all 0/1 READY. What is the cause?

- A) A selector/label mismatch between the Service and the Pod template
- B) Readiness is disqualifying every endpoint behind the Service
- C) A `port/targetPort` mismatch on the Service
- D) The EndpointSlice controller has failed

2. [retrieval: ch9] A Service is defined with `port: 80` and `targetPort: 8080`. Which port must the container inside the Pod be listening on?

- A) 80
- B) 8080
- C) Either — Kubernetes tries both
- D) Neither; the container port is set by `containerPort` and is independent of the Service

3. You port-forward directly to a Pod behind a failing Service and the application responds correctly. What have you established?

- A) The application is working, so the incident is resolved
- B) The application serves correctly, and the fault lies somewhere on the Service path
- C) The Service is correctly configured but the Pod is unhealthy
- D) Nothing — port-forward and the Service path are equivalent

4. [retrieval: ch5] A Pod is Running with a restart count of 0, and its readiness probe is failing. What is the state of its liveness probe, and what does that combination mean for traffic?

- A) Liveness must also be failing; the container will be restarted shortly
- B) Liveness is passing (or absent) — the container is not restarted, but it receives no Service traffic
- C) Liveness is irrelevant to readiness; the Pod receives traffic normally
- D) Readiness failure forces liveness failure after the failure threshold

5. A client Pod in namespace `frontend` calls `http://api/` and gets no response. The Service `api` exists in namespace `payments`, with three ready endpoints. What is the most likely cause?

- A) Services are not reachable across namespace boundaries without an Ingress
- B) The short name `api` resolves relative to the client's own namespace, not `payments`
- C) The Pods behind `api` are not Ready
- D) `port` and `targetPort` are mismatched on the `api` Service

Answers with Explanations:

1 — **B**. The endpoints are present in the slice, which means the Pods matched the selector; matching Pods are included regardless of readiness [source: [k8s-docs-endpointslices-2026-08-24](#)]. `ready: false` on all three, corroborated by `0/1 READY`, means readiness is disqualifying every one of them.

- **A is wrong** on two counts, and both are worth having. The label query returned Pods, so the selector matches; and a selector that matched nothing would leave the slice with **no endpoints at all**, not three unready ones. The trace on the screen distinguishes the two causes.
- **C is wrong** because a `port/targetPort` mismatch leaves the endpoints `ready` and the request failing further down. It is downstream of readiness; it cannot mark an endpoint not ready.
- **D is wrong** and it is the "blame the platform" distractor. What you are reading is the controller working correctly: it found the matching Pods, put them in the slice, and reported their condition faithfully. Reaching for controller failure is the reflex §1 exists to break.

2 — B. `targetPort` names the port on the Pod that traffic is delivered to, so the container must be listening on 8080. `port: 80` is the port clients use to reach the Service *[cross-bearing: see Ch 9 §3 — four ways to be reachable]*.

- **A is wrong** — that inverts the two, which is exactly the confusion this question exists to catch.
- **C is wrong** — there is no fallback behavior. Traffic goes to `targetPort` and nowhere else.
- **D is half-true and therefore the most dangerous option.** `containerPort` in the Pod spec is largely informational, and the container's actual listening port is set by the application, not by Kubernetes. But `targetPort` is absolutely not independent of it: `targetPort` is where traffic is sent, and if the container isn't listening there, the request fails. The two must agree.

3 — B. The narrowing step. The port-forward path shares no step with the Service path except the Pod itself, so a correct response proves the process serves correctly, and moves every remaining candidate onto the Service path.

- **A is wrong** and is the trap this section is built around. Users travel the Service path. You just proved the Service path is broken. Nothing is resolved.
- **C is wrong** — it inverts the finding. You reached the Pod directly and it answered; the Pod is demonstrably healthy in the way that matters.
- **D is wrong** — the two paths differ almost entirely, which is what gives the experiment its power. If they were equivalent, both would fail identically.

4 — B. A zero restart count tells you liveness is not failing; a failing liveness probe restarts the container and increments that counter. So liveness is passing or not configured, the container keeps running, and readiness independently disqualifies it as a target for every Service *[cross-bearing: see Ch 5 §7 — liveness, readiness, and startup probes]*.

- **A is wrong** — the restart count of 0 rules it out directly, and the two probes are independent. That independence is the design: readiness failing means "don't send me traffic," liveness failing means "kill me."
- **C is wrong** — readiness gates whether an endpoint is a valid target. A not-ready Pod gets no Service traffic at all.
- **D is wrong** — there is no such escalation. Neither probe influences the other's result.

5 — B. A short Service name resolves through the client's search domains, which include the client's own namespace *[source: k8s-docs-dns-pod-service-2026-08-23]*. A Pod in `frontend` asking for `api` looks for `api.frontend.svc.cluster.local` first. The fully qualified form `api.payments.svc.cluster.local`, or at minimum `api.payments`, is required to cross namespaces *[cross-bearing: see Ch 9 §7 — names, and where they resolve]*.

- **A is wrong**, and it is the most tempting option in the set because it sounds like a security posture. Namespaces scope *names*, not network reachability; a fully qualified Service name crosses a namespace boundary with no Ingress involved *[cross-bearing: see Ch 4 §3 — where a name lives]*.

Ingress is for traffic arriving from outside the cluster entirely [*cross-bearing: see Ch 10 §2 — routing by host and path*].

- **C is wrong** because the stem states the Service has three ready endpoints, which rules readiness out directly. Read the given facts before choosing a cause they exclude.
 - **D is wrong** because a port mismatch would produce a connection to the Service that fails at the Pod — a refused connection or a timeout from a real target — rather than the resolution failure the namespace crossing produces. Different signature, and distinguishing signatures is the skill.
-

How'd You Do?

If you scored 0–2: Re-read §4 first, then **Chapter 9 §4** ("the list behind the name"). Most misses at this checkpoint are Chapter 9's material rather than this chapter's; this section is a workflow built on a model you were given earlier, and the workflow does not stick without it. If question 2 was among the misses, add **Chapter 9 §3**.

If you scored 3–4: Re-read the why-wrong for anything you missed, paying particular attention to which stems already excluded the cause you picked. Half the skill here is reading the evidence before reaching for a diagnosis.

Checkpoint: You've Now Mastered

✓ Reading an EndpointSlice, and what "no ready endpoints" actually means ✓ Telling the two causes apart from the slice itself — empty means selector, not-ready means readiness ✓ Keeping upstream breaks (nothing ready) separate from downstream ones (ready endpoints, failed request) ✓ Using port-forward as an elimination step rather than a verdict

Two sections left. The first is what changes when the replicas are not interchangeable — where "the app is broken" turns out to mean "web-2 is broken, and the other two are fine."

§6 — When Each Replica Is Its Own

Everything so far has assumed something that is usually true and sometimes catastrophically false: that your replicas are interchangeable. That if you diagnose one, you have diagnosed all of them.

For a Deployment, that assumption holds. Three replicas of a stateless service are three instances of the same thing; whichever one you exec into will tell you the same story. For a StatefulSet it does not hold at all, and the four questions have to be asked of a *particular* replica rather than of the workload.

Three things make this different, and each is a retrieval you already have with a diagnostic turn on it.

Find out which one

A StatefulSet's Pods have stable ordinal identity — `web-0`, `web-1`, `web-2` — and *"each has a persistent identifier that it maintains across any rescheduling"* [source: `k8s-docs-statefulset-2026-08-24`] [*cross-bearing: see Ch 6 §6 — when Pods are not interchangeable*]. The diagnostic consequence: **"the app is broken" is very frequently "web-2 is broken, and web-0 and web-1 are fine."**

So the first move is not to investigate. It is to find out which replica you are investigating.

```
kubect1 get pods -l app.kubernetes.io/name=MyApp
```

The docs give exactly this form for listing a StatefulSet's Pods by label [source: `k8s-docs-debug-statefulset-2026-08-31`]. Look at the whole list before you pick one. A single unhealthy ordinal among healthy siblings is a completely different diagnosis from all three being unhealthy: the first says something is wrong with that replica's *state*, the second says something is wrong with the *workload*.

The docs also flag one specific case worth knowing: a Pod in *Unknown* or *Terminating* state can block the StatefulSet controller from making progress, because the controller's ordering guarantees mean it will wait rather than proceed past an uncertain replica [source: `k8s-docs-debug-statefulset-2026-08-31`]. A StatefulSet that seems frozen mid-rollout usually has one Pod in one of those states, and the freeze is the controller obeying its own rules [*cross-bearing: see Ch 6 §6 — when Pods are not interchangeable*].

The state that survives everything you try

This is the one that most looks like a platform fault and is not.

Each StatefulSet replica gets its own *PersistentVolumeClaim* from *volumeClaimTemplates* — *"for each VolumeClaimTemplate entry defined in a StatefulSet, each Pod receives one PersistentVolumeClaim"*, and *"the same PersistentVolumeClaim will be bound to a Pod throughout its lifecycle"* [source: `k8s-docs-statefulset-2026-08-24`] [*cross-bearing: see Ch 11 §6 — Pods that are not interchangeable, revisited*]. The claim is not deleted when the Pod is deleted. The default retention policy is *Retain* for both the scale-down and the delete case: *"Retain (default): PVCs from the volumeClaimTemplate are not affected when their Pod is deleted"* and *"The default for policies is Retain, matching the StatefulSet behavior before this new feature."* [source: `k8s-docs-statefulset-storage-2026-08-25`] And for the involuntary case: *"if a Pod associated with a StatefulSet fails due to node failure, and the control plane creates a replacement Pod, the StatefulSet retains the existing PVC. The existing volume is unaffected, and the cluster will attach it to the node where the new Pod is about to launch."* [source: `k8s-docs-statefulset-storage-2026-08-25`]

Now put that next to the most common debugging reflex in the industry.

Your application writes a corrupt record. `web-2` starts failing. You delete `web-2`. The controller recreates it, with the same name, the same DNS record, and **the same volume, containing the same corrupt record**. It fails again, identically. You delete it again. Same result.

⚠ Navigational Hazards

"Turn it off and on again" does not clear a StatefulSet replica's state, and the failure it leaves behind looks exactly like a platform bug.

The symptom is a replica that fails, gets deleted, comes back, and fails in precisely the same way — repeatedly, deterministically, immune to every restart. That signature reads as "something in the cluster is broken," and engineers have spent days on it from that angle.

It is not the cluster. The PVC survived, by design, because throwing away a stateful workload's data on a restart would be the worse failure by a wide margin. The state is the thing that is broken, and no amount of restarting will touch it. Go look at the data: exec into the replica and inspect what is on the volume, or mount the PVC into a debug Pod and read it there.

The diagnostic tell that separates this from a genuine platform fault: **it is deterministic and it is confined to one ordinal**. A platform problem would not preferentially afflict web-2 and leave web-0 and web-1 untouched across repeated rescheduling.

🔍 **Closer Look:** `.spec.persistentVolumeClaimRetentionPolicy` has two settings — `whenDeleted` and `whenScaled` — each accepting `Delete` or `Retain`, with `Retain` the default for both [source: k8s-docs-statefulset-storage-2026-08-25]. A workload configured with `whenScaled: Delete` behaves differently on scale-down than the default described above. Check the StatefulSet's actual policy before you reason about what a deletion did; the default is only the default.

Peers that find each other by name

The third difference is discovery. A StatefulSet uses a headless Service to give each Pod its own DNS name [cross-bearing: see Ch 9 §5 — *when you don't want a single address*], and the form is `$(podname).$(governing service domain)` — for example `web-0.nginx.default.svc.cluster.local` [source: k8s-docs-statefulset-2026-08-24]. Replicas use these names to find each other: a database forming a cluster, a queue electing a leader, a cache building a ring.

That creates failure modes a ClusterIP workload never sees. If web-1 cannot resolve web-2's name, the peer relationship fails while both Pods look perfectly healthy from outside. And there is a genuine timing trap here, which the docs call out directly: *"Depending on how DNS is configured in your cluster, you may not be able to look up the DNS name for a newly-run Pod immediately. This behavior can occur when other clients in the cluster have already sent queries for the hostname of the Pod before it was created. Negative caching (normal in DNS) means that the results of previous failed lookups are remembered and reused, even after the Pod is running, for at least a few seconds."* [source: k8s-docs-statefulset-2026-08-24]

So a peer that came up, failed to resolve a sibling that did not exist yet, cached the negative result, and gave up is a real and reproducible failure that has nothing to do with your code and everything to do with startup ordering. The diagnostic move is to resolve the peer names from inside a replica and see what comes back:

```
kubectl exec -it web-1 -- nslookup web-2.nginx
```

📌 **Snag:** A headless Service is required for a StatefulSet's network identity, and **you are responsible for creating it** — *"StatefulSets currently require a Headless Service to be responsible for the network identity of the Pods. You are responsible for creating this Service"* [source: k8s-docs-statefulset-2026-08-24]. Which means a `serviceName` pointing at a Service nobody created leaves you with Pods that run and cannot find each other, and nothing in a Pod's own status says why.

The unifying point across all three: for a StatefulSet, "which replica" is a question you have to answer before any of the other four questions mean anything.

🔧 §7 — Before You Ship It

The fastest debugging loop is the one that runs on your own machine, where you have a debugger, an IDE, and a rebuild that takes two seconds instead of a container build and a rollout.

The judgment call is knowing when that loop is worth building and when the reproduction is worthless before you start.

The dividing line

Some things about your application exist only in the cluster. Not "are easier in the cluster" — exist only there, and cannot be reproduced locally by definition:

- **Cluster-supplied identity.** The ServiceAccount token projected into the Pod, and everything it authorizes [*cross-bearing: see Ch 12 §2 — who you are*].
- **Cluster DNS.** Any name resolution through `svc.cluster.local`, including peer discovery.
- **Injected configuration.** ConfigMaps and Secrets mounted or projected into the container. You can copy the values locally, but you are then testing a copy, and if the bug is that the value *isn't what you think*, you have just reproduced your own misunderstanding.
- **Admission mutation.** Anything a mutating webhook or a sidecar injector added to your Pod after you submitted it. Your local process was never mutated [*cross-bearing: see Ch 8 §2 — three gates and a logbook*].
- **Service routing.** Everything in §4. A local process is not behind a Service, has no selector, and appears in no endpoint list.

Everything else — your business logic, your parsing, your request handling, your math — usually reproduces locally just fine, and reproducing it there is much faster than reproducing it in a cluster.

🔒 **Worth Securing:** Before you build a local reproduction, ask one question: *does the failing behavior depend on anything the cluster supplies?* If yes, a local reproduction will either fail to reproduce the bug or reproduce a different one, and either outcome is worse than not trying, because both are misleading. That question takes ten seconds and routinely saves an afternoon.

The pattern that resolves it

There is a third option between "reproduce it locally" and "debug it in the cluster," and it is the one worth knowing by shape: **proxy a local process into the cluster, so that your code runs on your machine with your debugger attached, while seeing the cluster's real dependencies.**

The Kubernetes documentation describes the motivation exactly: *"Kubernetes applications usually consist of multiple, separate services, each running in its own container. Developing and debugging these services on a remote Kubernetes cluster can be cumbersome, requiring you to get a shell on a running container in order to run debugging tools."* [source: k8s-docs-local-debugging-telepresence-2026-08-31] The tool the docs walk through for this is **Telepresence**, described as *"a tool to ease the process of developing and debugging services locally while proxying the service to a remote Kubernetes cluster,"* which *"allows you to use custom tools, such as a debugger and IDE, for a local service and provides the service full access to ConfigMap, secrets, and the services running on the remote cluster."* [source: k8s-docs-local-debugging-telepresence-2026-08-31]

That last clause is the whole pattern in one line: your process, their cluster's dependencies. The list above stops being a list of things you cannot reproduce, because you are not reproducing them. You are using the real ones.

Telepresence is one instance of the pattern and the one the Kubernetes docs happen to document. The tooling in this space changes; the pattern does not. Learn the shape — a local process, proxied into the cluster's network and configuration — and you will recognize whichever tool is current when you need one.

🔦 **Closer Look:** There is also a fourth option worth naming, which is running a small local cluster — kind, minikube, or k3s — rather than proxying into a shared one [*cross-bearing: see Ch 8 §5 — who owns the control plane*]. That gets you real cluster DNS, real ServiceAccounts, real admission, and real Services, on your laptop. What it does *not* get you is *their* cluster's config, *their* webhooks, and *their* network policy, so it reproduces the class of bug, not the instance. Useful for "does my manifest work at all," not for "why does it fail in staging."

That closes the practical arc. Four questions, five tools, one boundary, and one thing left to say about why the boundary was the point all along.

🕒 Taking Your Bearings: Identity, Storage, and the Limits of Reproducing It Locally

Five questions on §6 and §7. One tests material from an earlier chapter.

1. A three-replica StatefulSet has one failing Pod, db-1. You delete it. The controller recreates db-1, which fails identically. You delete it twice more with the same result. What is the most likely cause?

- A) A node-level fault on whichever node db-1 keeps landing on
- B) Corrupt or unexpected state on db-1's PersistentVolumeClaim, which survives every deletion
- C) The StatefulSet's image is broken and needs to be repulled
- D) An admission webhook is rejecting db-1 specifically

2. [retrieval: ch11] Chapter 11 taught what happens to a StatefulSet's PersistentVolumeClaim when a replica is rescheduled or deleted. What is the default — is the claim retained or removed — and why does that default make a failing replica's storage worth investigating before you delete the Pod again?

- A) Delete for both
- B) Retain for both
- C) Retain for whenDeleted, Delete for whenScaled
- D) There is no default; the field must be set explicitly

3. A StatefulSet's Pods run normally, but the replicas cannot discover each other and the cluster never forms. All Pods are Running and Ready, and they have been up for two hours. What should you check first?

- A) DNS negative caching from lookups issued before the peers existed
- B) The headless Service named by serviceName, and whether per-Pod DNS names resolve
- C) The port/targetPort pairing on the workload's ClusterIP Service
- D) The PersistentVolumeClaims for each ordinal

4. A bug appears only when the application runs in the cluster. The failing code path reads a value that a mutating admission webhook injects into the Pod. Is a local reproduction useful?

- A) Yes — copy the injected value into a local environment variable and run it locally
- B) No — the value's origin is the thing in question, and a local copy reproduces your assumption rather than the bug
- C) Yes — mutating webhooks run identically against local processes
- D) No — mutating webhooks apply non-deterministically, so the injected value differs on each admission and cannot be pinned down

5. Which of these is genuinely reproducible on a laptop, without any cluster or proxy?

- A) A ServiceAccount token's permissions against the API server
- B) A parsing error in the application's handling of a malformed request body
- C) A Service selector that fails to match its Pods
- D) Peer discovery through headless-Service DNS names

Answers with Explanations:

1 — B. The PVC follows the ordinal identity and survives Pod deletion by default [source: k8s-docs-statefulset-storage-2026-08-25]. Recreating the Pod reattaches the same volume with the same contents, so a state-caused failure reproduces exactly, every time.

- **A is wrong**, and the reasoning matters more than the answer. A node fault would not follow a specific ordinal across repeated rescheduling; the replacement Pod may well land somewhere else. Deterministic failure confined to one ordinal points at that ordinal's *state*, not at hardware.
- **C is wrong** — a broken image would fail all three replicas identically, since they share a Pod template. The confinement to one ordinal rules it out.
- **D is wrong** — an admission rejection would prevent the Pod from being created at all, producing a different signature entirely, and admission does not discriminate by ordinal.

2 — B. Retain for both, which is why deleting a replica does not clear its storage [source: k8s-docs-statefulset-storage-2026-08-25].

- **A is wrong** and would be actively dangerous as a default: it would mean a scale-down silently destroyed data.
- **C is wrong**, but it is a real configuration many teams choose deliberately. It is not the default, and assuming it is will cost you the diagnosis in question 1.
- **D is wrong** — the field is optional with documented defaults.

3 — B. Peer discovery in a StatefulSet runs on per-Pod DNS names provided by the headless Service, and you are responsible for creating that Service yourself [source: k8s-docs-statefulset-2026-08-24]. If it is missing or misnamed, the Pods run fine and simply cannot find each other.

- **A is wrong here, though it is a real failure and worth knowing.** Negative caching means a peer that queried a sibling's name before that Pod existed keeps getting the cached failure *"for at least a few seconds"* [source: k8s-docs-statefulset-2026-08-24]. Seconds — and the stem says two hours. A cluster that never forms is a structural problem, not a timing one.
- **C is wrong** — this is peer-to-peer traffic between replicas by individual name, not client traffic through a ClusterIP.
- **D is wrong** — a storage fault would show as a replica failing, not as healthy replicas that cannot see each other.

4 — B. The bug is about what the webhook injected. Copying a value you believe it injected tests your belief, not the system, and if your belief is the wrong part, the local run passes and tells you nothing.

- **A is wrong** for exactly that reason, and it is the appealing wrong answer because it *feels* like rigor.
- **C is wrong** — admission controllers run in the API server request path. A local process never passes through them [*cross-bearing: see Ch 8 §2 — three gates and a logbook*].
- **D is wrong**, and the reason is the same one that sinks C from the other direction. Webhooks are not random; a given webhook applied to a given submission produces a given result. The reason a local reproduction cannot reach this bug is not that the mutation is unpredictable. It is that the local process never gets mutated at all.

5 — B. Parsing logic is your code operating on your input, with no cluster-supplied dependency anywhere in the path. Reproduce it locally, fix it in seconds.

- **A, C, and D are all wrong** for the same reason, which is the point of the question: each depends on something only a cluster supplies — API-server authorization for a projected token, Service selection against real Pod labels, and cluster DNS respectively. All three are on §7's list.
-

How'd You Do?

If you scored 0–2: Re-read §6, then **Chapter 11 §6** for the retention policy. Question 1's diagnosis is unreachable without it, and question 2 is that chapter's fact stated plainly. If the misses were 4 and 5 rather than 1 through 3, the section to re-read is **§7**, not §6.

If you scored 3–4: Check whether your misses cluster in §6 or §7 before deciding what to review; the two halves of this checkpoint fail for different reasons.

Checkpoint: You've Now Mastered

✓ Asking "which replica" before asking anything else about a StatefulSet ✓ The surviving-PVC signature, and why it impersonates a platform fault ✓ Headless-Service peer DNS as a failure surface that ClusterIP workloads never have ✓ Which failures a local reproduction can and cannot reach, and the proxy pattern for the rest

⚓ **Safe Harbor reached** — the practical material of this chapter is behind you. One section remains, and it is about what the last two chapters were actually for.

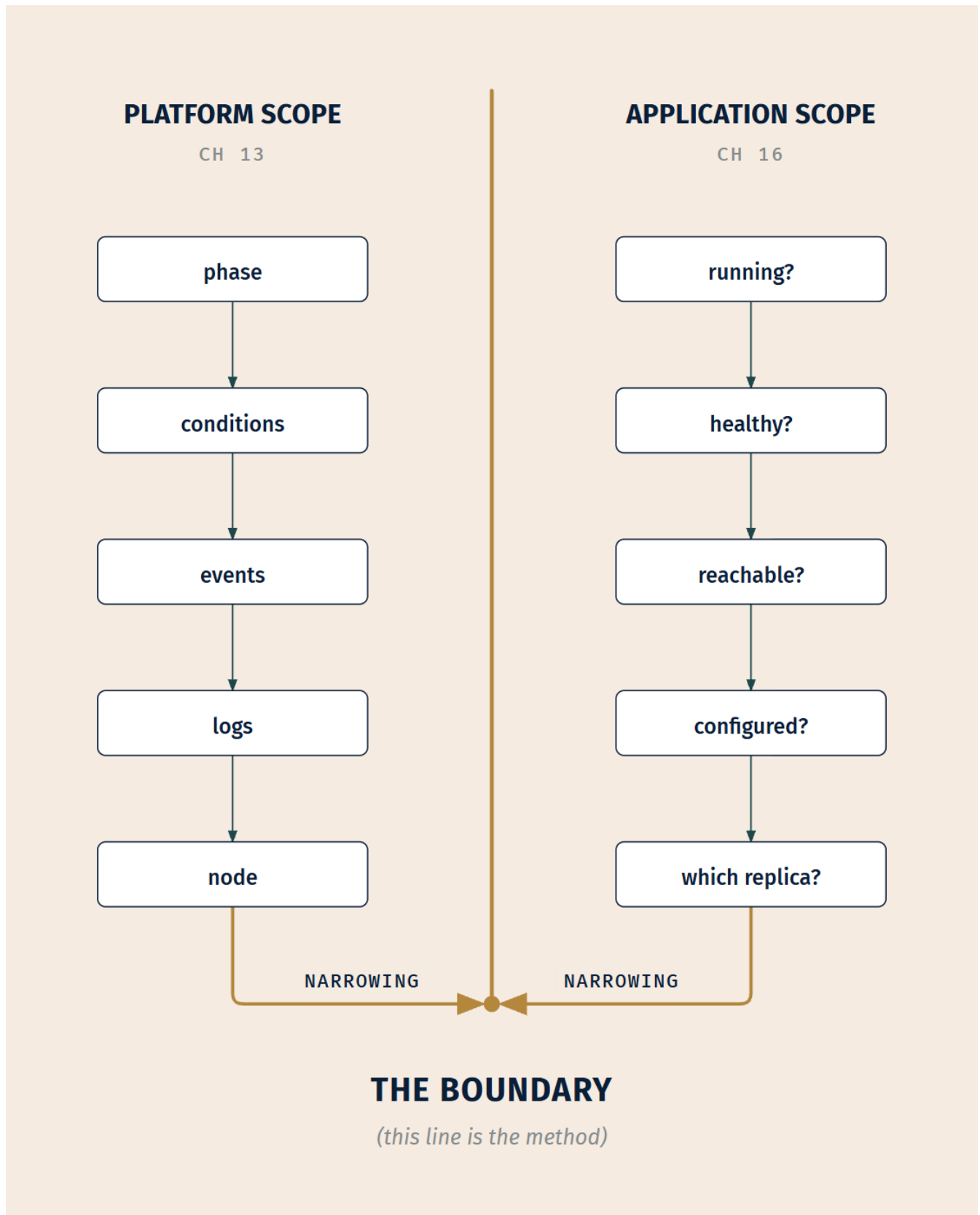
☀ §8 — Mine, or the Platform's

Here is the thing that has been true since Chapter 13 opened and has not been said outright until now.

These were never two chapters about two subjects. **The boundary is the method.**

Chapter 13 taught you to read the phase first, and the phase's last and most valuable answer, the one it works toward, is "*this is no longer mine.*" Every signature in that chapter was a way of narrowing until the platform's contribution was fully accounted for. Chapter 16 taught you four questions, and their real function is not to find the bug either. It is to keep narrowing until the bug has nowhere left to be.

Same move. Different altitude.



Look at what each half of the arc did with its tools. Chapter 13's tools — `describe`, `events`, `logs` -- previous, the node conditions — all answer questions about whether the cluster kept its promises. This chapter's tools — `exec`, `debug`, `port-forward`, `get endpointslices` — all answer questions about whether *you* kept yours. Neither set can answer the other's questions, and the reason the tools were split across two chapters is that the questions are split across one line.

The practitioner move is not knowing more commands. It is placing a failure on the correct side of that line quickly, and then staying on that side until the evidence moves you. Ninety seconds of scope triage saves an afternoon of confidently investigating the wrong half, and an engineer who does that reliably is doing the thing that separates someone who works with Kubernetes from someone who has read about it.

This is a shape you have met before in this book: narrowing by elimination, applied until only one candidate is left. A fix taken on two landmarks is worth more than a confident guess made from one. What is different here is that the narrowing crosses an ownership boundary, which means the last step is not "I found it" but "I know whose it is." On somebody else's cluster, those are equally valuable outcomes, and knowing which one you have reached, and being able to show your work for it, is what makes you good to work with.

The chapter title is "Your Application, Their Cluster." Both halves are true at once, permanently, and the whole skill is knowing which half you are standing on.

Exam Alert! 🚨

High-Priority Topics:

1. **Which verb answers which question.** This is the shape this book organizes the competency around. `logs` for what it said. `exec` for what it can see. `debug` for when there is nothing to `exec` into. `port-forward` for where the break is. `describe service` and `get endpointslices` for whether anything is even selected. You are being tested on the mapping, not on flag syntax.
2. **Ephemeral containers exist because a Pod's container list is immutable.** That single fact explains the entire mechanism — why it needs a dedicated API handler, why `kubect1 edit` cannot do it, why the container cannot be removed once added [source: k8s-docs-ephemeral-containers-concept-2026-08-31].
3. **`kubect1 debug` has three shapes.** Inject into the running Pod; `--copy-to a copy`; `node/` for the host. Each answers a different question and they are not interchangeable.
4. **No ready endpoints has two causes** — selector mismatch, or Pods not Ready — and they live in two different files [source: k8s-docs-debug-pods-2026-08-23]. The slice distinguishes them: a mismatch leaves it empty, while a readiness failure leaves the endpoints in place and marked not ready [source: k8s-docs-endpointslices-2026-08-24].

Common Traps:

Trap	The correct understanding
Assuming <code>kubectl exec ... - sh</code> works on any container	It requires a shell <i>in the image</i> . A distroless image has none, which is the whole reason ephemeral containers exist [source: k8s-docs-ephemeral-containers-concept-2026-08-31].
Expecting <code>kubectl debug</code> to repair the broken Pod	It adds a process to look with, or makes a copy to experiment on. It fixes nothing, and an ephemeral container cannot be removed once added [source: k8s-docs-ephemeral-containers-concept-2026-08-31].
Reading <code>--copy-to</code> as "debug the running Pod"	It is the opposite. The original is untouched, which is precisely the point on a workload you did not deploy.
Treating a working port-forward as proof the application is fine	It proves the application is fine and the Service path is not. Narrowing step, not clean bill of health.
Confusing port with <code>targetPort</code>	<code>targetPort</code> is the port the container listens on. A mismatch produces a Service that selects perfectly and routes into nothing.
Reading a Service with no ready endpoints as a broken Service	The Service is correct and finding nothing it will send traffic to. Two different bugs, two different files, and the slice tells you which: empty means selector, not-ready means readiness [source: k8s-docs-endpointslices-2026-08-24].
Believing a not-ready Pod is removed from its <code>EndpointSlice</code>	It is included and marked not ready. Matching Pods are in the slice regardless of readiness; readiness disqualifies an endpoint rather than deleting it [source: k8s-docs-endpointslices-2026-08-24].
Running plain <code>kubectl logs</code> on a Pod stuck in init	You get nothing useful. Name the init container with <code>-c</code> [source: k8s-docs-debug-init-containers-2026-08-31].
Assuming a rescheduled <code>StatefulSet</code> replica comes back with empty storage	The PVC follows the identity and is retained by default [source: k8s-docs-statefulset-storage-2026-08-25]. A corrupt write survives every restart you try, which is exactly why it impersonates a platform fault.
Assuming <code>kubectl debug</code> always succeeds if you have RBAC for it	A debug container is a container. Under restricted enforcement, admission can refuse it [cross-bearing: see Ch 12 §6 — three levels, three modes].
Treating D3 Debugging as a different subject from D2 Troubleshooting	The split is <i>scope</i> , not tooling. A question tagged to one may test the other's commands, because the commands do not respect the domain boundary — only the questions do.

Practice Questions

Q1. An application's Pod is Running, 1/1 Ready, has zero restarts, and sits on a node reporting no adverse conditions. The application returns stale data. Three other Deployments in the namespace are unaffected. Which is the correct first move?

A) File a platform ticket; every application-side signal is clean B) Treat it as application scope and begin the four triage questions C) Delete the Pod to force a reschedule onto a different node D) Check node conditions across the whole cluster before deciding

Q2. You are on a cluster where you cannot list nodes or read events in `kube-system`. A single workload of yours is failing while everything else in your namespace runs normally. What does §1's addition to the scope test say you should do?

A) Escalate immediately, because the scope test cannot be completed without node access B) Make the application-scope case from the evidence you can gather, and be ready to show it if you later need the platform team C) Assume platform scope by default, since the unverifiable half is the platform's D) Request node access before starting any diagnosis

Q3. A Pod reports STATUS: `Init:2/4`. What is true?

A) Two init containers have completed and the third is running or failing B) Two of four app containers have started C) Two init containers failed and two remain D) The Pod is Running with two containers ready

Q4. An init container runs `git clone` into a mounted volume. The workload deploys successfully. After the Pod is evicted and rescheduled, it will not start. What is the most likely cause?

A) The volume failed to reattach on the new node B) The init container is not idempotent — the clone target already exists C) The git repository became unreachable D) Init containers are skipped on rescheduled Pods, so setup never ran

Q5. A Pod sits at `Init:0/1` for twenty minutes. No errors, no restarts, and the init container's log reads `waiting for service endpoint....` The Service it is waiting on selects only this workload's Pods. What is happening?

A) A DNS failure is preventing the lookup B) An ordering deadlock — the init container waits for an endpoint that only this Pod could provide C) The init container image is being pulled slowly D) The readiness probe on the init container is failing

Q6. You need to inspect the filesystem of a running container built from a distroless image. Which is correct?

A) `kubectl exec -it <pod> -- /bin/sh` B) `kubectl debug -it <pod> --image=busybox --target=<container>` C) `kubectl logs <pod> --previous` D) `kubectl cp <pod>:/ ./local-copy`

Q7. Which statement about ephemeral containers is correct?

A) They may define a `readinessProbe` to report when the debug tooling is ready B) They may set resource requests so the debug session is guaranteed CPU C) They cannot be removed or changed once added to a Pod D) They are automatically restarted if the debug process exits

Q8. An application container crashes immediately on startup, before you can exec into it. Which `kubectl debug` shape is designed for this case?

A) An ephemeral container injected into the running Pod B) `--copy-to`, creating a copy of the Pod with the entrypoint changed C) `debug node/<node>` to inspect the host D) None — a crashing container cannot be debugged with `kubectl debug`

Q9. You run `kubectl debug` in a namespace enforcing the restricted Pod Security Standard, using a debug image that runs as root. The command is refused. You have verified you hold RBAC permission for the operation. What happened?

A) RBAC permissions do not cover the ephemeral containers subresource separately B) The debug container was rejected by admission, because it is a container and must meet the namespace's enforced standard C) `kubectl debug` is disabled in namespaces with Pod Security Admission enabled D) The node lacked capacity for an additional container

Q10. `kubectl get endpointslices -l kubernetes.io/service-name=web` returns a slice with zero endpoints. `kubectl get pods -l <the Service's selector>` returns no Pods at all. What is the cause?

A) The Pods are running but not Ready B) The Service's selector does not match any Pod's labels C) `targetPort` does not match the container's listening port D) The EndpointSlice controller is not running

Q11. A Service's EndpointSlice holds three endpoints, all of them ready. Requests to the Service time out. Which break point is most likely?

A) A selector/label mismatch B) The client is resolving a DNS name that does not point at this Service C) A port/`targetPort` mismatch delivering traffic to a closed port D) The EndpointSlice controller has stopped reconciling

Q12. [retrieval: ch9] A Service cache in namespace `data` has three ready endpoints. A client Pod in namespace `web` calls `http://cache:6379` and gets nothing. What is the most likely cause?

A) The Service's Pods are not Ready B) The short name resolves relative to the client's namespace, so cache resolves in `web`, not `data` C) port and `targetPort` are mismatched D) Redis requires a headless Service

Q13. [retrieval: ch9] A Service `web` declares `port: 80` and `targetPort: 80`. Requests through the Service fail. You run `kubectl port-forward pod/web-7f4d 8080:8080` and the application answers correctly on your local port 8080. What have you established?

A) The application is healthy and the Service is correctly configured B) The container is listening on 8080, so the Service's `targetPort` of 80 delivers traffic to a closed port C) The Pod is not Ready, so its endpoint was never a valid target D) Nothing — `port-forward` and the Service path deliver traffic identically

Q14. [retrieval: ch11] A three-replica StatefulSet has one replica, `queue-0`, that fails on start. You delete it; the recreated `queue-0` fails identically. `queue-1` and `queue-2` are healthy. What should you

investigate first?

A) The Pod template, since all replicas share it B) The contents of `queue-0`'s `PersistentVolumeClaim`, which survived the deletion C) The node `queue-0` was scheduled onto D) The `StatefulSet`'s image tag

Q15. A bug reproduces only in the cluster. The failing code path reads a config value mounted from a `ConfigMap`, and you suspect the mounted value differs from what the manifest declares. Which approach actually tests the hypothesis?

A) Copy the `ConfigMap`'s declared value into a local environment variable and run the app locally B) Exec into the running container and read the mounted file directly C) Re-apply the `ConfigMap` and restart the `Deployment` D) Reproduce in a local `kind` cluster using the same manifests

Q16. Three workloads owned by three different teams are all failing, and every one of them is on the same node; Pods rescheduled off that node recover immediately. You hold cluster-admin. Which move fits the evidence?

A) `kubectl debug <pod> --image=busybox --target=<container>` against one of the failing Pods B) `kubectl debug <pod> --copy-to=<pod>-debug --image=ubuntu` C) `kubectl debug node/<node> -it --image=ubuntu` D) `kubectl port-forward` to one of the failing Pods

Q17. Two workloads have the same class of bug: a wrong value for a key their configuration depends on. Workload A's Pod is stuck at `Init:0/1`, its init container exiting non-zero and printing the offending key's name. Workload B's Pod is `Running, 1/1 Ready`, and returns wrong answers. Which is the harder diagnosis, and why?

A) A — a Pod that will not start gives you nothing to inspect B) B — the value was present and read cleanly, so nothing failed and no signal names the problem C) Neither; both produce the same signature and the same diagnosis D) A — an init container's logs are not retrievable until the Pod reaches `Running`

Answers with Explanations:

Q1 — B. Every element of the mechanical test resolves to application scope: running, ready, stable, confined to one workload, no adverse platform signal.

- **A is wrong** because "application-side signals are clean" is a misreading. Those signals are *platform* signals reporting on your workload, and their cleanliness is what hands the problem to you.
- **C is wrong** — deleting a Pod to see what happens is the reflex the four questions exist to replace. Stale data is not a placement problem, and you would have destroyed the state you needed to inspect.
- **D is wrong** because a fault confined to one workload is already answered by the scope test; a cluster-wide node sweep is effort spent on the half you have already eliminated.

Q2 — B. The addition is that the boundary is also a statement about what you can see, and the practical response is to build the application-scope case from available evidence.

- **A is wrong** — the test completes fine here. Confinement to one workload plus clean workload-level signals is sufficient.
- **C is wrong** and is the failure mode this guidance exists to prevent: defaulting to "must be the platform" whenever you cannot check something. That reasoning would make every fault platform-scope on a restricted cluster.
- **D is wrong** — waiting on an access request before diagnosing is how a fifteen-minute problem becomes a two-day one.

Q3 — A. `Init:N/M` counts *completed init containers* out of the total [source: k8s-docs-debug-init-containers-2026-08-31]. `Init:2/4` means two are done and the third is where your attention belongs.

- **B is wrong** — the counter refers to init containers only. App containers have not started; the Pod is still Pending.
- **C is wrong** — the status does not count failures, and a failing init container reports `Init:Error` or `Init:CrashLoopBackOff` instead.
- **D is wrong** — *"a Pod that is initializing is in the Pending state but should have a condition Initialized set to false"* [source: k8s-docs-init-containers-2026-08-24] [cross-bearing: see Ch 5 §5 — Pod phase and container state].

Q4 — B. A clone into a directory that already contains a clone fails. Every init container runs again on every Pod start [source: k8s-docs-init-containers-2026-08-24], and the volume persisted across the reschedule, so the second run hits a populated target.

- **A is possible but is not the most likely given the stated sequence**, and the discriminator is available: a volume reattachment failure produces a Pod stuck on mounting with events saying so, not a Pod whose init container ran and exited non-zero.
- **C is possible in principle** but would have been equally likely on the first deploy. Nothing in the timeline points at the repository.
- **D is a factual error** — init containers run on every start, which is the entire reason B is the answer.

Q5 — B. The circular wait. The init container blocks until the Service has a ready endpoint; the Service cannot have one until this Pod's app container is ready; the app container cannot start until the init container exits. Perfectly stable, no error, waits indefinitely.

- **A is wrong** — a DNS failure produces resolution errors in the log, not a calm "waiting" message.
- **C is wrong** — an image pull in progress reports `Init:ImagePullBackOff` or a pulling event, and the init container's log would not exist yet [cross-bearing: see Ch 13 §2 — pods that never start].
- **D is wrong** on its facts: regular init containers *"do not support the Lifecycle, LivenessProbe, readinessProbe, or startupProbe fields"* [source: k8s-docs-init-containers-2026-08-24].

Q6 — B. A distroless image has no shell to exec into, so the move is to inject an ephemeral container that does have tools, targeting the container you want to inspect [source: k8s-docs-ephemeral-containers-2026-08-24].

concept-2026-08-31].

- **A is wrong** — that is the command that fails, and the failure is why this whole section exists.
- **C is wrong** — `logs --previous` returns the previous container instance's output. Useful for a crash loop, useless for inspecting a filesystem.
- **D is wrong** for a reason worth carrying: `kubectl cp` copies files *out* of the container rather than giving you a live view of the running process, and it also depends on tooling inside the image that a distroless image does not have.

Q7 — C. *"Like regular containers, you may not change or remove an ephemeral container after you have added it to a Pod."* [source: k8s-docs-ephemeral-containers-concept-2026-08-31]

- **A is wrong** — probes are explicitly disallowed, because ephemeral containers may not have ports [source: k8s-docs-ephemeral-containers-concept-2026-08-31].
- **B is wrong** — *"Pod resource allocations are immutable, so setting resources is disallowed."* [source: k8s-docs-ephemeral-containers-concept-2026-08-31]
- **D is wrong** — they *"will never be automatically restarted"* [source: k8s-docs-ephemeral-containers-concept-2026-08-31], which is one of the reasons they are unsuitable for building applications.

Q8 — B. *"you can't run `kubectl exec` to troubleshoot your container if your container image does not include a shell or if your application crashes on startup. In these situations you can use `kubectl debug` to create a copy of the Pod with configuration values changed to aid debugging."* [source: k8s-docs-kubectl-debug-copy-node-profiles-2026-08-31] Copy it, replace the entrypoint, and now the container sits still.

- **A is wrong** — an ephemeral container needs a Pod to attach to, and while the Pod exists, the *target container* is gone again before you can look at it.
- **C is wrong** — the node is not the problem, and that shape steps across the scope boundary for no reason.
- **D is wrong** — this is exactly the case `--copy-to` was built for.

Q9 — B. An ephemeral container is a container in that namespace, and admission enforces the namespace's standard against it. A root-running debug image has asked for more than restricted permits [cross-bearing: see Ch 12 §6 — *three levels, three modes*].

- **A is wrong** — the stem states RBAC is verified, and the refusal comes from a later gate. Authentication, then authorization, then admission [cross-bearing: see Ch 8 §2 — *three gates and a logbook*].
- **C is wrong** — a debug container that *meets* the standard is admitted normally. That is what the profile mechanism is for: a `--profile` is a preset for how much privilege the debug container asks for, and asking for less is what gets you through the gate.
- **D is wrong** — a capacity problem produces a scheduling failure with a distinct signature, not an admission refusal.

Q10 — B. The label query returned nothing, which means no Pod carries the labels the Service selects. Nothing matched, so nothing was placed in the slice.

- **A is wrong**, and this is the discriminator the section is built on. Not-ready Pods would still *appear* in a label query and would still be *in the slice*, marked not ready [source: k8s-docs-endpointslices-2026-08-24]. Zero endpoints alongside zero matching Pods is the selector's signature, not readiness!
- **C is wrong** — a port mismatch does not affect slice membership at all. It is downstream of readiness.
- **D is wrong** and is the reflexive platform-blame answer. The empty slice is the controller reporting correctly that nothing matched.

Q11 — C. Three ready endpoints means selection and readiness are both fine. The remaining candidate on that path is the port pairing: traffic arriving at a port nothing is bound to.

- **A is wrong** — a selector mismatch leaves the slice with no endpoints, and the stem says there are three.
- **B is wrong**, and it is the option that survives the stem's facts longest, which is why it is here. But the stem places you at the Service you are querying and reading endpoints from; a name resolving somewhere else means you never reached *this* Service, and the endpoints you are looking at would be irrelevant rather than informative. Break 4 is a failure to arrive, not a failure at the destination.
- **D is wrong** — a stalled controller would leave a stale or empty slice, not a correct one.

Q12 — B. A short name resolves through the client's search domains, which include the client's own namespace [source: k8s-docs-dns-pod-service-2026-08-23]. From web, cache means `cache.web.svc.cluster.local`, which is not the Service in question [*cross-bearing: see Ch 9 §7 — names, and where they resolve*].

- **A is wrong** — the stem states three ready endpoints, ruling readiness out.
- **C is wrong** — a port mismatch produces a connection failure against a real target, a different signature from a name that resolves elsewhere or not at all.
- **D is wrong** — a headless Service is for per-Pod DNS identity; a Redis client reaching a single ClusterIP Service needs no such thing.

Q13 — B. The port numbers are the whole story. The container answers on 8080, so 8080 is what it binds; the Service declares `targetPort: 80`, so every request through the Service is delivered to a closed port. Read the Service's declared `targetPort` with `kubectl describe service web` and compare it against the port you just forwarded to successfully. When those two numbers disagree and the forward works, you are looking at break 3 [*cross-bearing: see Ch 9 §3 — four ways to be reachable*].

- **A is wrong**, and it is the §5 trap in its most seductive form: a successful port-forward feels like an all-clear. Users travel the Service path, which you have just shown is broken.
- **C is wrong** — readiness would keep the Pod out of the Service path but has no effect on a port-forward, so it cannot explain a working forward. It also does not explain the port asymmetry, which is the actual evidence in the stem.

- **D is wrong** — port-forward is an API-server operation on the pods/portforward subresource [source: k8s-docs-port-forward-authorization-2026-08-31] and does not travel the Service path at all. If the two were equivalent, both would have failed.

Q14 — B. The PVC survives Pod deletion by default and reattaches to the recreated replica [source: k8s-docs-statefulset-storage-2026-08-25]. Deterministic failure confined to one ordinal, immune to restart, is the surviving-state signature.

- **A is wrong** — a Pod template problem would fail all three replicas, and two are healthy.
- **C is wrong** — the replacement Pod is not guaranteed to land on the same node, so a node fault would not track the ordinal so faithfully.
- **D is wrong** — the image is shared by all three replicas.

Q15 — B. The hypothesis is that the mounted value differs from the declared one. The only way to test it is to read what is actually mounted, in the running container.

- **A is wrong** and is the most instructive distractor: copying the *declared* value locally tests your assumption rather than the system. If the declared and mounted values differ, this reproduces the wrong one and passes.
- **C is wrong** — restarting may make the symptom vanish without ever telling you what was wrong, which means the next occurrence starts from zero.
- **D is wrong** for the same reason as A, at larger scale: a local cluster with the same manifests reproduces what the manifests say, not what the target cluster's mount actually produced.

Q16 — C. The scope test has already spoken: the fault is not confined to one workload, and it tracks a machine rather than an application [*cross-bearing: see Ch 13 §1 — whose problem is this, and what to read first*]. `debug node/` is the shape built for the host, and cluster-admin is what makes it available to you.

- **A is wrong** — an ephemeral container inspects one container's view of one Pod. Three teams' workloads failing together is the signature that sends you across the boundary, not deeper into one of them.
- **B is wrong** for the same reason, with an added defect: the copy may be scheduled somewhere else entirely, taking it away from the only thing the evidence implicates.
- **D is wrong** — port-forward is an application-scope elimination step. It can tell you whether a Service path is broken; it says nothing about a host.

Worth noticing about this question: the correct answer is the one this chapter spends a section arguing is out of scope. That is not a contradiction. `debug node/` is correct here precisely because the evidence has already moved you across the line, and on a cluster you did not own, the equally correct move would be handing the same evidence to whoever does [cross-bearing: see Ch 13 §5 — when the node is the problem].

Q17 — B. Same class of bug, two entirely different signatures, which is exactly why this chapter answers "is it configured" in two separate places. Workload A failed loudly and named the problem. Workload B

read a value that was present and well-formed and simply wrong, so nothing errored, no probe fired, and no status string is going to tell you anything.

- **A is wrong**, and it inverts the difficulty. A failing init container is the *easy* case: it exits non-zero, prints the reason, and `kubectl logs <pod> -c <init-name>` retrieves it [source: k8s-docs-debug-init-containers-2026-08-31].
 - **C is wrong** — if both produced the same signature, §2 and §3 would be one section. The whole doubling in §1's table exists because they do not.
 - **D is wrong** on its facts: an init container's logs are readable with `-c` while the Pod is still Pending, which is the entire point of that flag in this context [source: k8s-docs-debug-init-containers-2026-08-31].
-

Chapter Summary

Concept	Remember This
The scope boundary	Running + ready + confined to one workload = yours. On someone else's cluster, it is also a statement about what you can see.
The four questions	Running (§2), healthy/configured (§3), reachable (§4→§5), and for StatefulSets, <i>which replica</i> (§6).
Init container logs	<code>kubectl logs <pod> -c <init-name></code> . The plain form returns nothing useful and the nothing is misleading.
The three init failures	Ordering deadlock (waits indefinitely, no error) · non-idempotency (fails only on restart) · config error (exits loudly, tells you why).
A failing init container is retried	The kubelet restarts it until it succeeds — unless <code>restartPolicy: Never</code> , in which case the Pod is treated as failed. That is why <code>Init:CrashLoopBackOff</code> exists.
Termination messages	A container can write a one-line fatal reason to <code>/dev/termination-log</code> ; Kubernetes surfaces it in Pod status, so the failure stays legible from <code>describe</code> alone.
<code>exec</code>	Answers "what did the <i>process</i> actually read," which is where the manifest and reality diverge.
The field that never arrived	A misnested or misspelled key is silently dropped at apply. <code>exec</code> cannot find what the server never stored — read the object back with <code>-o yaml</code> and compare it against your file.
The distroless problem	No shell in the image means no shell in the container. Hardening win, debugging cost.
Ephemeral containers	Exist because you cannot add a container to a running Pod. No resources, no probes, no restart, no removal.
<code>kubectl debug</code> 's three shapes	Inject (what does it see now?) · <code>--copy-to</code> (what if I changed something?) · <code>node/</code> (is the machine the problem? — platform scope).
<code>--copy-to</code>	Makes a copy. The original is untouched, and that is the feature.
Debug profiles	A preset for how much privilege the debug container asks for. Asking for more than the namespace enforces is what gets it refused at admission.
No ready endpoints	Two causes, two files — and the slice separates them: empty means the selector matched nothing; present but not ready means readiness.
Ready endpoints, failed request	Not a selection problem. Look at <code>port</code> vs <code>targetPort</code> — <code>targetPort</code> is the one the container listens on.
<code>port-forward</code>	Bypasses the Service path via the API server. A working forward beside a failing Service <i>localizes</i> the fault; it does not clear the app.
StatefulSet debugging	Ask "which ordinal" first. The PVC survives deletion by default, so a bad write is immune to restarts and impersonates a platform fault.
Headless-Service peer DNS	You create the headless Service, not the controller. Missing one means healthy Pods that cannot find each other.

Concept	Remember This
Local reproduction	Anything cluster-supplied — identity, DNS, injected config, admission mutation, Service routing — is not reproducible locally. Everything else usually is.

The Voyage Ahead

Part IV closes here, and with it the two-chapter arc that began when the platform handed you back your own problem.

You now have both halves of a single skill: reading a cluster's report on your workload, and reading your workload once the cluster has nothing left to say. That skill is worth more than any individual command in either chapter, because commands change between releases and the boundary does not.

What comes next is a change of altitude. Part V steps back from the failing workload in front of you to the ecosystem the workload lives in: what "cloud native" actually names, who decides what belongs under that word, and the four places Kubernetes deliberately lets somebody else's software in. Chapter 17 finally answers the question Chapter 1 planted and left standing on purpose, and it collects a set of interfaces you have been meeting one at a time since Chapter 2 without ever seeing them side by side.

After that, the instruments. You have spent this chapter finding out what went wrong *after* someone told you something was wrong. Chapter 18 is about the systems that tell you first.

"The boundary is not a wall between two crews. It is the line on the chart that tells each of them where to look."

Chapter 17: The Fleet and Its Charts

"Meshes, functions, autoscalers, and the foundation that keeps the map"

Domain Weight: 12% | Complexity: Mixed | Novelty: Moderate

Attention Budget

Total time: ~155 minutes | Recommended: Split across 2 or 3 sessions

Section	Time	Attention Cost	Best Time to Study
📍 Soundings	10 min	Medium	Before you start reading properly
Why This Chapter Matters + What You'll Learn	6 min	Low	Anytime
§1 What "Cloud Native" Actually Names	8 min	Low	Anytime
§2 Sandbox, Incubating, Graduated, and Who Decides	12 min	Medium	Mid-session
§3 Small Pieces, Replaced Whole	6 min	Low	Anytime
🕒 Taking Your Bearings (1)	8 min	Medium	After a short break
§4 Every Place Kubernetes Lets You In	12 min	High	Peak attention
§5 A Network That Knows What It's Carrying	14 min	High	Peak attention
🕒 Taking Your Bearings (2)	7 min	Medium	After a short break
§6 Code Without a Server to Put It On	8 min	Medium	Mid-session
§7 Four Things That Scale	10 min	Medium	Mid-session
§8 How the Project Actually Runs, and How You'd Join	12 min	Low	Anytime — but do not skip
🕒 Taking Your Bearings (3)	7 min	Medium	After a short break
§9 One Pluggability Story	4 min	Low	Read it awake
Exam Alert, Practice Questions, Chapter Summary	30 min	High	A separate sitting

Attention Cost Key:

- **Low:** Concrete, familiar concepts — study anytime
- **Medium:** New concepts requiring focus — study when alert
- **High:** Abstract or complex material — study at peak attention

If you only have 15 minutes: read §4, then §9. Together they are the chapter's spine.

The natural split point is after Taking Your Bearings 2. §1 through §5 are the ecosystem's vocabulary; §6 through §9 are its shape. If you want a third session, take the Practice Questions on their own. Twenty-one items after two hours of reading is not a fair test of anything.

"We democratize state-of-the-art patterns to make these innovations accessible for everyone." — CNCF Cloud Native Definition v1.1 [source: cncf-cloud-native-definition-2026-08-23]

🔊 Soundings

Before reading this chapter, try these questions. Your score determines how to approach the content — no shame in any score, just different reading strategies.

1. For each of CRI, CNI, and CSI: which layer of the cluster does it serve?
2. Chapter 1 said that the common reading of *cloud native* — "it runs in a public cloud" — is not what the term means. In one sentence, and without looking it up: what do you think the published definition is actually about?
3. How many minor releases does the Kubernetes project support at any one time?
4. What makes a container a *sidecar*, and what does it share with the container beside it?
5. A HorizontalPodAutoscaler is created on a cluster where metrics-server was never installed. What happens?
6. A Pod that no node can satisfy: what phase is it in, and what does its continued existence state about the cluster?
7. Kubernetes organizes its contributors into named groups. What would you guess the primary, durable, topic-focused unit is called — and what would you call a group formed to work across several of them?
8. TLS terminates at the Ingress. What protects the leg from there to the Pod?

Click for answers + reading strategy

1.

Acronym	Layer
CRI	The container runtime on the node
CNI	Pod networking
CSI	Storage

[cross-bearing: see Ch 2 §4 — the container runtime interface], [cross-bearing: see Ch 9 §1 — four rules and a plugin], [cross-bearing: see Ch 11 §5 — who actually provides the storage].

2. Open-ended, and almost everyone gets this partly wrong, which is the point. Hold whatever sentence you wrote. §1 opens with the published definition verbatim, and you will be able to measure your answer against it word for word.

3. Three. The project maintains release branches for the most recent three minor releases [source: k8s-release-cycle-and-cadence-2026-08-31]. If you had this cold, Chapter 8 §6 stuck. If you did not, §8 repairs it, and pairs it with the fact that explains it.

4. A sidecar is a second container in the same Pod, running alongside the main application container. It shares the Pod's network namespace, meaning it reaches the main container on `localhost`, and it can share the Pod's volumes. *[cross-bearing: see Ch 5 §2 — more than one container aboard].*

5. The HorizontalPodAutoscaler object is created successfully. Nothing scales. The HPA needs the Metrics API to read utilization from, and on a cluster with no metrics-server, or any equivalent implementation, nothing is serving it. The object exists; the component that would act on it does not. *[cross-bearing: see Ch 13 §7 — numbers nobody collects by default].*

6. Pending. And its continued existence is a standing, machine-readable statement that the cluster is short of somewhere to put work. *[cross-bearing: see Ch 7 §2 — what makes a node feasible].*

7. The primary durable unit is a **SIG**, a Special Interest Group. A group formed to work across several of them is a **Working Group**. If you got the first and guessed at the second, you scored exactly what most readers score here, and §8 is where the guess becomes knowledge.

8. Nothing, by default. TLS terminates at the Ingress and the traffic continues to the Pod in the clear. NetworkPolicy can say who may talk to whom, but it cannot encrypt anything. *[cross-bearing: see Ch 10 §7 — what NetworkPolicy cannot do].* Hold onto this one; §5 is about the layer that closes that gap.

If you got 6+ right: Skim the technical sections. But read §2 and §8 properly, at normal pace. They are the two sections a strong technical score predicts nothing at all about, and they are where this chapter's points are most often left on the table.

If you got 3–5 right: Read at normal pace. This chapter is calibrated for you.

If you got 0–2 right: Read carefully, and before you start, go back and re-read **Chapter 2 §8** and **Chapter 11 §5**. Not alongside this chapter; before it. Those two sections are where the four-interface thread was last picked up, and §4 of this chapter is close to unreadable without them.

Why This Chapter Matters

Two chapters have just handed you a boundary and a method: whose problem a failure is, and how to work out what broke. This chapter does not change the subject. It changes altitude.

Everything from Chapter 2 forward has been *inside* one cluster: its components, its objects, its traffic, its storage, its failures. This chapter is about the thing that cluster is an instance of. And it opens by paying a debt that has been outstanding since page one.

Chapter 1 named the phrase *cloud native*. It said the near-universal reading, "it runs in a public cloud," is not what the term means. Then it declined to define it, and pointed here. That is the longest-running open loop in this book, and §1 closes it with the actual published document rather than a paraphrase.

But the chapter's real withheld question is larger and quieter, and it was planted back in Chapter 2. **Why does Kubernetes keep doing the same thing?**

Four times now you have watched Kubernetes define an interface and hand the implementation to somebody else. Runtimes, in Chapter 2. Networking, in Chapter 9. Storage, in Chapter 11. Object types themselves, in Chapter 6. Each time it looked like a local design decision about that one problem: a sensible way to handle runtimes, a sensible way to handle storage. §4 puts all four side by side. §9 says what they are evidence of.

Chapter 2 §8 promised you this would feel like recognition rather than a fourth list. Keeping that promise is this chapter's job, and there is only one way to keep it: you have to arrive already holding all four. If Soundings question 1 was uncomfortable, that is worth ten minutes of back-reading before you continue.

Dead Reckoning: This chapter covers Domain 4 of the KCNA curriculum, Cloud Native Architecture, which is 12% of the exam [source: [cncf-kcna-certification-page-2026-08-23](#)]. Domain 4 has three competencies. Observability belongs to Chapter 18. This chapter carries the other two: Cloud Native Ecosystem and Principles, and Cloud Native Community and Collaboration. It is the only chapter in this book that carries two whole competencies, which is why it is also the longest. CNCF publishes weights for the four domains and does not publish weights for the competencies inside them. The split of Domain 4's points between this chapter and the next is this book's judgment, not a published figure.

There is an identity shift buried in that dry paragraph, and it deserves naming. Up to here you have been a competent user of somebody else's software. This chapter puts you inside the community that produces it: a community with a published definition of its own terms, a graduation ladder its projects climb, a technical committee that decides what belongs, a contributor ladder that anyone reading this could start climbing on a Tuesday afternoon, and a release train that explains why the version numbers behave the way Chapter 8 said they do.

That last connection is the chapter's quietest payoff. Three minor releases a year, and three supported minor versions, are not two facts you have to remember separately. They are one fact stated twice. A reader who sees that will never lose either half again.

Now, the stakes, stated plainly rather than dramatized. Domain 4 is the smallest domain on the exam. It also has the highest ratio of names to mechanics: the most things to recognize, the fewest things to reason through. That combination makes it simultaneously the cheapest available points and the most commonly forfeited ones. Chapter 1 already said this to one class of reader's face: *Chapter 17's community and collaboration material is, in my experience, what technically strong candidates skip most often. It looks like soft content next to the technical chapters.*

§2 and §8 exist because of that sentence. They carry .1 Foundation difficulty markers for the same reason: not because the material is easy, but because the signal you need when scanning the left margin is *everyone needs this, not optional depth.*

What You'll Learn

By the end of this chapter, you'll be able to:

- **Quote** the CNCF's published definition of *cloud native*, and name the five characteristics it attaches to loosely coupled systems
- **Place** any CNCF project on the maturity ladder — and, more usefully, say who decides and where the criteria live
- **State** the shape that CRI, CNI, CSI and CRDs have in common, without being handed the four names first
- **Distinguish** a service mesh's data plane from its control plane, and both of those from the control plane you have been reading about since Chapter 3
- **Say** which axis each autoscaler moves — replicas, resources, or nodes — and which of them the cluster does not ship
- **Name** the difference between a SIG, a Working Group and a Committee, and explain why three releases a year and three supported versions are the same fact

You'll also stop treating the ecosystem as trivia around the edges of the technology, which is the habit that costs well-prepared candidates the most points on the smallest domain.

.1 §1 — What "Cloud Native" Actually Names

Chapter 1 used a phrase and refused to define it. Here is the document it was refusing to paraphrase.

Cloud native practices empower organizations to develop, build, and deploy workloads in computing environments (public, private, hybrid cloud) to meet their organizational needs at scale in a programmatic and repeatable manner. It is characterized by loosely coupled systems that interoperate in a manner that is secure, resilient, manageable, sustainable, and observable.

Cloud native technologies and architectures typically consist of some combination of containers, service meshes, multi-tenancy, microservices, immutable infrastructure, serverless, and declarative APIs — this list is non-exhaustive.

These techniques enable loosely coupled systems that are resilient, manageable, and observable. Combined with robust automation, they allow engineers to make high-impact changes frequently and predictably with minimal toil and clear separation of concerns.

The Cloud Native Computing Foundation seeks to drive adoption of this paradigm by fostering and sustaining an ecosystem of open source, vendor-neutral projects. We democratize state-of-the-art patterns to make these innovations accessible for everyone.

[source: cncf-cloud-native-definition-2026-08-23]

That is CNCF Cloud Native Definition v1.1. It is short enough to read twice, and you should.

Now notice what is in the first sentence, and what your Soundings answer to question 2 probably said instead.

"public, private, hybrid cloud."

Before the first sentence is half over, the definition rules out the most common reading of the term it defines. *Cloud native* does not mean *runs in a public cloud*. A workload running on three servers in a closet in your own building can be cloud native. A workload running on the largest public cloud on earth can fail to be. The phrase is about *how you build and operate*, not *where the machines are*.

That is the whole misconception, retired in one clause, and you have now met it directly rather than being lectured about it.

The clauses, one at a time

The definition does four distinct things, and pulling them apart makes it far easier to hold.

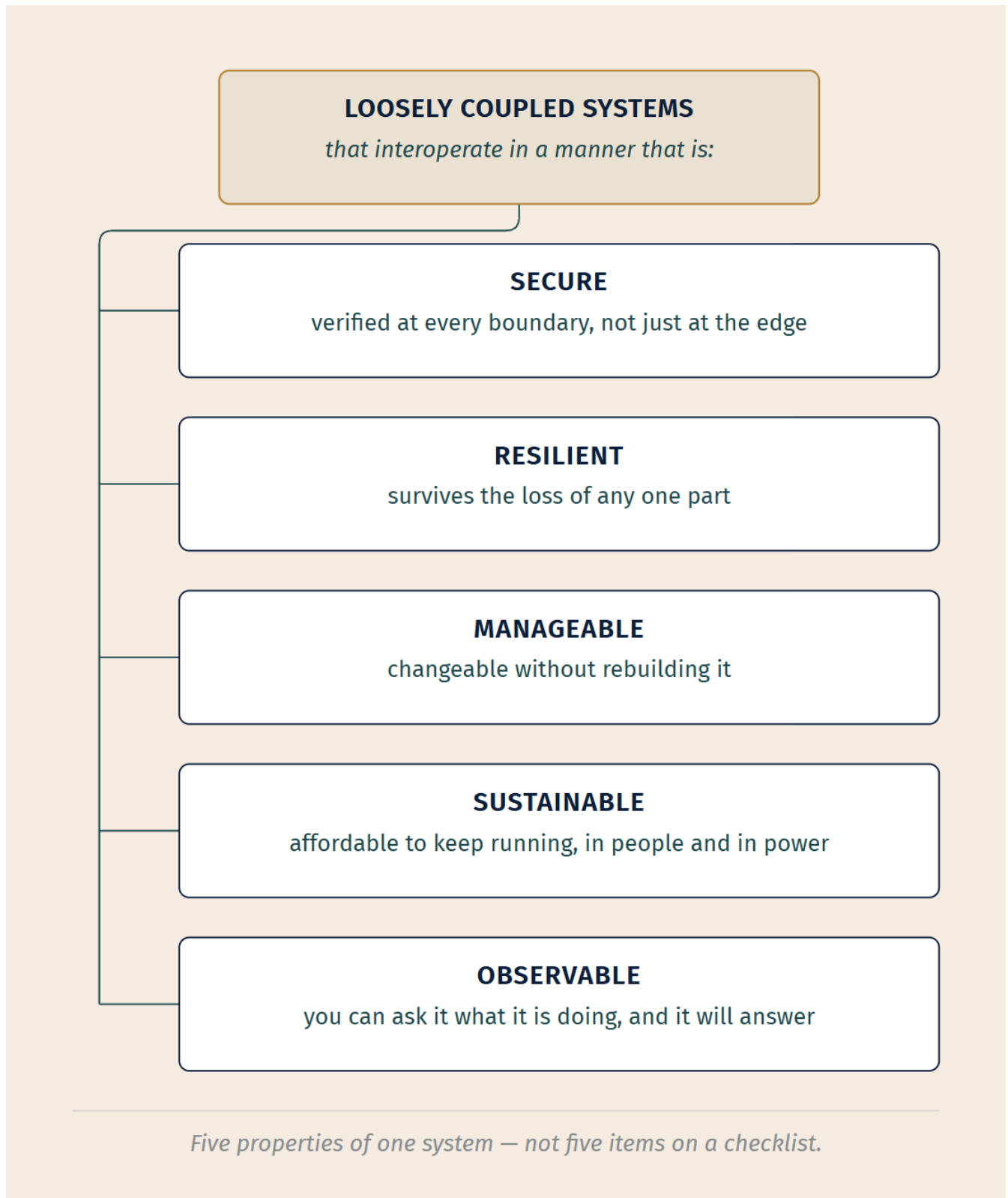
The practices clause. *Develop, build, and deploy workloads in computing environments to meet organizational needs at scale in a programmatic and repeatable manner.* Every load-bearing word here is about method. *Programmatic*: machines do it, not people typing. *Repeatable*: doing it again produces the same result. *At scale*: the method does not fall apart when there are a thousand of the thing instead of three. If you have ever wondered why this book has spent so much of its length on declarative objects and controllers that reconcile, that clause is why. [*cross-bearing*: see Ch 4 §1 — you file a declaration].

The characteristics clause. This is the part the exam can quote at you, so it is the part to know verbatim.

 ☆ **Fixed Point**

Cloud native is characterized by **loosely coupled systems** that interoperate in a manner that is **secure, resilient, manageable, sustainable, and observable**.

Five characteristics, attached to loosely coupled systems. Not five characteristics floating free. The loose coupling is the spine, and the five are what a loosely coupled system has to manage to be worth having. Each one is a real requirement with teeth:



The five characteristics of a cloud native system, hung on the loose coupling that makes them necessary. The list is short enough to memorize and worth memorizing; the annotations are what each one costs you if you skip it.

The technology clause. *Containers, service meshes, multi-tenancy, microservices, immutable infrastructure, serverless, and declarative APIs.* Seven items, and then, in the same sentence, **"this list is non-exhaustive."**

📌 **Snag:** Do not treat those seven as a closed set. The definition says outright that they are not one. It is a natural reader error, since a list of exactly seven things printed in an authoritative document looks like a complete enumeration, and it is a natural exam trap for exactly the same reason. What makes it worth flagging here specifically is that §4 of this chapter *does* teach a set that genuinely is closed. Two lists, three sections apart, and only one of them is finite.

The payoff clause. *High-impact changes made frequently and predictably with minimal toil and clear separation of concerns.* This is what all of it is for. Not elegance, not modernity: the ability to change important things often, without drama, without a person staying up all night to do it. If a practice does not move you toward that, it is cargo cult.

The institution behind the document

You met the CNCF in Chapter 1 as the body that issues this credential and publishes the exam blueprint. That is one true thing about it and not the interesting one.

The Cloud Native Computing Foundation is part of the nonprofit Linux Foundation [source: [cncf-who-we-are-2026-08-23](#)], and its mission is stated in one line: **to make cloud native computing ubiquitous** [source: [cncf-charter-governance-bodies-2026-08-31](#)]. It hosts critical components of global technology infrastructure, Kubernetes and Prometheus and Envoy among them: 227 projects at the time of this book's research, across four categories, with 715 member organizations and over 329,000 project contributors [source: [cncf-who-we-are-2026-08-23](#)].

The operative word in the definition's last paragraph is **vendor-neutral**. The CNCF does not sell you anything. It holds projects that no single company owns, so that a company adopting Kubernetes is not making a bet on one vendor's continued goodwill. That is not a marketing claim; it is a structural one, and §2 is about the structure that makes it true.

[cross-bearing: see Ch 2 §1 — what a container actually is], if you want to see the definition's first technology land on something concrete. And *[cross-bearing: see Ch 17 §3 — small pieces, replaced whole]* is where three more of them get their own treatment.

§2 — Sandbox, Incubating, Graduated, and Who Decides

The CNCF hosts hundreds of projects. Some of them are running production traffic for banks. Some of them are three people and a good idea. The foundation's answer to *how would a stranger tell which is which* is a maturity ladder, and the ladder is the durable, examinable fact of this section.

The three rungs

☆ Fixed Point

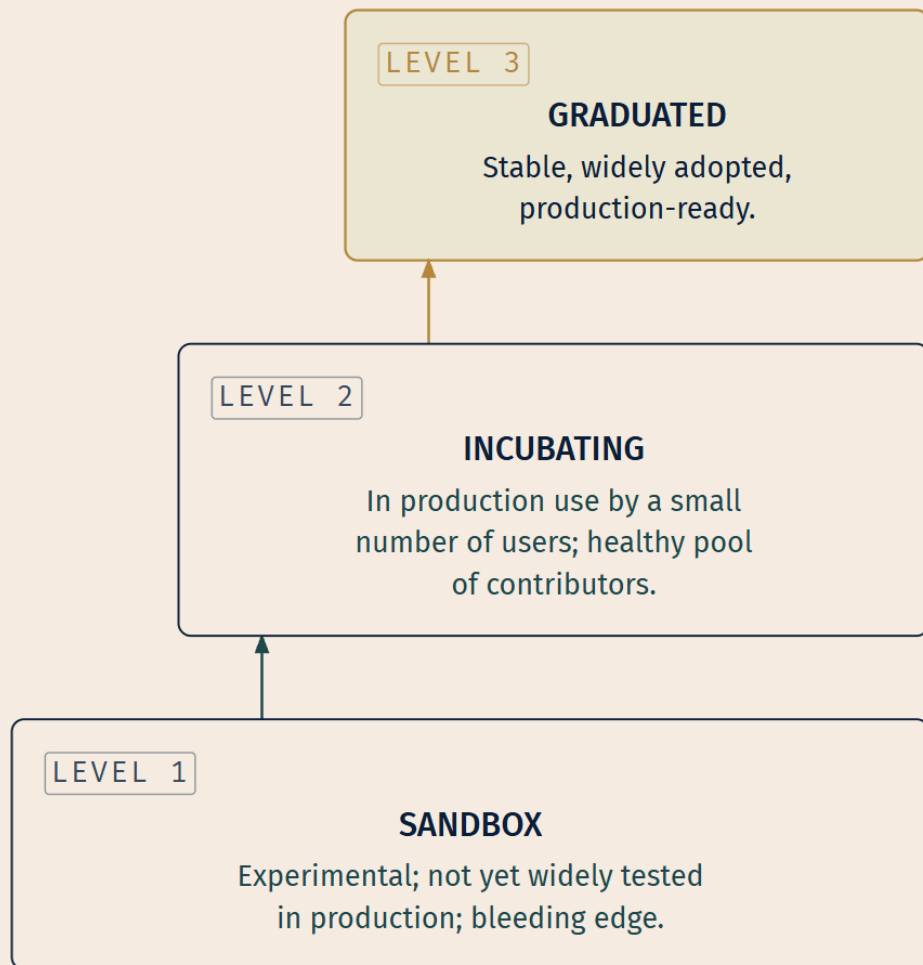
Sandbox → Incubating → Graduated, in that order.

- **Sandbox** — experimental projects, not yet widely tested in production, on the bleeding edge of technology.
- **Incubating** — used successfully in production by a small number of users, with a healthy pool of contributors.
- **Graduated** — stable, widely adopted, production ready, attracting thousands of contributors.

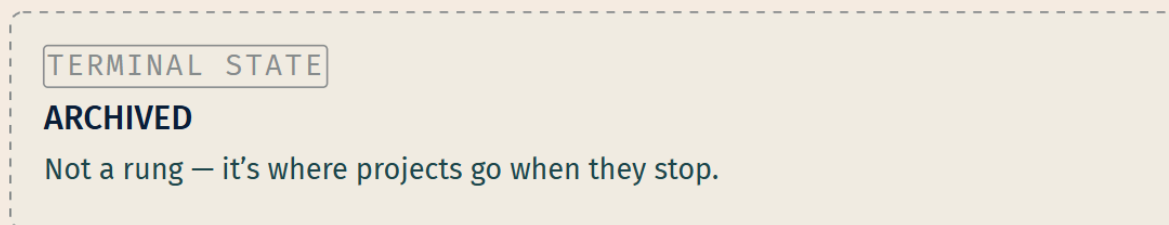
[source: cncf-project-maturity-levels-2026-08-23]

Read those three descriptions as claims about *evidence*, not about quality. A Sandbox project is not bad; it is unproven. An Incubating project is not half-finished; somebody is running it in production and there are enough hands on it to keep it alive. A Graduated project is one where the evidence is overwhelming enough that the foundation is willing to point at it and say: this is safe to build on.

There is a fourth word you will meet if you browse cncf.io, and it is not a rung. **Archived**, "inactive or low activity projects that are no longer supported" [source: cncf-toc-project-lifecycle-process-2026-08-31], is where projects land when nobody is maintaining them any more. Projects do not climb to it. The three-rung progression is correct as a progression, and Archived is the exit.



*Criteria for each level live with the TOC,
in the lifecycle docs — not the projects page.*



LEGEND

☐ MATURITY LEVEL

☐ ARCHIVED — NOT A RUNG

☐ FOCAL TIER

The ladder, annotated with what each level asserts. No project names appear on this figure deliberately — the roster changes, and a figure is the hardest thing in a book to update.

📖 **Worth Securing:** Learn the *levels*, not the roster. Which specific projects are Graduated on the day you sit the exam is a moving target, and no responsible study guide should ask you to memorize a list that changes faster than it prints. What does not move is what each level *asserts*, and that is what a well-built question tests.

That said, a ladder with nothing standing on it is hard to remember. Here are six Graduated projects you have already met by name in this book, as of the 2026-08-23 snapshot of the CNCF projects page [source: cncf-project-maturity-levels-2026-08-23]: **containerd**, **CoreDNS**, **etcd**, **Helm**, **Prometheus**, and **Argo**. Kubernetes itself is Graduated too, which is worth a moment's thought: it climbed the same ladder as everything else. Two or three more projects will have their level named later in this chapter, where the level does specific work in the argument. The instruction above still stands. Do not memorize the roster.

[cross-bearing: see Ch 3 §1 — how the cluster got the shape it has], if you want the history of how the foundation's first project got there.

Where the criteria actually live

📖 **Snag:** The projects page tells you what the levels *mean*. It does not tell you what a project must do to move between them. The criteria live in the **CNCF TOC's project lifecycle documentation**, in the TOC's own repository [source: cncf-project-maturity-levels-2026-08-23]: the due diligence, the adopter interviews, the governance and security requirements. A question that asks where graduation criteria are defined is testing whether you know these are two different documents.

The process itself is more concrete than most people expect. A project applies by opening an application issue on the TOC repository and completes an adopter interview form, naming real adopters willing to be interviewed. A TOC sponsor is assigned. There is a kickoff meeting, a due diligence document, actual interviews with those adopters, an internal TOC comment period, and then a public comment period. Finally the TOC votes [source: cncf-toc-project-lifecycle-process-2026-08-31].

That is not a rubber stamp. It is a months-long evidence-gathering exercise in which strangers are asked, on the record, whether they actually run the thing.

🔍 **Closer Look:** The exact numbers, for the curious, and not exam material: the adopter interview form asks for **five to seven adopters**, the internal TOC comment period runs about **one week**, the public comment period **two weeks**, and the vote requires a **two-thirds supermajority** of the TOC [source: cncf-toc-project-lifecycle-process-2026-08-31]. Learn the shape of the process; the numerals are the TOC's operating detail, not the blueprint's.

🔍 **Closer Look:** The lifecycle document says "Incubation" where the projects page says "Incubating": one names the process, the other names the level. This book uses **Incubating** throughout, matching the projects page and the level name. If you see both forms in the wild, they refer to the same rung.

Who decides what, and why the split matters

The CNCF has more than one governing body, and the exam cares about which does what. The clean version comes straight from the foundation's charter.

The Governing Board is "responsible for marketing and other business oversight and budget decisions for the CNCF" [source: cncf-charter-governance-bodies-2026-08-31]. Money, marketing, and the boundaries of what the foundation is for.

The Technical Oversight Committee, the TOC, is the technical governing body. The charter says the TOC facilitates "driving neutral consensus for: defining and maintaining the technical vision for the Cloud Native Computing Foundation" [source: cncf-charter-governance-bodies-2026-08-31]. Its own README expands the list: approving new projects **within the scope of the CNCF set by the Governing Board**; creating a conceptual architecture for the projects; aligning projects; removing or archiving them; accepting feedback from the end user technical advisory board and mapping it to projects [source: cncf-toc-and-tags-2026-08-23].

🧠 **Mnemonic: The Board draws the fence. The TOC decides what lives inside it.** The Board sets the scope and holds the budget; the TOC approves projects within that scope and owns the technical vision. That one sentence covers the most commonly confused pair in this section.

Technical Advisory Groups, TAGs, are the TOC's technical arms. They are "the primary organizational units within the CNCF that oversee and coordinate interests across projects, working groups, and the broader cloud native community," and they "serve as bridges between CNCF projects, end users, and the Technical Oversight Committee" [source: cncf-tags-current-structure-2026-08-31].

⚠ Navigational Hazards

The TAG list was restructured in 2025, and older study material has the old one.

The five current TAGs [source: [cncf-tags-current-structure-2026-08-31](#)]:

TAG	Scope, in the source's own words
Developer Experience	Databases, Microservices, Streaming, Messaging, API Management, Developer Frameworks
Infrastructure	Data, Storage, Network, DNS, Compute, Service Mesh, Infrastructure-as-Code, Edge, Sovereignty, Load Balancing
Operational Resilience	Observability, Management, Business Continuity, Resource Optimization, Cost Efficiency, Energy, Performance, Troubleshooting, Reliability, Day 2 Operations
Security and Compliance	Security Hygiene, Policy-as-Code, Compliance, Auditing, Threat Modeling, Secure Software Supply Chain
Workloads Foundation	Fundamental cloud native workload execution environments and lifecycle management

The **pre-2025** list — App Delivery, Contributor Strategy, Environmental Sustainability, Network, Observability, Runtime, Security, Storage [source: [cncf-toc-and-tags-2026-08-23](#)] — appears throughout older guides, blog posts and courses, which means you may well arrive holding it. The restructuring was approved by the TOC and announced in May 2025 [source: [cncf-tags-current-structure-2026-08-31](#)]. If a question offers you "TAG Observability," it is offering you the old map.

The End User Technical Advisory Board, the End User TAB, is the last piece, and it closes a loop. The charter says the TAB "will serve as the voice of End Users in the CNCF community, advance topics of concern to End Users, and raise awareness about the needs and perspectives of end users" [source: [cncf-charter-governance-bodies-2026-08-31](#)]. And the TOC's own list of responsibilities includes "accepting feedback from the end user technical advisory board and mapping it to projects" [source: [cncf-toc-and-tags-2026-08-23](#)]. The TAB collects what the people running this software actually need; the TOC turns that into project direction. It is a designed feedback path, not a suggestion box.

[cross-bearing: see Ch 17 §8 — how the project actually runs] for the Kubernetes-internal governance that sits inside all this, and which is a genuinely different structure with confusingly similar vocabulary.

The map of the terrain

The **CNCF Landscape** is the foundation's attempt to chart the whole space, not just its own projects. Its own repository describes it as "a map through the previously uncharted terrain of cloud native technologies," attempting to categorize most of the projects and product offerings in the space, "of which CNCF-hosted projects are a particularly well-traveled path" [source: [cncf-landscape-and-community-2026-08-23](#)].

The interactive version at landscape.cncf.io is generated from the repository's data files. Entries must represent cloud native technologies with meaningful community adoption, fit an existing category, and appear in the single category where they best fit. The categories group by layer [source: [cncf-landscape-and-community-2026-08-23](#)]:

- **Provisioning** — automation and configuration, container registries, security and compliance, key management
- **Runtime** — cloud native storage, container runtime, cloud native network
- **Orchestration and Management** — scheduling and orchestration, coordination and service discovery, service proxy, API gateway, service mesh
- **App Definition and Development** — databases, streaming and messaging, application definition and image build, continuous integration and delivery
- **Observability and Analysis** — monitoring, logging, tracing, chaos engineering
- **Platforms and Special** — Kubernetes distributions, hosted and installable platforms, PaaS and container services, serverless

Read that list slowly and you will notice something: it is roughly the table of contents of this book, in a different order. Runtime is Chapter 2. Cloud native network is Chapters 9 and 10. Cloud native storage is Chapter 11. Scheduling and orchestration is Chapters 3 through 8. CI/CD is Chapters 14 and 15. Service mesh and serverless are two sections from now. Observability is Chapter 18.

Entries are also tagged by the TAG that owns them, which is the connective tissue between the two halves of this section: the governance structure and the map are the same structure, seen from different sides.

[*cross-bearing: see Ch 2 §5 — the Open Container Initiative*], which is a different standards body with an overlapping mission and is worth keeping distinct in your head. And [*cross-bearing: see Ch 15 §6 — the other agent*], where you last met a Graduated project being named as such.

3 — Small Pieces, Replaced Whole

Three of the definition's technologies deserve their own treatment, and they deserve it *together*, because separately they are three definitions and together they are one argument.

Microservices

A microservices architecture is an architectural approach that breaks applications into individual independent (micro)services, with each service focused on a specific functionality.

[source: [cncf-glossary-microservices-monoliths-coupling-2026-08-31](#)]

The problem it addresses is stated most sharply as a scaling complaint. In a monolith, if one function of the application gets hammered, "the entire app would have to be scaled to accommodate the increase —

a very inefficient use of resources" [source: cncf-glossary-microservices-monoliths-coupling-2026-08-31]. Everything ships together, so everything scales together.

Breaking the app apart lets "different teams to work simultaneously on a small part of a bigger application without inadvertently negatively impacting the rest of the app" [source: cncf-glossary-microservices-monoliths-coupling-2026-08-31].

But the glossary is unusually honest about the bill: "it also creates operational overhead — the things you need to deploy and keep track of increase by order of magnitude" [source: cncf-glossary-microservices-monoliths-coupling-2026-08-31]. And the entry on **monolithic apps** goes further, arguing the other side outright: early in a product's lifecycle it may be advantageous to defer the complexity and build a monolith until the product is determined successful. "Crafting a microservices-based app before it has proven valuable may be premature spending of engineering effort. If the application yields no value, that effort becomes wasted" [source: cncf-glossary-microservices-monoliths-coupling-2026-08-31].

🔒 **Worth Securing:** The CNCF's own glossary says a well-designed monolith "can uphold lean principles by being the simplest way to get an application up and running." That is the foundation for cloud native computing saying, in writing, that microservices are not always the right answer. Take it as a license to think rather than a license to argue: the exam tests what microservices *are* and what they trade away, not which you should pick.

Loose coupling

Loosely coupled architecture is the property that makes any of it work: an architectural style where individual components are built independently of one another, each performing a specific function in a way that can be used by any number of other services. It is "generally slower to implement than tightly coupled architecture but has a number of benefits, particularly as applications scale," chiefly that teams can "develop features, deploy, and scale independently" [source: cncf-glossary-microservices-monoliths-coupling-2026-08-31].

This is the same loose coupling the definition in §1 hung its five characteristics on. It is not a separate concept that happens to share a name.

Immutable infrastructure

Immutable Infrastructure refers to computer infrastructure (virtual machines, containers, network appliances) that cannot be changed once deployed.

[source: cncf-glossary-immutable-infrastructure-2026-08-31]

You do not patch a running thing. You build a new thing and replace the old one. Enforcement comes either from automation that overwrites unauthorized modifications, or from systems that prevent changes altogether.

The benefit the glossary emphasizes is not security. Notice which one it reaches for first: "Operating such a system becomes a lot more straightforward because administrators can make assumptions about it" [source: cncf-glossary-immutable-infrastructure-2026-08-31]. When nothing drifts, what you deployed is what is running, and you can reason about it. Combined with version control, "there is a durable audit log of every authorized change to a system" [source: cncf-glossary-immutable-infrastructure-2026-08-31].

📌 **Snag:** This is the book's *second* immutability and it is not the same as the first. Chapter 2 §2 taught **image immutability**: an image, once built, has fixed content addressed by digest. **Immutable infrastructure** is the operational discipline of replacing rather than modifying deployed things. The image being immutable is what makes the infrastructure discipline possible; they are not the same claim. Always write the full two-word phrase. [cross-bearing: see Ch 2 §2 — *what's inside an image*].

Declarative APIs

You have known this one since Chapter 4 and it does not need re-teaching. What it needs is placement: **declarative APIs are named in the CNCF definition as a cloud native technology**, alongside containers and microservices [source: cncf-cloud-native-definition-2026-08-23]. The thing you have been doing since Chapter 4, describing desired state and letting a controller reconcile, is not a Kubernetes quirk. It is a named characteristic of the whole paradigm. [cross-bearing: see Ch 4 §1 — *you file a declaration*].

Why these three are one argument

Here is the connection, and it is the only thing in this section that is not available elsewhere in the book.

You can only replace a piece *whole* if the piece is small and independently deployable. That is microservices. You can only make replacement the default operation if the thing being replaced was never meant to be edited in the first place. That is immutable infrastructure. And you can only do that safely, repeatedly, at scale, if what you replace it with is *described* rather than *commanded*, if you can say "there should be five of these, at this version" and have something else work out the difference. That is declarative APIs.

Take any one away and the other two get much harder. Small pieces you have to patch in place are just a distributed monolith with extra network hops. Immutable replacement of a monolith means redeploying everything to change one line. Declarative desired state over components you edit by hand is a description that is constantly wrong.

That is what §1's "loosely coupled systems... combined with robust automation" is actually made of.

🔍 **Closer Look:** If several of these feel familiar from Chapter 15, that is not a coincidence: several of the twelve factors are these same principles under older names, written for a platform that did not exist yet. [cross-bearing: see Ch 15 §1 — *twelve factors*]. Also relevant: [cross-bearing: see Ch 5 §4 — *scheduled once, replaced never*], which is immutable infrastructure enacted at the Pod level, and where you first met the replacement discipline as a concrete behavior rather than a principle.

🧭 Taking Your Bearings: The Definition, the Ladder, and Who Decides

Six questions. One of them reaches back fifteen chapters.

1. Which of the following is stated *in* the CNCF Cloud Native Definition v1.1?

A) Cloud native architecture requires a service mesh, a container runtime, and declarative APIs at minimum B) Cloud native workloads are those that run in public cloud environments rather than on-premises C) Cloud native systems are characterized by loosely coupled systems that interoperate securely, resiliently, manageably, sustainably, and observably D) Cloud native applications must be decomposed into microservices before they qualify as cloud native

2. A colleague tells you a project is "CNCF Incubating." What has that project demonstrated?

A) Production use by a small number of users, with a healthy pool of contributors B) That it is experimental and not yet widely tested in production C) That it is stable, widely adopted, and production ready D) That it has been inactive long enough to lose support

3. Where are the criteria a project must meet to graduate defined?

A) On the [cncf.io](https://cncf.io/projects) projects page, beside each maturity level B) In the CNCF Landscape repository's data files C) In the CNCF charter, alongside the responsibilities of the governing bodies D) In the CNCF TOC's project lifecycle documentation

4. Which body approves new CNCF projects, and within what constraint?

A) The Governing Board, within the technical vision set by the TOC B) The TOC, within the scope of the CNCF set by the Governing Board C) The End User TAB, within the categories defined by the Landscape D) Any Technical Advisory Group, within its own technical area

5. The CNCF Cloud Native Definition lists containers, service meshes, multi-tenancy, microservices, immutable infrastructure, serverless, and declarative APIs. What does the definition say about this list?

A) These seven are the complete set of cloud native technologies B) That every cloud native system uses containers, and the other six are optional C) That the list is ordered by importance, containers first D) That the list is non-exhaustive

6. [retrieval: ch2] A container image, once built, cannot be altered — a change produces a new image with a new digest. A production environment practicing *immutable infrastructure* never patches a running server; it replaces it. What is the relationship between these two ideas?

A) They are the same principle, described at two different scales B) Immutable infrastructure is the CNCF's term for image immutability C) The first is a property of a build artifact; the second is an operational discipline for deployed systems, and the first makes the second practical D) Image immutability applies to containers; immutable infrastructure applies only to virtual machines

Answers with Explanations:**1. C.**

- **A is wrong.** No specific technology is required. Service meshes and declarative APIs appear in the technology list, which the definition explicitly calls non-exhaustive.
- **B is wrong,** and it is wrong in the most instructive possible way. The definition's very first sentence names "public, private, hybrid cloud." Reading *cloud native* as *public cloud* is the single most common misconception about the term, and the definition rules it out in its opening sentence.
- **C is correct** — verbatim from the definition's second sentence [source: cncf-cloud-native-definition-2026-08-23].
- **D is wrong** for the same reason as A. Microservices are listed as a typical component, not a requirement, and the glossary elsewhere argues that a monolith can be the right choice.

2. A. Straight from the projects page: Incubating projects are "used successfully in production by a small number of users with a healthy pool of contributors" [source: cncf-project-maturity-levels-2026-08-23]. **B** is Sandbox: experimental, bleeding edge, not widely tested. **C** is Graduated: stable, widely adopted, production ready. **D** is Archived, which is not a rung on the ladder at all but the terminal state for projects that are no longer maintained.

3. D. The projects page describes what the *levels* mean; the criteria for moving between them, due diligence and adopters and governance and security practices, are in the TOC's project lifecycle documentation in the TOC repository [source: cncf-project-maturity-levels-2026-08-23]. **A** is the trap, and it is a reasonable-sounding one: the levels and the criteria feel like they should live together, and they do not. **B** is the Landscape, which catalogs the terrain and has nothing to do with graduation. **C** is the charter, which establishes the bodies and their responsibilities, not the project criteria.

4. B. The TOC approves "new projects within the scope of the CNCF set by the Governing Board" [source: cncf-toc-and-tags-2026-08-23]. **A** inverts the relationship: the Board does not work within the TOC's vision; the TOC works within the Board's scope. **C** misassigns the End User TAB, which is the voice of end users in community decisions and feeds requirements to the TOC; it does not approve projects. **D** misassigns TAGs, which coordinate across projects and bridge to the TOC; project approval is the TOC's own vote.

5. D. "This list is non-exhaustive" appears in the definition itself, in the same sentence as the list [source: cncf-cloud-native-definition-2026-08-23].

- **A is wrong,** and it is the misreading the definition preemptively refutes. Seven items in an authoritative document read like an enumeration; the document says otherwise in the same breath.
- **B is wrong,** and it is the most widely held belief in this space. Containers are the first-named technology, not a requirement. The definition names no mandatory member of the list.
- **C is wrong.** No ordering claim appears anywhere in the document.

6. C. Two different immutabilities. Image immutability (*Ch 2 §2*) is a property of a build artifact: fixed content, addressed by digest. Immutable infrastructure is an operational discipline for deployed systems, infrastructure "that cannot be changed once deployed" [source: cncf-glossary-immutable-infrastructure-2026-08-31], and the reason it is practical at all is that the artifacts you replace things with are themselves fixed and identifiable.

- **A is wrong** because they are not the same principle; one is a property, the other is a practice, and you can have the first without adopting the second.
 - **B is wrong** — the CNCF glossary carries both concepts under different entries.
 - **D is wrong.** The glossary's own definition names "virtual machines, containers, network appliances" together. Neither concept is confined to one substrate.
-

How'd You Do?

5–6 correct: You have the ecosystem's vocabulary. §4 is the chapter's spine; go there rested.

3–4 correct: Solid. Review the ones you missed and continue. §4 does not depend on this section, so nothing is blocked.

0–2 correct: Stop before §4. Re-read **§1's characteristics clause** and **§2's three rungs**. Those are the two blocks the Exam Alert and the Chapter Summary will both come back to, and they are ten minutes of re-reading rather than a re-read of the section. If it was specifically question 6 you missed, the fastest repair is **Chapter 2 §2**.

Checkpoint: You've Now Mastered

✓ The published definition of *cloud native*, its five characteristics, and what it explicitly does not mean ✓ The maturity ladder, what each rung asserts, and where the criteria live ✓ The CNCF's governing bodies and which one does what ✓ Microservices, loose coupling, and immutable infrastructure as one mutually reinforcing argument

That is the ecosystem's vocabulary. The next two sections are the ones with teeth, and the first of them is the section more of this book points at than any other.

..1 §4 — Every Place Kubernetes Lets You In

Four times in this book you have watched the same thing happen.

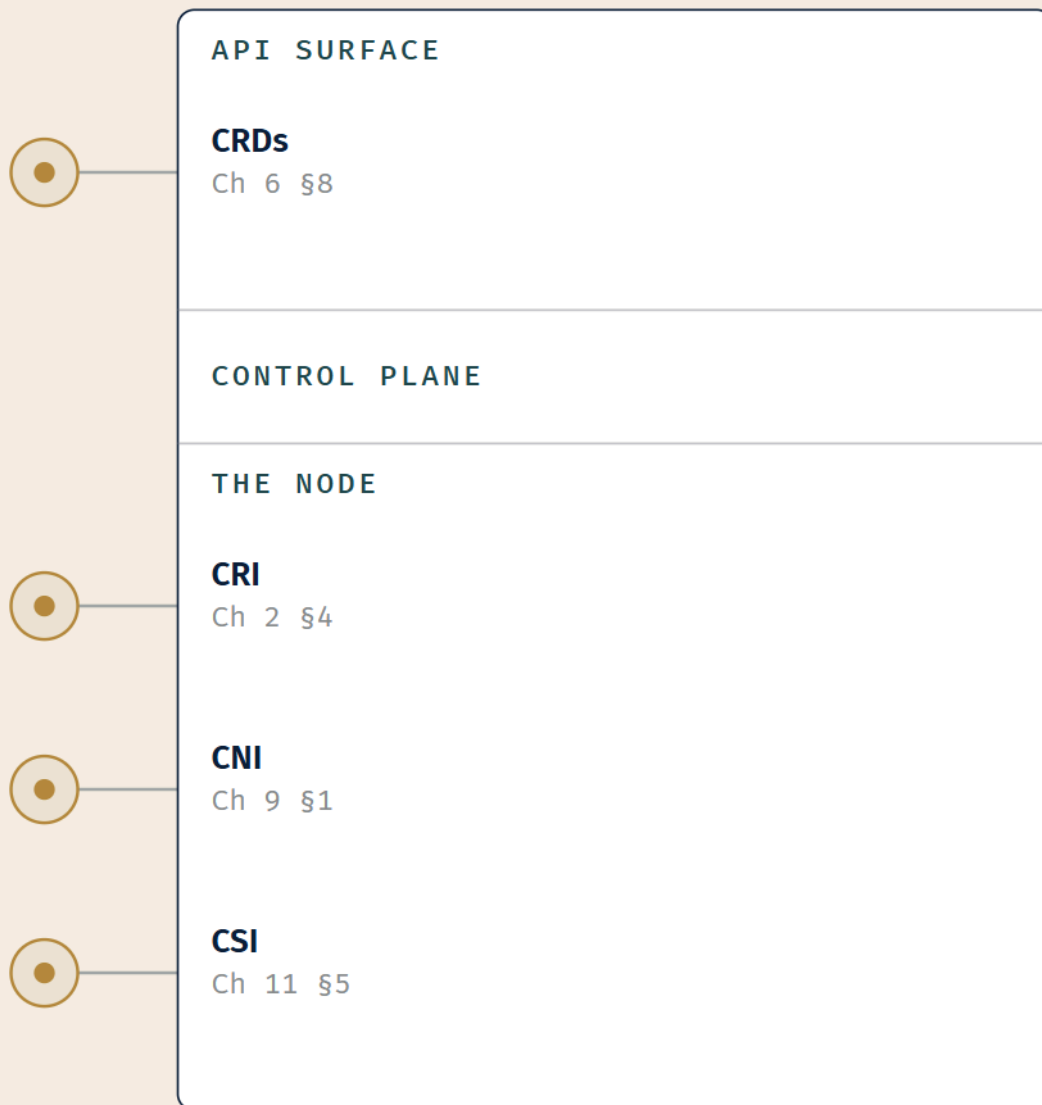
Chapter 2: Kubernetes needs to run containers. It does not implement a container runtime. It publishes the **Container Runtime Interface**, and containerd or CRI-O implements it. [*cross-bearing: see Ch 2 §4 — the container runtime interface*].

Chapter 9: Kubernetes needs pod networking. It does not implement it. It requires a plugin that satisfies the **Container Network Interface**, and Calico or Cilium or Flannel implements it. *[cross-bearing: see Ch 9 §1 — four rules and a plugin]*.

Chapter 11: Kubernetes needs to attach storage. It does not implement storage. It publishes — well, it *adopts* — the **Container Storage Interface**, and a driver from a storage vendor implements it. *[cross-bearing: see Ch 11 §5 — who actually provides the storage]*.

Chapter 6: Kubernetes needs object types it did not anticipate. It does not add them. It provides **CustomResourceDefinitions**, and you or a vendor define the kinds you need, with controllers to act on them. *[cross-bearing: see Ch 6 §8 — the control loop, extended]*.

This section puts those four side by side. It does not redefine any of them; each was taught in full where you met it, and if any of the four is fuzzy, the pointer above is where to go rather than the paragraph below.



*Kubernetes defines the shape.
Somebody else supplies the plug.*

LEGEND

● SOCKET = AN EXTENSION POINT

*The book's four pluggable interfaces, each marked at the layer it serves and the chapter that taught it.
Four sockets, four chapters, one shape.*

The judgment, stated as a judgment

☆ Fixed Point

At every one of the four pluggable interfaces — **CRI, CNI, CSI, and CRDs** — Kubernetes defines an interface and hands the implementation to somebody else.

That grouping is *this book's*, and honesty requires saying so. Kubernetes does not publish a document titled "the four pluggable interfaces." What Kubernetes publishes is a longer, differently-cut list, and Chapter 10 promised you would see both maps side by side. Here is the other one.

The documentation's own extension points [source: k8s-docs-extending-kubernetes-2026-08-23]:

#	Extension point	What it lets you replace or add
1	kubectl plugins and client credential providers	Client-side behavior
2	API access extensions	Authentication, authorization, dynamic admission control via webhooks (<i>Ch 8 §2</i>)
3	API extensions	CustomResourceDefinitions and the API aggregation layer
4	Scheduling extensions	Scheduler plugins, profiles, and custom schedulers (<i>Ch 7 §6</i>)
5	Controllers	Custom controllers over custom or built-in resources; the operator pattern
6	Infrastructure extensions	Device plugins; storage plugins (CSI); network plugins (CNI); container runtime (CRI); kubelet image credential providers

Six extension points, with five plugin types crammed into the sixth, and custom resources filed under a completely different heading from the storage and network plugins they sit beside in our four.

Both maps are correct. They are answering different questions. The documentation is answering *where can I hook in?*, and the honest answer is "in a great many places, at very different levels of the stack." This book is answering *what is the pattern?*, and the four we picked are the four where the same thing happens: Kubernetes writes down what a thing must do, and then declines to be the thing.

🔒 **Worth Securing:** When two authoritative-looking lists of the same subject have different lengths, the useful question is almost never "which is right." It is "what is each one for." Hold both. The exam can ask you either.

The extension surface beyond the four

Two members of the documentation's list deserve a sentence each, because you have not met them and they are in scope.

The API aggregation layer is the *other* way to add APIs. "The aggregation layer allows Kubernetes to be extended with additional APIs, beyond what is offered by the core Kubernetes APIs" [source: k8s-docs-api-aggregation-and-device-plugins-2026-08-31]. It runs in-process with the kube-apiserver and does

nothing until you register an `APIService` object, which "claims" a URL path in the Kubernetes API. From then on, the aggregation layer proxies anything sent to that path to the registered service.

And the documentation adds a sentence that settles a question this section would otherwise have to argue: "The aggregation layer is different from Custom Resource Definitions, which are a way to make the kube-apiserver recognize new kinds of object" [source: k8s-docs-api-aggregation-and-device-plugins-2026-08-31].

With a CRD, the API server stores and serves your objects for you. With aggregation, you run your own API server and Kubernetes routes to it. More work, more flexibility. You have already met one in the wild: metrics-server registers itself through the aggregation layer, and its own installation notes say the "kube-apiserver must enable an aggregation layer" [source: metrics-server-install-2026-08-31]. *[cross-bearing: see Ch 13 §7 — numbers nobody collects by default]*.

Device plugins "let you configure your cluster with support for devices or resources that require vendor-specific setup, such as GPUs, NICs, FPGAs, or non-volatile main memory" [source: k8s-docs-api-aggregation-and-device-plugins-2026-08-31]. A device plugin registers with the kubelet, "sends the kubelet the list of devices it manages, and the kubelet is then in charge of advertising those resources to the API server as part of the kubelet node status update" [source: k8s-docs-api-aggregation-and-device-plugins-2026-08-31].

If that shape looks familiar, it should. Kubernetes does not know what a GPU is. It knows how to advertise a resource somebody else told it about.

A note on the fourth interface's two names

Chapter 2 §4 called the set "CRI, CNI, CSI, and API extensions." This chapter, Chapter 6, Chapter 10 and Chapter 11 all call the fourth one **CRDs**.

Both are defensible, and the documentation is the reason: "API extensions" is the documentation's own heading for a category that contains two things, CRDs and the aggregation layer. When this book counts four interfaces, the fourth is specifically **CRDs**, because CRDs are the one where the interface-and-implementation shape is cleanest. The wider surface that Chapter 2 gestured at is the whole right-hand column above.

 **Mnemonic: Run it, wire it, store it, name it.** CRI runs your containers. CNI wires them up. CSI stores their data. CRDs name things Kubernetes had never heard of. Four verbs, four sockets, in the order you met them if you read the book front to back.

[cross-bearing: see Ch 8 §2 — three gates and a logbook] for admission webhooks, *[cross-bearing: see Ch 7 §6 — overruling the scheduler, and replacing it]* for scheduler plugins, and *[cross-bearing: see Ch 12 §8 — rules that watch]* for the policy engines that live on the admission hook. Each of those is a place Kubernetes lets somebody else in.

One more, which is a genuinely useful detail rather than a completeness note: Helm charts have a `crds/` directory precisely because a chart that ships custom resources has to install the definitions before the objects that use them [source: helm-crd-best-practices-2026-08-31]. The packaging format had to grow a special case for the extension mechanism. *[cross-bearing: see Ch 14 §6 — which one, when].*

That is the map. §9 says what it means.

§5 — A Network That Knows What It's Carrying

Soundings question 8 asked what protects the leg from the Ingress to the Pod once TLS has terminated at the edge. The answer was *nothing, by default*, and Chapter 10 said so twice while pointedly declining to name a remedy.

This is the remedy.

What a service mesh is

Start with the vendor-neutral definition, because it frames the problem before it sells the solution.

In a microservices world, apps are broken down into multiple smaller services that communicate over a network. Just like your wifi network, computer networks are intrinsically unreliable, hackable, and often slow. Service meshes address this new set of challenges by managing traffic (i.e., communication) between services and adding reliability, observability, and security features uniformly across all services.

[source: cncf-glossary-service-mesh-2026-08-31]

The glossary is specific about what makes this hard without a mesh. Once you have hundreds or thousands of services all talking over the network, individual applications "may need to encrypt communications to support regulatory requirements, provide common metrics to operations teams, or provide detailed insight into traffic to help diagnose issues. If built into the individual applications, each one of these features will cause friction between teams and slow down development of new features" [source: cncf-glossary-service-mesh-2026-08-31].

And then the payoff sentence, which contains the whole point:

☆ Fixed Point

Service meshes add reliability, observability, and security features uniformly across all services across a cluster **without requiring code changes**. Before service meshes, that functionality had to be encoded into every single service, becoming a potential source of bugs and technical debt.

[source: cncf-glossary-service-mesh-2026-08-31]

Istio, the mesh this section teaches, states its own version in almost the same words: a service mesh "gives applications capabilities like zero-trust security, observability, and advanced traffic management, without code changes" [source: istio-service-mesh-2026-08-23].

📌 **Snag:** *Without code changes* is not a marketing flourish. It is the defining property, and it is what a well-written exam question tests. If an answer option says a mesh requires the application to be modified, instrumented, or recompiled, that option is describing something else. The whole value proposition is that the application does not know the mesh is there.

Istio names three reasons you would want one [source: istio-service-mesh-2026-08-23]:

- **Security** — "a market-leading zero-trust solution based on workload identity, mutual TLS, and strong policy controls"
- **Observability** — telemetry generated within the mesh, integrating with tools such as Grafana and Prometheus
- **Traffic management** — service-level routing control, for use cases like A/B testing and canary deployments (*the strategy vocabulary is [cross-bearing: see Ch 15 §2 — ways to replace what's running]*)

The two planes — and the third one you already know

This is the most dangerous piece of vocabulary in the chapter, and it is dangerous for an honest reason: the two things share a name because they genuinely do the same *kind* of job, at different layers.

☆ Fixed Point

A mesh's **data plane** is the set of proxies that mediate and control all network communication between services. A mesh's **control plane** is what manages and configures those proxies.

Neither of them is the cluster's control plane from Chapter 3.

[source: istio-service-mesh-2026-08-23]

THE CLUSTER'S CONTROL PLANE

(Ch 3 §2)

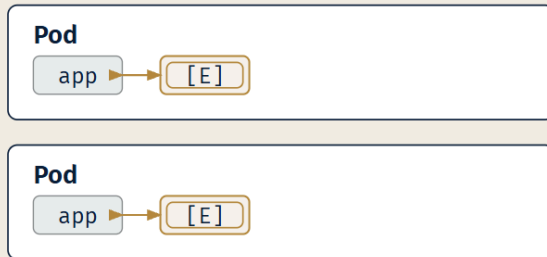
kube-apiserver • etcd • scheduler • controller-manager

Manages: **Kubernetes OBJECTS***(a different thing entirely)***THE MESH'S CONTROL PLANE**

Distributes policy + certificates to the proxies

Manages: **PROXIES****THE MESH'S DATA PLANE**

the proxies themselves — every byte between services passes through here

SIDECAR MODE*one Envoy per Pod***AMBIENT MODE**

*[E] = Envoy. Both columns are the **SAME** data plane,
arranged two ways — not two products.*

☒ ENVOY — same engine

☐ ZTUNNEL — not Envoy

☐ CONTROL PLANE (context)

The separation is the argument. Two mesh planes, and the cluster's control plane drawn deliberately apart from both.

The mesh's control plane is doing a recognizable job. Istio's security documentation describes it plainly: "The configuration API server distributes to the proxies: authentication policies, authorization policies, secure naming information" [source: [istio-security-mtls-identity-2026-08-31](#)]. It takes policy you wrote and pushes it out to the things that enforce it.

Compare that with the cluster's control plane, which takes objects you declared and drives the cluster toward them. Same *shape*, a central thing configuring distributed things, and completely different subject matter. One manages Kubernetes objects. The other manages proxies.

 **Mnemonic: The cluster's control plane manages objects. The mesh's control plane manages proxies.** When you see "control plane" bare in a question stem, it means the cluster's. When a question means the mesh's, a well-written one will say so.

[cross-bearing: see *Ch 3 §2 — the control plane*], for the one you have known since Chapter 3.

Sidecar and ambient: two arrangements, one data plane

Soundings question 4 asked what a sidecar shares with the container beside it. The answer, the Pod's network namespace and therefore `localhost`, is exactly what makes the sidecar mesh model work.

Envoy, the proxy at the heart of Istio's data plane, describes itself as "an L7 proxy and communication bus designed for large modern service oriented architectures" [source: [envoy-what-is-envoy-2026-08-31](#)]. And it describes its own deployment shape in a sentence that repays reading twice:

Envoy is a self contained process that is designed to run alongside every application server. All of the Envoy's form a transparent communication mesh in which each application sends and receives messages to and from `localhost` and is unaware of the network topology.

[source: [envoy-what-is-envoy-2026-08-31](#)]

Unaware of the network topology. That is "without code changes," told from the proxy's side. The application talks to `localhost`. Something else worries about TLS, retries, routing, and telemetry.

⚠ Navigational Hazards

"A service mesh means sidecars" is out of date, and the exam knows it.

Istio supports two data plane modes. In **sidecar mode**, an Envoy proxy is deployed alongside each Pod. In **ambient mode**, "Istio implements its features using a per-node Layer 4 (L4) proxy, and optionally a per-namespace Layer 7 (L7) proxy" [source: istio-ambient-mode-2026-08-31]. In ambient mode, "workload pods no longer require proxies running in sidecars in order to participate in the mesh."

The per-node L4 component is called **ztunnel**, "a purpose-built, per-node proxy that powers Istio's ambient data plane mode" [source: istio-ambient-mode-2026-08-31], and the set of L4 functions it implements is described as a **secure overlay**. The optional per-namespace L7 component is called a **waypoint proxy**, and here is the sentence that ties the two modes together: "The waypoint proxy is a deployment of the Envoy proxy; the same engine that Istio uses for its sidecar data plane mode" [source: istio-ambient-mode-2026-08-31].

Both modes use Envoy. They are two arrangements of the same data plane, not two competing products, and Pods in both modes can coexist in the same mesh [source: istio-ambient-mode-2026-08-31].

Chapter 5 promised you would meet the sidecar again in Chapter 17 [*cross-bearing: see Ch 5 §2 — more than one container aboard*]. Here it is, and here is the complication: the sidecar was the mesh's original answer, and it is now one of two.

mTLS, zero trust, and the leg that was unprotected

Zero trust is the security model a mesh is built to deliver, and the CNCF glossary states its core principle in four words: **never trust, always verify** [source: cncf-glossary-zero-trust-architecture-2026-08-31].

The glossary's framing of *why* is the part to internalize:

In many networks today, within the corporate network, systems and devices inside may freely communicate with each other as they are within the trusted boundary of the corporate network perimeter. Zero trust architecture takes the opposite approach where although inside the network perimeter, components within the system first have to pass verification before any communication is made.

[source: cncf-glossary-zero-trust-architecture-2026-08-31]

And in its account of the problem this addresses, the same entry puts the reasoning in a single sentence:

Zero trust architecture however, recognizes that **trust is a vulnerability**.

[source: cncf-glossary-zero-trust-architecture-2026-08-31]

The consequence it names is lateral movement: if an attacker takes one trusted device inside the perimeter, they can move sideways through everything else that trusts it.

Now go back to Soundings question 8. TLS terminates at the Ingress. The traffic continues to the Pod unencrypted. NetworkPolicy can say *who may reach whom*, and cannot encrypt a single byte [cross-bearing: see Ch 10 §7 — *what NetworkPolicy cannot do*]. That unprotected leg, inside the perimeter, between components that trust each other because they are both inside, is precisely the vulnerability the glossary is describing.

Mutual TLS is the mesh's answer. Istio's stated security goals are three, and they read as a direct response to that gap: "Security by default: no changes needed to application code and infrastructure; Defense in depth: integrate with existing security systems to provide multiple layers of defense; Zero-trust network: build security solutions on distrusted networks" [source: istio-security-mtls-identity-2026-08-31].

The mechanism has three named components: a **Certificate Authority** for key and certificate management; a **configuration API server** that distributes authentication policies, authorization policies, and secure naming information to the proxies; and **sidecar and perimeter proxies** acting as Policy Enforcement Points [source: istio-security-mtls-identity-2026-08-31].

And there is an identity model underneath it. "The Istio identity model uses the first-class service identity to determine the identity of a request's origin," flexible enough to represent "a human user, an individual workload, or a group of workloads" [source: istio-security-mtls-identity-2026-08-31]. That is what *mutual* means here: not just the client verifying the server, but the server verifying the client's workload identity too.

The handshake, in Istio's own summary: outbound traffic from a client is re-routed to the client's local sidecar Envoy; the client-side Envoy starts a mutual TLS handshake with the server-side Envoy; they establish the connection; the server-side Envoy authorizes the request [source: istio-security-mtls-identity-2026-08-31].

Four steps, and the application was not consulted at any of them.

🔔 **Closer Look:** Meshes also support a **permissive mode**, where "the server accepts both plaintext and mutual TLS traffic," which exists "to provide greater flexibility for the on-boarding process" [source: istio-security-mtls-identity-2026-08-31]. You cannot switch a hundred services to mandatory mTLS on a Tuesday afternoon; permissive mode is how a real migration happens. Worth knowing the concept exists; the configuration is well past associate tier.

What a mesh adds over what you already have

This is the boundary question, and you have been walking toward it since Chapter 9.

You already have	It gives you	It cannot give you
Service (Ch 9 §2)	A stable virtual address and load balancing across healthy backends	Encryption, retries, per-request routing, telemetry
Ingress / Gateway API (Ch 10 §2, §5)	L7 routing for north-south traffic; TLS termination at the edge	Anything about the leg from edge to Pod
NetworkPolicy (Ch 10 §6)	Allow-listed connectivity — who may talk to whom	Encryption, identity, observability, traffic shaping
A service mesh	mTLS between every pair of workloads; per-service telemetry; L7 traffic control east-west	It is not free — it is more moving parts, more resource use, and another control plane to run

The honest summary: Service gives you a name, NetworkPolicy gives you a fence, and a mesh gives you a *conversation you can inspect and trust*. The glossary is candid that the sidecar model "uses more computing resources and becomes more complex to manage as your system grows" [source: cncf-glossary-service-mesh-2026-08-31], which is the pressure ambient mode is responding to.

🔍 **Closer Look: Istio and Linkerd** are the two most widely deployed meshes in the CNCF ecosystem. This section teaches Istio's model because it is the most widely documented, and because the sidecar/ambient split is the fact most likely to be tested. At associate tier, know what a mesh *is* and what it *buys*, not how to configure one. If you find yourself learning the names of a mesh's own custom resources, you have gone past the exam.

[cross-bearing: see Ch 9 §6 — the component that makes it real] and [cross-bearing: see Ch 12 §4 — secrets are not encrypted], both of which are the same lesson in different clothes: the object is not the mechanism, and encoding is not encryption.

🕒 Taking Your Bearings: Extension Points, and the Layer That Knows What It's Carrying

Five questions. Two of them draw their answers from earlier chapters, and the first of those is the most important question in this chapter.

1. [retrieval: ch2, ch6, ch9, ch11] An organization runs Kubernetes on hardware with a proprietary storage array, a network fabric that requires a vendor's own routing agent, a container runtime hardened for their compliance regime, and an in-house database platform they want their developers to manage with `kubectl` the same way they manage Deployments.

Kubernetes ships an implementation for none of these four things. What does it ship instead, and what is the single sentence that covers all four cases?

A) Configuration flags for each, with no common pattern across the four cases B) Reference implementations that each vendor forks and modifies for their own hardware C) Admission webhooks for

each, with all four implemented as webhook backends D) A published interface for each — in all four cases Kubernetes defines what the thing must do and hands the implementation to somebody else

2. A team already runs their own API server for an internal platform, with its own storage and its own request handling. They want `kubectl` to reach it through the standard Kubernetes API path, without giving up their existing server. Which extension mechanism fits?

A) The API aggregation layer, registered with an `APIService` object that claims a URL path B) A `CustomResourceDefinition` C) A device plugin registered with the kubelet D) A mutating admission webhook

3. [retrieval: ch10] A cluster has `NetworkPolicy` enforced by a capable CNI plugin, with a strict default-deny posture and explicit allow rules. TLS terminates at the Ingress. A security review asks whether traffic between two Pods inside the cluster is encrypted. What is the accurate answer?

A) Yes — `NetworkPolicy` encrypts the traffic on the connections it allows B) No — `NetworkPolicy` controls which connections are permitted, not whether they are encrypted, and nothing else described provides encryption on that leg C) Yes — the connection originated over TLS at the Ingress, so it stays encrypted to the Pod D) No — but the Kubernetes network model requires every CNI plugin to encrypt pod-to-pod traffic anyway

4. In a service mesh, what is the relationship between the data plane and the control plane?

A) The data plane configures the proxies; the control plane carries the application traffic B) The data plane is the kube-apiserver and etcd; the control plane is the set of Envoy proxies C) The data plane is the proxies that mediate service-to-service communication; the control plane manages and configures those proxies D) They are two names for the same component, used in different documentation

5. Which statement about Istio's data plane modes is correct?

A) Ambient mode uses a per-node L4 proxy and optional per-namespace L7 proxies, and the L7 waypoint is a deployment of Envoy — the same engine sidecar mode uses B) Sidecar mode uses Envoy; ambient mode replaces Envoy with a purpose-built proxy at every layer C) A mesh must use sidecars; ambient mode is a non-Istio concept D) Sidecar and ambient mode Pods cannot coexist within the same mesh

Answers with Explanations:

1. D. This is the chapter's central claim, and if you got it without looking back, §9 is going to land properly.

- CRI (*Ch 2 §4*) — Kubernetes defines the protocol between kubelet and runtime; containerd or CRI-O implements it.
- CNI (*Ch 9 §1*) — Kubernetes states the network model's requirements; a plugin implements it.

- CSI (*Ch 11 §5*) — a published interface a storage vendor writes a driver against.
- CRDs (*Ch 6 §8*) — a mechanism for defining new object kinds, with your controller supplying the behavior.
- **A is wrong** and misses the entire point: these are not four unrelated configuration surfaces, they are four instances of one architectural decision.
- **B is wrong**. No forking is involved, which is precisely what a published interface exists to avoid. The whole benefit is that implementations are independent and interchangeable.
- **C is wrong**. Admission webhooks are a real extension point (*Ch 8 §2*), but they intercept API requests. They do not run containers, wire networks, mount volumes, or define object kinds.

2. A. The aggregation layer "allows Kubernetes to be extended with additional APIs, beyond what is offered by the core Kubernetes APIs," by registering an `APIService` object that "claims" a URL path, after which the layer proxies anything sent to that path to the registered service [source: [k8s-docs-api-aggregation-and-device-plugins-2026-08-31](#)]. The team already has the server; aggregation is how it gets reached.

- **B is the trap, and a good one.** A CRD also adds an API, but it works by having the kube-apiserver store and serve the objects itself. That is more service than this team wants, and it would strand the server they already run. The documentation flags the distinction directly: "The aggregation layer is different from Custom Resource Definitions."
- **C is wrong**. Device plugins advertise hardware resources to the kubelet. Different problem, different layer.
- **D is wrong**. A validating or mutating webhook can approve, reject, or modify objects; it cannot serve an API.

3. B. NetworkPolicy is an allow-list for connectivity. It decides which connections are permitted, using selectors (*Ch 10 §6*), and encryption is entirely outside its remit (*Ch 10 §7*).

- **A is wrong** and is the single most common misreading of NetworkPolicy: it permits and denies; it does not protect what it permits.
- **C is wrong**, and it is the misconception this question exists to catch. TLS *terminated* at the Ingress means the encrypted connection ended there. Whatever continues to the Pod is a new, separate connection, and by default it is plaintext.
- **D is wrong** on the requirement. Nothing in the Kubernetes network model requires encryption of pod-to-pod traffic, and nothing in the described setup provides it.

This is exactly the gap §5 is about. A service mesh's mTLS is what closes it.

4. C. Istio's own framing: the mesh consists of "a data plane (proxies that mediate and control all network communication between services) and a control plane (which manages and configures the proxies)" [source: [istio-service-mesh-2026-08-23](#)].

- **A inverts them.** The proxies carry traffic; the control plane configures.
- **B is the vocabulary collision this section exists to defuse.** kube-apiserver and etcd are the *cluster's* control plane (*Ch 3 §2*), a different structure at a different layer. A mesh's control plane distributes policy to proxies; the cluster's control plane reconciles Kubernetes objects.
- **D is wrong.** They are distinct components with distinct jobs.

5. A. In ambient mode "Istio implements its features using a per-node Layer 4 (L4) proxy, and optionally a per-namespace Layer 7 (L7) proxy," and the waypoint proxy "is a deployment of the Envoy proxy; the same engine that Istio uses for its sidecar data plane mode" [source: istio-ambient-mode-2026-08-31].

- **B is wrong** on the key detail, and it is the near-miss worth understanding. Ambient does add a purpose-built proxy, ztunnel, at L4, per node, but the L7 waypoint is Envoy. Ambient removes *sidecars*, not Envoy.
- **C is wrong** — ambient mode is an Istio feature, documented by the Istio project.
- **D is wrong.** The documentation states directly that "Pods and workloads using sidecar mode can co-exist within the same mesh as pods that use ambient mode."

How'd You Do?

4–5 correct: You are holding the chapter's hardest material. §9 will land.

2–3 correct: Review your misses and continue. §6 and §7 do not build on §4 or §5.

0–1 correct: Re-read **§4** in full before continuing. It is the section the rest of the book points at, and §9 is unreadable without it. If question 1 specifically was the miss, go further back first: **Chapter 2 §8** and **Chapter 11 §5**, in that order, and then §4.

Checkpoint: You've Now Mastered

✓ The four pluggable interfaces as one shape, and the documentation's wider extension map beside it ✓ CRDs versus API aggregation — two routes to a new API, with different costs ✓ What a service mesh is, and the property that defines it ✓ Data plane, mesh control plane, cluster control plane — three things, two names

Voyage Progress: 🏠 → 🌊 → ⚓ — you are in the passage. Four sections behind, four ahead, and the two hardest are done.

..11 §6 — Code Without a Server to Put It On

"Serverless" is the worst-named idea in this book, and correcting the name is most of what this section does.

What serverless actually means

Serverless Computing abstracts servers away from the user.

[source: cncf-glossary-serverless-2026-08-31]

Abstracts away from the user. Not eliminates. The servers are still there; somebody is still running them; you are simply not the one thinking about them.

The glossary gives three more defining properties [source: cncf-glossary-serverless-2026-08-31]:

- "Charges are based on a pay-per-use model."
- "Scaling and resource provisioning for computing, storage, or networking are automatically adjusted based on application demand without user intervention."
- "A serverless platform provider consolidates resources to serve multiple users on a single physical machine, ensuring isolation through virtualization."

And it names the problem this solves in terms that should sound familiar by now: without it, "users commit to a predefined capacity, resulting in charges for continuous server availability regardless of actual use," and "responsibility for adjusting server capacity to meet fluctuating demands falls on the user, maintaining active infrastructure even during idle periods" [source: cncf-glossary-serverless-2026-08-31].

The serverless answer is "activating services solely upon demand" [source: cncf-glossary-serverless-2026-08-31].

Knative, and the correction that matters

Knative is "a Kubernetes-based platform that provides a complete set of middleware components for building, deploying, and managing modern serverless workloads," and it is a CNCF Graduated project [source: knative-overview-2026-08-23].

It has three components, and they answer genuinely different questions.

Knative Serving is "an HTTP-triggered autoscaling container runtime that manages the complete lifecycle of stateless HTTP services, including deployment, routing, and automatic scaling (including scale to zero)" [source: knative-overview-2026-08-23]. Synchronous. A request arrives, something serves it.

Knative Eventing is "a CloudEvents-over-HTTP asynchronous routing layer that provides infrastructure for consuming and producing events, enabling loose coupling between event producers and consumers" [source: knative-overview-2026-08-23]. **CloudEvents** is itself a CNCF project: a specification for describing event data in a common way, so that an event emitted by one system can be understood by another without a bespoke adapter between them. Asynchronous. Something happened; interested parties get told.

Knative Functions "leverages Serving and Eventing to provide a simplified experience for building and deploying stateless functions" [source: knative-overview-2026-08-23]. It is built on the other two rather than being a third independent thing.

📌 **Snag:** Serving and Eventing answer different questions and a candidate who can only say "Knative does serverless" will lose the item. **Serving: synchronous HTTP, autoscaling, scale to zero.** **Eventing: asynchronous event routing over CloudEvents.** One sentence each, and keep them apart.

Now the correction this whole section exists for.

☆ Fixed Point

Serverless workloads on Kubernetes are still **containers in Pods**. Knative "builds on the Kubernetes Pod abstraction," and Serving and Eventing "are implemented as Kubernetes Custom Resource Definitions (CRDs)" [source: knative-overview-2026-08-23].

The serverless property is the **lifecycle** — driven by requests, scaled to zero when idle — not the absence of containers or servers.

Read that alongside the glossary's "abstracts servers away from the user" and the misconception dissolves completely. Nothing disappeared. A container image still gets pulled; a Pod still gets scheduled onto a node; a kubelet still starts it. What changed is that none of that happens until a request arrives, and it all goes away again when the requests stop.

🔍 **Closer Look:** Notice what Knative is *built out of*. It is implemented as CRDs, the fourth pluggable interface, two sections ago. A whole serverless platform, sitting on the extension mechanism, with the API server storing and serving its objects. [cross-bearing: see Ch 6 §8 — the control loop, extended]. This is the pattern from §4 doing real work rather than being an example.

Scale to zero

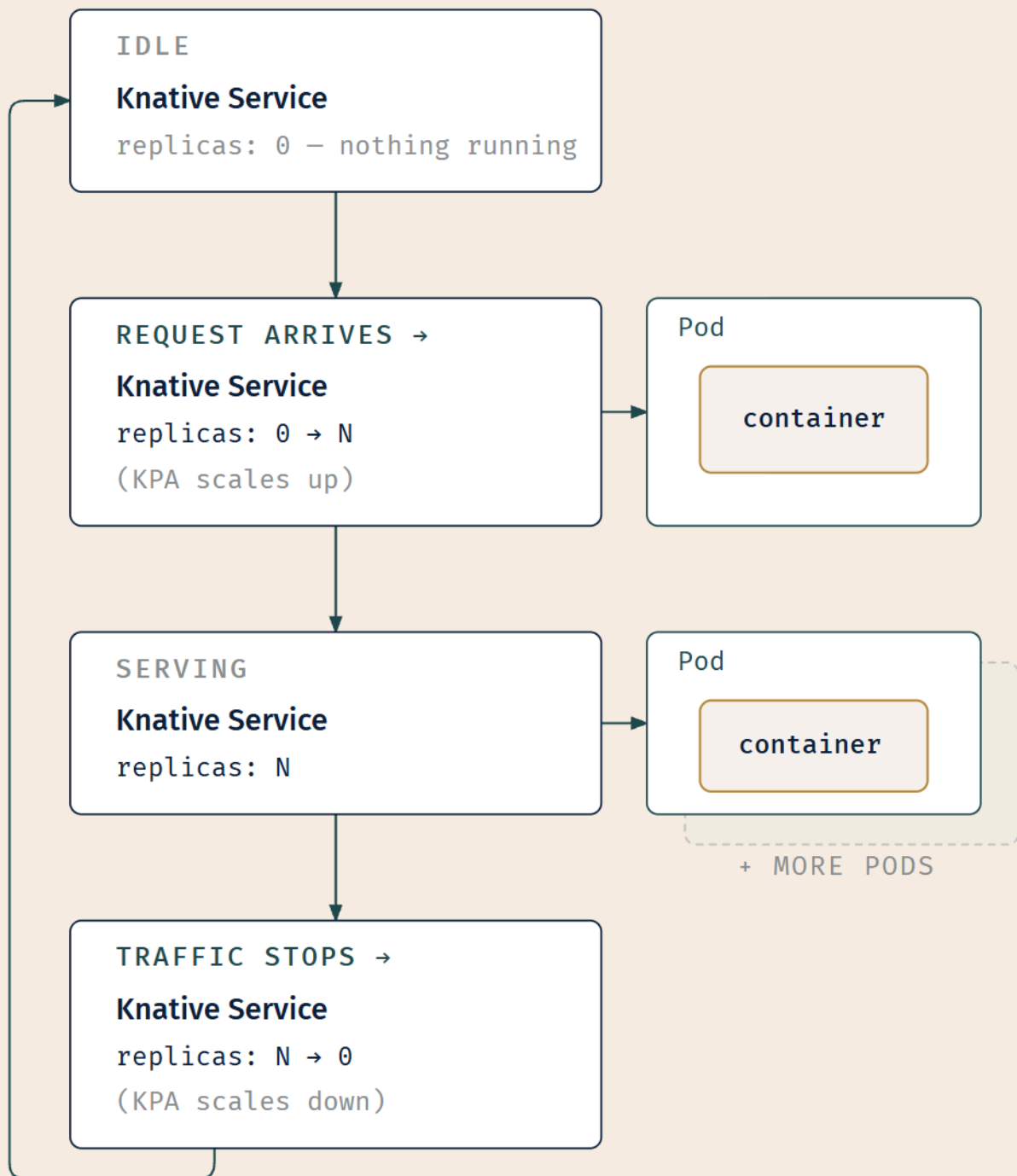
Knative Serving "provides automatic scaling, or *autoscaling*, for applications to match incoming demand," and this is provided by default "by using the Knative Pod Autoscaler (KPA)" [source: knative-serving-autoscaling-2026-08-31].

And the headline behavior:

If an application is receiving no traffic and scale to zero is enabled, Knative Serving scales the application down to zero replicas.

[source: knative-serving-autoscaling-2026-08-31]

Zero. Not one idle replica riding at anchor, burning memory in case somebody shows up. None.



The containers and Pods are real at every populated step – "serverless" describes the LIFECYCLE, not their absence.

A request arrives at an idle Knative Service; replicas go from zero to N; traffic stops; replicas return to zero. Containers in Pods are visible at both ends of the cycle – deliberately, because that is the section's Fixed

Point.

A **Knative Service** relates to the Deployment you already know the way a specialist relates to a generalist. A Deployment holds a replica count you or a controller sets (*Ch 6 §1*). A Knative Service holds the same underlying idea, Pods running your container, with an autoscaler that will take it all the way to zero and bring it back on a request. *[cross-bearing: see Ch 6 §1 — the resource that holds the intent]*, and *[cross-bearing: see Ch 5 §1 — the Pod as the unit of scheduling]* for the abstraction Knative is built on.

🔒 **Worth Securing:** Always write **Knative Service** in full. Chapter 9 §2 owns the Kubernetes Service, the stable virtual address with a selector, and the two are a completely different kind of object that happen to share a word. A sentence with a bare "Service" in a Knative context is a sentence somebody will misread.

Knative Serving can also be configured to use the Kubernetes HorizontalPodAutoscaler instead of the default KPA [source: knative-serving-autoscaling-2026-08-31], which is a neat bridge into the next section, where the whole autoscaling landscape gets laid out. *[cross-bearing: see Ch 17 §7 — four things that scale]*.

..1 §7 — Four Things That Scale

Chapter 6 gave you the HorizontalPodAutoscaler in one sentence and promised the landscape here. Chapter 7 planted an unschedulable Pod and said something could be watching for exactly that. Chapter 13 named metrics-server as what an HPA reads. Chapters 3 and 10 both told you the VPA is not shipped by default and pointed here.

All four of those debts come due in this section.

Two axes, then a third

Start with the distinction the documentation itself leads with, because everything else hangs off it.

☆ Fixed Point

Horizontal scaling changes the **number** of replicas. **Vertical** scaling changes the **resources** available to each replica.

"Horizontal scaling means that the response to increased load is to deploy more Pods. This is different from *vertical* scaling, which for Kubernetes would mean assigning more resources (for example: memory or CPU) to the Pods that are already running for the workload" [source: k8s-docs-hpa-2026-08-24].

More of them, or bigger ones. And then there is a third axis that is not about Pods at all: **node autoscaling** adds or removes the machines underneath.

The four autoscalers

HorizontalPodAutoscaler (HPA). Ships with Kubernetes. Implemented "as a Kubernetes API resource and a controller," where the controller "periodically adjusts the desired scale of its target (for example, a Deployment) to match observed metrics such as average CPU utilization, average memory utilization, or any other custom metric you specify" [source: k8s-docs-hpa-2026-08-24]. Moves the **replica count**, on **observed utilization**.

Two details worth carrying. The HPA runs as a control loop "that runs intermittently (it is not a continuous process)" [source: k8s-docs-hpa-2026-08-24], so a change in load does not produce an instant change in replicas. And it "does not apply to objects that can't be scaled (for example: a DaemonSet)" [source: k8s-docs-hpa-2026-08-24], which makes sense, because a DaemonSet's replica count is a function of the node count, not something you set.

VerticalPodAutoscaler (VPA). Does *not* ship with Kubernetes.

⚠ Navigational Hazards

The VPA is an add-on. It is not there unless somebody installed it.

The documentation says it twice, on two different pages. The autoscaling concepts page: unlike the HPA, the VPA "doesn't come with Kubernetes by default," and is an add-on you or a cluster administrator may need to deploy before you can use it. The VPA's own page: "Unlike HorizontalPodAutoscaler, which is part of the core Kubernetes API, VPA must be installed separately in your cluster" [source: k8s-docs-autoscaling-and-vpa-2026-08-31].

You have met this shape before, and you have met it under a name. Chapter 3 gave you the sentence and Chapter 10 §3 named it as a pattern: **an object without its component does nothing**. An Ingress with no Ingress controller is a document nobody reads. A `kubect1 top` with no metrics-server is a command with nothing to query. A VPA object on a cluster with no VPA installed is the same failure one layer over.

[cross-bearing: see Ch 10 §3 — the object is not the implementation].

The VPA also needs metrics-server installed to work at all, and it runs as three cooperating pieces: a recommender that analyzes usage, an updater that acts on the recommendations, and an admission controller webhook that applies them to new or recreated Pods [source: k8s-docs-autoscaling-and-vpa-2026-08-31]. Moves the **per-replica resources**.

Cluster Autoscaler, and **Karpenter** as a second implementation of the same idea. These do not touch Pods. They provision and remove **nodes**.

Automatically provision and consolidate the Nodes in your cluster to adapt to demand and optimize cost.

[source: k8s-docs-node-autoscaling-2026-08-31]

And here is the trigger, which is the second half of a sentence Chapter 7 left hanging:

If there are Pods in a cluster that can't be scheduled on existing Nodes, new Nodes can be automatically added to the cluster — *provisioned* — to accommodate the Pods.

[source: k8s-docs-node-autoscaling-2026-08-31]

Chapter 7 §2 told you an unschedulable Pod sits in Pending, and that its continued existence is "a standing, machine-readable statement that the cluster is short of somewhere to put work. Something could be watching for exactly that."

This is the something. *[cross-bearing: see Ch 7 §2 — what makes a node feasible]*.

The reverse operation is **consolidation**, and it is the half nobody puts on the brochure: the fleet gets smaller when the work does. "Nodes in your cluster can be automatically *consolidated* in order to improve the overall Node utilization, and in turn the cost-effectiveness of the cluster. Consolidation happens through removing a set of underutilized Nodes from the cluster" [source: k8s-docs-node-autoscaling-2026-08-31]. Node autoscalers also "need to interact with cloud provider APIs to provision and consolidate Nodes" [source: k8s-docs-node-autoscaling-2026-08-31]; they are not purely Kubernetes-internal, because buying a machine is not a Kubernetes operation.

Cluster Autoscaler and Karpenter "are the two Node autoscalers currently sponsored by SIG Autoscaling," and "from the perspective of a cluster user, both autoscalers should provide a similar Node autoscaling experience. Both will provision new Nodes for unschedulable Pods, and both will consolidate the Nodes that are no longer optimally utilized" [source: k8s-docs-node-autoscaling-2026-08-31]. Cluster Autoscaler works against pre-configured **node groups**; Karpenter describes itself as "an open-source node lifecycle management project built for Kubernetes" whose job is "to add nodes to handle unschedulable pods, schedule pods on those nodes, and remove the nodes when they are not needed" [source: karpenter-concepts-2026-08-31].

🔒 **Worth Securing:** Karpenter is sponsored by Kubernetes SIG Autoscaling [source: k8s-docs-node-autoscaling-2026-08-31]. It is **not** a CNCF project with a maturity level, and no official source assigns it one. Contrast Knative, whose own documentation states it is a CNCF Graduated project [source: knative-overview-2026-08-23], and KEDA, which kubernetes.io describes the same way [source: k8s-docs-autoscaling-and-vpa-2026-08-31]. Karpenter's documentation makes no such claim. If an answer option gives Karpenter a CNCF maturity level, be suspicious of it, and notice that §2 just taught you why that distinction is meaningful.

KEDA. Kubernetes Event-Driven Autoscaling, "a CNCF-graduated project enabling you to scale your workloads based on the number of events to be processed, for example the amount of messages in a queue. There exists a wide range of adapters for different event sources to choose from" [source: k8s-docs-autoscaling-and-vpa-2026-08-31].

KEDA also handles **schedule-based** scaling, through its Cron scaler, which "allows you to define schedules (and time zones) for scaling your workloads in or out" [source: k8s-docs-autoscaling-and-vpa-2026-08-31].

31], for reducing consumption during off-peak hours.

Four autoscalers, three axes — and the overlap is the lesson

The section is titled "Four Things That Scale," and an attentive reader will already have noticed that four autoscalers do not produce four axes. KEDA moves the **replica count**, same as the HPA.

That is not a flaw in the taxonomy. It is the most useful thing in the section.

AUTOSCALER COMPARISON • AXIS VS. TRIGGER

Four Autoscalers, Two Questions

AUTOSCALER	WHAT MOVES	WHAT TRIGGERS IT
HPA SHIPS WITH KUBERNETES	replica count	observed utilization
KEDA	replica count same axis as HPA	external EVENT (queue depth, schedule via cron)
VPA ADD-ON • NOT SHIPPED	per-replica resources (CPU, memory)	observed usage (needs metrics-server)
Cluster Autoscaler / Karpenter	the node pool	unschedulable Pods → provision underutilized nodes → consolidate

- VPA is the one that is not there by default.
- KEDA shares HPA's axis with a different trigger — axis and trigger are two separate questions.

LEGEND

☐ Ships with Kubernetes (built-in)

☐ Add-on • not shipped by default

Four autoscalers, three axes. The two marked cells carry most of this section's exam value.

🔊 **Mnemonic:** Ask two questions of every autoscaler, never one. **What moves? What triggers it?** HPA and KEDA give the same answer to the first and different answers to the second. Cluster Autoscaler and Karpenter give the same answer to both. Once you separate the two questions, the taxonomy stops being four things to memorize and becomes a small grid you can reconstruct.

Two more things sit at the edges of this landscape, named for completeness and not taught: the **Cluster Proportional Autoscaler**, which scales replica counts based on the number of schedulable nodes and cores (the classic use is cluster DNS), and its vertical counterpart [source: k8s-docs-autoscaling-2026-08-23]. Neither is graded material here.

The in-place resize question, stated to the conflict

There is a moving target in this material and you should see it as a moving target rather than as a fact.

In-place Pod vertical scaling lets you change a running Pod's CPU and memory requests and limits without recreating it. It went stable in Kubernetes v1.35, "more than 6 years after its initial conception," having been alpha in v1.27 and beta in v1.33 [source: k8s-docs-autoscaling-and-vpa-2026-08-31].

The question the exam might reach for is whether the VPA can use it. And here the official sources do not agree with each other.

The autoscaling concepts page states flatly: "As of Kubernetes 1.37, VPA does not support resizing pods in-place, but this integration is being worked on" [source: k8s-docs-autoscaling-and-vpa-2026-08-31]. But the VPA's own documentation page lists `InPlaceOrRecreate` and `InPlace` among its update modes, and the v1.35 release blog says VPA's "InPlaceOrRecreate update mode, which leverages this feature, has graduated to beta" [source: k8s-docs-autoscaling-and-vpa-2026-08-31].

📌 **Snag:** The safe statement, and the one this book will stand behind: **in-place Pod vertical resize is a stable Kubernetes feature, and full VPA support for it is not a settled story.** Do not write or believe "VPA now resizes in place" as an unqualified claim. If an exam item hinges on this, it will hinge on the durable half: that horizontal and vertical are different axes, and that VPA is an add-on.

The three debts, paid

Worth naming explicitly, because each was a pointer laid chapters ago:

metrics-server is what an HPA reads. The resource metrics pipeline exists so that "the HorizontalPodAutoscaler (HPA) and VerticalPodAutoscaler (VPA) use data from the metrics API to adjust workload replicas and resources to meet customer demand" [source: k8s-docs-resource-metrics-pipeline-2026-08-31]. `metrics-server` is a "cluster addon component" and "a reference implementation of the Metrics API" [source: k8s-docs-resource-metrics-pipeline-2026-08-31]. No Metrics API server, whether `metrics-server` or something equivalent serving that API, no HPA scaling. [cross-bearing: see Ch 13 §7 — *numbers nobody collects by default*].

An unschedulable Pod is what a node autoscaler watches for. Answered above; the sentence Chapter 7 left open is closed.

The VPA is the absent-component pattern by name. Not a new lesson, the same lesson, one layer over. *[cross-bearing: see Ch 10 §3 — the object is not the implementation].*

And a fourth connection that costs nothing: an HPA reasons about utilization *relative to what a Pod requested*, which is why Chapter 5's requests-and-limits material is load-bearing here and not merely adjacent. *[cross-bearing: see Ch 5 §8 — what a Pod is owed]*, and *[cross-bearing: see Ch 18 §3 — utilization relative to requests]* for where that gets examined properly.

[cross-bearing: see Ch 6 §2 — a loop you can watch working] for the HPA concept as you first met it.

.. §8 — How the Project Actually Runs, and How You'd Join

This is the section Chapter 1 warned you about, the one technically strong candidates skip because it looks like soft content. It is also, per point of exam weight, some of the cheapest material in the book. What follows is an org chart and an invitation, and the invitation is the more interesting half.

The four principles

Kubernetes states four community principles, and they are short enough to quote whole [source: k8s-community-governance-2026-08-23]:

- **Open** — Kubernetes is open source.
- **Welcoming and respectful** — see the Code of Conduct.
- **Transparent and accessible** — work and collaboration should be done in public.
- **Merit** — ideas and contributions are accepted according to their technical merit and alignment with project objectives, scope, and design principles.

Hold "transparent and accessible" in mind. In a moment you will meet the one group that is deliberately exempt from it, and that exemption is the section's sharpest exam item.

SIGs, Working Groups, and Committees

☆ Fixed Point

- A **SIG (Special Interest Group)** is the primary, **durable**, topic-focused unit. Members from multiple companies, common purpose of advancing the project on a specific topic.
- A **Working Group** is **time-bounded** and **crosses SIG lines**. Formed for a topic in scope for Kubernetes that spans multiple SIGs.
- A **Committee** does **not have open membership** and does not always operate in the open.

[source: k8s-community-governance-2026-08-23], [source: k8s-sig-list-and-groups-2026-08-31]

The project's framing: "Most community activity is organized into Special Interest Groups (SIGs) and time bounded Working Groups" [source: k8s-sig-list-and-groups-2026-08-31].

SIGs come in three orientations, and the examples make the taxonomy concrete: **vertical** (Network, Storage, Node), **horizontal** (Scalability, Architecture), or **project-support oriented** (Testing, Release, Docs) [source: k8s-community-governance-2026-08-23]. Each SIG "must have at least one and ideally two SIG chairs at any given time," who are "organizers and facilitators, responsible for the operation of the SIG and for communication and coordination with the other SIGs and the rest of the project" [source: k8s-community-governance-2026-08-23].

Within a SIG, "specific work is divided into **subprojects**, each with designated owners who serve as technical leaders for their respective areas" [source: k8s-community-governance-2026-08-23]. SIG Release, for instance, has a Release Engineering subproject "dedicated to the technical aspects of Kubernetes releases, for example its tooling and source code ownership" [source: k8s-release-cycle-and-cadence-2026-08-31].

Working groups "are primarily used to facilitate topics of discussion that are in scope for Kubernetes but that cross SIG lines. They are short-lived or address issues spanning multiple SIGs" [source: k8s-community-governance-2026-08-23].

And then Committees.

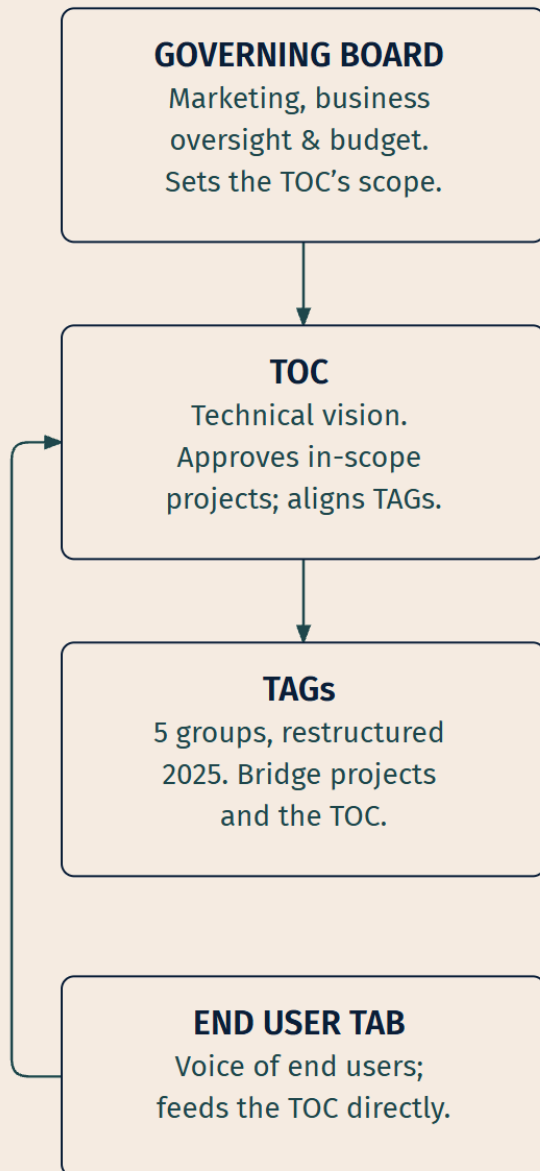
⚠ Navigational Hazards

Committees are the one community group that is not open, and that asymmetry is the whole item.

"Committees do not have open membership and do not always operate in the open. They are formed by the steering committee for specific topics requiring discretion (for example Security, Code of Conduct), have charters and chairs, and report periodically to the steering committee" [source: k8s-community-governance-2026-08-23].

A project whose stated principle is *transparent and accessible* has three closed bodies, and the reason is not hypocrisy. You cannot handle an unreported security vulnerability or a code-of-conduct complaint in a public meeting. Discretion is the requirement, and the exception is deliberate and chartered.

There are exactly **three** Committees: **Code of Conduct**, **Security Response**, and **Steering** [source: k8s-sig-list-and-groups-2026-08-31]. Steering is one of them, and it also holds overall project governance; it is Steering that charters the other two [source: k8s-community-governance-2026-08-23].

CNCF
THE FOUNDATION**KUBERNETES**
ONE CNCF PROJECT

*TAGs are CNCF-wide. SIGs are Kubernetes-internal.
Different organizations, different scopes.*

Two governance structures side by side. The pairing is the point — most of the confusion in this material comes from meeting them separately.

🔗 **Closer Look:** CNCF **TAGs** and Kubernetes **SIGs** are easy to confuse, and there is a historical reason that is more useful than any warning. The CNCF's own groups were *originally called SIGs*: "By June 2019, this number had grown to 37 projects and the TOC approved the creation of SIGs, later to be renamed Technical Advisory Groups" [source: cncf-tags-current-structure-2026-08-31]. They were the same word once, at two scales, and CNCF renamed theirs. TAGs operate across the whole foundation. SIGs operate inside the Kubernetes project. One foundation, many projects; Kubernetes is one of them.

A detail that makes the SIG list concrete rather than abstract: several SIGs are ones you have already met the work of. **SIG Network**, **SIG Storage**, **SIG Node**, **SIG Autoscaling** and **SIG Release** are all on the roster [source: k8s-sig-list-and-groups-2026-08-31], and mapping them onto this book's chapters is straightforward: Network to Chapters 9 and 10, Storage to Chapter 11, Node to much of Chapters 2 and 5, Release to the versions Chapter 8 taught you to reason about. SIG Autoscaling sponsors both of the node autoscalers from the section you just read [source: k8s-docs-node-autoscaling-2026-08-31]. Every interface in §4 has a group of people behind it, working in public.

The release train, and the fact you already half-know

Here is the section's quietest payoff, and possibly the chapter's.

Kubernetes releases happen approximately three times per year [source: k8s-release-cycle-and-cadence-2026-08-31].

The project maintains release branches for the most recent three minor releases [source: k8s-release-cycle-and-cadence-2026-08-31].

Look at those two numbers together. Three releases a year. Three maintained branches. Which means a release stays supported for roughly — do the arithmetic yourself before reading on.

Roughly a year. And the source confirms it independently, on the same page: "Kubernetes 1.19 and newer receive approximately 1 year of patch support" [source: k8s-release-cycle-and-cadence-2026-08-31].

☆ Fixed Point

Three minor releases a year, three supported minor versions, and roughly one year of patch support are not three facts. They are one fact stated three ways. Three releases per year × three maintained branches ≈ one year — and because the documentation states that year independently, you can *derive* the support window instead of memorizing it.

Chapter 8 §6 warned you that the version-skew numbers were the most forgettable material in the book, and it was right: they are three unrelated-looking integers. They are much harder to forget once they are

one relationship. *[cross-bearing: see Ch 8 §6 — versions that are allowed to disagree]*, and *[cross-bearing: see Ch 13 §6 — versions that don't agree]* for what happens when the window is violated.

SIG Release is the group that makes this happen — and this is where Chapter 8's fifteen weeks go *[cross-bearing: see Ch 8 §6 — release cadence and supported versions]*. Three minor releases a year, roughly one every fifteen weeks, is not an arbitrary rhythm: it is the length of a cycle that has to accommodate enhancement freeze, code freeze, testing and the staffing of the release roles below. Its charter lists "Production of Kubernetes releases on a reliable schedule" as its first responsibility, along with defining and staffing release roles, driving the development and release processes, and "managing the creation of release specific artifacts, including: Code branches, Binary artifacts, Container Images, Release notes" [source: k8s-release-cycle-and-cadence-2026-08-31].

The cycle itself has three phases — **Enhancement Definition, Implementation, Stabilization** — with an **Enhancements Freeze** around week 4, **Code Freeze** starting around week 12 and running about two weeks, during which "only critical bug fixes are accepted into the release codebase," and a post-release phase from week 14 [source: k8s-release-cycle-and-cadence-2026-08-31].

KEPs: how a change becomes a change

A **Kubernetes Enhancement Proposal** is "a way to propose, communicate and coordinate on new efforts for the Kubernetes project," using "a standard proposal format with useful metadata" [source: k8s-keps-and-feature-stages-2026-08-23].

A KEP is required for potentially controversial changes, most new features, major modifications to existing features, and changes that affect most of the project. The framework was "inspired by similar processes such as IETF RFCs and Python PEPs," and it provides "exposure through searchable websites, cross-referencing, and structured decision-making with a discoverable record" [source: k8s-keps-and-feature-stages-2026-08-23].

KEPs track a feature through **alpha → beta → stable (GA)**, with graduation criteria stated in the KEP itself. You have already seen this machinery from the outside: every "Feature state: Stable since Kubernetes v1.35" banner in the documentation is the visible end of a KEP that ran its course.

🔒 **Worth Securing:** The discoverable record is the real product. When you find yourself asking "why on earth does Kubernetes do it *this* way," there is very often a KEP with the argument written down, including the alternatives that were rejected and why. That is an unusually good property for a system you have to reason about under exam conditions, and an unusually good one for a system you have to operate.

How you'd actually join

This is not a metaphor. There is a published chart of the passage, and the first leg is shorter than it looks.

The Kubernetes contributor ladder has four rungs [source: k8s-community-membership-ladder-2026-08-23]:

Role	Responsibility	What it requires
Member	Active contributor in the community	Sponsored by 2 reviewers; multiple contributions to the project
Reviewer	Review contributions from other members	History of review and authorship in a subproject
Approver	Approve acceptance of contributions	Highly experienced active reviewer and contributor to a subproject
Subproject Owner	Set direction and priorities for a subproject	Demonstrated responsibility and excellent technical judgement

The Member requirements in full: two-factor authentication enabled on your GitHub account; multiple contributions "enough to demonstrate an ongoing and long-term commitment to the project"; subscribed to the dev mailing list; read the contributor guide; **sponsored by 2 reviewers from different companies**; and open a membership request issue [source: k8s-community-membership-ladder-2026-08-23].

Reviewer requires being a member for at least three months, primary reviewer on at least 5 PRs, and having reviewed or merged at least 20 substantial PRs. Approver requires three months as a reviewer, primary reviewer on at least 10 substantial PRs, 30 reviewed or merged, and nomination by a subproject owner [source: k8s-community-membership-ladder-2026-08-23].

Notice what is and is not on that list. There is no employer requirement. No seniority requirement. No credential. The gate is *contributions and two people willing to vouch for you*, and the sponsors have to be from different companies: a deliberate structural check against any one employer manufacturing members.

Logbook Entry: The most common misconception about contributing to a project this size is that the work is all deep systems programming, and that a first contribution has to be impressive.

It does not. SIG Docs exists. So does SIG Contributor Experience [source: k8s-sig-list-and-groups-2026-08-31]. In my experience, a great many people on the contributor ladder got their first merged PR by fixing documentation that was wrong, or by writing a test for a code path that had none, or by reproducing a bug report that a maintainer did not have the environment to reproduce.

The reason this matters for an exam chapter, and not just as encouragement: the project's own stated principle is that "work and collaboration should be done in public" [source: k8s-community-governance-2026-08-23], and the fastest way to stop finding "SIG, Working Group, Committee, Steering" abstract is to go and watch some of that work happen. Forty-five minutes spent following a group of engineers arguing about a KEP will do more for your retention of this section than reading it three times.

The CNCF side has its own on-ramps, and they are distinct from the Kubernetes ones. **LFX Mentorship** is "a mentoring initiative by the Linux Foundation"; **Google Summer of Code** is "a mentoring program for

the open source beginners"; **Outreachy** is "a mentoring initiative for the communities traditionally underrepresented in tech" [source: cncf-mentoring-and-community-groups-2026-08-31]. The foundation also "supports the worldwide community of the Cloud Native Community Groups (CNCGs)" [source: cncf-mentoring-and-community-groups-2026-08-31]: "free, volunteer-run meetups on the CNCF community platform, including Kubernetes Community Days" [source: cncf-landscape-and-community-2026-08-23].

CNCF Ambassadors are an extension of the CNCF, furthering the mission of making cloud native ubiquitous through community leadership and mentorship; many of them organize those local groups [source: cncf-landscape-and-community-2026-08-23].

And **KubeCon + CloudNativeCon** is the flagship conference series where the whole community convenes [source: cncf-landscape-and-community-2026-08-23].

The Code of Conduct

The **CNCF Community Code of Conduct** applies across all CNCF projects and events, and its scope statement is the examinable part because it is broader than people assume:

This code of conduct applies: within project and community spaces, in other spaces when an individual CNCF community participant's words or actions are directed at or are about a CNCF project, the CNCF community, or another CNCF community participant in the context of a CNCF activity.

[source: cncf-code-of-conduct-2026-08-31]

Project spaces, event spaces, *and* conduct outside both when it is directed at the community. The pledge commits to "making participation in the CNCF community a harassment-free experience for everyone," across a long enumerated list of dimensions of diversity [source: cncf-code-of-conduct-2026-08-31].

It is administered by the **CNCF Code of Conduct Committee**, reachable at a published address for incidents that are project-agnostic or span multiple projects, with a stated expectation of "a response within three business days" [source: cncf-code-of-conduct-2026-08-31].

The certification ladder

Chapter 1 deferred this here by name, and it is a short answer.

The KCNA "is a pre-professional certification designed for candidates interested in advancing to the professional level," and it "is intended to prepare candidates to work with cloud native technologies and pursue further CNCF credentials, including CKA, CKAD, and CKS" — the Certified Kubernetes Administrator, the Certified Kubernetes Application Developer, and the Certified Kubernetes Security Specialist [source: cncf-kcna-certification-page-2026-08-23]. The certification overview page puts it more directly: KCNA "lays the groundwork for further CNCF certifications like CKA, CKAD, and CKS" [source: cncf-kcna-certification-page-2026-08-23].

The shape of that ladder matters because the *format* changes, not just the difficulty. KCNA is "online and multiple-choice." **CKA** is "a performance-based exam where candidates interact with the command line to solve real-world challenges." **CKAD** is "a hands-on, command-line environment." **CKS** is "performance-based" [source: cncf-kcna-certification-page-2026-08-23].

CNCF also offers **KCSA**, the Kubernetes and Cloud Native Security Associate, and the Cloud Native Network Function certification [source: cncf-who-we-are-2026-08-23].

🕒 **Mnemonic: KCNA is the only one on this ladder — CKA, CKAD, CKS — you can pass by knowing things.** Everything above it requires doing things, at a terminal, against a live cluster, under time pressure. That is not a reason to underrate this exam. It is the reason the vocabulary you are building here has to be solid before you go on, because at the next tier nobody gives you four options to pick from.

🕒 Taking Your Bearings: Scaling, Serverless, and How the Project Runs

Five questions. At least two are about the material a strong technical reader skims, which is deliberate.

1. A team runs a workload whose load is driven by messages arriving in a queue, not by CPU utilization. They also have a nightly batch window where they want capacity increased on a schedule regardless of current load. Which autoscaler addresses both needs, and what axis does it move?

A) The VerticalPodAutoscaler, moving per-replica CPU and memory in response to observed usage B) Cluster Autoscaler, moving the node pool in response to the queue C) KEDA, moving the replica count on external events and on schedules via its Cron scaler D) The HorizontalPodAutoscaler, moving the replica count natively on queue depth

2. Which statement about the VerticalPodAutoscaler is accurate?

A) It ships with Kubernetes and is enabled by default on all clusters B) It adjusts the number of replicas of a workload in response to memory pressure C) It provisions new nodes when Pods cannot be scheduled on the existing ones D) It is an add-on that must be installed separately, and it requires metrics-server

3. What distinguishes a Kubernetes **Committee** from a SIG and a Working Group?

A) It does not have open membership and does not always operate in the open; Steering forms it for topics requiring discretion B) It is scoped to a single technical topic, where SIGs deliberately span several C) It is longer-lived than a SIG and outlasts any individual Working Group D) It is a CNCF body, where SIGs and Working Groups are Kubernetes bodies

4. [retrieval: ch8] A cluster runs Kubernetes 1.35. Two newer minor versions have since been released. Using the project's release cadence and support policy, what can you say about 1.35's patch support?

A) Three years of support remain, since the project maintains three years of release branches B) It is at or very near the end of its window — three branches are maintained, releases come about three times a year, and 1.19 and newer get about a year of patch support C) It is already out of support, because only the current release is maintained D) It depends entirely on the cloud provider; the upstream project makes no commitment

5. A developer says: "We moved to serverless, so we're not running containers any more." Correct them, with reference to how Knative works.

A) They are right — serverless workloads run as functions outside the container model B) They are right about Knative Functions but wrong about Knative Serving C) They are wrong — Knative replaces Kubernetes with its own runtime, which is not container-based D) They are wrong — Knative builds on the Pod abstraction and ships as CRDs, so the workloads are still containers in Pods

Answers with Explanations:

1. **C.** KEDA scales "based on the number of events to be processed, for example the amount of messages in a queue," and its Cron scaler "allows you to define schedules (and time zones) for scaling your workloads in or out" [source: k8s-docs-autoscaling-and-vpa-2026-08-31]. Both requirements, one tool. It moves the replica count, the same axis as the HPA, with a different trigger.

- **A is wrong** on both halves: VPA moves resources, not replicas, and it responds to observed usage, not queue depth or schedules.
- **B is wrong.** Cluster Autoscaler operates on nodes, reacting to unschedulable Pods. It has no view of a queue.
- **D is the near-miss worth understanding.** The HPA scales on "observed resource utilization such as CPU or memory usage" or custom metrics [source: k8s-docs-hpa-2026-08-24]. Getting queue depth in front of an HPA is possible through custom metrics plumbing, but it is not native, and it does not address the scheduling requirement at all.

2. **D.** Unlike the HPA, the VPA does not come with Kubernetes by default; it is an add-on a cluster administrator must deploy, and "You will need to have the Metrics Server installed to your cluster for the VPA to work" [source: k8s-docs-autoscaling-and-vpa-2026-08-31].

- **A is the misconception this section exists to prevent.** VPA is the standing example of the pattern Chapter 10 §3 named: the object can exist while nothing acts on it.
- **B describes the HPA's axis, not the VPA's.** Vertical means resources per replica.
- **C describes a node autoscaler.**

3. **A.** Committees "do not have open membership and do not always operate in the open. They are formed by the steering committee for specific topics requiring discretion (for example Security, Code of Conduct)" [source: k8s-community-governance-2026-08-23].

- **B inverts the SIG/WG relationship.** SIGs are the topic-scoped unit; Working Groups are the ones that cross SIG lines.
- **C is wrong** — longevity is not the distinction. SIGs are the durable unit; Working Groups are the time-bounded one. Committees are distinguished by closed membership.
- **D confuses the two organizations.** All three, SIGs and Working Groups and Committees, are Kubernetes bodies. CNCF's equivalent coordination units are TAGs.

4. B. The project "maintains release branches for the most recent three minor releases," and "Kubernetes 1.19 and newer receive approximately 1 year of patch support" [source: k8s-release-cycle-and-cadence-2026-08-31]. With approximately three releases per year, being two versions behind puts a release at the end of its supported window.

- **A misreads "three releases" as "three years."** The number three attaches to *minor versions*, not years.
- **C understates it — three branches are maintained, not one.**
- **D is wrong** about the upstream project, which publishes explicit end-of-life dates per release. Managed providers may offer extended support commercially, but that is in addition to the upstream policy, not instead of it.

This is the fact Chapter 8 §6 taught and §8 of this chapter explains: the cadence and the support window are the same fact.

5. D. Knative "builds on the Kubernetes Pod abstraction," and "Serving and Eventing are implemented as Kubernetes Custom Resource Definitions (CRDs)" [source: knative-overview-2026-08-23]. The CNCF glossary's own definition says serverless "abstracts servers away from the user" [source: cncf-glossary-serverless-2026-08-31]: abstracts away, not eliminates.

- **A is the misconception.** The name suggests absence; the architecture is a lifecycle.
- **B is wrong** — Functions is built on Serving and Eventing, so it inherits exactly the same container- and-Pod substrate.
- **C reaches the right verdict for the wrong reason**, which makes it the trap. Knative does not replace Kubernetes; it is Kubernetes-based and is built out of Kubernetes' own extension mechanism.

How'd You Do?

4–5 correct: Done. §9 asks nothing new of you — read it once, slowly.

2–3 correct: Review your misses. If they were questions 3 or 4, review them carefully: those two are §8's, and §8's material is the cheapest on the exam.

0–1 correct: Re-read **§7's autoscaler grid** and **§8's three groups**. Those are the chapter's two highest-density recall blocks and they reward a second pass more than any other pages here. Do it before the Practice Questions, not after.

Checkpoint: You've Now Mastered

✓ Four autoscalers across three axes, and the two questions to ask of each ✓ Serverless as a lifecycle claim, not a claim about the absence of containers ✓ SIG, Working Group, Committee, Steering — and which one is deliberately closed ✓ Why three releases a year and three supported versions are one fact ✓ The contributor ladder, its actual entry requirements, and the certification ladder above this exam

One section left, and it does not teach you anything new.

☀ §9 — One Pluggability Story

Go back to §4 and look at the figure again. Four sockets, at four layers, taught in four chapters, hundreds of pages apart.

Here is what you were actually looking at.

Kubernetes is not a system that happens to be extensible in four places. It is a system built on the premise that **it should not be the one implementing the parts that vary**.

THE SHAPE

**Kubernetes defines
WHAT MUST BE TRUE**

(THE SOCKET)

**Somebody else
SUPPLIES THE THING**

THE EVIDENCE

CRI (Ch 2 §4)

CNI (Ch 9 §1)

CSI (Ch 11 §5)

CRDs (Ch 6 §8)

**Four instances.
Not four decisions.**

*One decision, made four
times because it was
right four times.*

The same vocabulary as the extension-points map, one altitude higher — and altitude, to a navigator, is not a metaphor for importance. It is the angle you take on a fixed body to learn where you actually are.

Consider what it would have taken to do this the other way. Kubernetes ships a container runtime, and then every hardening requirement, every compliance regime, every sandboxed-isolation need becomes a feature request against the Kubernetes codebase. Kubernetes ships a network implementation, and every network fabric on earth becomes a patch somebody has to get merged. Kubernetes ships storage drivers, and the project maintains code for every array, appliance and cloud volume service in existence. Kubernetes ships a fixed set of object kinds, and the only way to model your database is to convince the API reviewers your database belongs in Kubernetes.

None of that scales. Not technically, and not organizationally, which §8 just made concrete, because now you know how a change becomes a change here. Every one of those hypothetical features would be a KEP, argued in a SIG, competing for review time against everything else.

The interface is how a project of this size stays a project rather than becoming a marketplace of forks.

And notice the second-order effect, which is the part that took four chapters to earn. Because the parts that vary are behind interfaces, they can vary *without permission*. A storage vendor writes a CSI driver on their own schedule. A network project ships a CNI plugin without asking. You define a CRD on a Tuesday afternoon and your cluster gains a new kind of object that upstream Kubernetes has never heard of and never needs to.

Chapter 2 §8 told you this would feel like recognition rather than a fourth list. If it did, that is because you were not learning four things. You were learning one thing, four times, in the four places it happened to show up.

[cross-bearing: see Ch 2 §8 — the crate, not the cargo], [cross-bearing: see Ch 6 §9 — nobody sails one pod], [cross-bearing: see Ch 9 §8 — a query with a name], [cross-bearing: see Ch 11 §7 — outliving the pod that asked].

That is the pluggability story, and it is one story.

Exam Alert! 🚨

High-Priority Topics:

1. **The four pluggable interfaces as one shape.** CRI, CNI, CSI, CRDs — and the ability to state what they have in common without being handed the list first. The most reused idea in this book.
2. **The maturity ladder in order, and where the criteria live.** Sandbox → Incubating → Graduated. Criteria in the TOC's project lifecycle documentation, not the projects page. The *levels* are examinable; the *roster* is dated data.
3. **Data plane versus control plane, twice.** A mesh's data plane is the proxies; its control plane configures them; neither is the cluster's control plane. Same vocabulary, different layer.
4. **Which autoscaler moves which axis, and which ones ship.** Replicas (HPA, KEDA), resources (VPA), nodes (Cluster Autoscaler, Karpenter). HPA is built in; VPA is an add-on.

5. **SIG, Working Group, Committee, Steering — and TAG is none of them.** Durable and topic-scoped; cross-SIG and time-bounded; closed-membership; overall governance. TAGs are CNCF-wide; SIGs are Kubernetes-internal.

Common Traps:

The trap	The correct understanding
Reading <i>cloud native</i> as "runs in a public cloud"	The definition's first sentence says public, private and hybrid. The term is about how you build and operate, not where it runs.
Treating the CNCF technology list as exhaustive	The definition says outright that the list is non-exhaustive.
Ordering the maturity levels wrong	Sandbox → Incubating → Graduated. Sandbox is bleeding edge; Incubating is production use by a small number of users; Graduated is stable and widely adopted.
Looking for graduation criteria on the projects page	They live in the TOC's project lifecycle documentation.
Memorizing which projects are Graduated	The roster changes. Learn the levels and what each one asserts.
Confusing the TOC with the Governing Board	The Board handles business oversight and budget and sets the scope; the TOC owns technical vision and approves projects within that scope.
Using the pre-2025 TAG list	TAGs were restructured in 2025. The old list is all over older study material.
Treating CNCF TAGs and Kubernetes SIGs as the same thing	Different organizations at different scopes. (<i>Easy to confuse — they shared a name historically, which is exactly why.</i>)
Confusing a SIG with a Working Group	SIGs are the primary, durable unit for a topic. Working Groups cross SIG lines and are time-bounded.
Assuming every community group is open	Committees are not. They are formed by Steering for topics requiring discretion, and do not always operate in the open.
"A service mesh needs application code changes"	The defining property is delivering security, observability and traffic management without them.
Confusing the mesh's data plane with its control plane — or with the cluster's	Data plane = the proxies mediating service-to-service traffic. Control plane = what configures them. Neither is Chapter 3's control plane.
"Service mesh means sidecars"	Sidecar mode puts an Envoy proxy beside each Pod; ambient mode uses per-node L4 proxies plus optional per-namespace Envoy waypoints. Both use Envoy.
"Knative replaces Kubernetes"	Knative is Kubernetes-based, builds on the Pod abstraction, and is implemented as CRDs.

The trap	The correct understanding
Confusing Knative Serving with Eventing	Serving: HTTP-triggered autoscaling container runtime with scale to zero. Eventing: CloudEvents-over-HTTP asynchronous routing.
"Serverless means no containers"	The workloads are still containers in Pods. Serverless describes the lifecycle. (<i>Easy to confuse — the name actively misleads.</i>)
Confusing horizontal with vertical scaling	Horizontal changes the number of replicas. Vertical changes the resources available to each.
"VPA ships with Kubernetes"	VPA is an add-on. The object can exist while nothing acts on it — the same pattern as kubectl top without metrics-server.
"In-place vertical resize means VPA now works in place"	In-place Pod vertical resize is a stable Kubernetes feature; full VPA support for it is not a settled story. Do not state it as fact.
Confusing Pod autoscaling with node autoscaling	HPA, VPA and KEDA scale workloads . Cluster Autoscaler and Karpenter scale the node pool .
"KEDA is a CPU autoscaler"	KEDA is event-driven — queue depth and similar external signals — plus schedule-based scaling through its Cron scaler.
Giving Karpenter a CNCF maturity level	No official source assigns it one. It is sponsored by Kubernetes SIG Autoscaling.
Expecting Observability as its own domain	The current blueprint has four domains — Kubernetes Fundamentals 44%, Container Orchestration 28%, Cloud Native Application Delivery 16%, Cloud Native Architecture 12% [source: cncf-kcna-certification-page-2026-08-23]. Observability is not one of them; it is competency material inside Cloud Native Architecture. Much third-party prep still targets an older, five-domain split.

Practice Questions

Twenty-one questions. Several deliberately cross domain boundaries. That is how the exam behaves, and it is how you find out whether a concept is anchored or merely adjacent to one you know.

Q1. The CNCF Cloud Native Definition v1.1 states that cloud native is characterized by loosely coupled systems that interoperate in a manner that is:

A) Scalable, portable, automated, distributed, and containerized B) Declarative, immutable, ephemeral, elastic, and stateless C) Secure, resilient, manageable, sustainable, and observable D) Reliable, available, performant, secure, and cost-effective

Q2. An engineering director claims their on-premises platform "cannot be cloud native, because it isn't in a cloud." What does the published definition say?

A) The definition's first sentence names "public, private, hybrid cloud," so an on-premises platform can absolutely be cloud native B) The director is right — cloud native by definition requires a public cloud provider C) The definition is silent on where workloads run, so the question cannot be settled from the document D) On-premises platforms can be cloud native only if they run a conformant Kubernetes distribution

Q3. According to the CNCF definition, what do cloud native techniques allow engineers to do when combined with robust automation?

A) Eliminate the need for operations staff B) Guarantee zero downtime during deployments C) Reduce infrastructure cost by a measurable percentage D) Make high-impact changes frequently and predictably with minimal toil and clear separation of concerns

Q4. A project is described as CNCF Sandbox. Which statement is supported?

A) It is stable, widely adopted, and production ready B) It is experimental, not yet widely tested in production, and on the bleeding edge C) It is used successfully in production by a small number of users D) It has completed the TOC's due diligence and adopter interviews

Q5. Which describes the relationship between the CNCF Governing Board and the Technical Oversight Committee?

A) The TOC sets the CNCF's scope; the Governing Board approves projects within it B) Both bodies vote jointly on every project application before it is accepted C) The Governing Board is elected by the TOC and reports to it on technical matters D) The Governing Board handles business oversight and budget and sets the CNCF's scope; the TOC defines the technical vision and approves projects within that scope

Q6. A candidate studying from a 2023 guide memorizes the CNCF TAGs as: App Delivery, Contributor Strategy, Environmental Sustainability, Network, Observability, Runtime, Security, and Storage. What is the problem?

A) TAGs were restructured in 2025; the current five are Developer Experience, Infrastructure, Operational Resilience, Security and Compliance, and Workloads Foundation B) Nothing — that is the current list C) Those are Kubernetes SIGs, not CNCF TAGs D) TAGs were abolished and their functions moved to the End User TAB

Q7. The CNCF glossary argues that a well-designed monolith "can uphold lean principles by being the simplest way to get an application up and running." What is the reasoning?

A) Monoliths outperform microservices once a system reaches production scale B) Microservices reduce total system complexity, so they only pay off on very large teams C) Microservices increase operational overhead, and building a microservices-based app before it has proven valuable may be premature

spending of engineering effort D) Monoliths are easier to make immutable, since there is only one artifact to replace

Q8. A cluster needs support for a storage array whose vendor Kubernetes has never heard of. What does Kubernetes provide, and what does the vendor provide?

A) Kubernetes provides a volume plugin compiled into the Kubernetes codebase, which the vendor configures B) The vendor submits a patch to the Kubernetes codebase for each release C) Kubernetes provides a device plugin framework the vendor registers against D) Kubernetes provides the Container Storage Interface — a published specification — and the vendor writes a CSI driver implementing it

Q9. [retrieval: ch6] A team needs the kube-apiserver to recognize and store a new object kind, served through the standard API so `kubectl get works`, without running any additional API server process. Which mechanism, and which of the four pluggable interfaces does it represent?

A) The API aggregation layer; it is the fourth pluggable interface B) A CustomResourceDefinition; it is the fourth pluggable interface C) A mutating admission webhook; it is not one of the four D) A scheduler plugin; it is the fourth pluggable interface

Q10. [retrieval: ch14] Why do Helm charts have a dedicated `crds/` directory rather than placing CustomResourceDefinitions in `templates/` with everything else?

A) CRDs cannot be templated, for security reasons B) CRDs are cluster-scoped, and Helm cannot manage cluster-scoped objects from templates C) A chart that ships custom resources must install their definitions before the objects that use them, which is an ordering problem the packaging format had to solve explicitly D) The directory is deprecated and exists only for backwards compatibility with Helm 2

Q11. The defining property of a service mesh, per both the CNCF glossary and Istio's own documentation, is that it adds reliability, observability and security features uniformly across services:

A) Without requiring code changes B) With a small, well-documented set of application code changes and a mesh-aware client library C) Only for services written in languages the mesh supplies an SDK for D) Only for north-south traffic entering the cluster from outside

Q12. [retrieval: ch10] A cluster enforces strict default-deny NetworkPolicy and terminates TLS at the Ingress. Which security gap does adding a service mesh with mTLS close?

A) It prevents unauthorized Pods from opening connections to a backend Service B) It encrypts the contents of Secrets as stored in etcd, closing the at-rest gap C) It replaces NetworkPolicy, which becomes unnecessary once identity is verified per request D) It encrypts and mutually authenticates service-to-service traffic inside the cluster — including the previously plaintext leg from the Ingress to the Pod, which is exactly the trusted-interior weakness zero trust names when it says trust is a vulnerability

Q13. Which statement about Istio's ambient mode is correct?

A) It removes Envoy from the architecture entirely, replacing it with ztunnel at every layer B) It uses a per-node L4 proxy called ztunnel, with optional per-namespace L7 waypoint proxies that are deployments of Envoy C) It is a competing product to Istio from a different project D) It requires every Pod in the cluster to migrate away from sidecars simultaneously

Q14. Knative Serving and Knative Eventing answer different questions. Which pairing is correct?

A) Serving handles asynchronous events; Eventing handles synchronous HTTP B) Serving runs the workload; Eventing is the autoscaler that scales it to and from zero C) Serving is an HTTP-triggered autoscaling container runtime including scale to zero; Eventing is a CloudEvents-over-HTTP asynchronous routing layer D) Serving is for stateful workloads; Eventing is for stateless ones

Q15. How does Knative relate to Kubernetes?

A) Knative is Kubernetes-based, builds on the Pod abstraction, and Serving and Eventing are implemented as CustomResourceDefinitions B) Knative replaces the Kubernetes control plane with its own scheduler and API server C) Knative runs containers outside of Pods, which is what makes scale to zero possible D) Knative is a fork of Kubernetes maintained separately by the CNCF

Q16. [retrieval: ch13] A HorizontalPodAutoscaler is created targeting a Deployment. The object is accepted, but the replica count never changes regardless of load. `kubectl top pods` returns an error. What is the most likely cause?

A) The Deployment's selector is misconfigured, so the HPA cannot find its Pods B) The HPA requires a VerticalPodAutoscaler to be installed alongside it C) The HPA control loop runs only once, at object creation D) Nothing is serving the Metrics API the HPA reads from — metrics-server is not installed

Q17. [retrieval: ch7] Pods are stuck in Pending because no existing node can satisfy their resource requests. Which component addresses this, and what does it do?

A) The HorizontalPodAutoscaler, by reducing the replica count until the remaining Pods fit B) The VerticalPodAutoscaler, by lowering the Pods' resource requests until they are schedulable C) A node autoscaler — Cluster Autoscaler or Karpenter — by provisioning new nodes to accommodate unschedulable Pods D) The scheduler, by preempting lower-priority Pods until room appears

Q18. Which correctly pairs each autoscaler with the axis it moves?

A) HPA → replica count; KEDA → replica count; VPA → resources per replica; Cluster Autoscaler → node pool B) HPA → resources per replica; VPA → replica count; KEDA → node pool; Cluster Autoscaler → node pool C) HPA → replica count; VPA → resources per replica; KEDA → node pool; Cluster Autoscaler → node pool D) All four move the replica count, differing only in what triggers them

Q19. [retrieval: ch8] The Kubernetes project ships approximately three minor releases per year and maintains release branches for the most recent three minor releases. What follows, and which group runs it?

A) Each release is supported for approximately three years; SIG Architecture runs the process B) Each release receives approximately one year of patch support — the two facts are the same fact — and SIG Release runs the process C) Each release is supported only until the next one ships; the Steering Committee runs the process D) Support duration varies by release and no single group is responsible for it

Q20. An engineer has six merged pull requests to the Kubernetes project, two-factor authentication enabled on their GitHub account, and a subscription to the dev mailing list. Two reviewers, both employed by the same company as the engineer, offer to sponsor their membership. What still stands between them and Member status?

A) Nothing — they meet every stated requirement, and the membership request issue is a formality B) Having reviewed or merged at least twenty substantial pull requests C) A nomination from a subproject owner D) Sponsorship by two reviewers **from different companies**

Q21. A candidate who passes the KCNA plans to continue with CKA, CKAD, and CKS. What changes about the exams as they move up that ladder?

A) The domain weights shift toward administration, but the format stays online multiple-choice throughout B) Each later exam adds a short hands-on section alongside a longer multiple-choice section C) KCNA is online and multiple-choice; CKA, CKAD, and CKS are performance-based, solved at a command line against a live environment D) The later exams remain multiple-choice but are proctored in person at a testing center

Answers and Explanations

Q1 — C. Verbatim from the definition [source: cncf-cloud-native-definition-2026-08-23]. **A** is a plausible-sounding list of cloud native buzzwords, none of which appear in the characteristics clause. **B** mixes real cloud native concepts, declarative and immutable are both in the *technology* list, with ones the definition does not name as characteristics at all. **D** is a list of general software quality attributes; "sustainable" and "observable" are the two that distinguish the real answer, and they are the two people most often drop.

Q2 — A. "public, private, hybrid cloud" appears in the definition's first sentence [source: cncf-cloud-native-definition-2026-08-23]. **B** is the misconception the definition preemptively rules out. **C** is wrong — the definition is explicit, not silent. **D** invents a requirement the definition does not make; a platform's conformance to a Kubernetes distribution standard has nothing to do with whether it meets this definition.

Q3 — D. The definition's closing clause: techniques combined with robust automation "allow engineers to make high-impact changes frequently and predictably with minimal toil and clear separation of concerns" [source: cncf-cloud-native-definition-2026-08-23]. **A** overclaims: "minimal toil" is not "no

operations staff." **B** promises a guarantee no architecture provides. **C** invents a cost claim the definition does not make.

Q4 — B. Sandbox projects are "experimental projects not yet widely tested in production, on the bleeding edge of technology" [source: cncf-project-maturity-levels-2026-08-23]. **A** is Graduated. **C** is Incubating. **D** is the trap worth understanding: the due-diligence and adopter-interview process is what a project completes to *leave* Sandbox [source: cncf-toc-project-lifecycle-process-2026-08-31]. A Sandbox project has not been through it.

Q5 — D. The Board is "responsible for marketing and other business oversight and budget decisions" [source: cncf-charter-governance-bodies-2026-08-31]; the TOC defines "the technical vision" and approves "new projects within the scope of the CNCF set by the Governing Board" [source: cncf-toc-and-tags-2026-08-23]. **A** inverts the relationship exactly, a common error because "Technical Oversight Committee" sounds more senior than "Board." **B** invents a joint process; project votes are the TOC's. **C** invents a reporting line that runs backwards.

Q6 — A. The restructuring was approved by the TOC and announced in May 2025 [source: cncf-tags-current-structure-2026-08-31]. **B** is the trap for anyone studying from pre-2025 material, and the old list is still all over blog posts and courses. **C** confuses the two organizations, though the confusion has a real historical root, since CNCF's groups were originally called SIGs before being renamed TAGs. **D** is fabricated; the End User TAB is a distinct body with a distinct role.

Q7 — C. The glossary states that devolving an application into microservices "increases its operational overhead," and that "crafting a microservices-based app before it has proven valuable may be premature spending of engineering effort. If the application yields no value, that effort becomes wasted" [source: cncf-glossary-microservices-monoliths-coupling-2026-08-31].

- **A is wrong.** The argument is about *timing and cost*, not raw performance at scale, and the microservices entry's own scaling complaint runs the other way.
- **B is the misconception worth catching.** Microservices do not reduce total complexity; the glossary says the operational surface "increase[s] by order of magnitude." What they redistribute is *who* carries which complexity.
- **D invents a connection** between monoliths and immutability that the glossary does not make; immutable infrastructure is orthogonal to how many services you have.

Q8 — D. CSI is a published interface; storage vendors implement drivers against it (*Ch 11 §5*). The specification's own stated objective is to "enable storage vendors (SP) to develop a plugin once and have it work across a number of container orchestration (CO) systems" [source: csi-spec-objective-2026-08-25], which is worth noticing: CSI is not a Kubernetes feature vendors happen to use, it is a cross-orchestrator standard Kubernetes implements.

- **A describes storage code living inside the Kubernetes codebase**, which is exactly what a published interface exists to avoid.

- **B is the same error, stated more explicitly.** The whole benefit is that vendors ship on their own schedule without touching Kubernetes.
- **C is a real mechanism for a different problem:** device plugins advertise hardware resources to the kubelet [source: k8s-docs-api-aggregation-and-device-plugins-2026-08-31], not storage volumes.

Q9 — B. A CustomResourceDefinition makes the kube-apiserver recognize new kinds of object, with the API server handling storage and serving [source: k8s-docs-api-aggregation-and-device-plugins-2026-08-31], and CRDs are the fourth of this book's four pluggable interfaces (*Ch 6 §8*).

- **A names a real mechanism that fails the stated constraint.** The aggregation layer proxies to a service you run, so you would be running an additional API server, precisely what the question rules out. Taking Your Bearings 2 asked this same distinction from the other direction, where aggregation was the right answer; the constraint in the stem is what separates them.
- **C is not an API-creation mechanism at all.** A mutating webhook modifies objects that already have a kind.
- **D is a scheduling extension,** at a different layer entirely.

Q10 — C. A chart shipping custom resources must install the CRDs before the objects using them; *crds/* exists to solve that ordering [*cross-bearing: see Ch 14 §6 — which one, when*]. **A** is wrong: the constraint is ordering, not security. **B** is wrong; Helm manages cluster-scoped objects routinely. **D** is fabricated.

Q11 — A. The CNCF glossary: mesh features are added "uniformly across all services across a cluster without requiring code changes" [source: cncf-glossary-service-mesh-2026-08-31]. Istio: "without code changes" [source: istio-service-mesh-2026-08-23]. **B** negates the defining property, and it is the option a reader who has only met SDK-based tracing will reach for. **C** is wrong, and is why the proxy model exists at all — Envoy's own documentation says it "works with any application language" [source: envoy-what-is-envoy-2026-08-31]. **D** inverts the traffic direction; a mesh's distinctive contribution is east-west, service-to-service.

Q12 — D. NetworkPolicy permits and denies connections; it cannot encrypt (*Ch 10 §7*). TLS terminated at the Ingress ends there. mTLS between workloads closes exactly that gap, with mutual authentication of workload identity [source: istio-security-mtls-identity-2026-08-31], and the reason it matters is the zero-trust principle that "trust is a vulnerability," where an attacker inside a trusted perimeter can move laterally through everything that trusts it [source: cncf-glossary-zero-trust-architecture-2026-08-31].

- **A is what NetworkPolicy already does;** the question asks what the mesh *adds*.
- **B confuses transit with rest.** Encryption at rest is a separate control on etcd (*Ch 12 §4*), and data does not stay encrypted because it was encrypted somewhere else. Each hop and each resting place needs its own control.
- **C overclaims.** Mesh and NetworkPolicy are complementary, and many clusters run both — Istio names defense in depth among its own security goals [source: istio-security-mtls-identity-2026-08-31].

Q13 — B. Ambient mode "implements its features using a per-node Layer 4 (L4) proxy, and optionally a per-namespace Layer 7 (L7) proxy"; ztunnel is "a purpose-built, per-node proxy"; and the waypoint proxy "is a deployment of the Envoy proxy; the same engine that Istio uses for its sidecar data plane mode" [source: istio-ambient-mode-2026-08-31]. **A** is the trap, and it is half-right in a way that makes it tempting: ambient removes *sidecars* and does introduce a purpose-built proxy, but only at L4. The L7 waypoint is Envoy. **C** is wrong; ambient is an Istio mode, documented by the Istio project. **D** is contradicted directly: sidecar and ambient Pods "can co-exist within the same mesh."

Q14 — C. Serving is "an HTTP-triggered autoscaling container runtime... including scale to zero"; Eventing is "a CloudEvents-over-HTTP asynchronous routing layer" [source: knative-overview-2026-08-23].

- **A swaps them**, which is the error a reader who knows both names but not their jobs will make.
- **B is the near-miss.** Serving *is* the thing that autoscales — the Knative Pod Autoscaler is part of Serving [source: knative-serving-autoscaling-2026-08-31] — so this option splits one component into two. Eventing is not an autoscaler; it routes events.
- **D inverts a stated property.** Knative Serving manages "the complete lifecycle of stateless HTTP services" [source: knative-overview-2026-08-23]; *stateless* is Serving's own description of its workloads, not Eventing's. Neither component is distinguished by statefulness.

Q15 — A. Knative "builds on the Kubernetes Pod abstraction," and Serving and Eventing "are implemented as Kubernetes Custom Resource Definitions (CRDs)" [source: knative-overview-2026-08-23]. **B** and **D** both assert replacement or forking; Knative does neither, and is built out of Kubernetes' own extension mechanism. **C** is the "serverless means no containers" misconception in mechanical dress: the containers are in Pods at every populated step of the cycle, which is exactly what makes scale-to-zero a lifecycle property rather than an architectural one.

Q16 — D. The HPA reads from the Metrics API, which metrics-server serves as a cluster addon and reference implementation [source: k8s-docs-resource-metrics-pipeline-2026-08-31]. The failing `kubectl top` is the giveaway; it consumes the same API (*Ch 13 §7*). **A** would cause different symptoms: the Deployment itself would fail to manage its Pods, not merely fail to scale, and `kubectl top` would still work. **B** is fabricated; the two autoscalers are independent, though both need the Metrics API. **C** contradicts the documented behavior — the HPA runs "as a control loop that runs intermittently (it is not a continuous process)" [source: k8s-docs-hpa-2026-08-24], which means periodically, not once.

Q17 — C. "If there are Pods in a cluster that can't be scheduled on existing Nodes, new Nodes can be automatically added to the cluster — *provisioned* — to accommodate the Pods" [source: k8s-docs-node-autoscaling-2026-08-31]. **A** is backwards; shrinking the workload to fit the cluster is not scaling to meet demand. **B** describes something the VPA does not do: it adjusts resources based on observed usage, not to satisfy the scheduler, and it is an add-on besides. **D** misuses preemption: preemption evicts lower-priority Pods to make room (*Ch 7 §2*), which relocates the shortage rather than resolving it, and does nothing when there is no lower-priority work to evict.

Q18 — A. HPA and KEDA move the replica count on different triggers, observed utilization versus external events. VPA moves per-replica resources. Cluster Autoscaler moves the node pool [source: k8s-docs-

autoscaling-and-vpa-2026-08-31], [source: k8s-docs-node-autoscaling-2026-08-31]. **B** swaps horizontal and vertical, the most common error in this material, and misfiles KEDA on top of it. **C** is the trap for a reader who has learned that KEDA is "the external one" and filed it with the cloud-provider autoscalers; KEDA scales workloads, not machines. **D** collapses a genuine distinction: the KEDA/HPA axis overlap is real, but VPA and the node autoscalers move genuinely different things.

Q19 — B. "Kubernetes releases currently happen approximately three times per year"; the project "maintains release branches for the most recent three minor releases"; and "Kubernetes 1.19 and newer receive approximately 1 year of patch support" [source: k8s-release-cycle-and-cadence-2026-08-31]. Three per year, three maintained, roughly a year: one fact, three ways. SIG Release owns "production of Kubernetes releases on a reliable schedule" [source: k8s-release-cycle-and-cadence-2026-08-31]. **A** misreads three versions as three years and misassigns the SIG. **C** understates the window badly; that would be about four months. **D** is wrong on both halves — the project publishes explicit end-of-life dates and a named group runs the process.

Q20 — D. The Member requirements include being "sponsored by 2 reviewers (from different companies)" [source: k8s-community-membership-ladder-2026-08-23]. Two sponsors from the same employer as the candidate satisfies the count and fails the constraint, and the constraint is deliberate, a structural check against any single company manufacturing members.

- **A is wrong** on the requirement, and also skips a step: the candidate must open a membership request issue in kubernetes/org. That is a real action, not a formality.
- **B is the Reviewer rung.** Twenty substantial reviewed or merged PRs, plus primary reviewer on at least five, plus three months as a Member.
- **C is the Approver rung.** Nomination by a subproject owner comes two rungs above Member.

Q21 — C. KCNA is described as "online and multiple-choice"; CKA is "a performance-based exam where candidates interact with the command line to solve real-world challenges"; CKAD is "a hands-on, command-line environment"; CKS is "performance-based" [source: cncf-kcna-certification-page-2026-08-23]. The format is what changes.

- **A gets it exactly backwards.** Domain weights differ per exam, but the format change is the headline: everything above KCNA on this ladder is performance-based.
- **B invents a hybrid** that none of the three exams uses.
- **D changes the wrong variable.** Proctoring is not the distinction, and none of the cached sources describes in-person testing centers for these credentials.

Chapter Summary

Concept	Remember This
Cloud native (v1.1)	Loosely coupled systems that interoperate securely, resiliently, manageably, sustainably, observably . Public, private, and hybrid — the term is about method, not location.
The technology list	Containers, service meshes, multi-tenancy, microservices, immutable infrastructure, serverless, declarative APIs — and explicitly non-exhaustive .
Maturity levels	Sandbox (bleeding edge) → Incubating (production use by a few, healthy contributors) → Graduated (stable, widely adopted). Archived is the exit, not a rung.
Where criteria live	The TOC's project lifecycle documentation — not the projects page. Application, adopter interviews, due diligence, public comment, TOC vote.
CNCF bodies	Board : business, budget, scope. TOC : technical vision, approves projects within that scope. TAGs : five, restructured 2025. End User TAB : voice of end users, feeds the TOC.
The three-in-one	Microservices + immutable infrastructure + declarative APIs. Small pieces, replaced whole, described rather than commanded. Remove one and the other two get much harder.
The four interfaces	CRI runs it, CNI wires it, CSI stores it, CRDs name it. Kubernetes defines the interface; somebody else implements it.
The wider surface	Six documented extension points; API aggregation is the <i>other</i> route to a new API — you run the server, Kubernetes proxies to it.
Service mesh	Security, observability, traffic management without code changes . Data plane = the proxies. Mesh control plane = what configures them. Neither is the cluster's.
Sidecar vs ambient	Sidecar: an Envoy per Pod. Ambient: per-node ztunnel at L4, optional per-namespace Envoy waypoints at L7. Both use Envoy . They coexist.
mTLS / zero trust	Never trust, always verify — because trust is a vulnerability and lateral movement is the consequence. mTLS closes the plaintext leg NetworkPolicy cannot touch.
Serverless	Abstracts servers away from the user . Knative builds on Pods and ships as CRDs. Serving = sync HTTP + scale to zero. Eventing = async CloudEvents routing.
Autoscaling	Two questions, always: what moves, what triggers it . HPA → replicas/utilization. KEDA → replicas/events. VPA → resources (add-on). Cluster Autoscaler & Karpenter → nodes/unschedulable Pods.
Kubernetes groups	SIG : durable, topic-scoped. Working Group : time-bounded, cross-SIG. Committee : closed membership, chartered by Steering. Three of them, Steering included.
The release fact	Three releases a year and three supported minors and ~one year of patch support are one fact stated three ways. SIG Release runs it.
Joining	Member → Reviewer → Approver → Subproject Owner. Member needs contributions and two sponsors from different companies . No employer, seniority, or credential gate.

Concept	Remember This
The ladder above	KCNA is pre-professional and multiple-choice; CKA, CKAD, CKS are performance-based. This is the only one you pass by knowing things.

⚓ Safe Harbor

You have finished the largest chapter in this book, and the one that reaches furthest back. Every thread earlier chapters left hanging here has now been answered.

More concretely: the phrase Chapter 1 refused to define is defined. The pattern Chapter 2 promised would feel like recognition has been collected and named. The unschedulable Pod Chapter 7 left sitting in Pending has something watching it. The unencrypted leg Chapter 10 identified and declined to fix has a fix. The version numbers Chapter 8 warned were forgettable are now one relationship instead of three integers. And the material Chapter 1 said technically strong candidates skip — you did not skip it.

The Voyage Ahead

Domain 4 has one competency left, and it is the one people are most surprised to find there.

Observability is not one of the four domains on the current blueprint — Kubernetes Fundamentals, Container Orchestration, Cloud Native Application Delivery, and Cloud Native Architecture [source: cncf-kcna-certification-page-2026-08-23]. It is competency material inside Cloud Native Architecture, which is where Chapter 18 will take it up. [*cross-bearing: see Ch 1 — The Curriculum That Moved Under Everyone's Feet*] for what changed and when, and for why so much third-party prep still teaches a different split.

That placement sounds like a demotion and is not, because the question observability answers has only got harder as systems have got more loosely coupled.

Which is the connection worth holding as you turn the page. Every architectural choice in this chapter made systems easier to change and harder to see. Break a monolith into microservices and one request becomes twenty. Replace machines instead of patching them and the machine you want to inspect is gone. Scale to zero and the thing you want to ask a question about is not running.

Chapter 18 is about asking anyway. Four signals, one collector, a database that pulls instead of being pushed to, and a distinction — *observability versus monitoring* — that turns out to be about what you knew to ask in advance.

You have spent this chapter learning the shape of the fleet. Next you learn to read its instruments.

"Trust is a vulnerability." — CNCF Cloud Native Glossary, on zero trust architecture [source: [cncf-glossary-zero-trust-architecture-2026-08-31](#)]. It is a fine principle for a network, and not a bad one for a study plan.

Chapter 18: Reading the Instruments

"Four signals, and the question they exist to answer"

Domain Weight: 12% (Cloud Native Architecture) [source: cncf-kcna-curriculum-pdf-2026-08-23] |

Competency: Observability | **Authored allocation for this chapter:** ~5% **Complexity:** Mixed | **Novelty:** Moderate

Attention Budget

Total time: ~125 minutes | **Recommended:** Split across 2 sessions

Section	Time	Attention Cost	Best Time to Study
📍 Soundings	8 min	Low	Anytime
§1 What You Can Ask, and What You Already Know	10 min	Medium	Mid-session
§2 Four Signals	6 min	Low	Anytime
§3 Numbers Over Time	10 min	Medium	Mid-session
🕒 Taking Your Bearings (1)	6 min	Medium	After brief break
§4 Pulling, Not Being Pushed	14 min	Medium	When alert
§5 Following One Request	12 min	High	Peak attention
🕒 Taking Your Bearings (2)	6 min	Medium	After brief break
§6 Lines From Everywhere	10 min	Medium	Mid-session
§7 Is the Service Doing What Users Expect	10 min	Medium	When alert
🕒 Taking Your Bearings (3)	6 min	Medium	After brief break
§8 One Question, Four Instruments	5 min	Low	Anytime
Exam Alert + Practice Questions	20 min	High	Peak attention

Attention Cost Key:

- **Low:** Concrete, familiar concepts — study anytime
- **Medium:** New concepts requiring focus — study when alert
- **High:** Abstract or complex material — study at peak attention

If you only have 15 minutes: read §2 and §7, then take Checkpoint 3. Those two carry two of this chapter's six high-priority exam topics between them, and §7 names the question the other five are all in service of.

"You must be conscious about what outputs are coming out of your system." — CNCF TAG
Observability, *Observability Whitepaper*

🕒 Soundings

Before reading, take depth.

Eight questions. None of them requires this chapter; every one is answerable from a chapter you have already finished. Your score tells you how to read what follows.

1. A liveness probe fails and the kubelet restarts the container. What record of that failure does the cluster hold an hour later?
2. Name one question about your workloads that metrics-server is not built to answer.
3. `kubectl top pods` returns an error on a cluster whose Pods are all `Running` and healthy. What is the most likely cause, and what is the general pattern it belongs to?
4. An autoscaler reports that a Pod is at 80% CPU utilization. Eighty percent of *what*?
5. Why does cluster log collection typically run one agent per node rather than one agent per application?
6. You need a container's log output from three restarts ago. Can `kubectl logs` give it to you?
7. A single user request crosses five services. Each of the five writes its own complete, well-formatted log file. What question can those five log files not answer between them?
8. A service mesh reports per-service latency for an application whose source code was never touched. How is that possible?

Answers + reading strategy

1. **None.** A probe is a yes/no signal to the kubelet, evaluated and acted on immediately. The restart itself surfaces as a container-state reason, a restart count, and an Event that expires. The probe result is never stored, trended, or queried. *[cross-bearing: see Ch 5 §7 — three probes, three jobs]*
2. **Anything historical, and anything that isn't CPU or memory.** metrics-server holds current resource readings for autoscaling decisions. It does not keep a record of what those readings were an hour ago, and it does not collect application-level measurements. *[cross-bearing: see Ch 13 §7 — the resource metrics pipeline]*
3. **metrics-server is not installed.** The `kubectl top` command exists in every `kubectl` binary; the component that answers it does not exist in every cluster. This is the pattern *an object without its*

component does nothing — the API surface being present tells you nothing about whether anything is behind it. *[cross-bearing: see Ch 10 §3 — the object is not the implementation]*

4. **Eighty percent of the container's resource request.** Not of the node's capacity, and not of the container's limit. *[cross-bearing: see Ch 5 §8 — what a Pod is owed]*
5. **Because container logs are written to the node's filesystem,** by the container runtime, for every container the node runs [source: k8s-docs-logging-architecture-2026-08-23]. Collecting them is a per-node job, and the workload resource that puts exactly one Pod on every node is the DaemonSet. *[cross-bearing: see Ch 6 §7 — one per node, and work that ends]*
6. **No.** `kubectl logs --previous` reaches one termination back, and the current log file is subject to rotation [source: k8s-docs-logging-architecture-2026-08-23]. Three restarts ago is gone from the node. *[cross-bearing: see Ch 13 §3 — looking inside]*
7. **What order things happened in, and where the time went.** Each file is internally complete and the five have nothing joining them: no shared identifier, no common clock the reader can trust, no way to know which line in service D corresponds to which line in service A. *[cross-bearing: see Ch 17 §3 — small pieces, replaced whole]*
8. **The traffic passes through a proxy the platform injected,** not through code the application team wrote. The proxy sees every request enter and leave, and reports on what it sees. *[cross-bearing: see Ch 17 §5 — a network that knows what it's carrying]*

If you got 6+ right: skim. Read §1's distinction carefully, memorize the four signals in §2, and spend your real attention on §4 and §7 — those two carry five of the eleven traps in this chapter's Exam Alert. Take all three checkpoints.

If you got 3–5 right: read at normal pace. This chapter builds almost entirely on chapters you have already finished, and the questions you missed point at exactly which section will feel like new material rather than extension.

If you got 0–2 right: read carefully, and read two things first. Ch 13 §7 (the resource metrics pipeline and metrics-server) and Ch 17 §3 (microservices) are the load-bearing prerequisites for this chapter. Almost everything here extends one or the other. Twenty minutes there will save you an hour here.

Why This Chapter Matters

Chapter 17 ended by handing you a bill.

Every architectural choice in that chapter made systems easier to change and harder to see. Break a monolith into microservices and one request becomes twenty. Replace machines instead of patching them and the machine you want to inspect is gone. Scale to zero and the thing you want to ask a question about is not running.

This is the chapter where the bill arrives.

Here is the part most study guides will not say out loud: **you cannot dashboard your way out of a question you did not know to ask.** Every dashboard you have ever built is a set of answers to questions somebody chose in advance. That is genuinely useful. It is also, on the night that matters, beside the point: the thing that broke is the thing nobody drew a graph for.

Practitioners do not add observability because a framework told them to. They add it because at some point they were asked a question about a running system and had no way to answer it. Somebody said *why was checkout slow between 2:10 and 2:14 for users in one region*, and there was no path from that sentence to an answer. Not a missing dashboard. A missing *capability*. That is the moment this chapter is about, and by the end of it you will have four instruments for it and a clear sense of which one to reach for.

Dead Reckoning: Observability lets you understand a system from the outside by letting you ask questions about that system without knowing its inner workings [source: opentelemetry-observability-primer-2026-08-23]. It is a property of the system rather than a product you install [source: cncf-glossary-observability-2026-08-31]. To produce the outputs that make it possible, the system must be instrumented — code in its components must emit signals [source: opentelemetry-instrumentation-2026-08-31]. This chapter covers the four signals OpenTelemetry defines, the two dominant collection tools (Prometheus for metrics, Jaeger for traces), how logs get off a node, and the vocabulary for saying whether a service is behaving acceptably: SLI, SLO, and the four golden signals.

One honest note before you start, because you may have met a contradiction already.

If you have been studying with material published before late 2025, you were probably told that Observability is its own KCNA domain with its own weight. On the current blueprint it is not. Observability is now a competency inside **Cloud Native Architecture**, a domain weighted at 12% [source: lf-kcna-program-changes-2026-08-23]. The Linux Foundation's own announcement puts it plainly: the domains remain mostly unchanged "except that observability will be rolled under Cloud Native Architecture" [source: lf-kcna-program-changes-2026-08-23].

Read that as a reorganization, not a demotion. The material did not stop being tested. It stopped being *separately weighted*, which is a different thing entirely, and a candidate who reads "rolled under" as "de-emphasized" and skims this chapter will find that out expensively. [cross-bearing: see Ch 1 — the curriculum that moved under everyone's feet]

What You'll Learn

By the end of this chapter, you'll be able to:

- **Distinguish** observability from monitoring by what each one lets you *ask*, not by which tool implements it
- **Name** all four OpenTelemetry signals — including the one most candidates drop
- **Explain** why Prometheus pulls, and the one narrow case in which anything pushes
- **Trace** a single request across service boundaries, and say what a span is that a trace is not
- **Distinguish** an SLI from an SLO, and name the four golden signals
- **Choose** the right instrument for a question, instead of reaching for the one you happen to know

You'll also stop treating "we have monitoring" as an answer to "can we find out?" — which is the shift that separates people who fix outages from people who watch them.

§1 — What You Can Ask, and What You Already Know

Start with the confusion, because you almost certainly have it.

Most people use *monitoring* and *observability* as synonyms, with a vague sense that observability is the newer, more expensive one: monitoring with nicer dashboards and a vendor invoice attached. That is the most common misconception in this domain, and it will cost you a question.

The difference is not tooling. It is not dashboards. It is **what you are able to ask**.

Monitoring, in its canonical definition, is "collecting, processing, aggregating, and displaying real-time quantitative data about a system" [source: sre-book-monitoring-definitions-2026-08-31]. Notice what that sentence assumes: somebody already decided *which* quantitative data. Google's own Site Reliability Engineering (SRE) text lists four reasons to monitor, and every one of them is a question chosen in advance [source: sre-book-monitoring-definitions-2026-08-31]:

- **Long-term trends** — "How big is my database and how fast is it growing?"
- **Alerting** — "Something is broken, and somebody needs to fix it right now!"
- **Building dashboards** — "Dashboards should answer basic questions about your service"
- **Debugging** — "Our latency just shot up; what else happened around the same time?"

Those are good questions. They are also, all four, questions somebody sat down and wrote out before the incident. A monitoring system is an instrument built to answer a fixed set of questions very well.

Observability is the other thing. The CNCF's TAG Observability whitepaper draws the line in one sentence: "Monitoring is called a system that can detect known unknowns — as opposed to observability which emphasizes being able to find and reason about unknown unknowns as well" [source: cncf-tag-observability-whitepaper-2026-08-31].

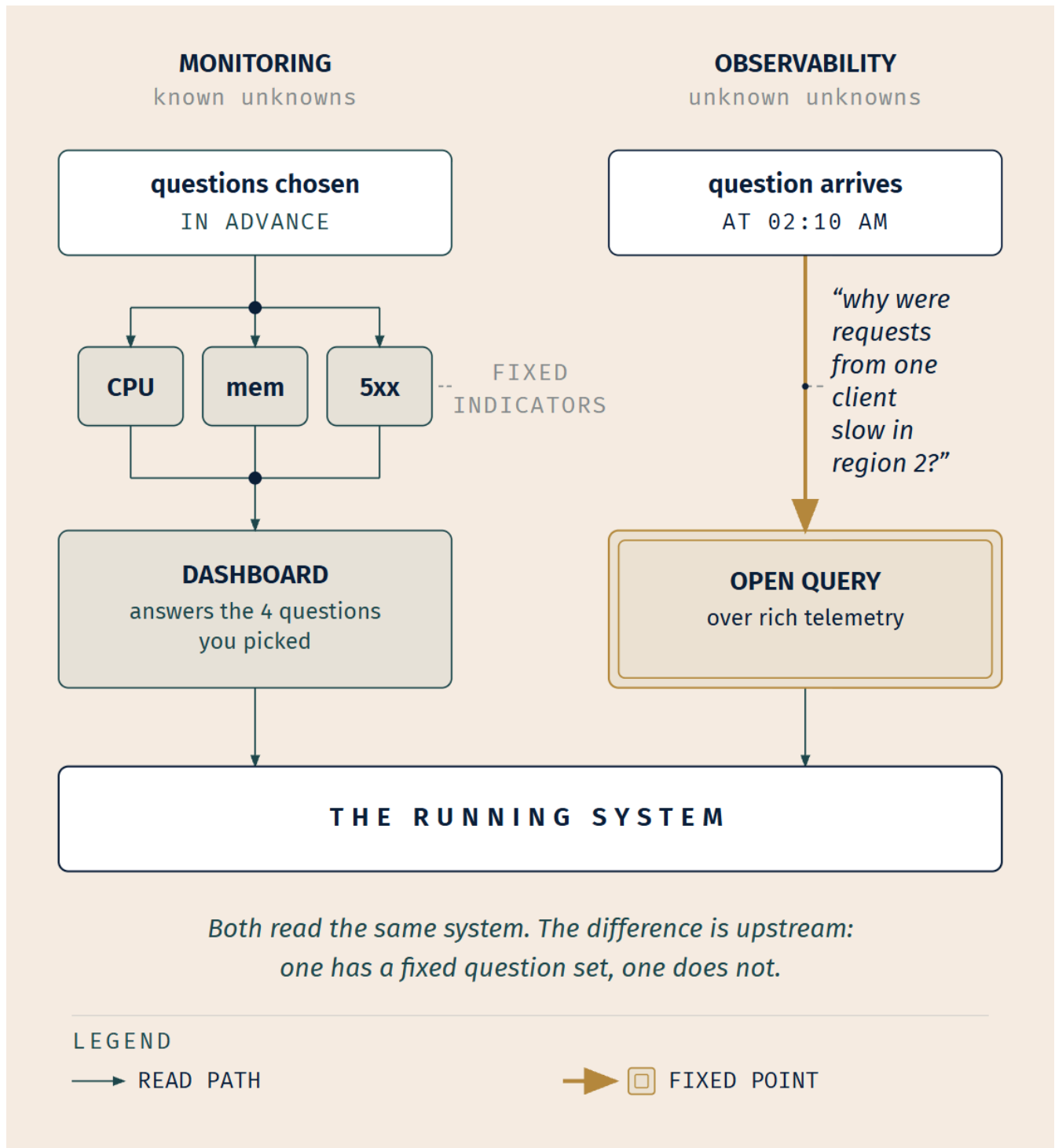
A **known unknown** is a question you have thought of but do not yet have the answer to. *Is the database near capacity?* You knew to ask; you built a graph; the graph tells you. An **unknown unknown** is a question you had not thought to ask until the moment you needed it. *Why did requests from one specific*

customer's mobile client start failing at 2:10, but only when they hit the region-two cache? Nobody built that graph. Nobody could have. There are more possible questions of that shape than there are dashboards in the world.

☆ Fixed Point

Observability is the ability to ask questions of a running system that you did not plan for in advance — to reason about *unknown unknowns*. Monitoring watches indicators you chose ahead of time — *known unknowns*. The distinction is about what you can ask, not about which tool you bought.

OpenTelemetry's own framing says the same thing from the other side: observability "lets you understand a system from the outside by letting you ask questions about that system without knowing its inner workings," which is what lets you "troubleshoot and handle novel problems, that is, 'unknown unknowns'" [source: opentelemetry-observability-primer-2026-08-23]. It also helps you answer the question "Why is this happening?" — as opposed to *whether* it is happening, which a dashboard can already tell you.



Note what that figure is *not*. It is not "Prometheus versus OpenTelemetry." Those are both tools, and both of them serve both postures. A Prometheus deployment with rich, well-labeled metrics and an ad-hoc query language is doing observability work; a pile of OpenTelemetry traces nobody can query is not. The posture is upstream of the tool.

Instrumentation is the precondition

None of this happens for free. A system is observable only to the degree it emits something worth reading.

The OpenTelemetry docs state the dependency directly: "For a system to be observable, it must be instrumented: that is, code from the system's components must emit signals" [source: opentelemetry-instrumentation-2026-08-31]. **Instrumentation** is the work of making a system emit those signals. **Telemetry** is the data that comes out — "data emitted from a system and its behavior" [source: opentelemetry-observability-primer-2026-08-23].

And here is the sharpest available test of whether you have done enough of it:

☆ Fixed Point

An application is properly instrumented when developers don't need to add more instrumentation to troubleshoot an issue, because they have all of the information they need [source: opentelemetry-observability-primer-2026-08-23].

Sit with that for a second. The bar is not "we emit metrics." The bar is: when something novel breaks, nobody has to ship a code change to find out what. If your debugging procedure routinely begins with *let me add a log line and redeploy*, you have a monitoring system and an aspiration.

Instrumentation comes in two kinds. **Code-based** instrumentation is written into the application and gives "deeper insight and rich telemetry from your application itself." **Zero-code** instrumentation attaches from outside and is "great for getting started, or when you can't modify the application you need to get telemetry out of," pulling telemetry "from libraries you use and/or the environment your application runs in" [source: opentelemetry-instrumentation-2026-08-31]. Hold onto that second kind. It returns in §5.

🔍 **Closer Look:** The CNCF glossary makes a point the other sources leave implicit: observability is "a system property," and consequently "how observable a system is will significantly impact its operating and development costs" [source: cncf-glossary-observability-2026-08-31]. It is not a product you install. It is a characteristic your system either has or lacks, and the cost of lacking it is paid in engineer-hours at three in the morning.

What a probe is not

You have already met something in this book that looks like observability, sits close to it, and is emphatically not it.

Probes.

A liveness probe asks a container a yes/no question on a schedule. A readiness probe asks a different yes/no question on a schedule. Both produce an immediate action — restart the container, remove the

Pod from endpoints — and then the answer is spent. *[cross-bearing: see Ch 5 §7 — three probes, three jobs]*

What a probe does not produce is a **record**. There is no trend. There is no history you can query. There is no way to ask "how often did this endpoint fail its readiness check last Tuesday, and did it correlate with the deploy?" The kubelet asked, got an answer, acted, and moved on. Health checking answers *is it up right now, according to one binary test I wrote in advance*, which is not just a different question from *why is this happening*. It is a different **kind** of question.

🔖 **Snag:** "We have liveness and readiness probes configured" is a true and useful statement about a workload's self-healing. It is not an answer to "is this service observable." A cluster full of perfect probes can still leave you unable to explain a single slow request. Health checking is not observability.

Everything from here on is about building the records that probes do not keep.

§2 — Four Signals

This section is short, and its only job is naming. What you need out of it is a list you can reproduce cold, under time pressure, with the right number of items in it.

The number is four.

A *signal*, in the ordinary sense of the word, is what you send when you want somebody else to know your state. The term is doing exactly that job here. OpenTelemetry is the project whose documentation this chapter has been quoting since §1, and its Signals page lists four [source: opentelemetry-signals-2026-08-23]:

Signal	What it is	What it answers	What it cannot answer alone
Traces	"the path of a request through your application"	<i>Where</i> did the time go, and in what order?	What the numbers looked like in aggregate
Metrics	"a measurement captured at runtime"	<i>Whether</i> something is happening, and how much	Which specific request was affected
Logs	"a recording of an event"	<i>What</i> the code said happened	Nothing about ordering across services, on its own
Baggage	"contextual information that is passed between signals"	— it is not a measurement; it is what lets the other three talk about the same request	Anything by itself

A **signal**, generally, is a system output describing the underlying activity of the platform and the applications on it — something you want to measure at a point in time, or an event moving through a distributed system that you would like to trace [source: opentelemetry-signals-2026-08-23].

☆ Fixed Point

OpenTelemetry's Signals page defines FOUR signals: traces, metrics, logs, and baggage [source: opentelemetry-signals-2026-08-23]. **Candidates reliably name three and drop baggage, because baggage is not itself a measurement — it is contextual information passed between the other signals.**

📌 **Snag — "four" is OpenTelemetry's count, and the attribution matters.** The CNCF TAG Observability whitepaper enumerates a different five: metrics, logs, traces, profiles and dumps [source: cncf-tag-observability-whitepaper-2026-08-31]. OpenTelemetry's own primer, mentioning signals in passing, names only three [source: opentelemetry-observability-primer-2026-08-23]. Two authorities, three lists, and all of them correct about their own document. When a question names **OpenTelemetry**, it wants the four.

🧠 **Mnemonic: T-M-L-B — Traces, Metrics, Logs, Baggage.** Or, if letters won't stick: three instruments and the thing that ties them together. The three you'd name unprompted are the measurements; the fourth is the string running through them.

Why baggage counts

Baggage is a key-value store that travels with a request, letting you "propagate any data you like alongside context" [source: opentelemetry-baggage-2026-08-31]. Its purpose is specific: "include information typically available only at the start of a request further downstream" — things like account identification, user IDs, product IDs, and origin IPs [source: opentelemetry-baggage-2026-08-31]. Service A knows which customer this is. Service E, four hops later, does not, unless something carried it there.

The detail that makes baggage a *separate* signal rather than a property of spans is this: "baggage is a separate key-value store and is unassociated with attributes on spans, metrics, or logs without explicitly adding them" [source: opentelemetry-baggage-2026-08-31]. It rides alongside. Attaching it to a span is a deliberate act. That separation is exactly why it earns its own row, and exactly why people forget it.

Baggage "means you can pass data across services and processes, making it available to add to traces, metrics, or logs in those services" [source: opentelemetry-baggage-2026-08-31]. Which is to say: it is the signal that makes the other three signals talk about *the same thing*. Hold that. §8 is built on it.

🔍 **Closer Look:** Baggage travels over the wire, which has a consequence worth one sentence of operational caution: "Sensitive Baggage items can be shared with unintended resources, like third-party APIs... making it visible to anyone inspecting your network traffic" [source: opentelemetry-baggage-2026-08-31]. Customer identifiers are the classic case. Not exam surface, but it is the reason mature teams have a policy about what goes in baggage.

One limit, stated early

Logs deserve an honest caveat before you meet them properly in §6, because it plants two later sections at once.

Logs "are not extremely useful for tracking code execution on their own, as they typically lack contextual information; they become far more useful when they are included as part of a span, or when they are correlated with a trace and a span" [source: opentelemetry-observability-primer-2026-08-23].

That is not a claim that logs are bad. It is a claim about what a log line *is*: a timestamped message from one process, which — unlike a trace — is not "necessarily associated with any particular user request or transaction" [source: opentelemetry-observability-primer-2026-08-23]. One line, no context about the journey it belonged to. Which is exactly the situation Soundings Q7 put you in, and exactly what \$5 exists to fix.

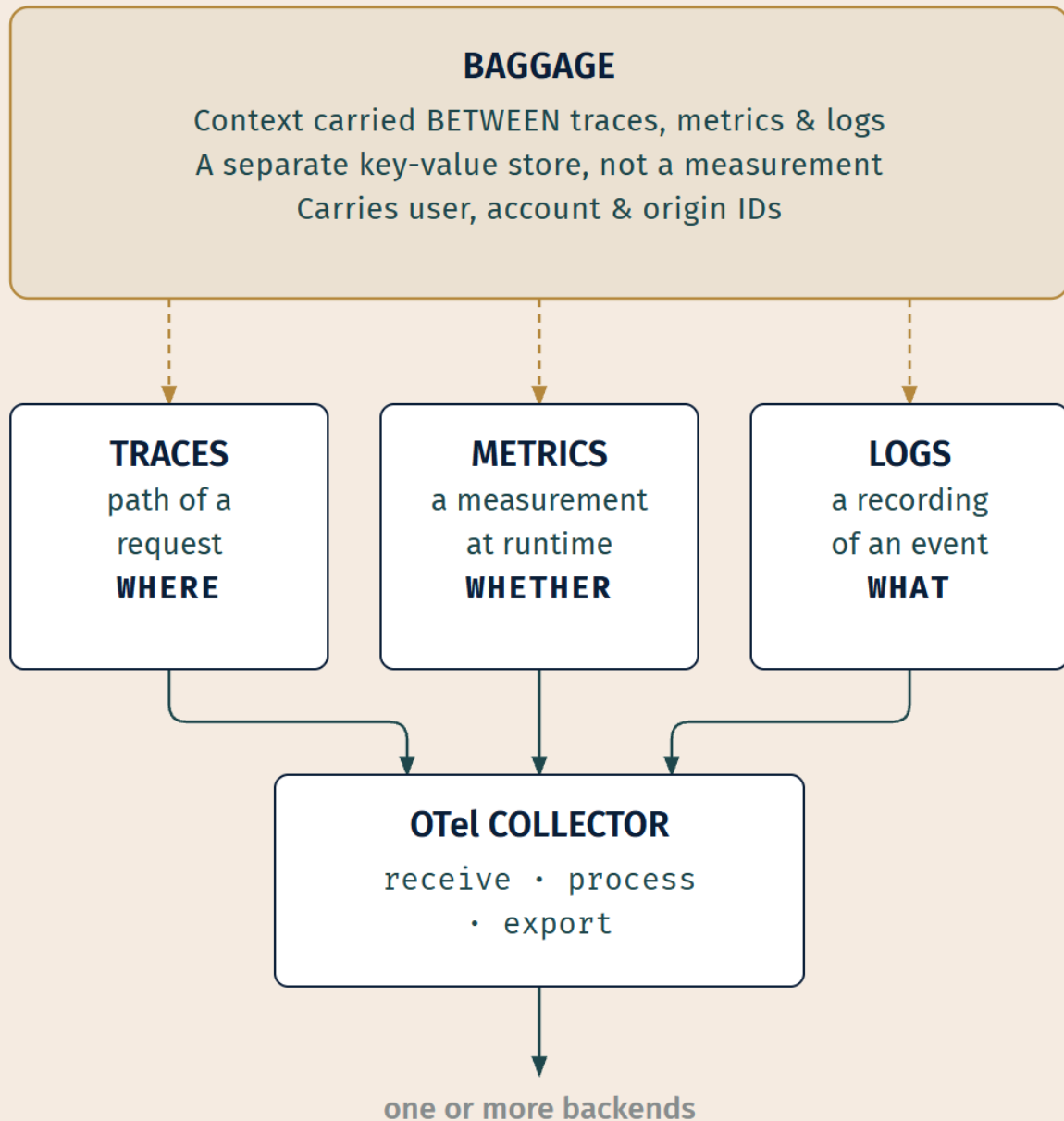
The collector

One more piece of vocabulary, because a signal is an *output*, not a system. Something has to receive the outputs.

The **OpenTelemetry Collector** is "a vendor-agnostic implementation of how to receive, process and export telemetry data" [source: opentelemetry-collector-2026-08-31]. Its value proposition is consolidation: it "removes the need to run, operate, and maintain multiple agents/collectors," and it supports "open source observability data formats (e.g. Jaeger, Prometheus, Fluent Bit, etc.) sending to one or more open source or commercial backends" [source: opentelemetry-collector-2026-08-31].

Read that second quote slowly, because it contains the architectural idea this whole chapter keeps re-encountering. One set of signals goes *in*. Multiple, swappable backends receive them going *out*. The thing that produces telemetry and the thing that stores it are separate, and either can be replaced without the other noticing.

OPENTELEMETRY • FOUR SIGNALS



The Collector is deployable "as an agent or collector with support for traces, metrics, and logs" from a single codebase [source: opentelemetry-collector-2026-08-31]: one implementation, several shapes, several signal types.

§3 — Numbers Over Time

You already know most of this section. That is the point of it.

Chapter 13 taught you the resource metrics pipeline: metrics-server, kubectl top, and the reason both exist. *[cross-bearing: see Ch 13 §7 — numbers nobody collects by default]* This section is not going to re-teach that. It draws the boundary on the *other* side of it — what a monitoring system does that metrics-server does not — because that boundary is what the exam actually reaches for.

A metric is a number with a shape

Start with what makes a metric different from a log line.

Prometheus "fundamentally stores all data as time series: streams of timestamped values belonging to the same metric and the same set of labeled dimensions" [source: prometheus-data-model-2026-08-31]. A **time series** is not one number. It is a stream of them, each stamped with when it was taken. Each individual reading is a **sample**, consisting of a value and a millisecond-precision timestamp [source: prometheus-data-model-2026-08-31].

That structure is what makes metrics cheap to keep and cheap to aggregate. You cannot ask "what was the 95th-percentile latency across all of last week" of a pile of log lines without reading all of them. You can ask it of a time series trivially, because the shape of the data was chosen for exactly that question.

Labels, and the cost of one

Time series are identified by more than a name.

"Every time series is uniquely identified by its metric name and optional key-value pairs called **labels**" [source: prometheus-data-model-2026-08-31]. So `http_requests_total{method="GET", status="200"}` and `http_requests_total{method="GET", status="500"}` are not one series with two flavors. They are two distinct series that happen to share a name.

🔖 **Snag:** A *metric label* and a *Kubernetes label* are different things that share a word. Kubernetes labels are the universal join between objects and selectors *[cross-bearing: see Ch 4 §5 — the universal join]*. Metric labels are dimensions on a time series. They are not related, they are not interchangeable, and reading a question quickly is exactly how you conflate them.

The identity rule has a consequence that follows directly from it: "The change of any label's value, including adding or removing labels, will create a new time series" [source: prometheus-data-model-2026-08-31].

🔗 **Closer Look — cardinality.** Because the label set *is* the identity, "every unique combination of key-value label pairs represents a new time series, which can dramatically increase the amount of data stored" [source: prometheus-naming-labels-cardinality-2026-08-31]. Hence the standing advice: "Do not use labels to store dimensions with high cardinality (many different label values), such as user IDs, email addresses, or other unbounded sets of values" [source: prometheus-naming-labels-cardinality-2026-08-31]. Add a `user_id` label to a busy endpoint's request counter and you have not added a dimension. You have multiplied your storage by your user count. This is not KCNA exam surface; it is the single most common way real teams break their own metrics stack, which is why it's here.

The denominator

Two earlier chapters sent you here for one number, so let's settle it.

When an autoscaler or a dashboard tells you a Pod is at "80% CPU utilization," the denominator is **the containers' resource request**. Kubernetes states it directly: "if a target utilization value is set, the controller calculates the utilization value as a percentage of the equivalent resource request on the containers in each Pod" [source: k8s-docs-hpa-utilization-vs-requests-2026-08-31].

Not the node's capacity. Not the container's limit. The request — the amount the Pod *asked for* when it was scheduled. [cross-bearing: see Ch 5 §8 — *what a Pod is owed*]

The cleanest proof of this comes from the failure case rather than the formula. "If some of the Pod's containers do not have the relevant resource request set, CPU utilization for the Pod will not be defined and the autoscaler will not take any action for that metric" [source: k8s-docs-hpa-utilization-vs-requests-2026-08-31].

Read that again. Not "utilization will be computed against the node." Not "it defaults to something." **Undefined.** A Pod with no request has no utilization percentage at all, which is only possible if the request is the denominator. Remove the denominator and the fraction ceases to exist.

🔗 **Mnemonic:** Utilization is a fraction, and the bottom of the fraction is what you **asked for**, not what you were **standing on**. Request, not node. No request, no fraction.

The boundary that gets tested

Now the other half.

metrics-server is the add-on that provides the `metrics.k8s.io` API, and Kubernetes notes it "needs to be launched separately" [source: k8s-docs-hpa-utilization-vs-requests-2026-08-31]. That is the Kubernetes documentation stating, from the autoscaling side, the same pattern you met from the `kubectl top` side: **an object without its component does nothing**. [cross-bearing: see Ch 10 §3 — *the object is not the implementation*] The `kubectl top` verb is in every `kubectl` binary ever shipped. Whether anything answers it is a separate question about your cluster.

Chapter 13 established what metrics-server is for and, just as importantly, what it is scoped to. What it is **not** is a monitoring system. A monitoring system, in the sense this chapter means, does four things metrics-server does not:

Question	metrics-server	A monitoring system
What is this Pod using <i>right now</i> ?	✓ yes	✓ yes
What was it using an hour ago?	✗ no history kept	✓ yes — time series
How many 500s did the checkout endpoint return?	✗ CPU and memory only	✓ yes — arbitrary metrics
Page someone when error rate crosses 2%	✗ not its job	✓ yes — alerting
Compute p99 (99th-percentile) latency over a rolling 7 days	✗ no query language	✓ yes — query over time

These two coexist. That is worth stating plainly, because candidates who learn the boundary sometimes overcorrect into "so you should replace metrics-server with Prometheus," which is wrong. metrics-server exists to feed autoscaling decisions with current readings, fast and cheaply. A monitoring system exists to keep history and answer arbitrary questions about it. Different jobs; both installed on plenty of real clusters.

⚠ Navigational Hazards

The exam does not test whether you can define metrics-server. It tests whether you know **which side of the boundary a given question falls on**. When you read a scenario, ask one thing: *does answering this require history, alerting, or a metric that isn't CPU or memory?* If yes, that is monitoring-system territory and metrics-server is the wrong answer — no matter how much the words in the question sound like "metrics."

The pipeline itself you have already seen. [cross-bearing: see Ch 13 §7 — the resource metrics pipeline, figure ch13-fig04-metrics-pipeline-and-metrics-server] Go back and look at it again if it has faded; there is no new diagram here, because there is no new architecture here. There is only a boundary you can now see the far side of.

🕒 Taking Your Bearings: What You Can Ask, and What the Numbers Report Against

Five questions. One of them tests material from an earlier chapter — that is deliberate, and it will keep happening for the rest of the book.

1. Your team has thorough dashboards: CPU, memory, request rate, error rate, and p99 latency per service, all with 90 days of history and alerting configured. At 02:10 a single enterprise customer reports that image uploads are failing, but only from their mobile app, and only in one region. None of the dashboards show anything unusual. Which best describes the situation?

- A) The dashboards are misconfigured; correctly configured monitoring would have caught this
- B) This is a known unknown that the alerting thresholds were set too high for
- C) This is an unknown unknown — a question nobody pre-registered, which no fixed dashboard set can answer
- D) The team has observability but lacks monitoring

2. A developer says: "Our app is properly instrumented — we emit metrics for request count and latency on every endpoint." By OpenTelemetry's own standard, what would actually establish that the application is properly instrumented?

3. For each question below, say whether metrics-server can answer it, whether it requires a monitoring system, or whether nothing in this chapter's toolkit records it at all:

- (a) How much memory is this Pod using at this instant?
- (b) Did this Deployment's memory use climb steadily over the past six hours?
- (c) How many requests did the payments service reject last night?
- (d) Which node currently has the most CPU consumed by Pods?
- (e) How many times did this endpoint fail its readiness check during last night's deploy?

4. Name all four OpenTelemetry signals. For the one that is *not* a measurement, state what it is and why it is nonetheless a signal.

5. *[retrieval: ch10]* You run `kubectl top nodes` on a freshly provisioned cluster. Every node is Ready, every Pod is Running, and the command returns an error. Name the cause, and name the general pattern this is an instance of — using the book's exact phrase for it.

Answers with Explanations:

1. C.

- **A is wrong**, and it is the tempting one. The dashboards are not misconfigured. They are correctly answering the five questions somebody chose. The failure is that this incident is not one of those five, and there is no configuration of a fixed dashboard set that anticipates every possible question. "Add another dashboard" is a response to a *specific* unknown unknown after it becomes known; it does not fix the class.
- **B is wrong** because a known unknown is a question you thought of. Nobody thought of "uploads from one customer's mobile client in one region." That is what makes it unknown.
- **C is correct**. This is precisely the "unknown unknowns" case [source: cncf-tag-observability-whitepaper-2026-08-31]: a novel question arriving unbidden, requiring the ability to interrogate the system rather than read a pre-built answer.
- **D is wrong** and inverts the situation. They clearly *have* monitoring: real-time quantitative data, displayed, with alerting [source: sre-book-monitoring-definitions-2026-08-31]. What they lack is the ability to ask the new question.

2. The standard is not about which metrics you emit. It is: **developers don't need to add more instrumentation to troubleshoot an issue, because they have all of the information they need** [source: opentelemetry-observability-primer-2026-08-23]. Request count and latency per endpoint is a good start and tells you *whether* something is wrong. It does not tell you *why*. If the team's debugging procedure routinely begins with adding a log line and redeploying, the application is not properly instrumented by this definition, however many metrics it emits.

3.

- **(a) metrics-server.** Current resource reading, CPU/memory. Exactly its scope.
- **(b) Monitoring system.** Requires history. metrics-server keeps none.
- **(c) Monitoring system.** Requires both history *and* an application-level metric that is neither CPU nor memory.
- **(d) metrics-server.** Current, resource-level, no history required — this is what `kubectl top nodes` reports.
- **(e) Neither, as posed.** A readiness probe is a yes/no question the kubelet asks, acts on, and discards; it produces no trend and no queryable history. [*cross-bearing: see Ch 5 §7 — three probes, three jobs*] Getting an answer means recording the *consequence* — the endpoint leaving and rejoining the Service, over time — which is monitoring-system work. Health checking is not observability.

The discriminator is the one in the Navigational Hazards box: history, alerting, or a non-resource metric ⇒ monitoring system. And a probe result is not a record at all, so it is on neither side of that boundary until something else writes it down.

4. Traces, metrics, logs, and baggage [source: opentelemetry-signals-2026-08-23].

The one that is not a measurement is **baggage**: "contextual information that is passed between signals" [source: opentelemetry-signals-2026-08-23], a key-value store that propagates data — user IDs, account identifiers, origin IPs — from the start of a request to services further downstream [source: opentelemetry-baggage-2026-08-31].

It counts as a signal because it is a distinct thing the system emits and propagates, with its own store: "baggage is a separate key-value store and is unassociated with attributes on spans, metrics, or logs without explicitly adding them" [source: opentelemetry-baggage-2026-08-31]. It is not a field on a span. It rides alongside, which is what makes it separable and what makes it forgettable.

5. The cause: metrics-server is not installed. The `metrics.k8s.io` API has no provider, so the command has nothing to query [source: k8s-docs-hpa-utilization-vs-requests-2026-08-31].

The pattern: an object without its component does nothing. The API surface existing tells you nothing about whether anything is behind it. [*cross-bearing: see Ch 10 §3 — the object is not the implementation*]

How'd You Do?

5 correct: move on with confidence. §4 will feel like extension, not new territory. **3–4 correct:** review the ones you missed before continuing. Understanding *why* the wrong answer was tempting is worth more here than the score. **0–2 correct:** stop and re-read §1 and §3 before going on. §4 assumes you can already tell a question that needs history from one that does not, and it will not re-explain the boundary.

Checkpoint: You've Now Mastered

✓ The observability/monitoring distinction, stated as what you can ask ✓ What "properly instrumented" actually means, and why most teams fail the test ✓ Why a probe leaves no record — and why health checking is not observability ✓ The metrics-server / monitoring-system boundary, and the question that discriminates them ✓ All four signals, including the one you'd otherwise drop

Four content sections to go, and the next one carries more sourced exam traps than any other section in this chapter.

..1 §4 — Pulling, Not Being Pushed

Everything the exam wants from you about Prometheus turns on one thing: **which way the arrow points**.

It is the same discipline as reading a bearing. The number tells you nothing until you know which end of the line you are standing on. Get that right and four separate traps stop working on you. Get it wrong and they all land.

The arrow

Prometheus is "an open-source systems monitoring and alerting toolkit originally built at SoundCloud" [source: prometheus-overview-2026-08-23]. It collects and stores metrics as time series — values with timestamps, plus optional key-value labels [source: prometheus-overview-2026-08-23], exactly the data model §3 described.

The collection mechanism is the part to memorize: "time series collection happens via a **pull model over HTTP**" [source: prometheus-overview-2026-08-23].

Prometheus reaches out. Your application does not report in. On an interval, the Prometheus server makes an HTTP request to each target's metrics endpoint and takes what it finds. This is called **scraping**. A **target** is "the definition of an object to scrape" [source: prometheus-glossary-2026-08-31], which in practice means one process exposing one source of metrics.

How does it know what to scrape? "Targets are discovered via **service discovery** or static configuration" [source: prometheus-overview-2026-08-23]. In a Kubernetes cluster this matters enormously, because Pods are not durable and their addresses are not stable [*cross-bearing: see Ch 9 §2 — the address that*

doesn't last]. Static configuration would be obsolete before you finished writing it. Service discovery lets Prometheus ask the cluster what exists right now, and scrape that.

☆ Fixed Point

Prometheus PULLS. It scrapes metrics from targets over HTTP on an interval; targets are found via service discovery or static configuration [source: prometheus-overview-2026-08-23]. **Pushing exists only through the Pushgateway, and "the only valid use case for the Pushgateway is for capturing the outcome of a service-level batch job"** [source: prometheus-pushgateway-practices-2026-08-31].

The one exception, and how narrow it is

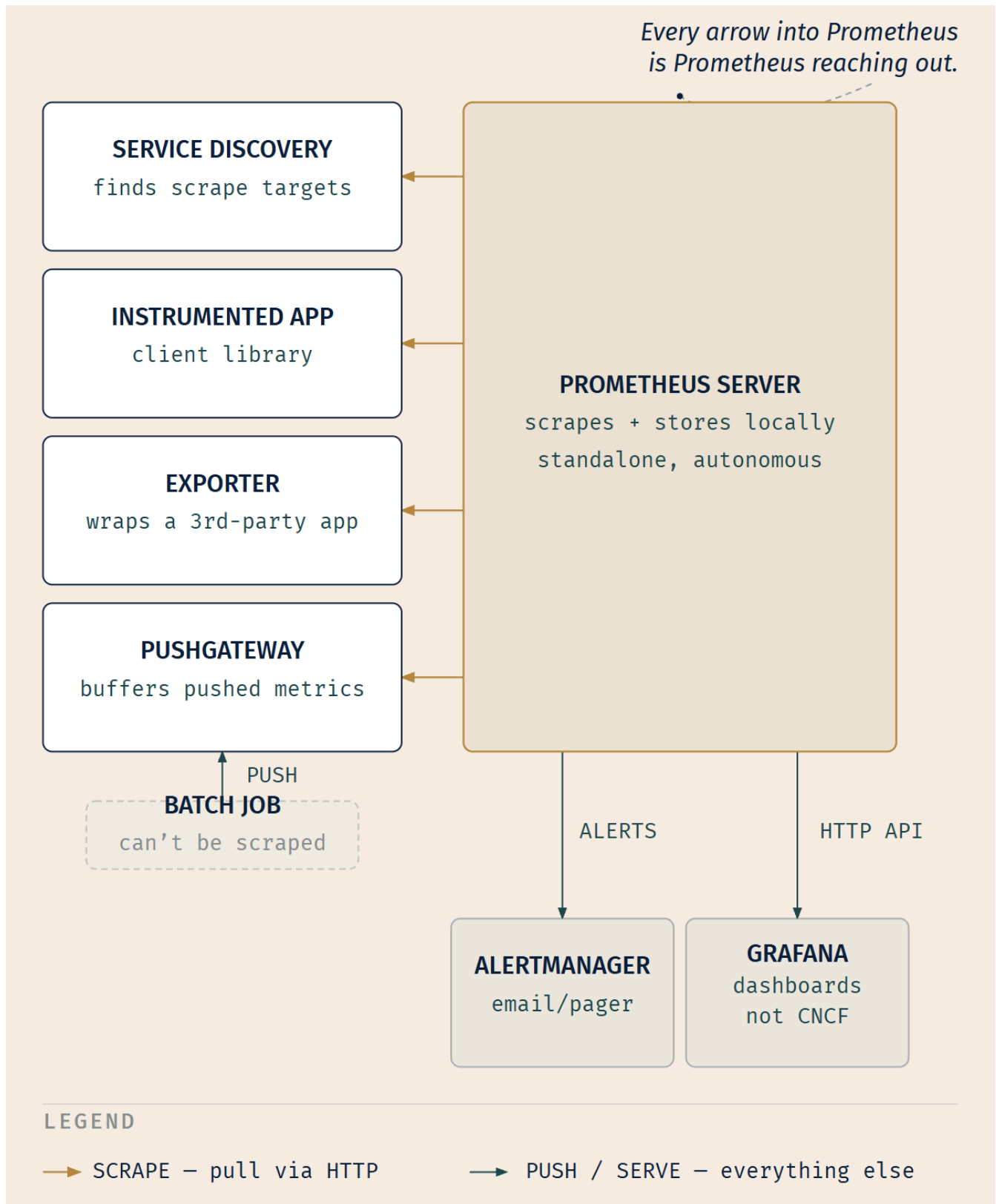
There is exactly one sanctioned way for something to push into Prometheus, and the project's own documentation goes out of its way to fence it in.

The **Pushgateway** is "an intermediary service which allows you to push metrics from jobs which cannot be scraped" [source: prometheus-pushgateway-practices-2026-08-31]. Note *intermediary*. Even here, the metric does not land in Prometheus by being pushed. It is pushed to the gateway, and Prometheus scrapes the gateway. The arrow into Prometheus never reverses.

And the licensed use is narrow to the point of being brusque: "We only recommend using the Pushgateway in certain limited cases," and "the only valid use case for the Pushgateway is for capturing the outcome of a service-level batch job" — where a service-level batch job is "one which is not semantically related to a specific machine or job instance" [source: prometheus-pushgateway-practices-2026-08-31].

The reason is straightforward once you see it: a job that runs for eleven seconds and exits cannot be scraped on a thirty-second interval. It is gone. Everything else — every long-running service, every process with an address that stays put long enough to be visited — gets scraped.

📌 **Snag:** "This service would rather push its metrics" is not a Pushgateway use case. Neither is "this job is tied to one specific machine" — the source explicitly scopes the Pushgateway to *service-level* batch jobs, which are the ones **not** semantically tied to a particular instance [source: prometheus-pushgateway-practices-2026-08-31]. Distractors are built from exactly these two plausible-sounding scenarios.



The pieces

Prometheus is a toolkit, not a single binary. Its main components [source: prometheus-overview-2026-08-23]:

The Prometheus server — scrapes targets and stores the time series data locally.

Client libraries — "a library in some language (e.g. Go, Java, Python, Ruby) that makes it easy to directly instrument your code" [source: prometheus-glossary-2026-08-31]. You add this to software *you* wrote, and it exposes a metrics endpoint for Prometheus to scrape.

Exporters — "a binary running alongside the application you want to obtain metrics from" [source: prometheus-glossary-2026-08-31]. This is for software you *didn't* write and can't modify. Special-purpose exporters exist for services like HAProxy, StatsD, and Graphite [source: prometheus-overview-2026-08-23]. The exporter speaks whatever the application speaks on one side and Prometheus's scrape format on the other.

Pushgateway — the narrow inbound path above. It "persists the most recent push of metrics from batch jobs" [source: prometheus-glossary-2026-08-31].

Alertmanager — "takes in alerts, aggregates them into groups, de-duplicates, applies silences, throttles, and then sends out notifications" [source: prometheus-glossary-2026-08-31], routing "to the correct receiver integration such as email, PagerDuty, or OpsGenie" [source: prometheus-alertmanager-2026-08-31].

🔒 **Worth Securing:** Client library versus exporter is the discrimination worth carrying into the exam. A **client library** instruments code from the *inside*: you added it, you own the code. An **exporter** is a *separate binary* that gets metrics out of something you did not write and cannot change. Same destination, opposite side of the code boundary. That is the same instrumentation-versus-backend separation you met with the Collector in §2, appearing again.

📌 **Snag — the arrow reverses exactly once, inside Prometheus's own architecture.** Alertmanager "handles alerts **sent by** client applications such as the Prometheus server" [source: prometheus-alertmanager-2026-08-31]. So Prometheus *pulls* from targets and *pushes* to Alertmanager. That is not a contradiction of the pull model: the pull model describes how metrics are **collected**, not how notifications are **dispatched**. Keep the two clauses separate and both facts fit.

Asking questions of what you collected

Storing time series is only half of it. **PromQL** — "the Prometheus Query Language" [source: prometheus-glossary-2026-08-31] — is the functional query language that "lets the user select and aggregate time series data in real time" [source: prometheus-promql-basics-2026-08-31].

That is the whole of what you need. PromQL syntax is not KCNA surface; what matters is knowing that a query language exists, that it is the thing turning stored series into answers, and that the ability to write

an *arbitrary* query is precisely what lifts a metrics store from "shows the dashboards we built" toward the \$1 posture.

Results are available in the Prometheus UI, and "other programs can fetch the result of a PromQL expression via the HTTP API" [source: prometheus-promql-basics-2026-08-31]. Which brings us to the dashboard layer.

Grafana is one such API consumer: "Grafana or other API consumers can be used to visualize the collected data" [source: prometheus-overview-2026-08-23]. It reads through that HTTP API. It is not part of Prometheus, and Prometheus does not depend on it.

📌 **Snag:** Grafana is Grafana Labs' own software — "open source software that enables you to query, visualize, alert on, and explore your metrics, logs, and traces wherever they're stored" [source: grafana-introduction-2026-08-23]. Contrast that with the two projects in this chapter whose own documentation states a CNCF donation outright: Prometheus joined in 2016 [source: prometheus-overview-2026-08-23], and Fluentd was accepted in November 2016 [source: fluentd-architecture-2026-08-31]. Carry the contrast, not a roster. Foundation membership moves, and anything graded on a *current* roster is graded on a moving target. [cross-bearing: see Ch 17 §2 — *sandbox, incubating, graduated, and who decides*]

Where Prometheus fits, and where it does not

Two statements from the project's own documentation are worth memorizing, because knowing a tool's stated *non-fit* is the kind of thing exams reward.

On independence: "Each Prometheus server is standalone, not depending on network storage or other remote services, so you can rely on it when other parts of your infrastructure are broken" [source: prometheus-overview-2026-08-23].

That is a deliberate design trade, not a limitation someone forgot to fix. The moment your monitoring system depends on your clustered storage, an outage in that storage takes down the thing that would have told you about the outage. Prometheus chooses to work alone so that it still works when nothing else does.

On accuracy: "Prometheus values reliability. If you need 100% accuracy, such as for per-request billing, Prometheus is not a good choice as the collected data will likely not be detailed and complete enough" [source: prometheus-overview-2026-08-23].

This follows directly from the pull model. Scraping samples state on an interval; it does not observe every event. If a counter increments 400 times between two scrapes, Prometheus sees the difference, not the four hundred events. For alerting, dashboards, and capacity work, that is exactly right and vastly cheaper. For invoicing a customer per API call, it is unacceptable, and the project says so itself.

⚠ Navigational Hazards

"Reliability over completeness" is the sentence to carry. A scenario that mentions billing, financial reconciliation, audit logs, or any phrase implying every single event must be counted is telling you that Prometheus is the *wrong* answer. Candidates who have learned "Prometheus is the metrics tool" pick it anyway, because the question is about numbers. The question is about *completeness*, and Prometheus explicitly trades that away for the ability to keep working during an outage.

🔒 **Worth Securing:** Prometheus "joined the Cloud Native Computing Foundation in 2016 as the **second** hosted project, after Kubernetes" [source: prometheus-overview-2026-08-23]. It was created at SoundCloud in 2012 [source: prometheus-overview-2026-08-23]. Second, not first. Kubernetes was first, and swapping those two is a real distractor.

🔊 §5 — Following One Request

This is the hardest section in the chapter, so we'll build it the way you actually hit the problem: not from the vocabulary down, but from the wall you run into.

You met the wall in Soundings Q7.

The wall

One user request enters your system. It hits the API gateway, which calls the auth service, which calls the user store; the gateway then calls the catalog service, which calls the pricing service, which calls a cache and then a database.

Seven processes. All seven log correctly. All seven log *well* — structured JSON, accurate timestamps, sensible messages. You have every log line from every service for the four seconds in question.

And you cannot answer "why did that request take four seconds."

Not because the data is missing. Because **nothing joins it**. There is no field in service E's log line that says *I am part of the same request as that line in service A*. You have seven complete stories about seven different subjects, and no way to know they were all about one customer pressing one button.

It is the oldest problem in navigation wearing new clothes: seven readings, and no way to fix a position from any of them, because a fix needs two lines drawn to the *same* landmark and nothing here agrees on what the landmark was.

That is the gap. Everything in this section is one idea for closing it: **carry an identifier across the boundary**.

Context, and moving it

Context is "an object that contains the information for the sending and receiving service, or execution unit, to correlate one signal with another" [source: opentelemetry-context-propagation-2026-08-31].

Propagation is "the mechanism that moves context between services and processes" [source: opentelemetry-context-propagation-2026-08-31]. Together, **context propagation** is what lets signals "be correlated with each other, regardless of where they are generated."

The concrete version is one sentence, and it is worth reading slowly because it is the whole mechanism: "Service A includes a trace ID and a span ID as part of the context. Service B uses these values to create a new span that belongs to the same trace" [source: opentelemetry-context-propagation-2026-08-31].

That is it. Service A stamps an identifier onto the outbound call. Service B reads it and stamps its own work as *belonging to that same identifier*. Repeat across seven hops and you have a chain that survives every process and network boundary in between. As OpenTelemetry puts it, context propagation "allows traces to build causal information about a system across services that are arbitrarily distributed across process and network boundaries" [source: opentelemetry-context-propagation-2026-08-31].

Arbitrarily distributed. That is the sentence closing the loop Chapter 17 opened. Break a monolith into twenty services and the request path becomes a thing you can no longer hold in your head, so you make the system carry the thread for you.

The identifier travels in HTTP headers; "the default propagator uses the headers specified by the W3C TraceContext specification" [source: opentelemetry-context-propagation-2026-08-31]. You do not need the header format for this exam. You need to know that the correlation rides *with the request*, in-band, rather than being reconstructed afterward by a clever log parser.

And baggage — the fourth signal from §2 — is the general form of the same trick: "Baggage allows you to propagate arbitrary key-value pairs" [source: opentelemetry-context-propagation-2026-08-31]. Trace and span IDs join the *work*. Baggage carries whatever else you want to travel with it: the customer ID, the origin IP, the feature-flag cohort.

Span and trace

Now the vocabulary lands on a need you can feel.

A **span** "represents a unit of work or operation." Spans "track specific operations that a request makes, painting a picture of what happened during the time in which that operation was executed," and a span "contains name, time-related data, structured log messages, and other metadata (that is, attributes) to provide information about the operation it tracks" [source: opentelemetry-observability-primer-2026-08-23].

A **trace** — formally a distributed trace — "records the paths taken by requests (made by an application or end-user) as they propagate through multi-service architectures, like microservice and serverless applications" [source: opentelemetry-observability-primer-2026-08-23].

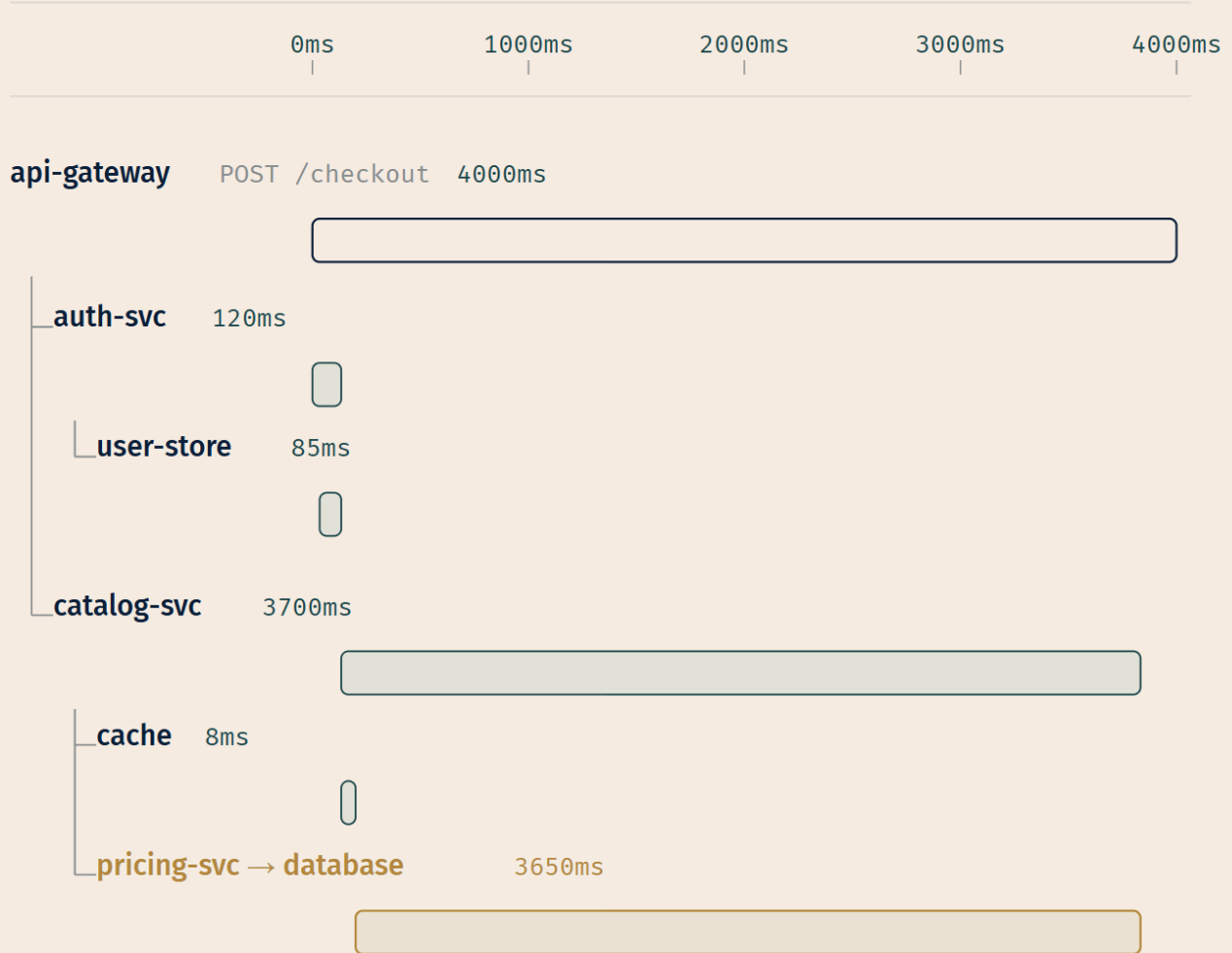
And the join between them, which is the exam's actual target:

☆ **Fixed Point**

A span is ONE unit of work. A trace is the WHOLE path a request took, and "a trace is made of one or more spans." "The first span represents the root span; each root span represents a request from start to finish" [source: opentelemetry-observability-primer-2026-08-23]. Spans beneath the root "provide a more in-depth context of what occurs during a request."

Note "one or more." A single-service request produces a trace with exactly one span in it. A trace is not defined by being multi-service. It is defined by being *the whole request*.

TRACE ID 4bf92f... — crosses every span below

**LEGEND**

○ Root span — full request

○ Child span

○ Focal — most of the time

└ Nested under parent

*Seven services. Seven sets of logs, each internally complete.**Only the trace ID makes them one story — and only then does**"where did the 4 seconds go?" have an answer.*

Look at what that figure gives you that seven log files cannot: **nesting** and **duration**. You can see that pricing-svc's database call accounts for nearly the entire four seconds, and you can see it *without* comparing timestamps across seven files and hoping the clocks agree. The structure is the answer.

🔗 **Mnemonic:** A **span** *spans* one operation. A **trace** *traces* the whole route. If you can point at a single box on the diagram, it's a span; if you mean the entire picture, it's a trace. And the picture always starts with one box that covers all the others — the root.

📌 **Snag:** "Span" and "trace" are used loosely and interchangeably in conversation all the time, including by people who know better. The exam does not use them loosely. If a question describes a single operation with a name and a duration, that is a span; if it describes the full journey of a request, that is a trace.

Jaeger

Spans have to go somewhere.

Jaeger is a distributed tracing backend: it "receives tracing telemetry data and provides processing, aggregation, data mining, and visualizations" [source: jaeger-overview-2026-08-23]. It "was originally designed to support the OpenTracing standard," and it is "OpenTelemetry compatible; terminology and concepts map directly between the two data models" [source: jaeger-overview-2026-08-23].

Keep the division of labor explicit, because it is going to matter in three paragraphs' time. OpenTelemetry — **OTel** in the shorthand the figure above uses, and the project's own — is the *instrumentation* side: the APIs and SDKs that produce spans and export them. Jaeger is the *backend*, the thing that receives them, stores them, and draws the picture above. The OTel Collector supports Jaeger as one of its output formats [source: opentelemetry-collector-2026-08-31], which is exactly the arrangement: producer, wire format, consumer, all separable.

If that shape feels familiar, it should. Prometheus stores and Grafana reads. OTel exports and Jaeger receives. Two different corners of this chapter, same architecture.

Spans you didn't ask for

One last thing, and it closes a loop from Chapter 17.

A service mesh injects a proxy alongside every workload, and that proxy sees every request enter and leave. Because it sits in the path, it can emit spans for traffic that passes through it — for an application whose source code nobody touched. [*cross-bearing: see Ch 17 §5 — a network that knows what it's carrying*]

OpenTelemetry names this category directly: **zero-code instrumentation** is "great for getting started, or when you can't modify the application you need to get telemetry out of," and it provides "rich telemetry from libraries you use and/or the environment your application runs in" [source: opentelemetry-instrumentation-2026-08-31]. The environment your application runs in is precisely what a mesh is.

There is a real limit to it, though, and it is the difference between the two kinds of instrumentation from §1. The proxy knows what crossed the network: this service called that service, it took 40 milliseconds, it returned a 503. It does not know what happened *inside* — which branch the code took, which cache

missed, which of three database queries was the slow one. Zero-code instrumentation gives you the shape of the request path for free. Code-based instrumentation is what fills in the boxes.

Logbook Entry: The order in which teams typically adopt this is worth knowing, because it is not the order the documentation implies.

Almost nobody starts with hand-instrumented traces. A team gets a mesh for mTLS or traffic management, notices they now have latency data per service for free, and builds a dashboard on it. That gets them to *which service is slow*, which resolves a large fraction of incidents, because "the pricing service is slow" is frequently all you needed.

The push to real, code-level instrumentation comes later, and it always comes from the same place: an incident where "the pricing service is slow" was the *beginning* of the question rather than the end. Somebody has to know which of the four things pricing-svc does was the slow one, the mesh cannot say, and that afternoon somebody adds an OpenTelemetry SDK.

The order matters for how you read exam scenarios. A question describing per-service latency in an uninstrumented app is describing the mesh. A question describing visibility *inside* a service is describing code-based instrumentation. The phrase "without changing the application" is doing real work whenever it appears.

🕒 Taking Your Bearings: Pull, Push, and the Path One Request Took

Five questions. Two draw on earlier chapters.

1. A team wants their long-running web service to report metrics to Prometheus. A developer proposes: "We'll have the service POST its metrics to the Pushgateway every 15 seconds — that way Prometheus doesn't have to reach into our network." Evaluate this proposal.
2. Your company bills customers per API call and needs the count to be exact for invoicing. An architect proposes Prometheus, reasoning that request counting is the canonical metrics use case. What is wrong with this, and what does the Prometheus documentation itself say?
3. A request enters your system, is handled by three services, and completes. Assuming full instrumentation, which is true?
 - A) Three traces are produced, one per service, each containing one span
 - B) One trace is produced, containing three or more spans, the first of which is the root span
 - C) One span is produced, containing three traces
 - D) Three root spans are produced and correlated afterward by timestamp
4. *[retrieval: ch13]* Your cluster already runs metrics-server. You now need Prometheus to collect metrics from two things: a PostgreSQL instance you cannot modify, and a Go service your own team wrote. Which pairing is correct?

- A) A client library for both
- B) An exporter for both
- C) A client library in the Go service, an exporter alongside PostgreSQL
- D) The Pushgateway for PostgreSQL, a client library for the Go service

Then, in one sentence: why does metrics-server, already installed, not remove the need for either?

5. [retrieval: ch17] A service mesh is deployed and the platform team now has per-service latency and error rates for applications that were never instrumented. An engineer investigating a slow endpoint asks: "Which of the three database queries in that handler is slow?" Can the mesh telemetry answer that? Why or why not?

Answers with Explanations:

1. The proposal is wrong, and it is wrong for a reason more specific than "Prometheus pulls."

Prometheus collects "via a pull model over HTTP" [source: prometheus-overview-2026-08-23], and the Pushgateway is not a general-purpose bypass for that. It is "an intermediary service which allows you to push metrics from jobs which cannot be scraped," and "the only valid use case for the Pushgateway is for capturing the outcome of a service-level batch job" [source: prometheus-pushgateway-practices-2026-08-31].

A long-running web service **can** be scraped, and that is the entire distinction. The correct design is a client library exposing a metrics endpoint, plus service discovery so Prometheus finds it. Note also that even the Pushgateway does not reverse the arrow into Prometheus: Prometheus scrapes *the gateway*.

2. The architect has matched on the word "metrics" and missed the requirement, which is **completeness**, not measurement.

Prometheus states its own non-fit: "If you need 100% accuracy, such as for per-request billing, Prometheus is not a good choice as the collected data will likely not be detailed and complete enough" [source: prometheus-overview-2026-08-23]. The billing example is the documentation's own. This falls out of the pull model: scraping samples state on an interval rather than observing every event.

The trade is deliberate: "Prometheus values reliability" [source: prometheus-overview-2026-08-23], and each server is standalone specifically so it works "when other parts of your infrastructure are broken." Billing needs an event stream that captures every transaction, which is a different system.

3. **B.** "A trace is made of one or more spans. The first span represents the root span; each root span represents a request from start to finish" [source: opentelemetry-observability-primer-2026-08-23]. One request, one trace, spans beneath the root for each unit of work.

- **A is wrong** and is the answer someone gives who is thinking in log files: one artifact per service. Traces are *per-request*, which is exactly what makes them useful across service boundaries.

- **C is wrong** and inverts the containment. Traces contain spans, not the reverse.
- **D is wrong** twice over. There is one root span per trace, and correlation comes from propagated trace and span IDs, not from comparing timestamps after the fact [source: opentelemetry-context-propagation-2026-08-31]. Timestamp correlation is precisely the fragile thing tracing exists to replace.

4. C. The discriminator is which side of the code boundary you are on.

- **A is wrong.** A client library "makes it easy to directly instrument your code" [source: prometheus-glossary-2026-08-31] — it goes *into* software you can change. You cannot add one to a PostgreSQL you do not build.
- **B is wrong.** An exporter is "a binary running alongside the application you want to obtain metrics from" [source: prometheus-glossary-2026-08-31]: right for PostgreSQL, but an unnecessary extra process beside a service you can instrument directly.
- **C is correct.** Client library inside the code you own; exporter beside the code you don't.
- **D is wrong**, and it is the strongest distractor here. It comes from correctly learning that the Pushgateway handles things Prometheus cannot reach into, and over-generalizing from *short-lived* to *unmodifiable*. PostgreSQL is long-running and perfectly scrapeable; "the only valid use case for the Pushgateway is for capturing the outcome of a service-level batch job" [source: prometheus-pushgateway-practices-2026-08-31].

On metrics-server: it holds current CPU and memory readings so autoscalers can make decisions. It keeps no history and collects no application-level metric, so connection counts, query latencies and rejected requests are outside its scope entirely: different word, different system, different boundary. [cross-bearing: see Ch 13 §7 — *numbers nobody collects by default*]

5. No.

The mesh proxy sits in the network path and reports on what crosses it: which service called which, how long the call took, what status came back. Database queries *inside* a handler never cross the proxy, so the proxy has no knowledge of them at all.

This is the boundary between zero-code and code-based instrumentation. Zero-code is "great for getting started, or when you can't modify the application," providing telemetry "from libraries you use and/or the environment your application runs in" [source: opentelemetry-instrumentation-2026-08-31] — but seeing inside a request handler requires instrumentation *in* the handler. Answering this engineer's question means code-based instrumentation. [cross-bearing: see Ch 17 §5 — *a network that knows what it's carrying*]

How'd You Do?

5 correct: you have the densest section in the chapter. Keep going. **3–4 correct:** review the misses now rather than later. §8 pays off both the arrow and the span/trace containment, and it will land harder if

they're solid. **0–2 correct:** re-read §4's opening (the arrow) and §5's Fixed Point (span inside trace) before continuing. Those two are the most reliably tested ideas in this chapter.

Checkpoint: You've Now Mastered

✓ The direction of the arrow, and the one narrow exception to it ✓ Prometheus's components — a client library inside code you own, an exporter beside code you don't ✓ Where the project says Prometheus does *not* fit ✓ Span versus trace versus root span, as a containment relationship ✓ What mesh-generated telemetry can and cannot see

Two content sections left. The next one is the most mechanical in the chapter, and the one after it names the question all of this has been in service of.

..l §6 — Lines From Everywhere

Start with the limit, because everything downstream is a consequence of it.

Kubernetes provides no native storage for log data [source: k8s-docs-logging-architecture-2026-08-23]. Container logs are written to the node's filesystem by the container runtime, and the kubelet manages them there using the CRI logging format [source: k8s-docs-logging-architecture-2026-08-23], and that is the end of the platform's involvement. The platform carries your logs as far as the pier and no further. If you want logs that outlive the node, the Pod, or the rotation window, you need a separate backend, and something to get the lines to it.

That is what cluster-level logging is: an architecture bolted alongside Kubernetes, not a feature inside it.

Why `kubectl logs` is not an archive

You have used `kubectl logs` since Chapter 13. [cross-bearing: see Ch 13 §3 — looking inside] It is a diagnostic, and it is bounded in three separate ways.

Rotation. The kubelet rotates container log files. The defaults are `containerLogMaxSize` of 10Mi and `containerLogMaxFiles` of 5, and critically, **only the contents of the latest log file are available through `kubectl logs`** [source: k8s-docs-logging-architecture-2026-08-23]. A chatty container blows through 10Mi in minutes, and the lines you wanted are in a rotated file the command will not read.

Restarts. If a container restarts, the kubelet keeps one terminated container with its logs, which is what `kubectl logs --previous` retrieves [source: k8s-docs-logging-architecture-2026-08-23]. One. Not three, not "since the Pod was created."

Eviction. "If a pod is evicted from the node, all corresponding containers are also evicted, along with their logs" [source: k8s-docs-logging-architecture-2026-08-23]. [cross-bearing: see Ch 13 §4 — pods that

start and then don't stay] The Pod that got OOMKilled and evicted at 3 a.m. took the evidence with it.

That last one is the reason the whole architecture exists. Every other limit is inconvenient; this one means the exact failure you most want to investigate is the one that most reliably destroys its own logs.

Navigational Hazards

The mistake is treating `kubectl logs` as a log store because it is the log command you know. It is a live-tail-and-recent-history diagnostic scoped to one container on one node. Any scenario involving *yesterday, across all replicas, searching for a pattern, or a Pod that no longer exists* is describing cluster-level logging, and `kubectl logs` is the wrong answer.

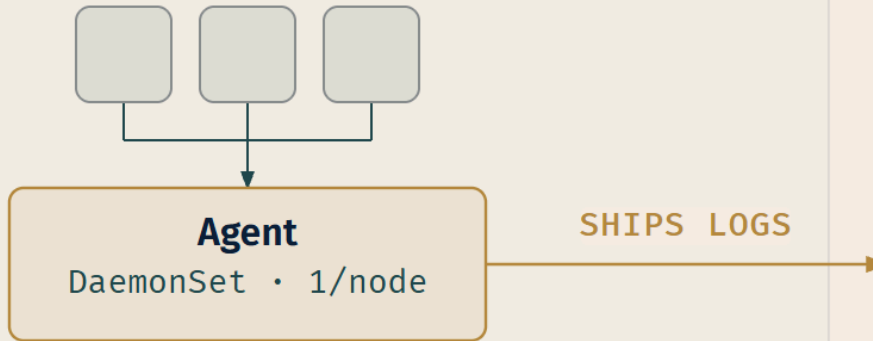
The three architectures

The Kubernetes documentation describes three approaches to getting logs off a node and into a backend [source: [k8s-docs-logging-architecture-2026-08-23](#)].

1 • NODE-LEVEL AGENT

the default answer

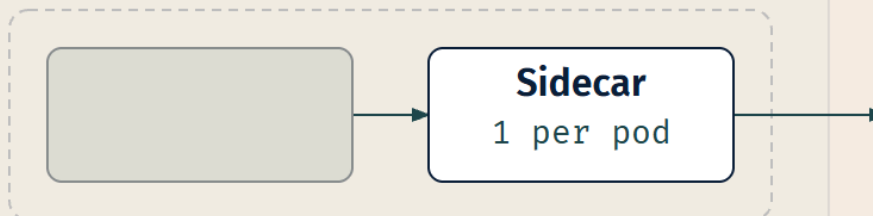
3 pods write to one node log file



2 • SIDECAR IN THE POD

one collector per pod

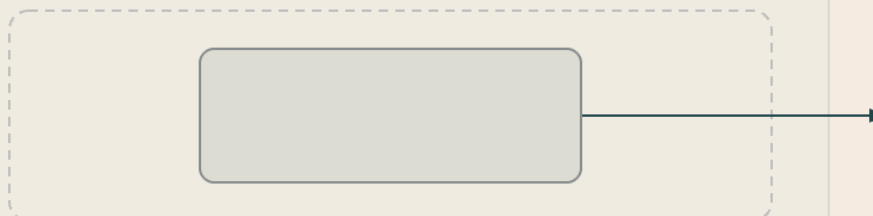
cost multiplies with pod count



3 • APP PUSHES DIRECTLY

no collector at all

app must know the backend



Logging Backend
outside k8s

1. A node-level logging agent that runs on every node, typically as a DaemonSet [source: k8s-docs-logging-architecture-2026-08-23], reading the container log files the runtime already writes and forwarding them to a backend. This is the default answer, and the reason is structural: the logs are already on the node's filesystem, for every container the node runs, whether or not the application cooperated. One agent covers everything.

The workload resource for "exactly one Pod on every node" is the **DaemonSet** [cross-bearing: see Ch 6 §7 — one per node, and work that ends], and node-level log collection is its canonical example. When a new node joins the cluster, it gets the agent automatically, because that is what a DaemonSet does.

2. A dedicated sidecar container for logging in the application Pod [source: k8s-docs-logging-architecture-2026-08-23]. This comes in two shapes: a streaming sidecar that reads the application's output and writes it to its own stdout/stderr, or a sidecar running a logging agent configured to pick logs up from the application container [source: k8s-docs-logging-architecture-2026-08-23]. Either is useful when an application writes to a file inside the container rather than to stdout, or needs per-application processing. The cost is one extra container per Pod rather than one per node, which on a node running forty Pods is a meaningfully different resource bill.

3. The application pushes logs directly to a backend from within the application [source: k8s-docs-logging-architecture-2026-08-23]. No collector at all; the app knows the backend's address and speaks its protocol. Simple, and it couples your application code to your logging vendor.

🔒 **Worth Securing:** "Which architecture?" almost always answers **node-level agent as a DaemonSet**, and the discriminator is worth stating as a rule: node-level collection is the only option that works *without the application's cooperation*. That is exactly why platform teams choose it. The platform team does not control what forty application teams write, and cannot make forty deploys to fix logging.

Fluentd and Fluent Bit

Two agents dominate this slot, and the exam-adjacent detail is their relationship.

Fluentd is a CNCF graduated project. Its design idea is the unified logging layer: it "tries to structure data as JSON as much as possible: this allows Fluentd to unify all facets of processing log data" [source: fluentd-architecture-2026-08-31], and it "connects dozens of data sources and data outputs" [source: fluentd-architecture-2026-08-31] through "a flexible plugin system that allows the community to extend its functionality" [source: fluentd-architecture-2026-08-31]. Fluentd was accepted into the CNCF in November 2016 and graduated in 2019 [source: fluentd-architecture-2026-08-31].

Fluent Bit is "an open source telemetry agent that processes logs, metrics, traces, and profiles," created in 2014 by Eduardo Silva "as a lightweight log processor, developed by the Fluentd team at Treasure Data for constrained environments such as embedded Linux"; it is "a sub-project of Fluentd" [source: fluent-bit-overview-2026-08-23].

Why two? Footprint. A vanilla Fluentd instance "runs on 30-40MB of memory" [source: fluentd-architecture-2026-08-31]: trivial on a server, less trivial multiplied across every node in a large fleet, and genuinely limiting on constrained hardware. Fluent Bit is the lightweight answer. Both "are commonly deployed on Kubernetes as node-level logging agents (DaemonSets) that collect container logs from each node and forward them to a backend" [source: fluent-bit-overview-2026-08-23].

🔍 **Closer Look — what a log agent actually does.** Fluent Bit's data pipeline runs six stages in order: **input** plugins gather from sources such as log files and OS metrics; a **parser** converts unstructured data to structured; **filters** alter the data before delivery; a **buffer** stores it in memory or on the filesystem; a **router** directs it through filters to one or more destinations using tags and matching rules; and **output** plugins define those destinations [source: fluent-bit-overview-2026-08-23]. This is deployment-level detail, not exam surface. It is here because it makes concrete what happens between reading a file off a node and writing it to a backend.

🧠 **Mnemonic: Fluentd** is one word. **Fluent Bit** is two. The parent is a single compound; the lightweight child is "Fluent" plus a "Bit" of it. That asymmetry looks like a typo and is not, and a question that renders one of them wrong is testing whether you noticed.

📌 **Snag:** Fluentd is CNCF graduated *as of the source cached for this book*. Project maturity levels change, and a question asking which projects are *currently* graduated is asking about a moving roster rather than a durable fact. What is durable and worth knowing: Fluentd is the CNCF project, Fluent Bit is its lighter sub-project, and both serve as node-level agents. [cross-bearing: see Ch 17 §2 — *sandbox, incubating, graduated, and who decides*]

..1 §7 — Is the Service Doing What Users Expect

Six sections of instruments. Here is the question they exist to answer.

Reliability answers the question: "Is the service doing what users expect it to be doing?" [source: opentelemetry-observability-primer-2026-08-23]

That is the whole thing. Not "is CPU below 80%." Not "are all Pods Running." Not "did the probes pass." Every one of those can be true while the service is failing the people using it, and every one of them can be false while users are perfectly happy.

The rest of this section is vocabulary for making that question answerable: turning "is it doing what users expect" from a feeling into a number somebody can be held to.

Three letters that get swapped

Take them in dependency order, because the dependency order is the memory hook.

An SLI — Service Level Indicator — "represents a measurement of a service's behavior. A good SLI measures your service from the perspective of your users" [source: opentelemetry-observability-primer-

2026-08-23]. Google's definition is complementary: "a carefully defined quantitative measure of some aspect of the level of service that is provided" [source: sre-book-service-level-objectives-2026-08-31].

The user-perspective clause is the part with teeth. "Average CPU across the fleet" is a measurement, and it is not an SLI in any useful sense, because no user has ever cared about it. "The proportion of checkout requests that completed successfully in under 400ms" is an SLI. The difference is whose experience is being measured.

An SLO — Service Level Objective — "is the means by which reliability is communicated to an organization/other teams. This is accomplished by attaching one or more SLIs to business value"

[source: opentelemetry-observability-primer-2026-08-23]. Or, mechanically: "a target value or range of values for a service level that is measured by an SLI" [source: sre-book-service-level-objectives-2026-08-31].

The SLI is the measurement. The SLO is the *target you commit to* on that measurement. 99.5% of checkout requests under 400ms, over a rolling 30 days — that is an SLO, and it takes an SLI as its input.

☆ Fixed Point

An SLI is the MEASUREMENT. An SLO is the OBJECTIVE — a target value for that measurement. The SLI is a number you observe; the SLO is a number you commit to.

One term travels with the SLO and is worth a clause. An **error budget** is the unreliability the objective leaves room for, and its value is organizational rather than mechanical: "as long as the uptime measured is above the SLO — in other words, as long as there is error budget remaining — new releases can be pushed" [source: sre-book-error-budgets-2026-08-31]. It exists because of a structural tension: "Product development performance is largely evaluated on product velocity, which creates an incentive to push new code as quickly as possible. Meanwhile, SRE performance is evaluated based upon reliability of a service, which implies an incentive to push back against a high rate of change" [source: sre-book-error-budgets-2026-08-31]. The budget is the referee: a number both sides agreed to in advance, in place of an argument about judgment.

An SLA — Service Level Agreement — is the third term, and it is here mostly because it is the distractor. SLAs are "an explicit or implicit contract with your users that includes consequences of meeting (or missing) the SLOs they contain" [source: sre-book-service-level-objectives-2026-08-31]. External, contractual, with teeth.

The discrimination has a procedure rather than requiring you to hold two definitions side by side: "An easy way to tell the difference between an SLO and an SLA is to ask 'what happens if the SLOs aren't met?': if there is no explicit consequence, then you are almost certainly looking at an SLO" [source: sre-book-service-level-objectives-2026-08-31].

🔗 **Mnemonic:** **I** for Indicator — what you *measure*. **O** for **O**bjective — what you *aim at*. **A** for **A**greement — what you *sign*, with consequences attached. And the containment runs the same direction as the letters: an SLA contains SLOs, which are measured by SLIs.

📌 **Snag:** SLI and SLO get swapped constantly, in the wild and on exams. When a question describes *a number being observed*, that is the SLI. When it describes *a threshold being committed to*, that is the SLO. When it mentions penalties, credits, or a customer contract, that is the SLA.

The four golden signals

If you can instrument only four things on a user-facing system, these are the four [source: sre-book-four-golden-signals-2026-08-23]:

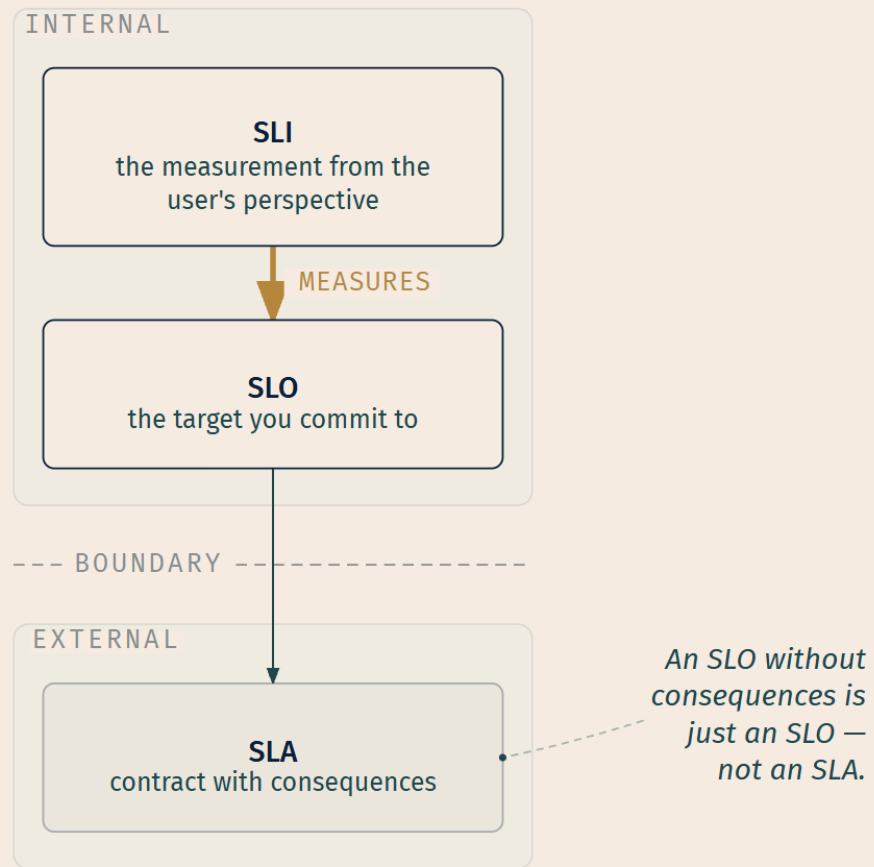
Latency — "the time it takes to service a request." With one crucial refinement: "It's important to distinguish between the latency of successful requests and the latency of failed requests." An HTTP 500 caused by a dropped database connection "might be served very quickly," so folding 500s into overall latency "might result in misleading calculations." And, memorably: "a slow error is even worse than a fast error!" [source: sre-book-four-golden-signals-2026-08-23]

Traffic — "a measure of how much demand is being placed on your system, measured in a high-level system-specific metric." Usually HTTP requests per second for a web service; network I/O rate or concurrent sessions for audio streaming; transactions and retrievals per second for a key-value store [source: sre-book-four-golden-signals-2026-08-23].

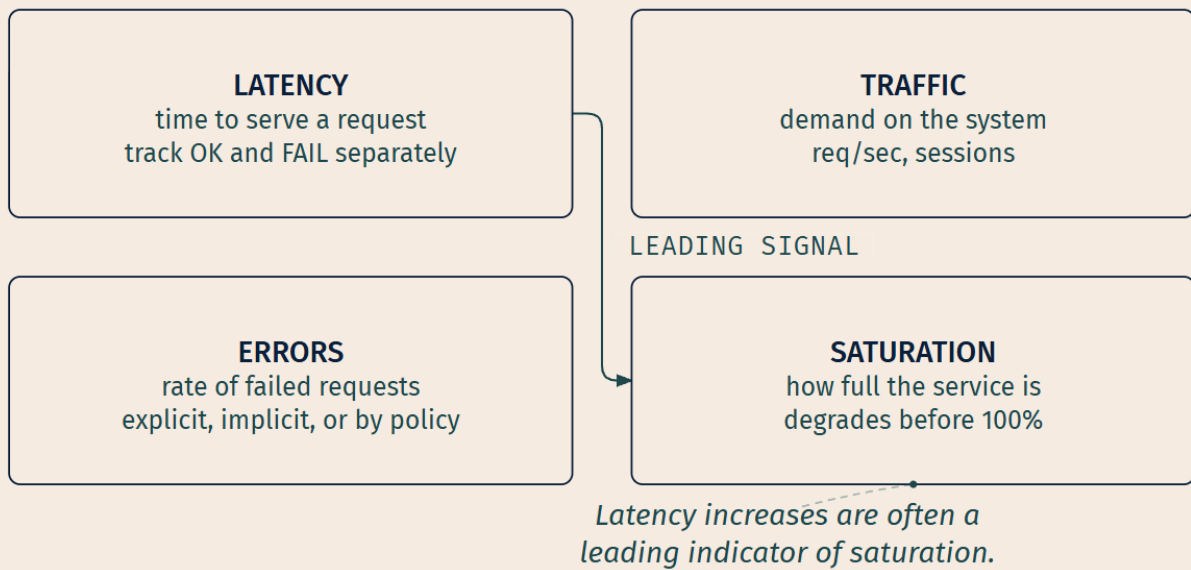
Errors — "the rate of requests that fail, either explicitly (e.g., HTTP 500s), implicitly (for example, an HTTP 200 success response, but coupled with the wrong content), or by policy (for example, 'If you committed to one-second response times, any request over one second is an error')" [source: sre-book-four-golden-signals-2026-08-23].

Saturation — "how 'full' your service is. A measure of your system fraction, emphasizing the resources that are most constrained." And the operational warning: "many systems degrade in performance before they achieve 100% utilization, so having a utilization target is essential" [source: sre-book-four-golden-signals-2026-08-23].

MEASUREMENT → COMMITMENT → CONTRACT



THE FOUR GOLDEN SIGNALS



🔊 **Mnemonic: L-T-E-S.** Latency, Traffic, Errors, Saturation. Or read the figure as a sentence: *how long, how much, how broken, how full.*

And the one relationship among the four that is worth carrying: **"Latency increases are often a leading indicator of saturation"** [source: sre-book-four-golden-signals-2026-08-23]. Response times creeping up before anything looks full is the system telling you it is about to be. Treat that as the early warning it is.

RED and USE

Two other framings appear alongside the golden signals, and both are named in the CNCF TAG Observability whitepaper [source: cncf-tag-observability-whitepaper-2026-08-31]. Their value here is the contrast between them, not their details.

USE — "the Utilization Saturation and Errors (USE) Method is a methodology for analyzing the performance of any system," directing "the construction of a checklist, which for server analysis can be used for quickly identifying resource bottlenecks or errors" [source: use-method-brendan-gregg-2026-08-31]. It is oriented at **resources**.

RED — Rate, Errors, Duration: "the number of requests per second," "the number of those requests that are failing," and "the amount of time those requests take" [source: red-method-tom-wilkie-2026-08-31]. It is oriented at **services**.

The person who created RED drew the line himself: "The USE Method doesn't really apply to services; it applies to hardware, network disks, things like this. We really wanted a microservices-oriented monitoring philosophy, so we came up with the RED Method" [source: red-method-tom-wilkie-2026-08-31].

That is the whole complementarity, and note what produced it. RED exists *because* one service became twenty. [cross-bearing: see Ch 17 §3 — *small pieces, replaced whole*] The architecture changed, and the measurement framing had to change with it, which is this chapter's thesis showing up in the methodology literature.

🔍 **Closer Look — one word, three meanings.** "Utilization" now means three different things in this chapter, and all three are correct in their own context. In §3, utilization is **a percentage of the containers' resource request** [source: k8s-docs-hpa-utilization-vs-requests-2026-08-31]. In the USE method, utilization is "the average time that the resource was busy servicing work" [source: use-method-brendan-gregg-2026-08-31], a duration fraction. And the golden-signals concept nearest to both is **saturation**, how full the service is [source: sre-book-four-golden-signals-2026-08-23]. When you meet the word in a question, check which system is speaking.

🕒 Taking Your Bearings: Lines, Signals, and the Question They Answer

Five questions. One draws on Chapter 6.

1. A Pod has restarted four times over the past hour. You need to see what the container printed before its *first* crash. Can you get it with `kubectl logs`? Explain, naming the specific mechanisms involved.
2. Name the three cluster-logging architectures. Which is the default answer for a platform team, and what is the structural reason?
3. A team commits: "99.9% of API requests will complete successfully within 300ms, measured over a rolling 30 days. If we miss this in a calendar quarter, affected enterprise customers receive a 10% service credit." Identify the SLI, the SLO, and the SLA in that statement.
4. Name the four golden signals. For one of them, state a relationship it has to another that makes it useful as an early warning.
5. *[retrieval: ch6]* Your log-collection agent must run on every node in the cluster, including nodes added tomorrow. Which workload resource, and why not a Deployment with `replicas` set to the node count?

Answers with Explanations:

1. No, on two independent grounds, and you should be able to name both.

Rotation: the kubelet rotates container logs, and only the contents of the latest log file are available through `kubectl logs` [source: k8s-docs-logging-architecture-2026-08-23]. Even within a single container run, older output can already be out of reach. (The specific defaults are in §6; you are not expected to recite them.)

Restart depth: if a container restarts, the kubelet keeps one terminated container with its logs, which is what `kubectl logs --previous` retrieves [source: k8s-docs-logging-architecture-2026-08-23]. One restart back, not four.

Four restarts ago is gone from the node. Retrieving it requires cluster-level logging: an agent that shipped those lines to a backend before the node discarded them. *[cross-bearing: see Ch 13 §3 — looking inside]*

2. The three: **a node-level logging agent** (typically a DaemonSet), **a dedicated sidecar container** in the application Pod, and **the application pushing directly** to a backend [source: k8s-docs-logging-architecture-2026-08-23].

The default is the **node-level agent**, and the structural reason is that it works *without the application's cooperation*. Container logs are already written to the node's filesystem for every container the node runs, whether the application team did anything or not [source: k8s-docs-logging-architecture-2026-08-23]. One agent per node covers all of them, including workloads owned by teams the platform team has never spoken to. Sidecars require a change to every Pod spec; direct push requires a change to every application.

- 3.

- **SLI:** the proportion of API requests completing successfully within 300ms. This is the measurement, and it is user-perspective: it measures what a caller experiences [source: opentelemetry-observability-primer-2026-08-23].
- **SLO:** 99.9% of them, over a rolling 30 days. The target value attached to the SLI [source: sre-book-service-level-objectives-2026-08-31].
- **SLA:** the 10% service credit. Apply the test — "what happens if the SLOs aren't met?" [source: sre-book-service-level-objectives-2026-08-31] There is an explicit consequence, so this clause is the agreement, not the objective.

4. Latency, traffic, errors, saturation [source: sre-book-four-golden-signals-2026-08-23].

The relationship worth naming: **"Latency increases are often a leading indicator of saturation"** [source: sre-book-four-golden-signals-2026-08-23]. Because many systems degrade before reaching 100% utilization, rising response times frequently appear *before* any saturation metric looks alarming.

(Also acceptable: latency's relationship to errors — failed-request latency must be tracked separately, because a fast 500 makes overall latency look good while "a slow error is even worse than a fast error" [source: sre-book-four-golden-signals-2026-08-23].)

5. A DaemonSet.

A DaemonSet's semantics are "one Pod per node," maintained by the controller. When a node joins the cluster, it gets a Pod automatically; when a node leaves, the Pod goes with it. [*cross-bearing: see Ch 6 §7 — one per node, and work that ends*] Kubernetes names this shape directly for logging: a node-level agent runs on every node, typically as a DaemonSet [source: k8s-docs-logging-architecture-2026-08-23].

A Deployment with `replicas: 12` gives you twelve Pods, placed by the scheduler according to feasibility and scoring. Nothing guarantees one per node; the scheduler may well put two on a roomy node and none on a constrained one [*cross-bearing: see Ch 7 §1 — one decision, made once*]. And the count is a number you set, which is stale the moment the cluster scales. Log collection is a per-node job, so it needs the resource whose contract is per-node.

How'd You Do?

5 correct: you have the whole chapter. §8 introduces nothing; read it for the shape, not for facts. **3–4 correct:** review your misses, then read §8 — it re-frames exactly the material you just tested. **0–2 correct:** re-read §6's three architectures and §7's SLI → SLO → SLA chain before the Practice Questions. Six of the seventeen depend on those two sections, and §8 will not re-teach them.

Checkpoint: You've Now Mastered

✓ Why `kubect1 logs` is a diagnostic and not an archive — rotation, restart depth, eviction ✓ The three cluster-logging architectures, and why one is the default ✓ SLI, SLO, and SLA, with a procedure for telling

the last two apart ✓ The four golden signals, and saturation's early warning

One section left. It introduces nothing.

☼ §8 — One Question, Four Instruments

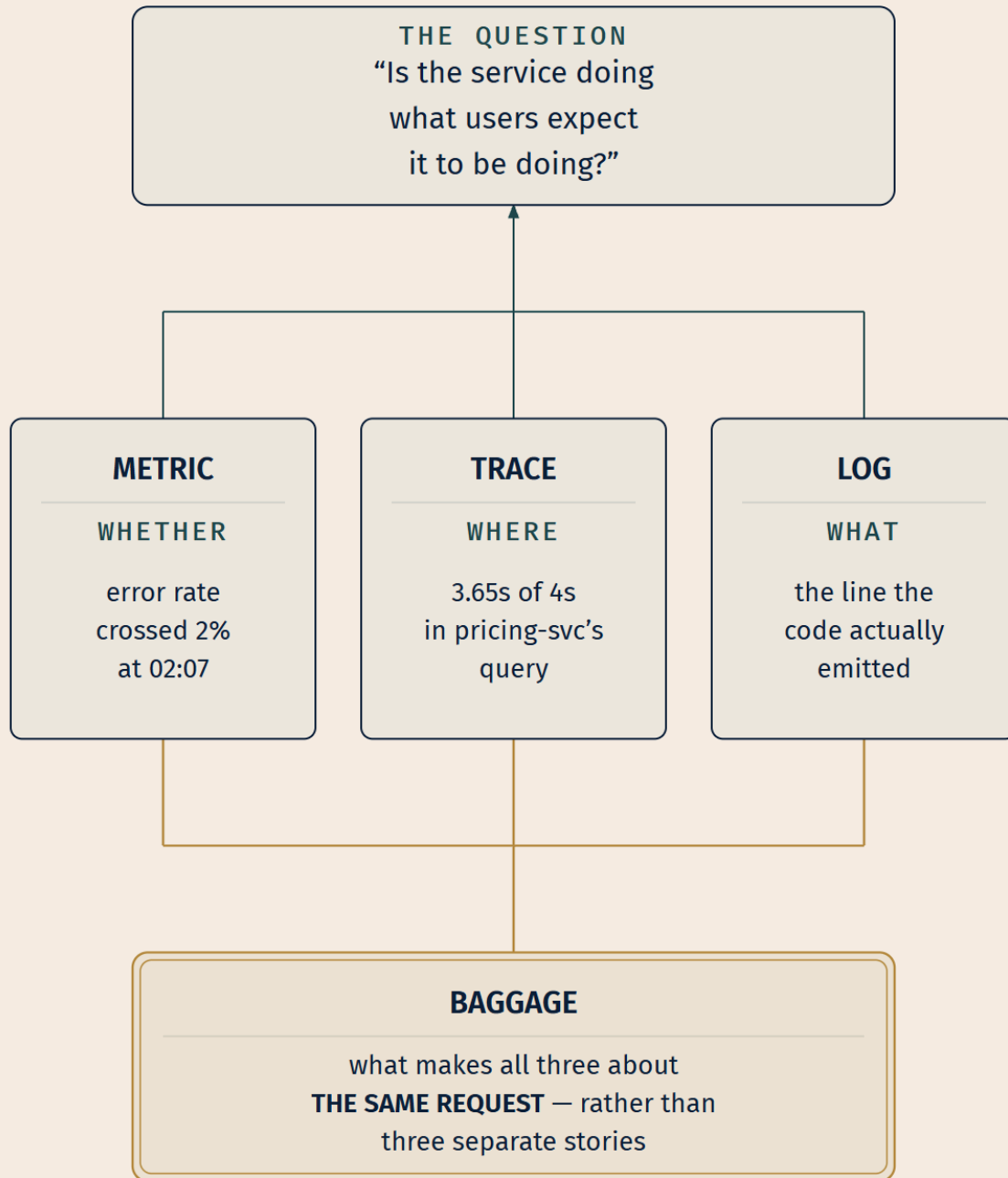
You have four instruments now. Here is the thing nobody said while you were collecting them.

They are not four topics. They are four ways of asking §7's one question.

Is the service doing what users expect it to be doing? [source: opentelemetry-observability-primer-2026-08-23]

Ask it of a **metric** and you get *whether*: a number over time, aggregated, that tells you the error rate crossed 2% at 02:07 and is still climbing. Ask it of a **trace** and you get *where*: the request took four seconds and 3.65 of them were in one database call inside pricing-svc. Ask it of a **log** and you get *what*: the specific line the code emitted at the moment it gave up, with the exception and the parameters. And **baggage** is what lets the first three be about the same request rather than three unrelated stories that happened to occur at the same time.

That is the payoff of the fourth signal, and it is why dropping it from the list is worse than a memory slip. Without something carrying context across the boundary, you have three readings of three different things. With it, you have one question examined at three resolutions.



Not four topics. One question, four resolutions.

LEGEND

→ RESOLVES THE QUESTION

— JOINED BY BAGGAGE

The other thing that kept happening

There is a second pattern in this chapter, quieter than the first, and you have now seen it four times.

OpenTelemetry exports; Jaeger receives. Prometheus stores; Grafana reads, through an HTTP API [source: [prometheus-promql-basics-2026-08-31](#)]. The container runtime writes container logs to the node [source: [k8s-docs-logging-architecture-2026-08-23](#)]; an agent ships them somewhere else. The OTel Collector receives one set of signals and exports to "one or more open source or commercial backends" [source: [opentelemetry-collector-2026-08-31](#)].

Every one of those is the same shape: **the thing that produces telemetry and the thing that stores it are separate, and either can be replaced without the other noticing**. Swap Jaeger for a commercial tracing vendor and your instrumented services do not change. Swap Fluentd for Fluent Bit and your applications do not change. Point the Collector at a different backend and nothing upstream of it knows.

You have met this argument before, in a domain you would not have expected it in. [*cross-bearing: see Ch 17 §4 — every place Kubernetes lets you in*] Chapter 17 made the case that Kubernetes' durability comes from defining interfaces rather than implementations, so that pieces can be replaced without the whole being redesigned. It turns out the observability ecosystem was built on the same conviction, by different people, for different reasons, arriving at the same architecture. That is not a coincidence to file away as trivia. It is the reason both ecosystems are still standing.

Where that leaves you

Chapter 17 handed you a bill: everything that made the system easier to change made it harder to see.

This chapter is the payment, and it is not a set of tools. It is a *disposition*, the habit of asking, before an incident rather than during one, *what would I need to be emitting for this question to have an answer?* The instruments follow from that. The dashboards follow from the instruments. Nothing works in the other direction, which is why teams who start by buying a dashboard product are so often the ones who cannot explain their own outages.

You know what the four signals are. You know which one tells you whether, which tells you where, which tells you what, and which makes them agree on the subject. You know that Prometheus reaches out rather than being reported to, that a trace is the whole path and a span is one piece of it, that logs leave the node by a route Kubernetes does not provide, and that all of it is in service of a single sentence about users.

The question was never "do we have monitoring." The question is: *is the service doing what users expect?* — and now you can find out.

Exam Alert! 🚨

High-Priority Topics:

1. **The four signals.** Traces, metrics, logs, **baggage** — OpenTelemetry's own count. The number is what gets tested; baggage is what gets dropped.
2. **Prometheus pulls.** It scrapes over HTTP. Push exists only via the Pushgateway, only for service-level batch jobs.
3. **Span vs trace.** A span is one unit of work. A trace is the whole path — one or more spans, starting at the root span.
4. **SLI vs SLO.** Measurement versus objective. SLA is the third term and the usual distractor.
5. **Observability vs monitoring.** New questions versus pre-chosen indicators. Not "dashboards versus no dashboards."
6. **metrics-server is not a monitoring system.** The exam tests the boundary, not either side.

Common Traps — these are distinctions that are easy to confuse, and they are the ones this material rewards getting right:

The trap	The correct understanding
"Observability is monitoring with better dashboards"	Observability handles unknown unknowns — questions you did not plan for. Monitoring detects known unknowns [source: cncf-tag-observability-whitepaper-2026-08-31]
Naming three signals	Four , on OpenTelemetry's Signals page. Baggage is contextual information passed <i>between</i> signals [source: opentelemetry-signals-2026-08-23]
Using "span" and "trace" interchangeably	One unit of work versus the whole request path [source: opentelemetry-observability-primer-2026-08-23]
"Logs are the richest signal, so lead with them"	Logs "are not extremely useful for tracking code execution on their own, as they typically lack contextual information" [source: opentelemetry-observability-primer-2026-08-23]
SLI and SLO swapped	SLI measures ; SLO commits . If there's a consequence attached, it's an SLA [source: sre-book-service-level-objectives-2026-08-31]
"Prometheus pushes" / "apps report to Prometheus"	It scrapes over HTTP . Pushgateway is an intermediary for jobs too short to scrape [source: prometheus-pushgateway-practices-2026-08-31]
"Prometheus for per-request billing"	Explicitly not — "if you need 100% accuracy... Prometheus is not a good choice" [source: prometheus-overview-2026-08-23]
"Prometheus needs clustered/network storage"	Each server is standalone by design , so it works "when other parts of your infrastructure are broken" [source: prometheus-overview-2026-08-23]
"Prometheus was CNCF's first project"	Second , in 2016. Kubernetes was first [source: prometheus-overview-2026-08-23]
Expecting Observability as a standalone domain	It is competency material inside Cloud Native Architecture [source: lf-kcna-program-changes-2026-08-23]. <i>[cross-bearing: see Ch 1 — the curriculum that moved under everyone's feet]</i>
kubect1 logs as a log archive	Rotation, one-restart depth, and eviction all bound it [source: k8s-docs-logging-architecture-2026-08-23]. Anything historical needs cluster-level logging

Practice Questions

Seventeen questions. Three are interleaved with earlier chapters and tagged. Explanations follow all seventeen; resist the urge to scroll.

Q1. Which statement best captures the difference between monitoring and observability?

- A) Observability is what you get when you add distributed tracing to your monitoring stack
- B) Monitoring detects known unknowns; observability additionally lets you find and reason about unknown unknowns
- C) Monitoring covers infrastructure; observability covers applications

- D) Monitoring is real-time; observability is historical
-

Q2. By OpenTelemetry's stated definition, an application is properly instrumented when:

- A) It emits at least the four golden signals
 - B) It has been onboarded to a CNCF-graduated observability backend
 - C) Developers do not need to add more instrumentation to troubleshoot an issue, because they have the information they need
 - D) It emits all four OpenTelemetry signals, including baggage
-

Q3. Which is the complete list of signals OpenTelemetry currently supports?

- A) Traces, metrics, logs
 - B) Traces, metrics, logs, baggage
 - C) Metrics, logs, traces, profiles, dumps
 - D) Traces, metrics, logs, alerts
-

Q4. Baggage is best described as:

- A) An attribute automatically attached to every span in a trace
 - B) A separate key-value store propagated between services, carrying contextual data that the other signals can incorporate
 - C) The buffered queue of telemetry awaiting export by the Collector
 - D) The metadata section of a log record
-

Q5. A dashboard shows a Pod at "85% CPU utilization." That percentage is calculated against:

- A) The node's total allocatable CPU
 - B) The container's CPU limit
 - C) The equivalent CPU resource request on the containers in the Pod
 - D) The namespace's ResourceQuota for CPU
-

Q6. *[retrieval: ch5]* A Deployment's Pod template declares no CPU requests. An HPA targets 60% CPU utilization for that Deployment. What happens?

- A) The HPA scales against the node's allocatable CPU, divided by Pod count
- B) The HPA treats the Pod as 0% utilized and scales it down to the minimum replica count
- C) CPU utilization is undefined for the Pod and the HPA takes no action on that metric
- D) The Pod is rejected at admission until a request is set

Q7. *[retrieval: ch13]* Your team needs to answer: "What was the memory usage of the payments Deployment at 3 a.m. last Tuesday, and did it correlate with the error spike?" The cluster runs metrics-server. What is required?

- A) Nothing further; `kubectl top` with a timestamp flag answers this
 - B) A monitoring system with time-series storage — metrics-server retains no history and does not collect application error metrics
 - C) Increase metrics-server's retention configuration
 - D) Enable the metrics-server aggregation layer for historical queries
-

Q8. How does the Prometheus server obtain metrics from an instrumented, long-running application?

- A) The application POSTs metrics to Prometheus on an interval
 - B) Prometheus scrapes an HTTP endpoint on the application, on an interval
 - C) An exporter running alongside the application pushes the application's metrics to the Prometheus server
 - D) The kubelet forwards metrics to Prometheus via the CRI
-

Q9. Which scenario is the documented, valid use of the Pushgateway?

- A) A high-traffic web API that prefers outbound connections to inbound ones
 - B) A nightly database-maintenance batch job, not tied to a specific machine, whose outcome must be recorded
 - C) A stateful application requiring exactly-once metric delivery
 - D) Any workload behind a firewall Prometheus cannot reach
-

Q10. A company needs an exact count of every API call for per-customer invoicing. What does the Prometheus documentation say about this use case?

- A) It is well suited; counters are the canonical Prometheus use case
 - B) It becomes suitable once the scrape interval is reduced to one second
 - C) It is explicitly a poor fit — Prometheus does not guarantee 100% accuracy, naming per-request billing as an example
 - D) It is suitable if the Pushgateway is used for every request
-

Q11. Which statement about Prometheus's storage architecture is correct?

- A) It requires a clustered backend such as Cassandra for durability

- B) Each server is standalone and does not depend on network storage or other remote services, so it works when other infrastructure is broken
 - C) It uses etcd, sharing the cluster's control-plane datastore
 - D) It stores data in the Kubernetes API server as custom resources
-

Q12. Which correctly describes the relationship between traces and spans?

- A) A span contains one or more traces; the first trace is the root trace
 - B) A trace contains one or more spans; the first span is the root span, representing the request from start to finish
 - C) Traces and spans are synonyms in OpenTelemetry
 - D) A trace contains exactly one span per service the request touches, never more
-

Q13. A request crosses six microservices. How do the spans emitted by service six come to be associated with the trace begun at service one?

- A) The tracing backend correlates them by comparing timestamps after ingestion
 - B) Each service includes a trace ID and span ID in the context passed to the next, which uses those values to create a span belonging to the same trace
 - C) Each service queries a central trace registry for the active trace
 - D) The service mesh control plane assigns trace IDs from a shared pool
-

Q14. *[retrieval: ch6]* A logging agent must run on every node, including nodes added after deployment. Which workload resource is correct?

- A) A Deployment with `replicas` equal to the current node count
 - B) A StatefulSet with one ordinal per node
 - C) A DaemonSet
 - D) A CronJob that reconciles node coverage every five minutes
-

Q15. Which correctly describes the relationship between Fluentd and Fluent Bit?

- A) Fluent Bit is a fork of Fluentd maintained by a different foundation
 - B) Fluent Bit is a lightweight telemetry agent and a sub-project of Fluentd, created for constrained environments; both are commonly deployed as node-level agents
 - C) Fluentd is the lightweight agent; Fluent Bit is the full-featured aggregator
 - D) They are unrelated projects that happen to share a name prefix
-

Q16. A team commits: "99.95% of search requests will return in under 200ms, measured monthly." No penalty is attached. This statement is:

- A) An SLA, because it is a public commitment
 - B) An SLI, because it describes a measurement
 - C) An SLO — a target value on an SLI, with no explicit consequence attached
 - D) An error budget
-

Q17. Which are the four golden signals of monitoring?

- A) Rate, errors, duration, saturation
 - B) Latency, traffic, errors, saturation
 - C) Utilization, saturation, errors, latency
 - D) Traces, metrics, logs, baggage
-

Answers and Explanations

Q1 — B.

- **A** is wrong, and it is the tool-acquisition misconception: the belief that observability is a product you bolt on. Adding traces to a stack nobody can query freely produces more telemetry, not more observability. The posture is upstream of the tool — a Prometheus deployment with rich labels and an ad-hoc query language is doing observability work, and a pile of unqueryable traces is not.
- **B** is correct. "Monitoring is called a system that can detect known unknowns — as opposed to observability which emphasizes being able to find and reason about unknown unknowns as well" [source: cncf-tag-observability-whitepaper-2026-08-31].
- **C** is wrong: the split is not by layer. You can monitor an application and observe infrastructure.
- **D** is wrong and inverts things if anything — monitoring is defined by "displaying **real-time** quantitative data" [source: sre-book-monitoring-definitions-2026-08-31], and observability is not restricted to historical questions.

Q2 — C. "An application is properly instrumented when developers don't need to add more instrumentation to troubleshoot an issue, because they have all of the information they need" [source: opentelemetry-observability-primer-2026-08-23].

- **A** is wrong: the golden signals are a monitoring prioritization heuristic, not an instrumentation completeness bar.
- **B** is wrong: a backend stores telemetry; it does not produce it. Instrumentation is upstream of any backend.
- **D** is tempting because it names the right four things, but emitting all four signals thinly still fails the test. The bar is *sufficiency for troubleshooting*, not signal-type coverage.

Q3 — B. OpenTelemetry's Signals page lists traces, metrics, logs, and baggage [source: opentelemetry-signals-2026-08-23].

- **A** is the three-signal answer commonly given. It is the OTel primer's *passing* list, not the Signals page's list.
- **C** is the CNCF TAG Observability whitepaper's five-signal enumeration — metrics, logs, traces, profiles and dumps [source: cncf-tag-observability-whitepaper-2026-08-31]. A genuinely different taxonomy from a different document, and the reason §2's Snag warned you to check *which* authority a question names. This one names OpenTelemetry.
- **D** is wrong: alerts are an output of a monitoring system acting on metrics, not a signal type.

Q4 — B. Baggage is "contextual information that is passed between signals" [source: opentelemetry-signals-2026-08-23] — a key-value store letting you "propagate any data you like alongside context" [source: opentelemetry-baggage-2026-08-31].

- **A** is precisely the misconception the OTel docs pre-empt: "baggage is a separate key-value store and is unassociated with attributes on spans, metrics, or logs without explicitly adding them" [source: opentelemetry-baggage-2026-08-31]. Adding baggage to a span is a deliberate act.
- **C** is wrong: buffering is a Collector implementation concern, not what baggage is.
- **D** is wrong: baggage propagates across service boundaries and is independent of any one log record.

Q5 — C. "The controller calculates the utilization value as a percentage of the equivalent resource request on the containers in each Pod" [source: k8s-docs-hpa-utilization-vs-requests-2026-08-31].

- **A** is wrong: node capacity is not the denominator for Pod utilization.
- **B** is the most common wrong answer, because limits are the more memorable number. Requests are what the Pod asked for, and requests are the denominator.
- **D** is wrong: a ResourceQuota caps namespace-wide consumption [*cross-bearing: see Ch 8 §3 — dividing a shared cluster*]; it is not an input to per-Pod utilization.

Q6 — C. "If some of the Pod's containers do not have the relevant resource request set, CPU utilization for the Pod will not be defined and the autoscaler will not take any action for that metric" [source: k8s-docs-hpa-utilization-vs-requests-2026-08-31].

- **A** is wrong: node allocatable is never the denominator for Pod CPU utilization.
- **B** is the sharpest distractor here, because "no data" and "zero" feel interchangeable and they are not. The controller does not compute a low value; it computes no value, and therefore takes no action *on that metric*. A Pod scaled to minimum and a Pod the autoscaler is ignoring look different in production and are graded differently on paper.
- **D** is wrong: a Pod with no resource requests is admitted normally. (A LimitRange may *default* a request at admission time [source: k8s-docs-limit-range-2026-08-24], a separate mechanism acting before the Pod exists, not the HPA improvising a denominator. [*cross-bearing: see Ch 8 §3 — dividing a shared cluster*])

Q7 — B. The question needs two things metrics-server does not provide: **history** and a **non-resource metric** (error counts). *[cross-bearing: see Ch 13 §7 — numbers nobody collects by default]*

- **A** is wrong: no such flag exists, and `kubectl top` reports current readings by design.
- **C** is wrong and is the trap for someone who thinks retention is a tuning knob. metrics-server is scoped to current resource readings for autoscaling; historical retention is not a setting it withholds.
- **D** is wrong: metrics-server registers with the aggregation layer to serve `metrics.k8s.io`; that is how it is reached, not a history feature.

Q8 — B. "Time series collection happens via a pull model over HTTP" [source: prometheus-overview-2026-08-23], with targets found by service discovery or static configuration.

- **A** is the single most common Prometheus misconception. Applications expose; Prometheus fetches.
- **C** inverts the exporter's arrow, which is worth being precise about because exporters *are* real and *are* how much of the ecosystem works. An exporter is "a binary running alongside the application you want to obtain metrics from" [source: prometheus-glossary-2026-08-31]: it **exposes** an endpoint that Prometheus scrapes, exactly as a client library does. It is also the wrong mechanism here, because an exporter is for software you did not write, and this application is instrumented.
- **D** is wrong: the CRI is the kubelet↔runtime boundary *[cross-bearing: see Ch 2 §4 — the container runtime interface]* and has nothing to do with Prometheus.

Q9 — B. "The only valid use case for the Pushgateway is for capturing the outcome of a service-level batch job," where service-level means "not semantically related to a specific machine or job instance" [source: prometheus-pushgateway-practices-2026-08-31]. A nightly maintenance job is exactly this: too short-lived to be scraped, and not tied to one instance.

- **A** is wrong: a long-running web API *can* be scraped, which disqualifies it. Preference is not a criterion.
- **C** is wrong: the Pushgateway "persists the most recent push" [source: prometheus-glossary-2026-08-31]; it is not a delivery-guarantee mechanism.
- **D** is wrong: network topology is a routing problem, not a Pushgateway use case. Its documentation recommends it only "in certain limited cases" [source: prometheus-pushgateway-practices-2026-08-31].

Q10 — C. "If you need 100% accuracy, such as for per-request billing, Prometheus is not a good choice as the collected data will likely not be detailed and complete enough" [source: prometheus-overview-2026-08-23] — the documentation's own example.

- **A** is the answer given by someone matching on "counting requests" and missing "exact."
- **B** targets the belief that sampling more often converges on completeness. It does not. Scraping observes *state* at intervals, not events; a counter that advanced four hundred times between two

scrapes is still one difference, however short the interval. The trade is deliberate — "Prometheus values reliability" [source: prometheus-overview-2026-08-23] — not a tuning oversight.

- **D** is wrong twice: the Pushgateway is for batch-job outcomes, not per-request events, and routing events through it would not make sampling exact.

Q11 — B. "No reliance on distributed storage — single server nodes are autonomous" [source: prometheus-overview-2026-08-23], and "each Prometheus server is standalone, not depending on network storage or other remote services, so you can rely on it when other parts of your infrastructure are broken" [source: prometheus-overview-2026-08-23]. Independence is a deliberate reliability choice.

- **A** inverts the design.
- **C** is wrong: Prometheus stores scraped samples locally and has nothing to do with etcd [*cross-bearing: see Ch 3 §2 — the control plane*].
- **D** is wrong: the API server stores API objects, not time series, and using it as a metrics store would be catastrophic for the control plane.

Q12 — B. "A trace is made of one or more spans. The first span represents the root span; each root span represents a request from start to finish" [source: opentelemetry-observability-primer-2026-08-23].

- **A** inverts the containment, which is exactly the confusion the question tests.
- **C** is wrong: they name different things — one unit of work versus the whole path.
- **D** is wrong on "exactly one per service." A single service commonly emits several spans for one request (the handler, a database call, a cache lookup), and "one or more" is the documented relationship.

Q13 — B. "Service A includes a trace ID and a span ID as part of the context. Service B uses these values to create a new span that belongs to the same trace" [source: opentelemetry-context-propagation-2026-08-31]. This is context propagation, which "allows traces to build causal information about a system across services that are arbitrarily distributed across process and network boundaries" [source: opentelemetry-context-propagation-2026-08-31].

- **A** is the *pre-tracing* approach and its failure is why tracing exists. Timestamp correlation across six independently clocked services under concurrent load is unreliable at best.
- **C** is wrong: there is no central registry; the correlation rides in-band with the request.
- **D** is wrong: a mesh may *inject* headers, but the mechanism is still in-band propagation of a trace ID with the request, not central assignment from a pool.

Q14 — C. A DaemonSet's contract is one Pod per node, maintained as nodes join and leave. [*cross-bearing: see Ch 6 §7 — one per node, and work that ends*] Kubernetes names this shape for logging directly: a node-level agent runs on every node, typically as a DaemonSet [source: k8s-docs-logging-architecture-2026-08-23].

- **A** is wrong on two counts: the scheduler does not guarantee one Pod per node, and the replica count goes stale the moment the cluster scales.

- **B** is wrong: StatefulSets provide stable ordinal identity [*cross-bearing: see Ch 6 §6 — when Pods are not interchangeable*], not per-node placement.
- **D** is wrong: a CronJob runs work that ends. Log collection is continuous, and reconciling coverage on a timer reinvents, badly, what a controller already does.

Q15 — B. Fluent Bit is "an open source telemetry agent," created in 2014 "as a lightweight log processor, developed by the Fluentd team at Treasure Data for constrained environments such as embedded Linux"; it is "a sub-project of Fluentd," and both "are commonly deployed on Kubernetes as node-level logging agents (DaemonSets)" [source: fluent-bit-overview-2026-08-23].

- **A** is wrong: sub-project, not fork, and both sit under the CNCF.
- **C** inverts them. Fluentd is the parent and heavier — "30-40MB of memory" for a vanilla instance [source: fluentd-architecture-2026-08-31].
- **D** is wrong: the shared prefix reflects a real parent-child relationship.

Q16 — C. A target value on a measurement, with no consequence attached. Apply the test: "ask 'what happens if the SLOs aren't met?': if there is no explicit consequence, then you are almost certainly looking at an SLO" [source: sre-book-service-level-objectives-2026-08-31].

- **A** is wrong: an SLA is a contract "that includes consequences of meeting (or missing) the SLOs they contain" [source: sre-book-service-level-objectives-2026-08-31]. Publicity is not what makes an SLA; consequences are.
- **B** is wrong but close, and worth being precise about. The **SLI** is the underlying measurement — the proportion of search requests returning under 200ms. The statement quoted adds a *target* and a *window*, which is what makes it an objective.
- **D** is wrong: the error budget is the allowance of unreliability this objective leaves room for [source: sre-book-error-budgets-2026-08-31], not the objective itself.

Q17 — B. "The four golden signals of monitoring are latency, traffic, errors, and saturation" [source: sre-book-four-golden-signals-2026-08-23].

- **A** mixes RED's rate/errors/duration [source: red-method-tom-wilkie-2026-08-31] with the golden signals' saturation. Plausible, and wrong.
- **C** is the USE method's three terms — utilization, saturation, errors [source: use-method-brendan-gregg-2026-08-31] — with latency appended. USE is a *resource*-oriented methodology; the golden signals are a user-facing-service list.
- **D** names the four OpenTelemetry signals [source: opentelemetry-signals-2026-08-23], which are signal *types*, not monitoring priorities. Two different fours, and the exam will happily offer you the wrong one.

Chapter Summary

Concept	Remember This
Observability	Asking questions you did not plan for — unknown unknowns. A property of the system, not a product
Monitoring	Collecting and displaying real-time data about indicators chosen in advance — known unknowns
Instrumentation	The precondition. Properly instrumented = you don't need to add more instrumentation to troubleshoot
Probes ≠ observability	A yes/no signal to the kubelet, acted on and discarded. No history, no trend, no query
The four signals	Traces, metrics, logs, baggage . Four on OpenTelemetry's Signals page, not three
Baggage	A separate key-value store propagated between services. Not a span attribute unless you make it one
OTel Collector	Receives, processes, exports. One input, swappable backends
Time series	Timestamped values sharing a metric name and label set. Change any label value, get a new series
Utilization's denominator	The container's resource request . No request, no defined utilization — undefined, not zero
metrics-server vs monitoring	Current resource readings for autoscaling vs history, arbitrary metrics, alerting, and queries over time
Prometheus	Pulls . Scrapes HTTP endpoints found via service discovery. CNCF's second project, 2016
Client library vs exporter	Library inside code you own; exporter beside code you don't. Same destination, opposite side of the boundary
Pushgateway	The only push path, and only for service-level batch jobs too short to scrape
Prometheus non-fit	Not for 100% accuracy — per-request billing is the documentation's own counter-example
Span vs trace	A span is one unit of work. A trace is the whole path: one or more spans, starting at the root span
Context propagation	Service A passes trace ID + span ID; service B creates a span in the same trace
Jaeger	The tracing backend. OTel instruments and exports; Jaeger receives and visualizes [source: jaeger-overview-2026-08-23]
kubect1 logs	Not an archive. Rotation, one-restart depth, and eviction all bound it
Logging architectures	Node-level agent (DaemonSet — the default), sidecar, or app pushes directly
Fluentd / Fluent Bit	Fluentd is CNCF; Fluent Bit is its lightweight sub-project. One word, two words
Reliability	"Is the service doing what users expect it to be doing?"
SLI / SLO / SLA	Measurement / objective / contract-with-consequences. "What happens if it's missed?"
Four golden signals	Latency, traffic, errors, saturation. Latency rising is a leading indicator of saturation

Concept	Remember This
RED vs USE	RED is service-oriented; USE is resource-oriented

Safe Harbor

Domain 4 is complete, and so is every content chapter in this book.

You started at Chapter 1 with a curriculum that had moved under everyone's feet, and you have now covered all four domains: Kubernetes Fundamentals, Container Orchestration, Cloud Native Application Delivery, and Cloud Native Architecture. At the top of this chapter you could name three signals; now you can name four and say why the fourth is the one that makes the other three cohere.

That is the last new thing this book has to teach you. Everything from here is consolidation.

The Voyage Ahead

Chapter 19 does not add material. It re-sees what you already have.

You have learned this book domain by domain, because that is how the exam is structured and because knowledge has to enter in some order. But that is not how it will be *tested*. A question does not announce which chapter it came from. It describes a situation and asks what is true, and answering it well means holding several chapters at once: the control loop from Chapter 3, the operator pattern from Chapter 6, and the GitOps agent from Chapter 15 turn out to be one idea seen three times, and a question can approach it from any of them.

So the next chapter runs the book the other way. Nine cross-cutting themes traced through twenty chapters. The confusion pairs that cost points, each with a question that discriminates between them. Ninety minutes of pacing, and what to do with a question you do not immediately know. A map of where the weight actually is, checked against your own Soundings and Bearings history rather than against a generic study plan. How to use The Lodestar in the last hour before you sit down. And a dated plan for the final week.

You have the instruments. Chapter 19 is the last stretch of open water before you sit down.

"The instruments do not tell you where you are. They tell you what you can find out." — Lodestar
Ledgers

Chapter 19: Bearings Before Landfall

"Everything that connects, and the traps that don't"

Domain Weight: — (synthesis; no new objectives) | **Complexity:** Mixed | **Novelty:** Familiar

Attention Budget

Total time: ~135 minutes | **Recommended:** Split across 2 sessions — \$1–\$2 in one, \$3–\$6 in another

Section	Time	Attention Cost	Best Time to Study
🕒 Soundings	10 min	Low	Anytime — but answer honestly
\$1 Nine Threads Through Eighteen Chapters	25 min	High	Peak attention
🕒 Taking Your Bearings: The Threads	8 min	Medium	Immediately after \$1
\$2 The Pairs That Cost Points	35 min	High	Peak attention, fresh session
🕒 Taking Your Bearings: The Pairs	7 min	Medium	Immediately after \$2
\$3 Ninety Minutes	10 min	Low	Anytime
\$4 Where the Weight Actually Is	10 min	Medium	With your checkpoint scores in front of you
\$5 Using The Lodestar	5 min	Low	Anytime
\$6 The Week Before	8 min	Low	Anytime
Practice Questions	17 min	Medium	After a break

Attention Cost Key:

- **Low:** Concrete, familiar concepts — study anytime
- **Medium:** New concepts requiring focus — study when alert
- **High:** Abstract or complex material — study at peak attention

If you only have 15 minutes: read \$2's pair tables and the ⚠ Navigational Hazards block. That is where the points are.

"The chart does not tell you where you are. Two bearings on two landmarks tell you where you are. The chart only tells you what you are looking at." — Lodestar Ledgers

🔊 Soundings

This is a synthesis chapter, so the Soundings changes shape. You are not measuring what you know about a new topic. You are measuring whether you are ready to sit the exam. Five questions, none of them multiple choice. Answer them out loud or in writing. Vague answers count as "no."

1. **Have you sat one full-length attempt under a clock — start to finish, no pauses, no looking anything up?** If yes: what was your score, and did you finish with time left or run out?
2. **Which of the four domains is your weakest, and what specifically will you do about it this week?** Naming the domain is the easy half. The readiness signal is the plan.
3. **Without looking anything up:** a PersistentVolumeClaim requests ReadWriteOnce. State in one sentence what that constrains, and what it does *not* constrain.
4. **Name the recurring patterns this book has been building across all eighteen chapters.** Not the chapter titles. The patterns that showed up in four or five different chapters wearing different names. How many can you list from memory?
5. **When you get a practice question wrong, what is your most common failure mode?** Misreading the stem? Coin-flipping between two survivors? Running out of clock and guessing? If you don't know, you have not been reviewing your wrong answers, only counting them.

Answers + readiness rubric

1. No answer key; this is self-report. Note the trap: an *untimed* full-length attempt tells you almost nothing about pacing, and pacing is a distinct failure mode from knowledge. The Linux Foundation publishes 90 minutes for its multiple-choice exams [source: lf-mc-exam-faq-2026-08-31], and the timer cannot be paused, even through a connection fault [source: lf-handbook2-taking-the-exam-2026-08-31].
2. No answer key. If you named a domain but not a plan, count this as half. §4 turns the plan half into something concrete.
3. No full derivation here; that is §2's work, and handing it over now would spend the discrimination before you have drilled it. Note only this much: access modes are node-count semantics, not permission semantics, and one of the four ⚠ Navigational Hazards in §2 exists precisely because the English misleads. If you hesitated on this one, §2's storage rows and the hazard block are your center of gravity. The underlying mechanism is at [cross-bearing: see Ch 11 §4 — access modes and reclaim policies].
4. No answer key here either. §1 supplies the list, and the point of asking first is that you try to generate it before you see it.
5. No answer key. If you cannot name your failure mode, that is itself the finding.

Readiness tiers — this rubric reads differently from every other chapter's. You are not choosing a reading pace. You are estimating a date.

5 of 5 — sit it. You have timed data, a named weakness with a plan, cold retrieval on a discriminator, the thematic map, and self-knowledge about your errors. Use §5 and §6 as your schedule and stop adding material. Adding new sources at this stage costs you consolidation and buys you nothing.

3–4 — one to two more weeks. You are close. §4 tells you where those weeks go: not "review everything," but the specific chapters your own checkpoint history flags.

0–2 — four or more weeks, and not a re-read. Rereading the book front to back is the least efficient thing you could do with four weeks. §4 plus your own Bearings scores will name five or six chapters. Work those, then re-run this Soundings.

Why This Chapter Matters

You have read eighteen chapters organized by domain, because that is how the exam is organized. It is not how the material is *shaped*.

The control loop is not a Chapter 3 fact. It is the thing Chapter 6 instantiates in a named controller, Chapter 11 re-uses for volume binding, Chapter 15 points at a Git repository, and Chapter 17 collects as an extension story. A reader who filed it under "Chapter 3" will not recognize it when it walks into a Chapter 15 question wearing different clothes. That reader has one fact. The reader who filed it as a *pattern* has a derivation, and derivations survive time pressure in a way that memorized rows do not.

This is the shift from *having studied* to *being ready*. Practitioners do not hold nine hundred facts. They hold a small number of patterns and derive the facts on demand. The RBAC four-way binding matrix is the clearest case in this book: candidates memorize four rows and hope; practitioners derive all four from a distinction the book settled back in Chapter 4, in about six seconds, under pressure, every time.

Here is what this chapter is honestly worth, stated plainly so you can decide how much time to give it: **it contains nothing new.** Every fact in it was taught somewhere in Chapters 2 through 18. Its entire value is the second pass: seeing the same material cut along a different axis, and converting recognition into discrimination. Recognizing that "ReadWriteOnce" is a term you have met is worth zero points. Being able to say what it constrains, and what its neighbor constrains, with ninety seconds on the clock, is worth all of them.

Dead Reckoning: This is a review chapter. Sections 1 and 2 re-present taught material in a new organization. Sections 3 through 6 are exam strategy and study scheduling, not technical content. There are two ☆ Fixed Points and both restate things you already learned. If you are short on time, §2 is the section that moves your score.

What You'll Learn

By the end of this chapter, you'll be able to:

- **Trace** each of the book's nine cross-cutting themes through the chapters that build it, and name the pattern rather than reciting the instances
- **Discriminate** between every confusion pair this book has flagged, using a one-line test rather than two competing definitions held side by side
- **Pace** a ninety-minute multiple-choice exam, including what to do with a question you cannot answer on first read
- **Allocate** your remaining study time against published domain weights and your own checkpoint history, rather than against whatever currently feels least comfortable
- **Use** the-lodestar.md in the last hour before the exam, and know what it deliberately leaves out
- **Plan** the final week — including the parts of it that consist of not studying

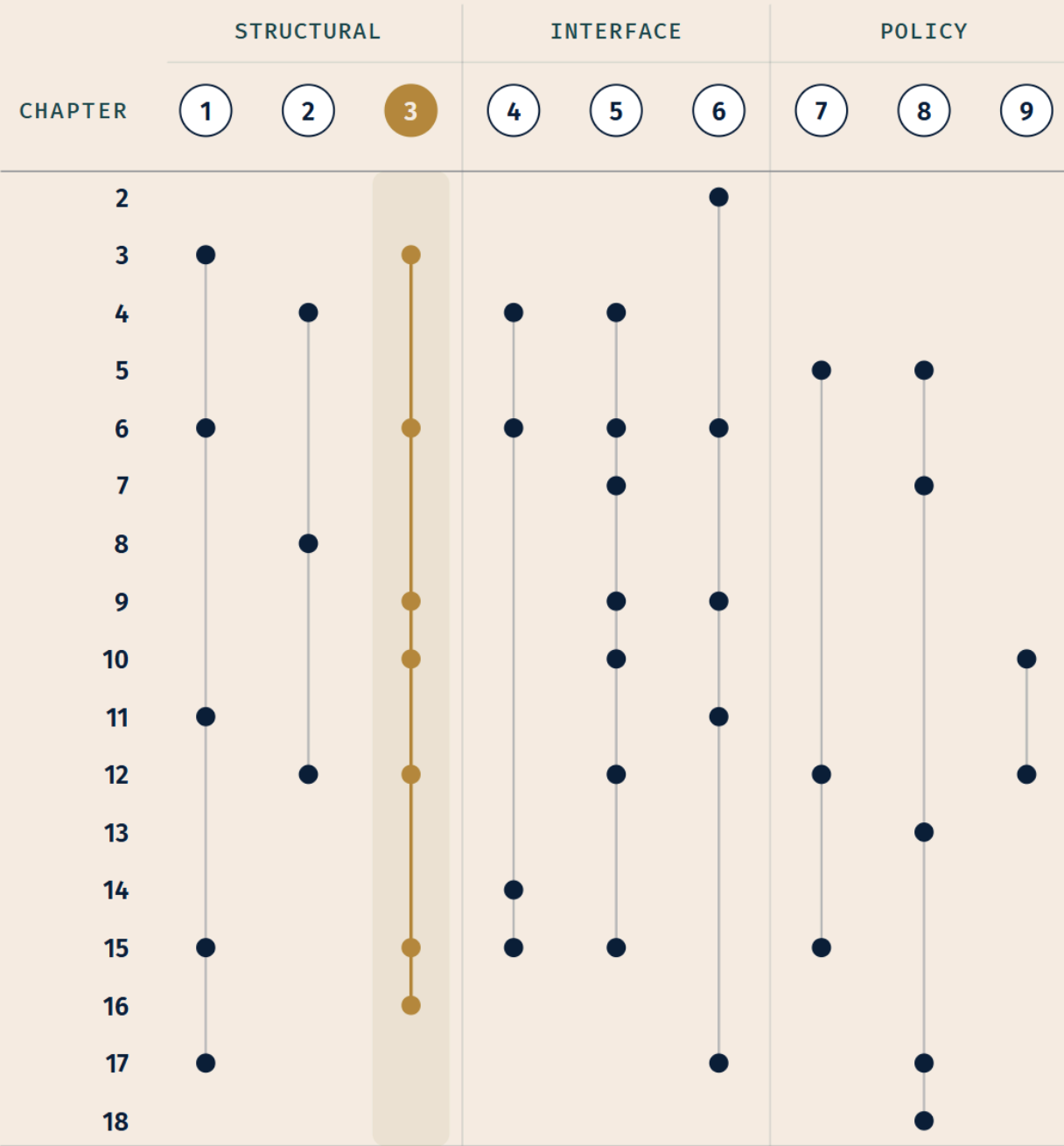
You'll also stop thinking of this material as eighteen chapters and start thinking of it as nine patterns with instances, which is how practitioners hold it.

☀ §1 — Nine Threads Through Eighteen Chapters

A book organized by exam domain serves the exam, not the material. Kubernetes did not grow four domains. It grew a handful of ideas that got applied over and over, in different subsystems, by different SIGs, across a decade. The exam blueprint cuts across those ideas at right angles.

So we cut the other way now. A coastline surveyed on a north-south run and the same coastline surveyed east-west produce two charts that do not look much alike, and a navigator wants both. Nine threads, each one named, then traced through the chapters that build it. Nothing here is new. Everything here you have read. What is new is the *axis*.

Read this section with one question running: **when I meet this pattern in a question stem, will I recognize it?**



Legend

STRUCTURAL TIER

- 1 · Control loop
- 2 · Scope boundary
- 3 · Absent component*

INTERFACE TIER

- 4 · Declarative
- 5 · Label join
- 6 · Pluggability

POLICY TIER

● 7 · Identity

● 8 · Requests/limits

● 9 · Additive/allow

* *Absent component is this chapter's stated Fixed Point.*

The three tiers are a reading aid, not doctrine. **Structural** threads describe how the cluster is built. **Interface** threads describe how its parts are joined and extended. **Policy** threads describe what is permitted. Density is the thing to notice: threads 1, 3 and 5 touch the most chapters, and they are correspondingly the most likely to appear in a question whose stem never names them.

Thread 1 — The control loop

Desired state, current state, reconciliation. A controller reads what you asked for, observes what exists, and acts to close the gap. Forever. That is the whole architecture, and it is the single most reused idea in the system.

Path: **Ch 3 §6** defines it [*cross-bearing: see Ch 3 §6 — controllers and the control loop*] → **Ch 4 §1** supplies the artifact the loop reads, the declared object → **Ch 6 §2** shows it instantiated in a controller you can watch working, with `.spec.replicas` as the desired state → **Ch 11 §2** applies it to storage, where a claim is a desired state and binding is the reconciliation → **Ch 15 §7** points the loop at a Git repository instead of etcd → **Ch 17 §4** collects it as the thing every extension point extends.

What makes Ch 15 §7 the payoff is that nothing changes except *where the desired state lives*. The loop is identical. Substituting Git for etcd is not a new mechanism; it is the same mechanism with a different store, which is precisely why the GitOps section could re-present Chapter 3's figure and expect it to land.

🔑 **Mnemonic:** Desired, observed, act. If you can say those three words about any Kubernetes component, you have found its control loop.

Thread 2 — Namespaced versus cluster-scoped

Some objects live inside a namespace. Some live above all namespaces. That single distinction, settled in Chapter 4, is load-bearing three chapters later in ways that look unrelated on the surface.

Path: **Ch 4 §3** defines it [*cross-bearing: see Ch 4 §3 — namespaced vs cluster-scoped*] → **Ch 8 §3** builds ResourceQuota (a namespace total) and LimitRange (a per-object default) on it → **Ch 12 §3** derives the entire RBAC binding matrix from it.

That last one is worth dwelling on, because it converts four memorized rows into one applied distinction. Roles and RoleBindings are namespaced. ClusterRoles and ClusterRoleBindings are cluster-scoped. Cross the two and you get four combinations, and the semantics of each fall straight out of the scoping:

	Role (namespaced)	ClusterRole (cluster-scoped)
RoleBinding (namespaced)	Grants the Role's permissions in that namespace	Grants the ClusterRole's permissions but only in the binding's namespace
ClusterRoleBinding (cluster-scoped)	Not valid — a namespaced Role cannot be granted cluster-wide	Grants the ClusterRole's permissions cluster-wide

You do not memorize this table. You derive it: a binding can never grant more scope than it has itself, and a namespaced Role can never escape its namespace. Both facts follow from Chapter 4.

Thread 3 — An object without its component does nothing

This is the pattern the book has hammered hardest, and it is the one most likely to be worth points on the day.

☆ Fixed Point

An object without its component does nothing.

Creating the API object is half the transaction. Something has to be *watching* for it. An Ingress with no Ingress controller installed routes no traffic. A NetworkPolicy on a CNI plugin that does not implement policy blocks nothing. A HorizontalPodAutoscaler with no metrics-server has no numbers to act on. A CustomResourceDefinition with no operator produces a resource you can create and nothing that happens afterward.

Path: **Ch 3 §4** — the phrase is coined where addons are introduced, because "optional component" is exactly where the reader first learns that parts of the cluster are not automatically present → **Ch 10 §3** names it as a pattern, at the Ingress-object-versus-Ingress-controller split [*cross-bearing: see Ch 10 §3 — the object is not the implementation*] → **Ch 11 §5** applies it to CSI drivers → **Ch 13 §7** applies it to `kubectl top` failing on a bare cluster → **Ch 17 §7** applies it to the VerticalPodAutoscaler, which is an add-on rather than something Kubernetes ships [source: [k8s-docs-autoscaling-and-vpa-2026-08-31](#)] → **Ch 18** applies it to the whole observability stack.

That is one rule doing the work of five separate gotchas. When a question describes an object that exists and a behavior that is not happening, this is the first thing to reach for.

📖 **Snag:** The inverse also holds and is less often taught. A component with no object also does nothing visible: an Ingress controller running in a cluster with no Ingress resources is idle, not broken. Question stems sometimes present the idle-controller case to see whether you have internalized both directions.

Thread 4 — Declarative desired state versus imperative command

You do not tell Kubernetes what to do. You tell it what you want and file the declaration; making reality match is the system's job. The imperative commands that exist (`kubectl scale`, `kubectl create`) are

conveniences that write a declaration on your behalf.

Path: **Ch 4 §1** defines it → **Ch 6** shows what happens when a declared replica count meets a controller → **Ch 14** packages declarations for distribution, as charts and overlays → **Ch 15** puts the declaration in version control and makes the repository the source of truth.

The thread's payoff is that GitOps is not a new idea bolted onto Kubernetes. It is what you get when you take Chapter 4 seriously and ask where the declaration ought to live.

Thread 5 — Labels and selectors as the universal join

Kubernetes has almost no foreign keys. What it has instead is labels on objects and selectors that match them, and nearly every relationship in the system is expressed that way.

Path: **Ch 4 §5** defines labels and selectors [*cross-bearing: see Ch 4 §5 — labels and selectors*] → **Ch 6 §3** uses selectors as the controller-to-Pod join, plus ownerReferences → **Ch 7 §3** uses node labels for nodeSelector and affinity → **Ch 9 §4** uses a Service's selector to build EndpointSlice membership → **Ch 10 §6** uses podSelector and namespaceSelector for NetworkPolicy → **Ch 16 §4** shows the join failing from the application side, where a selector that matches nothing produces an empty EndpointSlice [*cross-bearing: see Ch 16 §4 — is anything even selected*] → **Ch 12 §3** provides the contrast.

The contrast is the interesting part, and it is easy to miss. **RBAC does not select. RBAC names.** A RoleBinding lists its subjects explicitly; it does not match them by label. The asymmetry is deliberate: permission granted by label match would mean anyone who can set a label can escalate privilege. It is also exactly the kind of "why is this one different" detail that makes a good question.

Thread 6 — The pluggable interfaces

Kubernetes repeatedly declines to implement something itself, and defines an interface instead. Four of these are the canonical set:

Interface	What it delegates	Defined in
CRI — Container Runtime Interface	Running containers	Ch 2 §4
CNI — Container Network Interface	Pod networking	Ch 9 §1
CSI — Container Storage Interface	Providing storage	Ch 11 §5
CRDs and operators	Extending the API itself	Ch 6 §8

Ch 17 §4 collects all four and situates them alongside the wider extension surface: API aggregation, admission webhooks, device plugins, scheduler plugins [*cross-bearing: see Ch 17 §4 — the four pluggable interfaces, collected*]. That wider surface is sometimes called "API extensions," which is why you will see the fourth slot named both ways; Chapter 17 reconciled the two, and this chapter inherits that reconciliation rather than re-opening it.

Notice how thread 6 and thread 3 interlock. Every pluggable interface creates an object-without-component hazard, because "Kubernetes defines the interface" always implies "somebody else installs the implementation."

Thread 7 — Identity, from Pod to API to delivery agent

Everything that talks to the API server has an identity, and in this cluster non-human identity means ServiceAccount.

Path: **Ch 5 §6** introduces the ServiceAccount as the Pod's identity, with the default account carrying near-zero permissions → **Ch 12 §2** makes it the subject RBAC binds to [*cross-bearing: see Ch 12 §2 — ServiceAccounts as RBAC subjects*] → **Ch 15 §4** applies it to the delivery agent, which is a Pod like any other and therefore has a ServiceAccount and needs exactly the permissions its syncing requires, no more.

The reason this thread matters more than it looks: the GitOps blast-radius argument in Chapter 15 depends entirely on it. A pull-based agent holds cluster credentials *inside* the cluster; a push-based CI system holds them *outside*. That is an identity argument, not a networking one.

Thread 8 — Requests and limits, the numbers that reappear everywhere

You declare what a container needs (request) and the ceiling it may reach (limit). Two numbers, and they surface in five different subsystems.

Path: **Ch 5 §8** defines requests, limits, and the QoS classes they produce [*cross-bearing: see Ch 5 §8 — requests, limits, and QoS classes*] → **Ch 7 §2** makes requests the scheduler's filter input, which is why an over-requested Pod sits in Pending → **Ch 13 §4** makes QoS class the eviction ordering under node pressure → **Ch 17 §7** makes them the thing the VerticalPodAutoscaler adjusts → **Ch 18 §3** makes them the denominator for utilization.

Two numbers, five consequences. This is a thread worth being able to trace out loud, because a question can enter it at any of the five points.

Thread 9 — Additive, allow-only, no deny

RBAC and NetworkPolicy were built by different SIGs for different problems, and they share one semantic exactly.

Both are purely additive. Neither has a deny rule. In RBAC, permissions accumulate across every binding that names you; nothing subtracts. In NetworkPolicy, a Pod that no policy selects is fully open, a Pod that any policy selects is restricted to the union of what its policies allow, and there is no rule that says "block this."

Path: **Ch 10 §6** establishes it for NetworkPolicy [*cross-bearing: see Ch 10 §6 — allowing, never denying*] → **Ch 12 §3** establishes it for RBAC → **Ch 12 §9** names them as one shared semantic.

🔒 **Worth Securing:** When a question asks how to *stop* something in RBAC or NetworkPolicy, the answer is never "add a deny rule." It is either "remove the grant" or, for NetworkPolicy, "select the Pod with a policy that does not allow it," which restricts by selection, not by denial. An option offering a deny verb is testing whether you have imported firewall intuitions into a system that has none.

What to do with these nine

Do not memorize the paths. The paths are here so you can *check* your recall, not so you can recite them. What you want is the ability to hear a question stem and think: *this is thread 3*, or *this is the scope boundary again*. That recognition is what converts eighteen chapters of material into nine reusable moves.

🕒 Taking Your Bearings: The Threads

Three questions. All three are cross-chapter by construction — that is what a synthesis checkpoint is.

Q1. A cluster administrator installs a CustomResourceDefinition for a BackupSchedule resource. Users can create BackupSchedule objects successfully — `kubectl get backupschedules` returns them. No backups ever run. Name the pattern this is an instance of, and name two other places in this book where the same pattern appears. [retrieval: ch3, ch6, ch10, ch13, ch17]

Q2. A RoleBinding in namespace payments references a ClusterRole named secret-reader. What permissions does the subject receive, and where? Derive your answer from a distinction established in Chapter 4 rather than recalling the matrix. [retrieval: ch4, ch12]

Q3. A Pod is stuck in Pending. A colleague suggests checking `kubectl logs`. Explain why that is the wrong instrument, and name the **confusion pair** that tells you which field to read instead. [retrieval: ch5, ch7, ch13]

Answers with Explanations:

Q1 — The absent-component pattern: an object without its component does nothing.

A CRD defines a new resource *kind*. It does not supply anything that acts on instances of that kind. Without an operator or controller watching for BackupSchedule objects, creating one stores a record in etcd and nothing else. The API accepted it; nothing reconciled it.

Two other appearances (any two of these count):

- An **Ingress** object with no Ingress controller installed (Ch 10 §3)
- A **NetworkPolicy** on a CNI plugin that does not implement policy (Ch 10 §7)

- `kubectl top` failing because metrics-server is not installed (Ch 13 §7)
- The **VerticalPodAutoscaler**, which is an add-on rather than built in (Ch 17 §7)
- A **PersistentVolumeClaim** with no CSI driver to provision against (Ch 11 §5)

If you answered "the CRD is broken" or "the CRD needs a schema," you are diagnosing the object when the missing thing is the component. That is exactly the reflex this pattern exists to replace.

Q2 — Permissions from the ClusterRole, but only inside payments.

The derivation: a RoleBinding is a namespaced object. A namespaced object cannot grant anything outside its own namespace, because that is what "namespaced" means (Ch 4 §3). So even though `secret-reader` is a ClusterRole — defined once, cluster-scoped, reusable — binding it with a RoleBinding confines its grant to that RoleBinding's namespace.

This is the single most useful RBAC pattern in practice, incidentally: define a ClusterRole once, bind it per-namespace with RoleBindings, and you get consistent permission sets without duplicating Role definitions.

If you answered "cluster-wide secret read access," you inverted the rule. The scope of the *grant* is set by the binding, not by the role. If you couldn't answer without the table, that is the finding: go back to Ch 4 §3 and make sure the scope distinction is solid, because the matrix rests entirely on it.

Q3 — Because Pending means no container has started, so there are no logs to read.

A Pod in `Pending` has been accepted by the API server but has not been scheduled onto a node, or has been scheduled and its images have not been pulled. In the unscheduled case no container exists anywhere. `kubectl logs` asks the kubelet on the Pod's node for container output; there is no node and no container.

The pair is **Pod phase versus container state** (Ch 5 §5), the first row of §2's Domain 1 table. The phase tells you where in the lifecycle the Pod is, and the phase determines which instrument is even applicable. Read the phase first, then `kubectl describe` for events and conditions. For a `Pending` Pod the scheduler's message about why no node was feasible is the actual diagnostic [*cross-bearing: see Ch 13 §2 — diagnosing a Pod that will not start*].

If you said "logs would be empty," you are right but incomplete. The useful answer names what to read *instead*, and why the phase told you.

Checkpoint: You've Now Confirmed ✓ You can recognize the absent-component pattern from a symptom description ✓ You can derive the RBAC binding matrix rather than recall it ✓ You read the phase before you reach for an instrument

Missed one? Each of the three names its own repair, and none of them is "re-read §1." Q1 sends you to thread 3; Q2 sends you to thread 2 and Ch 4 §3; Q3 sends you to §2's Domain 1 pairs, which you are about

to read anyway. Work the named one and move on.

Now the harder half. §2 is where the points are.

§2 — The Pairs That Cost Points

Most lost points on a multiple-choice exam are not lost to ignorance. They are lost to *collapse*: two adjacent concepts filed as one thing, so that when both appear as options the reader has no way to separate them and picks by feel. Two landmarks that have merged into one on your chart cannot fix a position.

This section is the antidote, and it works one way only. For each pair: **the one-line test that separates them**, then **the tell** that shows it up in a stem. Never two competing definitions side by side. Definitions side by side is what caused the collapse in the first place; a discriminator is a *procedure*, and procedures survive pressure.

Work these actively. Cover the right column, read the pair, say the discriminator out loud, then check. Reading this section passively will feel productive and will do nothing.

The card below is a *selection*: the highest-value pairs, sized to be drilled in one pass. The tables underneath it carry the full set.

KUBERNETES & HELM • QUICK REFERENCE

One Question Beats Two Definitions

23 confusion pairs from the book, each resolved by one discriminating question — banded into the four Kubernetes & Helm exam domains.

PAIR → DISCRIMINATOR — THE QUESTION TO ASK

D1

Pod phase / container state

→ *Lifecycle, or one container?*

liveness / readiness

→ *Restart it, or stop sending traffic?*

labels / annotations

→ *Does anything select on it?*

ConfigMap / Secret

→ *Would you mind this in a log?*

Deployment / StatefulSet

→ *Does replica 0 need to BE replica 0?*

OCI / CRI

→ *Artifact format, or kubelet's API?*

D2

ClusterIP/NodePort/LB

→ *Reachable from where?*

Ingress object / controller

→ *The record, or the thing that acts?*

NetworkPolicy default

→ *Is this Pod selected by any policy?*

Role / ClusterRole binding

→ *What scope is the BINDING?*

PV / PVC

→ *Supply, or demand?*

RWO / RWOP

→ *One node, or one Pod?*

D3

chart / release / revision

→ *Package, install, or version of it?*

rollout undo / helm rollback

→ *Whose history is being rewound?*

push / pull delivery

→ *Where do the credentials live?*

OutOfSync

→ *Difference, or failure?*

D4

mesh CP / cluster CP

→ *Whose control plane?*

sidecar / ambient

→ *Proxy per Pod, or per node?*

HPA / VPA

→ *More replicas, or bigger ones?*

observability / monitoring

→ *New questions, or known ones?*

span / trace

→ *One hop, or the whole journey?*

SLI / SLO / SLA

→ *Measure, target, or consequence?*

TOC / Governing Board

→ *Technical, or business?*

LEGEND Aa OBJECT / FIELD → THE QUESTION — DOMAIN BAND

The question is the fix — when a pair blurs, ask it instead of re-reading two definitions.

Domain 1 — Kubernetes Fundamentals (Ch 2–8)

Pair	The one-line test	The tell in a stem
Pod phase vs container state	Phase describes the whole Pod's position in its lifecycle; state describes one container right now. Running as a phase means the Pod is bound to a node, all its containers have been created, and at least one is running or starting.	A stem that says a Pod is Running and something is still broken is testing this. Running is a lifecycle position, not a health verdict.
liveness vs readiness vs startup	Liveness failing restarts the container. Readiness failing removes it from Service endpoints , no restart. Startup suspends the other two until the app has finished booting.	A slow-starting app being killed repeatedly is a startup-probe question. Traffic reaching a not-yet-ready Pod is a readiness question.
labels vs annotations	Does anything <i>select</i> on it? Labels are selectable; annotations are not. Annotations hold arbitrary metadata for tools and humans.	If a stem describes attaching information that a controller must match on, it must be a label.
image tag vs digest	Mutable pointer, or identity? A tag can be moved to point at a different image tomorrow. A digest is the content hash and cannot be moved without becoming a different digest.	A stem about two clusters pulling "the same image" and getting different bytes is a tag question. A stem about a signature binding to something specific is a digest question. <i>[cross-bearing: see Ch 2 §3 — registries, tags, and digests]</i>
ConfigMap vs Secret	Would you mind seeing this value in a log or a describe output? If yes, it's a Secret. But Secret is <i>base64-encoded</i> , not encrypted, and encryption at rest is a separate cluster decision.	An option claiming Secrets are "encrypted by default" is wrong. <i>[cross-bearing: see Ch 12 §4 — Secrets are not encrypted]</i>
namespace vs label for separating versions	Namespace is a <i>scope</i> boundary (names, quota, policy). Label is a <i>selection</i> attribute. Separating v1 from v2 within one app is a label job; separating tenants or environments is a namespace job.	A stem about two versions needing different resource quotas has moved from label territory into namespace territory.
Resource Quota vs LimitRange	Total, or default? ResourceQuota caps a namespace's <i>aggregate</i> consumption. LimitRange sets <i>per-object</i> defaults and bounds, so a container that declares nothing still gets numbers.	"The namespace has used its whole CPU allowance" is quota. "Pods here default to 100m if unspecified" is LimitRange. <i>[cross-bearing: see Ch 8 §3 — dividing a shared cluster]</i>
Deployment vs StatefulSet	Does replica 0 need to <i>be</i> replica 0 — stable name, stable identity, ordered startup? If yes, StatefulSet. If replicas are interchangeable, Deployment.	The distinction is identity , not "does it write to disk." A stateless app can use a StatefulSet; a database can be run under a Deployment badly.
DaemonSet vs "set replicas"	DaemonSet means <i>one per node, tracked as nodes join and leave</i> . A replica count is a number that does not know what a node is.	Any stem where nodes are added or removed and the workload must follow is a DaemonSet.

Pair	The one-line test	The tell in a stem
to node count"		
Job vs CronJob	Job runs work that ends. CronJob creates Jobs on a schedule. CronJob is a factory; Job is the product.	"Runs nightly" is CronJob. "Runs once and must complete" is Job.
taints/tolerations vs node affinity	Who is doing the refusing? A taint is the <i>node</i> repelling Pods by default; a toleration is a Pod's permission to ignore that repulsion. Node affinity is the <i>Pod</i> asking for a kind of node. Repel-by-default versus attract-by-preference.	"These nodes should stay empty unless a workload explicitly opts in" is taints. "This workload wants GPU nodes" is affinity. <i>[cross-bearing: see Ch 7 §4 — when the berth refuses you]</i>
OCI vs CRI	OCI standardizes the artifact and its execution : image format, runtime spec, distribution. CRI standardizes the kubelet's API to a runtime . One is about the container; one is about the conversation.	Anything about image portability across registries is OCI. Anything about how the kubelet talks to containerd is CRI.
scheduler binds / kubelet starts	The scheduler makes a <i>decision</i> and writes it down. The kubelet on the chosen node does the work of starting containers.	A Pod with a node assigned but no running containers has passed the scheduler and stalled at the kubelet, so the diagnosis is on that node, not in the scheduler.
kubelet skew vs kubect1 skew vs supported releases	Three different numbers. kubelet may be up to three minor versions older than kube-apiserver and must never be newer [source: k8s-version-skew-policy-2026-08-31]. kubect1 is supported within one minor version either direction [source: k8s-version-skew-policy-2026-08-31]. The project maintains release branches for the most recent three minor releases [source: k8s-version-skew-policy-2026-08-31].	Stems mix these deliberately. Read which component is named before reaching for a number.
schedule d once vs rescheduled	A Pod is bound to a node once . It is never moved. What people call "rescheduling" is a <i>new</i> Pod created by a controller after the old one died.	A stem describing a Pod "moving to another node" is describing a replacement, and the controller is the actor.
restartPolicy scope	It governs containers within a Pod , not the Pod object. A Job whose Pod template sets restartPolicy: Never will still get a <i>replacement Pod</i> from the Job controller when the first one fails; the policy stopped the container from restarting in place, not the workload from being retried.	The trap is reading "Never" as "this workload will not come back." Note also that a Deployment's Pod template cannot use Never at all; the API requires Always there.

🔦 **Closer Look:** The version-skew numbers reward a moment of structure. Every rule is anchored to kube-apiserver, and every component except kubect1 must be **no newer** than it, which is why the upgrade order is control plane first. kubect1 is the only component allowed to be *newer*, because it is a client you run from your laptop and the project accepts you may have upgraded it casually [source: k8s-version-skew-policy-2026-08-31].

Domain 2 — Container Orchestration (Ch 9–13)

Pair	The one-line test	The tell in a stem
ClusterIP vs NodePort vs LoadBalancer	Reachable from where? ClusterIP: inside the cluster only. NodePort: any node's IP at a fixed port. LoadBalancer: an external address the provider allocates. Each type <i>includes</i> the ones below it.	"Should not be reachable from outside" points at ClusterIP. The inclusion property is what a stem about "also still reachable internally" is checking.
headless vs broken vs selectorless Service	Headless is clusterIP: None on purpose : you want the individual Pod addresses, not one virtual IP. Broken is a selector matching nothing. Selectorless is deliberate too: you manage endpoints yourself.	Headless plus StatefulSet is the stable-DNS-name pattern. A Service with no endpoints and a selector present is broken, not headless.
Ingress object vs Ingress controller	The record versus the thing that acts on it. This is thread 3.	An Ingress that "exists but routes nothing" is always the controller.
Ingress frozen vs deprecated	Frozen means feature-complete and still supported: no new features, but it works and is not going away on a schedule. Deprecated would mean scheduled for removal. Ingress is frozen; Gateway API is where new work happens.	An option saying Ingress is deprecated or removed is wrong.
NetworkPolicy default behavior	Ask: <i>is this Pod selected by any policy?</i> No policy selects it → fully open. Any policy selects it → restricted to the union of what its policies allow.	"Default deny" is achieved by <i>selecting</i> Pods with a policy that allows nothing, not by a deny rule.
NetworkPolicy needs both ends	For traffic A→B to flow when both are restricted, A needs egress permitting B and B needs ingress permitting A.	A stem where one policy was written and traffic still fails is usually the other end.
RBAC has no deny	Permissions are the union of every binding naming you. Nothing subtracts.	A "deny" verb in an option is testing whether you imported firewall intuitions.
The four-way binding matrix	Derive it. The binding's scope sets the grant's scope; a namespaced Role cannot be granted cluster-wide.	See §1 thread 2.
view / edit / admin	view reads (but not Secrets). edit writes workloads. admin adds the ability to manage RBAC within a namespace . None of them is cluster-admin.	A stem about someone who can grant permissions to others in one namespace is admin.
PV vs PVC	Supply versus demand. The PV is the piece of storage that exists; the PVC is the request a workload makes. Administrators think in PVs; developers write PVCs.	A Pod references a PVC , never a PV directly.
RWO vs RWOP	ReadWriteOnce = read-write by a single node , and multiple Pods on that node can share it. ReadWriteOncePod = read-write by a	The intuitive misreading of "Once" as "one Pod" is exactly what RWOP

Pair	The one-line test	The tell in a stem
	single Pod, cluster-wide [source: k8s-docs-persistent-volumes-depth-2026-08-25].	exists to name.
Retain / Delete / Recycle	Retain keeps the volume and its data after the claim goes, requiring manual reclamation. Delete removes both the PV object and the backing storage. Recycle is deprecated [source: k8s-docs-persistent-volumes-depth-2026-08-25].	Defaults differ by origin: Retain for manually created PVs, Delete for dynamically provisioned ones [source: k8s-api-ref-persistentvolume-v1-2026-08-25].
storageClassName: "" vs omitting it	"" explicitly means no class : bind only to a PV that also has no class. Omitting the field means "whatever the cluster's default behavior is," which depends on whether the DefaultStorageClass admission plugin is on [source: k8s-docs-persistent-volumes-depth-2026-08-25].	"" reading as "use the default" is a hazard; it means the opposite.
Pending → don't reach for logs	No containers yet, so nothing produced output. Read the phase, then events.	Covered in the checkpoint above. It recurs here because it is the instrument error that costs the most clock.

🔖 **Snag:** Released is not Available. When a PVC is deleted, its PV moves to Released: the claim is gone but the previous claimant's data is still there, so the volume is *not* free for a new claim [source: k8s-docs-persistent-volumes-depth-2026-08-25]. Under Retain, an administrator reclaims it by hand. A stem describing a PV that "should be reusable but isn't" is usually here.

Domain 3 — Cloud Native Application Delivery (Ch 14–16)

Pair	The one-line test	The tell in a stem
chart vs release vs revision	Chart = the package. Release = one <i>installed instance</i> of it, with a name. Revision = a numbered version of <i>that release</i> , incrementing on each upgrade.	Installing the same chart twice gives two releases. Upgrading one release gives it a second revision.
charts/ directory vs chart repository	charts/ is a directory inside a chart holding its subcharts, which are dependencies vendored in. A chart repository is a remote place charts are fetched from . Same word, completely different scope.	"Where do dependencies live?" is charts/. "Where did this chart come from?" is a repository.
Kustomize overlay vs Helm template	Patch, or render? Kustomize starts from a base that is already a valid, applicable manifest and patches it per environment. Helm starts from a template that is not a manifest until values are rendered into it.	If the un-customized files could be applied as-is, it is Kustomize. If they contain <code>{{ }}</code> and cannot, it is Helm. <i>[cross-bearing: see Ch 14 §6 — which one, when]</i>
kubectl rollout undo vs helm rollback	Whose history is being rewound? <code>rollout undo</code> walks a Deployment's revision history, inside Kubernetes. <code>helm rollback</code> takes a release name and a revision number and rolls that release back to that revision [source: helm-rollback-cli-2026-08-31]. Different histories, different scopes, same English word.	A stem mentioning a release name and a revision number is Helm. A stem about a Deployment's previous ReplicaSet is <code>rollout undo</code> .
rolling / Recreate / blue-green / canary	What happens to the old version? Rolling replaces it incrementally. Recreate stops it all first. Blue-green runs both and switches traffic at once. Canary sends a fraction of traffic to the new one and watches.	"Zero downtime, no spare capacity available" points at rolling. "The two versions must never be live simultaneously" points at Recreate. Ch 15 §2 owns the vocabulary; Ch 6 §4 owns the mechanics.
GitOps: push vs pull	Where do the credentials live? Push: an external CI system holds cluster credentials and reaches in. Pull: an agent inside the cluster holds them and reaches out to Git.	The whole blast-radius argument turns on this. GitOps is pull.
OutOfSync = drift, not error	It reports a difference between the repository and the cluster. Something changed on one side. That is a status, not a failure.	A stem treating OutOfSync as an error state is testing whether you know it is a comparison result.
troubleshooting scope: platform vs application	Is the Pod running the way Kubernetes intended? If not, platform. If it is running fine and the app misbehaves, application.	This is the Ch 13 / Ch 16 split. Running plus wrong output means you have crossed into application scope.
which instrument	The layer picks the instrument, and the phase tells you the layer. Platform-layer questions are answered by <code>kubectl describe</code> , events, and node conditions.	A stem where the Pod is Running and the suggested tool is <code>describe</code> has crossed layers in one direction; a stem where the Pod is Pending

Pair	The one-line test	The tell in a stem
nt, by layer	Application-layer questions are answered by logs, exec, ephemeral containers, and port-forward.	and the suggested tool is logs has crossed in the other. <i>[cross-bearing: see Ch 16 §3 – getting inside, and adding what isn't there]</i>

Domain 4 — Cloud Native Architecture (Ch 17–18)

Pair	The one-line test	The tell in a stem
mesh control plane vs cluster control plane	Whose control plane? The cluster's is kube-apiserver, etcd, scheduler, controller-manager. A mesh's is a separate thing configuring proxies.	The word alone is ambiguous; the stem's subject disambiguates.
sidecar vs ambient mode	Proxy per Pod , or per node ? Ambient mode uses "a per-node Layer 4 (L4) proxy, and optionally a per-namespace Layer 7 (L7) proxy" [source: istio-ambient-mode-2026-08-31].	Both are the same mesh with the same engine — the waypoint proxy "is a deployment of the Envoy proxy; the same engine that Istio uses for its sidecar data plane mode" [source: istio-ambient-mode-2026-08-31]. They are two arrangements, not two products. A stem asserting "a service mesh requires sidecars" is wrong.
Knative Serving vs Eventing	Serving is HTTP request-driven: it "manages the complete lifecycle of stateless HTTP services, including deployment, routing, and automatic scaling (including scale to zero)" [source: knative-overview-2026-08-23]. Eventing is asynchronous: "a CloudEvents-over-HTTP asynchronous routing layer" [source: knative-overview-2026-08-23].	Request in, response out → Serving. Event produced somewhere, consumed elsewhere, producers and consumers decoupled → Eventing.
maturity-level ordering	Sandbox → Incubating → Graduated. Entry, growing, proven.	The levels are the durable fact; which specific project sits at which level changes and is not a reliable memorization target.
TOC vs Governing Board	Technical or business? The Governing Board is "responsible for marketing and other business oversight and budget decisions" [source: cncf-charter-governance-bodies-2026-08-31]. The TOC facilitates "defining and maintaining the technical vision" [source: cncf-charter-governance-bodies-2026-08-31].	Money and marketing → Board. Technical direction → TOC.
End User TAB vs TOC	Advises, or holds the vision? The End User TAB "serve[s] as the voice of End Users in the CNCF community, advance[s] topics of concern to End Users, and raise[s] awareness about the needs and perspectives of end users" [source: cncf-charter-governance-bodies-2026-08-31]. The TOC holds the technical vision.	An option that has the TAB approving, blocking, or deciding anything is wrong — it advances topics and raises awareness; it does not decide.
SIG vs Working Group vs	SIGs are ongoing and topic-owning. Working Groups are "time bounded" and cross SIG lines [source: k8s-sig-list-and-groups-2026-08-31]. Committees have closed membership and do not always operate in the open [source: k8s-community-governance-2026-08-23].	Note there are exactly three Committees — Code of Conduct, Security Response, and Steering [source: k8s-sig-list-and-groups-2026-08-31] — and Steering holds overall project governance while chartering the other two

Pair	The one-line test	The tell in a stem
Commit tee		[source: k8s-community-governance-2026-08-23].
TAG vs SIG	TAGs are CNCF-level: "the primary organizational units within the CNCF that oversee and coordinate interests across projects" [source: cncf-tags-current-structure-2026-08-31]. SIGs are Kubernetes-project-level. Different organizations, different scope.	These are easy to confuse for a documented reason: CNCF's groups were originally called SIGs. The TOC "approved the creation of SIGs, later to be renamed Technical Advisory Groups" [source: cncf-tags-current-structure-2026-08-31]. The shared origin is why the names collide.
horizontal vs vertical scaling	More replicas, or bigger ones? Horizontal adds Pods. Vertical adjusts the resources of existing ones.	"Handle more concurrent requests" is usually horizontal. "This Pod keeps hitting its memory limit" is vertical.
HPA built-in vs VPA add-on	HPA is part of the core API. The VPA "doesn't come with Kubernetes by default, but is a a[sic]n add-on that you or a cluster administrator may need to deploy" [source: k8s-docs-autoscaling-and-vpa-2026-08-31], and additionally requires metrics-server installed [source: k8s-docs-autoscaling-and-vpa-2026-08-31].	This is thread 3 in Domain 4 clothing, and the absent component fails at a <i>different layer</i> here. With Ingress, the object is accepted and nothing happens. With VPA, the API rejects the kind outright, because without the add-on the CRD does not exist. Same pattern, two symptom shapes.
workload vs node autoscaling	HPA/VPA/KEDA change <i>workloads</i> . Cluster Autoscaler and Karpenter change the <i>cluster</i> — "Scaling the cluster infrastructure normally means adding or removing nodes" [source: k8s-docs-autoscaling-and-vpa-2026-08-31].	The trigger for node autoscaling is Pods that "can't be scheduled on existing Nodes" [source: k8s-docs-node-autoscaling-2026-08-31], which is thread 8 arriving from the other direction.
observability vs monitoring	New questions or known ones? Monitoring watches things you decided in advance to watch. Observability is whether you can ask a question you did not anticipate.	Health probes are neither: they are health checking, and a stem conflating probes with observability is testing that boundary.
span vs trace	One hop, or the whole journey? A span is a single unit of work; a trace is the tree of spans following one request across services.	Traces are one of OpenTelemetry's signals, alongside metrics, logs, and baggage [source: opentelemetry-signals-2026-08-23].
SLI vs SLO vs SLA	Measure, target, or consequence? An SLI is "a carefully defined quantitative measure of some aspect of the level of service." An SLO is "a target value or range of values for a service level that is measured by an SLI." An SLA is "an explicit or implicit contract with your users that includes consequences of meeting (or missing) the SLOs they contain" [source: sre-book-service-level-objectives-2026-08-31].	The clean test: " <i>what happens if the SLOs aren't met?</i> ": <i>if there is no explicit consequence, then you are almost certainly looking at an SLO</i> " [source: sre-book-service-level-objectives-2026-08-31].
Prometheus pull vs	Prometheus scrapes targets. The Pushgateway is "an intermediary service which allows you to push metrics from jobs which cannot be scraped" [source: prometheus-	Options offering the Pushgateway for a long-running service, or for a job tied to one specific machine, are ruled out by the source itself.

Pair	The one-line test	The tell in a stem
Pushgateway	pushgateway-practices-2026-08-31], and its use is deliberately narrow: "The only valid use case for the Pushgateway is for capturing the outcome of a service-level batch job" [source: prometheus-pushgateway-practices-2026-08-31].	

Surface-form homonyms

Thirteen words in this book carry two genuinely different concepts. When one of these appears in a stem, the first move is to establish *which sense*, because half the options will be constructed from the other one.

Word	Sense A	Sense B
namespace	Linux kernel isolation primitive (Ch 2)	Kubernetes scope object (Ch 4)
control plane	The cluster's (Ch 3)	A service mesh's (Ch 17)
sandbox	Sandboxed runtime — gVisor, Kata (Ch 2)	CNCF Sandbox maturity level (Ch 17)
revision	Deployment revision (Ch 6)	Helm release revision (Ch 14)
rollback	kubectl rollout undo (Ch 6)	helm rollback (Ch 14) — and rollback-by-revert (Ch 15)
label	Kubernetes label (Ch 4)	Prometheus metric label (Ch 18)
request	Resource request (Ch 5)	API request (Ch 8)
binding	Scheduler binding (Ch 7)	PV/PVC binding (Ch 11) — and RoleBinding (Ch 12)
release	A Kubernetes minor release (Ch 8)	A Helm release (Ch 14)
Service	The Kubernetes object (Ch 9)	A Knative Service (Ch 17)
immutable	Image immutability (Ch 2)	Immutable infrastructure (Ch 17)
operator	The operator pattern (Ch 6)	"Cluster operator" as a Gateway API role name (Ch 10)
volume	A volume in a Pod spec — a directory the containers share (Ch 11 §1)	A PersistentVolume — a cluster resource with its own lifecycle (Ch 11 §2)

One more word that is not a homonym but behaves like one: **plugin** is never used bare in this book. It is always qualified by its interface: CNI plugin, scheduler plugin, admission plugin, device plugin. An option that says "the plugin" without saying which is not telling you enough to answer, and that is usually deliberate.

⚠ Navigational Hazards

Four pairs where the *intuitive* answer is the wrong one. These are worth more attention than the rest of this section combined, because intuition will actively work against you and confidence will feel high.

ReadWriteOnce does not mean one Pod. It means one *node*, and multiple Pods on that node can mount it read-write [source: k8s-docs-persistent-volumes-depth-2026-08-25]. The mode that means one Pod is ReadWriteOncePod. The English is genuinely misleading, which is why ReadWriteOncePod exists to name the other case explicitly.

storageClassName: "" does not mean "use the default." It means *no class*: bind only to a PV that also has no class. It is an explicit opt-out [source: k8s-docs-persistent-volumes-depth-2026-08-25]. Omitting the field entirely is the thing that engages default behavior.

A second default IngressClass does not give you more coverage. Two defaults is an ambiguous configuration, not a redundant one. The rule to hold is that "default" is a singular role.

Running does not mean "the application works." It means the Pod was bound to a node, all its containers were created, and at least one is running or starting. The app inside may be crashing, misconfigured, returning 500s, or not listening on the port the Service targets. Phase is a lifecycle position, not a health verdict.

🧠 **Mnemonic:** For the four hazards: **Once is a node, empty is nothing, one default only, Running is not working.**

🕒 Taking Your Bearings: The Pairs

Two questions. Supply the **discriminator**, not the definition. If your answer reads like a dictionary entry, it will not survive the clock.

Q4. A colleague says: "Our Ingress is deprecated, so we need to migrate to Gateway API before our next upgrade or traffic will break." Identify every error in that sentence, and give the one-line discriminator for the pair being collapsed. [retrieval: ch10]

Q5. You're handed a stem containing the word "revision" and four options. Two options describe Kubernetes objects, two describe Helm concepts, so the ambiguity is *cross-tool*, not internal to either one. Before reading any option closely, what single question resolves which family the stem belongs to? Then state the same for "rollback." [retrieval: ch6, ch14]

Answers with Explanations:

Q4 — Two errors. Ingress is frozen, not deprecated; and nothing breaks on upgrade.

Frozen means feature-complete and still supported: the API is stable, it continues to work, and new development happens in Gateway API instead. *Deprecated* would mean scheduled for removal with a migration deadline. Those are different commitments and Ingress carries the first one.

The discriminator: **frozen means no new features; deprecated means an end date.** A frozen API is safe to keep using; a deprecated one is a countdown.

The second error follows: because Ingress is not deprecated, there is no upgrade at which traffic breaks. Migrating to Gateway API is a choice made for its capabilities, the role-oriented design and the richer routing, not a deadline being met.

If you caught "deprecated" but not "traffic will break," you got half. The practical consequence is the half that shows up in a stem about planning.

Q5 — For "revision": whose history? For "rollback": whose history is being rewound?

It is the same question both times, which is the useful part.

Revision. A Deployment revision is a point in that Deployment's rollout history, produced by changing its Pod template, and reachable with `kubectl rollout history`. A Helm revision is a numbered version of a *release*: the whole installed instance of a chart, which may contain a dozen Kubernetes objects. So: does the stem name a Deployment and its ReplicaSets, or a release name?

Rollback. `kubectl rollout undo` rewinds a Deployment to a previous revision of *that Deployment*. `helm rollback` takes "the name of a release" and "a revision (version) number" and rolls that release back to that revision [source: helm-rollback-cli-2026-08-31]. The scope differs by an order of magnitude even though the English word is identical.

There is a third sense worth having in reserve: GitOps *rollback by revert*, where you undo the commit and let the agent reconcile. That one is always written out in full precisely because the bare word is overloaded.

The tell that resolves it fastest: **a release name plus a number is Helm. A Deployment plus ReplicaSets is `rollout undo`. A commit is GitOps.**

If you reached for "*package, install, or version of it?*", you resolved the wrong split. That discriminator separates the three Helm terms from each other, which is a real and useful test, but it is internal to Helm. This stem's ambiguity was cross-tool. Both discriminators are correct; the stem tells you which one is in play, and reading the stem for that is the actual skill.

Checkpoint: You've Now Confirmed ✓ You can separate frozen from deprecated, and state the practical consequence ✓ You resolve a homonym by asking whose scope the stem is in, before reading options

Missed either? Q4 sends you to §2's Domain 2 table, Q5 to the homonym table just above. Both are twenty-minute repairs against material already in front of you, not re-reads.

Two sections of content behind you. The rest of this chapter is strategy: how to spend ninety minutes, where to spend your remaining study hours, and what to do with the last week.

§3 — Ninety Minutes

The Linux Foundation allows **90 minutes** for its multiple-choice exams [source: lf-mc-exam-faq-2026-08-31]. That number is published on the KCNA exam page [source: lf-kcna-exam-page-2026-08-23] and in the candidate handbook [source: lf-mc-exam-important-instructions-2026-08-31], and it does not move. Two further facts shape everything in this section:

The timer cannot be paused. "The system does not offer a way to pause the exam timer or to add time back to the exam during connection loss events" [source: lf-handbook2-taking-the-exam-2026-08-31]. Not for a bathroom break, not for a dropped connection. Ninety minutes is a hard, unrecoverable budget.

There is no scratch paper. "Candidate is not allowed to write or enter input on anything (whether paper, electronic device, etc.) outside of the Exam console screen" [source: lf-exam-rules-and-policies-2026-08-31]. You cannot write your pacing plan down when the clock starts. Whatever plan you use has to be executable from memory, which is the entire reason the rule below is short enough to memorize.

The rule

☆ Fixed Point

Read the question count off the screen. Divide the clock by it. Reach the end of the paper at roughly 60% of your total time, and hold the remaining 40% in reserve.

On a 90-minute exam, that means your first pass through every question ends at roughly the 54-minute mark, leaving about 36 minutes in reserve. You do not need to know the question count in advance — you read it, you divide, you go.

This is stated as a rule rather than a seconds-per-question number on purpose. The Linux Foundation publishes a 60-question format for its multiple-choice exams, in the candidate handbook, for that class of exam [source: lf-mc-exam-important-instructions-2026-08-31]. As a worked example: 60 questions in 90 minutes is 90 seconds apiece, and a first pass finishing at 54 minutes gives you about 54 seconds per question on the way through, with the balance held back. But the rule is what you memorize, because the rule survives a different count and the arithmetic does not.

📖 **Worth Securing:** The 60-question and 75% figures are published: in the Linux Foundation's *candidate handbook*, for multiple-choice exams as a class, not on the KCNA product page you probably read first [source: provenance-kcna-60-questions-2026-08-31]. That gap between where a fact lives and where a candidate looks is why the numbers circulate for years looking unsourced. Read the handbook, not just the product page.

What the reserve is for depends on the interface

The Linux Foundation does not publish how its multiple-choice exam console behaves: whether you can skip a question, mark one for review, navigate back, or change an answer you have already given. That silence is not an oversight in this book's research; it is confirmed absent on four separate official pages, each of which documents the exam in detail and says nothing about question navigation [source: lf-mc-exam-important-instructions-2026-08-31] [source: lf-mc-exam-faq-2026-08-31] [source: lf-exam-rules-and-policies-2026-08-31] [source: lf-handbook2-taking-the-exam-2026-08-31].

So establish it yourself, on the tutorial screen, before the clock matters. Look for a flag or mark control, and look for whether the console will let you move backward. Thirty seconds of checking buys you the shape of the next ninety minutes.

If you can navigate back, the reserve is a genuine second pass: you return to the questions you marked, in the order you marked them.

If you cannot, the reserve is a margin instead. It is the extra time you can afford to spend on a hard question *in place*, one at a time, without running out. You are still finishing the sweep at 60%; you are simply spending the balance as you go rather than at the end.

Either way the failure mode the rule guards against is identical, and it is the one that actually costs people the exam: spending four minutes on question nine.

The first pass

Three categories, decided fast:

Know it. Answer, move. Do not re-read to confirm. Confirmation on a question you knew is the single largest source of time leakage on a timed exam, and it almost never changes the answer.

Can get it. Two options survive, and you can see the discriminator if you think for twenty seconds. Give it the twenty seconds. If it resolves, answer and move. If it does not, pick the more likely one, mark it if the console allows, and move.

Don't know it. Answer anyway. Reason it through rather than taking it on faith: a question you leave blank scores zero with certainty, so putting an answer in cannot score worse *unless* there is a penalty for a wrong answer, and the Linux Foundation does not publish one in either direction. Given a certain zero against an uncertain penalty, answer. Then mark it and move immediately. Do not sit with it. The cost of sitting is that you spend three minutes on one question and then rush four you would have got.

Always leave an answer even when you mark. A marked question with an answer already in it is a question you can improve; a marked blank is a question you can lose to the clock.

The second pass

Your marked questions, in the order you marked them. Two things have changed since you first saw them:

You have read the whole exam, and later questions sometimes disambiguate earlier ones: a stem that names a concept precisely can tell you which sense an earlier ambiguous stem meant. And you are no longer under first-pass momentum, so you can afford the full discriminator procedure from §2.

Change an answer only when you can say why. A reason you can state out loud is evidence. "It feels wrong now" is usually fatigue wearing the costume of insight. "I misread the scope: it said RoleBinding, not ClusterRoleBinding" is a reason, and a reason is what the second pass is for.

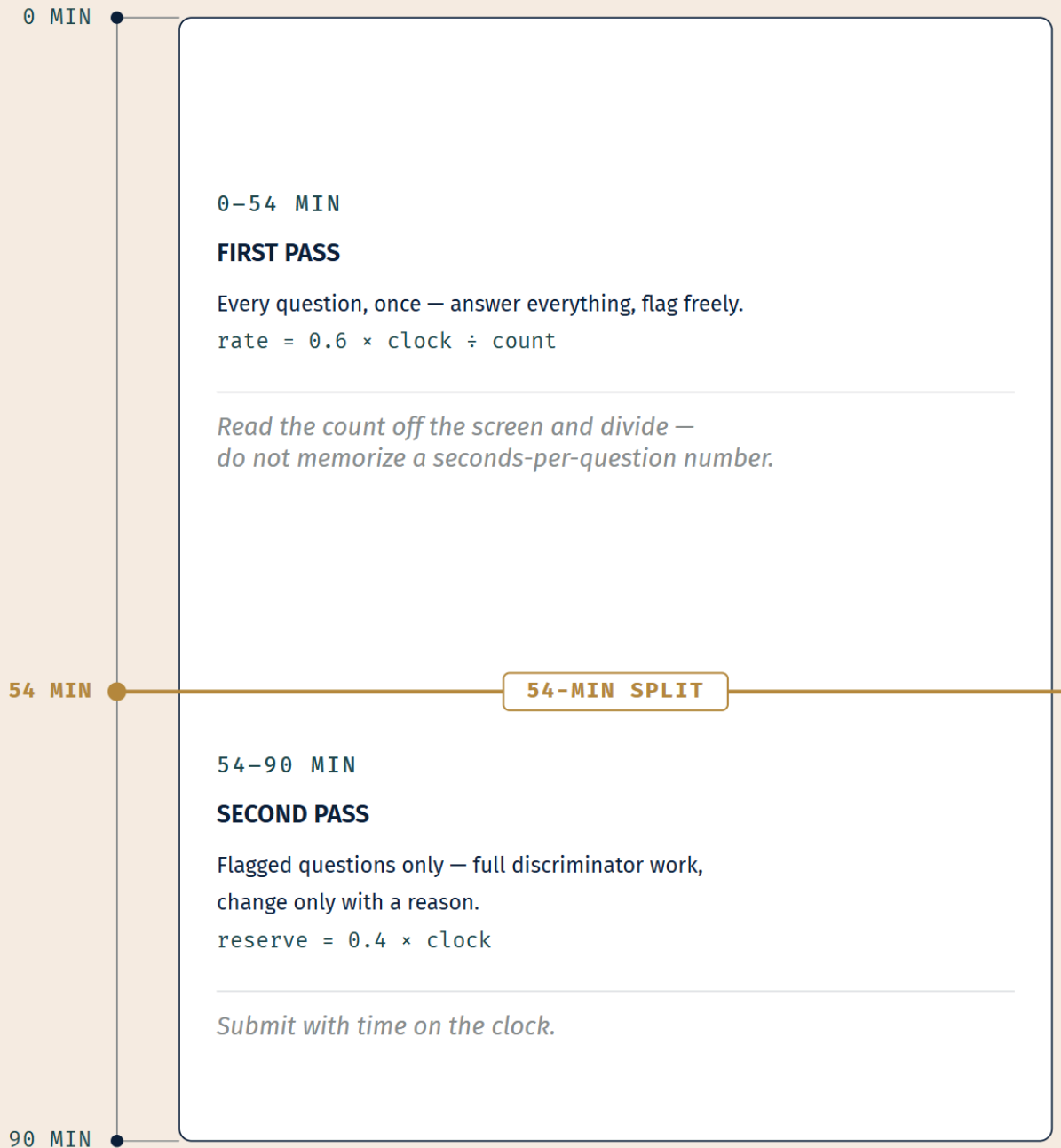
The three ways candidates run out of time

They have different fixes, which is why they are worth separating.

Re-reading stems. You read the question, read it again, then read it a third time before touching an option. Fix: read once, carefully, then go to the options. If the options reveal you missed something, you can come back, but you will usually find that reading the options *is* what clarifies the stem.

Refusing to leave a question unresolved. You will not move on, so you spend four minutes fighting one item. Fix: a navigator who takes a bearing, cannot resolve it, and comes back when the light is better has not given up on the fix; they have waited until the fix was possible. Marking is that, and nothing more.

Re-litigating settled answers. You keep returning to questions you already answered confidently. Fix: the reserve is for the ones you marked. If you did not mark it, you do not revisit it.

**LEGEND**

- Time mark
- Focal — 54-min split

Segment length is proportional to elapsed time.

One more thing about the environment, because it costs points every year: the exam is closed-book. "Candidates are NOT PERMITTED to access tools, resources or external sites when taking the Linux Foundation Multiple Choice OR SkillCred Exams" [source: [lf-certification-resources-allowed-2026-08-31](#)]. If you have read CKA preparation advice about having [kubernetes.io](#) open in a browser tab, that allowance is real, but only for the *performance-based* exams, which permit browsing the Kubernetes documentation during the exam [source: [lf-certification-resources-allowed-2026-08-31](#)]. It does not transfer to KCNA. Nothing is open. Plan accordingly.

§4 — Where the Weight Actually Is

Study time is not fungible with comfort. The hours you most want to spend are usually on material you already know, because reviewing what you know feels good and reviewing what you do not feels like failing. The stretch of water you most enjoy sailing is rarely the one that puts you on the rocks. This section is a correction for that.

Dead Reckoning: The four domains and their published weights are: Kubernetes Fundamentals 44%, Container Orchestration 28%, Cloud Native Application Delivery 16%, Cloud Native Architecture 12% [source: [cncf-curriculum-kcna-readme-2026-08-31](#)]. Those weights come from CNCF's own curriculum repository and agree exactly with the Linux Foundation exam page and the published curriculum PDF. The mapping of domains onto this book's chapters is editorial. It is how *this book* laid the material out, not something CNCF publishes: Kubernetes Fundamentals is Chapters 2–8, Container Orchestration is Chapters 9–13, Cloud Native Application Delivery is Chapters 14–16, and Cloud Native Architecture is Chapters 17–18.

Build the table yourself

Do not skip this. The value is in the filling-in, not the reading. You are going to see something about your own preparation that a printed table cannot show you.

Go back through the book and record your **Soundings score** and your **Taking Your Bearings scores** for each chapter. Then complete this:

Domain	Weight	Chapters	Your worst Bearings score in this range	Which chapter	Hours you've spent here
D1 Kubernetes Fundamentals	44%	2–8			
D2 Container Orchestration	28%	9–13			
D3 Cloud Native App Delivery	16%	14–16			
D4 Cloud Native Architecture	12%	17–18			

Then answer one question: **is the domain where you scored worst the domain where you have spent the most hours?**

If the answer is no, and for most readers it is no, that gap is your study plan. It is almost always more valuable than any general advice this book could give you, because it is measured on you.

Two calls this section has to make

Community and Collaboration is the competency easiest to leave until it is too late. It sits inside Cloud Native Architecture (Chapter 17 §2 and §8) and it is the material a reader who ran short of time will have stopped before. The reasons are in the material itself: it is institutional rather than technical, it is name-dense, and it does not *feel* examinable in the way a `kubectl` behavior does. It is examinable. It is on the published curriculum as one of Cloud Native Architecture's three competencies [source: [cncf-kcna-curriculum-pdf-2026-08-23](#)].

Here is the argument for spending an hour there, stated as arithmetic rather than as a claim about how often it is tested. Cloud Native Architecture is 12% of the exam, split across three competencies, so Community and Collaboration is roughly a twenty-fifth of the paper. What makes it worth the hour is not its share but its *shape*: the material is finite and bounded. Governance bodies and their remits, SIGs versus Working Groups versus Committees, the project lifecycle, the maturity levels. There is no depth to plumb. An hour covers a large fraction of the whole competency. An hour spent adding nuance to a Domain 1 topic you already score well on covers a much smaller fraction of a much larger surface, at a much lower marginal return.

The 2025-11-24 blueprint change is a study-allocation hazard, not just a historical fact. The KCNA exam was updated no earlier than November 24, 2025 [source: [lf-kcna-program-changes-2026-08-23](#)], and the change was not cosmetic: Cloud Native Application Delivery is now 16%, and observability was "rolled under Cloud Native Architecture" [source: [lf-kcna-program-changes-2026-08-23](#)] rather than being its own domain.

The practical consequence: **third-party study material produced before that date allocates its coverage against a blueprint that no longer exists.** If your practice questions came from a bank that still treats

observability as a domain in its own right, your sense of where the points are is wrong. And the rule for which blueprint you are tested against is unambiguous: "Any KCNA exam taken after the updated release will test on the new set of Domains and Competencies," and "The only date that matters is the date you sit for the exam" [source: lf-kcna-program-changes-2026-08-23]. Not your purchase date, not your first attempt, not your retake status.

📌 **Snag:** Check the copyright date on every non-CNCF resource you're using. The tell is where observability sits: if your material lists it as a **domain in its own right**, with its own weight, rather than as a competency under Cloud Native Architecture, that material predates the change.

Where the next block of hours goes

In priority order, and this order holds for most readers:

1. **Your worst-scoring chapter in Domain 1.** It is 44% of the exam. A weak chapter there costs more than a weak chapter anywhere else, arithmetically.
2. **Community and Collaboration**, if you have not deliberately studied it. Bounded material, so an hour covers a large share of it.
3. **§2 of this chapter**, worked actively rather than read. Discrimination failures are cheap to fix and expensive to leave.
4. **Your worst-scoring chapter in Domain 2.** 28% and the second-largest surface.
5. **A timed full-length attempt**, if you have not sat one. Pacing is a distinct skill and it is invisible until you meet the clock.

What is *not* on that list: re-reading chapters you scored well on. It feels like studying. It is not.

§5 — Using The Lodestar

the-lodestar.md ships with this book, at the repository root. It is the one-page reference: the concentrated form of everything worth having in front of you in the hour before you sit down, and nothing else.

What is in it

Domain weights and the exam's published shape. The 44/28/16/12 split, the 90 minutes, and the handbook-published question count and passing standard. This block is *lookup*: you are not memorizing it, you are checking it so the numbers are fresh.

The confusion-pair discriminators. A compressed form of §2: the pair and its one-line test, nothing else. This block is *drill*. Cover the right column, work down the list, uncover. This is the block that repays a second pass.

The four hazards where intuition is wrong. ReadWriteOnce is a node, "" is nothing, one default only, Running is not working. Four lines. Drill block.

The version-skew numbers. Three numbers that are easy to swap under pressure and cheap to refresh: kubelet three back, kubect1 one either way, three supported minors. Lookup block, but read it twice.

Exam-day pacing. The §3 rule in one line, because it must be executable from memory and there is no scratch paper.

Governance and institutional vocabulary. The Board/TOC split, the End User TAB's advisory role, SIG/WG/Committee, maturity levels. This is the Domain 4 material most likely to have gone soft, and it is the cheapest to refresh because it is bounded.

How to use it in the last hour

Two passes, about twenty minutes each, with a break between.

First pass — drill. Cover the answers. Work the discriminator blocks and the hazard block. Where you hesitate, mark it. Do not look anything up in the book; the point is to find soft spots, not to fix them.

Second pass — lookup. Read the marked items and the number blocks. Once, calmly. Then close it.

Then stop. Whatever is not in your head at that point is not going to arrive in the next twenty minutes, and the marginal value of a third pass is negative: it costs you composure and buys you nothing.

What it deliberately leaves out

The Lodestar is a **distillation, not a summary**. It contains no chapter recaps, no explanations, no worked examples, and no context. Every line assumes you have read the chapter it came from.

That is a design decision with a consequence worth stating plainly: **if you cannot reconstruct a chapter from its Lodestar lines, you have not found a defect in the Lodestar**. You have found a chapter you need to go back to. Use that as a diagnostic in the final week: a Lodestar line that means nothing to you names a chapter to re-read. In the last hour, though, it is too late to act on that, which is why the final week comes first.

§6 — The Week Before

The last week is the one most likely to be spent badly, because anxiety pushes toward *more*: more material, more sources, more hours. More is the wrong direction. Consolidation, not acquisition.

Seven days out

Sit one full-length timed attempt. Ninety minutes, no pauses, nothing open, in one sitting. Not to score, but to calibrate. You want to know whether you finish with time to spare, finish exactly on the buzzer, or run out. Those three outcomes have completely different fixes and you cannot guess which one you are.

Review the wrong answers properly. For each one, name *why*: knowledge gap, discrimination failure (a \$2 pair), misread stem, or clock. The four have different remedies, and sorting them is worth more than the raw score.

Six to three days out

Work the two or three chapters your attempt flagged. Not the whole book. Named chapters, from measured evidence.

Work \$2 of this chapter actively. Cover the right column, say the discriminator out loud, check. Twenty minutes a day. Discrimination is the thing that goes soft fastest and comes back quickest, which is what makes it the best use of a short daily block.

Give Community and Collaboration a dedicated hour if you have not. See \$4.

Check your equipment now, not on the morning. This is a genuinely important use of a final-week hour, because every item here can fail and every one takes time to fix. You need a supported OS, one active monitor ("Dual Monitors are NOT supported"), a microphone, and reliable internet; a wired connection is more stable than wireless [source: lf-handbook2-candidate-requirements-2026-08-31]. You also need a webcam you can physically move: "Ensure the webcam is capable of being moved as you will have to pan your surroundings" [source: lf-mc-exam-faq-2026-08-31]. You cannot use a virtual machine [source: lf-mc-exam-important-instructions-2026-08-31]. And avoid work-provided devices: the Linux Foundation states plainly that they "can result in technical challenges" [source: lf-mc-exam-faq-2026-08-31]. It does not say what those challenges are, but a managed endpoint and a secure browser are not natural allies, and the failure would land at exactly the wrong moment.

Check your ID against your registration. The first and last name on your government-issued photo ID must exactly match the name on your Linux Foundation account [source: lf-handbook2-taking-the-exam-2026-08-31]. A mismatch discovered at check-in is not something a proctor can wave through.

Two days out

Prepare the room. The space must be private, clutter-free, and well lit, with no papers or writing implements on the desk [source: lf-handbook2-candidate-requirements-2026-08-31] and none below it either: "no objects such as paper, trash bins, or other objects below the testing surface" [source: lf-mc-exam-faq-2026-08-31]. Walls must be clear of printouts, though "paintings and other wall décor is acceptable" [source: lf-mc-exam-faq-2026-08-31]; décor is fine, notes are not. Public spaces are prohibited outright: "coffee shops, stores, open office environments, etc. are not allowed" [source: lf-mc-exam-faq-2026-08-31]. If the room has a door, it must be closed [source: lf-handbook2-candidate-

requirements-2026-08-31]. Sorting this two days out costs ten minutes; sorting it while a check-in specialist waits costs composure you would rather spend on the exam.

Stop adding sources. No new question banks, no new videos, no new guides. A ship taking on stores in the last hour before sailing is not better provisioned; it is just harder to trim. A new source in the final 48 hours does not add knowledge. It adds a fresh sense of everything you do not know, which is the opposite of what you need.

The day before

Do not study new material. This is where Community and Collaboration typically gets crammed, and it typically does not stick. If you did not learn it in week one, the twelve hours before an exam are not the conditions under which it is going to arrive.

One Lodestar pass in the morning, then stop. Twenty minutes.

Rest. This sounds like the sort of advice a study guide includes because study guides include it. Take it as the specific recommendation it is: a tired reader misreads stems, and misread stems are one of the four failure modes you sorted your wrong answers into a week ago. It is the only one of the four you can still do anything about on the last night.

The morning

The morning of a departure is not the time to discover the chart is missing. Everything above was so that this morning consists only of casting off.

Check in early. The portal opens up to 30 minutes before your scheduled time, and you must start no later than 30 minutes after it [source: lf-handbook2-taking-the-exam-2026-08-31]. Expect a wait for a check-in specialist; it "should not exceed 15 minutes" [source: lf-handbook2-taking-the-exam-2026-08-31], but building that in means it is not a surprise. You will upload a photo of your ID and take a selfie for comparison [source: lf-handbook2-taking-the-exam-2026-08-31], and you will pan the room with your webcam.

Then, when the exam is released: read the count off the screen, divide, and go.

Logbook Entry: The final week has a texture that nobody warns you about, so here it is.

Around day four, you will probably have a bad session. You will get a run of practice questions wrong on material you were confident about three days earlier, and the conclusion that will arrive uninvited is that you have been fooling yourself and you are not ready. This is common enough to be worth naming in advance, and it is worth much less as a signal than it feels like. What is usually happening is that you have moved from recognition practice to retrieval practice, and retrieval is harder. Recognition was always the easier task and it was always flattering you.

The failure mode is what people do next. Faced with a bad session, the instinct is to widen: buy another question bank, find a different video series, start a fresh set of notes. Every one of those is a way of avoiding the specific thing that went wrong, which is generally three or four discriminations that have gone soft and could be repaired in forty minutes with the material already in front of you.

The other thing worth knowing: on exam day you will encounter a question you have no idea about. Not a hard question — a question about something you would swear was never covered. Everyone does. It is one question out of the sixty the handbook describes, it is worth the same as every other question, and the only mistake available to you is letting it take four minutes and your composure. Answer it, mark it, move. You will very likely never think about it again, because you will have passed.

Exam Alert! 🚨

Kept short deliberately. §2 is the trap section, and repeating it here would waste the pass you just made through it.

High-Priority Topics:

1. **The absent-component pattern, in its exact form:** *an object without its component does nothing*. It converts at least five separate gotchas — Ingress, NetworkPolicy, CSI, metrics-server, VPA — into one rule you apply on sight.
2. **The RBAC four-way binding matrix, derived rather than memorized.** The binding's scope sets the grant's scope. Everything else falls out of Chapter 4's namespaced/cluster-scoped distinction.
3. **Additive, allow-only, no deny.** One semantic shared by RBAC and NetworkPolicy, built independently by two different groups.
4. **The version-skew numbers:** kubelet up to three minor versions back and never newer, kubectl within one either way, three supported minor releases [source: k8s-version-skew-policy-2026-08-31].
5. **Community and Collaboration vocabulary.** Bounded, finite material, so an hour covers a large fraction of the whole competency, and it is the competency most likely to have been skipped.

Common Traps — the four where the intuitive answer is the wrong one:

- `ReadWriteOnce` reads as "one Pod." It is one *node* [source: k8s-docs-persistent-volumes-depth-2026-08-25]. `ReadWriteOncePod` is the one that means one Pod.
 - `storageClassName: ""` reads as "use the default." It means *no class* [source: k8s-docs-persistent-volumes-depth-2026-08-25].
 - **A second default IngressClass** reads as broader coverage. It is ambiguity.
 - `Running` reads as "the application works." It means bound to a node, all containers created, at least one running or starting. Nothing more.
-

Practice Questions

Ten questions, cross-domain by construction. Most require two chapters to answer, which is the point of a synthesis chapter. Work them under something like exam conditions: no looking back, one pass, then check.

1. A Pod has been in Pending for ten minutes. Which sequence of diagnostic steps is correct?

- A) `kubectl logs` → `kubectl exec` → read the container's stdout B) `kubectl top pod` → compare utilization to limits → raise the limit C) `kubectl describe pod` → read events → compare requests to node capacity D) Restart the kubelet on the target node, then re-check the phase
-

2. A cluster has a `NetworkPolicy` in namespace `api` selecting Pods labeled `tier=backend` and allowing ingress only from Pods labeled `tier=frontend`. A Pod labeled `tier=batch` in the same namespace can still reach the backend Pods. No other `NetworkPolicy` exists anywhere. What is the most likely cause?

- A) The installed CNI plugin does not implement `NetworkPolicy` B) The policy needs an explicit deny rule for `tier=batch` C) `namespaceSelector` must be set even for same-namespace traffic D) The frontend Pods need a matching egress rule
-

3. `kube-apiserver` is at 1.37. Which combination is supported?

- A) `kubelet` 1.38, `kubectl` 1.37 B) `kubelet` 1.33, `kubectl` 1.36 C) `kubelet` 1.37, `kubectl` 1.35 D) `kubelet` 1.34, `kubectl` 1.38
-

4. Which pairing correctly describes who holds the CNCF's technical vision and who makes its budget decisions?

- A) The Governing Board holds the technical vision; the TOC decides the budget B) The TOC holds the technical vision; the Governing Board decides the budget C) The End User TAB holds the technical vision;

the Governing Board decides the budget D) TAGs hold the technical vision; the TOC decides the budget

5. A `RoleBinding` in namespace `web` grants a `ClusterRole` called `pod-reader` to a `ServiceAccount` in namespace `ci`. A second `ClusterRoleBinding` grants the same `ClusterRole` to the same `ServiceAccount`. What can the `ServiceAccount` read?

A) Pods in `web` only — the `RoleBinding` constrains the broader binding
 B) Pods everywhere except `web`, where the `RoleBinding` takes precedence
 C) Nothing — the bindings conflict, and RBAC resolves conflicts by denying
 D) Pods in every namespace, including `web`

6. A `Deployment`'s Pods show phase `Running` and a readiness probe that is failing. What is the observable consequence?

A) The Pod is removed from the `Service`'s endpoints and stops receiving traffic
 B) The Pod is removed from endpoints and the container restarts after the failure threshold
 C) The container is restarted repeatedly until the probe succeeds
 D) The `Deployment`'s rollout is automatically rolled back to the previous revision

7. An application deployed via Helm has been upgraded three times. A bad configuration shipped in the third upgrade. The team wants the previous state of *all* objects the chart manages. Which is correct?

A) `kubectl rollout undo` on the `Deployment` the chart created
 B) Delete the release and reinstall the chart at the previous version
 C) `helm rollback <release> 2`, naming the release and the revision
 D) `kubectl apply` the previous manifests from the chart's templates/

8. A `PersistentVolume` with `persistentVolumeReclaimPolicy: Retain` had its bound PVC deleted. A new PVC with identical requirements is created and stays `Pending`. Why?

A) A `Retain` volume can only ever be bound once
 B) The PV is in `Released`, not `Available`, and must be reclaimed by hand
 C) The new PVC must set `volumeName` to bind to a specific PV
 D) The PV has `storageClassName: ""`, so it binds only to a PVC that omits the field

9. A contributor wants to organize work on a topic that spans SIG Node, SIG Network, and SIG Storage — the three groups behind the runtime, network, and storage interfaces — and the effort is expected to conclude once a design is ratified. Which body fits?

A) A new SIG, since an ongoing topic needs an owning group
 B) A Committee, since the effort crosses several groups' boundaries
 C) A Working Group, which is time bounded and crosses SIG lines
 D) A CNCF TAG, since it coordinates interests across multiple projects

10. A team runs an Argo CD instance that watches a Git repository and applies manifests to the cluster. Which statement correctly describes the security property this delivery model provides?

A) Cluster credentials live in the CI system, so a compromised cluster cannot reach the repository
 B) No credentials are needed, because Git is the source of truth
 C) The agent needs no ServiceAccount, since it acts on cluster-scoped resources
 D) Cluster credentials live inside the cluster, and the agent reaches out to Git

Answers with Explanations:

1 — C. Describe, read events, compare requests to node capacity.

Pending means no container has started, usually because no node was feasible. The scheduler records why, and `kubectl describe pod` surfaces it. Then you compare the Pod's *requests* to node Allocatable — requests are the scheduler's filter input (thread 8).

A is wrong. No containers exist, so there is nothing to log and nothing to exec into. This is the instrument error that costs the most clock, because it produces an empty result rather than an error and invites a second attempt. **B is wrong** for the same reason, and additionally `kubectl top` needs metrics-server installed, which is thread 3 hiding in a distractor. **D is wrong** because there is no target node yet; that is the problem. If you picked it, you have collapsed **scheduler binds / kubelet starts** (§2, Domain 1 table): Pending means the scheduler has not decided, so no kubelet is involved to restart.

Chapters: 5 (phase vs state), 7 (requests as filter input, Pending), 13 (triage order).

2 — A. The installed CNI plugin does not implement NetworkPolicy.

This is the absent-component pattern (thread 3) in Domain 2 clothing. The NetworkPolicy object is valid and accepted by the API server; nothing enforces it unless the installed CNI plugin implements policy. Creating the object is half the transaction, and the symptom of the missing half is precisely this: traffic that the policy should have blocked flowing anyway.

B is wrong and is the most instructive distractor: NetworkPolicy is additive and allow-only, with no deny rule at all. There is nothing to add for `tier=batch`; the policy already restricts the backend to frontend traffic *if it is enforced*. If you picked B, thread 9 is soft; go back to Ch 10 §6 and Ch 12 §3. **C is wrong.** `podSelector` alone covers same-namespace sources; `namespaceSelector` is for crossing namespaces. **D is wrong.** The batch Pods are not selected by any policy, so their egress is unrestricted regardless. The both-ends rule applies only when *both* ends are selected by policies.

Chapters: 10 (NetworkPolicy semantics, absent-component pattern), 9 (CNI as the pluggable interface).

3 — D. kubelet 1.34, kubectl 1.38.

kubelet may be up to three minor versions older than kube-apiserver and must not be newer [source: k8s-version-skew-policy-2026-08-31]; 1.34 is exactly three back from 1.37 and supported. kubect1 is supported within one minor version older or newer [source: k8s-version-skew-policy-2026-08-31]; 1.38 is one newer and supported.

A is wrong. kubelet 1.38 is newer than the API server, which is never allowed. **B is wrong.** kubelet 1.33 is four minor versions back, one too far. **C is wrong.** kubect1 1.35 is two minor versions back, one too far. Every distractor is off by exactly one on a different axis, which is worth noticing as a shape: when a numeric item's options differ by a single increment on separate dimensions, the discrimination being tested is which dimension the rule applies to, not the arithmetic.

Chapters: 8 (version skew), 13 (skew as a cause of misdiagnosed failures).

4 — B. The TOC holds the technical vision; the Governing Board decides the budget.

The charter is explicit on both halves. The Governing Board is "responsible for marketing and other business oversight and budget decisions for the CNCF" [source: cncf-charter-governance-bodies-2026-08-31], and the TOC facilitates "defining and maintaining the technical vision" [source: cncf-charter-governance-bodies-2026-08-31].

A is wrong: a clean inversion, and the most attractive distractor if you have the two bodies but not their remits. **C is wrong.** The End User TAB "serve[s] as the voice of End Users in the CNCF community" [source: cncf-charter-governance-bodies-2026-08-31]; it advances topics and raises awareness, it does not hold the technical vision. **D is wrong.** TAGs coordinate across projects and bridge to the TOC [source: cncf-tags-current-structure-2026-08-31]; they neither hold the vision nor set budgets.

Chapters: 1 (the CNCF as the body behind the credential you are sitting for), 17 §2 (governance bodies and their remits). Domain 4 Community and Collaboration — see §4.

5 — D. Pods in every namespace, including web.

RBAC is purely additive (thread 9). The ClusterRoleBinding grants pod-reader cluster-wide. The RoleBinding grants the same permissions within web. The union is cluster-wide. Nothing constrains, nothing subtracts.

A is wrong and is the trap: a narrower binding does not limit a broader one. Bindings only ever add. **B is wrong** in the opposite direction: it imagines a precedence rule, where the narrower binding somehow overrides inside its own namespace. RBAC has no precedence because it has no deny; with nothing that subtracts, there is nothing for precedence to arbitrate. **C is wrong** for the same underlying reason: no deny means no conflicts to resolve.

Note also what the ServiceAccount's own namespace (ci) did in this question: nothing. A subject's namespace determines *which* ServiceAccount is named, not what it may do.

Chapters: 4 (scope), 12 (binding matrix, additive semantics).

6 — A. Removed from Service endpoints, no restart.

Readiness governs endpoint membership. A failing readiness probe removes the Pod from the Service's EndpointSlice so it stops receiving traffic, and the container keeps running. That is the whole design: a Pod that is alive but not ready to serve should be quietly taken out of rotation rather than killed.

B is wrong, and it is the finest discrimination in the item: it has the endpoint effect right and then imports the restart from liveness. A partial collapse is harder to catch than a total one, which is why it is here. **C is wrong**. Restarting on failure is *liveness*, and collapsing the two probes entirely is the Domain 1 discrimination this item exists to test. **D is wrong**. A rollout can stall waiting on readiness, but nothing automatically rolls back on a probe failure.

Chapters: 5 (probes), 9 (readiness gating endpoint membership).

7 — C. helm rollback <release> 2.

The scope is the whole release. `helm rollback` takes "the name of a release" and "a revision (version) number" [source: `helm-rollback-cli-2026-08-31`] and rolls that release back to that revision.

A is wrong and is the homonym trap: `kubectl rollout undo` rewinds one Deployment's history, not the release's. If the chart also changed a ConfigMap, a Service, and an Ingress, those stay wrong. **B is wrong**. Reinstalling loses release history and is destructive where a rollback is not. **D is wrong**. `templates/` contains Go templates, not rendered manifests; you cannot apply them directly.

Chapters: 6 (rollout undo), 14 (chart/release/revision, helm rollback).

8 — B. The PV is Released, not Available.

When the bound PVC is deleted under `Retain`, the PV "still exists and the volume is considered 'released'" but "is not yet available for another claim because the previous claimant's data remains on the volume" [source: `k8s-docs-persistent-volumes-depth-2026-08-25`]. Reclaiming it is a manual, deliberate act. That is the entire point of `Retain`: it protects data by refusing to silently recycle.

A is wrong. The PV can be reused, after manual reclamation. **C is wrong**. `volumeName` binds a PVC to a named PV but does not change the PV's phase; a `Released` volume still will not bind. **D is wrong**, and it repays a careful read because it inverts one of the four hazards: `storageClassName: ""` on a PV means it binds only to a PVC that *also* explicitly sets `""`, not to one that omits the field. Omitting the field is what engages default behavior [source: `k8s-docs-persistent-volumes-depth-2026-08-25`].

Chapters: 11 (PV phases, reclaim policies), 13 (Pending as a diagnostic starting point).

9 — C. A Working Group.

Two properties in the stem settle it, and both are the Working Group's defining ones: the effort crosses SIG lines, and it is expected to end. The Kubernetes project describes its community as organized "into Special Interest Groups (SIGs) and time bounded Working Groups" [source: `k8s-sig-list-and-groups-2026-`].

08-31], and Working Groups exist to "facilitate topics of discussion that are in scope for Kubernetes but that cross SIG lines," being "short-lived or address[ing] issues spanning multiple SIGs" [source: k8s-community-governance-2026-08-23].

A is wrong. SIGs are ongoing and own a topic area indefinitely. "Concludes once a design is ratified" is the opposite of what a SIG is for, and creating one for a bounded effort would leave a permanent group with nothing to own. **B is wrong.** Committees "do not have open membership and do not always operate in the open" and are "formed by the steering committee for specific topics requiring discretion" [source: k8s-community-governance-2026-08-23]. There are exactly three: Code of Conduct, Security Response, and Steering [source: k8s-sig-list-and-groups-2026-08-31]. Crossing group boundaries is not what makes something a Committee; needing discretion is. **D is wrong** and tests the TAG/SIG collision directly: TAGs are CNCF-level bodies that "oversee and coordinate interests across projects" [source: cncf-tags-current-structure-2026-08-31], while this effort is internal to one project.

Chapters: 17 §8 (SIGs, Working Groups, Committees), 2 / 9 / 11 (the runtime, network, and storage interfaces the three named SIGs stand behind). Domain 4 Community and Collaboration — see §4.

10 — D. Credentials live inside the cluster; the agent reaches out.

This is the pull model, and it is thread 7 — identity — doing the security work. The agent is a Pod with a ServiceAccount holding exactly the permissions its syncing requires. No external system needs standing cluster credentials.

A is wrong. It describes push-based CI, which is the model GitOps is contrasted against, and it inverts the credential location. **B is wrong.** The agent needs credentials for both the cluster and the repository; "Git is the source of truth" is about *where desired state lives*, not about authentication. **C is wrong.** Every Pod has a ServiceAccount, and a delivery agent needs a deliberately-scoped one precisely because it modifies cluster resources.

Chapters: 5 and 12 (ServiceAccount identity), 15 (push vs pull, agent identity).

Chapter Summary

Concept	Remember This
The nine threads	The book is nine patterns with instances, not eighteen chapters with facts. Recognize the pattern in the stem.
The control loop	Desired, observed, act. Ch 3 defines it; Ch 15 points it at Git and nothing else changes.
Namespaced vs cluster-scoped	One Chapter 4 distinction derives the entire RBAC binding matrix. Derive, don't memorize.
Absent component	<i>An object without its component does nothing.</i> Five gotchas, one rule, and it fails at different layers, from a silent no-op to a rejected API kind.
Labels join, RBAC names	Nearly every relationship is a label selector. RBAC is the deliberate exception — permission by label match would be an escalation path.
Additive, no deny	RBAC and NetworkPolicy share one semantic. A "deny" option is testing for imported firewall intuitions.
Discriminators, not definitions	Two definitions side by side is what caused the collapse. One test separates them.
The four hazards	Once is a node, empty is nothing, one default only, Running is not working.
Pacing	Read the count, divide, reach the end at 60%, hold 40% back. No scratch paper, so it must be memorable. Check on the tutorial screen what the console lets you do with the reserve.
Weight allocation	44/28/16/12. Spend hours where your worst score meets the largest weight — usually not where you want to.
Community and Collaboration	Bounded, finite material, so an hour covers a large share of it, and it is the competency most frequently skipped.
The last week	Consolidate, don't acquire. A new source in the final 48 hours adds anxiety, not knowledge.

⚓ Safe Harbor

You have read every chapter, and now you have read the book a second time along a different axis. That second reading is the one that matters. The first gave you the material; this one gave you the shape of it.

Here is what is actually true about where you are. You are not carrying nine hundred facts, and you were never going to. You are carrying nine patterns, a set of discriminators, four weights, and one pacing rule — a load small enough to hand up on deck in the dark, which is the entire reason this chapter re-cut the book instead of summarizing it.

Everything from here is calibration.

The Voyage Ahead

One chapter left, and it is not a chapter — it is the instrument.

Chapter 20 is the full mock exam: a complete sitting, sized to the format the Linux Foundation publishes for its multiple-choice exams, weighted to match the published domain proportions, with answers and full walkthroughs in a separate section so you can attempt it cleanly first.

Take it under real conditions. Ninety minutes on a timer you do not pause. Nothing open. One monitor, closed door, no notes. The point is not the score. It is finding out which of the four failure modes is yours before it costs you anything, and whether the pacing rule in §3 survives contact with an actual clock.

Then, whatever the number says, come back to §4 with it. That is what §4 is for.

"Every navigator makes the same passage twice: once on the chart, and once on the water. The chart passage is the one that decides how the other goes."

Chapter 20: Full Mock Exam

"Ninety minutes. No notes. Find out."

Instructions

You have reached the last chapter, and it is the only one that does not teach you anything.

Nineteen chapters have explained. This one measures. There is no Soundings block here, no Fixed Points, no callouts in the margin: a diagnostic instrument that keeps interrupting itself with encouragement is not a diagnostic instrument. The voice comes back in the walkthroughs, after the clock stops.

Dead Reckoning: The Linux Foundation publishes both of the numbers this instrument is built around. Its candidate handbook gives them for multiple-choice exams as a class: the exam "consists of 60* multiple-choice questions" and candidates "have 90* minutes to complete" it [source: lf-mc-exam-important-instructions-2026-08-31]. The asterisks lead to a footnote naming CNPA, a different exam with 85 questions and 120 minutes; they are not a hedge about the class figures. KCNA belongs to that class: the Linux Foundation's own exam-code table puts KCNA in the multiple-choice column beside KCSA and LFCA, with CKA and CKAD in the performance-based column [source: lf-exam-user-interface-exam-codes-2026-08-31]. Neither number appears on the KCNA product page [source: provenance-kcna-60-questions-2026-08-31]. That is the provenance lesson Chapter 1 spent a section on. *[cross-bearing: see Ch 1 § Ninety Minutes: The Exam as Published]*

So: **sixty questions, ninety minutes.**

What this instrument is, and is not

It is sized to the published count and weighted to the published blueprint: 44% Kubernetes Fundamentals, 28% Container Orchestration, 16% Cloud Native Application Delivery, 12% Cloud Native Architecture [source: cncf-curriculum-kcna-readme-2026-08-31]. Twenty-six questions, seventeen, ten, and seven.

It was written by Lodestar Ledgers. It is not a leaked exam, not a reconstruction of one, and not a prediction of your score. What it gives you is a fix on your own position, taken from your own answers.

One shape here is this book's, not the Linux Foundation's. Every item below offers four options with one correct answer. The Linux Foundation does not publish how many options its multiple-choice items carry, whether any item is multi-select, whether unscored pretest items are mixed in, or whether a wrong answer costs anything [source: lf-examui-multiple-choice-2026-08-31]. Four options and one right answer is a reasonable authoring choice. Do not read it as a disclosure.

Conditions worth reproducing

One sitting. No notes, no cluster, no documentation, no search. Ninety minutes on a timer you can see.

A reader who looks things up is measuring how good this book's index is. That is a fine thing to measure, but it is not the thing you came here for.

How the real console behaves

The Linux Foundation documents the multiple-choice exam interface in its candidate handbook. You move between items with Previous and Next; you can flag an item for later review; flagged items are highlighted on a Review Screen, and you can return to one and change your answer. When you reach the final item you are prompted to review, and the Review Screen carries the Finish Exam button. A Pause Exam function exists, and using it does not stop the timer [source: lf-examui-multiple-choice-2026-08-31].

On paper you can do all of that and more. Flag, move on, come back — and finish the first pass at roughly 54 of the 90 minutes, keeping the rest for the flagged items. Chapter 19 owns the reasoning behind that split and what to do with the second pass; this chapter only asks you to run it. *[cross-bearing: see Ch 19 §3 — pacing and time discipline]*

One thing the real exam will not give you

Your score report arrives within 24 hours, by email and on the Portal. It reports the outcome. It does not tell you which domain you were weak in: the Linux Foundation states plainly that it "does not report performance on individual items and will not honor requests for more detailed information" [source: lf-exam-scoring-and-notification-2026-08-31].

That is the whole argument for the score sheet at the end of this chapter. The per-domain breakdown you are about to generate is not a convenience duplicating something the real exam hands you afterward. It is the only domain-level diagnostic you will get anywhere in this preparation, and you get it before the exam, which is the only point at which it can still change anything.

The answers

They are in the block after the exam, headed **Mock Exam Answers & Walkthroughs**. Do not read ahead. Every question there carries the correct answer, an explanation of why the wrong options are wrong, the domain it belongs to, and a pointer back to the section that taught it.

When you are ready, start the clock.

Exam

1. A container and a virtual machine both run an application in isolation from the host's other workloads. Which statement correctly distinguishes them?

A. A container runs its own kernel on virtualized hardware; a virtual machine shares the host's kernel with other workloads
B. A container's resource ceiling is set by a hypervisor; a virtual machine's is set by cgroups on the host
C. A container is a host process sharing the host's kernel; a virtual machine runs its own kernel on virtualized hardware
D. A container needs a hypervisor before it can start; a virtual machine needs a container runtime before it can start

2. A cluster has been bootstrapped, but no CNI plugin has been installed. What is the most likely observed symptom?

A. The API server refuses to start until a pod network has been configured
B. Pods are created but stay Pending, and pod networking does not function
C. Pods run normally but cannot reach any destination outside the cluster
D. Services are created, but the API server assigns none of them a ClusterIP

3. Which Kubernetes component is the only one that writes to cluster state?

A. kube-scheduler, which records each of its binding decisions directly
B. kubelet, which reports the status of its own node directly
C. kube-controller-manager, which writes the results of every reconciliation
D. kube-apiserver, which every other component reaches state through

4. In a Helm chart, what is the role of `values.yaml`?

A. It supplies the chart's default parameters, which templates consume and a user may override
B. It is the rendered manifest set that Helm submits to the cluster at install time
C. It declares the chart's name, version, and dependency list for the repository index
D. It holds the CustomResourceDefinitions the chart must install before its other objects

5. Which statement about a Kubernetes Namespace is correct?

A. It isolates the processes belonging to different namespaces from one another at the kernel level
B. It is a scope for names, and some resource kinds exist outside it entirely
C. It applies to every Kubernetes resource kind, without exception
D. Deleting one leaves the objects inside it in place, belonging to no namespace

6. In the four-layer cloud native security model, which layer is the outermost — the one whose compromise undermines every layer inside it?

- A. Code B. Container C. Cluster D. Cloud
-

7. Which is the correct characterization of Prometheus's data collection model?

- A. Applications push their metrics to Prometheus over HTTP as those metrics are produced B. Prometheus subscribes to a stream that an OpenTelemetry Collector forwards to it C. Prometheus scrapes metrics from targets it discovers, pulling on an interval D. Prometheus reads application metrics from the Kubernetes API server's aggregation layer
-

8.

```
apiVersion: v1
kind: Pod
metadata:
  name: report-worker
spec:
  containers:
    - name: worker
      image: example/worker:2.1
      resources:
        requests:
          cpu: "250m"
          memory: "256Mi"
        limits:
          cpu: "500m"
          memory: "512Mi"
```

Which QoS class does the Pod above receive?

- A. Burstable B. Guaranteed C. BestEffort D. Guaranteed for CPU and Burstable for memory
-

9. A Pod that mounts an `emptyDir` volume is deleted, and its controller recreates it on the same node. What happens to the data in the volume?

- A. It persists, because an `emptyDir` volume is bound to the node rather than to the Pod B. It is lost, because an `emptyDir` volume's lifetime is the Pod's C. It persists, provided the replacement Pod is given the same name as the original D. It is copied into the replacement Pod by the kubelet running on that node
-

10. Which statement about the relationship between a Deployment, a ReplicaSet, and a Pod is correct?

A. A Deployment creates Pods directly, and the ReplicaSet records what it created
B. A ReplicaSet manages Deployments, which in turn manage the individual Pods
C. A Deployment manages ReplicaSets, each of which manages a set of Pods
D. A Deployment and a ReplicaSet are one object exposed under two API versions

11. Which statement describes GitOps as distinct from a conventional CI/CD pipeline?

A. An agent inside the cluster pulls desired state from a repository and reconciles it continuously
B. The build system holds the cluster's credentials and applies manifests when a pipeline succeeds
C. Manifests are applied at merge time only, and are never reapplied to the cluster afterward
D. Declarative manifests are replaced by imperative deployment scripts that the agent executes

12.

```
apiVersion: v1
kind: Pod
metadata:
  name: check
spec:
  containers:
    - name: app
      image: example/app:1.0
```

The Pod above is applied and stays in Pending. Which of the following is a plausible cause?

A. The image tag does not exist in the registry that the node would pull it from
B. The container's main process started and then exited immediately on startup
C. The container was terminated by the kernel for exceeding its memory limit
D. No node passes the scheduler's filters, so the Pod has not been bound to one

13. An image is referenced as `registry.example.com/team/api:v2`. The same tag is later pushed again, pointing at different content. Which statement is true?

A. Tags are immutable under the OCI distribution spec, so the second push is rejected
B. The tag now resolves to the new content; a digest reference would still resolve to the old
C. Both manifests are retained under `:v2`, and the registry serves whichever copy is nearer
D. The original digest is updated to match the new content, so digest references follow it

14. An operator applies a NetworkPolicy that allows ingress to Pods labeled `app: db` from Pods labeled `app: api`, and then writes a second NetworkPolicy intended to deny ingress from `app: batch`. What is the effect?

- A. Both are honored, and traffic from `app: batch` is explicitly blocked at the target Pods B. The second policy replaces the first, because policies are evaluated in creation order C. NetworkPolicy has no deny rule; selection alone makes the target default-deny for anything not allowed D. The two policies conflict, so the API server rejects the second one at admission

15. A project has just been accepted into the CNCF at the entry level of its maturity ladder, before it has been asked to demonstrate broad production adoption. Which level is it at, and which level follows?

- A. Sandbox, and the next level is Incubating B. Incubating, and the next level is Sandbox C. Sandbox, and the next level is Graduated D. Incubating, and the next level is Graduated

16. Which of the following is the correct order of the three gates every request to the API server passes through?

- A. Admission, then authentication, then authorization B. Authentication, then admission, then authorization C. Authorization, then authentication, then admission D. Authentication, then authorization, then admission

17.

```
$ kubectl get pods -l app=web
```

NAME	READY	STATUS	RESTARTS	AGE
web-6d4f8b7c9d-2xk4p	1/1	Running	0	14m
web-6d4f8b7c9d-7hqnv	1/1	Running	0	14m
web-6d4f8b7c9d-lz8mt	0/1	Running	0	14m

A Service of type `ClusterIP` selects Pods with the label `app: web`. How many endpoints does the Service have?

- A. Three, because all three Pods are in the Running phase B. Zero, because one Pod that is not ready invalidates the whole EndpointSlice C. Two, because a Pod that is not ready is removed from the Service's endpoints D. Three, but kube-proxy silently drops the traffic it routes to the unready Pod

18. A `helm rollback` and a `kubectl rollout undo` both move a workload backward. Which statement correctly distinguishes them?

A. They are one operation, because Helm delegates the work to the Deployment's rollout machinery B. `helm rollback` restores a previous release revision; `kubectl rollout undo` reverts one Deployment to a previous ReplicaSet C. `kubectl rollout undo` works only on Helm-installed workloads, while `helm rollback` works on any D. `helm rollback` reads the Deployment's own ReplicaSet history, bounded by `revisionHistoryLimit`

19. A cluster administrator wants to take a node out of service for maintenance without losing the workloads running on it. Which sequence is correct?

A. `kubectl cordon`, then `kubectl drain` B. `kubectl drain`, then `kubectl cordon` C. `kubectl delete node`, then re-register the node afterward D. `kubectl taint node ... NoExecute`, then `kubectl uncordon`

20. A container reads its database password from a file mounted from a Secret. Which statement about that Secret is accurate?

A. The value is encrypted in etcd by default, so it is protected at rest without further work B. base64 encoding renders the value unreadable to anyone who can read etcd directly C. Secrets can only be injected as environment variables, and cannot be mounted as files D. The value is base64-encoded, which is encoding and not protection; encryption at rest is configured separately

21. Which statement about a StatefulSet is the reason it exists as a workload kind separate from a Deployment?

A. Only a StatefulSet is able to mount a PersistentVolumeClaim into the Pods it manages B. Only a StatefulSet supports rolling updates, rather than replacing every replica at once C. It gives each replica a stable ordinal identity and network name that survive rescheduling D. It places exactly one replica on each node, which is what stateful workloads require

22. A distributed trace shows one request passing through four services. What is the unit representing a single operation within that trace?

A. A metric, which records the operation's duration as a point in a time series B. A span, which records one operation's start, end, and position in the trace C. A log line, emitted by the service at the boundary of the operation D. Baggage, which carries the operation's identity from one service to the next

23. An init container in a Pod exits with a non-zero status. What happens?

A. The Pod's application containers start anyway, because init failures are advisory only B. The Pod is deleted and rescheduled onto a different node, where the init is retried C. The next init container in the

list runs, and the failure is recorded as an event D. No application container starts; the init container is retried per the Pod's restartPolicy

24.

```
$ kubectl get pod payment-7c9f4d8b6-nq2vs
```

NAME	READY	STATUS	RESTARTS	AGE
payment-7c9f4d8b6-nq2vs	0/1	CrashLoopBackOff	11 (22s ago)	38m

Which command retrieves the output of the container run that most recently failed?

- A. `kubectl logs payment-7c9f4d8b6-nq2vs --previous` B. `kubectl logs payment-7c9f4d8b6-nq2vs` C. `kubectl describe pod payment-7c9f4d8b6-nq2vs` D. `kubectl events --for pod/payment-7c9f4d8b6-nq2vs`
-

25. A team wants to deploy a new version to a small subset of production traffic before committing to a full rollout. Which strategy names this?

- A. Recreate, which stops every instance of the old version before starting the new one B. Blue/green, which stands up a complete parallel environment and then cuts over to it C. Canary, which sends a small share of production traffic to the new version first D. Rolling update, which replaces the running instances one batch at a time
-

26.

```
apiVersion: v1
kind: Node
metadata:
  name: node-07
spec:
  taints:
    - key: maintenance
      value: "true"
      effect: NoExecute
```

A Pod is already running on node-07 and declares no tolerations. What is the consequence of the taint above?

- A. Nothing, because the effect governs only scheduling decisions made after it is applied B. The Pod is evicted from node-07 C. The Pod keeps running, and the node is marked NotReady until the taint is tolerated D. The Pod is restarted in place, with a matching toleration added to it automatically
-

27. A PersistentVolumeClaim requests ReadWriteOnce. What does this constrain?

- A. The volume may be mounted read-write by Pods on a single node
 - B. Exactly one Pod anywhere in the cluster may mount the volume at any time
 - C. The volume may be read by many Pods, but written only by the first to mount it
 - D. Only one container inside a given Pod may mount the volume read-write
-

28. In a Kubernetes object, what is the relationship between `spec` and `status`?

- A. `spec` is written by the controller, and `status` is written by the user who submitted the object
 - B. Both are written by the user, and the API server validates that they agree before persisting
 - C. `status` is a copy of `spec` retained for rollback, and diverges from it only during an update
 - D. `spec` is the declared desired state; `status` is the observed state a controller reports
-

29. In a service mesh, what is the distinction between the data plane and the control plane?

- A. The data plane distributes policy, and the control plane carries the application traffic
 - B. The data plane is the proxies carrying application traffic; the control plane configures them
 - C. They name the same components, described once at cluster scope and once at Pod scope
 - D. The data plane is the Kubernetes API server, and the control plane is the Envoy proxy
-

30. A container image is built FROM a base image and adds two layers. A second image is built from the same base and adds one different layer. What does the registry store?

- A. Two complete and independent copies, one for each of the two images
 - B. Only the most recently pushed image; the older one is garbage collected
 - C. The shared base layers once, plus each image's distinct layers
 - D. A single merged image containing every layer from both of them
-

31. What does an Ingress object accomplish on a cluster where no Ingress controller is installed?

- A. Nothing; the object is accepted and stored, and no component acts on it
 - B. Traffic is routed by kube-proxy, which falls back to the Ingress rules
 - C. The API server rejects the object as invalid when it is submitted
 - D. A cloud load balancer is provisioned automatically for the Ingress host
-

32. A developer needs a shell inside a running container whose image contains no shell — a distroless build. Which approach fits?

- A. `kubectl exec -it <pod> -- /bin/sh`, which starts a shell in the target container
- B. `kubectl port-forward` to the container's port, then connect to it from the workstation
- C. Rebuild the image with a shell

included and redeploy, which is the only available option D. `kubectl debug`, which adds an ephemeral container running a different image alongside it

33. Which of the following best describes an operator in the Kubernetes sense?

A. A human administrator holding cluster-admin permissions across the whole cluster B. A plugin that extends the `kubectl` client with additional subcommands and output C. A controller for a custom resource, encoding operational knowledge in a reconciliation loop D. A webhook that mutates objects at admission time on an application's behalf

34. What distinguishes a headless Service from a standard ClusterIP Service?

A. It sets `clusterIP: None`, so DNS returns the Pod addresses instead of one virtual IP B. It carries no selector, so the addresses behind it must be maintained by hand C. It cannot be paired with a `StatefulSet`, which requires one Service per replica D. It is reachable only from outside the cluster, through each node's external address

35. A cluster has 3 control plane nodes and 20 worker nodes. Where does the authoritative record of cluster state live?

A. Replicated across all 23 nodes by the kubelet running on each of them B. In each controller's in-memory cache, reconciled between the controllers C. On whichever node the objects' Pods have been scheduled onto D. In etcd, which only the API server reads from and writes to

36.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: payments
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list"]
```

A `ServiceAccount` in `payments` is bound to the Role above by a `RoleBinding` in the same namespace. What may it do?

A. Get and list Pods in every namespace, since the rules name no namespace of their own B. Get and list Pods in `payments`, and nothing else C. Get, list, and delete Pods in `payments`, since read verbs imply the

write verbs D. Everything except delete Pods, since RBAC refuses only what a rule explicitly forbids

37. What does an Argo CD Application in state OutOfSync indicate?

A. The live cluster state differs from the desired state in the tracked repository revision B. The Argo CD controller cannot reach the cluster's API server to compare the two states C. The repository the Application tracks is unreachable, or its revision cannot be resolved D. The application's Pods are running but are failing their configured readiness probes

38. A Pod has a liveness probe and a readiness probe, both HTTP GETs against different paths. The readiness probe begins failing while the liveness probe continues to succeed. What happens?

A. The container is restarted by the kubelet on the node where it is running B. The Pod is evicted from the node and rescheduled somewhere with more headroom C. The Pod is removed from its Services' endpoints, and the container keeps running D. Nothing happens until both probes have failed their configured thresholds

39. Which describes the relationship between the four golden signals and the RED method?

A. They are the same four measures, published under two different names B. RED superseded the golden signals when OpenTelemetry standardized its signals C. RED is the client-side view of a service, and the golden signals the server-side view D. Complementary framings: latency, traffic, errors, and saturation against rate, errors, and duration

40. A cluster operator sets a ResourceQuota on a namespace limiting total memory requests to 10Gi, and a LimitRange in the same namespace setting a default container memory request of 256Mi. What does each accomplish?

A. They are redundant; either one alone would bound the namespace's memory consumption B. The quota caps the namespace's aggregate; the LimitRange supplies per-object defaults and bounds C. The LimitRange caps the namespace's aggregate; the quota applies to each container in turn D. The LimitRange applies only to Pods that already declare requests of their own

41. A Pod's container is terminated and the container state's Reason reads OOMKilled. What does this indicate?

A. The node came under memory pressure, and the kubelet evicted the Pod from it B. The container's liveness probe failed, and the kubelet restarted the container C. The scheduler could not find a node

with enough allocatable memory for the Pod D. The container exceeded its own memory limit and was terminated by the kernel

42. Which statement about `imagePullPolicy` is accurate?

A. `IfNotPresent` skips the pull when the image is already on the node; the effective default depends on the tag, with `:latest` defaulting to `Always` B. The default is `Always` for every image, whatever tag the reference carries or omits C. `Never` causes the kubelet to build the image on the node from a local build context D. `Always` re-checks the registry on every start, and requires the image to be given by digest

43. A cluster's storage is provisioned on demand when a `PersistentVolumeClaim` is created, without an administrator pre-creating volumes. What makes this possible?

A. The claim's `accessModes` field, which selects between the provisioning modes B. The `hostPath` volume type, which creates the directory on the node at first use C. A `StorageClass` naming a provisioner, which creates the `PersistentVolume` dynamically D. A reclaim policy of `Retain`, which preserves the volume so it can be reused

44. An organization wants configuration that varies between staging and production without maintaining two copies of the manifests, and prefers not to use a templating language. Which tool matches?

A. Helm, using a separate values file for each of the two environments B. Kustomize, using a shared base and one overlay per environment C. Argo CD sync waves, which order the objects applied to each environment D. A `ConfigMap` generator in Helm, which emits per-environment `ConfigMaps`

45. The scheduler is placing a newly created Pod. Which sequence describes what it does?

A. It scores every node, discards the ones scoring below a threshold, and binds to one of the rest B. It binds the Pod to a node, then filters and scores to confirm that the choice was feasible C. It asks each kubelet whether it can accept the Pod, and binds to the first one that answers D. It filters nodes down to the feasible ones, scores those, and binds the Pod to the highest scorer

46. The Gateway API splits responsibility for north-south traffic across three resource kinds, each aimed at a different role. Which mapping is correct?

A. `GatewayClass` to the infrastructure provider, `Gateway` to the cluster operator, `HTTPRoute` to the application developer B. `GatewayClass` to the application developer, `Gateway` to the infrastructure provider, `HTTPRoute` to the cluster operator C. All three are cluster-scoped, and all three are the cluster

operator's alone to manage D. HTTPRoute to the infrastructure provider, with Gateway and GatewayClass to the application developer

47. A microservices architecture is chosen over a monolith. Which trade-off is the honest characterization?

A. Decomposition reduces the system's total complexity, because each part is smaller than the whole B. Service boundaries remove coupling between components, because each service owns its own data C. Components can be deployed and scaled independently, at the cost of operational and network complexity D. Independent deployment follows automatically once each service has its own repository and pipeline

48. A CronJob is defined with `schedule: "0 2 * * *"`. What does it create when it fires?

A. A Pod directly, labeled with the CronJob's name and the timestamp of the run B. A Job, which in turn creates one or more Pods to run the work to completion C. A Deployment scaled to a single replica, which is deleted when the work finishes D. A DaemonSet on whichever node has the most allocatable capacity at that moment

49. An auditor needs to know which software components are present in a shipped image, so that a newly disclosed vulnerability can be checked against them. Which artifact answers that question directly?

A. The image's SBOM, an inventory of the components the image contains B. A vulnerability scan report from the last time that image was scanned C. A signed provenance attestation, recording how and where the image was built D. The image's digest, which uniquely identifies the content that was shipped

50. `kubectl port-forward` to a Pod works and the application answers, but requests sent through the Pod's Service time out. What does that pair of results establish?

A. The application is unhealthy, and the Service is correctly declining to send it traffic B. The cluster's CNI plugin is failing to pass traffic between nodes C. The Service's ClusterIP has been reused by another Service in the same namespace D. The application is reachable; the fault is on the Service path — selector, ports, or endpoints

51. Which statement about labels and selectors is correct?

A. They are interchangeable with annotations, and which to use is a matter of house style B. Labels are for identification and selection; annotations carry metadata and are not selectable C. Selectors can

match on annotations when a label value would exceed the 63-character limit D. Labels must be unique within a namespace, whereas annotations may repeat freely

52. A Pod in the namespace `orders` resolves the name `billing`. Which Service does that name reach, and why?

A. The `billing` Service in default, because unqualified names resolve in the default namespace B. Nothing, because a Service name must be given fully qualified before cluster DNS resolves it C. The `billing` Service in `orders`, because the Pod's search domains try its own namespace first D. Whichever `billing` Service was created first, because cluster DNS resolves names cluster-wide

53. A contributor wants to propose a substantial change to a Kubernetes subsystem. Through which structure and document does that proposal proceed?

A. A pull request against the release branch, reviewed by the Steering Committee B. An issue on the CNCF Landscape repository, triaged by the Technical Oversight Committee C. A CNCF Technical Advisory Group charter, ratified by the Governing Board D. A Kubernetes Enhancement Proposal, brought to the SIG that owns that subsystem

54. What does the Container Runtime Interface standardize?

A. The boundary between the kubelet and the container runtime, so runtimes are interchangeable B. The on-disk format of a container image and the ordering of the layers inside it C. The protocol by which registries distribute images to the nodes that pull them D. The mechanism by which a container is given a network address on its node

55. Two containers run in one Pod. Which statement about what they share is correct?

A. They share a filesystem root, so a file written by one is visible to the other B. They share a process namespace by default, so each can see the other's processes C. They share a network namespace, so each reaches the other on `localhost` D. They share one memory limit, so either may consume whatever the other leaves unused

56. The twelve-factor app says configuration belongs in the environment rather than in the build. Which Kubernetes practice implements that?

A. Baking each environment's settings into a separate image, one tag per environment B. Supplying settings from ConfigMaps and Secrets, so one image runs in every environment C. Mounting a `hostPath`

volume holding the node's local copy of the settings file D. Rebuilding and redeploying the application whenever a single setting has to change

57.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
spec:
  replicas: 4
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
```

What is the maximum number of Pods this Deployment may have in existence during a rolling update?

- A. Four, because `maxUnavailable: 0` forbids the running count from changing at all B. Eight, because the new ReplicaSet is created at full size beside the old one C. Three, because one Pod is taken down before each replacement is created D. Five, because `maxSurge: 1` permits one Pod above the desired count
-

58. A workload should run on nodes labeled `disk=ssd` where possible, but should still be scheduled when no such node is available. Which mechanism expresses that?

- A. Node affinity with a `preferredDuringSchedulingIgnoredDuringExecution` rule B. A `nodeSelector` naming the label, which the scheduler treats as a preference C. A taint on every node lacking the label, with a matching toleration on the Pod D. Node affinity with a `required` rule, which the scheduler relaxes when nothing matches
-

59.

```
$ kubectl get svc,endpointslice -l app=orders
NAME                TYPE        CLUSTER-IP    PORT(S)    AGE
service/orders      ClusterIP   10.96.140.22  80/TCP     6m

NAME                ADDRESSTYPE  ENDPOINTS    AGE
endpointslice/orders-x8k2p  IPv4        <none>       6m

$ kubectl get pods -l app=order-api
NAME                READY  STATUS   RESTARTS   AGE
order-api-5d7c9b4f-jm2wq  1/1    Running  0          6m
order-api-5d7c9b4f-t4nlv  1/1    Running  0          6m
```

Both Pods are ready, and the Service has no endpoints. What is the most likely cause?

- A. The Pods' readiness probes pass, but their startup probes have not yet completed
 - B. The Service's selector does not match the labels the Pods actually carry
 - C. The Service's targetPort names a port the containers do not expose
 - D. No Ingress controller is installed, so the Service has nothing to route traffic for
-

60. Which of the following accurately describes how a Kubernetes controller behaves?

- A. It runs a sequence of steps once, at the moment the object it manages is created
 - B. It is invoked by the scheduler once a Pod has been bound to a particular node
 - C. It loops continuously, comparing desired state against observed and closing the difference
 - D. It applies its changes only when an administrator runs `kubectl apply` against the cluster
-

Stop. Note your time. The answers begin below.

Mock Exam Answers & Walkthroughs

Work through these in order, and record two things per question: whether you got it right, and which domain it belonged to. The domain tag is on every answer for exactly that purpose.

1. C — D1.4 · Ch 2 §1 · ...

A container is a host process placed in kernel namespaces and cgroups, sharing the host's kernel. A VM runs its own kernel on virtualized hardware. That single difference explains the startup-time gap and the isolation-strength gap at once. *[cross-bearing: see Ch 2 §1 — what a container actually is]*

- **A is wrong** because it inverts the two. The container shares the kernel; the VM has its own.
 - **B is wrong** because a container's ceiling comes from cgroups on the host, and a VM's from the hypervisor's allocation. It swaps the mechanisms.
 - **D is wrong** on both halves. A hypervisor belongs to the VM; a container runtime belongs to the container.
-

2. B — D2.1 · Ch 9 §1 · ...

Without a CNI plugin the cluster has no implementation of the pod network. Pods do not get addresses and do not become ready. *[cross-bearing: see Ch 9 §1 — four rules and a plugin]*

- **A is wrong** because the control plane comes up independently of the pod network. That is why you can inspect a broken cluster at all.
 - **C is wrong** because the failure is not egress-specific; pod-to-pod networking does not exist either.
 - **D is wrong** because ClusterIP assignment is done by the API server from a configured range, and does not depend on CNI.
-

3. D — D1.1 · Ch 3 §5 · ..

The API server is the sole mutator of cluster state. Everything else — scheduler, controllers, kubelets — reads and writes through it. *[cross-bearing: see Ch 3 §5 — the only door in]*

- **A is wrong** because the scheduler decides a binding and then *submits* it to the API server; it does not write state itself.
 - **B is wrong** for the same reason: the kubelet reports its node's status through the API server.
 - **C is wrong** because controllers act by making API requests. It is the pattern's whole point that they have no back channel.
-

4. A — D3.1 · Ch 14 §2 · ..

`values.yaml` holds the chart's default parameters, which templates consume and users override. *[cross-bearing: see Ch 14 §2 — what a chart contains]*

- **B is wrong** because the rendered manifest is the output of templating, not an input file in the chart.
 - **C is wrong** because name, version, and dependencies live in `Chart.yaml`.
 - **D is wrong** because CustomResourceDefinitions live in the `crds/` directory, which exists to solve an ordering problem. *[cross-bearing: see Ch 14 §6 — which one, when]*
-

5. B — D1.1 · Ch 4 §3 · ..

A Namespace is a scope for names, and several important kinds live outside it: Nodes, PersistentVolumes, StorageClasses, ClusterRoles. *[cross-bearing: see Ch 4 §3 — where a name lives]*

- **A is wrong**, and it is the trap this question is built for. Kernel isolation is the *Linux* namespace from Chapter 2, a different mechanism that shares an English word.
 - **C is wrong** because cluster-scoped kinds exist, and knowing which is which is directly testable.
 - **D is wrong** because namespace deletion cascades: the objects inside are deleted with it. There is no orphaned state left behind, which is exactly why deleting a namespace is not the tidying-up it looks like.
-

6. D — D2.2 · Ch 12 §1 · ..

Cloud, Cluster, Container, Code, from outside in. The cloud (or datacenter) layer is the foundation; a compromise there makes every layer above it moot. *[cross-bearing: see Ch 12 §1 — four layers and four phases]*

- **A is wrong** because Code is the innermost layer, the one every other layer sits outside of.
 - **B is wrong** because Container is the second-innermost, one step out from Code.
 - **C is wrong** because Cluster is the third, sitting inside Cloud rather than outside it.
-

7. C — D4.1 · Ch 18 §4 · ..

Prometheus pulls. It discovers targets and scrapes their metrics endpoints on an interval. *[cross-bearing: see Ch 18 §4 — pulling, not being pushed]*

- **A is wrong** as the general model. Pushgateway exists for the narrow case of jobs too short-lived to be scraped, and using it generally is an anti-pattern.
 - **B is wrong** because an OTEL Collector can *export* to Prometheus, but the transfer is still a scrape of the collector; Prometheus does not subscribe to a stream.
 - **D is wrong** because the API server is one scrape target among many, not the source of application metrics.
-

8. A — D1.1 · Ch 5 §8 · ..

Requests and limits are both set, and they are not equal, which is Burstable. Guaranteed requires request equal to limit for every resource in every container. *[cross-bearing: see Ch 5 §8 — what a Pod is owed]*

- **B is wrong** because $250m \neq 500m$ and $256Mi \neq 512Mi$. This is the single most common QoS error, and it is arithmetic rather than recall.
 - **C is wrong** because BestEffort means no requests and no limits at all, which this manifest plainly has.
 - **D is wrong** because QoS is a property of the Pod, computed once across every resource in every container. There is no per-resource QoS class.
-

9. B — D2.4 · Ch 11 §1 · ..

An `emptyDir` lives and dies with the Pod. Same node makes no difference; a new Pod gets a new empty directory. *[cross-bearing: see Ch 11 §1 — three lifetimes, and the volumes that have them]*

- **A is wrong** because node-scoped persistence is what `hostPath` does, and it is a hazard for exactly the reasons that chapter gives.

- **C is wrong** because Pod name has no bearing on volume lifetime. Per-replica persistence requires a PVC.
 - **D is wrong** because nothing copies or migrates an `emptyDir`; the kubelet deletes it when the Pod is removed.
-

10. C — D1.1 · Ch 6 §1 · ..

Deployment owns ReplicaSets; each ReplicaSet owns Pods. A rolling update works by scaling a new ReplicaSet up while scaling the old one down, which is why old ReplicaSets stick around and why rollback is possible at all. *[cross-bearing: see Ch 6 §5 — every rollout is a revision]*

- **A is wrong** because it removes the layer that makes revision history work.
 - **B is wrong** because it inverts the ownership chain; a ReplicaSet has no knowledge of Deployments.
 - **D is wrong** because they are distinct kinds with distinct jobs, both in `apps/v1`.
-

11. A — D3.1 · Ch 15 §3 · ..

Pull, not push. An in-cluster agent reconciles continuously against a repository that holds desired state. *[cross-bearing: see Ch 15 §3 — push, or pull]*

- **B is wrong** because it describes the push model precisely — and it inverts the security argument. Holding cluster credentials in the build system is the push model's cost, and avoiding it is a principal reason to adopt pull.
 - **C is wrong** because continuous reconciliation is one of the OpenGitOps principles. The agent corrects drift that no merge caused.
 - **D is wrong** because GitOps depends on declarative manifests; it does not replace them with scripts.
-

12. D — D2.3 · Ch 13 §2 · ..

Pending is the phase of a Pod that has not been scheduled. The scheduler found no feasible node, or the Pod has not yet been considered. *[cross-bearing: see Ch 13 §2 — Pods that never start]*

- **A is wrong** because a missing tag surfaces as `ImagePullBackOff`, which happens *after* scheduling. The image is pulled on a node, so a Pod cannot fail to pull an image it was never assigned a node for.
 - **B is wrong** because a container that starts and exits gives you `CrashLoopBackOff`, not Pending.
 - **C is wrong** because an OOM kill requires a running container. Reading the phase first is what separates these four instantly. *[cross-bearing: see Ch 13 §1 — whose problem is this]*
-

13. B — D1.4 · Ch 2 §3 · ...

A tag is a mutable pointer into a repository. A digest is content-addressed and identifies exactly one image. *[cross-bearing: see Ch 2 §3 — registries, tags, and digests]*

- **A is wrong** because tag immutability is a registry policy some registries offer, not a property of the reference grammar.
 - **C is wrong** because a tag resolves to one manifest; there is no "nearer" copy behind the same tag.
 - **D is wrong** because it misunderstands what a digest is. The digest is the content's identity: new content has a new digest, and the old digest still resolves to the old content.
-

14. C — D2.1 · Ch 10 §6 · ...

NetworkPolicy is allow-only. There is no deny rule to write. What produces isolation is *selection*: once any policy selects a Pod, that Pod becomes default-deny for the direction the policy covers, and only the allowed sources get through. *[cross-bearing: see Ch 10 §6 — allowing, never denying]*

- **A is wrong** because the second policy cannot be written in the first place, so there is nothing to honor.
 - **B is wrong** because policies are additive and unordered; a later one never overrides an earlier one.
 - **D is wrong** because there is no conflict to detect, and nothing for admission to reject. This is the same additive semantics RBAC uses, which is not a coincidence. *[cross-bearing: see Ch 12 §9 — additive, never deny]*
-

15. A — D4.3 · Ch 17 §2 · ...

Sandbox, Incubating, Graduated, in that order. Sandbox is the entry point; Incubating and then Graduated each raise the bar on adoption, governance, and sustainability. *[cross-bearing: see Ch 17 §2 — Sandbox, Incubating, Graduated, and who decides]*

- **B is wrong** because it reverses the first two levels. Incubating is the middle rung, not the entry point.
- **C is wrong** because it skips Incubating. A project does not go from Sandbox straight to Graduated.
- **D is wrong** on both halves: it names the wrong entry level and the wrong successor.

Worth carrying into the real exam: the *levels and their order* are the durable fact. Which projects sit at which level changes constantly, and a memorized Graduated roster is memorization with an expiry date on it.

16. D — D1.2 · Ch 8 §2 · ...

Authentication (who are you), then authorization (may you), then admission (should this specific object be allowed, and does it need modifying). *[cross-bearing: see Ch 8 §2 — three gates and a logbook]*

- **A is wrong** because admission cannot run first: it inspects an object whose submitter has not been identified and whose permission has not been checked. There is nothing yet to admit.
 - **B is wrong** because admission cannot precede authorization. A mutating webhook would otherwise modify an object the requester may turn out to have no permission to create.
 - **C is wrong** because authorization cannot precede authentication. Policy is evaluated against an identity, and there is no identity yet.
-

17. C — D2.1 · Ch 9 §4 · ...

Readiness gates endpoint membership. `web-...-1z8mt` shows 0/1 ready, so it keeps running but is taken out of the Service's endpoint set — precisely the behavior the probe exists to provide. Two endpoints. *[cross-bearing: see Ch 9 §4 — the list behind the name]*

- **A is wrong** because it reads the STATUS column and ignores READY. Running is the phase; readiness is a separate condition, and it is the one endpoints follow.
 - **B is wrong** because endpoint membership is per-Pod, not all-or-nothing. The other two Pods remain addressable.
 - **D is wrong** because kube-proxy does not drop traffic to unready Pods. The Pod is not in the endpoint list for it to route to in the first place.
-

18. B — D3.1 · Ch 14 §3 · ...

Two mechanisms wearing the same word. `helm rollback` moves the *release* to a previous *release revision*, potentially every object the chart manages. `kubectl rollout undo` moves one Deployment back to a previous ReplicaSet. *[cross-bearing: see Ch 14 §3 — chart, release, revision]*

- **A is wrong** because Helm maintains its own release history and applies the previous manifest set; it does not call the Deployment's rollout machinery.
 - **C is wrong** because `kubectl rollout undo` works on any Deployment regardless of how it was installed, which is part of why the confusion is easy to have.
 - **D is wrong** because `revisionHistoryLimit` bounds a Deployment's ReplicaSet history, which is not where Helm keeps release revisions.
-

19. A — D1.2 · Ch 8 §4 · ...

Cordon marks the node unschedulable so nothing new lands there; drain then evicts what is already running so it can be rescheduled elsewhere. *[cross-bearing: see Ch 8 §4 — taking a node out of service]*

- **B is wrong** in ordering. Draining first without cordoning invites the scheduler to place new Pods on the node you are emptying. (In practice `kubectl drain` cordons for you. The exam tests the conceptual order, and knowing *why* cordon precedes drain is the point.)
 - **C is wrong**, and destructive: deleting the Node object does not gracefully move the workloads off it first.
 - **D is wrong** because `uncordon` returns a node to service, which is the opposite of taking it out.
-

20. D — D2.2 · Ch 12 §4 · ...

base64 is an encoding. Anyone who can read the Secret can decode it in one command. Encryption at rest is a separate cluster-level configuration. *[cross-bearing: see Ch 12 §4 — Secrets are not encrypted]*

- **A is wrong** because encryption at rest is not on by default; it requires an `EncryptionConfiguration` on the API server.
 - **B is wrong** because that is the misconception this whole item targets. Decoding base64 requires no key and no privilege beyond reading the value.
 - **C is wrong** because file mounts are not only possible but generally preferred over environment variables, which leak into process listings and crash dumps.
-

21. C — D1.1 · Ch 6 §6 · ...

Stable identity is the distinction. Each replica gets an ordinal, a stable name, and a stable network identity that survive rescheduling. *[cross-bearing: see Ch 6 §6 — when Pods are not interchangeable]*

- **A is wrong**, and it is the most common belief about `StatefulSets`. Any Pod can mount a PVC. What a `StatefulSet` adds is a `volumeClaimTemplate` giving each replica *its own* claim. *[cross-bearing: see Ch 11 §6 — Pods that are not interchangeable, revisited]*
 - **B is wrong** because `Deployments` support rolling updates too; `StatefulSets` add ordering constraints to theirs, but they did not invent the capability.
 - **D is wrong** because one replica per node is a `DaemonSet`, a different kind with a different purpose.
-

22. B — D4.1 · Ch 18 §5 · ...

A span represents one operation within a trace, carrying its own start, end, and place in the tree. *[cross-bearing: see Ch 18 §5 — following one request]*

- **A is wrong** because a metric is a different signal entirely: aggregate numbers over time, with no identity tying them to one request.
- **C is wrong** because logs are a third signal. A log line may reference a trace, but it is not the trace's unit of structure.
- **D is wrong** because baggage is contextual data propagated alongside a trace, not a unit within it.

23. D — D1.1 · Ch 5 §3 · ...

Init containers run to completion, in order, before any application container starts. A failure stops the sequence and is retried per the Pod's `restartPolicy`. *[cross-bearing: see Ch 5 §3 — everything that must happen first]*

- **A is wrong** because the ordering guarantee is the entire feature. If failures were advisory, an init container could not be used to wait for a dependency.
 - **B is wrong** because a failing init container does not trigger rescheduling; the Pod stays where it is and retries in place.
 - **C is wrong** because init containers do not proceed past a failure. The sequence halts on the one that failed.
-

24. A — D2.3 · Ch 13 §3 · ...

`--previous` retrieves the logs of the previous terminated container instance. With eleven restarts and the most recent 22 seconds ago, that is the run that actually failed. *[cross-bearing: see Ch 13 §3 — looking inside]*

- **B is wrong** because in a crash loop the current instance is either seconds old or not running at all. You get an empty stream or the first lines of a fresh start.
 - **C is wrong** because `describe` shows events, conditions, and state transitions — useful context, but not the container's own output.
 - **D is wrong** for the same reason. `kubectl events --for pod/<pod>` filters events to a resource *[source: k8s-docs-kubectl-events-2026-08-31]*, which tells you what the cluster did, not what the process printed.
-

25. C — D3.1 · Ch 15 §2 · ...

Canary: a small share of production traffic to the new version first, with the rest still on the old one. *[cross-bearing: see Ch 15 §2 — ways to replace what's running]*

- **A is wrong** because `Recreate` takes everything down and brings everything up, which is the opposite of a graduated exposure.
 - **B is wrong** because `blue/green` stands up a complete parallel environment and cuts over to it wholesale, rather than splitting traffic by proportion.
 - **D is wrong** because a rolling update replaces instances incrementally without steering any particular share of traffic to the new version.
-

26. B — D1.3 · Ch 7 §4 · ...

NoExecute evicts running Pods that do not tolerate the taint, as well as preventing new ones from landing. The Pod on node-07 declares no tolerations, so it goes. *[cross-bearing: see Ch 7 §4 — when the berth refuses you]*

- **A is wrong** because that describes NoSchedule, which affects only future placement. The distinction between the three effects is exactly what this manifest is testing.
 - **C is wrong** because node conditions and taints are different mechanisms. A condition can cause a taint; a taint does not set a condition.
 - **D is wrong** because tolerations are declared by the Pod's author. Nothing adds one on the Pod's behalf.
-

27. A — D2.4 · Ch 11 §4 · ...

Access modes are node-count semantics. ReadWriteOnce means read-write by a single *node*; multiple Pods on that node can share the volume. *[cross-bearing: see Ch 11 §4 — access modes and what happens after]*

- **B is wrong** because it reads "Once" as "one Pod." The mode that means one Pod is ReadWriteOncePod, which exists precisely because RWO does not mean that.
 - **C is wrong** because it treats the mode as permission semantics — who may write — rather than as a constraint on how many nodes may mount it.
 - **D is wrong** for the same category error at a smaller scope. Containers within one Pod are already on one node, so the mode has nothing to say about them.
-

28. D — D1.1 · Ch 4 §2 · ...

spec is the declaration a user files; status is what a controller observes and reports back. Every control loop in the cluster is the gap between those two fields being closed. *[cross-bearing: see Ch 4 §2 — the anatomy of a record]*

- **A is wrong** because it swaps the writers. A user who edits status is writing into a field the controller owns and will overwrite.
 - **B is wrong** because the API server does not require them to agree — a newly created object has a spec and an empty or absent status, and that disagreement is the work queue.
 - **C is wrong** because rollback history lives in revisions on the workload object, not in status. *[cross-bearing: see Ch 6 §5 — every rollout is a revision]*
-

29. B — D4.2 · Ch 17 §5 · ...

The data plane is the proxies carrying traffic; the control plane configures them and distributes policy. *[cross-bearing: see Ch 17 §5 — a network that knows what it's carrying]*

- **A is wrong** because it inverts them. The plane that carries the packets is the data plane, by definition.
 - **C is wrong** because they are genuinely different components with different jobs, not one thing viewed at two scopes.
 - **D is wrong** on both halves. Note also that "control plane" here is the *mesh's*, not the cluster's — two different things wearing one phrase. *[cross-bearing: see Ch 3 §2 — the control plane]*
-

30. C — D1.4 · Ch 2 §2 · ..

Layers are content-addressed and shared. Two images from the same base share the base's layers on disk and in the registry. *[cross-bearing: see Ch 2 §2 — what's inside an image]*

- **A is wrong**, and it misses the reason layering exists at all. Independent copies would make a registry of similar images ruinously large.
 - **B is wrong** because images are independent objects; pushing one does not evict another that happens to share its base.
 - **D is wrong** because layers stack within one image. There is no merge across images.
-

31. A — D2.1 · Ch 10 §3 · ..

The object exists; nothing happens without the component. Kubernetes accepts and stores the Ingress, and no traffic moves until an Ingress controller is watching for it. *[cross-bearing: see Ch 10 §3 — the object is not the implementation]*

- **B is wrong** because kube-proxy implements Services, not Ingress rules, and has no L7 routing to fall back to.
 - **C is wrong** because the object is valid, which is what makes this failure quiet and confusing. Nothing warns you.
 - **D is wrong** because that is a LoadBalancer Service's behavior — and it too requires a component, the cloud-controller-manager, to act.
-

32. D — D3.2 · Ch 16 §3 · ..

An ephemeral container added by `kubectl debug` runs a separate image inside the target Pod, so you get tools the target image does not have. *[cross-bearing: see Ch 16 §3 — getting inside, and adding what isn't there]*

- **A is wrong** because `exec` runs a binary that must already exist in the container. Distless means it does not.
- **B is wrong** because port-forward moves traffic to a port. It gives you the application's interface, not a shell.

- **C is wrong** because ephemeral containers exist precisely so that rebuilding is not the only option — and rebuilding changes the thing you were trying to observe.
-

33. C — D1.1 · Ch 6 §8 · ...

An operator is a controller for a custom resource, encoding what a human operator would know about running a particular application. *[cross-bearing: see Ch 6 §8 — the control loop, extended]*

- **A is wrong**, and it is a genuine vocabulary hazard: "operator" is never used for a person in this book. Where the Gateway API says "cluster operator" it means a *role name*, not an individual. *[cross-bearing: see Ch 10 §5 — roles, not just routes]*
 - **B is wrong** because a kubect1 plugin extends the client on your workstation, not the cluster's reconciliation behavior.
 - **D is wrong** because an admission webhook acts once per request, at admission time. An operator runs a loop continuously, long after the object was persisted.
-

34. A — D2.1 · Ch 9 §5 · ...

clusterIP: None means no virtual IP. DNS returns the Pod addresses directly, which is what a client needs when it must address individual replicas. *[cross-bearing: see Ch 9 §5 — when you don't want a single address]*

- **B is wrong** because a headless Service usually *does* have a selector. A Service without selectors is a separate case with a separate purpose.
 - **C is wrong** because headless Services are the standard pairing for StatefulSets, supplying exactly the per-replica DNS names those workloads need. *[cross-bearing: see Ch 6 §6 — when Pods are not interchangeable]*
 - **D is wrong** because headless Services are an internal DNS mechanism. Nothing about them exposes anything externally.
-

35. D — D1.1 · Ch 3 §2 · ..

etcd holds cluster state, and only the API server talks to it. That is why etcd is the one thing whose backup matters. *[cross-bearing: see Ch 8 §7 — the one backup that matters]*

- **A is wrong** because kubelets hold no authoritative state; they report and obey.
 - **B is wrong** because controller caches are derived from watches and are disposable. Losing one costs a resync, not data.
 - **C is wrong** because object state is not distributed to the nodes running the workloads. A node knows about its own Pods, not about the cluster.
-

36. B — D2.2 · Ch 12 §3 · ..

A Role is namespaced, and a RoleBinding grants it within that namespace only. The rule lists two verbs on one resource, so that is exactly the access granted: get and list Pods in payments. *[cross-bearing: see Ch 12 §3 — what you may do]*

- **A is wrong** because a Role's scope comes from its own `metadata.namespace`, not from what its rules mention. Cluster-wide access would require a ClusterRole bound with a ClusterRoleBinding.
 - **C is wrong** because RBAC verbs are enumerated, never implied. `delete` is absent, so `delete` is refused.
 - **D is wrong** because it inverts the model. RBAC is purely additive: there are no deny rules, and anything not granted is refused. *[cross-bearing: see Ch 12 §9 — additive, never deny]*
-

37. A — D3.1 · Ch 15 §4 · ..

`OutOfSync` means live state differs from the desired state in the tracked revision. *[cross-bearing: see Ch 15 §4 — an agent that watches a repository]*

- **B is wrong** because a controller that cannot reach the cluster reports a connectivity condition. It cannot make a sync verdict at all without seeing live state.
 - **C is wrong** because an unreachable repository is likewise its own error state. With no desired state to compare against, there is nothing to call out of sync.
 - **D is wrong** because Pod health is application state. `OutOfSync` is strictly about the declared-versus-live comparison, and a perfectly healthy application can be out of sync.
-

38. C — D1.1 · Ch 5 §7 · ..

Readiness controls endpoint membership. The container keeps running; traffic stops arriving. *[cross-bearing: see Ch 5 §7 — three probes, three jobs]*

- **A is wrong** because restart is the *liveness* probe's consequence. Three probes, three distinct jobs, and the exam will test whether you can keep them apart.
 - **B is wrong** because eviction is a node-pressure mechanism, unrelated to probe results.
 - **D is wrong** because each probe acts independently on its own failure. There is no combined threshold.
-

39. D — D4.1 · Ch 18 §7 · ..

Complementary framings. Golden signals: latency, traffic, errors, saturation. RED: rate, errors, duration. *[cross-bearing: see Ch 18 §7 — is the service doing what users expect]*

- **A is wrong** because RED has three measures and omits saturation — the one that tells you about headroom rather than symptoms.
 - **B is wrong** because OpenTelemetry standardizes signal *transport and semantics*, not which measures you should care about. Neither framing was retired.
 - **C is wrong** because both framings are written from the service's own perspective. RED's rate, errors, and duration are measured at the service, not at its callers.
-

40. B — D1.2 · Ch 8 §3 · ...

ResourceQuota caps the namespace's aggregate; LimitRange supplies per-object defaults and bounds. They work together: the LimitRange's default request is often what makes objects countable against the quota in the first place. *[cross-bearing: see Ch 8 §3 — dividing a shared cluster]*

- **A is wrong** because they operate at different scopes and neither substitutes for the other. A quota alone cannot stop one Pod claiming the whole 10Gi.
 - **C is wrong** because it swaps them. Nothing about a LimitRange sums across the namespace.
 - **D is wrong** because supplying defaults for objects that *don't* declare requests is a LimitRange's central job.
-

41. D — D2.3 · Ch 13 §4 · ...

OOMKilled is the container exceeding its own memory limit and being terminated by the kernel's OOM killer. *[cross-bearing: see Ch 5 §8 — what a Pod is owed]*

- **A is wrong**, and this is the distinction the question exists for. Node-level memory pressure produces Evicted, and eviction order follows QoS class. *[cross-bearing: see Ch 13 §4 — Pods that start and then don't stay]*
 - **B is wrong** because a liveness failure restarts the container without that reason string; you would see the restart count climb with a different cause.
 - **C is wrong** because a scheduling failure leaves the Pod Pending, with no container running for anything to kill.
-

42. A — D1.4 · Ch 2 §6 · ...

IfNotPresent uses the copy already on the node when there is one. The effective default is not fixed: it is IfNotPresent for a normal tag, and Always when the reference is :latest or carries no tag at all. *[cross-bearing: see Ch 2 §6 — when Kubernetes pulls, and when it doesn't]*

- **B is wrong** because the default is conditional on the tag, which is exactly the detail this item tests. Assuming an unconditional Always is how people convince themselves a stale image is impossible.

- **C is wrong** because `Never` means use only what is already on the node, and fail if nothing is there. The kubelet builds nothing.
- **D is wrong** because `Always` re-checks the registry on every container start but places no requirement on how the image is referenced. A digest reference is a separate, stronger guarantee.

43. C — D2.4 · Ch 11 §3 · ..

A `StorageClass` names a provisioner, and that provisioner creates the `PersistentVolume` when a claim asks for the class. That is what "dynamic" means: supply is manufactured in response to demand, rather than pre-staged by an administrator. *[cross-bearing: see Ch 11 §3 — provisioning on demand]*

- **A is wrong** because `accessModes` describes how many nodes may mount the volume and in what mode. It is a compatibility constraint on matching, not a switch between static and dynamic supply. *[cross-bearing: see Ch 11 §4 — access modes and what happens after]*
- **B is wrong**, and it is the answer that sounds most like "storage appears by itself." `hostPath` ties the data to one node's filesystem, which is the opposite of what a claim is for.
- **D is wrong** because a reclaim policy governs what happens to a volume *after* its claim is deleted. It acts at the end of the lifecycle, not the beginning.

44. B — D3.1 · Ch 14 §5 · ..

Kustomize patches a shared base with one overlay per environment, and it does so without a templating language — the overlays are ordinary YAML fragments merged onto the base. The question's two constraints, no duplication and no templating, name Kustomize exactly. *[cross-bearing: see Ch 14 §5 — patching instead of templating]*

- **A is wrong** on the second constraint only, which is what makes it the strongest distractor. Separate values files genuinely solve the duplication; Helm reaches that answer *through* a template language, which the question rules out. *[cross-bearing: see Ch 14 §6 — which one, when]*
- **C is wrong** because `sync` waves order the objects within an apply. Ordering is not variation between environments. *[cross-bearing: see Ch 15 §5 — ordering the sync]*
- **D is wrong** because it is still Helm, and a generator does not remove the templating the question excludes.

45. D — D1.3 · Ch 7 §1 · ..

Filter, then score, then bind. Filtering reduces the nodes to the feasible ones; scoring ranks what survives; binding writes the choice. The order matters because each stage operates on the output of the one before it. *[cross-bearing: see Ch 7 §1 — one decision, made once]*

- **A is wrong** because it inverts the two stages. Scoring every node and then discarding low scorers would let an infeasible node be ranked at all, and feasibility is not a matter of degree. *[cross-bearing: see Ch 7 §2 — what makes a node feasible]*
- **B is wrong** because binding is the last act, not the first. A Pod is bound once, to a node already chosen.

- **C is wrong** because the kubelet is not consulted for placement. It acts on Pods already assigned to its node. *[cross-bearing: see Ch 3 §3 — node components in context]*

46. A — D2.1 · Ch 10 §5 · ..

The Gateway API's three kinds are modeled on three roles: GatewayClass for the infrastructure provider, Gateway for the cluster operator, HTTPRoute for the application developer. The split is the design, not an accident of the API. *[cross-bearing: see Ch 10 §5 — roles, not just routes]*

- **B is wrong** because it scrambles the mapping. Reading it against the responsibilities is the check: an application developer does not supply the cluster's networking infrastructure.
- **C is wrong** on both counts. HTTPRoute is namespaced and belongs to the team that owns the application, which is the whole point of a role-oriented API.
- **D is wrong** for the same reason as B — it inverts the outermost and innermost roles.

47. C — D4.2 · Ch 17 §3 · ..

Independent deployment and independent scaling are the gains; operational and network complexity is the price. The honest characterization keeps both halves, because a trade-off with only an upside is not a trade-off. *[cross-bearing: see Ch 17 §3 — small pieces, replaced whole]*

- **A is wrong**, and it is the most seductive option here. Decomposition *relocates* complexity into the spaces between services; it does not reduce the total. What was a function call becomes a network call that can fail.
- **B is wrong** because service boundaries change the *shape* of coupling rather than removing it. Two services that must be released together are still coupled, now across a network.
- **D is wrong** because a repository and a pipeline per service are the mechanics of independent deployment, not its cause. Shared databases and lockstep releases defeat it regardless.

48. B — D1.1 · Ch 6 §7 · ..

A CronJob creates a Job on its schedule, and the Job creates the Pod or Pods that run the work to completion. Two objects, one for the schedule and one for the run. *[cross-bearing: see Ch 6 §7 — one per node, and work that ends]*

- **A is wrong** because it collapses the layer that makes retries and completion tracking work. Without a Job there is nothing holding the notion of "this run finished."
- **C is wrong** because a Deployment is for work that should keep running. Its controller replaces a Pod that exits, which is the opposite of what run-to-completion needs.
- **D is wrong** because a DaemonSet is about node coverage, not scheduled work.

49. A — D2.2 · Ch 12 §7 · ..

An SBOM is the inventory: it records which components the image contains, which is exactly the question an auditor is asking when a new vulnerability is disclosed. *[cross-bearing: see Ch 12 §7 — trusting what you ship]*

- **B is wrong** because a scan report is a point-in-time answer against the vulnerabilities known on the day it ran. A vulnerability disclosed afterwards is not in it, which is the scenario the question describes.
- **C is wrong**, and the distinction is worth holding: provenance records *how and where* the artifact was built. It is the build-process half; the SBOM is the components half.
- **D is wrong** because a digest identifies the image uniquely without saying anything about what is inside it. *[cross-bearing: see Ch 2 §3 — registries, tags, and digests]*

50. D — D3.2 · Ch 16 §5 · ..

A working port-forward proves the application is running and answering on its port. The Service path is the only thing left between that Pod and the client, so the fault is in the Service path — selector, ports, or endpoint membership. *[cross-bearing: see Ch 16 §5 — bypassing the Service on purpose]*

- **A is wrong**, and it is the answer that feels responsible. If the application were unhealthy it would not have answered the port-forward. A successful port-forward rules this out rather than supporting it.
- **B is wrong** because port-forward and Service traffic are not separable that way at this level; a CNI failing between nodes would not leave a single Pod cleanly reachable while its Service silently times out.
- **C is wrong** because cluster IPs are allocated from a range and not handed out twice. Inventing an allocator bug is a heavier explanation than the ordinary one.

51. B — D1.1 · Ch 4 §5 · ..

Labels identify and select; annotations carry metadata that nothing selects on. That is the whole distinction, and it is why a selector is a query over labels specifically. *[cross-bearing: see Ch 4 §5 — the universal join]*

- **A is wrong** because the difference is mechanical rather than stylistic. Put a value in an annotation and no selector can find it.
- **C is wrong** because selectors cannot match annotations at all, whatever the length. Length is a real constraint on label values, but it does not open a door to annotations.
- **D is wrong** because labels are not unique keys. Non-uniqueness is the point: a selector matches a *set*.

52. C — D2.1 · Ch 9 §7 · ..

An unqualified name resolves in the client Pod's own namespace, because that namespace comes first in the Pod's DNS search list. A Pod in `orders` reaches the `billing` Service in `orders`. *[cross-bearing: see Ch 9 §7 — names, and where they resolve]*

- **A is wrong**, and it is the most common wrong model. `default` is a namespace like any other; it has no privileged position in resolution.

- **B is wrong** because the search list exists precisely so that short names work. Requiring an FQDN everywhere would make the convenience pointless.
- **D is wrong** because resolution is not first-come-first-served across the cluster. Two namespaces may each hold a `billing` Service, and each resolves within its own.

53. D — D4.3 · Ch 17 §8 · ..

A substantial change goes through a Kubernetes Enhancement Proposal, brought to the SIG that owns the subsystem. The KEP is the document; the SIG is the structure. *[cross-bearing: see Ch 17 §8 — how the project actually runs, and how you'd join]*

- **A is wrong** because a pull request is how a change *lands*, not how it is proposed and agreed. The Steering Committee handles project governance rather than technical review of subsystems.
- **B is wrong** because the Landscape is a catalog of the ecosystem, not an intake path for Kubernetes changes.
- **C is wrong** because a TAG operates at CNCF level across projects. It is the wrong altitude for a change to one Kubernetes subsystem.

54. A — D1.4 · Ch 2 §4 · ..

CRI standardizes the boundary between the kubelet and the container runtime. Because the interface is fixed, runtimes behind it are interchangeable — which is what let the ecosystem change runtimes without changing Kubernetes. *[cross-bearing: see Ch 2 §4 — the container runtime interface]*

- **B is wrong** because image format and layer ordering are the OCI image specification's business. *[cross-bearing: see Ch 2 §5 — the Open Container Initiative]*
- **C is wrong** because registry distribution is the OCI distribution specification.
- **D is wrong** because giving a container a network address is CNI's job. Keeping the four interfaces apart by *what they stand between* is the reliable way to hold them. *[cross-bearing: see Ch 17 §4 — every place Kubernetes lets you in]*

55. C — D1.1 · Ch 5 §1 · ..

Containers in a Pod share a network namespace, so they reach each other on `localhost` and share one IP address and one port space. *[cross-bearing: see Ch 5 §1 — the Pod as the unit of scheduling]*

- **A is wrong** because filesystems are not shared by default. Sharing files between containers in a Pod takes a volume, mounted into both. *[cross-bearing: see Ch 11 §1 — three lifetimes, and the volumes that have them]*
- **B is wrong** because process-namespace sharing is opt-in, not the default.
- **D is wrong** because limits are set per container. What the Pod has is a QoS class derived from all of them together. *[cross-bearing: see Ch 5 §8 — what a Pod is owed]*

56. B — D3.1 · Ch 15 §1 · ..

ConfigMaps and Secrets supply configuration at run time, so one image runs unchanged in every environment. That is twelve-factor III implemented with Kubernetes objects. *[cross-bearing: see Ch 15 §1 — twelve factors, and the ones Kubernetes already solved]*

- **A is wrong** and is the practice the factor exists to forbid. An image per environment means the artifact you tested is not the artifact you shipped. *[cross-bearing: see Ch 2 §2 — what's inside an image]*
- **C is wrong** because a `hostPath` makes the configuration a property of the node, which is worse than baking it in: now it varies invisibly.
- **D is wrong** because rebuilding on a configuration change is exactly the coupling between build and config that the factor separates.

57. D — D1.1 · Ch 6 §4 · ...

Five. `maxSurge: 1` permits one Pod above the desired count of four, so the ceiling during a rolling update is five. `maxUnavailable: 0` sets the floor at four ready Pods rather than the ceiling. *[cross-bearing: see Ch 6 §4 — changing the fleet under way]*

- **A is wrong** because it reads `maxUnavailable: 0` as freezing the total. It constrains how far the count may fall, not whether it may rise.
- **B is wrong** because that is a blue/green shape, not a rolling update. `maxSurge` is one, not the replica count. *[cross-bearing: see Ch 15 §2 — ways to replace what's running]*
- **C is wrong** because taking a Pod down first is precisely what `maxUnavailable: 0` forbids. This configuration adds before it removes.

58. A — D1.3 · Ch 7 §3 · ...

`preferredDuringSchedulingIgnoredDuringExecution` expresses a preference: the scheduler favors matching nodes when they exist and schedules the Pod anyway when they do not. Soft rules score; they do not filter. *[cross-bearing: see Ch 7 §3 — asking for a particular berth]*

- **B is wrong** because `nodeSelector` is a hard requirement. No matching node means the Pod stays Pending — the exact outcome the question rules out.
- **C is wrong** because taints and tolerations work the other way round: a toleration lets a Pod onto a tainted node, it does not express a preference among untainted ones. *[cross-bearing: see Ch 7 §4 — when the berth refuses you]*
- **D is wrong** because a required rule is the hard form, and hard rules filter.

59. B — D3.2 · Ch 16 §4 · ...

The Service selects `app=orders`; the Pods carry `app=order-api`. The `EndpointSlice` exists and is empty, which is the signature of a selector that matches nothing. *[cross-bearing: see Ch 16 §4 — is anything even selected]*

Read the output in the order it is printed. The slice was created, so the Service has a selector and the control plane is doing its job. Its `ENDPOINTS` column reads `<none>`, so nothing matched. And both Pods are `1/1 READY`, which rules out readiness as the cause — a Pod that matched but was not ready would still be *listed*, carrying a condition saying it must not receive traffic. **An empty list is a selector problem; a list that serves nothing is a readiness problem.** *[cross-bearing: see Ch 9 §4 — the list behind the name]*

- **A is wrong** on the evidence in front of you. `1/1 READY` is the readiness probe passing; a startup probe still running would show `0/1`.
- **C is wrong** because a `targetPort` mismatch produces endpoints that exist and refuse connections. The slice would not be empty.
- **D is wrong** because an Ingress controller has nothing to do with a Service's endpoint list. A ClusterIP Service does not route through one. *[cross-bearing: see Ch 10 §3 — the object is not the implementation]*

60. C — D1.1 · Ch 3 §6 · ..

A controller loops continuously, comparing desired state against observed state and acting to close the difference. Not once, not on demand, not when invoked — continuously. *[cross-bearing: see Ch 3 §6 — controllers and the control loop]*

- **A is wrong** because a one-shot sequence at creation cannot explain the behavior you have watched all book: delete a Pod under a Deployment and it comes back, with nobody running anything.
- **B is wrong** because it inverts the relationship. The scheduler is itself one of these loops; it does not invoke the others. *[cross-bearing: see Ch 7 §1 — one decision, made once]*
- **D is wrong** because `kubectl apply` writes the desired state and stops. What acts on it afterwards is the loop, running whether or not anyone is at a terminal. *[cross-bearing: see Ch 4 §6 — a declaration, not an order]*

Scoring Rubric

Mark each item right or wrong, then tally by domain. The per-domain maximums below come from the published blueprint weighting applied to sixty items [source: `cncf-curriculum-kcna-readme-2026-08-31`]; the item lists are this instrument's own.

Domain	Items	Max	Your score
D1 — Kubernetes Fundamentals (44%)	1, 3, 5, 8, 10, 13, 16, 19, 21, 23, 26, 28, 30, 33, 35, 38, 40, 42, 45, 48, 51, 54, 55, 57, 58, 60	26	—
D2 — Container Orchestration (28%)	2, 6, 9, 12, 14, 17, 20, 24, 27, 31, 34, 36, 41, 43, 46, 49, 52	17	—
D3 — Cloud Native Application Delivery (16%)	4, 11, 18, 25, 32, 37, 44, 50, 56, 59	10	—
D4 — Cloud Native Architecture (12%)	7, 15, 22, 29, 39, 47, 53	7	—
Total		60	—

Dead Reckoning: The Linux Foundation publishes the passing standard for its multiple-choice exams in the candidate FAQ: a score of 75% or above must be earned to pass [source: lf-mc-exam-faq-2026-08-31]. That is 45 of 60 on an instrument this size. The figure is published for multiple-choice exams as a class rather than on the KCNA product page, which is the provenance point Chapter 1 made and this chapter's Instructions restated.

What your score means here. This is a calibrated practice instrument, not a predictor. Treat the bands as a reading of where your study time should go next, and nothing more.

Band	Reading	What to do next
52–60	Comfortable across all four domains.	Work the items you missed rather than re-reading whole chapters. Then Chapter 19 §6's final-week plan.
45–51	Above the published standard, with soft spots.	Find the domain where you lost the most <i>proportionally</i> , not absolutely — losing 3 of 7 in D4 is worse than 5 of 26 in D1. Re-read that domain's chapters' Fixed Points and Bearings.
36–44	Below the standard, and the gap is addressable.	Do not re-read the book front to back. Take the two weakest domains, re-read only their ☹ Taking Your Bearings checkpoints and ☆ Fixed Points, then retake those items.
Under 36	The material has not consolidated yet.	Go back to the chapters, not the questions. Read the Zenith section of each chapter first — they are the syntheses — and use Chapter 19 §1's nine threads as the map.

The per-domain reading matters more than the total. A total that clears 75% while D4 sits at 3 of 7 is a real risk: Cloud Native Architecture is 12% of the exam and the competency technically strong candidates most reliably under-study. The tally exists so that a passing total cannot hide a failing domain. [cross-bearing: see Ch 19 §4 — where the weight actually is]

One thing the score cannot tell you. The real exam reports an outcome, not a domain breakdown. This is the last time you will see one, so use it.

And if you cleared the band comfortably with even sub-scores, you are not looking for more study material. You are looking for a date.

Calculated Length: n/a (content-driven) | Chapter type: mock_exam

The Lodestar — KCNA

One page. Read it the week before. Work it in the last hour. Then close it.

This is a **distillation, not a summary**. There are no chapter recaps here, no explanations, no worked examples, and no context — every line assumes you have read the chapter it came from. If you cannot reconstruct a chapter from its lines here, you have not found a defect in this page. You have found a chapter to go back to. *[cross-bearing: see Ch 19 §5 — using The Lodestar]*

How to work it in the last hour. Two passes, about twenty minutes each, with a break between. **First pass — drill:** cover the right-hand column of the discriminator blocks and the hazard block, work down, mark every hesitation. Do not look anything up; the point is to *find* soft spots, not fix them. **Second pass — lookup:** read the marked items and the number blocks, once, calmly. Then stop. Whatever is not in your head by then will not arrive in the next twenty minutes, and a third pass costs composure and buys nothing.

1. The exam's published shape — *lookup*

Fact	Value	Where it is published
Duration	90 minutes , timer cannot be paused	KCNA exam page and candidate handbook
Questions	60 , multiple choice	Candidate handbook, "Multiple Choice Exams: Important Instructions"
Passing score	75%	Candidate handbook, MC exam FAQ
Format	Online, proctored, multiple choice	KCNA exam page
Attempts	Two included · 12-month eligibility · valid 2 years	KCNA exam page
Prerequisites	None	KCNA exam page

The provenance point, because it is the one people get wrong. All three numbers are published by the Linux Foundation — but for multiple-choice exams **as a class**, in the candidate handbook, not on the KCNA product page most candidates read. They are official. They are just not where you would look. *[cross-bearing: see Ch 1 § Ninety Minutes: The Exam as Published]*

Domain weights

#	Domain	Weight	Chapters
1	Kubernetes Fundamentals	44%	2–8
2	Container Orchestration	28%	9–13
3	Cloud Native Application Delivery	16%	14–16
4	Cloud Native Architecture	12%	17–18

The 2025-11-24 revision **doubled** Application Delivery from 8% to 16% and folded the old standalone Observability domain into Cloud Native Architecture. Material written against the five-domain blueprint under-serves D3 badly. *[cross-bearing: see Ch 1 § What Changed, and What It Costs You]*

2. Exam-day pacing — *memorize; there is no scratch paper*

☆ **Read the question count off the screen. Divide the clock by it. Reach the end of the paper at roughly 60% of your total time, and hold the remaining 40% in reserve.**

On 90 minutes that is the end of the first pass at about the **54-minute mark**, with ~36 minutes left. You do not need the count in advance — you read it, you divide, you go.

- **Don't know it?** Answer anyway, then mark it. A blank scores zero with certainty; no wrong-answer penalty is published in either direction.
- **The failure mode this guards against** is spending four minutes on question nine.
- **Check the console on the tutorial screen**, before the clock matters: is there a flag/mark control, and will it let you move backward? Thirty seconds of checking buys you the shape of the next ninety minutes.

3. The four hazards where intuition is wrong — *drill*

The intuition	What is actually true
ReadWriteOnce means one Pod	It means one node . Multiple Pods <i>on that node</i> can share it. ReadWriteOncePod is the one that means one Pod
storageClassName: "" is the same as omitting it	"" means no class — bind only to a PV that also has none. Omitting it means "whatever the cluster's default behavior is"
A second default IngressClass is ambiguous	It is worse: it removes the default, and the Ingress can no longer be created at all
Phase Running means working	It means bound, containers created, at least one running or starting or restarting . A crash-looping Pod reports Running

4. The version-skew numbers — *lookup, but read it twice*

Component	Rule
kubelet	Up to three minor versions older than kube-apiserver. Never newer
kubect1	Within one minor version, either direction . The only component permitted to be newer
Supported releases	Three supported minors · ~1 year of patch support · ~3 minor releases per year

⚠ The kubelet rule and the kubect1 rule are **different rules**, and candidates routinely apply the first to the second. *kubelet: three minors, older only. kubect1: one minor, either direction*. Not the same number, not the same shape.

5. The confusion-pair discriminators — *drill*

Cover the right column. Work down. Uncover. This is the block that repays a second pass.

Domain 1 — Kubernetes Fundamentals

Pair	The one-line test
Pod phase vs container state	Phase = the whole Pod's position in its lifecycle. State = one container, right now
liveness vs readiness vs startup	Liveness restarts . Readiness removes from endpoints , no restart. Startup suspends the other two while booting
labels vs annotations	Does anything <i>select</i> on it? Labels yes, annotations no
image tag vs digest	Mutable pointer, or identity? A tag can be moved; a digest cannot
ConfigMap vs Secret	Would you mind seeing it in a log? Secret is base64- encoded , not encrypted
namespace vs label	Namespace is a <i>scope</i> boundary. Label is a <i>selection</i> attribute
ResourceQuota vs LimitRange	Total, or default? Quota caps the namespace's aggregate. LimitRange sets per-object defaults
Deployment vs StatefulSet	Does replica 0 need to be replica 0? It is about identity , not disk
DaemonSet vs "replicas = node count"	DaemonSet tracks nodes joining and leaving. A number does not know what a node is
Job vs CronJob	CronJob is the factory; Job is the product
taints/tolerations vs node affinity	Who refuses? Taint = the node repels. Affinity = the Pod asks
OCI vs CRI	OCI = the artifact and its execution. CRI = the kubelet's API to a runtime
scheduler binds / kubelet starts	Scheduler decides and writes it down. Kubelet does the starting
scheduled once vs rescheduled	A Pod is bound once and never moved. "Rescheduling" is a <i>new</i> Pod
restartPolicy scope	Governs containers within a Pod , not the Pod object

Domain 2 — Container Orchestration

Pair	The one-line test
ClusterIP / NodePort / LoadBalancer	Reachable from where? Each type includes the one beneath it
headless vs broken vs selectorless	Headless is clusterIP: None on purpose . Broken is a selector matching nothing
Ingress object vs controller	The record versus the thing that acts on it
Ingress frozen vs deprecated	Frozen = feature-complete, still supported. Deprecated would mean scheduled for removal. Ingress is frozen
NetworkPolicy default	Is this Pod selected by <i>any</i> policy? No → fully open. Yes → union of what its policies allow
NetworkPolicy needs both ends	A→B needs A's egress and B's ingress
RBAC has no deny	Permissions are the union of every binding naming you. Nothing subtracts
view / edit / admin	view reads (not Secrets). edit writes workloads. admin manages RBAC within a namespace . None is cluster-admin
PV vs PVC	Supply versus demand. A Pod references a PVC , never a PV
Retain / Delete / Recycle	Retain keeps it for manual reclamation. Delete removes both. Recycle is deprecated
empty slice vs slice that serves nothing	Empty list = selector problem. Listed but not ready = readiness problem
Pending → don't reach for logs	No containers yet, so nothing produced output. Read the phase, then events

Domain 3 — Cloud Native Application Delivery

Pair	The one-line test
chart vs release vs revision	Chart = package. Release = one installed instance. Revision = a numbered version of that release
charts/ vs chart repository	charts/ is a directory inside a chart holding subcharts. A repository is a remote place charts come from
Kustomize overlay vs Helm template	Patch, or render? Kustomize patches a valid manifest. Helm renders one that was not a manifest until values arrived
rollout undo vs helm rollback	Whose history? A Deployment's revisions, or a release's
rolling / Recreate / blue-green / canary	What happens to the old version? Incremental / all stopped first / both then switch / a fraction
GitOps push vs pull	Where do the credentials live? Outside reaching in, or inside reaching out
OutOfSync = drift, not error	It reports a difference . That is a status, not a failure
platform vs application scope	Is the Pod running the way Kubernetes intended? If yes and the app misbehaves, it is the application

Domain 4 — Cloud Native Architecture

Pair	The one-line test
mesh vs cluster control plane	The cluster's is kube-apiserver/etcd/scheduler/controller-manager. A mesh's configures proxies
sidecar vs ambient mode	Proxy per Pod , or per node
Knative Serving vs Eventing	Serving = HTTP request-driven, scale to zero. Eventing = asynchronous routing
maturity-level ordering	Sandbox → Incubating → Graduated. <i>Which project sits where is not a memorization target</i>
TOC vs Governing Board	Technical, or business? Board = marketing, business oversight, budget
End User TAB vs TOC	The TAB advises and voices end users. The TOC holds the technical vision
SIG vs Working Group vs Committee	SIGs are ongoing and own a topic. WGs are time-bounded and cross SIGs. Committees have closed membership
TAG vs SIG	TAGs are CNCF-level across projects. SIGs are Kubernetes-project-level
horizontal vs vertical scaling	More replicas, or bigger ones
HPA vs VPA	HPA is core API. VPA is an add-on you deploy
workload vs node autoscaling	HPA/VPA/KEDA change <i>workloads</i> . Cluster Autoscaler and Karpenter change the <i>cluster</i>
observability vs monitoring	New questions, or known ones
span vs trace	One unit of work, or the whole tree following one request
SLI vs SLO vs SLA	Measure, target, or consequence
Prometheus pull vs Pushgateway	Prometheus scrapes. The Pushgateway is for jobs that cannot be scraped

Surface-form homonyms — one word, two meanings

Word	Sense A	Sense B
namespace	Linux kernel isolation primitive	Kubernetes scope object
control plane	The cluster's	A service mesh's
sandbox	Sandboxed runtime (gVisor, Kata)	CNCF Sandbox maturity level
revision	Deployment revision	Helm release revision
rollback	<code>kubectl rollout undo</code>	<code>helm rollback</code> — and rollback by revert
label	Kubernetes label	Prometheus metric label
request	Resource request	API request
binding	Scheduler binding	PV/PVC binding — and RoleBinding
release	A Kubernetes minor release	A Helm release
Service	The Kubernetes object	A Knative Service
immutable	Image immutability	Immutable infrastructure
operator	The operator pattern	"Cluster operator" as a Gateway API role
volume	A directory the containers share	A PersistentVolume with its own lifecycle

6. Governance and institutional vocabulary — *lookup; the cheapest block to refresh*

This is the Domain 4 material most likely to have gone soft, and it is bounded — which is exactly why it is worth a pass.

Body	What it does
CNCF Governing Board	Marketing, business oversight, budget decisions
TOC (Technical Oversight Committee)	Technical vision; project acceptance and lifecycle
End User TAB	The voice of end users ; advisory, not governing
TAG	CNCF-level, coordinates interests across projects
Kubernetes SIG	Project-level, ongoing, owns a topic area
Working Group	Time-bounded , spans SIGs
Committee	Closed membership; does not always operate in public
Steering Committee	Kubernetes project governance

- **Maturity levels:** Sandbox → Incubating → Graduated.

- **Proposing a change:** a **KEP**, brought to the SIG that owns the subsystem.
 - **Release cadence:** three minor releases a year, roughly every fifteen weeks.
 - **The certification ladder:** KCNA is pre-professional, feeding CKA (Certified Kubernetes Administrator), CKAD (Application Developer) and CKS (Security Specialist).
-

7. The one sentence to carry in

An object without its component does nothing.

A type: LoadBalancer Service with no provider. A Service whose selector matches no Pods. An Ingress with no Ingress controller. A NetworkPolicy on a plugin that does not implement it. An HPA with no metrics-server. One rule, five instances, and the first instinct is always "something is misconfigured" when the answer is "something is not installed."

Chart → Passage → Dawn. Close the book. Go and pass it.

Glossary

Every term the book uses in a load-bearing way, plus the ones you will meet in the wild and want a one-line answer for. Skim it, or `Ctrl+F` it. Entries name the chapter that owns the idea, so a term you cannot place sends you somewhere specific rather than back to the beginning.

Where a word means two different things in this material — and several do — both senses appear together, because the collision is the thing worth knowing. Those are marked  **two senses**.

A

ABAC (Attribute-Based Access Control) — An authorization mode that grants access through policies combining attributes. Kubernetes supports it; RBAC is the one the curriculum teaches and essentially every cluster uses. (Ch 12)

Access mode — A PersistentVolume/PersistentVolumeClaim field describing **how many nodes** may mount a volume and in what mode: ReadWriteOnce (RWO), ReadOnlyMany (ROX), ReadWriteMany (RWX), ReadWriteOncePod (RWOP). Node-count semantics, not permission semantics. (Ch 11)

Addon — Optional cluster software that is not part of the core control plane: CoreDNS, a CNI plugin, metrics-server, an Ingress controller. Kubernetes runs without most of them, badly. (Ch 3)

Admission controller — A plugin in the API server that intercepts a request after authentication and authorization but before persistence. It can validate (yes/no) or mutate (yes, in modified form). The third of the three API access gates. (Ch 8)

Admission webhook — An admission controller implemented outside the API server and called over HTTP. **Validating** webhooks accept or reject; **mutating** webhooks may also modify the object. (Ch 8, Ch 12)

Aggregation layer — The API-server mechanism that lets an extension server serve an additional API group, as metrics-server does for the Metrics API. Distinct from CRDs, which extend the API server in place. (Ch 17)

Alertmanager — The Prometheus component that handles alerts: deduplicating, grouping, and routing them to receivers. Prometheus evaluates rules; Alertmanager decides who hears about it. (Ch 18)

Ambient mode — A service-mesh data-plane arrangement using a per-node L4 proxy and optionally a per-namespace L7 proxy, instead of a sidecar in every Pod. (Ch 17)

Annotation — Key/value metadata on an object that **nothing selects on**. For tools, humans, and controllers that look things up by name. Contrast **label**. (Ch 4)

API group — A collection of related resource kinds. The core group is "" (Pods, Services, ConfigMaps); apps holds Deployments and StatefulSets; rbac.authorization.k8s.io holds Roles and bindings. (Ch 4)

API server (kube-apiserver) — The control-plane component that serves the Kubernetes API. The only component that writes to etcd, and the only way into the cluster. (Ch 3)

APIService — An object registering an extension server to serve a particular API group through the aggregation layer. (Ch 17, Ch 18)

Argo CD — A declarative GitOps continuous-delivery tool. A controller inside the cluster watches a Git repository and reconciles the cluster toward it. (Ch 15)

Attestation — A signed statement about an artifact — how it was built, what it contains, who approved it. Provenance is one kind. (Ch 12)

Autoscaling — Adjusting capacity automatically. **Horizontal** adds replicas (HPA, KEDA); **vertical** resizes existing ones (VPA); **cluster** autoscaling adds and removes nodes (Cluster Autoscaler, Karpenter). (Ch 6, Ch 17)

B

Baggage — An OpenTelemetry signal: contextual key/value data propagated alongside a trace so downstream services can read it. The signal candidates most reliably forget. (Ch 18)

Base (Kustomize) — A directory of valid, applicable manifests that overlays patch. Contrast a Helm **template**, which is not a manifest until values are applied. (Ch 14)

Binding —  **two senses.** (1) The scheduler's act of writing a chosen node onto a Pod. (2) The association of a PersistentVolumeClaim with a PersistentVolume. A **RoleBinding** is a third, unrelated object. (Ch 7, Ch 11, Ch 12)

Blue/green deployment — Running the old and new versions side by side and switching traffic between them at once. Contrast **canary**, which shifts a fraction at a time. (Ch 15)

C

Canary deployment — Exposing a subset of users to a new version while the rest continue on the old one, then widening if it holds. (Ch 15)

cgroup (control group) — The Linux kernel feature that limits and accounts for a process's resource use. What makes a container's CPU and memory limits real. (Ch 2)

Chart — Helm's packaging format: a directory of files describing a related set of Kubernetes resources. The **package**, not an installation of it. (Ch 14)

Chart repository — A remote location charts are fetched from. ⚠ Not to be confused with a chart's own charts/ directory, which holds vendored subcharts. (Ch 14)

chart.yaml — A chart's required metadata file. Carries `apiVersion`, `name` and `version` as required fields, and `appVersion` — the version of the *application* the chart installs — as an optional one, explicitly unrelated to `version`. (Ch 14)

CI/CD — Continuous Integration and Continuous Delivery/Deployment. A pipeline that builds, tests, and ships. In the **push** model it holds cluster credentials and reaches in; GitOps inverts that. (Ch 15)

Cilium — A CNI plugin providing a flat Layer 3 network with an eBPF data plane. A CNCF graduated project. (Ch 9)

Cloud native — Per the CNCF definition, an approach building and running scalable applications in modern, dynamic environments, exemplified by containers, service meshes, microservices, immutable infrastructure and declarative APIs. Never hyphenated. (Ch 17)

CloudEvents — A CNCF specification for describing event data in a common way, so an event from one system is intelligible to another without a bespoke adapter. (Ch 17)

Cluster Autoscaler — Adds and removes **nodes** in response to unschedulable Pods and underused capacity. Contrast HPA/VPA, which change workloads. (Ch 17)

ClusterIP — The default Service type. Exposes the Service on a cluster-internal virtual IP, reachable only from inside the cluster. (Ch 9)

ClusterRole / ClusterRoleBinding — RBAC objects that are not namespaced. A ClusterRole may be bound cluster-wide (ClusterRoleBinding) or into one namespace (RoleBinding). (Ch 12)

CNI (Container Network Interface) — The interface a network plugin implements to provide Pod networking. **A CNI plugin is required to implement the Kubernetes network model**; the container runtime loads it. (Ch 9)

ConfigMap — An object holding non-confidential configuration as key/value data, consumed as environment variables or mounted as a volume. (Ch 4)

Container — A runnable unit packaging an application with its dependencies, isolated by kernel features rather than by a separate operating system. Contrast a **virtual machine**. (Ch 2)

containerd — A container runtime implementing CRI. A CNCF graduated project. (Ch 2)

Container runtime — The software that runs containers on a node. The kubelet talks to it through CRI. (Ch 2, Ch 3)

Context (kubeconfig) — One named bundle of cluster, user and namespace. The **current context** is the one `kubectl` uses when you do not say otherwise. (Ch 8)

Control loop — A non-terminating loop that regulates a system: observe current state, compare against desired state, act to close the gap. The animating idea of Kubernetes. (Ch 3)

Control plane —  **two senses.** (1) The cluster's: API server, etcd, scheduler, controller manager. (2) A service mesh's: the component configuring the mesh's proxies. Never use the bare phrase where both are in play. (Ch 3, Ch 17)

Controller — A control loop that watches one or more resource types and works to bring current state closer to the desired state recorded in `.spec`. (Ch 3)

Controller manager (kube-controller-manager) — The control-plane component running the built-in controllers as a single binary. (Ch 3)

CoreDNS — The cluster DNS add-on that serves Service and Pod records. (Ch 9)

CrashLoopBackOff — A container that starts, exits, and is restarted, with the kubelet backing off between attempts. The Pod's **phase** is still `Running`. (Ch 13)

CRD (CustomResourceDefinition) — An object that adds a new resource kind to the API. Storage is all you get: nothing acts on those objects unless a controller is watching for them. (Ch 6)

`crds/` — A Helm chart directory whose CustomResourceDefinitions are installed before the templates render, because a chart shipping custom resources must define them before objects use them. (Ch 14)

CRI (Container Runtime Interface) — The interface standardizing the boundary between the kubelet and the container runtime, which is what makes runtimes interchangeable. (Ch 2)

CRI-O — A lightweight container runtime implementing CRI. (Ch 2)

`crictl` — A CLI for talking to a CRI-compatible runtime directly on a node. Useful when the cluster's own view is the thing in doubt. (Ch 13)

CronJob — An object that creates **Jobs** on a schedule. The factory; the Job is the product. (Ch 6)

CSIDriver — An object describing a CSI driver installed in the cluster, so Kubernetes knows how to interact with it. (Ch 11)

CSI (Container Storage Interface) — The interface a storage driver implements to provide volumes. The third of the four pluggable interfaces. (Ch 11)

D

DaemonSet — A controller ensuring a copy of a Pod runs on every eligible node, tracked as nodes join and leave. Its Pods automatically tolerate the unschedulable taint, so they keep running on cordoned nodes. (Ch 6)

Dead Reckoning — A branded marker in this book: a facts-only block, stated without interpretation.

Declarative API — An API where you record the state you want and something else reconciles reality toward it. Contrast **imperative**, where you issue the steps. (Ch 4)

Deployment — A controller managing a ReplicaSet to run a stated number of interchangeable Pods, with rolling updates and revision history. (Ch 6)

Desired state — What you asked for, recorded in an object's `.spec`. Contrast **current state**, reported in `.status`. (Ch 3, Ch 4)

Digest — A content hash identifying an image immutably. Contrast a **tag**, which is a mutable pointer. (Ch 2)

Distroless image — A container image containing the application and its runtime dependencies and nothing else — no shell, no package manager. Excellent for security, awkward for `kubectl exec`, which is why ephemeral containers exist. (Ch 16)

Drift — A difference between the declared state in a repository and the actual state in the cluster. Argo CD reports it as `OutOfSync`, which is a status, not an error. (Ch 15)

E

eBPF — A Linux kernel technology allowing sandboxed programs to run in kernel space, used by some CNI plugins as a data plane and by security tools such as Falco. (Ch 9, Ch 12)

EndpointSlice — An object listing the network endpoints backing a Service. The control plane creates them for any Service with a selector, and they reference **all** the Pods the selector matches — each endpoint carrying `ready` / `serving` / `terminating` conditions. (Ch 9)

Endpoints (legacy object) — The older object EndpointSlice replaced. You will meet it in `kubectl api-resources` output and in older material. This book teaches EndpointSlice. (Ch 9)

Envoy — A proxy commonly used as a service mesh's data plane. A CNCF graduated project. (Ch 17)

Ephemeral container — A container added to a **running** Pod for debugging. It exists because a Pod's container list is fixed at creation and cannot be changed. (Ch 16)

etcd — The consistent, distributed key-value store holding all cluster data. The only stateful component of the control plane. (Ch 3)

Eviction — The termination of a Pod by the kubelet or by the node controller, typically under resource pressure or after a node stops reporting. Shows as Evicted. (Ch 13)

ExternalName — A Service type mapping a name to an external hostname via a CNAME record. ⚠ Not a rung on the ClusterIP/NodePort/LoadBalancer ladder: no cluster IP, no endpoints, no proxying. (Ch 9)

F

Falco — A CNCF runtime-security project that detects anomalous behavior at run time. (Ch 12)

Finalizer — A key on an object that blocks deletion until a controller has done its cleanup and removed the key. (Ch 11)

Fixed Point — A branded marker in this book: a must-memorize concept.

Flannel — A CNI plugin providing an overlay network. (Ch 9)

Fluentd / Fluent Bit — Log collectors, usually run as a DaemonSet. Fluentd is one word; Fluent Bit is two. Both are CNCF projects. (Ch 18)

Flux — A GitOps delivery agent, alternative to Argo CD. A CNCF graduated project. (Ch 15)

FQDN (fully qualified domain name) — A complete DNS name. In-cluster Services take the form `<service>.<namespace>.svc.<cluster-domain>`. (Ch 9)

G

Gateway API — The role-oriented successor to Ingress, shipped as CRDs rather than built in.

GatewayClass belongs to the infrastructure provider, **Gateway** to the cluster operator, **HTTPRoute** to the application developer. (Ch 10)

GitOps — An operating model where the desired state lives in Git and an agent continuously reconciles the cluster toward it. Its four OpenGitOps principles are: declarative, versioned and immutable, pulled automatically, continuously reconciled. (Ch 15)

Golden signals — Latency, traffic, errors, saturation. The four things worth watching on almost any service. (Ch 18)

Grafana — A visualization and dashboarding tool commonly paired with Prometheus. (Ch 18)

Graduated — The most mature CNCF project level. The ladder is Sandbox → Incubating → Graduated. ⚠ Which project sits at which level changes; the ordering is the durable fact. (Ch 17)

gVisor — A sandboxed container runtime providing stronger isolation than shared-kernel containers, selected via RuntimeClass. (Ch 2)

H

Harbor — A CNCF registry project with vulnerability scanning and signing. (Ch 12)

Headless Service — A Service with `clusterIP: None`. Deliberate, not broken: DNS returns the Pod addresses directly instead of one virtual IP. Required by StatefulSets for per-Pod network identity. (Ch 9)

Helm — The package manager for Kubernetes. Charts are packages; installing one produces a **release**. (Ch 14)

hostPath — A volume type mounting a path from the node's filesystem. Ties data to one node, and a hazard in most cluster contexts. (Ch 11)

HPA (HorizontalPodAutoscaler) — A built-in controller that adjusts a workload's replica count based on observed metrics. Requires the Metrics API, hence metrics-server. (Ch 6, Ch 17)

HTTPRoute — The Gateway API resource expressing application-level routing rules. Owned by the application developer. (Ch 10)

I

imagePullPolicy — When the kubelet pulls an image: Always, IfNotPresent, or Never. The **default is conditional on the tag** — IfNotPresent normally, Always for `:latest` or an omitted tag. (Ch 2)

imagePullSecrets — A Pod or ServiceAccount field naming Secrets holding registry credentials, of type `kubernetes.io/dockerconfigjson`. (Ch 12)

Immutable infrastructure — Servers and containers that are replaced rather than modified in place. ⚠ Distinct from **image immutability**, which is about a digest identifying fixed content. (Ch 17)

Incubating — The middle CNCF project maturity level. (Ch 17)

Ingress — An API object defining HTTP/HTTPS routing rules into the cluster. **Frozen**, not deprecated: feature-complete and supported, receiving no new features, with Gateway API as the recommended successor. Does nothing without an Ingress controller. (Ch 10)

Ingress controller — The component that reads Ingress objects and implements them, usually with a load balancer or edge router. Not shipped by default; you install one. (Ch 10)

IngressClass — The object naming which controller should fulfill an Ingress, referenced by `ingressClassName`. ⚠ A second default IngressClass does not create ambiguity — it removes the default, and the Ingress can no longer be created. (Ch 10)

Init:N/M — A Pod status showing that init container *N* of *M* has completed. A Pod stuck here has an init container that has not finished. (Ch 16)

Init container — A container that runs to completion before the Pod's application containers start, in order. (Ch 5)

in-toto — A CNCF framework for securing a software supply chain by making the steps of its production verifiable. (Ch 12)

iSCSI — A protocol carrying SCSI storage commands over IP networks; one of the volume types a PersistentVolume may be backed by. (Ch 11)

J

Jaeger — A CNCF distributed-tracing backend: it receives, stores and visualizes traces. Contrast **OpenTelemetry**, which produces them. (Ch 18)

Job — A controller running one or more Pods to completion. Its Pod template may use `restartPolicy` of `Never` or `OnFailure` only. (Ch 6)

K

Karpenter — A node-provisioning autoscaler that adds and removes cluster capacity. (Ch 17)

Kata Containers — A sandboxed runtime running each container in a lightweight VM. (Ch 2)

KCNA / KCSA / CKA / CKAD / CKS — The CNCF certification ladder: Kubernetes and Cloud Native Associate, Kubernetes and Cloud Native Security Associate, Certified Kubernetes Administrator, Certified Kubernetes Application Developer, Certified Kubernetes Security Specialist. (Ch 17)

KEDA (Kubernetes Event-Driven Autoscaling) — A CNCF graduated project scaling workloads on event-source metrics such as queue depth. (Ch 17)

KEP (Kubernetes Enhancement Proposal) — The document and process through which a substantial change to Kubernetes is proposed, brought to the SIG that owns the area. (Ch 17)

kind — A tool running Kubernetes clusters in Docker containers, for local development and testing. (Ch 8)

kubeadm — The officially supported tool for bootstrapping a cluster: installing the control plane and joining nodes. (Ch 8)

kubeconfig — The file holding cluster addresses, credentials and contexts. `kubectl` looks for `$HOME/.kube/config`, overridable by `KUBECONFIG` or `--kubeconfig`. (Ch 8)

kubectl — The command-line client for the Kubernetes API. ⚠ Version skew: within **one** minor version of the API server, in **either** direction — the only component permitted to be newer. (Ch 8)

kubelet — The node agent that runs containers described for its node and reports node and Pod status. ⚠ Version skew: up to **three** minor versions older than the API server, and never newer. (Ch 3, Ch 8)

kube-proxy — The node component implementing the Service virtual-IP mechanism, by watching Services and EndpointSlices and programming node rules. Modes on Linux: **iptables (default)**, IPVS, nftables; on Windows, kernel space. Optional — some CNI plugins do the work themselves. (Ch 9)

Kubernetes network model — The four requirements every implementation must satisfy: every Pod gets its own cluster-wide IP; Pods can reach all other Pods without NAT; node agents can reach Pods on their node; and Pods see their own address as others see it. (Ch 9)

Kustomize — A tool patching a shared base with per-environment overlays, without a templating language. Built into `kubectl` as `apply -k`. (Ch 14)

Kyverno — A CNCF policy engine whose policies are Kubernetes resources written in YAML and CEL. Contrast **OPA Gatekeeper**, which uses Rego. (Ch 12)

L

Label — ⚠ **two senses**. (1) Kubernetes: key/value metadata that selectors query. (2) Prometheus: a dimension on a time series. In Kubernetes, labels are selectable and annotations are not. (Ch 4, Ch 18)

LimitRange — A namespaced policy setting **per-object** defaults and bounds for resource requests and limits. Contrast **ResourceQuota**, which caps the namespace's aggregate. (Ch 8)

Liveness probe — A probe whose failure **restarts** the container. Contrast readiness (removes from endpoints) and startup (suspends the other two while booting). (Ch 5)

LoadBalancer — The Service type requesting an external load balancer from the provider. Kubernetes provides no load balancer itself; without an integration the address stays pending indefinitely. (Ch 9)

LUN (Logical Unit Number) — In a traditional storage array, an addressable unit of storage. The pre-Kubernetes analogue of a PersistentVolume. (Ch 11)

M

Manifest —  **two senses.** (1) A YAML or JSON file describing a Kubernetes object. (2) In OCI, the document listing an image's layers and configuration. (Ch 2, Ch 4)

maxSurge / maxUnavailable — Deployment rolling-update bounds. **maxSurge** is how far above the desired replica count the total may go; **maxUnavailable** is how far below the ready count may fall. (Ch 6)

Metrics API — The API serving CPU and memory metrics for autoscaling and `kubectl top`. Requires an extension server — normally `metrics-server` — registered through the aggregation layer. (Ch 13, Ch 18)

metrics-server — The cluster addon collecting resource metrics from each kubelet and serving the Metrics API. **Not installed by default** in many distributions, which is why `kubectl top` so often fails. Meant for autoscaling, not monitoring. (Ch 13)

Microservices — An architecture decomposing an application into independently deployable services. The trade: independent deployment and scaling, at the cost of operational and network complexity. (Ch 17)

minikube — A tool running a local single-node cluster for learning and development. (Ch 8)

Mirror Pod — The API-server representation of a static Pod, so that `kubectl` can show it. (Ch 13)

Monitoring — Collecting, processing, aggregating and displaying real-time quantitative data about a system — questions you decided in advance to ask. Contrast **observability**. (Ch 18)

Mutating admission webhook — An admission webhook that may modify an object before persistence. (Ch 8, Ch 12)

N

Namespace —  **two senses.** (1) Linux kernel: an isolation primitive giving a process its own view of a resource. (2) Kubernetes: a scope for object names, quota and policy. (Ch 2, Ch 4)

NAT (Network Address Translation) — Rewriting addresses as packets traverse a boundary. The Kubernetes network model forbids it between Pods. (Ch 9)

NetworkPolicy — A namespaced object restricting Pod traffic at layer 3/4. **Additive and allow-only:** a Pod selected by no policy is fully open; a Pod selected by any policy is restricted to the union of what its policies permit. There is no deny rule, and it does nothing unless the network plugin implements it. (Ch 10)

NFS (Network File System) — A protocol for sharing filesystems over a network; a volume type supporting multiple simultaneous writers, and therefore `ReadWriteMany`. (Ch 11)

Node — A worker machine, virtual or physical, running the kubelet, a container runtime, and usually kube-proxy. (Ch 3)

Node affinity — A Pod's expression of preference or requirement for nodes carrying particular labels. required... filters; preferred... scores. Contrast **taints**, where the node does the refusing. (Ch 7)

Node condition — A node's self-reported status: Ready, MemoryPressure, DiskPressure, PIDPressure, NetworkUnavailable. ⚠ Ready=False means the node is reporting a problem; Ready=Unknown means it has stopped reporting at all. (Ch 8, Ch 13)

NodePort — A Service type exposing the Service on a static port on every node, drawn from --service-node-port-range (default **30000–32767**). It **adds** a cluster IP rather than replacing it. (Ch 9)

nodeSelector — The simplest node-selection field: a hard requirement that the node carry the given labels. (Ch 7)

O

Observability — Being able to understand a system from the outside, and to ask questions of it you did not anticipate. Contrast **monitoring**. (Ch 18)

OCI (Open Container Initiative) — The body publishing the image, runtime and distribution specifications. ⚠ OCI standardizes the artifact and its execution; CRI standardizes the kubelet's conversation with a runtime. (Ch 2)

OOMKilled — The reason recorded when a container exceeds its memory limit and is terminated. The kernel does the killing; the kubelet observes and applies the restartPolicy. (Ch 5, Ch 13)

OPA (Open Policy Agent) — A CNCF policy engine; with Gatekeeper it enforces policy at admission, using the Rego language. (Ch 12)

OpenTelemetry (OTel) — The CNCF project providing APIs, SDKs and a Collector for producing telemetry. Its signals are **traces, metrics, logs and baggage**. Contrast Jaeger, a backend. (Ch 18)

Operator — ⚠ **two senses**. (1) The operator *pattern*: a custom resource plus a controller that acts on it, encoding operational knowledge. (2) "Cluster operator" as a **role name** in Gateway API. Never use the bare word for a person. (Ch 6, Ch 10)

OutOfSync — Argo CD's report that the cluster differs from the repository. A status describing drift, not an error. (Ch 15)

Overlay (Kustomize) — A directory patching a base for one environment. (Ch 14)

P

PersistentVolume (PV) — A piece of storage in the cluster, provisioned by an administrator or dynamically by a StorageClass. Cluster-scoped: it belongs to no namespace. **Supply**. (Ch 11)

PersistentVolumeClaim (PVC) — A workload's request for storage. **Namespaced** — it must live in the same namespace as the Pod using it. **Demand**. A Pod references a PVC, never a PV directly. (Ch 11)

Phase (Pod) — A Pod's high-level position in its lifecycle: Pending, Running, Succeeded, Failed, Unknown.  Running means bound with at least one container running, starting **or restarting** — a crash-looping Pod reports Running. Contrast **container state**. (Ch 5)

Pod — The smallest deployable unit: one or more containers sharing a network namespace, an IP address and storage volumes. Not a container. (Ch 5)

Pod Security Admission (PSA) — The built-in admission controller enforcing the Pod Security Standards, configured per namespace by label. Its **modes** are enforce, audit, warn. (Ch 12)

Pod Security Standards (PSS) — Three cumulative policies: privileged, baseline, restricted.  Levels say what is checked; modes say what happens. Independent axes. (Ch 12)

PodDisruptionBudget (PDB) — A policy limiting how many replicas of a workload may be voluntarily disrupted at once, which a drain must respect. (Ch 8)

port / targetPort / nodePort — port is exposed by the Service; targetPort is on the container; nodePort is on every node. targetPort defaults to port, which is why the distinction is easy to miss. (Ch 9)

Prometheus — A CNCF monitoring system that **pulls** metrics by scraping targets, stores them as time series, and queries them with PromQL. (Ch 18)

Provenance — A verifiable record of how and where an artifact was built. The build-process half of supply-chain security; the **SBOM** is the components half. (Ch 12)

Pushgateway — A Prometheus intermediary for jobs that cannot be scraped — short-lived batch work. Not a general workaround for the pull model. (Ch 18)

Q

QoS class — A Pod's quality-of-service classification, derived from its containers' requests and limits: Guaranteed, Burstable, BestEffort. Determines eviction order under node pressure. (Ch 5)

R

RBAC (role-based access control) — The authorization mode regulating access by role. **Additive and allow-only**: permissions are the union of every binding naming you, and nothing subtracts. ⚠ After a binding is created you cannot change the Role it refers to. (Ch 12)

Readiness probe — A probe determining whether a container should receive traffic. Failure removes the endpoint from service **without restarting** the container. (Ch 5)

Reclaim policy — What happens to a PersistentVolume after its claim is deleted: `Retain` (kept for manual reclamation), `Delete` (object and backing storage removed), `Recycle` (**deprecated**). (Ch 11)

Release (Helm) — One installed instance of a chart, with a name. Installing the same chart twice gives two releases. ⚠ Distinct from a Kubernetes minor **release**. (Ch 14)

ReplicaSet — The controller maintaining a stated number of identical Pods. Normally managed by a Deployment rather than created directly. (Ch 6)

Requests and limits — A container's declared resource floor and ceiling. **Requests** are what the scheduler counts when placing a Pod; **limits** are what the kubelet and kernel enforce at run time. (Ch 5)

ResourceQuota — A namespaced policy capping **aggregate** consumption. Contrast **LimitRange**, which sets per-object defaults. (Ch 8)

restartPolicy — A Pod field governing restarts of **containers within the Pod**, not of the Pod object: `Always`, `OnFailure`, `Never`. (Ch 5)

Revision — ⚠ **two senses**. (1) A Deployment's numbered rollout history entry. (2) A Helm release's numbered version. (Ch 6, Ch 14)

Reverse proxy — A server that accepts a client's request and forwards it to a backend on the client's behalf. What an Ingress controller is, mechanically. (Ch 10)

Rollback — ⚠ **three senses**. `kubectl rollout undo` walks a Deployment's revisions; `helm rollback` returns a release to a revision; **rollback by revert** changes a commit and lets a GitOps agent reconcile. (Ch 6, Ch 14, Ch 15)

Rolling update — Replacing a workload's Pods incrementally, bounded by `maxSurge` and `maxUnavailable`, so the service stays available throughout. (Ch 6)

Role / RoleBinding — Namespaced RBAC objects: a Role holds permissions, a RoleBinding grants them to subjects within that namespace. (Ch 12)

runC — The reference OCI runtime that actually creates containers. (Ch 2)

RuntimeClass — An object selecting which runtime configuration a Pod uses, the mechanism by which sandboxed runtimes are chosen. (Ch 2)

S

Sandbox —  **two senses.** (1) A sandboxed **runtime** such as gVisor or Kata, giving stronger isolation. (2) The entry-level **CNCF maturity level**. (Ch 2, Ch 17)

SBOM (Software Bill of Materials) — A standardized inventory of the components an artifact contains. The two dominant standards are **SPDX** (Linux Foundation) and **CycloneDX** (OWASP). Answers "what is in this image," which a scan report from last week does not. (Ch 12)

Scheduler (kube-scheduler) — The control-plane component that assigns Pods to nodes: **filter** to the feasible nodes, **score** those, **bind** to the winner. It decides; the kubelet does the starting. (Ch 7)

Secret — An object holding confidential data.  Base64-**encoded**, not encrypted; encryption at rest is a separate cluster configuration. (Ch 4, Ch 12)

Selector — A query over labels. Equality-based or set-based. What a Service, a ReplicaSet and a NetworkPolicy each use to name a set of Pods. (Ch 4)

Service —  **two senses.** (1) The Kubernetes object giving a set of Pods a stable address and name. (2) A **Knative Service**, always written in full. (Ch 9, Ch 17)

ServiceAccount — A Pod's identity to the API server, and an RBAC **subject**. Every namespace has a default one, which can do almost nothing. (Ch 5, Ch 12)

Service mesh — A layer handling service-to-service communication through proxies, with a **data plane** (the proxies) and a **control plane** (what configures them). Provides mTLS, retries, and traffic policy without application changes. (Ch 17)

SIG (Special Interest Group) — An ongoing Kubernetes group owning a topic area. Contrast a **Working Group** (time-bounded, spans SIGs) and a **Committee** (closed membership). (Ch 17)

SLI / SLO / SLA — A **service level indicator** is a measure; an **objective** is a target for it; an **agreement** is the external contract with consequences. Measure, target, consequence. (Ch 18)

SNI (Server Name Indication) — A TLS handshake extension carrying the requested hostname, letting one address serve several hostnames' certificates. (Ch 10)

Span — A single unit of work in a distributed trace. A **trace** is the tree of spans following one request; a **root span** is the first. (Ch 18)

spec / status — An object's two halves: **spec** is the desired state you write, **status** is the observed state the system reports. (Ch 4)

Startup probe — A probe that suspends liveness and readiness checks until an application has finished starting. (Ch 5)

Static Pod — A Pod managed directly by a kubelet from a file on the node, represented in the API by a **mirror Pod**. (Ch 13)

StatefulSet — A controller for Pods that are **not interchangeable**: stable ordinal names, ordered startup, and a per-replica PersistentVolumeClaim from a volumeClaimTemplate. Requires a headless Service for network identity. △ The distinction from Deployment is *identity*, not "writes to disk." (Ch 6, Ch 11)

StorageClass — An object describing a class of storage and naming a provisioner, enabling dynamic provisioning. volumeBindingMode of Immediate binds on claim creation; WaitForFirstConsumer waits for a Pod, so the volume matches its scheduling constraints. (Ch 11)

Subject (RBAC) — Whatever a grant is made to: a user, a group, or a ServiceAccount. (Ch 12)

T

TAG (Technical Advisory Group) — A CNCF-level group coordinating interests across projects. Contrast a Kubernetes **SIG**, which is project-level. (Ch 17)

Taint / toleration — A **taint** is a node repelling Pods by default (NoSchedule, PreferNoSchedule, NoExecute); a **toleration** is a Pod's permission to ignore it. The node refuses; affinity is the Pod asking. (Ch 7)

Tag — A mutable, human-readable pointer to an image. Can be moved to a different image tomorrow; a **digest** cannot. (Ch 2)

TOC (Technical Oversight Committee) — The CNCF body holding the technical vision and overseeing project lifecycle. Contrast the **Governing Board** (marketing, business oversight, budget) and the **End User TAB** (advisory voice of end users). (Ch 17)

Tiller — The cluster-side component Helm 2 used to hold release state and apply changes. Removed in Helm 3, which talks to the API server directly as the user. A frequent distractor. (Ch 14)

TokenRequest — The API through which short-lived, audience-scoped ServiceAccount tokens are issued, replacing long-lived token Secrets. (Ch 12)

Topology spread constraints — A Pod field distributing replicas across failure domains, bounded by maxSkew. (Ch 7)

Trace — The tree of spans following one request across services. One of OpenTelemetry's signals. (Ch 18)

TUF (The Update Framework) — A CNCF framework for securing software update systems. (Ch 12)

Twelve-factor app — A methodology for building services. Factor III, configuration in the environment, is what ConfigMaps and Secrets implement: one image, every environment. (Ch 15)

V

Validating admission webhook — An admission webhook that accepts or rejects an object without modifying it. (Ch 8, Ch 12)

Version skew — The supported version differences between components. **kubelet**: up to three minors older than the API server, never newer. **kubect1**: within one minor, either direction. Three supported minor releases; roughly three releases a year. (Ch 8)

Volume —  **two senses**. (1) A directory in a Pod spec that the Pod's containers share, living as long as the Pod. (2) A **PersistentVolume**, a cluster resource with its own lifecycle. (Ch 11)

volumeClaimTemplate — A StatefulSet field producing one PersistentVolumeClaim per replica, so each Pod keeps its own storage across rescheduling. (Ch 11)

VPA (VerticalPodAutoscaler) — Adjusts the resource requests of existing Pods.  Not part of core Kubernetes — an add-on you deploy. (Ch 17)

W

Waypoint proxy — In ambient mode, the optional per-namespace Layer 7 proxy. (Ch 17)

Working Group — A **time-bounded** Kubernetes group spanning SIG boundaries. (Ch 17)

Workload — An application running on Kubernetes, expressed through a controller: Deployment, StatefulSet, DaemonSet, Job, CronJob. (Ch 6)

Acronyms

Acronym	Expansion
ABAC	Attribute-Based Access Control
API	Application Programming Interface
BGP	Border Gateway Protocol
CA	Cluster Autoscaler
CD	Continuous Delivery / Continuous Deployment
CEL	Common Expression Language
CI	Continuous Integration
CKA	Certified Kubernetes Administrator
CKAD	Certified Kubernetes Application Developer
CKS	Certified Kubernetes Security Specialist
CNAME	Canonical Name (DNS record)
CNCF	Cloud Native Computing Foundation
CNI	Container Network Interface
CRD	CustomResourceDefinition
CRI	Container Runtime Interface
CSI	Container Storage Interface
DNS	Domain Name System
eBPF	extended Berkeley Packet Filter
EBS	Elastic Block Store
FaaS	Functions as a Service
FQDN	Fully Qualified Domain Name
GUID	Globally Unique Identifier
HPA	HorizontalPodAutoscaler
iSCSI	Internet Small Computer Systems Interface
IPVS	IP Virtual Server
KCNA	Kubernetes and Cloud Native Associate
KCSA	Kubernetes and Cloud Native Security Associate
KEDA	Kubernetes Event-Driven Autoscaling
KEP	Kubernetes Enhancement Proposal
LTS	Long-Term Support

Acronym	Expansion
LUN	Logical Unit Number
mTLS	mutual Transport Layer Security
NAT	Network Address Translation
NFS	Network File System
OCI	Open Container Initiative
OIDC	OpenID Connect
OOM	Out Of Memory
OPA	Open Policy Agent
OSI	Open Systems Interconnection
OTel	OpenTelemetry
OWASP	Open Worldwide Application Security Project
PDB	PodDisruptionBudget
PSA	Pod Security Admission
PSS	Pod Security Standards
PV	PersistentVolume
PVC	PersistentVolumeClaim
QoS	Quality of Service
RBAC	Role-Based Access Control
ROX	ReadOnlyMany
RWO	ReadWriteOnce
RWOP	ReadWriteOncePod
RWX	ReadWriteMany
SBOM	Software Bill of Materials
SIG	Special Interest Group
SLA	Service Level Agreement
SLI	Service Level Indicator
SLO	Service Level Objective
SNI	Server Name Indication
SPDX	Software Package Data Exchange
SRE	Site Reliability Engineering
TAB	Technical Advisory Board

Acronym	Expansion
TAG	Technical Advisory Group
TLS	Transport Layer Security
TOC	Technical Oversight Committee
TUF	The Update Framework
VM	Virtual Machine
VPA	VerticalPodAutoscaler
WG	Working Group